1C:ENTERPRISE 8.3

Practical Developer Guide

Examples and best practices

Contents

Overview 20
Target audience 20
How to read this book 21
Distribution package contents 21
Limitations of the training platform version 24
Lesson 1 (0:40). Introduction. Creating an infobase 25
Programming or development? 25
General overview 25
Configurations and applied solutions 26
System operation modes 27
Creating an infobase 28
In Designer mode 32
Introduction to Designer 32
Configuration object tree 33
Understanding configuration objects 34
Adding configuration objects 35
Property palette 38
Debugging in 1C:Enterprise mode 40
In 1C:Enterprise mode 40
Applied solution interface appearance 40
Quiz 41
Lesson 2 (0:45). Subsystems 42
Understanding subsystems 42
Adding subsystems 43
In Designer mode 43
Configuration object name and synonym 45
Adding subsystem icons 45
Applied solution sections panel 50
In 1C:Enterprise mode 50
Specifying section order 50
In Designer mode 50
In 1C:Enterprise mode 52
Theory. Configuration object editor window and property palette 53
Quiz 53
Lesson 3 (2:10). Catalogs 54
Understanding catalogs 54
Catalog forms 59

Simple catalogs 64
In Designer mode 65
Configuration object presentations 65
Object affiliation with subsystems 67
Catalog code and description 69
Item creation command 71
In 1C:Enterprise mode 73
Current section commands panel 73
Creating catalog items 75
Customizing applied solution panel view 78
In Designer mode 78
In 1C:Enterprise mode 80
Catalogs with tabular sections 82
In Designer mode 82
Tabular sections 85
In 1C:Enterprise mode 88
Filling tabular sections 89
Hierarchical catalogs 91
In Designer mode 91
In 1C:Enterprise mode 96
Creating items in a hierarchical catalog 96
Moving items between groups 99
Catalogs with predefined items 102
In Designer mode 102
Quick choice property 104
Predefined items 105
In 1C:Enterprise mode 107
Theory 109
Predefined items 109
Base configuration and database configuration 109
Quiz 112
Lesson 4 (1:30). Documents 114
Understanding documents 114
Document forms 117
Theory. Data types. Type-defining configuration objects 118
The Goods receipt document 119
In Designer mode 119
Adding a document 119
Attributes of the reference type 122
"Fill value" property of a configuration object attribute 122

Tabular section fill check 124
In 1C:Enterprise mode 126
Adding Goods receipt documents 127
Theory. Catalogs and documents 129
Automatic recalculation of document totals 130
In Designer mode 130
Document form 130
Event handler 134
In 1C:Enterprise mode 137
A single procedure as a hander for multiple events 137
In Designer mode 137
Common module 137
In 1C:Enterpise mode 140
The Services document 140
In Designer mode 141
In 1C:Enterprise mode 143
Quiz 144
Lesson 5 (2:00). Theory 145
Default forms 145
Event handlers 146
Modules 146
Module types 146
Form module context 152
A form as a script object 161
Event handler procedures in form modules 161
Understanding form module content 164
Understanding 1C:Enterprise script 164
Using Syntax Assistant to analyze script 166
Method #1 166
Method #2 175
Analyzing script with Debugger 178
Objects, objects, objects 184
Server and clients 185
Common module compilation 188
Compilation directives 189
Script execution on the client and on the server 189
Lesson 6 (0:50). Accumulation registers 191
Accumulation register purpose 191
Understanding accumulation registers 192
Adding an accumulation register 194

In Designer mode 194
Creating register records of a document 198
In 1C:Enterprise mode 204
Register records opening command in a document form 207
In Designer mode 207
In 1C:Enterprise mode 208
Creating register records of the Services document 210
In Designer mode 210
In 1C:Enterprise mode 212
Theory. Accessing collection items 214
Quiz 215
Lesson 7 (0:25). A simple report 216
Understanding reports 216
Adding a report 216
In Designer mode 216
Template 218
Data composition schema 219
Data set 219
Query text 219
Report settings 222
In 1C:Enterprise mode 225
Quiz 226
Lesson 8 (0:40). Templates. Editing templates 227
Understanding templates 227
Print form template 227
In Designer mode 227
In 1C:Enterprise mode 232
Editing a template 234
In Designer mode 234
In 1C:Enterprise mode 237
Quiz 238
Lesson 9 (0:50). Periodic information registers 239
Information register purpose 239
Understanding information registers 239
Adding a periodic information register 241
In Designer mode 242
Dimensions and resources 243
In 1C:Enteprise mode 245
Creating information register records 246
Filling prices in documents upon selecting materials or services 247

In Designer mode 248
Creating a function that returns material or service price 248
Calling the function upon material or service selection for filling the price . 250
In 1C:Enterprise mode 252
Quiz 254
Lesson 10 (0:40). Enumerations 255
Understanding enumerations 255
Adding enumerations 255
In Designer mode 255
Linking materials and services to enumeration values 256
In Designer mode 256
In 1C:Enterprise mode 257
Custom presentations for materials and services 258
In Designer mode 260
In 1C:Enterprise mode 261
Recording expenses for materials only 262
In Designer mode 262
In 1C:Enterprise mode 264
Quiz 265
Lesson 11 (1:20). Posting documents across multiple registers 266
Why post a document across multiple registers 266
Adding another accumulation register 266
In Designer mode 266
Posting goods receipts across two registers 268
In Designer mode 268
Modifying the posting procedure 268
Adding the command that opens register records 273
In 1C:Enterprise mode 273
Posting Services documents across two registers 274
In Designer mode 275
Adding the document attribute 275
Modifying the posting procedure 277
In 1C:Enterprise mode 279
Quiz 282
Lesson 12 (0:40). Turnover accumulation registers 283
Why we need another register 283
Understanding turnover accumulation registers 283
Adding turnover accumulation registers 285
In Designer mode 285
Posting Services documents across three registers 286

In Designer mode 287
In 1C:Enterprise mode 289
Quiz 291
Lesson 13 (4:30). Reports 292
Theory. Data access methods 292
Using queries 293
Query data sources 293
Query language 294
Data composition system 295
Selecting data from a single table 298
In Designer mode 299
Query for a data set 300
Field aliases 301
Records order 302
Analyzing query text 302
Settings 304
In 1C:Enterprise mode 306
Selecting data from two tables 308
In Designer mode 308
Query for a data set 308
Left join of two tables 308
Record filter condition 311
Field aliases 312
Records order 313
Analyzing query text 313
Resources 314
Parameters 315
Settings 319
Quick user settings 320
In 1C:Enterprise mode 323
Settings in Designer and settings in 1C:Enterprise mode 325
Conditional appearance 326
In Designer mode 326
In 1C:Enterprise mode 329
User settings 331
In Designer mode 331
Filter 332
In Designer mode 332
In 1C:Enterprise mode 332
Breaking down by days in the selected period 336

In Designer mode 337
Query for a data set 337
Virtual table parameters 337
Analyzing query text 338
Resources 339
Parameters 339
Settings 339
In 1C:Enterprise mode 343
Breaking down by date within the selected period 344
In Designer 344
In 1C:Enterprise mode 346
Report options 347
Charts 348
In Designer mode 349
In 1C:Enterprise mode 352
Getting current values from a periodic information register 353
In Designer mode 354
Query for a data set 354
Virtual table parameters 354
Left join of tables 355
Field aliases 356
Analyzing query text 356
Resources 357
Parameters 357
Settings 358
In 1C:Enterprise mode 360
Using calculated fields in reports 361
In Designer mode 362
Query for a data set 362
Calculated fields 362
Resources 363
Settings 363
In 1C:Enterprise mode 365
Displaying data as a spreadsheet 367
In Designer mode 367
Query for a data set 368
Analyzing query text 368
Resources 368
Settings 368
In 1C:Enterprise mode 370

Theory. Virtual query tables 373
Quiz 373
Lesson 14 (3:20). Optimization of posting the Services document 375
Theory. Referential data usage specifics 375
Improving posting performance 379
In Designer mode 379
In 1C:Enterprise mode 392
Automatic cost calculation 392
In Designer mode 394
In 1C:Enterprise mode 412
Theory 412
Quick viewing of a query result 412
Real-time and regular document posting 414
"Point in time" concept 415
Checking the balances 417
In Designer mode 417
Locking data that is read and modified during posting 420
In Designer mode 421
Marking custom module regions 422
In 1C:Enterprise mode 427
Theory. Understanding cache 427
Standard cache 428
Transactional cache 429
Quiz 430
Lesson 15 (2:50). Charts of characteristic types 431
The task at hand 431
Understanding charts of characteristic types 431
Logical relation between the objects 435
Creating new configuration objects 435
In Designer mode 435
Description of material option characteristics 440
Modifying configuration objects 442
The MaterialOptions catalog 443
In 1C:Enterprise mode 443
In Designer mode 445
In 1C:Enterprise mode 446
In Designer mode 447
In 1C:Enterprise mode 449
The MaterialPropertyValues information register 450
In 1C:Enterprise mode 450

In Designer mode 451
In 1C:Enterprise mode 453
In Designer mode 454
In 1C:Enterprise mode 455
Creating material characteristic types 455
In 1C:Enterprise mode 455
Modifying accounting functionality 462
The BalanceOfMaterials register 463
In Designer mode 463
The GoodsReceipt document 464
In Designer mode 464
The Services document 467
In Designer mode 467
Material receipt and expense with property sets specified 469
In 1C:Enterprise mode 469
Report with characteristics 471
In Designer mode 472
Query for a data set 472
Resources 473
Settings 474
In 1C:Enterprise mode 475
Quiz 479
Lesson 16 (1:50). Accounting 481
Using charts of characteristic types in accounting 481
Adding a chart of characteristic types 481
In Designer mode 481
Understanding charts of accounts 484
Adding a chart of accounts 485
In Designer mode 485
Understanding accounting registers 489
Adding an accounting register 490
In Designer mode 490
Modifying the GoodsReceipt document 493
In Designer mode 493
In 1C:Enterprise Mode 496
Modifying the Services document 497
In Designer mode 497
In 1C:Enterprise mode 499
Trial balance 500
In Designer mode 501

Query for a data set....501
Resources....502
Parameters....503
Settings....503
In 1C:Enterprise mode....505
Quiz....505
Lesson 17 (1:00). Charts of calculation types and calculation registers....507
Chart of calculation types and calculation register purposes....507
Dependency by base period....507
Displacement by action period....508
Understanding charts of calculation types....509
Adding a chart of calculation types....511
In Designer mode....511
Understanding calculation registers....513
Periodicity....514
Displacement by action period....515
Dependency by base period....517
Dependency by action period....517
Dependency by registration period....518
Adding a calculation register....519
In Designer mode....519
In 1C:Enterprise mode....520
In Designer mode....521
Quiz....524
Lesson 18 (3:40). Using calculation registers....526
Adding a document that describes accruals....526
In Designer mode....526
In 1C:Enterprise mode....531
Examples of displacement and dependency by base period....533
Recalculation report....533
In Designer mode....533
Dependency by base period....534
In 1C:Enterprise mode....534
Displacement by action period....535
In 1C:Enterprise mode....535
Procedure for calculating calculation register records....537
In Designer mode....537
In 1C:Enterprise mode....544
Employee accruals report....547
In Designer mode....548

Query for a data set....548
Resources....549
Settings....549
In 1C:Enterprise mode....550
Recalculations....551
In Designer mode....551
In 1C:Enterprise mode....555
Gantt chart....558
In Designer mode....560
In 1C:Enterprise mode....566
Quiz....567
Lesson 19 (0:40). Database search....569
Full-text search basics....569
Creating a full-text index....570
In Designer mode....570
In 1C:Enterprise mode....572
Database search examples....574
In 1C:Enterprise mode....574
Search during input by string....579
In Designer mode....579
In 1C:Enterprise mode....581
Quiz....581
Lesson 20 (0:35). Scheduled jobs....582
The task at hand....582
Understanding scheduled jobs....583
Creating scheduled jobs....583
In Designer mode....583
Running scheduled jobs....590
In 1C:Enterprise mode....590
Quiz....591
Lesson 21 (0:40). Editing register records in document forms....592
In Designer mode....592
In 1C:Enterprise mode....596
Editing register records from 1C:Enterprise script....597
Writing register records from a form....597
In Designer mode....597
In 1C:Enterprise mode....597
Writing register records from 1C:Enterprise script....598
In Designer mode....598
In 1C:Enterprise mode....600

Choosing handler locations 600
Quiz 600
Lesson 22 (1:50). Users and their roles 601
Understanding roles 601
Creating roles 601
In Designer mode 601
Administrator 602
CEO 604
Technician 605
Payroll accountant 606
Accountant 606
Rights to run client applications 607
Adding users 608
In Designer mode 608
Record and database field level data access restrictions 611
In Designer mode 612
In 1C:Enterprise mode 615
In Designer mode 616
In 1C:Enterprise mode 618
In Designer mode 620
In 1C:Enterprise mode 623
In Designer mode 623
Quiz 623
Lesson 23 (1:10). Start page and command interface customization 625
Customizing section command interfaces 625
In Designer mode 625
In 1C:Enterprise mode 635
Quick menu. Customizing the start page 638
In Designer mode 639
In 1C:Enterprise mode 645
Quiz 650
Lesson 24 (6:10). Data exchange 651
Understanding data exchange 651
Understanding exchange plans 652
XML serialization 654
Reading and writing XML documents 654
Universal data exchange 654
Creating a constant for data exchange 655
In Designer mode 655
Updating configuration objects involved in data exchange 656

In Designer mode 656
Adding an exchange plan 657
In Designer mode 657
Creating data exchange procedures 663
In Designer mode 663
Data writing procedure 665
Data reading procedure 670
Testing data exchange 684
In Designer mode 684
In 1C:Enterprise mode 685
Running the branch infobase 687
In Designer mode 688
In 1C:Enterprise mode 689
Distributed infobases 691
Overview 692
The task at hand 693
Interactive exchange 693
In Designer mode 693
In 1C:Enterprise mode 695
Running the department infobase 697
In Designer mode 697
In 1C:Enterprise mode 698
Script-based exchange 701
In Designer mode 701
Modifying distributed infobase node structure 712
Quiz 715
Lesson 25 (0:30). Functional options 716
Accounting and Payroll options 716
In Designer mode 717
In 1C:Enterprise mode 722
Accounting by customers option 724
In Designer mode 725
In 1C:Enterprise mode 729
Quiz 731
Lesson 26 (2:00). Picking list items, avoiding modal windows, and generating data based on other data 732
Picking list items 732
Picking a single item 732
In Designer mode 732
In 1C:Enterprise mode 735

Picking multiple items 735
In Designer mode 735
In 1C:Enterprise mode 736
Picking with multiple selection 736
In Designer mode 736
In 1C:Enterprise mode 737
Picking multiple items with multiple selection 737
In Designer mode 737
In 1C:Enterprise mode 738
Using the NotifyChoice() method 738
Avoiding modal windows 738
Theory 738
Requesting user input in a form command 739
In Designer mode 739
In 1C:Enterprise mode 742
Requesting user input in an event handler 742
In 1C:Enterprise mode 744
Generating data based on other data 745
Creating a document generation command 745
In Designer mode 745
In 1C:Enterprise mode 747
Generated objects 748
Understanding filter criteria 748
Getting generated objects 748
In Designer mode 749
In 1C:Enterprise mode 750
Quiz 750
Lesson 27 (2:10). Form development techniques 751
Form data and controls 751
Form data types 757
Related lists 760
In Designer mode 760
In 1C:Enterprise mode 761
Formatting rows in list forms 763
In Designer mode 763
In 1C:Enterprise mode 766
In Designer mode 767
In 1C:Enterprise mode 768
Calculated columns in lists 769
In Designer mode 769

In 1C:Enterprise mode 774
Selection list in a text box 775
In Designer mode 776
In 1C:Enterprise mode 777
Choice form for a field with a reference attribute 777
In Designer mode 778
In 1C:Enterprise mode 782
Attribute fill check 783
Automatic fill check 783
In Designer mode 783
In 1C:Enterprise mode 784
1C:Enterprise script-based fill check 784
In Designer mode 784
In 1C:Enterprise mode 786
Using parameterized commands 786
In Designer mode 787
In 1C:Enterprise mode 789
Opening a list form with the specified filter 790
In Designer mode 791
In 1C:Enterprise mode 792
In Designer mode 793
In 1C:Enterprise mode 795
Quiz 796
Lesson 28 (2:00). Form customization techniques 797
Form editor 797
Implementing a custom form for the Services document 801
In Designer mode 801
In 1C:Enterprise mode 814
In Designer mode 815
In 1C:Enterprise mode 820
In Designer mode 821
In 1C:Enterprise mode 824
Adding a tumbler switch to a form 826
In Designer mode 826
In 1C:Enterprise mode 828
Grouping list data 829
In Designer mode 829
In 1C:Enterprise mode 831
Quiz 831
Quick developer reference 832

1C:Enterprise script objects used for operations with applied data 832
Manipulating object data 833
Constants 836
1C:Enterprise script objects used for operations with constants 836
Event sequence for writing constants from constant forms (save and close) .. 838
Catalogs 839
1C:Enterprise script objects used for operations with catalogs 839
Event sequence for writing catalog items from item forms (save and close) .. 843
Documents 843
1C:Enterprise script objects used for operations with documents 843
Event sequence for writing documents from document forms 847
Event sequence for posting documents from document forms (post and close) 848
Event sequence for canceling document posting from document forms 849
Enumerations 849
1C:Enterprise script objects used for operations with enumerations 849
Charts of characteristic types 851
1C:Enterprise script objects used for operations with charts of characteristic types 851
Event sequence for writing characteristic types from item forms (save and close) 852
Charts of accounts 852
1C:Enterprise script objects used for operations with charts of accounts 852
Event sequence for writing accounts from account forms (save and close) 854
Charts of calculation types 854
1C:Enterprise script objects used for operations with charts of calculation types 854
Event sequence for writing calculation types from calculation type forms (save and close) 856
Information registers 857
1C:Enterprise script objects used for operations with information registers ... 857
Event sequence for writing data from information register record forms (save and close) 861
Event sequence for writing data from information register record set forms (save and close) 862
Accumulation registers 863
1C:Enterprise script objects used for operations with accumulation registers. 863
Event sequence for writing accumulation register record sets from record set forms 866
Accounting registers 866
1C:Enterprise script objects used for operations with accounting registers 866
Event sequence for writing accounting register record sets from record set forms 868

Calculation registers 868
1C:Enterprise script objects used for operations with calculation registers 868
Event sequence for writing calculation register record sets from record set forms 870
Exchange plans 871
1C:Enterprise script objects used for operations with exchange plans 871
Event sequence for writing exchange plan nodes from node forms (save and close) 872
Glossary 873

This book is a quick tutorial on development and modification of applied solutions based on the 1C:Enterprise 8.3 platform.

The tutorial demonstrates the structure of various platform objects, their purposes, and usage scenarios based on an actual applied solution. It provides examples of 1C:Enterprise script procedures with detailed comments, including examples that utilize the query language.

The book can serve both as a practical guide and a reference manual. The applied solution described in the book is based on accumulated 1C:Enterprise 8 experience enhanced by new features and capabilities introduced in version 8.3.

The tutorial is designed for beginner developers who are not yet familiar with 1C:Enterprise.

Russian text: M. Radchenko, E. Khrustaleva

Translation: Y. Ivanova

Translation editor: S. Polikarpov

Copyright (C) 2014, 1C Company.

All rights reserved. No part of this document or the related files may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording, or otherwise) without the prior written permission of the publisher except for the use of brief quotations in a book review.

Limit of Liability and Disclaimer of Warranty: The publisher has used its best efforts in preparing this book, and the information provided herein is provided "as is." 1C Company makes no representation or warranties with respect to the accuracy or completeness of the contents of this book and specifically disclaims any implied warranties of merchantability or fitness for any particular purpose and shall in no event be liable for any loss of profit or any other commercial damage, including but not limited to special, incidental, consequential, or other damages.

Trademarks: This book identifies product names and services known to be trademarks, registered trademarks, or service marks of their respective holders. They are used throughout this book in an editorial fashion only. In addition, terms suspected of being trademarks, registered trademarks, or service marks have been appropriately capitalized, although 1C Company cannot attest to the accuracy of this information. Use of a term in this book should not be regarded as affecting the validity of any trademark, registered trademark, or service mark. 1C Company is not associated with any other product mentioned in this book, except for 1C:Enterprise and applied solutions developed by 1C Company or in association with 1C Company. 1C Company is not associated with any other vendor mentioned in this book.

Overview

The new edition of this book is prepared for the release of 1C:Enterprise 8.3.

1C:Enterprise 8.3 features significant improvements of the platform architecture in the areas of cross-platform design, mobility, and Internet services. Since these improvements require knowledge of advanced development techniques, they are out of scope of this book.

However, the new platform version also includes a lot of new capabilities available to beginner developers. Among these, the book describes the new Taxi interface, the interface fully composed of nonmodal windows, new search options, new 1C:Enterprise script editor options, and more.

The authors hope this book will prove useful for everyone who wants to learn how to develop applied solutions based on the 1C:Enterprise 8.3 platform.

If you need any help with the lessons included in this book, feel free to ask at 1C:Developer Network forum (http://1c-dn.com/forum) to get support both from 1C specialists and the developer community.

Target audience

The tutorial is based on an actual applied solution developed for a small company that provides consumer services. During your advancement through the tutorial you will learn the basic 1C:Enterprise development techniques for various business automation areas, accounting, payroll calculation, and more.

Why have we picked this particular example?

On the one hand, most of us are familiar with consumer services. One way or another, we constantly use such services, which include home appliance repair, car services, laundry and dry cleaning, hair stylists and beauty consultants, and many more.

On the other hand, the activities in a maintenance business lend themselves to a demonstration of 1C:Enterprise capabilities. Here you find a range of customer services, the need to supply the company with the required materials, and expenses in the process of providing services. The company routines include human resource and payroll management. We can also illustrate the accounting process. Due to this broad range of activities, we can offer detailed demonstration of the capability to create various account statements and summary reports based on the applied solution data.

This book is intended mostly for novice developers making their first steps in applied solution development. The explanations in the book are detailed and clear enough even for those readers who only know the very basics of programming.

If you have just started learning 1C:Enterprise or you are not familiar with it at all but willing to master the system, the book is intended for you. The goal of the book is to take you step by step through the major stages of

applied solution development in 1C:Enterprise and to demonstrate that nothing is impossible for an intelligent human being.

This book will also be useful to experienced developers because it will help them refresh their skills or improve their knowledge of specific development techniques.

How to read this book

This book is a tutorial consisting of individual lessons. At the beginning of each lesson we provide an estimated time for lesson completion.

Each lesson ends with a quiz to test your new knowledge.

The book has a brief table of contents listing the lessons and their durations. The lessons themselves include detailed tables of contents, you can use them to navigate to specific sections of completed lessons. A detailed table of contents for all lessons is available at the end of the book.

Each lesson is a logically complete part of applied solution development. Hence, while the lessons have different durations, we recommend completing each lesson in full, from the beginning to the end. Otherwise as a novice developer you may face difficulties while resuming your activities from some point in the middle of a lesson.

Each lesson starts with simple concepts, which are followed by more complex ones. The lessons provide step-by-step descriptions of basic techniques and various fields of 1C:Enterprise development.

The book includes numerous figures and 1C:Enterprise script examples accompanied by detailed explanations. If such explanations seem to be unnecessary or too detailed, you can skip them.

The lessons also include theory that you can either read while you work on a lesson or leave for later. Whatever you choose, it will not influence the completion of the development example that is covered in the book.

"Lesson 5 (2:00). Theory" on page 145 provides detailed examples of using Debugger and Syntax Assistant. This will help you to further learn 1C:Enterprise script on your own and deal with the typical mistakes when you work through this tutorial.

The applied solution created in the book is provided as a demo infobase in the book distribution package. You can use this demo when you need to check whether you perform the tasks described in the book correctly.

Since the example studied in the book is quite large, the demo contains four infobases instead of one. These infobases correspond to completing lessons 8, 13, 20, and 27. This will help you quickly find the applied solution fragments that you need.

Distribution package contents

The book is a part of a distribution package with materials intended for learning and individual use.

First, these materials include 1C:Enterprise 8.3. Training version (see the next section to learn about the limitations of the training version).

Second, they include four demo applied solutions that correspond to different stages of the example applied solution development. You can use these demo solutions with both the training version and the full version of 1C:Enterprise.

All the applied solutions are stored in a single installation package. When you run the setup file, the applied solution templates are installed to the current template directory. These applied solutions have been developed in 1C:Enterprise version 8.3.5.1086.

The demo applied solutions should not be treated as complete solutions and they are not intended for real-life accounting purposes. They are merely a set of examples that explain the content of the book.

The distribution package also contains all script listings that are included in the book. They may be useful both while you read the book and in your future work. This is why we have arranged them as a 1C:Enterprise text template file example.st.

You can attach this file to any configuration.

To attach the file

1. In Designer, on the **Tools** menu, click **Text templates**.
2. In the **Text templates** window, click **Actions** and then click **Template Setup**.
3. In the **Template settings** window, click the **Add** button (fig. 0.1).

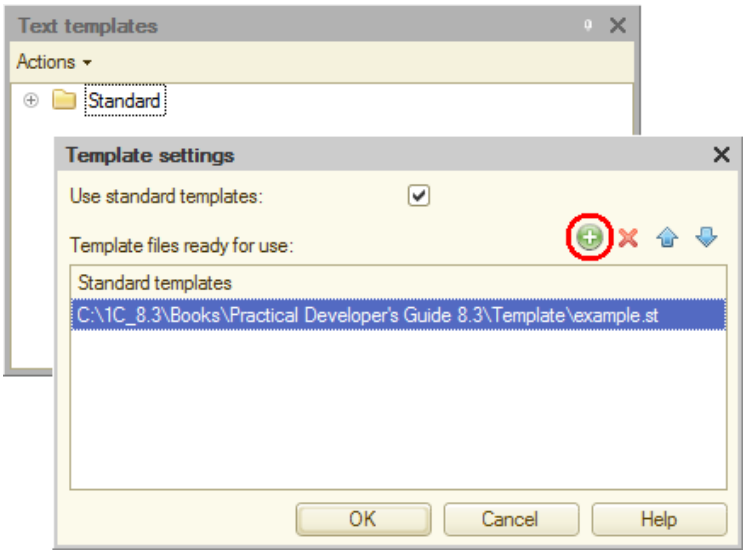


Fig. 0.1. Attaching the templates

The script fragments are grouped by lessons (fig. 0.2).

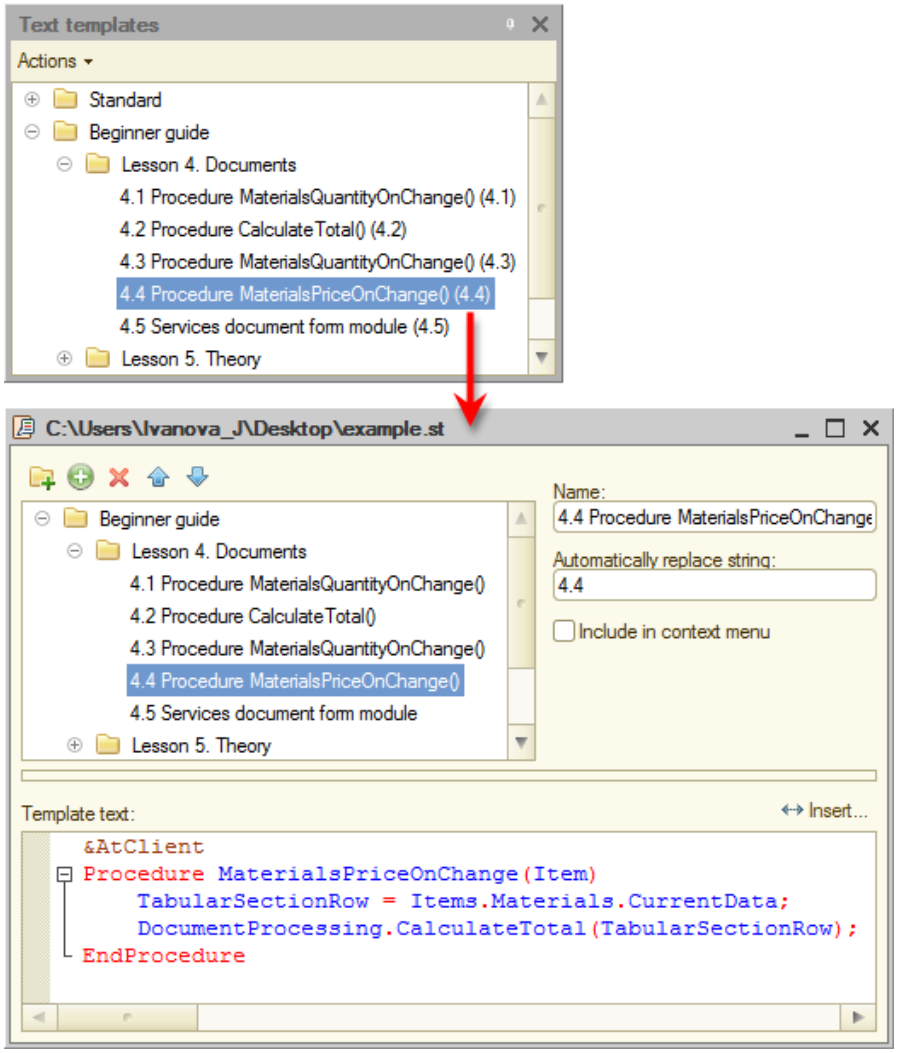


Fig. 0.2. Script fragments as templates

For each script fragment, its number in the book serves as a keyword that will be replaced by the template content. You can have those numbers replaced automatically as you type (provided that you enable the autoreplacement feature), or you can replace typed numbers manually by pressing Ctrl+Q.

To enable autoreplacement

1. On the **Tools** menu, click **Options**.
2. Click the **Modules** tab.
3. Click the **Edit** tab.
4. In the **Autoreplacement** list, select **Enable**.

You can also drag any text template to any location in a module.

The distribution package also includes the Pictures folder. It contains the icons that will represent configuration subsystems in your applied solution.

Limitations of the training platform version

The training version of the platform has some limitations that will affect execution of some examples provided in the book. There are few limitations and none of them are critical, but they should be mentioned anyway.

In lesson 22 you will create a list of software users. The training version of the platform does not support setting user passwords and using operating system authentication. But this is not vital for studying because you will still be able to run the software on behalf of each of these users. However, the users will not have passwords.

Lesson 24 covers data exchange. The training version of the platform will not let you test the second part of the example (the distributed infobase). But you will be able to fully study the first, more general part (the universal exchange mechanism).

Lesson 1 (0:40). Introduction. Creating an infobase

The first lesson is an introduction to 1C:Enterprise 8 and its primary development tool, Designer.

You will learn the meaning behind the *platform*, *applied solution*, and *application* terms, 1C:Enterprise 8 startup modes, the definition of configuration objects, and how to create objects and define their properties.

Finally, you will create an empty infobase for the training applied solution development.

Programming or development?

"What exactly am I doing?" This question comes up from time to time for everyone who has worked in 1C:Enterprise development or merely looked into it.

"I'm programming," is the most frequent response. "In what?" ... "1C." "What are you working on?" ... "1C." "What is it coded in?" ... "1C." "Wanted: accountant with 1C knowledge," "part-time 1C programmer needed," and so on.

For a casual listener these phrases draw no particular attention. However, for those who know something about 1C:Enterprise development, questions like these are baffling, since the term 1C is used to refer to entirely different things, and the term program is just deceiving.

If your goal is to learn programming in 1C, this is not the right wording. 1C:Enterprise includes a built-in programming language (1C:Enterprise script) but this language is not the most important part of the development process. And this book does not teach you programming as people normally understand it. This book teaches development of applied solutions on the 1C:Enterprise platform. This process definitely includes programming but only as one of the many development tools.

It is important that you understand it from the very beginning, before you take your first steps with 1C:Enterprise.

To help you understand exactly what you will be creating in this tutorial, we will start with an overview of 1C:Enterprise.

General overview

1C:Enterprise is universal software for automating financial and operational activities of a company. Since these activities can be quite diverse, 1C:Enterprise has the capability to adapt to the specific needs of the field where it is used. This capability is summarized in the term configurability, which describes the ability to customize the system based on the needs of a specific company and a specific set of tasks.

The key to configurability is the fact that 1C:Enterprise is more than just a program composed of a set of files that cannot be changed. Rather, it is a diverse set of software tools employed by developers and users. Logically, it can be divided into two major components that are closely interrelated: an applied solution and the platform that controls the functioning of the applied solution.

To explain the relationship between these 1C:Enterprise components, we compare it to a CD player. Everyone needs a player in order to listen to music. However, given the variety of "different strokes for different folks", there are lots of music CDs to suit any taste.

To listen to a track, you need to put a CD into the player, and the player will reproduce the music that is recorded on it. In addition, modern CD players will even let you record your own selection of music, in other words, create a new CD.

A player is useless without a CD, and likewise, a CD is useless if you do not have a player (except perhaps as a coaster for your coffee cup).

Getting back to 1C:Enterprise, the platform is a "player," and an applied solution is a "CD." The platform provides the environment for creating, modifying, and running applied solutions.

There is a single platform (1C:Enterprise) and many applied solutions. To run an application, you always need to have both the platform and an applied solution (fig. 1.1).

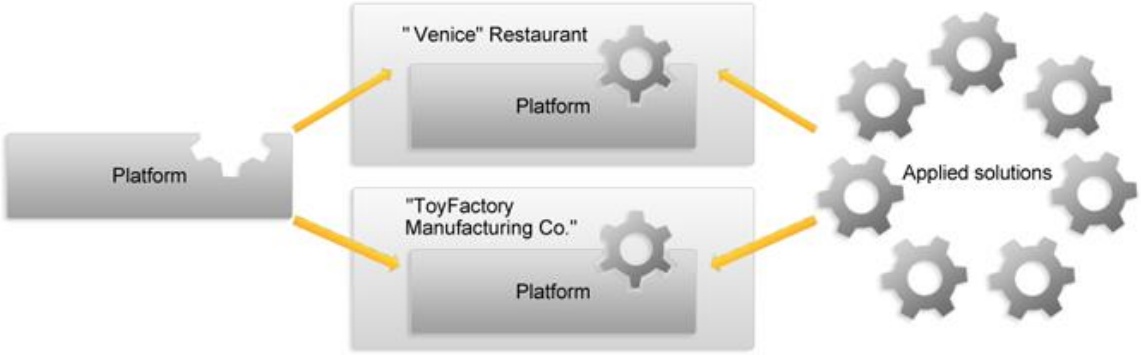


Fig. 1.1. A single platform with many applied solutions

The platform does not perform any automation tasks on its own because its purpose is to provide an environment for running applied solutions. The same is true for an applied solution: to execute the tasks it is intended for, it needs to have a platform that manages its operation.

Configurations and applied solutions

Now we can finally answer the question that was raised in the previous section. In the process of reading this book and implementing the demo example, we will develop an applied solution.

A word should be said about a minor ambiguity in the terminology that will be used later in this book. The ambiguity comes from the use of different terms to refer to a single object: *configuration* and *applied solution*.

Both terms refer to the part of the 1C:Enterprise system that is managed by the platform and is available to all end users, and should be used depending on the context.

When talking about developer work, the term "configuration" is used, since this is a more precise term within 1C:Enterprise. On the contrary, the term "applied solution" is more widely used and understood by 1C:Enterprise users.

Therefore, since the automation tasks can be quite varied, as we mentioned above, 1C Company and its partners develop applied solutions, each of which is designed to automate a specific area of human activities. Examples of applied solutions are 1C:AccountingSuite and 1C:Small Business.

There are many other standard applied solutions. For the detailed list, visit https://1c-dn.com/partners/solutions.

Standard applied solutions satisfy the requirements of the widest range of companies working in the same field. That is a good thing.

On the other hand, usually companies use only a fraction of the applied solution features, while some of the features they need will be missing (you cannot please everyone).

This is where configurability comes into play, since, in addition to managing applied solution features, the platform has resources for customizing the applied solution deployed in a company. Additionally, the platform allows you to create custom applied solutions from scratch if for some reason you decide against using one of the standard applied solutions.

If we return to the CD player example, one can alter whatever melody was previously recorded on the CD to suit their tastes, and they can even create disks with their own music. And they do not need any musical instruments because everything they need to create a melody is included in the CD player.

System operation modes

In order to provide the functionality described in the previous section, 1C:Enterprise offers two operation modes: 1C:Enterprise and Designer.

1C:Enterprise mode is the basic one, it allows end users to work with applied solutions. In this mode users enter data, have them processed, and obtain the results.

Designer mode is used by developers and infobase administrators. This is the mode that provides the tools required to modify an applied solution or create a new one.

Since the purpose of this book is to teach you how to create applied solutions and customize third-party ones, the discussion that follows deals mainly with using the system in Designer mode. You will only run the system in 1C:Enterprise mode from time to time to check your work.

We assume that you already have 1C:Enterprise version 8.3 installed on your computer. If you do not have it yet, now is the time to install it, since the text that follows describes how to use 1C:Enterprise.

Creating an infobase

You should have no problem installing 1C:Enterprise. For installation details, see 1C:Enterprise 8. Administrator Guide.

You should also have no trouble with running the platform and creating an infobase with an empty configuration, which is described later in this section.

Important! To create the example described in this book, you need an infobase intended to develop a new configuration, rather than an infobase created from a template. Let us create that infobase.

1. Run 1C:Enterprise.

   The **1C:Enterprise startup** window displays the list of available infobases.

2. If the list is empty, the platform prompts you to create an infobase.

   -OR-

   If the list is not empty, click **Add** to create an infobase (fig. 1.2).

   The list might not be empty if you installed a configuration earlier (for example, a demo configuration delivered with this book).

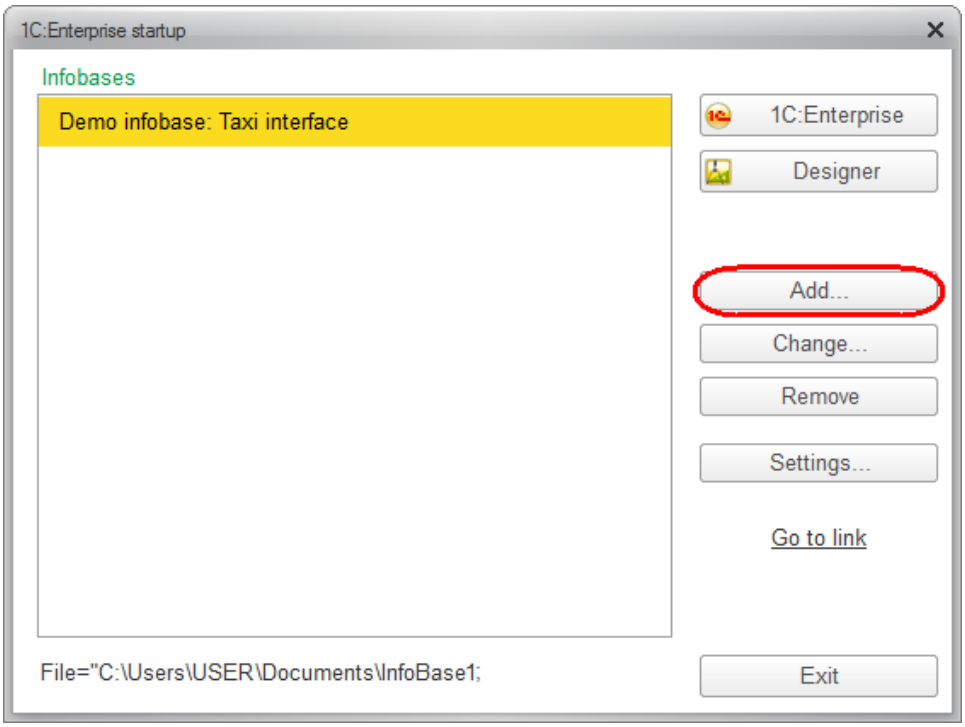


Fig. 1.2. Creating an infobase, step 1

3. In the **Add infobase/group** window, click **Create infobase** (fig. 1.3).

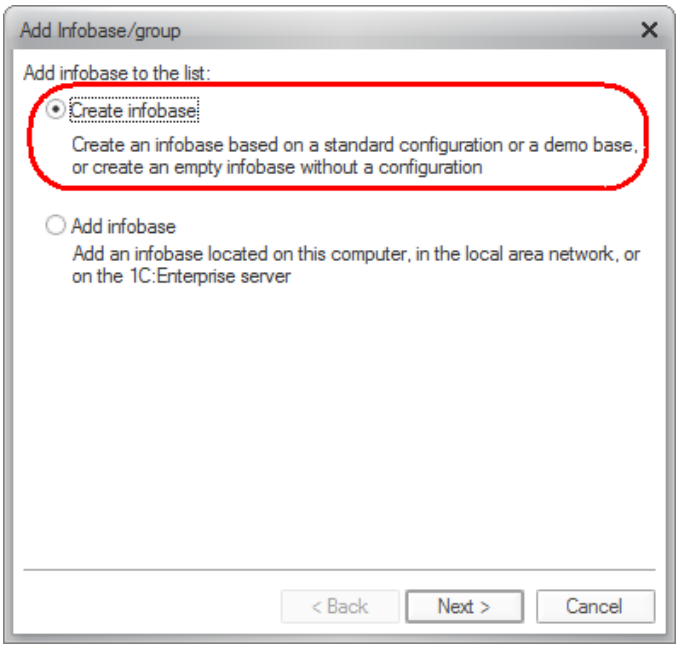


Fig. 1.3. Creating an infobase, step 2

4. Click **Next**, then click **Create an infobase without a configuration**... (fig. 1.4).

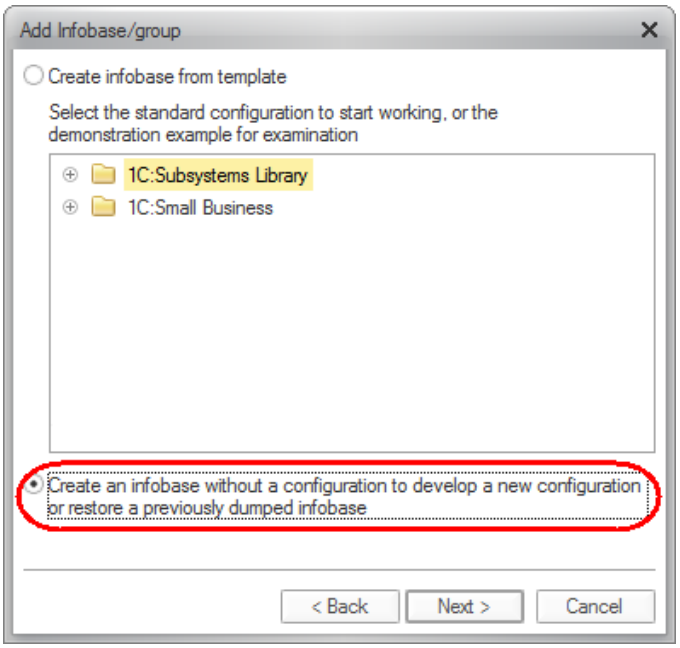


Fig. 1.4. Creating an infobase, step 3

5. Click **Next**, type the infobase name, and click **This computer or LAN computer** for its location type (fig. 1.5).

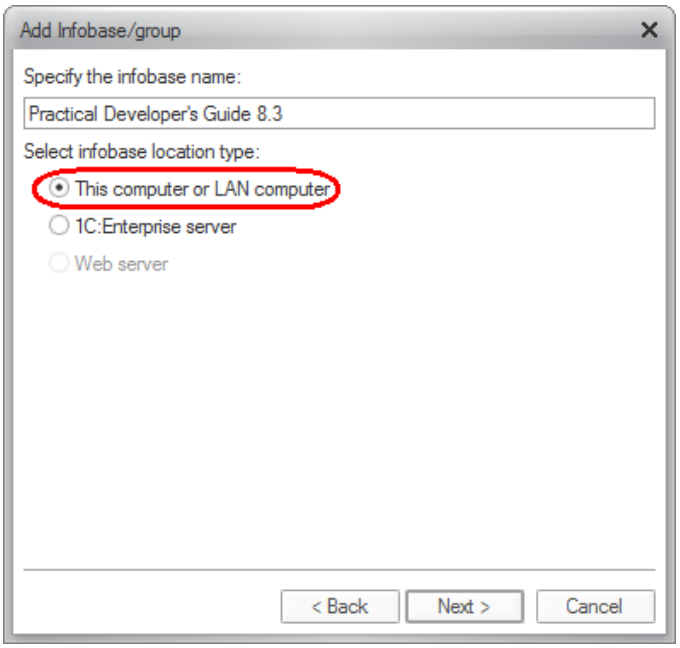


Fig. 1.5. Creating an infobase, step 4

6. Click **Next** and specify the infobase directory. Ensure that English is the default language (fig. 1.6).

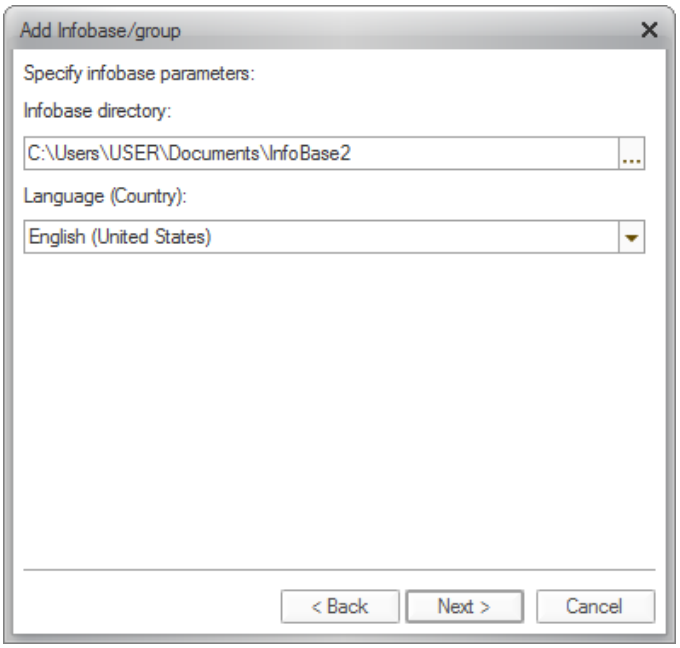


Fig. 1.6. Creating an infobase, step 5

7. Click **Next**, then click **Finish** (fig. 1.7).

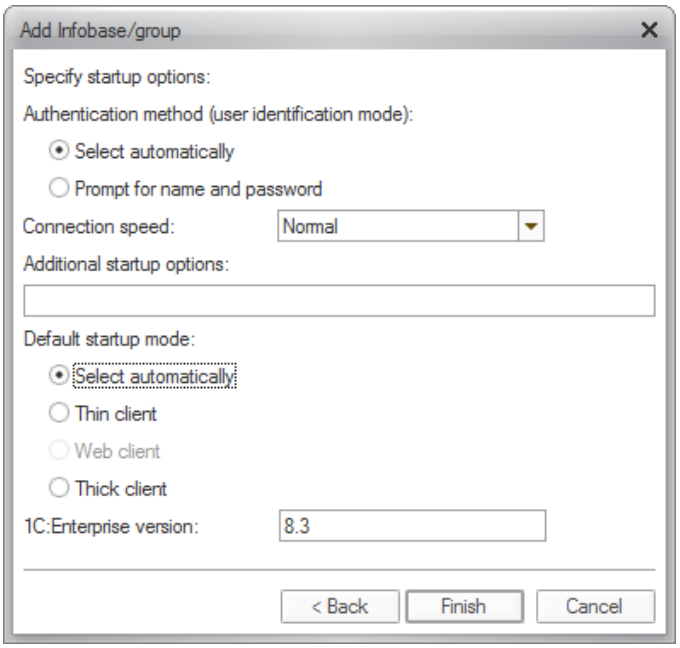


Fig. 1.7. Creating an infobase, step 6

The empty infobase that you created is now added to the list of infobases in the 1C:Enterprise startup window (fig. 1.8).

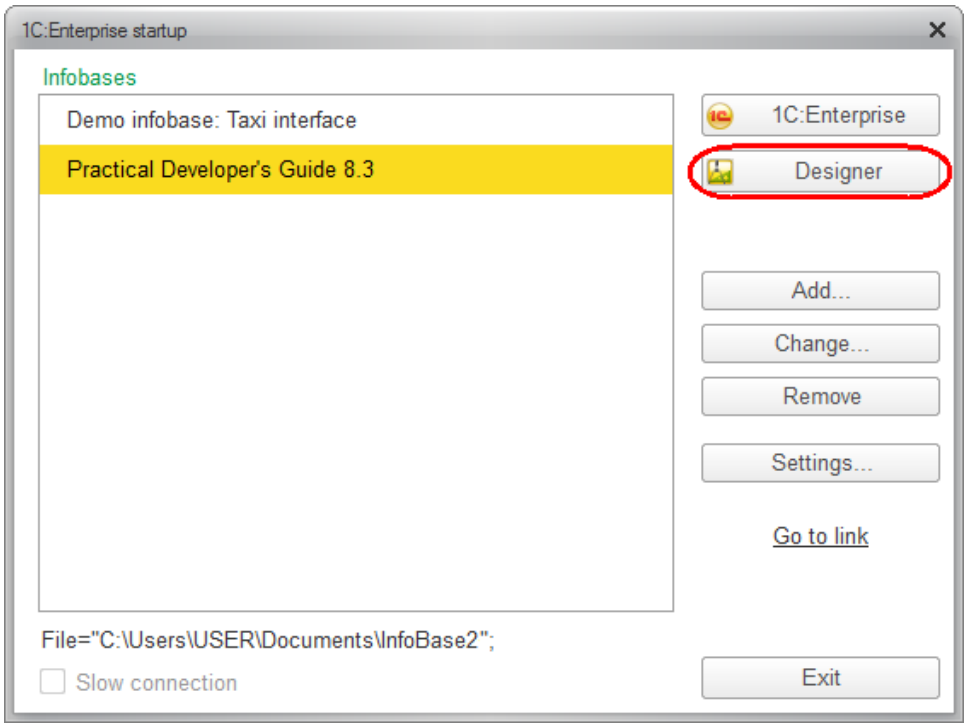


Fig. 1.8. Starting 1C:Enterprise in Designer mode

In Designer mode

Introduction to Designer

Now let us start 1C:Enterprise in Designer mode.

- In the **1C:Enterprise startup** window, select the newly created infobase, and click **Designer** (see fig. 1.8).

The **Designer** window is displayed (fig. 1.9).

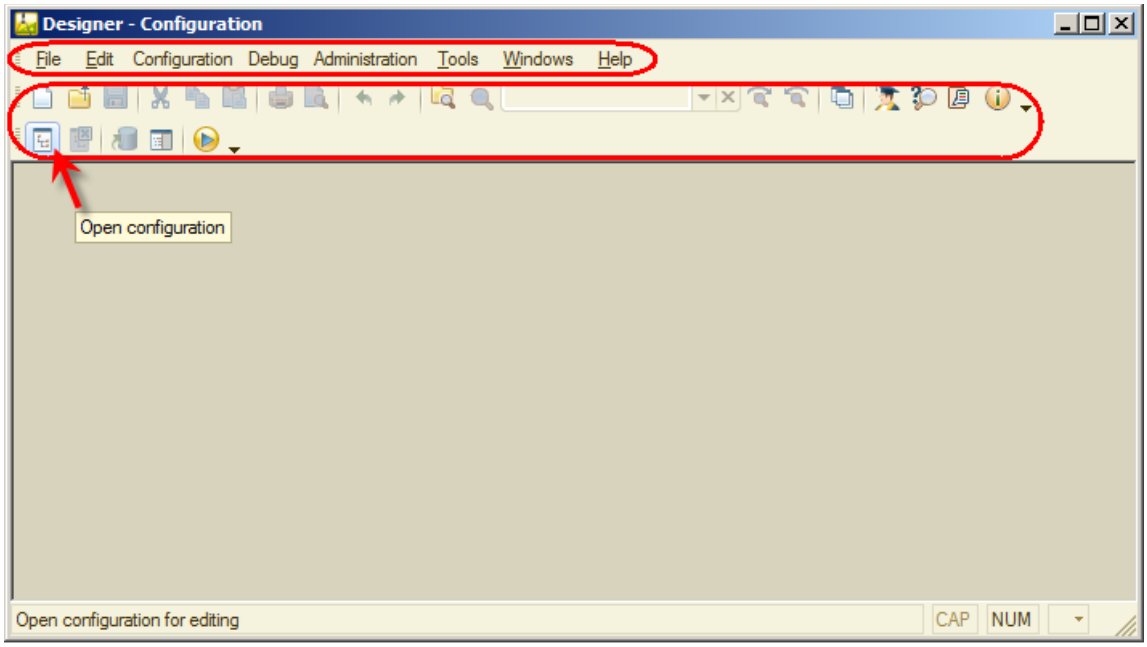


Fig. 1.9. Designer window

This is the tool you will use to create your applied solution. The Designer menu bar is located right below the window title. It includes menus: File, Edit, Configuration, Debug, and more. Each menu contains commands that perform various actions in Designer.

The Designer toolbar is located under the menu bar. The toolbar contains the icons for accessing frequently used actions that are also available in the menus.

Hence, many common actions can be performed using any of the two methods: by selecting a menu command or by clicking a toolbar button.

A large number of unfamiliar icons often confuse a novice developer. But there is nothing to worry about. In time, you will be able to distinguish them easily. Simply point to any button to view a tooltip that describes the button function (see fig. 1.9).

Chances are that you will initially prefer the menus but eventually you will begin to use the toolbar more because it saves clicks. You have the option to customize the toolbar by adding or removing buttons (fig. 1.10).

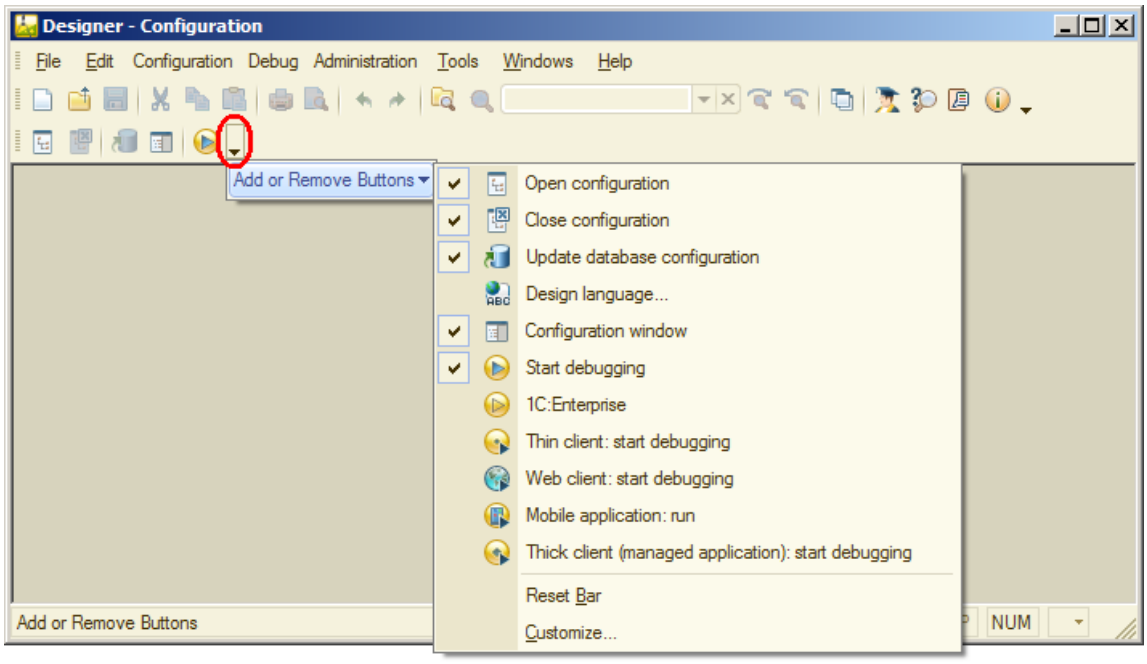


Fig. 1.10. Customizing Designer toolbar

Configuration object tree

To be able to perform any operations on your configuration, you have to open it first.

- On the **Configuration** menu, click **Open configuration**.

  -OR-

  Click the **Open configuration** button on the toolbar (see fig. 1.9).

  This opens the configuration object tree (fig. 1.11).

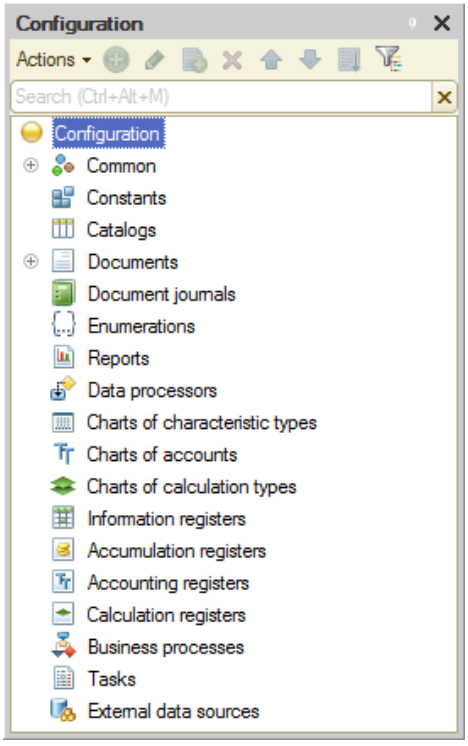


Fig. 1.11. Configuration object tree

The configuration object tree is an important developer tool. It contains virtually all the information about the configuration structure.

You may already be asking yourself: why does the tree contain items before you have created any?

The answer is that, for developer convenience, all configuration items are combined into groups, and at this point the tree contains these groups. If you run along the tree and click the ⊕ icons, you will find that all the groups are empty. The only exception is the Common / Languages group where you can see the English item. The platform created that item on its own because, in this case, Designer is using the English interface.

You are ready to begin actually doing things, but first we need to go over some terminology. You may already have noticed that when talking about the contents of the configuration we have consciously avoided using specific terms. Now is a good time to define some terms and talk about configuration objects.

Understanding configuration objects

A configuration is essentially a definition. It defines the data that users can access in 1C:Enterprise mode.

In addition, a configuration defines all of the data processing operations available. Besides, a configuration defines how the data is displayed on screen and in printouts, and so forth. The 1C:Enterprise platform then uses this definition to create a database that has the appropriate structure and provides user access to the data.

To ensure quick and easy customization of 1C:Enterprise for specific applied solutions, all of the definitions stored within a configuration are represented as logical items named *configuration objects*. Perhaps you have had a chance to flip through 1C:Enterprise. Developer Guide, which provides a brief definition of a configuration object.

We will not repeat that definition in this book, since our task is not to present the strategies behind the 1C:Enterprise system design as metadata structures focused on business essentials, but to teach you how to use 1C:Enterprise features correctly and efficiently.

Therefore, we will explain the nature of configuration objects at the mundane level. This explanation will nonetheless allow you to understand the role of objects in the tasks that you will solve in this tutorial.

On the one hand, configuration objects are the pieces of a set of building blocks the configuration is assembled of. A set of building blocks normally comes with a certain assortment of pieces. The pieces can be different: long, short, rectangular, square, and so on. Now imagine that you can make as many of each type of pieces as you need (say, 5 long pieces and 3 short ones). You can connect the pieces to each other in various ways.

The same is true for configuration objects. You can only create configuration objects of certain types. However, you can create as many objects of each type as you need. The difference between objects of different types is that they have different properties (to be exact, different sets of properties). Objects can interact with each other and you can define these interactions.

What else makes configuration objects similar to pieces in a set of building blocks? A set of building blocks usually includes pieces that can be attached to each other, as well as other pieces, such as wheels, that cannot be attached to each other, but can be connected to an axle so that the wheel will be able to rotate. In other words, different pieces of a single set behave in a different manner.

Likewise, configuration objects have different behaviors, and that behavior depends on the object type. Some objects can perform certain actions, while others cannot do so, but have their own specific set of actions.

We will explain another characteristic of configuration objects using a car as an example. A car consists of a large number of parts. An engine is one of those parts. In turn, however, an engine also consists of a set of parts, while a single part can be used in different engines.

In the same way, "complex" configuration objects consist of "simple" objects and the same "simple" objects may be used to make up "complex" objects. This structure simplifies the work with configuration objects, since, if we know how a certain "simple" object works, we will deal with it in the same manner when it is a part of a "complex" object.

And finally, the most important property of a configuration object is its practical application. Configuration objects are not merely abstract constructs the developers use to describe a task at hand. Rather, they correspond to the actual objects involved in company operations.

For example, all companies use various documents that describe the facts of business transactions. In the same manner, a configuration contains objects of the Document type.

Additionally, all companies must maintain lists of employees and catalogs of goods or services. Configurations have objects of the Catalog type, which allow developers to create digital versions of these lists.

As mentioned above, based on the configuration objects, the platform creates database tables that store data. In the documentation, as a rule, a single name is used for describing a configuration object and the corresponding set of database tables.

For example, if a configuration contains the Employees catalog object, the set of tables created by the platform based on this configuration object is also referred to as the Employees catalog.

We will stay away from that sort of "fuzzy" writing style, and whenever we are talking about the configuration, we will use the unambiguous term: the Employees catalog configuration object. In those places where we are talking about the database, we will simply refer to the Employees catalog.

Adding configuration objects

Before you begin adding your first configuration objects, note that in order to develop a configuration that automates a company’s economic activity, a developer can use only a limited set of configuration objects that is built into the platform. A developer cannot create custom configuration objects. They only can add standard objects available in the system to a configuration.

Before you begin, you have to learn how to perform some basic actions in Designer.

To open a configuration

- On the **Configuration** menu, click **Open configuration**.

To close a configuration

- On the **Configuration** menu, click **Close configuration**.

Respective toolbar buttons are available as well.

Once the configuration is open, its content is displayed in the configuration object tree window (see fig. 1.11). You can close this window just like any other window, but if you do so, the configuration itself remains open (in other words, it can still be edited).

To view the configuration tree window

- On the **Configuration** menu, click **Configuration window**.

There are several methods of adding configuration objects. You can use any method that you like.

To add configuration objects (method 1)

- Click the required configuration object tree branch, click the **Actions** button in the configuration window command bar, and then click **Add** (fig. 1.12).

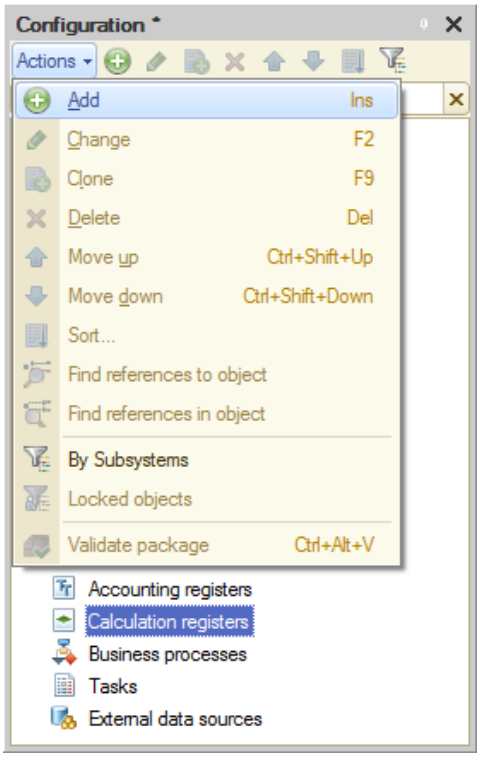


Fig. 1.12. Adding a configuration object

To add configuration objects (method 2)

- Right-click the required configuration object tree branch, then click **Add** (fig. 1.13).

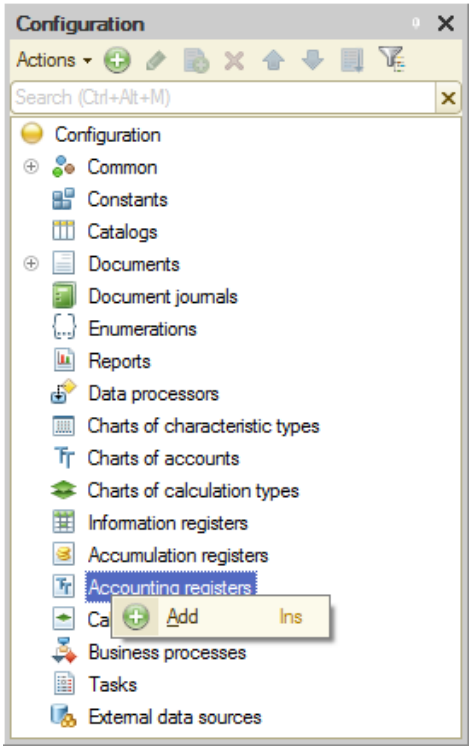


Fig. 1.13. Adding a configuration object

To add configuration objects (method 3)

- Click the required configuration object tree branch, and then click the **Add** button in the configuration window command bar (fig. 1.14).

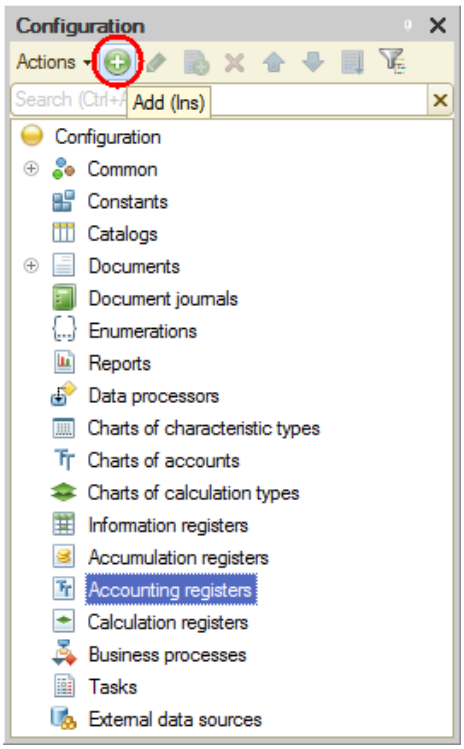


Fig. 1.14. Adding a configuration object

We prefer the last method so we will mainly use it in the future.

Property palette

So, let us begin.

Let us define a name for our configuration and use this example to introduce the property palette. Developers can use the property palette to define various properties of configuration objects that they create.

A property palette is a dedicated auxiliary window that is intended for editing configuration object properties and other related data. Since different configuration objects have very different properties, the content of this window varies depending on the selected configuration object.

1. In the configuration object tree, double-click the **Configuration** root item.

   This opens the configuration property palette.

2. In the **Name** field, type *BeginnerGuide*.

   The synonym is filled automatically (fig. 1.15) but you can specify it explicitly if needed. The synonym serves as a title for the main applied solution window.

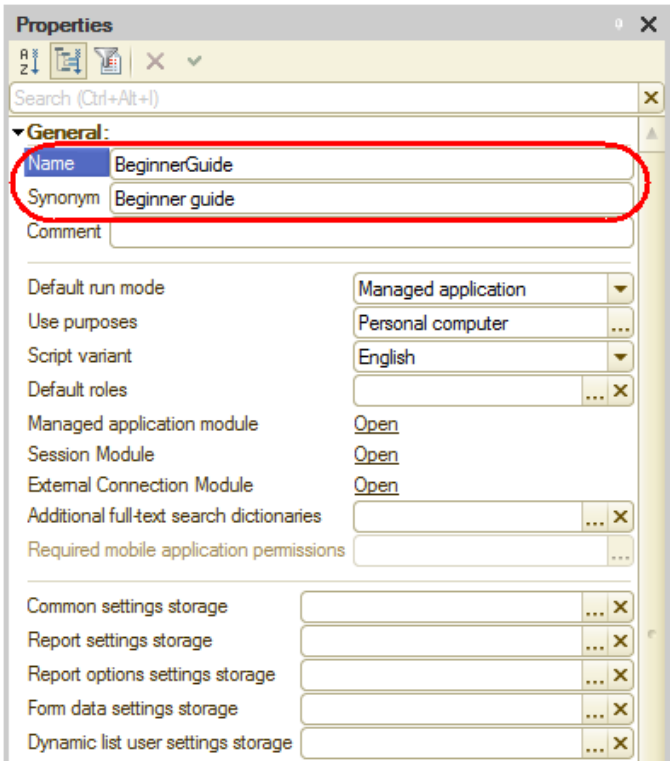


Fig. 1.15. Configuration property palette

Important! This tutorial is designed and thoroughly tested using 1C:Enterprise version 8.3.5. It includes a few synchronous method calls. In 1C:Enterprise versions 8.3.6 or later synchronous method calls are disabled by default.

3. If you are using 1C:Enterprise version 8.3.6 or later, scroll down the property palette to find the **Synchronous call usage mode for extensions and add-ins** property and change its value to **Use**.

Sometimes the property palette is opened automatically in response to certain developer actions. But you can always open the property palette manually.

To open a property palette

- Right-click an object and then click **Properties**.

In this case the property palette is opened in a similar manner and docks within the Designer workspace. It means that when you select any configuration object, its property palette is always opened. However, there is a handy option to undock the property palette using the push-pin icon in the title of the property palette window (fig. 1.16).

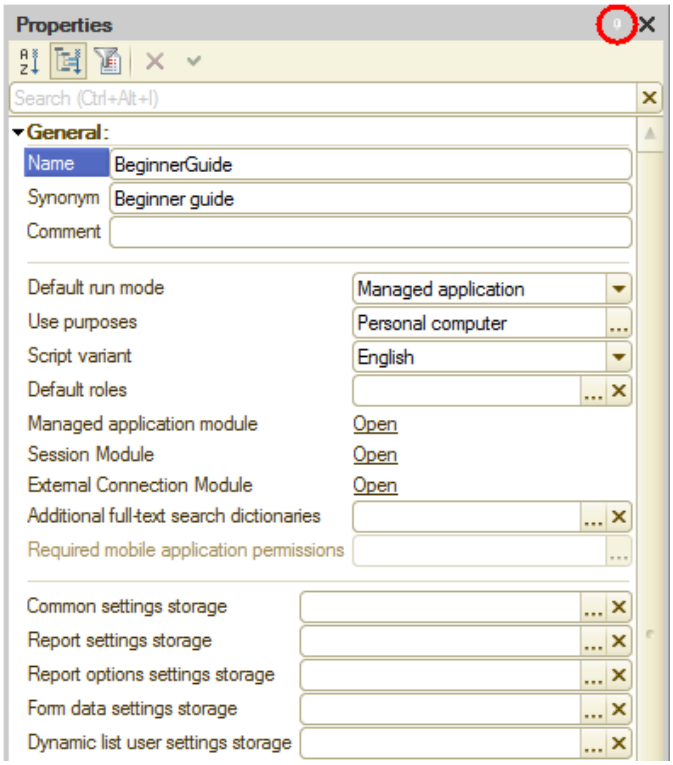


Fig. 1.16. Undocking the property palette

In the undocked state, when you move the pointer to any other window, the property palette is minimized to a button near the right edge of the Designer window (fig. 1.17).

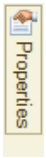


Fig. 1.17. The property palette button

When you point to the button, the property palette is opened up again. In addition to the property palette, other Designer windows (such as the configuration object tree window) behave in a similar way (they can be docked, hidden and so on).

Debugging in 1C:Enterprise mode

Now let us check your first changes in 1C:Enterprise mode.

1. On the **Debug** menu, click **Start debugging**.

   -OR-

   On the Designer toolbar, click the **Start debugging** button .

   The platform checks whether the configuration was changed and prompts you to update the database configuration (fig. 1.18).

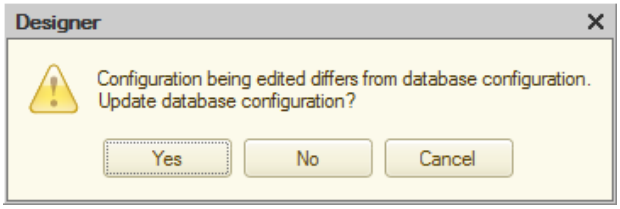


Fig. 1.18. Database configuration update dialog box

   At this point we will not discuss what it means as it will be explained in section "Base configuration and database configuration" on page 109.

2. Click **Yes**.

   Then the 1C:Enterprise window is displayed (fig. 1.19).

In 1C:Enterprise mode

Applied solution interface appearance

The window title displays the configuration name. The empty space is the work area of the application. This area is not yet filled with any items.

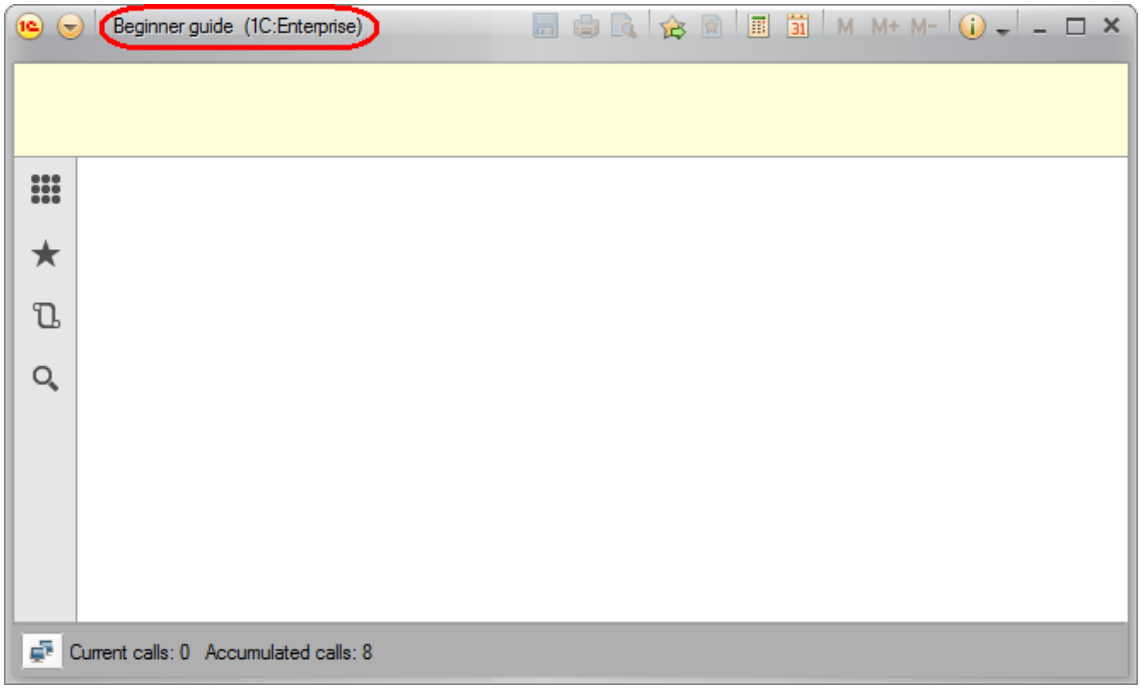


Fig. 1.19. 1C:Enterprise mode

Note. Besides the title, the application window contains the area of system commands, which are available in all applications. They are intended for opening the main application menu, opening the **About** window, printing, and so on.

Currently nothing is displayed in the 1C:Enterprise window except the configuration title. And this is exactly what we should have expected.

You have created neither any configuration objects, nor subsystems to display such objects yet. The subsystems as the basis of the 1C:Enterprise interface development are discussed in the next lesson. For now, take a quiz to test your knowledge and see how comfortable you are with the material that has been discussed.

Quiz

- What is configurability of the 1C:Enterprise system?
- What are the major components of the 1C:Enterprise system?
- What is a platform and what is a configuration?
- What are the two modes of running 1C:Enterprise?
- What is the configuration object tree?
- What are configuration objects?
- What does the platform create based on configuration objects?
- How can one add a configuration object?
- What is the property palette used for?
- How does one start 1C:Enterprise in the debug mode?

Lesson 2 (0:45). Subsystems

In this lesson you will study the Subsystem configuration object, which is the basis for declarative description of the 1C:Enterprise 8 interface.

You will create multiple subsystems that define the logical structure of your applied solution. You will also customize their appearance and order in the applied solution interface.

Understanding subsystems

Subsystems serve as primary 1C:Enterprise interface building blocks. Hence, the first thing you do when you begin to develop a configuration is specifying a set of subsystems.

Developers face a crucial task at this stage: they have to think over the assortment of subsystems carefully and then link the configuration objects being created to the subsystems while paying attention to all the details.

You do not have to use subsystems in simple applications but we will consider a generic scenario that includes subsystems.

Subsystem configuration objects are intended to describe functional configuration parts, which represent logical parts of the applied solution.

These objects are located in the Common object branch. You can create a tree-like structure that consists of subsystems and subordinate subsystems (fig. 2.1).

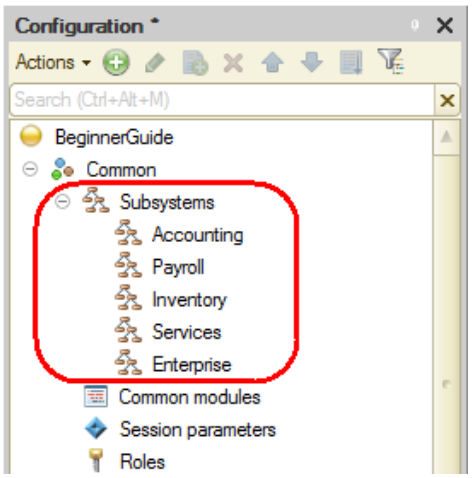


Fig. 2.1. Subsystem structure in a configuration

The top-level subsystems are major interface items because they represent applied solution sections (fig. 2.2).

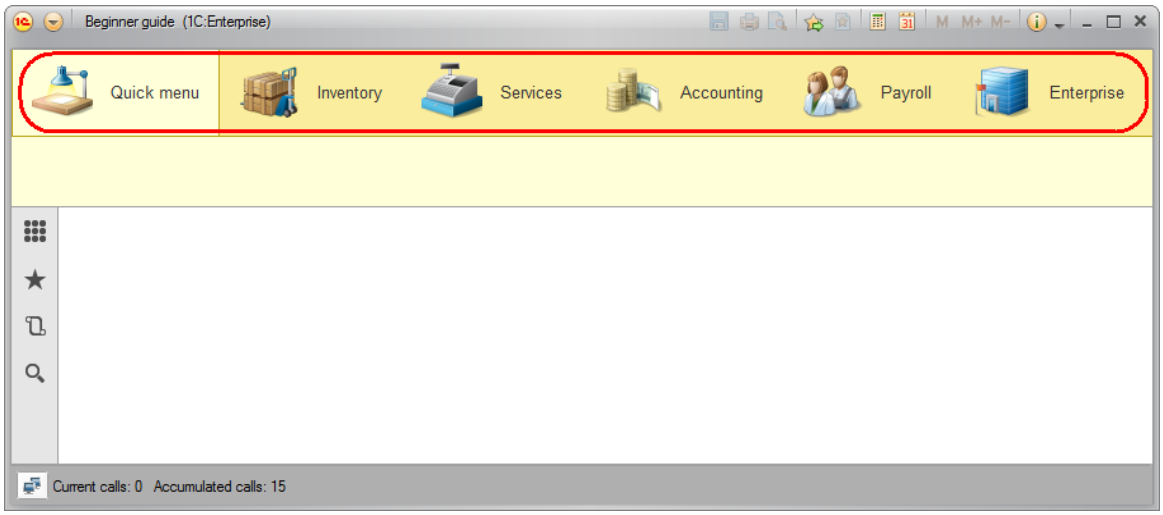


Fig. 2.2. Applied solution sections

Each configuration object can be included in one or several subsystems. In other words, it will be available in each of the selected subsystems.

Looking ahead, we will mention that subsystems combined with visibility by roles allow developers to provide users with an easy-to-use and feature-rich interface free from any unnecessary items. For example, warehouse clerks need the ability to accept and issue products, but they have no need for anything related to accounting or service provisioning.

Hence, subsystems determine the applied solution structure, arrange the entire user interface, and enable sorting various documents, catalogs, and reports by logically related categories where users can find them easily. Note that users will only see the sections (i.e. applied solution features) that they need for their work.

Even for a small applied solution like yours you can define several functional components that represent distinct subject areas.

You can put everything related to accounting into a single subsystem, and everything related to payroll calculation to another subsystem.

The production activities of the Jack of All Trades, Ltd. company can be divided into recording of materials and rendering of services, which gives you two more subsystems.

And the need to perform administrative database operations suggests adding another subsystem that will only be available to the administrator.

So you will need five Subsystem objects with the following names: Accounting, Payroll, Inventory, Services, and Enterprise.

Adding subsystems

In Designer mode

Let us create the subsystems.

1. Close the application and return to Designer.

2. In the configuration object tree, expand the **Common** branch by clicking the ⊕ icon to the left of the branch name.
3. Right-click the **Subsystems** branch and then click **Add**.

   -OR-

   In the command bar of the configuration window, click the **Add** button (fig. 2.3).

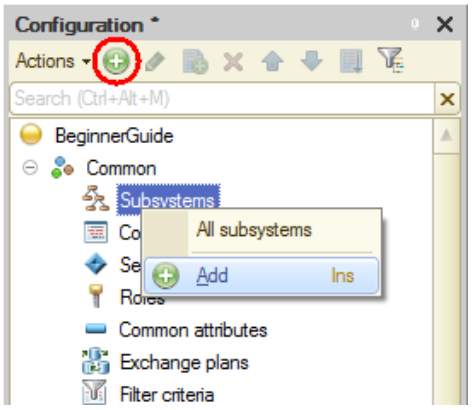


Fig. 2.3. Adding a subsystem to the configuration object tree

This opens the configuration object editor window. The editor provides step-by-step guidance for quick creation of complex configuration objects.

The **Next** and **Back** buttons in the bottom part of the editor guide you through the steps. When you move through the wizard pages by clicking **Next**, you set object properties in the right order (so that you do not skip over to a location where data from a previous step is needed). If you find that you missed something or entered incorrect data, use the **Back** button.

You can also move between the pages by clicking the tabs (for example, you can fill the object properties on the **Data** tab). When you open the configuration object editor window, the **Main** tab is selected.

Note. You might want to open the configuration object editor window later for editing object properties. To open the configuration object editor window, in the configuration object tree, select an object and then, in the configuration window command bar, click the **Change current item** button, or select the object and press F2, or double-click the object.

In the **Name** field, enter *Accounting*.

The platform automatically generates the *Accounting* synonym based on the name (fig. 2.4).

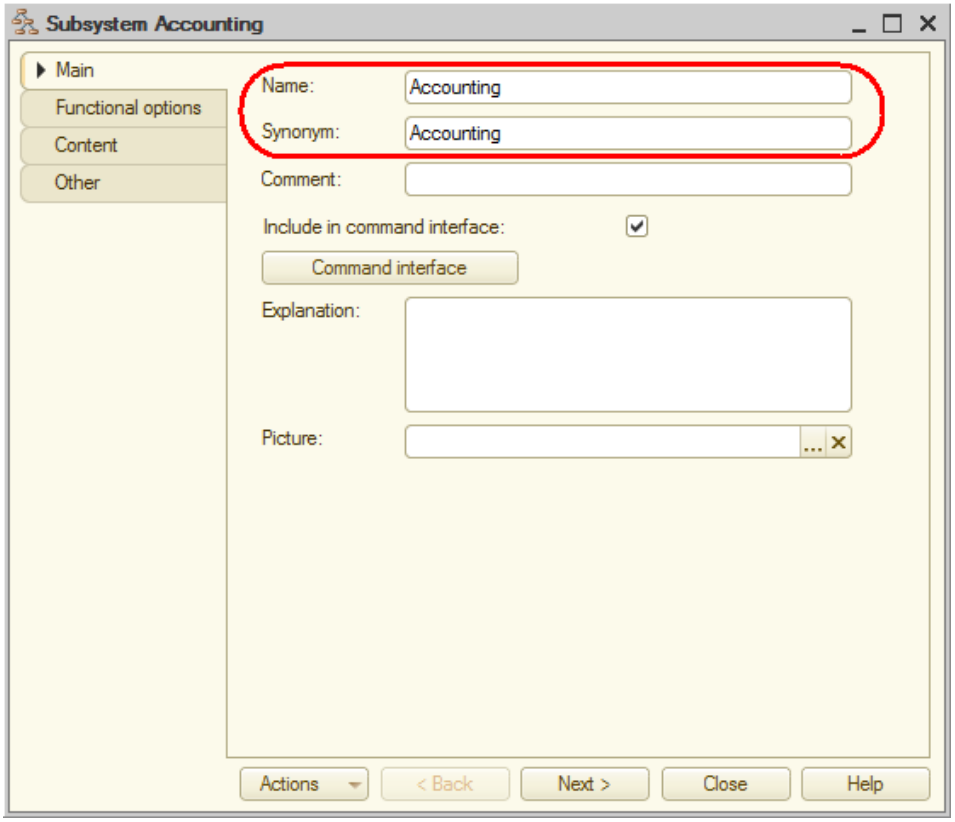


Fig. 2.4. Defining subsystem name and synonym

Configuration object name and synonym

The name is the main property of any configuration object. When you create an object, the platform generates its name automatically.

You can keep the generated name but we recommend that you change it to a name that makes sense to you. You can choose any name, provided that it begins with a letter and does not contain certain special characters (such as spaces).

It is a common practice to use human-readable names and, in case of names that consist of multiple words, to take out the spaces and begin each word with a capital letter. Object names are unique and you can use them to call object properties and methods from 1C:Enterprise script.

Each configuration object also has the Synonym property. It stores an alternate description of a configuration object. This description is displayed in the user interface. In other words, it is what users will see. That is why there are almost no limitations to the synonym format and you can always make a synonym human-readable.

Adding subsystem icons

Let us enhance the applied solution interface by adding subsystem pictures.

1. In the **Picture** field, click the selection button ... (see fig. 2.4).
2. In the **Select picture** window, on the **From configuration** tab, click **Add** (fig. 2.5).

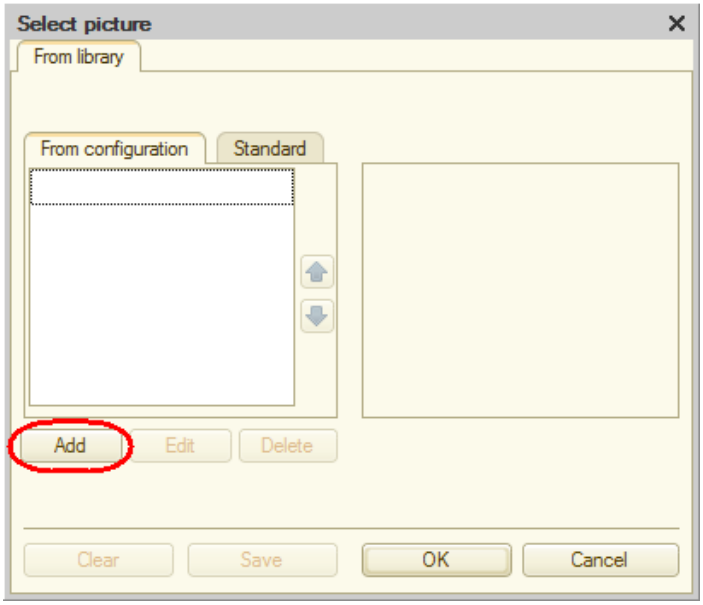


Fig. 2.5. Adding a subsystem picture

The platform creates a Common picture configuration object and opens its editor window.

3. In the **Name** field, enter *Accounting*.
4. Click **Load from file** (fig. 2.6).

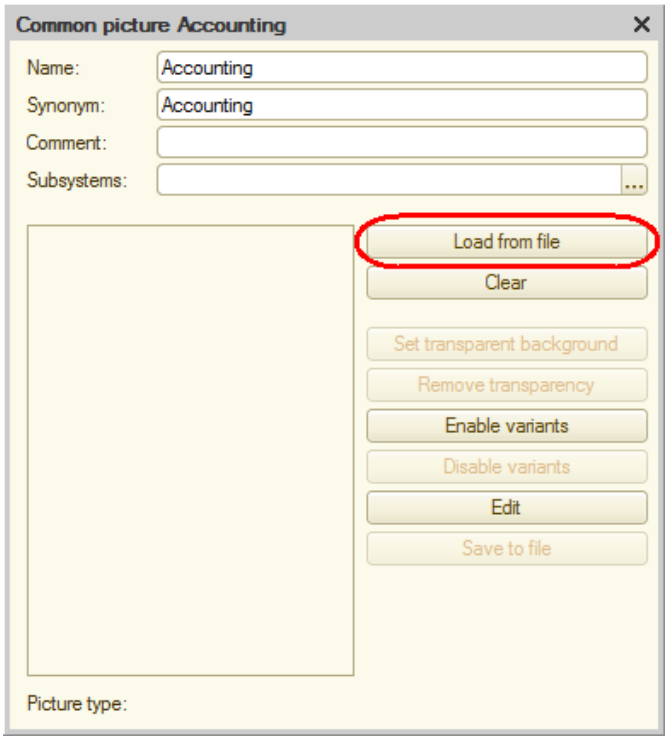


Fig. 2.6. Configuration object editor of a common picture

5. Browse to the **Pictures** folder in the book distribution kit, select the Accounting.png file, and click **Open** (fig. 2.7).

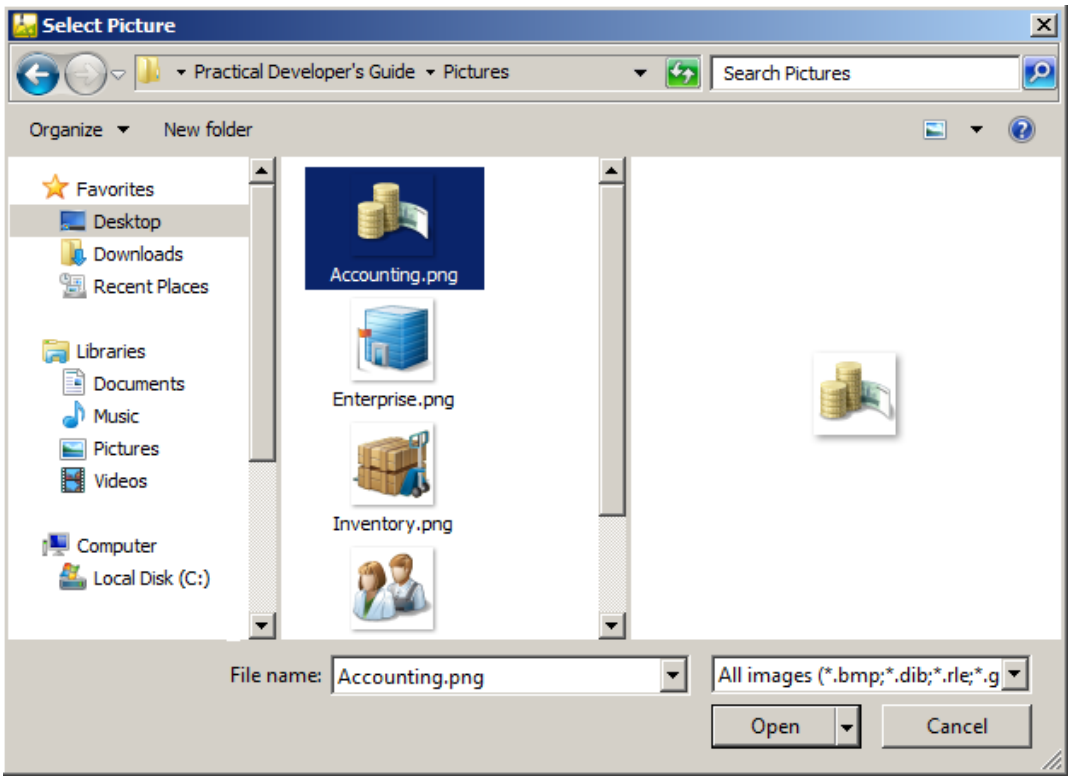


Fig. 2.7. Selecting a subsystem picture

The selected picture is displayed in the common picture editor window.

6. Close the Common picture configuration object editor.

   You can see that the list in the **Select picture** window now contains the picture that you added.

7. Click **OK** (fig. 2.8).

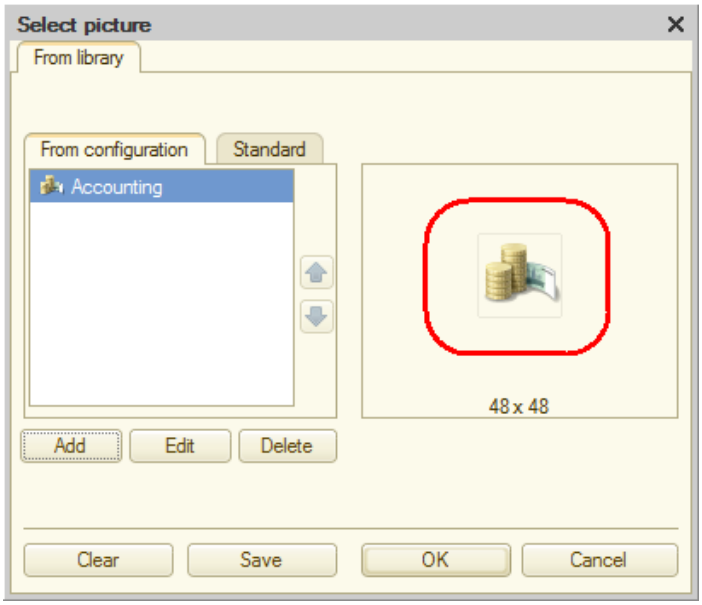


Fig. 2.8. Selecting a subsystem picture

Once you are done, the Accounting picture is added to the Common pictures branch of the configuration object tree. You can edit this picture and use it in your configuration (fig. 2.9).

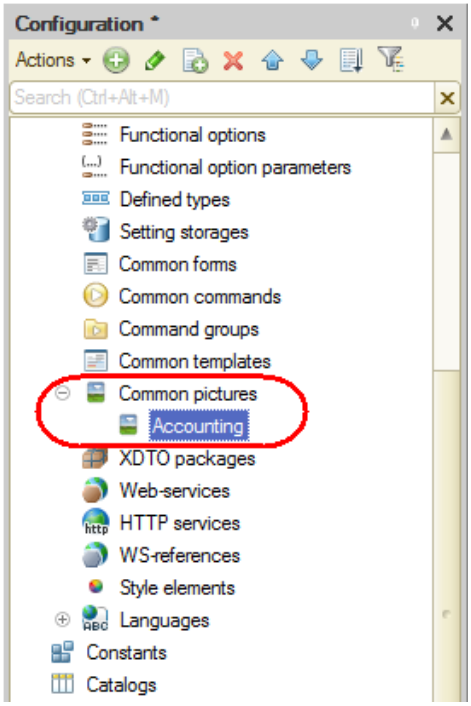


Fig. 2.9. Common pictures in the configuration object tree

When you return to the Accounting subsystem configuration object editor, you can see that the picture you selected is now assigned as the subsystem picture (fig. 2.10).

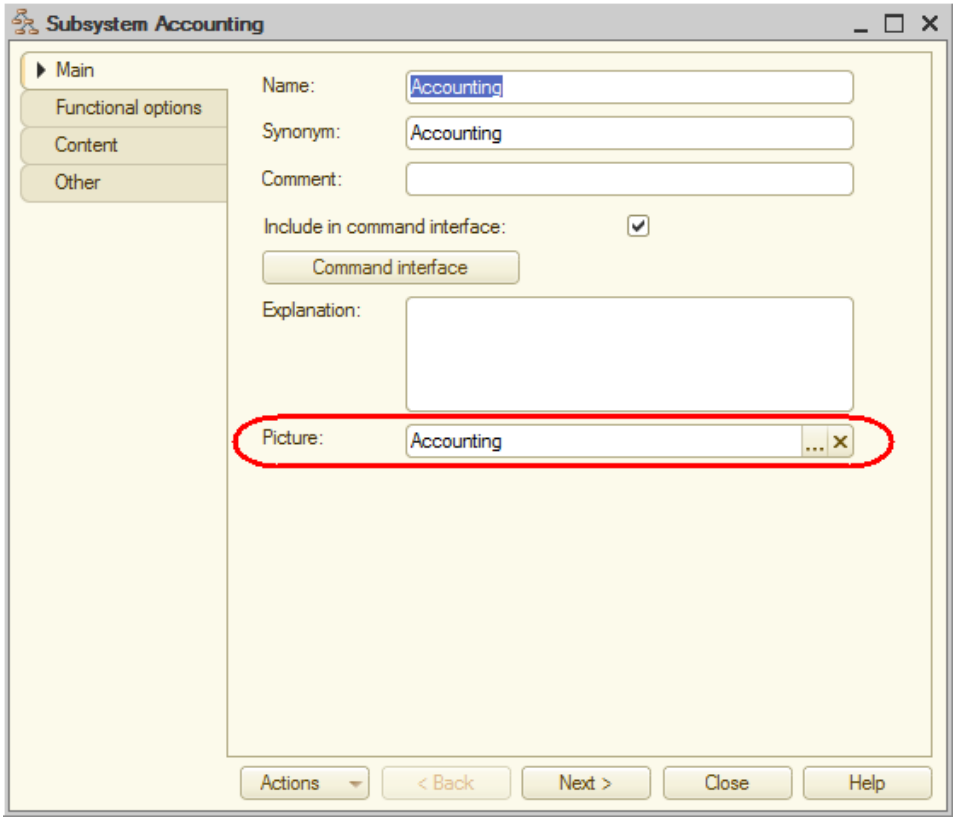


Fig. 2.10. The selected picture is assigned to the subsystem

In the 1C:Enterprise interface the subsystem synonym serves as the section name, while the picture is displayed near that name.

If a subsystem does not have a picture, the section is displayed in the interface with the default picture.

8. Click the **Subsystems** branch and then, in the configuration object tree, click the **Add** button to create the **Inventory** and **Services** subsystems. Assign the **Inventory** and **Services** common pictures to these subsystems (you have to load them from the Inventory.png and Services.png files first) using the same procedure that you used for the Accounting subsystem.

   Now let us use another method of adding subsystems.

9. Right-click any of the available subsystems and point to **Add**.

   This menu item includes two subitems. Clicking **Subsystem** adds a subsystem of the same hierarchy level as the selected one. Clicking **Subordinate Subsystem** adds a subsystem that is subordinate to the selected one (fig. 2.11).

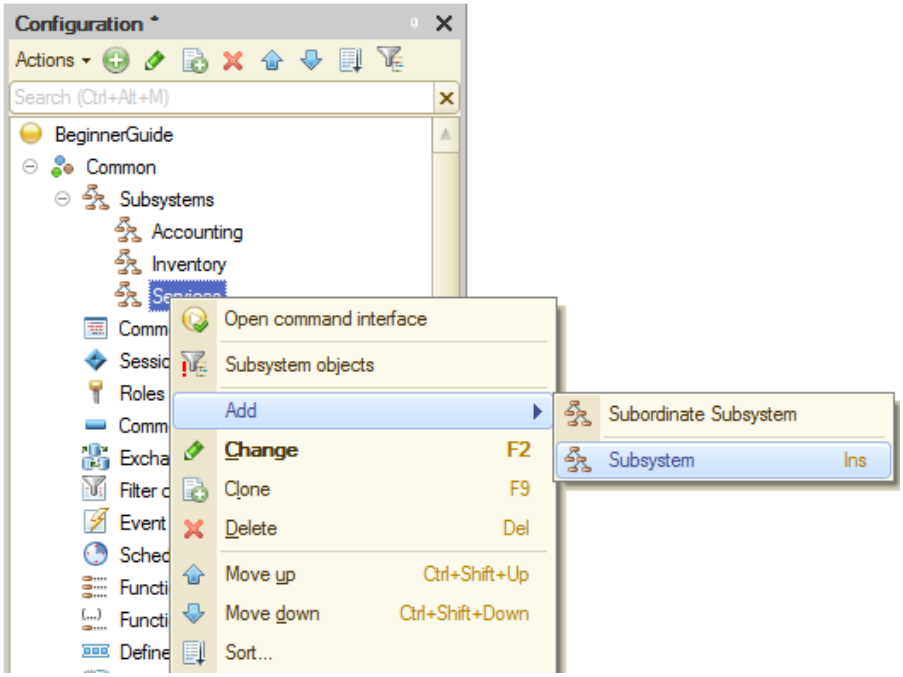


Fig. 2.11. Adding a subsystem

Since we are not planning to have a complex multilevel structure in this configuration, the first option will do.

10. Click **Subsystem** and add the **Payroll** subsystem. Assign the **Payroll** picture to this subsystem (you have to load it from the Payroll.png file first).

11. Add the **Enterprise** subsystem, and then add a picture to it.

Applied solution sections panel

In 1C:Enterprise mode

Start 1C:Enterprise in the debug mode and note the result of your modifications. The applied solution appearance has changed (fig. 2.12).

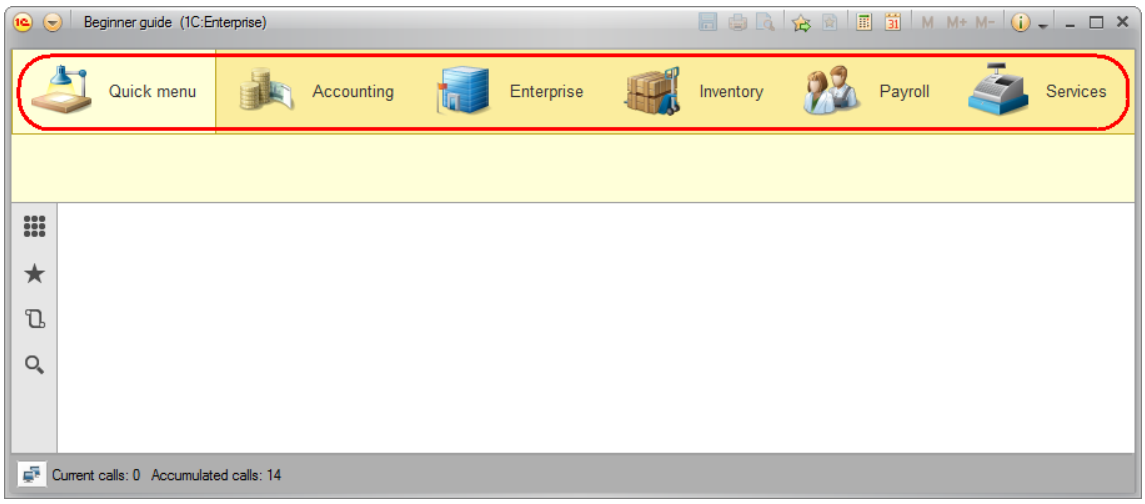


Fig. 2.12. 1C:Enterprise mode

Right under the window title and the system command area you can see the sections panel where all the created subsystems are displayed. All the sections have pictures that you selected in their properties.

The section names are hyperlinks. Clicking a hyperlink opens the documents, catalogs, reports, and other items included in the section. For now the sections are empty because you have not yet created any configuration objects to populate them.

Note. The Quick menu section is generated automatically. It is designed to store documents, reports, and other items that are frequently accessed by users.

Specifying section order

In Designer mode

The current order of subsystems is not optimal. Let us change it.

1. Close the applied solution and return to Designer.
2. In the configuration object tree, right-click the **BeginnerGuide** root item and then click **Open configuration command interface** (fig. 2.13).

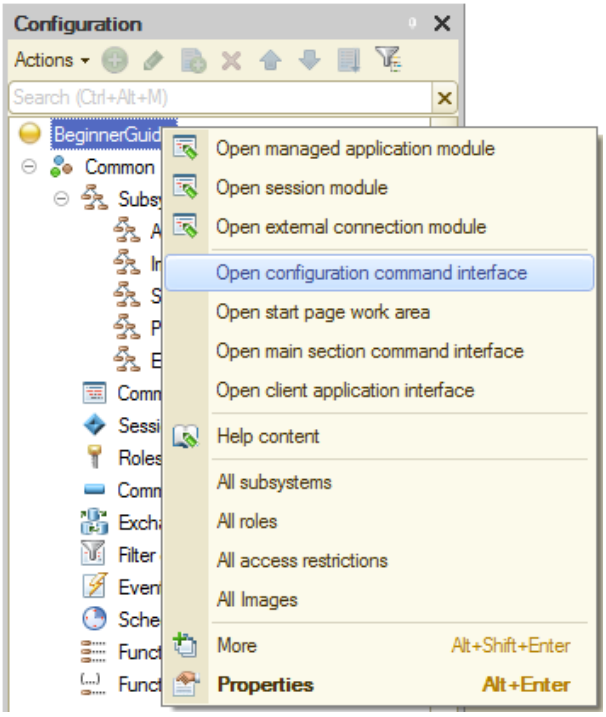


Fig. 2.13. Opening configuration command interface settings

The **Command interface** window displays the list of subsystems (applied solution sections). You can change the subsystem order using the **Move up** and **Move down** buttons.

3. Order the subsystems as follows. First **Inventory** and **Services**, which reflect the business activities of the company. Then **Accounting** and **Payroll**, and finally **Enterprise** (fig. 2.14).

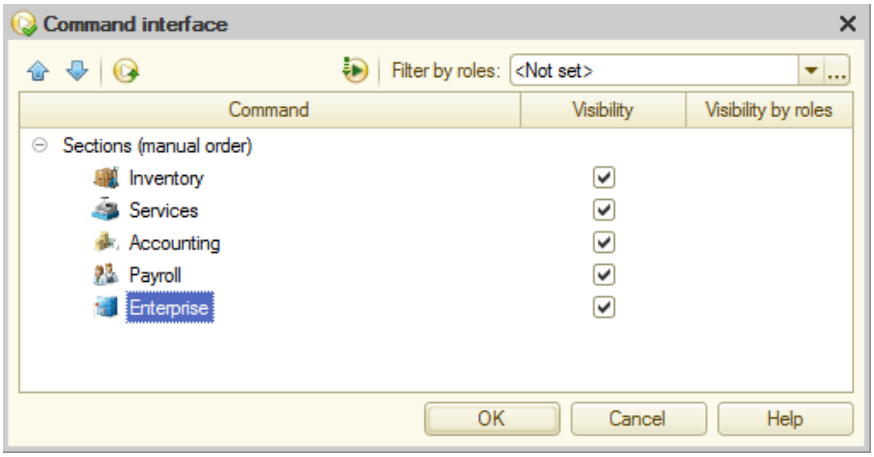


Fig. 2.14. Specifying subsystem order

In 1C:Enterprise mode

Start 1C:Enterprise in the debug mode and note that the order of subsystems in the sections panel has changed (fig. 2.15).

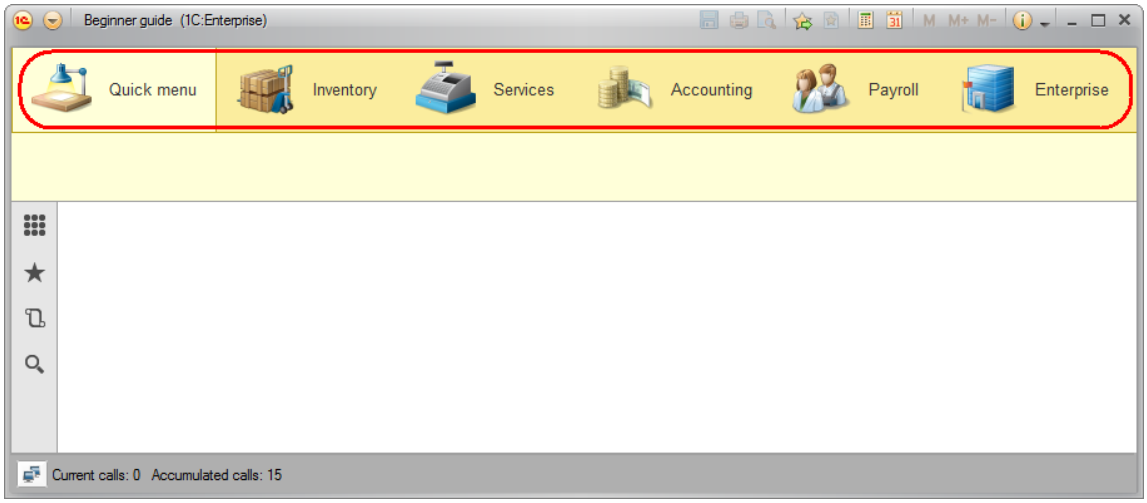


Fig. 2.15. 1C:Enterprise mode

Note. In 1C:Enterprise mode you also have the option to change the subsystem order using the main menu.

To change subsystem order

- Click in the upper left corner of the applied solution window, point to **View**, and click **Sections panel setup**.

  Note that the order of sections specified in Designer is available to all applied solution users, while the order of sections specified in 1C:Enterprise mode is only available to the user who made that change.

If the list of subsystems does not fit the sections panel width, a scrollbar is added to the panel.

In the next lesson you will create your first configuration objects, assign them to subsystems, and learn how they look in the 1C:Enterprise interface.

Important! Once you complete a lesson, we recommend that you create a configuration backup.

To back up a configuration

- On the **Administration** menu, click **Dump infobase**.

If you encounter any issues later, you can revert to an earlier version that does not yet have them.

To revert to an earlier version

- On the **Administration** menu, click **Restore infobase**.

Theory. Configuration object editor window and property palette

At first glance, the configuration object editor window and the property palette seem redundant. Indeed, the property palette displays all of the configuration object properties. So why create the object editor window on top of that? And if the object editor window is available, why do we need the property palette, which provides the same data in a different layout?

The configuration object editor window is primarily intended for quick object creation. This involves entering exhaustive information about the object and therefore requires a thorough knowledge of the object structure. Learning all that takes time, so how are we supposed to create an object quickly?

Easy enough. The object editor window is implemented as a wizard. It takes you through the proper sequence for entering the required data. The data entry sequence is designed so that previously entered data serves as the basis for entering new data. You can use the **Next** and **Back** buttons for navigation. At each step, you are prompted to enter a group of logically related items of information.

Now imagine that you are already comfortable with the object structure or you only need to change a few object properties. So not to "replay" the entire process from the very beginning, the object editor has tabs that let you go directly to the required step. Hence, the object editor window is handy for creating configuration objects and it provides easy access to specific properties that require editing.

But when it comes to the property palette, it provides an absolutely irreplaceable feature. Its structure is not tied to any specific configuration object type. So its content varies depending on the selected object. The property palette also "remembers" which property is selected, and when you move through the tree to another object, this property stays selected, while the property palette displays its value for the new object.

This feature of the property palette is necessary when, for instance, you have a few dozens of catalogs and you need to quickly find those that are subordinate to some particular catalog. In this case, you would click the Owner property in the property palette of any catalog. Then switch to the configuration object tree and simply scan through it using the Up Arrow and Down Arrow keys.

Quiz

- What is the Subsystem configuration object intended for?
- How does one define the configuration logical structure using Subsystem objects?
- How does one manage the order used to display subsystems in a configuration?
- What is a configuration object editor and how is it different from the property palette?

Lesson 3 (2:10). Catalogs

In this lesson you will study the Catalog configuration object. You will learn the purpose of this object, its structure, and its main properties.

Using practical examples, you will learn how to create catalogs, define their structure, and populate them with data.

Additionally, you will learn more about the Form configuration object, including form types available for Catalog configuration objects and the scenarios where these forms are used.

Finally, at the end of the lesson, we will discuss some theory related to configuration updates.

Understanding catalogs

The Catalog configuration object is intended for managing data lists. All companies have lists of employees, products, customers (clients), vendors (suppliers), and so on. Configuration objects of the Catalog type define the structure and properties of those lists. The platform uses Catalog objects to create database tables that store data available in the catalogs.

A catalog consists of items. For example, an employee is an item of the Employees catalog, while a product is an item of the Products catalog, and so on. Users can add catalog items, which can include adding new employees, creating new products, or entering new customers.

In a database each catalog item is an individual record in the main table, which stores data of that catalog (fig. 3.1).

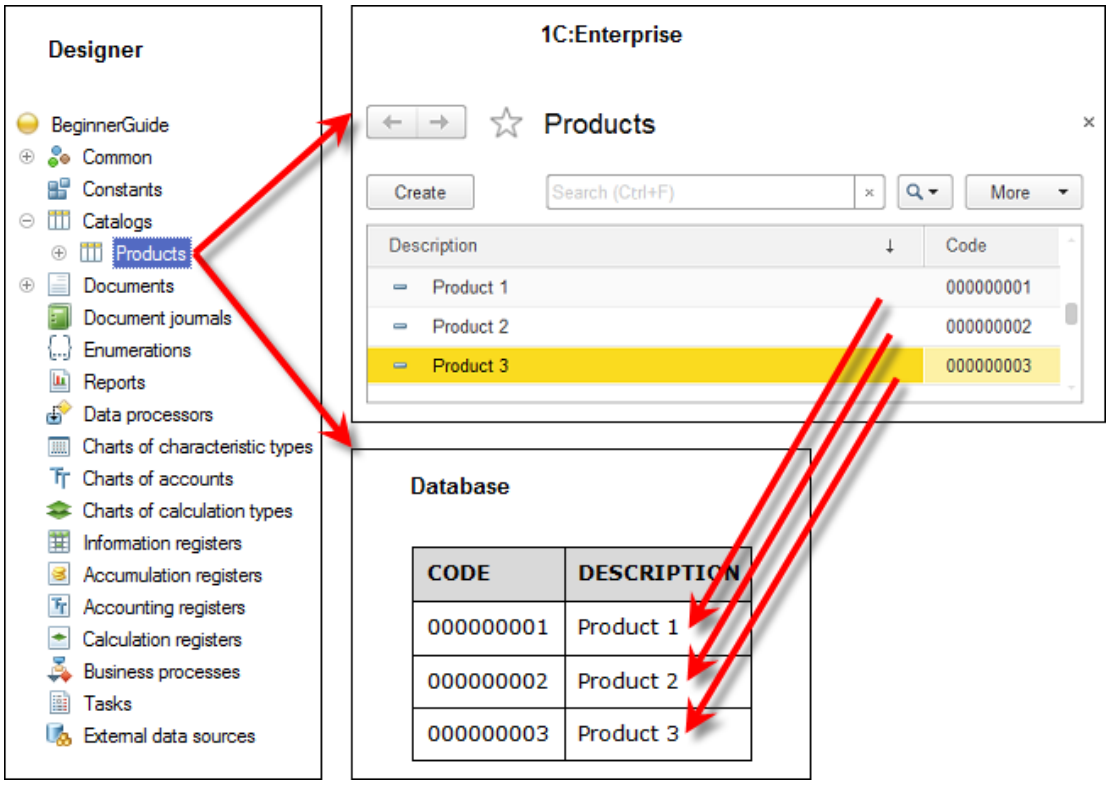


Fig. 3.1. Products catalog in Designer mode, in 1C:Enterprise mode, and in the database

Normally, each catalog item includes additional data that further defines the item. For example, all of the items in the Products catalog can contain additional information about the manufacturer, shelf life, and so on. This data set is identical for all the items in a catalog, and it is defined using the attributes of the Catalog configuration object, which are, in turn, configuration objects (fig. 3.2).

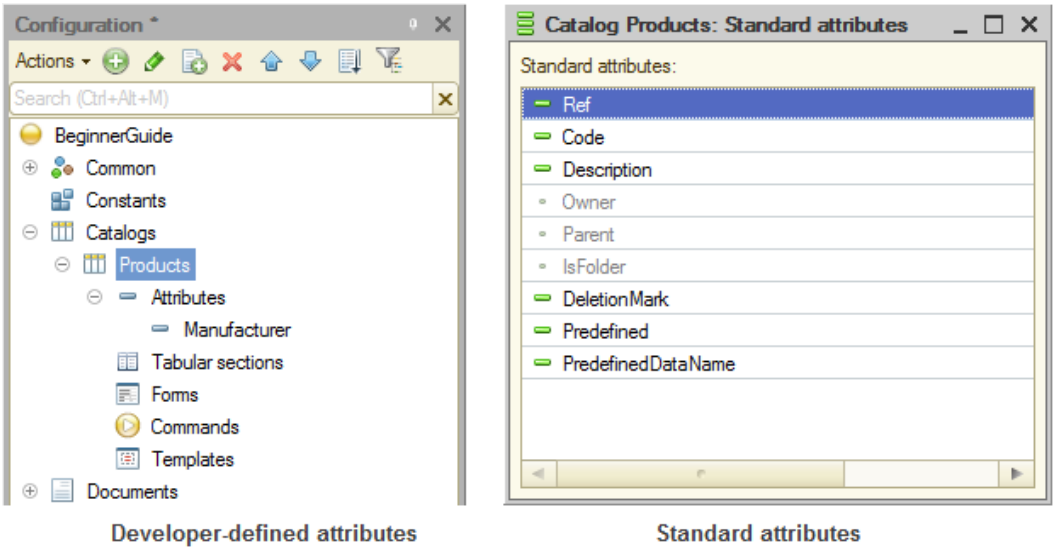


Fig. 3.2. Standard catalog attributes and developer-defined attributes

Since these configuration objects are logically connected to the Catalog object, they are referred to as subordinate to this object.

A developer creates the majority of attributes manually. However, each Catalog configuration object has a set of standard attributes: Code, Description, and more (see fig. 3.2). Note that the availability of standard attributes depends on the catalog properties.

For example, a hierarchical catalog has the Parent standard attribute available. If a catalog is subordinate to another configuration object, it has the Owner attribute. If you select zero for the length of the standard Code attribute, the catalog will not have this attribute available. The same is true for the Description attribute. But at least one of the two attributes (either Code or Description) should be available because otherwise the catalog makes no sense.

A catalog is stored in a database as a table, its rows store list items and columns represent attributes (either default ones or developer-defined). Hence, each table cell stores a value of a specific attribute for a certain catalog item (fig. 3.3).

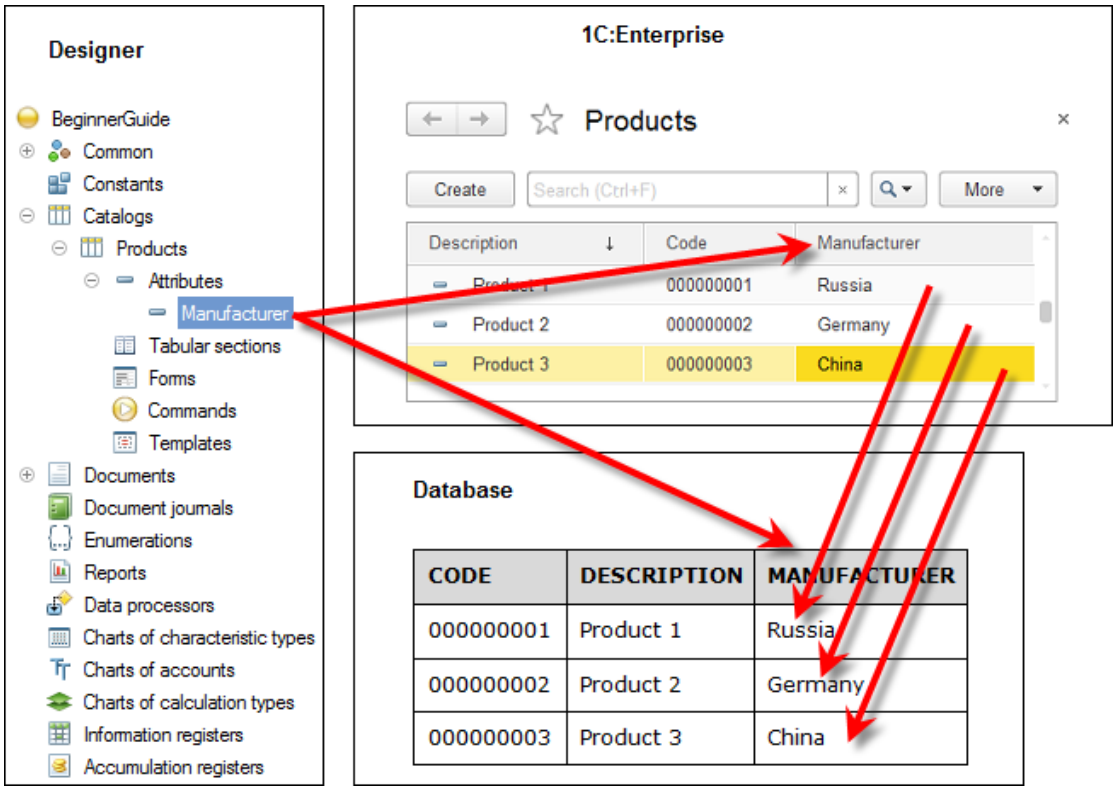


Fig. 3.3. Products catalog in Designer mode, in 1C:Enterprise mode, and in the database

Catalog items can also store certain data sets that have identical structure but vary in quantity (different quantities for different catalog items).

For example, each item of the Employees catalog can include information about employee family members. For one employee that is only a spouse, while another family can include a spouse, a son, and a daughter.

You can use the tabular sections of the Catalog configuration object to store such data. These sections are configuration objects subordinate to the catalog. In this scenario the database has additional tables for storing tabular sections subordinate to a specific catalog item (fig. 3.4).

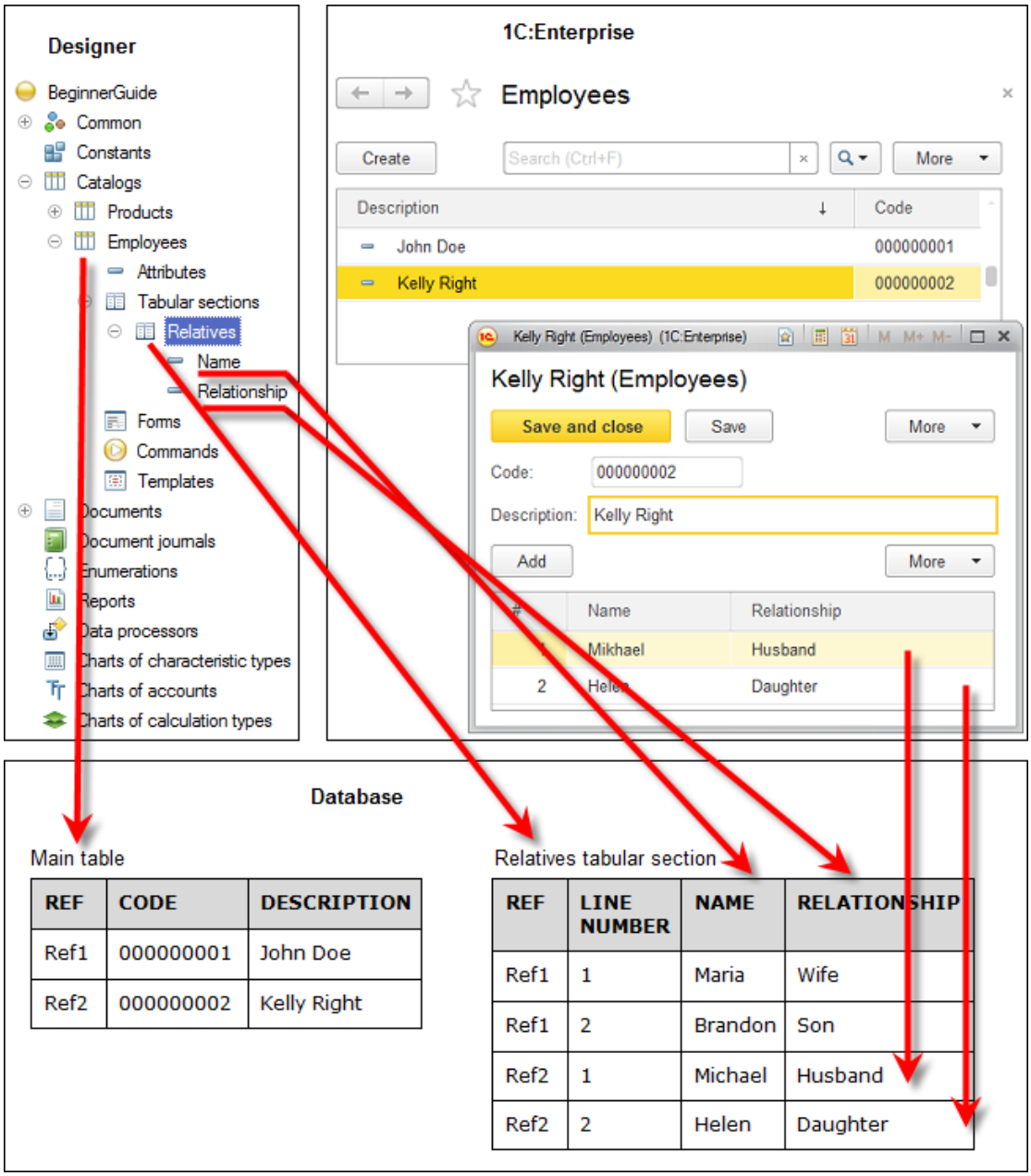


Fig. 3.4. Employees catalog in Designer mode, in 1C:Enterprise mode, and in the database

Note that the software hides the technical details related to data storage from developers: multiple catalog tables in the database, connections between tables based on a unique Ref field, specific table field types, and so on. The platform handles all of these automatically. The only thing you have to do is add a subordinate Tabular section object to the Catalog configuration object.

For user convenience, catalog items can be grouped.

For example, you can create the following groups in the Household appliances catalog: Refrigerators, TV sets, Washing machines, and so on. The Hierarchical property of the Catalog configuration object defines whether one can create groups in the catalog. A catalog item that is identified as a group is the parent for all items and groups that are included in it. This hierarchy type is referred to as folder and item hierarchy (fig. 3.5).

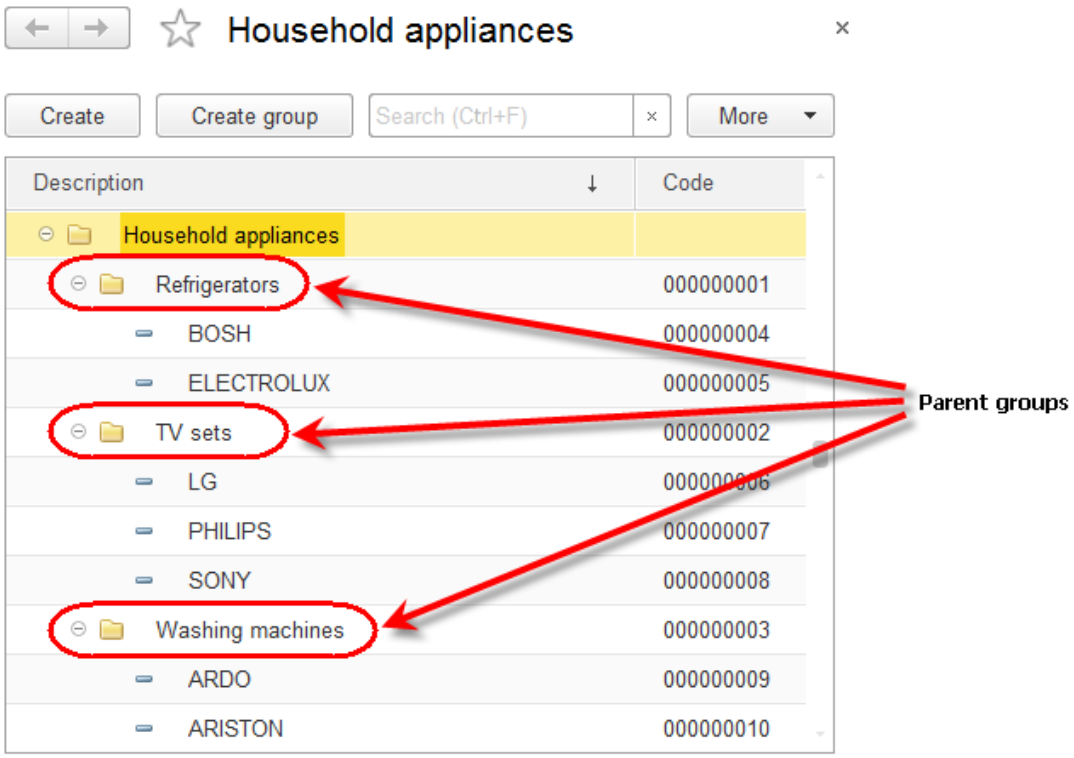


Fig. 3.5. Hierarchical catalog with folder and item hierarchy

Another available hierarchy type is item hierarchy. In this scenario the parent is not a group of catalog items but one of the catalog items itself. For example, you can use this hierarchy type in the Departments catalog where a single department can be the parent for several subdepartments (fig. 3.6).

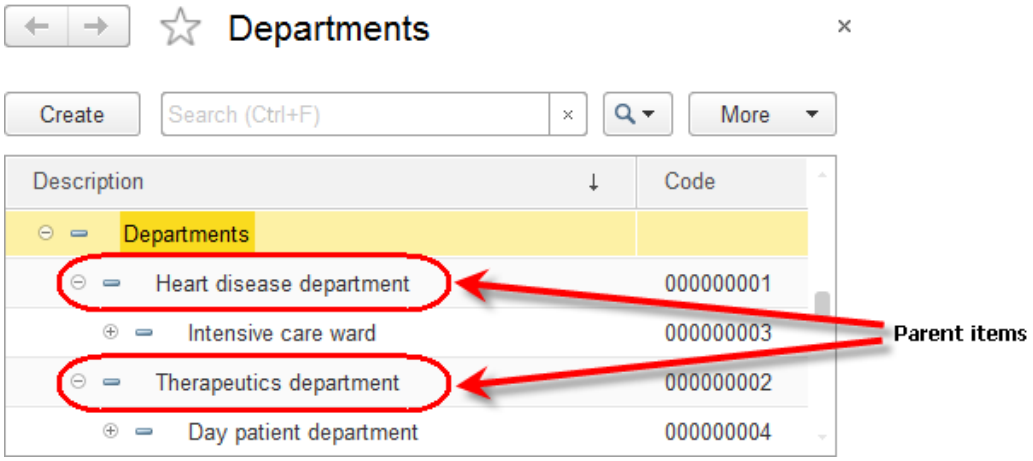


Fig. 3.6. Hierarchical catalog with item hierarchy

Catalog items can be subordinate to items or groups belonging to another catalog. For example, the UnitsOfMeasure catalog can be subordinate to the Products catalog. Then you can specify a unit of measure for each item of the Products catalog.

In 1C:Enterprise this is accomplished by defining a list of owners for each Catalog configuration object. In this case the Products catalog should be the owner of the UnitsOfMeasure catalog (fig. 3.7).

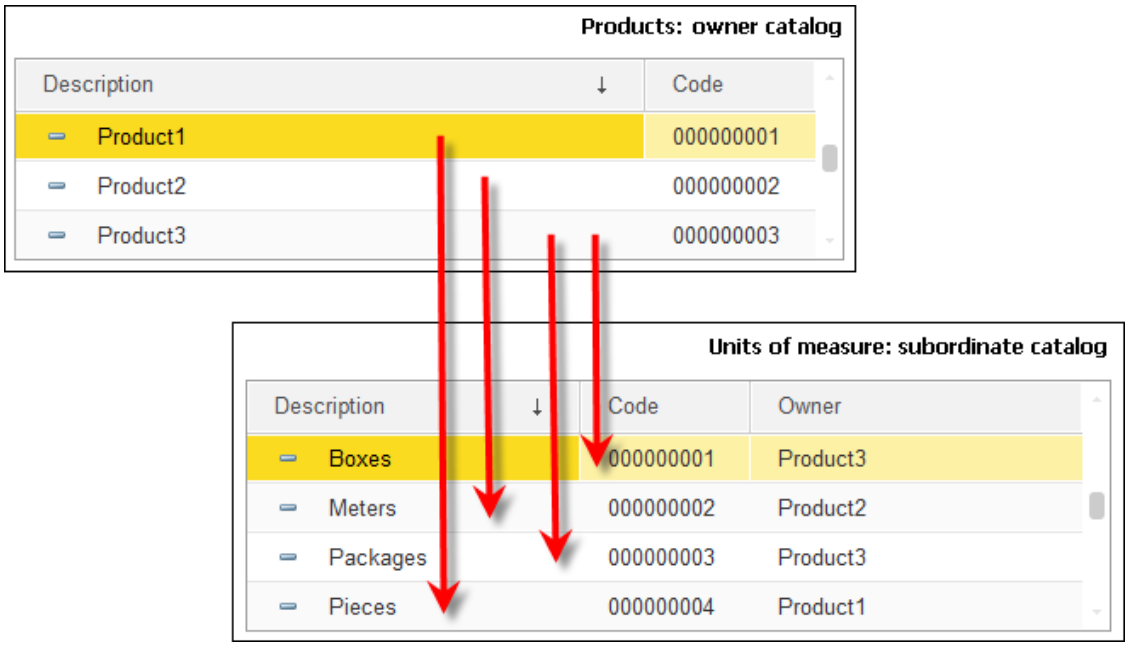


Fig. 3.7. Products catalog is the owner of the UnitsOfMeasure catalog

In some scenarios a catalog must always contain certain items, regardless of user actions. For example, standard business processes in a company assume that all products are first received at the main warehouse, and then they might be shipped to other warehouses. In this scenario the Warehouses catalog must contain the Main item at all times, otherwise the receipt of goods would not be accomplished correctly. The Catalog configuration object supports defining any number of such catalog items. They are referred to as predefined catalog items (fig. 3.8).

Fig. 3.8. Warehouses catalog with the Main predefined item

Unlike regular items, predefined items are created in Designer and they can be accessed using 1C:Enterprise script. In the user interface predefined catalog items are marked with a special icon (see fig. 3.8).

Catalog forms

Depending on the actions you want to perform on a catalog, you might need to display the catalog in various views. For example, selecting a catalog item is easier when you view the catalog as a list, while modifying a catalog item is easier when you have all of the catalog item attributes in a single form (fig. 3.9).

List form of the Employees catalog

Item form of the Employees catalog

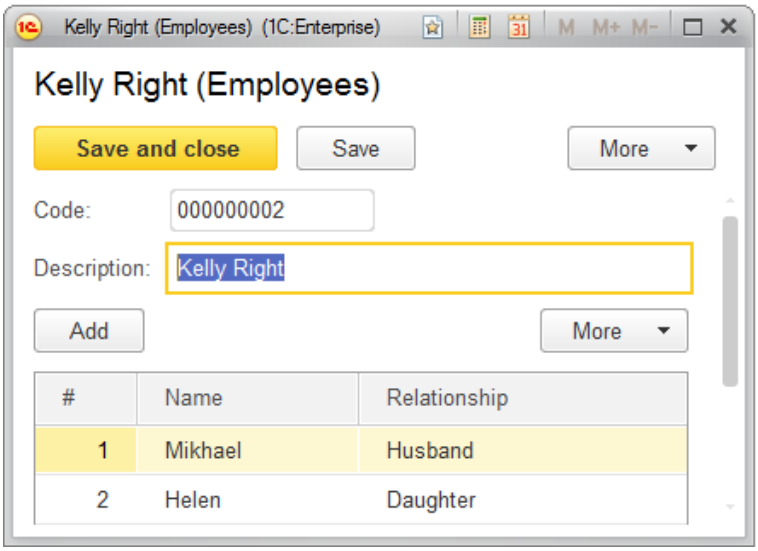


Fig. 3.9. List form and item form of the Employees catalog

All of the forms that you need to view catalog data can be generated automatically. The platform automatically selects the required form.

Catalogs have five form types for displaying their data in various scenarios. Note that a single form can have slightly different names in various Designer windows (table 3.1). The table below lists the form name differences:

- in the catalog pop-up menu, in the configuration tree, and in the catalog property palette,
- in the form wizard,
- on the **Forms** tab of the catalog editor.

Table 3.1. Catalog forms

IN THE POP-UP MENU AND IN THE PROPERTY PALETTE (FIG. 3.12)	IN THE FORM WIZARD (FIG. 3.11)	ON THE FORMS TAB (FIG. 3.10)
Default Object Form	Catalog item form	Item
Default Folder Form	Catalog folder form	Group
Default List Form	Catalog list form	List
Default Choice Form	Catalog choice form	Choice
Main group selection form	Catalog folder choice form	Group choice

This is because configuration object property names are displayed in the pop-up menu and in the property palette. They are the same for all configuration objects. At the same time the form wizard and the Forms tab display property names that are clear to developers. So they vary for different configuration objects (fig. 3.10, 3.11, 3.12).

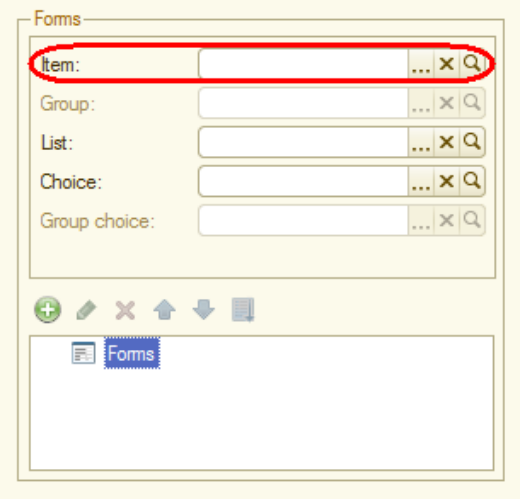


Fig. 3.10. Catalog form names on the Forms tab

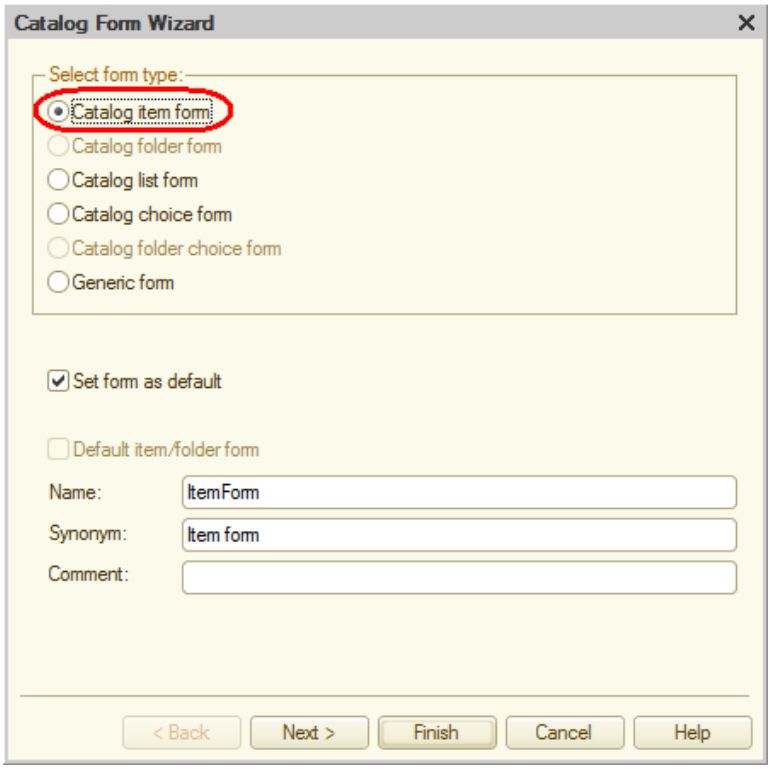


Fig. 3.11. Catalog form names in the Form wizard

Fig. 3.12. Catalog form names in the property palette

An item form is used to create or edit a catalog item.

A group form is used to create or edit a catalog group. A group normally contains much less information than a regular catalog item, so it requires a separate form that is different from the item form (fig. 3.13).

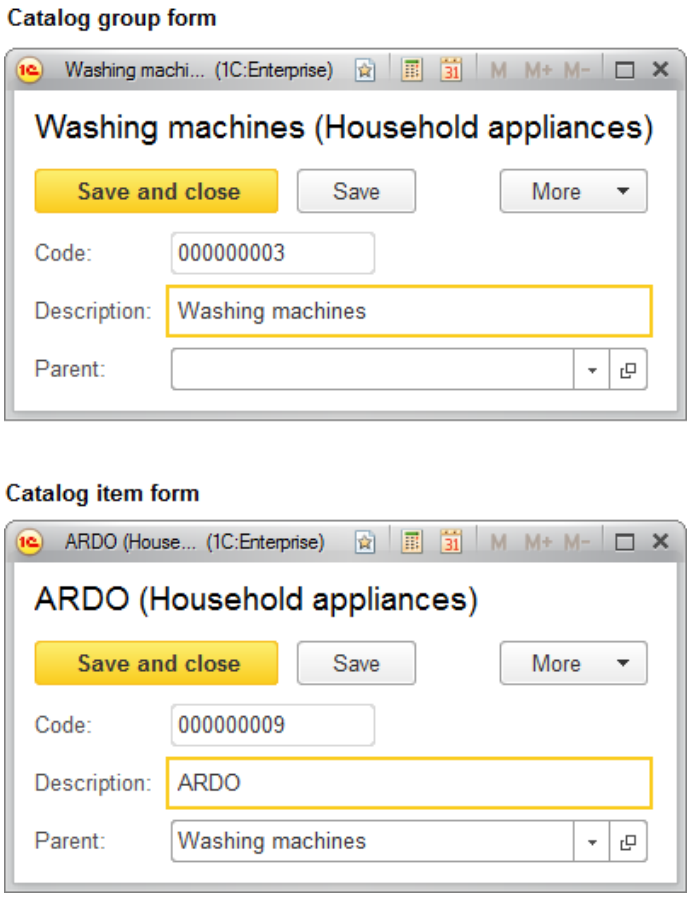


Fig. 3.13. An item form and a group form of a catalog

A list form is used to display a list of catalog items.

A choice form is used to select one of the catalog items in a form field. The choice form is simpler than the list form because a list form can display a lot of attributes. And when you select an item (for example, in a document), you only need to know its name. That is why a separate simpler form is available for item selection (fig. 3.14).

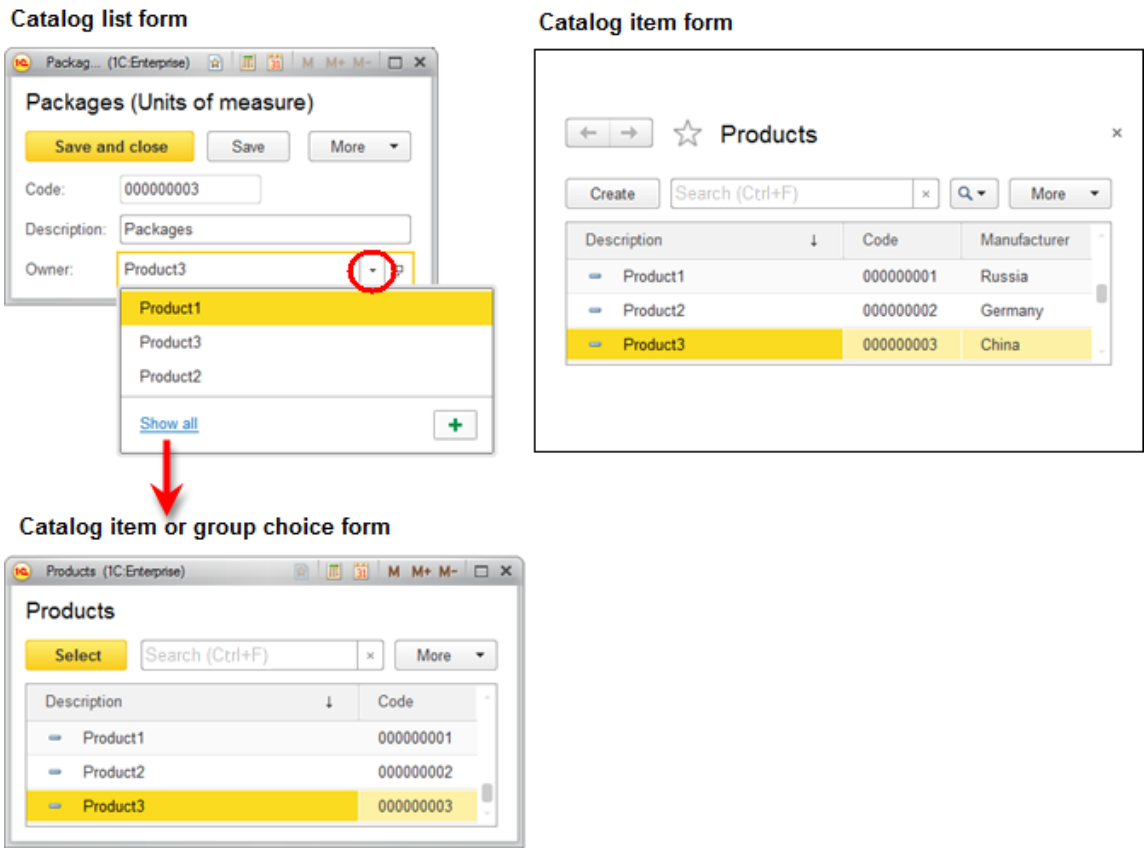


Fig. 3.14. A choice form and a list form of a catalog

A group selection form is used to select one of the catalog groups instead of an item in a form field. A group selection form is simpler than an item choice form because a group normally contains much less data than a catalog item.

Object forms are available for all referential configuration objects (catalogs, documents, and so on) but note that when we mention an object we refer to an infobase object (i.e. one of the items described by this object). For a catalog, it is a catalog item; for a document, it is a document; for a chart of accounts, it is an account, and so on.

You can define forms in Designer. A subordinate Form object is available for this (fig. 3.15).

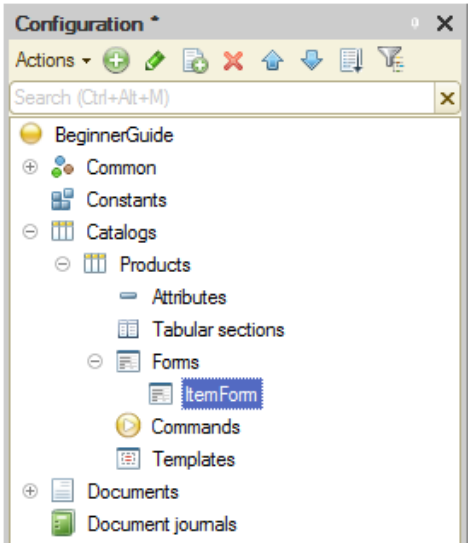


Fig. 3.15. A catalog item form in Designer

As a rule, the Form configuration object is subordinate to one of the applied objects, but it can also exist independently.

Whenever needed, the 1C:Enterprise platform uses the definition stored in the Form configuration object to create the Form script object, and the user will work with this script object.

Hence, forms serve as visual presentations of data stored in the database. Forms present this data in user-friendly format and provide options for defining algorithms that will assist users in working with the displayed data.

Learn more! For details on the structure of 1C:Enterprise script objects intended for catalog operations, see section "Quick developer reference. Catalogs" on page 839.

Simple catalogs

Now that you are somewhat familiar with the features of Catalog configuration objects, you will create several objects of this type. They will describe the catalogs available in your database.

Since your company, Jack of All Trades, provides household appliance services, you obviously need to create a certain number of lists for keeping the records.

First of all, you need a list of company employees who provide the services.

You also need a list of customers.

Then, you need a list of services provided by your company, and a list of materials that can be consumed while providing the services. Finally, you need a list of warehouses where Jack of All Trades stores materials.

Let us start with the simplest things: the employee list and the customer list. The first goal is the customer list.

In Designer mode

Let us create the list of customers.

1. In Designer, open your configuration.
2. In the configuration object tree, click the **Catalogs** branch.
3. In the configuration window command bar, click the **Add** button (fig. 3.16).

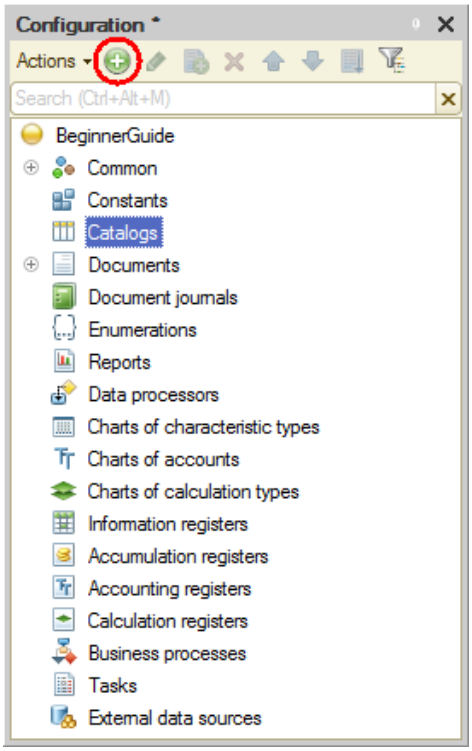


Fig. 3.16. Creating a Catalog configuration object

4. In the configuration object editor, in the **Name** field, enter **Customers**.

Based on the name, the platform generates the Customers synonym. As a reminder, the Synonym property defines the object presentation in the applied solution interface.

You can also specify additional properties that define various object presentations. These properties are optional. If they are not defined, the synonym of the Catalog configuration object is used to represent this object in the 1C:Enterprise interface. But this is not always the best option as you will see further.

Configuration object presentations

An object can have multiple presentations.

1. **Object presentation** determines the object name in the singular and is used in the standard commands. For example, it is used in the command that creates objects of this type: Customer: create.

The object presentation is required when the object synonym is defined in the plural or when it describes multiple objects because commands that open the catalog list and create a catalog item are automatically generated in the interface.

If a synonym is defined in the plural, it is absolutely suitable for the command that opens the Customers list (the list of all customers).

But it is not a good option for the command that creates a catalog item (a single customer). This command requires a presentation in the singular: Customer. The object presentation describes the name of the command that adds a customer in the user interface. It is also shown in the title of the customer form (unless extended object presentation is available) and in the presentation of a reference to a customer.

2. **Extended object presentation** defines the title of the object form (for example, a catalog item creation form). If this property is omitted, the Presentation property of the object is used instead.
3. **List presentation** defines the name of a list of objects, and it is also used in the names of standard commands (for example, the Customers command that opens the list of objects). List presentation is required when the synonym is defined in the singular.

   This is often needed for documents (such as Goods receipt). In this case the list presentation must include the configuration object name in the plural (Goods receipts).
4. **Extended list presentation** defines the title of the list form, such as the catalog list form. If this property is omitted, the List presentation property is used instead.

Now let us define some of these properties.

1. In the **Object presentation** field, type *Customer*.
2. In the **List presentation** field, type *Customers*.

   You can omit setting this property because the catalog synonym matches the List presentation property (fig. 3.17).

The list presentation seems to suggest List of customers as a name but in the application interface the lines List of employees, List of customers, and List of warehouses following each other will not look good.

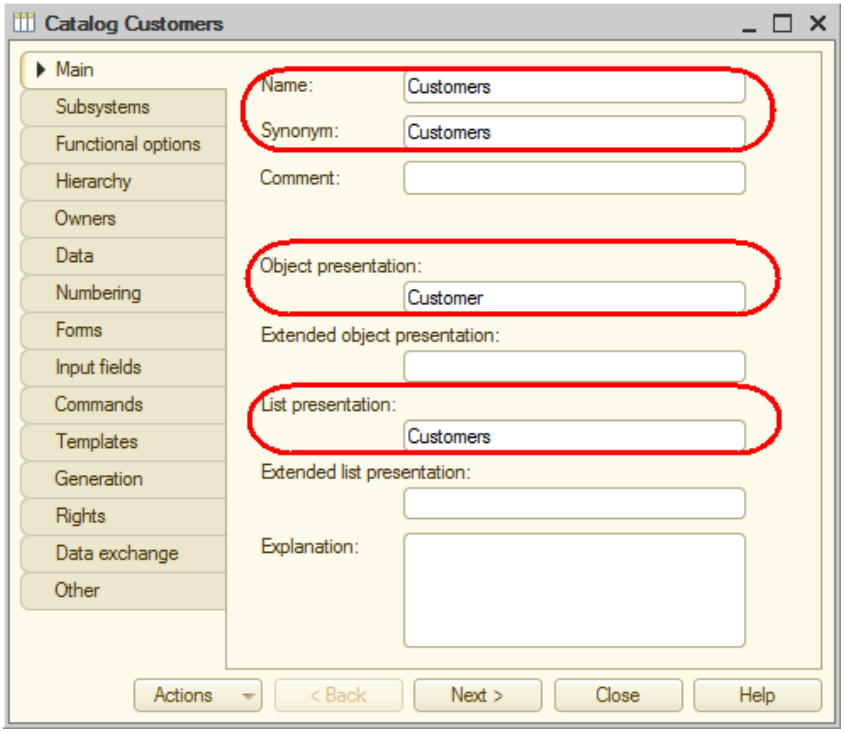


Fig. 3.17. Specifying main catalog settings

Object affiliation with subsystems

Now let us include the object in subsystems.

1. Click **Next**.

   This opens the **Subsystems** tab of the Catalog configuration object editor. Here you can define the subsystems where this catalog is available.

   The subsystem list includes the subsystems that you created earlier in the process of defining the application structure. Based on the business logic, the list of customers should be available in the Services section because each service is related to a specific customer. Accounting records generated in the Accounting section can also be filtered by customers.

2. Select the **Accounting** and **Services** check boxes (fig. 3.18).

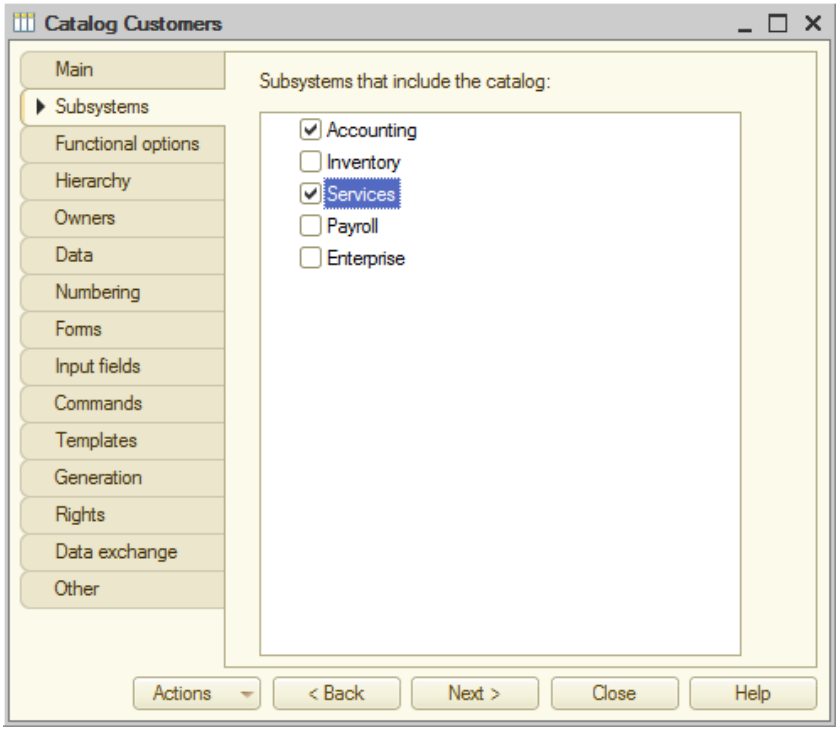


Fig. 3.18. Specifying subsystems where the catalog is available

3. Open the editor for one of these subsystems (for example, **Accounting**) and click the **Content** tab.

   You can see that the objects of this subsystem now include the new configuration object: the Customers catalog (fig. 3.19).

   Note that on the **Content** tab you can also modify the list of objects included in the subsystem.

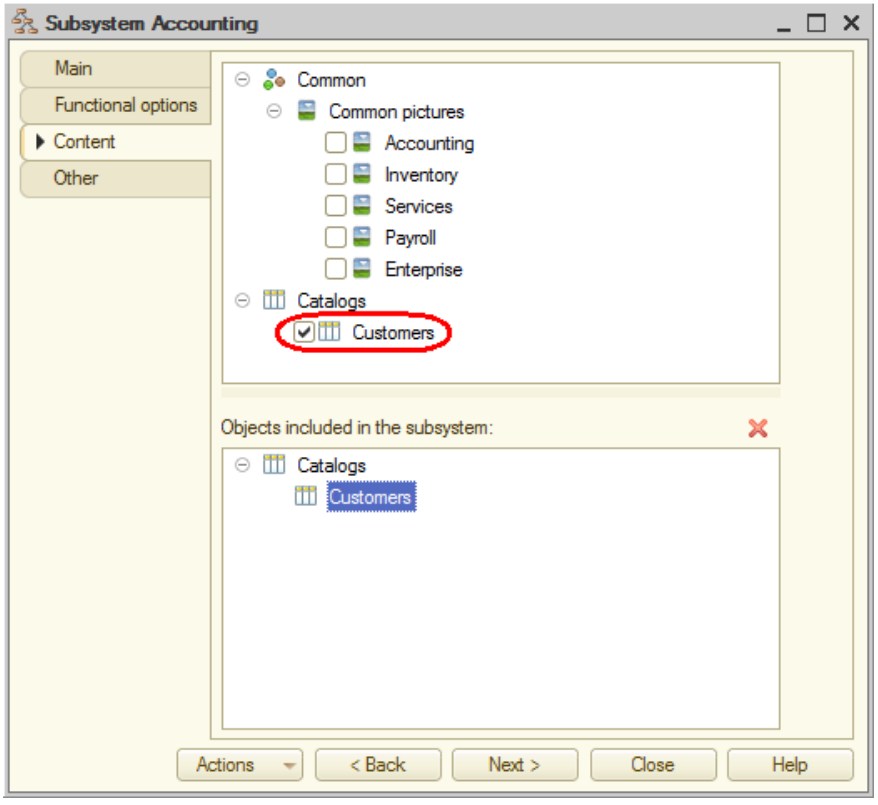


Fig. 3.19. Objects included in a subsystem

Catalog code and description

In this section you will specify the catalog code and description settings.

1. Return to the Catalog configuration object editor and click the **Data** tab.

   Note the **Code length** and **Description length** fields.

   The code length is an important catalog property. As a rule, the catalog code is used to identify a catalog item, and it contains a unique value for each catalog item. The platform can check the codes for uniqueness and perform automatic numbering of catalog items. So the code length defines the total number of items that can be stored in a catalog.

   The maximum code length is 9 characters. This means that codes between 1 and 999999999 are available, which is okay for the small company Jack of All Trades.

   Now let us look at the description length. 25 characters would clearly be too few.

2. In the **Description length** field, type *50* (fig. 3.20).

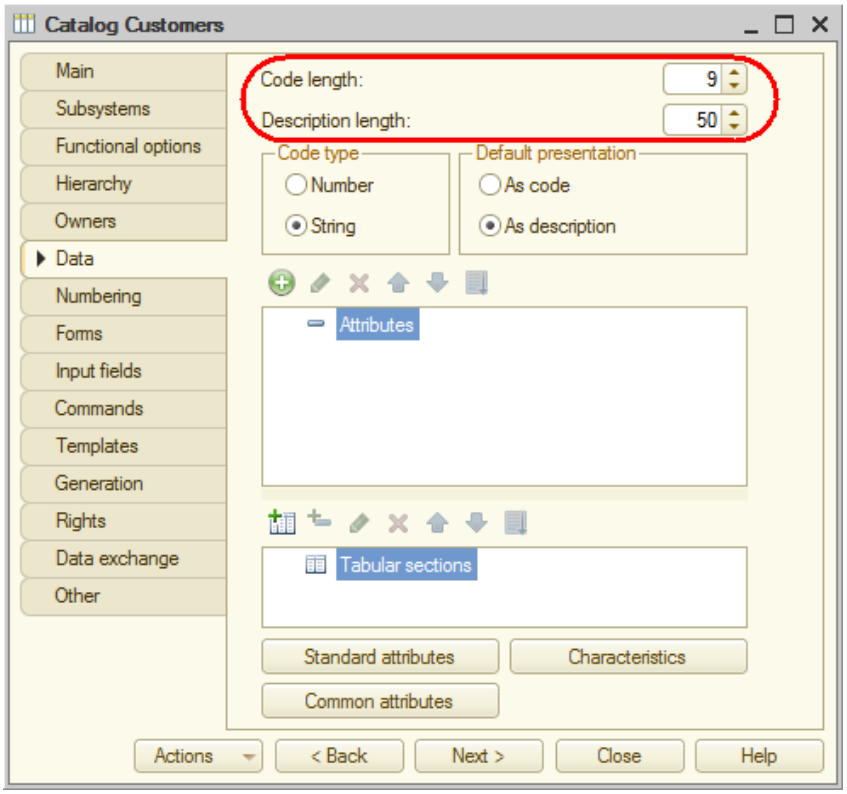


Fig. 3.20. Specifying catalog code length and description length

You also need a better name for the Description standard attribute, which will be displayed in the user interface. The word "description" is suitable for inanimate objects but our customers are people. Since the user interface displays object synonyms, let us change the Synonym property of the Description catalog attribute.

3. At the bottom of the window, click the **Standard attributes** button.

4. Right-click the **Description** attribute and then click **Properties** (fig. 3.21).

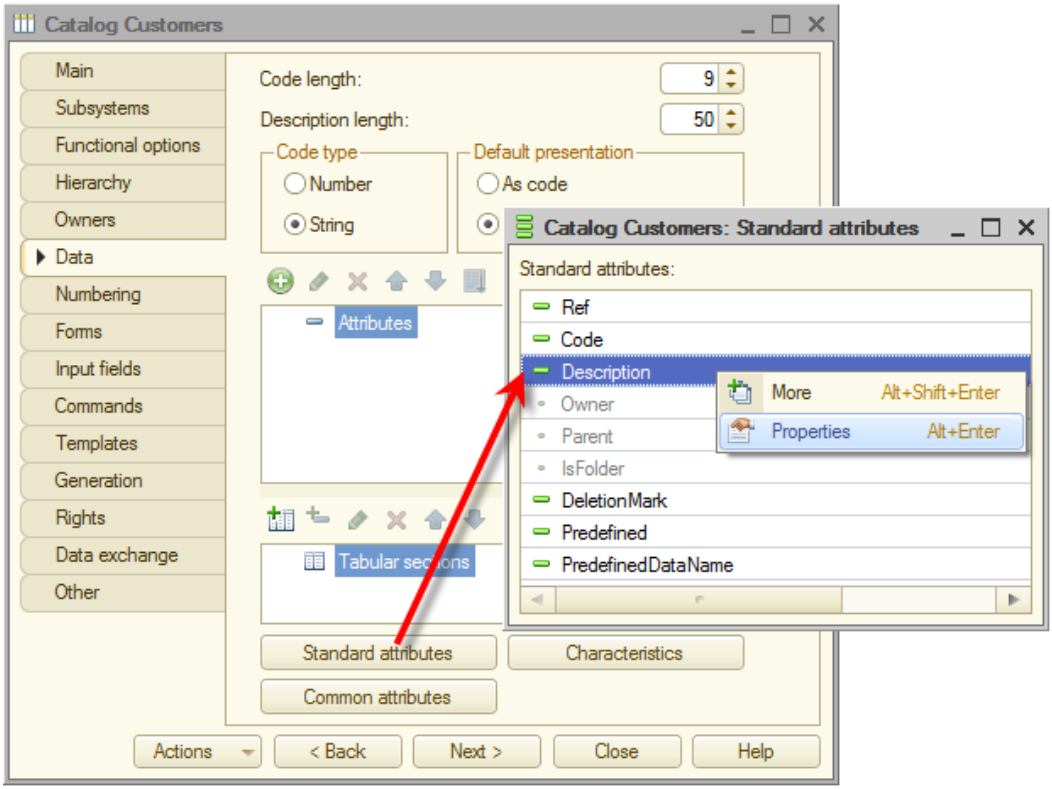


Fig. 3.21. List of standard catalog attributes

5. In the property palette of the **Description** standard attribute, in the **Synonym** field, type *Full name* (fig. 3.22).

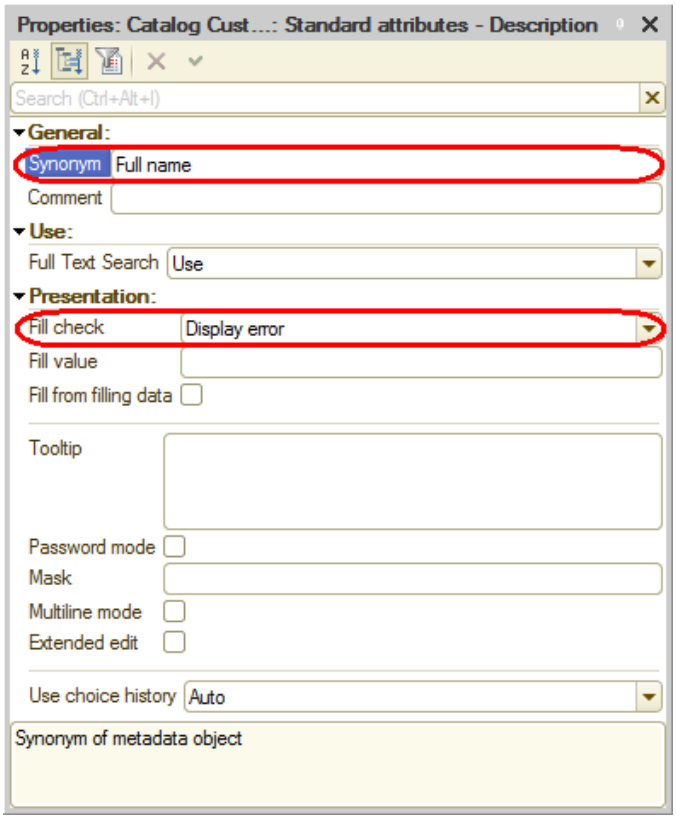


Fig. 3.22. Property palette of the Description standard attribute

Note that you changed the synonym of the configuration object attribute instead of changing the synonym of the form attribute. Actually, in this case the platform automatically generates the item form for the Customers catalog.

So this attribute will have its synonym set in all types of forms, unless you decide to change it during the development of a custom form.

Also note that the default value of the Fill check property is *Display error*. It means that leaving the Description attribute blank will generate an error message (see fig. 3.29).

Item creation command

Before you start the applied solution in 1C:Enterprise mode, let us configure the application interface to simplify creation of catalog items.

1C:Enterprise features a common algorithm for arranging standard commands that open lists and create configuration objects. You will learn this algorithm using catalogs as an example, but it also applies to documents, chart of accounts, and so on.

The command that opens a catalog list and the command that creates catalog items are added to the interface of the sections (subsystems) that include the catalog. But the command that creates items is hidden from the application interface by default.

This is because viewing catalog lists is a frequent operation, while creation of catalog items is less frequent. So the item creation command should only be enabled for Catalog configuration objects in sections where item creation is the major user activity.

Let us make the standard customer creation command available on the actions panel of the Services section.

1. Open the editor window of the **Services** subsystem and click **Command interface**.

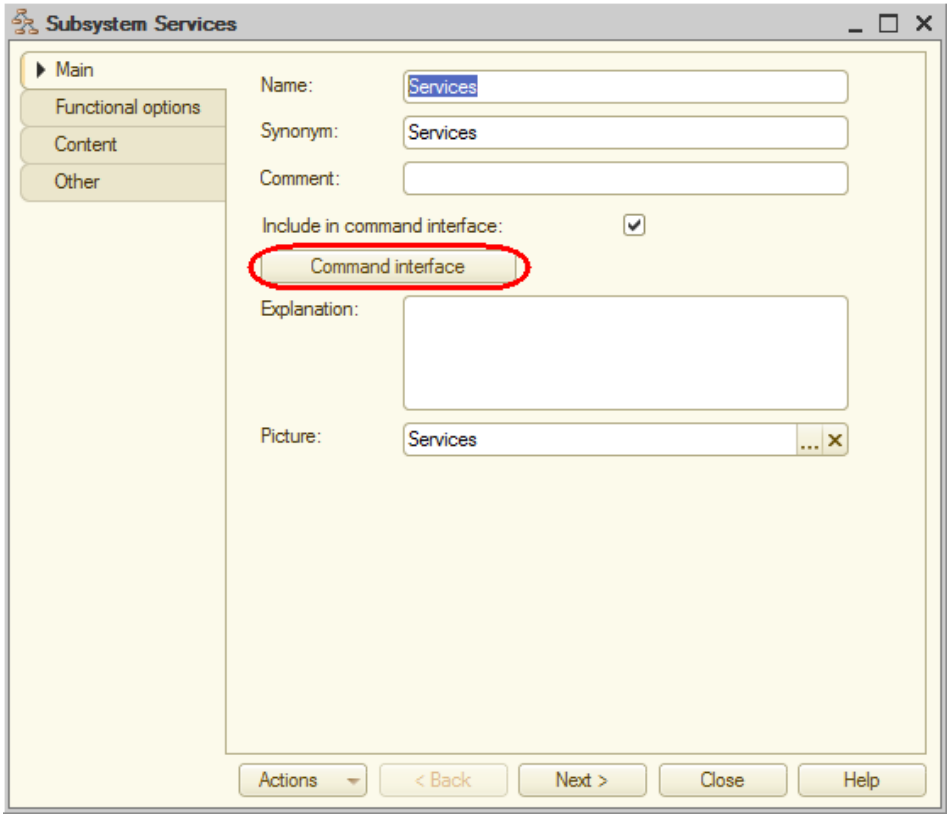


Fig. 3.23. Opening subsystem command interface settings

This opens the **Command interface** window, which lists all of the subsystem commands.

When you have created the Customers catalog, the Customers command (used to open the catalog list) was automatically added to the Navigation panel.Normal group and made visible by default. Also, the Customer: create command was added to the Actions panel.Create group and made hidden by default.

2. Select the **Visibility** check box for the **Customer: create** command (fig. 3.24).

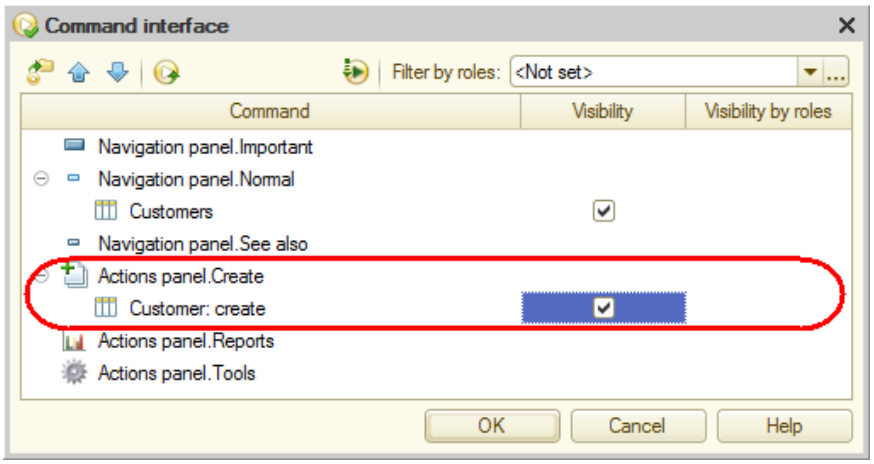


Fig. 3.24. Subsystem command interface settings window

You do not have to add any item creation commands to the Accounting subsystem because the business logic does not imply item creation in this section.

We assume that everyday operations with customers are mostly executed by the manager responsible for provision of services. These operations include creation of customers in the database when needed. And the accounting department simply processes the data in the database to obtain the reports they need.

This is why the customer creation command is added to the Services subsystem where the manager works. And the command is hidden from the accountants because we do not expect them to add customers.

But an accountant still has the option to create a customer using the list of customers (by opening the list and adding an item). The availability of the item creation command without the need to open the list of items is a matter of user convenience, it is not related to access restrictions. This is why we provide this handy feature to the manager instead of the accountant.

3. Close the **Customers** catalog editor and start the applied solution in the debug mode.
4. Confirm that you want to update the configuration.

   The platform displays the window that lists the changes in the configuration structure (the new Customers catalog).
5. Click **Accept** (fig. 3.25).

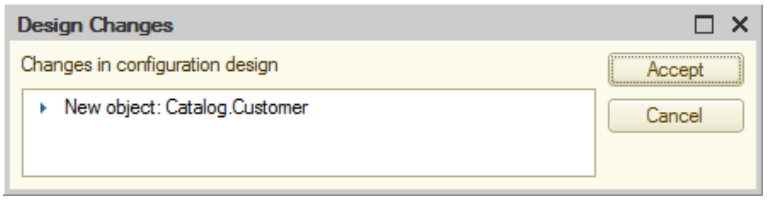


Fig. 3.25. List of changes in the configuration structure

In 1C:Enterprise mode

Current section commands panel

Let us look at the applied solution window in 1C:Enterprise mode. If you click the Services or Accounting section, you will see the current section commands panel below the sections panel (fig. 3.26).

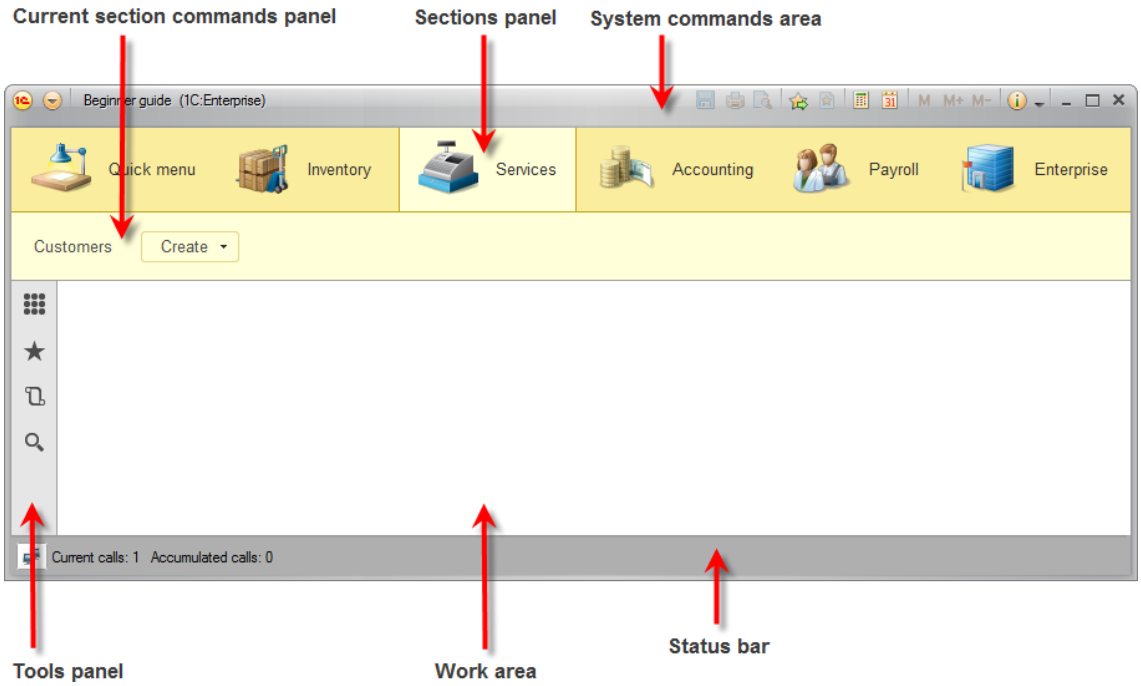


Fig. 3.26. The 1C:Enterprise window

The current section commands panel contains commands that belong to the selected section. First commands that open lists, then commands that create items, and finally commands that generate reports or run data processors.

In the command interface settings window for the Services subsystem (see fig. 3.24), in the Navigation panel.Normal group, you kept the default visibility of the command that opens the Customers catalog. This command is also visible by default in the Accounting subsystem. Therefore, both sections contain the Customers command that opens your first list.

Note that the Customers command name is taken from the List presentation catalog property. If the property value is not specified, the synonym of the Catalog configuration object is used instead.

Section commands that perform actions other than opening lists are combined into groups that are represented by menus in the user interface: Create, Reports, Tools, and custom developer-created groups. The Create menu contains commands that create database objects, such as documents or catalog items.

Note that the Services section, in addition to the Customers command, contains the Create menu (see fig. 3.26). This menu contains the command that creates items in your first catalog, you made the command visible in the section interface (in the Actions panel.Create section, see fig. 3.24). Now you can use this command to create catalog items without opening the list of customers.

Note that the default item creation command name is defined by the Object presentation property that you specified for this catalog. If you had not specified this property, the command name would be based on the synonym of the Customers catalog (Customers). This is both inconvenient because it is identical to the command that opens the list and not exactly correct because when you create catalog item, you create a single customer.

Note that the other applied solution sections have empty commands panels because you did not make the Customers catalog visible in these subsystems.

Creating catalog items

For now your catalog is empty, let us populate it with some items.

1. In the **Services** subsystem, in the commands panel, on the **Create** menu, click **Customer**.

   This opens the catalog item creation form (fig. 3.27).

2. In the **Full name** field, type *Michael Johnson*.

   You do not have to type the code because it is generated automatically.

3. Click **Save and close**.

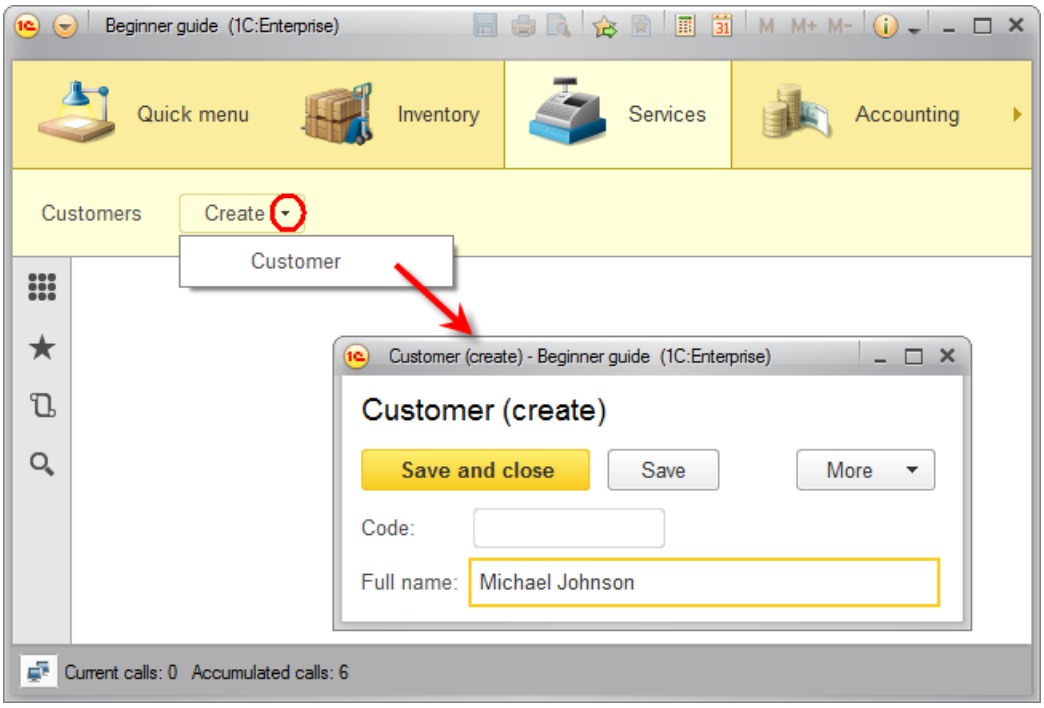


Fig. 3.27. Creating a catalog item

A message is displayed in the lower right corner informing you that an item is created or modified. This is handy because you do not have to open the list to ensure that the item is created.

4. Add another customer named *Peter Smith*.

   And now let us add the last customer Shelley Jackson using the customer list form.

5. In the **Services** section, click the **Customers** command.

   This opens the customer list form (see fig. 3.28).

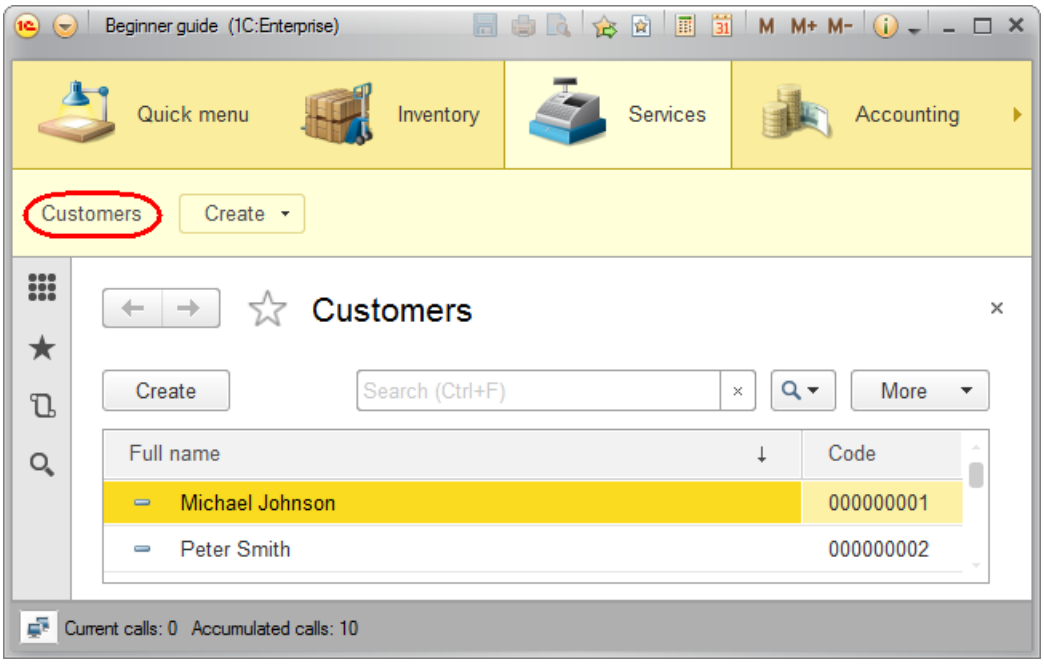


Fig. 3.28. Default customer list form

You can add a catalog item by clicking the **Create** button in the form command bar or by pressing the Insert key.

6. Click the **Create** button.

   Note that the **Full name** field is marked by a red dotted line. This means that the platform performs the automatic fill check for this field. If you attempt to save a customer with an empty full name, you get an error message (fig. 3.29).

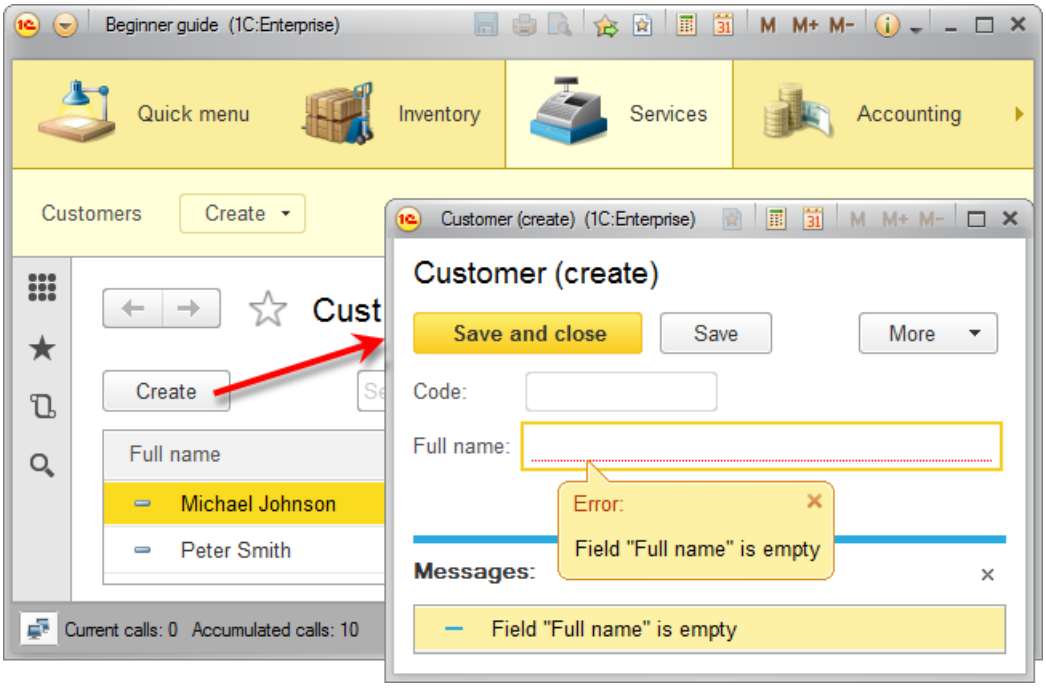


Fig. 3.29. Error message during catalog item creation

The error message is displayed because the platform performs automatic fill checks for some standard object attributes (for

example, for the catalog description, provided that the default catalog presentation is set to As description).

7. In the **Full name** field, type *Shelley Jackson*.

Once you have added the items, the catalog looks as follows (fig. 3.30).

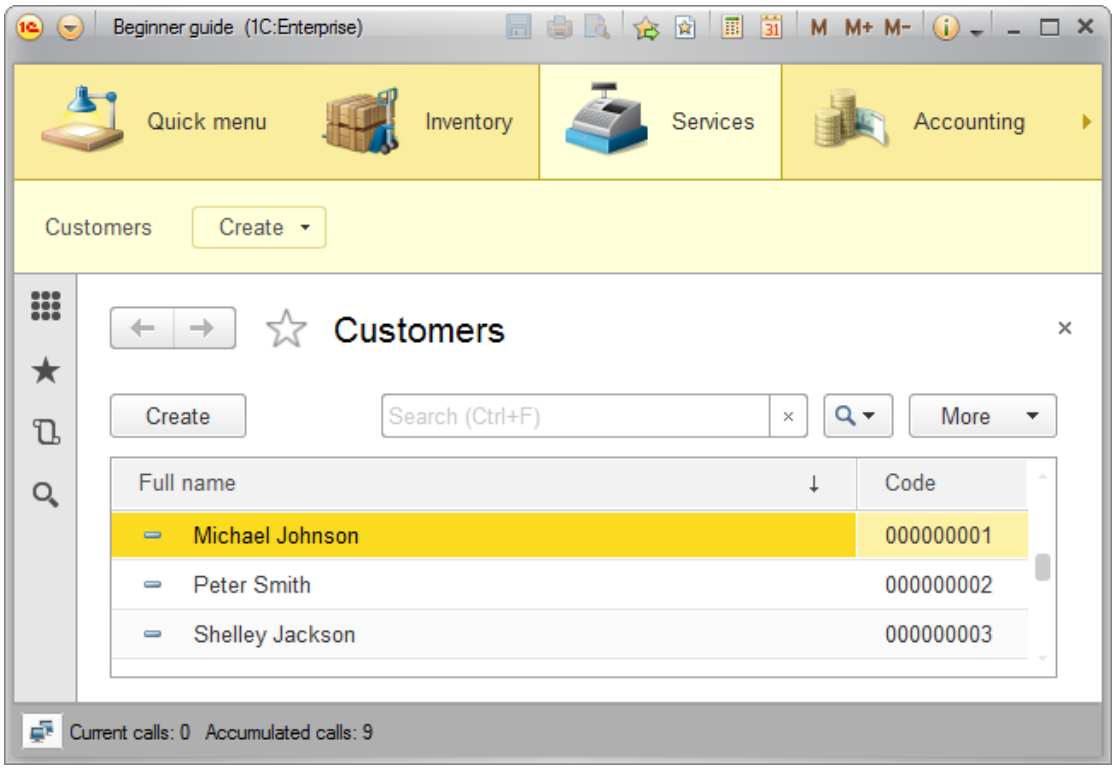


Fig. 3.30. The list of customers

You can open a catalog item for editing by double-clicking it.

For quick access to recently opened, created, or modified database objects (documents, catalog items, and so on) you can use the built-in history of user actions. The icon that opens the history is located on the tools panel (fig. 3.31).

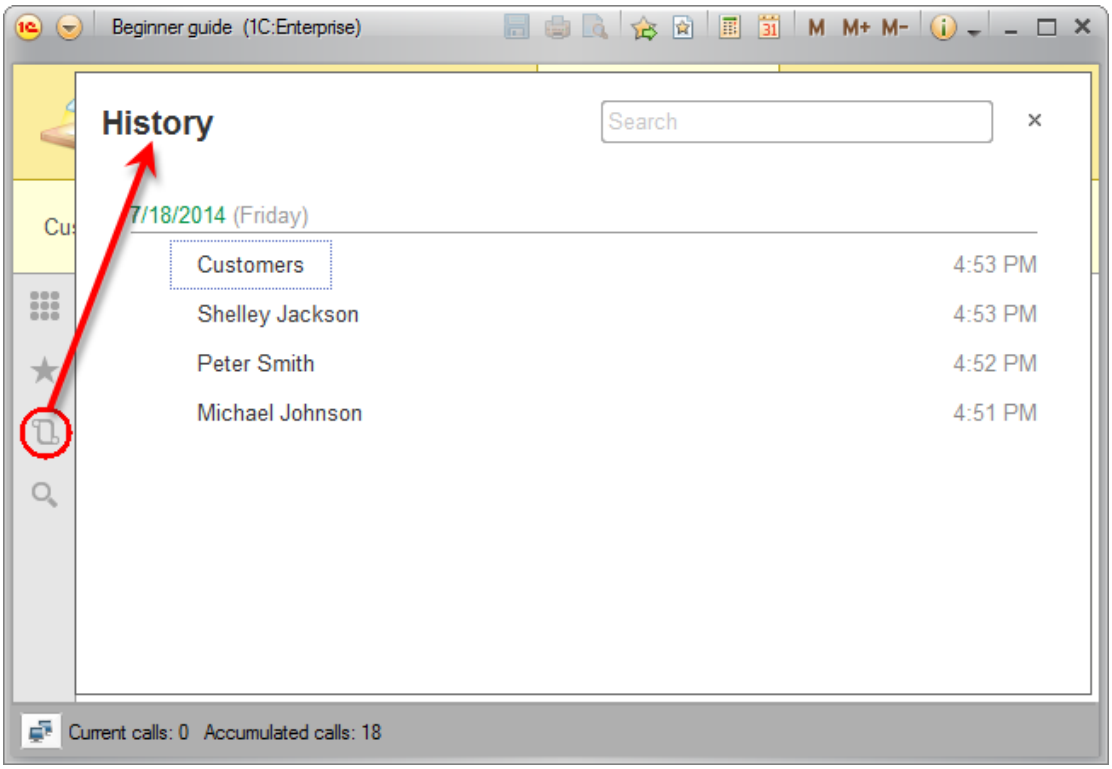


Fig. 3.31. History of user actions

Using the history panel, you can open a recently created database object (for example, a catalog item) or run an applied solution command. If the history is long, you can use the search field at the top of the history panel. Novice developers often prefer to keep the history panel open at all times.

Customizing applied solution panel view

By default, the history panel is not displayed in the applied solution interface. In 1C:Enterprise mode users can add this panel to the interface (on the main menu, point to **View** and then click **Pane settings**). Alternatively, developers can add the history panel to the interface using Designer.

In Designer mode

Let us add the history panel using Designer.

1. Close the applied solution and return to Designer.
2. In the configuration object tree, right-click the **BeginnerGuide** root item and then click **Open client application interface** (fig. 3.32).

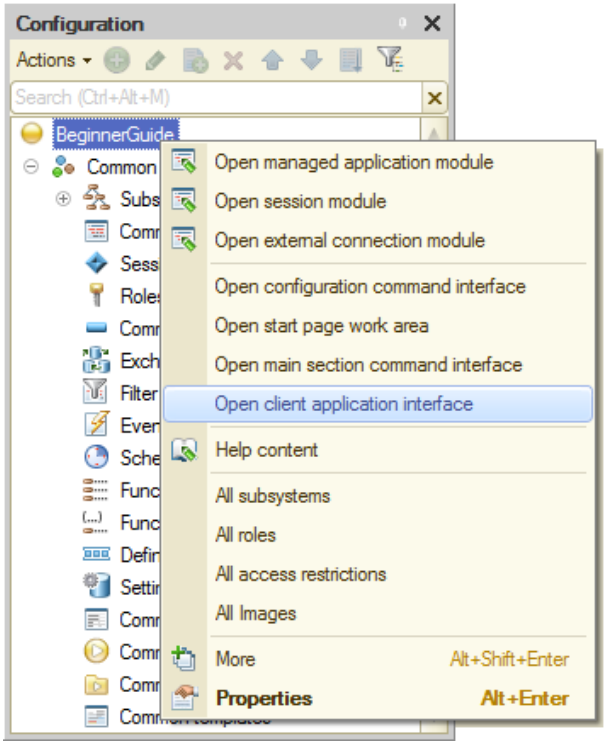


Fig. 3.32. Opening the applied solution panel layout settings

This opens the **Client application interface window**. The right part of the window contains the list of all standard panels available in the applied solution.

3. Drag the **History pane** item to the left part of the window, to the **Bottom** folder (fig. 3.33).

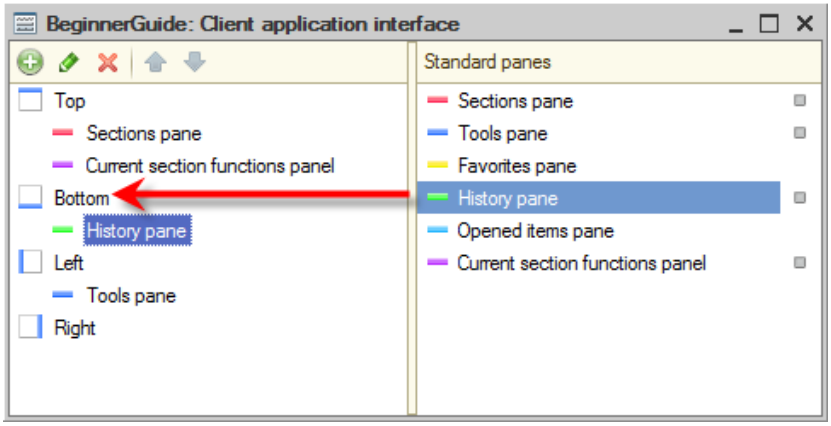


Fig. 3.33. Specifying applied solution panel layout settings

If you want to hide any of the standard panels, you can drag it to the right part of the window or click the **Delete current item** button in the window command bar. Dragging items between the groups (**Left**, **Right**, and so on) changes the panel positions in the applied solution window.

In 1C:Enterprise mode

Let us check the history panel position.

1. Start 1C:Enterprise in the debug mode.

   Note that the history panel is displayed at the bottom of the applied solution window. You can open any of the recently modified catalog items by clicking a link in this pane (fig. 3.34).

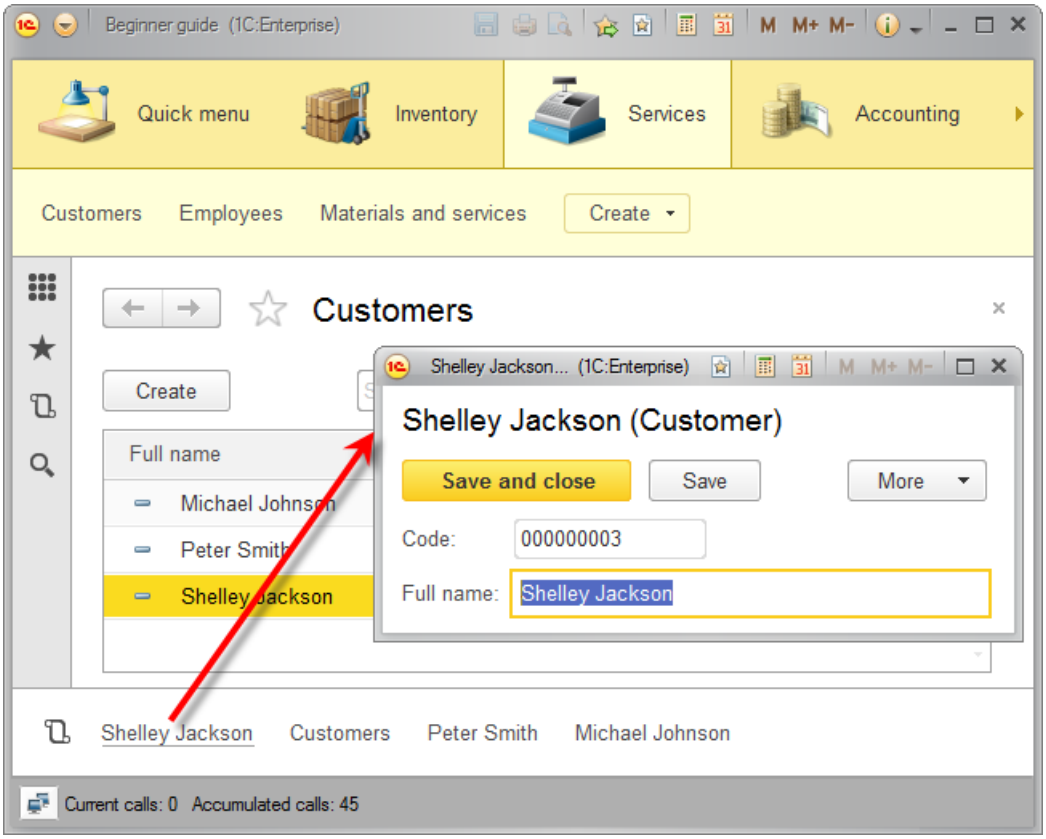


Fig. 3.34. Opening customers from the history panel

   Now let us hide the history panel in 1C:Enterprise mode because you do not always need it and it takes up the screen space.

2. On the main menu, point to **View** and click **Pane settings** (fig. 3.35).

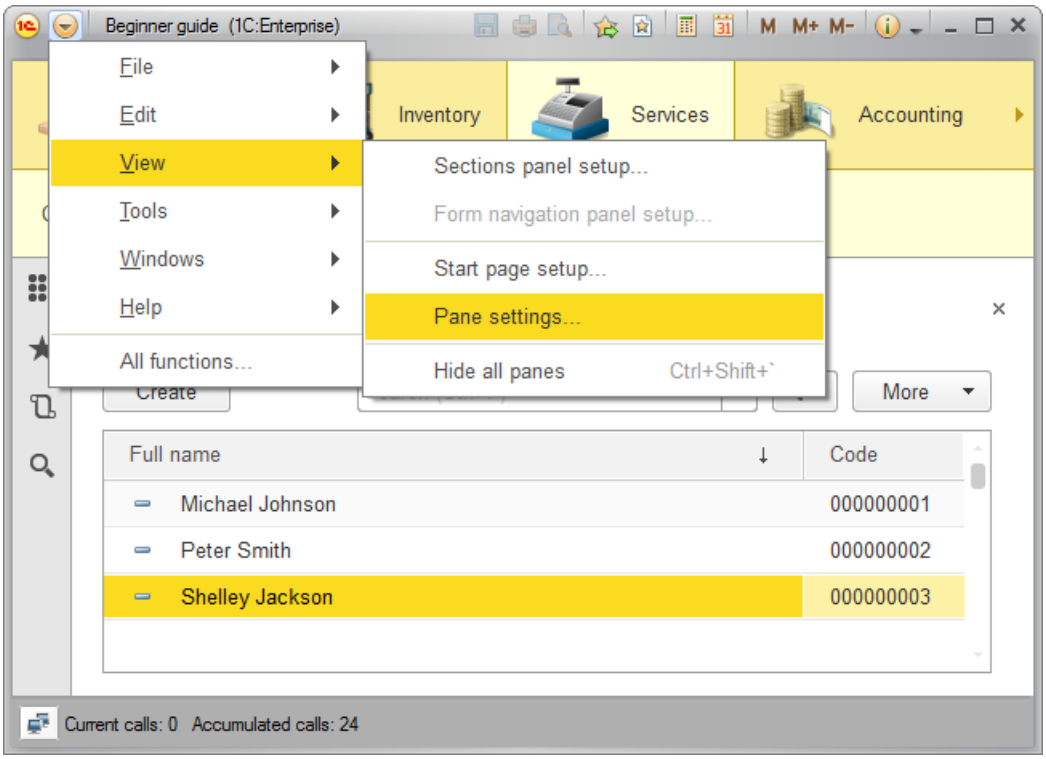


Fig. 3.35. Opening applied solution panel layout settings

3. In the **Customize panes** window, drag the history panel to the gray area at the bottom of the window.

   The gray area contains hidden panels (fig. 3.36).

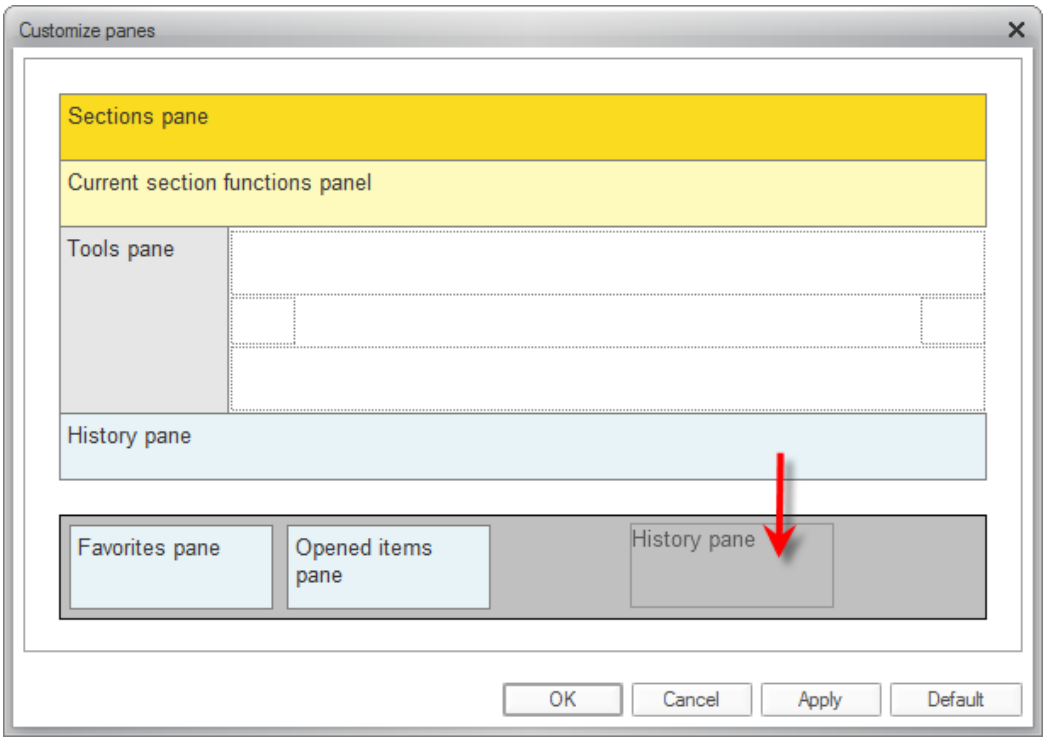


Fig. 3.36. Specifying applied solution panel layout settings

4. Click **OK**.

   This hides the panel in the default applied solution layout but you can always open it by clicking the **History** button on the tools pane.

In addition to changing panel visibility, you can change panel positions in the applied solution window by dragging them.

Now you learned how to change the panel layout in Designer and 1C:Enterprise modes on the example of the history panel. Note that specifying panel settings in Designer changes these settings for all of the applied solution users, while specifying panel settings in 1C:Enterprise only changes them for the user that made that change.

Catalogs with tabular sections

Now you can proceed to creating the second catalog: Employees.

This catalog will be more complex than the Customers catalog. It will store not only the full employee name, but also their past employment data.

This data is uniform in structure (company, start and end dates, and job title), but the number of previous jobs varies. You will use a tabular section for storing this kind of data.

In Designer mode

Let us add the Employees catalog.

1. In the configuration object tree, right-click the **Catalogs** branch and then click **Add**.
2. In the **Name** field, type *Employees*.

   The synonym is filled automatically based on the name.
3. In the **Object presentation** field, type *Employee*.
4. In the **Extended list presentation** field, type *Employee list*.

   Note that you do not have to fill the List presentation field (fig. 3.37).

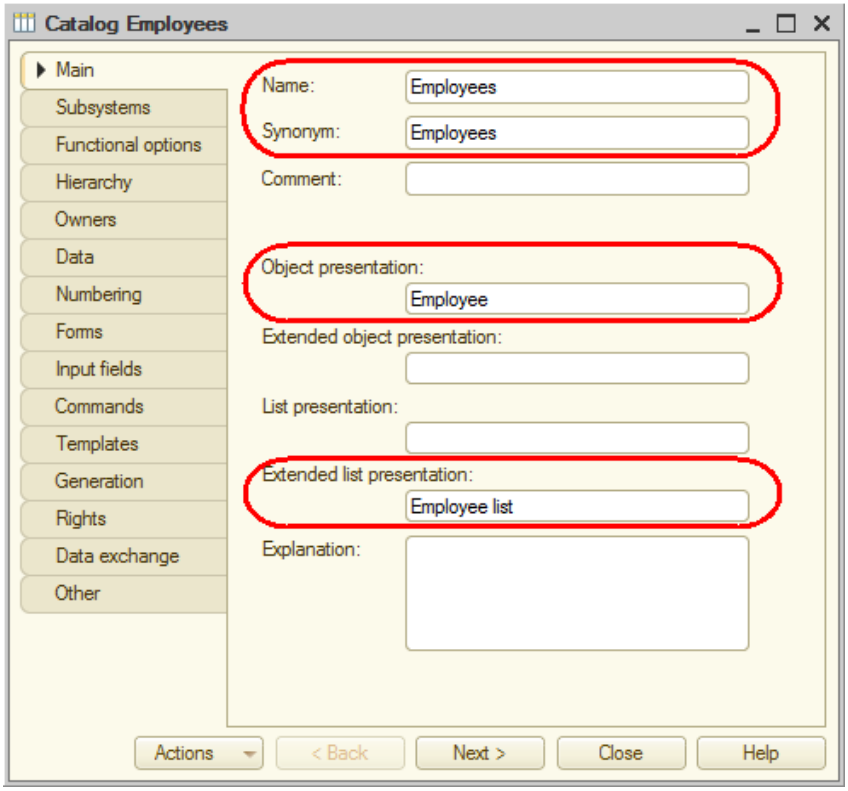


Fig. 3.37. Specifying main catalog properties

5. Click **Next**.

   This opens the **Subsystems** tab.

   The configuration logic suggests that the list of employees should be available in the Services and Payroll sections. Indeed, when a service is provided you need to specify the employee in charge. This information is used for calculating employee wages.

6. Select the **Services** and **Payroll** check boxes (fig. 3.38).

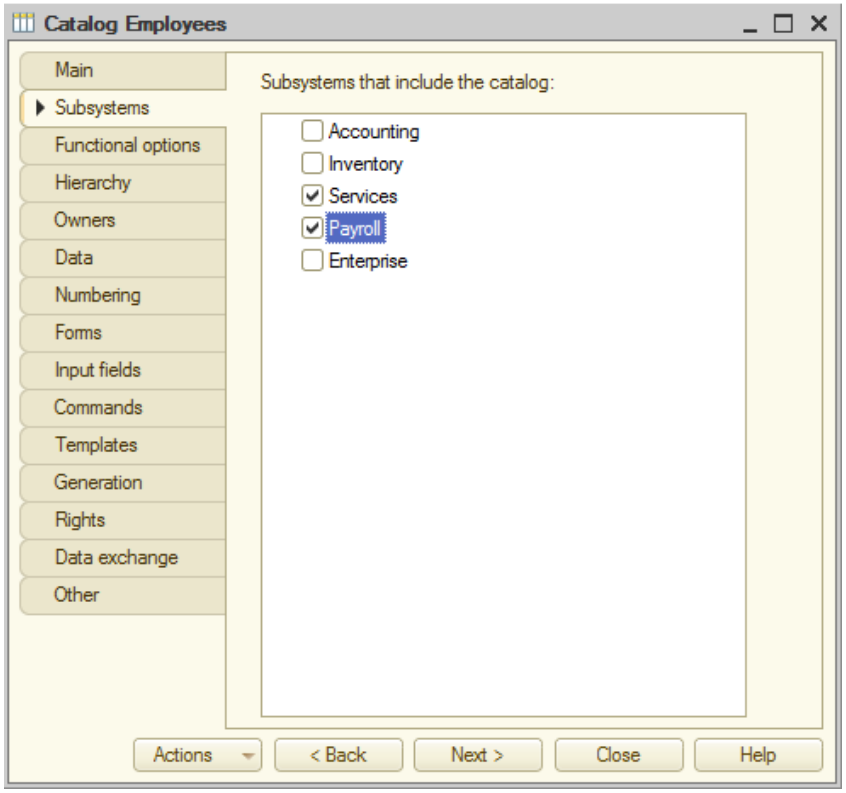


Fig. 3.38. Specifying subsystems where the catalog is available

7. Click the **Data** tab.
8. Leave the default values for the code length and type unchanged, and in the **Description length** field, type *50.*

   Let us change the synonym of the Description standard attribute, just like you did for the Customers catalog.
9. Click the **Standard attributes** button.
10. Double-click the **Description** attribute and then, in the property palette, in the **Synonym** field, type *Full name* (fig. 3.39).

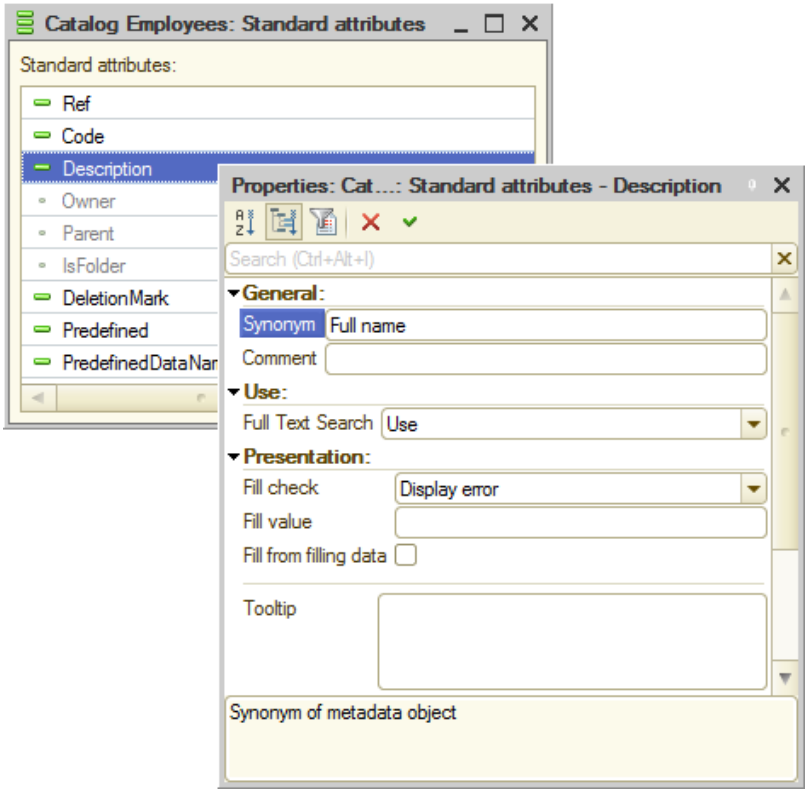


Fig. 3.39. Specifying synonym for the Description standard attribute

Tabular sections

Let us add the PastEmployment tabular section.

1. Click the **Add tabular section** button (fig. 3.40).

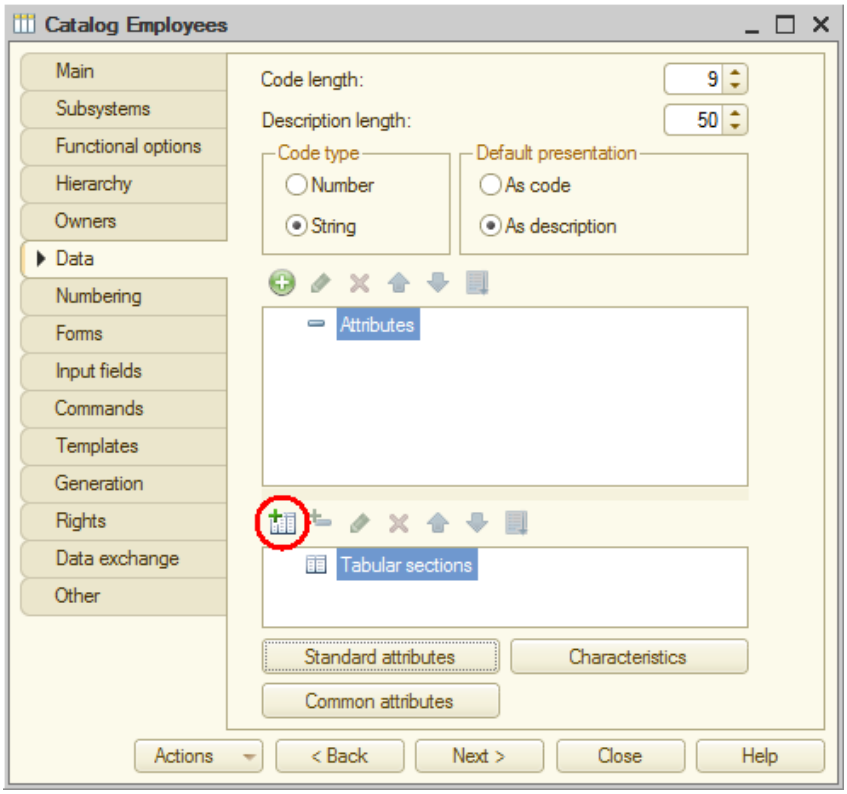


Fig. 3.40. Adding a tabular section

2. In the **Name** field, type *PastEmployment* (fig. 3.41).

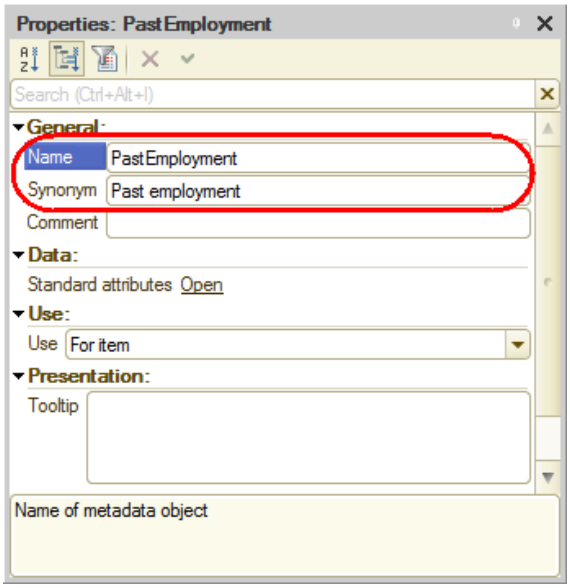


Fig. 3.41. Adding a tabular section

Then let us create the tabular section attributes.

3. Click the **Add attribute** button (fig. 3.42)

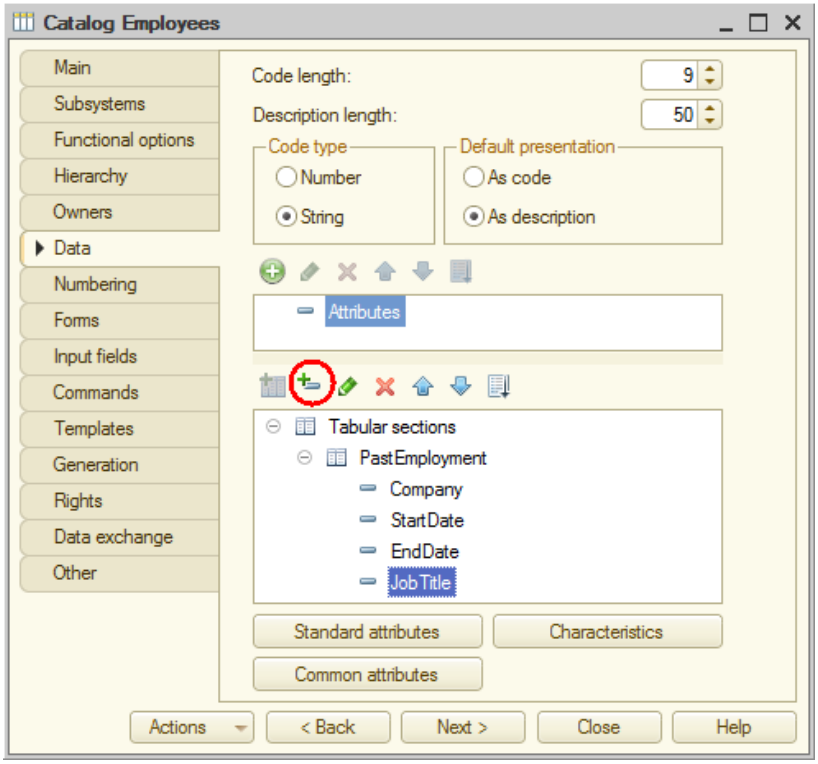


Fig. 3.42. Adding a tabular section attribute

4. Add the following attributes:
   - **Company**. Type: String, length: 100
   - **StartDate**. Type: Date, date contents: Date
   - **EndDate**. Type: Date, date contents: Date
   - **JobTitle**. Type: String, length: 100.

You selected the Date contents for the StartDate and EndDate attributes (fig. 3.43) because in 1C:Enterprise values of Date type contain both date and time. In this case the time of day when the jobs started or ended is not important.

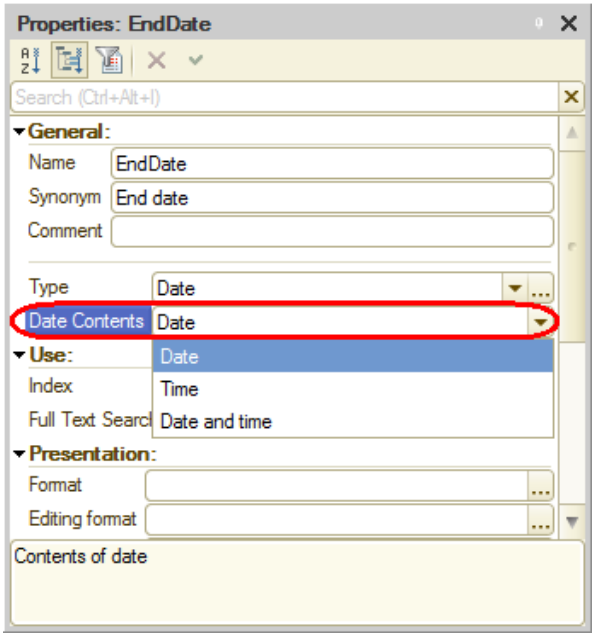


Fig. 3.43. Tabular section attribute properties

Finally, let us edit the command interface of the Payroll section to simplify creation of catalog items (make the standard employee creation command visible).

5. Open the editor of the **Payroll** subsystem configuration object and click the **Command interface** button.

   This opens the **Command interface** window that lists all of the subsystem commands.

6. In the **Action panel.Create** group, enable visibility for the **Employee: create** command.

   You can also see that the Navigation panel.Normal group contains the Employees command. It is visible by default (fig. 3.44).

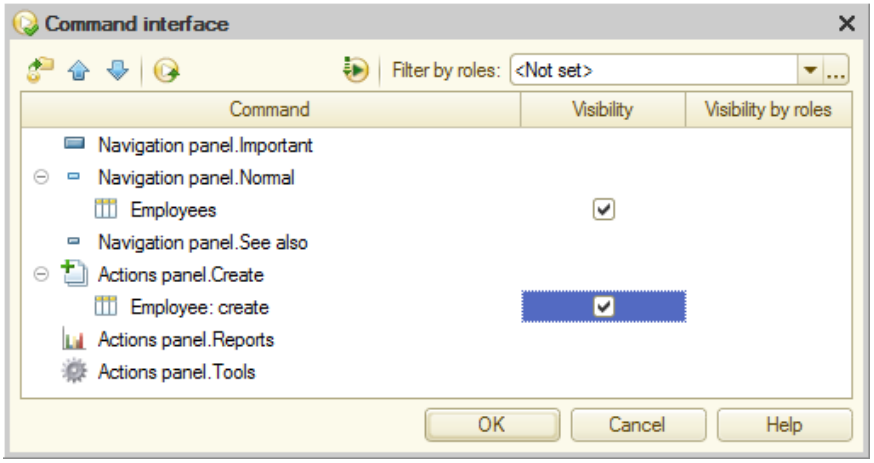


Fig. 3.44. Subsystem command interface settings window

You do not have to add any commands to the actions panel of the Services subsystem because adding employees in this section is not a typical scenario.

You completed the creation of the Employees catalog.

7. Close the **Employees** catalog editor and start 1C:Enterprise in the debug mode.
8. Confirm that you want to update the configuration.

   The platform displays the window that lists the changes in the configuration structure (the new Employees catalog).
9. Click **Accept** (fig. 3.45).

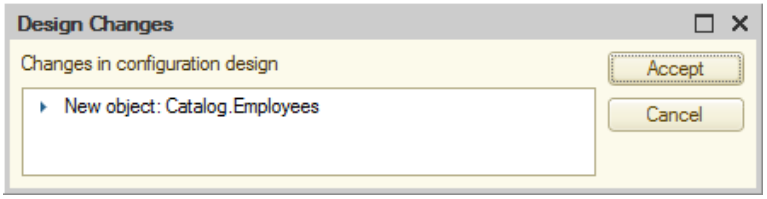


Fig. 3.45. List of changes in the configuration structure

In 1C:Enterprise mode

In the 1C:Enterprise window you can see that the Employees command is now available in the command panels of the Services and Payroll sections. This command opens the list of employees (fig. 3.46).

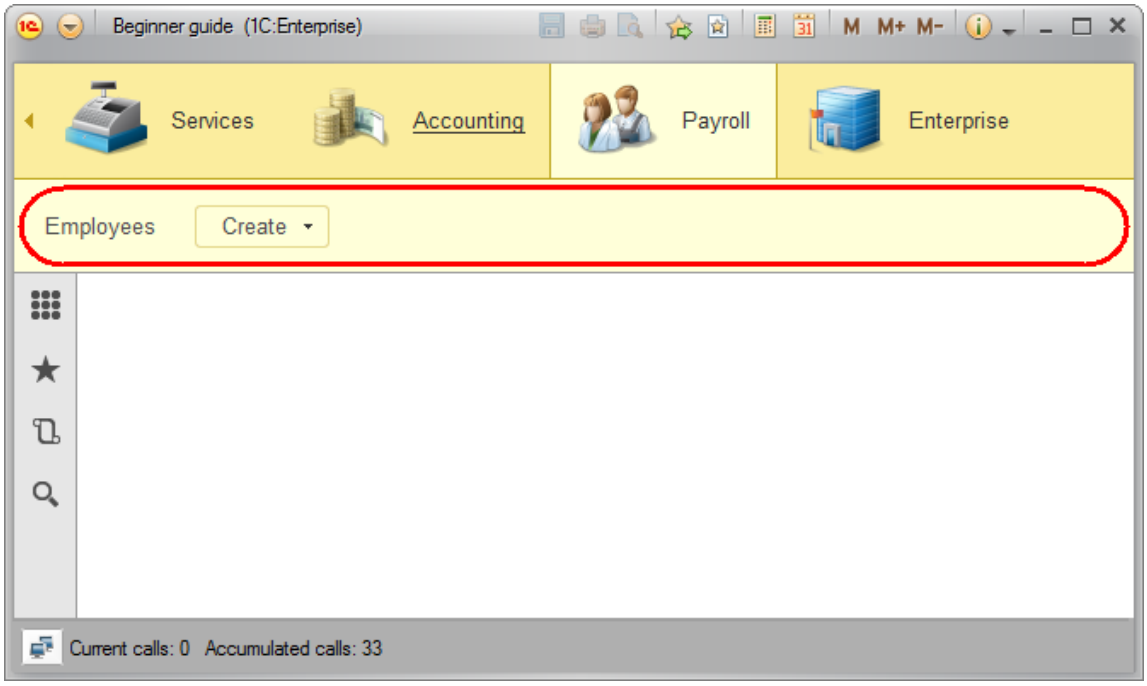


Fig. 3.46. Payroll section

The command name is determined by the object synonym because you have not defined the List presentation for this catalog.

Also the actions panel of the Payroll section now has the Employee command that creates employees (see fig. 3.46). The command name is

determined by the Object presentation property that you defined for this catalog.

Now let us use this command to create catalog items without opening the list of employees.

Filling tabular sections

Let us fill the tabular section.

1. Run the **Employee** command.

   This opens the catalog item creation form. The form title is defined by the Object presentation property. This form includes a tabular section with the attributes that you defined in Designer.

2. Create the following employees (fig. 3.47, 3.48, 3.49):
   - Alexander D. Johnson
     - Past employment:
       - Company: NTI Company
       - Start date: 2/1/2000
       - End date: 4/16/2003
       - Job title: Senior technician
   - John A. Walkman
     - Past employment 1:
       - Company: Global Automation Solutions, LLC
       - Start date: 01/22/2006
       - End date: 12/31/2012
       - Job Title: Engineer
     - Past employment 2:
       - Company: QDA Telecommunications Group
       - Start date: 06/20/1996
       - End date: 01/21/2006
       - Job title: Production manager
   - Michelle V. Turner
     - Past employment:
       - Company: Wilson and Beane Construction
       - Start date: 2/6/2001
       - End date: 4/3/2004
       - Job title: Master builder

   You can add rows to the catalog tabular section by clicking the **Add** button (fig. 3.47).

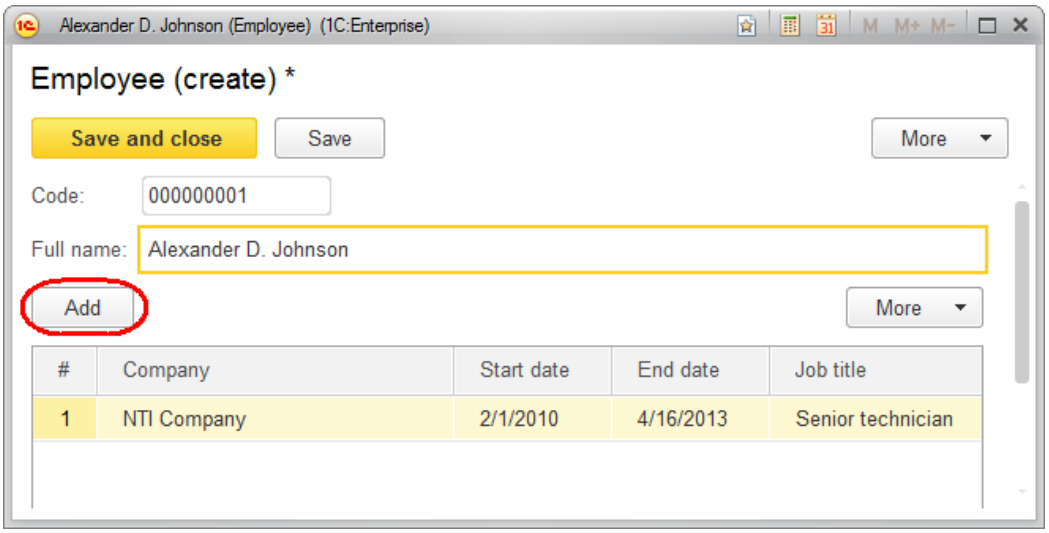


Fig. 3.47. Filling an Employees catalog item

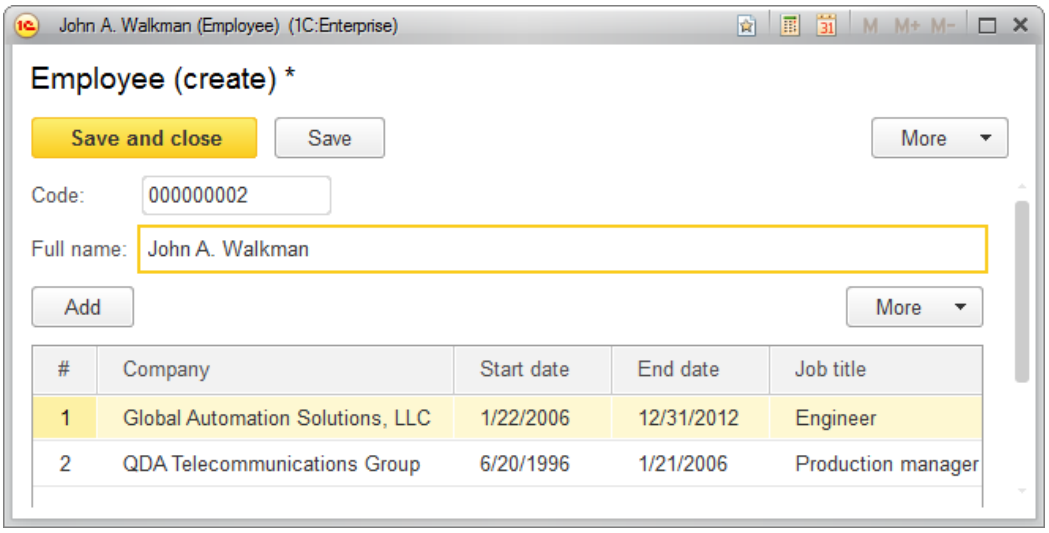


Fig. 3.48. Filling an Employees catalog item

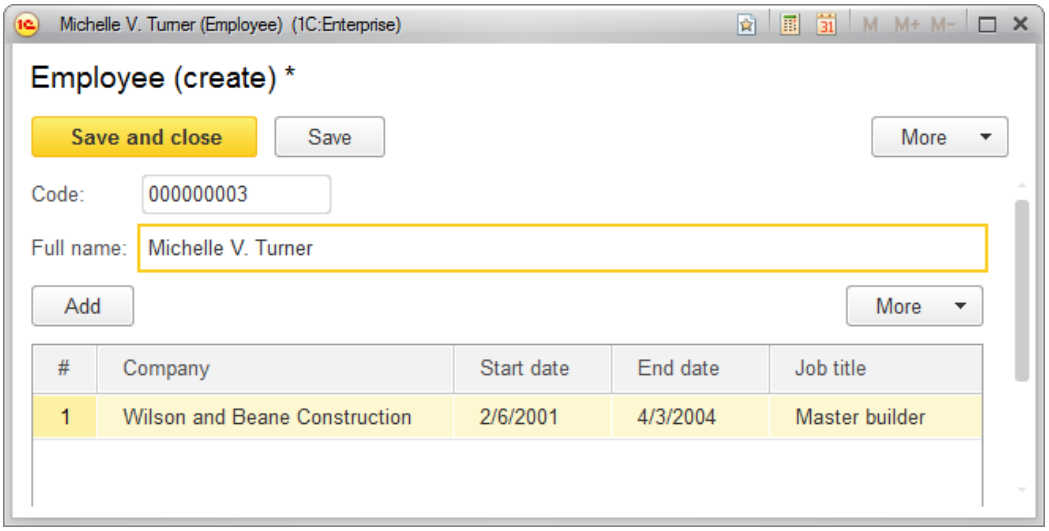


Fig. 3.49. Filling an Employees catalog item

3. Run the **Employees** command.

It opens the employee list form where you can view the list of added employees. Note that the form title is determined by the Extended list presentation property that you defined for this catalog (fig. 3.50).

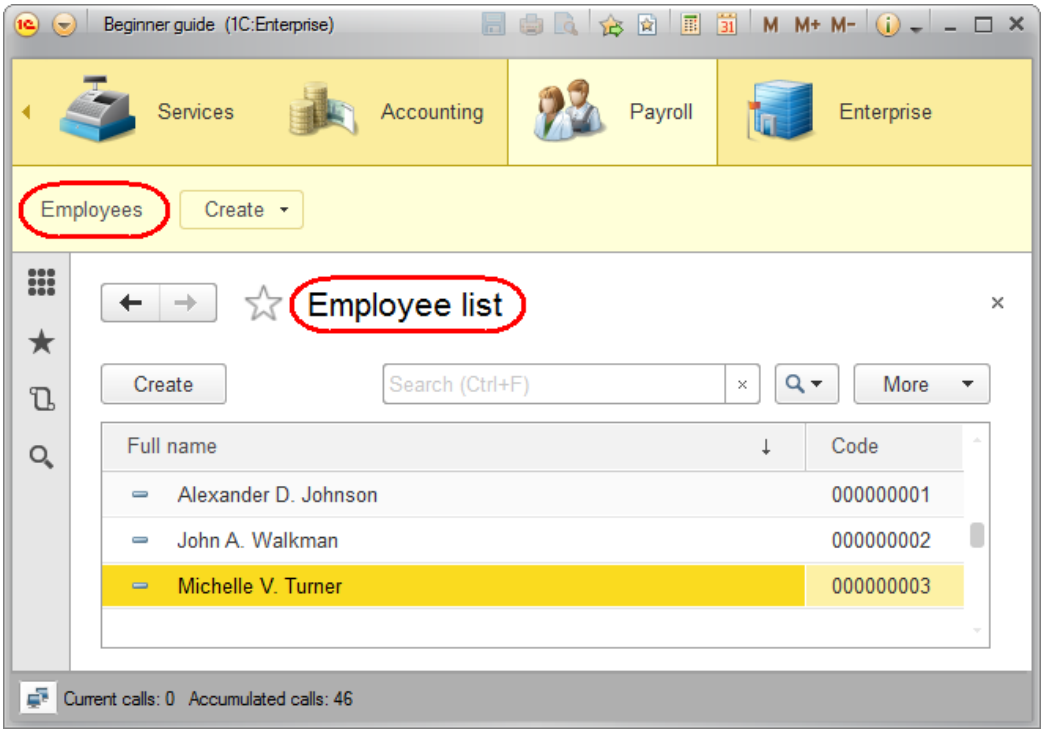


Fig. 3.50. Employee list

Now you can proceed to creating the next catalog: MaterialsAndServices.

Hierarchical catalogs

The MaterialsAndServices catalog will store information about services that the Jack of All Trades company provides, and resources that might be consumed while providing services.

This catalog will not be complex. Its only distinctive feature is its hierarchical structure. To make the catalog user-friendly, you will combine services into a group and resources into another group.

Additionally, since Jack of All Trades provides a wide range of services, you will break them down into several groups. The same is true for resources.

In Designer mode

Let us create the catalog.

1. Create a Catalog configuration object and name it *MaterialsAndServices*.

   The platform automatically generates the synonym based on the name.

2. In the **Object presentation** field, type *Material or service*.

You do not have to fill the List presentation because the catalog synonym Materials and services will be used as a list name.

3. Click the **Subsystems** tab.

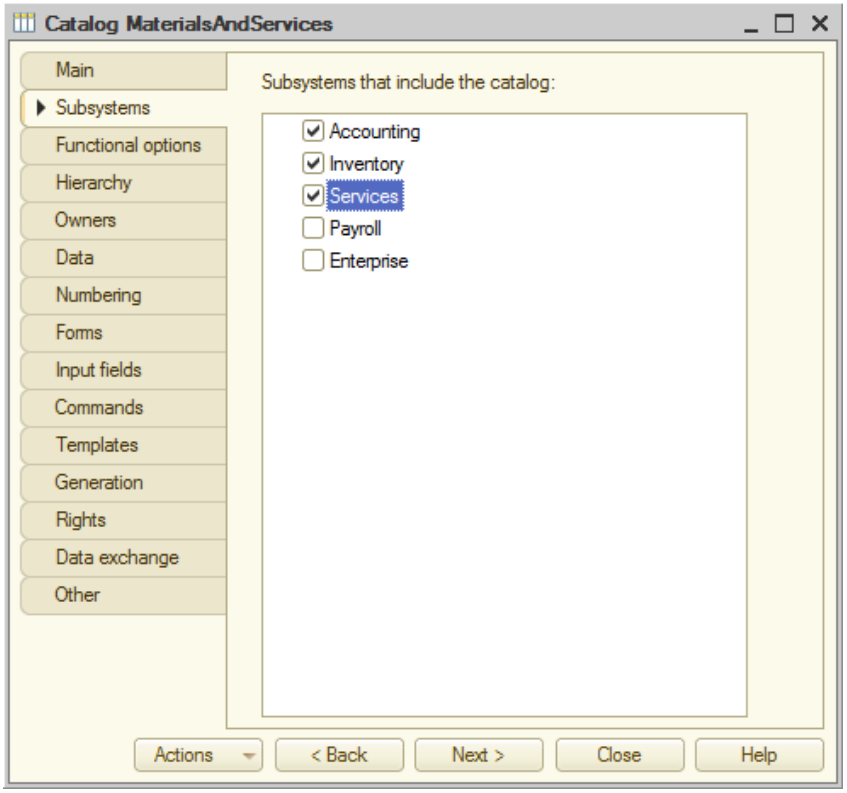


Fig. 3.51. Specifying subsystems where the catalog is available

The configuration logic suggests that the list of materials and services should be available in the Inventory, Services, and Accounting sections. Indeed, this catalog is directly related to the first two sections and the list of materials and services might be frequently needed for accounting analysis.

4. Select the **Accounting**, **Inventory**, and **Services** check boxes (fig. 3.51).
5. Click the **Hierarchy** tab and select the **Hierarchical** catalog check box (fig. 3.52).

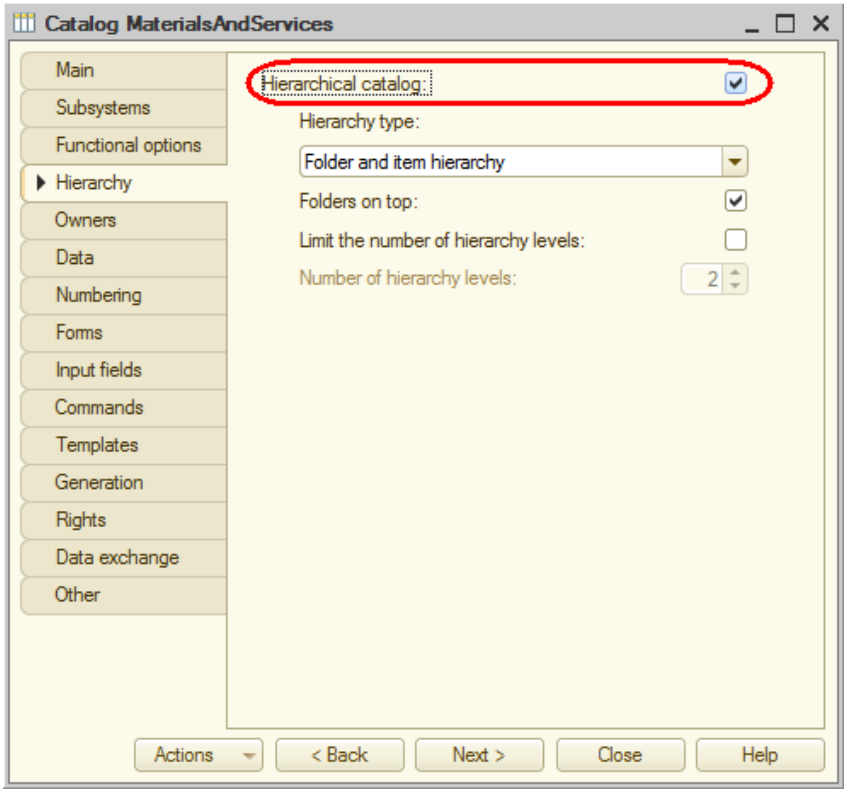


Fig. 3.52. Specifying the Hierarchical catalog property

6. Click the **Data** tab.
7. Leave the default values for the code length and type unchanged, and in the **Description** length field, type *100*.

   Let us also change the synonym of the standard Parent attribute to a more clear word.
8. Click the **Standard attributes** button.
9. Double-click the **Parent** attribute and then, in the property palette, in the **Synonym** field, type *Material or service group*.

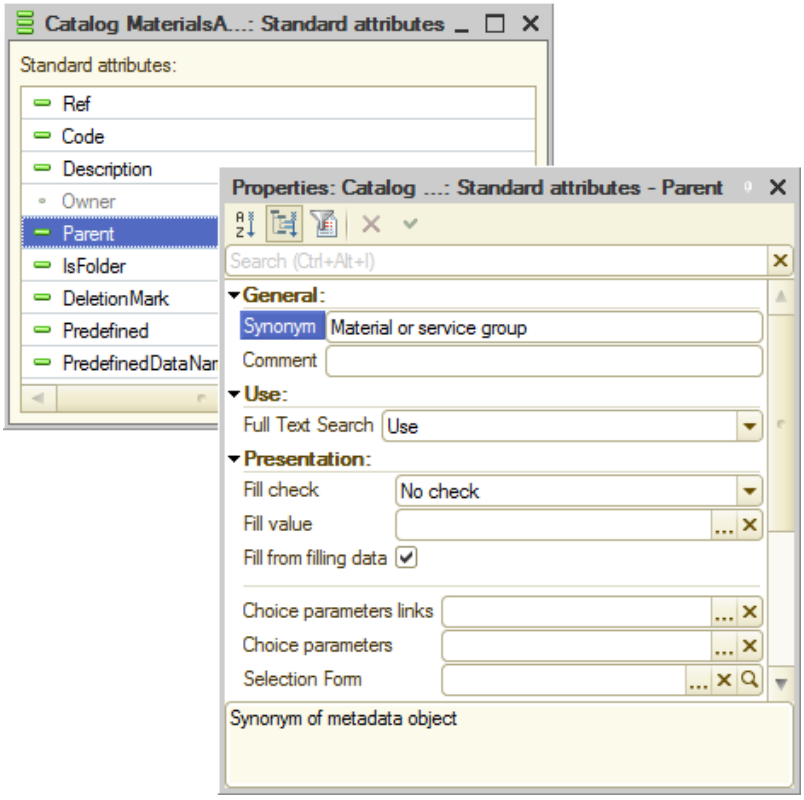


Fig. 3.53. Specifying a synonym for the standard Parent attribute

Before you start 1C:Enterprise, let us configure the applied solution interface to simplify creation of catalog items (make the standard material or service creation command visible on the action panels of the Inventory and Services sections).

10. In the configuration object tree, right-click the **Subsystems** branch and then click **All subsystems** (fig. 3.54).

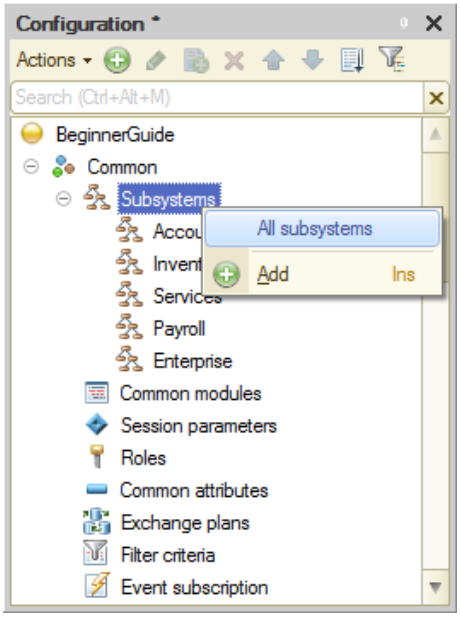


Fig. 3.54. Opening the subsystem settings window

11. In the **All Subsystems** window, click the **Inventory** subsystem.

    The **Command interface** list on the right displays all of the commands of the selected subsystem.

12. In the **Action panel.Create** group, enable visibility of the **Material or service: create** command.

    You can also see that the Navigation panel.Normal group contains the Materials and services command that opens the list of materials and services. It is visible by default (fig. 3.55).

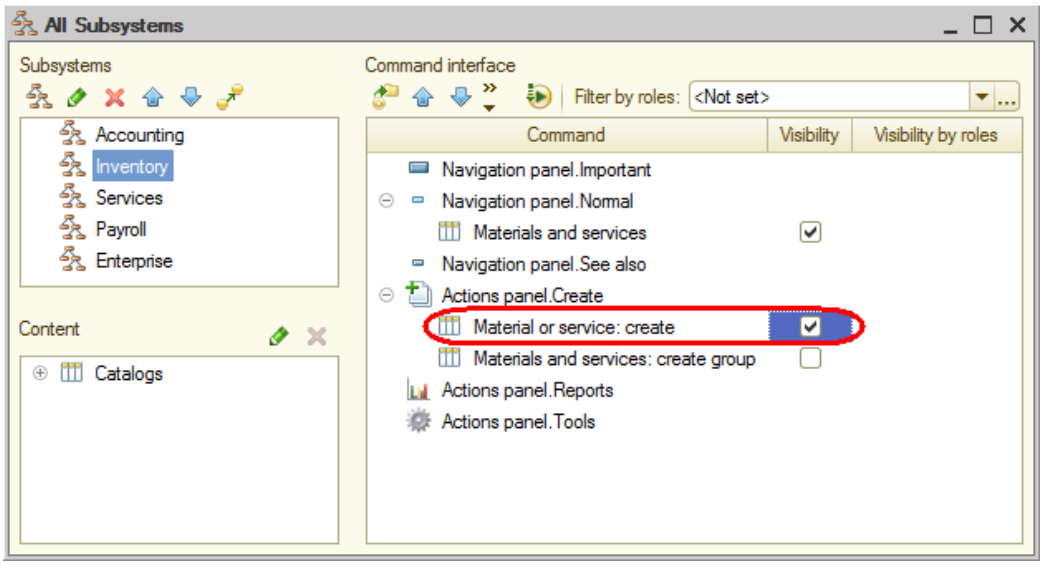


Fig. 3.55. Subsystem settings window

13. In the list of subsystems, click **Services** and repeat step 12.

    You do not have to add any commands to the actions panel of the Accounting subsystem because adding materials or services in this section is not a typical scenario.

    Note. You can also open and customize the command interface for a specific subsystem from the editor of this subsystem by clicking the **Command interface** button. But if you need to customize the command interface for multiple subsystems at once, the **All Subsystems** editor comes handy.

    Next, let us populate the MaterialsAndServices catalog. On this example you will learn how to create groups and move items between groups.

14. Close the **MaterialsAndServices** catalog editor and start the applied solution in the debug mode.

15. Confirm that you want to update the configuration.

    The platform displays the window that lists the changes in the configuration structure (the new MaterialsAndServices catalog).

16. Click **Accept** (fig. 3.56).

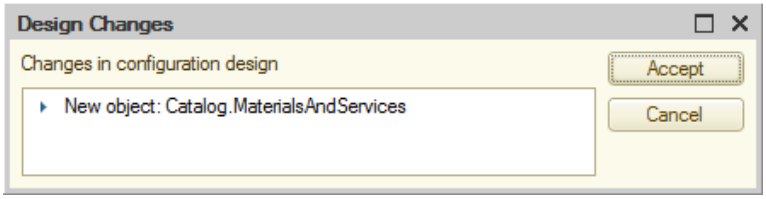


Fig. 3.56. List of changes in the configuration structure

In 1C:Enterprise mode

In the 1C:Enterprise window you can see that the command panels of the Inventory, Services, and Accounting sections now include the Materials and services command (see fig. 3.57).

The command name is determined by the object synonym because you have not defined any other presentations for this catalog.

- In the **Inventory** section, in the navigation panel, click the **Materials and services** command.

  This opens the Materials and services list form.

Creating items in a hierarchical catalog

Let us create two groups in the catalog root: Materials and Services.

1. In the command bar of the list form, click **Create group** (fig. 3.57).

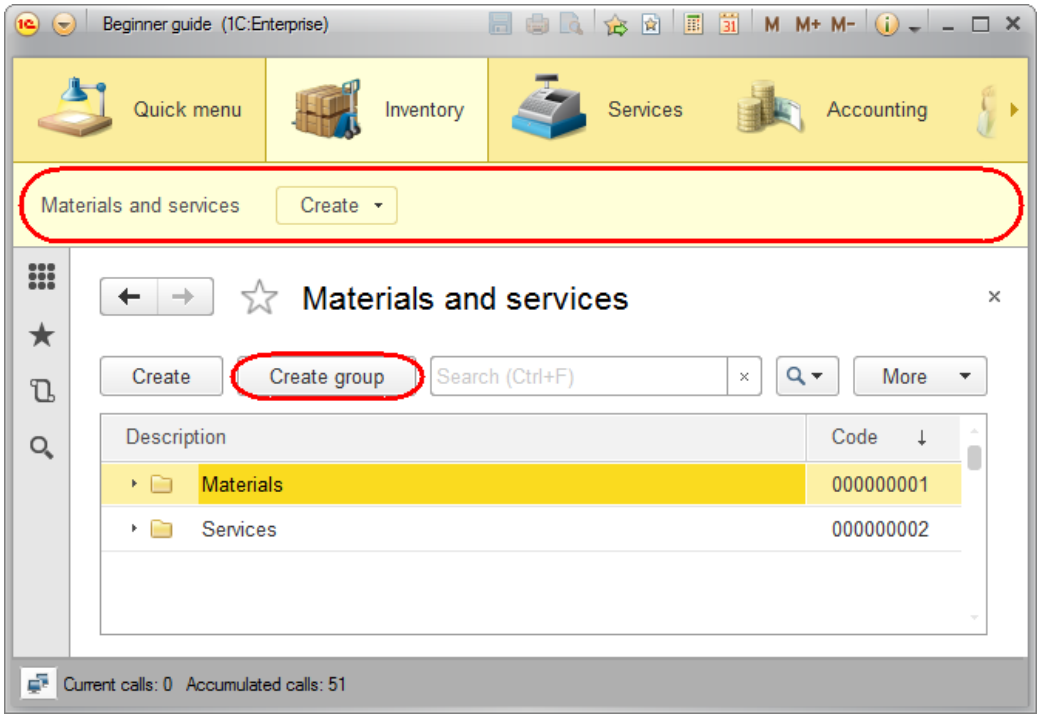


Fig. 3.57. Creating groups in the MaterialsAndServices catalog

2. Type the following group names: *Materials* and *Services*. Leave the **Parent** and **Code** fields empty.

   Next, let us fill the groups with items.

You can add an item to a group that is currently open by clicking **Create** in the command bar of the catalog list form. This opens a catalog item creation form. Note that if you add an item to a group that is currently open, this group is automatically added to the **Material or service group** field (fig. 3.58).

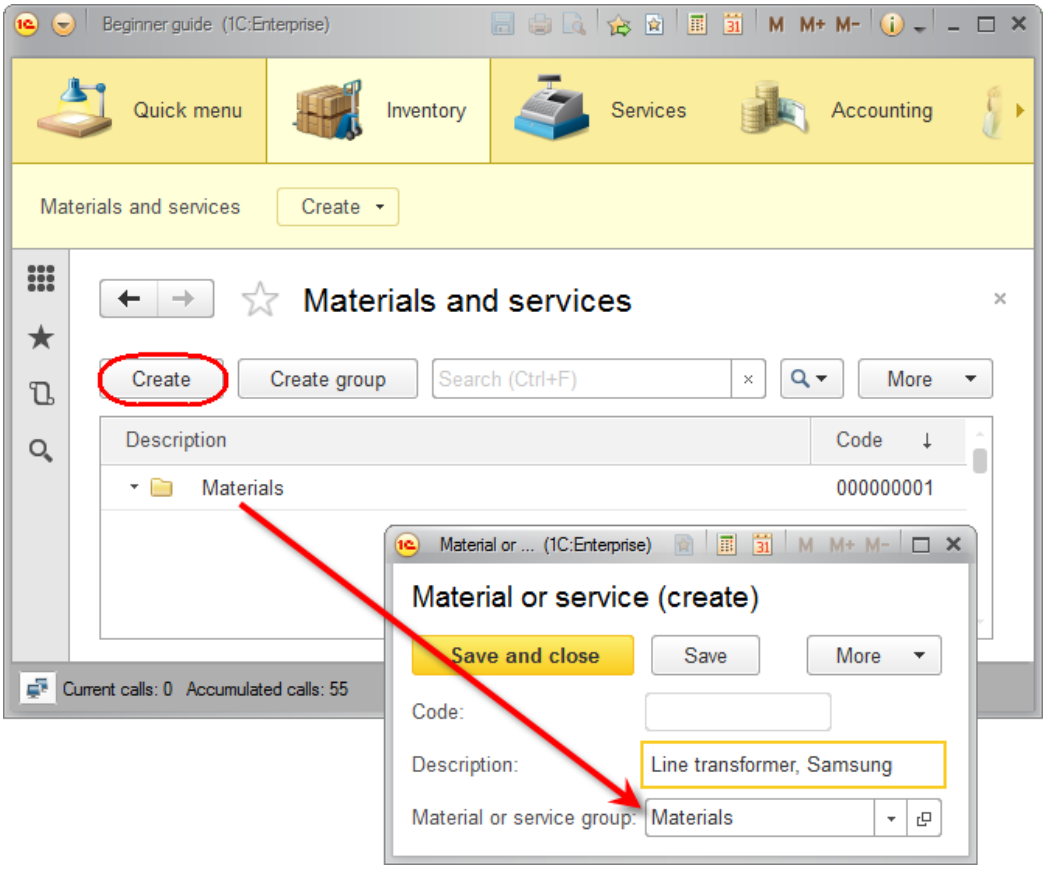


Fig. 3.58. Creating items in the Materials group

To create a catalog item, you can also use the **Material or service** command that is available in the **Create** menu of the **Inventory** and **Services** sections.

If you add an item using this method, it is not connected to the materials and services list in any way. So the platform cannot determine the item group automatically, and you have to specify it manually (fig. 3.59).

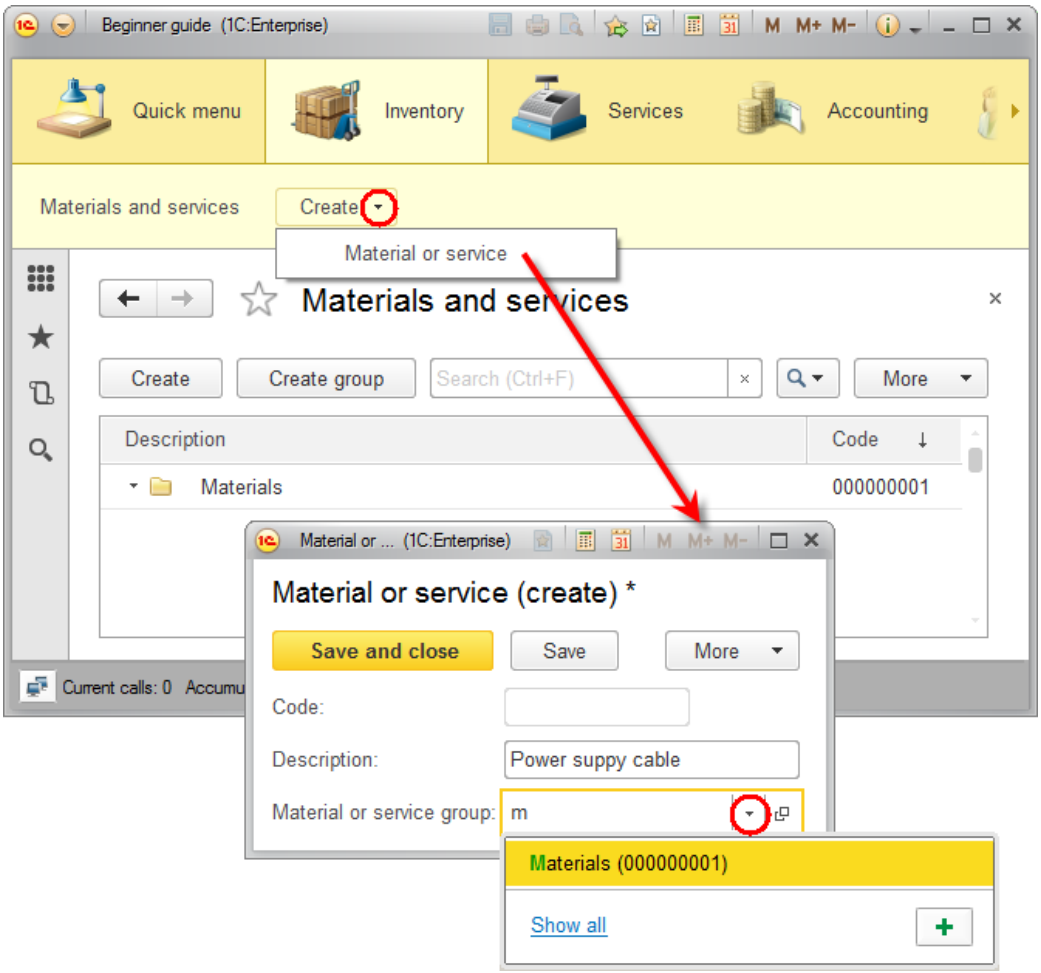


Fig. 3.59. Creating items in the Materials group

When specifying a value in the **Material or service group** field, you can use the handy "input by string" option. Once you type a few letters, the platform prompts you to select from a list of matching values.

You do not need to open the materials or services list when you create materials or services. The fact that an item is added to the catalog is confirmed by a message in the bottom corner of the applied solution window.

3. Double-click the **Materials** group to expand it and create the following items in this group:
   - Line transformer, Samsung
   - Line transformer, LG
   - Transistor 2N2369, Philips
   - Tubing, rubber
   - Power supply cable
4. Expand the **Services** group and create the services related to TV set repair (fig. 3.60):
   - Diagnostics
   - Domestic TV set repair

- Imported TV set repair

And the services related to washing machine installation:

- Water hookup
- Electrical hookup

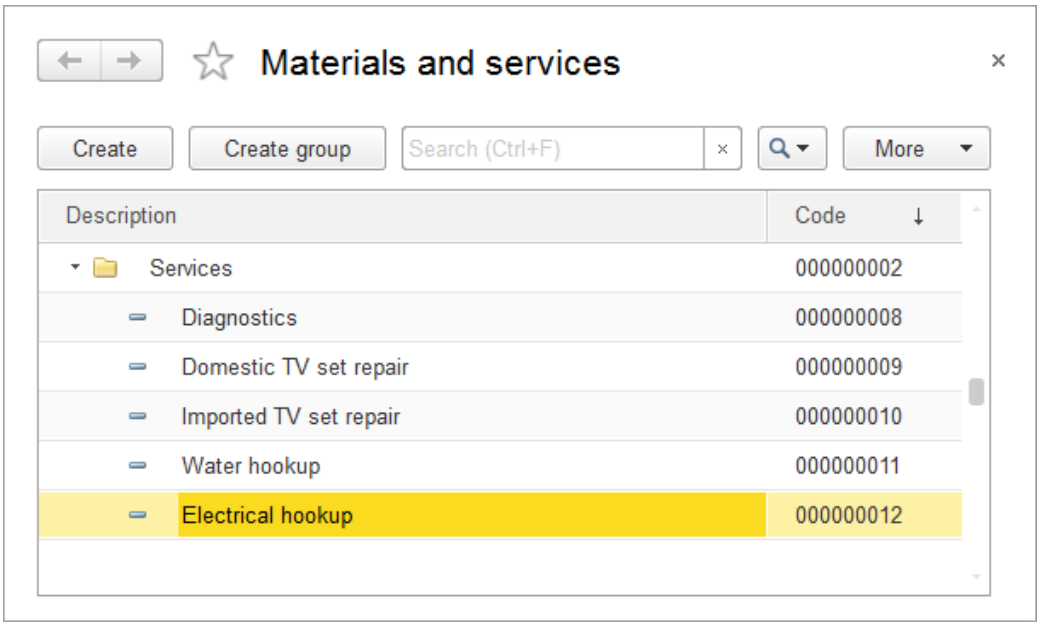


Fig. 3.60. Creating items in the Services group

Moving items between groups

Now let us divide the services into logical groups.

1. Create two more groups in the **Services** group: *TV sets* and *Washing machines* (fig. 3.61).

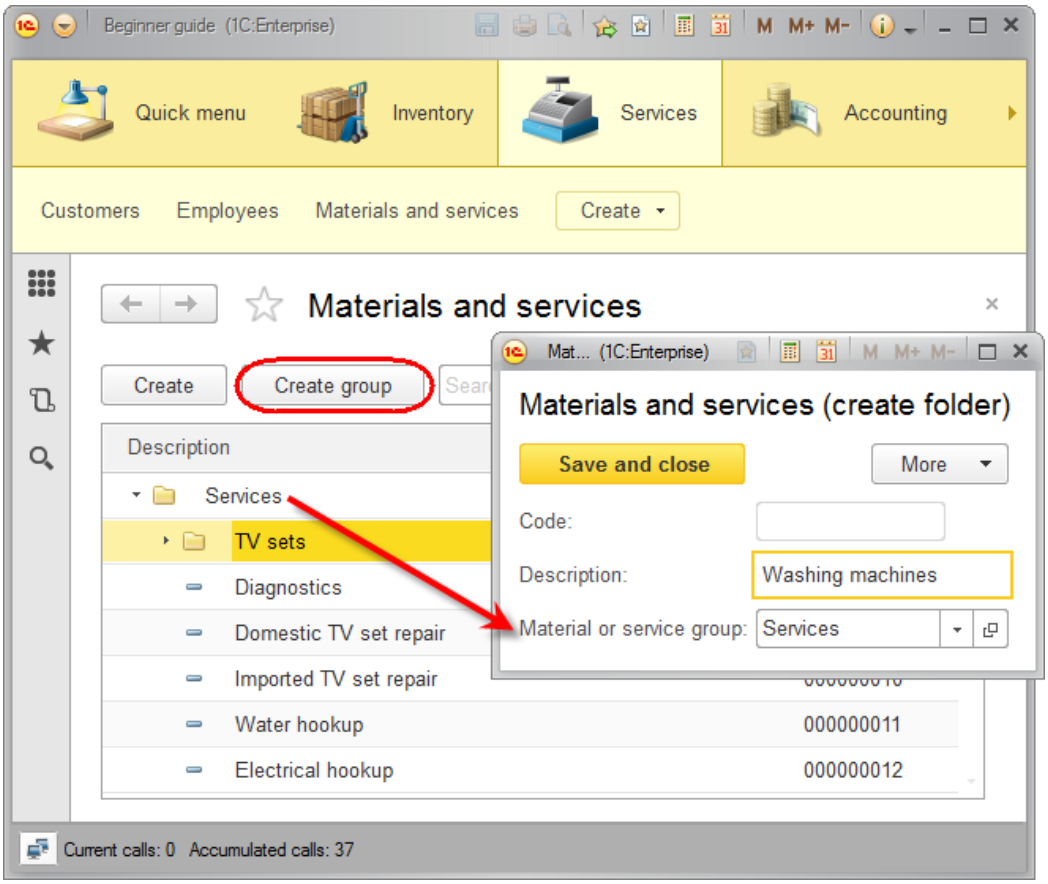


Fig. 3.61. Creating a group in the Services group

You can move a service to a group by dragging an item to a group, or by selecting an item and then clicking **Move to folder** in the **More** menu of the list form command bar.

You can also open a catalog item and change the value in the **Material or service group** field (fig. 3.62).

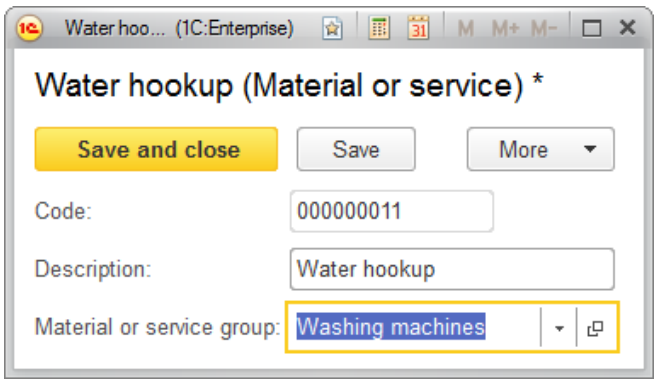


Fig. 3.62. Moving a catalog item to another group

2. Move the **Diagnostics**, **Domestic television repair**, and **Imported television repair** services to the **TV sets** group.
3. Move the **Water hookup** and **Electrical hookup** services to the **Washing machines** group.
4. In the **Materials** group, create the *Radio parts* group.

5. Move **Line transformer, Samsung**, **Line transformer, LG**, and **Transistor 2N2369, Philips** to the **Radio parts** group.

   Now let us move the rest of the items to the Miscellaneous group. This group is not yet available and now you will learn how to create a catalog item when you fill a field that should contain a reference to this item.

6. In the **Materials** group, open the **Power supply cable** item.

7. In the value selection list, click the button.

   The platform detects that you want to create a group and opens the catalog item creation form (fig. 3.63).

8. In the **Description** field, type *Miscellaneous*.

9. In the **Material or service group** field, type *Materials*.

10. Click **Save and close**.

    The platform creates the Miscellaneous group in the Materials group, and then it adds a reference to the Miscellaneous group to the Material or service group field of the Power supply cable item.

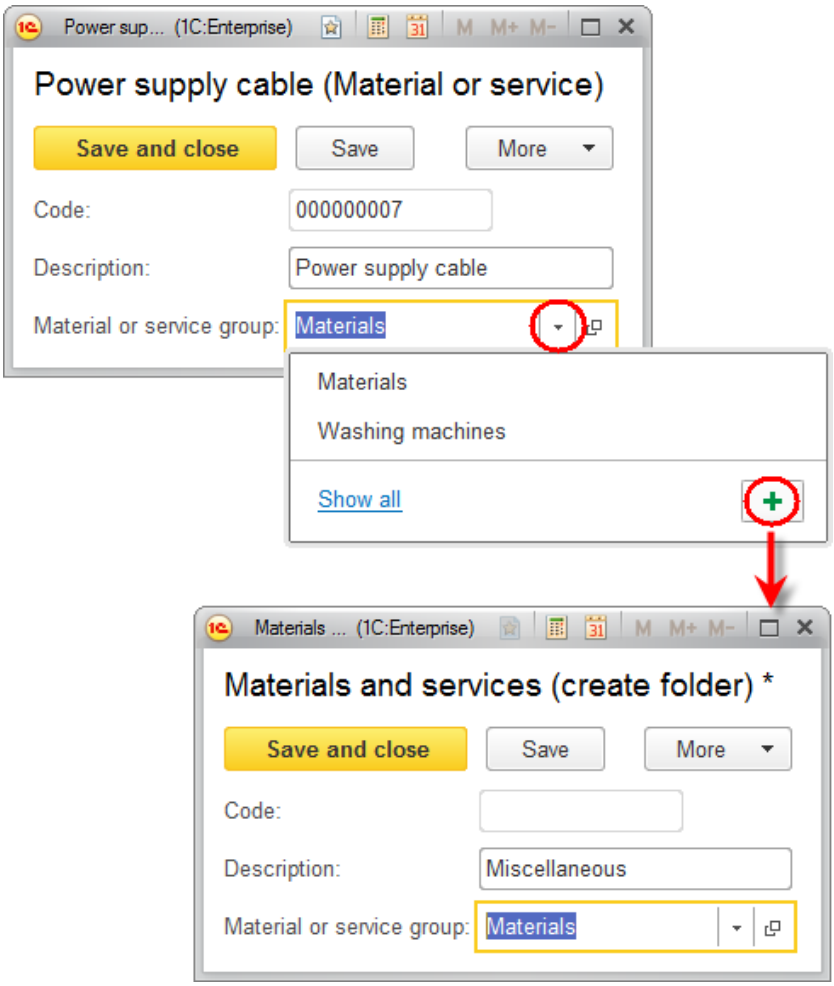


Fig. 3.63. Creating a material or service item from the Material or service group field

11. Move the **Tubing, rubber** item to the **Miscellaneous** group.

    Now let us view the list of products and services as a tree.

12. In the command bar of the Materials and services list, in the **More** menu, point to **View mode** and click **Tree**.

    The Materials and services list now looks as follows (fig. 3.64).

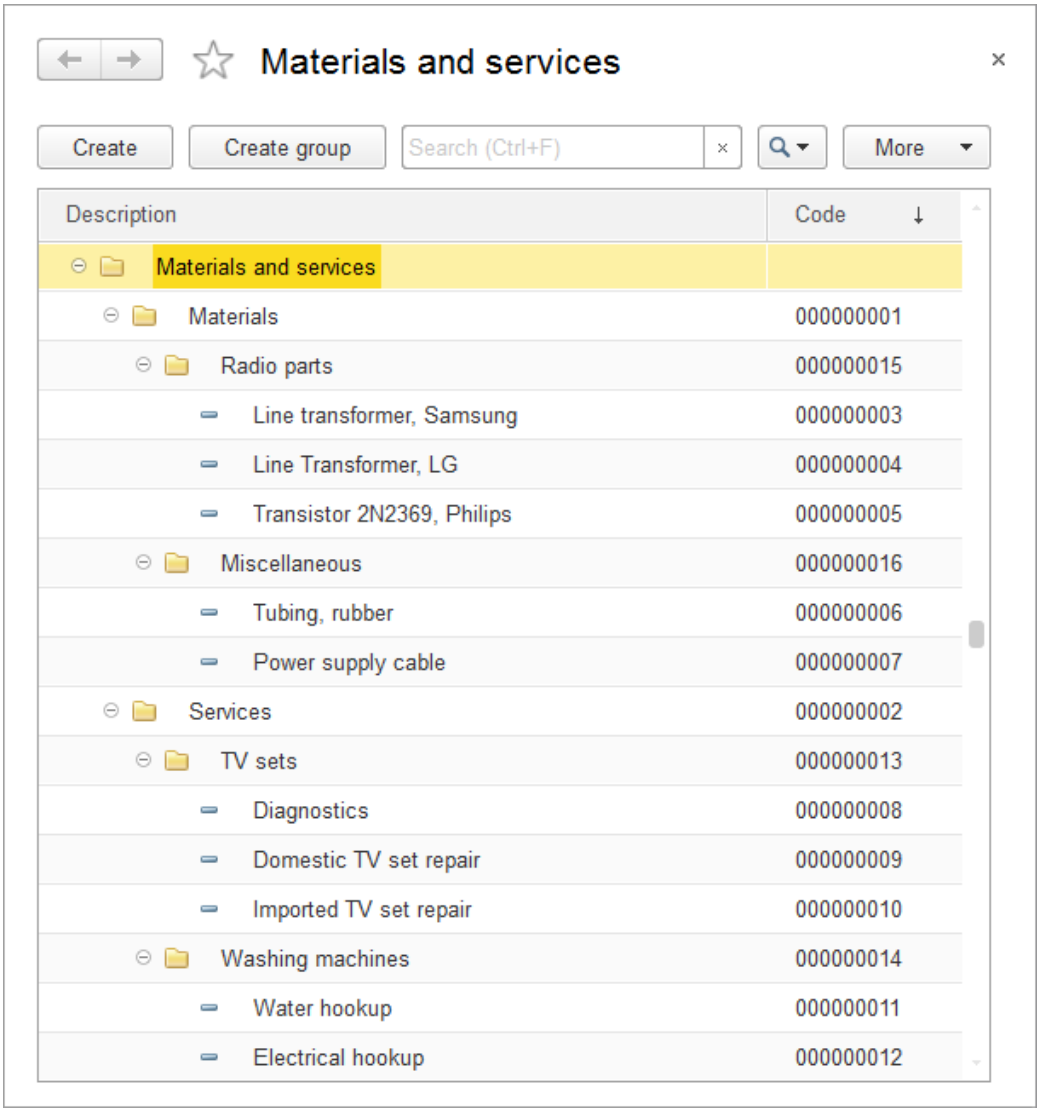


Fig. 3.64. Tree view of the Materials and services list

Catalogs with predefined items

Finally, you will create the Warehouses catalog, which will store warehouses used by Jack of All Trades.

This catalog will contain a predefined item: the Main warehouse where all the materials are delivered.

In Designer mode

Let us create the catalog.

1. Open Designer and create a Catalog configuration object named *Warehouses*.

   The platform fills the synonym automatically based on the name.

2. In the **Object presentation** field, type *Warehouse*.

   You do not have to fill the List presentation property because the platform will use the Warehouses synonym (fig. 3.65).

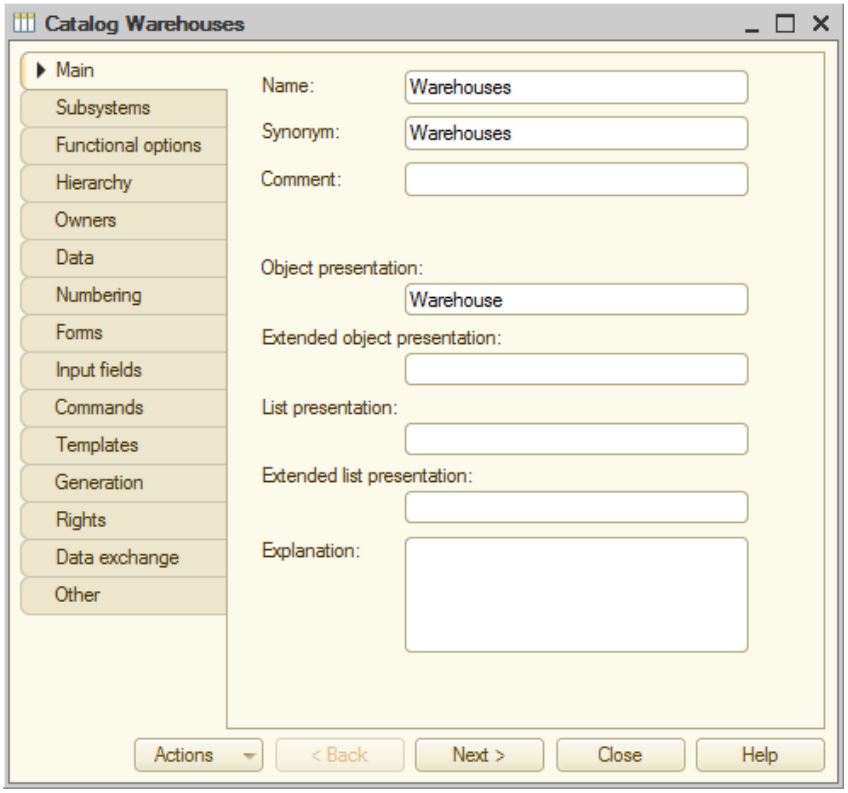


Fig. 3.65. Specifying main catalog properties

3. Click the **Subsystems** tab.

   The configuration logic suggests that the list of warehouses should be available in the Services and Inventory sections since deliveries of materials and rendering of services are normally recorded for warehouses.

4. Select the **Inventory** and **Services** check boxes (fig. 3.66).

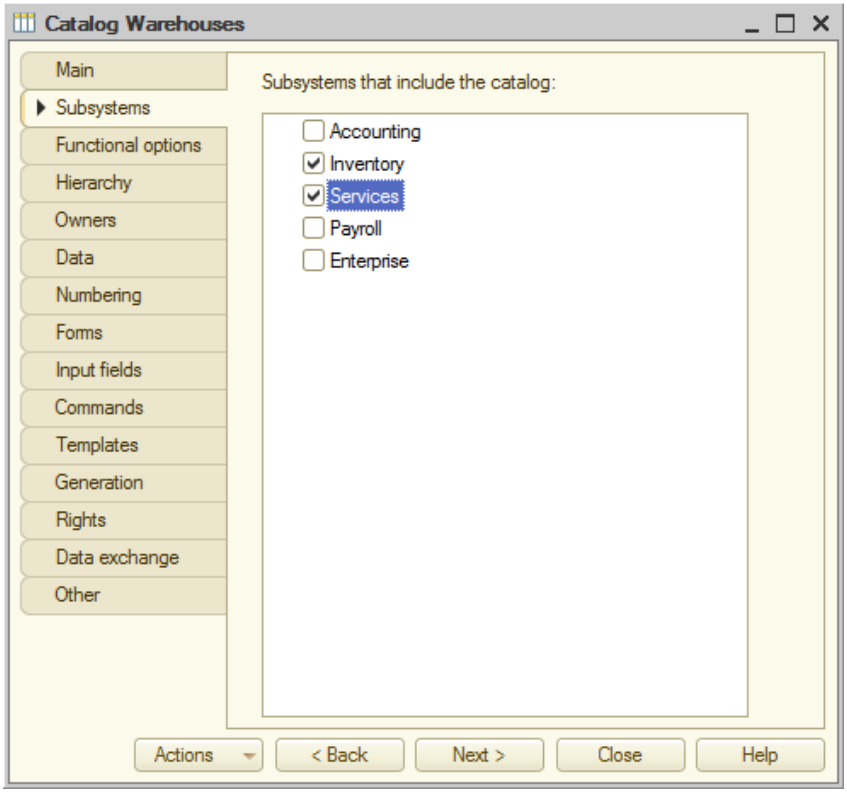


Fig. 3.66. Specifying subsystems where the catalog is available

Quick choice property

Let us fill another Warehouses catalog property: Quick choice.

- Click the **Input fields** tab and select the **Quick choice** check box (fig. 3.67).

Fig. 3.67. Specifying the Quick choice property

By default, this check box is not selected and clicking a selection button in the field that references to a catalog item opens the catalog item choice form. It might not be user-friendly, especially when the catalog is not hierarchical and is known to include only a small number of items.

The Quick choice property allows selecting items from a small drop-down list filled with items of this catalog instead of using a dedicated form (fig. 3.68).

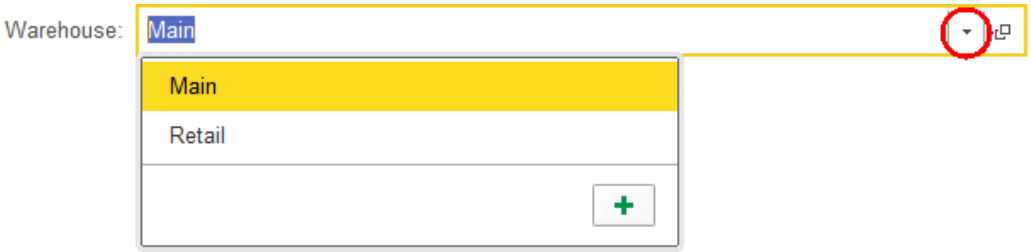


Fig. 3.68. Selecting a warehouse from a drop-down list

This option is handy for the list of warehouses because this list will likely contain just a few items.

You have not set the Quick choice property for the other catalogs. This is because Materials and services is a hierarchical catalog and quick selection does not make sense for this catalog. The list of employees and especially the list of customers can be very large and the drop-down list might require too much scrolling, which is inconvenient.

Predefined items

Let us create a predefined item.

1. Click the **Other** tab and then click the **Predefined data** button.

   This opens the list of predefined catalog items. At this point the list is empty.

2. Click the **Add** button and create a predefined item named *Main* (fig. 3.69).

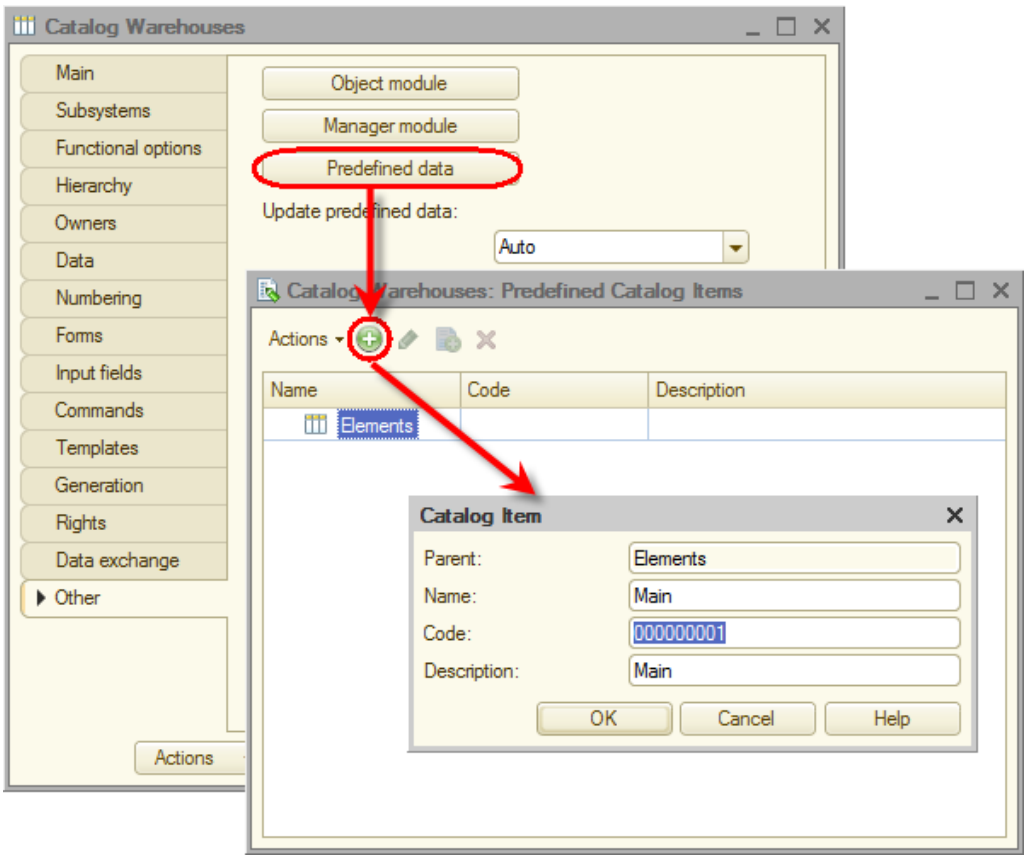


Fig. 3.69. Creating a predefined catalog item

Note that in addition to the description the predefined catalog item has a name.

In the future, when you start learning 1C:Enterprise script, you will be able to refer to this catalog item using the name that you assigned in Designer. While a predefined catalog item description can be edited by users, the name is not visible to users and therefore they cannot edit it.

Before you start 1C:Enterprise, let us configure the applied solution interface to streamline catalog item creation (make the standard warehouse creation command visible in the Inventory section).

3. In the configuration object tree, double-click the **Inventory** subsystem and then click the **Command interface** button.

   This opens the **Command interface** window, which lists all of the subsystem commands.

4. In the **Actions panel.Create** group, enable visibility for the **Warehouse: create** command.

   You can also see that the Navigation panel.Normal group contains the Warehouses command, which opens the list of warehouses. The command is visible by default (fig. 3.70).

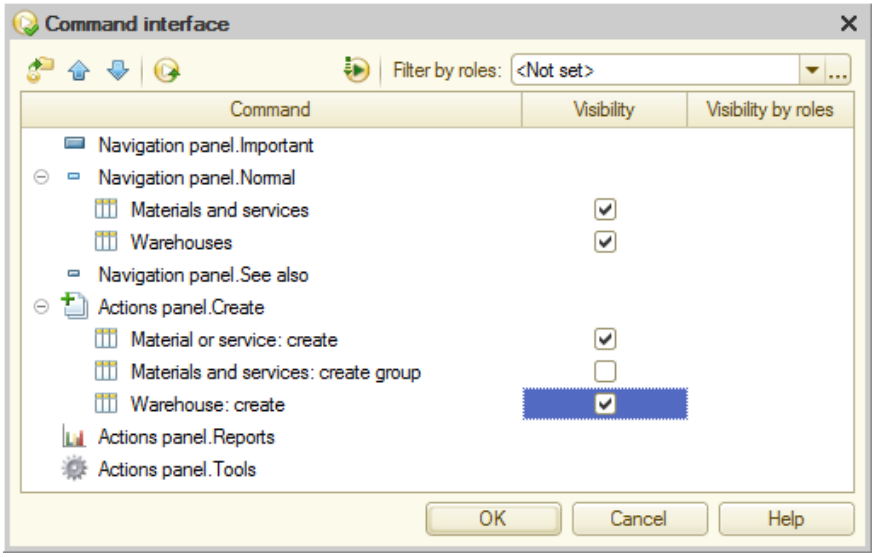


Fig. 3.70. Subsystem command interface settings window

5. Close the **Warehouses** catalog editor and start 1C:Enterprise in the debug mode.
6. Confirm that you want to update the configuration.

   The platform displays the window that lists the changes in the configuration structure (the new Warehouses catalog).
7. Click **Accept** (fig. 3.71).

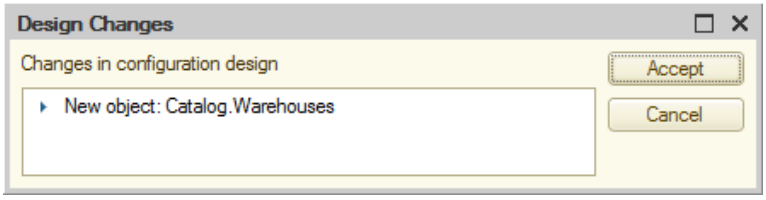


Fig. 3.71. List of changes in the configuration structure

In 1C:Enterprise mode

In the 1C:Enterprise window you can see that the **Create** menu of the **Inventory** section contains the **Warehouse** command that creates warehouses (fig. 3.72).

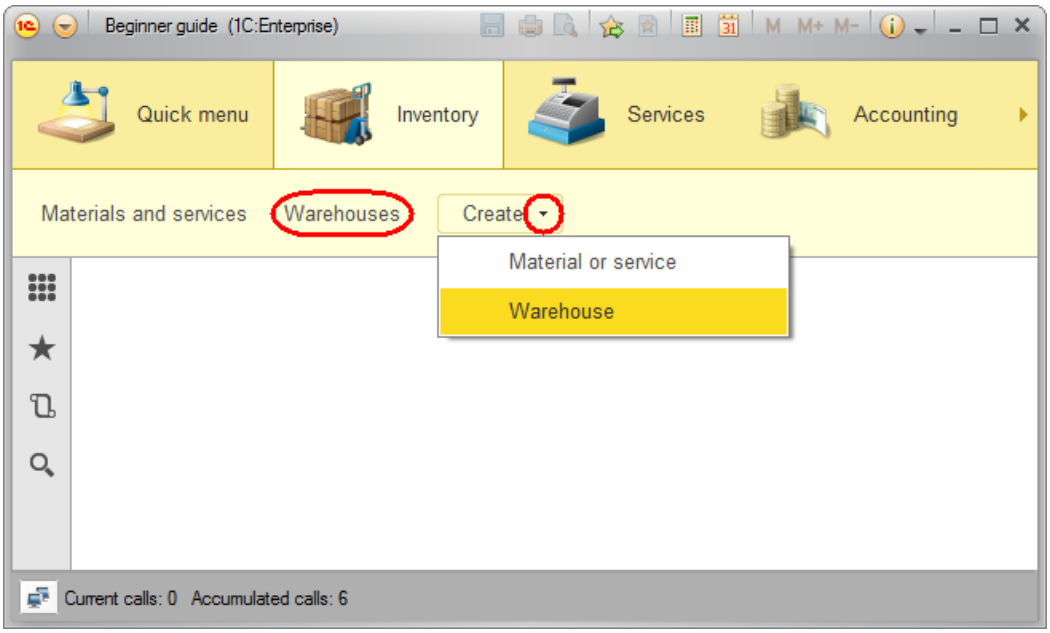


Fig. 3.72. The Inventory section

The command name is defined by the Object presentation property that you specified for this catalog.

Also, the Warehouses command that opens the list of warehouses is added to the Services and Inventory sections (see fig. 3.72). The command name is defined by the configuration object synonym because you have not specified the List presentation for the catalog.

1. In the **Inventory** section, run the **Warehouses** command.

   This opens the warehouse list form. The list of warehouses contains the **Main** item. This is the predefined item that you created in Designer.

2. On the **Create** menu, run the **Warehouse** command and create another item named *Retail* (fig. 3.73).

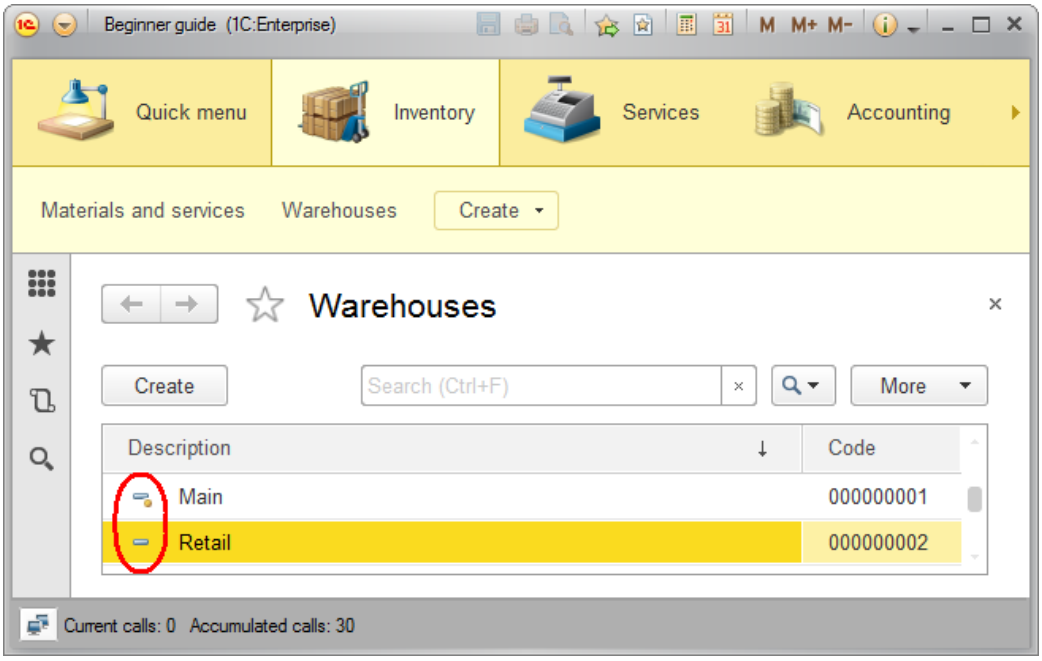


Fig. 3.73. Items of the Warehouses catalog

With that we will wrap up the preliminary work of creating catalogs, and will discuss some theory to explain the predefined items and the prompts that you see on the screen when you start or continue debugging.

Theory

Predefined items

Note that the platform uses different icons for regular and predefined catalog items (see fig. 3.73).

Despite the fact that users can edit the code or description for both items, they cannot edit the predefined item names that you specify in Designer. Later you will use these names to call the predefined items from 1C:Enterprise script.

So, the configuration algorithms can be based on predefined items.

Now you can see the main difference between regular and predefined catalog items from the configuration point of view.

Regular items are not permanent. Users can create and delete them. Although the configuration can distinguish them from each other, you cannot use them in the algorithms because they can disappear at any time.

Predefined items are permanent. They are always available, regardless of user actions.

In theory, users can delete predefined items but we will not give them rights for interactive deletion of predefined objects (or interactive deletion of any objects at all, see "Lesson 22. Users and their roles" on page 601.

Therefore, the configuration algorithms can rely on predefined items. This is why each predefined item has a unique name, so that it can be referenced using 1C:Enterprise script.

Base configuration and database configuration

Up to this point we have not dug deeply into the structure of the 1C:Enterprise system, but this is a good point for a few words on the subject.

You remember that, from the user perspective, a 1C application consists of the platform and the applied solution (configuration). We have already mentioned that each applied solution is a single configuration. But at this point we have to say this is not exactly true.

Why not true? Because each infobase has a minimum of two configurations.

Why not exactly true? Because in practice a user always works with a single configuration. The other configuration is intended for developers or for those who modify the configuration (for example, infobase administrators). It is not visible to users.

The configuration intended for developers is referred to as *base configuration* (or simply *configuration*). This is the one that you edit in Designer.

The configuration intended for user work is referred to as *database configuration*.

A base configuration can be edited, while a database configuration cannot be edited. Instead, a database configuration can be updated based on the base configuration.

But you may naturally ask the question: if I have two configurations, one that can be edited and another one where users work, why the editable one is referred to as the base configuration? From the end-user perspective, the base configuration is the one they work with.

But the base name is given from the developer perspective and makes great practical sense.

In general, a 1C:Enterprise infobase can store more than two configurations: the base configuration, the database configuration, and multiple vendor configurations.

Besides, a repository can exist outside of an infobase. It stores a configuration that is intended for shared development.

A certain number of configuration files can exist outside of the infobase as well, including files belonging to a new distribution package (fig. 3.74).

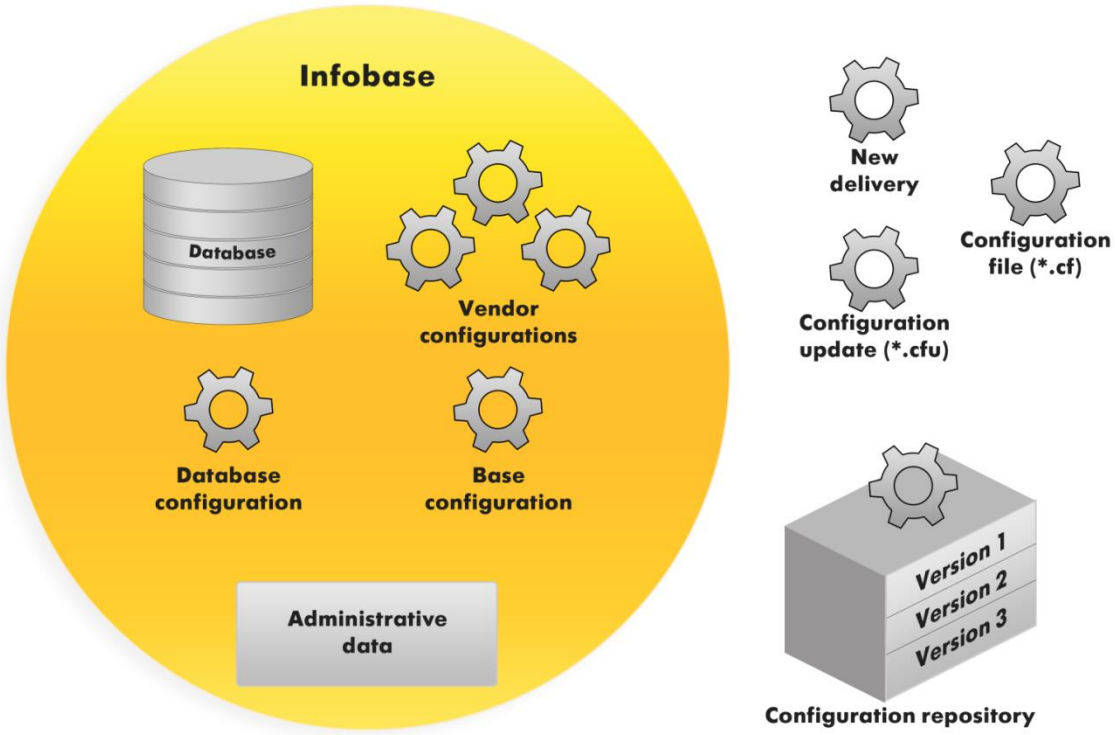


Fig. 3.74. Configuration structure

A vendor configuration within an infobase stores the previous delivery states. There are also scenarios where a configuration is supported by multiple vendors, each of them supporting their part as a standalone configuration. In this case the infobase contains multiple vendor configurations (the previous delivery state for each vendor).

New delivery files can exist in the form of configuration files (full distribution) or update files (update distribution).

A configuration repository contains a configuration intended for shared development. This configuration is stored as a set of individual objects assigned to specific configuration versions, rather than as a complete configuration. This allows developers to get a configuration version from the repository by assembling it from the individual objects.

Now imagine that there is a way to perform comparisons and updates between all these configuration types. In this case, it is easy for things to get into a muddle, so the name *base configuration* is the best way to reflect the overall goal of these changes.

Now that we are back to the base configuration and database configuration, note that internal division into two configurations gives developers the option to make modifications without any user work interruptions. This is possible because all the modifications are introduced into the base configuration that is not available to users.

Then, when the developer is confident that all the modifications are correct, they can use the base configuration to quickly update the database configuration.

If such modifications do not affect the database structure (for example, database tables do not need to be modified, only module script is changed), a database configuration can be updated without interrupting user work. This process is referred to as *dynamic update*.

Users will only see the changes after they restart their application. The DataBaseConfigurationChangedDynamically() method of 1C:Enterprise script is intended for checking whether the applied solution restart is required.

But if such modifications affect database structure (for example, a catalog has a new attribute or an existing attribute has a new type), all the user sessions must be terminated. The developer can compare the base configuration and the database configuration at any time, and they can roll back to the original version of the base configuration by reading the database configuration (if, for example, the introduced changes are a total mess).

So, the relationship between the two configurations can be depicted as follows (fig. 3.75).

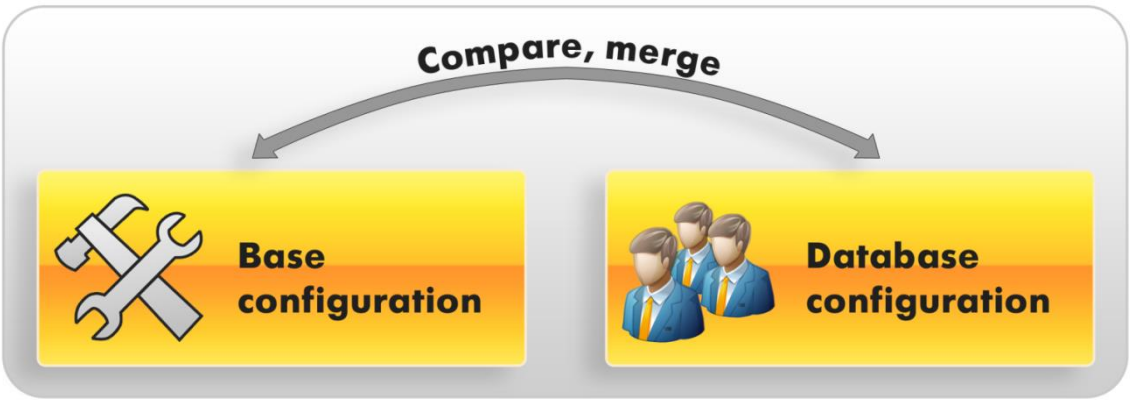


Fig. 3.75. Relationship between base and database configurations

When a developer is working in the base configuration, the platform provides them with an indication whether the current version of the base

configuration is different from the saved version, and whether the saved version of the base configuration is different from the database version.

If the developer is editing the base configuration and the edited version of the base configuration differs from the one that is saved, an asterisk (*) is displayed in the title of the configuration tree window, indicating that the configuration has been modified (fig. 3.76).

Fig. 3.76. Configuration tree window title

If the saved version of the base configuration is different from the database configuration, an exclamation mark (<!>) is displayed in the title of the configuration tree window (fig. 3.77).

Fig. 3.77. Configuration tree window title

To save the base configuration, on the **Configuration** menu, click **Save configuration**.

To update a database configuration, on the **Configuration** menu, click **Update database configuration**.

Once you click **Start debugging** on the **Debug** menu, the platform saves the base configuration and then compares it with the database configuration. If the configurations are different, you are prompted to update the database configuration as you saw in the preceding examples. Once you click **Continue Debugging** on the **Debug** menu, after the actions described above, the platform also prompts you to restart the applied solution in order to terminate the current debug session.

This is how the platform simplifies configuration development and automates frequently used actions.

When the database configuration is updated, the platform creates (or modifies) the database data structures that are defined as configuration objects.

When users add regular catalog items, they add them to the database structure based on the Catalog configuration object.

The platform adds predefined catalog items to that database structure according to their descriptions in the Catalog configuration object.

Again, this proves the fact that regular catalog items are not important from the configuration point of view, while predefined items are important because they might be used in the configuration algorithms.

Quiz

- What are the uses of the Catalog configuration object?
- What are the distinctive catalog features?
- How does one use the attributes and tabular sections of a catalog?
- What is the purpose of hierarchical catalogs and meaning of parents?

- What is the purpose of subordinate catalogs and meaning of owners?
- What are the default catalog forms?
- What are predefined items?
- What is the difference between regular catalog items and predefined items from the configuration perspective?
- How can users tell the difference between regular catalog items and predefined items?
- How does one create a Catalog configuration object and define its structure?
- How does one specify a synonym for a standard attribute?
- How does one add items to a catalog?
- How does one create a catalog group?
- How does one move items between catalog groups?
- What are the uses of the base configuration and database configuration?
- How does one modify the database configuration?
- What is the relationship between configuration objects and database objects?
- What is a subordinate configuration object?
- What is the purpose of catalog attribute fill check?
- What is quick choice and when should one use it?
- How does one display a catalog and define a presentation for the catalog in the applied solution interface sections?
- How does one display the commands that create catalog items in the subsystem interface?
- How does one edit the command interface of subsystems?
- What are the standard panels used in the applied solution interface and how does one set up the panel positions in Designer and in 1C:Enterprise mode?

Lesson 4 (1:30). Documents

In this lesson you will study the Document configuration object. You will learn the purpose of this object, its structure, and its main properties.

Then you we will create several documents and learn how a developer can introduce custom algorithms that enhance document functionality.

Additionally, you will learn how to create a document form, familiarize yourself with some of the 1C:Enterprise script expressions, and learn what a type-specifying configuration object is.

Understanding documents

The Document configuration object is intended for storing data related to completed financial operations or events that occurred within a company. Normally, every company uses such documents as goods receipts, hiring orders, sales invoices, or bills. The Document configuration object describes the structure and properties of these documents. The platform uses these objects to create database tables that store document data.

The document logic is different from the logic of other configuration objects. A document can be posted. Events recorded in posted documents affect the accounting records.

When a document is not yet posted, it does not affect the accounting records, at this stage the document is nothing more than a draft or a blank. Once a document is posted, the accounting operations described in the document take effect and the accounting records are changed accordingly.

Since a document changes accounting records, it is always tied to a specific point in time. This ensures reflecting the actual sequence of events in database records.

The next important observation, which follows from the previous two, is that the 1C:Enterprise system includes functionality for monitoring the accuracy of accounting records. If you change any of the previously posted documents and then repost them, 1C:Enterprise can monitor whether your changes would have an effect on subsequently posted documents and, if so, it can repost all of the affected documents.

Users can create documents such as sales and purchase invoices, bills, and so on.

In the database each document is an individual record in the main table that stores information about this document type (fig. 4.1).

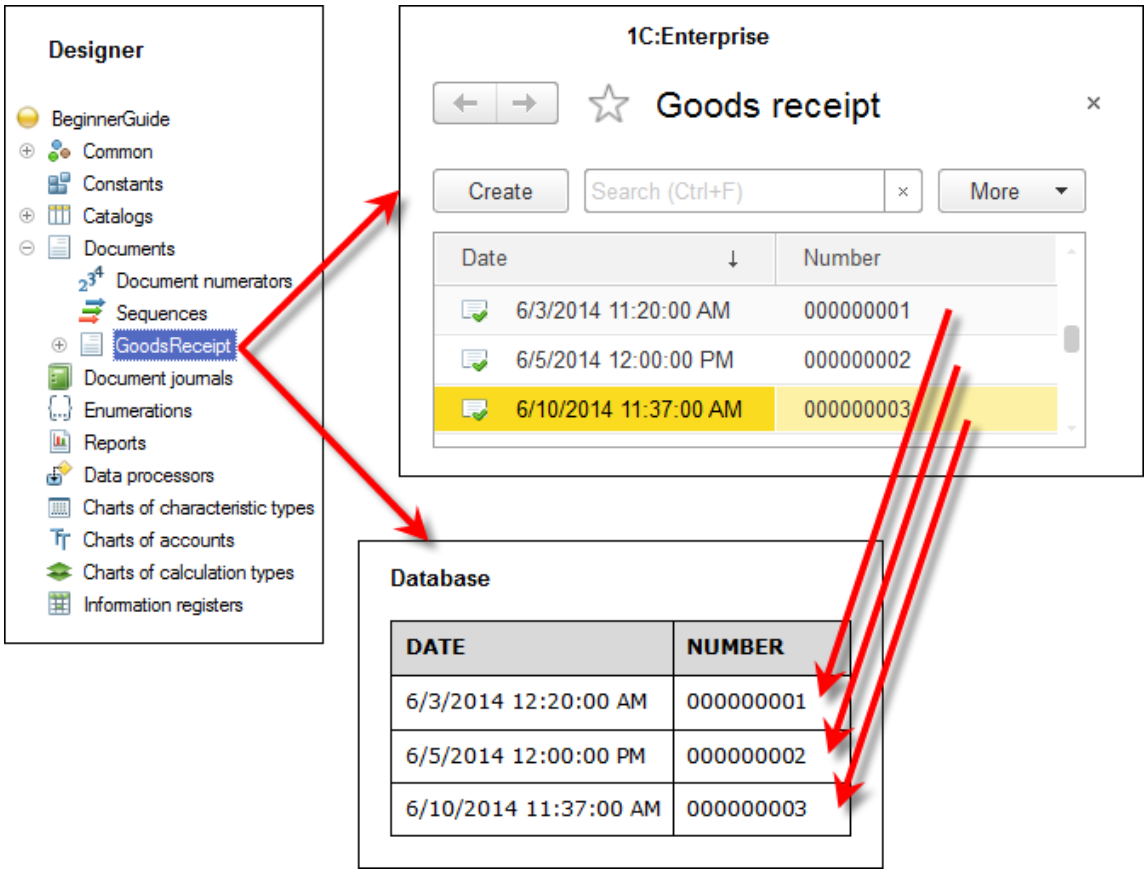


Fig. 4.1. Standard attributes of a Goods receipt document in Designer mode, in 1C:Enterprise mode, and in the database

As a rule, each document contains data that describes the document in detail. For example, each Goods receipt can include vendor info, the warehouse where the goods are delivered, and so on.

This data set is the same for all documents of the same type, it is defined using the attributes of Document configuration objects (subordinate configuration objects). Developers create the majority of attributes manually but each Document configuration object also has a set of standard attributes, which are available by default. Two most important attributes are Date and Number. Since the Date data type contains dates and times with accuracy of one second, this attribute also generally serves as the document time stamp (fig. 4.2).

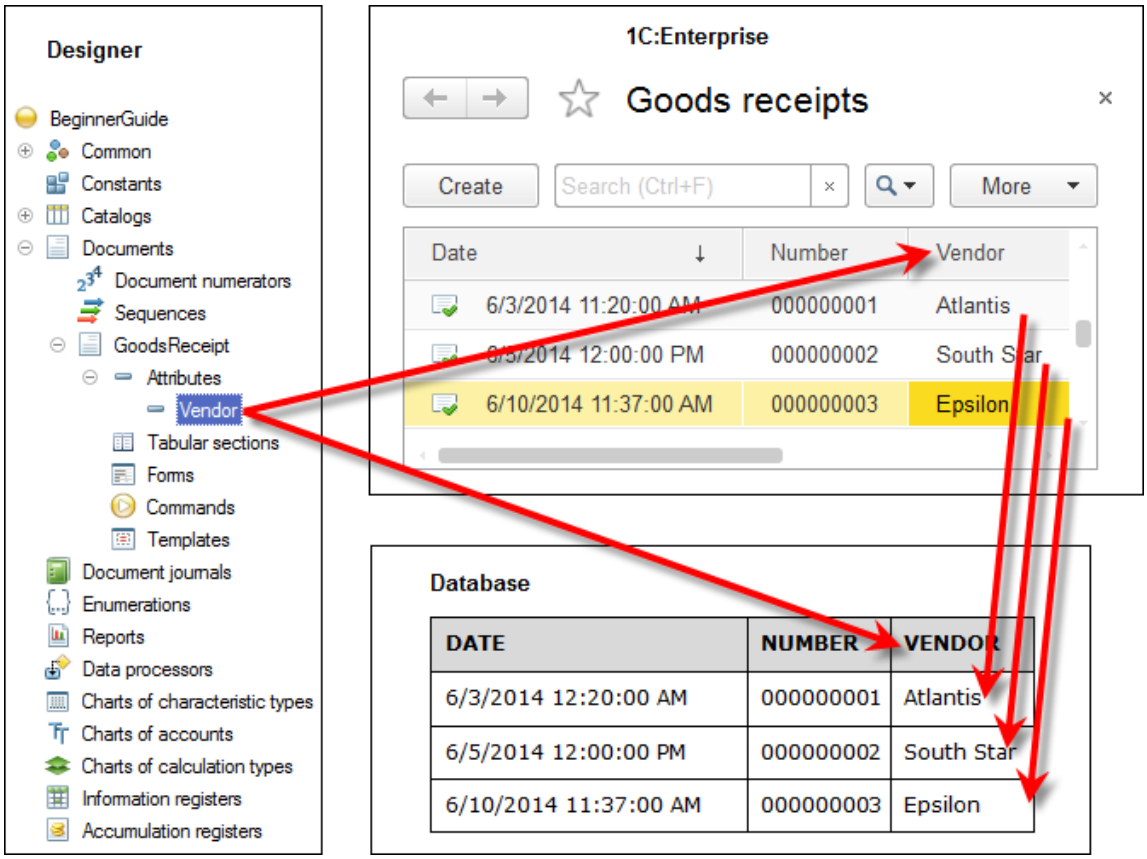
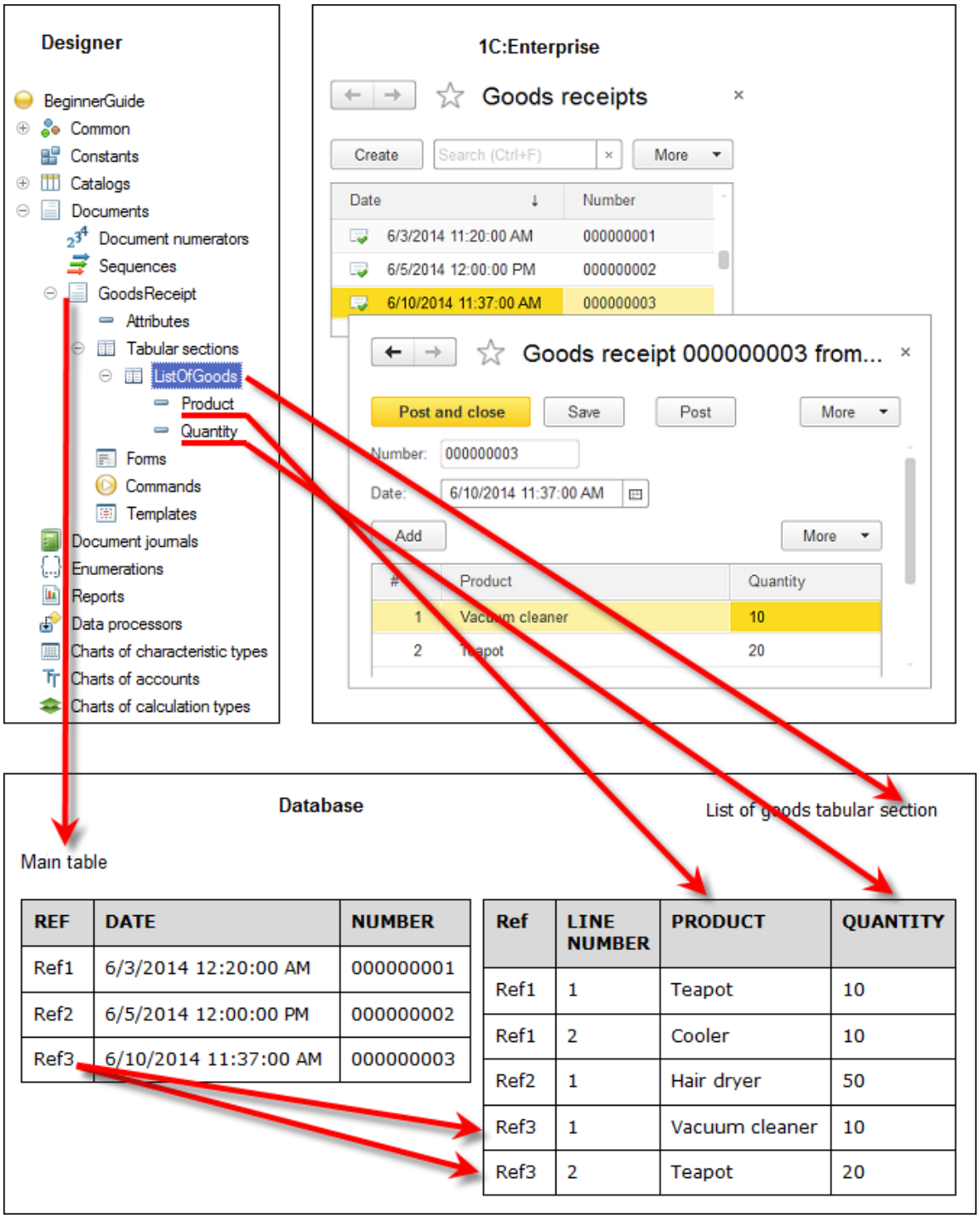


Fig. 4.2. Vendor attribute of a Goods receipt document in Designer mode, in 1C:Enterprise mode, and in the database

Documents can also store certain data sets that have identical structure but vary in quantity (different quantities for different documents). For example, each Goods receipt document can contain a list of received goods.

The Document configuration object uses tabular sections to store such data. In this case the database has additional tables for storing the tabular sections subordinate to a specific document (fig. 4.3).



REF	DATE	NUMBER
Ref1	6/3/2014 12:20:00 AM	000000001
Ref2	6/5/2014 12:00:00 PM	000000002
Ref3	6/10/2014 11:37:00 AM	000000003

Ref	LINE NUMBER	PRODUCT	QUANTITY
Ref1	1	Teapot	10
Ref1	2	Cooler	10
Ref2	1	Hair dryer	50
Ref3	1	Vacuum cleaner	10
Ref3	2	Teapot	20

Fig. 4.3. List of goods tabular section in a Goods receipt document in Designer mode, in 1C:Enterprise mode, and in the database

Document forms

Several default forms are available for displaying document data. As we mentioned earlier, each form might have multiple names (table 4.1).

Table 4.1. Default document forms

IN THE POP-UP MENU AND IN THE PROPERTY PALETTE	IN THE FORM WIZARD	ON THE FORMS TAB
Default Object Form	Document form	Document
Default List Form	Document list form	List
Default Choice Form	Document choice form	Choice

Learn more! For details on the structure of 1C:Enterprise script objects intended for document operations, see section "Quick developer reference. Documents" on page 843.

Theory. Data types. Type-defining configuration objects

Before you start creating documents, let us talk about the data types that are available in the 1C:Enterprise system.

In the previous lesson, when you created catalog or tabular section attributes, you always specified the attribute data type. Those were the primitive data types: Number, String, Date, and Boolean. There are just a few predefined primitive types within the system.

In addition to the predefined types that are available in all configurations, you can define custom data types in your configuration. These are the types that do not exist in a configuration permanently but only appear when specific configuration objects are added.

For example, once you created the Warehouses Catalog configuration object, several new data types related to that catalog became available. These types include, for example, CatalogRef.Warehouses. Now, if you set this data type for some attribute, you can use this attribute for storing a reference to a specific Warehouses catalog object.

Configuration objects that can add new data types are referred to as type-defining objects.

For example, once you created a MaterialsAndServices catalog, the following data types became available:

- CatalogManager.MaterialsAndServices
- CatalogRef.MaterialsAndServices
- CatalogObject.MaterialsAndServices
- CatalogSelection.MaterialsAndServices

Again, note that these data types were not supported by the platform initially, they only exist in a specific applied solution.

This brief discussion was required because right when you create your first document, you will have to deal with CatalogRef.Warehouses and CatalogRef.MaterialsAndServices data types, which are available in your configuration because you created the Warehouses and MaterialsAndServices catalogs.

The Goods receipt document

Now that you are familiar with the Document configuration object, you will create a few such objects for recording the events that take place within Jack of All Trades.

The company provides television repair and washing machine installation services. In both cases, there is a requirement for certain materials that are expended during service provisioning. Therefore, two of the critical events in the life of our company are receipt of materials and rendering of services.

To reflect these events in the database, you will create two documents: Goods receipt and Service.

The Goods receipt document records the delivery of the materials your company needs, while the Service document records the provided services and expended materials.

In Designer mode

Adding a document

Let us add the Goods receipt document.

1. Open Designer and add a Document configuration object.
2. On the **Main** tab, in the **Name** field, type *GoodsReceipt*.

   The platform fills the synonym based on the name.

   Let us also define the document presentations in the 1C:Enterprise interface. You do not have to fill Object presentation because the synonym will be used instead. This is totally suitable because the document name is specified in the singular.

   On the contrary, you have to specify List presentation in the plural.
3. In the **List presentation** field, type *Goods receipts* (fig. 4.4).

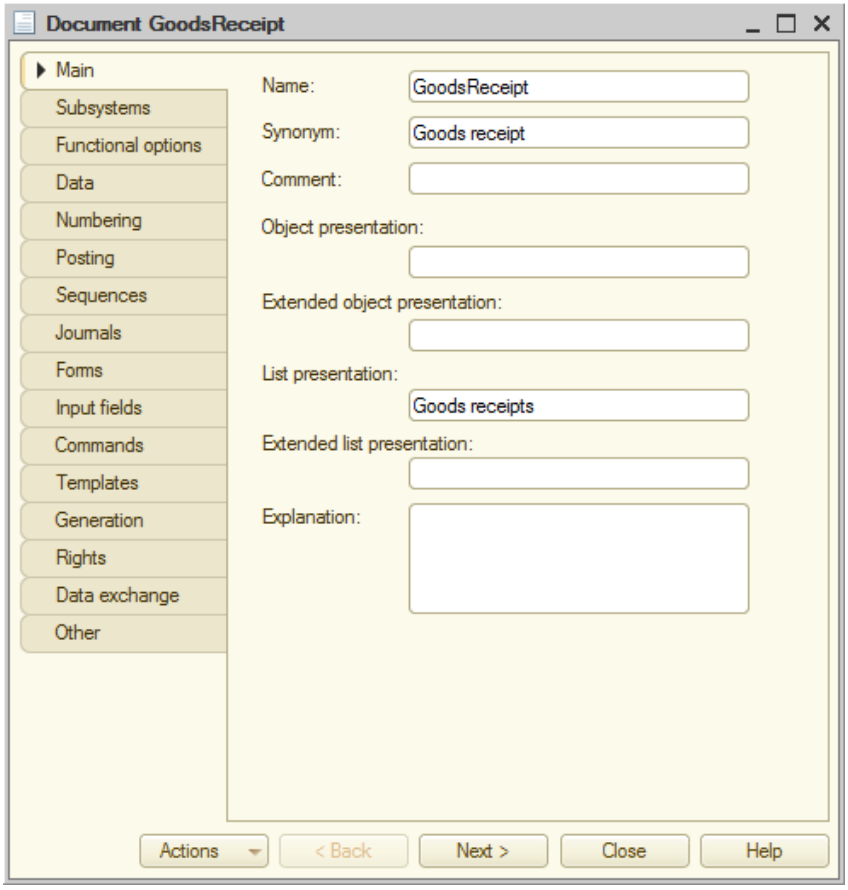


Fig. 4.4. Specifying main document properties

4. Click **Next**.

   This opens the **Subsystems** tab.

   The configuration logic suggests that the list of goods receipts should be available in the Inventory and Accounting sections. Indeed, this document is directly related to the first section and the list of documents that record delivery of materials can be frequently needed for accounting analysis.

5. Select the **Accounting** and **Inventory** check boxes (fig. 4.5).

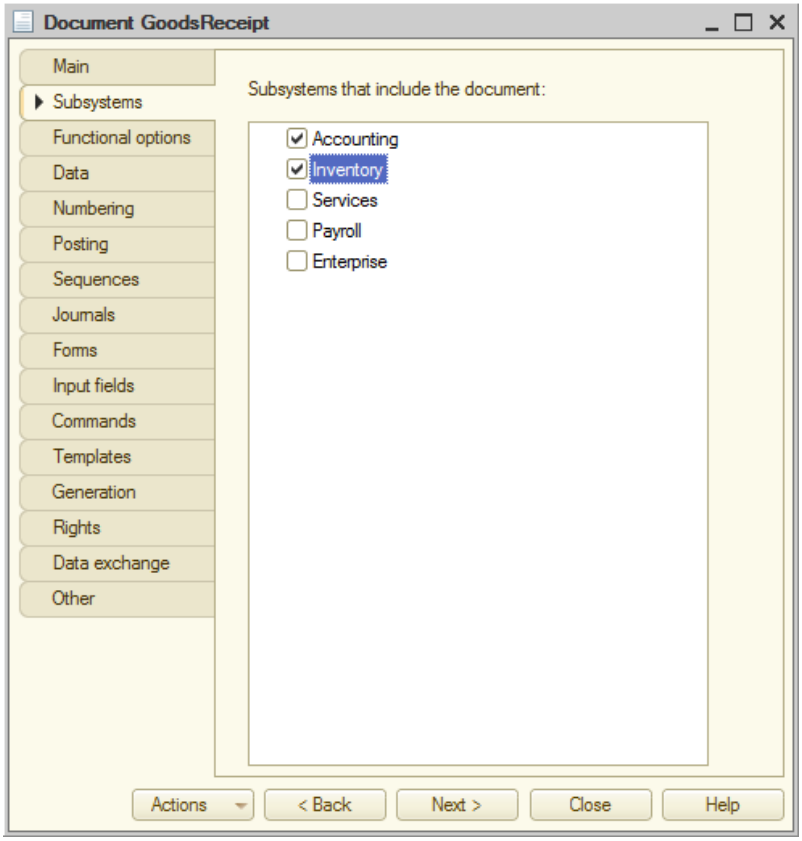


Fig. 4.5. Specifying subsystems where the document is available

6. Click the **Data** tab, click **Add** above the list of document attributes, and create the **Warehouse** attribute (fig. 4.6).

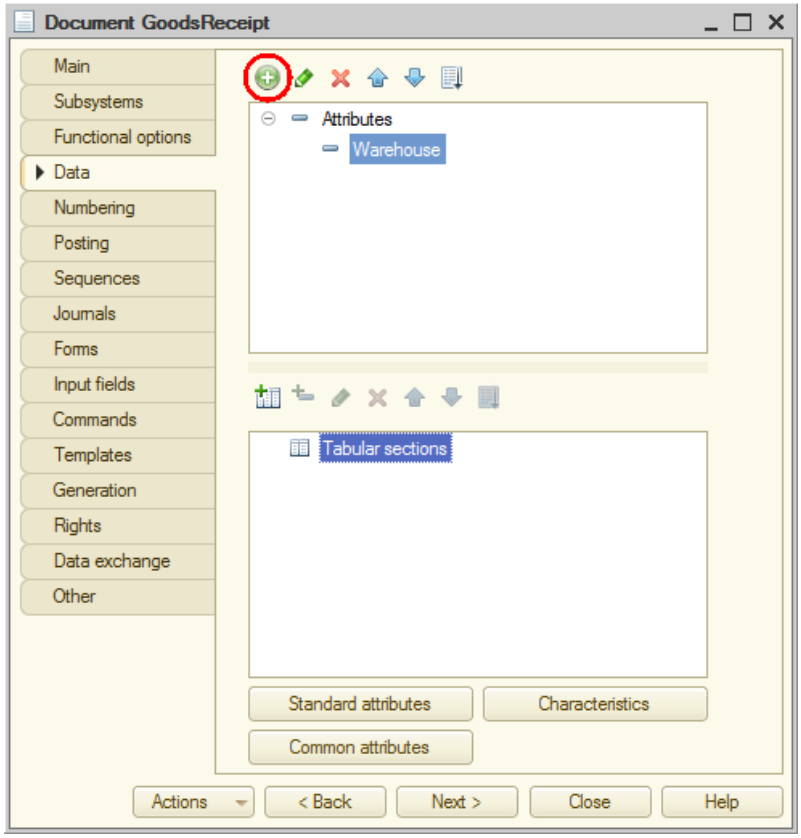


Fig. 4.6. Creating a document attribute

Attributes of the reference type

Let us select a reference data type for the Warehouse attribute.

- Double-click the **Warehouse** attribute and then, in the property palette, in the **Type** list, select **CatalogRef.Warehouses**.

  This data type is available in the configuration because you created the Warehouses catalog configuration object (fig. 4.7).

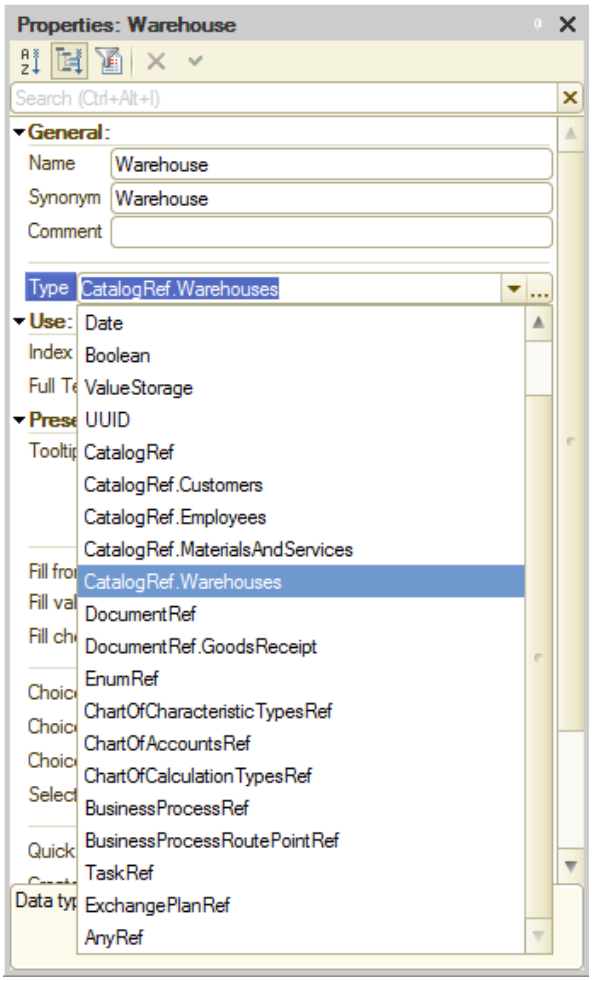


Fig. 4.7. Document attribute properties

"Fill value" property of a configuration object attribute

Now let us streamline the document creation. In the company that you are automating all the goods are received at the main warehouse.

1. In the property palette of the **Warehouse** attribute, in the **Fill value** field, select the **Main** predefined item.

   When a user starts creating a document, its Warehouse field is filled with the Main value, so they do not have to enter it manually (fig. 4.8).

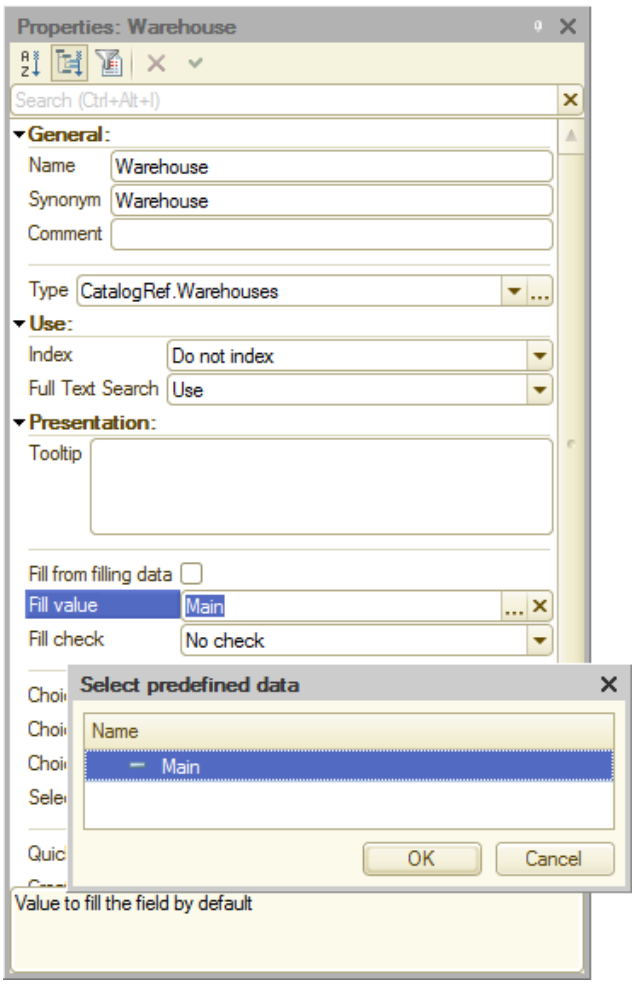


Fig. 4.8. Selecting the default fill value for the Warehouse attribute

Now let us add the Materials tabular section.

2. Click **Add tabular section** above the list of tabular sections (fig. 4.9).

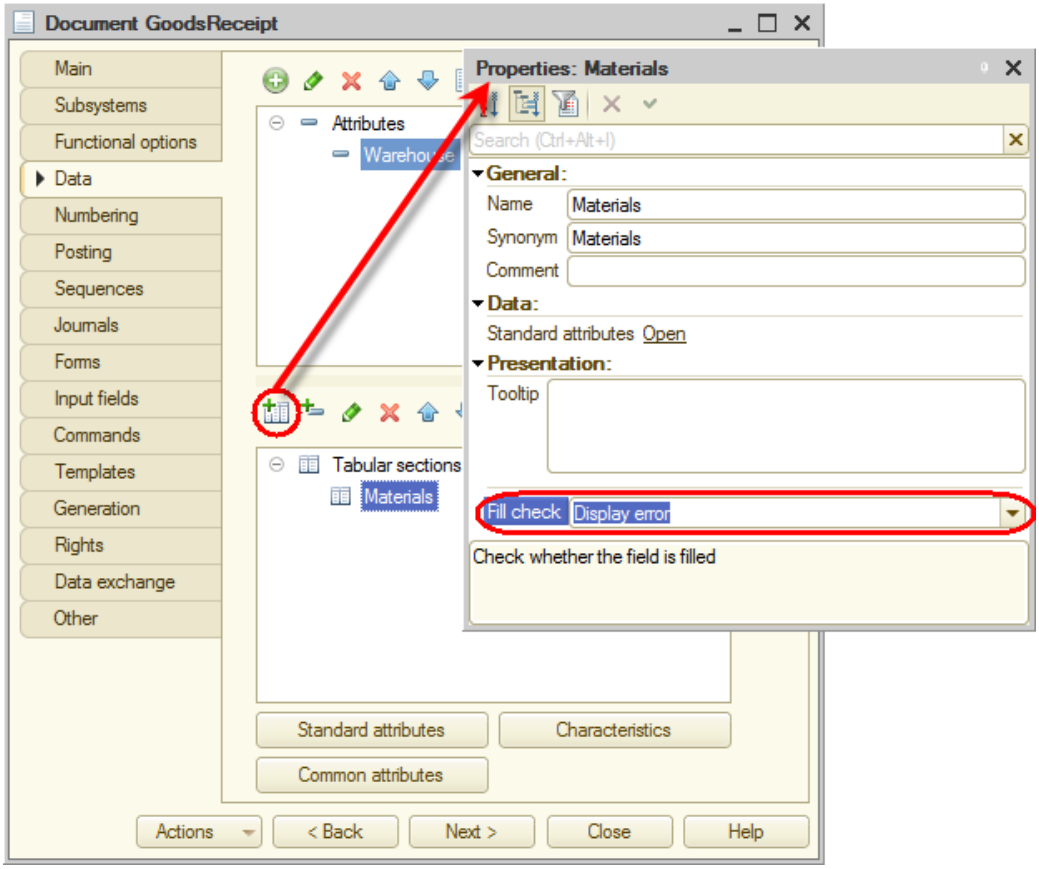


Fig. 4.9. Creating a tabular section for a document

Tabular section fill check

Let us prohibit saving goods receipts with empty tabular sections.

1. In the **Fill check** field, select **Display error**.

   In doing so you specify that a GoodsReceipt document must include a tabular section (the list of received materials). Otherwise an error message is displayed when a user attempts to save the document.

   Let us create the attributes for the Materials tabular section.

2. On the **Data** tab, click **Add attribute** above the list of tabular sections and add the following attributes (fig. 4.10):

   - **Material**. Type: CatalogRef.MaterialsAndServices
   - **Quantity**. Type: Number, length: 15, precision: 3, non-negative
   - **Price**. Type: Number, length: 15, precision: 2, non-negative
   - **Total**. Type: Number, length: 15, precision: 2, non-negative

3. For each tabular section attribute, in the **Fill check** field, select **Display error**.

   In doing so you specify that when a document is saved, in addition to the availability of tabular section records, the platform checks that all of the attributes of each record are filled.

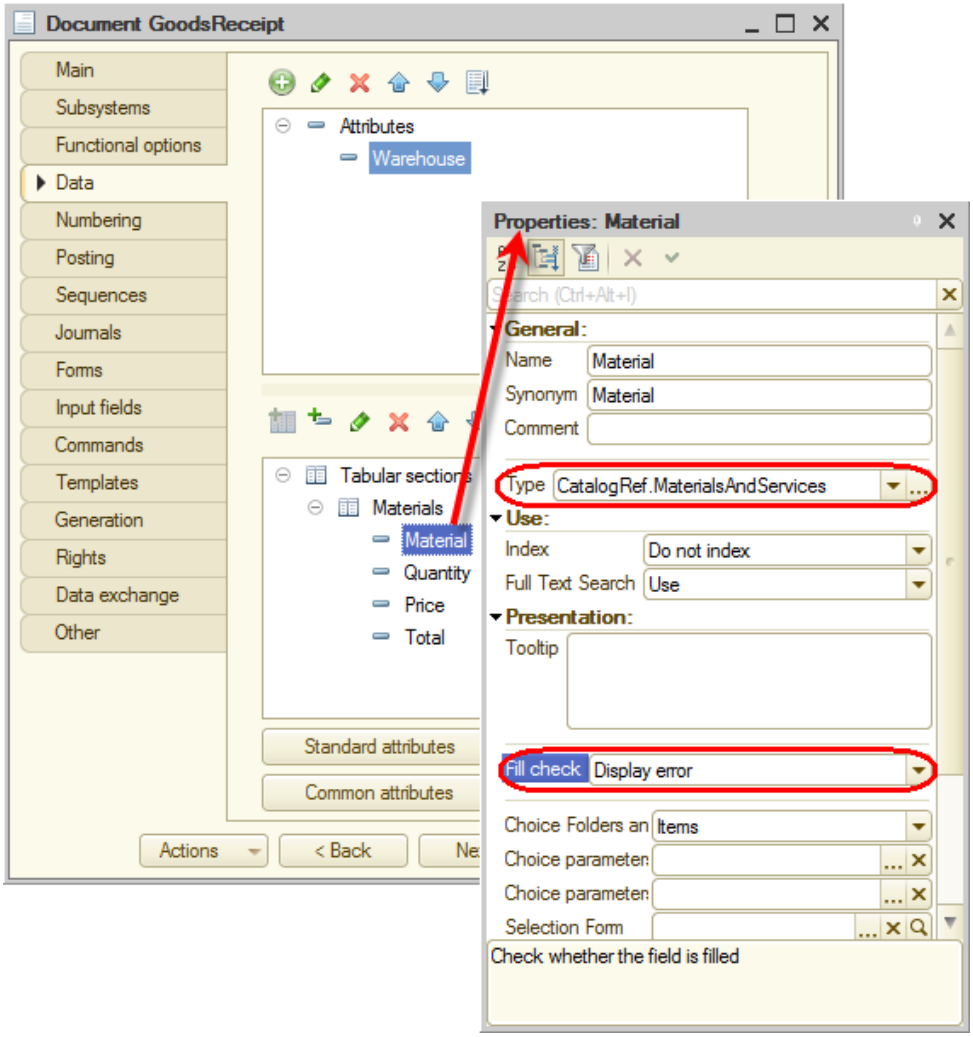


Fig. 4.10. Creating document tabular section attributes

4. Click the **Numbering** tab and ensure that the **Autonumbering** check box is selected.

   This property defines automatic generation of unique numbers for documents that users create.

   Finally, let us add the document creation command to the Inventory subsystem.

5. In the configuration object tree, right-click the **Subsystems** branch and then click **All subsystems**.

6. In the **All Subsystems** window, click the **Inventory** subsystem.

   The **Command interface** list on the right displays all of the commands of the selected subsystem.

7. In the **Action panel.Create** group, enable visibility for the **Goods receipt: create** command (fig. 4.11).

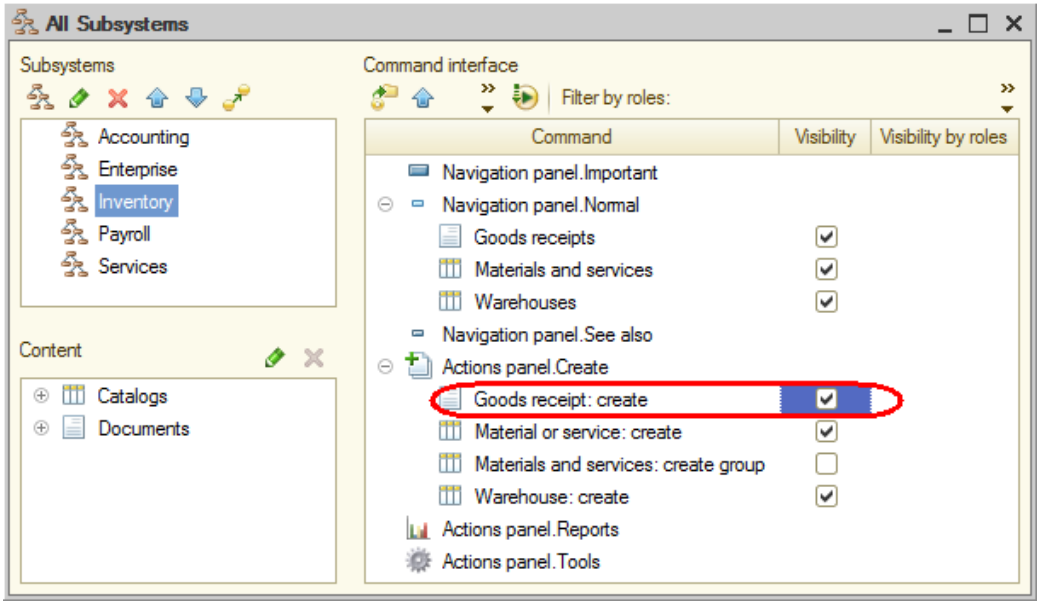


Fig. 4.11. Specifying subsystem command interface settings

In 1C:Enterprise mode

Let us test the resulting document.

- Start 1C:Enterprise in the debug mode.

  You can see that the Goods receipts command is now available in the Accounting and Inventory sections (fig. 4.12).

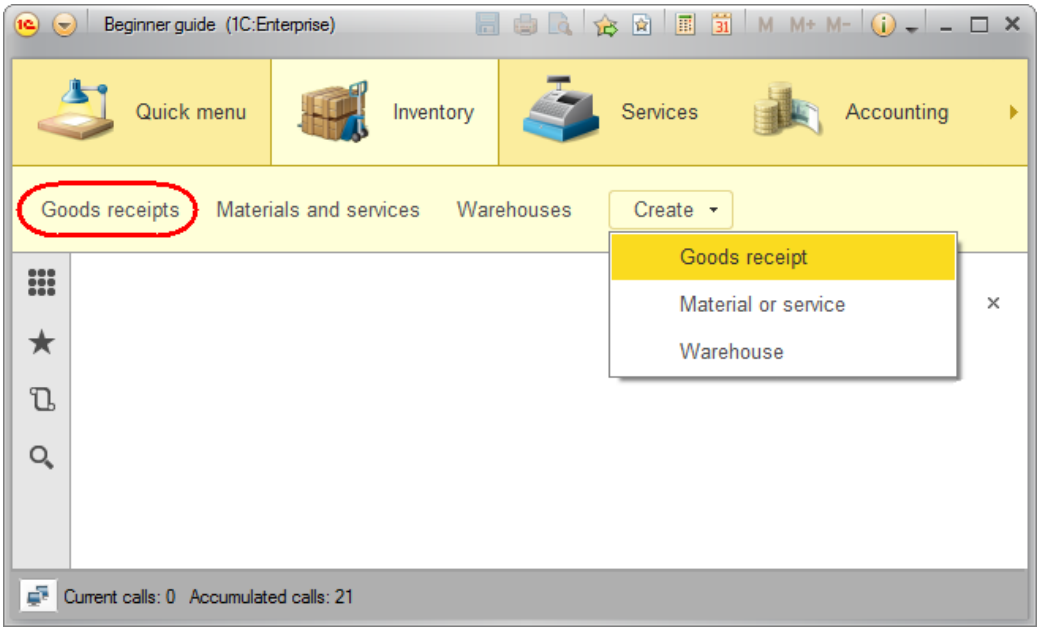


Fig. 4.12. Inventory section

The command name is determined by the List presentation property that you defined for this document.

Also the Goods receipt command is added to the Create menu of the Inventory section. The command name is determined by the synonym because you have not defined the Object presentation for this document (see fig. 4.12).

Adding Goods receipt documents

For now there are no Goods receipt documents in the database. Let us create one.

1. In the **Inventory** section, on the **Create** menu, click **Goods receipt**.

   This opens the default object form. The Goods receipt form title matches the document synonym.

   The date field is automatically filled with the current date and zero time because the document has not yet been posted. Once you post the document, it is changed to the actual time stamp.

   The Number field is empty but the platform will generate a unique number for the document because the Autonumbering property is enabled for the document. The new number is generated when you save the document.

   Note that the Warehouse field is already filled with the Main value exactly as you selected in the properties of this attribute.

2. Fill the tabular section of the goods receipt with the television repair materials as shown on the figure (fig. 4.13).

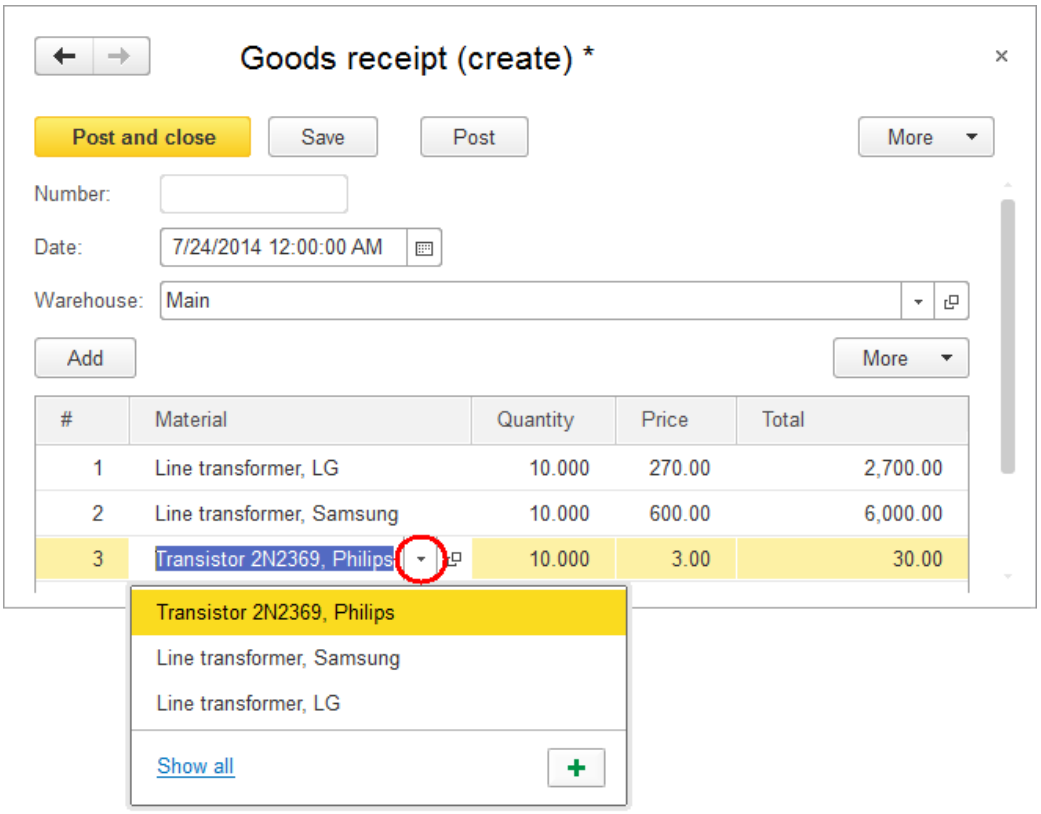


Fig. 4.13. Creating a Goods receipt document

   Note that clicking the Material column, which is not yet filled, opens the list of items from the MaterialsAndServices catalog. This is because this attribute has reference data type and references the MaterialsAndServices catalog (see fig. 4.13).

3. Click **Post and close**.

This saves and posts the document. An automatically generated number and the posting time are assigned to the document.

4. Follow the same steps to create a second document with washing machine installation materials received to the Main warehouse (fig. 4.14), then click **Post and close**.

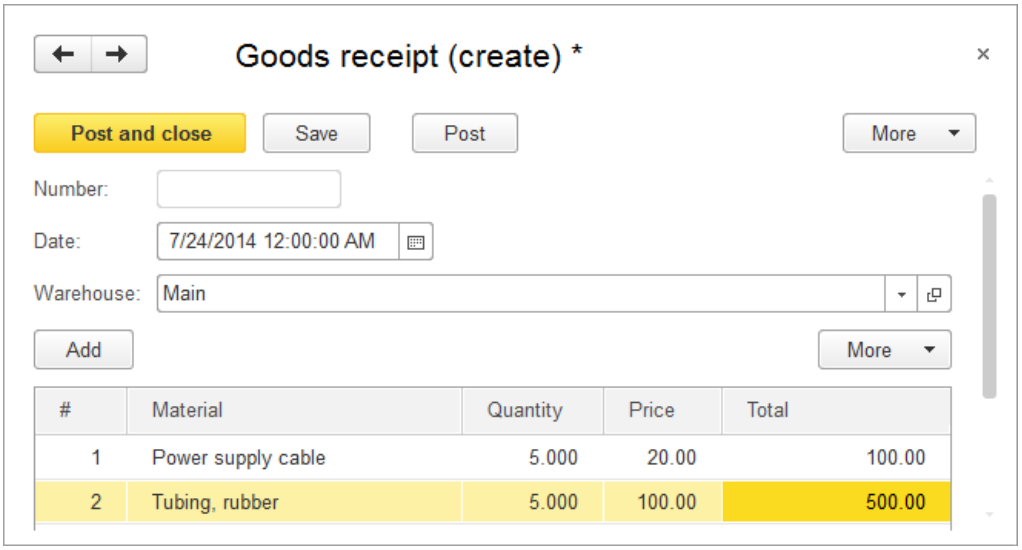


Fig. 4.14. Creating the second Goods receipt document

This saves and posts the document. An automatically generated number and the posting time are assigned to the document.

Note that when you fill the document, the entire tabular section and each column are marked by red dotted line. It means that the fill check is applied to them. If you do not add any rows to the tabular section or leave any tabular section column empty and attempt to save the document, an error message is displayed (fig. 4.15).

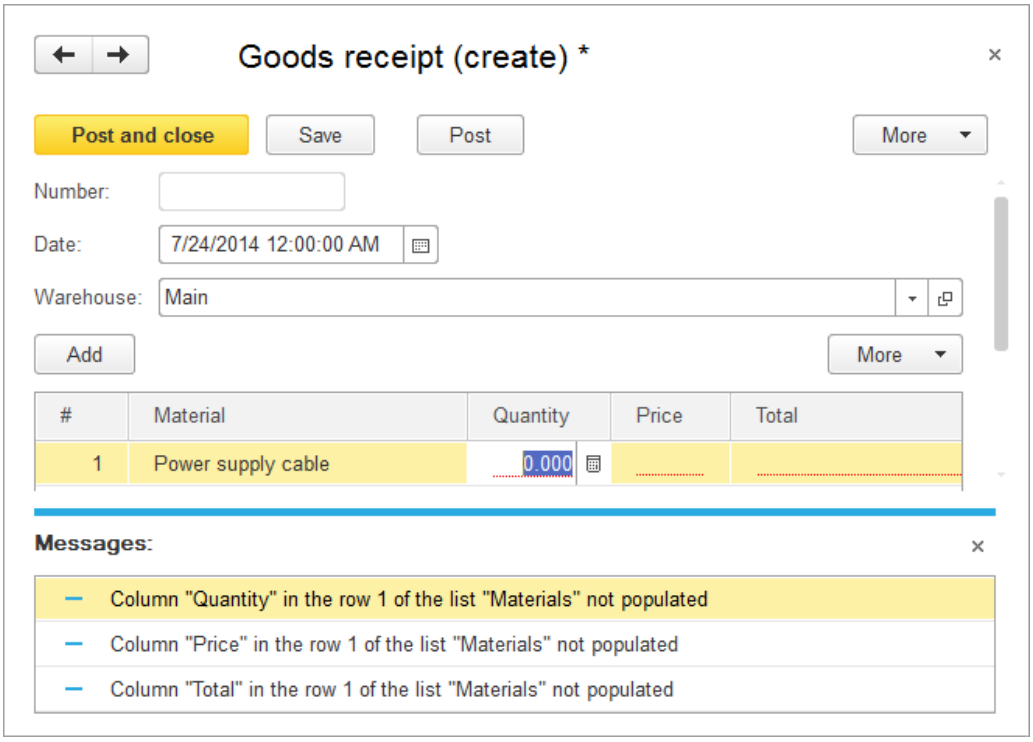


Fig. 4.15. Error message when saving a document

Let us view the list of created documents.

5. Run the **Goods receipts** command.

   It opens the list form where you can see two documents marked with icons indicating that these documents have been posted (a green checkmark over the document icon, fig. 4.16).

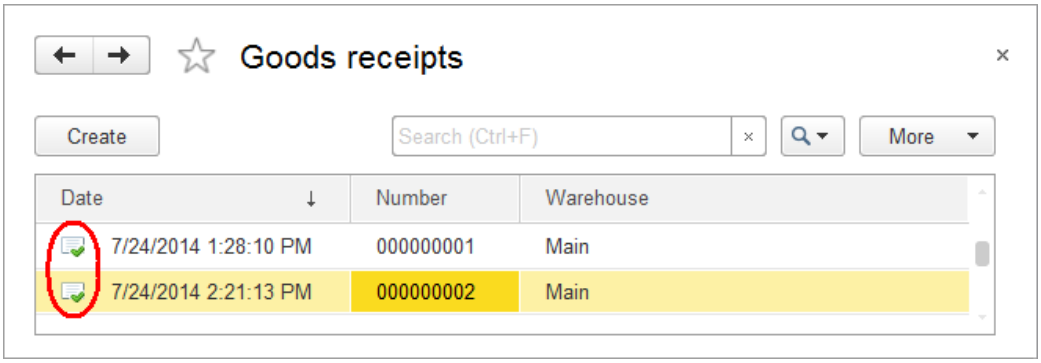


Fig. 4.16. List of goods receipts

Theory. Catalogs and documents

It is interesting to note the differences in the use of singular and plural terms when referring to Catalog and Document objects.

If you open the tree of any standard configuration, you can see that the objects in the Catalogs branch are generally named in the plural, while the objects in the Documents branch are generally named in the singular.

It may seem that when you create a Catalog configuration object you create a storage to keep the items of this catalog, whereas when you create a Document configuration object you create a template for a single specific document. This is not actually the case.

When you create a Catalog object in Designer, you name it in the plural (MaterialsAndServices). And you imply that in the database this object will store items describing specific materials and services (each item in the singular).

When you create a Document object, you name it in the singular (GoodsReceipt), even though you are creating the same sort of a storage as in the case of a catalog. Each record in that storage will describe a single document: a goods receipt (in the singular). Therefore, in Designer it would be conceptually correct to name Document objects in the plural, which would emphasize the fact that they describe a set of documents of that type (for example, GoodsReceipts).

However, human thinking is such that, when opening the Documents branch, people expect to see them listed in the singular rather than in the plural. So we end up with a situation where it is difficult to come up with an appropriate term to describe a whole set of documents of specific type (a set of records of a single type is far easier: a catalog, a chart, and so on). Therefore, the corresponding configuration object branch is named Documents, while the configuration objects created in this branch are named in the singular despite the fact that the branch actually contains

descriptions of storages for various document types and each item in the branch describes the set of all the documents of a certain type.

Automatic recalculation of document totals

You have probably noticed that, when filling the document, you needed to enter the total in each row. That is a nuisance and the natural impulse is to automate the document, so that the total is automatically calculated every time there is a change to the price or quantity of materials in a row.

It is actually easily accomplished. To do so, you will first create a custom document form, and then use the 1C:Enterprise script.

So far you always used the predefined object forms generated by 1C:Enterprise. Now you have come to the point when you need a slight modification of the document form logic, and that means you need to create a custom form for the GoodsReceipt document where you will define the required algorithm using 1C:Enterprise script. The platform will use that custom form instead of the default form.

In Designer mode

Document form

Let us create a custom document form.

1. In Designer, open the editor of the **GoodsReceipt** document configuration object.
2. Click the **Forms** tab.

   As you can see, no default forms are available yet.
3. In the **Document** field, click the **Open** button.

   -OR-

   Click the **Add** button above the list of forms (fig. 4.17).

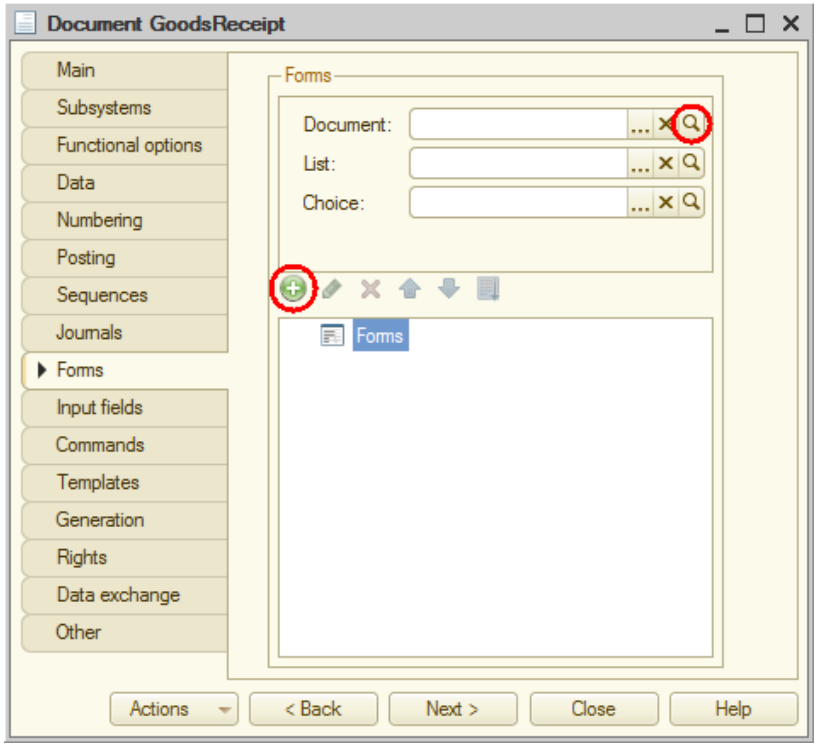


Fig. 4.17. Creating a document form

The platform opens yet another helpful developer tool: the form wizard (fig. 4.18).

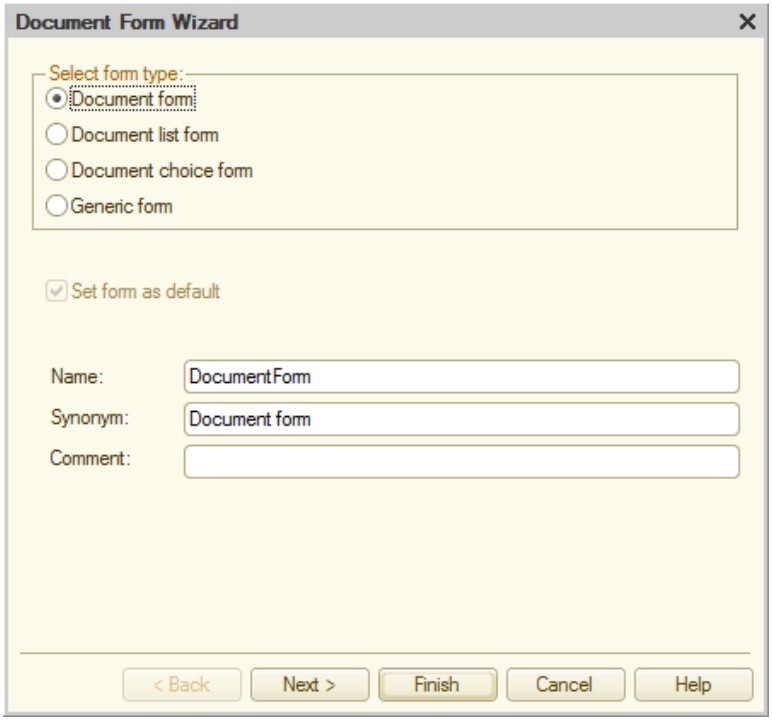


Fig. 4.18. The form wizard

It is a typical wizard where you enter data in a specific order and use the **Next** and **Back** buttons for navigation.

4. Click **Document form**, keep the default values in all other fields, and click **Finish** (see fig. 4.18).

Note that the DocumentForm form subordinate object is added to the GoodsReceipt document configuration object (fig. 4.19), and that the form editor displaying this form is now open (fig. 4.20).

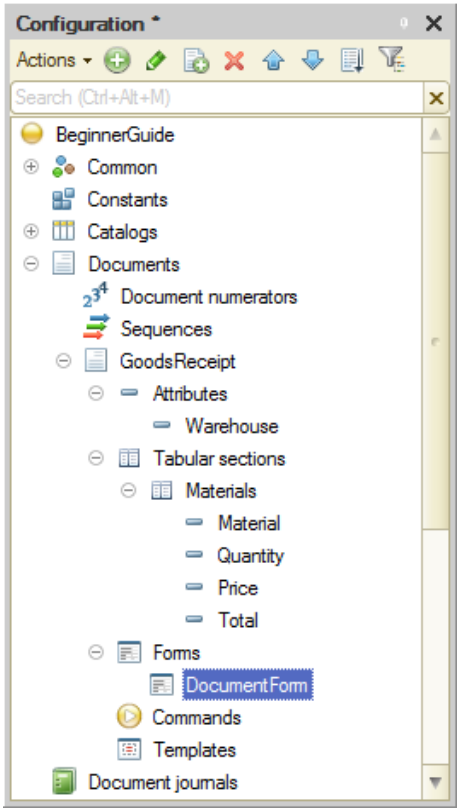


Fig. 4.19. New document form in Designer

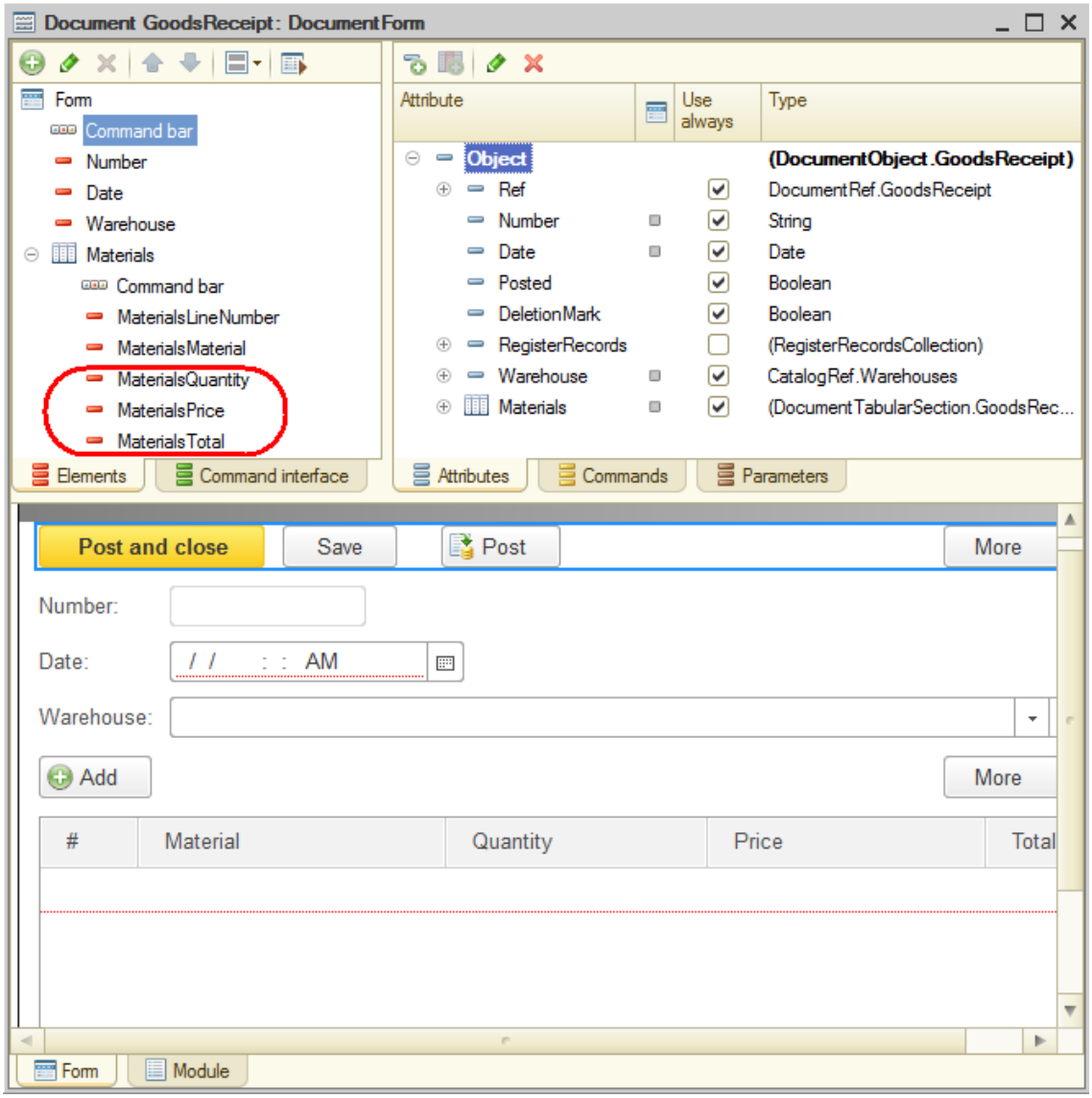


Fig. 4.20. Form editor

The form editor combines multiple windows of interconnected editors. We will not discuss all of the actions available in the form editor and will only cover the issues that are required to complete the basic operations related to our task.

For detailed description of the form editor, see the "Form Editor" section in the "Development Tools" chapter of 1C:Enterprise. Developer Guide.

For now we will only discuss the document form preview pane (located at the bottom) and the form controls editor pane (in the top left corner).

When you develop configuration object forms, you cannot simply draw a form. You only can specify the form controls, which are automatically arranged within the form.

Form controls in the upper left pane are arranged in a tree. The controls are arranged top-to-bottom and left-to-right within the form, with higher-priority items going first.

You can edit the tree on the **Elements** tab. This is how you can edit the form layout.

You see that based on the configuration description of the GoodsReceipt document the platform created a structure of controls that defines the form layout.

These controls serve various purposes and have different behaviors. However, all of them display data stored in the database and provide interaction with that data.

You can drag the tree items to change the positions of form controls (such as tabular section attributes). The result is immediately displayed in the document form preview at the bottom of the form editor. Note that you do not have to think of specific locations (in pixels) for the form controls, their sizes and dockings to other controls. The platform takes care of this.

You can use the property palette to modify control properties, which can affect their view in the form. You can also modify the structure of form controls: create a field, a group of fields, or a tabular section, and link these controls to form data.

But for now you do not have to do any of this. You only need to focus on the tabular section items: MaterialsQuantity, MaterialsPrice, and MaterialsTotal (see fig. 4.20).

Every time a user changes a value in the Quantity or Price field, the Total field value should automatically change to Quantity*Price.

Obviously, to do so you have to use 1C:Enterprise script to write a command that executes something like Total = Quantity*Price when a value in the Quantity or Price field is changed. But how are we going to "capture" that point of change?

Event handler

As you have already seen, the platform can handle the objects described in the configuration tree: it displays their data, provides the options for adding items, and so on. So the platform has some standard algorithms for this.

But developers are only satisfied with these standard algorithms in the very basic scenarios. Actual tasks are much more diverse. This is why the platform features events that are connected to various moments of standard operations. These include the events related to user interaction with forms and form controls.

Using 1C:Enterprise script, developers can interpose in the events and define custom algorithms that are executed when those events occur. This is exactly what you will do now.

1. Double-click the **MaterialsQuantity** form control.

   -OR

   Right-click the **MaterialsQuantity** form control and then click **Properties**.

2. Scroll down the property palette window to see the list of events that can be linked to this field.

   It is obvious that you need the OnChange event that occurs when the field value is changed.

3. In the **OnChange** field, click the **Open** button (fig. 4.21).

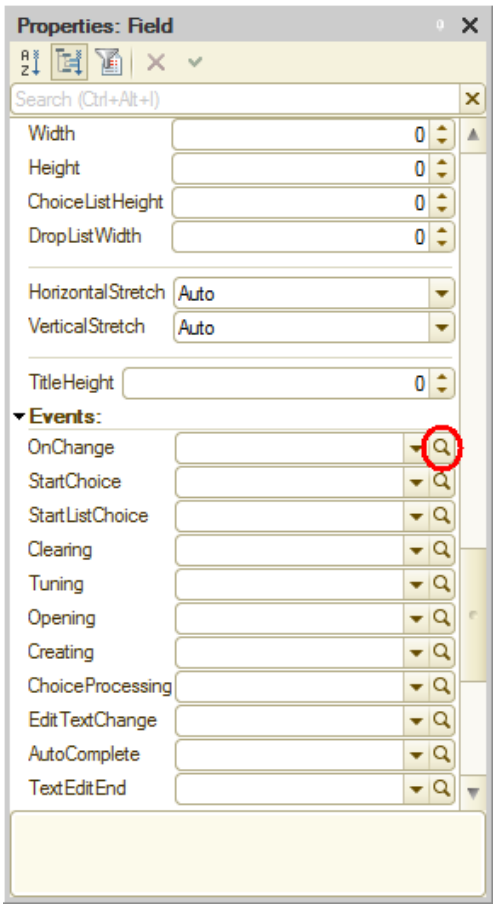


Fig. 4.21. Creating the OnChange event handler for the Quantity tabular section field

The platform prompts you to select the event handler type (fig. 4.22).

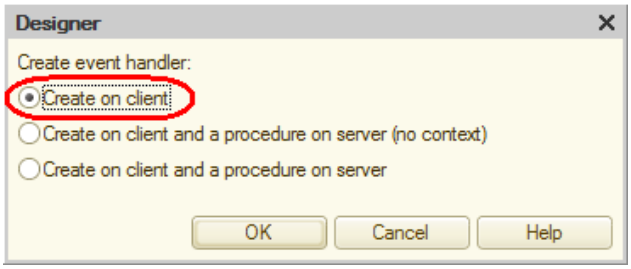


Fig. 4.22. Selecting event handler type

4. Keep the default **Create on client** option.

   We will not explain these options for now, as they are described in section "Compilation directives" on page 189.

   Then the platform generates a handler procedure template for this event in the form module and opens the **Module** tab of the form editor.

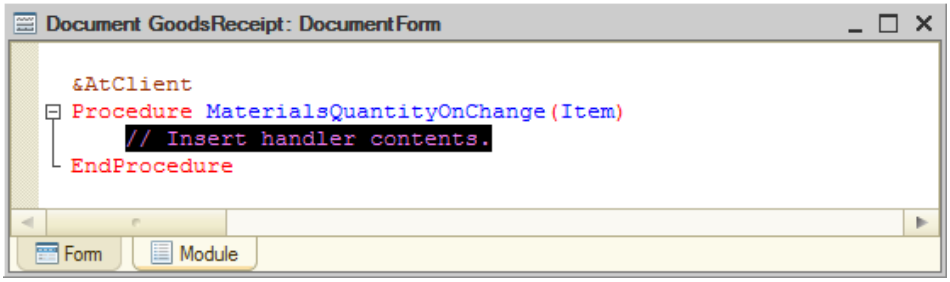


Fig. 4.23. OnChange handler template for the Quantity tabular section field

Modules store 1C:Enterprise script algorithms. There are several module types intended for storing algorithms related to various aspects of applied solution functioning. In this case it is a form module because handlers of all interactive events linked to form controls are stored in the form module.

5. Add the following script to the MaterialsQuantityOnChange() procedure body (listing 4.1).

Listing 4.1. Procedure MaterialsQuantityOnChange()

```
TabularSectionRow = Items.Materials.CurrentData;

TabularSectionRow.Total = TabularSectionRow.Quantity
    * TabularSectionRow.Price;
```

Let us examine this script.

The first line creates the TabularSectionRow variable that will store the object containing data of the tabular section row that requires recalculation.

1C:Enterprise script features weak data typing, which allows using variables without declaring them first (without specifying their types beforehand). In this line the variable is created on the fly and its type is defined by the type of the value assigned to it.

Since we are now in the form module, all the properties and methods of the ManagedForm 1C:Enterprise script object are available within this module. Hence you can call them directly. In this line after the equality sign a collection of form controls is called using one of the ManagedForm object attributes: Items.

The collection of form controls is a FormAllItems 1C:Enterprise script object that contains all of the form controls. This is basically a script analog of the root of the form control tree.

You can get a form control by specifying its name as the property of this object, separated by "." (dot). In this example the tabular section of the Materials document is called (Items.Materials).

A tabular section of a document is a FormTable 1C:Enterprise script object. You can get the row that is being edited using the CurrentData property (Items.Materials.CurrentData) of the FormTable script object.

So once the first line of the handler procedure is executed, the TabularSectionRow variable stores a FormDataStructure object. This object contains data stored in the current row of the document tabular section (Items.Materials.CurrentData).

Once you get this object, you can call the data of a specific column of the tabular section using the column name as the object property. For example,

using TabularSectionRow.Quantity allows you to get the number located in the Quantity cell of the row being edited.

Hence, the second line of the handler procedure calculates the value of the Total column by multiplying the values in the Quantity and Price columns.

In 1C:Enterprise mode

Now let us see how it works.

- Start 1C:Enterprise in the debug mode, open the list of Goods receipt documents, and open any of the two documents created earlier.

If you change the quantity in any document row, the row total is recalculated automatically.

A single procedure as a hander for multiple events

So you checked that once you change the quantity in any row of a Goods receipt document, the platform automatically recalculates the total.

You also need the Price field to behave in a similar manner. Taking a look ahead though, you will see that similar automatic calculation of Total fields might come in handy for other documents as well.

Therefore, it is better to move the calculation of totals to some universally accessible location, so that various documents with similar attributes in their tabular sections will be able to use the algorithm.

The Common module configuration objects serve for defining these universally accessible locations. You can add them in the Common / Common modules branch of the configuration object tree. The procedures and functions stored in these modules are accessible from all the configuration objects.

So you will create a common module and move the total calculation procedure to this module. And in the document you will simply call that procedure from the common module.

In Designer mode

Common module

Let us add a Common module configuration object.

1. In the configuration object tree, expand the **Common** branch by clicking the ⊕ icon to the left of the branch name.
2. Right-click the **Common modules** branch and click the **Add** button in the command bar of the configuration window (fig. 4.24).

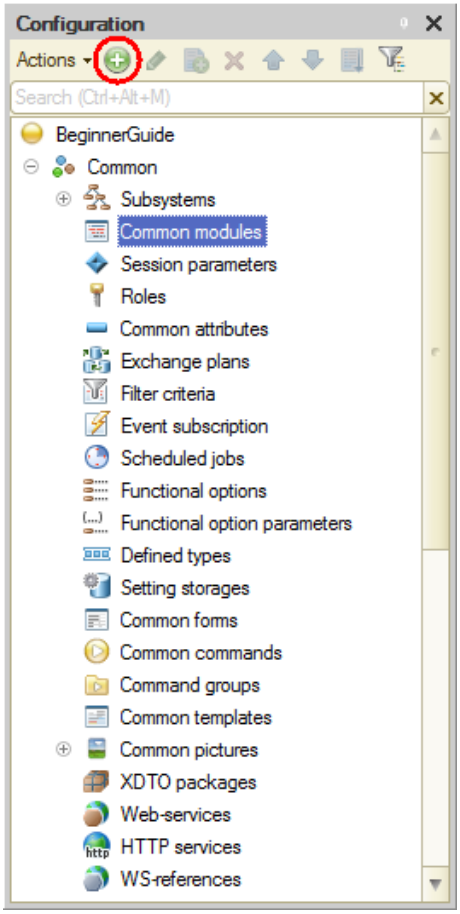


Fig. 4.24. Creating a common module in the configuration object tree

This opens the window where you can enter the module text and the module property palette.

3. In the property palette, name the module *DocumentProcessing*, select the **Client (managed application)** check box, and clear the **Server** check box.

   It means that instances of this module will be compiled in the thin client context and in the web client context (fig. 4.25).

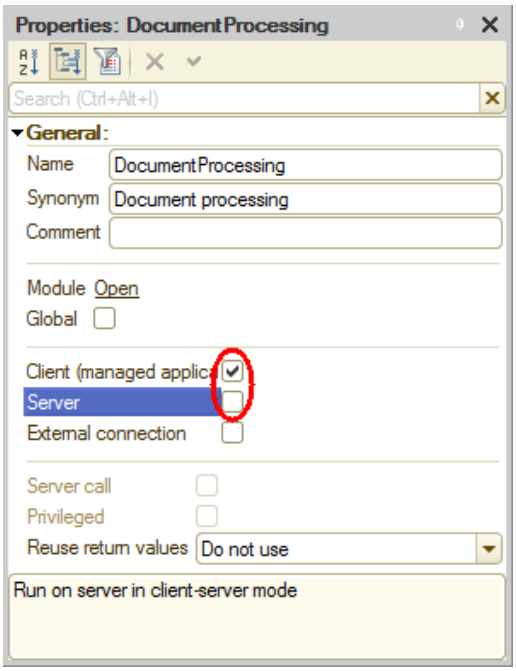


Fig. 4.25. Common module properties

4. Add the following script to the module (listing 4.2).

Listing 4.2. Procedure CalculateTotal()

```
Procedure CalculateTotal (TabularSectionRow) Export

    TabularSectionRow.Total = TabularSectionRow.Quantity
        * TabularSectionRow.Price;

EndProcedure
```

Now let us examine this script. The TabularSectionRow variable that you defined in the OnChange event handler of the Quantity field is passed to the CalculateTotal() procedure. This variable stores data of the tabular section row of the Goods receipt document that is being edited.

You can use this variable to access tabular section column data and calculate the total by multiplying the price and quantity.

The Export keyword in the procedure title indicates that the procedure can be accessed from other applied solution modules.

5. Change the handler script in the form module (listing 4.3).

Listing 4.3. Procedure MaterialsQuantityOnChange()

```
&AtClient
Procedure MaterialsQuantityOnChange(Item)
    TabularSectionRow = Items.Materials.CurrentData;
    DocumentProcessing.CalculateTotal(TabularSectionRow);
EndProcedure
```

You can see that the first line of the procedure remains unchanged. And in the second line instead of directly calculating the total the CalculateTotal() procedure from the DocumentProcessing common

module is called. The current tabular section row is passed to the procedure as a parameter.

6. Start 1C:Enterprise in the debug mode and ensure that nothing has changed from the user perspective.

   Finally, let us implement a similar handler for the Price field. Since you have already coded the required procedure in the form module, you might want to reuse it with a different form control. However, the 1C Company configuration development standards do not permit that approach.

   Learn more! According to the 1C Company development standards, each event must have its own handler. If identical actions are to be performed in response to changes to different controls (for example, in response to clicking any of the several buttons), these should be handled as follows:

   - A dedicated procedure or function that performs the required actions is created.
   - For each control, a separate handler with a default name is created.
   - The procedure or function is called from each handler.

   This is why you will create the OnChange event handler for the MaterialsPrice field of the tabular section in the same manner as you have done it for the MaterialsQuantity field and repeat the call of the CalculateTotal() procedure from the common module.

7. Add the **OnChange** event with the following script to the **Price** field (listing 4.4).

   Listing 4.4. Procedure MaterialsPriceOnChange()

```
&AtClient
Procedure MaterialsPriceOnChange(Item)
    TabularSectionRow = Items.Materials.CurrentData;
    DocumentProcessing.CalculateTotal(TabularSectionRow);
EndProcedure
```

In 1C:Enterpise mode

Start 1C:Enterprise in the debug mode and ensure that the tabular section row totals in Goods receipt documents are recalculated in response to changes in both quantity and price.

The Services document

Now let us create the second document: Services. You will follow the same procedure that you used to create the Goods receipt document (page 119).

In Designer mode

Let us create the Services document.

1. Create a Document configuration object and name it Services.

   On the **Main** tab let us define the document names used in 1C:Enterprise interface. Do not specify Object presentation, the platform will use the object synonym instead.

2. In the **List presentation** field, type Services.
3. On the **Subsystems** tab, specify that the document is available in the **Services** and **Accounting** subsystems.
4. On the **Data** tab, add the following document attributes:
   - **Warehouse**. Type: CatalogRef.Warehouses, fill value: Main (the predefined item of the Warehouses catalog)
   - **Customer**. Type: CatalogRef.Customers, fill check: display error
   - **Technician**. Type: CatalogRef.Employees, fill check: display error
5. Create a tabular section named *MaterialsAndServices* with the following attributes:
   - **MaterialOrService**. Type: CatalogRef.MaterialsAndServices
   - **Quantity**. Type: Number, length: 15, precision: 3, non-negative
   - **Price**. Type: Number, length: 15, precision: 2, non-negative
   - **Total**. Type: Number, length: 15, precision: 2, non-negative
6. For the entire tabular section and for all its attributes, set the **Fill check** property to **Display error**.
7. On the **Forms** tab, create the default document form.
8. For the **MaterialsAndServicesQuantity** field, create the **OnChange** event handler that is executed on the client.

   Later you will add a call of the CalculateTotal procedure from the DocumentProcessing common module to the handler.

   Once you create a handler, the form module with a template for the MaterialsAndServicesQuantityOnChange event handler is opened. Do not fill the handler procedure body at that point.

9. In the form editor window, click the **Form** tab and create the **OnChange** event handler for the **MaterialsAndServicesPrice** field in a similar manner.
10. Replace the script in the Services document form module with the following (listing 4.5).

Listing 4.5. Services document form module

```
&AtClient
Procedure MaterialsAndServicesQuantityOnChange(Item)
        TabularSectionRow = Items.MaterialsAndServices.CurrentData;
        DocumentProcessing.CalculateTotal(TabularSectionRow);
    EndProcedure

    &AtClient
    Procedure MaterialsAndServicesPriceOnChange(Item)
        TabularSectionRow = Items.MaterialsAndServices.CurrentData;
        DocumentProcessing.CalculateTotal(TabularSectionRow);
EndProcedure
```

Finally, let us edit the command interface to make the document creation commands available in the Services section.

11. Open the editor of the **Services** subsystem configuration object and click the **Command interface** button.

    This opens the window that lists all commands of the selected subsystem.

12. In the **Actions panel.Create** group, select the check box next to the **Services: create** command (fig. 4.26).

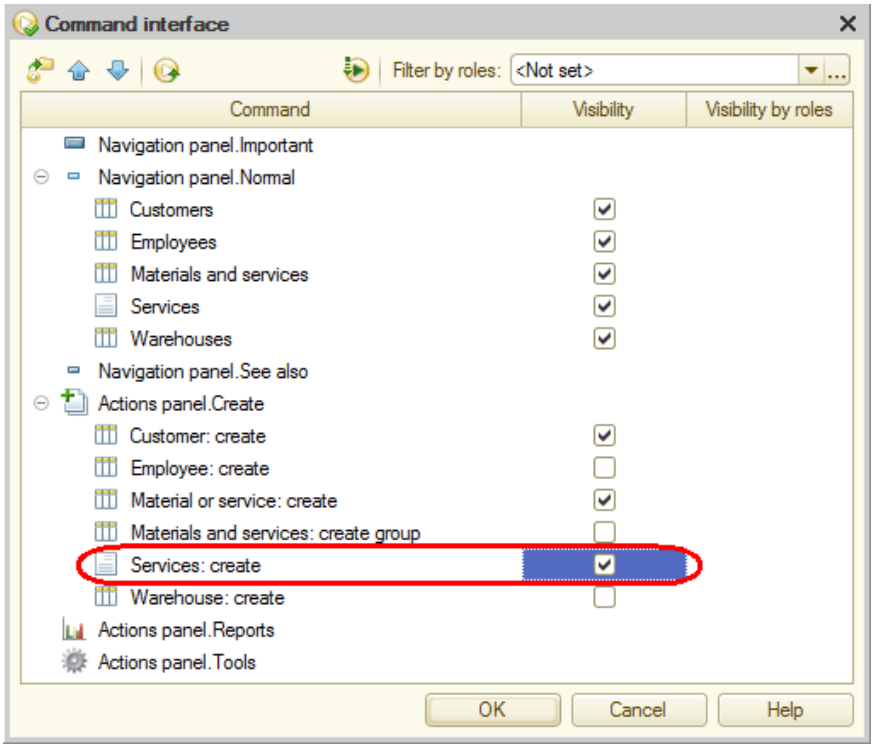


Fig. 4.26. Specifying subsystem command interface settings

The Services document now looks as follows in the configuration object tree (fig. 4.27).

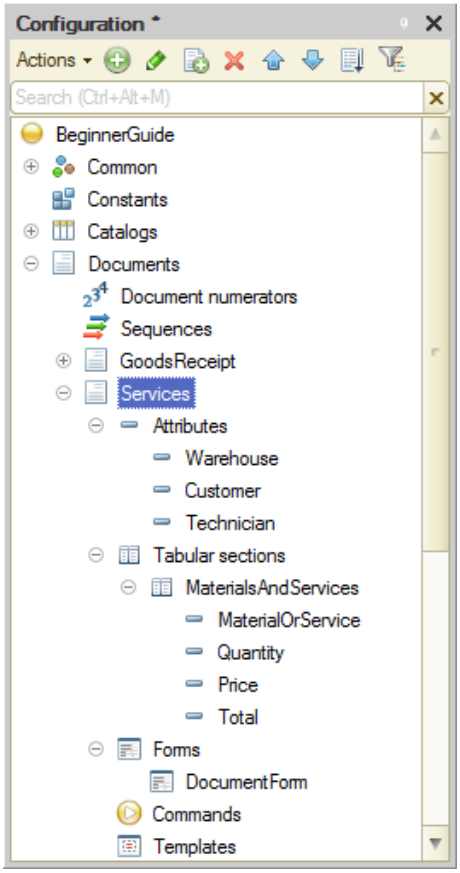


Fig. 4.27. The Services document in the configuration object tree

In 1C:Enterprise mode

Let us check the Services document.

1. Start 1C:Enterprise in the debug mode.
2. In the **Services** section, on the **Create** menu, click the **Services** command.
3. Fill the document as follows (fig. 4.28).

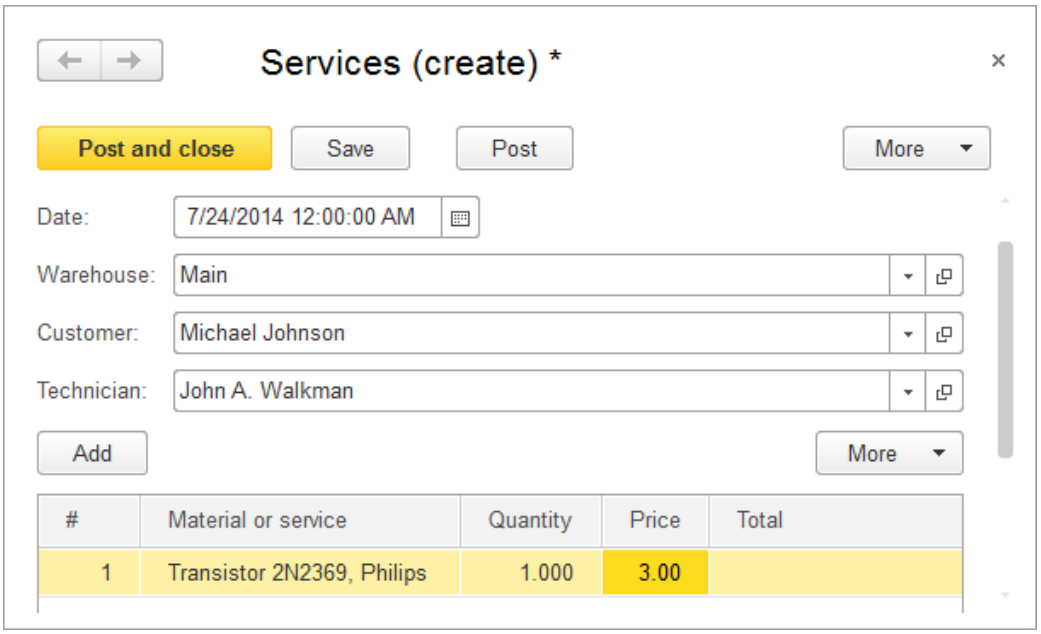


Fig. 4.28. Creating a Services document #1

Note that the Main warehouse is selected by default and the fill check is applied to the Customer and Technician fields. Also when you enter the price and quantity in the tabular section of the Services document, the total is recalculated according to the specified algorithm.

Quiz

- What are the uses of the Document configuration object?
- What are the distinctive document features?
- What are the uses of document attributes and tabular sections?
- What default document forms are available?
- What is posting of a document?
- How does one create a Document configuration object and define its basic structure?
- How does one create a document and populate it with data?
- How does one create a custom document form?
- What is the form wizard?
- What is the form editor?
- What is a form control?
- What is an event? What are events linked to?
- What is an event handler? How does one create an event handler?
- What is a module? What is a module used for?
- Why do we need common modules?
- What are type-defining objects?

Lesson 5 (2:00). Theory

To avoid complex explanations in the large previous lesson covering Documents, we have decided to discuss all the theoretical issues here in an individual lesson. Here we will pay attention to the issues we have omitted or only briefly covered earlier.

This theory is very important for understanding why we use the specific approaches described in the previous lessons. This information is vital for those readers who are willing to develop configurations themselves in the future.

But if you find this section too complicated, you can skip it and return here later. This will in no way damage the development of the training configuration that we discuss in this book.

Default forms

In the previous lesson (page 117) you created a form of the Goods receipt document and made this form the default one. What does it mean?

Each applied object in a configuration has a certain number of default forms. These forms offer various ways of displaying the object data.

If a developer does not specify custom default forms, the platform generates required object forms automatically when they are called.

This feature reduces the development time because you can use the default forms instead of creating them manually.

The platform generates the forms dynamically, on the fly. Forms are created at the moment they are called (both interactively and from 1C:Enterprise script).

For example, the list form of the Customers catalog is created both when, on the main menu, you point to **All functions**, point to **Catalogs**, and click **Customers**, and when the GetForm() global method is called (listing 5.1).

Listing 5.1. Calling the GetForm() method

```
ListForm = GetForm("Catalog.Customers.ListForm");
```

Another fact worth mentioning is that the assortment of default forms available for a configuration object might not match the list of forms that you can create for that object using the form wizard.

For example, for most registers you can create a default list form in Designer. However, if you open the form wizard for a register, you will see that, in addition to a list form, you have the option to create a register record set form, which is not available in the list of default forms.

This is because the assortment of default forms is determined based on how the data may need to be displayed to users during their work. Developers have the option to create custom forms and make them the default forms, or they can use the automatically generated forms.

In contrast, the form wizard is based on the developer needs. If a developer needs to use a record set form for some register rather than the usual list form, they can do it by creating the form in the wizard and then specifying it as the default register form. But this will not make any real difference in the system operation logic.

Event handlers

In the previous lesson (page 134) you created the OnChange event handler for some form controls. What is it?

When working with events in the 1C:Enterprise platform, you need to distinguish between two types of events: events connected to a form and its controls and all the other events.

The difference is that the event handlers related to forms and its controls are assignable, whereas the handlers of all other events are fixed.

A fixed event handler must have a name that matches the event name. This is the only way it can be called when the respective event occurs.

An assigned handler can have a custom name. Simply having a procedure name match the name of a form or its control is not enough to call the event handler procedure with that name. The handler procedure must be explicitly assigned to that event in the property palette.

Therefore any procedure located in a form module can be assigned as a handler of one or several events related to the form or the form controls. In this scenario the procedure name is irrelevant. The only thing that matters is that it is assigned to handle a specific event.

Handlers can be assigned either interactively when you edit the form in Designer, or from 1C:Enterprise script, using the SetAction() method for a form or a form control.

Modules

In the previous lesson (page 134) we discussed event handler script. You have learned that event handler procedures are located in form modules, which are basically storages for 1C:Enterprise script algorithms.

Now let us discuss the modules in more detail and get to know form module nature better.

Module types

Configuration can include various module types. The modules can belong to certain objects (such as forms), or they can exist independently (belong to a configuration as a whole).

The platform uses the script stored in the modules at predefined points of 1C:Enterprise operation. These points are referred to as events and we have discussed them earlier.

The following module types are available in 1C:Enterprise.

1. **Managed application module**. It is executed when 1C:Enterprise is started in the thin client or web client modes.

You can declare variables and define procedures and functions in this module, and they will be available to all configuration modules (except the external connection module). They are also available for nonglobal common modules with the Client (managed application) property enabled. Exported procedures and functions of common modules are available in the managed application module context.

To open a managed application module

- In the configuration object tree, right-click the **BeginnerGuide** root item and click **Open managed application module** (fig. 5.1).

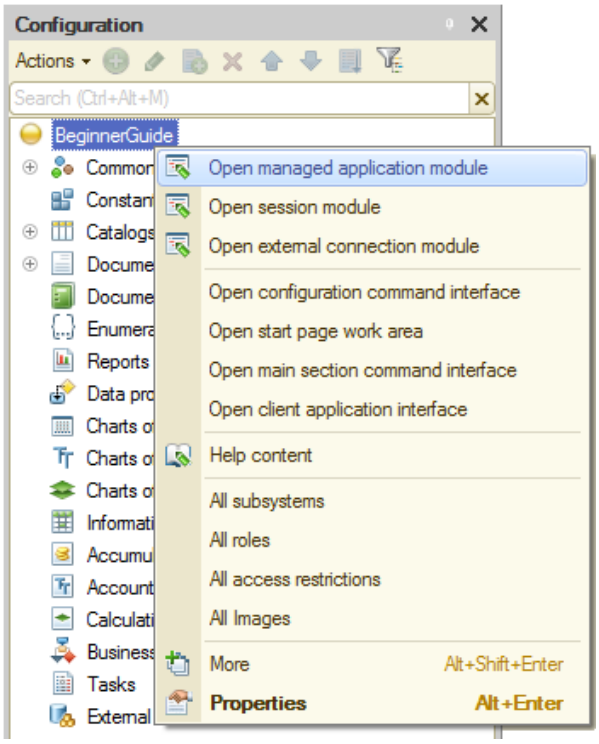


Fig. 5.1. Opening the managed application module

2. **Common modules**. They store the procedures and functions that are called from other modules. A common module itself is never executed, it simply stores the procedures and functions.

To open a common module

- In the configuration object tree, expand the **Common** branch, then expand the **Common modules** branch and double-click the module (fig. 5.2).

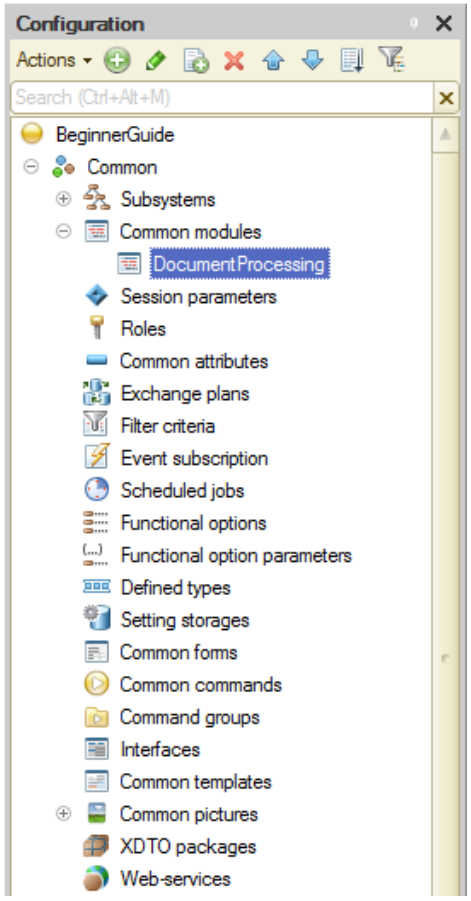


Fig. 5.2. Opening a common module

3. **Object modules**. Examples of such modules are a catalog item module and a document module.

An object module is called when an object is created using 1C:Enterprise script (for example, the CreateItem() method of a catalog manager is called, or the CreateDocument() method of a document manager), or when a user creates an object (for example, a catalog item or a document).

When modified object data is recorded to the database, event handlers located in the object module are called.

For details on the event sequence during the writing of modified configuration objects, see chapter "Quick developer reference" on page 832.

To open an object module

- In the configuration object editor, click the **Other** tab and then click **Object module** (fig. 5.3).

  -OR-

  In the configuration object tree, right-click the object and then click **Open object module**.

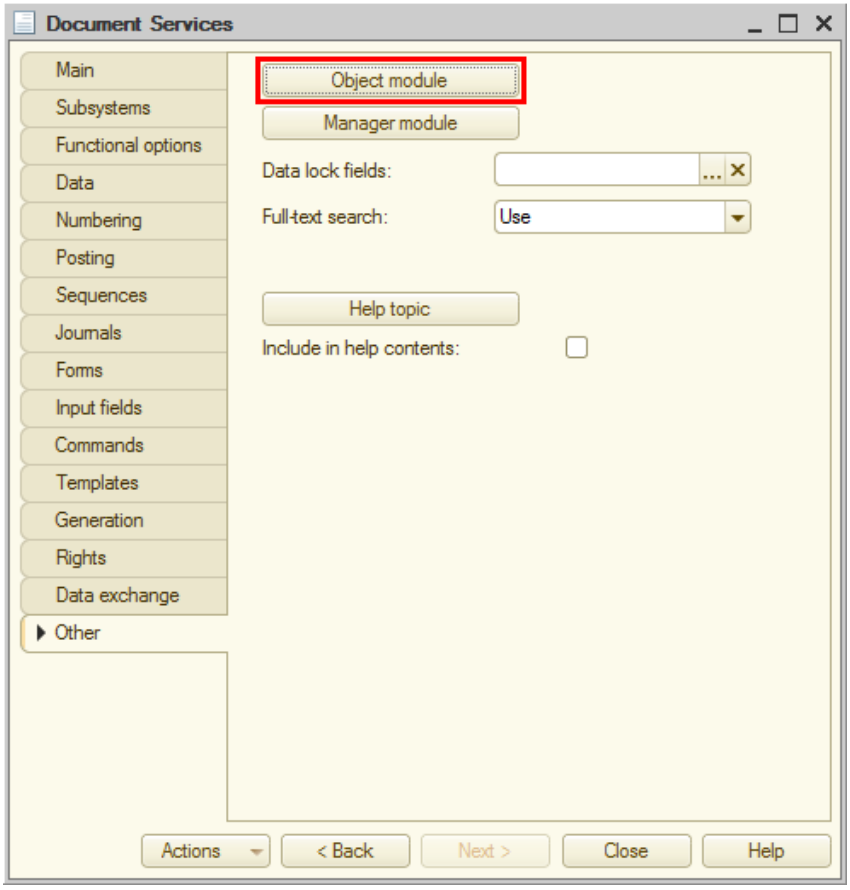


Fig. 5.3. Opening an object module

4. **Form modules**. Each form that is defined in a configuration has its own module. This module is executed when the ManagedForm object of 1C:Enterprise script is created. The platform creates this object in 1C:Enterprise mode when a form is opened either using 1C:Enterprise script (the GetForm() or OpenForm() method) or interactively.

To open a form module

- In the configuration object tree, expand the object and open its subordinate Form object, then click the **Module** tab (fig. 5.4).

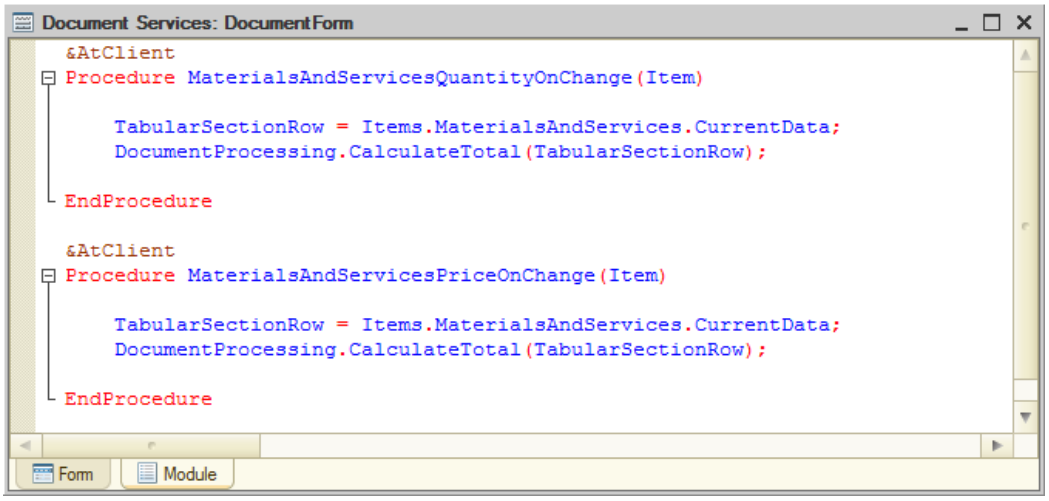


Fig. 5.4. Opening a form module

5. **Session module**. It is executed when the configuration is loaded during 1C:Enterprise startup. A session module is intended to initialize the session parameters and execute the session-related actions. It cannot contain exported procedures or functions. It can call procedures from the common modules.

To open a session module

- In the configuration object tree, right-click the **BeginnerGuide** root item and **click Open session module** (fig. 5.5).

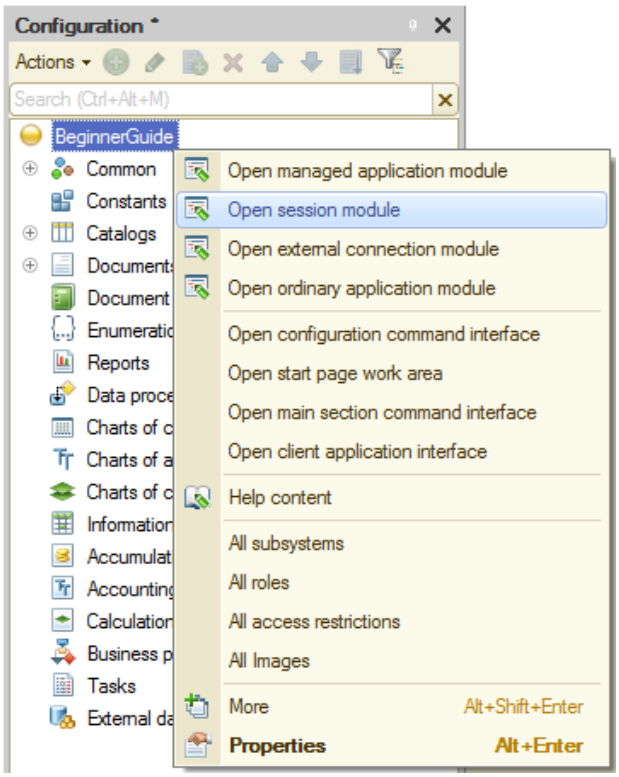


Fig. 5.5. Opening a session module

6. **External connection module**. It is intended for storing functions and procedures that can be called in an external connection session.

To open a session module

- In the configuration object tree, right-click the **BeginnerGuide** root item and click **Open external connection module** (see fig. 5.5).

7. **Manager modules**. Each application object has a manager that manages the corresponding configuration object. Using this manager, you can create objects and perform operations with their forms and templates. A manager module allows you to extend the functionality of default managers provided by the platform by writing procedures and functions in 1C:Enterprise script.

In fact, this allows you to define custom methods for a configuration object (for example, a catalog) that do not refer to a specific database object instance, but rather to the configuration object itself.

To open a manager module

- In the configuration object editor, click the **Other** tab and click **Manager module**.

  -OR-

  In the configuration object tree, right-click the object and click **Open manager module**.

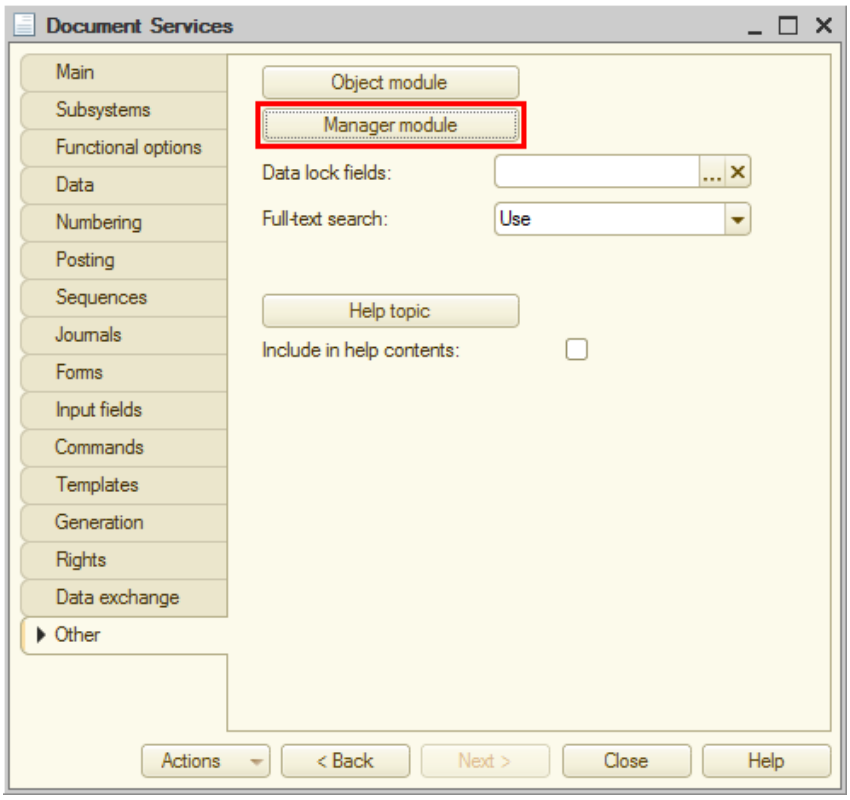


Fig. 5.6. Opening a manager module

8. **Command modules**. The entire configuration and its applied objects can have subordinate configuration objects referred to as Commands. Each command has a command module. You can use this module to create a predefined CommandProcessing() procedure, which is called when the command is executed.

To open the module of a command subordinate to a configuration object

- In the configuration object editor, click the **Commands** tab and double-click the command (fig. 5.7).

  -OR-

  In the configuration object tree, right-click the command and click **Open command module**.

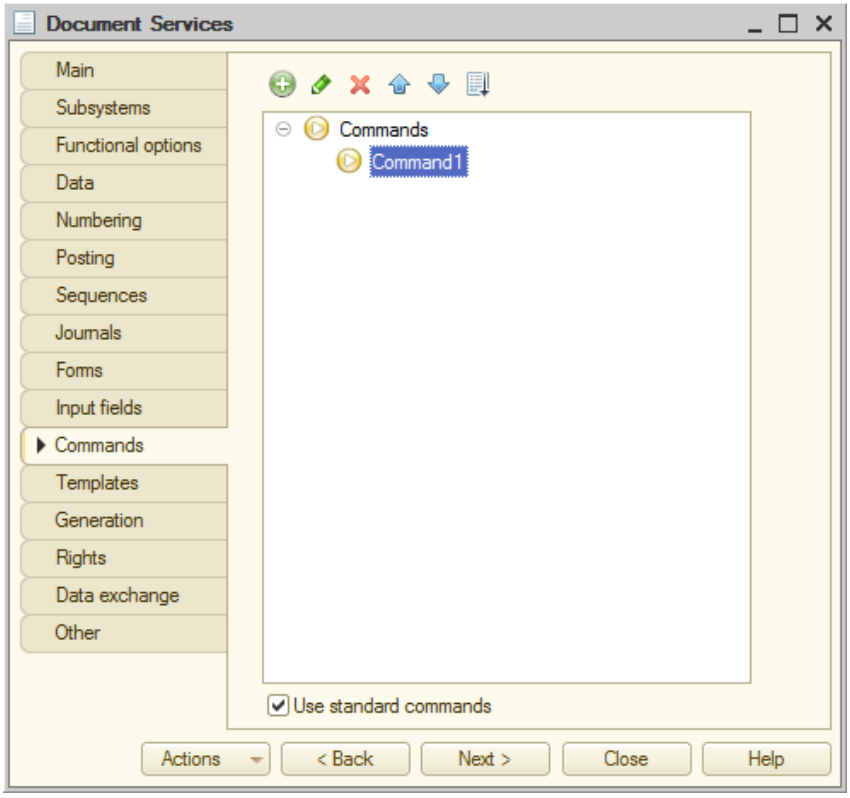


Fig. 5.7. Opening a module of a configuration object command

Form module context

Each module is connected to other configuration parts. This relationship is referred to as module context.

The module context defines the assortment of objects, variables, procedures, and functions available when the module is executed.

Since further we will talk about the handler that you created in the form module, let us discuss the content of form module context in detail.

A form module context consists of:

1. Local form module context
2. Module form attributes
3. Properties and methods of the ManagedForm 1C:Enterprise script object
4. Properties and methods of the form extension that is defined by the type of the object whose data is stored in the main form attribute
5. Global context, including nonglobal common modules and exported functions and procedures of global common modules
6. Exported variables, procedures, and functions of the managed application module

Now let us discuss all the listed items in detail.

1. Local form module context.

   The local form module context consists of variables, procedures, and functions declared in that module.

For example, inside a form module you can call the GetTotal() procedure that is declared in this module using its name (listing 5.2).

Listing 5.2. Form module

```
&AtClient
Procedure Command1()
    GetTotal();
    EndProcedure

&AtServerNoContext
Procedure GetTotal()
    ...
EndProcedure
```

Also, inside a form module you can directly address the InternalVariable variable that is declared in this module using its name (listing 5.3).

Listing 5.3. Form module

```
&AtClient
Var InternalVariable;

&AtClient
Procedure Command1()
    InternalVariable = 3;
EndProcedure
```

2. Module form attributes.

   For example, if a form has an InternalAttribute attribute (fig. 5.8), you can directly address it using its name (listing 5.4).

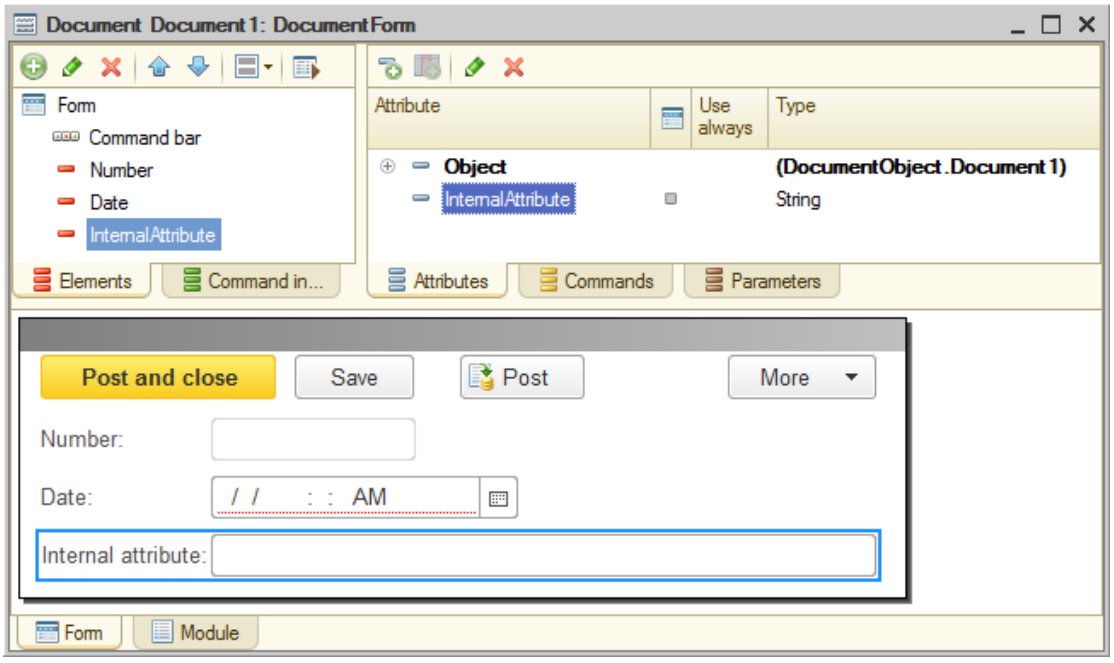


Fig. 5.8. Form attribute: InternalAttribute

Listing 5.4. Form module

```
&AtClient
Procedure Command1()
    InternalAttribute = 3;
EndProcedure
```

3. Properties and methods of the ManagedForm object.

   Properties and methods of the ManagedForm 1C:Enterprise script object are described in the Syntax Assistant (page 164), in section Interface(managed) / Managed form / ManagedForm (fig. 5.9).

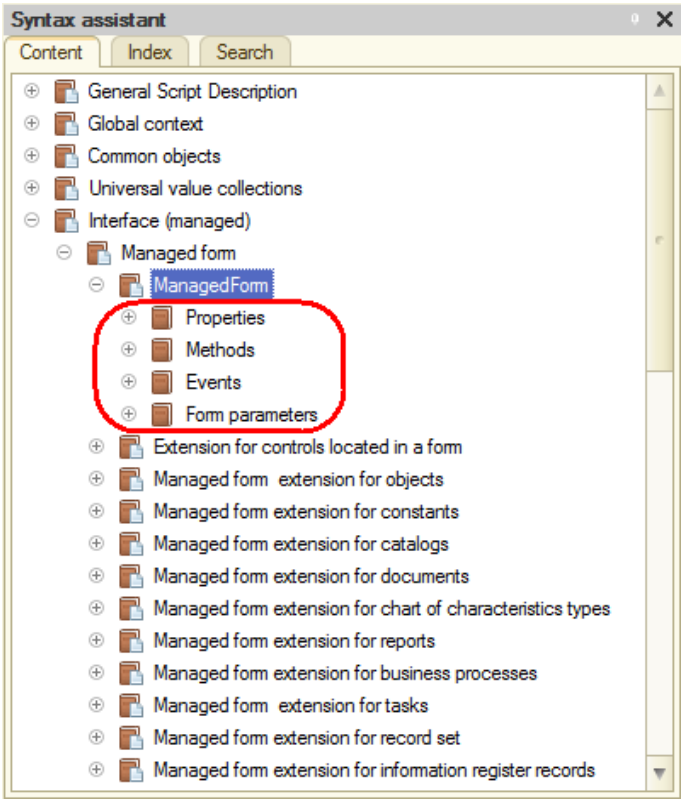


Fig. 5.9. Managed form properties and methods in the Syntax Assistant

You can address them directly using their names. For example, you can define a form header (listing 5.5).

Listing 5.5. Form module

```
&AtClient
Procedure Command1()
    Title = "New form title";
EndProcedure
```

Or you can close a form (listing 5.6).

Listing 5.6. Form module

```
&AtClient
Procedure Command1()
    Close();
EndProcedure
```

4. Properties and methods of the form extension that is defined by the type of the object whose data is stored in the main form attribute.

   One of the form attributes can be the default one. It is displayed in bold in the attribute list. Normally the default form attribute contains the data of the object displayed in the form. For example, in a catalog form the default attribute stores the data of the CatalogObject.<name> object (fig. 5.10).

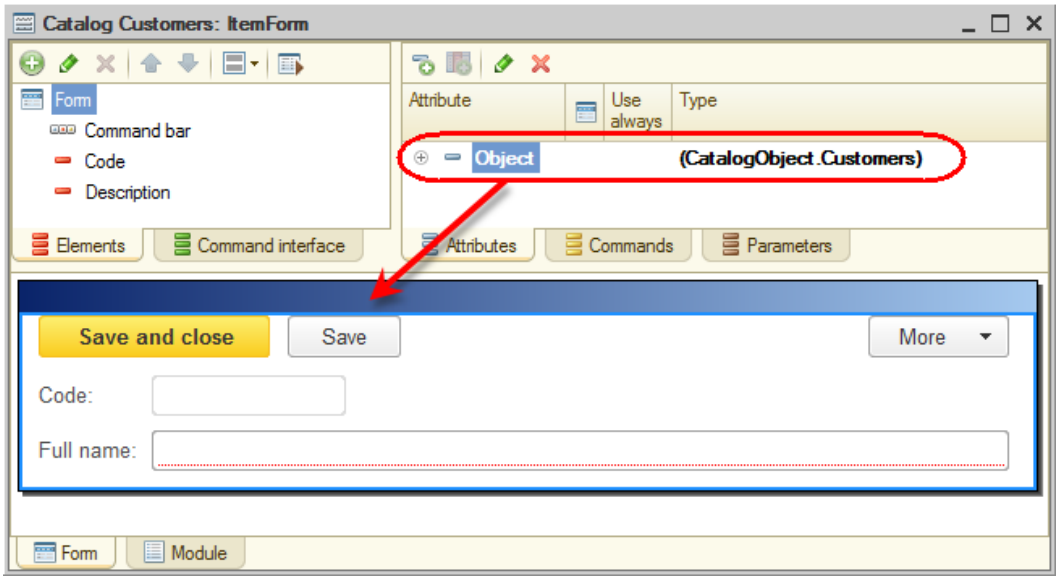


Fig. 5.10. Default form attribute

And in a document form the default attribute stores the data of the DocumentObject.<name> object (fig. 5.11).

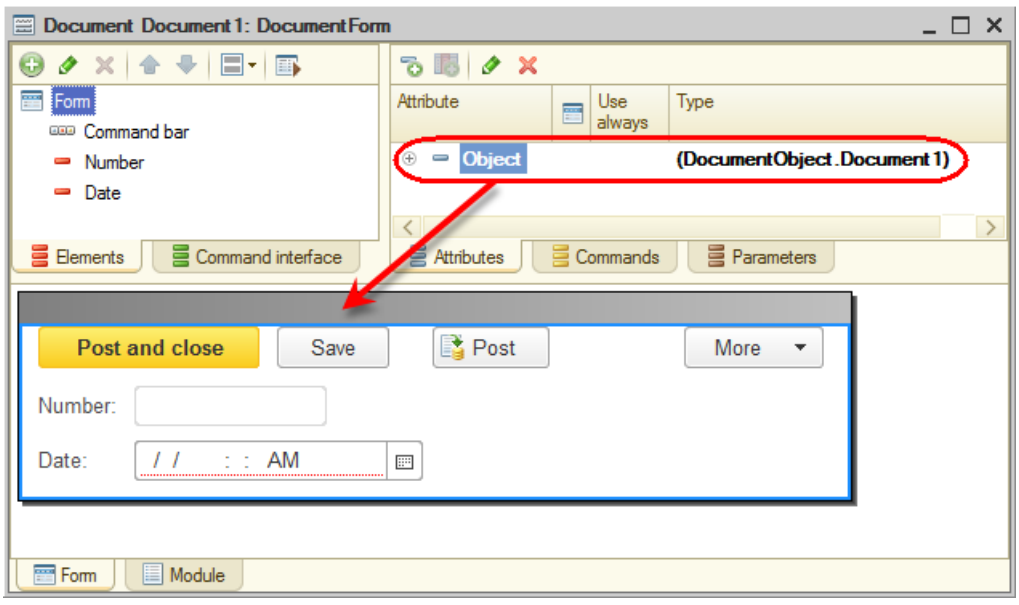


Fig. 5.11. Default form attribute

Here we should clarify why the default attribute type is displayed in brackets in the list of attributes: (DocumentObject.Document1). This is because it is not the real attribute type. In this case the real type is FormDataStructure (fig. 5.12).

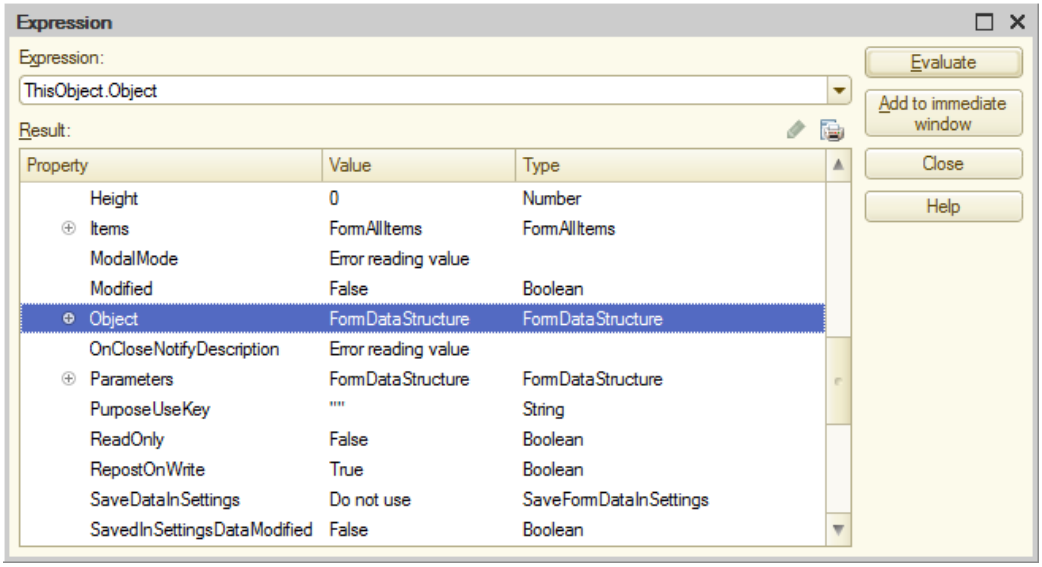


Fig. 5.12. Object type of the default form attribute

But FormDataStructure is a universal type that can store data of various application objects. So to provide a clear description of the application object displayed in the form, the Type column of the editor shows the type of the object whose data is stored in this attribute instead of the actual form attribute type (FormDataStructure). And this "fake" type is displayed in the brackets.

The type of the object whose data is stored in the default form attribute affects some form behavior specifics.

For example, if the default form attribute stores document data, the user is prompted to confirm saving and posting of this document when the form is closed. And if the default form attribute stores catalog data, the platform does not display that prompt when the form is closed.

Depending on the type of the object whose data is stored in the default attribute, the context of the ManagedForm script object includes the context of the respective extension.

For example, if the default attribute is CatalogObject.<name>, the properties and methods of the Managed form extension for catalogs 1C:Enterprise script object are available in the form module (in the Syntax Assistant, see section Interface (managed) / Managed form / Managed form extension for catalogs, fig. 5.13).

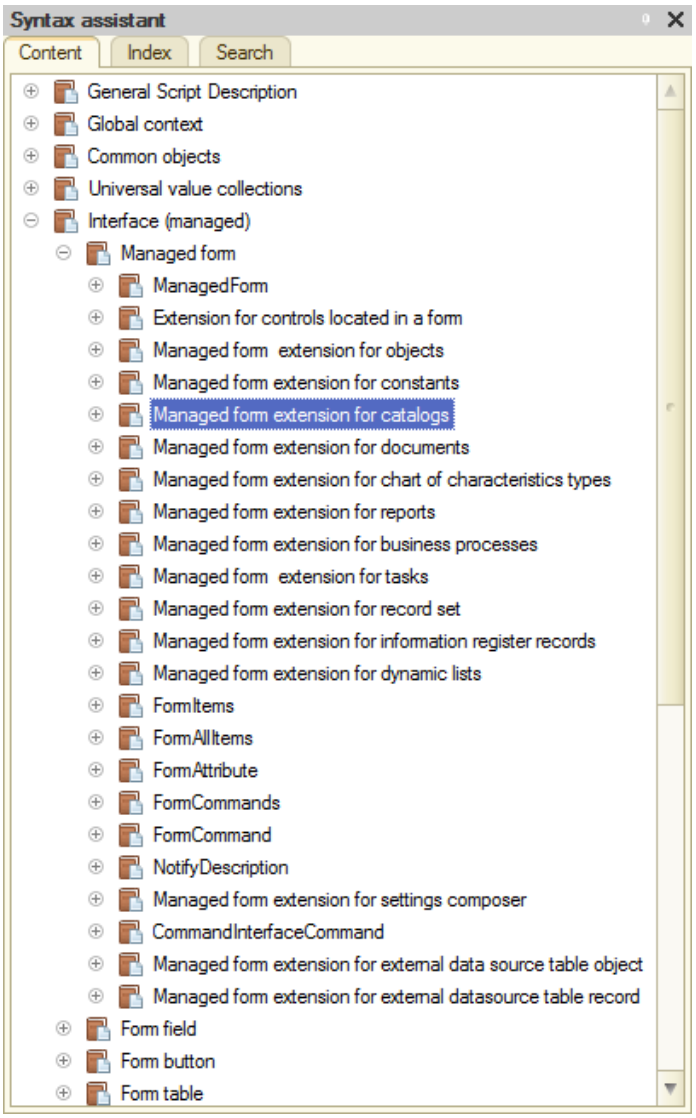


Fig. 5.13. Object descriptions in Syntax Assistant

And if the default attribute is DynamicList (fig. 5.14), the properties and methods of the Managed form extension for dynamic lists 1C:Enterprise script object are available in the form module (in the Syntax Assistant, see section Interface (managed) / Managed form / Managed form extension for dynamic lists, fig. 5.15).

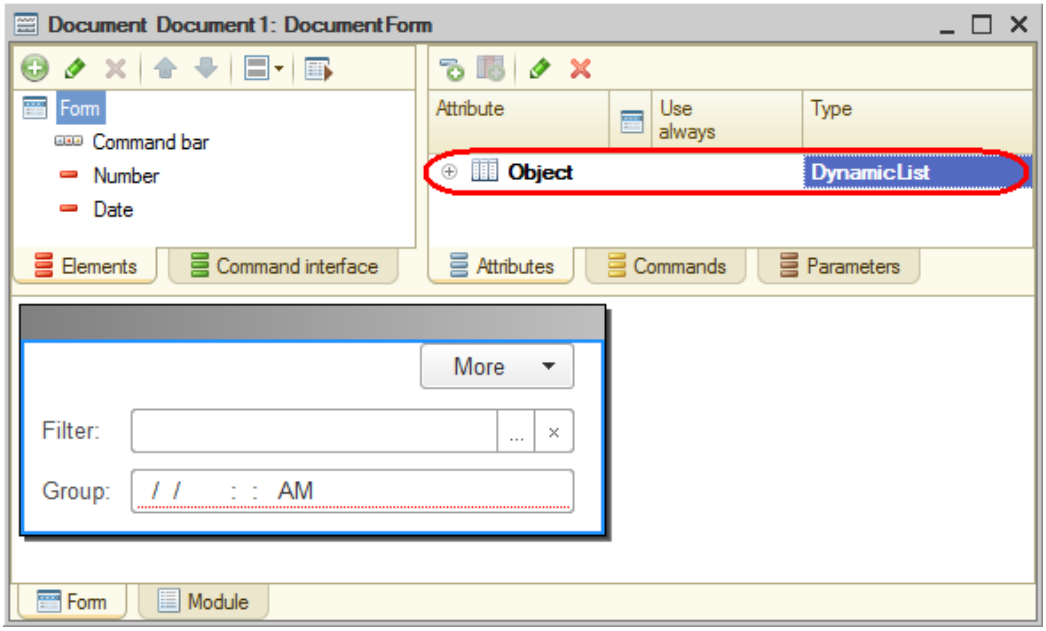


Fig. 5.14. Default form attribute

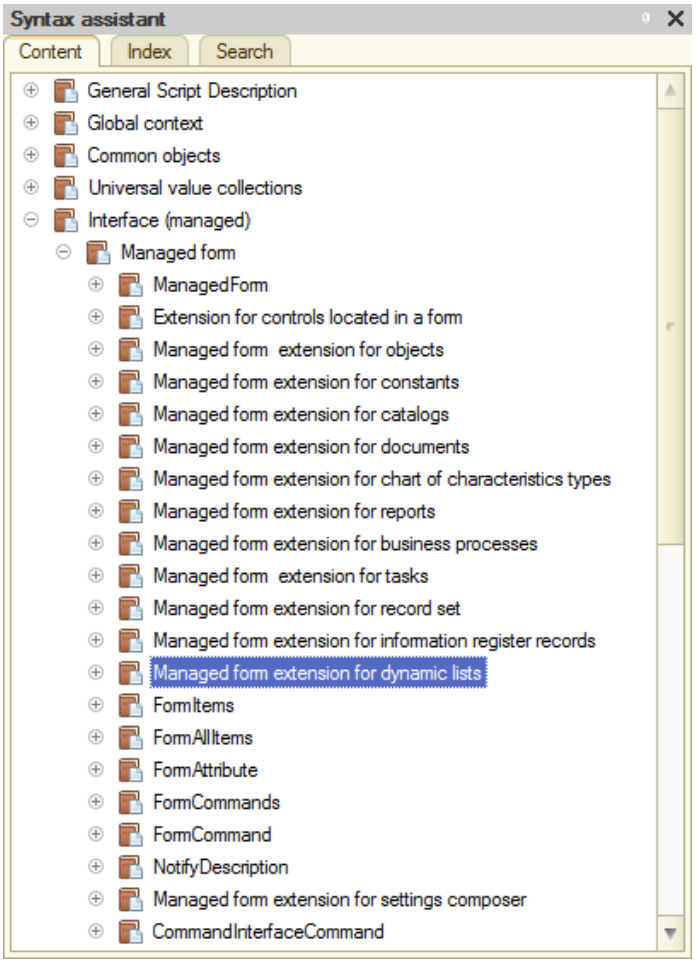


Fig. 5.15. Object descriptions in Syntax Assistant

So in a form module where the default attribute stores document data (fig. 5.16) you can call the AutoTime property of Managed form extension for documents (listing 5.7).

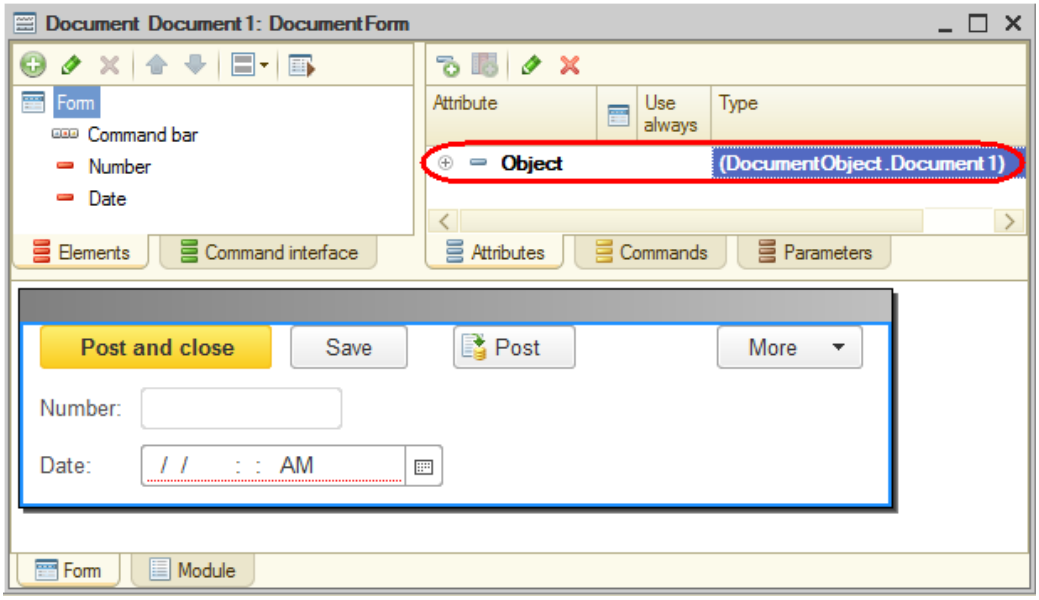


Fig. 5.16. Default form attribute

Listing 5.7. Form module

```
&AtClient
Procedure Command1()
    AutoTime = AutoTimeMode.First;
EndProcedure
```

Or you can save the document using the Write() method of Managed form extension for documents (listing 5.8).

Listing 5.8. Form module

```
&AtClient
Procedure Command1()
    Write(DocumentWriteMode.Posting);
EndProcedure
```

5. Global context, including nonglobal common modules and exported functions and procedures of global common modules.

   In a form module you can get the system date by calling the CurrentDate() built-in function (listing 5.9).

Listing 5.9. Form module

```
&AtClient
Procedure Command1()
    Message(CurrentDate());
EndProcedure
```

Or you can get the history of user actions by calling the UserWorkHistory property of the global context (listing 5.10).

Listing 5.10. Form module

```
&AtClient
Procedure Command1()
    History = UserWorkHistory.Get();
EndProcedure
```

If a global common module (for example, DataExchange) defines the GetNumberPrefix() export procedure (listing 5.11), you can call it using its name in the form module (listing 5.12).

Listing 5.11. Global common module

```
Function GetNumberPrefix() Export
    Return Constants.NumberPrefix.Get();
EndFunction
```

Listing 5.12. Form module

```
&AtClient
Procedure Command1(Prefix)
    Prefix = GetNumberPrefix();
EndProcedure
```

If such common module is nonglobal (for example, DocumentProcessing), when you call a procedure you have to specify the procedure name after the module name, separated by dot (.), see listing 5.13.

Listing 5.13. Form module

```
&AtClient
Procedure QuantityOnChange(Item)
    DocumentProcessing.CalculateTotal(TabularSectionRow);
EndProcedure
```

The latter method is better because, unlike global modules, nonglobal common modules are not compiled at the system startup. Instead, they are compiled when they are called.

Of course, it is important to ensure that the procedure directive in the form module (&AtClient, &AtServer, and so on) matches the properties of the common module (Client (managed application), Server, and so on).

6. Exported variables, procedures, and functions of the managed application module.

   If a TestMessage() export procedure is defined in an application module (listing 5.14), you can call it using its name in the form module (listing 5.15).

Listing 5.14. Application module

```
Procedure TestMessage() Export
    Message("Test message");
EndProcedure
```

Listing 5.15. Form module

```
&AtClient
Procedure Command1()
    TestMessage();
EndProcedure
```

A form as a script object

In addition to the fact that you can access various configuration parts from a form, you can also access a form (as a script object) from other configuration parts.

And, together with standard properties and methods of the ManagedForm 1C:Enterprise object, a form can have other developer-defined properties and methods.

For example, if the module of the DocumentForm form of the GoodsReceipt document describes the MaterialsQuantityOnChange() exported procedure (listing 5.16), you can call this procedure as follows (listing 5.17).

Listing 5.16. Form module

```
&AtClient
Procedure MaterialsQuantityOnChange(Item)
    TabularSectionRow = Items.Materials.CurrentData;
    DocumentProcessing.CalculateTotal(TabularSectionRow);
EndProcedure
```

Listing 5.17. Form module

```
Form = GetForm("Document.GoodsReceipt.Form.DocumentForm");
Form.MaterialsQuantityOnChange();
```

Event handler procedures in form modules

In addition to variable declarations and the main program, a form module can contain procedures that are handlers of the events related to the form. The events of opening and closing the form window are the most common events that can be handled in the form module (listing 5.18).

Listing 5.18. Form module

```
&AtServer
Procedure OnCreateAtServer(Cancel, StandardProcessing)
    // Insert handler content
EndProcedure

&AtClient
Procedure OnOpen(Cancel)
    // Insert handler content
EndProcedure
```

It is important that the names of these procedures are not fixed, you can name them as you like. And it is not sufficient, for example, to define the OnCreateAtServer or OnOpen procedures in the module. You also have to link them to the respective form events in Designer (fig. 5.17).

Fig. 5.17. Form events

Novice developers often forget about this aspect. To avoid such situations, we recommend that you use the opening buttons in the property palette to generate the procedures (fig. 5.18).

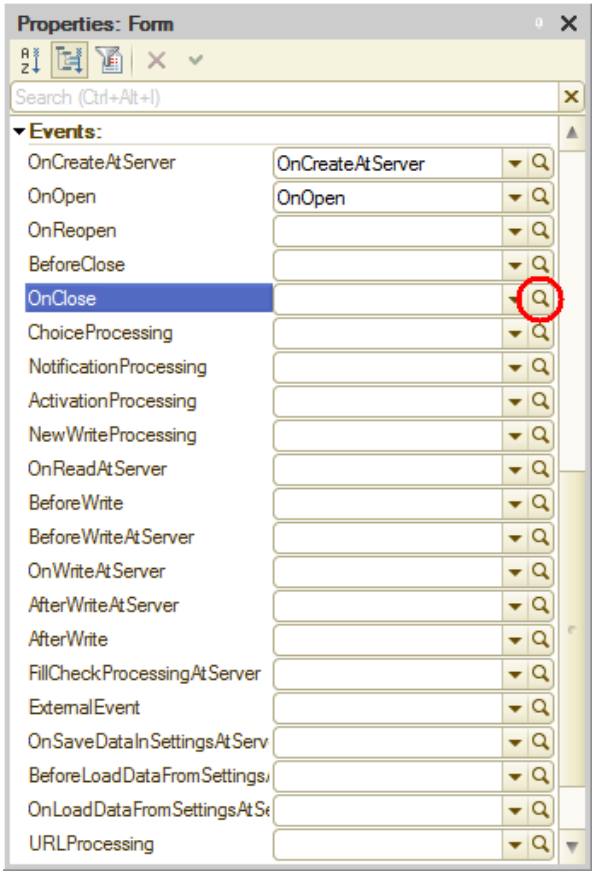


Fig. 5.18. Form events

Alternatively, you can select them from the drop-down list that is available when you are in the form module (fig. 5.19).

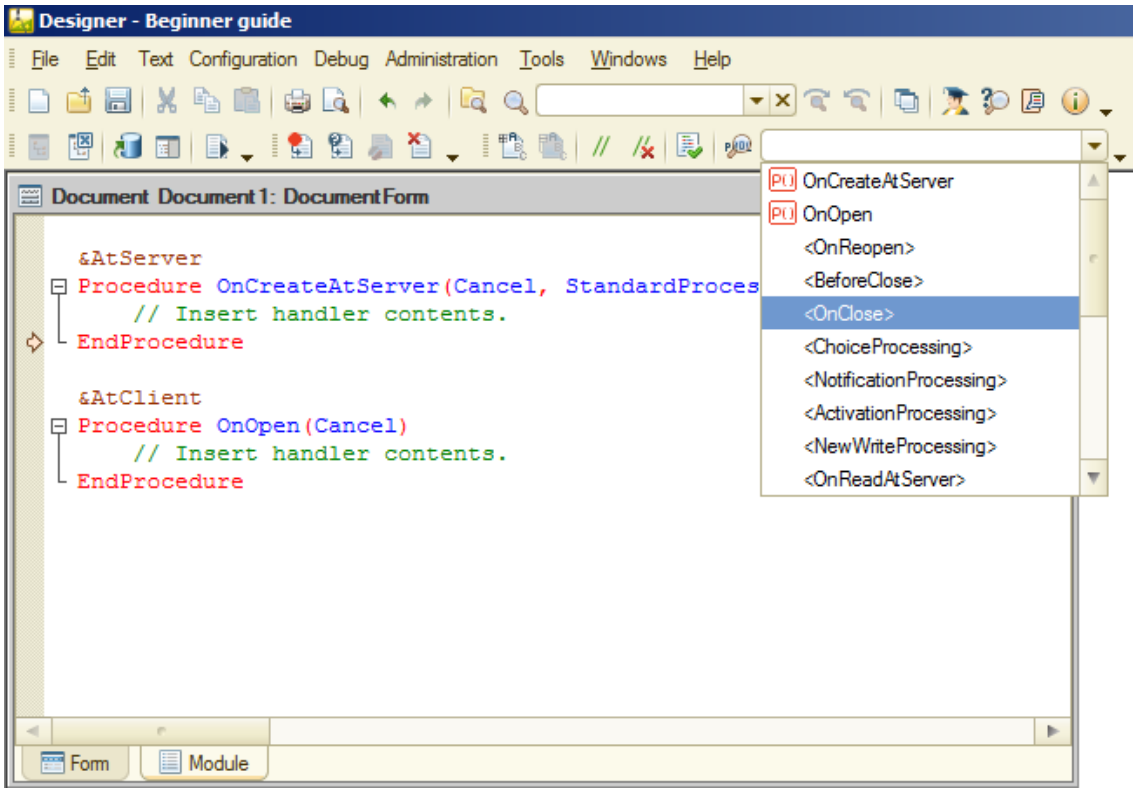


Fig. 5.19. List of form events

In this case the procedure definitions are automatically generated and linked to the respective form properties.

Understanding form module content

Finally we will briefly describe the general algorithm for understanding the form module script on the example of the MaterialsQuantityOnChange event handler that you created earlier.

Suppose you see the following expression in the form module:

```
TabularSectionRow = FormItems.Materials.CurrentData;
```

How does one understand the meaning of TabularSectionRow? Let us remember the form context (page 152) parts.

1. Local form module context
2. Module form attributes
3. Properties and methods of the ManagedForm 1C:Enterprise script object
4. Properties and methods of the form extension that is defined by the type of the object whose data is stored in the main form attribute
5. Global context, including nonglobal common modules and exported functions and procedures of global common modules
6. Exported variables, procedures, and functions of the managed application module

And check these positions one by one:

1. Is the TabularSectionRow variable declared in the form module? No.
2. Does the form have the TabularSectionRow attribute? No.
3. Does the ManagedForm object have the TabularSectionRow property? No.
4. Does the form extension have the TabularSectionRow property? No.
5. Does the TabularSectionRow global context property exist? No.
6. Does the managed application module contain exported TabularSectionRow variable? No.

This means that TabularSectionRow is a local variable that is defined directly in this assignment operator.

Understanding 1C:Enterprise script

In the previous lesson (page 134) you wrote the handler of the MaterialsQuantityOnChange event (listing 4.1) and we provided a brief description of its functioning.

Now we will describe two approaches to understanding numerous unfamiliar configuration object properties and methods, which will help you study script fragments and create custom 1C:Enterprise script procedures.

The Syntax Assistant is a developer tool. It stores descriptions of all script objects available in the platform along with their methods, properties, events, and other related information.

To open the Syntax Assistant

- Click the button on the Designer toolbar (fig. 5.20).

  -OR-

  On the **Help** menu, click **Syntax Assistant**.

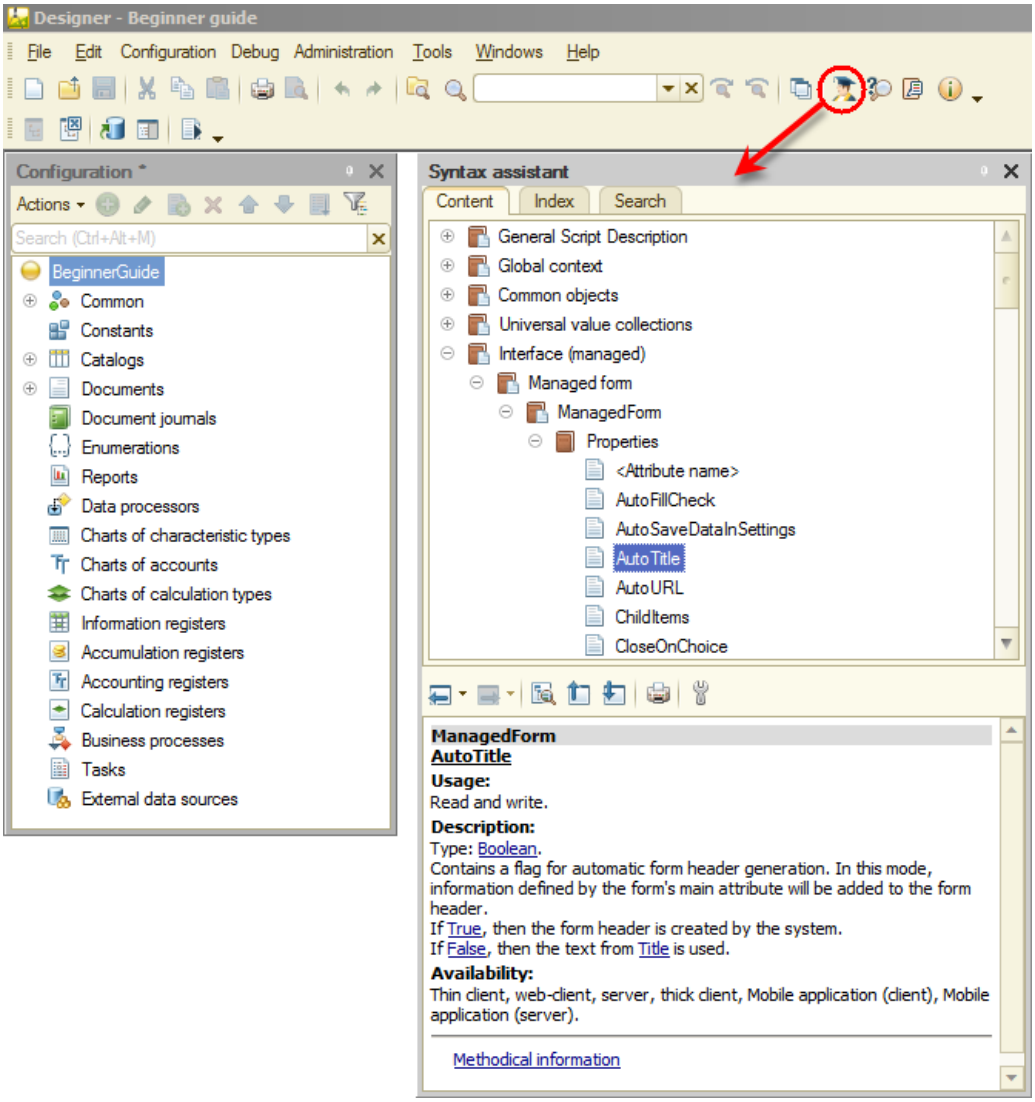


Fig. 5.20. Opening the Syntax Assistant

Same as any other help system, the Syntax Assistant has a tree-like structure with chapters, sections, subsections, and so on. Using the Syntax Assistant is more convenient than using any of the printed 1C:Enterprise script documentation because it is readily available and supports context-sensitive help (Ctrl+F1).

In addition to that, each Syntax Assistant page has a Methodical information hyperlink at the bottom (see fig. 5.20). Clicking this link opens a browser window with a list of methodical materials related to the current topic. These may include 1C:Enterprise knowledge base articles, 1C:Developer Network articles and forum topics, and more. New methodical materials are added as soon as they become available online. This provides

developers with additional options for getting answers to their questions without interrupting their work.

Using Syntax Assistant to analyze script

The Syntax Assistant is handy when you need to understand the script written by another developer. We will explain how to use it on the example of the MaterialsQuantityOnChange event handler that you wrote earlier (see listing 4.1).

Method #1

The first method is searching for a topic in the table of contents by following down the tree while expanding the subsections, properties, links, and so on.

Let us take a look at the first line of our handler (listing 5.19).

Listing 5.19. MaterialsQuantityOnChange procedure (first line)

```
TabularSectionRow = Items.Materials.CurrentData;
```

To understand this script, you need to know the execution context. The program context depends on the module where the script is located. In this case the handler procedure is located in the form module, so we are in the form module context.

Let us review this script line from left to right. What is TabularSectionRow? If something is located to the left of the assignment operator (=), it is either a property that is available directly in this context or a variable.

According to the algorithm described in the previous section (page 164), you have to make the following checks:

1. Is the TabularSectionRow variable declared in the form module?

   Open the form module (to learn how to open a module, see section "Module types" on page 146). There is no variable description line (Variable TabularSectionRow;) so this is not a form module variable.

2. Does the form have the TabularSectionRow attribute?

   Open the form of the GoodsReceipt document and check the form attributes pane located in the upper right part of the form editor (fig. 5.21).

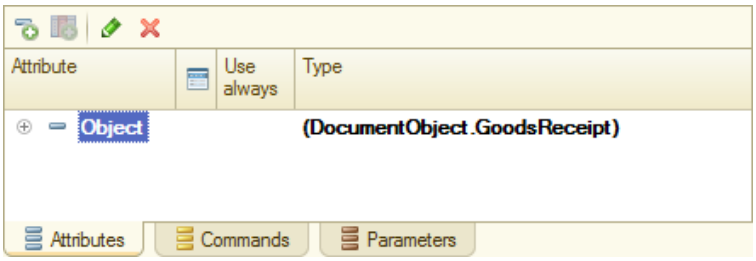


Fig. 5.21. List of attributes of the GoodsReceipt document form

   You see that this form has a single Object default attribute (displayed in bold). This means that the form does not have the TabularSectionRow attribute.

3. Does the ManagedForm object have the TabularSectionRow property?

   Let us look at the list of the managed form properties in the Syntax Assistant.

   Open the Syntax Assistant and click the **Contents** tab.

   A managed form is an interface object of a managed application, so expand the **Interface (managed)** / **Managed form** section, then expand the **ManagedForm** object and its **Properties** (fig. 5.22).

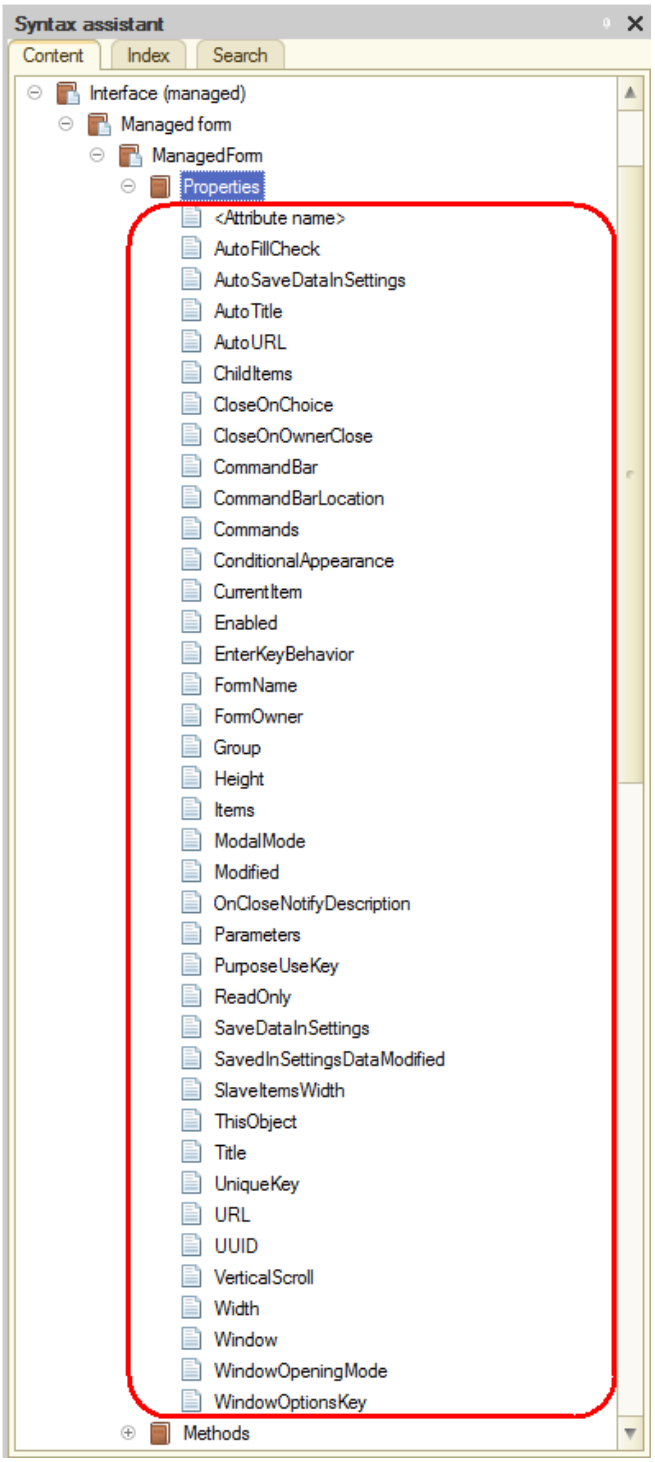


Fig. 5.22. List of ManagedForm object properties in the Syntax Assistant

The properties are sorted alphabetically. You can see that there is no TabularSectionRow property among them.

4. Does the form extension have the TabularSectionRow property?

   You know that the default form attribute stores data of the DocumentObject.GoodsReceipt object (see fig. 5.21). This means that the properties and methods of the 1C:Enterprise script object Managed form extension for documents (Syntax Assistant / Interface (managed) / Managed form / Managed form extension for documents) are available in the form module. Let us review them (fig. 5.23).

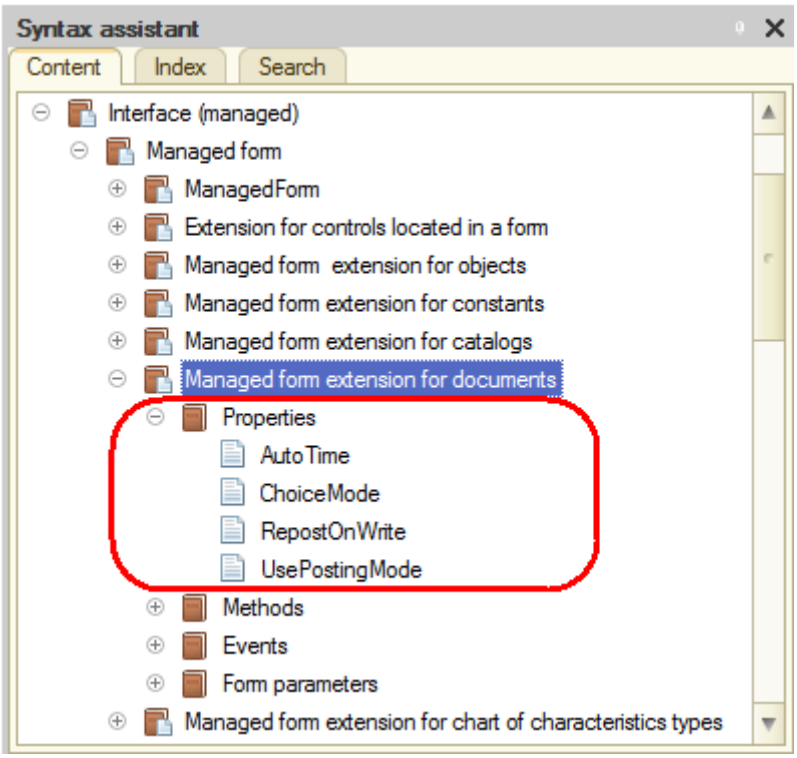


Fig. 5.23. List of properties of the managed form extension for documents in the Syntax Assistant

You can see that there is no TabularSectionRow property among them.

5. Does the TabularSectionRow global context property exist?

   Open the global context properties in the Syntax Assistant (fig. 5.24).

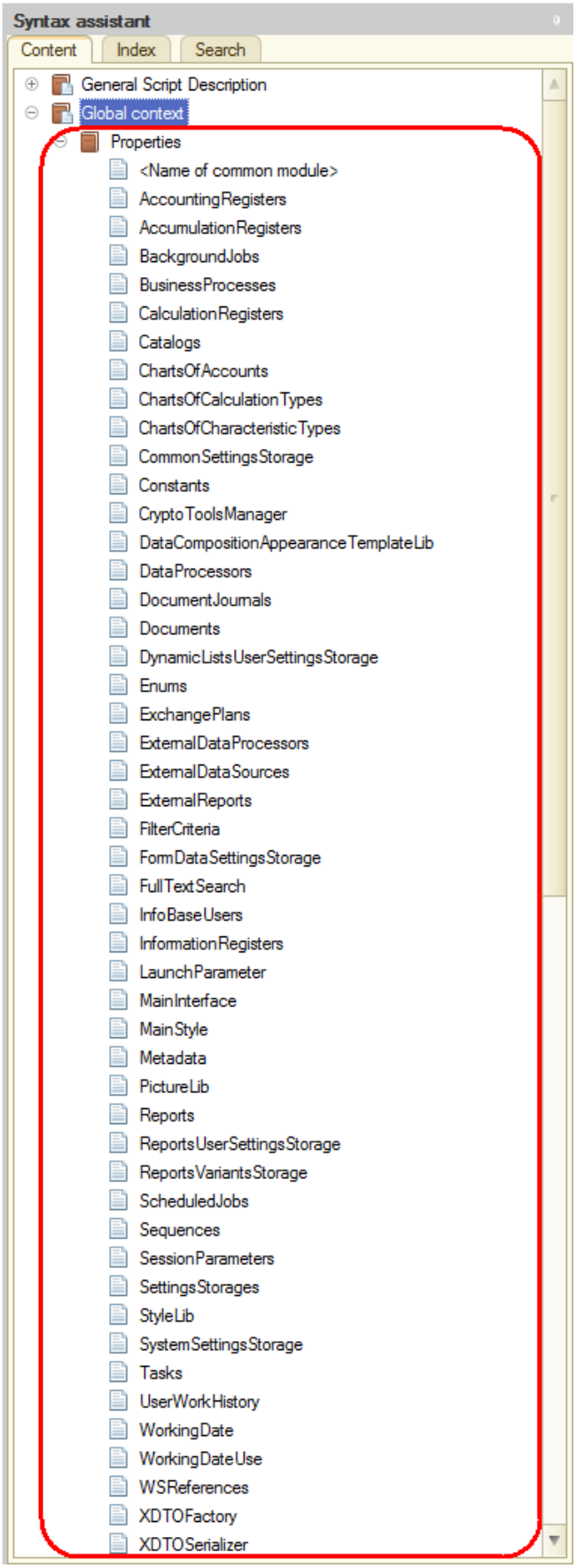


Fig. 5.24. List of global context properties in the Syntax Assistant

You can see that there is no TabularSectionRow property among them.

The TabularSectionRow expression also cannot be a name of a nonglobal common module because its procedures should be called using a dot (TabularSectionRow.).

This expression also cannot be an exported procedure of a global common module because in this case one would call the procedure as TabularSectionRow() instead of assigning anything to it.

6. Does the managed application module contain the TabularSectionRow exported variable?

   Open the managed application module (to learn how to open a module, see section "Module types" on page 146). There is no variable description line (Variable TabularSectionRow Export), so this is not a variable of a managed application module.

Now you see that the TabularSectionRow expression is a local variable of the MaterialsQuantityOnChange procedure. During the script execution a value is assigned to the variable. The 1C:Enterprise script type system allows assigning a value of any type. If a variable is local, i.e. it is only used in the context of this particular procedure, it does not require explicit declaration. The variable is declared when it is first used.

To the right of the assignment operator you can see the Items.Materials.CurrentData expression. To learn the meaning of Items, let us use the same algorithm as with the TabularSectionRow local variable.

1. Is the Items variable declared in the form module? No.
2. Does the form have the Items attribute? No.
3. Does the ManagedForm object have the Items property?

   Let us look at the list of properties of the ManagedForm object again. There is the Items property among them, so Items is one of the managed form properties. To learn its meaning, double-click it (fig. 5.25).

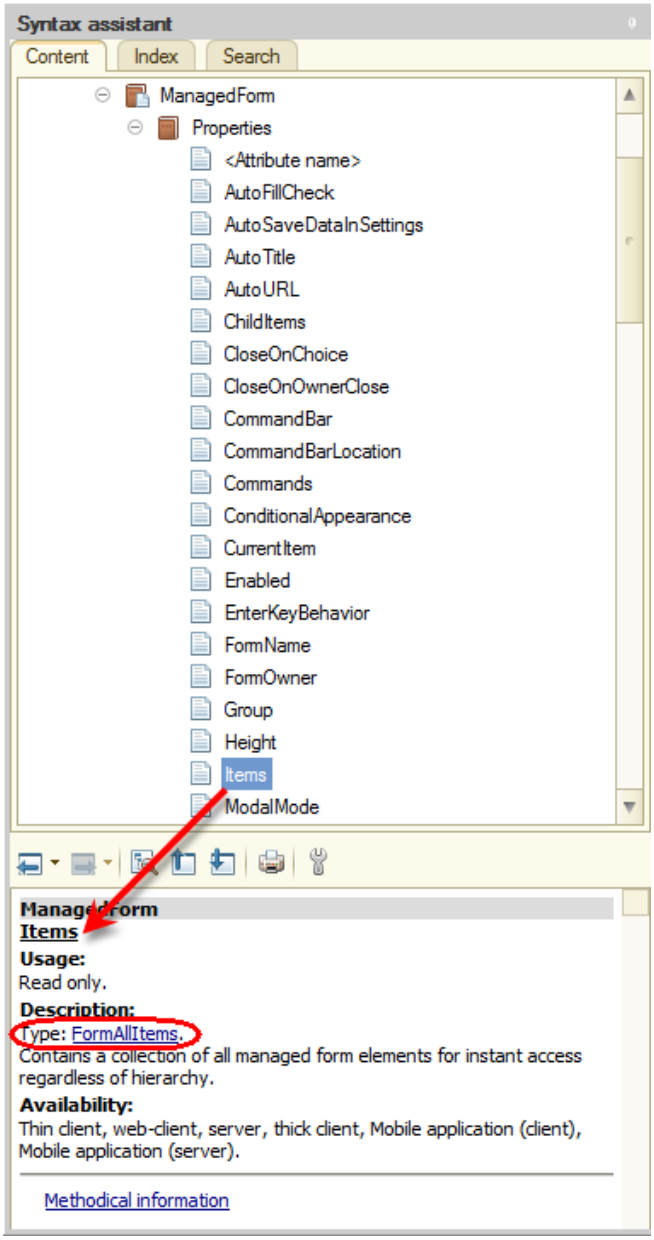


Fig. 5.25. Description of the Items property of the ManagedForm object in the Syntax Assistant

The bottom pane of the Syntax Assistant window displays the description of the selected property. This description states that the Items property contains the FormAllItems object that stores a collection of all form controls.

To learn its meaning, click the **FormAllItems** link. This opens the description of the FormAllItems collection (fig. 5.26).

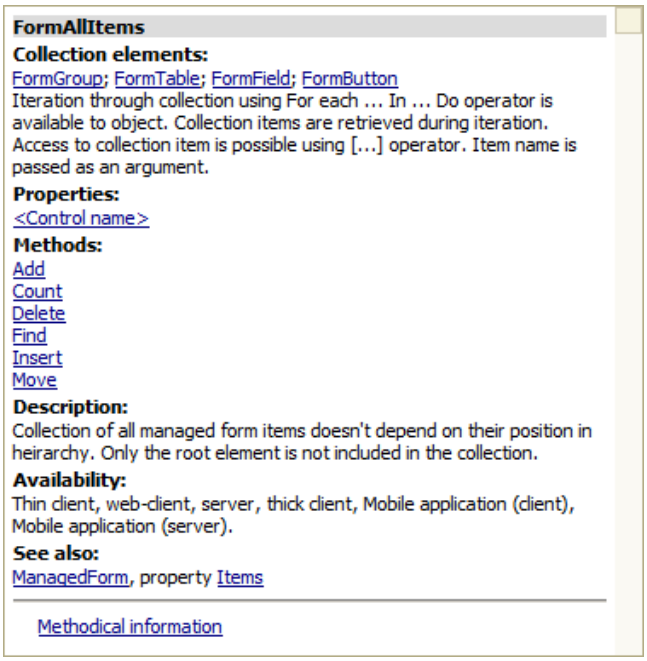


Fig. 5.26. Description of the FormAllItems collection in the Syntax Assistant

Note. The Syntax Assistant tree above is not changed. To locate the tree branch that stores the current topic, in the Syntax Assistant, click the **Find current element in the tree** button above the object description pane.

This collection contains the controls available in a managed form. You can access an item by its name.

So now you know what Items mean. The next word is separated from Items by a dot: Items.Materials. The collection has a <Control name> property, so Materials is a form control name. Let us look at the structure of form controls. Open the GoodsReceipt document form and look at the form controls pane located in the upper left part of the form editor (fig. 5.27).

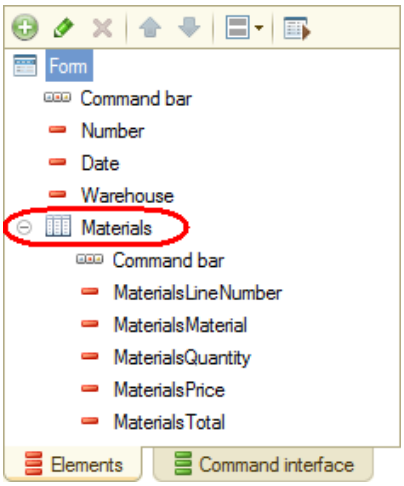


Fig. 5.27. Structure of form controls of the GoodsReceipt document

The form controls include the Materials table. If you open the property palette of this table, you can see that its title is Properties: Table (fig. 5.28).

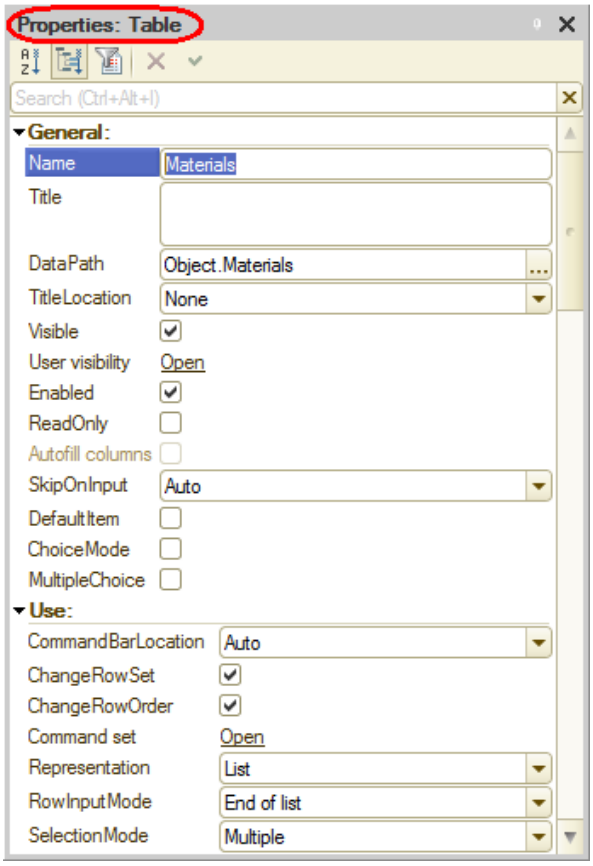


Fig. 5.28. Property palette of a table form control

It means that this form control is a table, so we need an object of the **FormTable** collection. To learn its meaning, click the FormTable link (see fig. 5.26). You will see a list of properties of the FormTable object. The properties are sorted alphabetically (fig. 5.29).

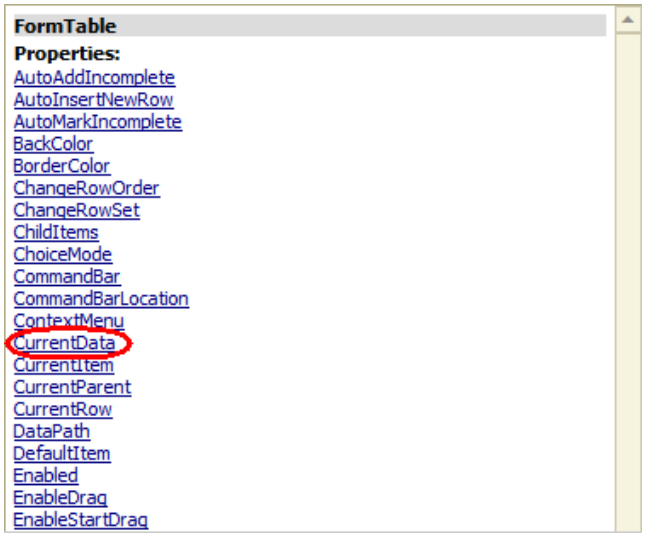


Fig. 5.29. List of properties of the FormTable object in the Syntax Assistant

So now you know what Items.Materials mean. The next word is separated from Items.Materials by a dot: Items.Materials.CurrentData. Scroll through the list of managed form table properties to find the CurrentData property. It is one of the properties of the FormTable object. To learn its meaning, double-click it (fig. 5.30).

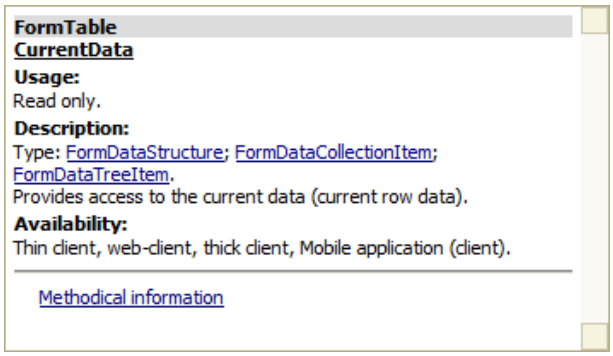


Fig. 5.30. Description of the Current Data property of the FormTable object in the Syntax Assistant

The bottom pane of the Syntax Assistant window displays the description of the selected property. This description states that the CurrentData property contains the FormDataStructure object that contains data stored in the current table row.

So once the first line of the handler (TabularSectionRow = Items.Materials.CurrentData;) is executed, the TabularSectionRow variable contains an object of the FormDataStructure type.

Now let us review the next line of the handler (listing 5.20).

Listing 5.20. MaterialsQuantityOnChange procedure (second line)

```
TabularSectionRow.Total = TabularSectionRow.Quantity
    * TabularSectionRow.Price;
```

It is reasonable to suggest that Total, Quantity, and Price are properties of the FormDataStructure object that is recorded to the TabularSectionRow variable during the execution of the first line. To learn its meaning, click the **FormDataStructure** link (see fig. 5.30, 5.31).

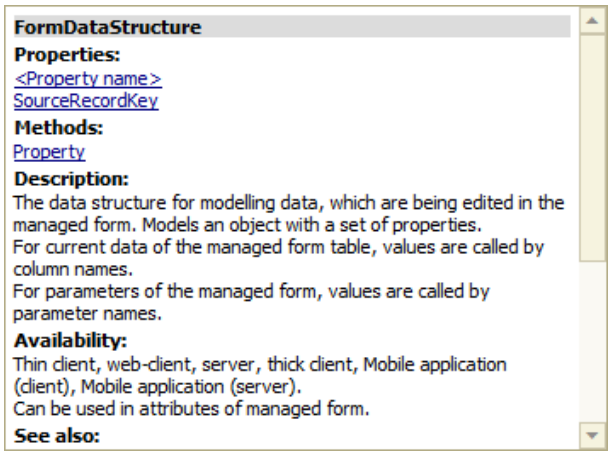


Fig. 5.31. Description of the FormDataStructure object in the Syntax Assistant

The description of this object states that you can use the FormDataStructure object to access the data of a specific tabular section column using the column name as the object property. So using the expression TabularSectionRow.Total you access the data stored in the Total column of the current table row. And the product of data in the Quantity column and data in the Price column is recorded to the Totals column.

Method #2

The second method involves using context help provided by the Syntax Assistant. You can do it by opening a module, moving the cursor to a 1C:Enterprise script clause, and pressing Ctrl+F1.

1. Open the GoodsReceipt document form, click the **Module** tab, and find the code of the MaterialsQuantityOnChange procedure.

   You already know that the TabularSectionRow expression located to the left of the assignment operator is a local variable.

   To the right of the assignment operator you can see the Items.Materials.CurrentData expression.

2. Move the cursor to the Items word and press Ctrl+F1.

   The Syntax Assistant searches all of the 1C:Enterprise script expressions for Items and sorts the results alphabetically (fig. 5.32).

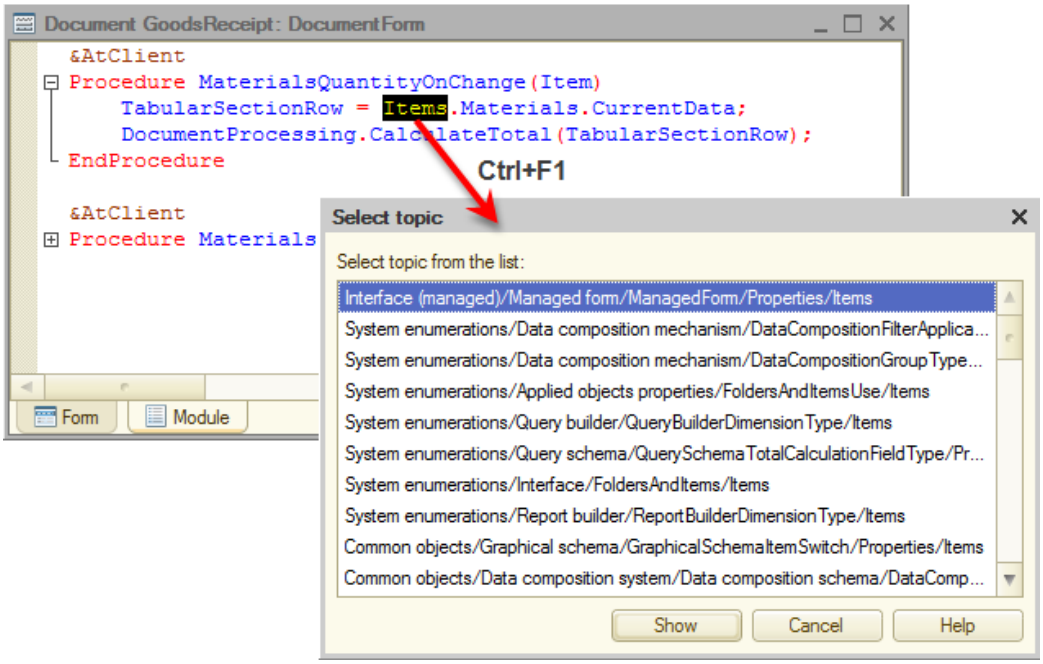


Fig. 5.32. Opening context help in the Syntax Assistant

You can see that the expression is found because a new window with the list of topics that contain the expression is displayed. Each topic includes its full path in the Syntax Assistant tree structure. Slash (/) is used as a separator between structure levels. You can move the pointer over the list of topics to find the one you need.

But the list of topics is quite lengthy so it may be difficult to search through it manually. Let us find the Items property that is used in the managed form.

3. In the list of topics, press Ctrl+F and then, in the **Find** field, enter *ManagedForm*.

4. Click **Find**.

   The selection is moved to the required topic (fig. 5.33).

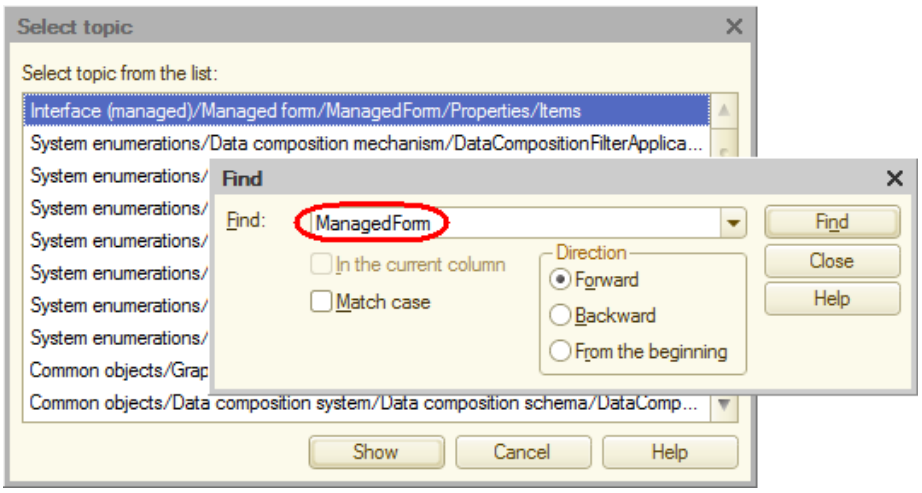


Fig. 5.33. Searching the list of topics in the context help

5. Click **Show**.

   The selected topic is opened in the bottom pane of the Syntax Assistant (fig. 5.34). Note that the Syntax Assistant tree above is not changed. You can locate the topic in the tree by clicking **the Find current element in the tree** button above the pane that displays the topic.

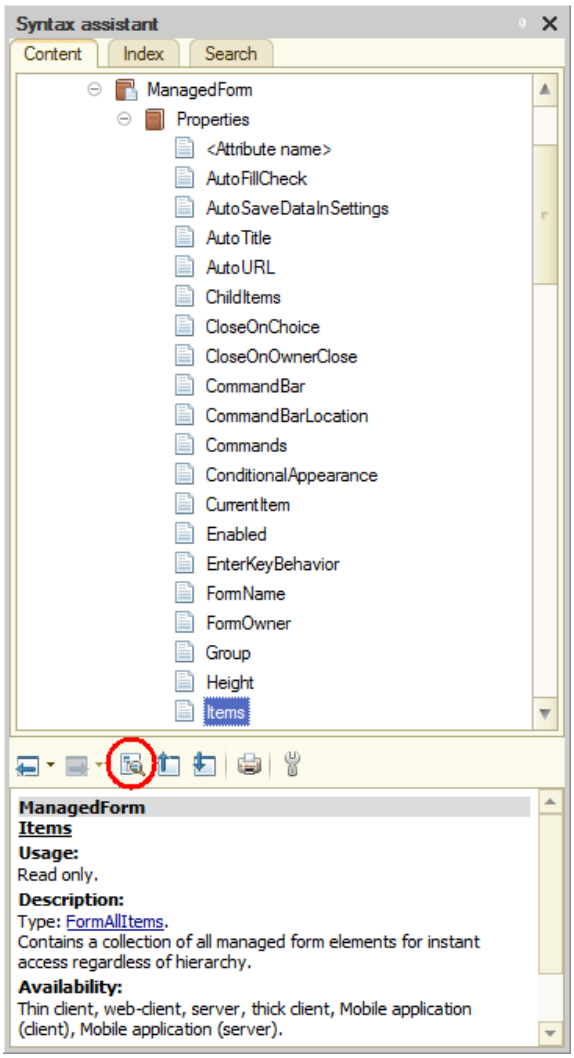


Fig. 5.34. Description of the Items property of the ManagedForm object

The **Contents** tab of the Syntax Assistant displays the topic position in the tree. So you can see that Items is a property of the ManagedForm object.

Then, when you click the links as described in method #1, you will learn what is assigned to the TabularSectionRow variable and how you can access data in its columns.

The Syntax Assistant has another handy feature. You can filter the assortment of objects included in the Syntax Assistant. Since you are currently in a client and in a form, it is reasonable to view only the 1C:Enterprise script objects that are available in the thin client and web client modes.

1. On the **Tools** menu, click **Options**.

   -OR-

   Click the **Enables you to change settings** button above the object topic in the Syntax Assistant.

2. On the **Help** tab, select or clear the check boxes that represent available modes (fig. 5.35).

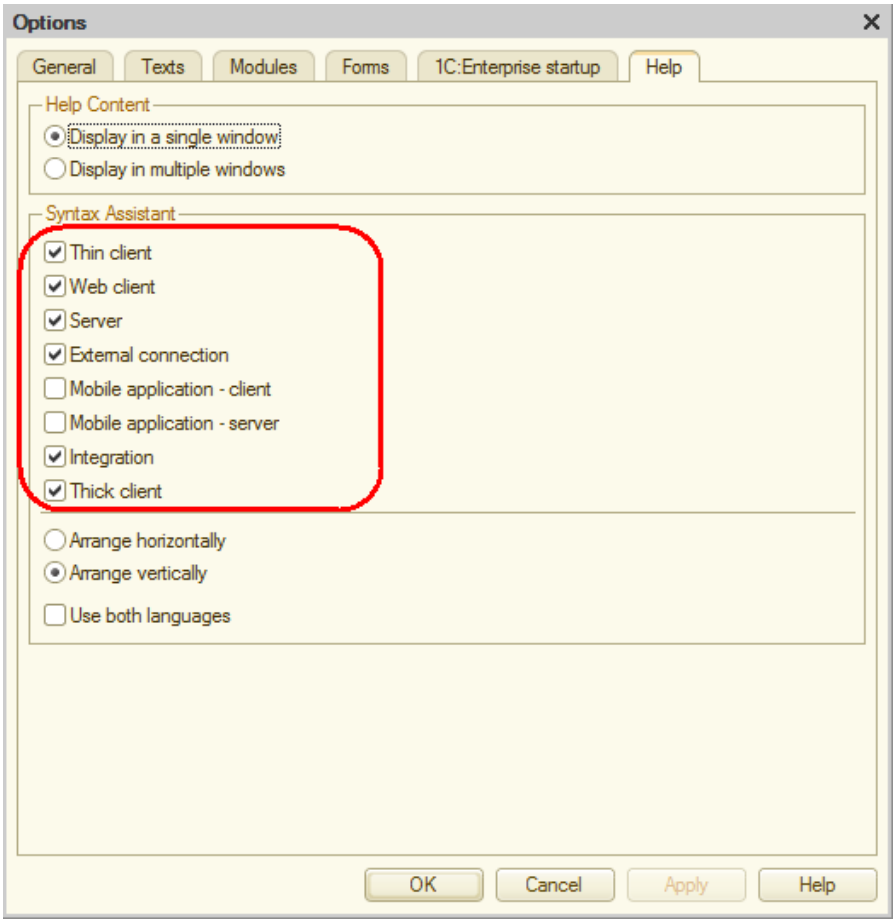


Fig. 5.35. Filtering objects displayed in the Syntax Assistant

Important! If you want to use the Syntax Assistant when you are writing a procedure that will be executed, for example, on the server, remember to select the respective run mode to have the Syntax Assistant display objects available in this mode.

Analyzing script with Debugger

The Debugger tool comes handy when you write custom script. While the Syntax Assistant requires you to understand the execution context, the object structure, and more, Debugger requires none of these.

You can simply stop at any point of your program and view the available properties or script objects.

Debugger is a tool that facilitates development and debugging of 1C:Enterprise modules. It provides the following capabilities:

- Step-by-step module execution
- Setting breakpoints
- Pausing and resuming module execution
- Debugging multiple modules simultaneously
- Calculating expressions to analyze variable states
- Viewing procedure and function call stack
- Pausing upon error
- Editing modules during the debugging

For now we will not discuss all these features in detail but will instead discuss Debugger use cases on the example of the MaterialsQuantityOnChange event handler (listing 4.1).

When you are editing module script in Designer, the commands of the **Debug** menu that set and clear breakpoints are available. The script execution is paused at lines that have breakpoints. During a pause you can analyze the current values and types of module expressions and variables and continue the execution up to the next breakpoint, and so on.

1. Open the GoodsReceipt document form, click the **Module** tab, and look at the MaterialsQuantityOnChange procedure.

   Note that the commands for breakpoint operations are now available in the **Debug** menu and on the Designer toolbar (fig. 5.36).

Fig. 5.36. Breakpoints toolbar

To set a breakpoint

- Double-click in the vertical bar to the left of the script line.

  -OR-

  Move the cursor to the module line and then, on the **Debug** menu, click **Breakpoint**.

  -OR-

  Click the **Breakpoint** button on the toolbar.

  This adds a breakpoint icon to the left of the script line.

To clear a breakpoint

- Double-click the breakpoint icon to the left of the script line.

  -OR-

  Move the cursor to the script line with the breakpoint and then, on the **Debug** menu, click **Breakpoint**.

  -OR-

  Click the **Breakpoint** button on the toolbar.

To clear all breakpoints

- On the **Debug** menu, click **Clear all breakpoints**.

  -OR-

  Click the **Clear all breakpoints** button on the toolbar.

Let us set a breakpoint.

2. Double-click in the vertical bar to the left of the first line of the MaterialsQuantityOnChange procedure (fig. 5.37).

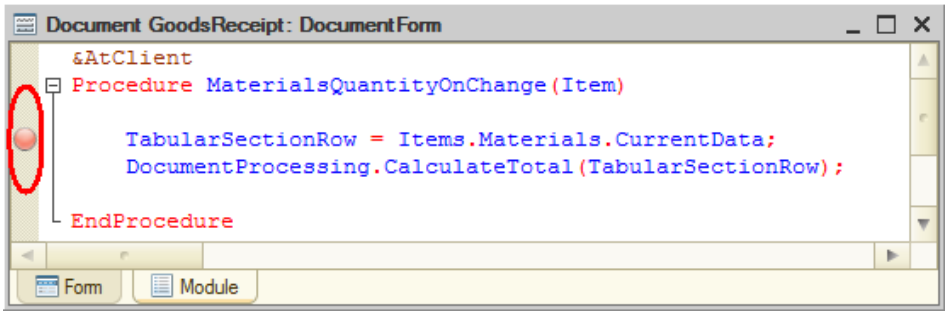


Fig. 5.37. Setting a breakpoint in the MaterialsQuantityOnChange procedure

To debug a 1C:Enterprise script algorithm, you have to run the application that executes that algorithm in the debug mode.

3. On the **Debug** menu, click **Start debugging**.

   -OR-

   Click the **Start debugging** button on the toolbar.

   Designer starts 1C:Enterprise in the debug mode. Actually this is exactly what you have done before but you have not set any breakpoints and the applied solution execution was never paused.

4. Open the list of Goods receipt documents and open any of the two documents that you created.

5. Edit the **Quantity** field in any document row.

   Once you do it, the program execution is interrupted and the MaterialsQuantityOnChange procedure is opened in Designer at the breakpoint location. An arrow over the breakpoint icon indicates the module line that is being executed (fig. 5.38).

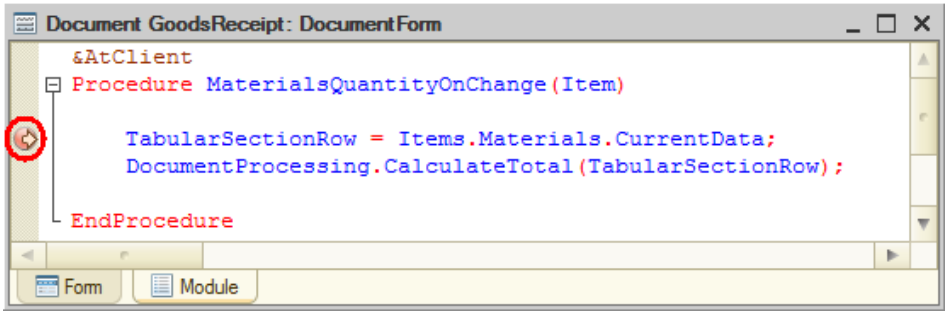


Fig. 5.38. Pausing a program at a breakpoint

Note that the commands for debugging the configuration are now available in the Debug menu and on the Designer toolbar (fig. 5.39).

Fig. 5.39. Configuration debugging toolbar

The **Step Over**, **Step In**, and **Step Out** buttons continue step-by-step script execution.

The **Continue Debugging** button continues debugging up to the next breakpoint.

The **Immediate Window** and **Evaluate expression** buttons are intended for viewing expression values at the current breakpoint.

The **Call Stack** button is intended for tracing the sequence of procedure and function calls.

But for now the program is already paused at the specified line of the MaterialsQuantityOnChange procedure. Note that this line has not been executed yet so the variable values are not assigned yet. You can use the **Step Over** button to view their values after the line is executed.

Let us look at the Items.Materials.CurrentData expression and view the content of that object at the current breakpoint.

6. Double-click the **Items** word and then click the **Evaluate expression** button (Shift+F9) on the configuration debugging toolbar.

   The *Items* value is added to the **Expression** field. The **Value** and **Type** columns display the value and type of this object. You can see that the Items object is a FormAllItems value collection, which stores all of the form controls. Expand this object branch (fig. 5.40).

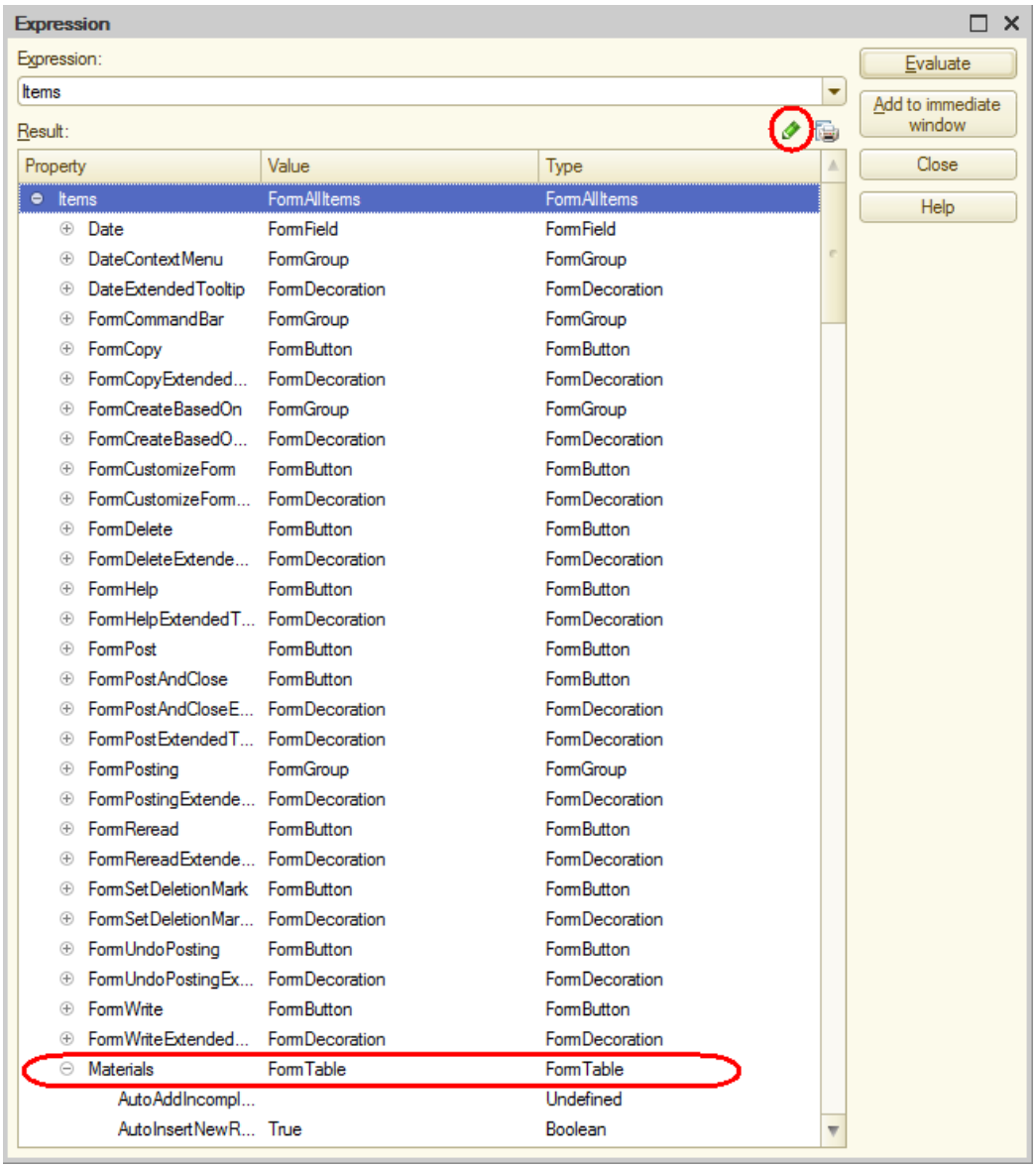


Fig. 5.40. The Items object

Above the **Result** pane you can see **the Show Values in Separate Window** button. If you click this button (or press F2), you will be able to view the collection contents.

Next, let us check the **Materials** expression.

7. Find **Materials** in the list of form controls.

   You can see that it is a FormTable object. Expand this object and review its properties. The property we are interested in is CurrentData.

8. Find the **CurrentData** branch and expand it.

   You can see the data stored in the current tabular section row, with values and types. Note that Quantity has the value that you have just set (fig. 5.41).

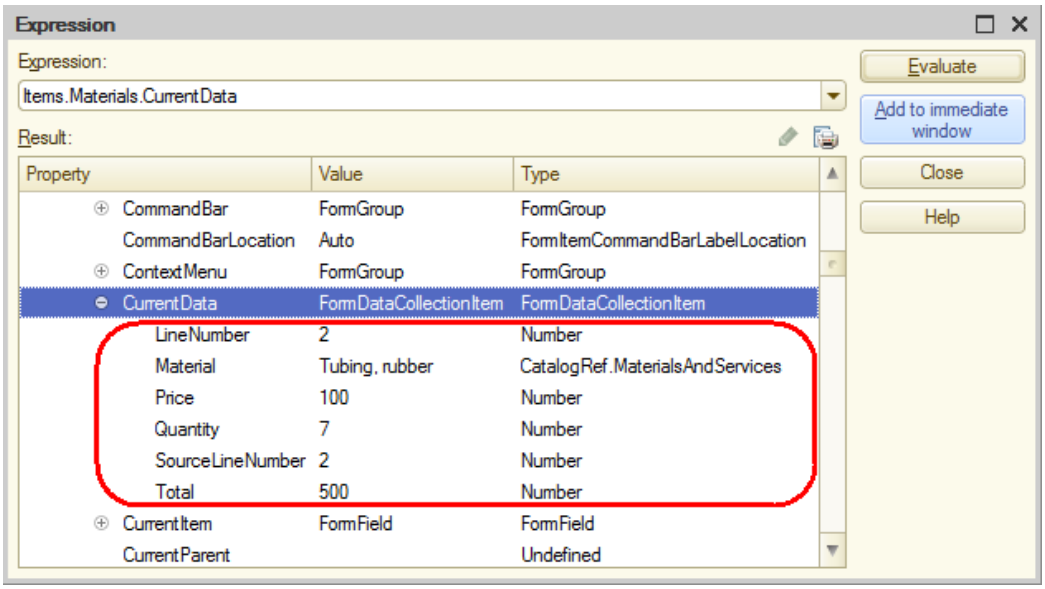


Fig. 5.41. Items.Materials.CurrentData object

9. Close the **Expression** window.

   Then let us step to the procedure of the common DocumentProcessing module where the TabularSectionRow value is calculated.

10. Click **Step In** on the configuration debugging toolbar twice. This program execution is now paused at the CalculateTotal procedure of the common DocumentProcessing module.

11. Double-click **TabularSectionRow** to select it and then click **Evaluate expression**.

12. Expand the **TabularSectionRow** object (fig. 5.42).

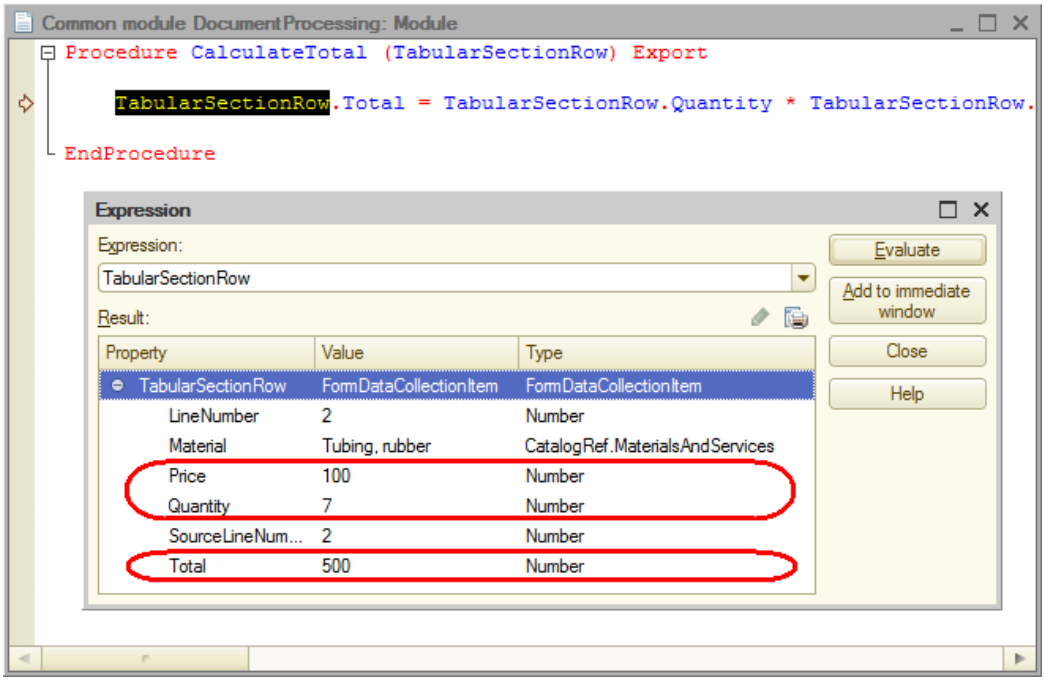


Fig. 5.42. The TabularSectionRow object

You can see that the TabularSectionRow variable contains a FormDataCollectionItem object. But the values in the Total column are not recalculated because the second line has not been executed yet.

13. Close the **Expression** window and click the **Step In** button on the configuration debugging toolbar.

    The platform executes the CalculateTotal procedure of the common DocumentProcessing module and pauses at the end of the procedure.

14. Move the pointer over the **Quantity** or **Total** word to view a tooltip with its current value (fig. 5.43).

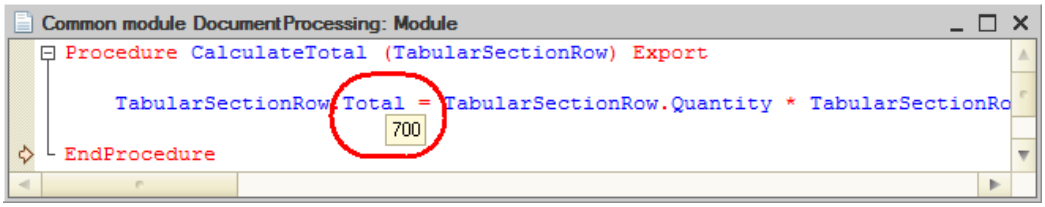


Fig. 5.43. The TabularSectionRow object

You can see that the values in the Total column are recalculated, so the Items.Materials.CurrentData object now contains new values.

After you analyze the variables and expressions, you can continue with debugging.

To continue debugging

- Click the **Continue Debugging** button.

To stop debugging

- On the **Debug** menu, click **Stop**.

Note that you can set a breakpoint inside a loop and see how values are changed during each iteration.

You can edit the configuration and save the changes during the debugging.

Important! Although you can edit a module that is being debugged, Debugger does not compile the modified script, and debugging is continued for the database configuration script (the script that was available at the moment when Debugger was started or when the connection was opened). To debug the changes that you made, you have to update the database configuration first.

Finally, let us discuss some handy debugging techniques.

When you are in a form module and you want to write a handler, you can use the ThisObject property to view the context properties of the current form along with its available extensions and other available items.

To view ThisObject property

1. After you stop at a breakpoint, click the **Evaluate expression** button (Shift+F9) on the configuration debugging toolbar.
2. In the **Expression** field, enter *ThisObject* and click **Evaluate** (fig. 5.44).

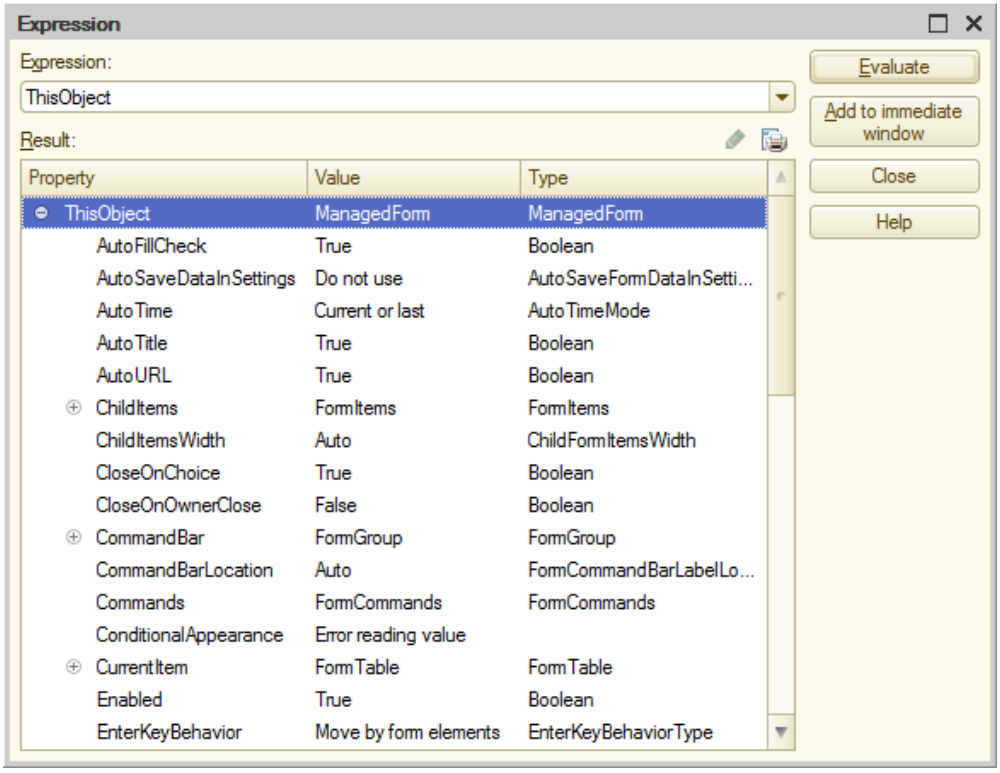


Fig. 5.44. ThisObject property of a managed form

If you expand this item, you will see the type and properties of the 1C:Enterprise script objects available at the current breakpoint.

And when you are in an object or record set module, you can use ThisObject property in a similar manner to view the properties of the object or record set module context.

Objects, objects, objects

What exactly is an object in 1C? This question can puzzle not just novice developers, but also people with a fair amount of experience in 1C:Enterprise development.

The hard part is to make sure that you clearly understand the context where the term is used.

As a rule, the "object" term is used in one of the three contexts:

- Configuration
- Database
- 1C:Enterprise script

When speaking about the configuration, we use the "configuration object" term to describe a certain set of data definitions and data processing algorithms. For example, a configuration can contain a Catalog object named Employees.

For each configuration object, a data structure is created in the database.

So speaking about a database, what we mean by "object" is simply an item in that data structure. A reference to that item (that data set) is always available, and it can be stored in a field of another data structure.

For example, a database contains the Employees catalog, which includes an employee named Johnson. The catalog item that contains information about Johnson is a database object. And if you have a GoodsReceipt document that includes the PersonResponsible attribute, the value type for that attribute is a reference to a database object, i.e. the catalog item containing Johnson's information.

However, if we start talking about 1C:Enterprise script and the catalog management tools that it provides, "object" means the data type that is used for accessing catalog data and has a set of properties and methods.

There are quite a number of 1C:Enterprise script objects intended for catalog management: CatalogsManager, CatalogManager.<name>, CatalogRef.<name>, and more. Among these there is an object that provides access to catalog objects in the database: CatalogObject.<name> (fig. 5.45).

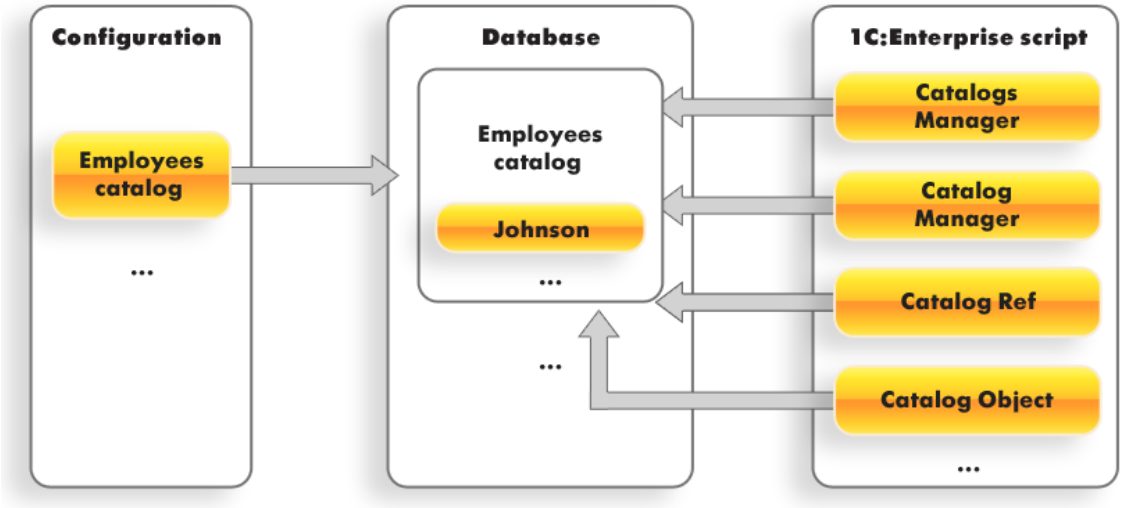


Fig. 5.45. Configuration object, database object, and 1C:Enterprise script objects

Server and clients

In the previous lesson (page 137) you created a single procedure for handling multiple events and stored it to the common module named DocumentProcessing. You selected the **Client** and **Server** check boxes in the properties of this common module. Now let us discuss why 1C:Enterprise has clients and servers.

1C:Enterprise supports two operation modes: file mode and client/server mode.

The file mode is intended for a single user or a small number of users in a local network. In this mode all the infobase data (configuration, database, and administrative data) is located in a single file (fig. 5.46).

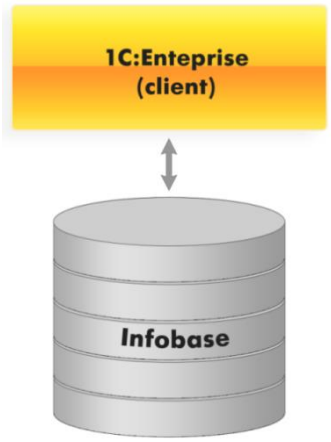


Fig. 5.46. File mode

The main purpose of the file mode is quick and easy installation and start. You might need it to review something or work on a specific task at home on a laptop. The file mode supports performing actual accounting operations but it does not offer the same level of scalability and data security options as offered by the client/server version. So if you handle accounting on your own or you have a small team, you do not need data protection from unauthorized use by other employees, and you have a relatively small volume of data, you can certainly work in the file mode. In other cases we recommend that you use the client/server mode.

The client/server mode is intended for use in workgroups or in the entire enterprise. This mode is based on a three-tier client/server architecture (fig. 5.47).

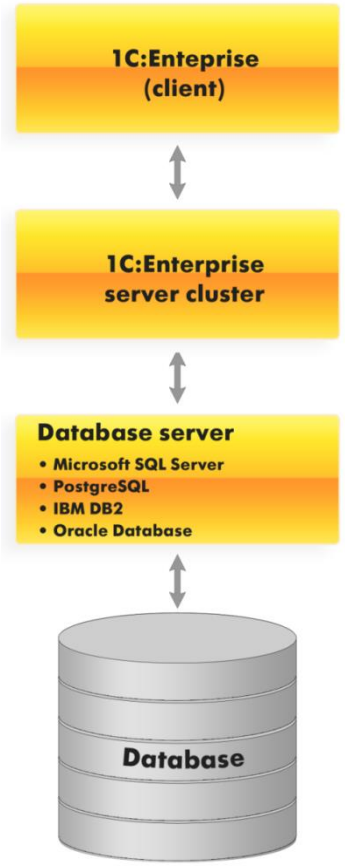


Fig. 5.47. Three-tier 1C:Enterprise architecture

The client/server mode is the primary mode in a multiuser environment with a large volume of data. It provides a complete range of features for scalability, administration, and data security. But it also requires significant installation and administration efforts.

Note that in physical sense the client and server parts of 1C:Enterprise can be located either on different computers or on a single one. The most important part is that a user does not have direct access to the database server, which ensures data security. And in the file mode the database is located in a shared network location available to all users.

1C:Enterprise is primarily intended for running in the client/server mode. And while right now you are developing your training configuration in the file mode, it will also work in the client/server mode without any extra efforts.

All applied solutions are capable of working in both modes and this does not require additional configuration modifications.

This is possible because a configuration is always developed for the client/server architecture. In 1C:Enterprise you simply cannot develop a configuration that cannot work in a client/server mode. And in the file mode the system emulates a server on a client computer.

The client/server architecture divides the entire system into three parts that interact with each other: client, 1C:Enterprise server, and database server.

A client application is a program that is a part of 1C:Enterprise. Its main purpose is providing the user interface and displaying data that users can modify. Besides, a client application can execute 1C:Enterprise script algorithms (custom developer-created algorithms) but it can handle a very limited range of 1C:Enterprise script types. This approach makes client applications very lean and not resource-consuming, they are capable of browsing the Internet and even running in a web browser environment.

Client applications interact with 1C:Enterprise server. 1C:Enterprise server is also a program and a part of 1C:Enterprise.

One of the main goals of this program is transferring requests from a client application to the database server and returning the results to the client.

Another server goal is execution of the vast majority of 1C:Enterprise script algorithms, preparation of data for displaying it in forms, reports, and so on. Hence, all the complicated calculations that involve direct operations with data are executed on the server. Note that almost the entire range of 1C:Enterprise script types is available on the server (interface-specific types are a natural exception because the server has no interface part as it does not interact with users, it only exchanges data with other programs: the client applications and the database server).

A database server is a program as well. But this program is not a part of 1C:Enterprise, it is a third-party software. Its major goal is arrangement and maintenance of databases where a database is a structured and organized data set that describes characteristics of some physical or virtual systems.

Currently 1C:Enterprise supports the following database servers:

- Microsoft SQL Server
- PostgreSQL

- IBM DB2
- Oracle Database

Common module compilation

In the previous lesson (page 137) you created a single procedure for handling multiple events and stored it to the common DocumentProcessing module. This module, just like any other common module, has the following properties: Client (managed application), Server, and External connection. The values of these properties (True or False) define where the instances of these modules are compiled.

Let us discuss what changes when you set or clear these common module properties.

First you need to learn why compilation is needed. Everything that you have developed and coded in the configuration by now is nothing more than a "workpiece". When you start the platform in 1C:Enterprise mode, all this is transformed into a program capable of being executed on a computer. This process is referred to as compilation. Note that as we have mentioned above, script can be executed in different parts of the system: on the server and in client applications. So, for common modules you have to explicitly specify the side where they are compiled: on the server or on the client.

In the common DocumentProcessing module you set the Client (managed application) property to True by selecting its check box (see fig. 4.24). It means that instances of this module are only compiled on the client side in the thin client context and in the web client context (fig. 5.48).

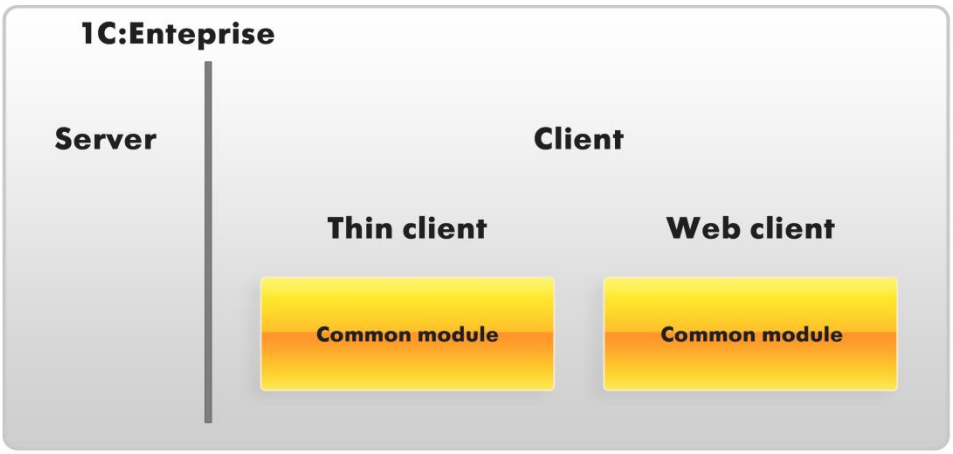


Fig. 5.48. Common module compilation on the server and on the client

If a module only has the Server property set to True, the module is only compiled at the server side.

The platform has two client applications available: the thin client and the web client.

The thin client is the 1cv8c.exe file. This file is executed when you start debugging.

The web client does not have an executable file because it is not executed in the operating system environment, the web browser environment is used

instead. A user should only start their web browser and enter the address of the web server where the infobase is published, and the web client is downloaded to the computer and started.

Compilation directives

In the previous lesson (page 137) you created handlers for OnChange events of some form controls. The generated declarations of the handler procedures were preceded by &AtClient directives. Here we will explain their meaning.

A form exists simultaneously on the server and on the client. So for each procedure in the form module you have to explicitly specify its execution context, in other words, where it is executed: on the server or on the client. Availability of the &AtClient, &AtServer, and &AtServerNoContext directives reflects the fact that when 1C:Enterprise script is used in a form module, the client script must be clearly separated from the server script. So when you specify one of the directives, you specify a client or a server part of the program.

You can use procedures with different execution directives within a single form module and you can pass script execution from the client to the server. You can also call a common module procedure from a client procedure (which is exactly what you did earlier in this tutorial), and that common module procedure can be executed either on the client or on the server, depending on the values of the Client and Server module properties.

Script execution on the client and on the server

In the previous lesson (page 137) you created a single procedure for handling multiple events and stored it to the common DocumentProcessing module. In particular, you set the **Client (managed application)** check box and cleared the **Server** check box in the module properties. The procedure is called from the form module where the event handler also has a directive that specifies the execution context for this handler: &AtClient. Now we will explain how script is executed in this scenario.

When an application is started, script execution is always started on the client. Then the script execution can be passed to the server by calling a procedure of a common module compiled on the server.

When a procedure or function is called, the platform searches for it on a client first. If the compiled client context does not contain this procedure, the search continues on the server. If the procedure is found, the script execution is passed to the server. When the procedure is execution is completed, the script execution continues on the client.

For example, when you change a price in a goods receipt form, the PriceOnChange() procedure of the form module is called. It is executed on the client in the thin client mode. The second line of this handler calls the CalculateTotal() procedure. This procedure is located in the common DocumentProcessing module, which is also compiled on the client. It is

executed in the thin client mode. Once the procedure is executed, the execution is passed back to the form module (fig. 5.49).

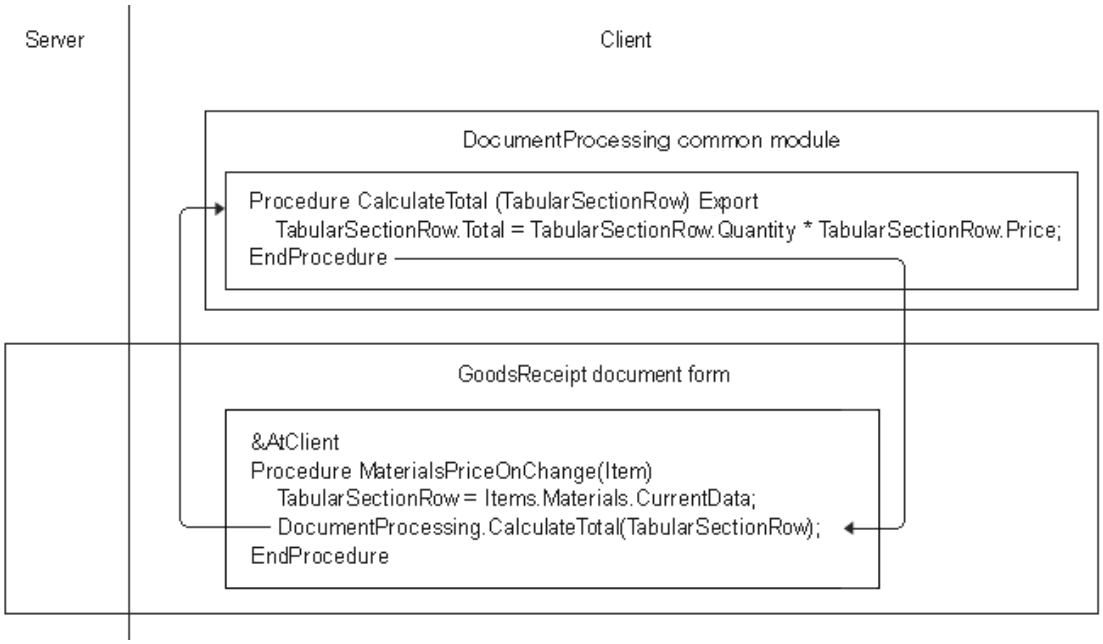


Fig. 5.49. Passing script execution between client modules

Lesson 6 (0:50). Accumulation registers

In this lesson you will study the Accumulation register configuration object. You will learn the object purpose, its structure, and its main properties.

Then you will create an accumulation register in your configuration. The register will store the data changes introduced by the documents that you created earlier.

Accumulation register purpose

You have come to one of the key points in configuration development: writing the functionality for storing accumulated data.

It seems that you have already created everything you need. You know what is received and expended (catalogs), and you have a means of receiving and expending (documents). Adding several reports would complete the automation of Jack of All Trades.

However, this is not the case.

First of all, you can certainly obtain all the required output data by analyzing documents. But imagine that tomorrow Jack of All Trades decides to slightly modify its business processes and you will need to introduce a new document (or multiple documents!) to your configuration.

For instance, currently Jack of All Trades receives goods and then expends them. Then the management wants to strengthen its resource management and decided to receive goods at the company's main warehouse and then issue them to accountable officers. In this scenario you need to add another document to the configuration to reflect the movement of resources between the main warehouse and the accountable officers. Obviously, that will force you to revise all the reports that you have created up to that point in order to process the changes introduced in the new document. And imagine what that would mean if your configuration had twenty documents instead of two!

Second, reports that rely on the document analysis operate pretty slowly, which would irritate users and dissatisfy management.

To avoid these issues, 1C:Enterprise features configuration objects intended for creating database structures that accumulate data in a format that facilitates subsequent analysis. Using these data storages lets you accumulate data provided by various documents (or other database objects), as well as easily create reports based on this data, or use this data in the configuration algorithms (fig. 6.1).

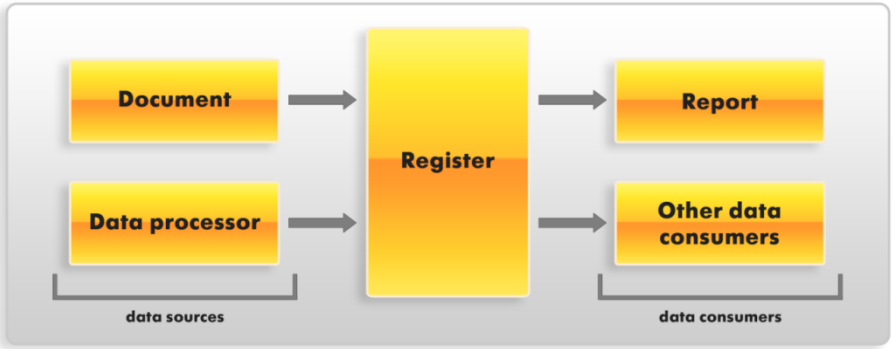


Fig. 6.1. Generic configuration operation algorithm

The platform features several objects that are referred to as registers. These objects are intended for describing such storages. Let us discuss one of them.

Understanding accumulation registers

The Accumulation register configuration object is intended to describe the data accumulation structure. The platform uses the Accumulation register configuration object as the basis for creation of database tables that accumulate data provided by various database objects.

This data is stored in the tables as individual records. All of the records have identical structure, which is specified in Designer (fig. 6.2).

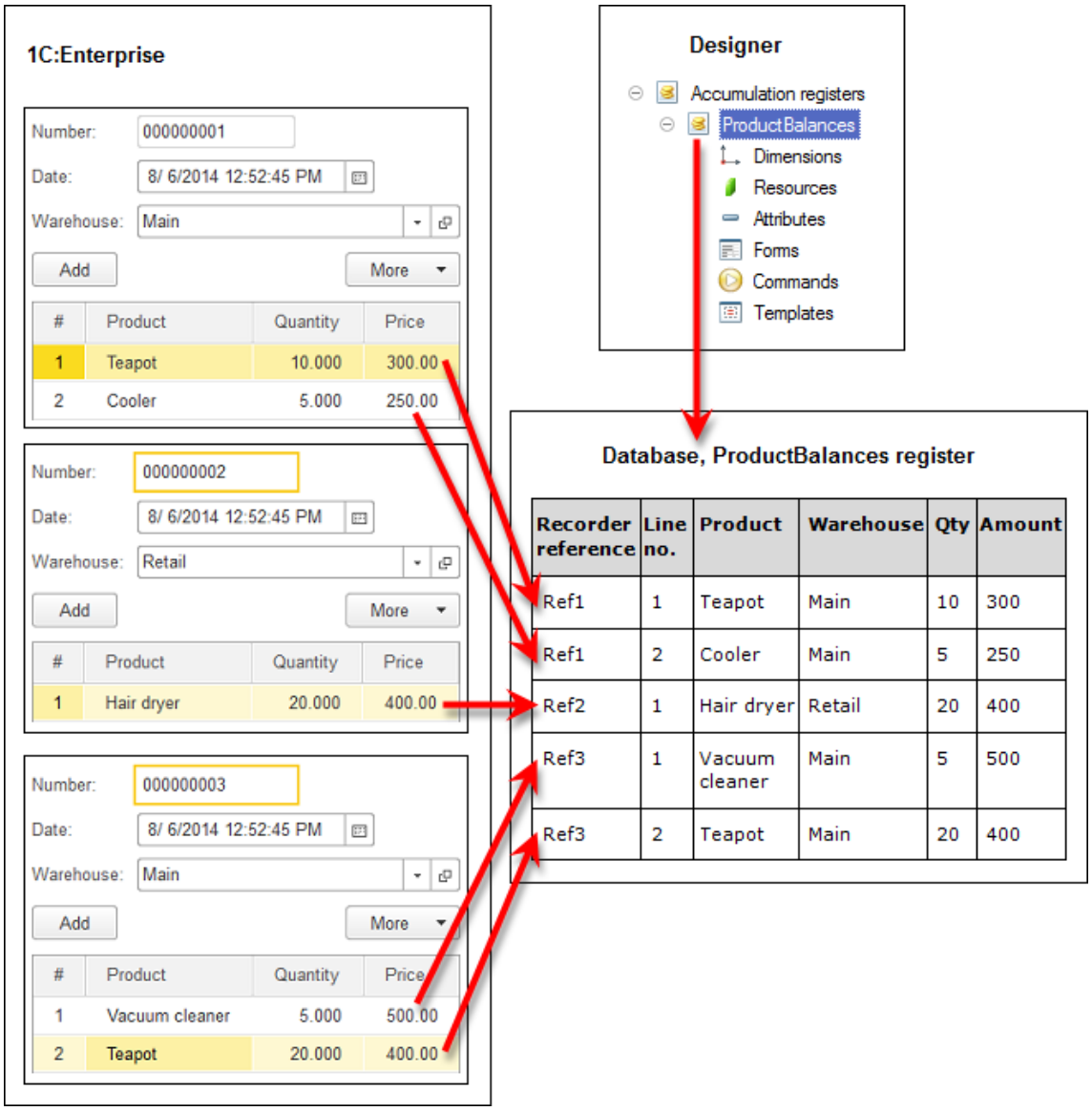


Fig. 6.2. The Product balances accumulation register in Designer and in the database

Based on the accumulation register table of register records, the platform calculates the table of register totals that stores the totals for the time of the last register record in the database (the current totals).

The accumulation register distinctive feature is that it is not designed to be edited directly by users.

The developer might allow users to edit the accumulation register if required. However, the purpose of the accumulation register is to be modified based on modifications of other database objects, rather than through direct user intervention.

The main accumulation register purpose is accumulating numeric data in multiple dimensions, which are declared by the developer in the Accumulation register object. Dimensions are configuration objects that are subordinate to the accumulation register.

The types of numeric data stored in an accumulation register are referred to as resources. Resources are also configuration objects that are subordinate to the Accumulation register object, they are declared in Designer.

For example, an accumulation register can accumulate data on the quantity and total number of products in the warehouses. In this scenario it will have the Product and Warehouse dimensions and the Quantity and Total resources (see fig. 6.2).

Changes to the accumulation register generally occur when a document is posted, usually this adds a certain number of records. Each record contains the dimension values, incremented values for the resources, a link to the document that caused the change (the recorder) and the "direction" of the increment (receipt or expense). This set of records is referred to as register records of an accumulation register. A recorder is an infobase object (normally a document) that causes a register record. Each register record has a recorder.

An accumulation register can also contain additional data that describes each register record. The structure of this additional data is specified by the developer using the attributes of the Accumulation register configuration object.

Learn more! For details on the structure of 1C:Enterprise script objects intended for accumulation register operations, see section "Quick developer reference. Accumulation registers" on page 863.

Adding an accumulation register

Now that you know the purpose of accumulation registers, let us discuss how to use them in our example.

We are mostly concerned with the quantity and type of materials available in the warehouses. To accumulate this information, let us create the BalanceOfMaterials register.

In Designer mode

Let us create the accumulation register.

1. Open your configuration in Designer.
2. In the configuration object tree, click the **Accumulation registers** branch and then click the **Add** button in the configuration window command bar.

   This opens the configuration object editor window.
3. On the **Main** tab, in the **Name** field, type *BalanceOfMaterials*.
4. In the **Extended list presentation** field, type *Register records for Balance of materials*.

   It will be displayed in the title of the register records list window.
5. Click **Next**.

   This opens the **Subsystems** tab. The logic of your configuration suggests that the register should be available in the Inventory, Services, and Accounting sections.
6. Select the **Accounting**, **Inventory**, and **Services** check boxes (fig. 6.3).

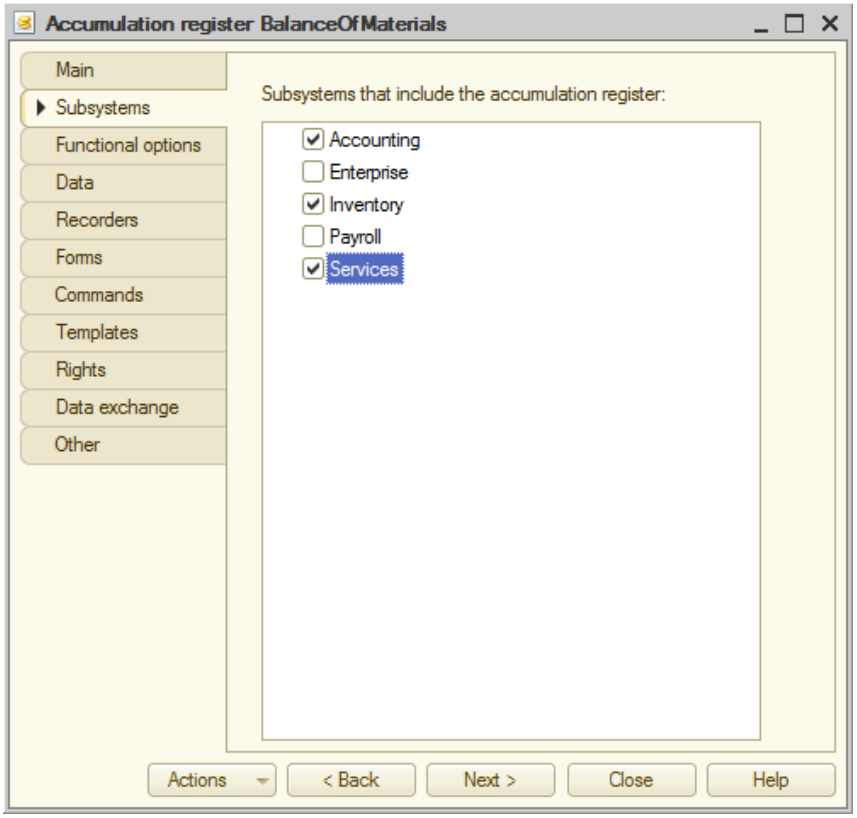


Fig. 6.3. Specifying subsystems where a register is available

7. Click the **Data** tab.

   Here you can define the register structure.

8. Click the **Dimensions** branch and then click the **Add** button in the command bar of the window (fig. 6.4) to add the following register dimensions:

   - **Material**. Type: CatalogRef.MaterialsAndServices
   - **Warehouse**. Type: CatalogRef.Warehouses

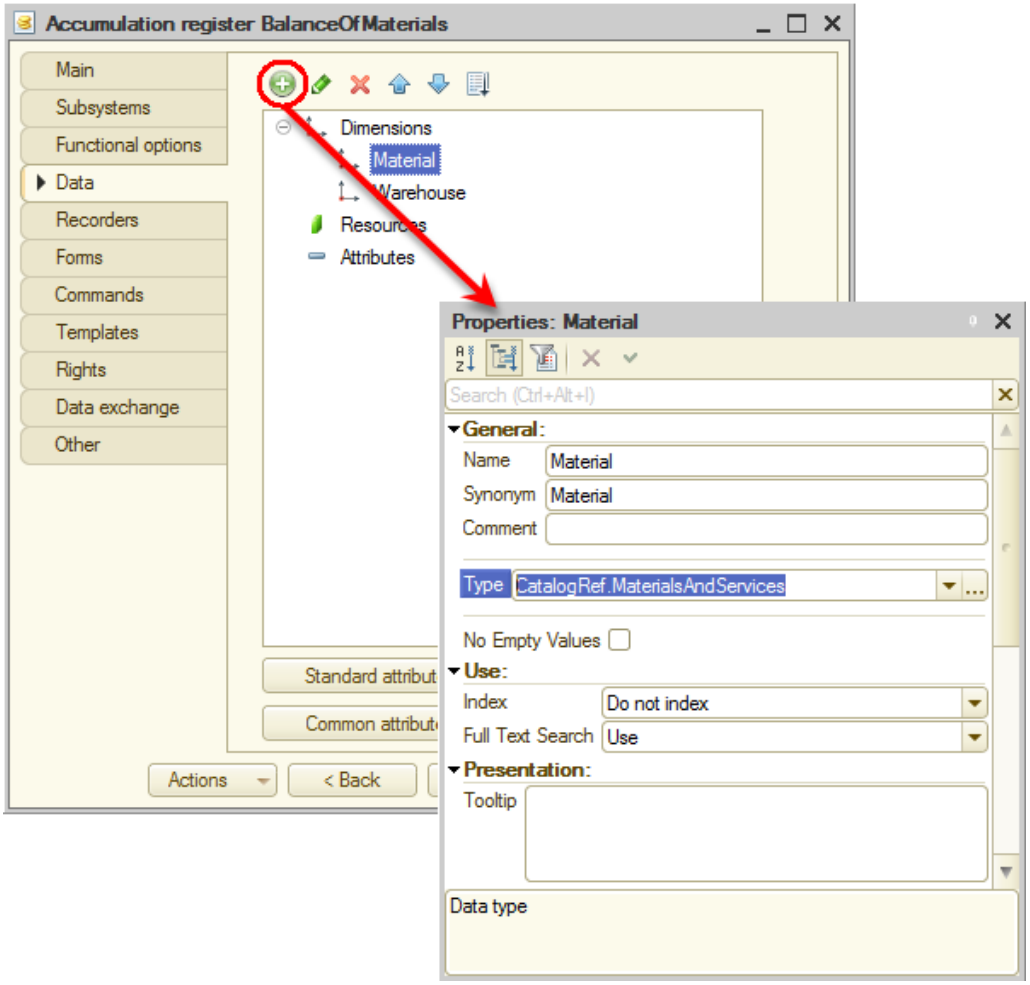


Fig. 6.4. Creating register dimensions

9. Click the **Resources** branch and click the **Add** button in the command bar of the window (fig. 6.5) to create the following resource:
   - **Quantity**. Length: 15, precision: 3

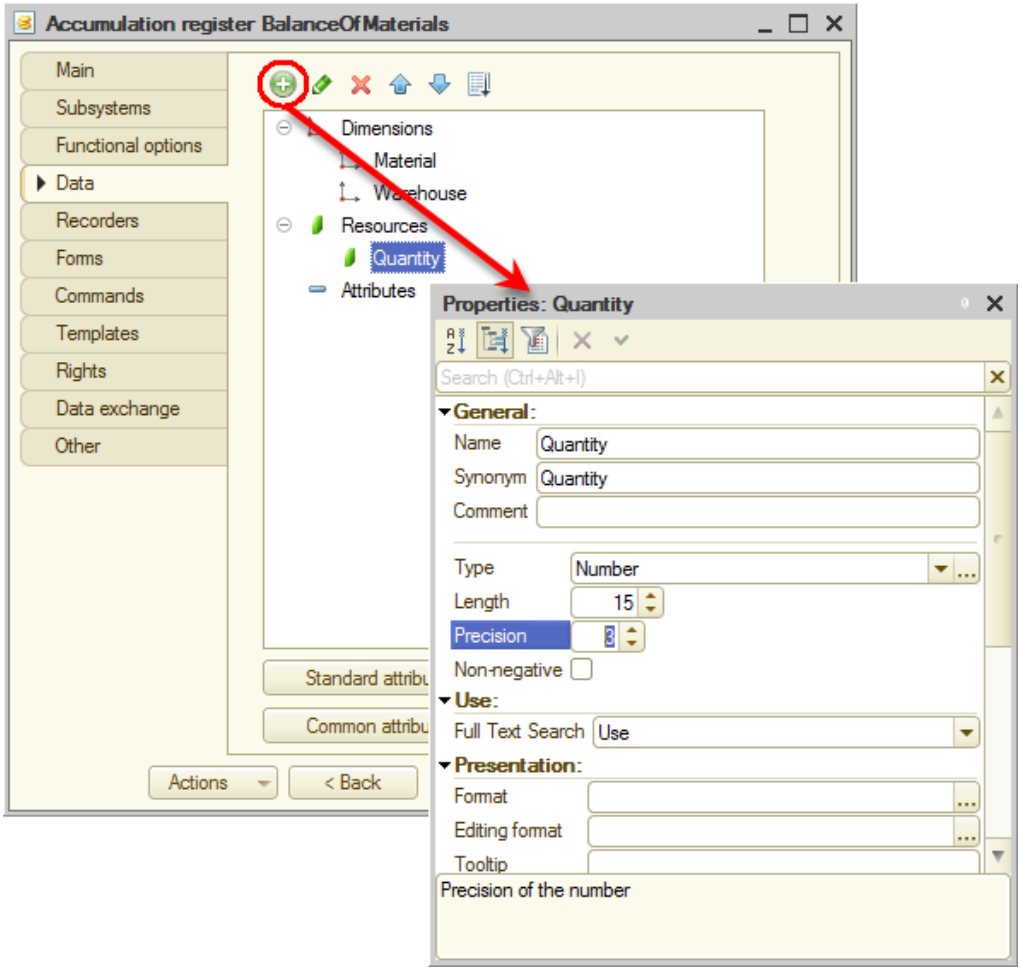


Fig. 6.5. Creating register resources

Once you are done with these steps, the BalanceOfMaterials register should look as shown in fig. 6.6.

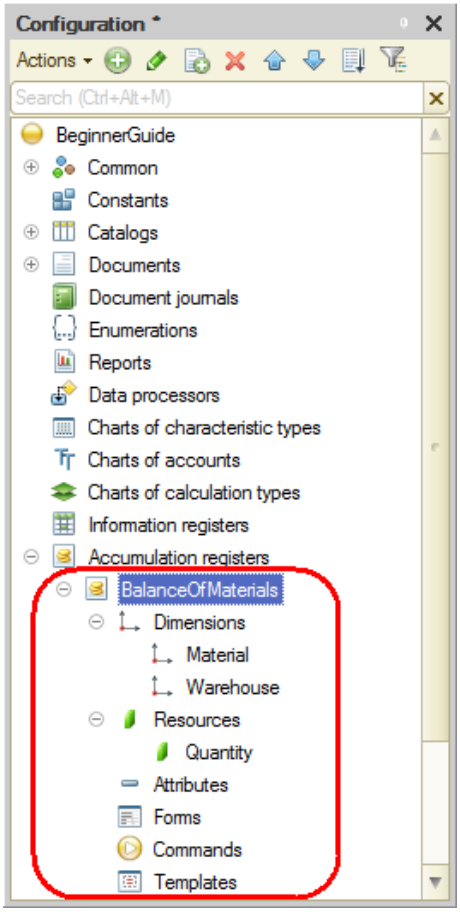


Fig. 6.6. BalanceOfMaterials register

At this point, if you attempt to start 1C:Enterprise in the debug mode, you get the error message: "AccumulationRegister.BalanceOfMaterials: None of the documents is a recorder for the register." This message confirms the fact that the accumulation register purpose is accumulating data provided by documents.

So let us set up generation of register records in the BalanceOfMaterials accumulation register during the posting of the two documents that you created: GoodsReceipt and Services.

Creating register records of a document

A register record of a document is a record generated in a register when a document is posted. It reflects the changes introduced by that document.

1. Open the configuration object editor for the GoodsReceipt document.
2. Click the **Posting** tab, expand the **Accumulation registers** list, and select the **BalanceOfMaterials** check box (fig. 6.7)

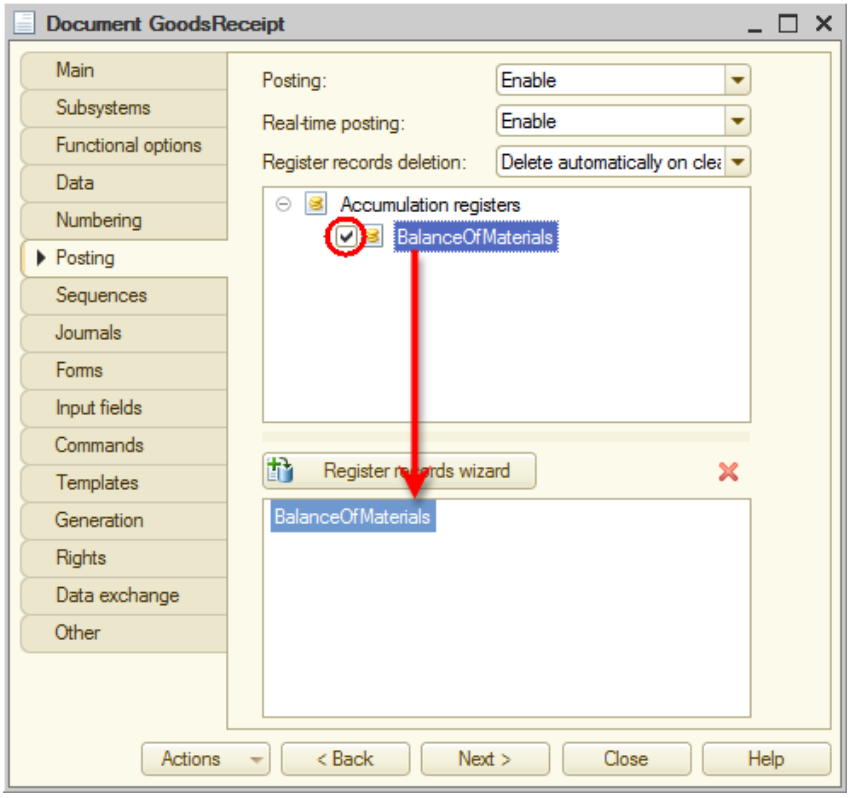


Fig. 6.7. Creating register records of the GoodsReceipt document in the BalanceOfMaterials register

Note that once you select the check box, the **Register records wizard** button becomes available.

3. Click the **Register records wizard** button.

   The wizard is straightforward. The **Registers** list displays the registers where the document can create records. In this case, the only register available is BalanceOfMaterials.

   The **Document attributes** list contains the data used to generate register records, that is, the attributes of the GoodsReceipt document.

   The table with **Field** and **Expression** columns stores the formulas used to calculate register dimension and resource values when a register record is generated (fig. 6.8).

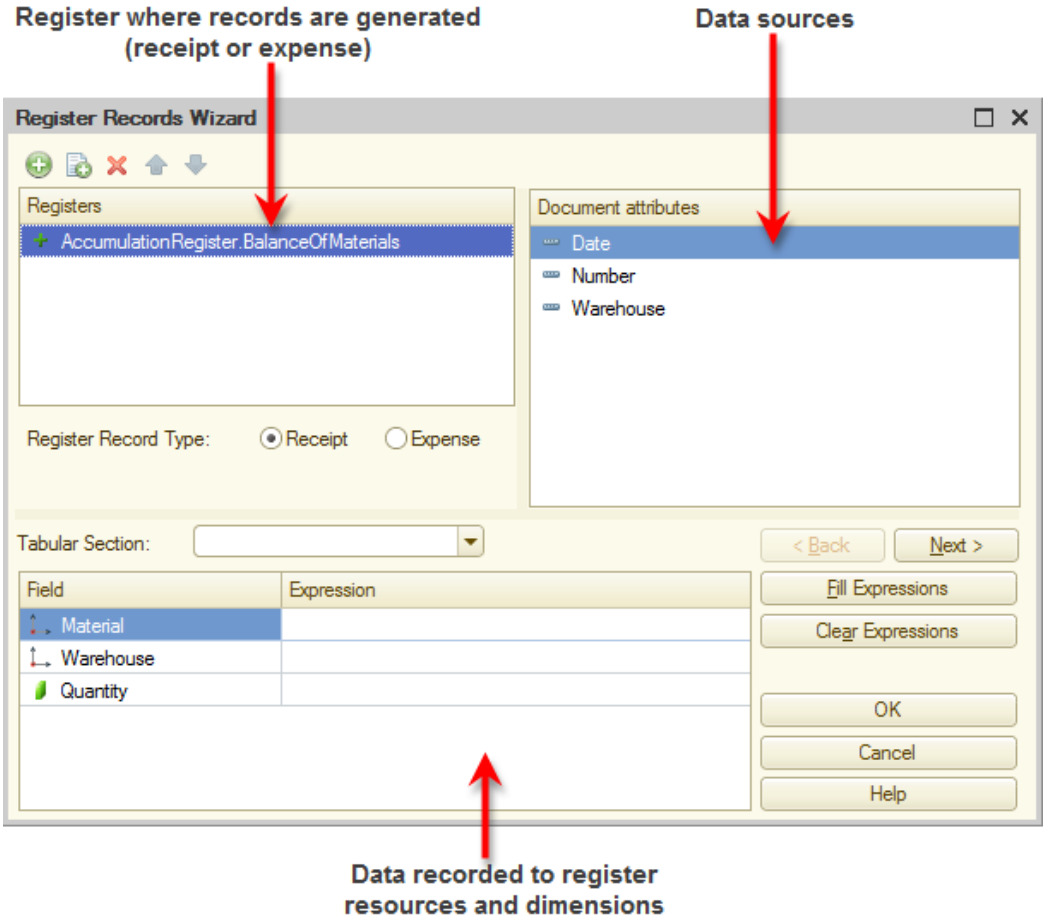


Fig. 6.8. Register records wizard

Note that by default the wizard prompts you to create a receipt register record for the BalanceOfMaterials register (**Register Record Type** is set to **Receipt**, and the + icon is displayed next to the register name). This is fine with us since the document is GoodsReceipt and is intended to record received materials.

4. In the **Tabular Section** list, select **Materials**.

   It is the only tabular section in the document. Note that its attributes are added to the **Document attributes** list.

5. Click **Fill Expressions**.

   In the bottom pane the mapping between the register fields (dimensions and resources) and the expressions for calculating their values is generated (fig. 6.9).

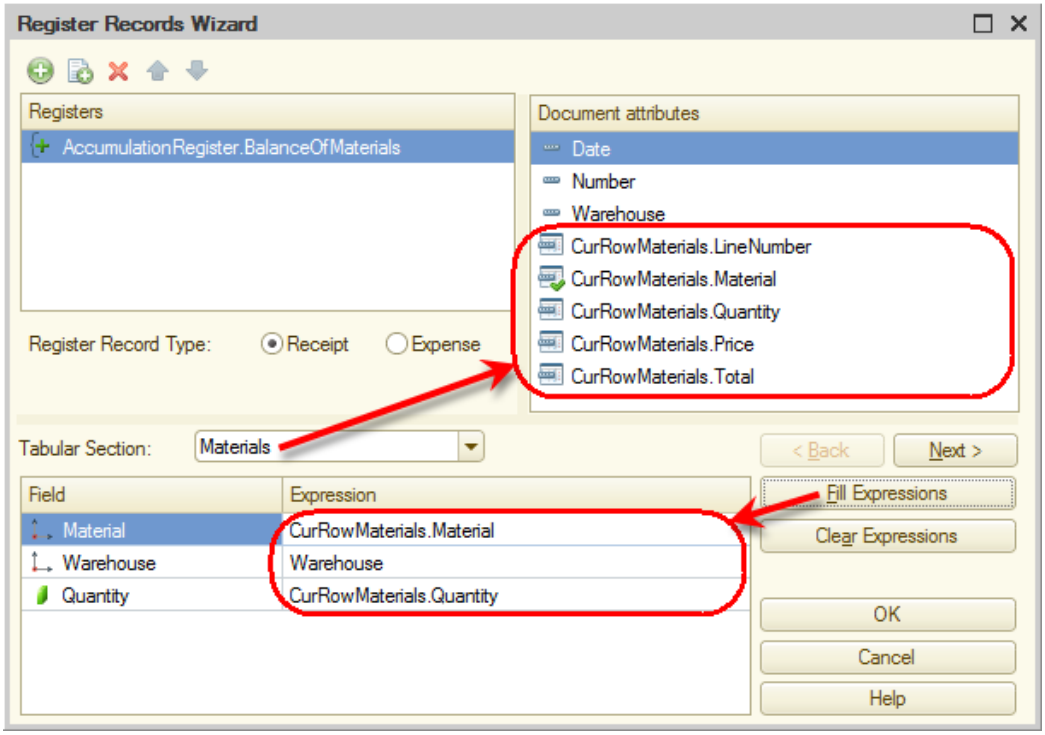


Fig. 6.9. Selecting a document tabular section and filling expressions for calculating register records

As you can see, the register records wizard defined the mapping appropriately: it took material from the document tabular section for material, warehouse specified in the document header for warehouse, and quantity from the document tabular section for quantity.

6. Click **OK**.

The wizard generates a procedure in the GoodsReceipt document module and opens the module (listing 6.1).

Listing 6.1. Posting() procedure

```
Procedure Posting(Cancel, Mode)
    //{{__REGISTER_REGISTERRECORDS_WIZARD
    // This fragment was built by the wizard.
    // Warning! All manually made changes will be lost next time you
use the wizard.

    // register BalanceOfMaterials Receipt
    RegisterRecords.BalanceOfMaterials.Write = True;
    For Each CurRowMaterials In Materials Do
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Receipt;
        Record.Period = Date;
        Record.Material = CurRowMaterials.Material;
        Record.Warehouse = Warehouse;
        Record.Quantity = CurRowMaterials.Quantity;
    EndDo;
```

```
    //}}__REGISTER_REGISTERRECORDS_WIZARD
EndProcedure
```

It is the Posting event handler for the GoodsReceipt document configuration object.

The Posting event is one of the most important document events. This event occurs when a document is posted. The main purpose of this event handler is generating register records for the document. The data operations in the handler procedure affect the accounting records. So this is the place where developers implement custom algorithms that are executed during document posting.

Now lets us study the handler procedure script.

The DocumentObject 1C:Enterprise script object has the RegisterRecords property. It returns the RegisterRecordsCollection object, which contains a collection of register record sets that this document can use to generate register records.

You can address a specific record set of this collection by writing the name of the register that stores the record set after a dot (.). For example, RegisterRecords.BalanceOfMaterials.

Next, after a dot you can use register record sets methods, for example, RegisterRecords.BalanceOfMaterials.Add(). The Add() method adds a record to the record set.

The first line of the procedure assigns the True value to the Write property of the register record set. So it explicitly specifies that once the posting is completed the platform writes this record set to the database.

The handler includes the loop For Each … In … Do. This loop is intended to iterate through the document tabular section rows.

In the loop the document tabular section is called by its name (Materials). The CurRowMaterials variable contains an object storing the data of the current document tabular section row. This variable is created at the beginning of the loop and changes with each iteration.

In the first line of the loop body the Add() method adds a record to the record set that the document creates in the register. To be exact, it creates an AccumulationRegisterRecord object and saves it to the RegisterRecord variable.

Using this object, you can call the fields of the record by writing the field name after the variable, separated by a dot (for example, RegisterRecord.Quantity).

Note that Record.Material and Record.Warehouse are register dimensions, Record.Quantity is a register resource, and Record.RecordType and Record.Period are standard register attributes that are generated automatically.

In order to assign values to the fields of the new register record, tabular section fields are addressed by writing their names after the CurRowMaterials variable and a dot (for example, CurRowMaterials.Material).

Note that Warehouse is an attribute of the document header, while Date is a standard document attribute that is generated automatically. Note that only the values of the document tabular section are changed within the loop (CurRowMaterials.Material and CurRowMaterials.Quantity). Other fields are not changed because they belong to the entire document and do not depend on the document tabular section row.

AccumulationRecordType.Receipt is the value of the system enumeration that defines the type of the accumulation register record type as Receipt.

So the required values are assigned to all the fields of the new record. After iterating through all document rows (when the loop is completed), the record set (RegisterRecords.BalanceOfMaterials) contains the number of records that is equal to the number of rows in the tabular section of the posted document.

If you open the configuration object editor for the BalanceOfMaterials accumulation register and click the Recorders tab, you will see that the **GoodsReceipt** check box is now selected because you set up generation of register records for the BalanceOfMaterials register in the document module (fig. 6.10).

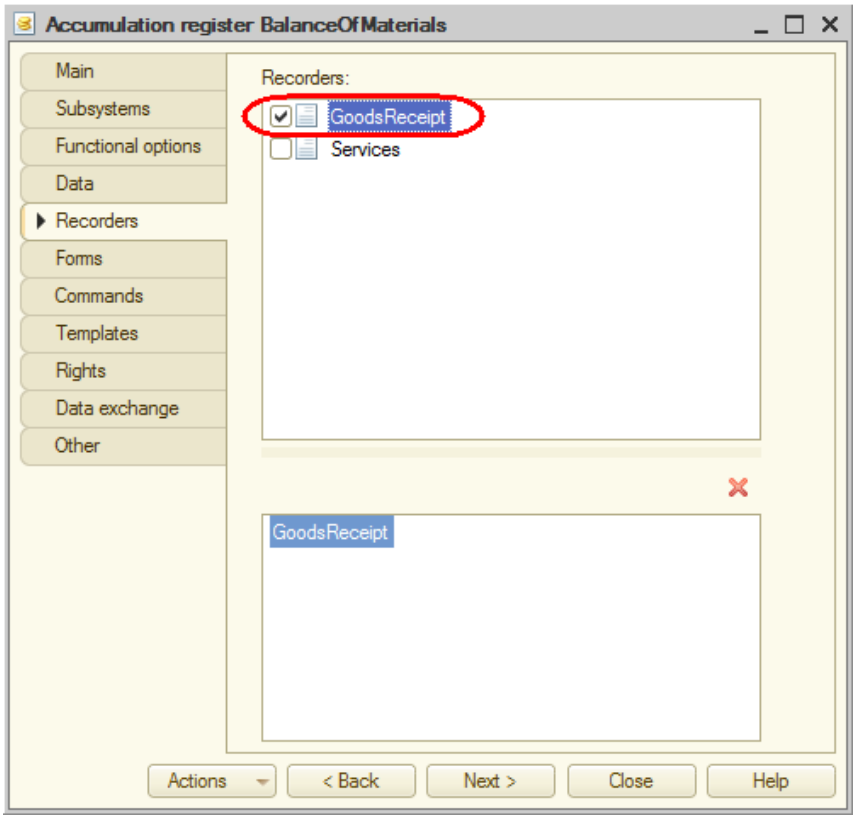


Fig. 6.10. Recorders of the BalanceOfMaterials register

Finally, let us edit the command interface to add the link for viewing the accumulation register records to the Accounting, Services, and Items subsystems.

Commands that open registers are added to the section command panels but, unlike commands that open catalogs or documents, they are hidden by default.

1. In the configuration object tree, right-click the **Subsystems** branch and then click **All subsystems**.

2. In the **All Subsystems** window, in the **Subsystems** list, click **Inventory**.

   The **Command interface** list displays all the commands of the selected subsystem.

3. In the **Navigation panel.Normal** group, enable visibility for the **Balance of materials** command and drag it to the **Navigation panel.See also** group (fig. 6.11).

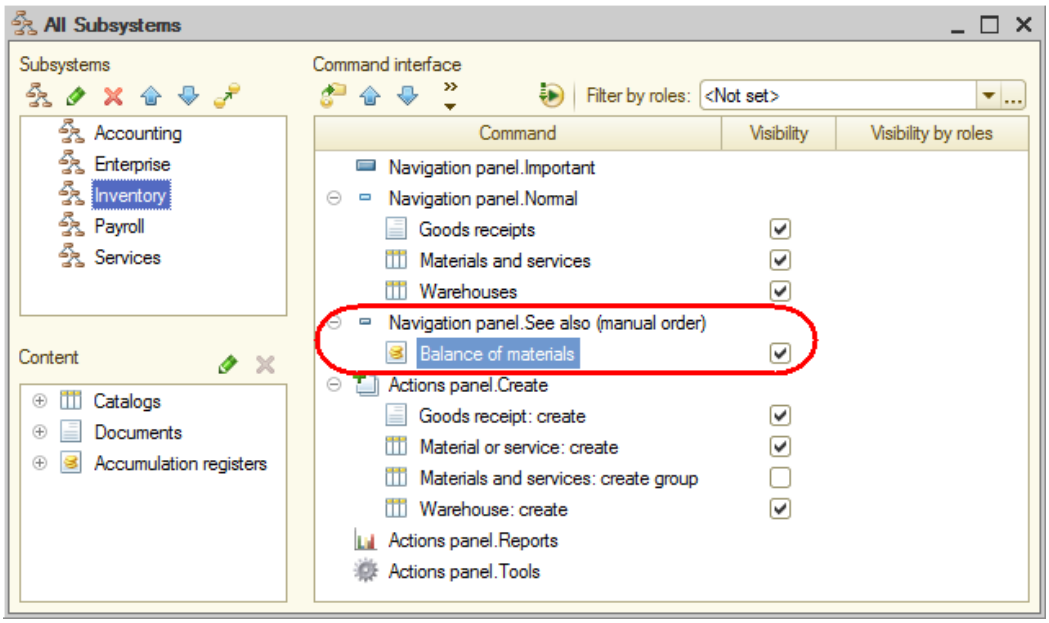


Fig. 6.11. Specifying subsystem command interface settings

Indeed, the commands that open accumulation registers are used less frequently so it is better to move them to the bottom of the section command list.

4. Perform the same procedure for the **Services** and **Accounting** subsystems: in the **Navigation panel.Normal** group, enable visibility for the **Balance of materials** command and drag it to the **Navigation panel.See also** group.

In 1C:Enterprise mode

Let us test the changes that you made.

1. Start 1C:Enterprise in the debug mode.

   You can see that the Balance of materials command that opens the register list is added to the Inventory, Services, and Accounting subsystems. It is located after other list opening commands and before the Create and Reports menus (if these menus are available, fig. 6.12).

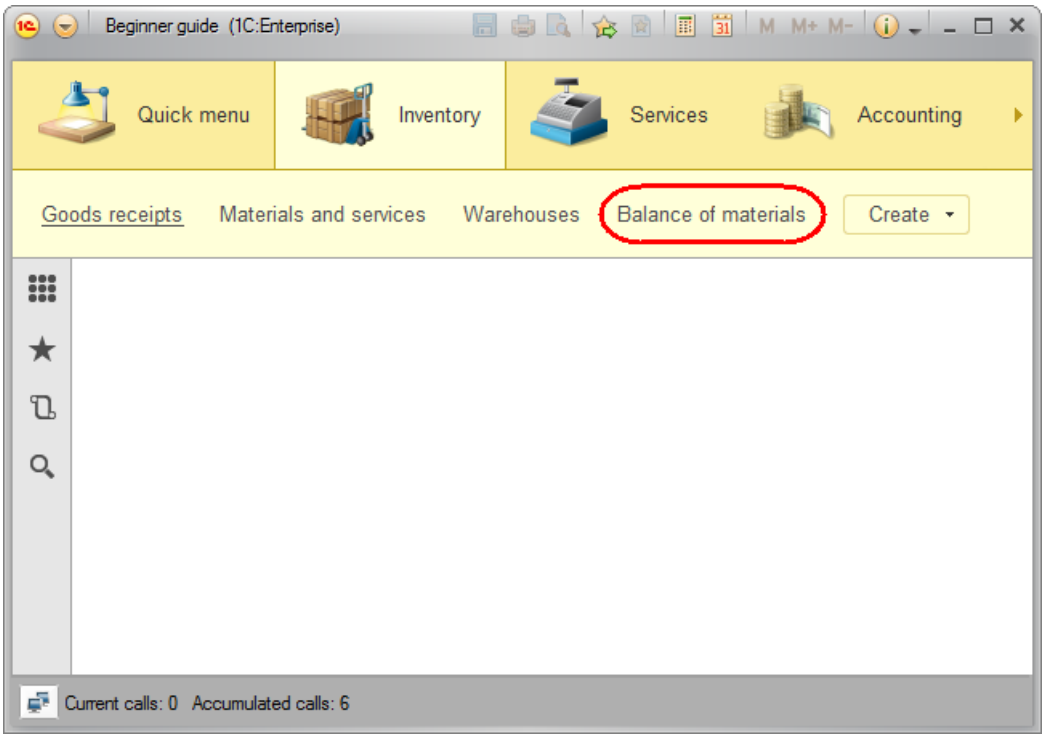


Fig. 6.12. List of the BalanceOfMaterials accumulation register

Now let us track the connection between document posting and data accumulation in the register.

2. In the **Accounting** section, run the **Goods receipts** command.
3. Open the first Goods receipt and click **Post and close**, then do the same for the second Goods receipt.

   This reposts the documents. You can also repost documents without opening them. To repost a document, select it in the list (you can select multiple documents by clicking them while holding down Ctrl), click **More** in the command bar of the list form, and click **Post** (fig. 6.13).

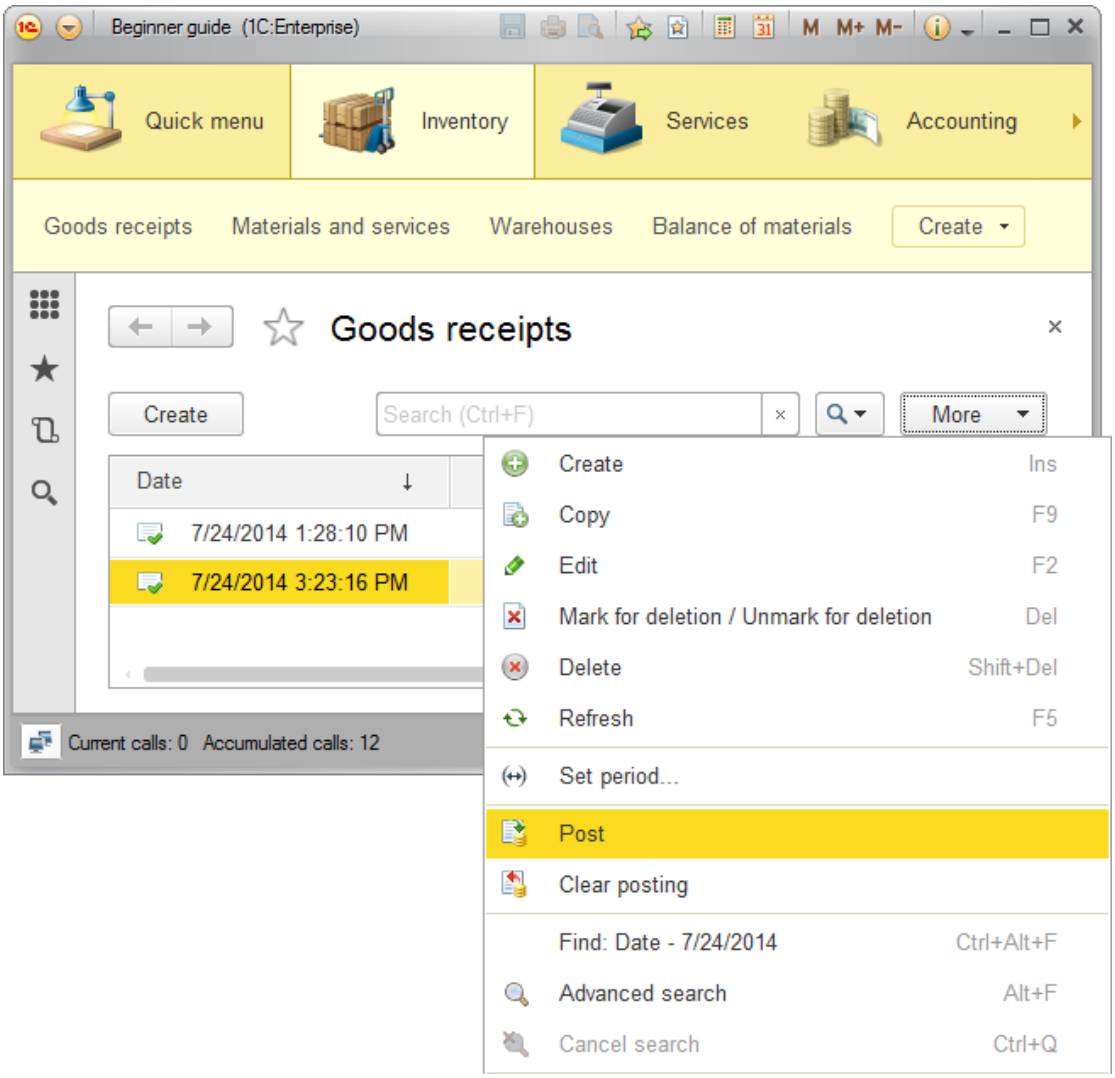


Fig. 6.13. Posting a document

4. Run the **Balance of materials** command.
   This opens the accumulation register list (fig. 6.14).

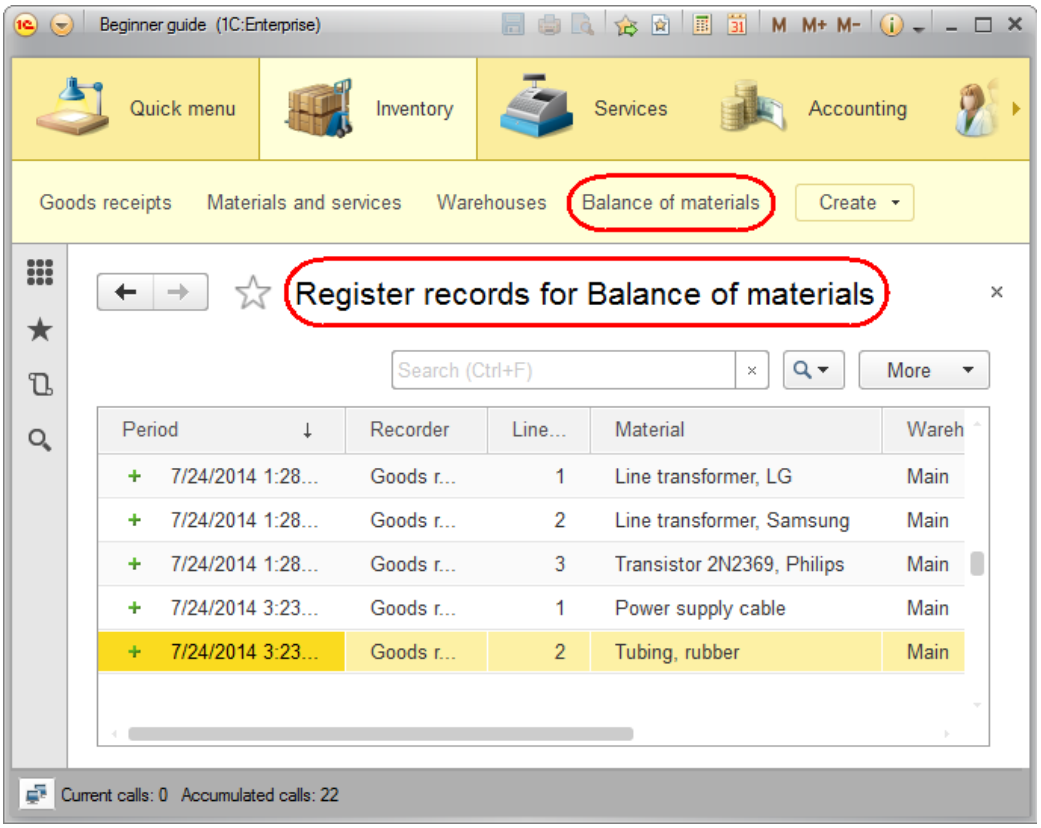


Fig. 6.14. List of the BalanceOfMaterials accumulation register

You can see that when you post goods receipts, respective records are added to the Balance of materials accumulation register. Note that five records are added: three records from posting the first document (this number matches the number of rows in its tabular section) and two records from posting the second document.

All register fields are filled with the document data exactly as specified in the posting handler of the GoodsReceipt document. The + icon to the left of each record indicates that the register record type is Receipt. As you can see, the title of the register record list form matches the one that you specified in the Extended list presentation property for this register.

Register records opening command in a document form

In Designer mode

In real-life work the number of records in the BalanceOfMaterials register can be very large, so it might be difficult to understand the relations between records and specific documents.

This is why in addition to the full register list you need the option to open a register list that only contains the records generated by a specific document from the document form. Let us implement this.

1. Return to Designer and open the GoodsReceipt document form.

2. In the upper left pane, click the **Command Interface** tab.
3. In the **Navigation panel** branch, expand the **Go to** node.

   It contains the command that opens the list of the Balance of materials accumulation register. The command is automatically included in the navigation panel of the document form because this document is a recorder, that is, it creates register records in that register.
4. Select the **Visible** check box for this command (fig. 6.15).

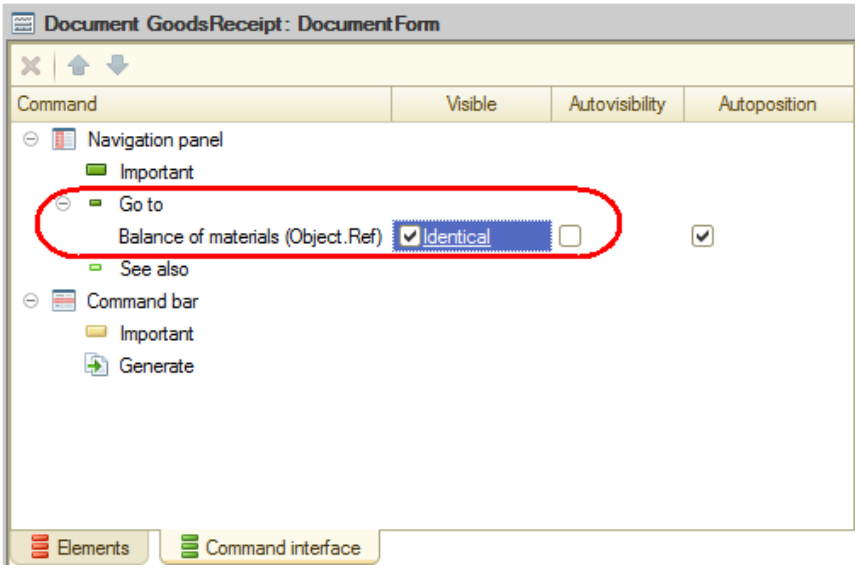


Fig. 6.15. Document form command interface setup

In 1C:Enterprise mode

Let us test the changes.

- Start 1C:Enterprise in the debug mode and open the second Goods receipt (fig. 6.16).

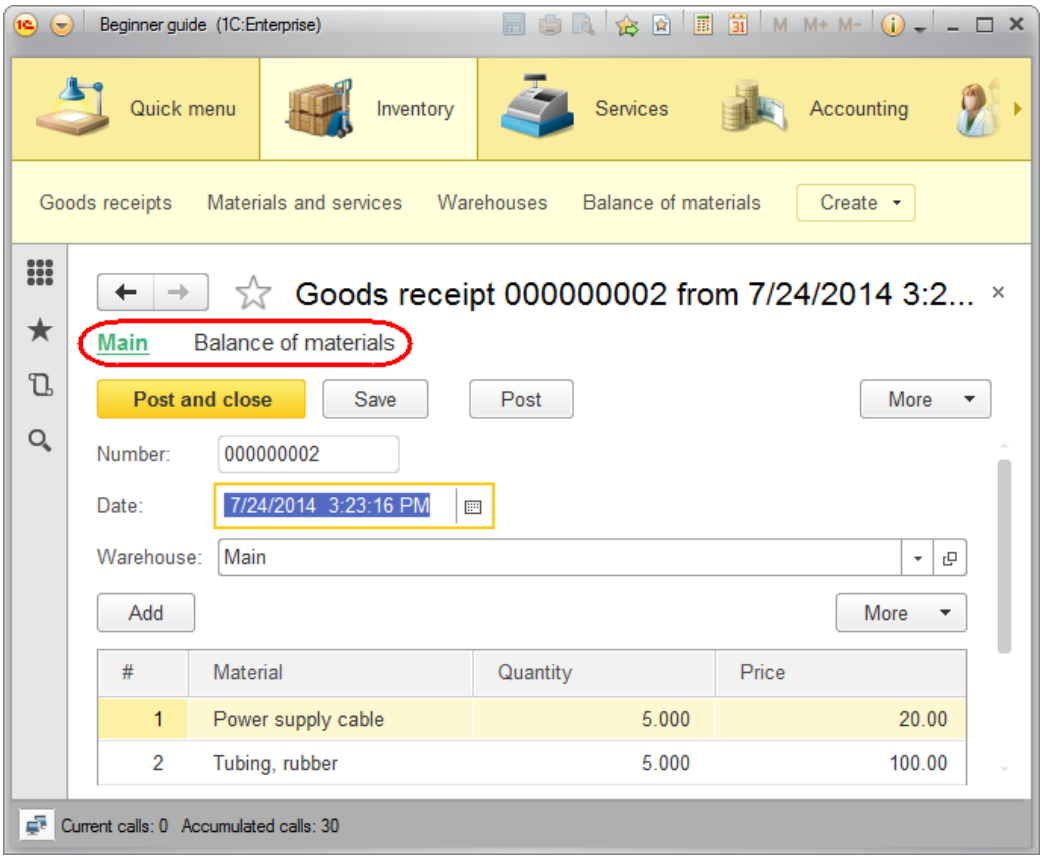


Fig. 6.16. Navigation panel of the Goods receipt document form

The document form now has a navigation panel where you can switch to the list of Balance of materials register records related to this document (fig. 6.17) and then switch back to the document content (the Main link).

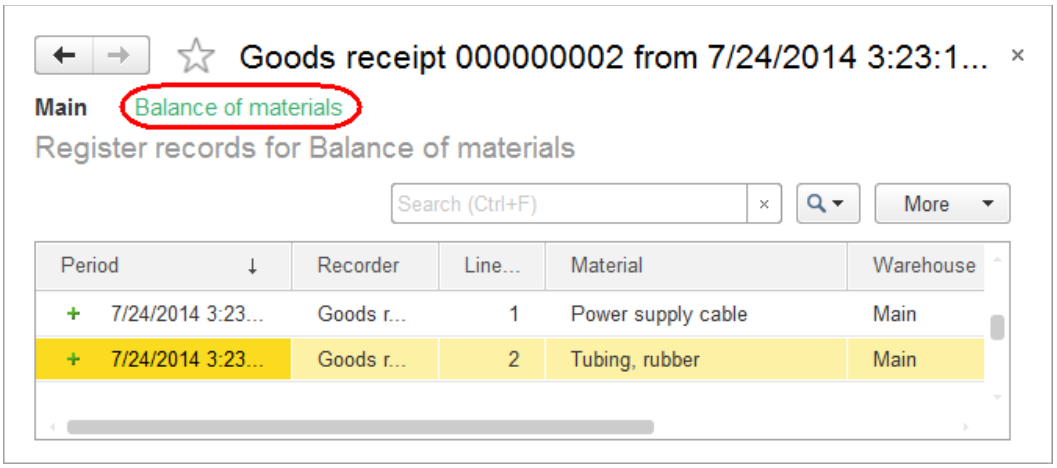


Fig. 6.17. Switching to the accumulation register from the document form

Note that the navigation panel of the goods receipt form was previously hidden because it did not contain any commands.

Creating register records of the Services document

Now let us set up generation of register records for the Services document.

In Designer mode

Let us use a procedure similar to setting up register records for the Goods receipt document.

1. Open the configuration object editor for the Services document.
2. Click the **Posting** tab and select the **BalanceOfMaterials** check box.
3. Click **Register records wizard**.
4. In the wizard window, set **Register Record Type** to **Expense**.

   Indeed, the Services document is intended to record expenses of materials. Note that the icon to the left of the register name changed to the minus sign −.
5. In the **Tabular Section** list, select **MaterialsAndServices**.

   It is the only tabular section in the document. Its attributes are added to the list of document attributes.
6. Click **Fill Expressions**.

   In the bottom pane, the mapping between the register fields (dimensions and resources) and the expressions for calculating their values is generated. But the **Material** field is not filled automatically.

   This is because the name of the tabular section field (MaterialOrService) does not match the name of the register dimension (Material). If you leave it as is, products will not be recorded to accumulation register rows with Expense register record type. Let us fix that.
7. Select the **Material** register field and then, in the **Document attributes** list, double-click **CurRowProductList.MaterialOrService**.

   This ensures that the materials for the accumulation register records are selected from the document tabular section (fig. 6.18).

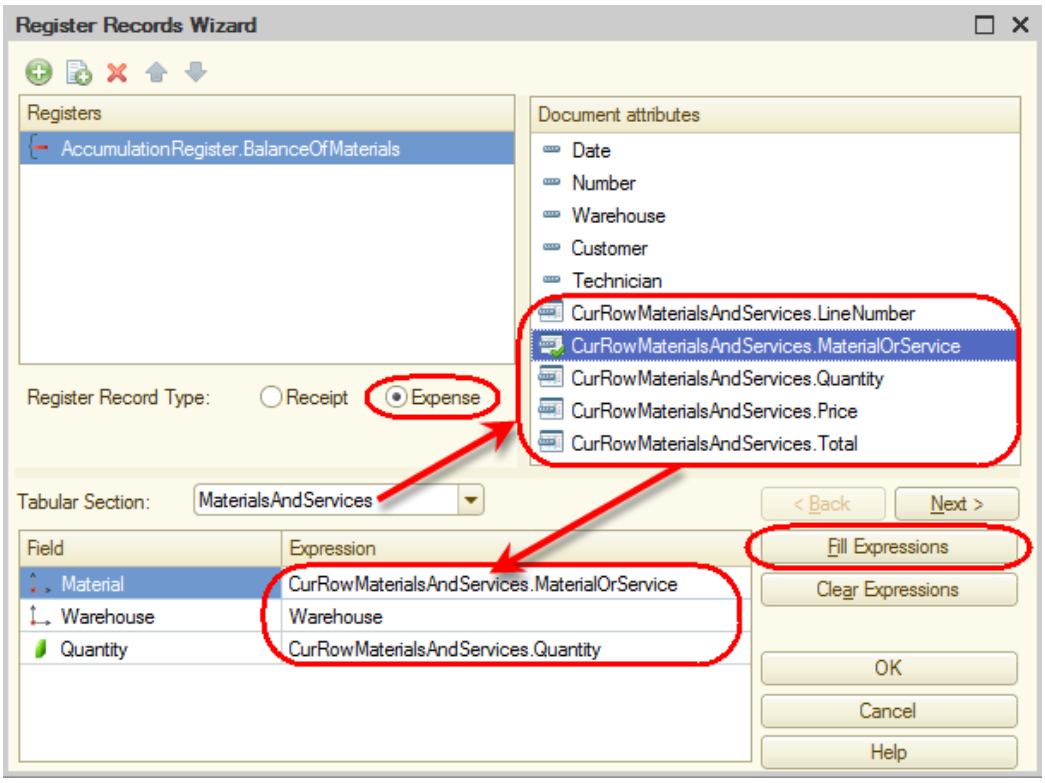


Fig. 6.18. Selecting a document tabular section and filling the expressions for calculating register records

8. Click **OK**.

The wizard generates a procedure in the Services document module (listing 6.2).

Listing 6.2. Posting() procedure

```
Procedure Posting(Cancel, Mode)
    //{{__REGISTER_REGISTERRECORDS_WIZARD
    // This fragment was built by the wizard.
    // Warning! All manually made changes will be lost next time you
use the wizard.

    // register BalanceOfMaterials Expense
    RegisterRecords.BalanceOfMaterials.Write = True;
    For Each CurRowMaterialsAndServices In MaterialsAndServices Do
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material =
            CurRowMaterialsAndServices.MaterialOrService;
        Record.Warehouse = Warehouse;
        Record.Quantity = CurRowMaterialsAndServices.Quantity;
    EndDo;

    //}}__REGISTER_REGISTERRECORDS_WIZARD
EndProcedure
```

It is the Posting event handler for the Services document configuration object.

Note that the line Record.RecordType = AccumulationRecordType.Expense defines the type of the accumulation register record generated by this document as Expense. The rest of the procedure is similar to the posting handler of the GoodsReceipt document (see listing 6.1) that we have discussed in detail earlier.

Finally, let us edit the command interface of the document. You will add the command that opens the Balance of materials list with records generated by that document to the navigation panel.

1. Open the Services document form.
2. In the upper left pane, click the **Command interface** tab.
3. In the **Navigation panel** branch, expand the **Go to** node and select the **Visible** check box for the command that opens the Balance of materials accumulation register.

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. In the **Services** section, open the first Services document and click **Post and close**.

   This reposts the document.
3. Run the **Balance of materials** command.

   This opens the accumulation register list (fig. 6.19).

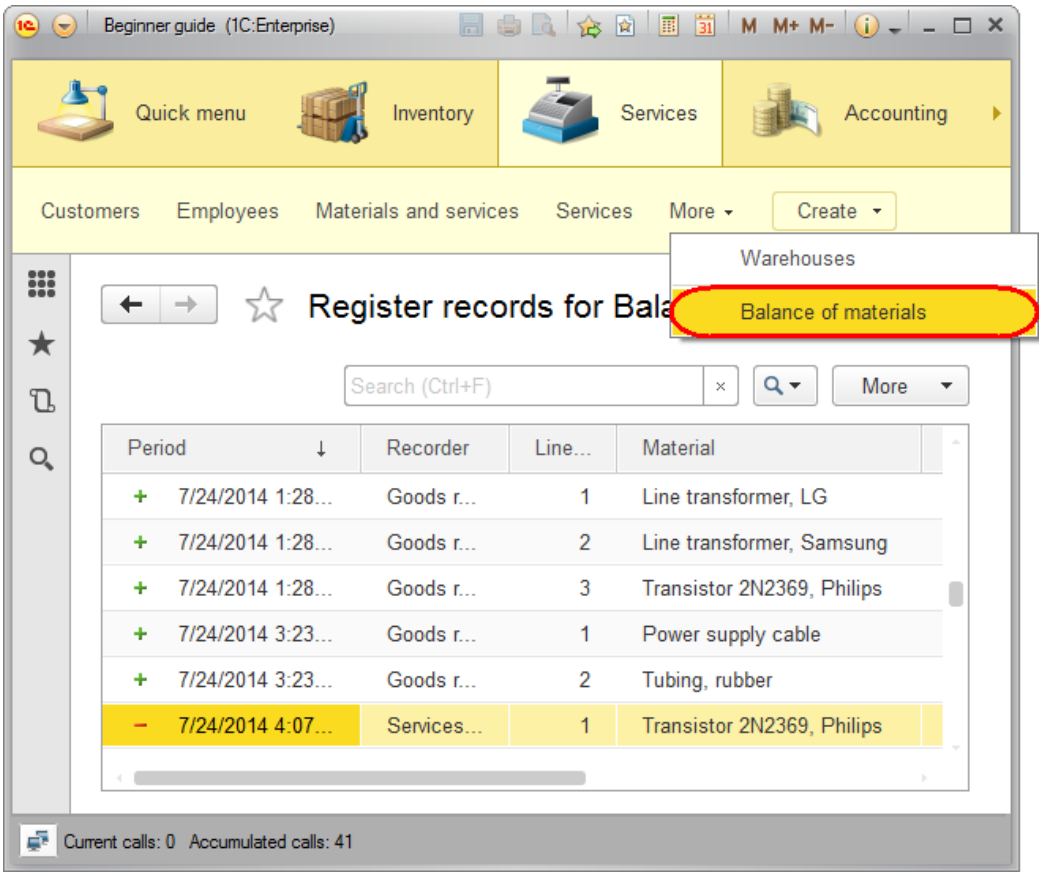


Fig. 6.19. List of the BalanceOfMaterials accumulation register

You can see that the Balance of materials accumulation register now has a new record (a single record because the tabular section of the posted document has a single row).

All register fields are filled with the document data exactly as specified in the posting handler of the Services document.

The – icon to the left of the record indicates that the register record type is Expense (see fig. 6.19).

Now you see the full list of register records. You can have it filtered by recorder document if you open it from a document form.

4. Open the first Services document again.

   The document form now has a navigation panel where you can switch to the list of Balance of materials register records related to this document (fig. 6.20) and then switch back to the document content (the **Main** link).

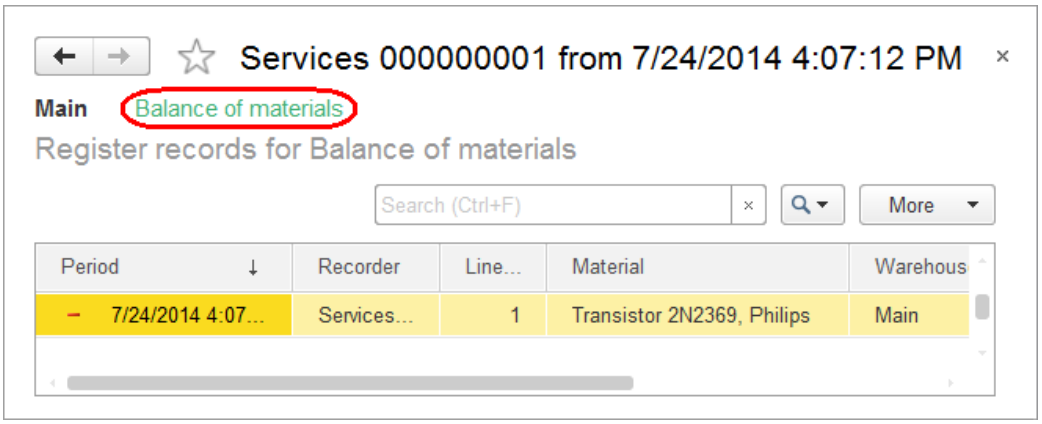


Fig. 6.20. Switching to the accumulation register from the document form

But the register records generated by this document might be inaccurate.

In contrast to the Goods receipt document, the Services document can list not only the materials but the services as well. Therefore, records that describe rendered services can appear in the Balance of materials register, which is not what we want.

For now, we will not change the register records generated by the wizard, but as soon as we proceed to enumerations, you will make the necessary modifications to the posting handler.

Theory. Accessing collection items

While describing the document register records, you have encountered a 1C:Enterprise script object that is a collection. It is used in the loop that processes the tabular sections of the GoodsReceipt and Services documents.

Many of the 1C:Enterprise script objects are collections. A collection is a set of objects. There are some general principles that apply to collection operations.

First, you can access each collection object by iterating through the collection items in a loop using the following syntax: For Each … In … Do … (listing 6.3).

Listing 6.3. Iterating through collection items in a loop

```
For Each TabularSectionRow In TabularSection Do
    Message(TabularSectionRow.MaterialOrService);
EndDo;
```

In this example TabularSection is a collection of tabular section rows of a configuration object. In each iteration, the TabularSectionRow variable contains the next row of the collection.

Second, you can access a collection item directly, without iterating through all items in a loop. These two approaches can be combined.

1. Some collections are named. These are the collections where each item has a unique name, and you can access an item by its name (listing 6.4).

Listing 6.4. Accessing a collection item

```
Catalogs.Employees;
// -OR-
Catalogs["Employees"];
```

In this example Catalogs is a collection of managers of all catalogs available in the configuration. Since each catalog in a configuration has a unique name, a specific item of this collection (a manager of a specific catalog) can be referenced by the catalog name: Catalogs["Employees"].

2. If naming the collection items makes no sense (the collection is not named), you can access a collection item by its index (the index for the first item in a collection is zero, listing 6.5).

Listing 6.5. Accessing a collection item by index

```
TabularSection[0];
```

In this example TabularSection is a collection of tabular section rows of a configuration object. The first item in the collection is accessed by its index 0.

Note that there are collections where both access methods are combined. For example, you can access a collection of columns in a value table either by column name or by index.

Quiz

- What are the uses of the Accumulation Register configuration object?
- Why should one use registers while the required data is available in other objects?
- What are the uses of register dimensions, resources, and attributes?
- What is a register record? What is a recorder?
- How does one create an accumulation register and define its structure?
- How does one create register records of a document using the register records wizard?
- How does one iterate through a document tabular section and access its data using 1C:Enterprise script?
- How does one display the commands that open register lists in the configuration interface and in the form interface?

Lesson 7 (0:25). A simple report

In this lesson you will study the Report configuration object. You will learn the purpose of this object, and then you will create a report that shows the register records and the material balance in your company.

The goal of this lesson is limited to explaining the basics of report creation using a simple example. This is why we will not discuss the details of the data composition schema that is used in report generation. This aspect will be covered in "Lesson 13. Reports" on page 292.

Understanding reports

The Report configuration object is intended for defining algorithms that users run to obtain the output data they need. You can define the algorithms that generate output data using visual design tools or using 1C:Enterprise script. In practice, Report objects describe various tables with output data, summaries, charts, and so on.

Adding a report

In Designer mode

Now you have everything you need to obtain some output data. Let us create a report that will display receipts, expenses, and the balance of materials (fig. 7.1).

Data parameters: Begin of period: 7/1/2014 12:00:00 AM
End of period: 7/30/2014 12:00:00 AM

Warehouse	Material	Quantity Starting balance	Quantity Receipt	Quantity Expense	Quantity Ending balance
Main	Line transformer, Samsung		10.000		10.000
Main	Line transformer, LG		10.000		10.000
Main	Transistor 2N2369, Philips		10.000	1.000	9.000
Main	Tubing, rubber		5.000		5.000
Main	Power supply cable		5.000		5.000

Fig. 7.1. Resulting report

Using this example, we will show how you can develop a report quickly and easily using only visual development tools and without writing any single script line.

Let us open your practice configuration in Designer and create a Report configuration object.

1. In the configuration object tree, click the **Reports** branch.

2. In the configuration window command bar, click the **Add** button (fig. 7.2).

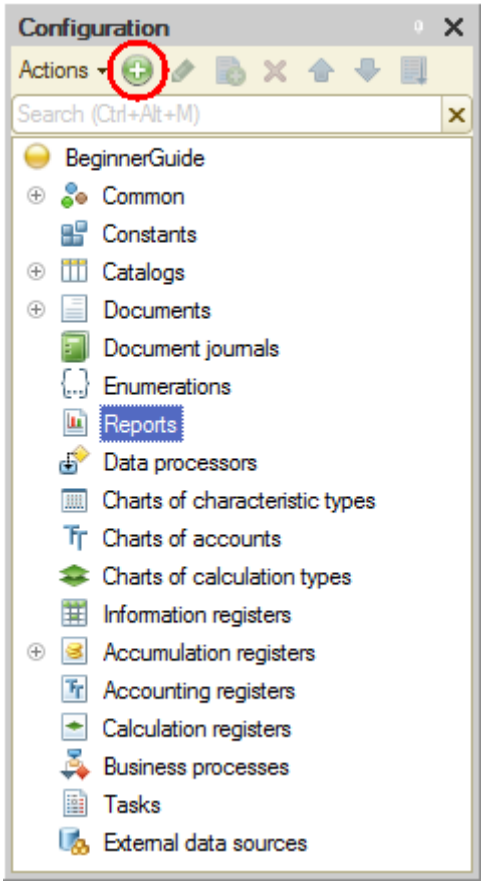


Fig. 7.2. Creating a Report configuration object

3. In the configuration object editor window, on the **Main** tab, in the **Name** field, type *Materials*.

   Do not set other properties that define the object presentation in the applied solution interface. The platform will use the automatically generated synonym instead.

   Next let us create the data composition schema, which serves as a basis for report generation.

4. Click **Open data composition schema** (fig.7.3).

   -OR-

   Click the **Open** button (fig. 7.3).

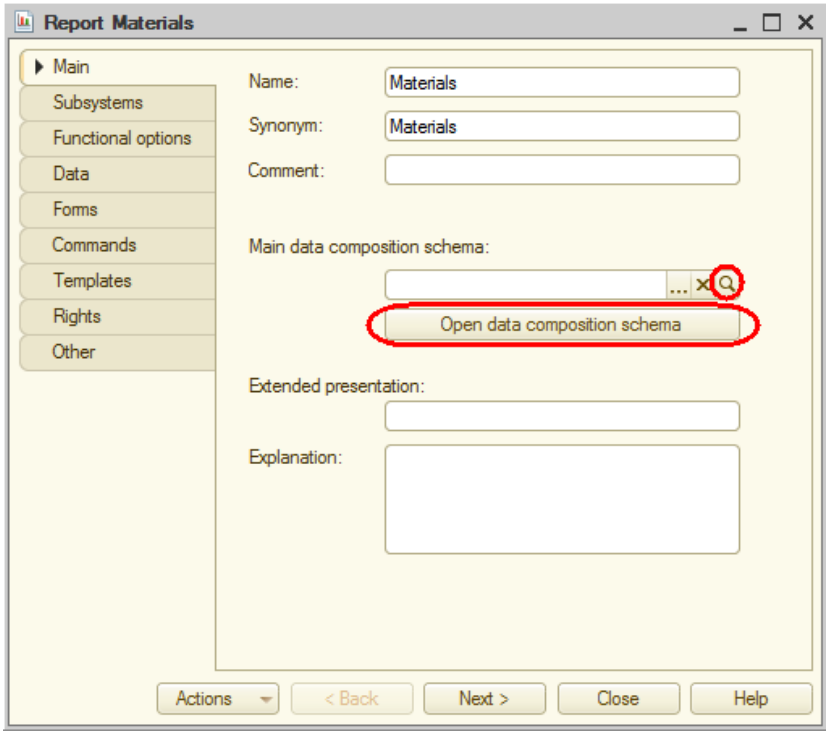
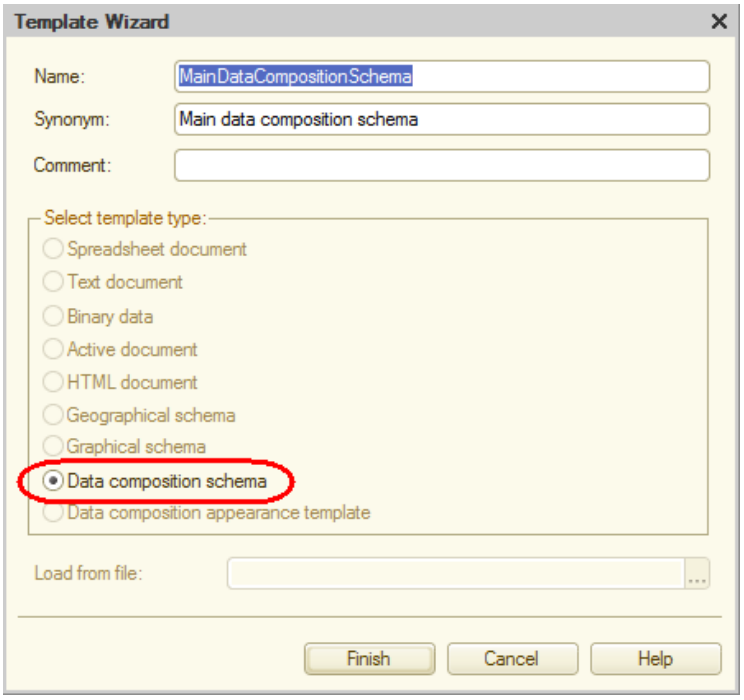


Fig. 7.3. Creating a data composition schema for a report

Template

Since the report that you are creating does not have any data composition schema yet, the platform prompts you to create a schema. From the configuration perspective, a data composition schema is a template, and the platform opens the template wizard with the only template type available: **Data composition schema** (fig. 7.4).



7.4. Creating a data composition schema for a report

- Click **Finish**.

Data composition schema

The platform creates a template that contains the data composition schema and then opens the data composition schema wizard.

The wizard provides a wide range of features for visual report design, but now you will use just a few basic features to specify the data that will be included in the report.

Data set

Let us add a data set (a query) to the data composition schema.

- Click the **Add data set** button (fig. 7.5) and then click **Add data set - query**.

  -OR-

  In the left pane, right-click **Data Sets** and then click **Data set - query**.

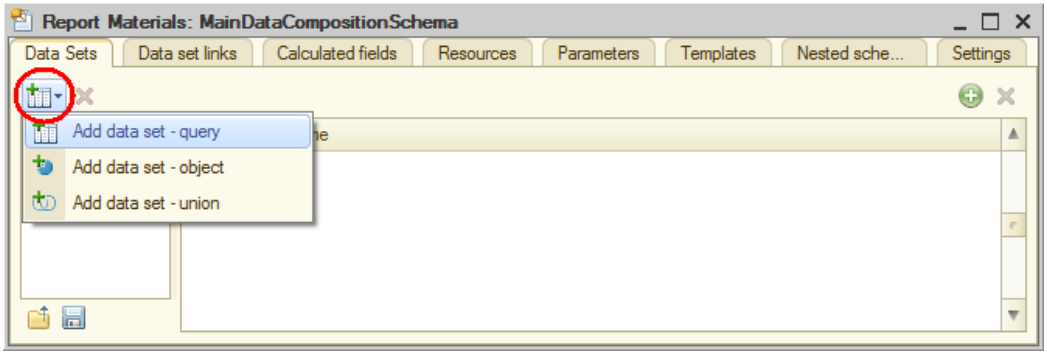


Fig. 7.5. Adding a data set in the data composition schema wizard

Query text

Let us create the query text.

1. Click **Query Builder** (fig. 7.6).

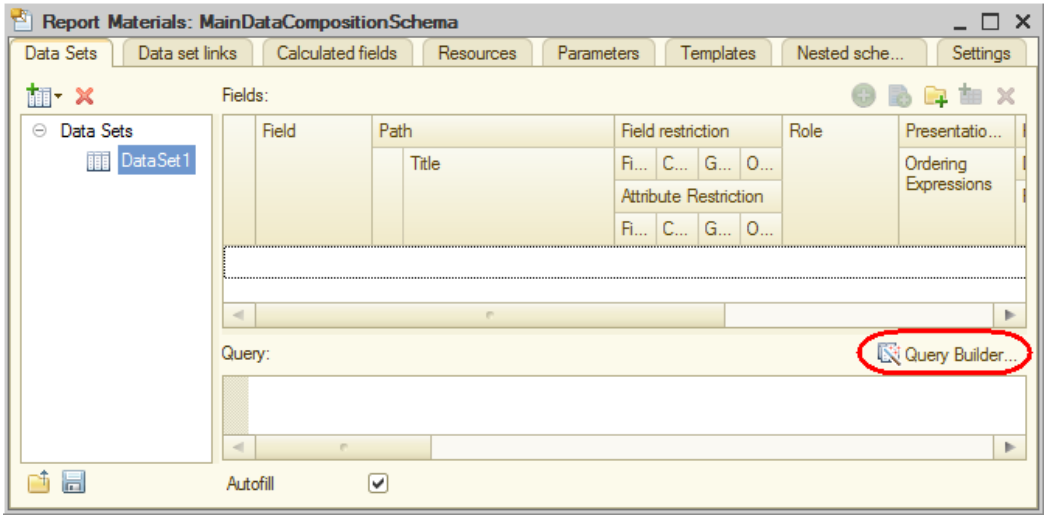


Fig. 7.6. Opening the Query builder

The Query builder is intended for generating queries using visual tools. Even users who are unfamiliar with the query language can use the wizard to generate valid queries.

The **Database** list displays the tables used to generate a query. The report is generated based on the data stored in these tables.

If you expand the **AccumulationRegisters** branch, you will see that, in addition to the **BalanceOfMaterials** register table, this branch contains several virtual tables that are generated automatically (fig. 7.7).

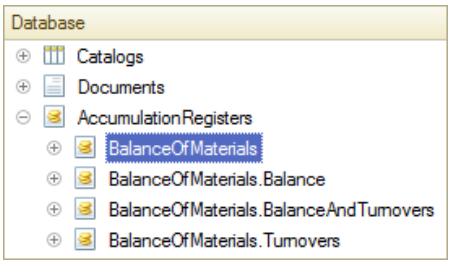


Fig. 7.7. Tables that you can use in a query

The register virtual tables are generated automatically, they are mainly used in report generation. Since we want to see both the balance of materials and information about material receipts and expenses, let us opt for the BalanceOfMaterials.BalanceAndTurnovers virtual table.

2. Drag the **BalanceOfMaterials.BalanceAndTurnovers** table to the **Tables** list and expand its structure (fig. 7.8).

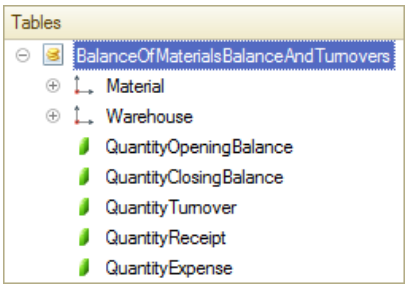


Fig. 7.8. The BalanceOfMaterials.BalanceAndTurnovers table

As you can see, this table contains the dimensions of the BalanceOfMaterials register (Material and Warehouse), the opening and closing balance, as well as receipt, expense, and turnover values for all the resources of the BalanceOfMaterials register.

Let us select the table fields in the required order.

3. Double-click the following items in the specified order:
   - **Warehouse**
   - **Material**
   - **QuantityOpeningBalance**
   - **QuantityReceipt**
   - **QuantityExpense**
   - **QuantityClosingBalance**

Note. Instead of double-clicking, you can move the selected items between the lists by dragging them of by clicking the >, <, >>, and << buttons.

At this point the **Fields** list looks as follows (fig. 7.9).

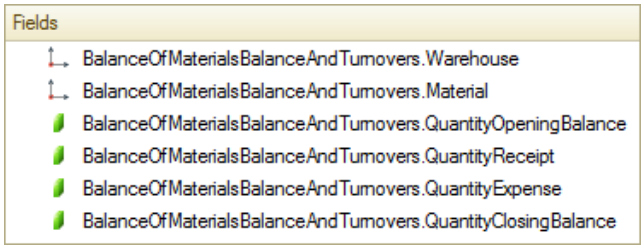


Fig. 7.9. Selected fields

4. Click **OK** to return to the data composition schema wizard (fig. 7.10).

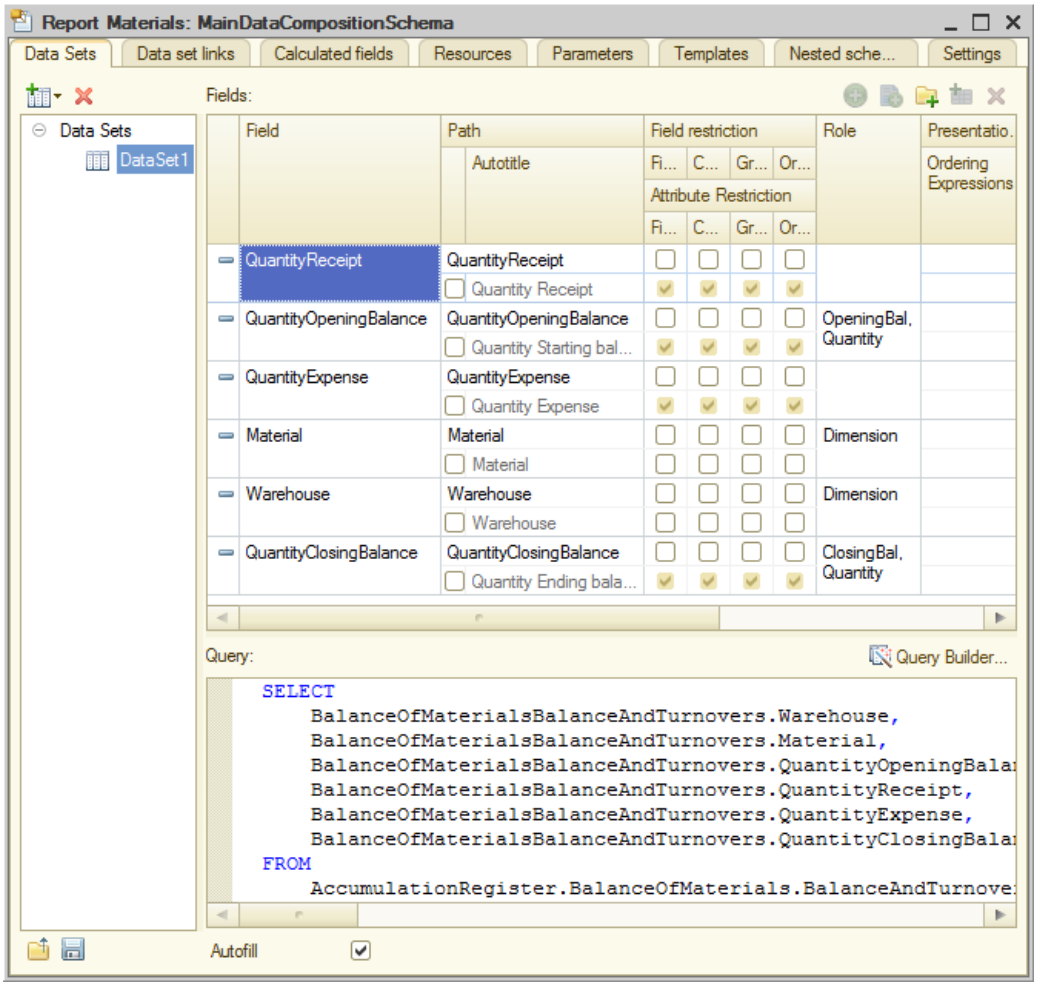


Fig. 7.10. Data composition schema wizard

The platform populates the **Query** field with the query text generated by the Query builder. This field is basically a text editor where you can edit the resulting query. Alternatively, you can reopen the Query builder and use it to edit the query.

To simplify things, we will not discuss the query text here. This aspect will be covered in Lesson 13. Reports (in the parts dedicated to the query

language. In this scenario (just like with many other reports) you can avoid analyzing and editing the query text.

Note the automatically populated list of data composition system fields in the upper part of the wizard window. It lists the fields available in the current data set. In this case the platform populated the list automatically based on the query text, so you do not need to adjust it manually.

So we have discussed how to extract data that is used in a report. But to have this data displayed in a report, you have to create the default report settings. Let us create simple report settings suitable for viewing typical detailed infobase records. These include the records of the BalanceOfMaterials accumulation register virtual table selected in the order that you used to add them into the table.

Report settings

Let us create the report settings.

1. Click the **Settings** tab.

   The upper right pane displays the hierarchical structure of the report.

2. Right-click **Report** and then click **New grouping**.

   -OR-

   In the command bar of the window, click the **Add** button and click **New grouping**.

   -OR-

   Click **Report** and then press Insert.

3. Do not enter the grouping field, simply click **OK**.

   You have specified that the report includes detailed infobase records, that is, the records that are retrieved when a query is executed without totals (fig. 7.11).

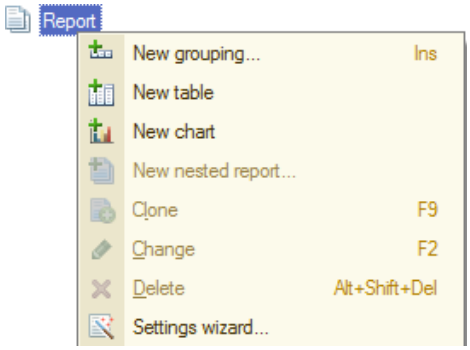


Fig. 7.11. Adding a grouping to a report

The **Detailed records** grouping is added to the report.

Now let us specify the fields that will be displayed in the report.

4. In the bottom pane, on the **Selected fields** tab, drag the following fields from the **Available fields** list to the **Selected fields** list:
   - **Warehouse**
   - **Material**
   - **QuantityOpeningBalance**
   - **QuantityReceipt**
   - **QuantityExpense**
   - **QuantityClosingBalance**

   Note. You can also add available fields to the list of selected fields by double-clicking the fields or by clicking **Add** to the right of the list of selected fields. You can change the order of selected fields using the **Move up** and **Move down** buttons or by dragging them.

   Now the report settings window looks as follows (fig. 7.12).

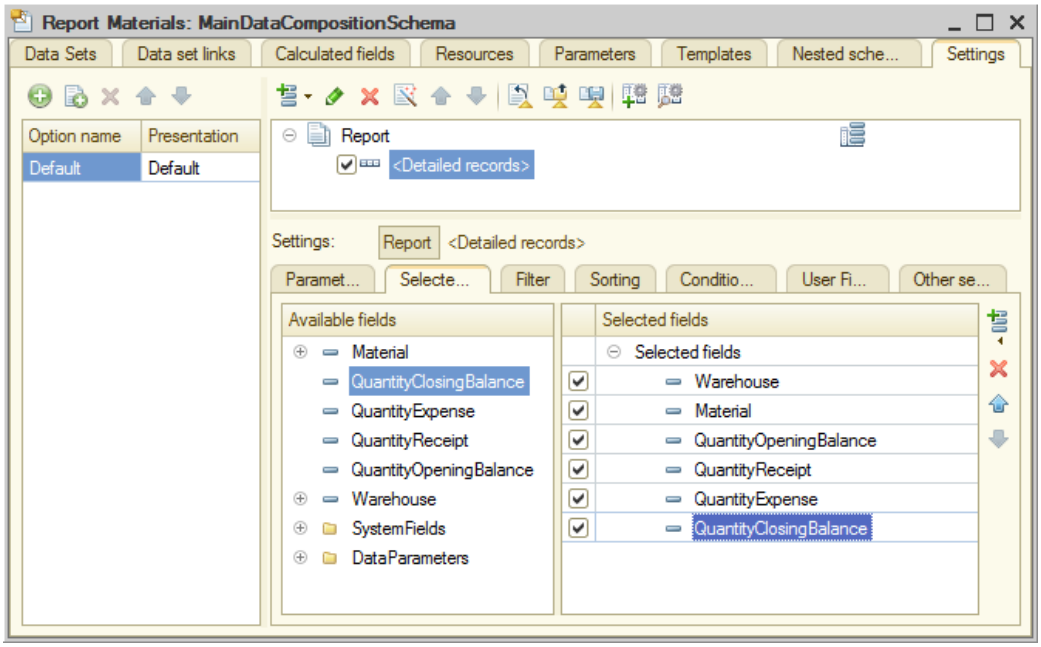


Fig. 7.12. Report settings window

5. In the bottom pane, click the **Parameters** tab.

   Let us specify that the Begin of period and End of period report parameters are included in the list of custom settings and these settings are located directly in the report form (they will serve as quick settings).

6. In the lower pane, select the **Begin of period** and **End of period** check boxes.

   The check boxes indicate that both parameters are used in the report.

7. For each parameter, select it, click the **Custom settings item property** button, and select the **Include in custom settings** check box (fig. 7.13).

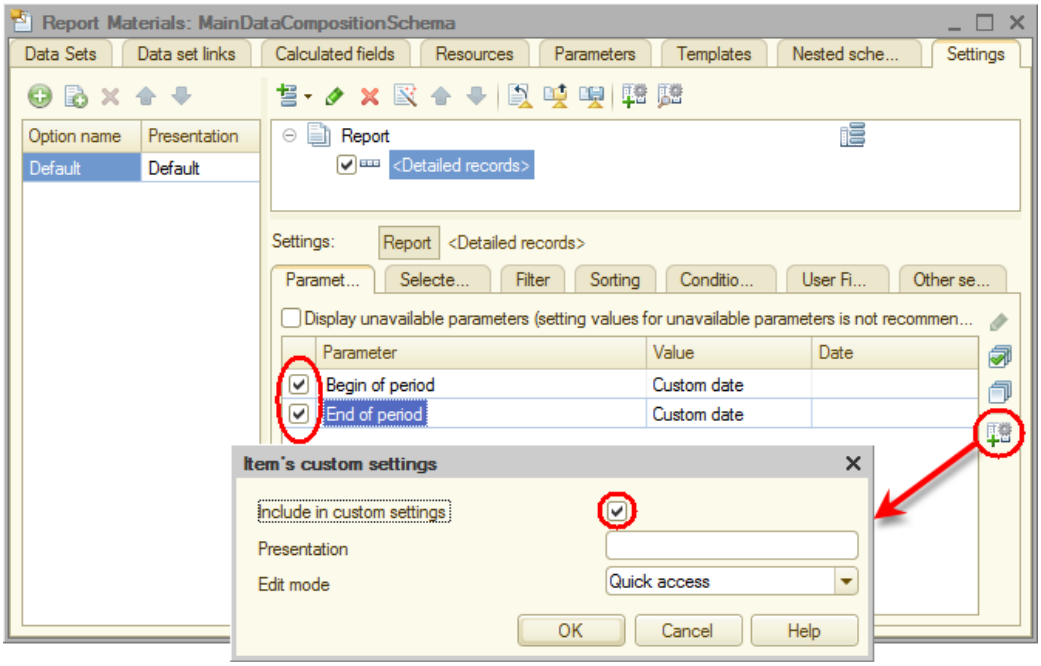


Fig. 7.13. Report settings window

So before a report is actually generated, a user can specify the report period. For detailed explanation of report parameters and custom settings, see "Lesson 13. Reports" on page 292.

Finally, let us specify the subsystems where the report is available.

8. Close the data composition schema wizard and then, in the editor of the **Materials** report configuration object, click the **Subsystems** tab.
9. Select the **Accounting**, **Inventory**, and **Services** check boxes (fig. 7.14).

   This adds the link that opens the report to the **Reports** menu in the command panels of these subsystems.

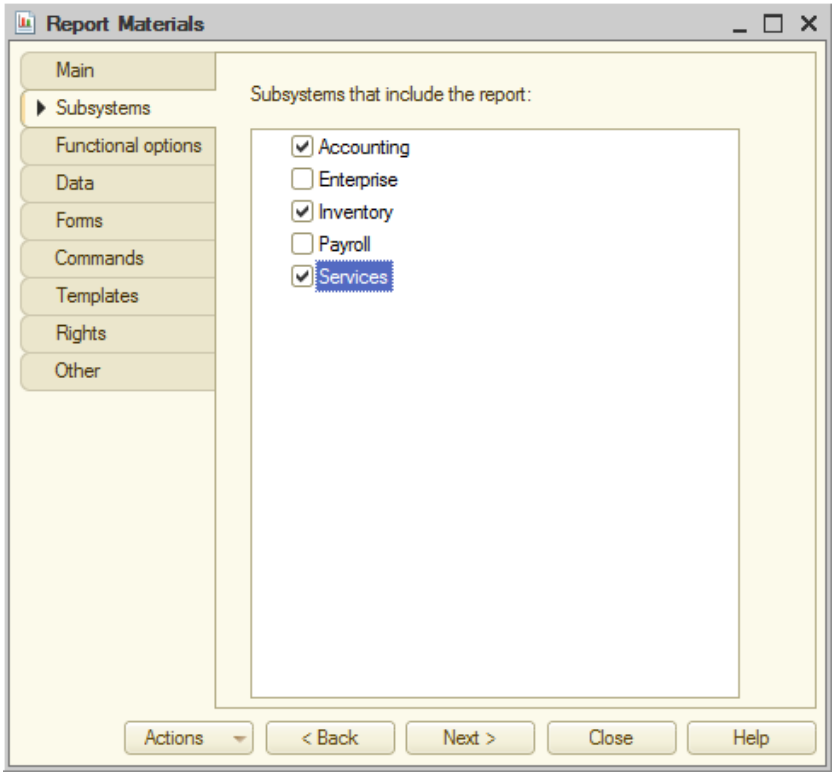


Fig. 7.14. Specifying the list of subsystems where a report is available

In 1C:Enterprise mode

Let us see how the report works.

1. Start 1C:Enterprise in the debug mode.

   You can see that the Accounting, Services, and Items sections now include the **Reports** menu, which contains the command that generates the Materials report (fig. 7.15).

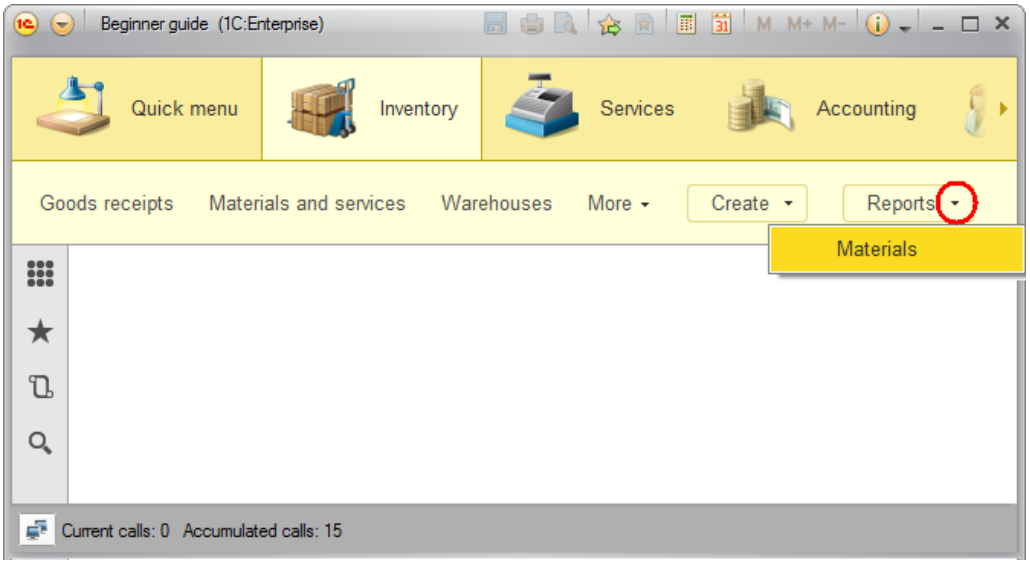


Fig. 7.15.Command that generates the Materials report

2. Click the **Materials** command.

   This opens an automatically generated report form.

3. Specify the beginning and end dates of the report period and click **Create** (fig. 7.16).

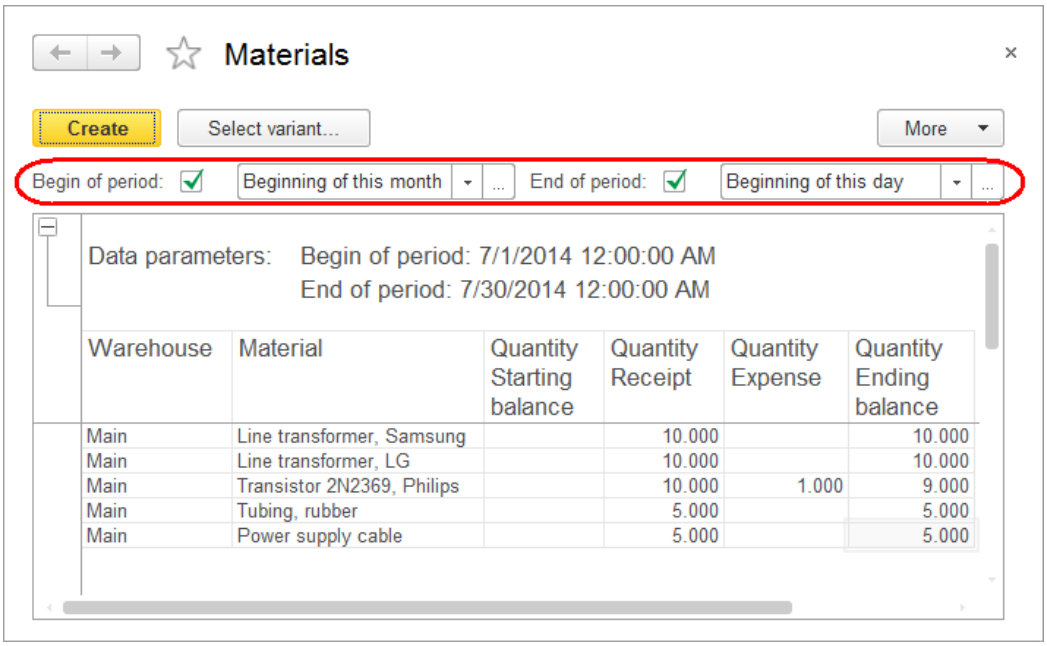


Fig. 7.16. Materials report

As you can see, the report looks good enough and fully reflects the movement of materials within your company.

Quiz

- What are the uses of the Report configuration object?
- How does one create a report using the data composition schema wizard?
- How does one add a report to various applied solution sections?

Lesson 8 (0:40). Templates. Editing templates

In this lesson you will study the Template configuration object. You will learn the object purpose and then you will create a document template that will serve as a basis for generation of a print document form.

Understanding templates

The Template configuration object is intended for storing various data presentation forms, which might be required by certain configuration objects or by the entire applied solution.

A template can store a spreadsheet or a text document, binary data, an HTML document or an Active Document, a graphical or geographical schema, a data composition schema, or an appearance template for a data composition schema.

A template can exist independently (a common template), or it can be subordinate to some configuration object.

One of the uses of a template that is subordinate to a configuration object and contains a spreadsheet document is generating a print form for this object. The print form creation consists of laying out its component parts (named areas that are used to assemble a print form).

The methods of populating the areas with data and outputting them into the final form are defined using 1C:Enterprise script. A print form can include various graphic objects: images, OLE objects, charts, and more.

In addition to manual template creation, Designer provides developers with the print wizard, which simplifies the template creation.

Print form template

In Designer mode

Let us create a print form for the Services document.

1. In Designer, open the configuration object editor for the **Services** document.
2. Click the **Templates** tab, click **Wizards**, and then click **Print wizard** (fig. 8.1).

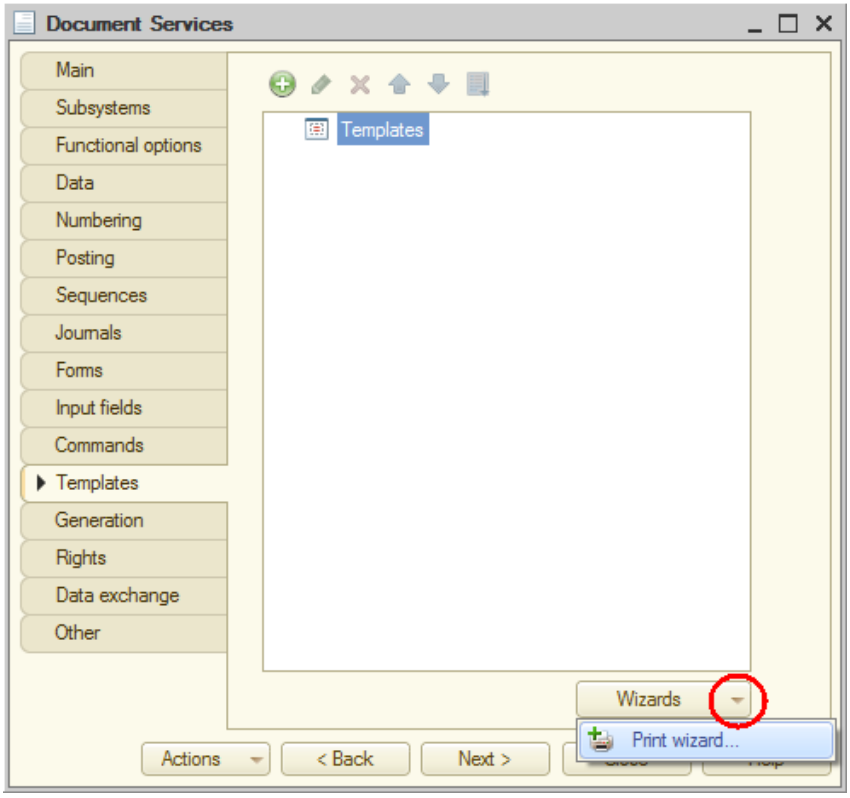


Fig. 8.1. Starting the Print wizard

3. On the first page, keep the default settings and click **Next**.
4. Click **Create new command** (fig. 8.2).

   By doing this you specify that the wizard will create the Print command that generates the document print form.

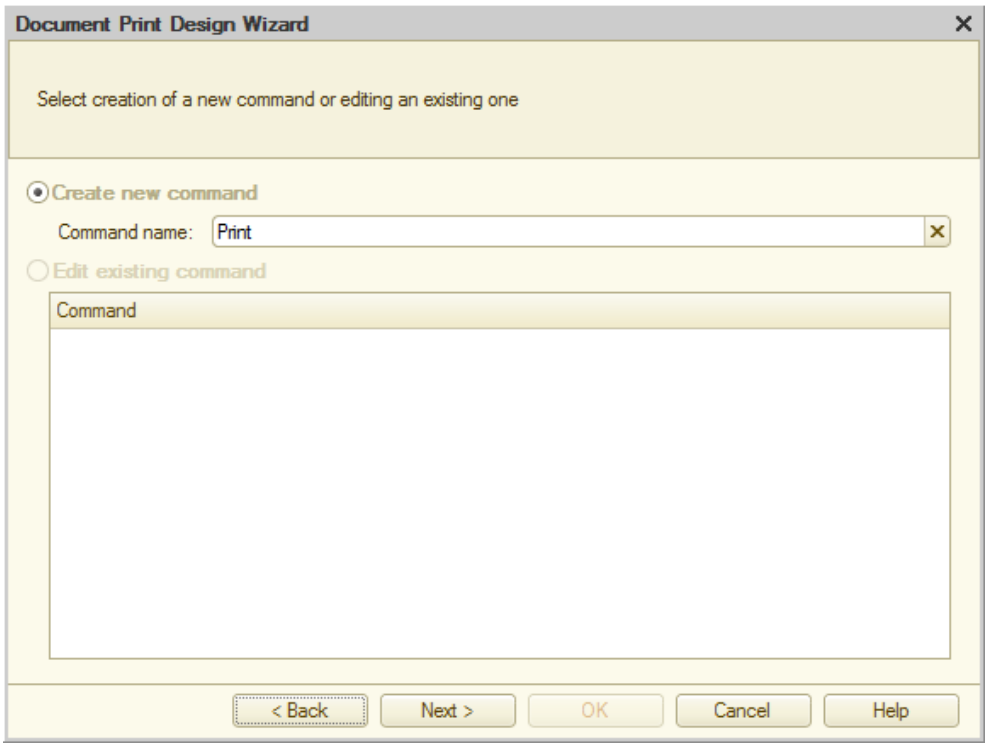


Fig. 8.2. The Print wizard, step 1

5. Click the >> button (fig. 8.3) and click **Next**.

   By doing this you specify that all of the document attributes will be displayed in the print form header.

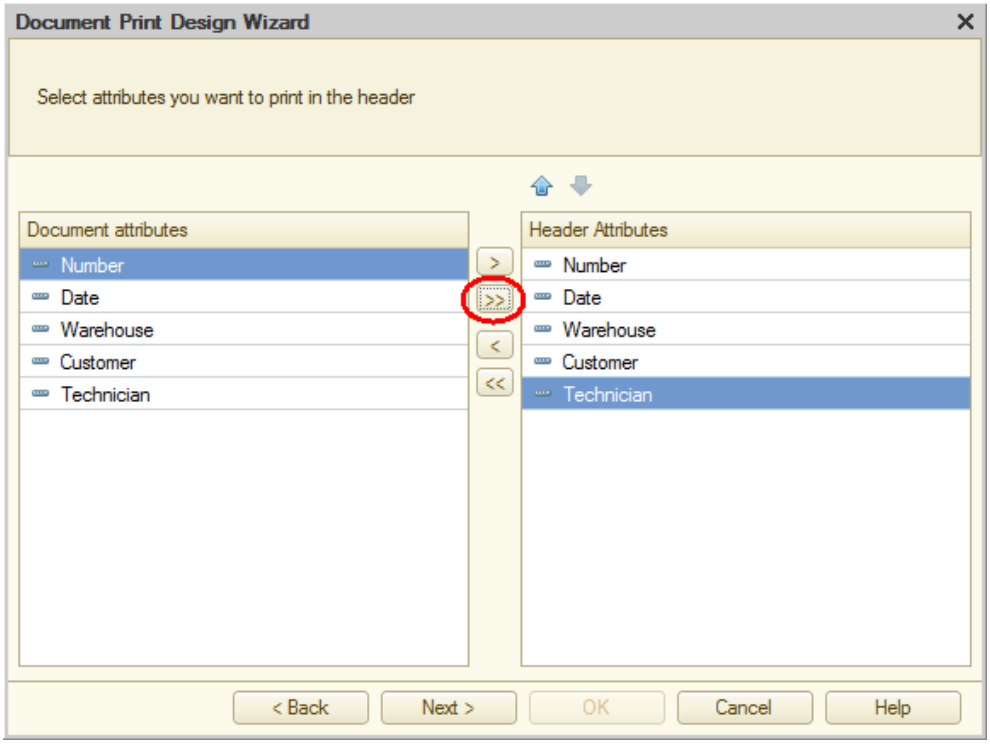


Fig. 8.3. The Print wizard, step 2

6. Click the >> button (fig. 8.4) and click **Next**.

   By doing this you specify that all of the document tabular section attributes will be displayed in the print form.

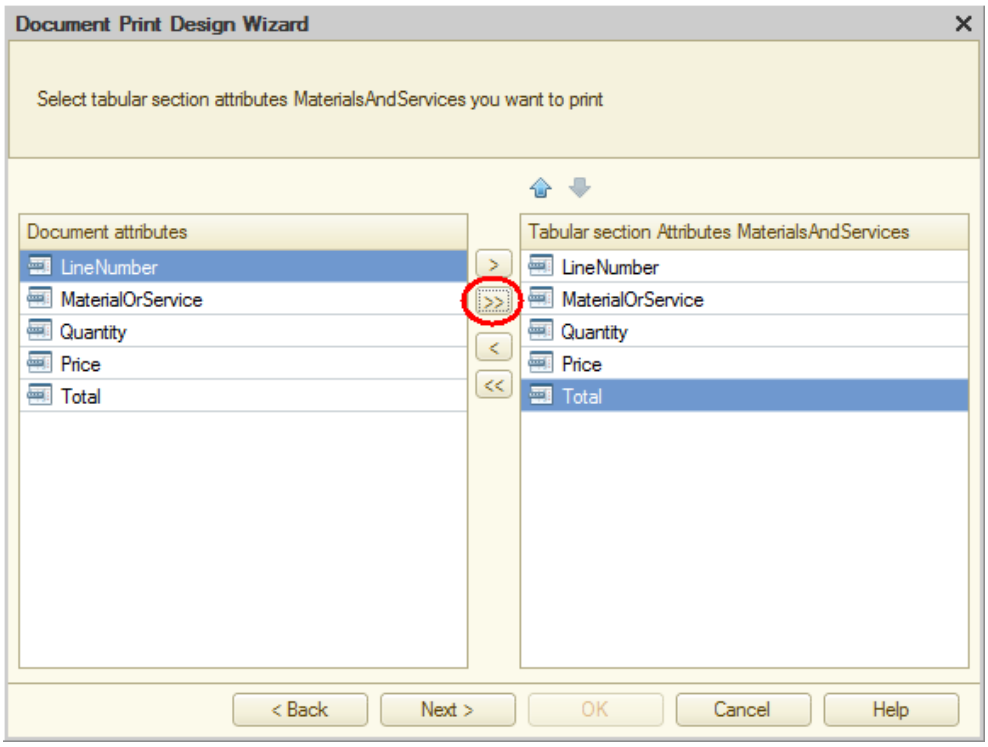


Fig. 8.4. The Print wizard, step 3

Then the wizard prompts you to generate a footer (the bottom part) of the print form.

7. Keep the default settings and click **Next**.

   The default settings specify that the print form does not have a footer. You actually do not need a footer in this form. The wizard proceeds to the next step (fig. 8.5).

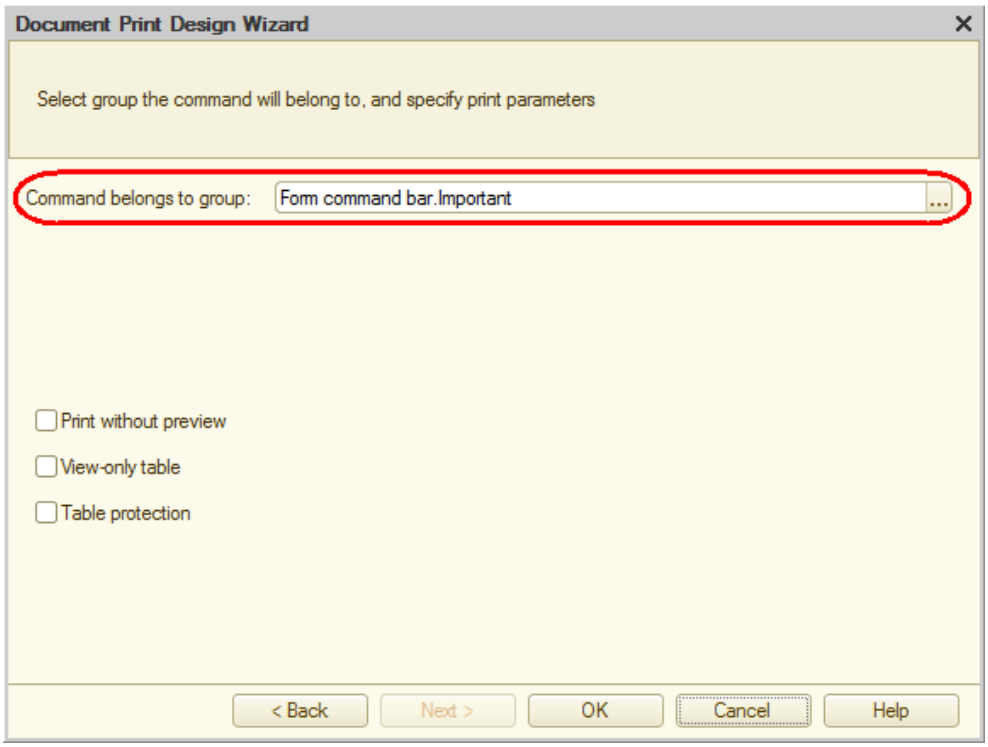


Fig. 8.5. The Print wizard, step 5

8. Leave the default settings and click **Next**.

   The default settings specify that the command that generates the print form is added to the form commands panel, to the Important section.

9. Click **OK**.

   Designer opens the module of the Print command, the module of the Services document manager, and the template of this document (fig. 8.6).

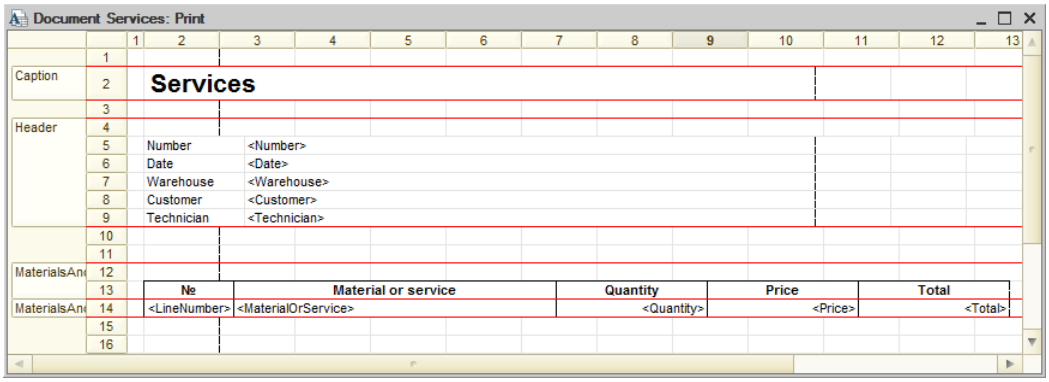


Fig. 8.6. The Services document template

Note that you have the option to create a print form template, the respective command, and the button in the document form manually. But in this scenario the Print wizard does all the work:

- The print form template named Print is created for the Services document (see fig. 8.6).
- The Print command is created for the Services document. The module of this command includes a handler that calls the document print procedure, which is executed on the server. The print procedure itself is located in the Services document manager module (fig. 8.7).

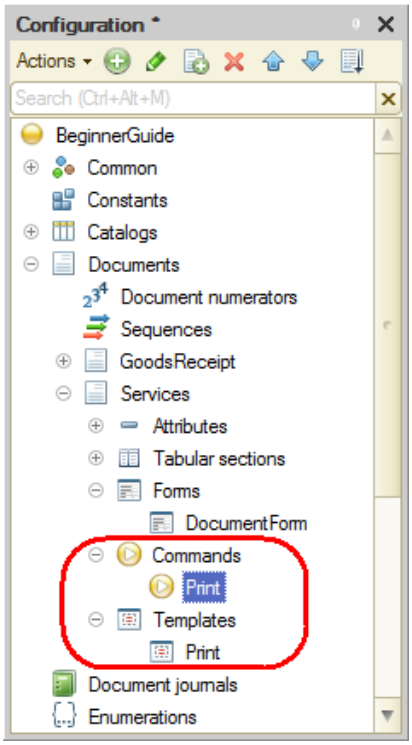


Fig. 8.7. The structure of the Services document in the configuration object tree

- The Print command that generates the document print form is added to the command bar of the Services document (fig. 8.8).

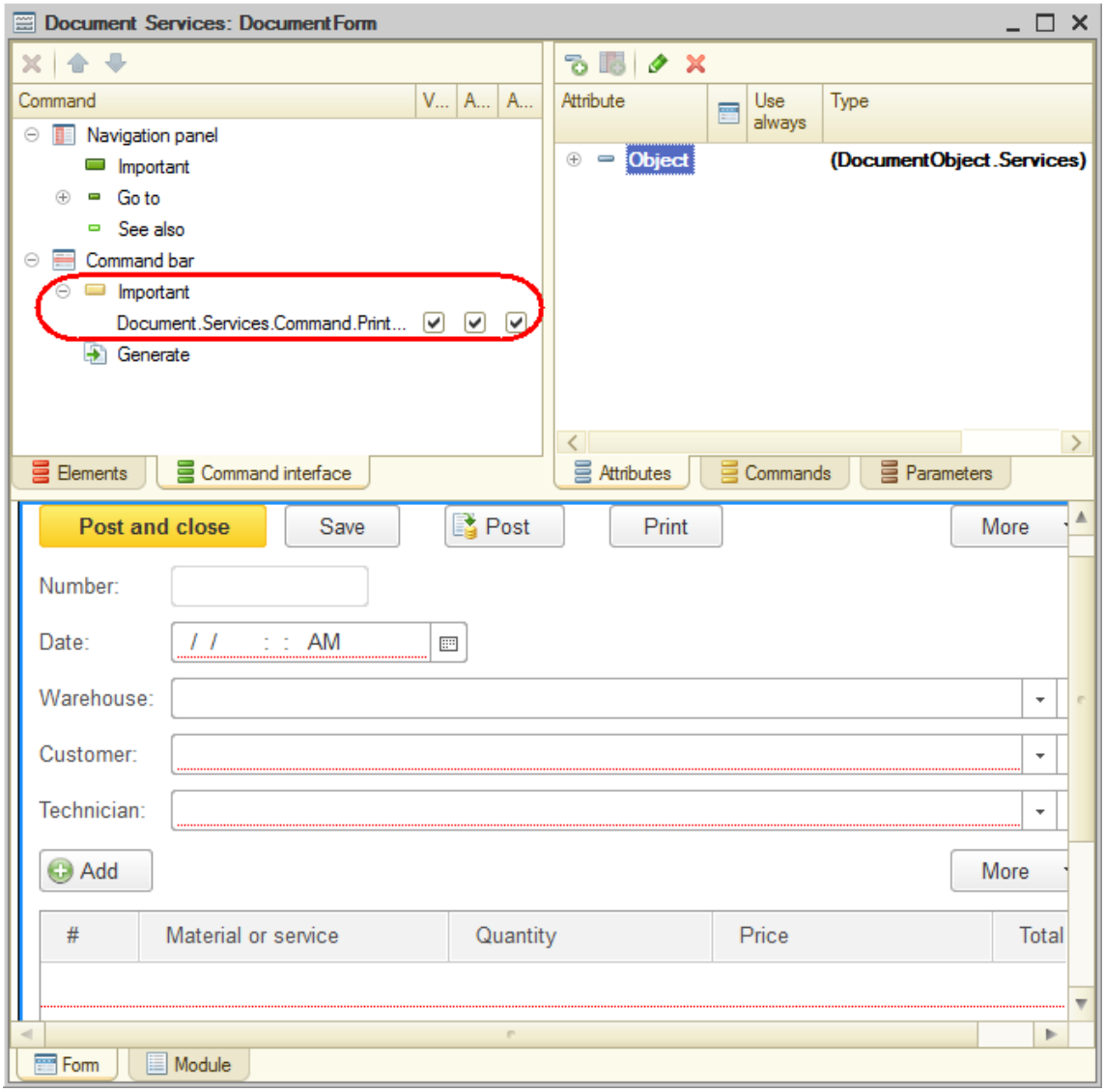


Fig. 8.8. The Services document form

Since the Print command belongs to the entire Services document instead of a specific form of this document, this command can be added to any form created for this document.

In the future you will create command handler procedures manually and add their respective buttons to the form but for now let us use the result of the print wizard.

In 1C:Enterprise mode

Let us test the template.

1. Start 1C:Enterprise in the debug mode and open the first Services document.

   Note that the document commands panel now has a new **Print** button (fig. 8.9).

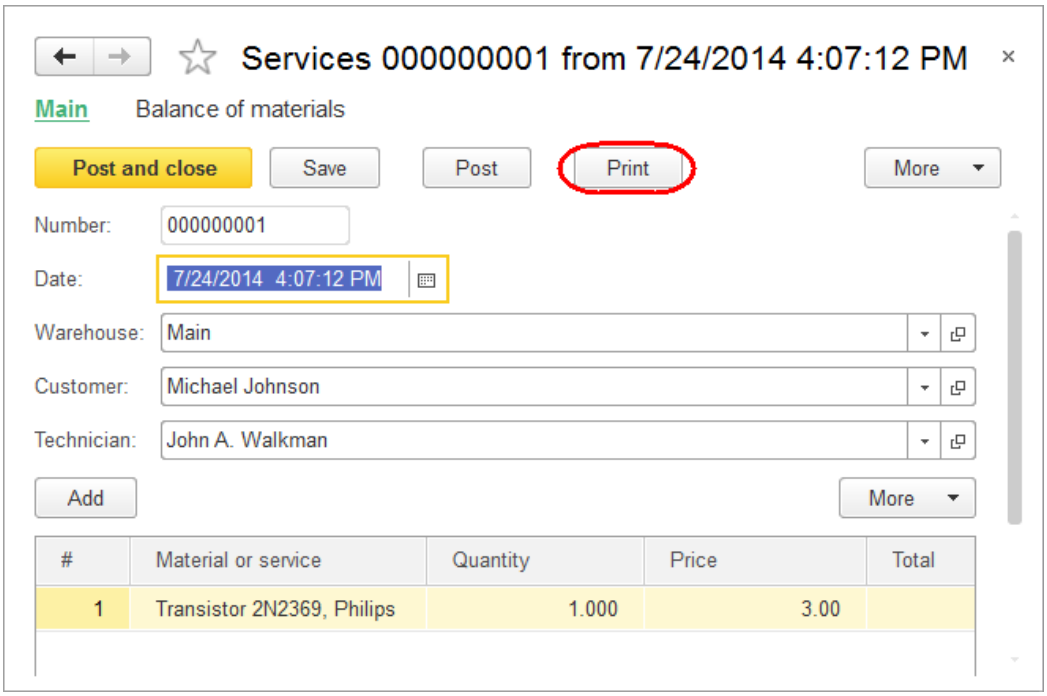


Fig. 8.9. The Services document form with a new Print button

The **Print** button is also added to the commands panel of the document list form. This is handy because you do not have to open a document to print it. You can simply select the document in the list and click **Print**.

2. Click the **Print** button either in the document or in the document list.

   It opens the print form of the document (fig. 8.10).

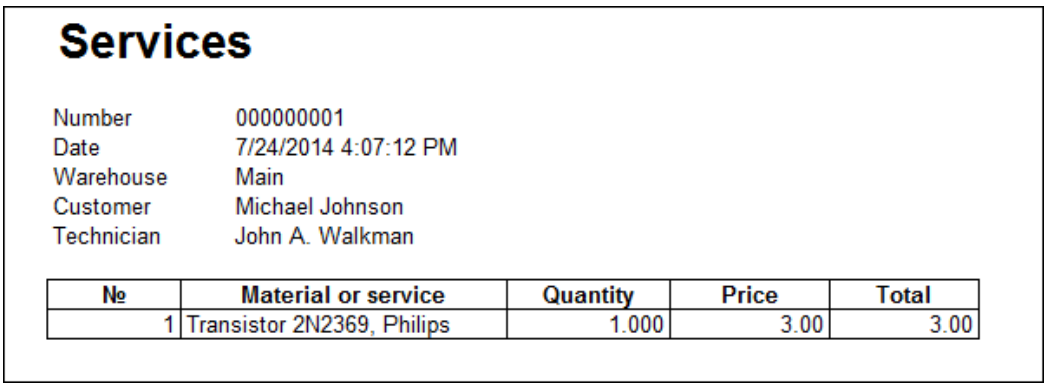


Fig. 8.10. The print form of the Services document

As you can see, the wizard generated a perfectly usable print form. The only thing this form lacks is the total amount for the document. In the next section you will learn how to edit forms and templates of configuration objects on the example of adding totals.

Editing a template

In Designer mode

First of all, let us add the total amount to the print form of the Services document.

1. Open Designer, expand the **Services** document branch, and double-click the **Print** template.

   As you can see, the document template consists of named areas that are printed in a certain order.

   The named areas listed at the left are generated by the wizard. You can create or delete areas, rename them, and so on.

   Let us create a new area for displaying the total.

2. Select two empty rows under the document tabular section and then, on the **Table** menu, point to **Names** and click **Set name** (fig. 8.11).

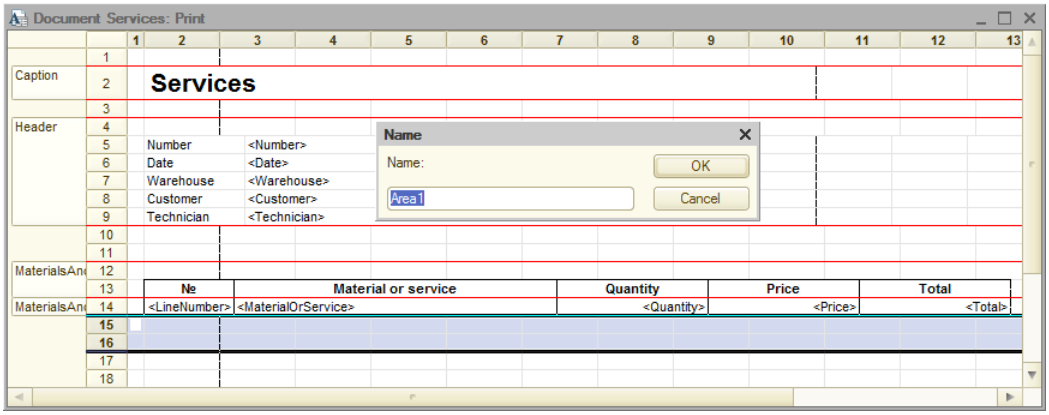


Fig. 8.11. Creating a cell area for displaying totals

3. In the **Name** field, type *Total* and click **OK**.

   To have the format of the added rows match the format of the document header and tabular section, let us change the column width.

4. Drag the right border of column 2 in the spreadsheet header to align it with the **№** column.

   The platform prompts you to create a new format for the selected lines.

5. Confirm the creation of the new format.

6. Perform steps 4-5 for columns 3, 4, 5, and 6.

7. Within the created area, under the **Price** column, enter *Total*, and then, under the **Total** column, enter *DocumentTotal* (fig. 8.12).

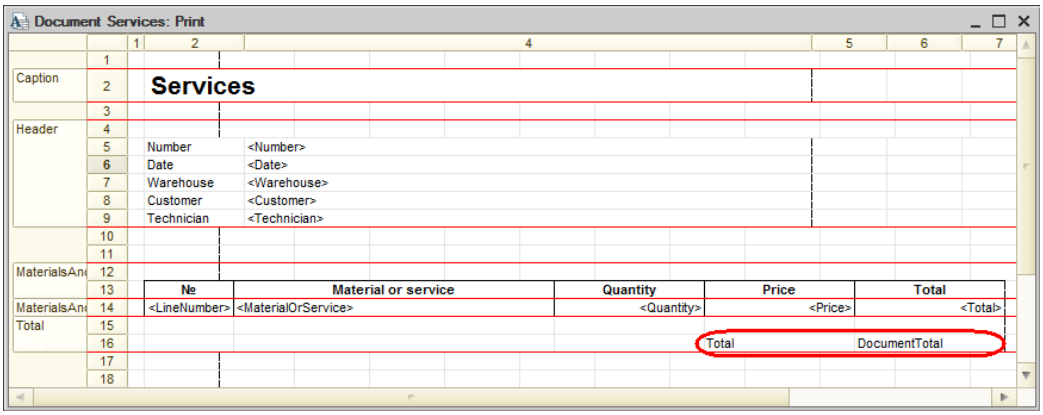


Fig. 8.12. Filling cells that display totals

8. Right-click the **DocumentTotal** cell and click **Properties**.

   This opens the cell property palette.

9. In the **FillType** property, select **Parameter**.

   This indicates that the cell contains a parameter rather than text (fig. 8.13).

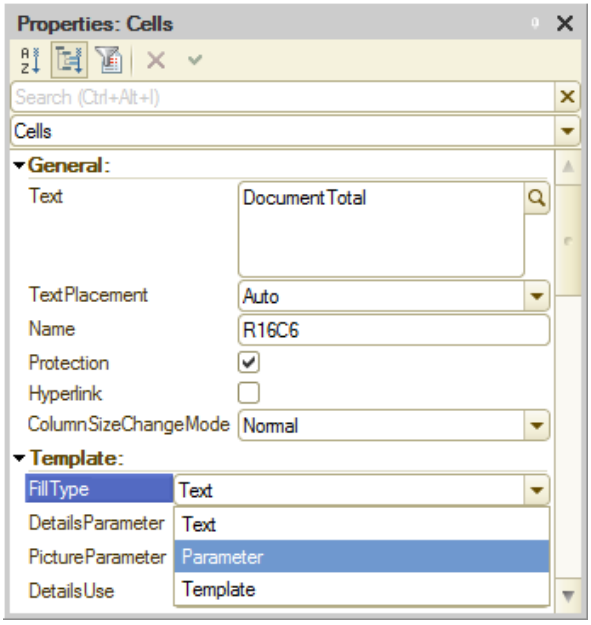


Fig. 8.13. Properties of the DocumentTotal cell

Each cell of the spreadsheet document can contain either a text, or a parameter, or a template.

A text is displayed on the screen as is.

A parameter is replaced by a value assigned to it using 1C:Enterprise script. The cell text is actually the parameter name.

A template is a text string with parameter values at predefined locations.

So, by setting the cell property to **Parameter**, you defined the DocumentTotal parameter of the area, which is replaced by a value when the print form is generated.

Let us open the Services document manager module.

10. In the configuration object editor of the Service document, click the **Other** tab and then click **Manager module** (fig. 8.14).

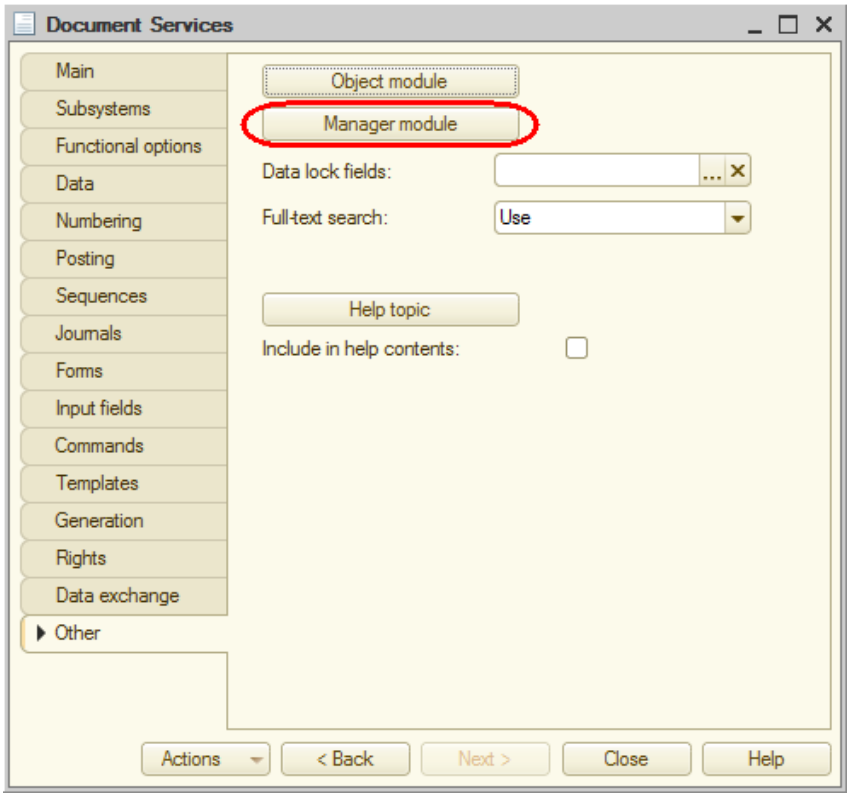


Fig. 8.14. Opening the manager module of the Services document

11. In the module, find the Print procedure and edit it as shown in listing 8.1 (new lines are marked with comments).

Listing 8.1. Printing a document form (fragment)

```
...
AreaCaption = Template.GetArea("Caption");
Header = Template.GetArea("Header");
AreaMaterialsAndServicesHeader =
    Template.GetArea("MaterialsAndServicesHeader");
AreaMaterialsAndServices = Template.GetArea("MaterialsAndServices");
AreaTotal = Template.GetArea("Total"); // New line
Spreadsheet.Clear();

InsertPageBreak = False;
While Selection.Next() Do
    If InsertPageBreak Then
        Spreadsheet.PutHorizontalPageBreak();
    EndIf;

    Spreadsheet.Put(AreaCaption);

    Header.Parameters.Fill(Selection);
    Spreadsheet.Put(Header, Selection.Level());
```

```
    Spreadsheet.Put(AreaMaterialsAndServicesHeader);
    SelectionMaterialsAndServices =
        Selection.MaterialsAndServices.Select();


    TotalSum = 0; // New line
    While SelectionMaterialsAndServices.Next() Do
        AreaMaterialsAndServices.Parameters.Fill(
            SelectionMaterialsAndServices);
        Spreadsheet.Put(AreaMaterialsAndServices,
            SelectionMaterialsAndServices.Level());
        TotalSum = TotalSum +
            SelectionMaterialsAndServices.Total; // New line
    EndDo;


    AreaTotal.Parameters.DocumentTotal = TotalSum; // New line
    Spreadsheet.Put(AreaTotal); // New line
    InsertPageBreak = True;
EndDo;
...
```

The meaning of the added code is simple: it calls the Services document template using its name (Template).

The Total area (the one you have added to the template) is obtained using the GetArea() method and then saved to the AreaTotal variable.

In the loop that iterates through the document tabular section rows obtained as a result of query execution, the sum of the values in the Total column of the tabular section is calculated and stored to the TotalSum variable.

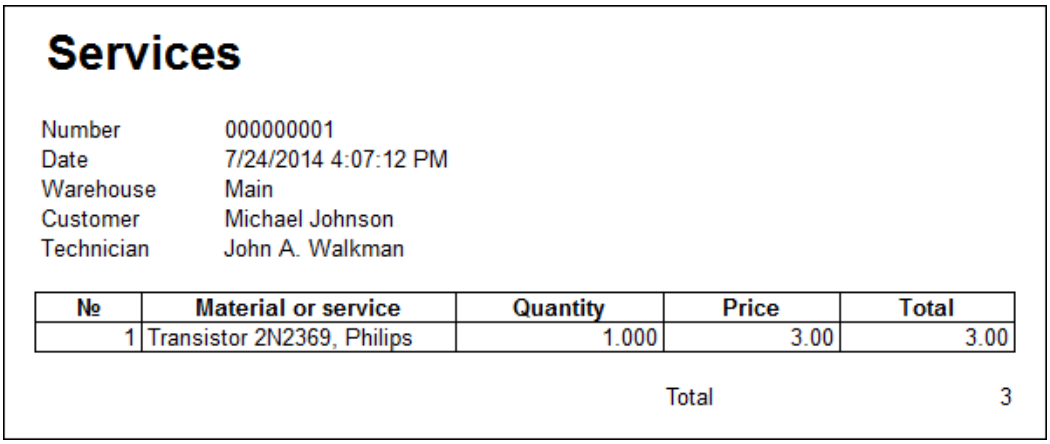
Then the value of the TotalSum variable is assigned to the DocumentTotal parameter (AreaTotal.Parameters.DocumentTotal), which is displayed in the Total area.

Finally, the total area is output to the spreadsheet document that will be displayed on the screen and printed out by the user (Spreadsheet.Put(AreaTotal)).

A spreadsheet document is displayed on the screen using the Print command handler in the client module of this command, while the print procedure itself is defined in the document manager module and executed on the server.

In 1C:Enterprise mode

Start 1C:Enterprise in the debug mode and test the result of your modifications (fig. 8.15).



Services

Number 000000001
Date 7/24/2014 4:07:12 PM
Warehouse Main
Customer Michael Johnson
Technician John A. Walkman

№	Material or service	Quantity	Price	Total
1	Transistor 2N2369, Philips	1.000	3.00	3.00

Total 3

Fig. 8.15. Print form of the Services document

A developer can create a print form with a custom layout in a similar manner by using named areas and template cells, editing their properties, and managing their output order using 1C:Enterprise script tools.

Quiz

- What are the uses of the Template configuration object?
- What is a print wizard?
- How does one create a template using the print wizard?
- How does one modify a spreadsheet document?
- What is the difference between populating cells of a spreadsheet document with text, parameters, or templates?
- How does one use 1C:Enterprise script to insert an area into a spreadsheet document?

Lesson 9 (0:50). Periodic information registers

In this lesson you will study the Information register configuration object on the example of a specific information register type, a periodic information register. You will learn the object purpose and structure.

Then you will create a periodic information register and learn how to process data stored in the register using 1C:Enterprise script tools.

Information register purpose

Let us look at the Service document. As you remember, in this document you can select a rendered service and enter its price.

Obviously, the Jack of All Trades company has a price list for its services. It would seem that the price of services is an integral part of the services themselves and should be included as an attribute of the MaterialsAndServices catalog.

However, service prices can change over time. And you might need to make changes or corrections to previously posted Services documents. In this scenario you will be unable to get the correct price for the service because the catalog attribute stores the latest value.

Moreover, the Jack of All Trades management might request a report showing the dependency of the company profit on the service prices. So the ability to analyze the price changes over time is essential.

Therefore, to store the service prices, you will use a new object: *Information register*.

Understanding information registers

The Information register configuration object is intended to describe a multidimensional data storage structure. The platform uses the Information register configuration object as the basis for creation of database tables that store custom multidimensional data (fig. 9.1).

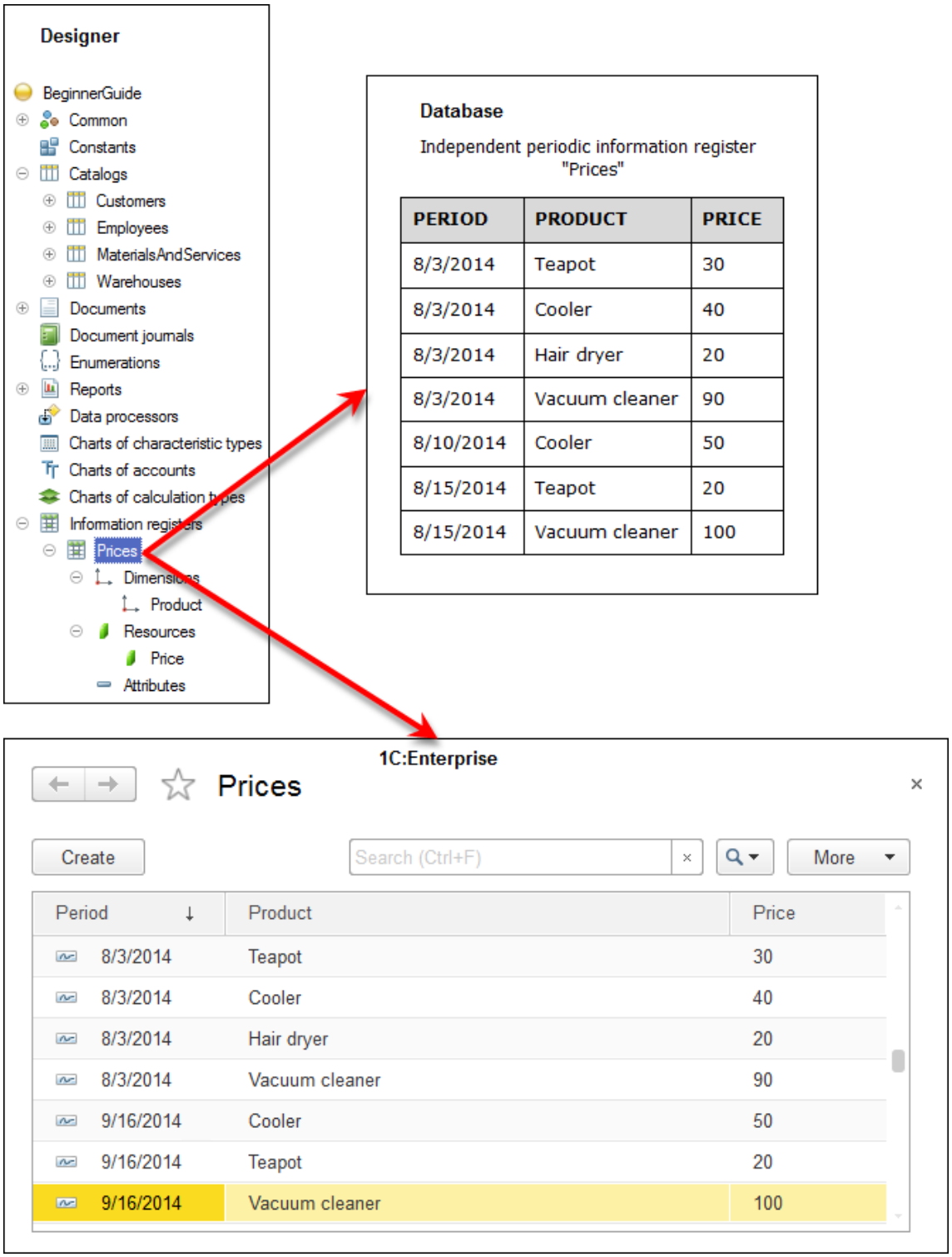


Fig. 9.1. The Prices independent periodic information register in Designer and in the database

The main difference between an information register and an accumulation register is the fact that each information register record sets a new resource value, while each accumulation register record modifies the current resource value. Because of this difference, an information register can store data of any type (rather than just numeric, as was the case with an accumulation register).

Another important characteristic of an information register is its ability to store data with time stamps if required. So it can store not only the current

data values but also their changes with time. If an information register uses time stamps, it is referred to as a *periodic information register*.

The information register periodicity can have one of the following values:

- Within a second
- Within a day
- Within a month
- Within a quarter
- Within a year

By recorder position (if the write mode is set to Subordinate to recorder)

Periodic information registers always contain the system Period field, which is added automatically. It has the Date type and specifies the period where a record belongs. When writing data to a register, the platform changes the value of the Period field to the beginning of the register period that includes that value.

For example, if data is written to an information register with periodicity "within a month" and the period for that data is set to 4/8/2014, the register stores the data with the period value 4/1/2014.

As with other registers, the platform ensures the uniqueness of information register records. However, while other registers use the recorder and row number as unique identifiers, information registers use a different key value generation method.

An information register record key, which uniquely identifies a record, consists of all register dimensions and a period (in case of a periodic information register). For example, the key of the periodic information register that has the Product dimension and the Price resource (see fig. 9.1) is a set of Period and Product field values. An information register cannot contain multiple records with identical keys.

Compared to an accumulation register, an information register offers greater freedom in editing the data it contains. In addition to information registers that are subordinate to recorders (their records are "bound" to the recording documents), information registers that can be freely edited by users are available. An information register that is not subordinate to a recorder is referred to as *independent information register*.

Learn more! For details on the structure of 1C:Enterprise script objects intended for information register operations, see section "Quick developer reference. Information registers" on page 857.

Adding a periodic information register

Let us create a periodic information register that will store the retail prices of materials and the prices of services provided by Jack of All Trades, together with the periods when the prices are effective.

In Designer mode

Let us create the information register.

1. Open your configuration in Designer.
2. In the configuration object tree, click the **Information registers** branch and then click the **Add** button in the configuration window command bar.

   This opens the configuration object editor window.
3. On the **Main** tab, in the **Name** field, type *Prices*.
4. In the **Periodicity** field, select **Within a second**.

   With this periodicity you can specify multiple prices during a single day. But if you are not planning to change prices that often, you can set **Periodicity** to **Within a day**.

   Now let us define the object presentation in the application interface.
5. In the **Record presentation** field, type *Price*.
6. In the **List presentation** field, type *Material and service prices* (fig. 9.2).

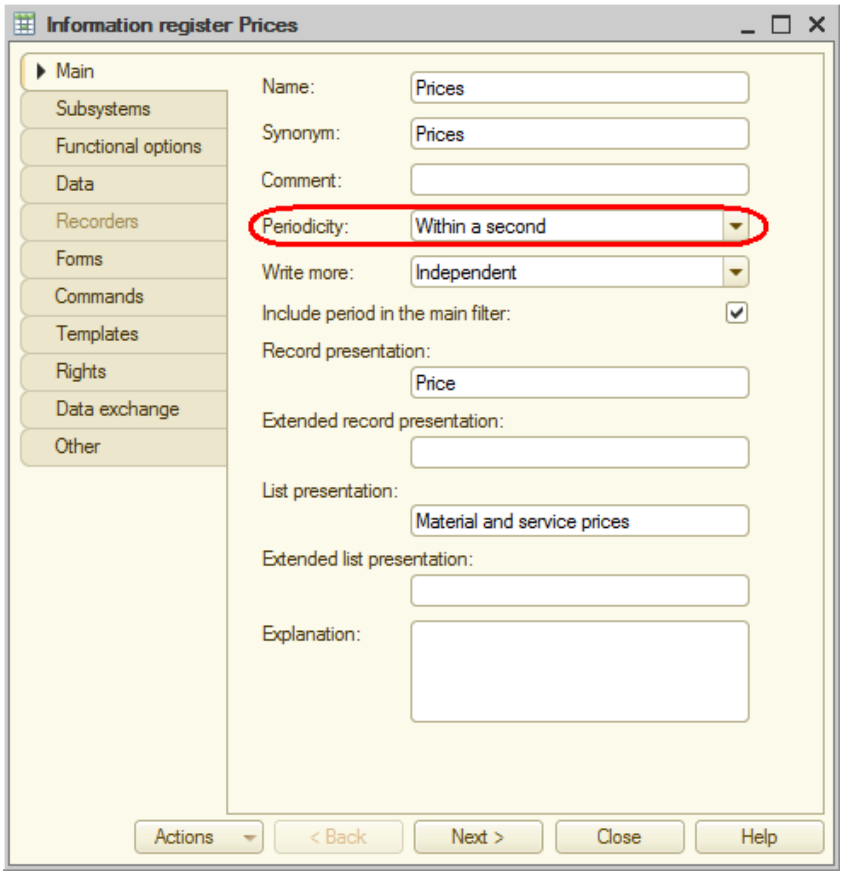


Fig. 9.2. Specifying main information register settings

Note the **Write mode** property of the information register. By keeping the default **Independent** value, you create an independent information register. Later you will be able to enter register data directly, without using a recorder.

7. Click **Next**.

   This opens the **Subsystems** tab. The logic of your configuration suggests that the register should be available in the Accounting, Inventory, and Services sections.

8. Select the **Accounting**, **Inventory**, and **Services** check boxes (fig. 9.3).

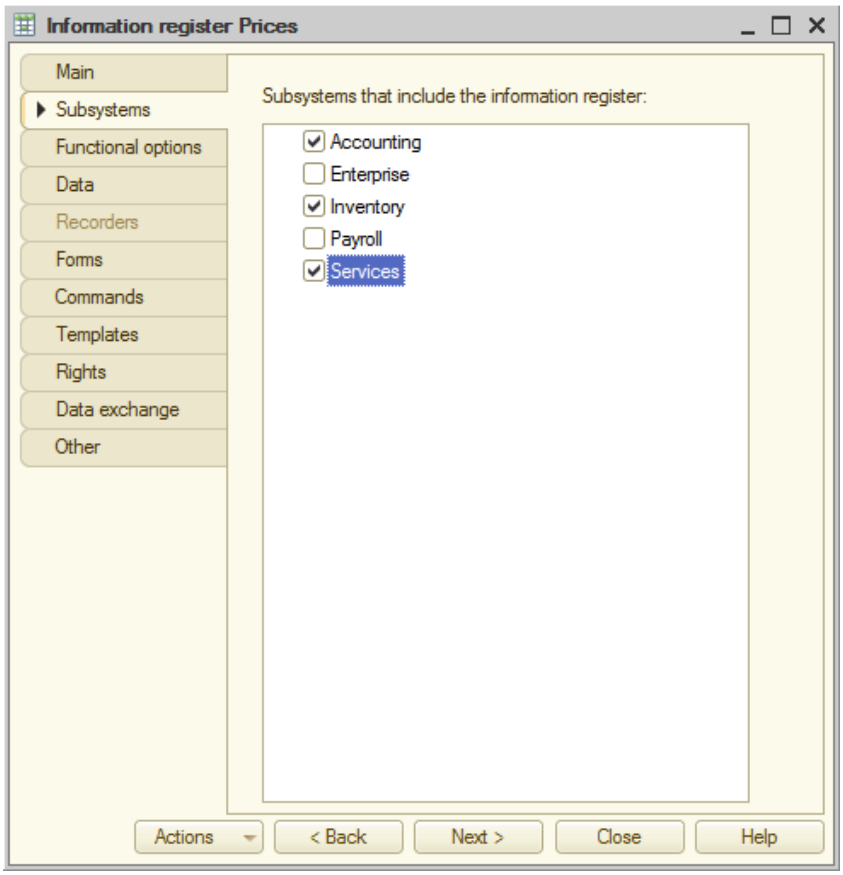


Fig. 9.3. Specifying subsystems where a register is available

Dimensions and resources

Let us specify the register dimensions and resources.

1. Click the **Data** tab.
2. Click the **Dimensions** branch, click the **Add** button in the command bar of the window, and create the **MaterialOrService** dimension with the **CatalogRef.MaterialsAndServices** type.
3. In the dimension properties, select the **Master** check box (fig. 9.4).

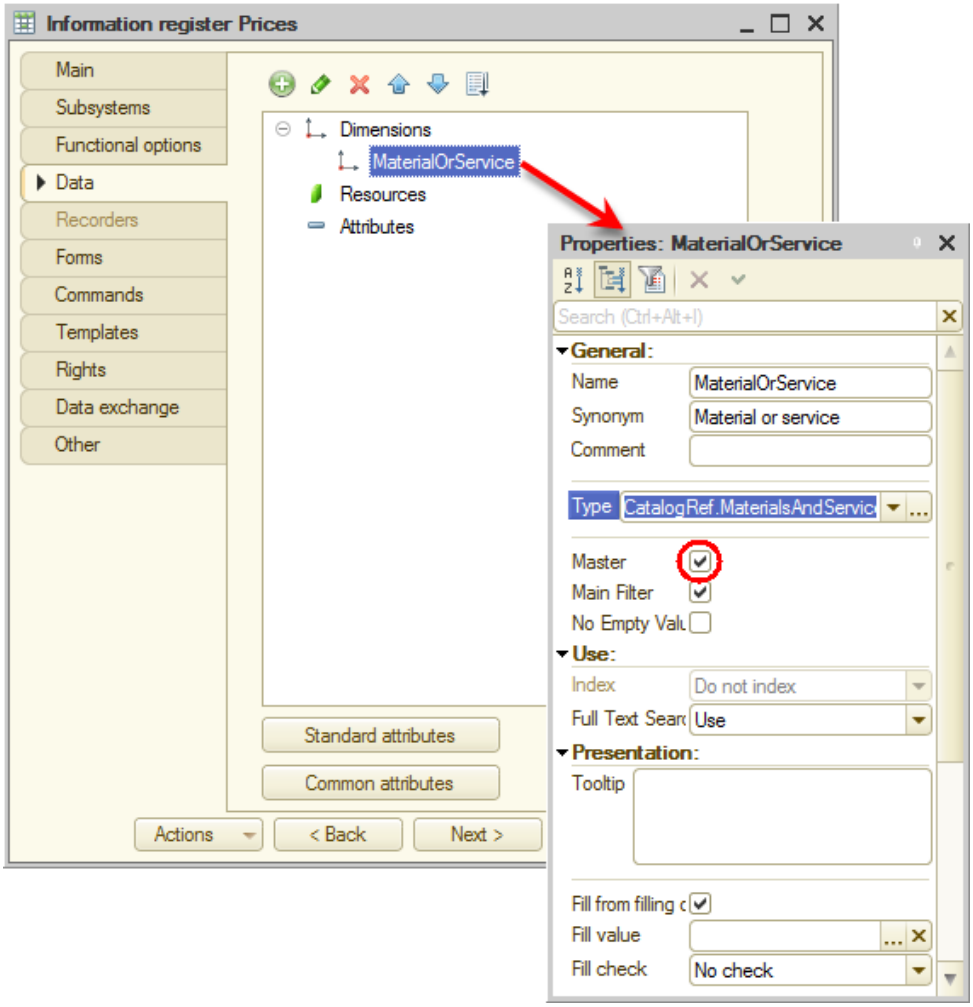


Fig. 9.4. Creating a master register dimension

The Master property makes sense for dimensions whose type is a reference to a database object. If this property is set to True, the register record is considered meaningful only while the database object is available. Once the object is deleted, the platform deletes all information register records related to that object.

Also, once you set the Master property to True, a link is added to the navigation panel of the Materials and services catalog item form. Clicking this link opens the register records where the MaterialOrService dimension value is a reference to that catalog item.

4. Click the **Resources** branch and click the **Add** button in the command bar of the window (fig. 6.5) to create the following resource:
   - **Price**. Type: Number, length: 15, precision: 2, non-negative.

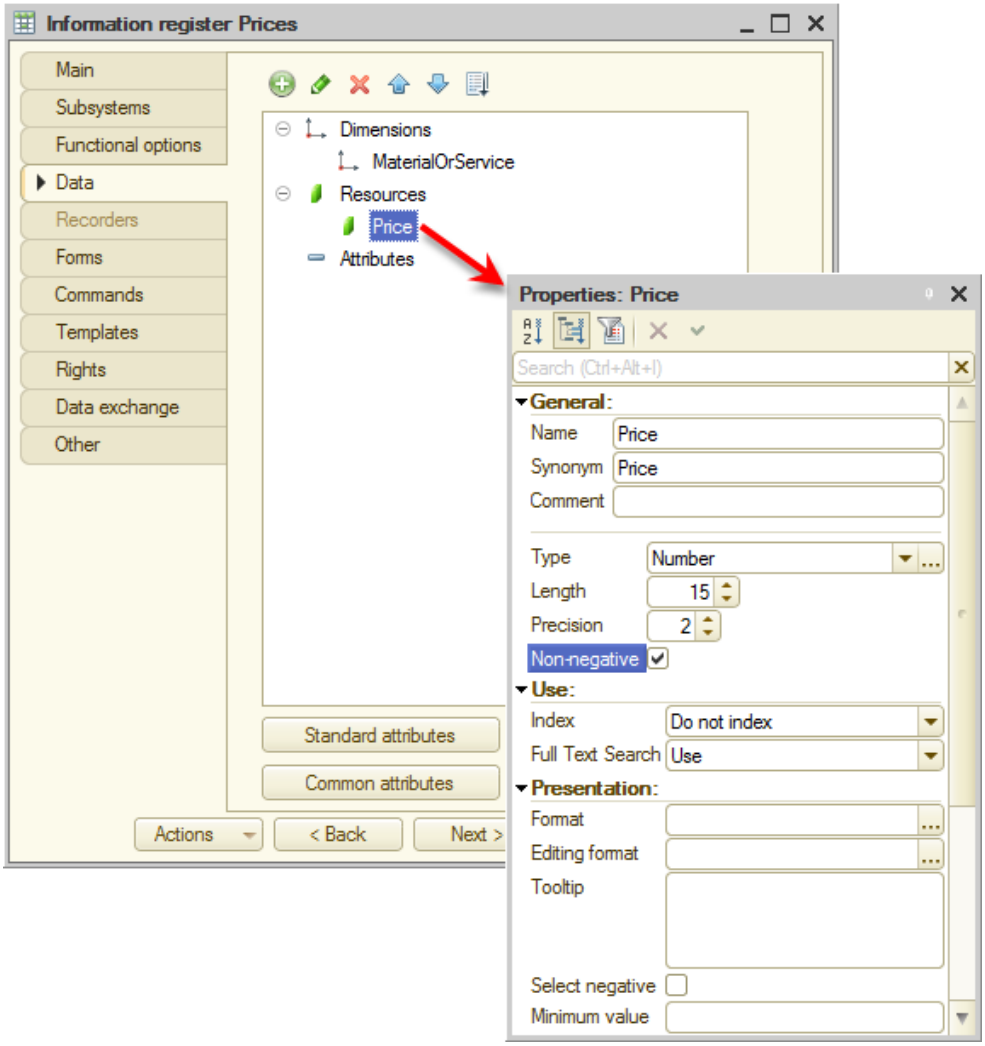


Fig. 9.5. Creating an information register resource

In 1C:Enteprise mode

Let us test the Prices information register.

- Start 1C:Enterprise in the debug mode.

  You can see that the Material and service prices command that opens the register list is added to the Inventory, Services, and Accounting subsystems (fig. 9.6).

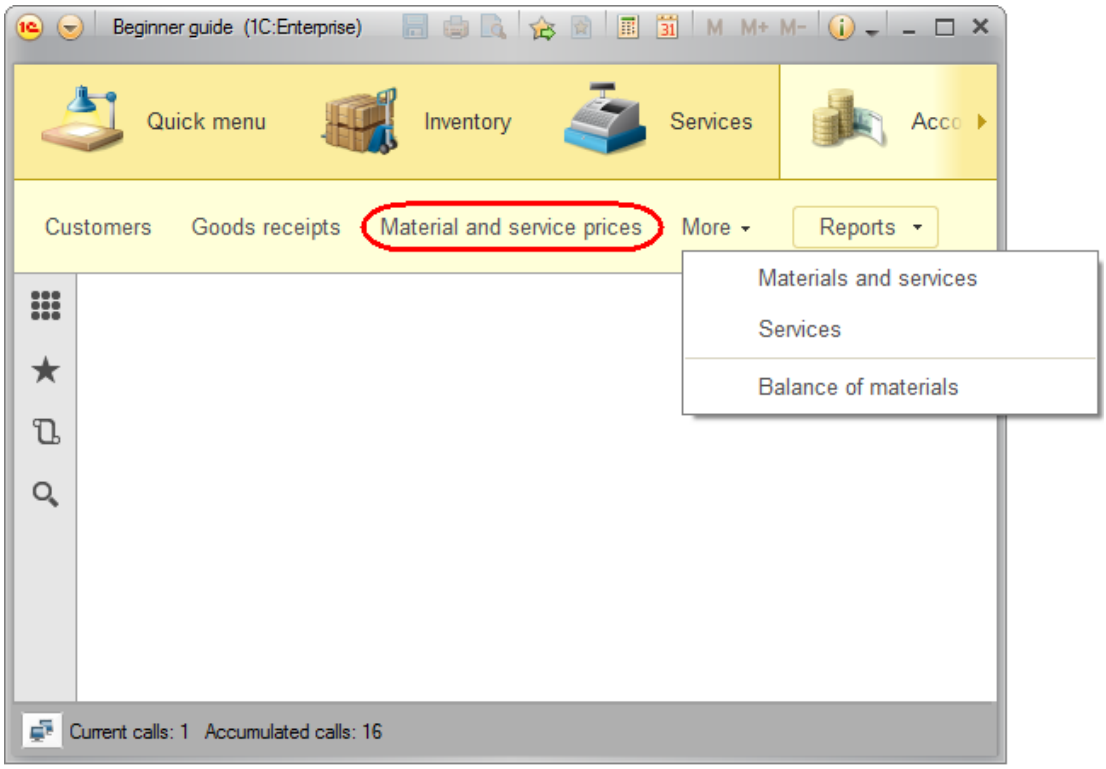


Fig. 9.6. Command that opens the periodic information register

The command that opens the information register is available by default in the interface of sections where the register is available because users should be able to modify information register data (note that commands that open accumulation registers are not available by default because users should not modify their data). Some command names that do not fit the current section command panel are moved to the **More** menu.

Creating information register records

Let us create records in the Material and service prices information register.

1. Click **Material and service prices**.
2. Click **Create**.
3. Specify the service prices as shown in fig. 9.7.

   Note that you have to enter periods that are less or equal to the Services document creation date (in the example used in this book the document was created on 7/24/2014).

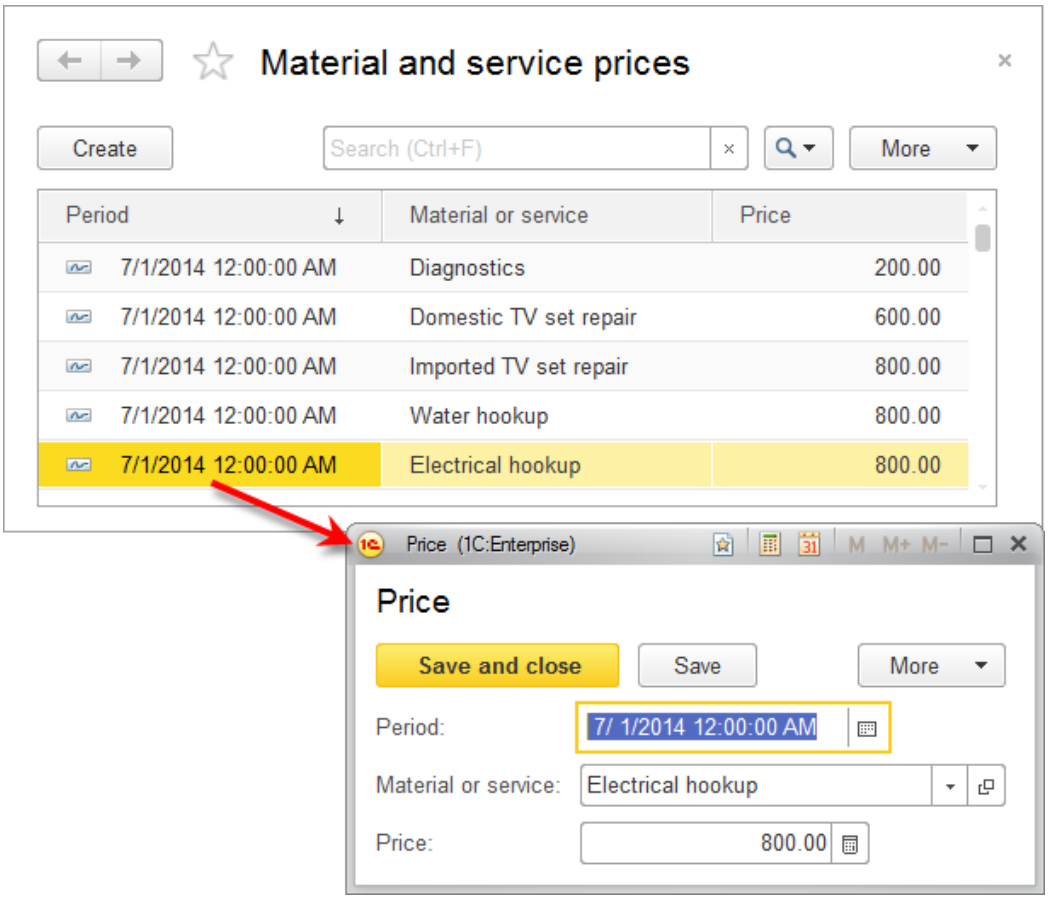


Fig. 9.7. Service prices in the Prices information register

4. Specify the material retail prices as shown on fig. 9.8.

Period	Material or service	Price
7/1/2014 12:00:00 AM	Line transformer, Samsung	900.00
7/1/2014 12:00:00 AM	Line transformer, LG	400.00
7/1/2014 12:00:00 AM	Transistor 2N2369, Philips	5.00
7/1/2014 12:00:00 AM	Tubing, rubber	150.00
7/1/2014 12:00:00 AM	Power supply cable	30.00

Fig. 9.8. Material prices in the Prices information register

So now your applied solution features setting prices for materials and services. Since the prices have time stamps, you can set new prices in advance and be certain that they take effect on the specified date.

Filling prices in documents upon selecting materials or services

Let us proceed to the next goal. The material and service prices are stored in an information register. When a user creates a Services document or modifies a list of services within a Services document, it is good to have the

service price added or updated automatically based on the information register record that matches the document date.

This includes two steps. First, write a function that returns the current material or service price. Second, call the function when a service is added to the document and fill the price field with the returned value.

This feature will hardly be used exclusively in this document. You might need it for the development of other documents that contain materials and services in their tabular sections. So let us store the function to a place that can be accessed from any applied solution part: a common module.

In Designer mode

Creating a function that returns material or service price

First, let us create the RetailPrice() function that returns the current material or service retail price, and let us store it to a common module.

1. In Designer, in the configuration object tree, in the **Common** / **Common modules** branch, add a Common module configuration object and name it *CatalogProcessing*.

   In the module property palette the **Server** check box is selected by default. This means that the module instances are only compiled on the server.

2. Select the **Server call** check box.

   This allows calling export procedures and functions of the module from the client (fig. 9.9).

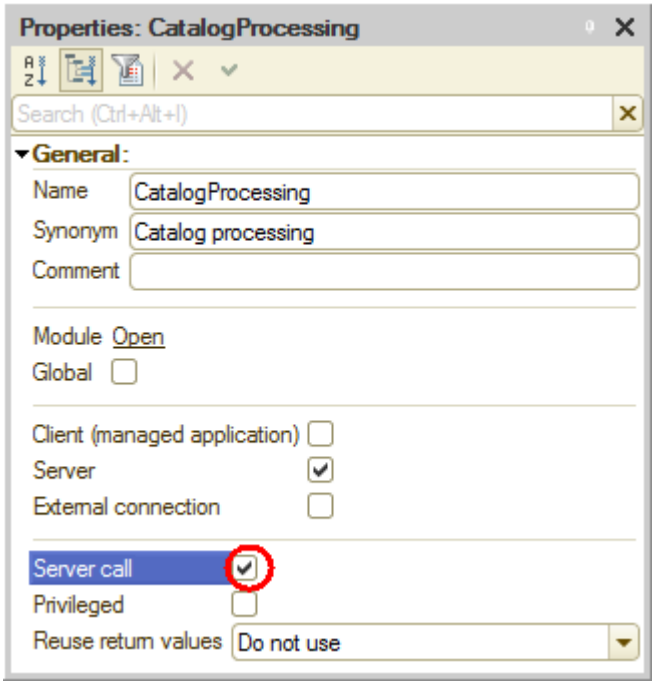


Fig. 9.9. Common module properties

3. Add the following script to the module (listing 9.1).

Listing 9.1. Function RetailPrice()

```
Function RetailPrice(EffectiveDate, MaterialOrServiceItem) Export

    //Creating auxiliary Filter object

    Filter = New Structure("MaterialOrService",
        MaterialOrServiceItem);


    //Getting effective register resource values

    ResourceValues =
        InformationRegisters.Prices.GetLast(EffectiveDate, Filter);

    Return ResourceValues.Price;

EndFunction
```

Now let us examine this script.

The function has two parameters:

- EffectiveDate, type: Date. The point on the time axis for which you want to know the retail price.
- MaterialOrServiceItem, type: reference to an item of the MaterialsAndServices catalog. The item whose price you want to know.

The first line in the function body creates the auxiliary Filter object.

It is a structure that contains a filter by register dimensions. It includes the register records where the MaterialOrService register dimension is equal to the catalog item reference passed to the function.

The structure key name MaterialOrService must be identical to the register dimension name that you specified in Designer. The structure item value MaterialOrServiceItems is the value filtered by the specified dimension.

The second line calls the manager of the Prices information register (InformationRegisters.Prices) and executes its method GetLast(), which returns the resource values for the latest register record that corresponds to the date passed to the function (EffectiveDate) and for the specified register dimension values (Filter).

The GetLast method retrieves a structure that contains the resource values. This structure is stored to the ResourceValues variable. In general a register can have multiple resources. The Prices register has a single resource, so the method retrieves a structure with a single element.

This is why in the next line you can get the required retail price by specifying the register resource name after a dot (ResourceValues.Price). The function returns this value.

Now let us have this function called at some point in document processing algorithms.

Calling the function upon material or service selection for filling the price

Your next task is implementing automatic filling of the Price field once a user selects a service in the Services document. And the service price must be based on the document date.

1. In Designer, find the **Services** document and open its **DocumentForm**.
2. Double-click the **MaterialsAndServicesMaterialOrService** form item.

   -OR-

   Right-click the **MaterialsAndServicesMaterialOrService** form item and then click **Properties**.

   This opens the property palette of that item.
3. Scroll down the property palette to find the **OnChange** event.

   This event occurs when the field value is changed.
4. In the **OnChange** field, click the **Open** button.
5. In the window that prompts you to select the handler type, keep the default **Create on client** option (fig. 9.10).

   This option indicates that you are creating a client handler of an event that is triggered by interactive user actions.

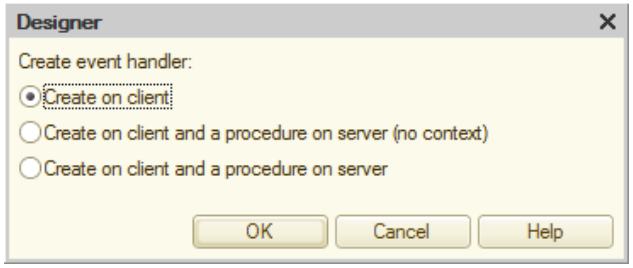


Fig. 9.10. Selecting handler type for a form event

   The platform generates a handler procedure template for this event in the form module and opens the **Module** tab of the form editor.
6. Add the following script to the handler procedure body (listing 9.2).

Listing 9.2. Procedure MaterialsAndServicesMaterialOrServiceOnChange()

```
// Getting current tabular section row
TabularSectionRow = Items.MaterialsAndServices.CurrentData;

// Setting price
TabularSectionRow.Price = CatalogProcessing.RetailPrice(Object.Date,
    TabularSectionRow.MaterialOrService);

//Recalculating row total
DocumentProcessing.CalculateTotal(TabularSectionRow);
```

Now let us examine this script.

You are already familiar with the first line of the handler after the MaterialsAndServicesQuantityOnChange and MaterialsAndServicesPriceOnChange procedures. It gets the current tabular section row of the document and stores it to the TabularSectionRow variable for future use.

The next line calls the RetailPrice() function from the CatalogProcessing common module.

The first function parameter is the document date, which is used to retrieve the price. The date is taken from the Object.Date default form attribute.

The second parameter is a reference to an item of the MaterialsAndServices catalog that is stored in the current tabular section row (TabularSectionRow.MaterialOrService).

The function returns the latest price value, which is assigned to the Price field of the current tabular section row (TabularSectionRow.Price).

The next line calls the CalculateTotal procedure from the DocumentProcessing common module. It is the procedure that you created earlier for recalculating the document row total when the quantity or price is changed.

Note that the MaterialsAndServicesMaterialOrServiceOnChange procedure execution starts in the form module on the client because it is a handler of an interactive form event. During the generation of the procedure template the platform added the &AtClient compilation directive before the procedure description.

Then the RetailPrice() function is called. Since this function is not found on the client, the execution is passed to the CatalogProcessing common module that is executed on the server. Once the function execution is completed, the script execution continues on the client.

So why do we need this trick? Why is it necessary to pass the script execution to the server?

It is because all database operations (reading or writing data) can only be performed on the server. Here you need to read the latest data for some material or service from the information register.

This action can only be performed on the server, and if you check the description of the GetLast() method of an information register in the Syntax Assistant, you will see that this method is only available on the server, in the thick client, and in external connections.

Thick client and external connection are client applications of the previous platform version that are supported for compatibility with legacy applied solutions.

But you are developing a new applied solution that uses the thin client or the web client. So in order to get some data from the database you need to pass the script execution to the server, get the required data there, and return the data to the client. And this is exactly what the handler does.

In 1C:Enterprise mode

Let us test the document.

1. Start 1C:Enterprise in the debug mode.
2. Open the **Prices** information register.
3. Add another price with a different date for the Philips transistor (fig. 9.11).

Period	Material or service	Price
7/1/2014 12:00:00 AM	Diagnostics	200.00
7/1/2014 12:00:00 AM	Domestic TV set repair	600.00
7/1/2014 12:00:00 AM	Imported TV set repair	800.00
7/1/2014 12:00:00 AM	Water hookup	800.00
7/1/2014 12:00:00 AM	Electrical hookup	800.00
7/1/2014 12:00:00 AM	Line transformer, Samsung	900.00
7/1/2014 12:00:00 AM	Line transformer, LG	400.00
7/1/2014 12:00:00 AM	Transistor 2N2369, Philips	5.00
7/1/2014 12:00:00 AM	Tubing, rubber	150.00
7/1/2014 12:00:00 AM	Power supply cable	30.00
9/17/2014 1:00:00 AM	Transistor 2N2369, Philips	7.00

Fig. 9.11. Prices information register

4. Open the Service document #1.

   As you remember, this document records the expense of one Philips transistor.

5. Leave the document date unchanged and reselect the transistor in the **Material or service** column of the document tabular section.

   The platform fills the Price field with the price value effective as of 7/1/2014. This is the latest price value for the document date (fig. 9.12).

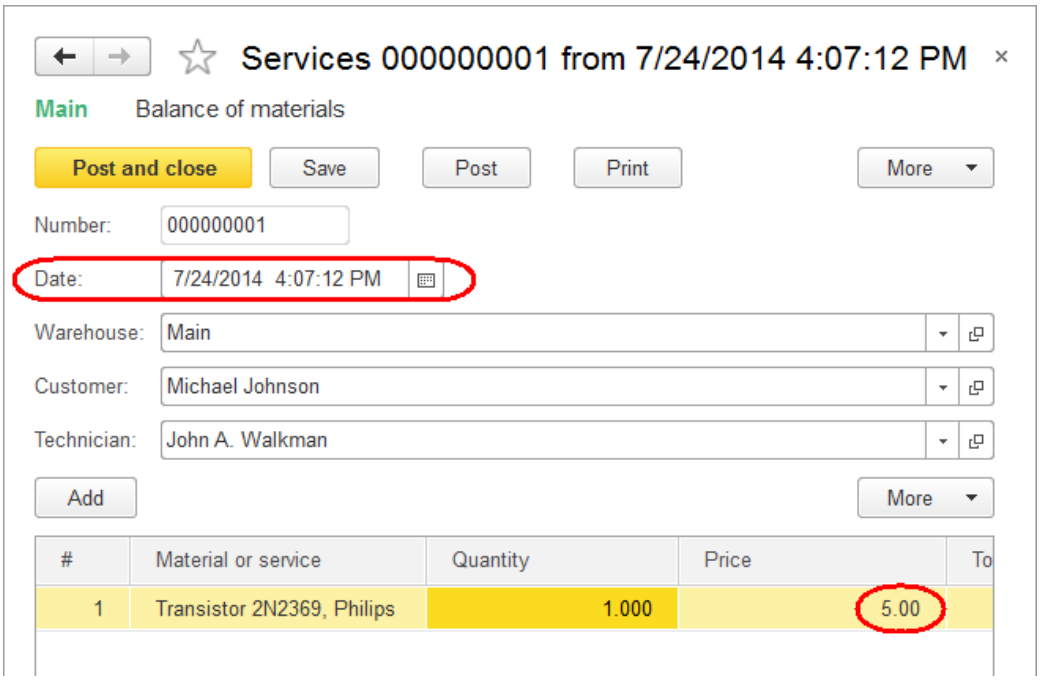


Fig. 9.12. Filling the Services document

6. Change the document date to 9/17/2014 and reselect the transistor.

   The platform fills the Price field with the price value effective as of 9/17/2014 (fig. 9.13).

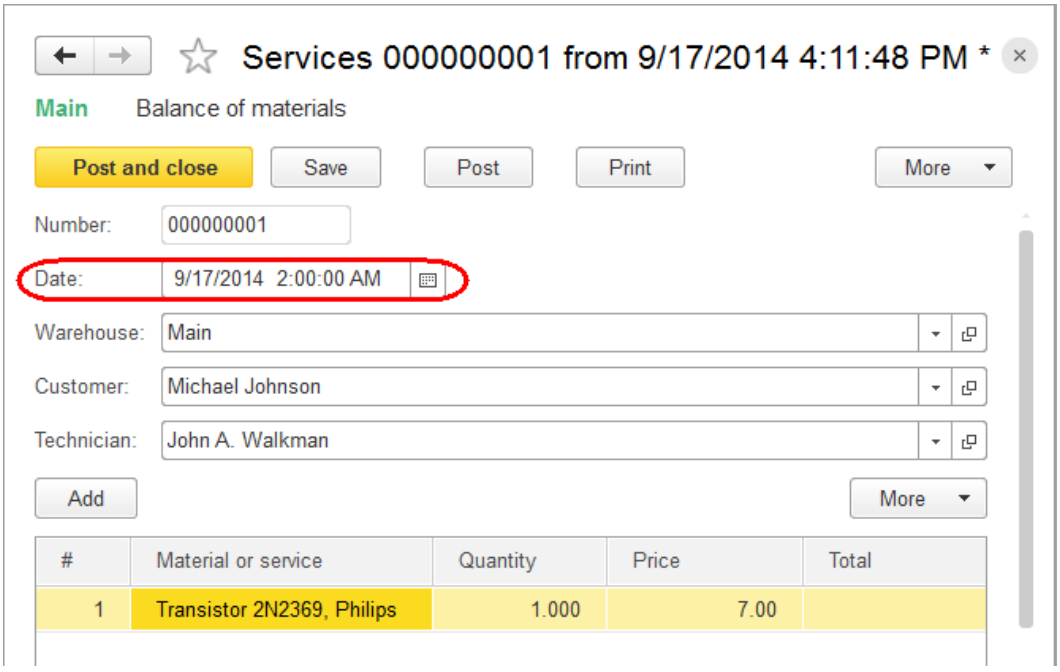


Fig. 9.13. Filling the Services document

So you can see that the Price field is automatically filled with the price effective on the document creation date.

Quiz

- What are the uses of the Information register configuration object?
- What are the distinctive information register features?
- What are the main differences between an information register and an accumulation register?
- Which fields determine the uniqueness key for an information register?
- What is a periodic information register?
- What is an independent information register?
- How does one create a periodic information register?
- What is the master register dimension?
- How does one get the resource values for the latest information register records using 1C:Enterprise script?

Lesson 10 (0:40). Enumerations

Up to this point we have ignored the fact that we have no sure way of determining whether a specific item in the MaterialsAndServices catalog is a material or a service. The fact that you have arranged all catalog items into certain groups is not a reliable basis for such a decision, since groups can be deleted or renamed, items can be grouped by another principle, and so on.

Therefore, we need some kind of a marker that will clearly define whether a catalog item belongs to materials or to services, irrespective of changes to the catalog hierarchical structure.

In this lesson you will create an attribute for the MaterialsAndServices catalog using the data type described by a configuration object that you have not encountered yet, an Enumeration. As you move forward, this will let you easily classify catalog items of the MaterialsAndServices catalog as services or materials.

Additionally you will update the posting procedure of the Services document and learn how to work with enumerations using 1C:Enterprise script tools.

Understanding enumerations

The Enumeration configuration object is intended to describe a structure for storing constant value sets that cannot be modified at run time. On the basis of an Enumeration object the platform creates a database table that stores a set of constant values.

In practice, this object can store options for calculating prices (such as "VAT included" or "without VAT"). The assortment of enumeration values is defined during configuration development and users cannot add, modify, or delete values.

This leads us to an important enumeration feature: enumerated values are significant to the configuration, and 1C:Enterprise script algorithms can rely on them.

Learn more! For details on the structure of 1C:Enterprise script objects intended for enumeration operations, see section "Quick developer reference. Enumerations" on page 849.

Adding enumerations

In Designer mode

Let us create an enumeration.

1. In Designer, create an Enumeration object named *MaterialServiceTypes*.
2. On the **Data** tab, click the **Add** button and add the **Material** and **Service** enumeration values.

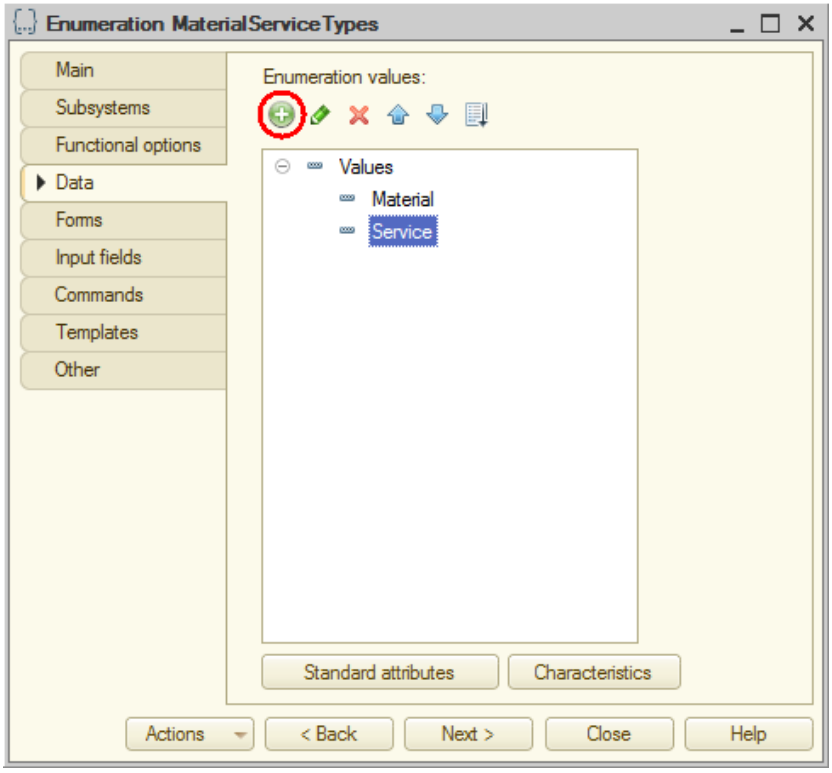


Fig. 10.1. MaterialServiceTypes enumeration values

Linking materials and services to enumeration values

To link materials and services to enumeration values, you have to do the following:

1. In Designer, create a MaterialsAndServices catalog attribute that will store enumeration values.
2. In 1C:Enterprise mode, specify the values of this attribute for all the items in the MaterialsAndServices catalog.

In Designer mode

Let us add the attribute to the MaterialsAndServices catalog.

1. Open the object editor window for the **MaterialsAndServices** catalog.
2. On the **Data** tab, click the **Add** button and add the **MaterialServiceType** attribute with type *EnumRef.MaterialServiceTypes* and synonym *Type* (fig. 10.2).

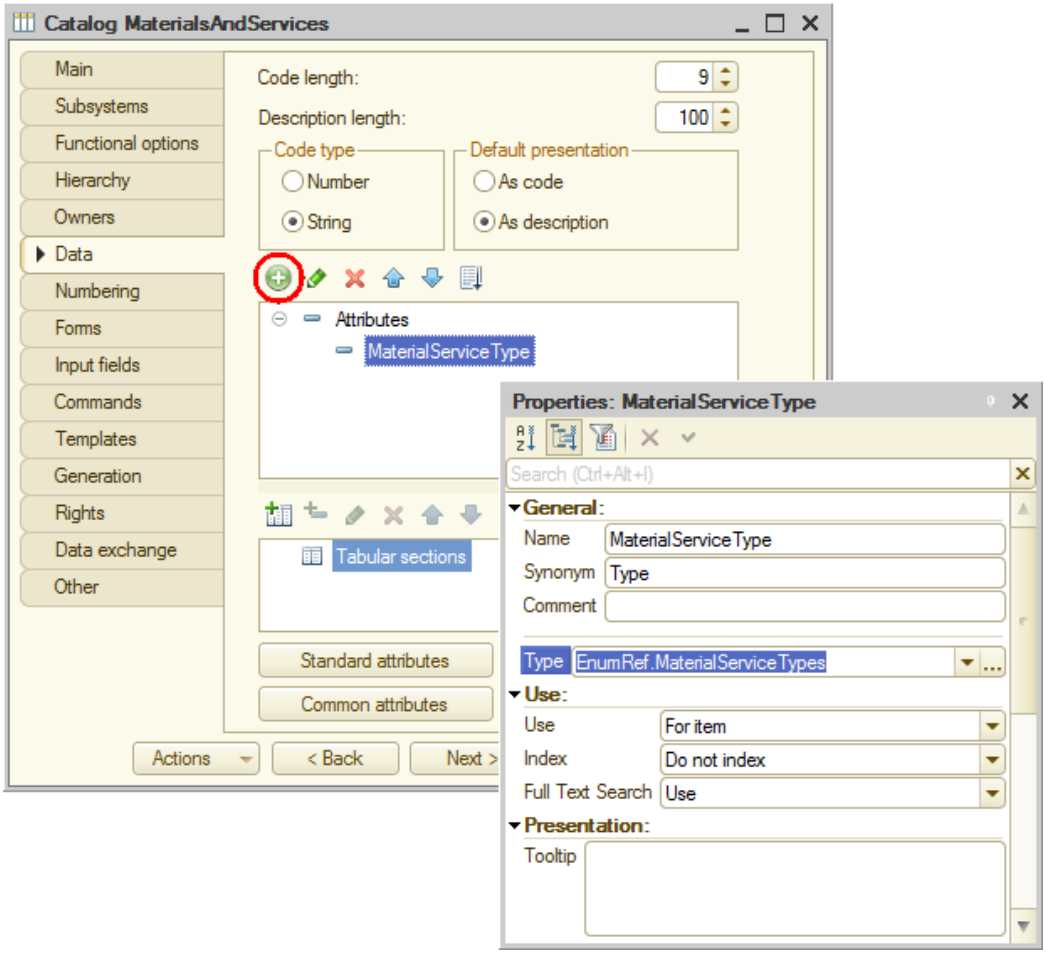


Fig. 10.2. MaterialsAndServices catalog attributes

In 1C:Enterprise mode

Let us set the attribute values.

1. Run 1C:Enterprise in the debug mode.
2. For each catalog item, specify its **Type** (fig. 10.3).

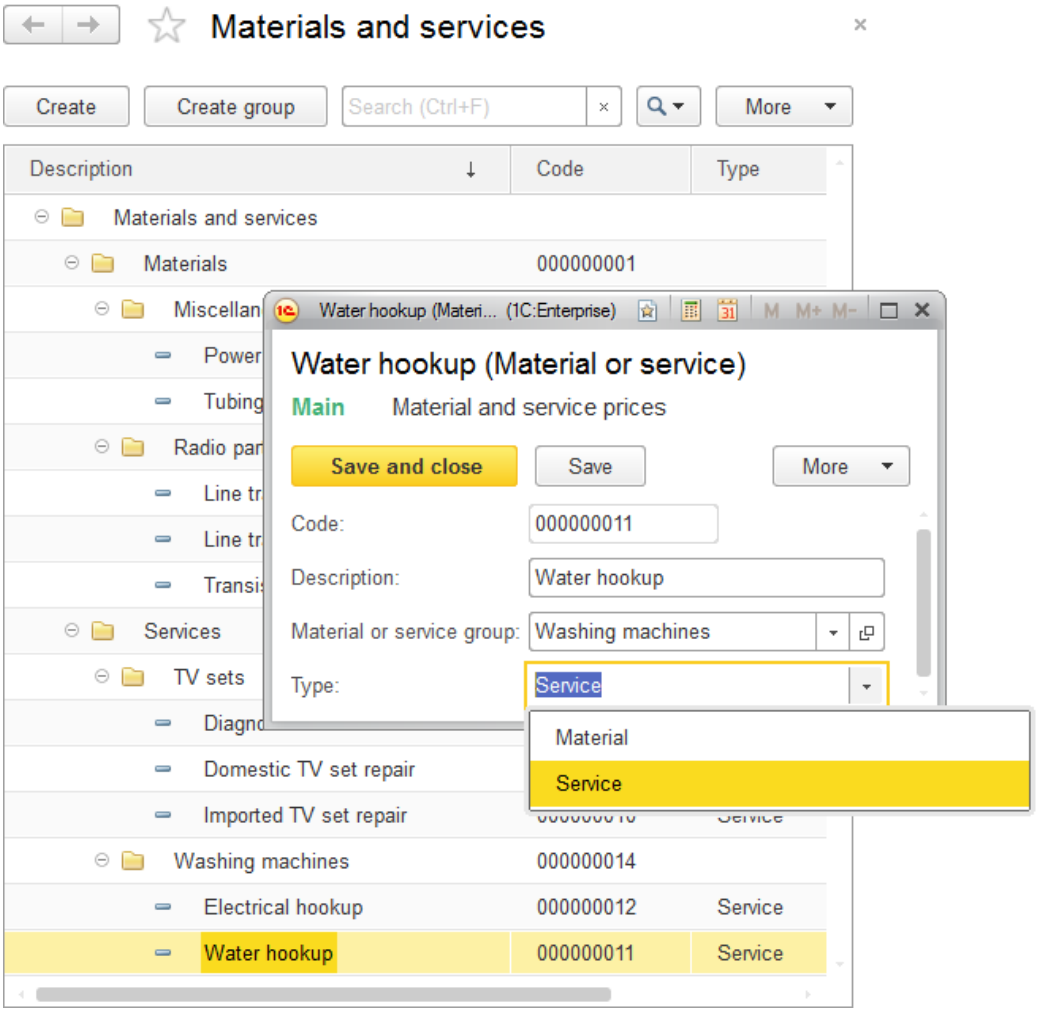


Fig. 10.3. Materials and services catalog items

Now let us discuss the uses of the new data stored in the MaterialServiceType enumeration.

Custom presentations for materials and services

Let us use the MaterialServiceType attribute for specifying custom presentations for materials and services in 1C:Enterprise interface.

Material and service presentations are displayed in all fields that refer to MaterialsAndServices catalog items. This includes document tabular section fields, information register fields, accumulation register fields, and more.

As with any catalog item, the default material or service presentation is defined by the Default presentation property. The default property value is As description (fig. 10.4).

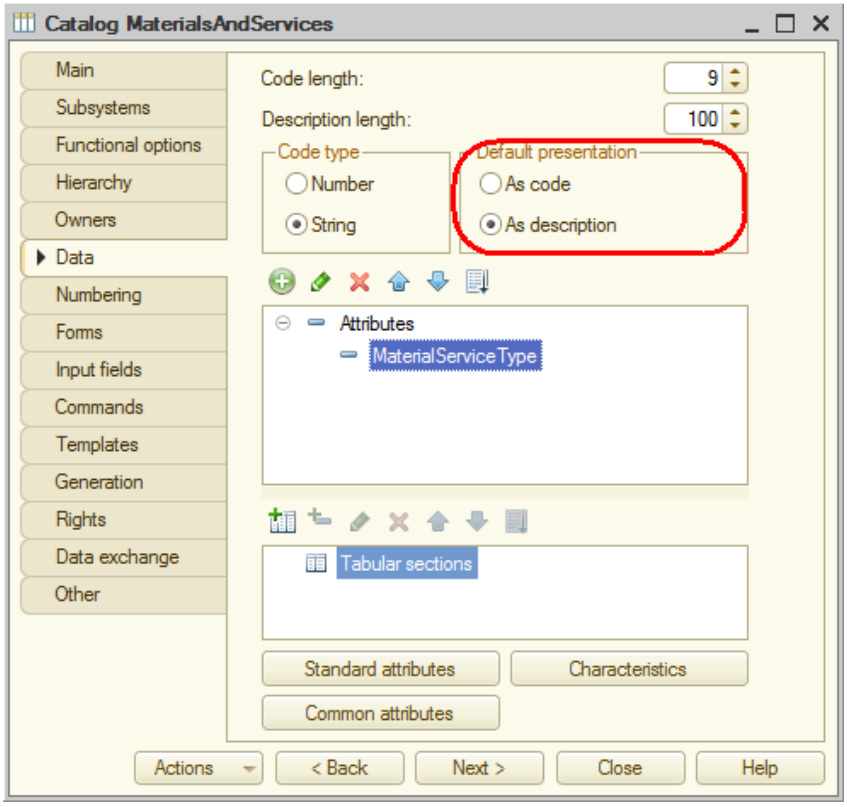


Fig. 10.4. Default presentation property of a catalog

That is why the **Material or service** column in the document tabular section shows the material or service names instead of references (fig. 10.5).

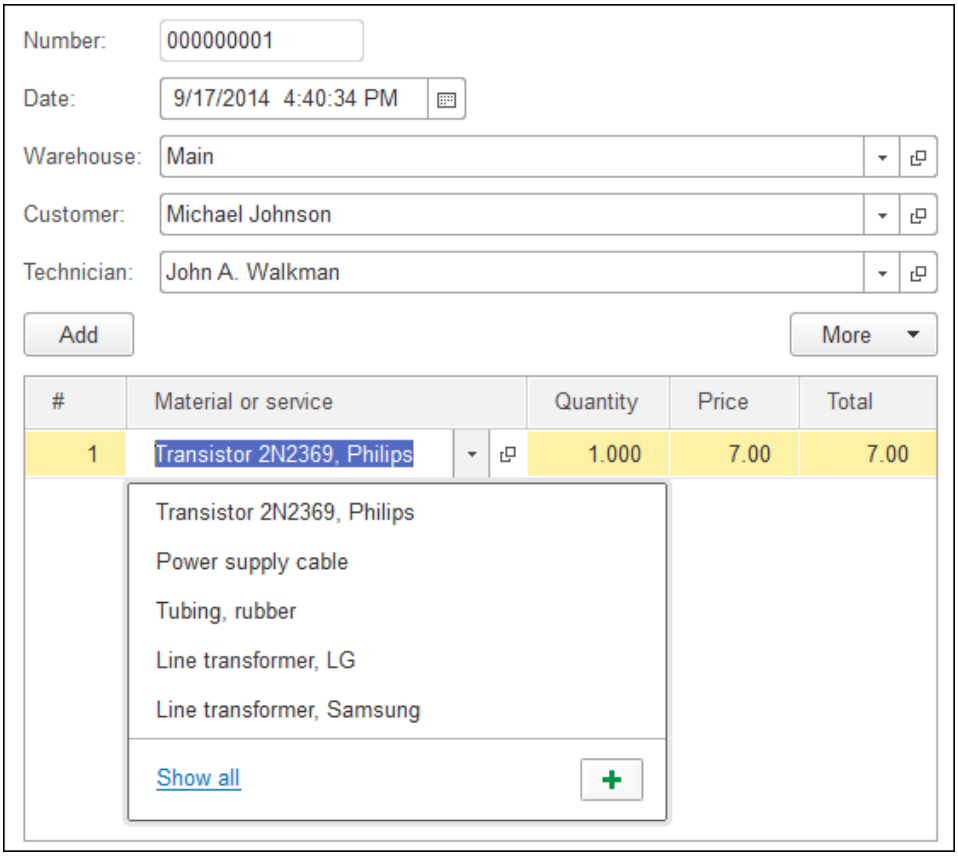


Fig. 10.5. Services document

Let us modify the view of references to MaterialsAndServices catalog items so that they display the item type: material or service.

In Designer mode

The generation of configuration object presentations consists of two stages: first, defining the attributes that affect the presentation, and then the generation itself. You can use the following object manager event handlers for this: PresentationFieldsGetProcessing and PresentationGetProcessing.

1. In Designer, in the configuration object tree, right-click the **MaterialsAndServices** catalog and click **Open manager module**.
2. Add the procedures from listings 10.1 and 10.2 to the manager module.

Listing 10.1. Procedure PresentationFieldsGetProcessing()

```
Procedure PresentationFieldsGetProcessing(Fields, StandardProcessing)


    StandardProcessing = False;
    Fields.Add("Description");
    Fields.Add("MaterialServiceType");


EndProcedure
```

Listing 10.2. Procedure PresentationGetProcessing()

```
Procedure PresentationGetProcessing(Data, Presentation,
    StandardProcessing)


    StandardProcessing = False;
    If ValueIsFilled(Data.MaterialServiceType) Then
        Presentation = Data.Description + " ("
            + Lower(String(Data.MaterialServiceType)) + ")";
    Else
        Presentation = Data.Description;
    EndIf;


EndProcedure
```

Now let us examine this script.

The PresentationFieldsGetProcessing handler describes the attributes that affect the document presentation. First, the handler sets the StandardProcessing parameter to False, and then it adds the required catalog attributes to the Fields array. Note that if StandardProcessing is not set to False, the Fields array contains the attributes used to generate the default object presentation (in this case it is the Description attribute).

The PresentationGetProcessing handler describes the generation of the Presentation parameter based on the array of fields specified in the previous

handler. The data required to generate the presentation is passed in the Data parameter that has Structure type. First, StandardProcessing is set to False, and then the material or service presentation is generated by adding the type enclosed in parentheses to the description. Note that if StandardProcessing is not set to False, the platform attempts to generate the default object presentation based on the data that is passed to the handler.

In 1C:Enterprise mode

Let us test the changes that you made.

1. Run 1C:Enterprise in the debug mode.
2. Open the Services document and ensure that the material or service presentations are generated according to the algorithm that you specified (fig. 10.6).

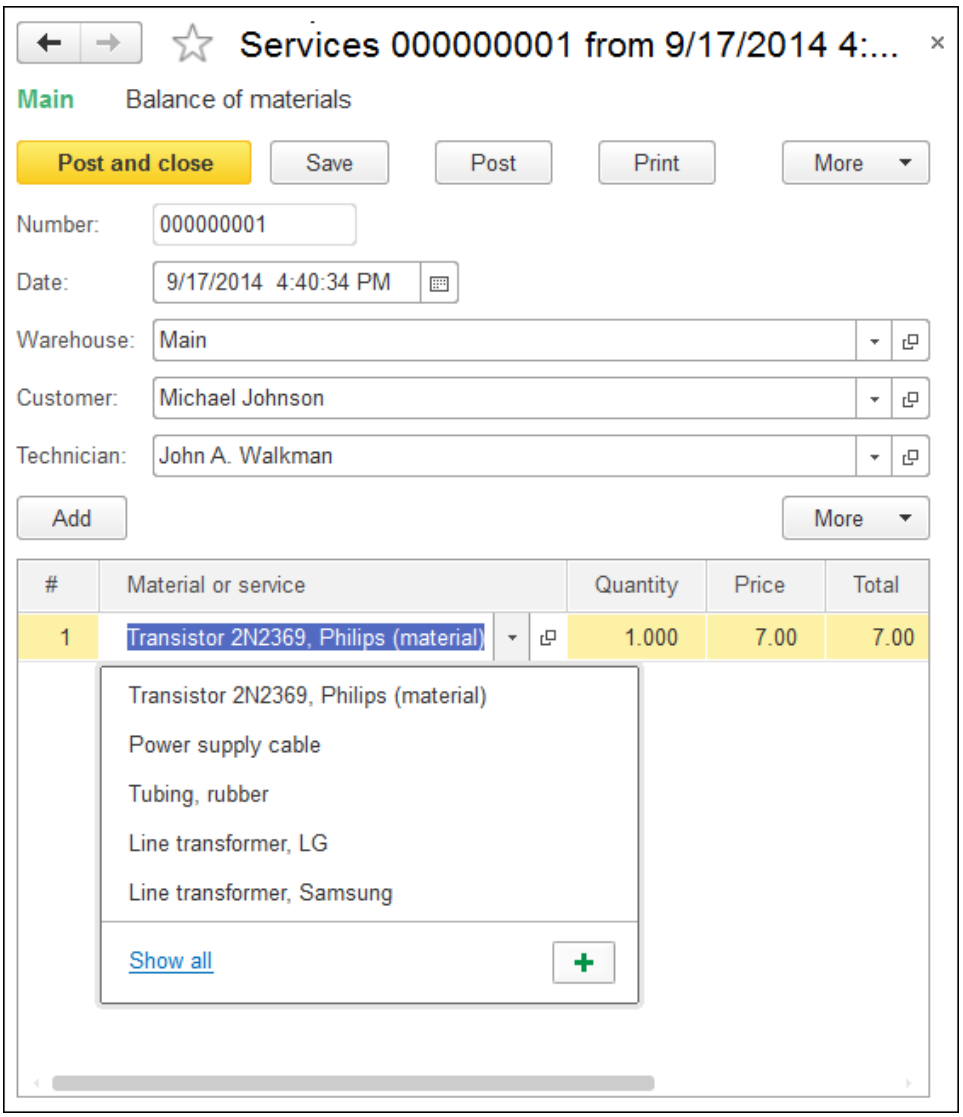


Fig.10.6. The Services document

You can use this method to set custom presentations not only for catalogs, but also for documents, charts of characteristic types, charts of accounts, and other configuration objects.

Recording expenses for materials only

In lesson 6 (page 210), when you created the Services document records in the BalanceOfMaterials accumulation register, we mentioned that these records were not entirely correct because they included not only expended materials but also rendered services (listing 10.3).

Listing 10.3. The Posting() procedure of the Services document

```
Procedure Posting(Cancel, Mode)
    //{{__REGISTER_REGISTERRECORDS_WIZARD
    // This fragment was built by the wizard.
    // Warning! All manually made changes will be lost next time you use the
wizard.
    // register BalanceOfMaterials Expense

    RegisterRecords.BalanceOfMaterials.Write = True;
    For Each CurRowMaterialsAndServices In MaterialsAndServices Do
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = CurRowMaterialsAndServices.MaterialOrService;
        Record.Warehouse = Warehouse;
        Record.Quantity = CurRowMaterialsAndServices.Quantity;
    EndDo;

    //}}__REGISTER_REGISTERRECORDS_WIZARD
EndProcedure
```

Let us modify the document so that only material expense records are added to the register.

First, you have to change the posting procedure in Designer to add only material expense records to the register, and then you have to repost all the Services documents in 1C:Enterprise mode to have the register data modified in compliance with the new document posting algorithm.

This enhancement is not optimal in terms of performance but it ensures that the BalanceOfMaterials register stores the correct data.

Note. We will discuss a more efficient algorithm of posting this document in lesson 14 on page 375, after we study how 1C:Enterprise processes queries.

In Designer mode

Let us refine the document register records by excluding any tabular section rows that contain services.

1. In Designer, right-click the Services document and then click **Open object module**.

This opens the Services document module.

2. Add a condition to the handler of the Posting event, to the beginning of the loop that iterates through the tabular section rows, after the following line:

```
For Each CurRowMaterialsAndServices In MaterialsAndServices Do
```

The resulting procedure should look as shown in listing 10.4.

Listing 10.4. The Services document records

```
Procedure Posting(Cancel, Mode)


 //{{__REGISTER_REGISTERRECORDS_WIZARD
 // This fragment was built by the wizard.
 // Warning! All manually made changes will be lost next time you use
the wizard.


 // register BalanceOfMaterials Expense
 RegisterRecords.BalanceOfMaterials.Write = True;
 For Each CurRowMaterialsAndServices In MaterialsAndServices Do


   If CurRowMaterialsAndServices.MaterialOrService.MaterialServiceType
     = Enums.MaterialServiceTypes.Material Then
      Record = RegisterRecords.BalanceOfMaterials.Add();
      Record.RecordType = AccumulationRecordType.Expense;
      Record.Period = Date;
      Record.Material =
        CurRowMaterialsAndServices.MaterialOrService;
      Record.Warehouse = Warehouse;
      Record.Quantity = CurRowMaterialsAndServices.Quantity;
   EndIf;
 EndDo;


 //}}__REGISTER_REGISTERRECORDS_WIZARD
EndProcedure
```

The added script excludes the tabular section rows where the item type is not Material from the loop. Let us examine this in detail.

In each loop step the CurRowMaterialsAndServices variable contains the data of the current row of the MaterialsAndServices tabular section.

The MaterialOrService column name specified after a dot (CurRowMaterialsAndServices.MaterialOrService) calls a reference to the material or service item stored in the current tabular section row.

The MaterialServiceType specified after a dot (CurRowMaterialsAndServices.MaterialOrService.MaterialServiceType) calls the MaterialServiceType attribute of this item of the MaterialsAndServices catalog.

The comparison operator (=) compares the obtained value with the Material value of the MaterialServiceTypes enumeration (Enums.MaterialServiceTypes.Material).

If the values match, the operators included in the loop are executed. Otherwise the next loop iteration, which processes the next tabular section row, is started.

In 1C:Enterprise mode

Let us check the changes in the posting procedure of the Services document.

1. Start 1C:Enterprise in the debug mode.
2. In the **Services** section, click the **Services** command.

   This opens the list of documents, which contains a single Services document.
3. Open the document and make the following changes:
   - Remove the Philips transistor.
   - Add the **Water hookup** service.
   - Add the **Tubing, rubber** material (fig. 10.7).

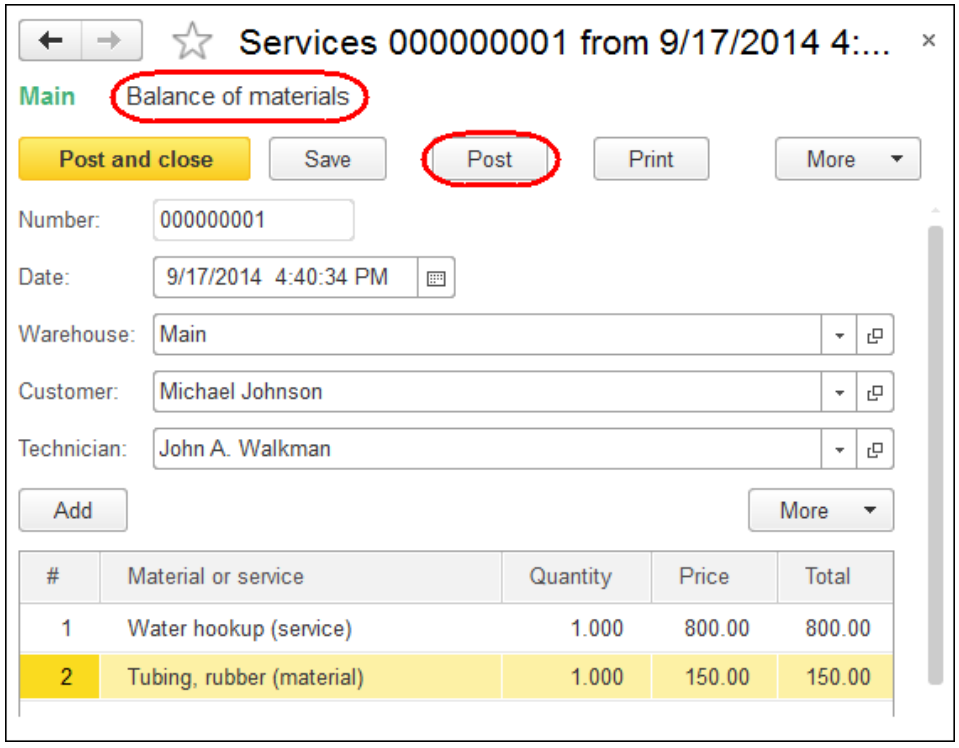


Fig. 10.7. The modified Services document

   Note that the prices are filled automatically based on the Prices information register data.
4. In the document command bar, click **Post**.
5. In the form navigation panel, run the **Balance of materials** command.

This opens the list of Balance of materials register records related to this document (fig. 10.8).

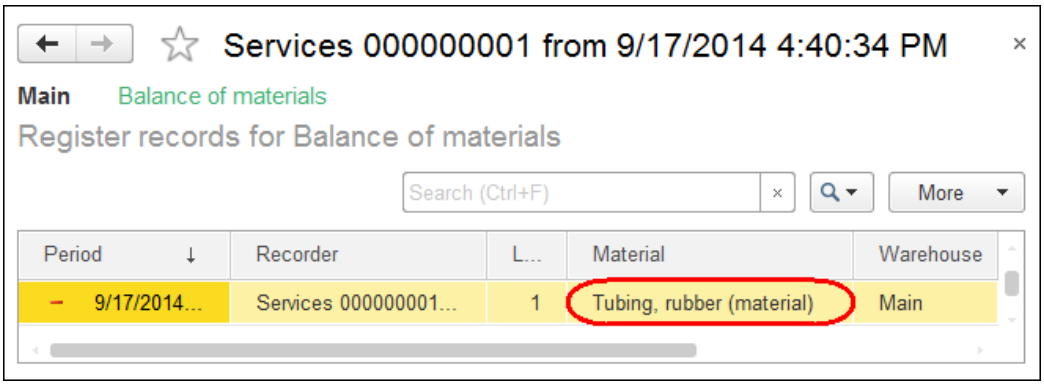


Fig. 10.8. Balance of materials register records

As you can see, the records for the Balance of materials register only include the rows containing the materials. There is no record for the water hookup service.

Also note that the item presentation in the Material field of the Balance of materials register includes the item type (material), exactly as you specified earlier.

Quiz

- What are the uses of the Enumeration configuration object?
- How does one create an enumeration?
- How does one use enumerations to declare that catalog items belong to a specific logical group?
- How does one reference an enumeration value using 1C:Enterprise script?
- How does one specify a custom configuration object presentation?

Lesson 11 (1:20). Posting documents across multiple registers

This lesson describes how a single document can provide data for multiple configuration registers and when this feature might be required.

In this lesson you will create another accumulation register for your configuration and modify document posting procedures so that the documents record data to both registers.

This will also provide a basis for studying the next lesson.

Why post a document across multiple registers

Up to this point you only considered the changes of material quantities within Jack of All Trades. You have created the BalanceOfMaterials accumulation register for material quantity accounting.

However, as you have probably guessed, this type of accounting alone will hardly meet the needs of your company. Clearly, you also need to know what monetary resources were expended to purchase the materials and what the company's material reserves are worth in cash.

Once you started automating your company, the Jack of All Trades management asked you to implement the average cost method for all material balances accounting.

In other words, when materials are acquired, they are accounted for in terms of purchase prices, while when expended they are accounted for at an average cost, which is calculated based on the aggregate purchase of a given material and the total material quantity available in Jack of All Trades.

Since this data structure is absolutely different from the data structure used in material quantity accounting, you will use a new accumulation register named CostOfMaterials, which will store data regarding the overall cost of the materials available in the company.

So, in order to reflect the balance changes, the GoodsReceipt and Services documents should create records simultaneously in the BalanceOfMaterials register and in the CostOfMaterials register.

Adding another accumulation register

In Designer mode

The CostOfMaterials register is very simple so we will not go into detail during its creation.

1. Create an accumulation register configuration object named *CostOfMaterials*.
2. Set **Extended list presentation** to *Cost of materials register records*.

The extended list presentation will serve as the title of the register record list window.

3. On the **Subsystems** tab, include the register in the **Accounting**, **Inventory**, and **Services** subsystems.
4. On the **Data** tab, create the following dimension:

   - **Material**. Type: CatalogRef.MaterialsAndServices

   and the following resource:

   - **Cost**. Type: Number, length: 15, precision: 2.

   Once created, the CostOfMaterials register should look as follows in the configuration tree (fig. 11.1).

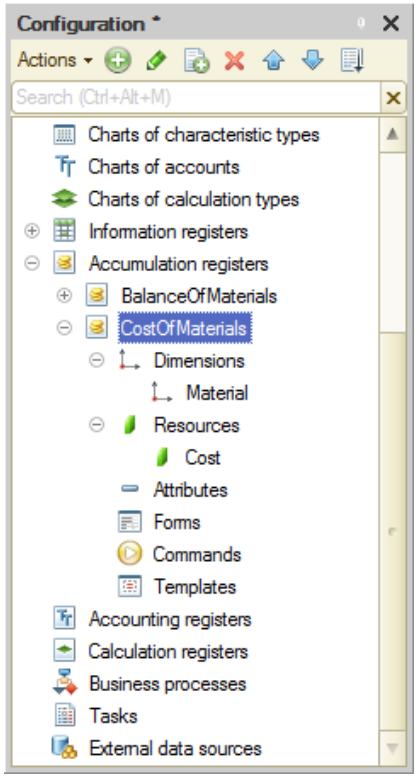


Fig. 11.1. CostOfMaterials accumulation register

Now let us edit the command interface to add the command that opens the accumulation register to the Accounting, Inventory, and Services sections.

5. In the configuration object tree, right-click the **Subsystems** branch and then click **All subsystems**.
6. In the **All Subsystems** window, in the **Subsystems** list, click **Accounting**.

   The **Command interface** list displays all the commands of the selected subsystem.

7. In the **Navigation panel.Normal** group, enable visibility for the **Cost of materials** command and drag it to the **Navigation panel.See also** group (fig. 11.2).

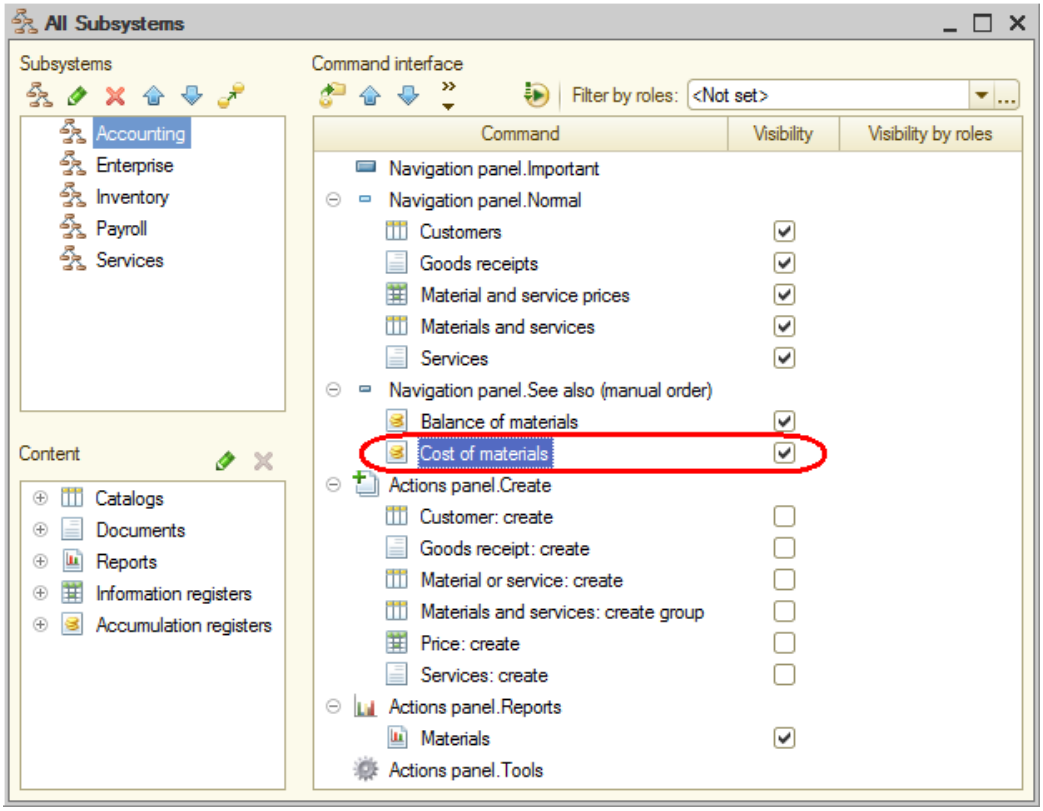


Fig. 11.2. Specifying subsystem command interface settings

8. Perform the same procedure for the **Inventory** and **Services** subsystems: in the **Navigation panel.Normal** group, enable visibility for the **Cost of materials** command and drag it to the **Navigation panel.See also** group.

Then let us proceed to updating the document posting procedures, starting with the simplest GoodsReceipt document.

Posting goods receipts across two registers

In Designer mode

Modifying the posting procedure

Let us modify the document posting procedure.

1. In Designer, open the **GoodsReceipt** document editor window and click the **Posting** tab.
2. Add the **CostOfMaterials** register to the list of registers affected by the document (fig. 11.3).

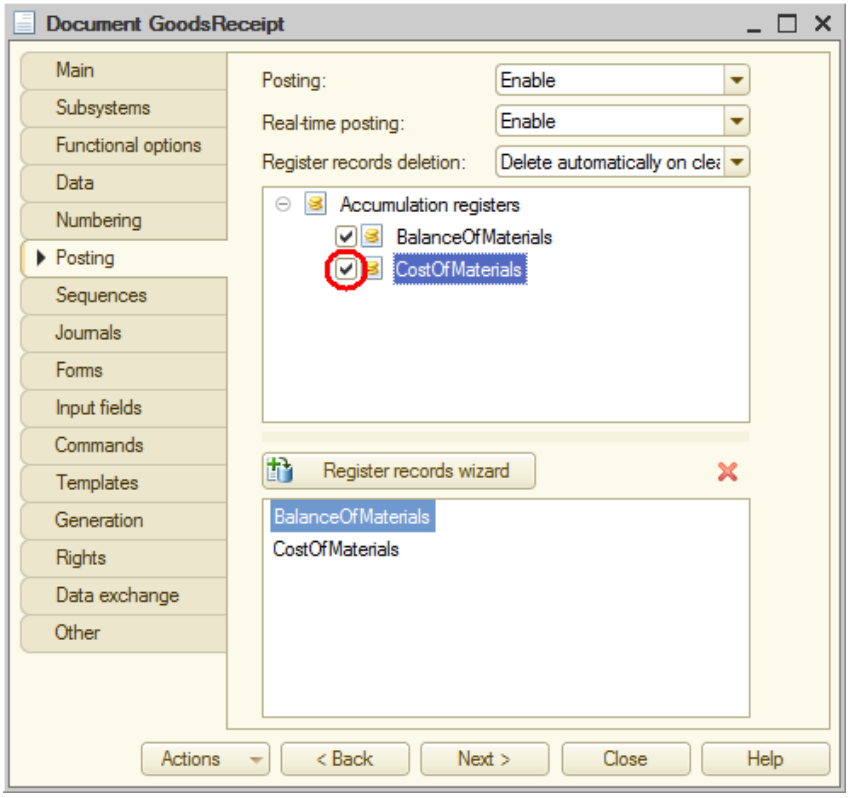


Fig. 11.3. Creating records of the GoodsReceipt document in the CostOfMaterials register

3. Click **Register records wizard**.
4. Confirm that you want to replace the previously generated posting procedure.

   By doing this you do not lose any data because you have not changed the previously generated procedure and the new one will add records to both registers.

   In the **Register Records Wizard** window you can see that all of the fields related to the BalanceOfMaterials register contain data that you specified earlier, when you set up record generation for this register in section "Creating register records of a document" on page 198.
5. Click the **Add** button above the **Registers** list and add the **CostOfMaterials** register (fig. 11.4).

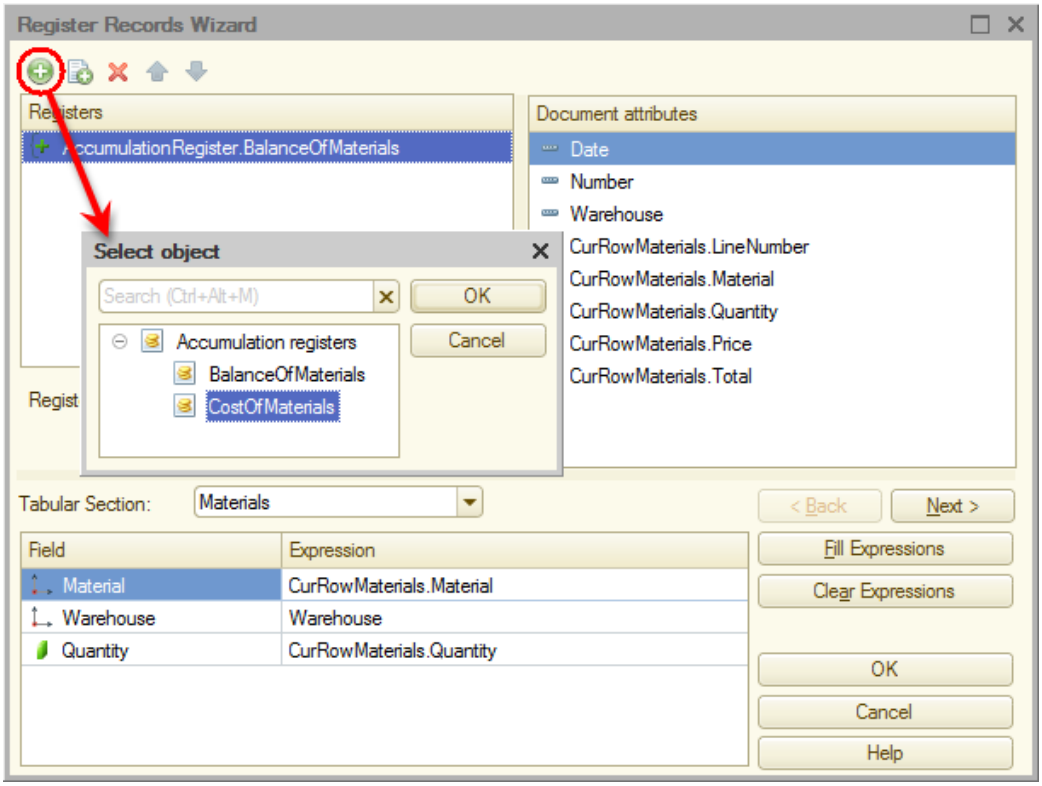


Fig. 11.4. Register records wizard

The default register record type is Receipt (the + icon is displayed next to the register name).

6. In the **Tabular Section** list, select **Materials**.

   It is the tabular section of the GoodsReceipt document.

   Once you select the tabular section, its attributes are added to the **Document attributes** list, which already contains the document header attributes.

7. Click **Fill expressions**.

   In the bottom pane the mapping between the register fields (dimensions and resources) and the expressions for calculating their values is generated.

   Let us specify the mapping for the Cost register field.

8. In the **Field** column, click the **Cost** field, and then, in the **Document attributes** list, double-click the **CurRowMaterials.Total** attribute (fig. 11.5).

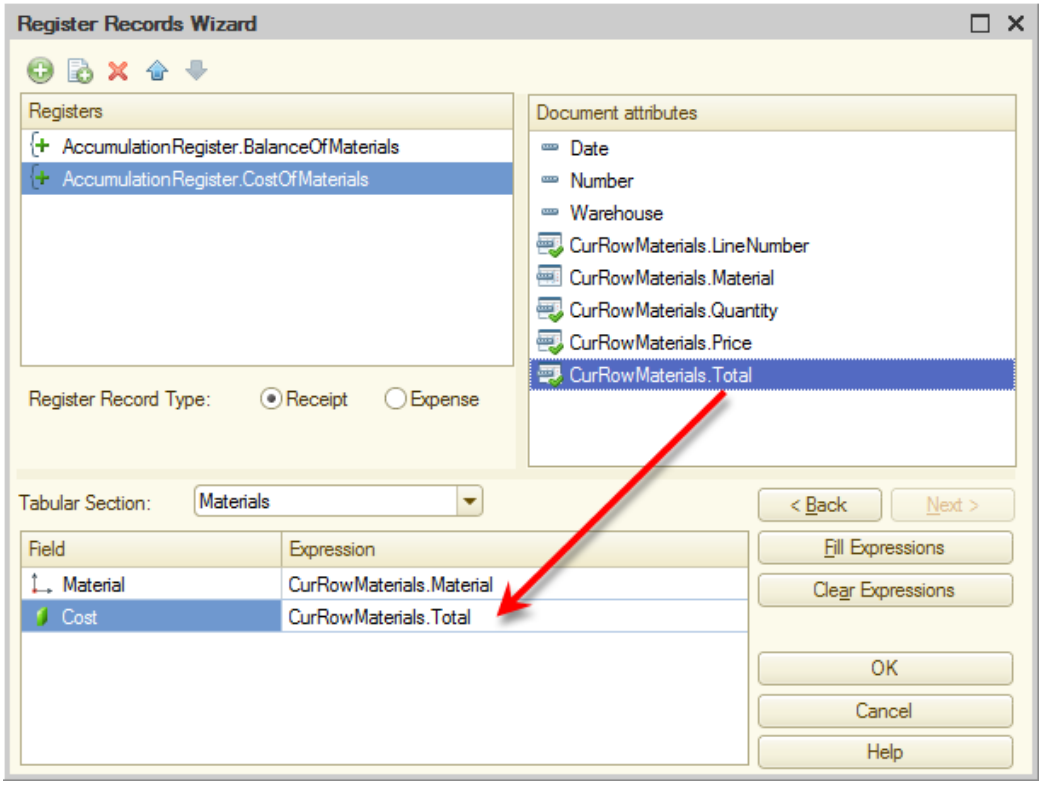


Fig. 11.5. Register records wizard

9. Click **OK**.

The wizard generates a procedure in the GoodsReceipt document module and opens the module. You can see that the new handler procedure includes the generation of records in two registers (listing 11.1).

Listing 11.1. Records of the GoodsReceipt document

```
Procedure Posting(Cancel, Mode)
    //{{__REGISTER_REGISTERRECORDS_WIZARD
    // This fragment was built by the wizard.
    // Warning! All manually made changes will be lost next time you
use the wizard.

    // register BalanceOfMaterials Receipt
    RegisterRecords.BalanceOfMaterials.Write = True;
    For Each CurRowMaterials In Materials Do
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Receipt;
        Record.Period = Date;
        Record.Material = CurRowMaterials.Material;
        Record.Warehouse = Warehouse;
        Record.Quantity = CurRowMaterials.Quantity;
    EndDo;

    // register CostOfMaterials Receipt
```

```
    RegisterRecords.CostOfMaterials.Write = True;
    For Each CurRowMaterials In Materials Do
        Record = RegisterRecords.CostOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Receipt;
        Record.Period = Date;
        Record.Material = CurRowMaterials.Material;
        Record.Cost = CurRowMaterials.Total;
    EndDo;

    //}}__REGISTER_REGISTERRECORDS_WIZARD
EndProcedure
```

You can see that the script fragment that describes the record generation for the CostOfMaterials register is similar to the script fragment that describes the record generation for the BalanceOfMaterials register.

But having two loops that iterate through the Materials tabular section is not optimal, you can use a single loop for this.

10. Combine the two loops into one and delete the comments generated by the wizard.

    Then your procedure should look as shown in listing 11.2.

Listing 11.2. Records of the GoodsReceipt document

```
Procedure Posting(Cancel, Mode)

    RegisterRecords.BalanceOfMaterials.Write = True;
    RegisterRecords.CostOfMaterials.Write = True;

    For Each CurRowMaterials In Materials Do
    // register BalanceOfMaterials Receipt
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Receipt;
        Record.Period = Date;
        Record.Material = CurRowMaterials.Material;
        Record.Warehouse = Warehouse;
        Record.Quantity = CurRowMaterials.Quantity;

    // register CostOfMaterials Receipt
        Record = RegisterRecords.CostOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Receipt;
        Record.Period = Date;
        Record.Material = CurRowMaterials.Material;
        Record.Cost = CurRowMaterials.Total;
    EndDo;
EndProcedure
```

Adding the command that opens register records

Finally, let us edit the document form command interface to add the link for viewing the CostOfMaterials register records related to the document to the form navigation panel.

1. Open the **GoodsReceipt** document form.
2. In the upper left pane, click the **Command interface** tab.
3. In the **Navigation panel** branch, expand the **Go to** node.

   It contains the command that opens the list of the Cost of materials accumulation register.
4. Select the **Visible** check box for this command (fig. 11.6).

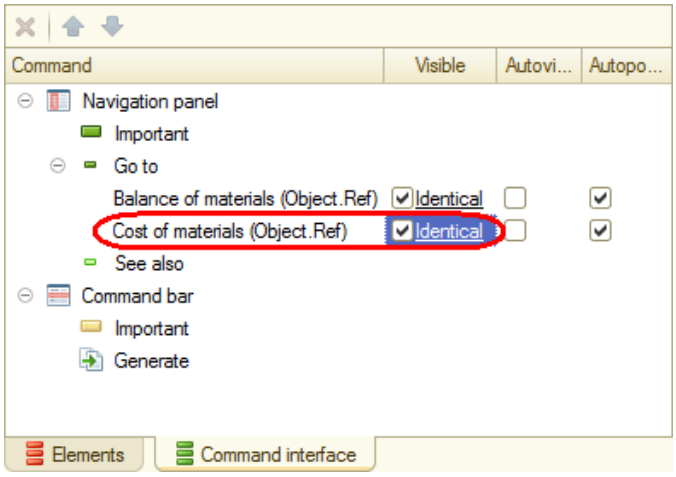


Fig. 11.6. GoodsReceipt document form command interface setup

In 1C:Enterprise mode

In 1C:Enterprise mode you have to repost all goods receipts. This is required for the documents to create new register records according to the posting algorithm you have just modified.

1. Start 1C:Enterprise in the debug mode.
2. In the **Inventory** section, run the **Goods receipts** command.

   This opens the list of goods receipts.
3. Click each goods receipt while holding down Ctrl to select all of them, and then, on the **More** menu, click **Post**.
4. Open the first document (fig. 11.7) and run the **Balance of materials** and **Cost of materials** commands.

   You can see that the document created records in both accumulation registers (fig. 11.8 and fig. 11.9).

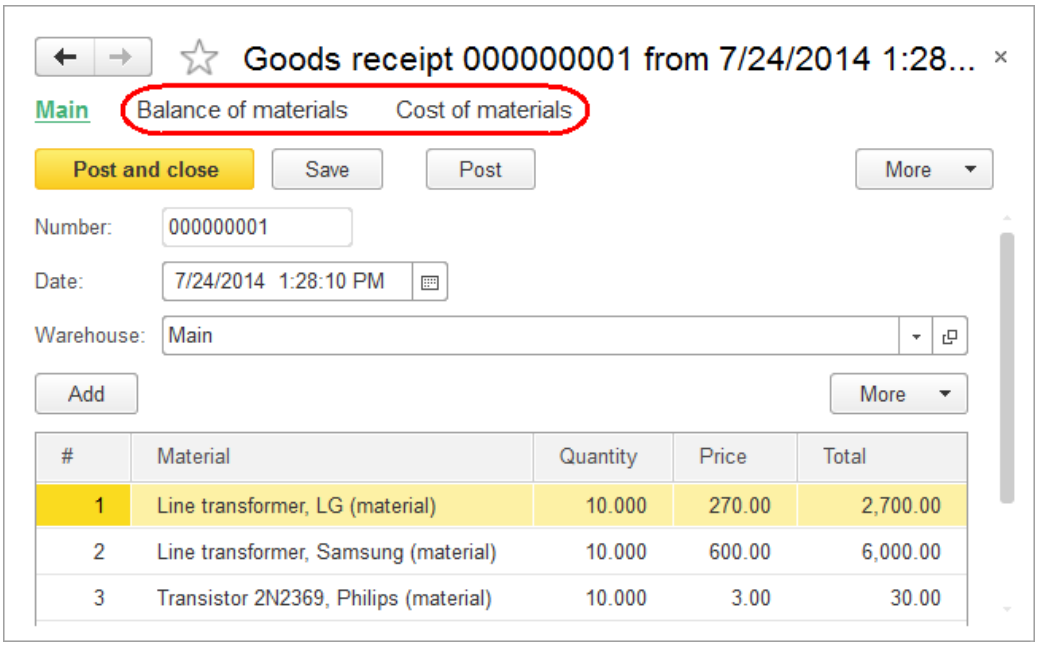


Fig. 11.7. Goods receipt #1

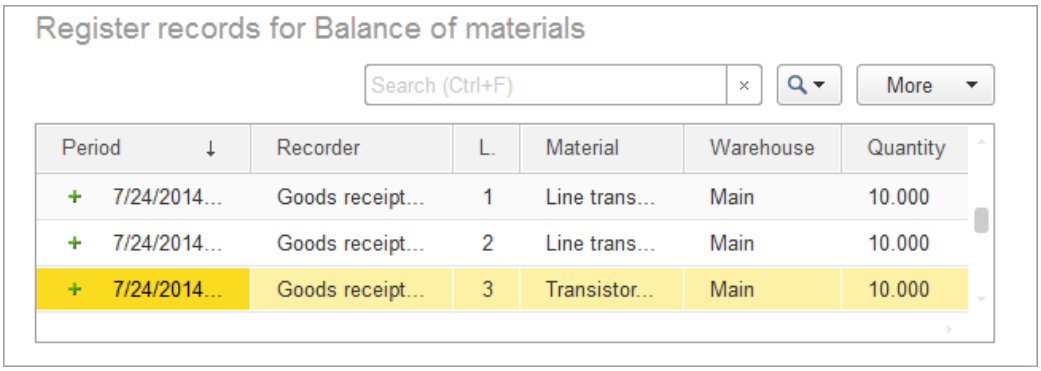


Fig. 11.8. Balance of materials register records

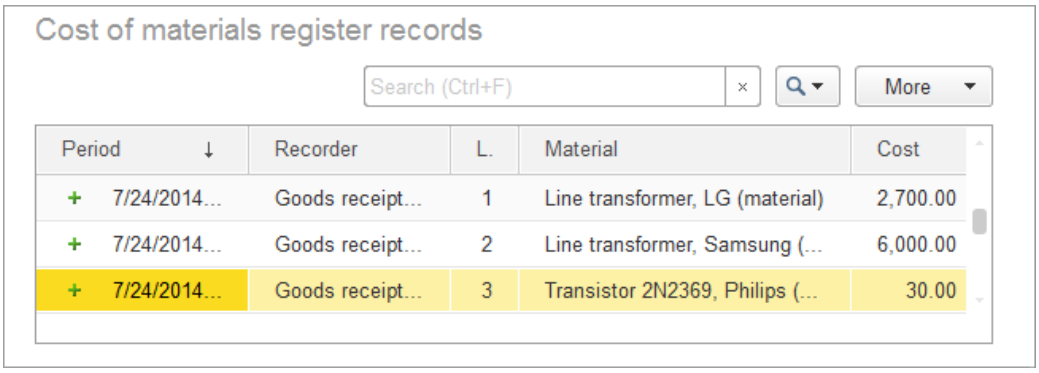


Fig. 11.9. Cost of materials register records

Posting Services documents across two registers

Finally let us modify the posting procedure for the Services documents.

This posting procedure will be based on a Jack of All Trades management request. When writing off materials expended in the course of rendering

services, there should be a way to specify a custom cost for each material, which is calculated by the management on the basis of current market conditions.

Since the Services document only reflects material prices, you have to:

1. Add another attribute, which stores the material cost, to the document tabular section.
2. Modify the posting procedure of the Services document.
3. In 1C:Enterprise mode repost all Services documents to execute the new posting algorithm for each document.

In Designer mode

Adding the document attribute

Let us add a new attribute to the Services document.

1. In Designer, open the **Services** configuration object editor window and click the **Data** tab.
2. Create the following tabular section attribute (fig. 11.10):
   - **Cost**. Type: Number, length: 15, precision: 2, non-negative

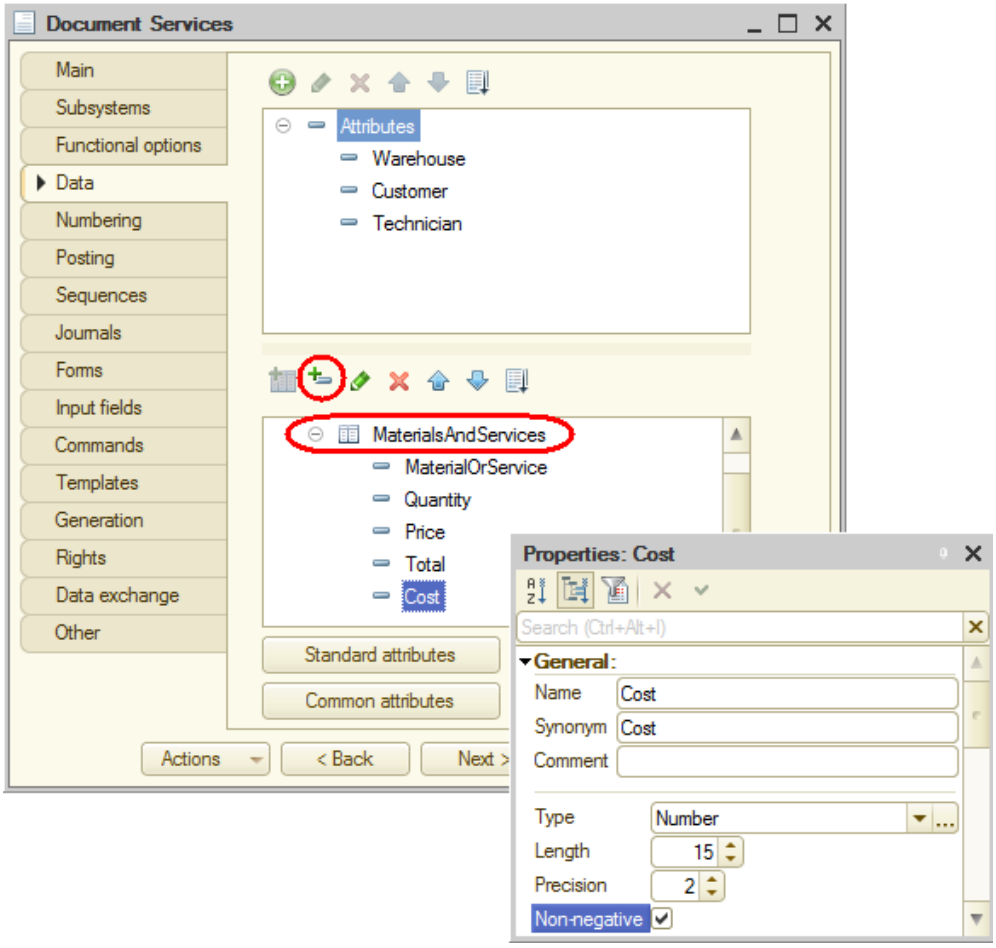


Fig. 11.10. Editing the Services document

Let us add the Cost field linked to the new attribute to the tabular section of the Services document form.

3. Open the **Services** document form.
4. In the upper left pane of the form editor, on the **Attributes** tab, expand the **Object** form attribute (fig. 11.11).

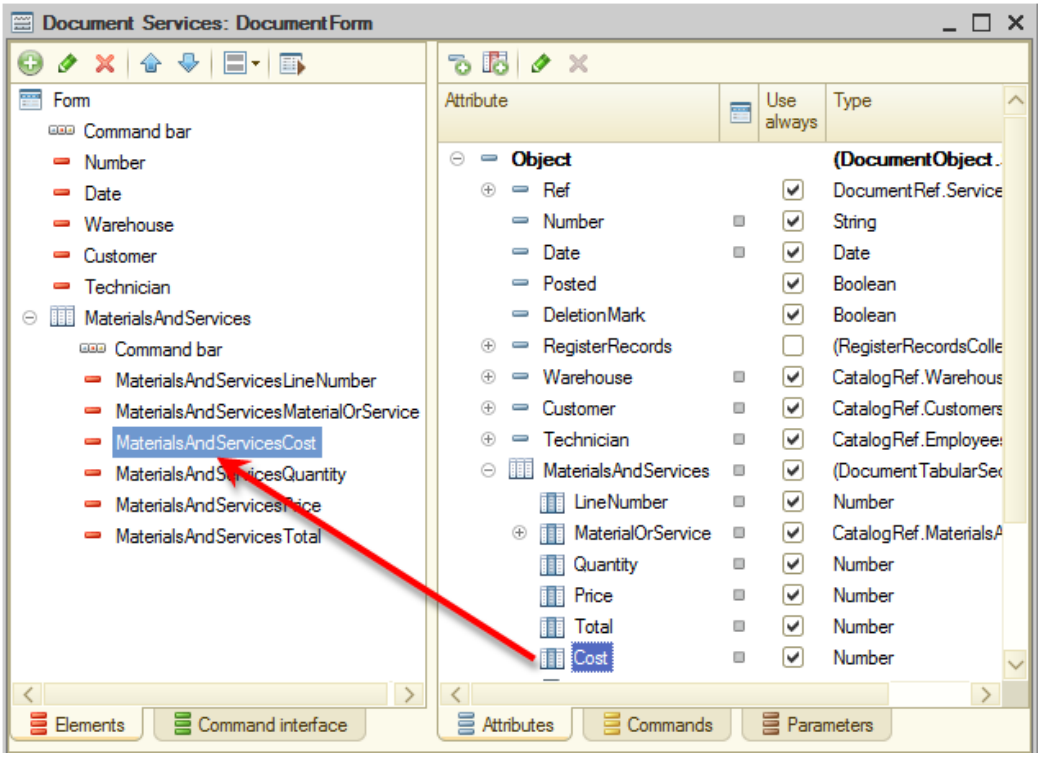


Fig. 11.11. Editing the Services document form

You can see that it contains all of the attributes of the Services document.

5. Locate the **Cost** attribute in the tabular section and drag it to the form controls pane in the upper left corner of the form editor.
6. Move the new control to the position after the **MaterialOrService** field and keep the default properties of the control.

The new attribute is immediately displayed in the form preview in the bottom of the form editor window (fig. 11.12).

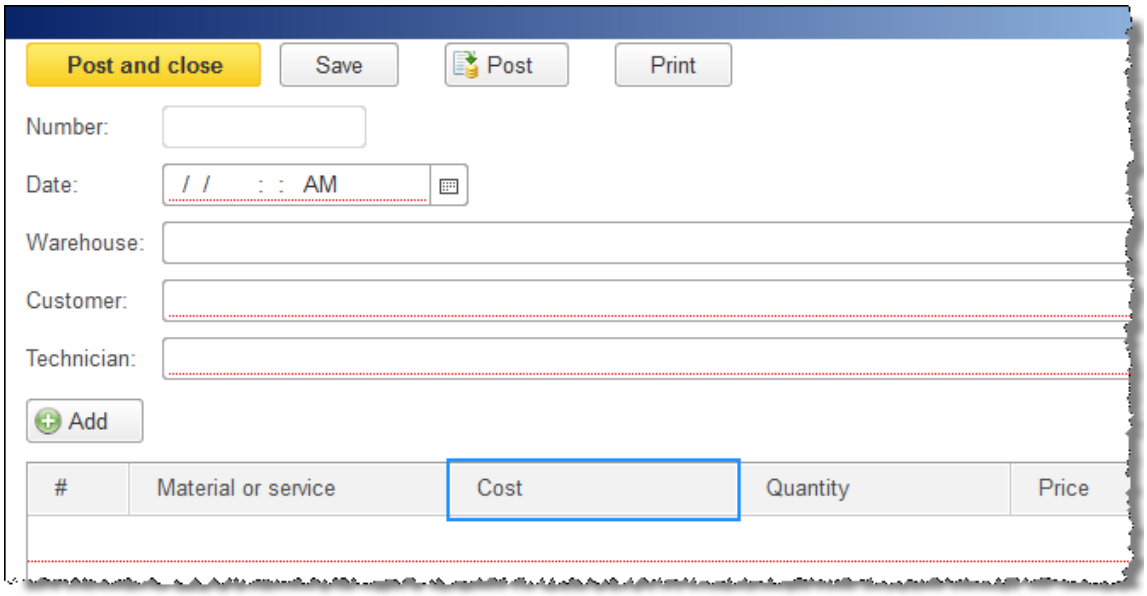


Fig. 11.12. Editing the Services document form

Modifying the posting procedure

Let us modify the document posting procedure.

1. In Designer, open the **Services** document editor window and click the **Posting** tab.
2. Add the **CostOfMaterials** register to the list of registers affected by the document.

   To preserve the changes that you made in the posting procedure in the previous lesson (page 262), do not use the Register records wizard this time. Instead, let us make the required changes in the Posting event handler of the Services document manually.
3. Click the **Other** tab and then click **Object module**.
4. Add the script lines that create CostOfMaterials register records for the Services document to the handler of the Posting event, to the end of the loop, right before the EndIf operator (listing 11.3).

   Listing 11.3. Records of the Services document (fragment)

```
// register CostOfMaterials Expense
Record = RegisterRecords.CostOfMaterials.Add();
Record.RecordType = AccumulationRecordType.Expense;
Record.Period = Date;
Record.Material = CurRowMaterialsAndServices.MaterialOrService;
Record.Cost = CurRowMaterialsAndServices.Quantity
    * CurRowMaterialsAndServices.Cost;
```

5. Before the loop set the Write property of the register record set to True and remove the comments generated by the wizard. The resulting procedure should look as shown in listing 11.4.

Listing 11.4. Records of the Services document

```
Procedure Posting(Cancel, Mode)

 RegisterRecords.BalanceOfMaterials.Write = True;
 RegisterRecords.CostOfMaterials.Write = True;

 For Each CurRowMaterialsAndServices In MaterialsAndServices Do
   If CurRowMaterialsAndServices.MaterialOrService.MaterialServiceType
     = Enums.MaterialServiceTypes.Material Then

     // register BalanceOfMaterials Expense
     Record = RegisterRecords.BalanceOfMaterials.Add();
     Record.RecordType = AccumulationRecordType.Expense;
     Record.Period = Date;
     Record.Material = CurRowMaterialsAndServices.MaterialOrService;
     Record.Warehouse = Warehouse;
     Record.Quantity = CurRowMaterialsAndServices.Quantity;

     // register CostOfMaterials Expense
     Record = RegisterRecords.CostOfMaterials.Add();
     Record.RecordType = AccumulationRecordType.Expense;
     Record.Period = Date;
     Record.Material = CurRowMaterialsAndServices.MaterialOrService;
     Record.Cost = CurRowMaterialsAndServices.Quantity
         * CurRowMaterialsAndServices.Cost;

   EndIf;
 EndDo;

EndProcedure
```

Note that the Cost register dimension is calculated by multiplying the cost and the quantity specified in the document tabular section.

Finally, let us edit the document form command interface to add the link for viewing the CostOfMaterial register records related to the document to the form navigation panel.

1. Open the **Services** document form.
2. In the upper left pane, click the **Command interface** tab.
3. In the **Navigation panel** branch, expand the **Go to** node.

   It contains the command that opens the list of the Cost of materials accumulation register.
4. Select the **Visible** check box for this command.

In 1C:Enterprise mode

In 1C:Enterprise mode you have to repost all Services documents. This is required for the documents to create new register records according to the posting algorithm you have just modified.

1. Start 1C:Enterprise in the debug mode.
2. In the **Services** section, run the **Services** command.

   This opens the list of services.
3. Open the Services document #1 and set the cost of rubber tubing to 100 (fig. 11.13).

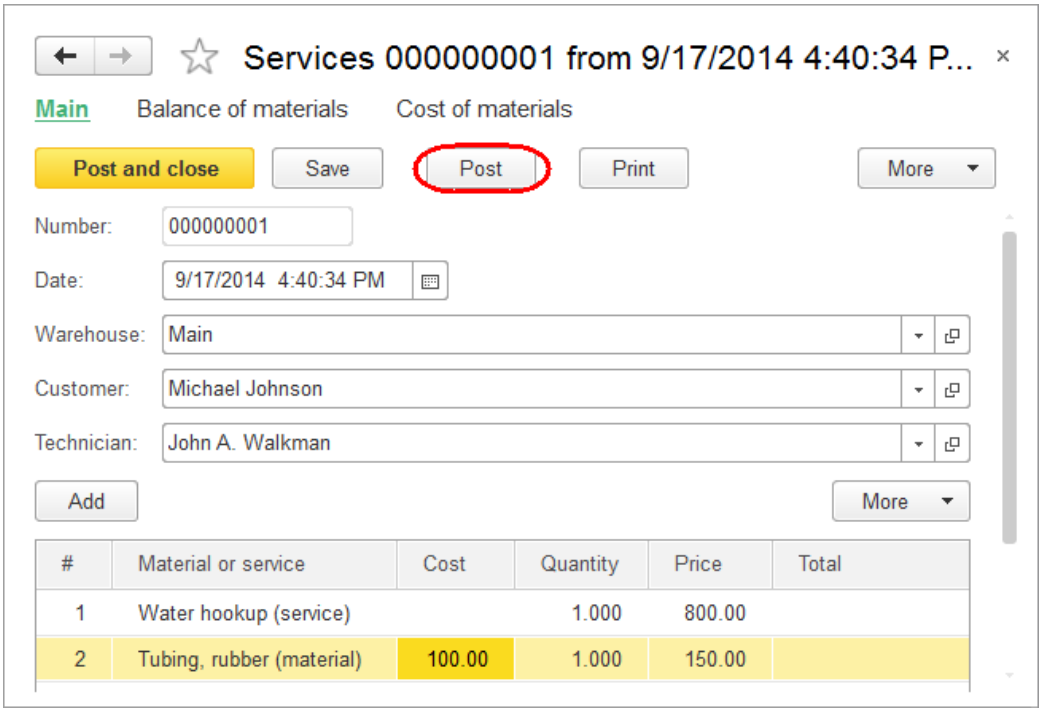


Fig. 11.13. Services document #1

Now let us post the document and see how the Cost of materials register records change.

4. Click **Post** and then run the **Cost of materials** command (fig. 11.14).

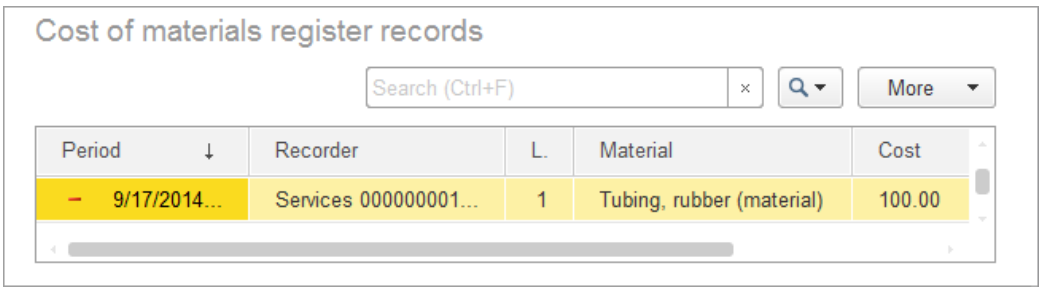


Fig. 11.14. Cost of materials register records

Now let us create and post two more Services documents.

5. In the document list form, click the **Create** button.

   -OR-

   In the **Services** section, on the actions panel, on the **Create** menu, click **Service** (fig. 11.15).

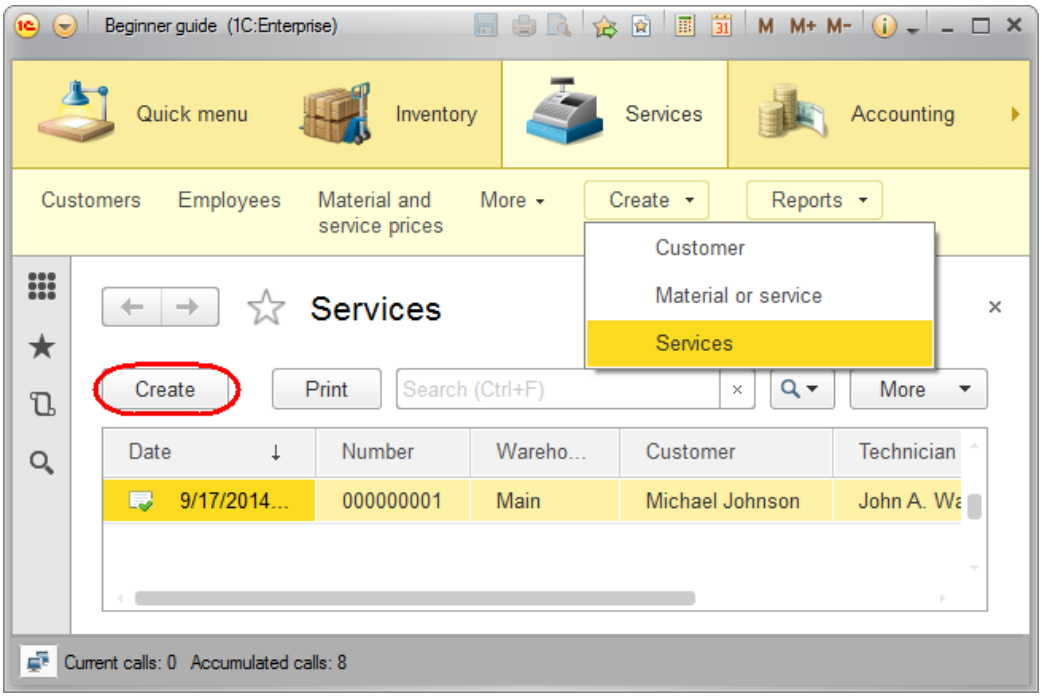


Fig. 11.15. Creating the documents

6. Fill the document as shown in fig. 11.16 and post it, and then create another document, fill it as shown in fig. 11.17, and post it.

   Note that the dates of these documents are different from the dates of the first document, we will utilize this difference in a future lesson.

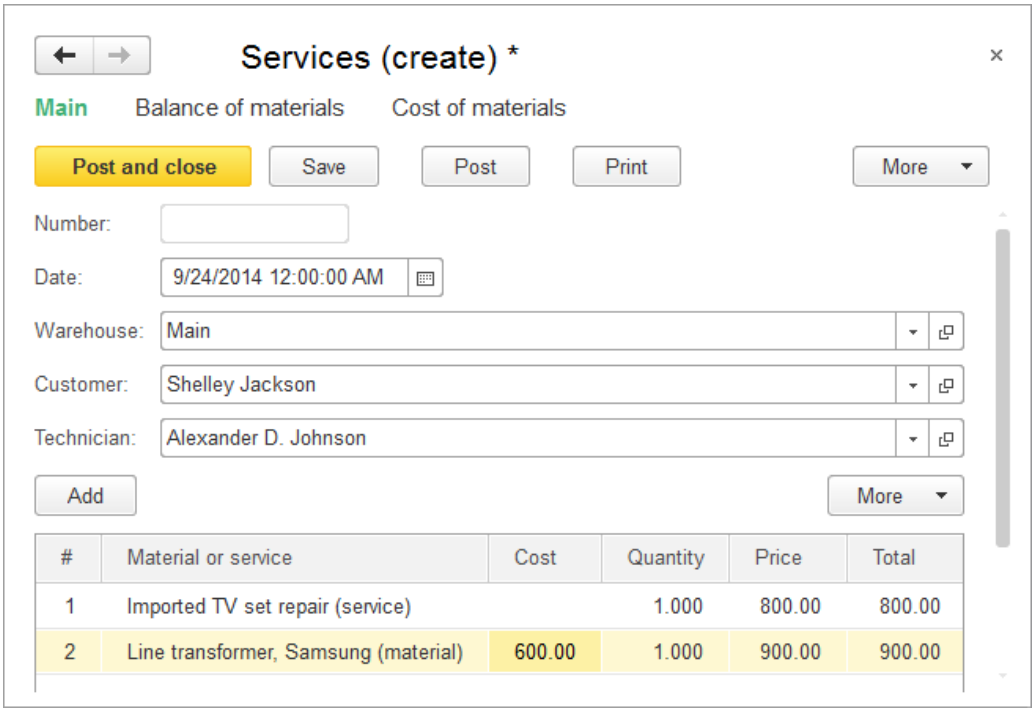


Fig. 11.16. Services document #2

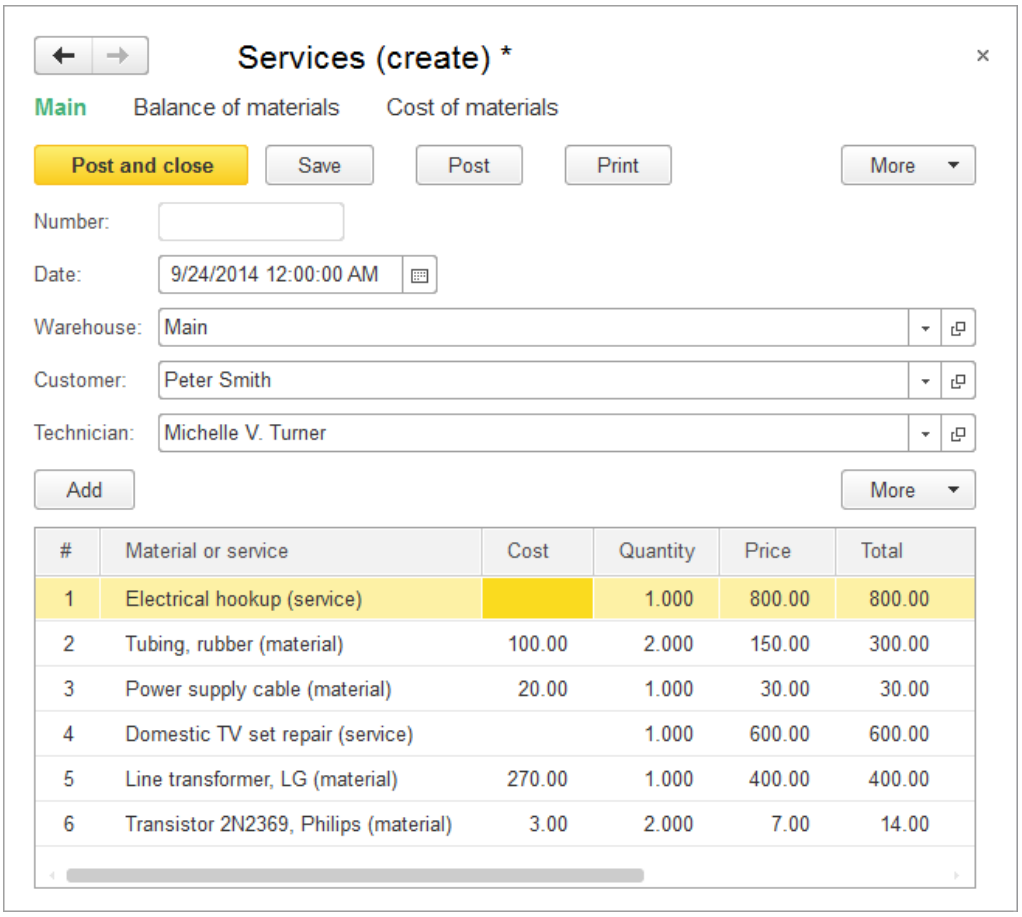


Fig. 11.17. Services document #3

The Cost of materials register records for the Services documents #2 and #3 should look as shown in fig. 11.18 and 11.19.

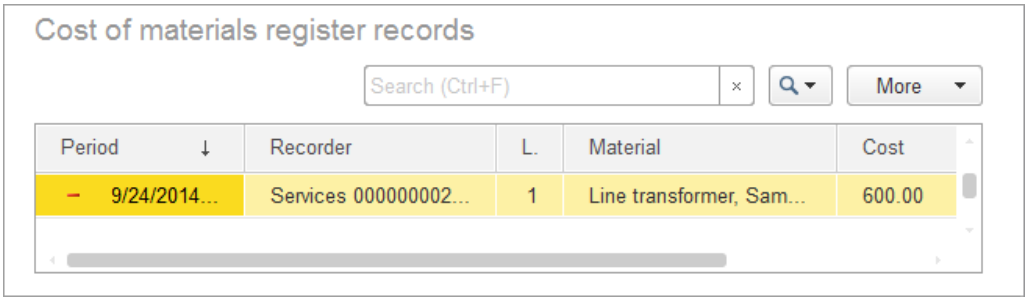


Fig. 11.18. Services document #2 register records

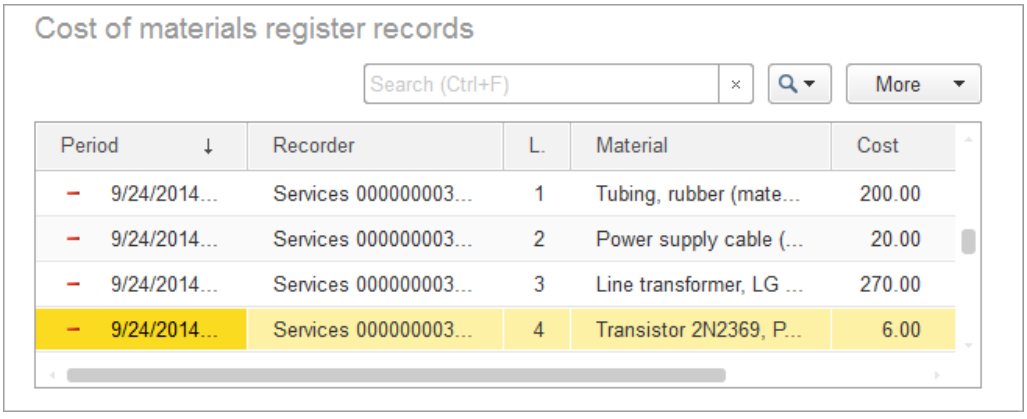


Fig. 11.19. Services document #3 register records

Quiz

- Why might one need to post a document across multiple registers?
- How does one create records for a document across multiple registers in a document posting event handler?
- How does one create document register records without using the register records wizard?
- How does one generate document register records and write them into an accumulation register using 1C:Enterprise script tools?
- How does one add an attribute to a document form?

Lesson 12 (0:40). Turnover accumulation registers

In this lesson you will study another accumulation register type: turnover accumulation register.

You will also learn the rules of selecting dimensions and attributes for accumulation registers.

Then you will create a turnover accumulation register and modify one of your documents to post records to that register.

Why we need another register

Let us look into the Services document again.

So far you have created accumulation register records only for document rows that contained materials. You have not yet implemented the accounting of services listed in the document.

This is because the service accounting logic is quite different from the material accounting logic.

First of all, there is no sense talking about how many services were rendered and how many of them remain. Instead, you need to know the quantity of services that were rendered within a given time period and the total payment for those services.

Additionally you might want to track the following:

- Service type (for building a service popularity rating)
- Customer (for offering discounts based on the volume of services the customer bought in the past)
- Technician who rendered the service (for payroll purposes)

Obviously, the accumulation registers that you have so far are not suitable for the task. Therefore, you have to create one more data storage in your configuration: the Sales turnover accumulation register.

Understanding turnover accumulation registers

In the lessons where you created the BalanceOfMaterials and CostOfMaterials registers we deliberately skipped over discussion of register types available in 1C:Enterprise. Now it is time for a few words on the subject.

Accumulation registers can serve as balance or turnover registers.

The registers that you already have in your tutorial configuration, BalanceOfMaterials and CostOfMaterials, are balance registers.

You can remember that during the creation of the Materials report in the query wizard you saw three virtual tables generated for a register of this

type: the balance table, the turnovers table, and the combined balance and turnovers table.

A turnover accumulation register is very similar to the balance register that you are familiar with, except that the idea of balance is meaningless here. A turnover register only accumulates turnovers and ignores balances entirely. Therefore, the only virtual table the platform generates for this register type is a turnover table. A turnover register is absolutely similar to a balance register in every other respect.

We should mention one distinctive feature in the design of accumulation registers, which is directly related to the ability to get balances. When you create a turnover accumulation register, it is easy to determine which data should serve as register dimensions, and you can assign any data you want to be a register dimension.

The situation is entirely different for an accumulation register that supports accumulation of balances. When selecting dimensions, you should remember that the register has two record directions: receipt and expense. When you specify dimensions, choose data for which both types of register records are totally possible.

For example, if materials are recorded by material type and warehouse, material type and warehouse can obviously be dimensions since both receipt and expense of materials are always made with a reference to a specific material type and a specific warehouse. But if you also need to record materials by vendor, your implementation should be based on the specific accounting procedures used within the company.

Probably a vendor is always specified for material receipts, while for expenses a vendor is most certainly missing. In the majority of scenarios this information is not needed at all.

Therefore, a vendor should be included as an attribute of an accumulation register instead of a dimension.

However, in scenarios where a vendor is certainly specified when materials are expended, adding a vendor register dimension makes sense.

In other words, resources of each balance accumulation register dimension must support changes in both directions: receipt and expense. Resources that support only receipt or only expense are not allowed.

Neglecting this principle of accumulation register design leads to inefficient use of system resources, which impacts the applied solution performance.

This principle is less important for register attributes. Entities that are represented by register attributes can be both only received or only expended.

Adding turnover accumulation registers

In Designer mode

Now that you know "almost everything" about accumulation registers, let us create one.

1. In Designer, create an accumulation register configuration object named *Sales* and set its type to **Turnovers**.
2. Set **Extended list presentation** to *Sales register records* (fig. 12.1).

   The extended list presentation will serve as the title of the register record list window.

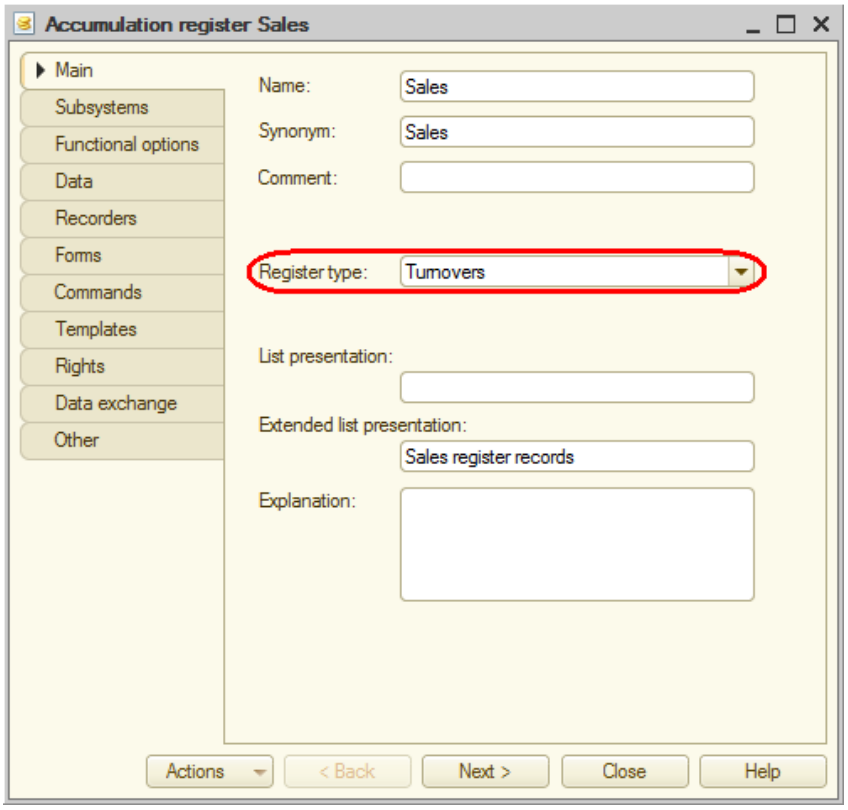


Fig. 12.1. Creating a turnover accumulation register

3. On the **Subsystems** tab, include the register in the **Accounting**, **Inventory**, and **Services** subsystems.
4. On the **Data** tab, create the following dimensions:
   - **MaterialOrService**. Type: CatalogRef.MaterialsAndServices
   - **Customer**. Type: CatalogRef.Customers
   - **Technician**. Type: CatalogRef.Employees

   and the following resources:
   - **Quantity**. Type: Number, length: 15, precision: 3
   - **Revenue**. Type: Number, length: 15, precision: 2
   - **Cost**. Type: Number, length: 15, precision: 2

Once created, the Sales register should look as follows in the configuration tree (fig. 12.2).

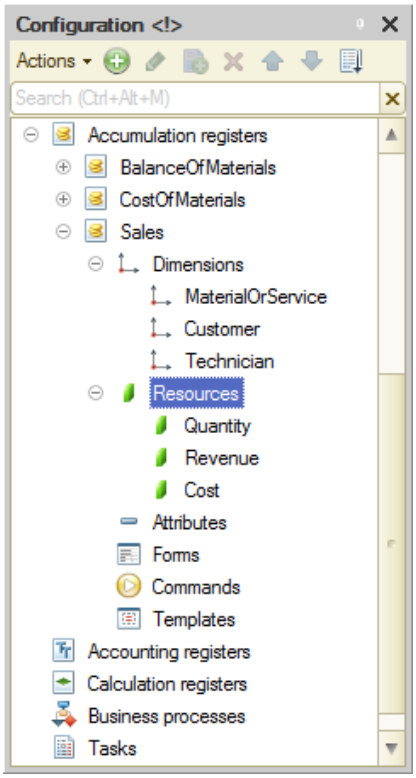


Fig. 12.2. Sales turnover accumulation register

Now let us edit the command interface to add the command that opens the turnover accumulation register to the Accounting, Inventory, and Services sections.

5. In the configuration object tree, right-click the **Subsystems** branch and then click **All subsystems**.
6. In the **All Subsystems** window, in the **Subsystems** list, click **Accounting**.

   The Command interface list displays all the commands of the selected subsystem.
7. In the **Navigation panel.Normal** group, enable visibility for the **Sales** command and drag it to the **Navigation panel.See also** group.
8. Perform the same procedure for the **Inventory** and **Services** subsystems: in the **Navigation panel.Normal** group, enable visibility for the **Sales** command and drag it to the **Navigation panel.See also** group.

Posting Services documents across three registers

Let us first modify the posting procedure of the Services document and then repost all Services documents in 1C:Enterprise mode to create new register records according to the modified posting algorithm.

In Designer mode

Let us modify the document posting procedure.

1. In Designer, open the Services document editor window and click the **Posting** tab.
2. Add the **Sales** register to the list of registers affected by the document.
3. Click the **Other** tab and then click **Object module**.
4. Add the script lines that create Sales register records for the Services document to the handler of the Posting event, to the end of the loop, right after the EndIf operator and before the EndDo operator (listing 12.1).

Listing 12.1. Records of the Services document (fragment)

```
// register Sales
Record = RegisterRecords.Sales.Add();
Record.Period = Date;
Record.MaterialOrService =
    CurRowMaterialsAndServices.MaterialOrService;
Record.Customer = Customer;
Record.Technician = Technician;
Record.Quantity = CurRowMaterialsAndServices.Quantity;
Record.Revenue = CurRowMaterialsAndServices.Total;
Record.Cost = CurRowMaterialsAndServices.Quantity
    * CurRowMaterialsAndServices.Cost;
```

5. Before the loop set the Write property of the register record to True. The resulting procedure should look as shown in listing 12.2.

Listing 12.2. Records of the Services document

```
Procedure Posting(Cancel, Mode)
 RegisterRecords.BalanceOfMaterials.Write = True;
 RegisterRecords.CostOfMaterials.Write = True;
 RegisterRecords.Sales.Write = True;

 For Each CurRowMaterialsAndServices In MaterialsAndServices Do
   If CurRowMaterialsAndServices.MaterialOrService.MaterialServiceType
     = Enums.MaterialServiceTypes.Material Then

     // register BalanceOfMaterials Expense
     Record = RegisterRecords.BalanceOfMaterials.Add();
     Record.RecordType = AccumulationRecordType.Expense;
     Record.Period = Date;
     Record.Material = CurRowMaterialsAndServices.MaterialOrService;
     Record.Warehouse = Warehouse;
     Record.Quantity = CurRowMaterialsAndServices.Quantity;
```

```
      // register CostOfMaterials Expense
      Record = RegisterRecords.CostOfMaterials.Add();
      Record.RecordType = AccumulationRecordType.Expense;
      Record.Period = Date;
      Record.Material = CurRowMaterialsAndServices.MaterialOrService;
      Record.Cost = CurRowMaterialsAndServices.Quantity
        * CurRowMaterialsAndServices.Cost;

    EndIf;

    // register Sales
    Record = RegisterRecords.Sales.Add();
    Record.Period = Date;
    Record.MaterialOrService =
      CurRowMaterialsAndServices.MaterialOrService;
    Record.Customer = Customer;
    Record.Technician = Technician;
    Record.Quantity = CurRowMaterialsAndServices.Quantity;
    Record.Revenue = CurRowMaterialsAndServices.Total;
    Record.Cost = CurRowMaterialsAndServices.Quantity
      * CurRowMaterialsAndServices.Cost;

  EndDo;
EndProcedure
```

You already know all the added expressions well.

Just note that the turnover register does not have a RecordType property, since specifying the record type (receipt or expense) only makes sense for balance accounting. As for turnover registers, we are only interested in the value that is written to the register resource.

Also note that you added the commands that create Sales register records to the end of the loop that iterates through the document tabular section, to the part that is not affected by the condition that checks whether an item is a material. This is important because in this register records are created for both materials and services.

Finally, let us edit the document form command interface to add the link for viewing the Sales register records related to the document to the form navigation panel.

1. Open the **Services** document form.
2. In the upper left pane, click the **Command interface** tab.
3. In the **Navigation panel** branch, expand the **Go to** node.

   It contains the command that opens the list of the Sales accumulation register.
4. Select the **Visible** check box for this command.

In 1C:Enterprise mode

Let us repost all of the Services documents in 1C:Enterprise mode and check that they create correct records in the Sales register.

1. Start 1C:Enterprise in the debug mode.
2. For each Services document, open it, click **Post**, and then open the Sales register records.

   The sales records should look as shown in fig. 12.3a, 12.3b, 12.4a, 12.4b, 12.5a, and 12.5b.

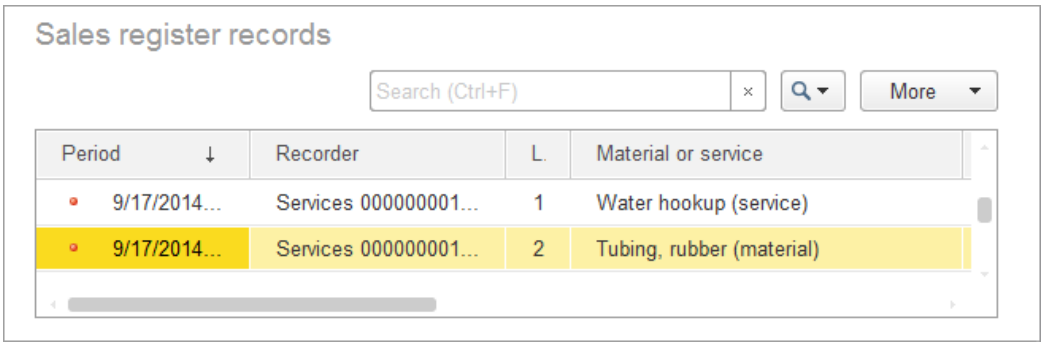


Fig. 12.3a. Services document #1 records in the Sales register

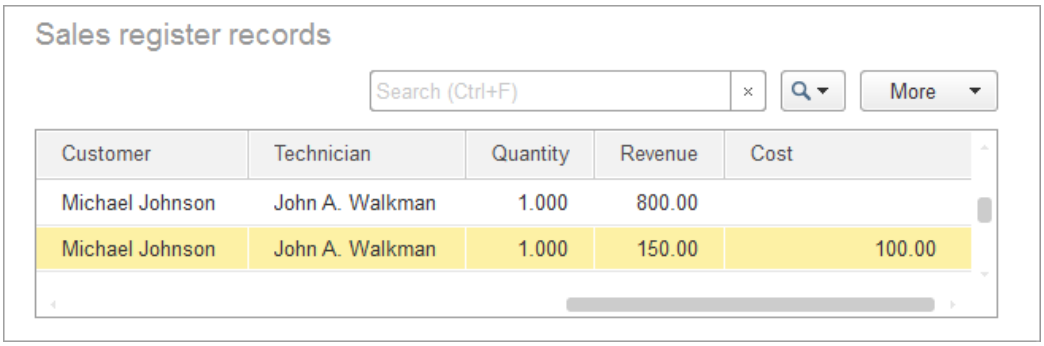


Fig. 12.3b. Services document #1 records in the Sales register

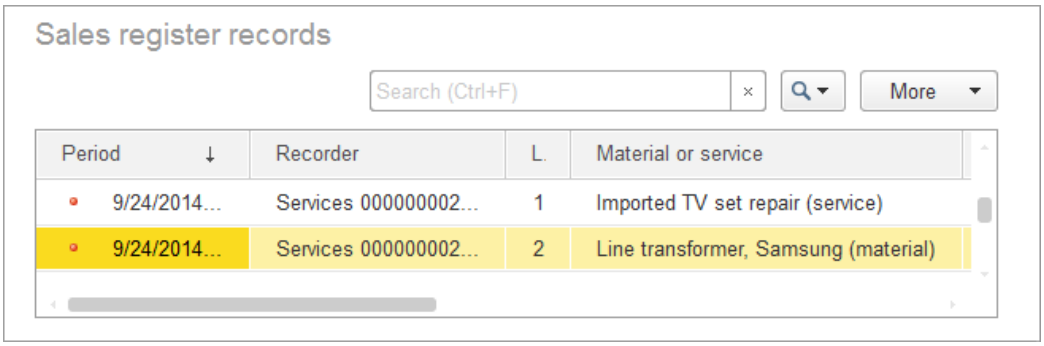


Fig. 12.4a. Services document #2 records in the Sales register

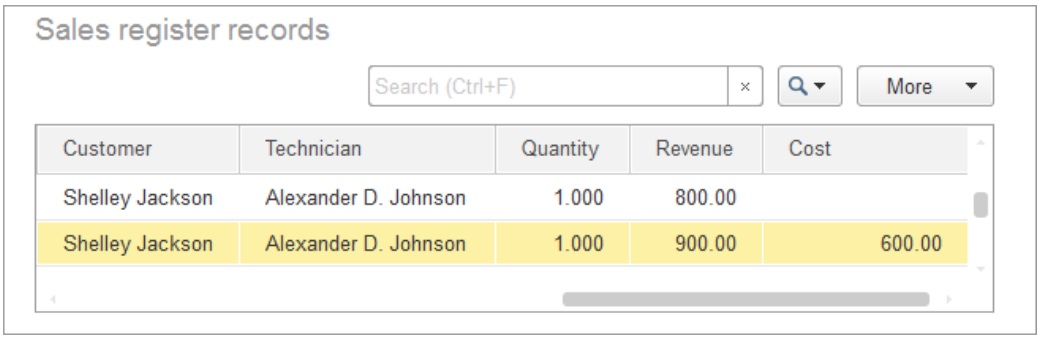


Fig. 12.4b. Services document #2 records in the Sales register

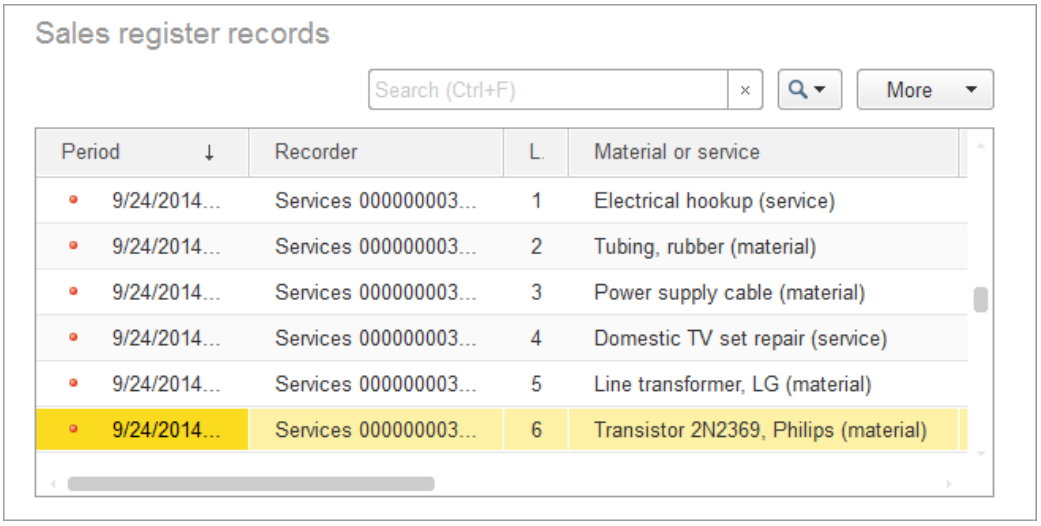


Fig. 12.5a. Services document #3 records in the Sales register

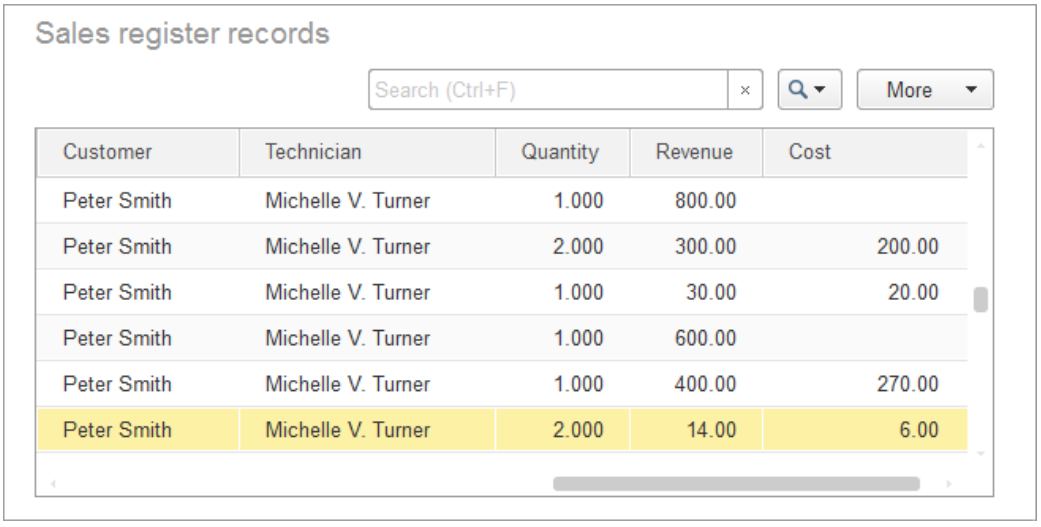


Fig. 12.5b. Services document #3 records in the Sales register

You can see that the Sales register stores both material and service records.

Now you have virtually all the data you need to analyze operations of Jack of All Trades.

In the next lesson you will create some reports that will provide you with summary information on the company operations.

Quiz

- What is a turnover accumulation register?
- What is the difference between balance accumulation registers and turnover accumulation registers?
- How does one select attributes and dimensions when creating accumulation registers?
- How does one create a turnover accumulation register?

Lesson 13 (4:30). Reports

Since the overall duration of the lesson is quite long, you can go through the lesson in parts and pause after the development of each of the six reports included in the lesson.

Now it is time to learn the data composition system, which is a significant part of the 1C:Enterprise platform. In this lesson you will implement several reports for your configuration. We will explain the main features of the data composition system on the example of these reports.

As a rule, a report is based on a complex selection of data that is grouped and ordered as specified. The data composition system is a powerful and flexible tool covering all major report functionality, from obtaining data from various sources to displaying the data in a user-friendly manner.

In most scenarios the report source data is located in the database. You can use 1C:Enterprise query language for specifying the data sources for the data composition system and which data is retrieved from these sources.

On the report development stage you can define the default report settings, so that users can run the report right away. At the same time a user can change the report settings manually and then run the report. At that the data composition system generates another query and presents the result in a different manner, according to the user-specified settings.

At the beginning of the lesson we will provide general information about the 1C:Enterprise query language and the data composition system.

Then we will explain how to use the data composition system for solving practical tasks on the example of several reports.

Theory. Data access methods

1C:Enterprise supports two methods of accessing data stored in a database:

- Object method (used for reading and writing)
- Table method (used for reading)

The object method involves 1C:Enterprise script objects. You have encountered some of these objects in the previous lessons.

An important feature of object data access method is that when you call a 1C:Enterprise script object, you call a data set stored in the database as a whole.

For example, the DocumentObject.Services object contains the values of all attributes and tabular sections of the Services document.

The object method features object caching, ensures object integrity and execution of appropriate event handlers, and so on.

The table method involves database queries that are written in the query language. You can use this method to access specific database table fields.

The table method is intended for retrieving data from the database based on certain conditions, such as filters, grouping, ordering, merging several selections, calculation of totals, and so on. The table method is optimized

for processing large volumes of data stored in the database, and for obtaining data that matches specific criteria.

Using queries

Queries are based on the 1C:Enterprise script object named Query. It is intended for getting data stored in database fields in the form of selections based on the specified rules.

Query data sources

Queries obtain their source data from a set of tables. These tables provide data from actual database tables in a form that is convenient for analysis.

All the tables that can be accessed using the query language can be divided into two large groups: actual tables and virtual tables (fig. 13.1).

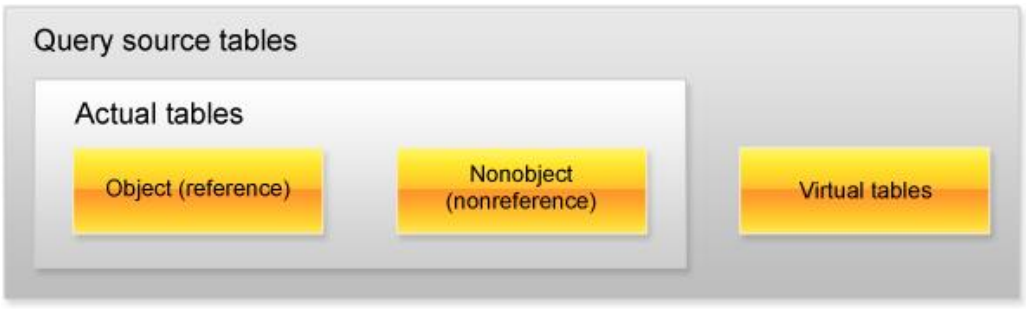


Fig. 13.1. Query tables

You can view the assortment of tables that can be accessed using queries along with their descriptions in the Syntax Assistant, in section Working with queries / Query tables.

Each actual table contains data from a single database table.

For example, the Catalog.Customers actual table corresponds to the Customers catalog, while the AccumulationRegister.BalanceOfMaterials table corresponds to the BalanceOfMaterials accumulation register.

Virtual tables are usually generated based on data available in multiple database tables. For example, AccumulationRegister.BalanceOfMaterials.BalanceAndTurnovers is a virtual table that is generated based on multiple tables of the BalanceOfMaterials register.

In some cases virtual tables can be generated based on a single actual table (for example, the Prices.SliceLast virtual table is generated based on the Prices information register table).

However, all the virtual tables share one common feature: they can be assigned a number of parameters that define which data is included in these tables. The assortment of parameters may vary for different virtual tables, it is defined by the data stored in the source database tables.

There are two actual table types: object (referential) and nonobject (nonreferential). Object (referential) tables store data of referential types: catalogs, documents, charts of characteristic types, and so on. Nonobject

(nonreferential) tables store all the other data types: constants, registers, and so on.

The distinctive feature of object (referential) tables is the fact that they contain the Ref field, which is linked to the current record. You can obtain user object presentations from such tables as well. These tables can be hierarchical and their fields can include nested tables (tabular sections).

Query language

The algorithm that selects data from query source tables is defined using the query language.

A query text can include several parts:

- Query definition
- Query union
- Ordering query result
- Autoordering
- Description of totals

The only mandatory part is the first one, the query definition. All the other parts are optional.

A query definition specifies data sources, selection fields, groupings, and so on.

A query union specifies how the results of several queries are merged.

Ordering query result specifies how query result rows are ordered.

Autoordering enables automatic ordering of query result rows.

Description of totals specifies which totals are calculated in the query and how the results are grouped.

Note that when the query language is used to describe data sources in the data composition system, the description of totals query part is not used because the data composition system calculates the totals on its own based on the settings provided by the developer or by the user.

The query language syntax structures are described in detail in the built-in Designer help.

To open the built-in help

1. On the **Help** menu, click **Help contents**.
2. In the help contents, open section **1C:Enterprise** / **Script** / **Working with queries**.

Another source of information about query language syntax is chapter "Work with Queries" of the 1C:Enterprise Developer Guide.

We will discuss the query language in more detail later, during the creation of specific reports.

You will not create any queries manually. For the majority of reports developed using the data composition system, you can create a query using the query wizard.

Your goal for this lesson is to learn how to read and understand query text, so that you can edit queries in the future.

Data composition system

The data composition system is intended for creating custom reports in 1C:Enterprise system. It consists of several major parts.

A data composition schema stores the source data for report composition, which includes data sets and data processing methods (fig. 13.2).

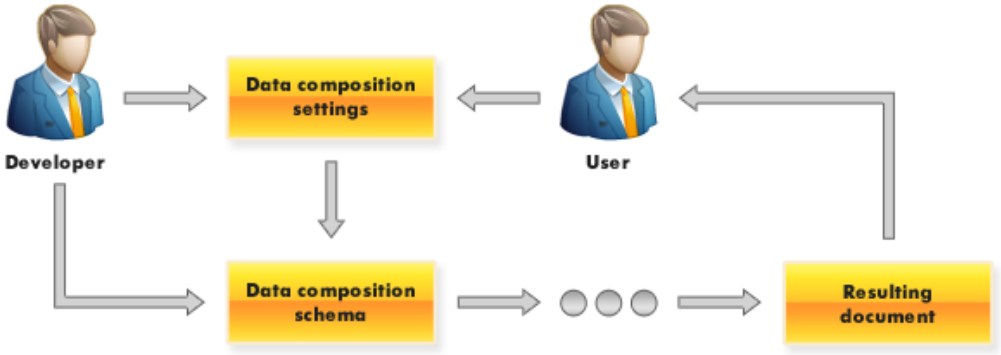


Fig. 13.2. General data composition system flowchart

A developer creates a data composition schema, which includes the query text, data sets and connections between them, available fields, data access parameters, and also default composition settings, such as report structure or data appearance template.

For example, a composition schema can include a data set as shown in fig. 13.3.

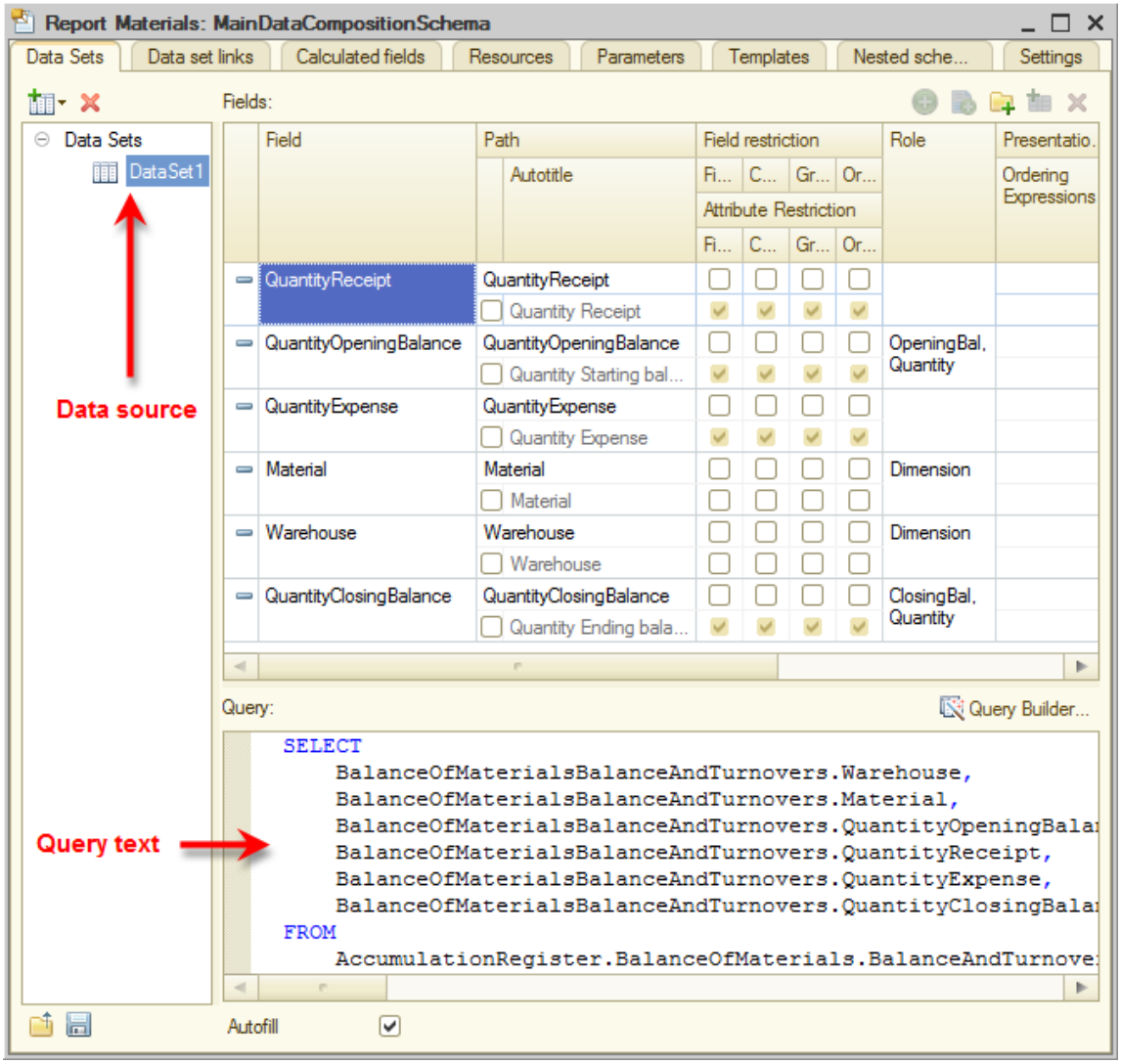


Fig. 13.3. Sample composition schema (a data set and a query that uses the set)

The figure shows the data composition schema wizard window, which contains the data source, the query text, and the fields selected by the query.

When a user runs a report based on the data composition system, what they get is not simply a table of records that satisfy the query conditions. A composition system report has a complex hierarchical structure, which can include a variety of items: groupings, tables, charts, and more.

A user can edit the report structure or create an entirely new report structure. A user can also customize the selection used in the report and the appearance of report structure items, view details of each item, and more.

For example, a report structure can include a single table and a single chart, as shown in fig. 13.4.

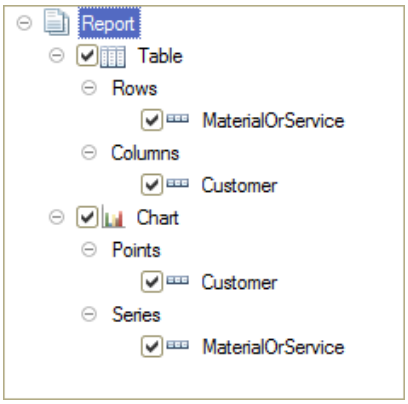


Fig. 13.4. Sample report structure

In this case the generated report looks as shown in fig. 13.5.

Material or service	Michael Johnson Revenue Turnover	Peter Smith Revenue Turnover	Shelley Jackson Revenue Turnover	Total Revenue Turnover
Domestic TV set repair (service)		600.00		600.00
Electrical hookup (service)		800.00		800.00
Imported TV set repair (service)			800.00	800.00
Line transformer, LG (material)		400.00		400.00
Line transformer, Samsung (material)			900.00	900.00
Power supply cable (material)		30.00		30.00
Transistor 2N2369, Philips (material)		14.00		14.00
Tubing, rubber (material)	150.00	300.00		450.00
Water hookup (service)	800.00			800.00
Total	950.00	2,144.00	1,700.00	4,794.00

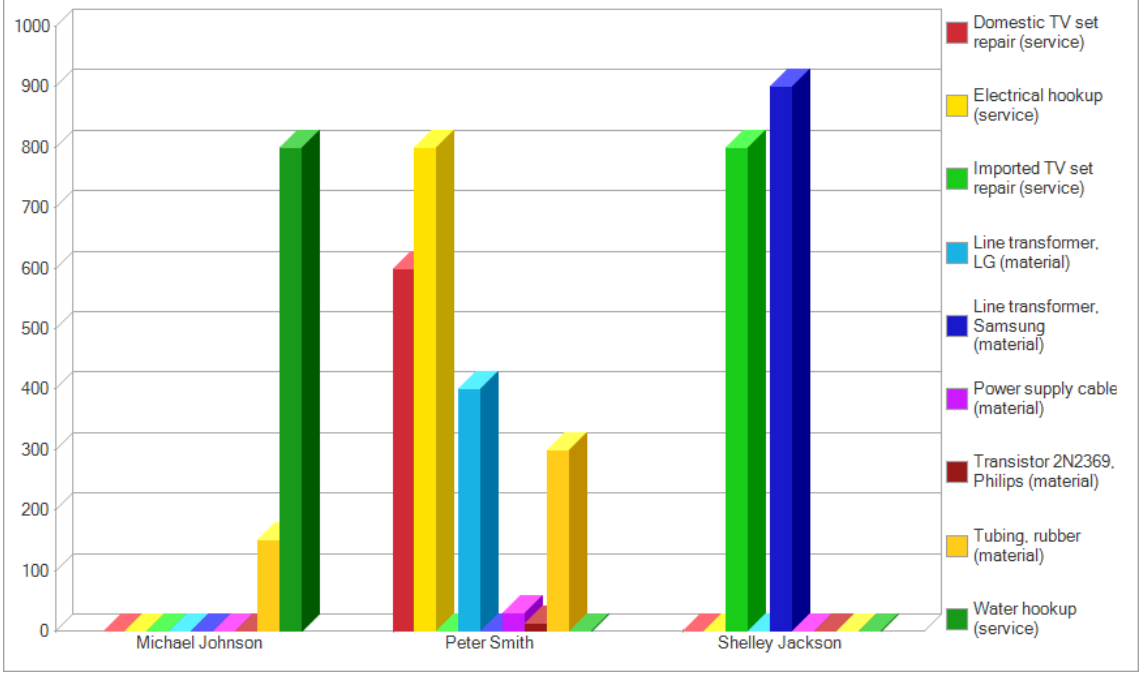


Fig. 13.5. Sample report

The sample report contains customer and service data from the SalesTurnover accumulation register, which is displayed as a table and as a chart.

As we have mentioned earlier, the data composition system is an aggregate of multiple objects. When a user runs a report, data is transferred consecutively from one data composition system object to another until the resulting report (a document displayed to the user) is generated.

The interaction between the objects is based on the following algorithm:

- A developer creates a *data composition schema* and the *default settings*. Usually a single data composition schema can serve as a basis for a large variety of reports, while data composition settings (either created by a developer or modified by a user) define the report layout.
- The *template composer* creates a *template* based on the composition schema and available settings. This step is a preparation for report execution. A data composition template is a complete assignment for the composition processor. It includes the required queries, report area templates, and more.
- The *data composition processor* selects data from the infobase based on the composition template, then aggregates and arranges that data.
- The *output processor* processes the composition result and the user obtains the resulting spreadsheet document.

This algorithm can be presented as a flowchart (fig. 13.6).

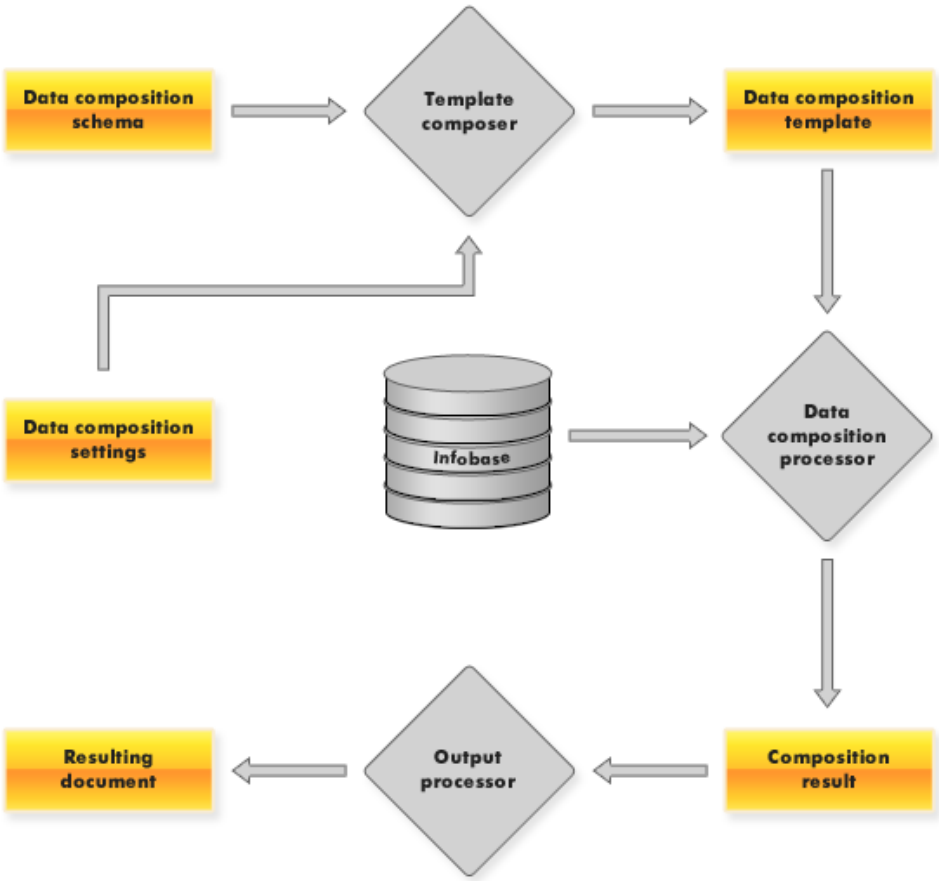


Fig. 13.6. Flowchart of composition system operations

Selecting data from a single table

Let us create a report named Services document register using the data composition system.

Using this report as an example, we will show how to select data from a single database table and output the data in a specific order. You will also get acquainted with the specifics of the drilldown feature.

This report displays the list of Services documents available in the database, sorted by their date and number (fig. 13.7).

Document	Warehouse	Technician	Customer
Services 000000002 from 9/24/2014 4:03:16 PM	Main	Alexander D. Johnson	Shelley Jackson
Services 000000003 from 9/24/2014 4:12:46 PM	Main	Michelle V. Turner	Peter Smith
Services 000000001 from 9/17/2014 4:40:34 PM	Main	John A. Walkman	Michael Johnson

Fig. 13.7 Resulting report

In Designer mode

Let us repeat the first steps of report generation that are described in lesson 7 (page 216).

1. Create a report configuration object.
2. On the **Main** tab, set **Name** to *ServicesDocumentRegister*.
3. Set **Extended presentation** to *List of rendered services*.

   This report name will be displayed in the applied solution interface.

   Now let us create the data composition system.
4. Click **Open data composition schema**.

   -OR-

   In the **Main data composition schema** field, click the **Open** button (fig. 13.8).

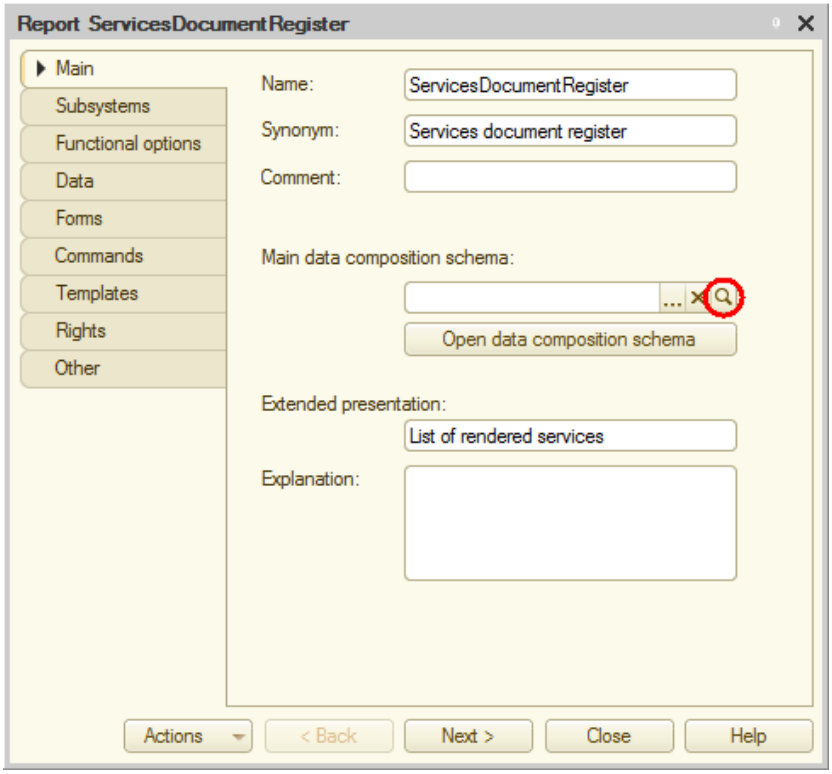


Fig. 13.8. Specifying main report properties

5. In the **Template Wizard** window, click **Finish**.
6. In the data composition schema wizard, add **Data set – query** (fig. 13.9).

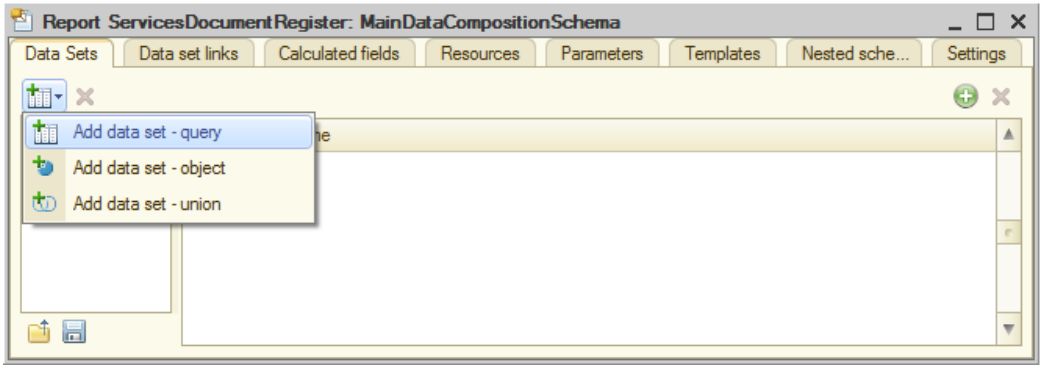


Fig. 13.9. Adding data set (query)

Query for a data set

Let us specify the query data set.

1. Click the **Query Builder** button.
2. Select the **Services** table as the data source.

   It is the object (referential) table of the Services document.
3. Select the following fields from this table (fig. 13.10):
   - **Warehouse**
   - **Technician**

- **Customer**
- **Ref**

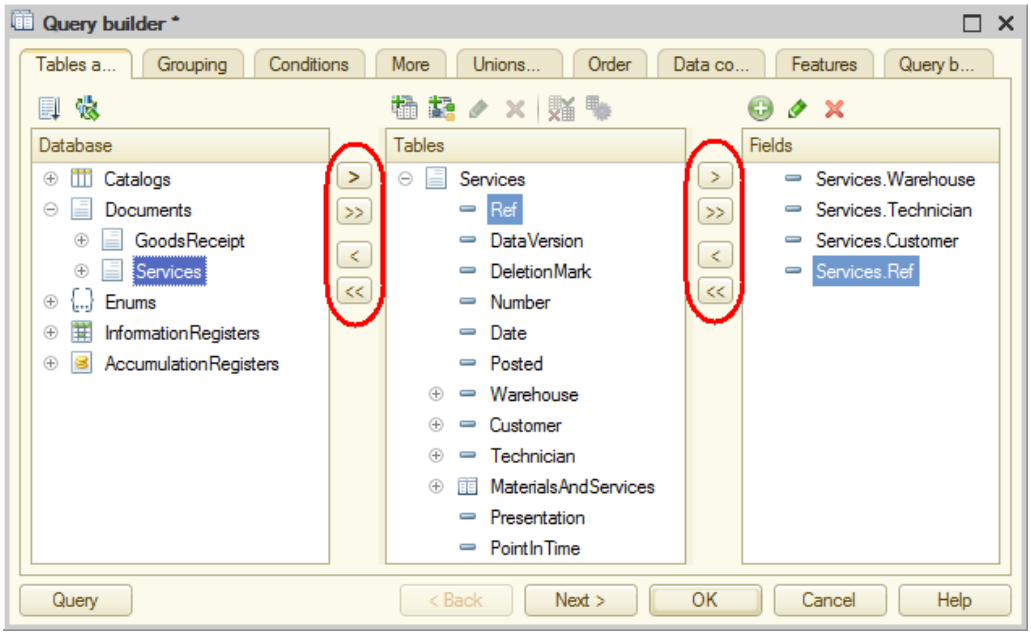


Fig. 13.10. Fields selected for the query

Note. You can move the selected items between the lists by dragging them or by double-clicking them. You can also use the >, <, >>, and << buttons.

Field aliases

Let us set an alias for the Ref field.

1. Click the **Unions/Aliases** tab.
2. In the **Field name** column, change *Ref* to *Document* (fig. 13.11).

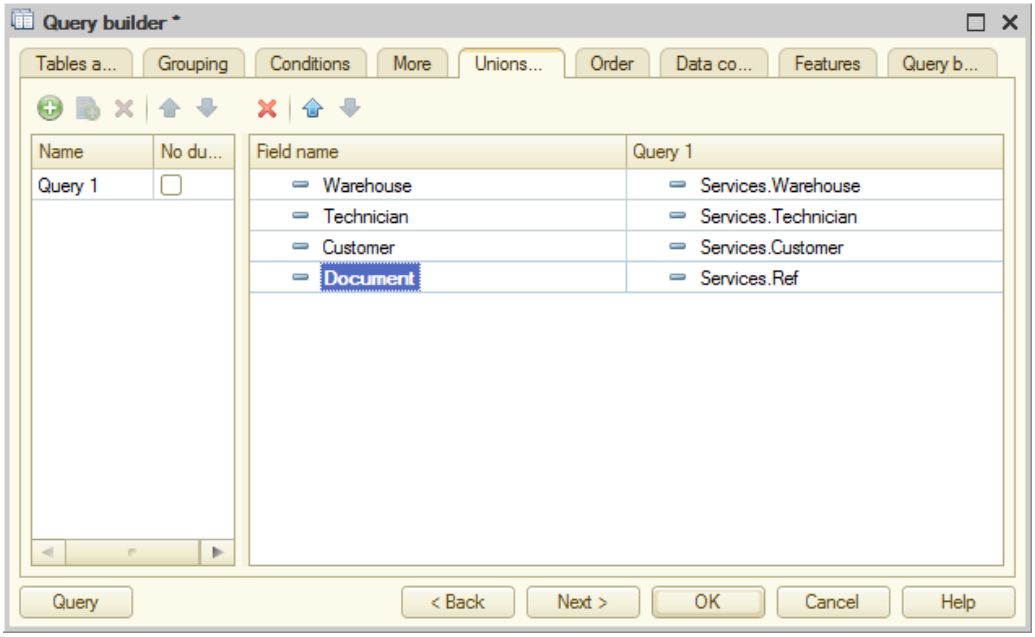


Fig. 13.11. Specifying query field aliases

Note. If you want to change a field name, we recommend that you do it in a query because field names are replicated to three data composition schema columns at once: Field, Path, and Title. This eliminates the need to change them in three locations.

Records order

Let us specify the order of report records.

- Click the **Order** tab and specify that the query result is ordered by the value of the **Document** field (fig. 13.12).

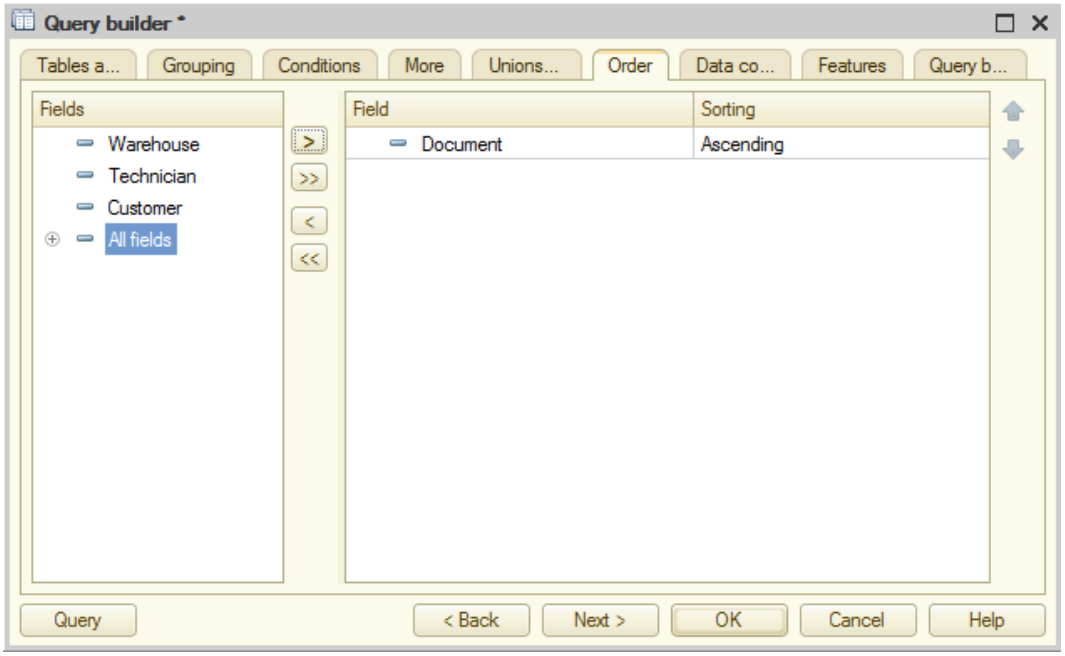


Fig. 13.12. Specifying the order of records in a query

Analyzing query text

Let us analyze the generated query text.

- Click **OK** and review the query generated by the query wizard (listing 13.1).

Listing 13.1. Query text

```
SELECT
    Services.Warehouse,
    Services.Technician,
    Services.Customer,
    Services.Ref AS Document
FROM
    Document.Services AS Services


ORDER BY
    Document
```

The query text begins with the query description (listing 13.2).

Listing 13.2. Query description

```
SELECT
    Services.Warehouse,
    Services.Technician,
    Services.Customer,
    Services.Ref AS Document
FROM
    Document.Services AS Services
```

A query description always begins with the SELECT keyword followed by the list of selection fields. This list describes the fields that are included in the query result. The list can contain the fields themselves as well as certain expressions calculated based on the field values.

The data sources (the source query tables whose contents are processed in the query) are specified after the FROM keyword. In this case the data source is the Document.Services object (referential) table.

Aliases of the source data are specified after the AS keyword. In this case the alias is Services. Thereafter, the query text can reference that data source using the alias (listing 13.3).

Listing 13.3. Description of selection fields

```
SELECT
    Services.Warehouse,
    Services.Technician,
    Services.Customer,
    Services.Ref AS Document
FROM
    Document.Services AS Services
```

Selection fields can also have aliases that you can use to address these fields in the query text. In this case the alias of the Ref field is Document.

After the query description section, the query continues with the result ordering section (listing 13.4).

Listing 13.4. Ordering query results

```
ORDER BY
    Document
```

The ORDER BY clause is intended for sorting query result rows. It is followed by the ordering expression, which is generally a list of fields or expressions with their output order options.

In this case the result is ordered by the Document field (which is actually the Services.Ref field). The result is sorted in ascending order (if sorting order is not specified explicitly, the ascending order is applied).

Now let us proceed to specifying data composition schema settings.

Settings

Let us specify the default report view settings.

1. Click the **Settings** tab.

   A hierarchical report structure can include various combinations of three key items:

   - **Grouping**, used for displaying data as a regular linear report
   - **Table**, used for displaying data as a table
   - **Chart**, used for displaying data as a chart

2. Right-click **Report** and then click **New grouping**.

   -OR-

   In the window command bar, click the **Add** button and then click **New grouping**.

   -OR-

   Click **Report** and then press Insert (fig. 13.13).

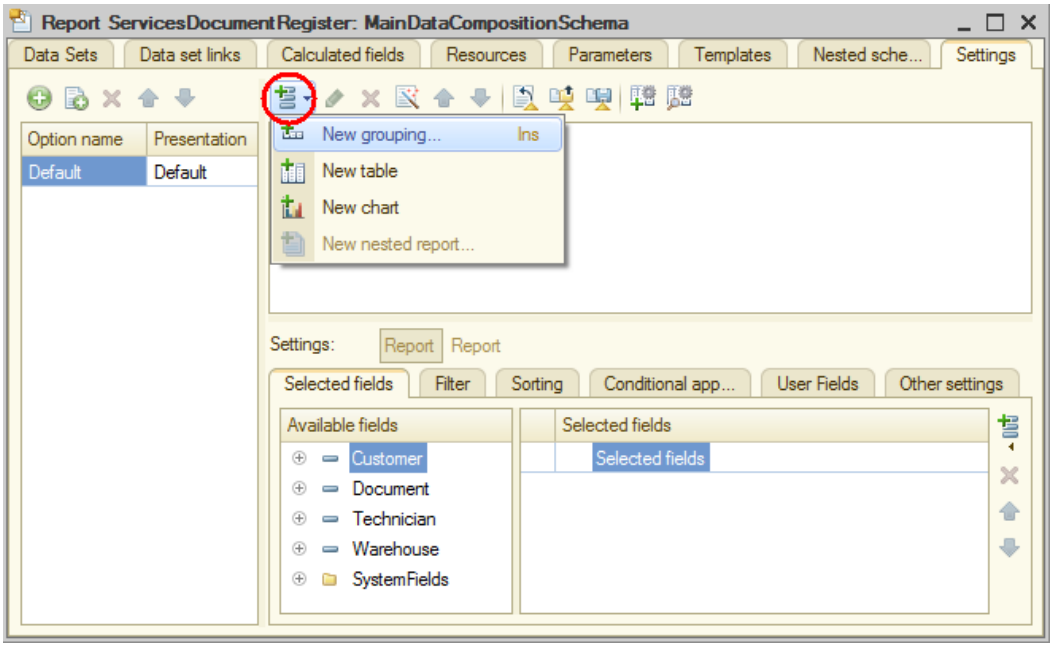


Fig. 13.13. Adding a grouping to a report

3. Do not enter the grouping field, simply click **OK**.

   This specifies that the report includes detailed infobase records.

   The *Detailed records* grouping is added to the report.

4. In the bottom pane, on the **Selected fields** tab, drag the following fields from the **Available fields** list to the **Selected fields** list:

   - **Document**
   - **Warehouse**
   - **Technician**
   - **Customer**

Note. You can also add available fields to the list of selected fields by double-clicking the fields or by clicking **Add** to the right of the list of selected fields. You can change the order of selected fields by dragging them or by using the **Move up** and **Move down** buttons.

Now the report settings window should look as follows (fig. 13.14).

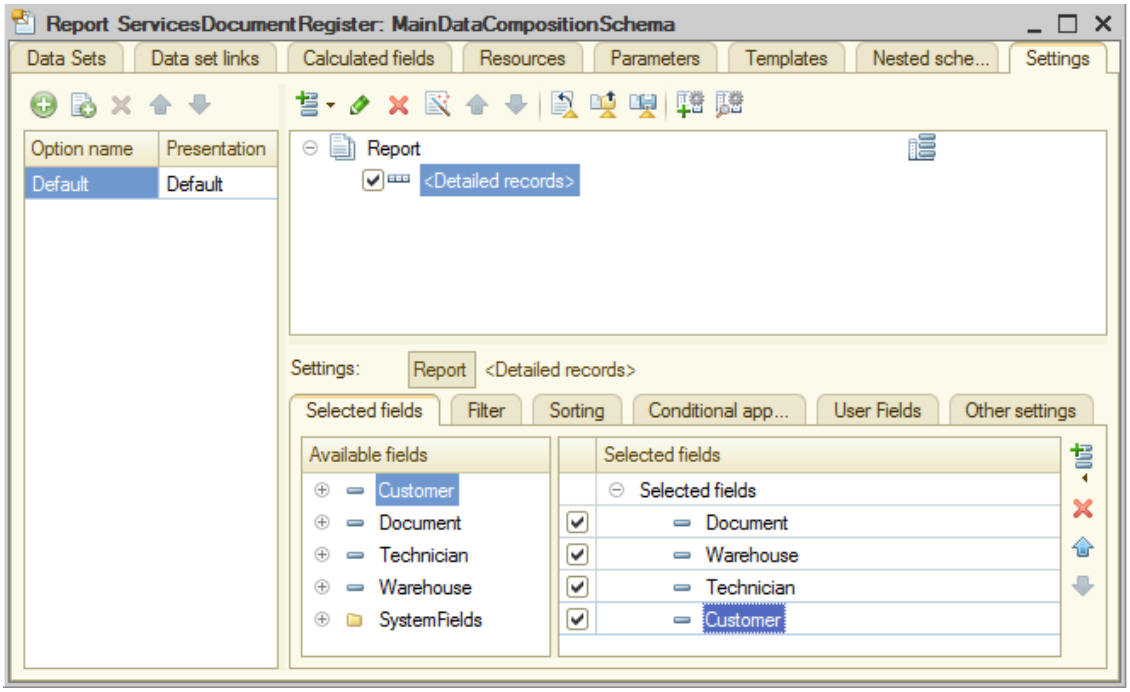


Fig. 13.14. Report settings window

This completes the report creation.

Finally, let us specify the subsystems where the report is available.

5. Close the data composition schema wizard and then, in the editor of the **ServicesDocumentRegister** report configuration object, click the **Subsystems** tab.
6. Select the **Services** check box (fig. 13.15).

   This adds the link that opens the report to the **Reports** menu in the command panel of this subsystem.

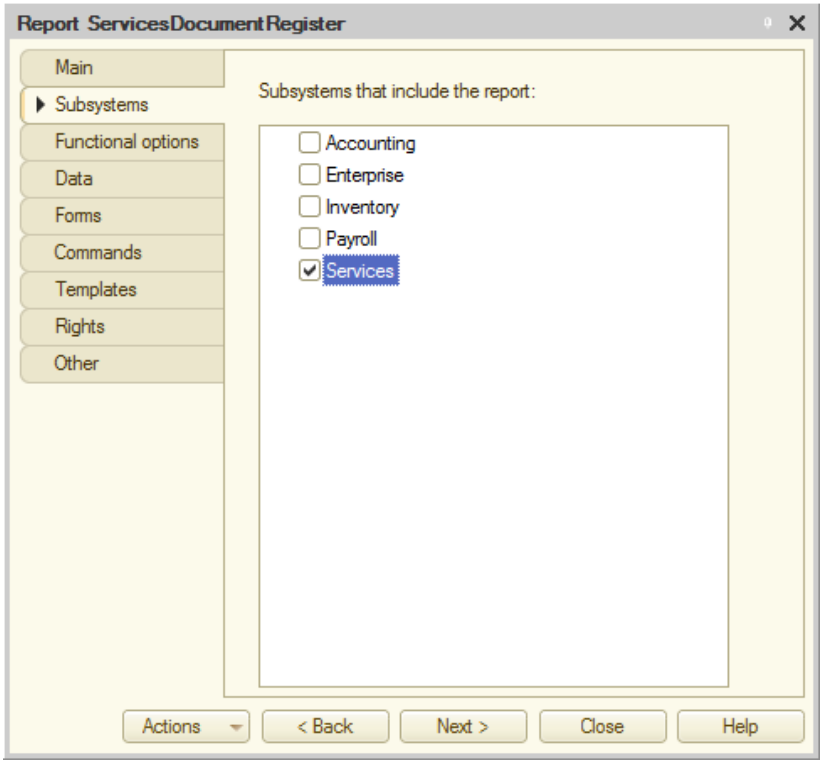


Fig. 13.15. Specifying the list of subsystems where the report is available

In 1C:Enterprise mode

Let us check the report.

1. Start 1C:Enterprise in the debug mode.

   You can see that the Services section now includes the **Reports** menu, which contains the command that generates the Services document register report.

2. Click the **Services document register** command (fig. 13.16).

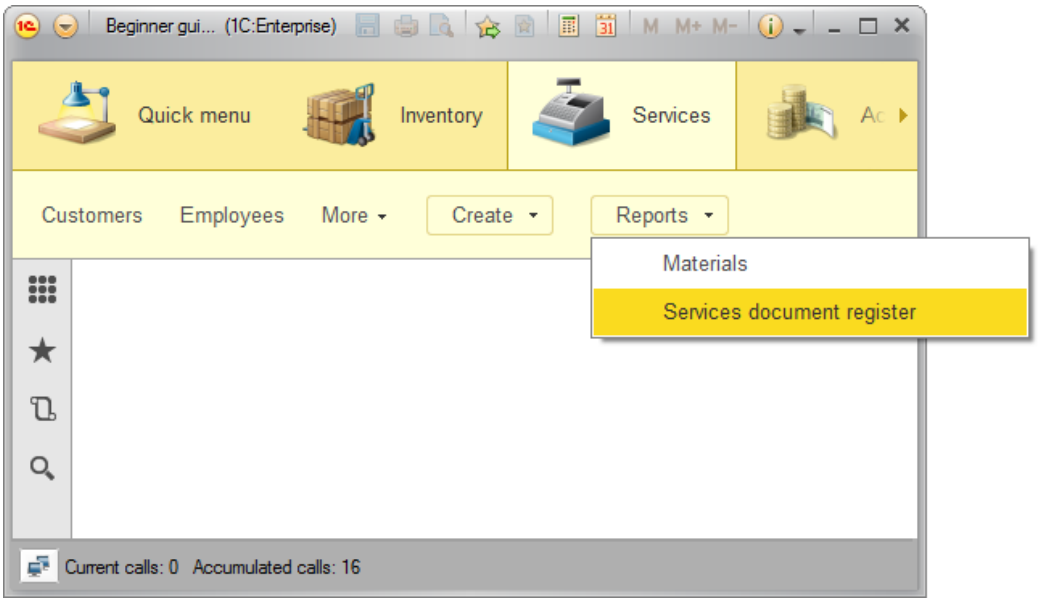


Fig. 13.16. The command that generates the Services document register report

This opens an automatically generated report form.

Note that the form title is List of services rendered, it is defined by the Extended presentation property that you specified for the report.

3. Click **Create** (fig. 13.17).

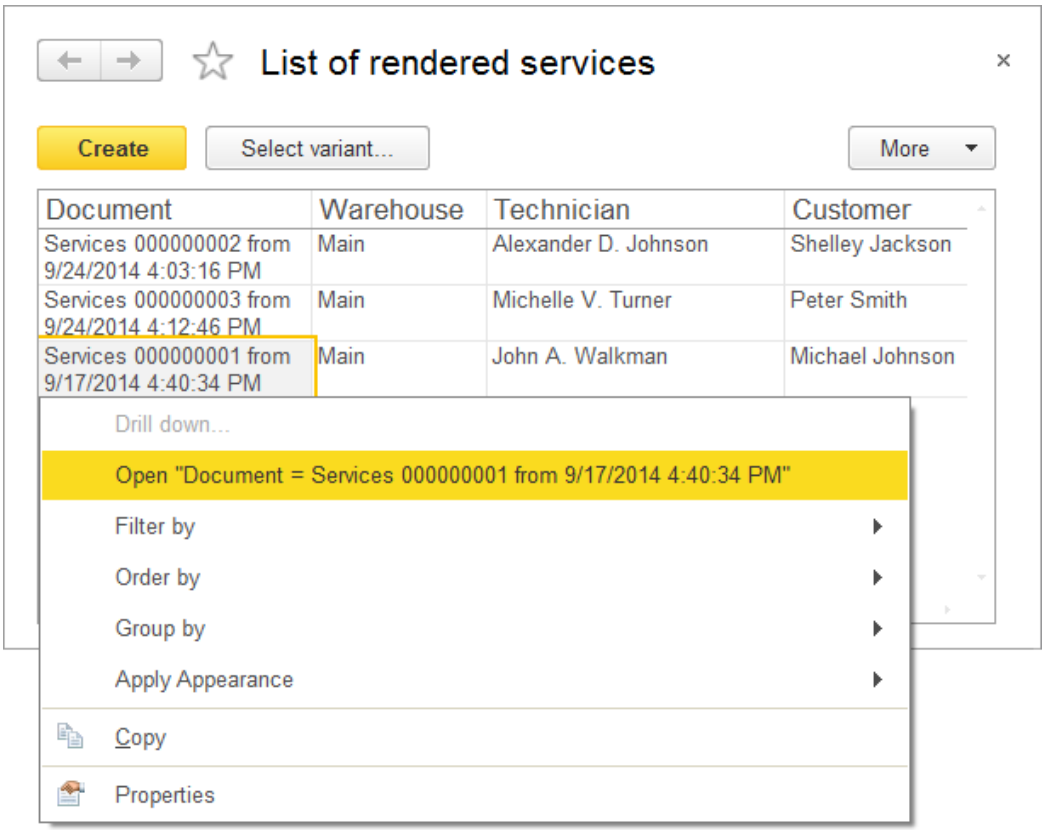


Fig. 13.17. List of rendered services report

As you can see, the report lists the Services documents.

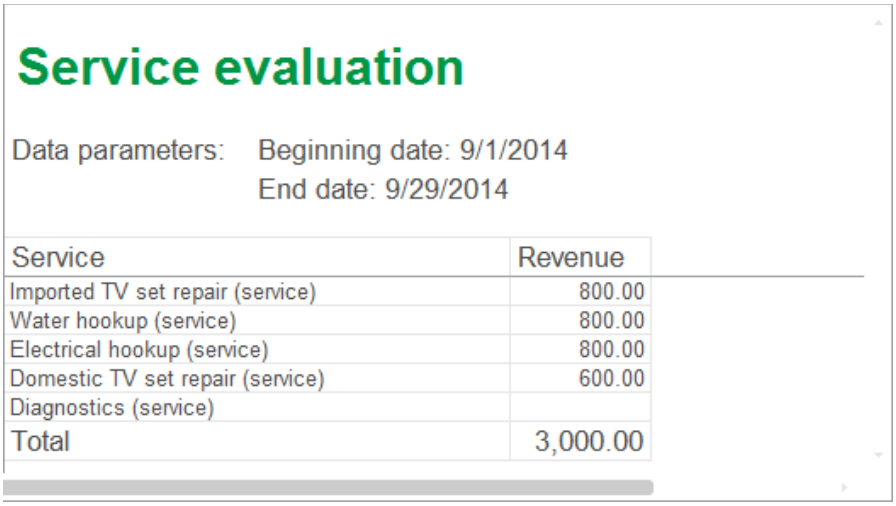
Double-clicking in the **Document** column opens the respective document.

Right-clicking a report cell opens a pop-up menu with commands that apply to the cell value: **Open**, **Drill down**, and so on.

In the report development exercise you learned how to use the data composition schema wizard and made yourself familiar with a few basic query language statements.

Selecting data from two tables

The Service evaluation report displays the most profitable services for Jack of All Trades within a given period (fig. 13.18).



Service evaluation

Data parameters: Beginning date: 9/1/2014
End date: 9/29/2014

Service	Revenue
Imported TV set repair (service)	800.00
Water hookup (service)	800.00
Electrical hookup (service)	800.00
Domestic TV set repair (service)	600.00
Diagnostics (service)	
Total	3,000.00

Fig. 13.18. Resulting report

Using this report as an example, we will show how to select data within a given period, how to set query parameters, how to use data from multiple tables in a query, and how to include all the data from a single source to the query result.

You will also learn how to work with data composition system parameters and how to use default dates, and you will familiarize yourself with quick user report settings.

Finally, you will learn more about the report filter and conditional appearance settings.

In Designer mode

Let us proceed to report creation.

1. Create a report configuration object named *ServiceEvaluation*.
2. Run the Data composition schema wizard.
3. Add **Data set – query** and run the Query builder.

Query for a data set

Left join of two tables

Let us specify two tables as query data sources.

1. Drag the **MaterialsAndServices** object (referential) table and the **Sales.Turnovers** accumulation register virtual table to the **Tables** list.

In order to avoid name ambiguity in the query, let us rename the MaterialsAndServices table.

2. In the **Tables** list, right-click **MaterialsAndServices**, click **Rename Table**, and set the table name to *MaterialsAndServicesCat* (fig. 13.19).

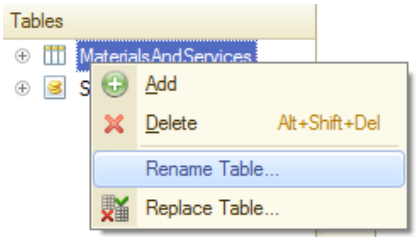


Fig. 13.19. Renaming a table in a query

3. Drag the following fields from these two tables to the field list:
   - **MaterialsAndServicesCat.Ref**
   - **SalesTurnovers.RevenueTurnover** (fig. 13.20)

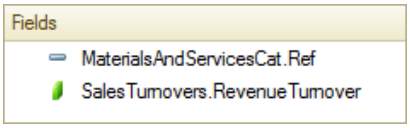


Fig. 13.20. Selected fields

4. Click the **Links** tab.

   Since the query includes multiple tables, you have to specify how they are linked to each other.

   By default the platform creates a link using the MaterialsAndServices field. This means that the value of the MaterialOrService dimension of the Sales register is equal to a reference to an item of the MaterialsAndServices catalog.

5. Clear the **All** check box for the **SalesTurnovers** table and select the **All** check box for the **MaterialsAndServicesCat** table.

   This is how you specify Left join as the link type. In other words, the query result will include all the records of the MaterialsAndServices catalog and those records of the Sales register that meet the link condition based on the MaterialOrService field.

   So the query result will contain all of the services, and sales revenues will be available for some services. No revenue values are displayed for services that were not rendered within the selected period.

The following example visually describes how the two tables shown above are linked to each other (fig. 13.21).

Ref
Materials
Services
Line transformer, Samsung
Line transformer, LG
Transistor 2N2369, Philips
Tubing, rubber
Power supply cable
Diagnostics
Domestic TV set repair
Imported TV set repair
Water hookup
Electrical hookup
TV sets
Washing machines
Radio parts
Miscellaneous

MaterialOrService	Revenue (Turnover)
Line transformer, Samsung	900.00
Line transformer, LG	400.00
Transistor 2N2369, Philips	14.00
Tubing, rubber	450.00
Power supply cable	30.00
Domestic TV set repair	600.00
Imported TV set repair	800.00
Water hookup	800.00
Electrical hookup	800.00

Ref	Revenue (Turnover)
Materials	
Services	
Line transformer, Samsung	900.00
Line transformer, LG	400.00
Transistor 2N2369, Philips	14.00
Tubing, rubber	450.00
Power supply cable	30.00
Diagnostics	
Domestic TV set repair	600.00
Imported TV set repair	800.00
Water hookup	800.00
Electrical hookup	800.00
TV sets	
Washing machines	
Radio parts	
Miscellaneous	

Fig. 13.21. Links between the table records in the query

Once you complete the procedure described in this section, the Links tab should look as shown in fig. 13.22.

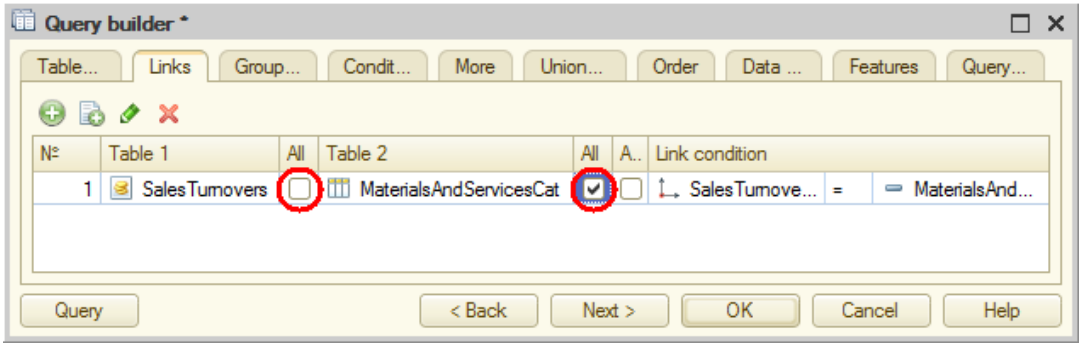


Fig. 13.22. Links between the tables

Record filter condition

Let us set a filter that excludes the folders of the MaterialsAndServices catalog from the report.

1. Click the **Conditions** tab.
2. Expand the **MaterialsAndServicesCat** table and drag its **IsFolder** field to the list of conditions.
3. Select the **Arbitrary** check box for this condition and type the following text in the **Condition** column (listing 13.5).

Listing 13.5. Query condition

```
MaterialsAndServicesCat.IsFolder = FALSE
```

This is how you specify that only those records of the MaterialsAndServices catalog that are not folders are selected.

This condition can be visually described using the following example. On the left you see the source table of the MaterialsAndServices catalog while on the right you see the records that are selected from this table (fig. 13.23).

Ref	Revenue (Turnover)
Materials	
Services	
Line transformer, Samsung	900.00
Line transformer, LG	400.00
Transistor 2N2369, Philips	14.00
Tubing, rubber	450.00
Power supply cable	30.00
Diagnostics	
Domestic TV set repair	600.00
Imported TV set repair	800.00
Water hookup	800.00
Electrical hookup	800.00
TV sets	
Washing machines	
Radio parts	
Miscellaneous	

Ref	Revenue (Turnover)
Line transformer, Samsung	900.00
Line transformer, LG	400.00
Transistor 2N2369, Philips	14.00
Tubing, rubber	450.00
Power supply cable	30.00
Diagnostics	
Domestic TV set repair	600.00
Imported TV set repair	800.00
Water hookup	800.00
Electrical hookup	800.00

Fig. 13.23. Filtering material and service records in a query

The second condition should state that the selected items must be services. Such conditions are referred to as simple conditions. Let us create it.

4. Drag the **MaterialServiceType** field to the list of conditions.

The platform automatically creates a condition stating that the material or service type must match the value of the MaterialServiceType parameter.

Later, before the query execution, you will pass the Service enumeration value to the MaterialServiceType parameter.

This condition can be visually described using the following example. On the left you see the records of the MaterialsAndServices catalog filtered using the first condition. On the right you see the service records only (fig. 13.24).

Ref	Revenue (Turnover)
Line transformer, Samsung	900.00
Line transformer, LG	400.00
Transistor 2N2369, Philips	14.00
Tubing, rubber	450.00
Power supply cable	30.00
Diagnostics	
Domestic TV set repair	600.00
Imported TV set repair	800.00
Water hookup	800.00
Electrical hookup	800.00

×

Ref	Revenue (Turnover)
Diagnostics	
Domestic TV set repair	600.00
Imported TV set repair	800.00
Water hookup	800.00
Electrical hookup	800.00

Fig. 13.24. Filtering material and service records in a query

Once you complete the procedure described in this section, the **Conditions** tab should look as shown in fig. 13.25.

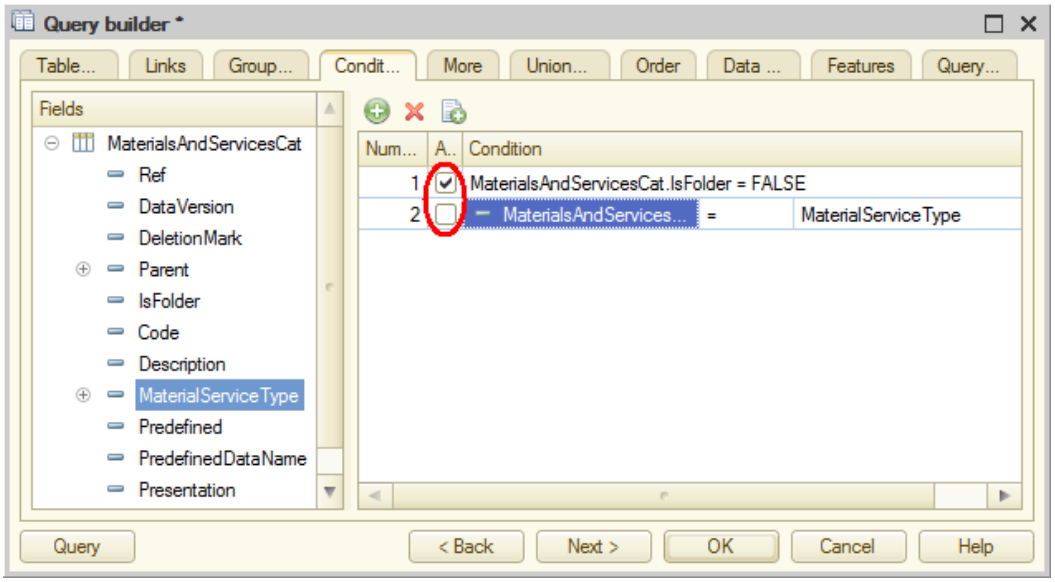


Fig. 13.25. Creating query conditions

Note. You can specify filters both in the query itself and in the report settings. The same is true for sorting and grouping. We recommend that you specify filters in queries when the records that do not meet the filter condition are definitely not needed in the report. We also recommend that you use sorting and grouping in the report settings to make the report more flexible.

Field aliases

Let us set an alias for the catalog item presentation (Ref field) and for the register field.

1. Click the **Unions/Aliases** tab.
2. In the **Field name** column, change *Ref* to *Service* and change *RevenueTurnover* to *Revenue* (fig. 13.26).

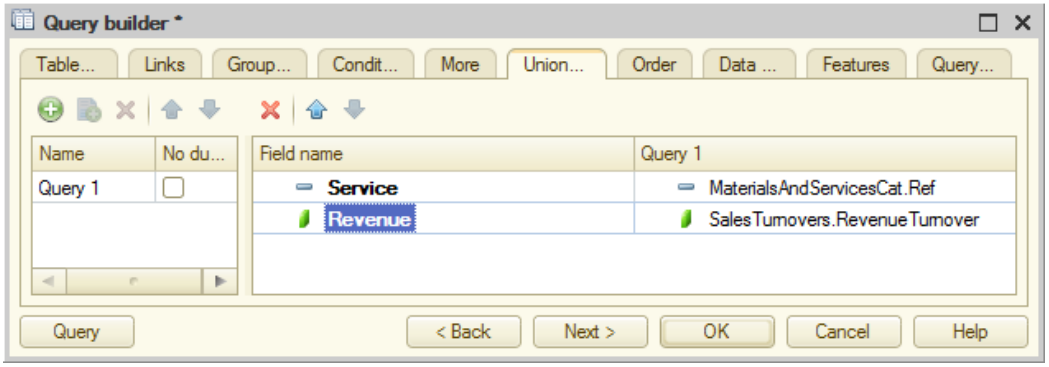


Fig. 13.26. Specifying query field aliases

Records order

Let us specify the order of report records.

1. Click the **Order** tab and specify that the query result is ordered by the value of the **Revenue** field (fig. 13.27).

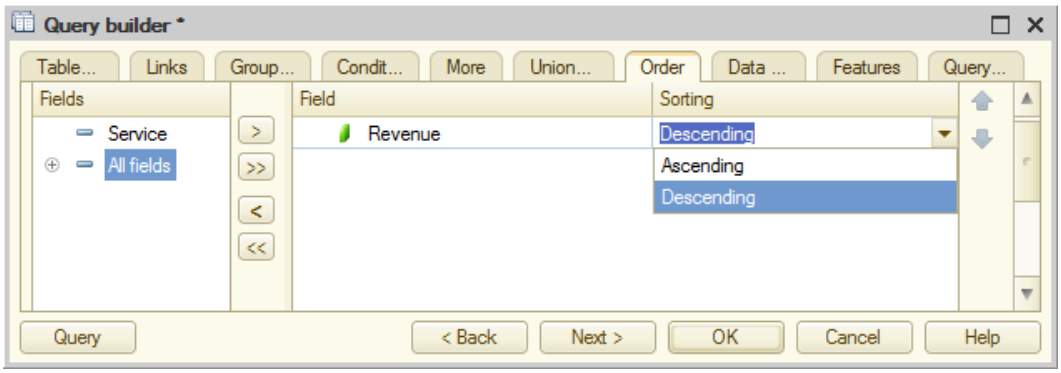


Fig. 13.27. Specifying the order of records in a query

   This completes the query creation.

2. Click **OK**.

   This returns you to the Data composition schema wizard.

Analyzing query text

Let us analyze the generated query text (listing 13.6).

Listing 13.6. Query text

```
SELECT
    MaterialsAndServicesCat.Ref AS Service,
    SalesTurnovers.RevenueTurnover AS Revenue
FROM
    Catalog.MaterialsAndServices AS MaterialsAndServicesCat
        LEFT JOIN AccumulationRegister.Sales.Turnovers AS SalesTurnovers
        ON SalesTurnovers.MaterialOrService = MaterialsAndServicesCat.Ref
WHERE
    MaterialsAndServicesCat.IsFolder = FALSE
```

```
    AND MaterialsAndServicesCat.MaterialServiceType = &MaterialServiceType
ORDER BY
    Revenue DESC
```

The query begins with a usual query description part, this time it contains statements you have not yet encountered.

The query source description (after the FROM keyword) specifies multiple sources (listing 13.7).

Listing 13.7. Multiple query sources description

```
FROM
    Catalog.MaterialsAndServices AS MaterialsAndServicesCat
        LEFT JOIN AccumulationRegister.Sales.Turnovers AS SalesTurnovers
        ON SalesTurnovers.MaterialOrService = MaterialsAndServicesCat.Ref
```

In this case records are selected from two sources (MaterialsAndServicesCat and SalesTurnover), while the LEFT JOIN … ON keywords describe how the records from these two sources are combined.

LEFT JOIN means that the query result includes a combination of records from both sources that satisfy the condition stated after the ON keyword. Additionally, the query result includes records from the first source (which is specified to the left of the JOIN keyword) that are not linked to any records that match the condition in the second source.

Let us review the next lines of the query text. In the query definition section there is one more statement that is new to you: specifying the filter conditions for source table data (listing 13.8).

Listing 13.8. Filter conditions

```
WHERE
    MaterialsAndServicesCat.IsFolder = FALSE
    AND MaterialsAndServicesCat.MaterialServiceType = &MaterialServiceType
```

A filter condition is always preceded by the WHERE keyword. The condition itself follows the keyword. Note that the source table fields that the condition is applied to do not have to be present in the selection list (this is the case here). And the condition includes the MaterialServiceType query parameter preceded by ampersand (&).

Now that we are done reviewing the query text, let us proceed to data composition schema generation.

Resources

The report should display the total revenue values for each service. For this you have to specify the report resource fields.

When we talk about resources in the data composition system, we refer to the fields whose values are calculated based on detailed records included in a grouping. Actually, resources represent group or overall totals of a report.

You can set up the totals on the **Resources** tab.

- Click the **Resources** tab and click >>.

  This selects all of the resources whose totals are available for calculation. In this case you have a single Revenue resource. The platform suggests that you set up the totals calculation for values stored in this field and this is exactly what you need (fig. 13.28).

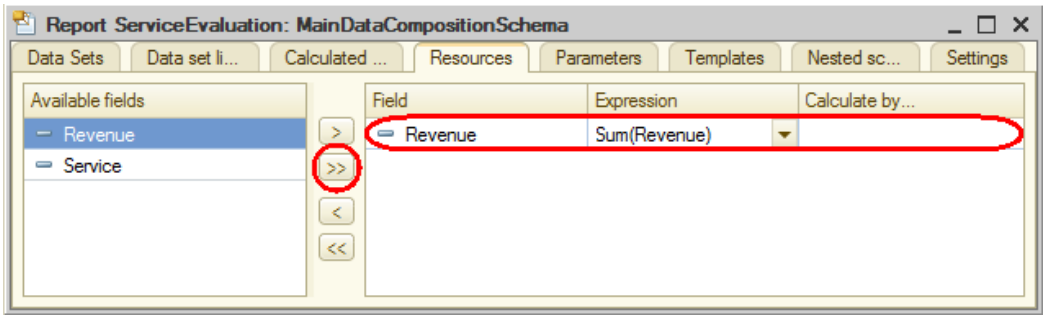


Fig. 13.28. Data composition schema resources

Parameters

A user is normally interested in economic activity data over a specific period. This is why almost any report includes parameters that specify the beginning and the end of the report period.

Report parameters specify the conditions for including records in the report. In the data composition schema you can define report parameters on the **Parameters** tab (fig. 13.29).

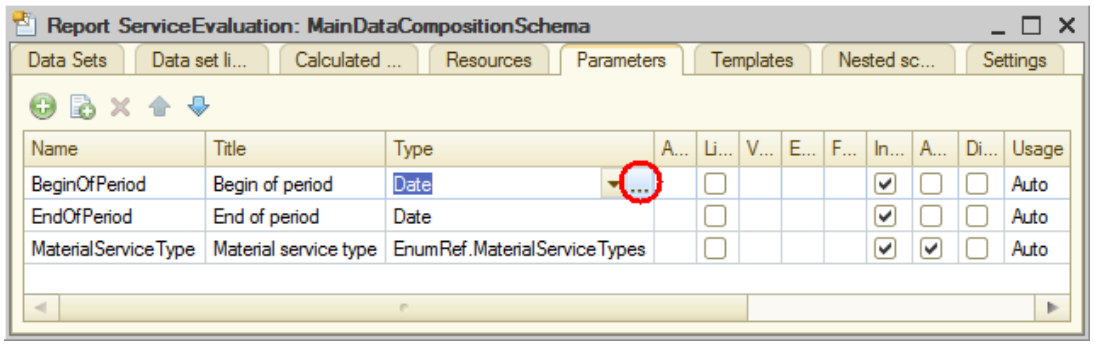


Fig. 13.29. Data composition parameters

The tab contains three parameters: BeginOfPeriod, EndOfPeriod, and MaterialServiceType. You might ask why you have three parameters here while you only specified one (MaterialServiceType) in the query.

The data composition system analyses query text on its own and in addition to the explicitly specified parameters (MaterialServiceType) provides the option to customize parameters of virtual tables that participate in the query.

This is the case for the BeginOfPeriod and EndOfPeriod parameters. They are the two first parameters of the AccumulationRegister.Sales.Turnovers virtual table, which is used in the query, in the left join.

If you select this table in the list of tables in the Query builder and then click the **Virtual table parameters** button, a dialog box with the BeginOfPeriod and EndOfPeriod parameters is displayed (fig. 13.30).

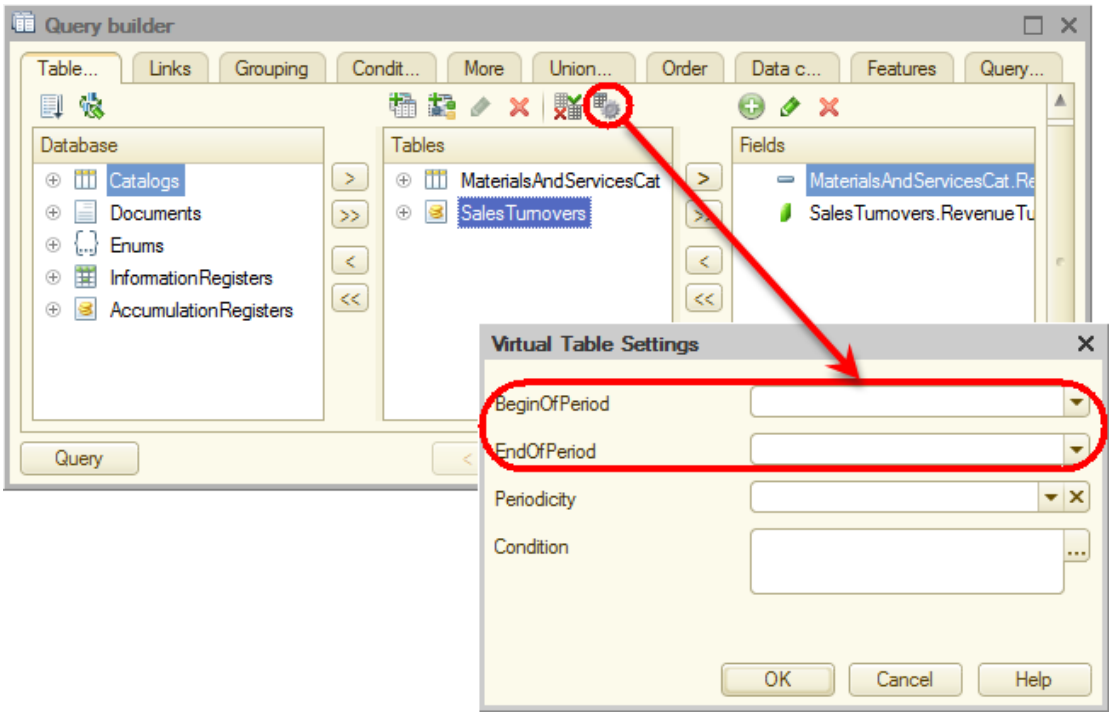


Fig. 13.30. Virtual table parameters

The first parameter stores the beginning of the totals calculation period, while the second parameter stores the end of the period. So the source table contains only turnovers calculated for the specified period.

Remember that when you pass a date to any of these parameters (it is what you will do later in this example), the date includes time with a one-second precision.

Suppose that you know in advance that the user does not need the report results for periods specified in seconds. Then you have to do the following two things.

First, eliminate the need to enter time when a user enters report period dates. To implement this, let us modify the type definition for the BeginOfPeriod parameter.

1. In the Data composition schema wizard, click the **Parameters** tab.
2. For the **BeginOfPeriod** parameter, double-click in the **Type** cell.
3. Click the **Select** ... button and then, in the **Date format** list, select **Date**.
4. Click **OK** (fig. 13.31).

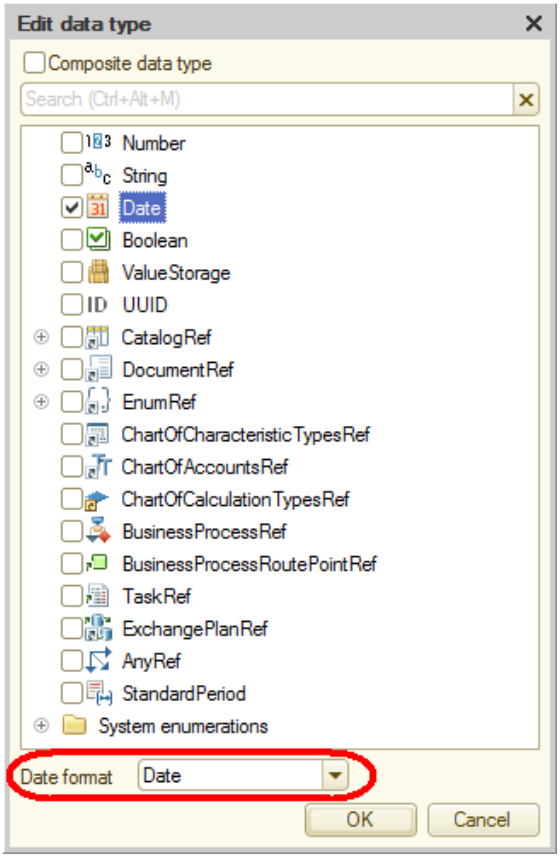


Fig. 13.31. Editing date format

Second, the default time value in a date is 00:00:00. If a user specifies a report period from 6/1/2014 to 6/11/2014, the register totals are calculated from the beginning of the day 6/1/2014 00:00:00 to the beginning of the day 6/11/2014 00:00:00. The data for the 11th day of the month that is not available at the beginning of the day is not included in the report, which might confuse users.

To eliminate this issue, let us add a new EndDate parameter for storing the end date entered by the user. And let us set up automatic calculation of the EndOfPeriod parameter, so that it points to the end of the day specified by the user.

1. For the **EndOfPeriod** parameter, select the **Availability restriction** check box (fig. 13.32).

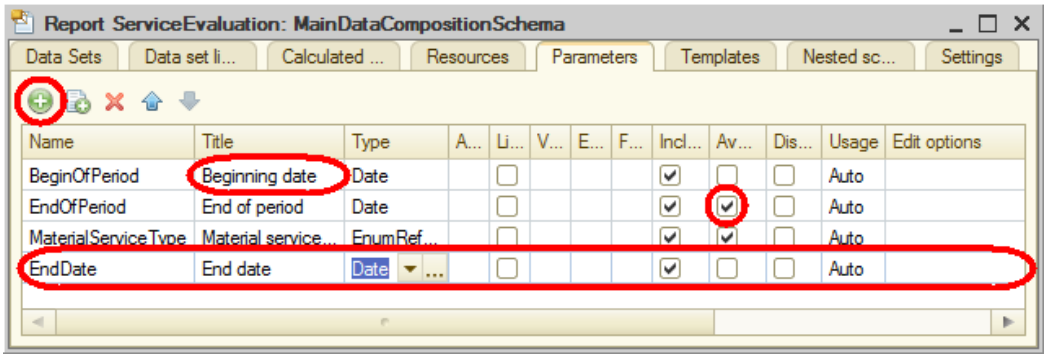


Fig. 13.32. Adding the EndDate parameter

When the check box is cleared, the parameter is available for user customization. And when the check box is selected, the parameter is hidden from the user interface.

2. Click the **Add** button in the command bar and add the *EndDate* parameter.

   The platform automatically generates the *End date* title for this parameter.

3. For the **EndDate** parameter, set the type to **Date**, and then set **Date format** to **Date**, just like you did for the **BeginOfPeriod** parameter.

4. For the **BeginOfPeriod** parameter, set the title to **Beginning date**.

   Note that by default the added parameter is available to users (the **Availability restriction** check box is cleared). This is fine with us.

   Then let us look at the EndOfPeriod parameter. You have selected its **Availability restriction** check box because you will implement the calculation of the parameter value based on the EndDate parameter value entered by the user.

   Let us use the expression language of the data composition system for specifying the EndOfPeriod calculation formula. This language includes the EndOfPeriod() function, which gets a date that corresponds to the end of some period, for example, the end of the specified day.

5. For the **EndOfPeriod** parameter, in the **Expression** cell, type the expression shown in listing 13.9.

Listing 13.9. Expression for calculating the EndOfPeriod parameter value

```
EndOfPeriod(&EndDate,"Day")
```

Note. The expression language available in the data composition system is described in detail in the built-in help in Designer.

To open the built-in help

1. On the **Help** menu, click **Help contents**.
2. In the help contents, open section **1C:Enterprise** / **Data Composition System** / **Data Composition System Expression Language**.

Once you are done with these steps, the composition parameters should look as shown in fig. 13.33.

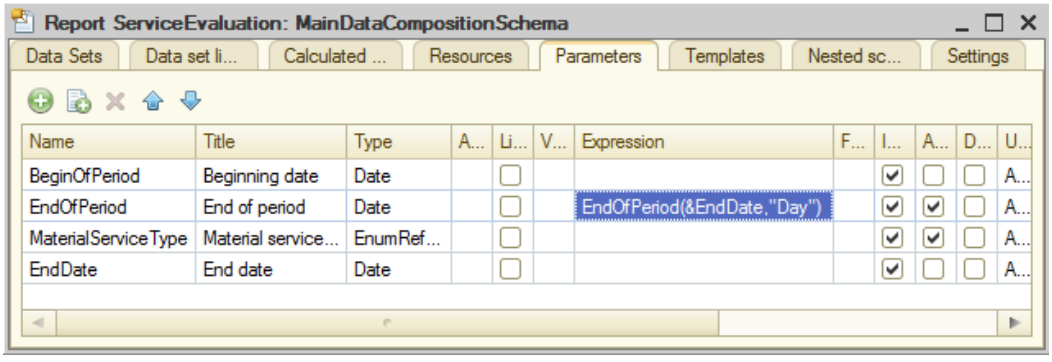


Fig. 13.33. Composition system parameters

Finally, let us set up the MaterialServiceType parameter.

Since the report should display the revenue generated by sales of services only, the value of the MaterialServiceType parameter should not be edited by users. It should be specified directly in the composition schema as Enumeration.MaterialServiceType.Service.

For the **MaterialServiceType** parameter, the **Availability restriction** check box is selected by default, so you only need to specify the required value of the MaterialServiceTypes enumeration in the **Value** cell.

6. For the **MaterialServiceType** parameter, in the **Value** cell, click the **Select** ... button and select **Service** (fig. 13.34).

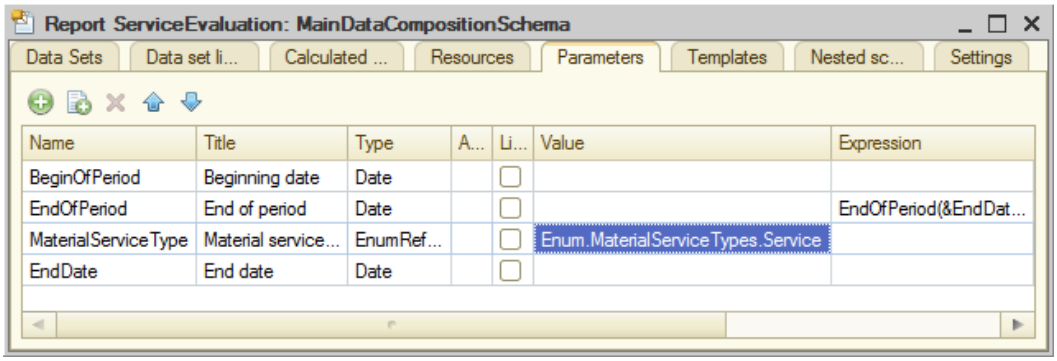


Fig. 13.34. Specifying MaterialServiceType parameter value

Settings

Let us specify the report structure.

1. On the **Settings** tab, add a grouping without specifying the grouping field.
2. On the **Selected fields** tab, select the **Service** and **Revenue** fields (fig. 13.35).

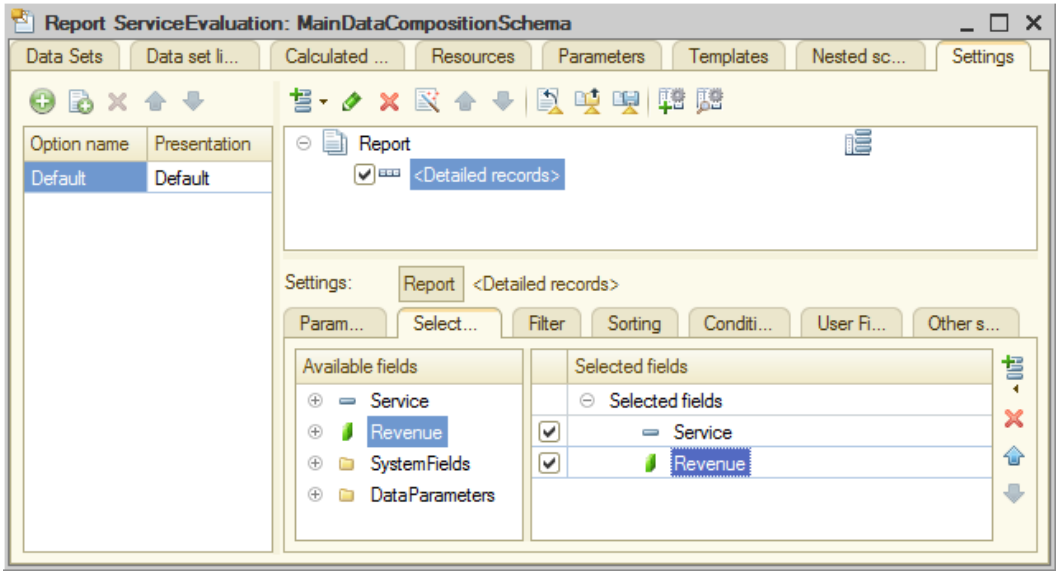


Fig. 13.35. Service evaluation report structure

3. On the **Other settings** tab, set the **Title** parameter to *Service evaluation* (fig. 13.36).

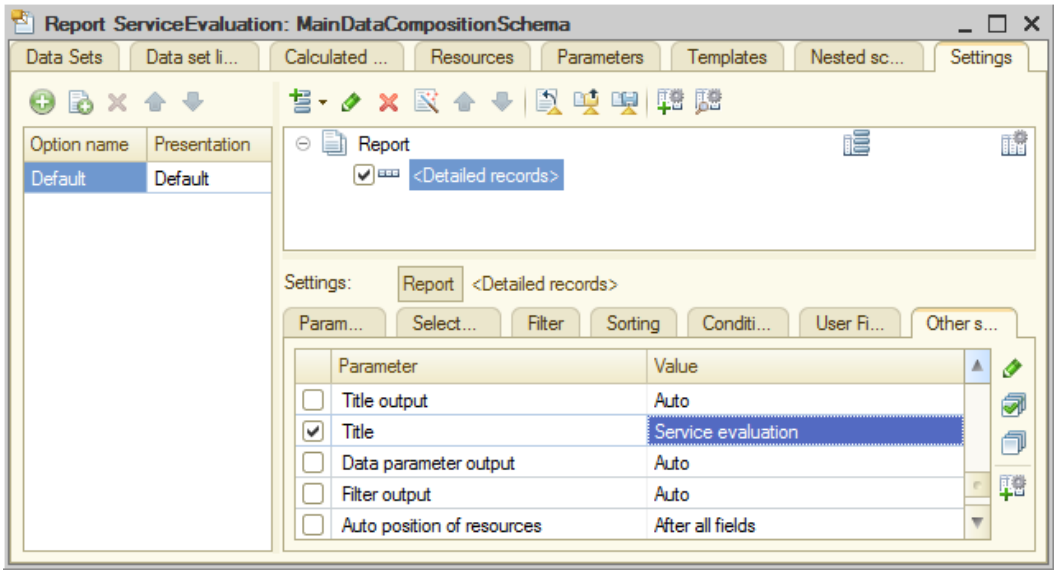


Fig. 13.36. Specifying report title

Note. To modify settings that require a value selection, double-click the **Value** field, click the **Select** ... button, and select a value from the list. Once you do, the usage check box is automatically selected. You can also select or clear the check box manually.

Quick user settings

Finally, let us give users the option to specify the report period prior to report generation. This means including the Beginning date and End date parameters to the list of user settings.

These settings should be available directly in the report form because users almost always specify the report period.

The **Parameters** tab lists the parameters that users can edit (those that have the **Availability restriction** check box cleared).

1. For each parameter, select it, click the **Custom settings item property** button, select the **Include in custom settings** check box, and, in the **Edit mode** list, keep the default **Quick access** value (fig. 13.37).

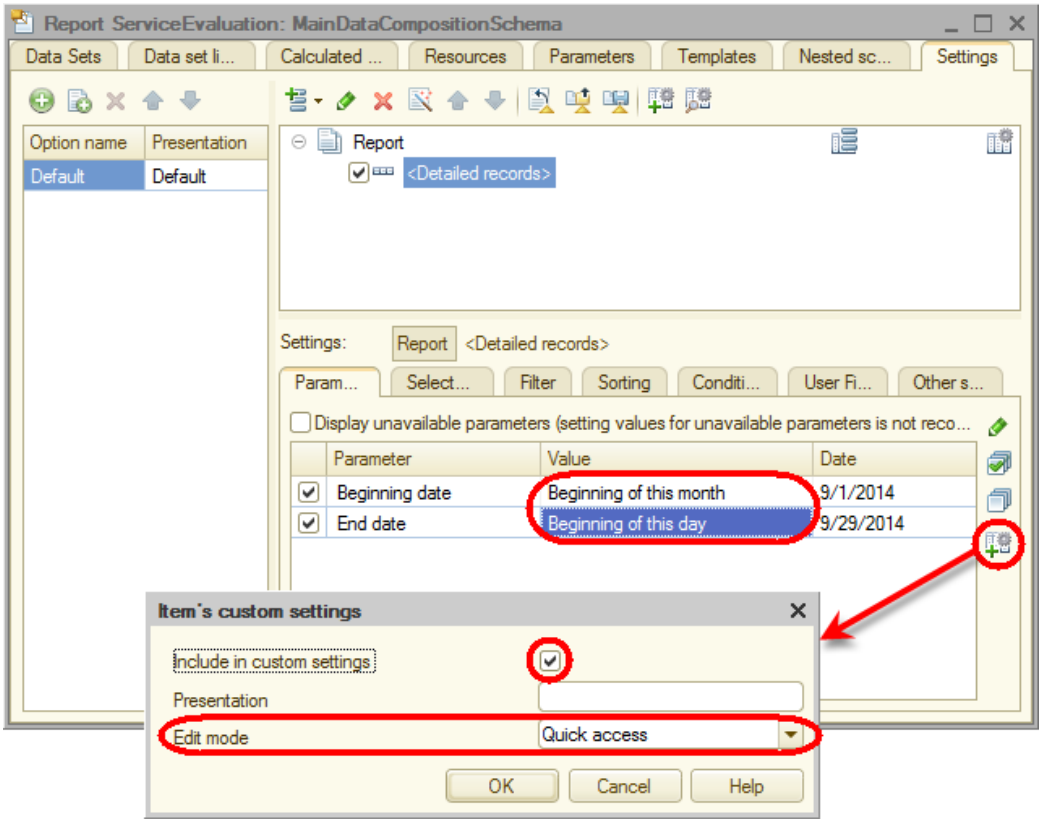


Fig. 13.37. Specifying user settings

Note that the **Include in custom settings** check box means that this setting is available in a new window (2) that is opened when the user clicks **Settings** on the **More** menu (so this setting is available to users but not intended for frequent use, fig. 13.38).

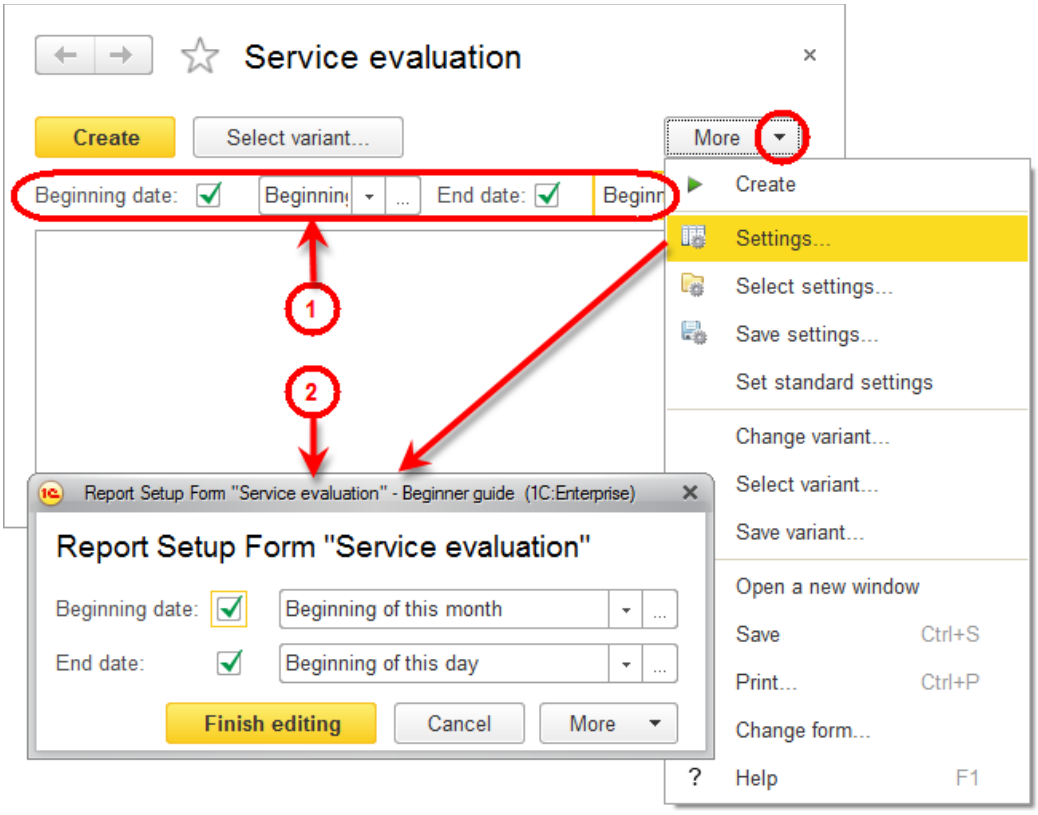


Fig. 13.38. Quick (1) and regular (2) user settings

Setting **Edit mode** to **Quick access** means that this setting is also displayed directly in the report form (1). It is a quick user setting (the setting that users need frequently, almost every time the report is executed), so it should be always available.

Let us specify the default values for the beginning and end dates to improve the report usability.

2. Set **Beginning date** to **Beginning of this month** and set **End date** to **Beginning of this day** (see fig. 13.37).

   This means that when the report is generated, the beginning and end dates of the report period change dynamically so that the report displays data from the beginning of the current month up to the current day and the user will probably never need to modify them manually.

   Finally, let us specify the subsystems where the report is available.

3. Close the data composition schema wizard and then, in the editor of the **ServiceEvaluation** report configuration object, click the **Subsystems** tab.

4. Select the **Accounting** and **Services** check boxes.

   This adds the links that open the report to the **Reports** menus in the command panels of these subsystems (fig. 13.39).

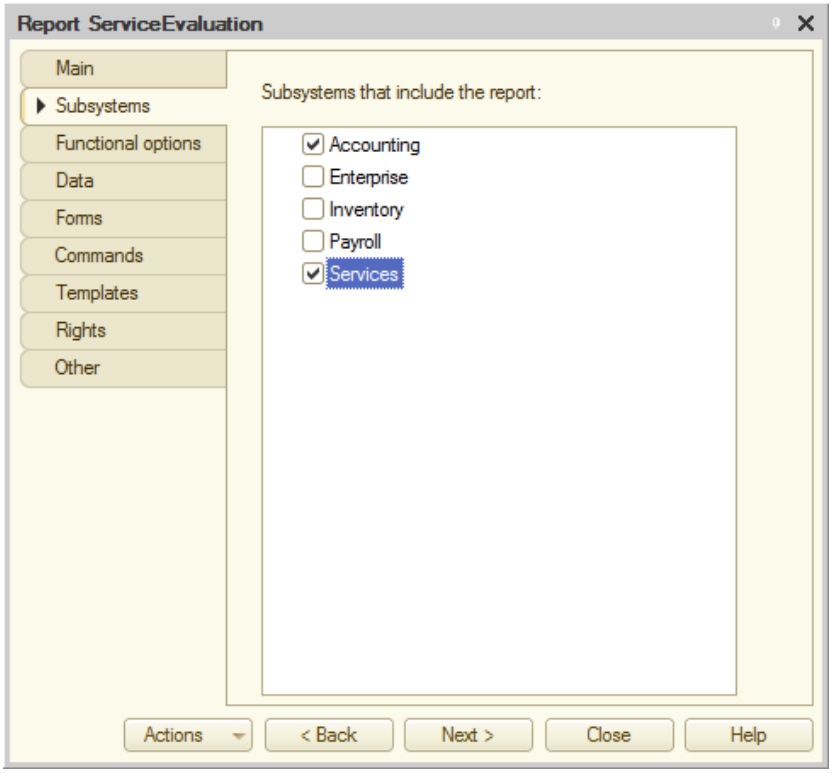


Fig. 13.39. Specifying the list of subsystems where the report is available

In 1C:Enterprise mode

Let us check the report.

1. Start 1C:Enterprise in the debug mode.

   You can see that the **Reports** menus of the Accounting and Services sections now include the command that generates the Service evaluation report (fig. 13.40).

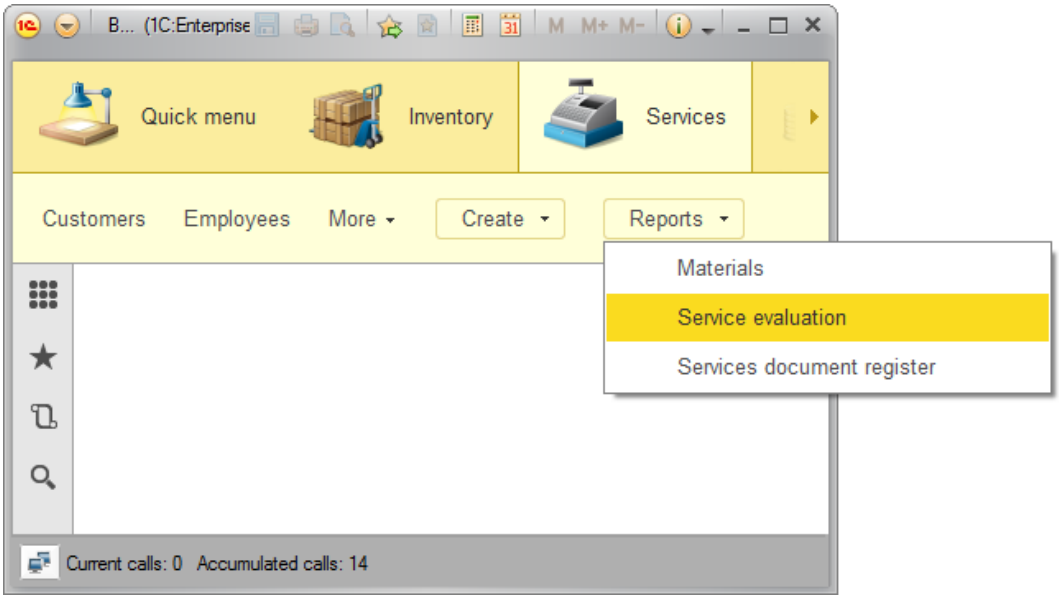


Fig. 13.40.Command that generates the Service evaluation report

2. Click **Service evaluation**.

   This opens an automatically generated report form.

   The report window displays the parameters that define the report period. The default period is from the beginning of the month till the current date. You can change this period by clicking the calendar button.

3. Click **Create** (fig. 13.41).

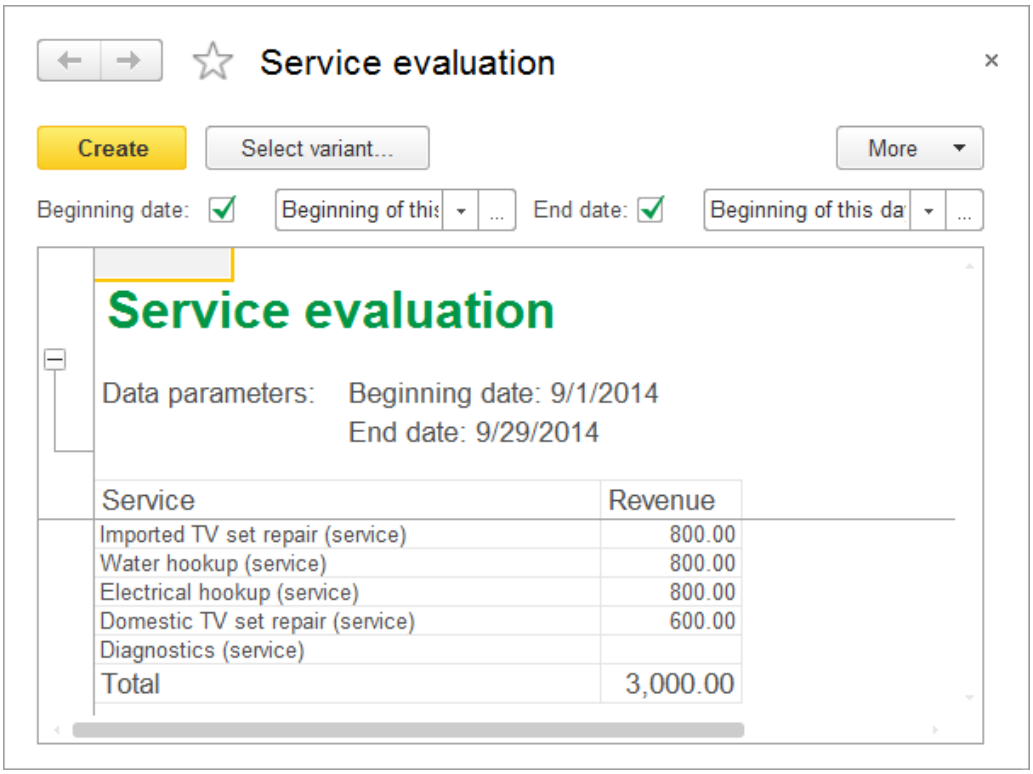


Fig. 13.41. Service evaluation report

As you can see, the revenue for the services listed in all three Services documents is included in the report because the document dates fit the specified period.

Note that the report header displays the title that you specified and the parameters that define the report period.

Also note that the header position remains fixed when you scroll down the report. The platform automatically freezes the top rows of the report spreadsheet document. You can also specify the freeze settings for report rows and columns manually using the Fix left and Fix top output parameters.

The service names (the Ref field of the MaterialsAndServices catalog) include both the item name and the item type because you specified this format for their presentations.

4. Change the end date to 9/20/2014.

   You can see that only data of the Services document #1, which is created on September 17th, is now included in the report (fig. 13.42).

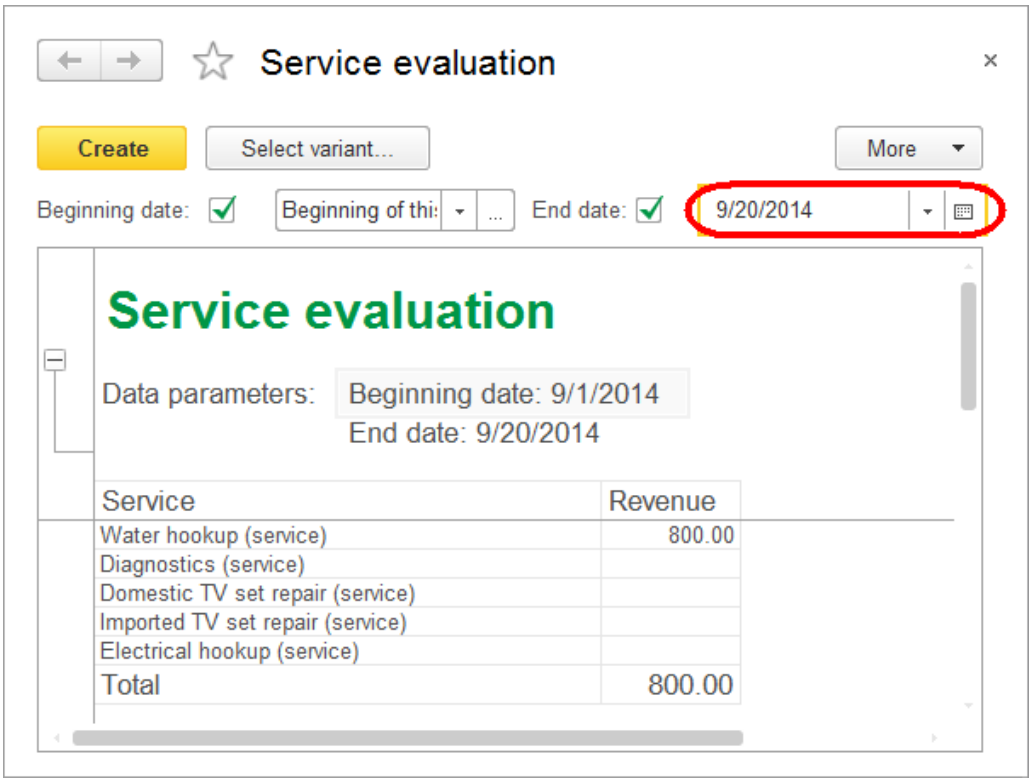


Fig. 13.42. Report execution result

So the data as of the last date of the report period is included in the report, exactly as you specified earlier using the EndOfPeriod() function.

Note that since the report query links the materials and services table to the sales register table using left join, the services that do not have any sales data are still displayed in the report.

Settings in Designer and settings in 1C:Enterprise mode

Using this report as an example, we will show how to create and use the following report settings: Conditional appearance and Filter.

You will create these settings in Designer and then switch to 1C:Enterprise mode to check the result. But actually all the settings that you specify in Designer are available in 1C:Enterprise mode. When a user clicks **Change variant** on the **More** menu, they see a report settings window that is very similar to the **Settings** tab of the data composition schema.

But these settings are not the same as Designer settings. The settings specified in Designer are the default settings, they are stored in the data composition schema itself, in other words, they are a part of the configuration. Any configuration user will see the report layout exactly as you have specified it in Designer.

The same settings are available in 1C:Enterprise mode but they are not a part of the configuration. They are only available to a single user of a specific infobase, the one who specifies them.

Note. You can implement a settings exchange between the configuration users. But this is not a simple task and we will not discuss it in this book, just know that this option is available.

The report options feature in 1C:Enterprise mode is not intended for regular users (quick settings and user settings are sufficient for them). Instead, this feature is intended for configuration developers, administrators, or advanced users.

It is obvious that the settings applied in 1C:Enterprise mode override the default settings. If a user changes the report, making it totally different from the original version, they can always return to the default settings using the **Set standard settings** command on the **More** menu.

Since right now we want to customize the report for all the users who will have access to it, let us do it in Designer.

But if one day the chief accountant asks you to make a "better-looking" report, you will be able to customize the report at their workstation, without modifying the configuration.

Conditional appearance

When it comes to the Service evaluation report, it is convenient to highlight some report records. These can be the records that contain services with the lowest or the highest revenues, or based on some other condition.

In Designer mode

Let us implement conditional appearance.

1. Return to Designer, open the data composition schema, and click the **Settings** tab.
2. In the bottom of the window, click the **Conditional appearance** tab.
3. Click the **Add** button on the right (fig. 13.43).

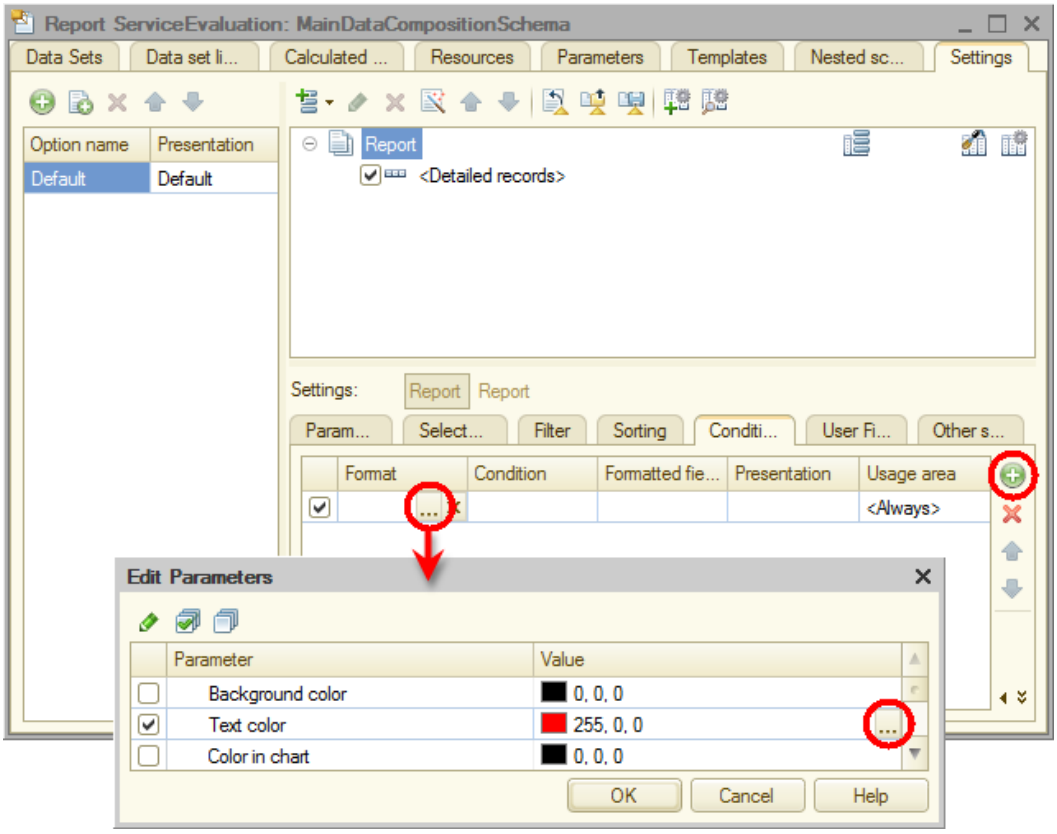


Fig. 13.43. Specifying conditional appearance settings

First let us specify the appearance, that is, how the fields are highlighted.

4. In the **Format** field, click the **Select** ... button, select red text color (see fig. 13.43), and click **OK**.

   Then let us specify the condition when the appearance is applied (when the text turns red).

5. In the **Condition** field, click the **Select** ... button.
6. In the **Filter** window, click the **Add** button and then click **New item** (fig. 13.44).

   Note that each filter item specifies a single condition and you can add multiple conditions.

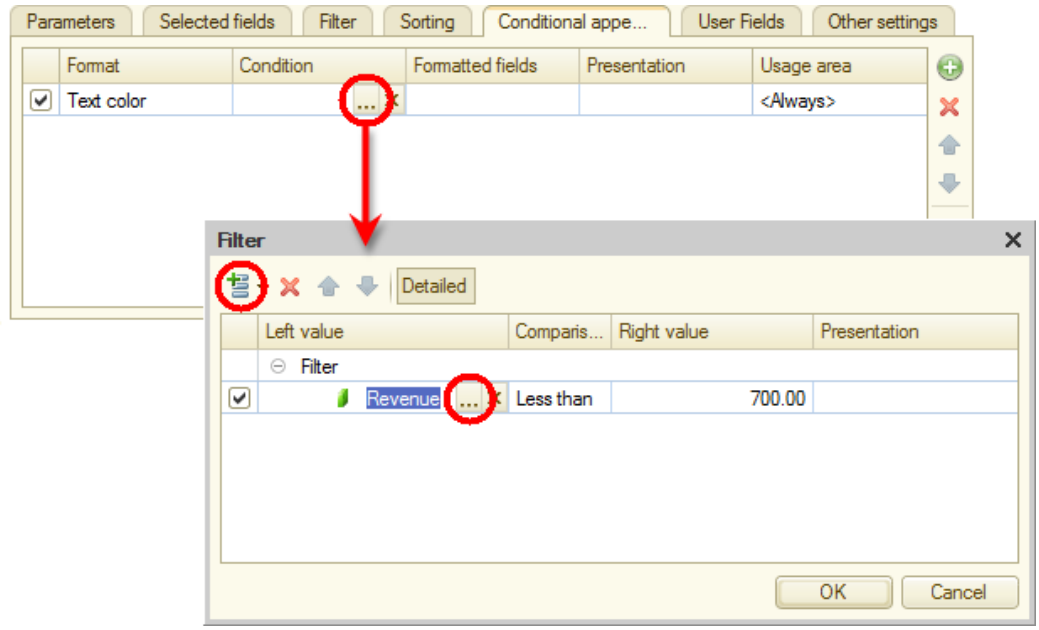


Fig. 13.44. Specifying conditional appearance settings

7. Specify the following filter values and then click **OK**:
   - Left value = Revenue
   - Comparison type = Less than
   - Right value = 700

   This means that when the Revenue field has a value under 700, red text color is applied to something. Now let us define that "something" by specifying the list of formatted fields.

   If you want to highlight the entire report row, you can leave this list blank.

   Let us select the report fields to be highlighted.

8. In the **Formatted fields** cell, click the **Select** button and then, in the window that is opened, click the **Add** button to select the **Service** and **Revenue** fields, and then click **OK** (fig. 13.45).

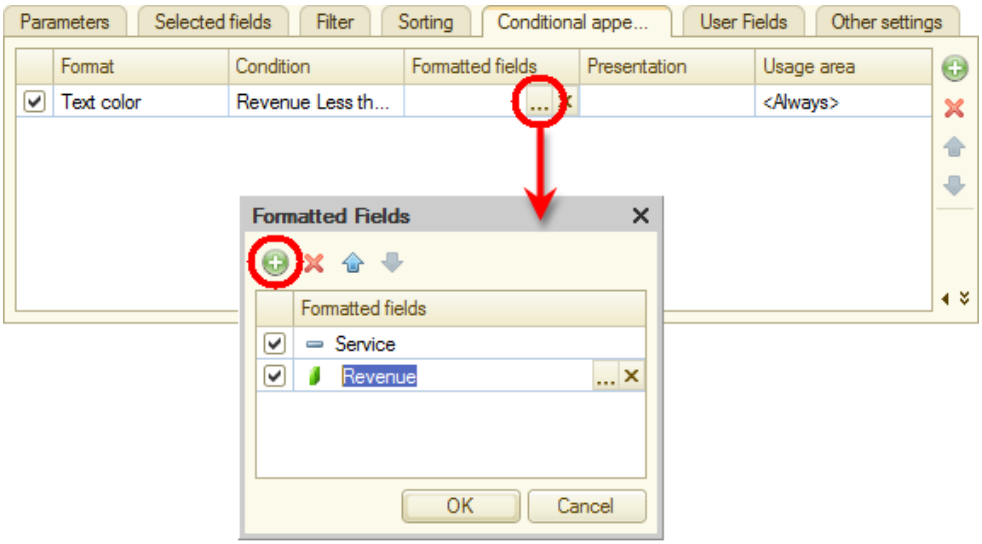


Fig. 13.45. Specifying conditional appearance settings

Actually, in this case you can skip this step because Service and Revenue are the only report fields.

Finally let is specify a presentation for the conditional appearance.

9. In the **Presentation** cell, type *Unpopular service* (fig. 13.46).

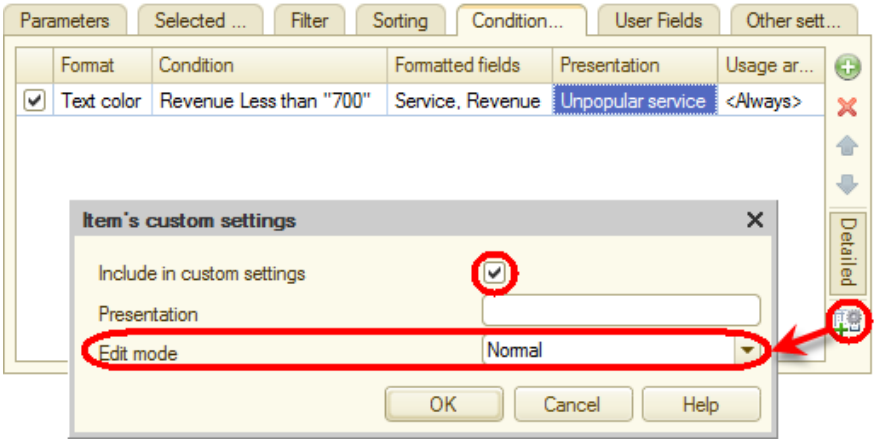


Fig. 13.46. Specifying conditional appearance settings

"Unpopular service" is what users will see in their settings. So instead of the messy line "Revenue under 700…" users will see the human-readable text that you specify in the **Presentation** field.

You have defined a conditional appearance for the report where all the services with revenues under 700 dollars are considered unpopular and highlighted by red text color.

Let us add this condition to the user settings.

10. Click the **Custom settings item property** button, select the **Include in custom settings** check box, and then, in the **Edit mode** list, select **Normal** (see fig. 13.46).

    So you included the conditional appearance in the regular user settings. Unlike quick settings, regular user settings are not available in the report form and can only be accessed in a new window that is opened by clicking **Settings** on the **More** menu. This is because such settings are used much less frequently than, for example, report period settings.

In 1C:Enterprise mode

Let us check the report.

1. Start 1C:Enterprise in the debug mode and open the report.
2. Set the **End date** to **Beginning of this day** and click **Create** (fig. 13.47).

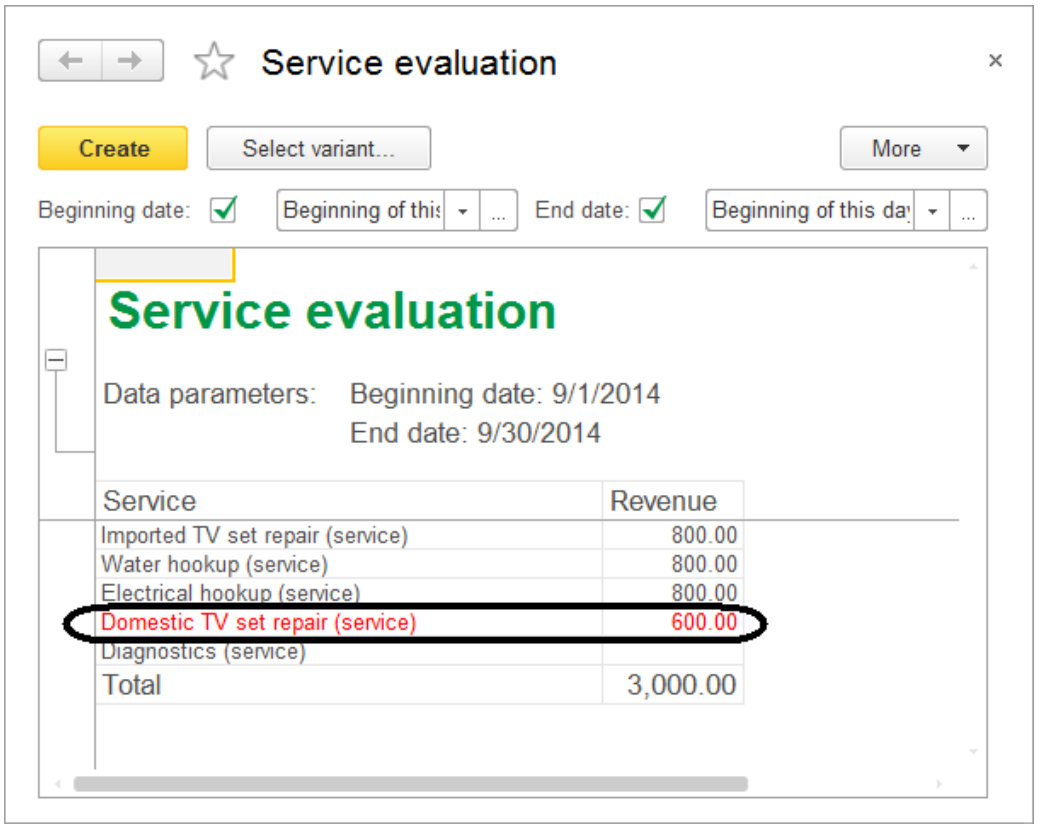


Fig. 13.47. Report execution result

You can see that the services with revenues under 700 dollars are displayed in red.

3. On the **More** menu, click **Settings**.

   This opens the window with user settings, which include the report period parameters and the conditional appearance setting for unpopular services.

4. Clear the **Unpopular service** check box, click **Finish editing** (fig. 13.48), and run the report.

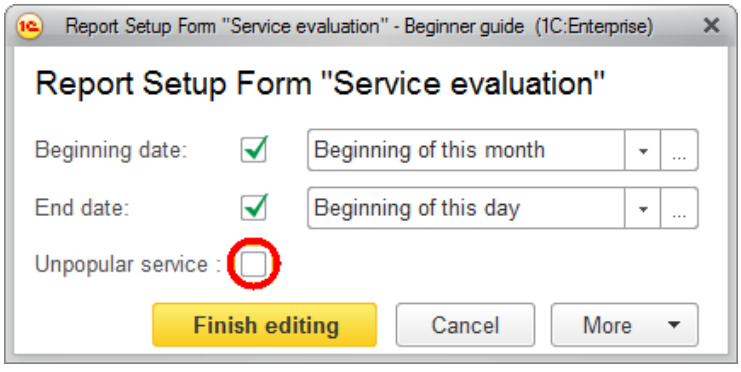


Fig. 13.48. User settings window

You can see that the color highlighting disappeared. The Unpopular service setting is not available in the report form because you selected the Normal edit mode for it instead of Quick access.

But this conditional appearance setting is hardcoded, so that users can only choose between using and not using it. Normally it suits the needs of regular users.

You can offer greater flexibility for advanced users. For example, you can let them specify their own report settings, such as filters, sorting, or conditional appearance.

Let us try this.

User settings

In Designer mode

Let us add the user settings to the report.

1. Return to Designer.
2. In the report data composition schema, click the **Settings** tab.

   This tab lists all of the report settings defined by the developer. Some of these settings can be available to users, so that they can create custom filters, conditional report appearance, and so on.
3. In the settings window command bar, click the **Custom settings item properties** button (fig. 13.49).

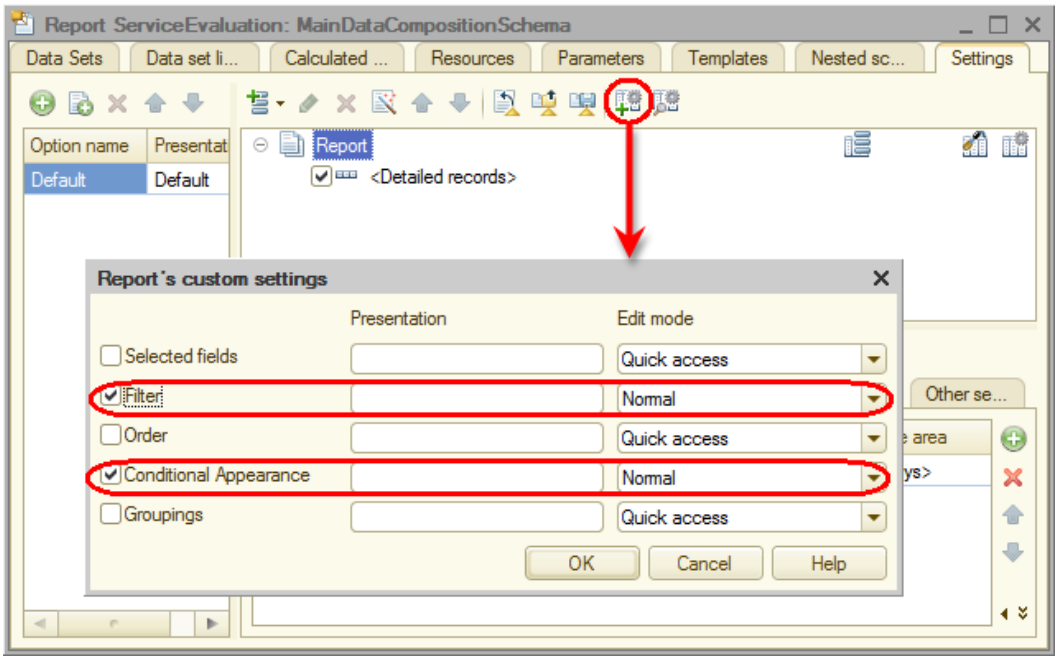


Fig. 13.49. List of user settings

In the window that is opened you can edit the assortment of user settings for the report.

4. Select the **Filter** and **Conditional Appearance** check boxes.
5. For the filter and conditional appearance, set **Edit mode** to **Normal**.

   So you have included the filter and conditional appearance settings in the user settings. The users can specify these settings in the window that is opened by clicking **Settings** on the **More** menu.

Filter

In Designer mode

Let us create a filter setting in the report.

1. In the bottom of the **Settings** tab, click the **Filter** tab.

   The list on the left contains available report fields

2. Expand the **Service** field and double-click the **Parent** field.

   This moves the Parent field to the list of filter conditions on the right (fig. 13.50).

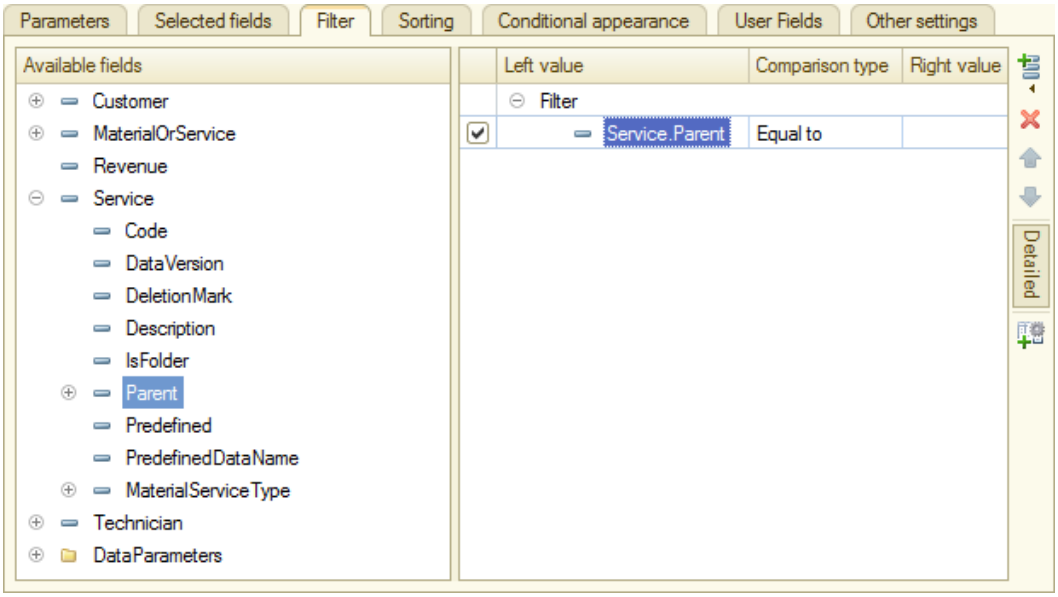


Fig. 13.50. Specifying filter settings

So you implemented the option to filter the report by service category, which is available to users in 1C:Enterprise mode.

In 1C:Enterprise mode

Let us check the report.

1. Start 1C:Enterprise in the debug mode and open the report.
2. On the **More** menu, click **Settings**.

   The user report settings window now includes the **Filter** and **Conditional Appearance** settings that you have just implemented (fig. 13.51).

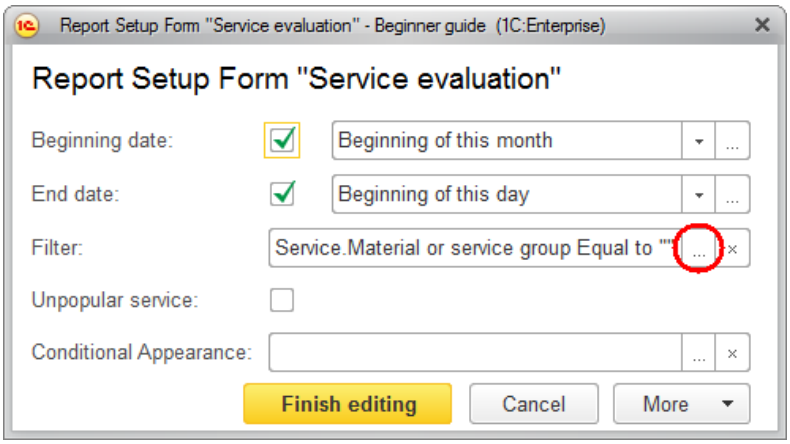


Fig. 13.51. User settings window

Actually, there are two conditional appearance settings in this window.

The first one is the Unpopular service setting that you implemented in Designer. And now that you have added a setting for general conditional appearance, you provided the user with the option to create any number of their own conditions for conditional appearance, just like you have done it in Designer. We will skip it now but you can try it yourself.

Let us specify the report filter to display only services related to washing machine installation.

3. In the user settings window, in the **Filter** field, click the **Select** button (see fig. 13.51).

   In the **Edit filter** window that is opened you can see the filter condition that you created earlier in Designer.

4. In the **Value** cell, click the **Select from list** button, start typing *Washing machines*, and select the **Washing machines** item from the drop-down list (fig. 13.52).

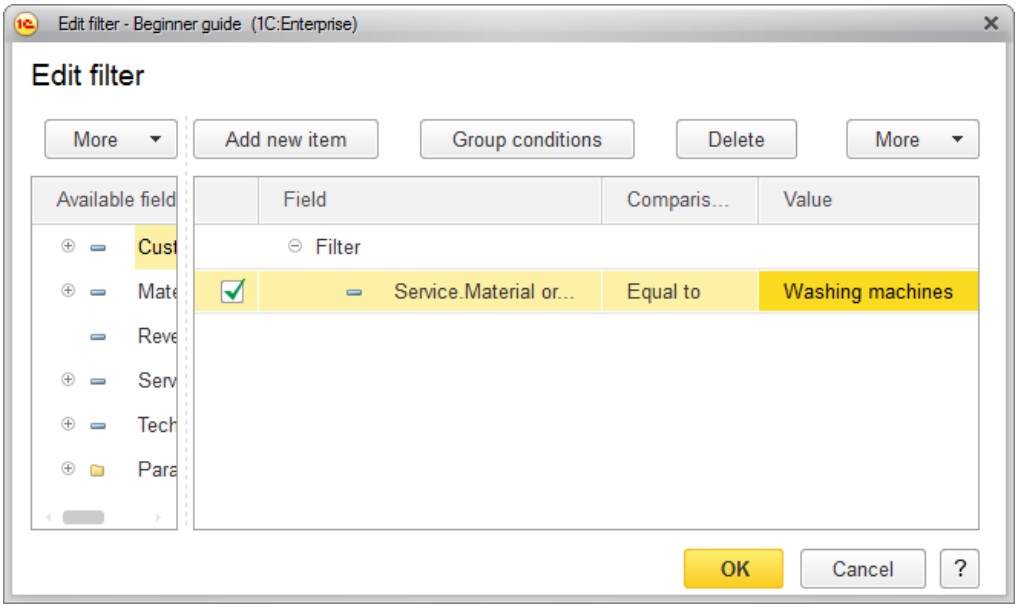


Fig. 13.52. Specifying filter settings

5. Click **OK**.

   You have defined a filter to display only services that have the Washing machines group of the Materials and services catalog as their parent.

6. In the user settings window, click **Finish editing** and then click **Create** to run the report (fig. 13.53).

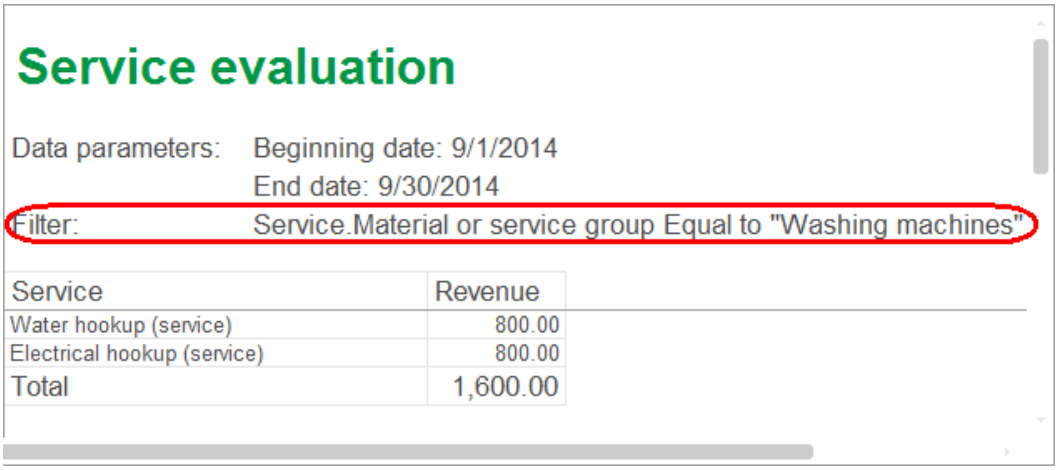


Fig. 13.53. Report execution result

You can see that the report lists only services related to washing machine installation and the report header includes the filter description.

When you close the report, the user settings are saved and they become the default report settings for the current user.

In the user settings window you can clear the filter by clicking the **Clear** button, or you can create another condition by clicking the **Select** button in the **Filter** field (fig. 13.54).

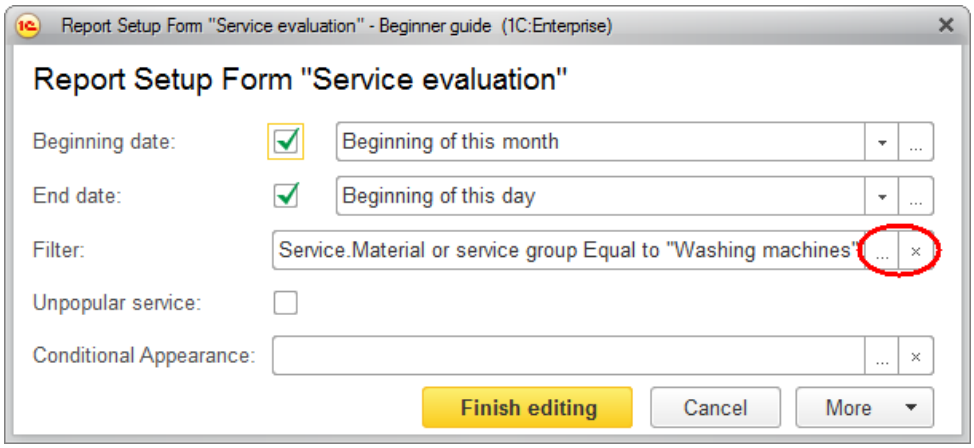


Fig. 13.54. User settings window

So advanced users can customize a lot of settings. And if the users do not need this or do not have required skills, it is better to specify the settings explicitly, so that they can only enable or disable them.

In fact, often the users need just the report period and maybe a few other important settings, and such settings should be obviously located directly in the report form.

If the user priorities of settings usage are different from those implemented by the developer in the data composition schema, users can customize the settings by clicking **Change settings assortment** on the **More** menu (fig. 13.55).

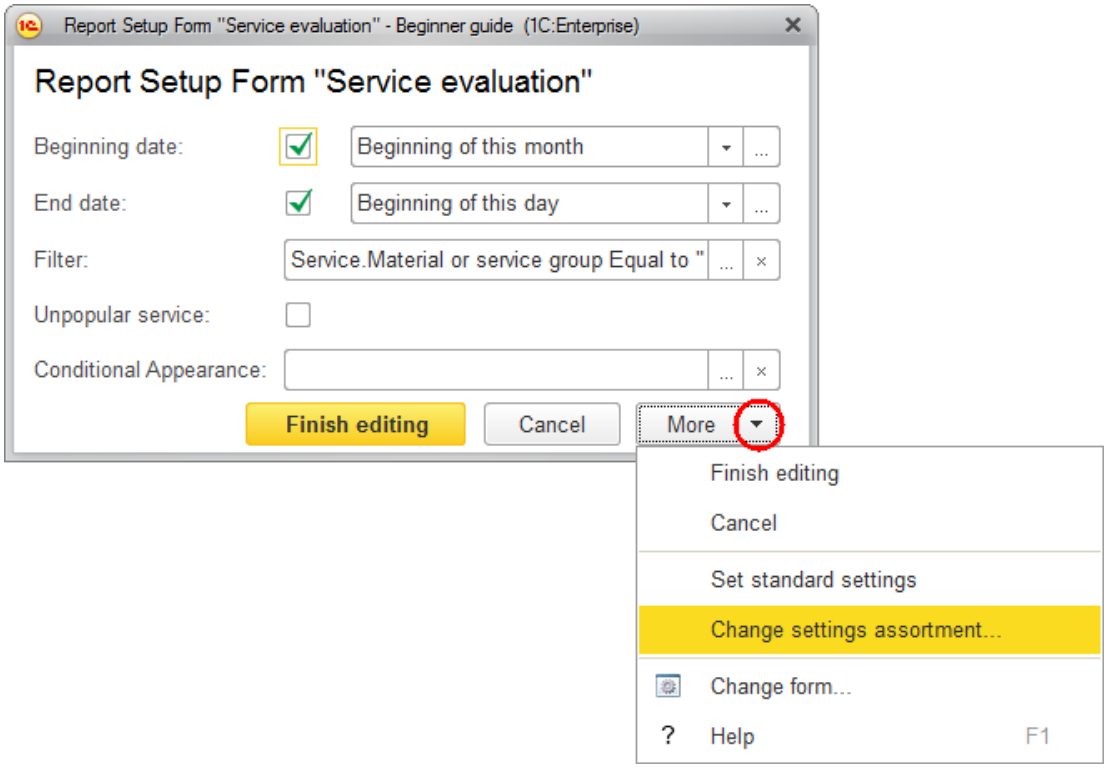


Fig. 13.55. User settings of a report in 1C:Enterprise mode

In the **Settings content** window that is opened, a user can specify which settings are edited in the report form as quick settings (the list on the right) and which settings are available by clicking **Settings** on the **More** menu (the list on the left). They can use the **Add** and **Remove** buttons, as well as double-click the list items to move them between the lists.

1. Move the filter setting to the quick settings list (fig. 13.56).

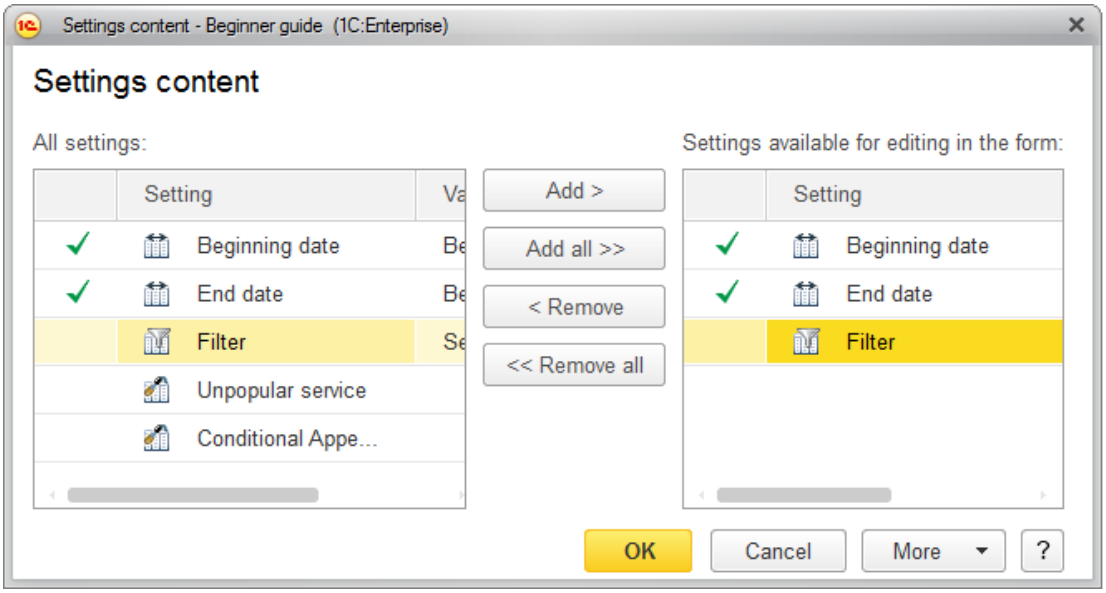


Fig. 13.56. Customizing the settings in 1C:Enterprise mode

The filter setting is now available directly in the report form.

2. Remove the filter setting from the report window, return to Designer, and clear the usage check box for the filter condition.

   You will need this in the next examples.

Breaking down by days in the selected period

Let us add the next report: Revenue by technician.

This report displays the amount of revenue received by Jack of All Trades, Ltd. through the performance of each of its technicians, with a daily breakdown within a selected period and detailed data on the customers served during each of those days (fig. 13.57).

Revenue by technician

Data parameters: Begin of period: 9/17/2014
End of period: 9/24/2014

Technician	Period	Customer	Revenue
Total			4,794.00
Alexander D. Johnson			1,700.00
	9/17/2014 12:00:00 AM		
	9/18/2014 12:00:00 AM		
	9/19/2014 12:00:00 AM		
	9/20/2014 12:00:00 AM		
	9/21/2014 12:00:00 AM		
	9/22/2014 12:00:00 AM		
	9/23/2014 12:00:00 AM		
	9/24/2014 12:00:00 AM		1,700.00
		Shelley Jackson	1,700.00
John A. Walkman			950.00
	9/17/2014 12:00:00 AM		950.00
		Michael Johnson	950.00
	9/18/2014 12:00:00 AM		
	9/19/2014 12:00:00 AM		
	9/20/2014 12:00:00 AM		
	9/21/2014 12:00:00 AM		
	9/22/2014 12:00:00 AM		
	9/23/2014 12:00:00 AM		
	9/24/2014 12:00:00 AM		
Michelle V. Turner			2,144.00
	9/17/2014 12:00:00 AM		
	9/18/2014 12:00:00 AM		
	9/19/2014 12:00:00 AM		
	9/20/2014 12:00:00 AM		
	9/21/2014 12:00:00 AM		
	9/22/2014 12:00:00 AM		
	9/23/2014 12:00:00 AM		
	9/24/2014 12:00:00 AM		2,144.00
		Peter Smith	2,144.00

Fig. 13.57. Resulting report

Using this report as an example, we will show how to build multilevel groupings in a query and how to iterate through all the days of the selected period.

We will also show how to set up individual report structure items, export their data into a chart, and create multiple report options in Designer.

In Designer mode

Let us begin the report creation.

1. Create a report configuration object named *RevenueByTechnician*.
2. Run the Data composition schema wizard.
3. Add **Data set – query** and run the Query builder.
4. Select the **Sales.Turnovers** accumulation register virtual table as the data source.

Query for a data set

Virtual table parameters

Let us specify the Periodicity virtual table parameter.

1. In the **Tables** field, click the **SalesTurnovers** table, and then click the **Virtual table parameters** button (fig. 13.58).

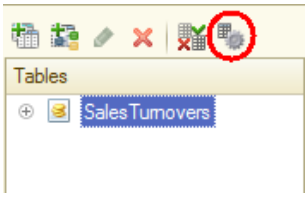


Fig. 13.58. Specifying virtual table parameters

2. In the **Virtual table parameters** window, set **Periodicity** to **Day** (fig. 13.59).

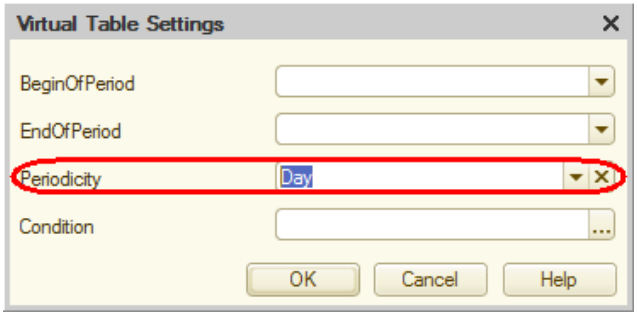


Fig. 13.59. Specifying virtual table parameters

3. Click **OK**.

4. Select the following table fields (fig. 13.60):
   - **SalesTurnovers.Technician**
   - **SalesTurnovers.Period**
   - **SalesTurnovers.Customer**
   - **SalesTurnovers.RevenueTurnover**

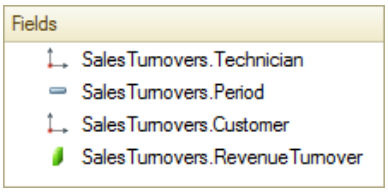


Fig. 13.60. Selected fields

5. Click the **Unions/Aliases** tab and type *Revenue* as the alias for the **SalesTurnovers.RevenueTurnover** field (fig. 13.61).

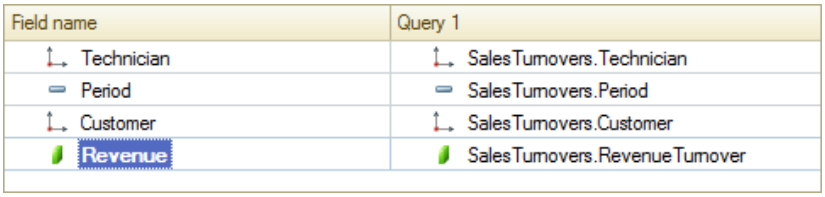


Fig. 13.61. Unions/aliases

6. Click **OK**.

Analyzing query text

Let us analyze the generated query text (listing 13.10).

Listing 13.10. Query text

```
SELECT
    SalesTurnovers.Technician,
    SalesTurnovers.Period,
    SalesTurnovers.Customer,
    SalesTurnovers.RevenueTurnover AS Revenue
FROM
    AccumulationRegister.Sales.Turnovers( , , Day, ) AS SalesTurnovers
```

Note that in the query description the periodicity of data selected from the data source is set to Day (listing 13.11).

Listing 13.11. Specifying virtual table periodicity

```
FROM
    AccumulationRegister.Sales.Turnovers( , , Day, ) AS SalesTurnovers
```

This is why you can use the Period field in the selection.

Resources

Let us proceed to editing the data composition schema.

- Click the **Resources** tab and click >>.

  The wizard selects the only available resource: Revenue.

Parameters

Let us specify the data composition parameters, just like you did for the previous report.

1. Click the **Parameters** tab.
2. For the **BeginOfPeriod** and **EndOfPeriod** parameters, in the **Type** field, select **Date** and specify its format as **Date**.
3. For the **EndOfPeriod** parameter, define the expression as shown in listing 13.12.

Listing 13.12. Expression for calculating the EndOfPeriod parameter value

```
EndOfPeriod(&EndOfPeriod,"Day")
```

Once you are done, the data composition parameters should look as shown in fig. 13.62.

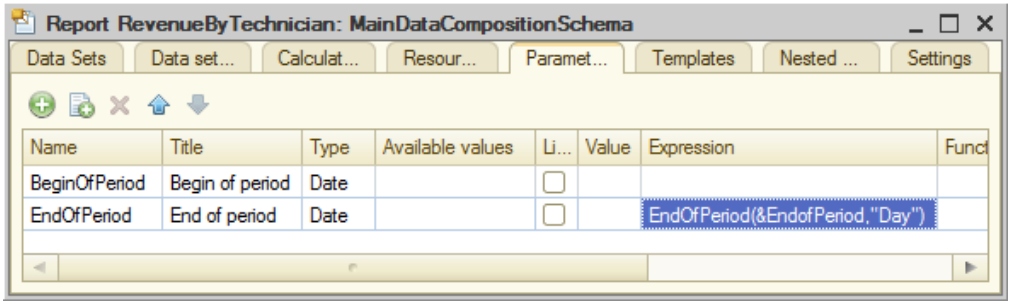


Fig. 13.62. Data composition parameters

Settings

Let us specify the report structure.

You need two nested groupings for this report:

- Top-level grouping by Technician field
- Nested grouping by Period field

Let us create the groupings.

1. On the **Settings** tab, in the command bar, click the **Add** button to add a grouping and then, in the **Field** field, select **Technician** and click **OK** (fig. 13.63).

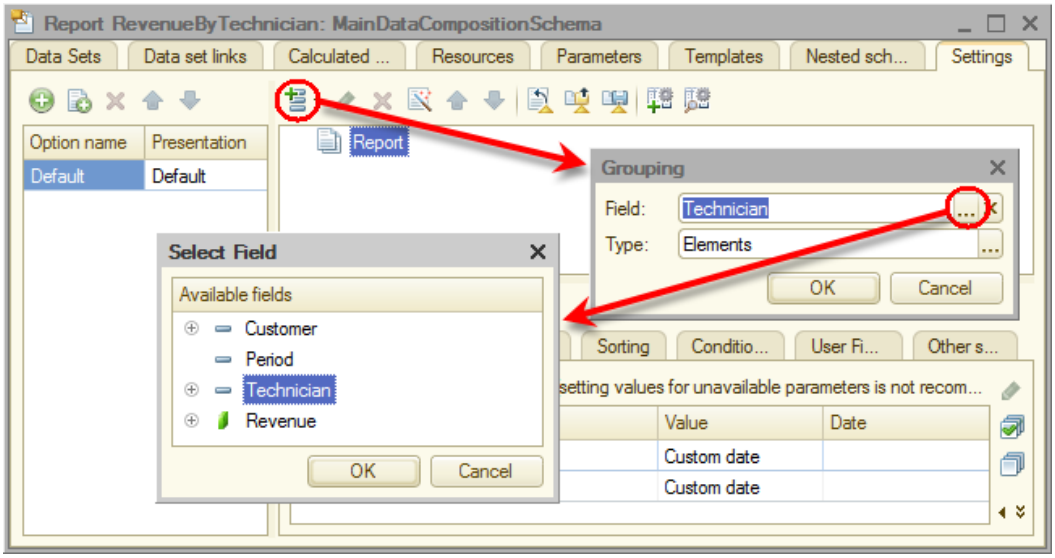


Fig. 13.63. Specifying the grouping field

Let us add the Period grouping that is subordinate to the Technician grouping.

2. Click the **Technician** grouping, click the **Add** button to add a grouping and then, in the **Field** field, select **Period** and click **OK**.

   Then let us add the Detailed records grouping that is subordinate to the Period grouping (without selecting a grouping field).

3. Click the **Period** grouping, click the **Add** button to add a grouping, do not enter the grouping field, and click **OK**.

4. Click the **Selected fields** tab and add **Customer** and **Revenue** to the list of selected fields.

   You do not have to select the Technician and Period fields because these fields are used for grouping data, so their values are automatically displayed.

   Once you perform these steps, the report structure should look as shown in fig. 13.64.

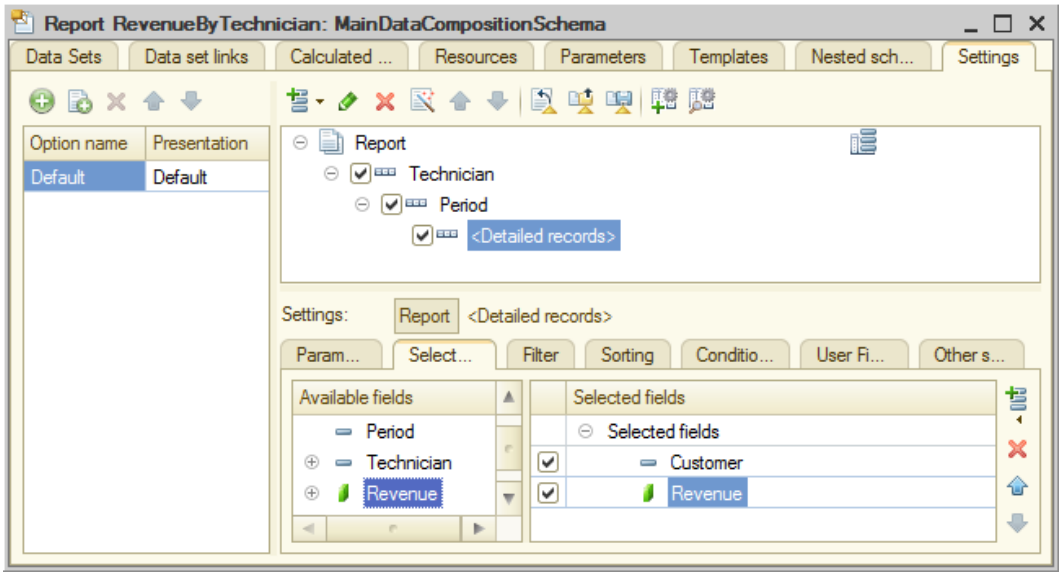
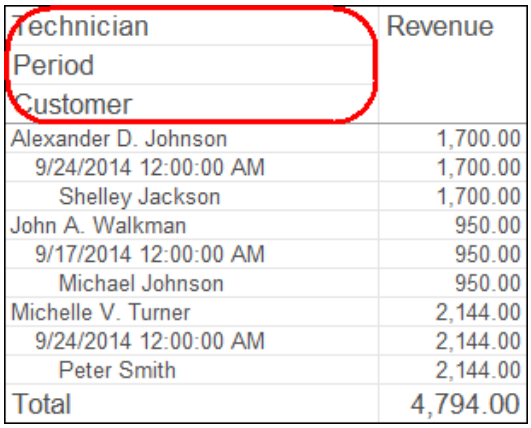


Fig. 13.64. Report structure and fields

5. Click the **Other settings** tab.
6. In the **Grouping field placement** field, select **Separately and only in totals**.

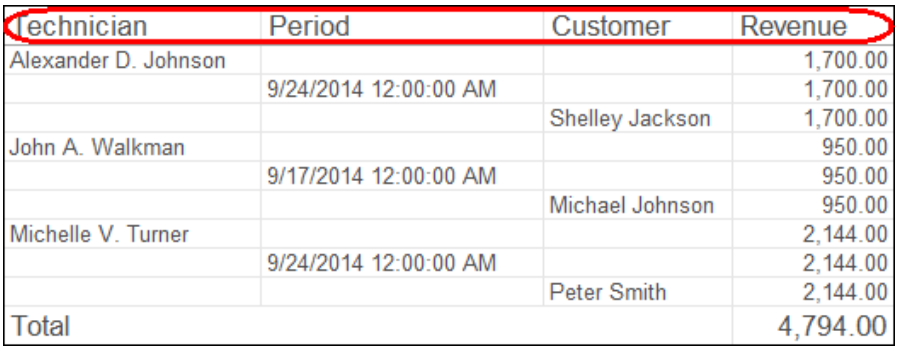
   By default grouping fields are arranged vertically in the report (fig. 13.65).



Technician Period Customer	Revenue
Alexander D. Johnson	1,700.00
9/24/2014 12:00:00 AM	1,700.00
Shelley Jackson	1,700.00
John A. Walkman	950.00
9/17/2014 12:00:00 AM	950.00
Michael Johnson	950.00
Michelle V. Turner	2,144.00
9/24/2014 12:00:00 AM	2,144.00
Peter Smith	2,144.00
Total	4,794.00

13.65. Default vertical arrangement of grouping and total fields

   Selecting Separately and only in totals as the value for this property means that each grouping is located in a separate report area from left to right and its name is displayed only in this grouping (fig. 13.66).



Technician	Period	Customer	Revenue
Alexander D. Johnson			1,700.00
	9/24/2014 12:00:00 AM		1,700.00
		Shelley Jackson	1,700.00
John A. Walkman			950.00
	9/17/2014 12:00:00 AM		950.00
		Michael Johnson	950.00
Michelle V. Turner			2,144.00
	9/24/2014 12:00:00 AM		2,144.00
		Peter Smith	2,144.00
Total			4,794.00

Fig. 13.66. Arrangement of grouping fields as Separately and only in totals

7. In the **Vertical placement of overalls** field, select **Begin**.

   By default totals are located at the end of a vertical list (see fig. 13.66). Selecting this value for the property means that totals are displayed at the top, before the grouping rows (fig. 13.67).

Technician	Period	Customer	Revenue
Total			4,794.00
Alexander D. Johnson			1,700.00
	9/24/2014 12:00:00 AM		1,700.00
		Shelley Jackson	1,700.00
John A. Walkman			950.00
	9/17/2014 12:00:00 AM		950.00
		Michael Johnson	950.00
Michelle V. Turner			2,144.00
	9/24/2014 12:00:00 AM		2,144.00
		Peter Smith	2,144.00

Fig. 13.67. Vertical arrangement with totals at the top

After these changes the report settings on the Other settings tab should look as shown in fig. 13.68.

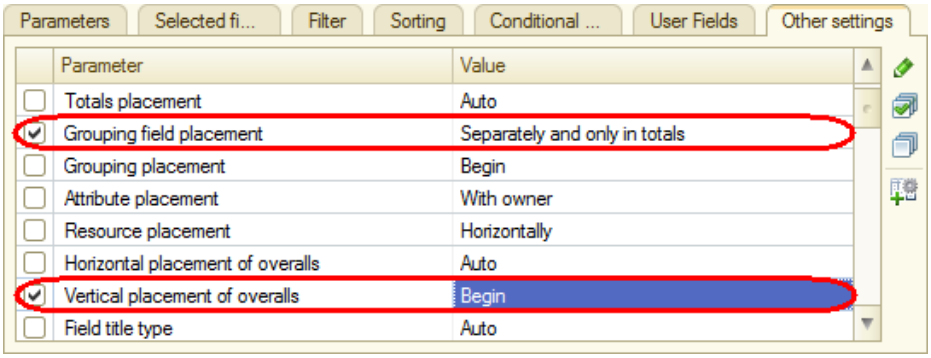


Fig. 13.68. Parameters of report display settings

8. In the **Title** field, type *Revenue by technician*.
9. Specify that the **Begin of period** and **End of period** parameters are included in the list of user settings and displayed directly in the report form (in other words, they serve as quick settings).

   It means that a user can define the report period before generating the report (fig. 13.69).

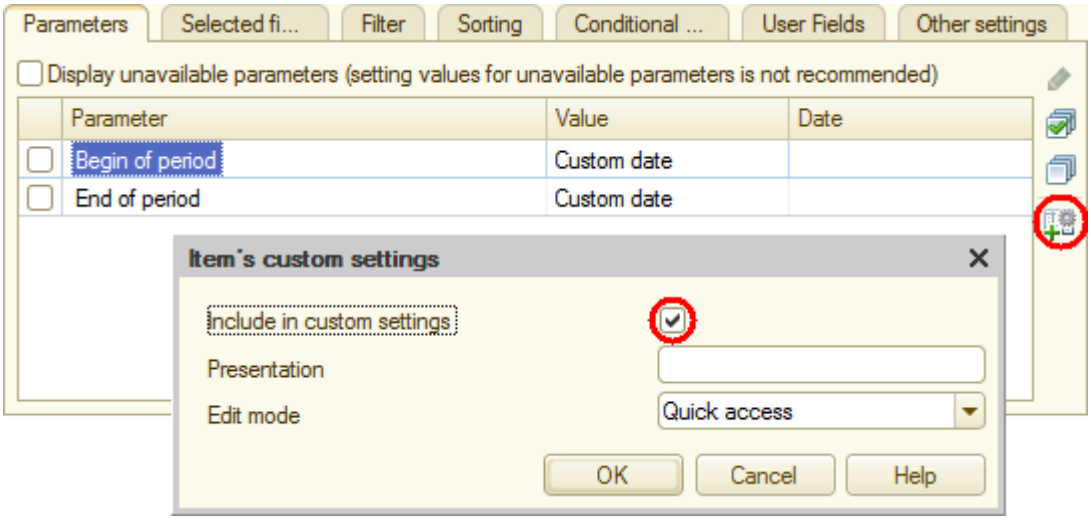


Fig. 13.69. Creating quick report period settings

Finally, let us specify the subsystems where the report is available.

10. Close the data composition schema wizard and then, in the editor of the **RevenueByTechnician** report configuration object, click the **Subsystems** tab.
11. Select the **Payroll** and **Services** check boxes.

    This adds the links that open the report to the Reports menus in the command panels of these subsystems.

In 1C:Enterprise mode

Let us check the report.

1. Start 1C:Enterprise in the debug mode.

   You can see that the Revenue by technician command is now available in the Reports menus of the Services and Payroll sections.
2. Click **Revenue by technician**, specify the report period between 9/17/2014 and 9/24/2014, and generate the report (fig. 13.70).

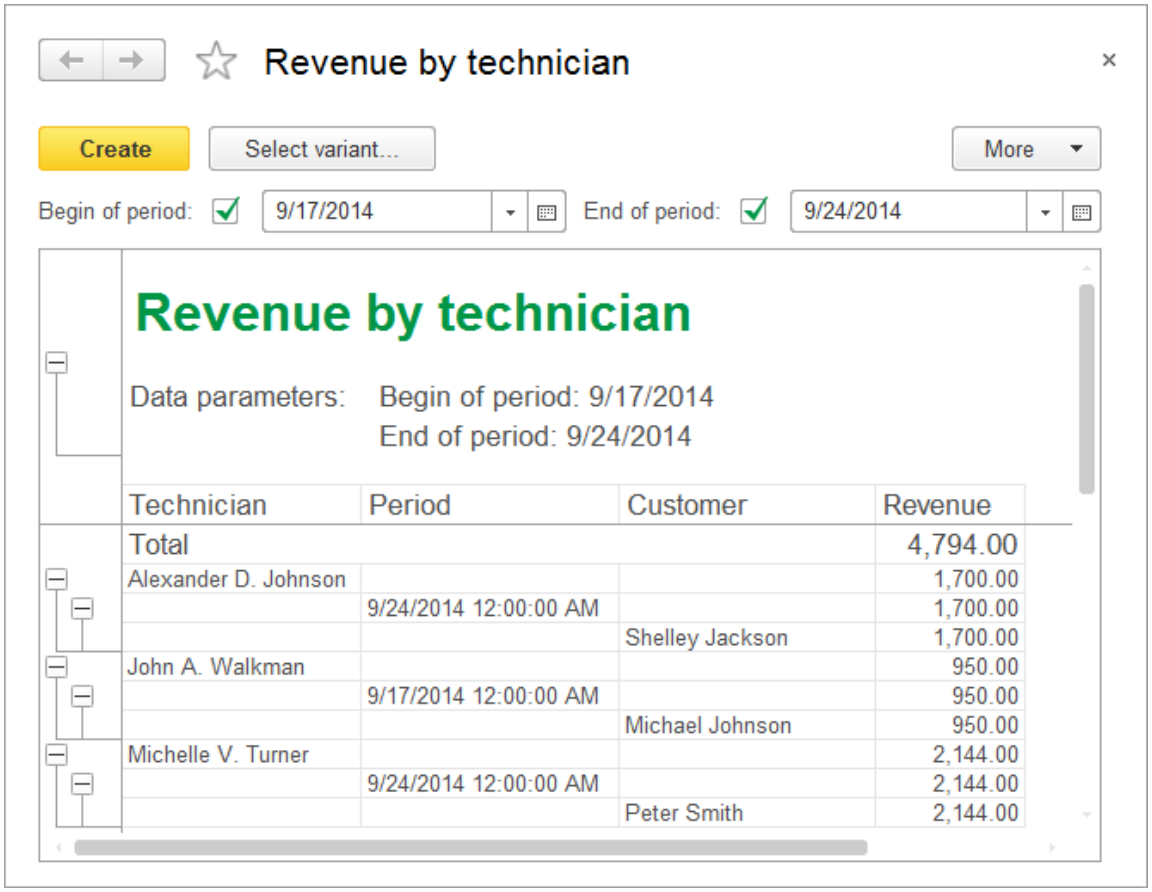


Fig. 13.70. Resulting report

If the report period is not important to users, they can clear the **Begin of period** and **End of period** check boxes. Then the report is generated based on all of the Sales register records available in the database (in this specific case this does not make any difference for the resulting report).

Breaking down by date within the selected period

At the beginning of the section we mentioned that this report should provide breakdown by date within a given period.

So far the report only displays the days that have nonzero records in the Sales accumulation register table.

For adding detailed data to the report, the data composition system features period completion for groupings with a specified periodicity within a specified time period.

So let us modify the report settings to include each day of the report period into the report.

In Designer

Let us fine-tune the report structure.

1. In Designer, open the data composition schema, and click the **Settings** tab.

   Up to this point all the structure settings that you implemented were applied to the entire report. But the data composition system features customization of each report structure item as well.

   Important! When you customize the report settings, pay attention to the buttons in the middle of the window, under the report structure tree. When the **Report** button is selected, the customization applies to the entire report. When the button with a grouping name (such as **Detailed records**) is selected, the customization applies to the grouping.

   Let us customize the settings for the Period grouping.

2. In the report structure tree, click the **Period** grouping and then click the **Period** button in the middle of the window, under the report structure tree.

   The bottom pane displays the settings available for this grouping.

3. Click the **Grouping Fields** tab.

4. For the **Period** field, set **Addition type** to **Day** (fig. 13.71).

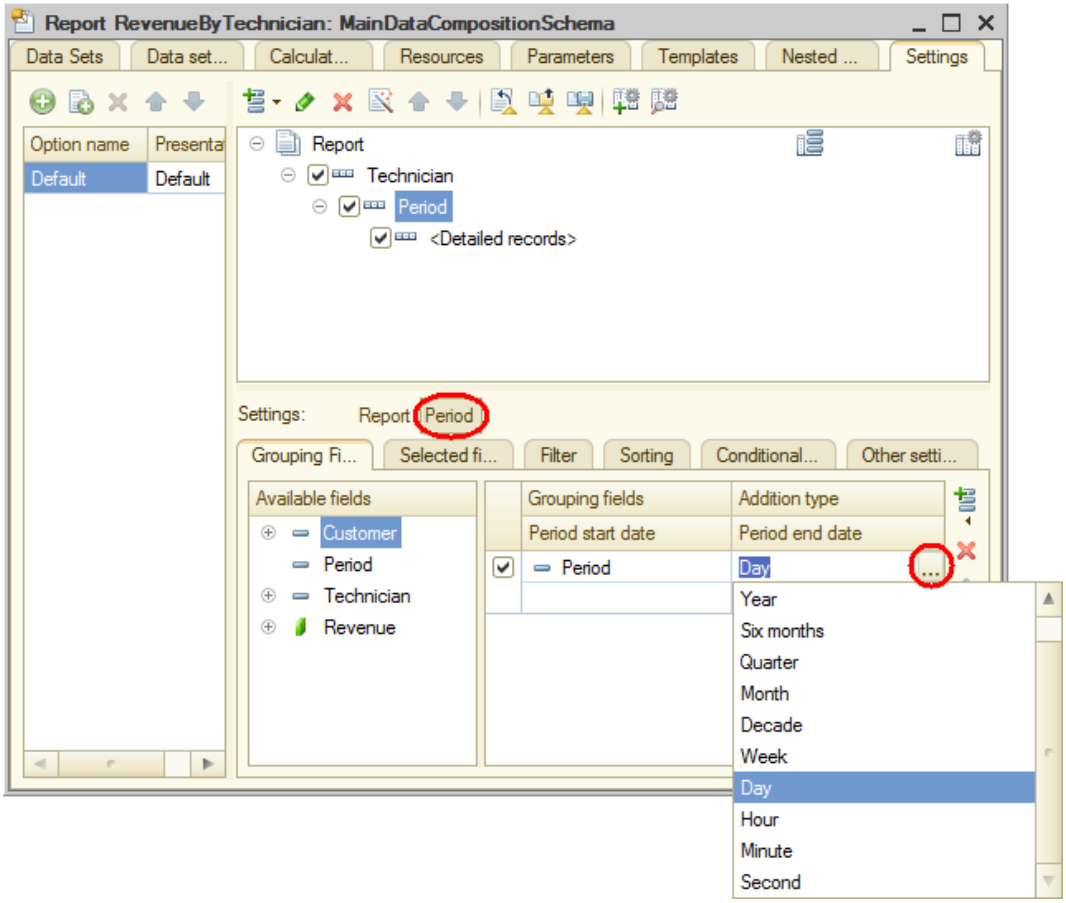


Fig. 13.71. Specifying period completion type

This specifies that records with nonzero values available in this grouping are complemented by daily records.

Next let us specify the period where the completion is performed.

In the fields of the row below you can enter the beginning and end dates for this period. But specifying the dates explicitly does not suit our goal because a user might generate a report for a custom period. So we want the date completion to be applied to the report period selected by the user instead of some fixed period.

5. Double-click the **Period start date** field and then click the **Clear** button.

   Then let us create the data type for this field.

6. Click the **Select** button, select **Data composition field** (fig. 13.72), and click **OK**.

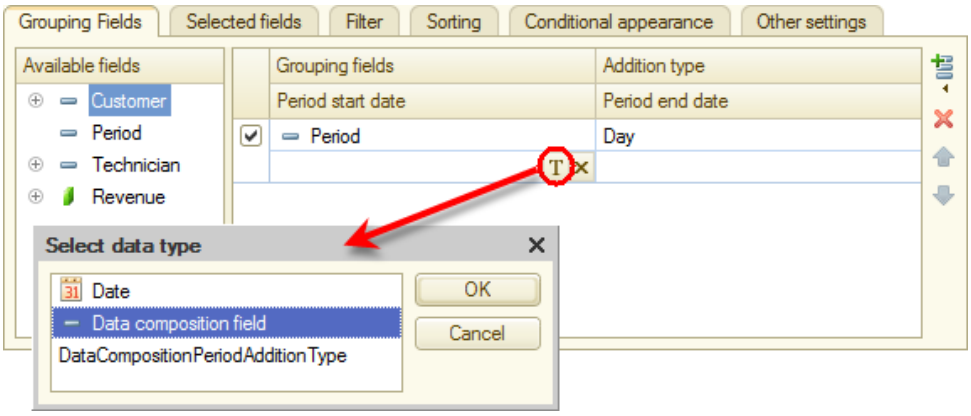


Fig. 13.72. Selecting data type

7. Double-click the **Period start date** field, click the **Select** ... button, and then, in the **Select Field** window, select **BeginOfPeriod** (fig. 13.73) and click **OK**.

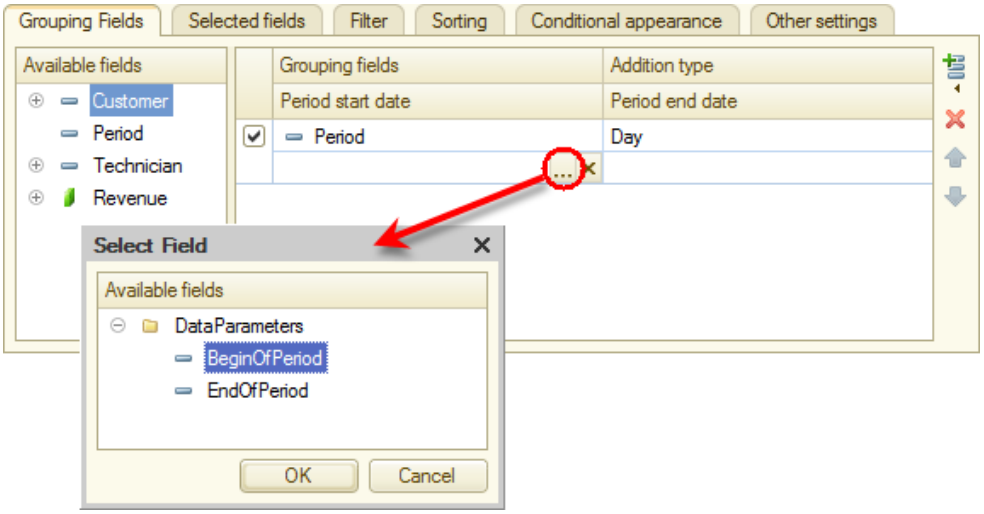


Fig. 13.73. Selecting field

8. In a similar manner specify that the period end date is obtained from the **EndDate** parameter (fig. 13.74).

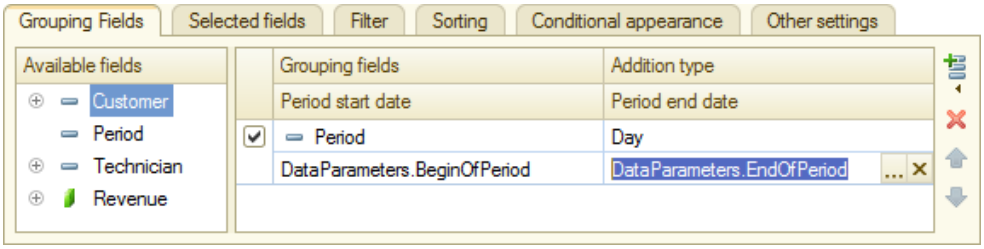


Fig. 13.74. Specifying settings of the Period grouping

In 1C:Enterprise mode

Let us check the report.

1. Run 1C:Enterprise in the debug mode and run the **Revenue by technician** report for the period from 9/17/2014 to 9/24/2014 (fig. 13.75).

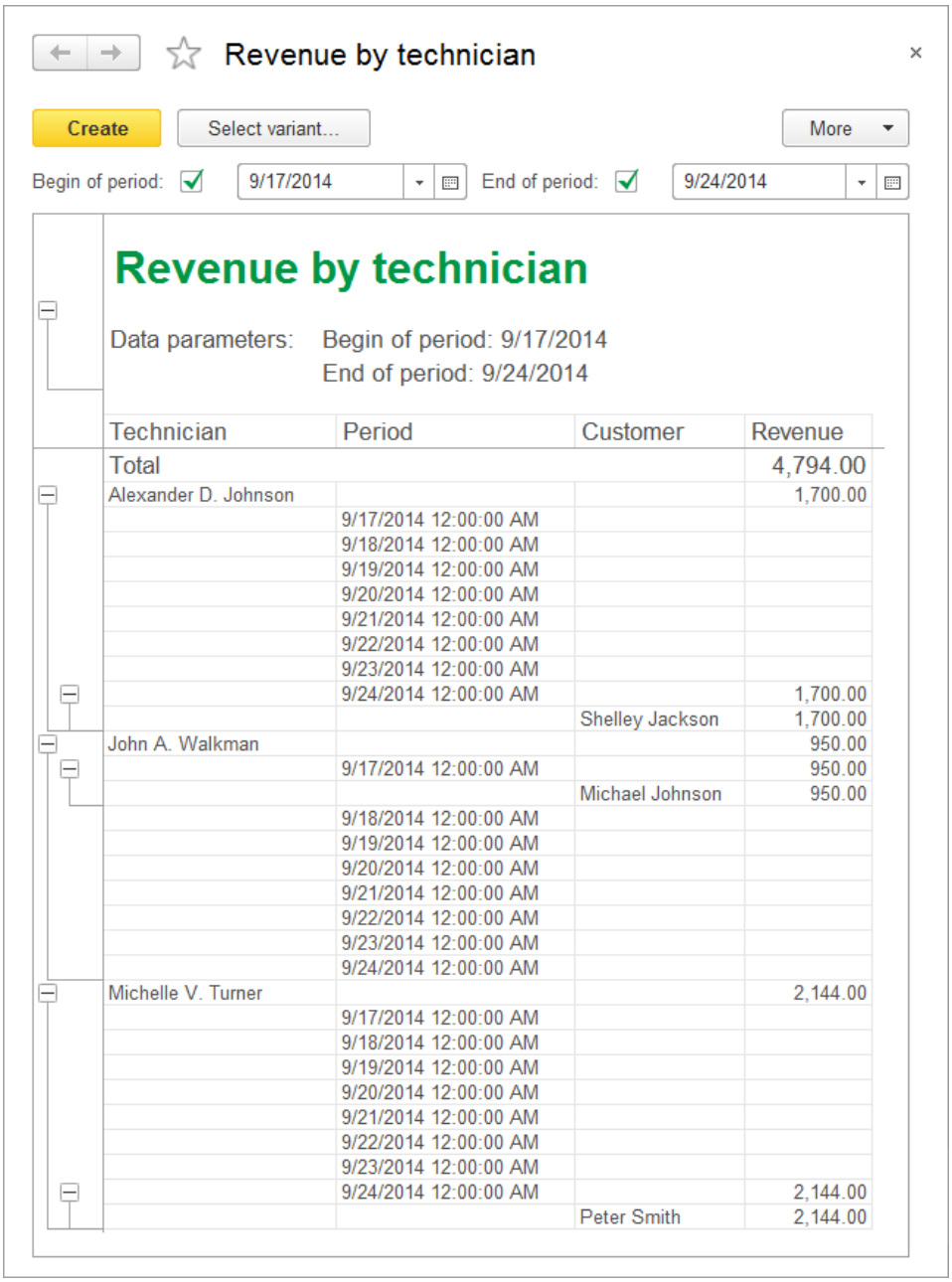


Fig. 13.75. Resulting report

The resulting report displays the accumulation register data for the specified period broken down by date. The report includes the days when no services were rendered.

Report options

To analyze technician performance over a specific period, you might want to have another view of that data. For example, when your CEO makes payroll decisions, they might want to see a chart displaying the share of each technician in the entire company's revenue over a specific period to understand who works better.

Let us create a report option for the RevenueByTechnician report, which displays the data as a chart.

Charts

The Chart object is intended for displaying charts and diagrams in spreadsheets and forms.

Generally speaking, a chart consists of points, series, and series values in those points (fig. 13.76).

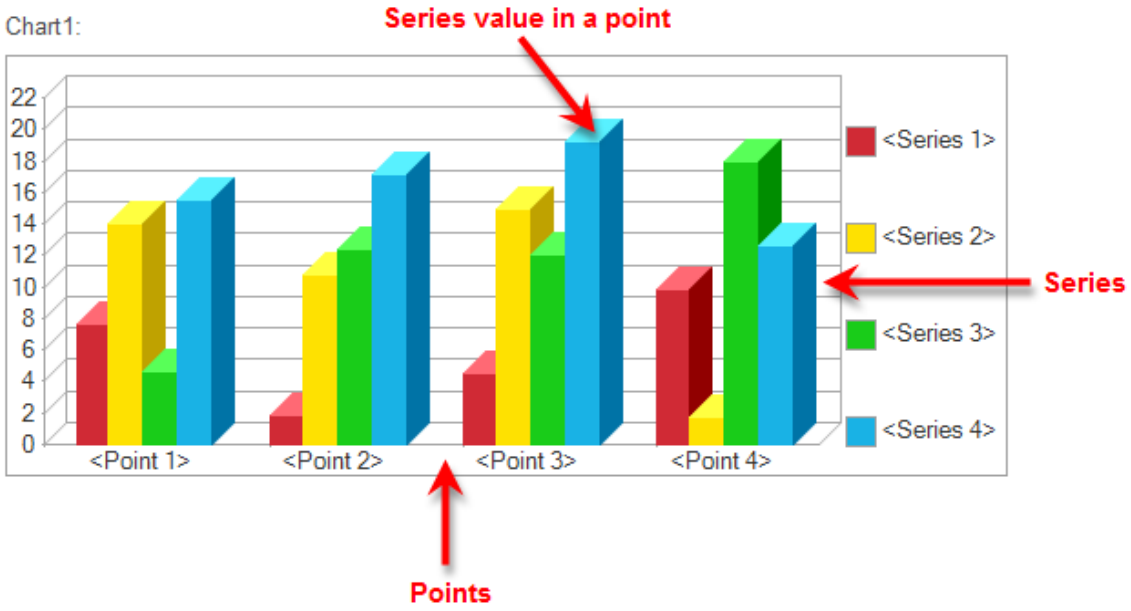


Fig. 13.76. Sample chart

As a rule, points represent moments of time or objects that have characteristics whose values you obtain, while series represent the characteristics. Chart values are located at intersections of series and points.

For example, a chart depicting monthly sales for service types consists of points (months), series (service types), and values (sales revenues).

The Chart 1C:Enterprise script object has three areas that define the chart appearance: the plot area, the header area, and the legend area (fig. 13.77).

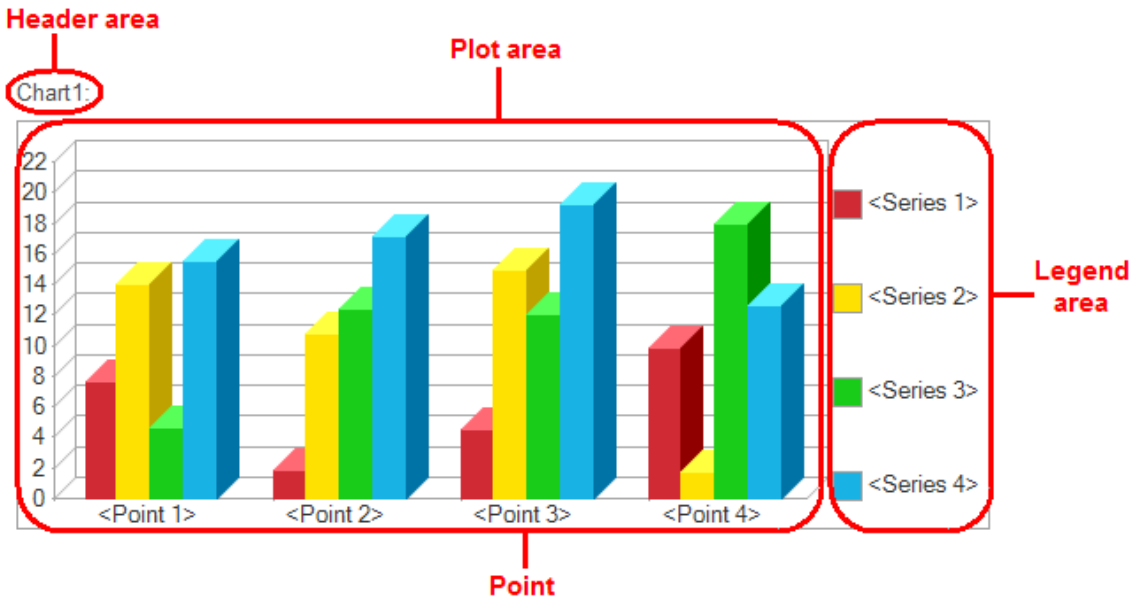


Fig. 13.77. Chart areas

You can add a chart as an independent report structure item. In the RevenueByTechnician report option you will include a chart in the data composition schema settings structure.

In Designer mode

Let us create a report option.

1. In Designer, open the data composition schema, and click the **Settings** tab.

   The left pane displays a list of report options.

   When you first create report settings, the data composition system automatically creates the Default settings option. You can see this option in the list. Let us add another option.

2. Click the **Add** button above the list.
3. For the report option name, type *RevenueAmounts*. For the report option presentation, type *Revenue amounts* (fig. 13.78).

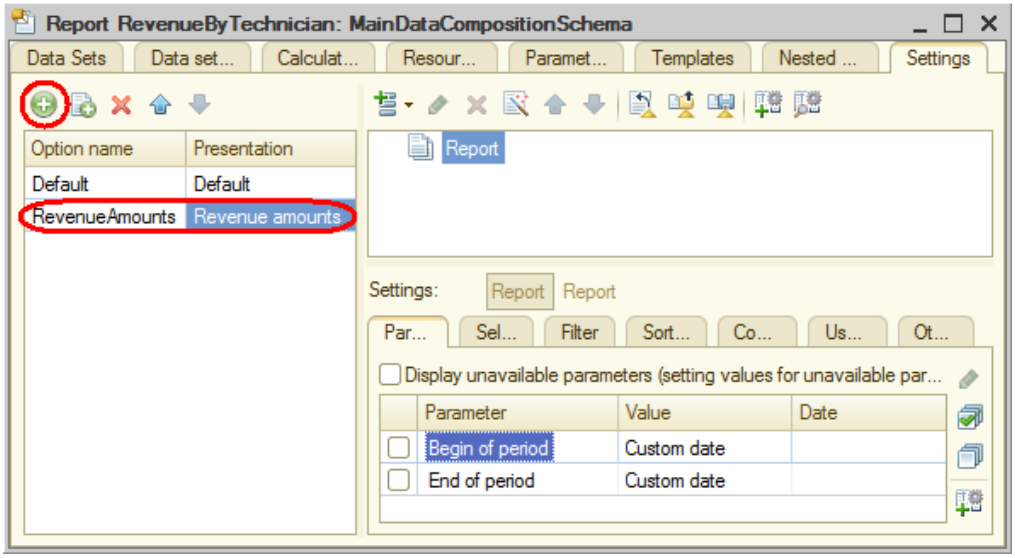


Fig. 13.78. Adding a settings option

You can see that the report structure and all report settings are cleared.

But they are not gone. They are simply invisible because they belong to the Default settings option.

If a report has multiple options, you can only see and edit the settings for the currently selected option. And the other parts of the data composition schema (resources, parameters, and data sets) are not changed. The report data is obtained using the same database query. Only the settings that define the report view are different.

Let us add a chart to the report structure.

4. Right-click the **Report** root item and click **New chart** (fig. 13.79).

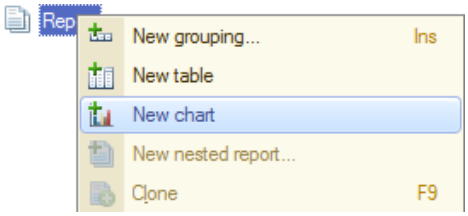


Fig. 13.79. Adding a chart to report structure

5. Click the **Points** branch and add a grouping by the **Technician** field to this branch.

   A gauge chart is handy for displaying the share of each technician in the entire revenue amount. For this chart type it is enough to specify points, you do not have to specify series.

   One or several report resources are always used as chart values. You have a single Revenue resource (the resource field is marked with a specific icon, so you can easily distinguish it from other fields).

6. Click the **Selected** fields tab and select the **Revenue** field to be displayed in the report.

   The report structure should look as follows (fig. 13.80).

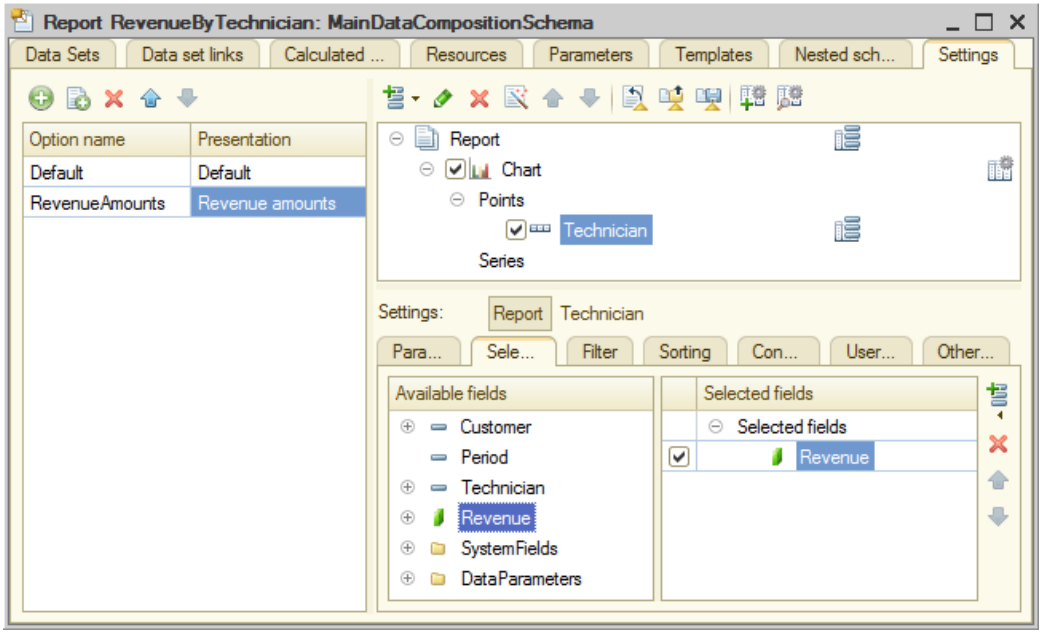


Fig. 13.80. Report structure and chart settings

Important! A chart must always display one or several report resources, otherwise it will return an error.

7. On the **Other settings** tab, select **Gauge** as the chart type (fig. 13.81).

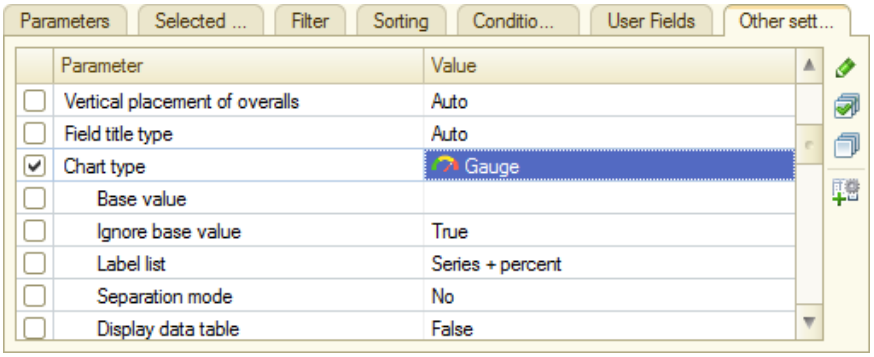


Fig. 13.81. Specifying chart type

8. Scroll down the list of gauge properties and define the gauge bands: *Bad*, *Good*, and Excellent (fig. 13.82).

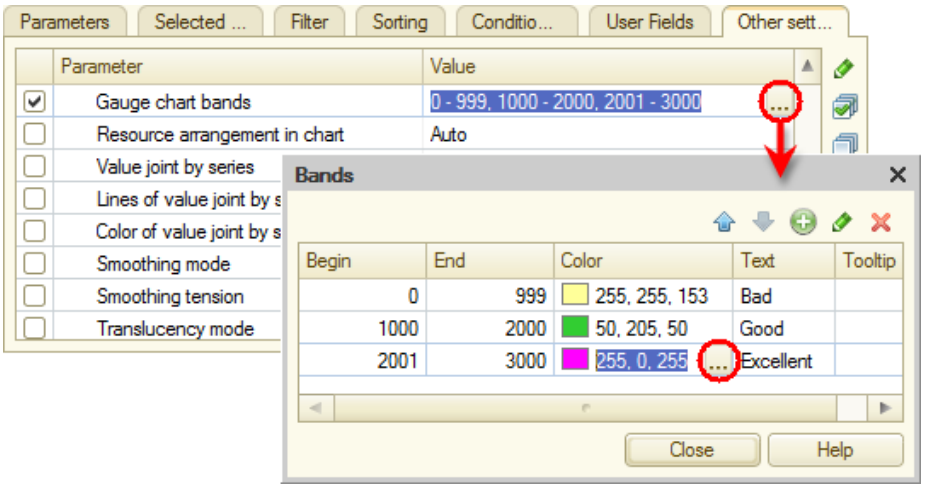


Fig. 13.82. Specifying gauge band settings

9. Include **Begin of period** and **End of period** to the list of user settings and set their **Edit mode** to **Quick access**.

   Important! You have to specify the user settings for each report option independently because each option has its own user settings.

In 1C:Enterprise mode

Let us check the report.

1. Run 1C:Enterprise in the debug mode.
2. In the **Payroll** section, on the **Reports** menu, click **Revenue by technician**.
3. Click **Select variant** (fig. 13.83).

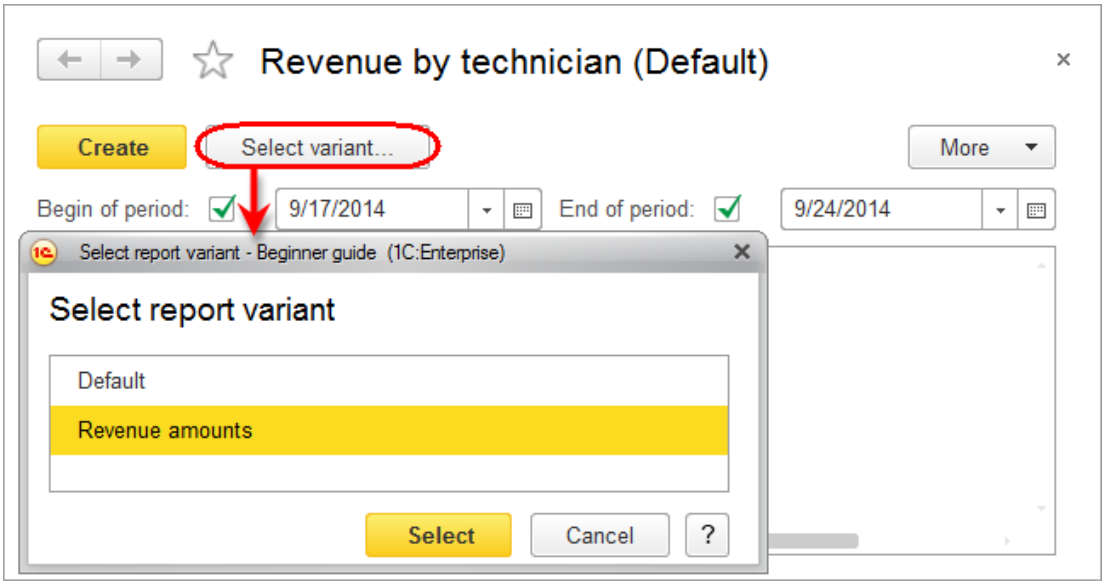


Fig. 13.83. Specifying report option

The **Select report variant** window contains two options: **Default** and **Revenue amounts** (which you have just created).

4. Click **Revenue amounts** and click **Select**.
5. Click **Create** (fig. 13.84).

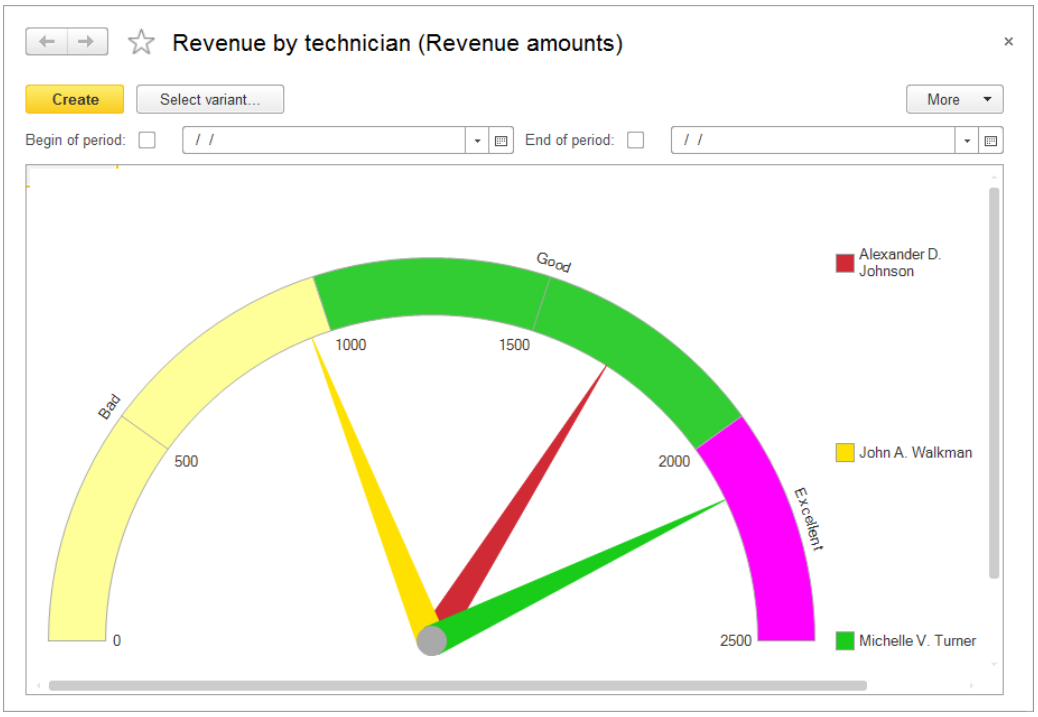


Fig. 13.84. Resulting report

This report option displays the same data as the default option but arranged as a gauge. The chart clearly displays the share of each technician in the entire revenue volume. Tooltips are available when you move the pointer over chart arrows.

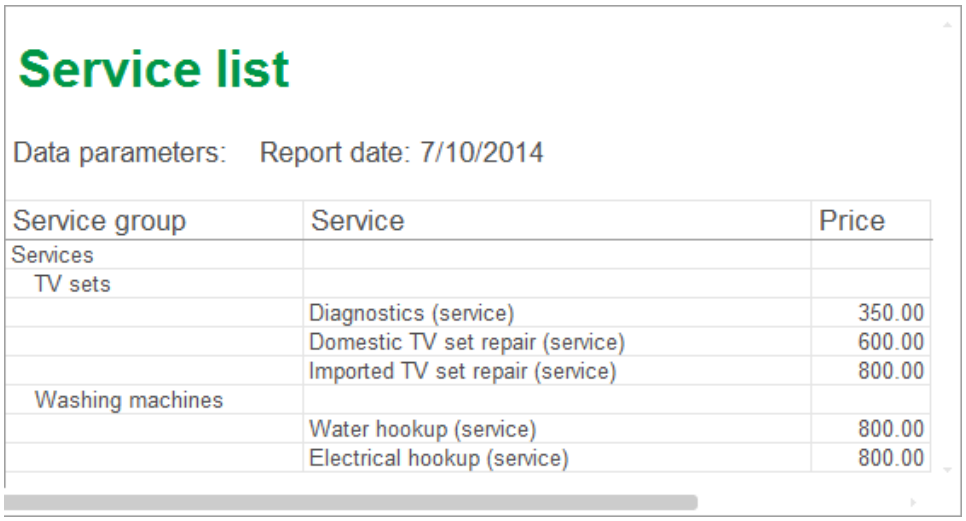
Note that you have not specified the report period here, thus including all service-related data to the report. In a live system the number of documents is much greater than three, that is why you need the option to set the report period. For example, one might want to view monthly technician revenues.

But if you need to view data for a specific technician broken down by days and customers, it is sufficient to select the Default report option and generate the report.

In the Revenue by technician report development exercise you learned how to create and use report options to display technician performance data using the view that suits your needs best.

Getting current values from a periodic information register

The Service list report displays the list of services provided by Jack of All Trades with their prices (fig. 13.85).



Service list

Data parameters: Report date: 7/10/2014

Service group	Service	Price
Services		
TV sets		
	Diagnostics (service)	350.00
	Domestic TV set repair (service)	600.00
	Imported TV set repair (service)	800.00
Washing machines		
	Water hookup (service)	800.00
	Electrical hookup (service)	800.00

Fig. 13.85. Resulting report

Using this report as an example, we will show how to obtain the latest values from a periodic information register and how to display hierarchical catalogs.

In Designer mode

Let us begin the report creation.

1. Create a report configuration object named *ServiceList*.
2. Run the Data composition schema wizard.
3. Add **Data set – query** and run the Query builder.

Query for a data set

Let us create a query for a data set.

1. As a data source for the query, select the object (referential) table of the **MaterialsAndServices** catalog and the virtual table of the **Prices.SliceLast** information register.

   To avoid ambiguous names in the query, let us change the name of the MaterialsAndServices table.

2. In the **Tables** list, right-click **MaterialsAndServices**, click **Rename Table**, and type *MaterialsAndServicesCat*.

Virtual table parameters

Let us specify virtual table parameters.

1. In the **Tables** list, click **PricesSliceLast** and then click the **Virtual table parameters** button.
2. Specify that the period is passed in the *ReportDate* parameter (fig. 13.86).

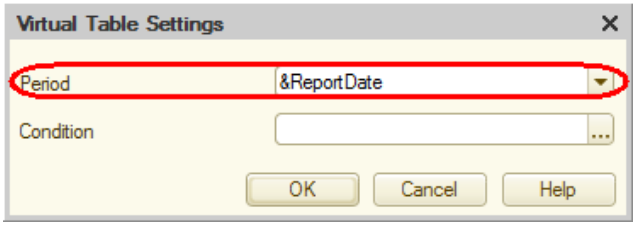
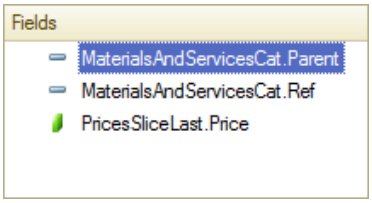


Fig. 13.86. Virtual table parameters

3. Select the following fields from the tables (fig. 13.87):
   - **MaterialsAndTablesCat.Parent**
   - **MaterialsAndTablesCat.Ref**
   - **PricesSliceLast.Price**



13.87. Selected fields

Left join of tables

Let us specify the table join.

1. Click the **Links** tab.

   The platform automatically generates a link condition stating that the value of the MaterialsAndServices dimension of the information register is equal to the reference to the MaterialsAndServices catalog item. It is okay with us.

2. Clear the **All** check box for the register table and select the **All** check box for the catalog table.

   This is how you specify Left join as the link type for the catalog table (fig. 13.88).

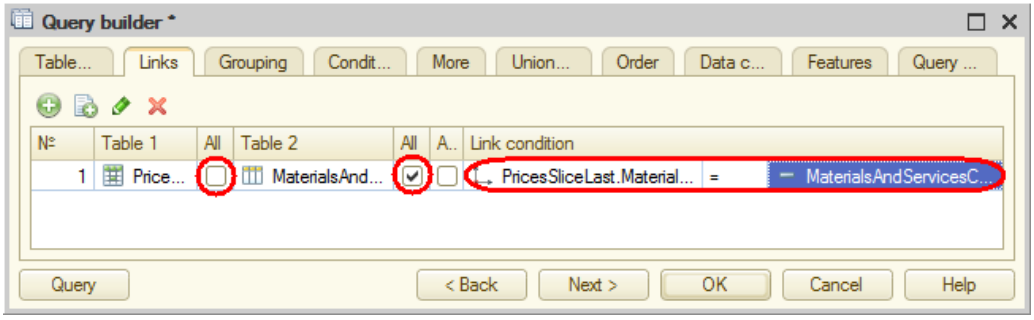


Fig. 13.88. Link between the tables in a query

3. On the **Conditions** tab, specify the criteria for selecting items from the **MaterialsAndServices** catalog: select only items that have the type passed in the **MaterialServiceType** parameter (fig. 13.89).

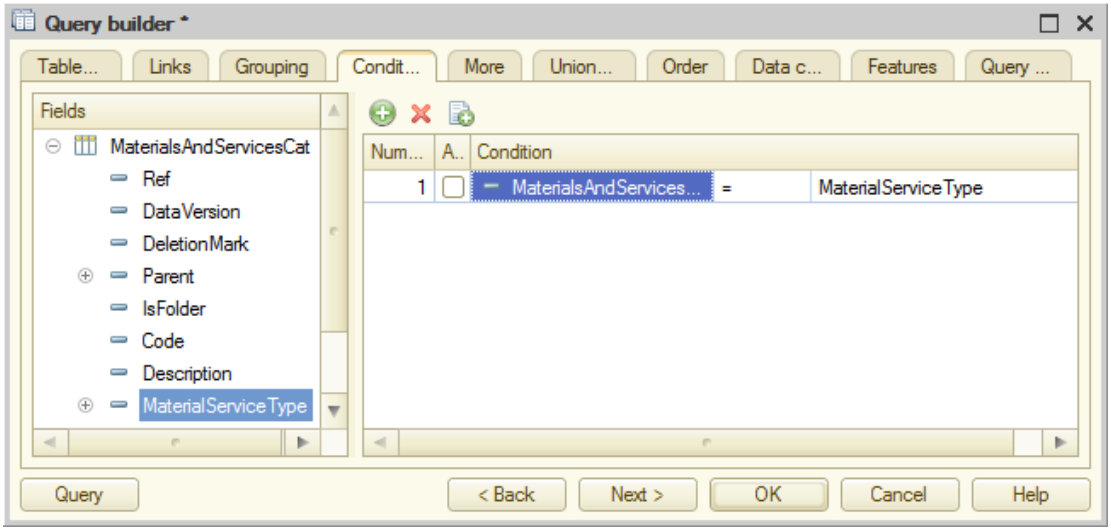


Fig. 13.89. Item selection condition

Field aliases

Let us set aliases for the catalog item presentation (Ref field) and for the register field.

1. Click the **Unions/Aliases** tab.
2. In the **Field name** column, change *Ref* to *Service* and change *Parent* to *ServiceGroup* (fig. 13.90).

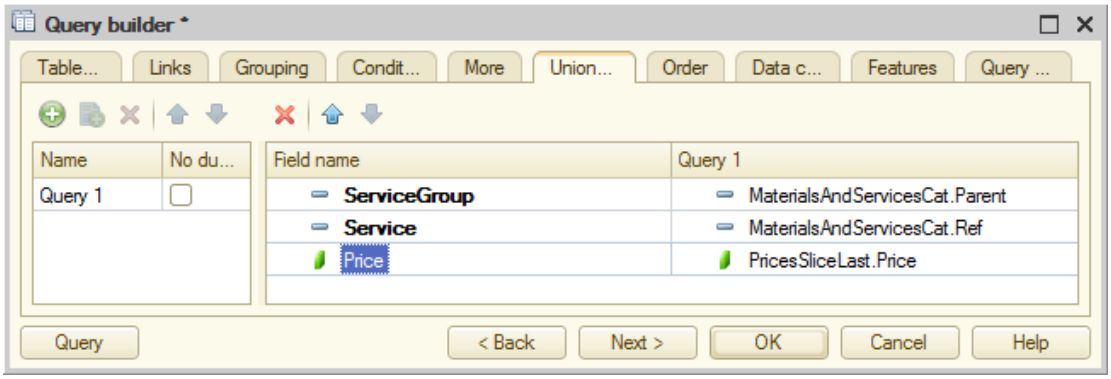


Fig. 13.90. Specifying query field aliases

3. Click **OK**.

   You have completed the query creation.

Analyzing query text

Let us analyze the generated query text (listing 13.13).

Listing 13.13. Query text

```
SELECT
    MaterialsAndServices.Parent AS ServiceGroup,
    MaterialsAndServicesCat.Ref AS Service,
    PricesSliceLast.Price
```

```
FROM
    Catalog.MaterialsAndServices AS MaterialsAndServicesCat
        LEFT JOIN InformationRegister.Prices.SliceLast(&ReportDate, ) AS
            PricesSliceLast
        ON (PricesSliceLast.MaterialOrService = MaterialsAndServicesCat.Ref)
WHERE
    MaterialsAndServicesCat.MaterialServiceType = &MaterialServiceType
```

Almost all of the clauses used in this query are already familiar to you. Let us proceed to editing the data composition schema.

Resources

Let us add a resource.

1. Click the **Resources** tab and click >>.

   The wizard selects the only available resource: Price.

2. In the **Calculate by** column, click the **Select** ... button and select the **Service** field (fig. 13.91).

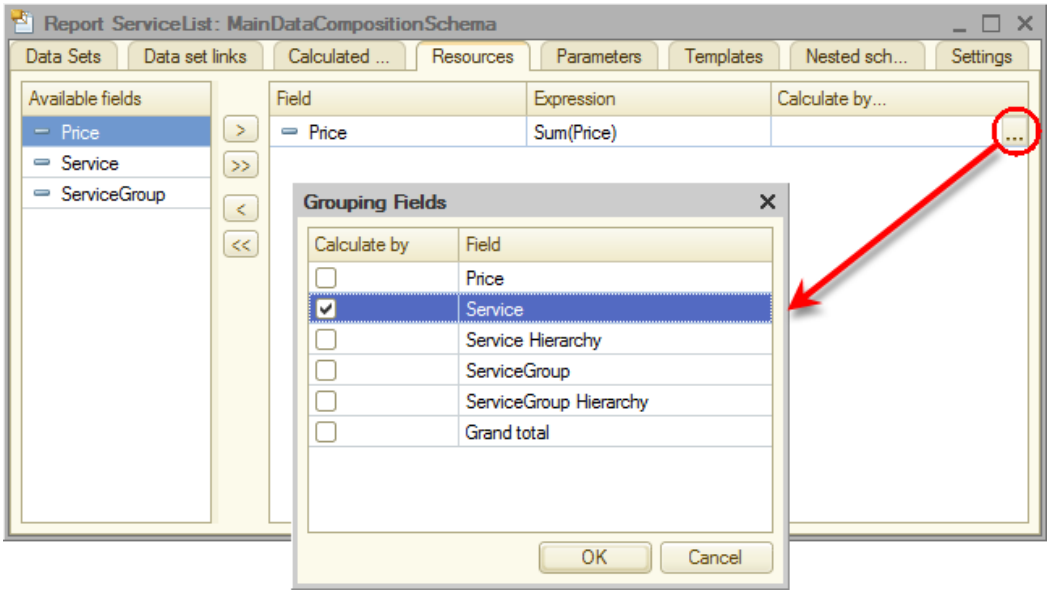


Fig. 13.91. Data composition resources

This is required to have the price totals displayed for specific services, since calculating prices for groupings and overall totals does not make sense.

Parameters

Let us specify the data composition parameters.

1. On the **Parameters** tab, set the **MaterialServiceType** parameter value to *Enum.MaterialServiceTypes.Service*.

2. For the **ReportDate** parameter, clear the **Availability restriction** check box.
3. For the **ReportDate** parameter, in the **Type** field, select the **Date** date format.
4. For the **Period** parameter, select the **Availability restriction** check box (fig. 13.92).

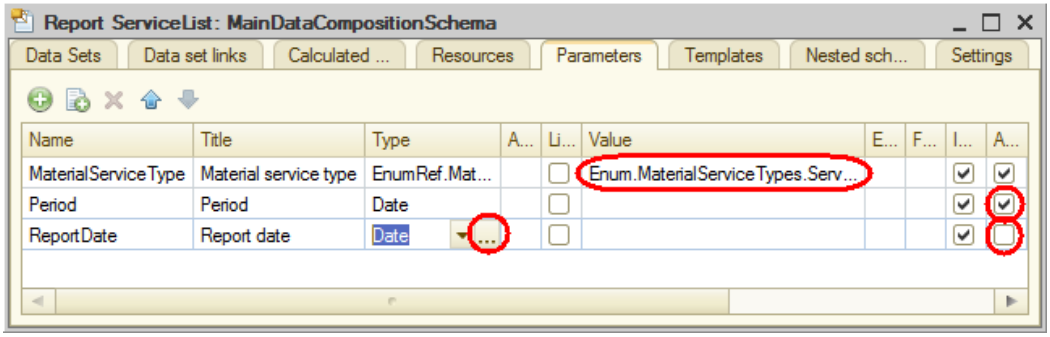


Fig. 13.92. Composition schema parameters

Settings

Let us specify the report structure.

1. On the **Settings** tab, create a grouping by the **ServiceGroup** field with the **Hierarchy** type (fig. 13.93).

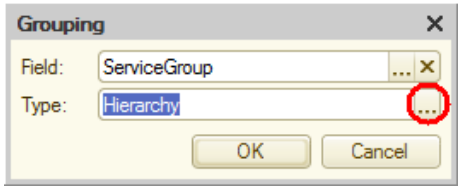


Fig. 13.93. Selecting grouping field and type

Note. Previously you always used the default hierarchy type: Elements. The following hierarchy types are available for report groupings:

- **Elements**. The grouping only includes nonhierarchical records.
- **Hierarchy**. The grouping includes both hierarchical and nonhierarchical records.
- **Hierarchy only**. The grouping only includes hierarchical records.

2. Create another grouping inside this one, without specifying the grouping field.

   It will contain detailed report records.
3. Click the **Selected fields** tab and specify that the **Service** and **Price** fields are included in the report (fig. 13.94).

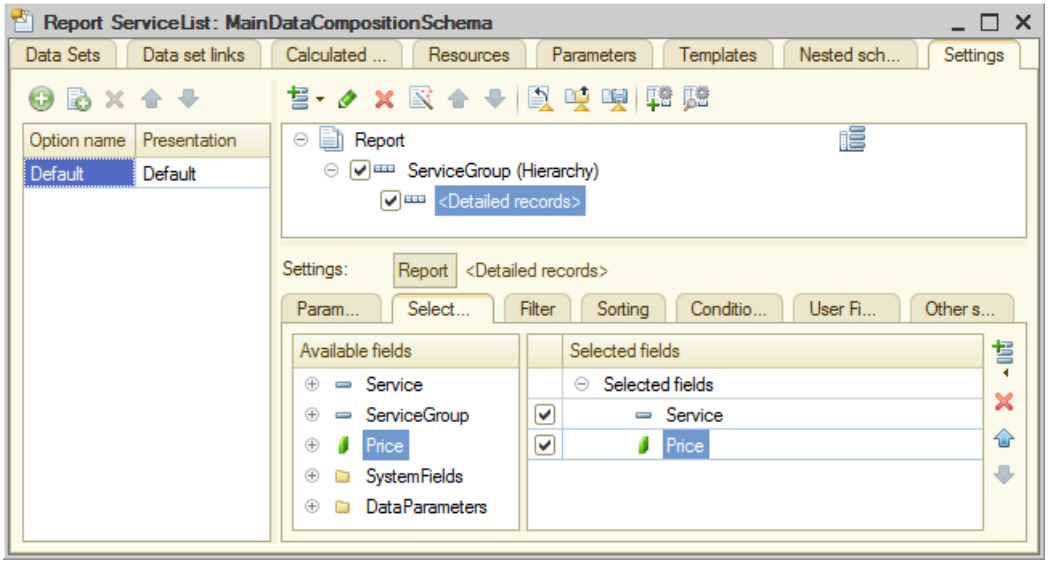


Fig. 13.94. Report structure and fields

Let us customize the report appearance.

4. Click the **Other settings** tab and set **Vertical arrangement of overalls** to **None**.

   This disables the display of overall totals in the report.

5. Set **Grouping field placement** to **Separately and only in totals** (fig. 13.95).

   This improves the report usability.

6. Set the report title to *Service list*.

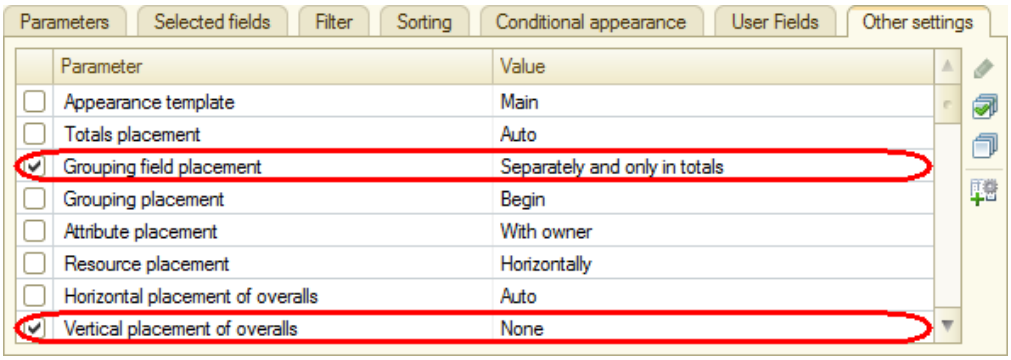


Fig. 13.95. Parameters of report display settings

7. Include the **Report date** parameter in the user settings and set its **Edit mode** to **Quick access**.

   Finally, let us specify the subsystems where the report is available.

8. Close the data composition schema wizard and then, in the editor of the **ServiceList** report configuration object, click the **Subsystems** tab.

9. Select the **Accounting** and **Services** check boxes.

In 1C:Enterprise mode

Let us check the report.

1. Run 1C:Enterprise in the debug mode.
2. Open the **Prices** periodic register.
3. Add a record for the **Diagnostics** service: price 350 as of 7/10/2014 (fig. 13.96).

   This is required for testing the report.

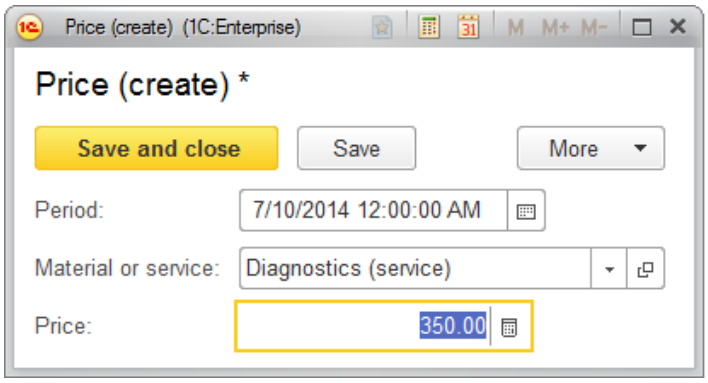


Fig. 13.96. Adding a Prices register record for the Diagnostics service

4. Run the **Service list** report as of 7/7/2014 (fig. 13.97).

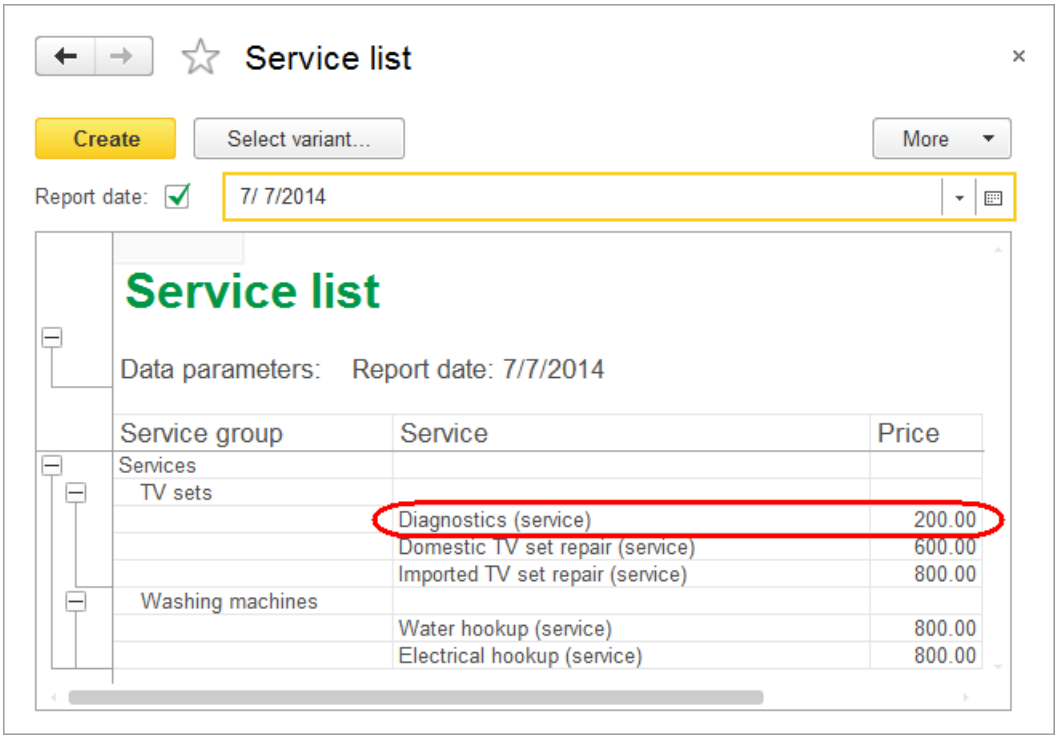


Fig. 13.97. Resulting report

The report displays the Diagnostics service price as of 7/7/2014, which is 200 dollars.

5. Run the report as of 7/10/2014 (fig. 13.98).

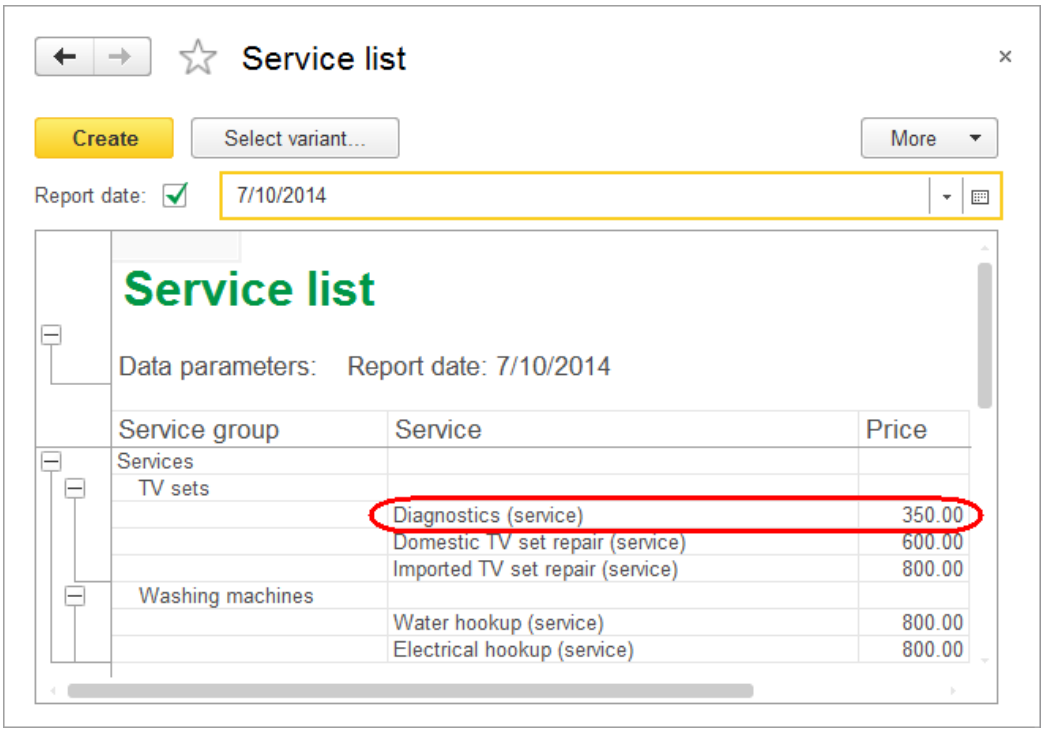


Fig. 13.98. Resulting report

The report displays the new Diagnostics service price: 350 dollars.

This example shows how the data composition system gets the latest values from a periodic information register and how to display groupings based on the catalog hierarchy.

Using calculated fields in reports

Let us create the next report: Profit by customer. It visualizes the lifetime profit of Jack of All Trades by customer (fig. 13.99).

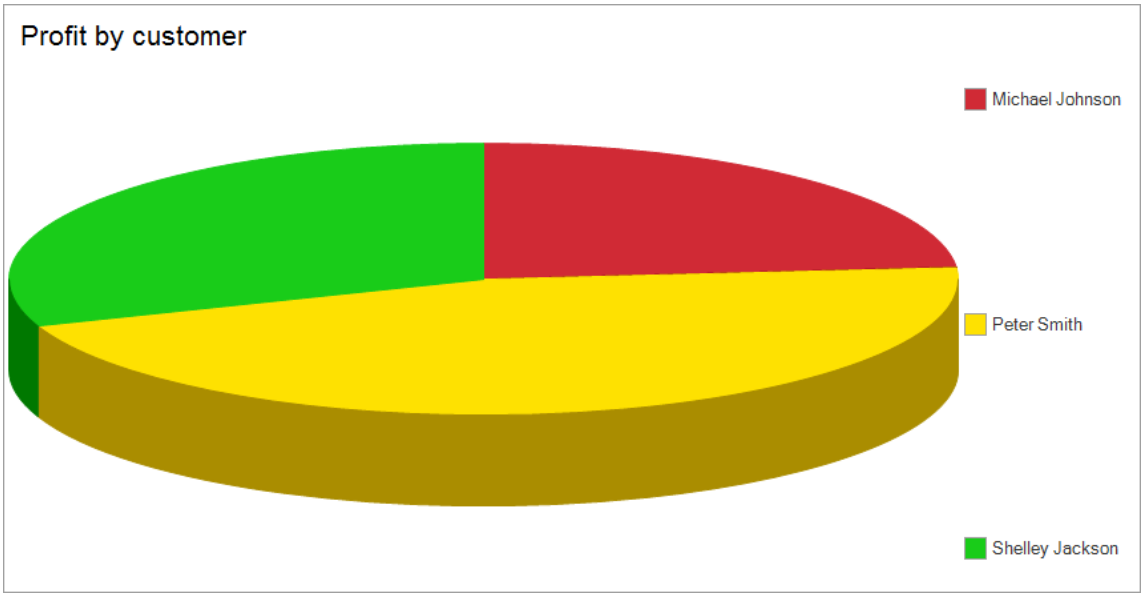


Fig. 13.99. Resulting report

Using this report as an example, we will show how to use calculated fields and how to display a report as a pie chart and as a bar chart.

In Designer mode

Let us begin the report creation.

1. Create a report configuration object named *ProfitByCustomer*.
2. Run the Data composition schema wizard.
3. Add **Data set – query** and run the Query builder.

Query for a data set

Let us specify the query data set.

1. Select the **Sales.Turnovers** virtual table as the data source.
2. Select the following fields from this table (fig. 13.100):
   - **SalesTurnovers.Customer**
   - **SalesTurnovers.RevenueTurnover**
   - **SalesTurnovers.CostTurnover**

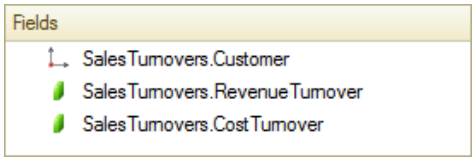


Fig. 13.100. Fields selected for the query

3. Click the **Unions/Aliases** tab.
4. Change *RevenueTurnover* to Revenue and change *CostTurnover* to *Cost*.
5. Click **OK**.

   This completes the query creation. Since the query contains only expressions that you already know, we will not explain it in detail. Let us proceed to editing the data composition schema.

Calculated fields

This is the first time you need to display a field that is not available in the data set. Previously you always used fields defined in the data sets. To display the profit for rendered services by customer, you need an additional field calculated as revenue minus service cost.

For this purpose the data composition system provides the option to define a calculated field.

Calculated fields are additional data composition schema fields whose values are calculated according to the specified formula.

1. Click the **Calculated fields** tab.

2. Click the **Add** button.

   This adds a calculated field.

3. In the **Data path** column, type **Profit**.

4. In the **Expression** column, type the expression for calculating the field value (listing 13.14).

Listing 13.14. Expression for calculating the Profit field value

```
Revenue - Cost
```

The platform fills the calculated field title (which is displayed in the report) automatically, you have the option to change it (fig. 13.101).

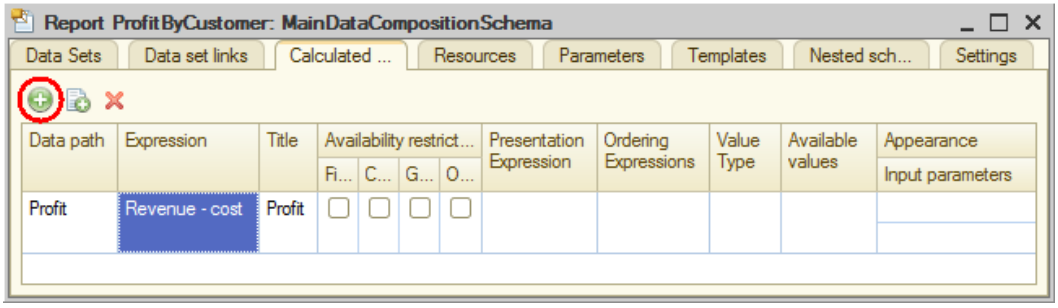


Fig. 13.101. Creating a calculated field

You can add a calculated field to report resources in order to calculate group and overall totals by this field.

Resources

Let us add the resources.

- Click the **Resources** tab and click >>.

  You can see that the calculated field is included in the list of resources (fig. 13.102).

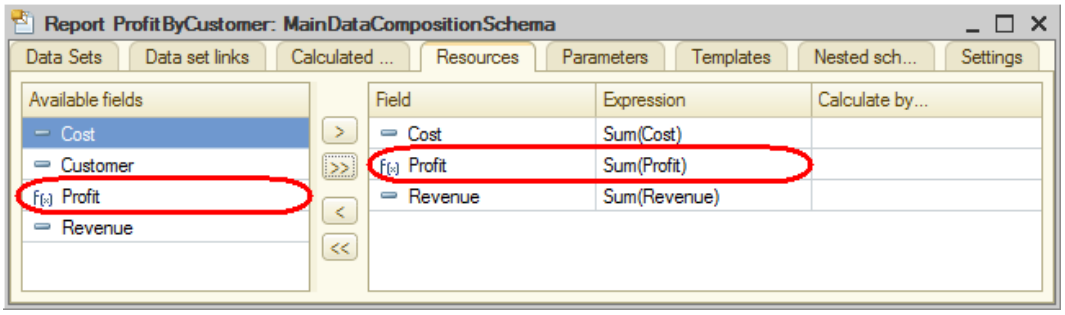


Fig. 13.102. Data composition schema resources

Settings

Let is add a chart to the report structure.

1. On the **Settings** tab, in the command bar, click the **Add** button and then click **New chart** (fig. 13.103).

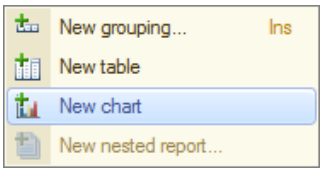


Fig. 13.103. Adding a chart to report structure

2. Click the **Points** branch and add a grouping by the **Customer** field to this branch.

   Keep the chart series unchanged. A pie chart is perfect for displaying profit by customers and this chart type only requires specifying points, so you do not have to specify series.

   The chart values always represent one or several report resources.

3. Click the **Selected fields** tab and include the **Profit** field in the report.

   The report structure should look as shown in fig. 13.104.

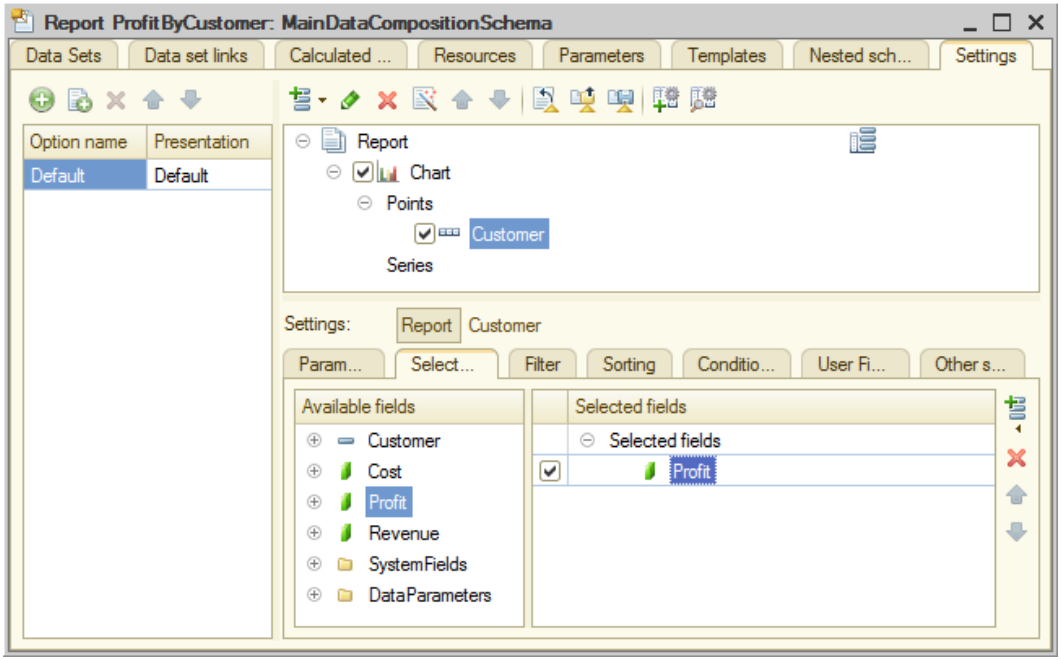


Fig. 13.104. Report structure and chart settings

4. Click the **Other settings** tab, set **Chart type** to **Pie 3D** and include **Chart type** in the list of quick user settings (fig. 13.105).

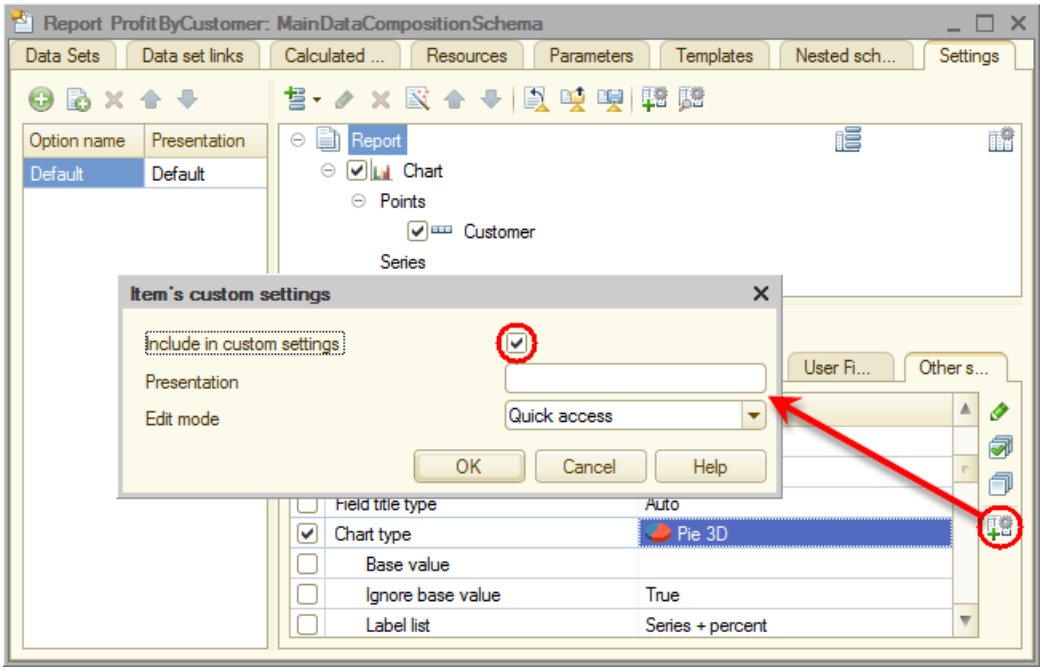


Fig. 13.105. 3D pie chart

5. Enter the **Profit by customer** report title.

   Finally, let us specify the subsystems where the report is available.

6. Close the data composition schema wizard and then, in the editor of the **ServiceList** report configuration object, click the **Subsystems** tab.

7. Select the **Accounting** and **Services** check boxes.

In 1C:Enterprise mode

Let us check the report.

1. Run 1C:Enterprise in the debug mode.

2. In the **Accounting** section, on the **Reports** menu, click **Profit by customer**.

3. Click **Create**.

   You can see that the profit generated by rendering services to specific clients is displayed as a pie chart (fig. 13.106).

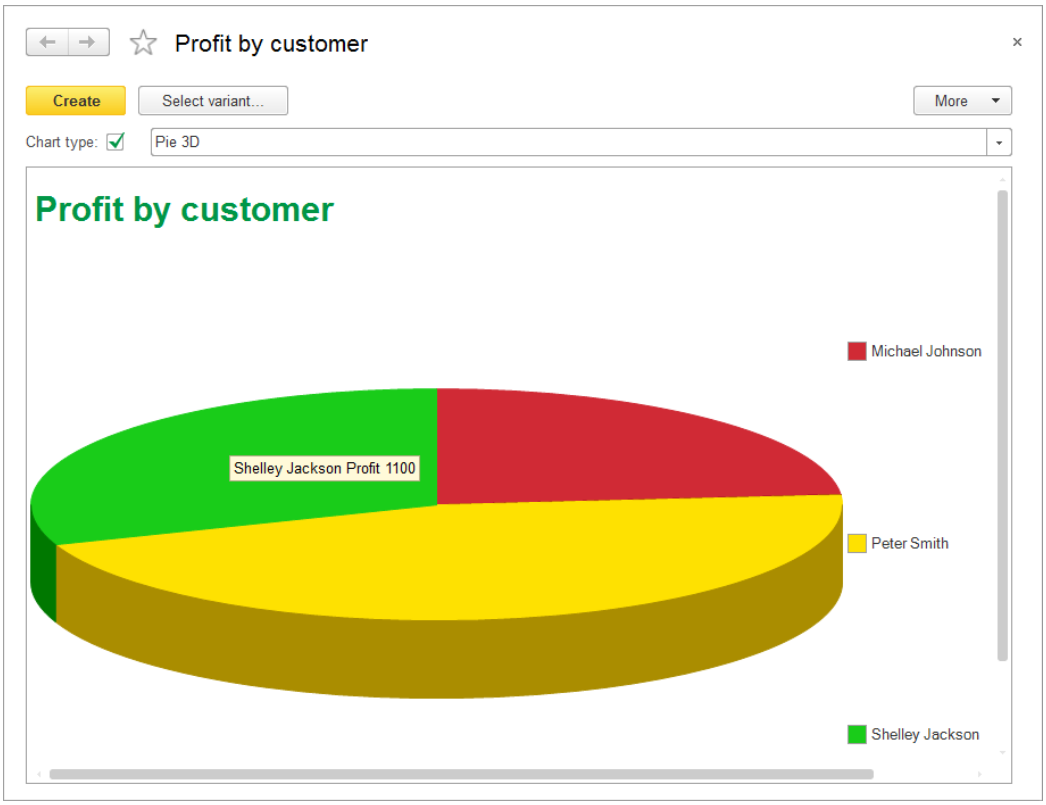


Fig. 13.106. 3D pie chart

4. In the report form, change **Chart type** to **Column 3D** and create the report.

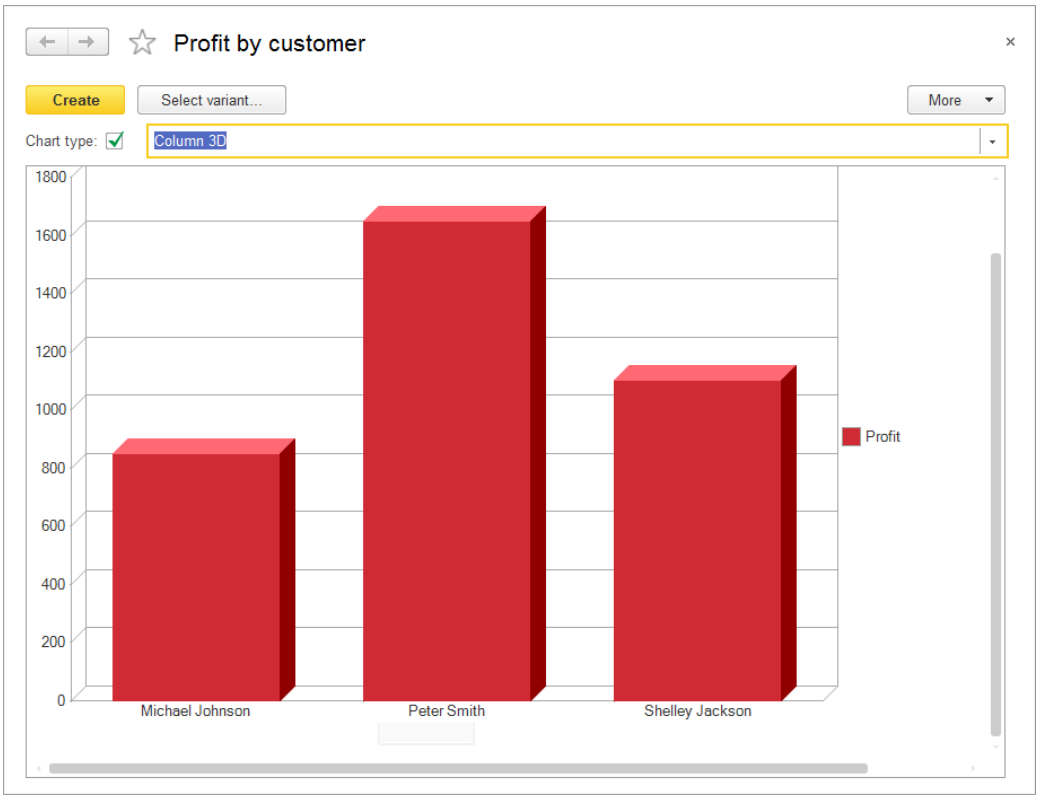


Fig. 13.107. 3D column report

So we have shown how you can use various chart types for viewing report data.

Displaying data as a spreadsheet

Let us create a general-purpose report. Using this report as an example, we will show how to output data to a table (fig. 13.108).

Material or service	Alexander D. Johnson	John A. Walkman	Michelle V. Turner	Total
	Revenue Turnover	Revenue Turnover	Revenue Turnover	Revenue Turnover
Materials	900.00	150.00	744.00	1,794.00
Miscellaneous		150.00	330.00	480.00
Power supply cable (material)			30.00	30.00
Tubing, rubber (material)		150.00	300.00	450.00
Radio parts	900.00		414.00	1,314.00
Line transformer, LG (material)			400.00	400.00
Line transformer, Samsung (material)	900.00			900.00
Transistor 2N2369, Philips (material)			14.00	14.00
Services	800.00	800.00	1,400.00	3,000.00
TV sets	800.00		600.00	1,400.00
Domestic TV set repair (service)			600.00	600.00
Imported TV set repair (service)	800.00			800.00
Washing machines		800.00	800.00	1,600.00
Electrical hookup (service)			800.00	800.00
Water hookup (service)		800.00		800.00
Total	1,700.00	950.00	2,144.00	4,794.00

Fig. 13.108. Resulting report

We will show how to make a report as flexible as possible, so that users can edit the report structure and appearance in 1C:Enterprise mode without using the full report settings (the **Select variant** command on the **More** menu). For example, users can change the order of spreadsheet columns and rows or change the data displayed in spreadsheet cells.

In Designer mode

Let us begin the report creation.

1. Create a report configuration object named *GenericReport*.
2. Run the Data composition schema wizard.
3. Add **Data set – query** and run the Query builder.

Query for a data set

Let us specify the query data set.

1. Select the **Sales.Turnovers** virtual table as the data source.
2. Select all fields from this table (fig. 13.109).

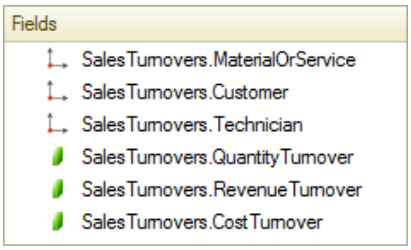


Fig. 13.109. Selected fields

3. Click **OK**.

Analyzing query text

Let us check the generated query text (listing 13.15).

Listing 13.15. Query text

```
SELECT
    SalesTurnovers.MaterialOrService,
    SalesTurnovers.Customer,
    SalesTurnovers.Technician,
    SalesTurnovers.QuantityTurnover,
    SalesTurnovers.QuantityRevenue,
    SalesTurnovers.CostTurnover
FROM
    AccumulationRegister.Sales.Turnovers AS SalesTurnovers
```

Resources

Let us add the resources.

- Click the **Resources** tab and click >>.

  This adds all available resources.

Settings

Let us add a table to the report structure.

1. On the **Settings** tab, in the command bar, click the **Add** button and then click **New table** (fig. 13.110).

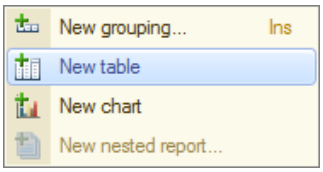


Fig. 13.110. Adding a table to report structure

Let us skip specifying spreadsheet rows and columns and the list of selected fields because our goal is to provide users with full freedom in their actions.

2. In the report item structure, click the **Table** item and then click the **Custom settings item properties** button on the command bar.

   This opens the window where you can edit the assortment of user settings for the spreadsheet.

3. Select the **Selected Fields**, **String groupings**, and **Column groupings** check boxes, and in the **Edit mode** column, leave their default **Quick access** values (fig. 13.111).

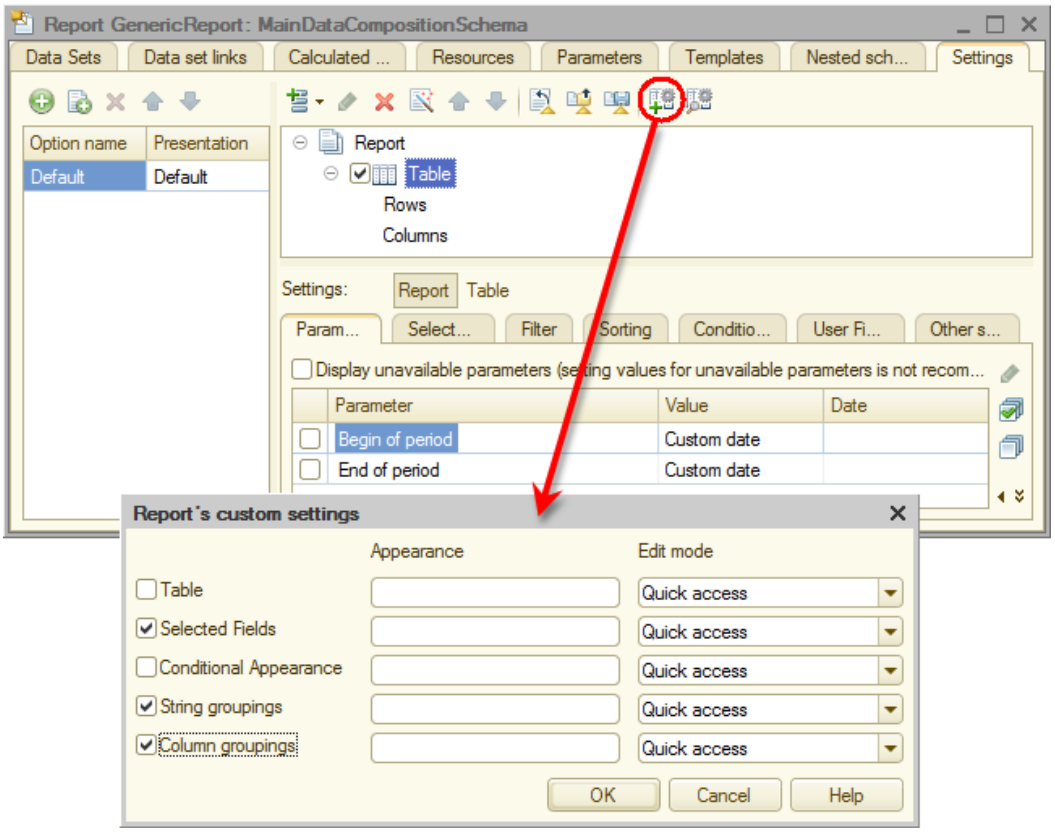


Fig. 13.111. Editing user settings

So you gave users the option to define the assortment of selected fields, as well as spreadsheet row and column groupings, directly in the report form prior to report generation.

Finally, let us specify the subsystems where the report is available.

4. Close the data composition schema wizard and then, in the editor of the **GenericReport** configuration object, click the **Subsystems** tab.
5. Select the **Services** check box.

In 1C:Enterprise mode

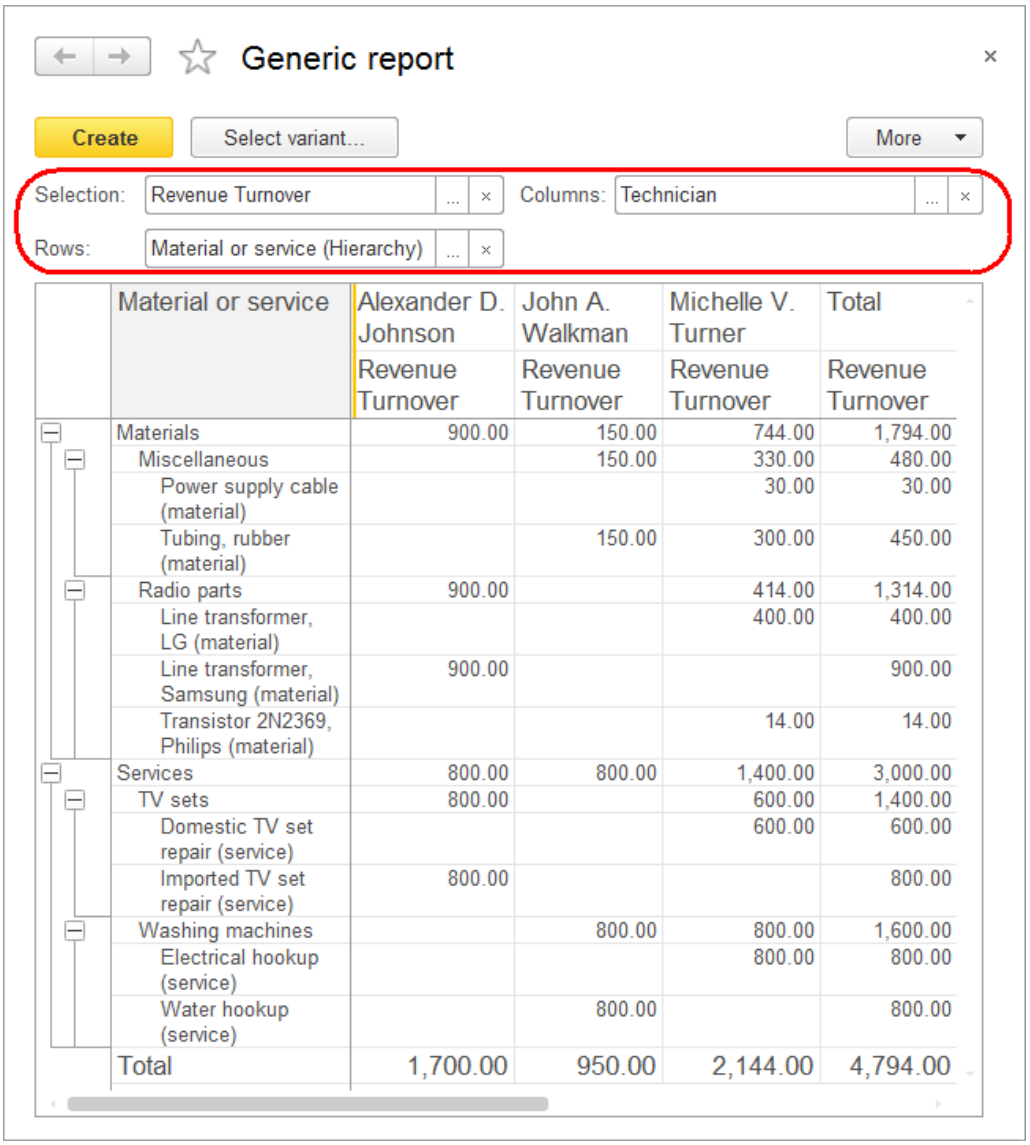
Let us check the report.

1. Run 1C:Enterprise in the debug mode.
2. In the **Services** section, on the **Report** menu, click **Generic report**.
3. Click **Create**.

   You can see that the resulting report is empty because row groupings, column groupings, and the list of selected fields are empty. Let us specify these quick user settings.

4. In the **Selection** field, click the **Select** button and select **Revenue Turnover**.
5. In the **Rows** field, click the **Select** button and add a grouping by the **Material or service** field with the **Hierarchy** type.
6. In the **Columns** field, click the **Select** button and add a grouping by the **Technician** field.
7. Click **Create**.

   The report should look as shown in fig. 13.112.



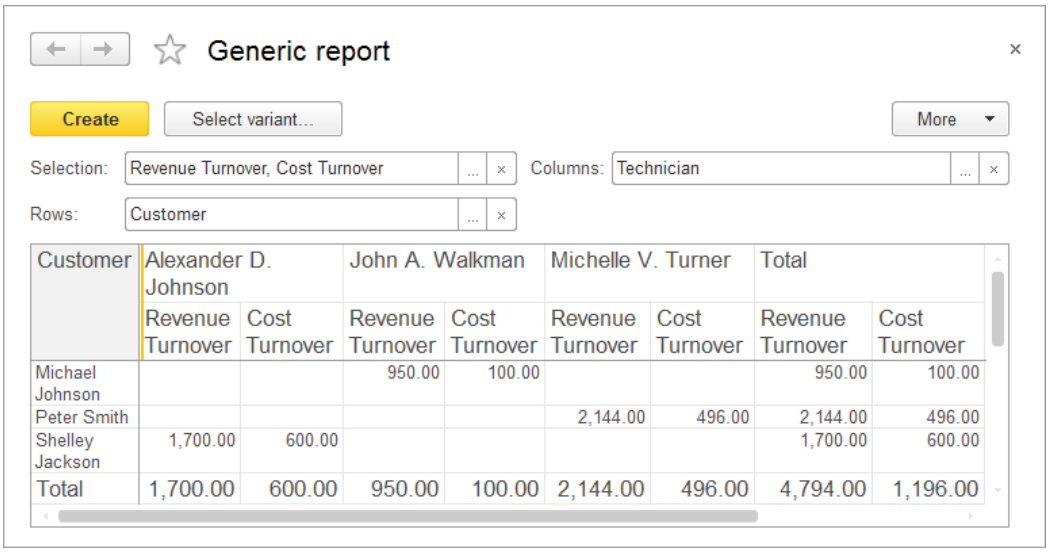
Material or service	Alexander D. Johnson	John A. Walkman	Michelle V. Turner	Total
	Revenue Turnover	Revenue Turnover	Revenue Turnover	Revenue Turnover
Materials	900.00	150.00	744.00	1,794.00
Miscellaneous		150.00	330.00	480.00
Power supply cable (material)			30.00	30.00
Tubing, rubber (material)		150.00	300.00	450.00
Radio parts	900.00		414.00	1,314.00
Line transformer, LG (material)			400.00	400.00
Line transformer, Samsung (material)	900.00			900.00
Transistor 2N2369, Philips (material)			14.00	14.00
Services	800.00	800.00	1,400.00	3,000.00
TV sets	800.00		600.00	1,400.00
Domestic TV set repair (service)			600.00	600.00
Imported TV set repair (service)	800.00			800.00
Washing machines		800.00	800.00	1,600.00
Electrical hookup (service)			800.00	800.00
Water hookup (service)		800.00		800.00
Total	1,700.00	950.00	2,144.00	4,794.00

Fig. 13.112. Resulting report

Note that scrolling the report horizontally or vertically does not move the report header and its leftmost column. The platform automatically freezes the top and left report areas for usability purposes. Alternatively, you can specify the rows and columns to freeze using the FixLeft and FixTop output parameters.

8. Add **Cost Turnover** to the list of selected fields.
9. Change the row grouping from **Material or service** to **Customer** and create the report.

   The report should look as shown in fig. 13.113.

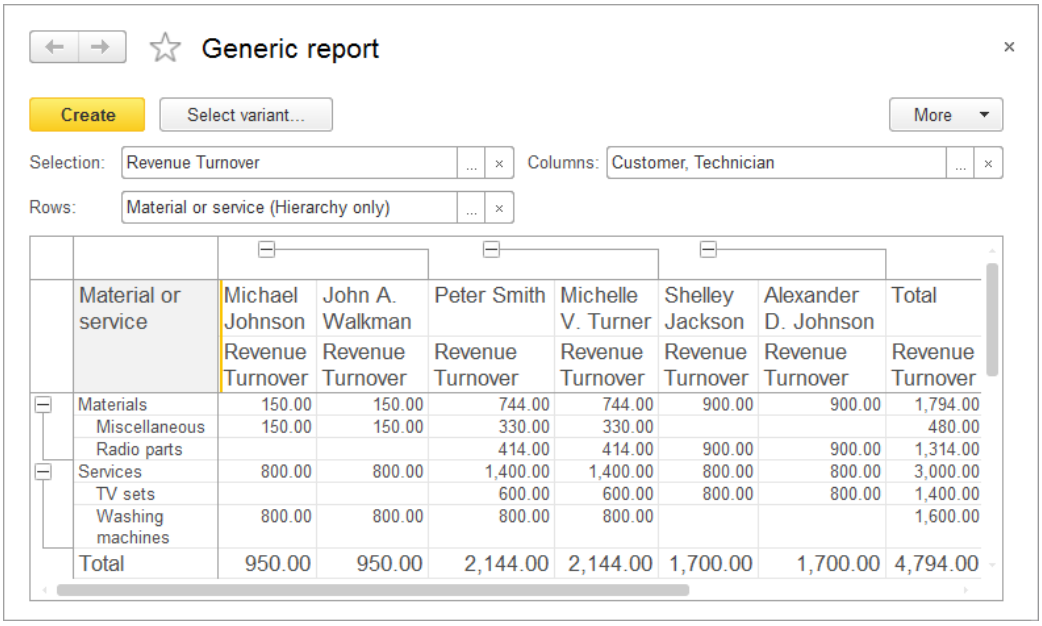


Customer	Alexander D. Johnson		John A. Walkman		Michelle V. Turner		Total	
	Revenue Turnover	Cost Turnover	Revenue Turnover	Cost Turnover	Revenue Turnover	Cost Turnover	Revenue Turnover	Cost Turnover
Michael Johnson			950.00	100.00			950.00	100.00
Peter Smith					2,144.00	496.00	2,144.00	496.00
Shelley Jackson	1,700.00	600.00					1,700.00	600.00
Total	1,700.00	600.00	950.00	100.00	2,144.00	496.00	4,794.00	1,196.00

Fig. 13.113. Resulting report

10. Exclude **Cost Turnover** from the list of selected fields.
11. Change the row grouping to **Material or service** with the **Hierarchy only** type.
12. Add a column grouping by the **Customer** field, move it to the beginning of the grouping list, and click **Create**.

    The report should look as shown in fig. 13.114.



Material or service	Michael Johnson	John A. Walkman	Peter Smith	Michelle V. Turner	Shelley Jackson	Alexander D. Johnson	Total
	Revenue Turnover	Revenue Turnover	Revenue Turnover	Revenue Turnover	Revenue Turnover	Revenue Turnover	Revenue Turnover
Materials	150.00	150.00	744.00	744.00	900.00	900.00	1,794.00
Miscellaneous	150.00	150.00	330.00	330.00			480.00
Radio parts			414.00	414.00	900.00	900.00	1,314.00
Services	800.00	800.00	1,400.00	1,400.00	800.00	800.00	3,000.00
TV sets			600.00	600.00	800.00	800.00	1,400.00
Washing machines	800.00	800.00	800.00	800.00			1,600.00
Total	950.00	950.00	2,144.00	2,144.00	1,700.00	1,700.00	4,794.00

Fig. 13.114. Resulting report

So you have created a report that offers users the option to create a variety of custom reports based on the Sales register.

Theory. Virtual query tables

As you learned in this lesson, when you create a query, the platform provides a number of virtual tables as data sources. "Virtual" is a good name for them because a virtual table is a result of a query that the platform generates during the script execution.

Actually, developers can implement retrieving data available in virtual tables without using any virtual tables. However, the algorithm for retrieving that data will not be optimal for the following two reasons.

First, all of the virtual tables are parameterized. In other words, developers can define some parameters that the platform uses to generate a query that creates a virtual table.

When a developer specifies virtual table parameters, the parameters are not always simply plugged into the query text. Depending on the virtual table parameters, the platform might generate DIFFERENT queries for getting a single virtual table, which are optimized based on the assortment of parameters.

Second, a developer is not always able to access all the data available to the platform. For example, they basically have access to all information register data that the platform uses to generate a virtual table query, but accumulation register virtual tables are quite different. Here the platform generates queries dynamically based not only on the parameters, but also on the period of the totals calculated in the register. The resulting query depends on the calculated totals, which are simply not available at the query development stage.

Of course a developer can implement a search through all the accumulation register records and get the data that is normally provided in a virtual table. But obviously this query would be less efficient and would require more of the developer’s time and effort.

Quiz

- What are the uses of the Query 1C:Enterprise script object?
- What are the uses of the data composition system?
- What are the uses of the data composition schema?
- What are the uses of data composition settings?
- What is the difference between actual tables and virtual tables?
- What sections does a query text have? Which sections are mandatory?
- What are basic syntax statements of the query language?
- What data sources are used for a query?
- What is an alias in the query language?
- What are query parameters?
- What are virtual table parameters?

- What is the meaning of left join?
- How does one use the Query builder?
- How does one select data belonging to a specific period for a report?
- How does one order data in a report?
- How does one use data from multiple tables in a report?
- How does one use groupings in the report structure?
- How does one get the latest values from an information register?
- How does one display hierarchical data in a report?
- How does one manage the display of grouping and overall totals?
- How does one create a report containing a chart?
- How does one use parameters in the data composition system?
- What are resources in the data composition system?
- What are calculated fields in the data composition system?
- How does one add all the days of a grouping period to report data?
- How does one create user settings for a report?
- What is the difference between quick settings and regular user settings?
- How does one define the assortment of user settings for a report?
- How does one display data as a spreadsheet?
- How does one make a generic report?

Lesson 14 (3:20). Optimization of posting the Services document

After studying the previous lesson, you learned a lot about the query language. You are ready to proceed to one of the most important lessons in this book, which describes the optimization of the Services document. In particular, you will completely refactor its Posting event handler.

You might ask why you have to do it. There are three reasons.

First, the Posting event handler accesses the MaterialServiceType attribute of the MaterialsAndServices catalog using dot (.), which can slow down the procedure execution for documents with large tabular sections.

Second, the management at Jack of All Trades finally decided to wrap up the practice of entering the costs of consumed materials manually and now wants you to implement automatic calculation of the average cost of materials.

Third, when a Services document is posted, you need to monitor the item balances in the warehouses. If the number of items is not sufficient, a warning should be displayed and the document should not be posted.

Therefore, the changes you will make to the Services document will accomplish three purposes:

- Improve the procedure performance
- Calculate the cost of consumed materials automatically when a document is posted
- Split the document posting algorithm into real-time posting and regular posting and implement the check of balances for real-time posting.

Before you proceed to these tasks, let us briefly discuss the specifics of storing and using reference data in 1C:Enterprise.

Theory. Referential data usage specifics

This section describes the referential data usage specifics. Using queries to access this data can significantly improve the document posting performance.

We will use the "referential data" term to describe data that is stored in a database and accessed by means of 1C:Enterprise script objects of the Ref type: CatalogRef.<name>, DocumentRef.<name>, and so on. To simplify the explanation, we will use an example of getting a reference to a material or service type when posting a Services document.

Not all the data stored in the database is referential. This is due to the fact that within the 1C:Enterprise data structure there is a division between data that represents object entities (catalogs, charts of accounts, documents, and so on), and data that represents nonobject entities (information registers, accumulation registers, and so on).

From the platform perspective, a certain set of object-based data is defined not only by the field values but by the very fact of its existence as well. In other words, once you delete a set of object-based data from the database, you will not be able to get the system to the condition it had prior to the deletion. Even if you recreate the same set of object-based data with exactly the same field values, from the platform perspective it will be a DIFFERENT set of object-based data.

Such set of object-based data, which is uniquely recognized by the platform, is referred to as a database object.

To ensure that the platform can distinguish one database object from another, each database object (a set of object-based data) has an internal identifier. Different database objects always have different identifiers. An identifier is stored together with the rest of the object data in a field named Ref.

Nonobject data is stored as records, and from the platform perspective it is fully defined by the values of the record fields. If you delete a certain record and then write a new one with exactly the same values in all the fields, the database will end up in the same state that it had before the deletion.

Therefore, since you can unambiguously point to each database object, you can store that pointer in fields within other database tables, select it in text boxes, use it as a query parameter when searching by reference, and so on.

All these scenarios use 1C:Enterprise script objects of the Ref type. In fact, this object only stores an internal identifier located in the Ref field.

If we use the Services document as an example, the field that stores the MaterialOrService attribute of the tabular section actually stores an internal identifier that points to an item in the MaterialsAndServices catalog (fig. 14.1).

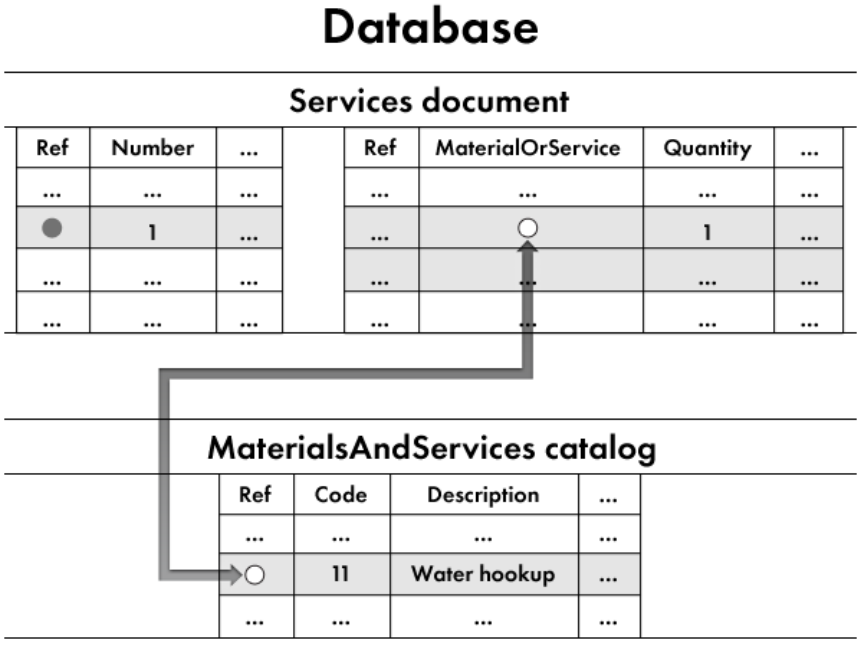


Fig. 14.1. Reference to a MaterialsAndServices catalog item

When the Posting event handler of the Services document assigns the value of the MaterialOrService tabular section attribute to some variable, it

actually deals with 1C:Enterprise script object named DocumentObject.Services.

This object contains the values of all document attributes and all document tabular section attributes.

The script that accesses the object in listing 14.1 simply reads the data stored in that 1C:Enterprise script object from RAM (fig. 14.2).

Listing 14.1. Reference to an object attribute

```
Record.Material = CurRowMaterialsAndServices.MaterialOrService;
```

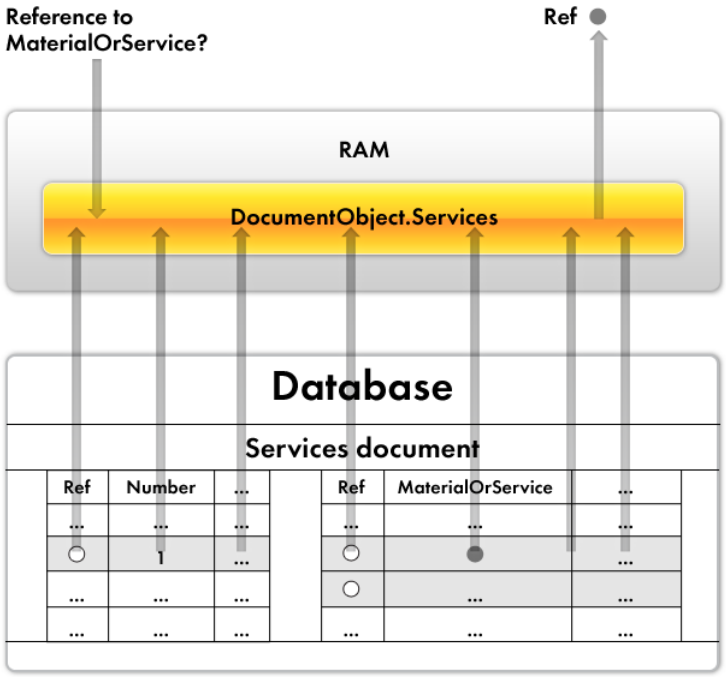


Fig. 14.2. Reading data from RAM

However, when you refer to a material or service type as an attribute of the catalog item that is referenced to in the document tabular section (listing 14.2), the following happens (fig. 14.3).

Listing 14.2. Reference to a reference attribute

```
If CurRowMaterialsAndServices.MaterialOrService.MaterialServiceType =
    Enums.MaterialServiceTypes.Material Then
```

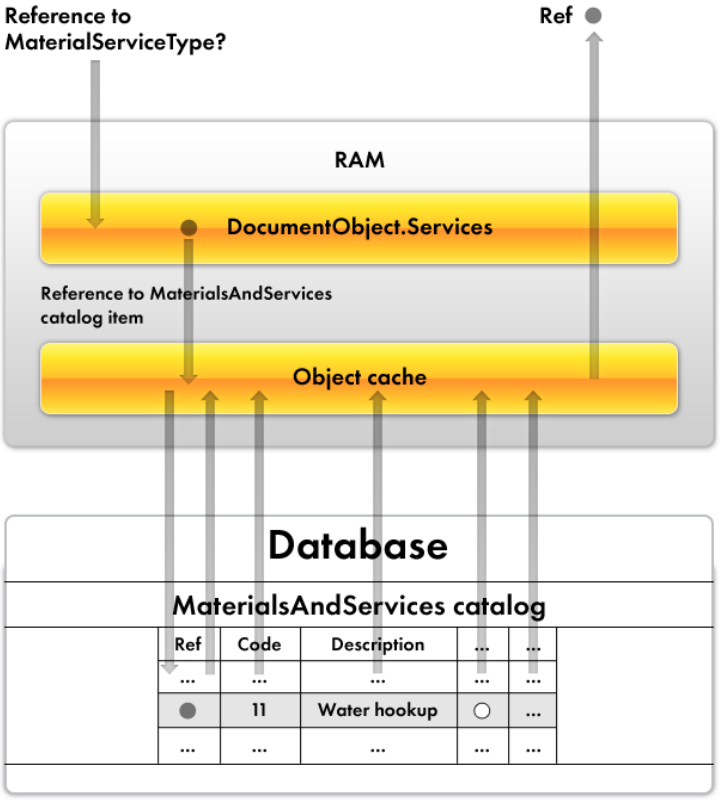


Fig. 14.3. Using object cache

Since the DocumentObject.Services object is only a reference to an item of the MaterialsAndServices catalog and there is no additional information about this item, the platform uses the reference to search the object cache for the data of the object that corresponds to that reference.

If the object cache does not contain the required data, the platform accesses the database to read all the data of the referenced object.

Once all the data stored in the attributes of the required catalog item and in the attributes of its tabular sections is read into the object cache, the object cache returns the requested reference stored in the MaterialServiceType attribute of the MaterialsAndServices catalog.

It is obvious that such query to the database takes far longer than simply reading from RAM. When a document is being filled interactively, such lags go unnoticed, compared to the user work speed. However, in a scenario with a large number of calculations (such as posting large documents containing thousands of rows), the time difference might be noticeable.

From the facts listed above you can conclude that if the document posting algorithm uses only the data available in the document attributes and tabular sections, using the document register records wizard is totally sufficient (as was the case with the GoodsReceipt document).

However, if a posting algorithm includes the analysis of additional object attributes that are referenced in the document, and also requires the use of calculated register totals, you need to use queries to select data from the database faster.

The same is true for execution of any parts of the program where performance is critical. Queries provide better options for reading the

infobase, they are capable of selecting only the required data in a single operation. That is why standard applied solutions almost never use 1C:Enterprise objects named CatalogSelection.<name>. They use database queries instead.

Improving posting performance

Your first task in this lesson is eliminating the CurRowMaterialsAndServices.MaterialOrService.MaterialServiceType statement, which impacts the performance.

In Designer mode

Open the module of the Services document.

The current posting procedure is shown in listing 14.3.

Listing 14.3. Posting() procedure

```
RegisterRecords.BalanceOfMaterials.Write = True;
RegisterRecords.CostOfMaterials.Write = True;
RegisterRecords.Sales.Write = True;

For Each CurRowMaterialsAndServices In MaterialsAndServices Do
    If CurRowMaterialsAndServices.MaterialOrService.MaterialServiceType =
        Enums.MaterialServiceTypes.Material Then

        // register BalanceOfMaterials Expense
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = CurRowMaterialsAndServices.MaterialOrService;
        Record.Warehouse = Warehouse;
        Record.Quantity = CurRowMaterialsAndServices.Quantity;

        // register CostOfMaterials Expense
        Record = RegisterRecords.CostOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = CurRowMaterialsAndServices.MaterialOrService;
        Record.Cost = CurRowMaterialsAndServices.Quantity
            * CurRowMaterialsAndServices.Cost;
    EndIf;

    // register Sales
    Record = RegisterRecords.Sales.Add();
    Record.Period = Date;
    Record.MaterialOrService = CurRowMaterialsAndServices.MaterialOrService;
```

```
    Record.Customer = Customer;
    Record.Technician = Technician;
    Record.Quantity = CurRowMaterialsAndServices.Quantity;
    Record.Revenue = CurRowMaterialsAndServices.Total;
    Record.Cost = CurRowMaterialsAndServices.Quantity *
        CurRowMaterialsAndServices.Cost;

EndDo;
```

All the data required for posting the document is taken from the document itself. The platform accesses the database and reads the data of the entire MaterialsAndServices object only to determine whether an item is a material or a service (listing 14.4).

Listing 14.4. Accessing the MaterialsAndServices object

```
If CurRowMaterialsAndServices.MaterialOrService.MaterialServiceType
```

Leaping ahead, we will mention that the correct document posting requires some other data that is not stored in the document itself.

So let us implement this as follows: all the material or service-related data contained in the document tabular section is retrieved using a database query. And the data related to the document itself (such as document date or warehouse) is still obtained from the document. This approach focuses on reading only the required data and therefore reaching the maximum document posting performance.

The following data is retrieved using a query:

- MaterialOrService
- Quantity
- Total
- Cost

The following data is retrieved from the document:

- Date
- Customer
- Technician
- Warehouse

Let us proceed to creating the query.

1. Right-click in the module window, in the line before the loop that iterates through the document tabular section, and then click **Query Wizard with result processing** (fig. 14.4).

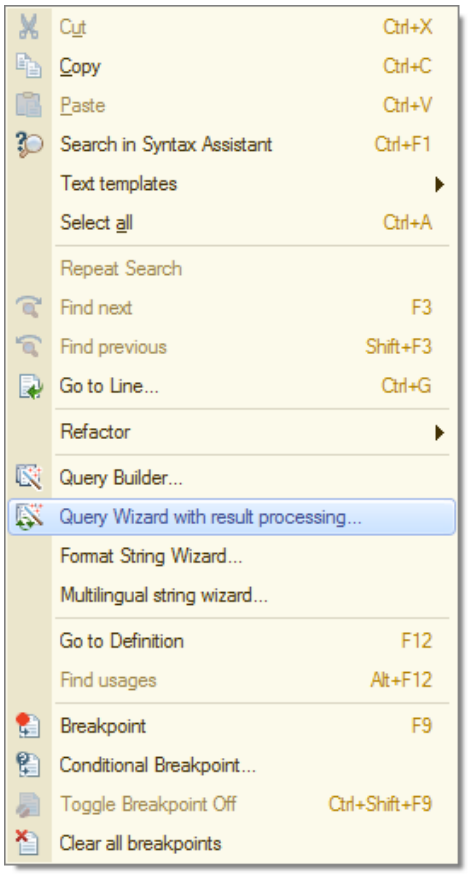


Fig. 14.4. Opening the query wizard

2. Confirm that you want to create a new query.
3. In the query wizard, click the **Tables and fields** tab and select the **ServicesMaterialsAndServices** table.

   It is the tabular section of the Services document.
4. Select the following fields from the table (fig. 14.5):
   - **MaterialOrService**
   - **MaterialOrService.MaterialServiceType**
   - **Quantity**
   - **Total**
   - **Cost**

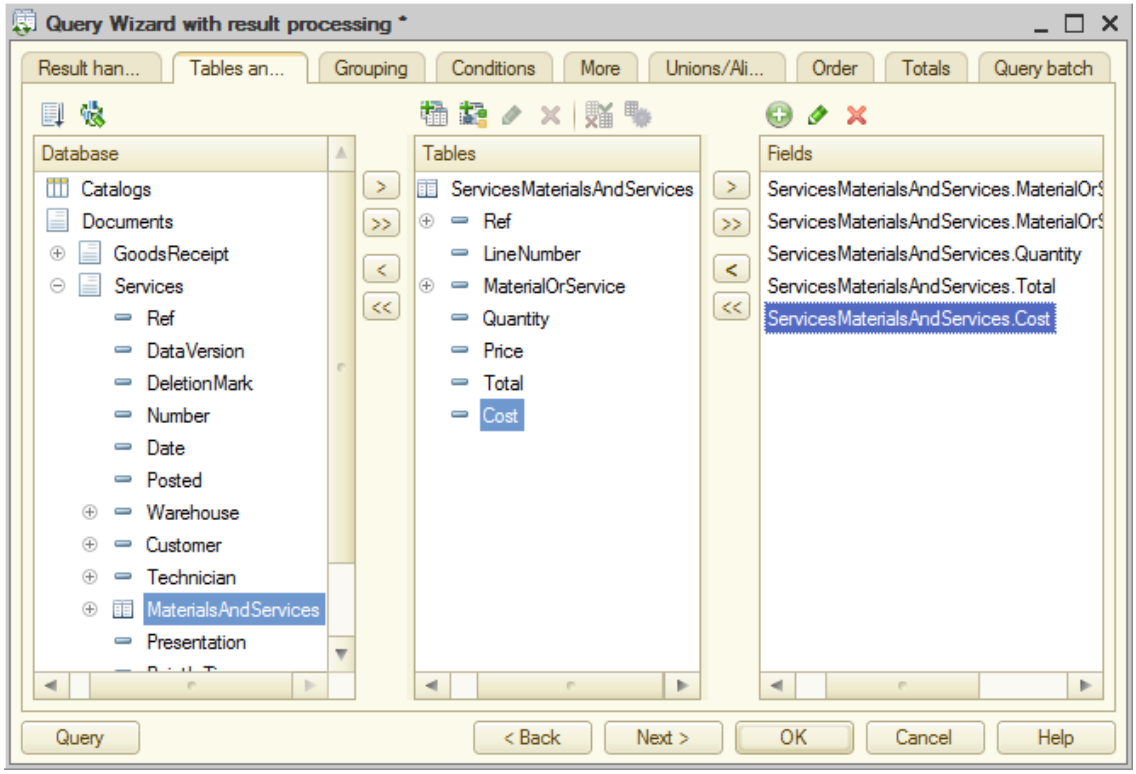


Fig. 14.5. Selected fields

But you do not need all the records of this table. You only need the records related to your document, so let us set a filter condition for selecting only the rows of the document being posted from the document table.

5. Click the **Conditions** tab, drag the **Ref** field to the list of filter criteria, and set the condition as shown in listing 14.5.

Listing 14.5. Filter condition for the document table

```
ServicesMaterialsAndServices.Ref = &Ref
```

A reference to this document is passed to the Ref query parameter (fig. 14.6).

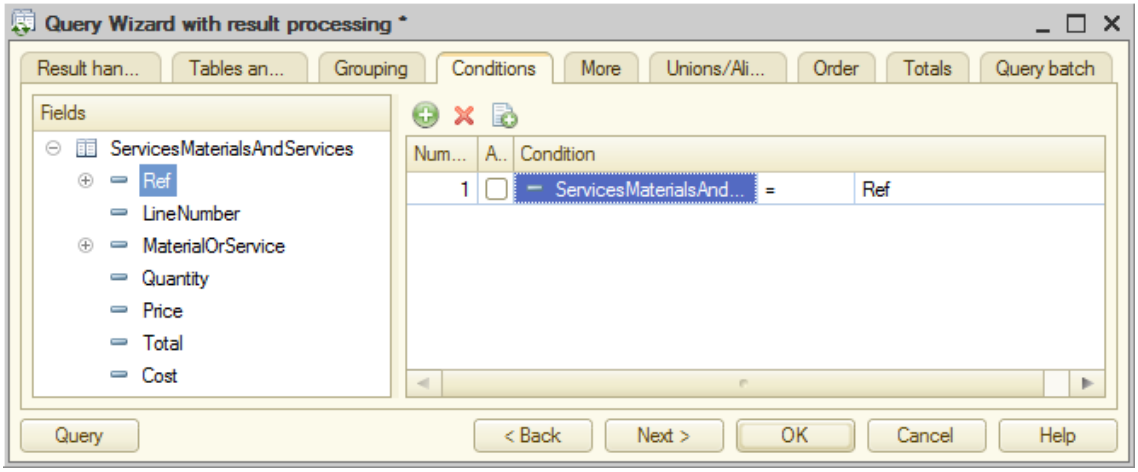


Fig. 14.6. Filter condition for the document table

Also note that a single material or service might have multiple occurrences in a document tabular section. Let us reflect this in the query.

6. On the **Grouping** tab, group the records by the **MaterialOrService** and **MaterialOrService.MaterialServiceType** fields.
7. Specify that if the document tabular section contains multiple rows with a single product or service, totals are calculated by the **Quantity** and **Total** fields.

   This ensures that each result row contains a unique material or service, and if a material or service is encountered in multiple document rows, the corresponding result row stores its total by the Quantity and Total fields.
8. Move the **Cost** field to the list of totaled fields.

   This field will be used to calculate the Max function.

   We imply that the cost is the same for all document rows that contain the same material or service, so the Max function is only needed to obtain one of the available cost values (fig. 14.7).

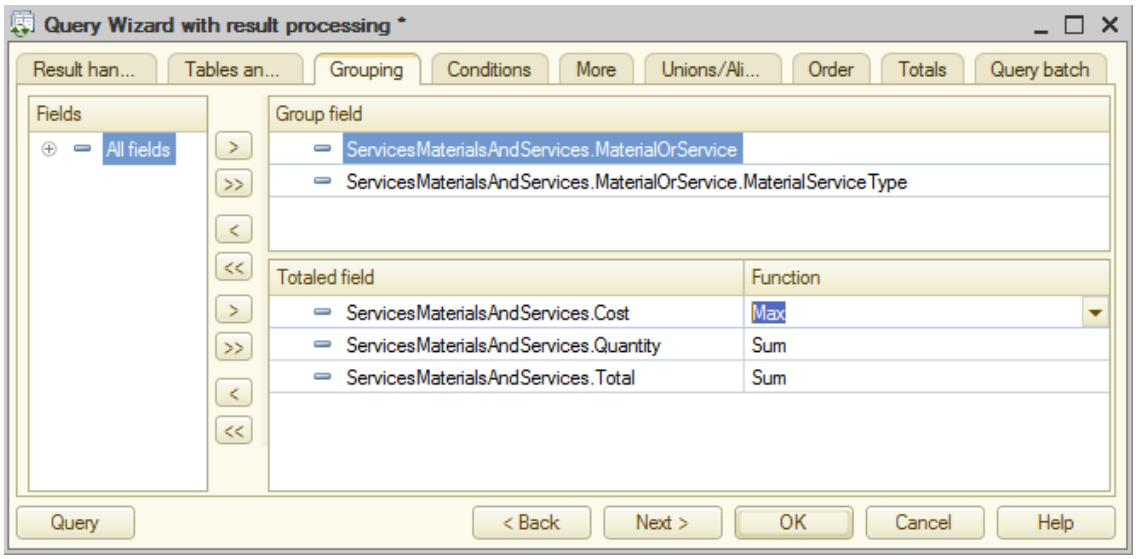


Fig. 14.7. Grouping document table rows

9. On the **Unions/Aliases** tab, specify the aliases for the **Quantity** and **Total** fields as *QuantityInDocument* and *TotalInDocument* respectively.
10. For the **MaterialOrServiceMaterialServiceType** field, specify *MaterialServiceType* as an alias (fig. 14.8).

    This alias makes the query easier to read.

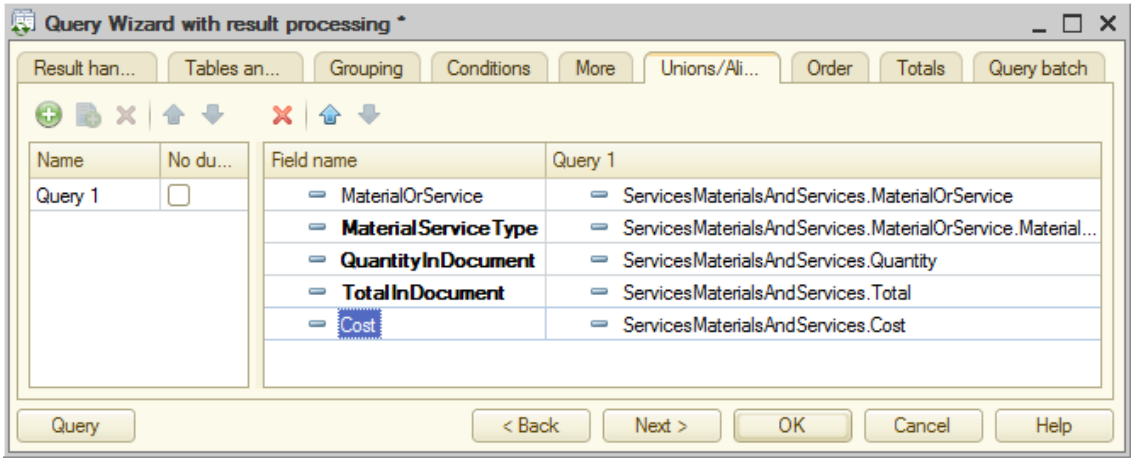


Fig. 14.8. Field aliases

11. Click **OK**.

    The query text generated by the wizard is shown in listing 14.6.

Listing 14.6. Query text

```
//{{QUERY_BUILDER_WITH_RESULT_PROCESSING
// This fragment was built by the wizard.
// Warning! All manually made changes will be lost next time you use
the wizard.

Query = New Query;
Query.Text =
 "SELECT
 | ServicesMaterialsAndServices.MaterialOrService,
 | ServicesMaterialsAndServices.MaterialOrService.MaterialServiceType
 |     AS MaterialServiceType,
 | SUM(ServicesMaterialsAndServices.Quantity)
 |     AS QuantityInDocument,
 | SUM(ServicesMaterialsAndServices.Total) AS TotalInDocument,
 | MAX(ServicesMaterialsAndServices.Cost) AS Cost
 |FROM
 | Document.Services.MaterialsAndServices
 |     AS ServicesMaterialsAndServices
 |WHERE
 | ServicesMaterialsAndServices.Ref = &Ref
 |
 |GROUP BY
 | ServicesMaterialsAndServices.MaterialOrService,
 | ServicesMaterialsAndServices.MaterialOrService.MaterialServiceType
 |";

Query.SetParameter("Ref", Ref);
```

```
QueryResult = Query.Execute();

SelectionDetailRecords = QueryResult.Select();

While SelectionDetailRecords.Next() Do
    // Insert selection processing SelectionDetailRecords
EndDo;

//}}QUERY_BUILDER_WITH_RESULT_PROCESSING
```

You can delete the comments generated by the query wizard at the beginning and at the end of the script fragment.

Since you used the query wizard with result processing to generate the query, the wizard has also generated the script that executes the query and iterates through the query records. Let us analyze the generated script. As you already know, the query processing involves the use of the 1C:Enterprise script object named Query. First, a new Query object is created and stored to the Query variable. Then the query text is stored to the Text property of the Query object (Query.Text = …).

Then a reference to the document whose module is being executed is stored to the &Ref query parameter (listing 14.7).

Listing 14.7. Specifying the query parameter

```
Query.SetParameter("Ref", Ref);
```

Then the query is executed (Query.Execute()), the QueryResult object is obtained, and its method Select() is executed. This method generates a selection of records from the query result.

This is how the QueryResultSelection object is generated. This object is stored to the SelectionDetailRecords variable.

Then the Next() method of this object (SelectionDetailRecords.Next()) is used to iterate through the selection of query records within a loop.

Each execution of the query selection method SelectionDetailRecords.Next() moves the pointer to the next selection record until the end of the selection is reached.

You can get a value of a field that belongs to the selection from the query result using the SelectionDetailRecords variable, which contains the current row of the query selection, followed by a dot. Example: SelectionDetailRecords.MaterialOrService.

Now you only need to move the lines that generate the register records into the loop that iterates through the query result (listing 14.8).

Listing 14.8. Loop that iterates through the query records

```
While SelectionDetailRecords.Next() Do
    // Insert selection processing SelectionDetailRecords
EndDo;
```

1. Delete the comment "// Insert selection processing SelectionDetailRecords" and move the condition that checks whether

an item is a material or a service and the script that generates records for the BalanceOfMaterials and CostOfMaterials registers into the loop (listing 14.9).

Listing 14.9. Creating register records

```
While SelectionDetailRecords.Next() Do
   If CurRowMaterialsAndServices.MaterialOrService.MaterialServiceType
      = Enums.MaterialServiceTypes.Material Then


      // register BalanceOfMaterials Expense
      Record = RegisterRecords.BalanceOfMaterials.Add();
      Record.RecordType = AccumulationRecordType.Expense;
      Record.Period = Date;
      Record.Material =
         CurRowMaterialsAndServices.MaterialOrService;
      Record.Warehouse = Warehouse;
      Record.Quantity = CurRowMaterialsAndServices.Quantity;


      // register CostOfMaterials Expense
      Record = RegisterRecords.CostOfMaterials.Add();
      Record.RecordType = AccumulationRecordType.Expense;
      Record.Period = Date;
      Record.Material = CurRowMaterialsAndServices.MaterialOrService;
      Record.Cost = CurRowMaterialsAndServices.Quantity
         * CurRowMaterialsAndServices.Cost;
   EndIf;


EndDo;
```

2. In the condition replace CurRowMaterialsAndServices.MaterialOrService with SelectionDetailRecords because the material or service type is now obtained from the query.
3. In the register records also replace CurRowMaterialsAndServices with SelectionDetailRecords (listing 14.10).

Listing 14.10. Creating register records

```
While SelectionDetailRecords.Next() Do


    If SelectionDetailRecords.MaterialServiceType =
        Enums.MaterialServiceTypes.Material Then


        // register BalanceOfMaterials Expense
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Warehouse = Warehouse;
```

```
        Record.Quantity = SelectionDetailRecords.Quantity;


        // register CostOfMaterials Expense
        Record = RegisterRecords.CostOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Cost = SelectionDetailRecords.Quantity
            * SelectionDetailRecords.Cost;
    EndIf;


EndDo;
```

4. Replace Quantity with QuantityInDocument because you use that alias in the query (listing 14.11).

Listing 14.11. Creating register records

```
While SelectionDetailRecords.Next() Do


    If SelectionDetailRecords.MaterialServiceType =
        Enums.MaterialServiceTypes.Material Then


        // register BalanceOfMaterials Expense
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Warehouse = Warehouse;
        Record.Quantity = SelectionDetailRecords.QuantityInDocument;


        // register CostOfMaterials Expense
        Record = RegisterRecords.CostOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Cost = SelectionDetailRecords.QuantityInDocument
            * SelectionDetailRecords.Cost;


    EndIf;


EndDo;
```

5. Move the script that creates Sales register records into the loop (listing 14.12).

Listing 14.12. Creating register records

```
While SelectionDetailRecords.Next() Do
```

```
    If SelectionDetailRecords.MaterialServiceType =
        Enums.MaterialServiceTypes.Material Then

        // register BalanceOfMaterials Expense
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Warehouse = Warehouse;
        Record.Quantity = SelectionDetailRecords.QuantityInDocument;

        // register CostOfMaterials Expense
        Record = RegisterRecords.CostOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Cost = SelectionDetailRecords.QuantityInDocument
            * SelectionDetailRecords.Cost;
    EndIf;

    // register Sales
    Record = RegisterRecords.Sales.Add();
    Record.Period = Date;
    Record.MaterialOrService =
        CurRowMaterialsAndServices.MaterialOrService;
    Record.Customer = Customer;
    Record.Technician = Technician;
    Record.Quantity = CurRowMaterialsAndServices.Quantity;
    Record.Revenue = CurRowMaterialsAndServices.Total;
    Record.Cost = CurRowMaterialsAndServices.Quantity
        * CurRowMaterialsAndServices.Cost;

EndDo;
```

This part requires the same replacements.

6. Replace CurRowMaterialsAndServices with SelectionDetailRecords (listing 14.13).

Listing 14.13. Creating register records

```
While SelectionDetailRecords.Next() Do

    If SelectionDetailRecords.MaterialServiceType =
        Enums.MaterialServiceTypes.Material Then

        // register BalanceOfMaterials Expense
        Record = RegisterRecords.BalanceOfMaterials.Add();
```

```
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Warehouse = Warehouse;
        Record.Quantity = SelectionDetailRecords.QuantityInDocument;

        // register CostOfMaterials Expense
        Record = RegisterRecords.CostOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Cost = SelectionDetailRecords.QuantityInDocument
            * SelectionDetailRecords.Cost;
    EndIf;

    // register Sales
    Record = RegisterRecords.Sales.Add();
    Record.Period = Date;
    Record.MaterialOrService =
        SelectionDetailRecords.MaterialOrService;
    Record.Customer = Customer;
    Record.Technician = Technician;
    Record.Quantity = SelectionDetailRecords.Quantity;
    Record.Revenue = SelectionDetailRecords.Total;
    Record.Cost = SelectionDetailRecords.Quantity
        * SelectionDetailRecords.Cost;

EndDo;
```

7. Replace the Total and Quantity query fields with TotalInDocument and QuantityInDocument respectively (listing 14.14).

Listing 14.14. Creating register records

```
While SelectionDetailRecords.Next() Do

    If SelectionDetailRecords.MaterialServiceType =
        Enums.MaterialServiceTypes.Material Then

        // register BalanceOfMaterials Expense
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Warehouse = Warehouse;
        Record.Quantity = SelectionDetailRecords.QuantityInDocument;

        // register CostOfMaterials Expense
```

```
        Record = RegisterRecords.CostOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Cost = SelectionDetailRecords.QuantityInDocument
            * SelectionDetailRecords.Cost;
    EndIf;

    // register Sales
    Record = RegisterRecords.Sales.Add();
    Record.Period = Date;
    Record.MaterialOrService =
        SelectionDetailRecords.MaterialOrService;
    Record.Customer = Customer;
    Record.Technician = Technician;
    Record.Quantity = SelectionDetailRecords.QuantityInDocument;
    Record.Revenue = SelectionDetailRecords.TotalInDocument;
    Record.Cost = SelectionDetailRecords.QuantityInDocument
        * SelectionDetailRecords.Cost;

EndDo;
```

8. Remove the remaining part of the loop that iterates through the tabular section (listing 14.15).

Listing 14.15. Lines to remove

```
For Each CurRowMaterialsAndServices In MaterialsAndServices Do

EndDo;
```

The resulting posting procedure should look as shown in listing 14.16.

Listing 14.16. Posting() procedure

```
Procedure Posting(Cancel, Mode)

   RegisterRecords.BalanceOfMaterials.Write = True;
   RegisterRecords.CostOfMaterials.Write = True;
   RegisterRecords.Sales.Write = True;

   Query = New Query;
   Query.Text =
      "SELECT
      | ServicesMaterialsAndServices.MaterialOrService,
      | ServicesMaterialsAndServices.MaterialOrService.MaterialServiceType
      |     AS MaterialServiceType,
      | SUM(ServicesMaterialsAndServices.Quantity) AS QuantityInDocument,
      | SUM(ServicesMaterialsAndServices.Total) AS TotalInDocument,
```

```
    | MAX(ServicesMaterialsAndServices.Cost) AS Cost
    |FROM
    |    Document.Services.MaterialsAndServices
    |        AS ServicesMaterialsAndServices
    |WHERE
    | ServicesMaterialsAndServices.Ref = &Ref
    |
    |GROUP BY
    | ServicesMaterialsAndServices.MaterialOrService,
    | ServicesMaterialsAndServices.MaterialOrService.MaterialServiceType";

Query.SetParameter("Ref", Ref);

QueryResult = Query.Execute();

SelectionDetailRecords = QueryResult.Select();

While SelectionDetailRecords.Next() Do

   If SelectionDetailRecords.MaterialServiceType =
      Enums.MaterialServiceTypes.Material Then

         // register BalanceOfMaterials Expense
         Record = RegisterRecords.BalanceOfMaterials.Add();
         Record.RecordType = AccumulationRecordType.Expense;
         Record.Period = Date;
         Record.Material = SelectionDetailRecords.MaterialOrService;
         Record.Warehouse = Warehouse;
         Record.Quantity = SelectionDetailRecords.QuantityInDocument;

         // register CostOfMaterials Expense
         Record = RegisterRecords.CostOfMaterials.Add();
         Record.RecordType = AccumulationRecordType.Expense;
         Record.Period = Date;
         Record.Material = SelectionDetailRecords.MaterialOrService;
         Record.Cost = SelectionDetailRecords.QuantityInDocument
             * SelectionDetailRecords.Cost;

   EndIf;

   // register Sales
   Record = RegisterRecords.Sales.Add();
   Record.Period = Date;
   Record.MaterialOrService = SelectionDetailRecords.MaterialOrService;
   Record.Customer = Customer;
```

```
        Record.Technician = Technician;
        Record.Quantity = SelectionDetailRecords.QuantityInDocument;
        Record.Revenue = SelectionDetailRecords.TotalInDocument;
        Record.Cost = SelectionDetailRecords.QuantityInDocument
            * SelectionDetailRecords.Cost;

    EndDo;

EndProcedure
```

In 1C:Enterprise mode

Run 1C:Enterprise in the debug mode, repost the Services documents, and ensure that nothing has changed.

You have accomplished the first step: eliminated the reading of all the data of the MaterialsAndServices object and thus streamlined the posting procedure.

Automatic cost calculation

Let us proceed to the second step.

Up to this point you entered the cost of consumed materials to the Services document manually during its creation.

Now let us define the cost of materials using average values: for each material its total cost is divided by the number of materials available in order to get an average cost of one material unit.

To perform this calculation, you need additional data that you do not have for now.

For each material type in the tabular section you need:

- Material cost, which is stored in the CostOfMaterials register
- Total number of material units in all warehouses, which is stored in the BalanceOfMaterials register

So you need to modify your query to get this data from the database.

You need the query to retrieve the following fields for each material in the document (fig. 14.9).

Fig. 14.9. Query field descriptions

The first four fields can be retrieved (in fact, they are already retrieved) from the tabular section of the document itself, while the latter two fields should be retrieved from other database tables:

- Cost is retrieved from the CostOfMaterials register

- Total balance in all warehouses is retrieved from the BalanceOfMaterials register (fig. 14.10)

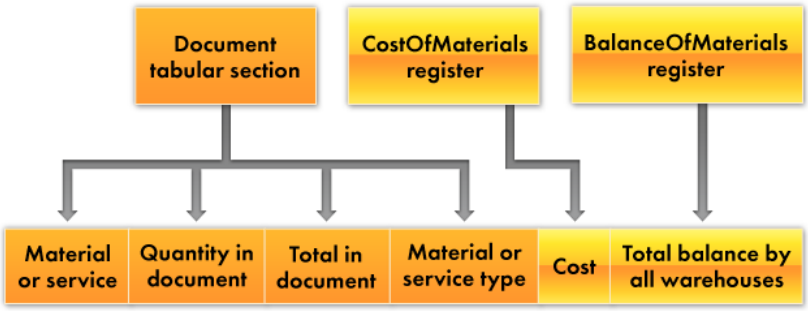


Fig. 14.10. Query field and table descriptions

This means that the query should include two left joins of the document table with other tables: the left join with the AccumulationRegister.CostOfMaterials.Balance table, and the left join with the AccumulationRegister.BalanceOfMaterials.Balance table.

Now it seems that everything is ready to generate the query.

But note one important aspect: in the suggested arrangement virtual tables return costs and balances for all the materials. And you only need this data for the materials available in the document.

This might not be noticeable in a small database because the number of unique materials in a catalog is almost equal to the number of unique materials in a document.

But imagine a real-life database. Suppose the MaterialsAndServices catalog includes 15 000 items. And a document only includes 5 items. The virtual table works heavily to calculate the cost (or balance) for all the 15 000 items, and when this table is connected with the document table using the left join, only 5 rows that correspond to the materials specified in the document are used. The remaining 14 995 rows are simply discarded so it is a waste of efforts to calculate them.

In real life such wastage is not acceptable and this query is far from optimal. So you should add a filter condition to all the virtual tables that you use in order to select only the materials contained in the tabular section of the document. In this scenario cost and balance are only calculated for the required materials instead of all the materials.

This gives you the following query pattern (fig. 14.11).

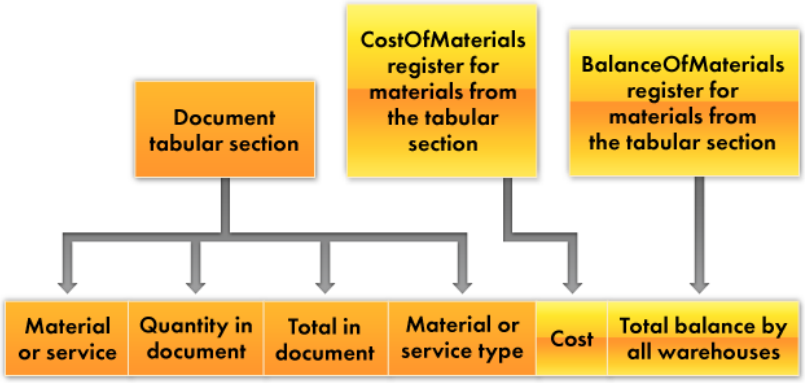


Fig. 14.11. Query pattern

Note that both virtual tables use the list of materials from the document tabular section (the data is calculated only for this list).

Besides, not all the data is retrieved from the document tables: instead, only the data related to a specific document is retrieved. In order to avoid generating this list three times (once for the document and again for each virtual table), you can have it generated in advance for later use in query conditions.

Temporary tables will help here.

A temporary table is a script object that is created and populated with data by a developer. Then queries can use this data when needed. For example, they can use it to apply some complex condition, which is the case in our scenario.

This gives you the following query pattern (fig. 14.12).

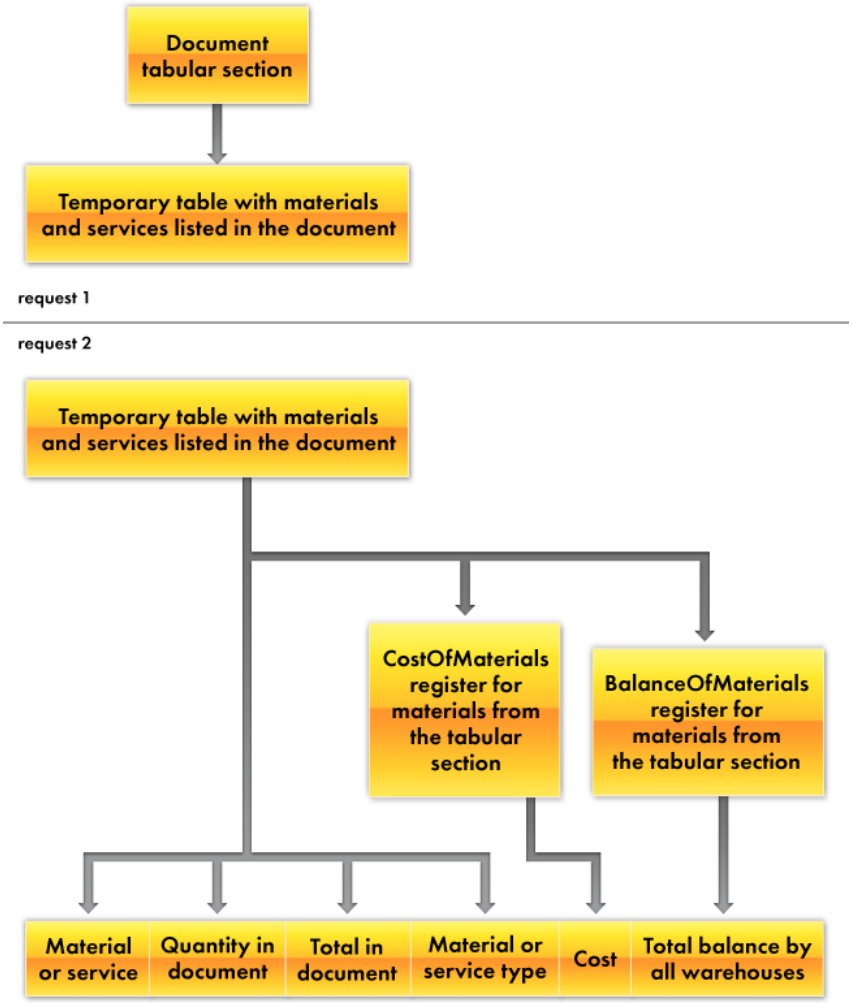


Fig. 14.12. Query pattern

Let us proceed to implementing the automatic cost calculation.

In Designer mode

The first thing you have to do is deleting the Cost attribute of the Services document because you do not need it any more.

1. Open the editor of the **Services** document configuration object, click the **Data** tab, expand the list of document tabular section attributes, click **Cost**, and click the **Delete current item** button in the command bar (fig. 14.13).

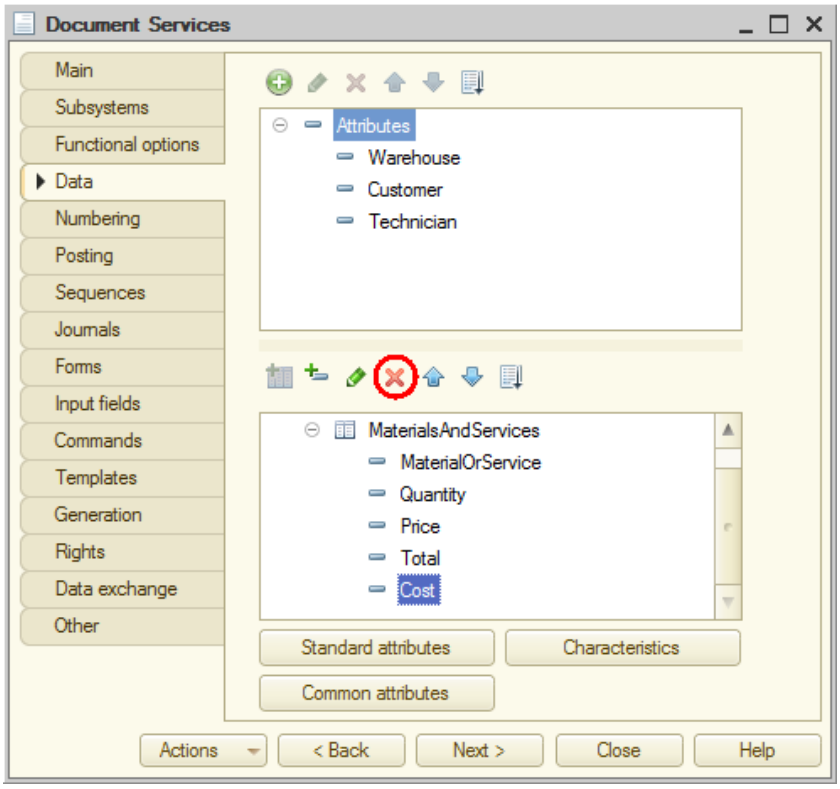


Fig. 14.13. Deleting a tabular section attribute

You also have to delete the corresponding field from the MaterialsAndServices table located in the document form.

2. For the **Services** document, open the **DocumentForm** form and then, in the list of form elements, click **MaterialsAndServicesCost**, and click the **Delete current item** button in the command bar (fig. 14.14).

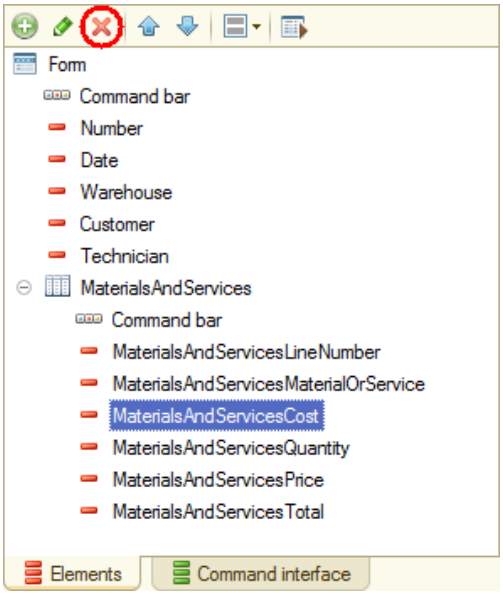


Fig. 14.14. Deleting a tabular section field

Then let us deal with the query. First, let us create the temporary table using the query that you have created earlier.

3. Open the **Services** document module.
4. In the Posting() procedure, before the query generation, add the lines that create a temporary tables manager and specify that the query uses the manager (listing 14.17).

Listing 14.17. Using a temporary tables manager

```
RegisterRecords.BalanceOfMaterials.Write = True;
RegisterRecords.CostOfMaterials.Write = True;
RegisterRecords.Sales.Write = True;

// Creating temporary tables manager
TTManager = New TempTablesManager;

Query = New Query;

// Specifying the temporary tables manager used by the query
Query.TempTablesManager = TTManager;

Query.Text =
    "SELECT
    |    ServicesMaterialsAndServices.MaterialOrService,
```

Let us modify the query so that it creates a temporary table to be stored in the TTManager temporary tables manager.

5. Delete the query line shown in listing 14.18.

   This is required to make the query suitable for opening with the query wizard (as you do not have the Cost field anymore).

Listing 14.18. Line to remove

```
|    MAX(ServicesMaterialsAndServices.Cost) AS Cost
```

6. Delete the comma in the end of the previous line (listing 14.19).

Listing 14.19. Line to modify

```
|    SUM(ServicesMaterialsAndServices.Total) AS TotalInDocument
```

7. Right-click anywhere inside the query (for example, on the SELECT word) and click **Query Builder**.

   This opens the query text in the form of the query wizard (fig. 14.15).

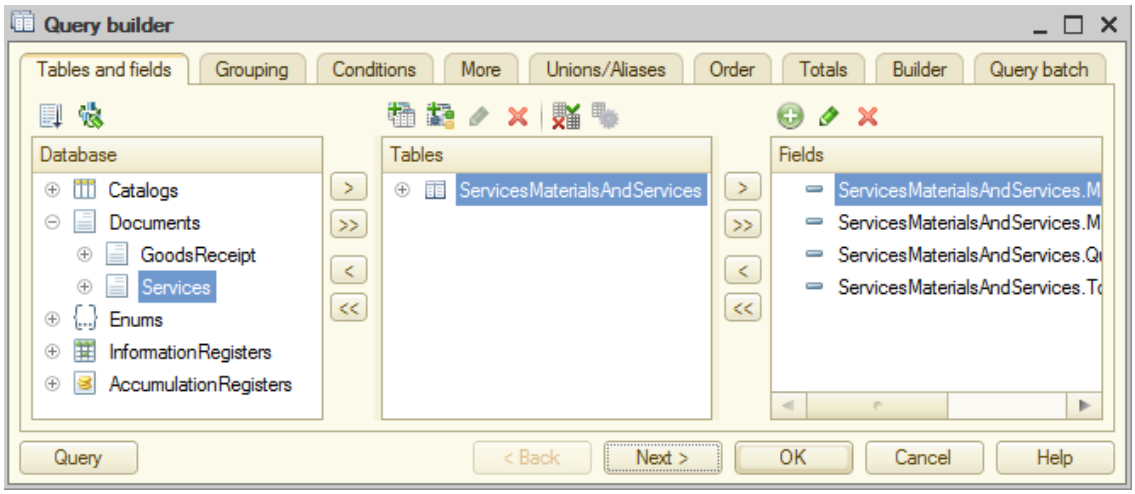


Fig. 14.15. Query wizard

Let us store the query result to a temporary table.

8. Click the **More** tab and then click **Create temporary table**.
9. In the **Temporary table name** field, enter *DocumentMaterialsAndServices* (fig. 14.16).

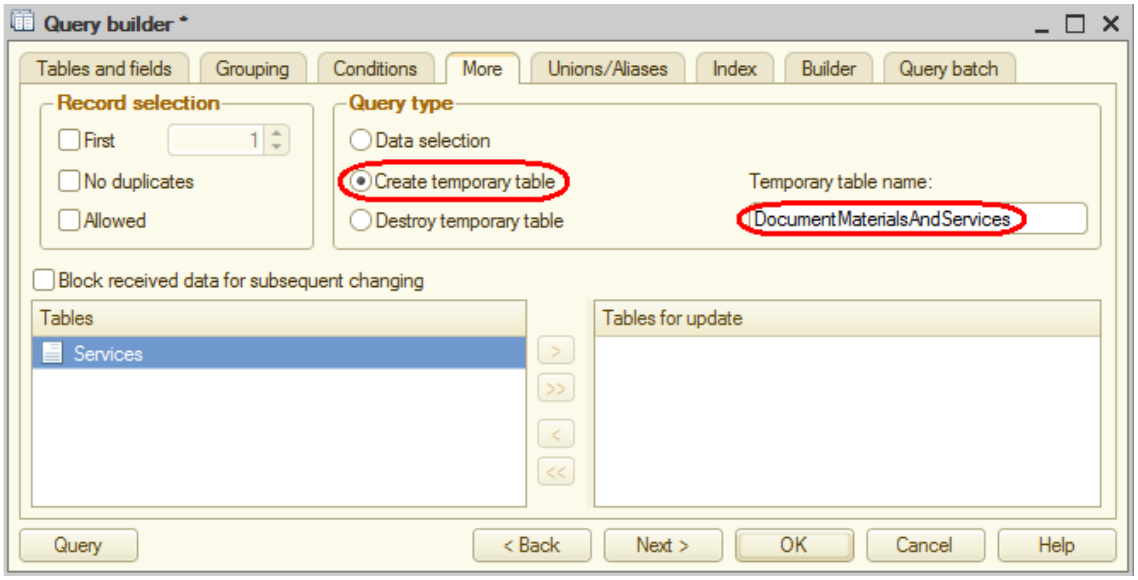


Fig. 14.16. Creating a temporary table

10. Click **OK** and review the text generated by the query wizard (listing 14.20).

Listing 14.20. Query text

```
"SELECT
|   ServicesMaterialsAndServices.MaterialOrService,
|   ServicesMaterialsAndServices.MaterialOrService.MaterialServiceType
|     AS MaterialServiceType,
|     SUM(ServicesMaterialsAndServices.Quantity) AS QuantityInDocument,
|     SUM(ServicesMaterialsAndServices.Total) AS TotalInDocument
|INTO DocumentMaterialsAndServices
|FROM
```

```
|  Document.Services.MaterialsAndServices
|    AS ServicesMaterialsAndServices
|WHERE
|  ServicesMaterialsAndServices.Ref = &Ref
|GROUP BY
|  ServicesMaterialsAndServices.MaterialOrService,
|  ServicesMaterialsAndServices.MaterialOrSevice.MaterialServiceType";
```

There is only one new line here (listing 14.21).

Listing 14.21. Creating a temporary table

```
|INTO DocumentMaterialsAndServices
```

This line means that the query result is stored to the DocumentMaterialsAndServices temporary table.

If you specify the same temporary tables manager (TTManager) for another query, you will be able to access the data of this temporary table in that query.

So you have completed a part of the task: created a query that stores the document tabular section data to a temporary table (fig. 14.17).

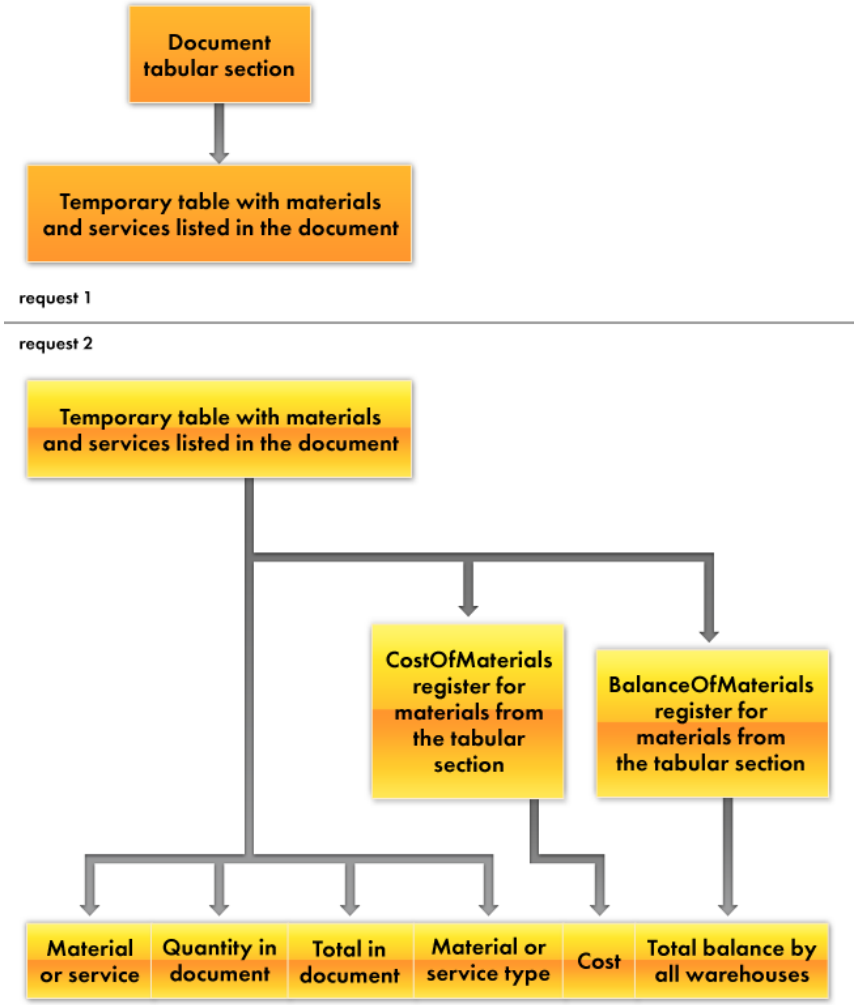


Fig. 14.17. Creating the first query

Let us proceed to designing the second query.

11. Put the cursor into the line following the clause QueryResult = Query.Execute(); (this is where the temporary table is created) and create a template for your future query (listing 14.22).

Listing 14.22. Creating the second query

```
Query2 = New Query;
Query2.TempTablesManager = TTManager;
Query2.Text = "";
```

You have created a new Query object and specified the same temporary tables manager for this query. Now the query can access the temporary table that you created earlier.

12. Right-click the area between the quotation marks and click **Query Builder**.
13. Confirm that you want to create a new query.

    Since you are going to select data from your temporary table, let us add the temporary table description to the query.

14. Above the **Tables** list, click the **Create a temporary table description** button (fig. 14.18).

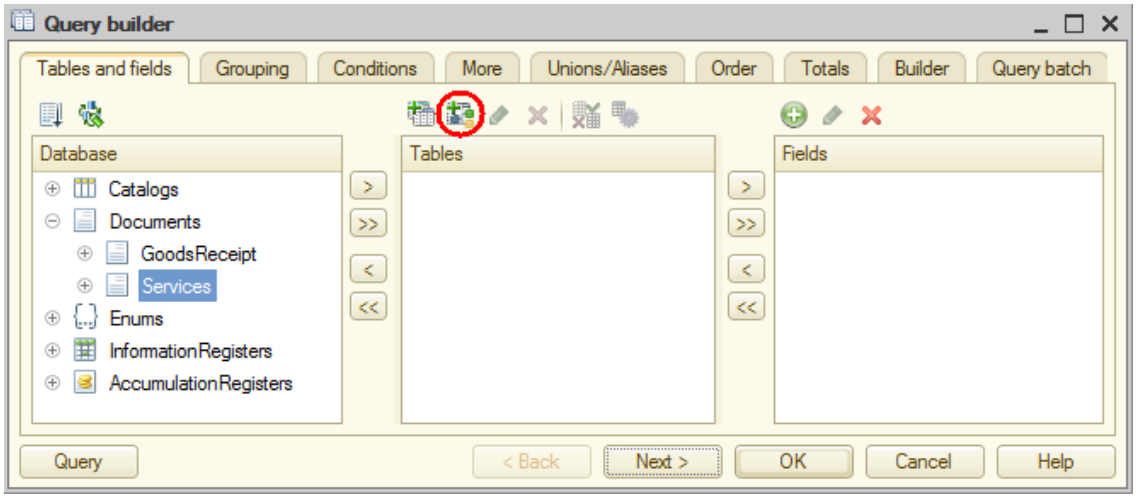


Fig. 14.18. Creating a temporary table description

15. In the window that is opened, in the **Table name** field, enter *DocumentMaterialsAndServices*.
16. Add the following field descriptions:
    - **MaterialOrService**. Type: CatalogRef.MaterialsAndServices
    - **MaterialServiceType**. Type: EnumRef.MaterialServiceTypes
    - **QuantityInDocument**. Type: Number, length: 15, precision: 3
    - **TotalInDocument**. Type: Number, length: 15, precision: 2

    The resulting description of the temporary table should look as shown in fig. 14.19.

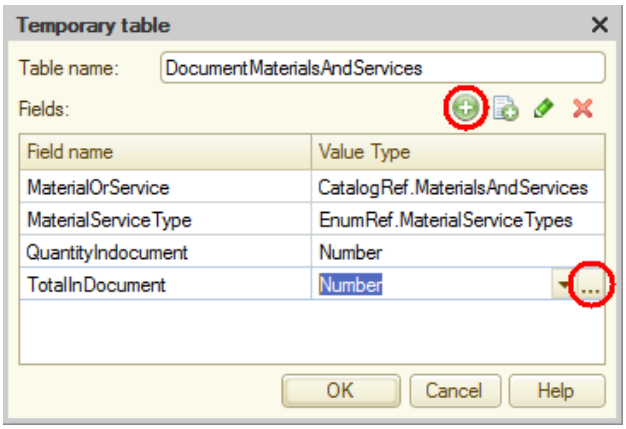


Fig. 14.19. Creating a temporary table description

17. Click **OK**.
18. Select all the fields from this table (fig. 14.20) and click the **Query** button in the bottom left corner of the query wizard.

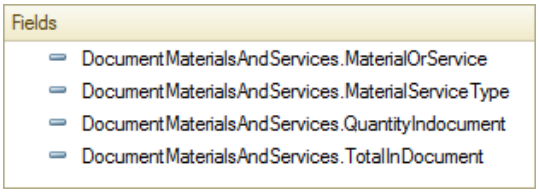


Fig. 14.20. Selected temporary table fields

The query text should look as shown in listing 14.23.

Listing 14.23. Text of the second query

```
SELECT
    DocumentMaterialsAndServices.MaterialOrService,
    DocumentMaterialsAndServices.MaterialServiceType,
    DocumentMaterialsAndServices.QuantityIndocument,
    DocumentMaterialsAndServices.TotalInDocument
FROM
    DocumentMaterialsAndServices AS DocumentMaterialsAndServices
```

So you have created the first part of the second query, which selects data from the temporary table (fig. 14.21).

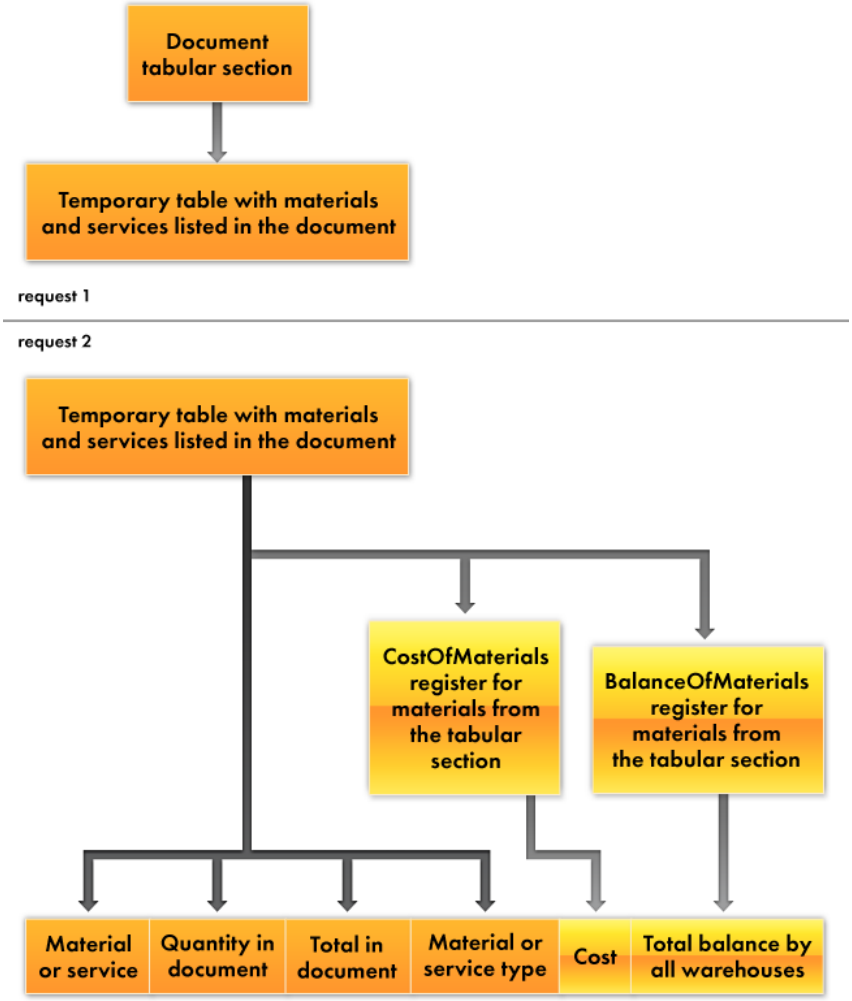


Fig. 14.21. Creating the second query

Now you need left joins to connect this query part with the balance tables.

Let us begin with cost of materials.

19. Add the **AccumulationRegister.CostOfMaterials.Balance** virtual table to the list of query tables. Select the **CostBalance** field from this table.

    Let us define the relation between the tables.

20. Click the **Links** tab and specify that all the records are selected from the temporary table and the MaterialOrService field of the temporary table matches the Material field of the balances table (fig. 14.22).

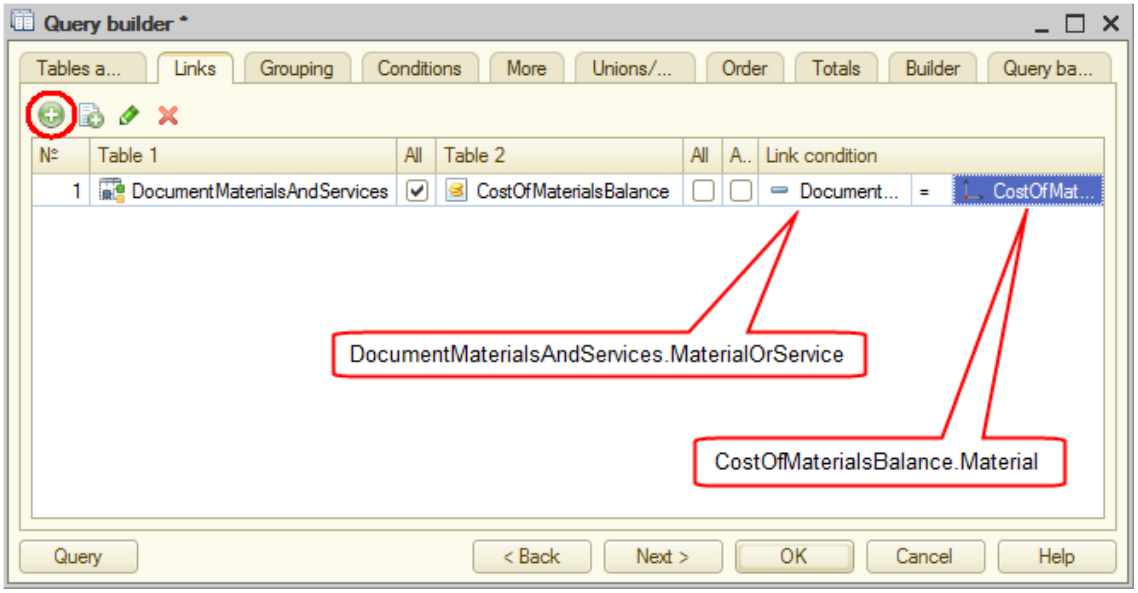


Fig. 14.22. Relation between the tables

You also need to filter the virtual table data, so that only materials available in the temporary table are selected.

21. Click the **Tables and fields** tab, click the **CostOfMaterialsBalance** table, and click the **Virtual table parameters** button above the list of tables.

22. In the **Condition** field, enter the line shown in listing 14.24.

Listing 14.24. Virtual table condition

```
Material IN (SELECT DocumentMaterialsAndServices.MaterialOrService
FROM DocumentMaterialsAndServices)
```

This means that material should be in the list of materials and services selected from the temporary table.

Learn more! Be careful in how you use virtual query tables. Be particularly mindful of using virtual table parameters as much as possible.

For example, in this scenario you could avoid using the Condition parameter and filter the field selection in the query itself, by using the BY condition with materials and services from the document equal to materials from the balance table. Technically it gives exactly the same result, but not the same performance.

In the scenario that you implemented the number or records retrieved from the virtual table is equal to the number of unique materials and services in the document being posted. And if you set no conditions, the virtual table provides you with the records for all the materials that are available in the accumulation register, and you have to filter that large list of records in the query to get the several records that you need.

Clearly, the second option would take longer to complete, and the execution time for such query would depend less on the amount of

data contained in the document (the actual amount of data to be processed), but rather on the size of the accumulation register.

Aside from the fact that this option affects the configuration performance, it may happen that the two approaches produce different results. For example, this might happen when you use the SliceLast virtual table of an information register. For more information, see Using filters in queries with virtual tables.

23. Click the **Query** button and review the text generated by the query wizard (listing 14.25).

Listing 14.25. Query text

```
SELECT
    DocumentMaterialsAndServices.MaterialOrService,
    DocumentMaterialsAndServices.MaterialServiceType,
    DocumentMaterialsAndServices.QuantityIndocument,
    DocumentMaterialsAndServices.TotalInDocument,
    CostOfMaterialsBalance.CostBalance
FROM
    DocumentMaterialsAndServices AS DocumentMaterialsAndServices
        LEFT JOIN AccumulationRegister.CostOfMaterials.Balance( ,
            Material IN (
        SELECT
            DocumentMaterialsAndServices.MaterialOrService
        FROM
            DocumentMaterialsAndServices)) AS CostOfMaterialsBalance
ON DocumentMaterialsAndServices.MaterialOrService =
    CostOfMaterialsBalance.Material
```

So you added the material cost to the selection fields (fig. 14.23).

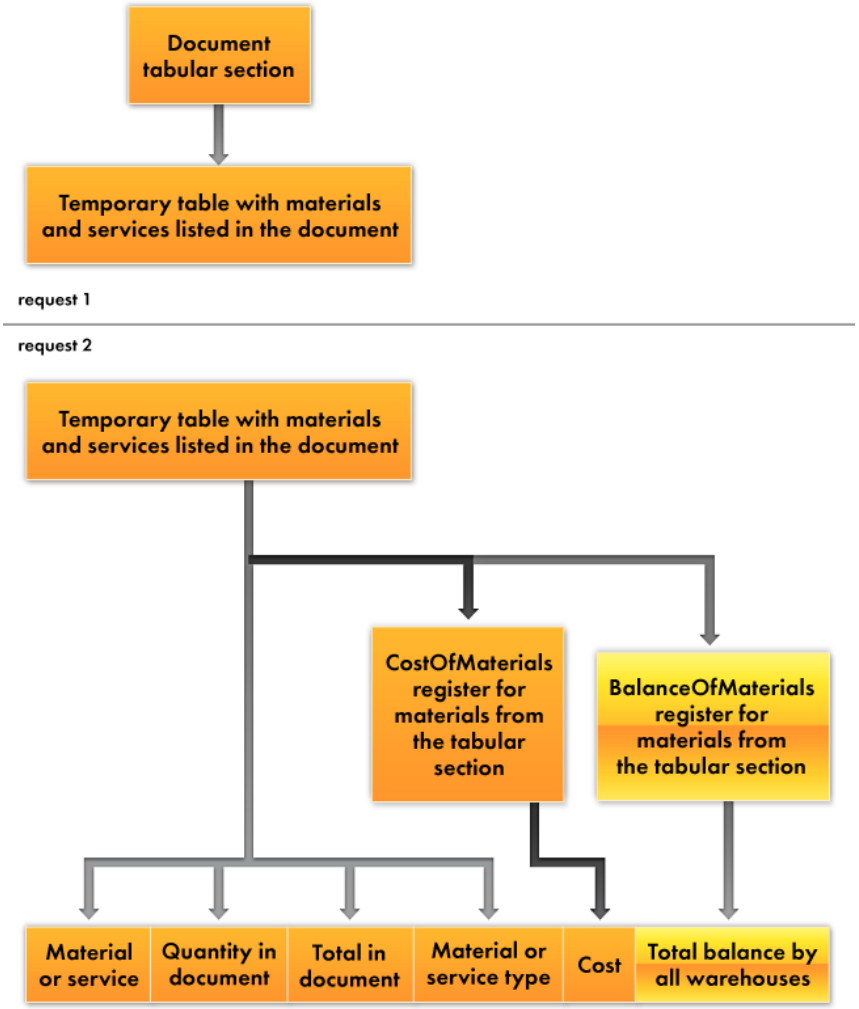


Fig. 14.23. Creating the second query

24. Add the **Balances** virtual table of the **BalanceOfMaterials** register to the list of query tables.
25. Select the **QuantityBalance** field from this table.

    Let us define the relation between the tables.
26. Click the **Links** tab and specify that all the records are selected from the temporary table and the MaterialOrService field of the temporary table matches the Material field of the balances table (fig. 14.24).

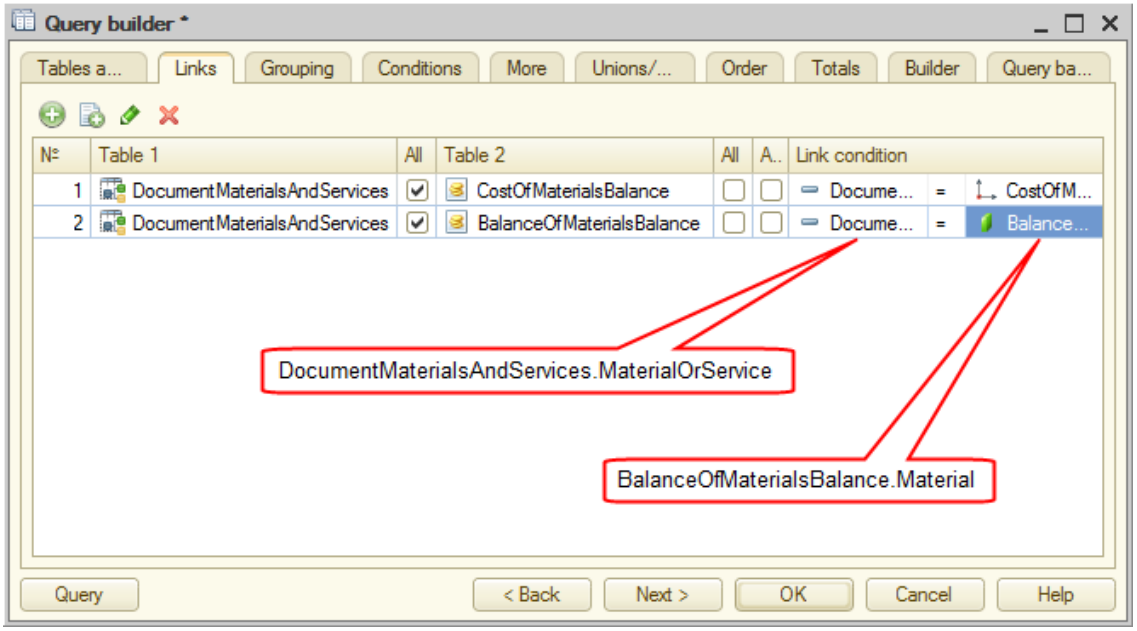


Fig. 14.24. Relation between the tables

Let us specify the parameters for the BalanceOfMaterialsBalance virtual table.

27. In the **Condition** field, enter the line shown in listing 14.26.

Listing 14.26. Virtual table condition

```
Material IN (SELECT DocumentMaterialsAndServices.MaterialOrService
FROM DocumentMaterialsAndServices)
```

This gives you the following query text (listing 14.27).

Listing 14.27. Query text

```
SELECT
    DocumentMaterialsAndServices.MaterialOrService,
    DocumentMaterialsAndServices.MaterialServiceType,
    DocumentMaterialsAndServices.QuantityIndocument,
    DocumentMaterialsAndServices.TotalInDocument,
    CostOfMaterialsBalance.CostBalance,
    BalanceOfMaterialsBalance.QuantityBalance
FROM
    DocumentMaterialsAndServices AS DocumentMaterialsAndServices
        LEFT JOIN AccumulationRegister.CostOfMaterials.Balance( ,
            Material IN (
            SELECT
                DocumentMaterialsAndServices.MaterialOrService
            FROM
                DocumentMaterialsAndServices))
                    AS CostOfMaterialsBalance
ON DocumentMaterialsAndServices.MaterialOrService =
    CostOfMaterialsBalance.Material
LEFT JOIN AccumulationRegister.BalanceOfMaterials.Balance( ,
    Material IN (
```

```
            SELECT

                DocumentMaterialsAndServices.MaterialOrService

            FROM

                DocumentMaterialsAndServices))
                    AS BalanceOfMaterialsBalance

ON DocumentMaterialsAndServices.MaterialOrService =
    BalanceOfMaterialsBalance.Material
```

So you added the material balance in all the warehouses to the selection fields (fig. 14.25).

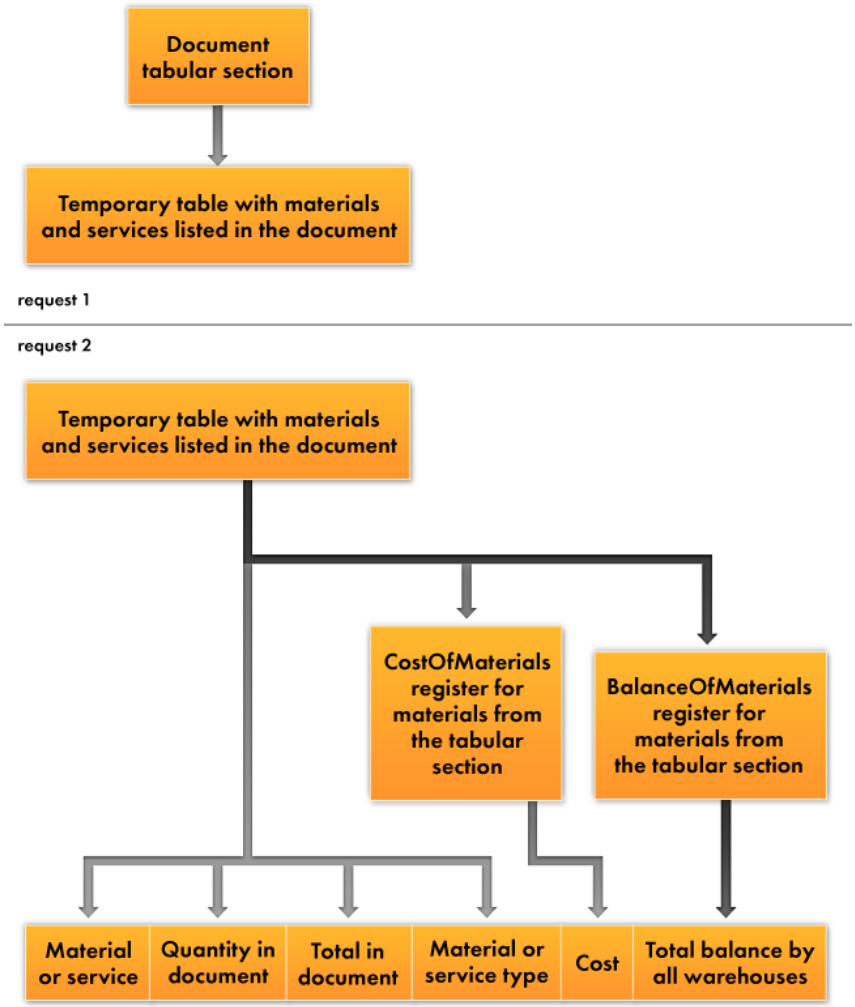


Fig. 14.25. Creating the second query

28. Click the **Unions/Aliases** tab, set the *CostBalance* alias to *Cost*, and set the **QuantityBalance** alias to Quantity (fig. 14.26).

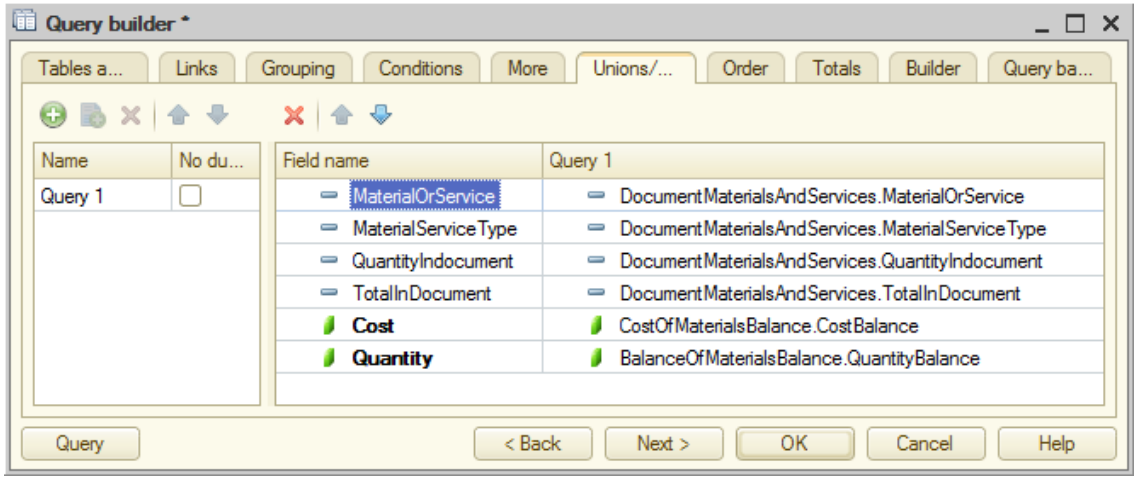


Fig. 14.26. Field aliases

The query text generated by the wizard is shown in listing 14.28.

Listing 14.28. Query text

```
SELECT
    DocumentMaterialsAndServices.MaterialOrService,
    DocumentMaterialsAndServices.MaterialServiceType,
    DocumentMaterialsAndServices.QuantityInDocument,
    DocumentMaterialsAndServices.TotalInDocument,
    CostOfMaterialsBalance.CostBalance AS Cost,
    BalanceOfMaterialsBalance.QuantityBalance AS Quantity
FROM
    DocumentMaterialsAndServices AS DocumentMaterialsAndServices
        LEFT JOIN AccumulationRegister.CostOfMaterials.Balance( ,
            Material IN (
            SELECT
                DocumentMaterialsAndServices.MaterialOrService
            FROM
                DocumentMaterialsAndServices))
                    AS CostOfMaterialsBalance
        ON DocumentMaterialsAndServices.MaterialOrService =
            CostOfMaterialsBalance.Material
        LEFT JOIN AccumulationRegister.BalanceOfMaterials.Balance( ,
        Material IN (
            SELECT
                DocumentMaterialsAndServices.MaterialOrService
            FROM
                DocumentMaterialsAndServices))
                    AS BalanceOfMaterialsBalance
        ON DocumentMaterialsAndServices.MaterialOrService =
            BalanceOfMaterialsBalance.Material
```

The final thing to do with the query is specifying how to handle the situation where a material is available in the catalog but without any balance or cost data. For example, this can happen when a material has already been added to the catalog but has not been delivered to the company yet.

In this situation left joins with virtual tables return nothing. In the query language it means that the cost and quantity fields store NULL values.

For future convenience, let us eliminate such values right in the query.

Let us apply the ISNULL() function to the Cost and Quantity fields. If a field value is NULL, the function returns 0. Otherwise it returns the field value.

29. Click the **Tables and fields** tab, click the **CostOfMaterialsBalance.CostBalance** field, and click the **Change current item** button (fig. 14.27).

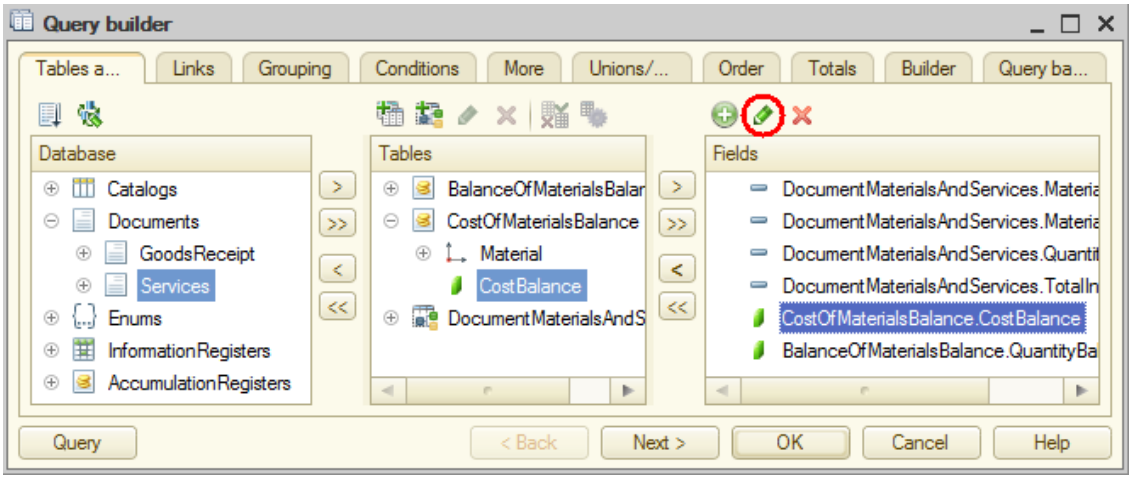


Fig. 14.27. Changing a field value in a query

30. In the window that is opened, edit the field value as shown in listing 14.29 (fig. 14.28).

Listing 14.29. Expression for calculating a field in a query

```
ISNULL(CostOfMaterialsBalance.CostBalance, 0)
```

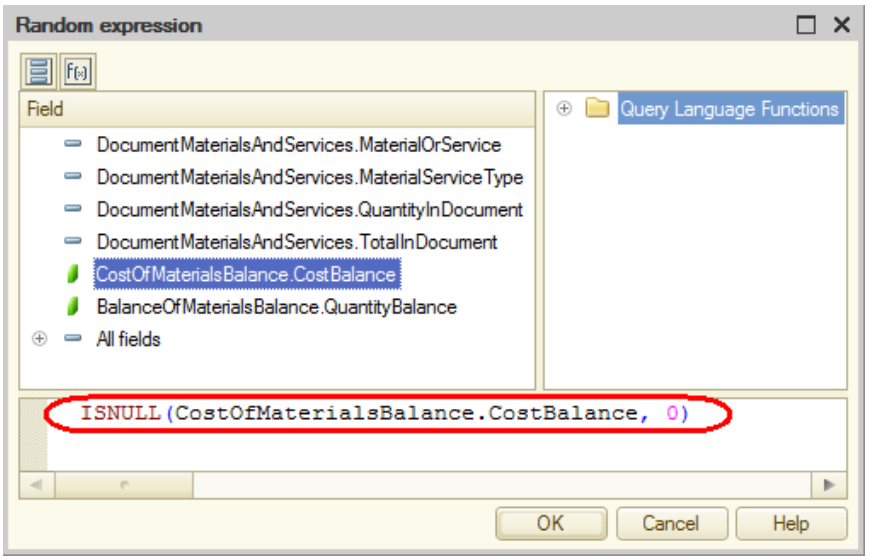


Fig. 14.28. Changing a field value in a query

31. Set a similar expression for calculating the **BalanceOfMaterialsBalance.QuantityBalance** field and click **OK**.

    The query text is added to the module (listing 14.30).

Listing 14.30. Query text

```
Query2.Text = "SELECT
|    DocumentMaterialsAndServices.MaterialOrService,
|    DocumentMaterialsAndServices.MaterialServiceType,
|    DocumentMaterialsAndServices.QuantityInDocument,
|    DocumentMaterialsAndServices.TotalInDocument,
|    ISNULL(CostOfMaterialsBalance.CostBalance, 0) AS Cost,
|    ISNULL(BalanceOfMaterialsBalance.QuantityBalance, 0) AS Quantity
|FROM
|    DocumentMaterialsAndServices AS DocumentMaterialsAndServices
|        LEFT JOIN AccumulationRegister.CostOfMaterials.Balance( ,
|            Material IN (
|            SELECT
|                DocumentMaterialsAndServices.MaterialOrService
|            FROM
|                DocumentMaterialsAndServices))
|                    AS CostOfMaterialsBalance
|        ON DocumentMaterialsAndServices.MaterialOrService =
|            CostOfMaterialsBalance.Material
|        LEFT JOIN AccumulationRegister.BalanceOfMaterials.Balance( ,
|            Material IN (
|            SELECT
|                DocumentMaterialsAndServices.MaterialOrService
|            FROM
|                DocumentMaterialsAndServices))
|                    AS BalanceOfMaterialsBalance
```

```
|        ON DocumentMaterialsAndServices.MaterialOrService =
|            BalanceOfMaterialsBalance.Material";
```

The only thing you need to do is add the query execution statement after the query text (listing 14.31).

Listing 14.31. Posting() procedure (fragment)

```
...
Query2 = New Query;
Query2.TempTablesManager = TTManager;
Query2.Text = "SELECT
    |    DocumentMaterialsAndServices.MaterialOrService,
...
    |        ON DocumentMaterialsAndServices.MaterialOrService =
    |            BalanceOfMaterialsBalance.Material";

QueryResult = Query2.Execute();

SelectionDetailRecords = QueryResult.Select();
...
```

Now let us deal with writing register records.

You can use the script lines that you wrote earlier. The only thing you need to change is how cost is obtained. You used to take it directly from the document but now you need to calculate it based on the value retrieved in the query.

The material cost is calculated by dividing the total cost retrieved in the query (Cost) by the total quantity of material units available in all warehouses (Quantity).

But we have already mentioned that the Quantity field might contain zero and one cannot divide by zero. So right after the beginning of the loop that iterates through the query result add the script that calculates the cost for the current material (listing 14.32).

Listing 14.32. Posting() procedure (fragment)

```
While SelectionDetailRecords.Next() Do

    If SelectionDetailRecords.Quantity = 0 Then
        MaterialCost = 0;
    Else
        MaterialCost = SelectionDetailRecords.Cost
            / SelectionDetailRecords.Quantity;
    EndIf;

    If SelectionDetailRecords.MaterialServiceType =
        Enums.MaterialServiceTypes.Material Then

...
```

32. Replace the cost calculation expressions in the script fragment that writes CostOfMaterials and Sales register records (listing 14.33).

Listing 14.33. Posting() procedure (fragment)

```
While SelectionDetailRecords.Next() Do


    If SelectionDetailRecords.Quantity = 0 Then
        MaterialCost = 0;
    Else
        MaterialCost = SelectionDetailRecords.Cost
            / SelectionDetailRecords.Quantity;
    EndIf;


    If SelectionDetailRecords.MaterialServiceType =
        Enums.MaterialServiceTypes.Material Then


        // register BalanceOfMaterials Expense
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Warehouse = Warehouse;
        Record.Quantity = SelectionDetailRecords.QuantityInDocument;


        // register CostOfMaterials Expense
        Record = RegisterRecords.CostOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Cost = SelectionDetailRecords.QuantityInDocument
            * MaterialCost;


    EndIf;


    // register Sales
    Record = RegisterRecords.Sales.Add();
    Record.Period = Date;
    Record.MaterialOrService =
        SelectionDetailRecords.MaterialOrService;
    Record.Customer = Customer;
    Record.Technician = Technician;
    Record.Quantity = SelectionDetailRecords.QuantityInDocument;
    Record.Revenue = SelectionDetailRecords.TotalInDocument;
    Record.Cost = SelectionDetailRecords.QuantityInDocument
        * MaterialCost;


EndDo;
```

Now everything seems to be fine but there is still one important issue.

The first posting of a document gives the correct result but the second posting makes it incorrect because at the time of the second posting the database contains the register records written during the first posting.

So when the cost and balance of materials are read from the database, the retrieved values are based on the register records written by this document earlier. And this is absolutely wrong.

In order to read database data in the posting event handler without reading the register records written by this document earlier, you have to write empty record sets to the registers before reading data from them.

In this example you have to write empty record sets to the CostOfMaterials and BalanceOfMaterials accumulation registers.

33. Add two lines before the execution of the second query, as shown in listing 14.33a.

Listing 14.33a. Posting() procedure (fragment)

```
...
|        ON DocumentMaterialsAndServices.MaterialOrService =
|            BalanceOfMaterialsBalance.Material";


// Writing empty record sets to read balances without the data added
by this document
RegisterRecords.CostOfMaterials.Write();
RegisterRecords.BalanceOfMaterials.Write();


QueryResult = Query2.Execute();
```

In 1C:Enterprise mode

Run 1C:Enterprise in the debug mode, repost the Services documents, and ensure that the document data is recorded to the registers correctly.

Theory

Quick viewing of a query result

During the modification of the posting procedure of the Services document you went through creating quite a complex query.

You might need to ensure that some data read from the database in a query is correct, and view this data as a table.

You can do it easily in Designer. We will discuss this feature using the query from your posting handler.

Suppose that you want to dump the execution result of Query2 to a value table (VT).

1. Modify the script as shown in listing 14.34.

   Listing 14.34. Posting() procedure (fragment)

```
...
QueryResult = Query2.Execute();

VT = QueryResult.Unload();

SelectionDetailRecords = QueryResult.Select();
...
```

2. Set a breakpoint at the statement SelectionDetailRecords = Result.Select().
3. Run 1C:Enterprise in the debug mode and repost one of the Services documents (for example, Services #2).

   The script execution is interrupted at that point and the module text is opened in Designer.
4. Double-click the **VT** word to select it, and then, on the **Configuration Debug** toolbar, click the **Evaluate expression (Shift + F9)** button.

   This opens the expression preview window with the VT value table.

   A value table is a collection, so you can use the standard method of viewing collection data.
5. In the **Result** list, click the **VT** row and then click the **Show Values in Separate Window (F2)** button above the list (fig. 14.29).

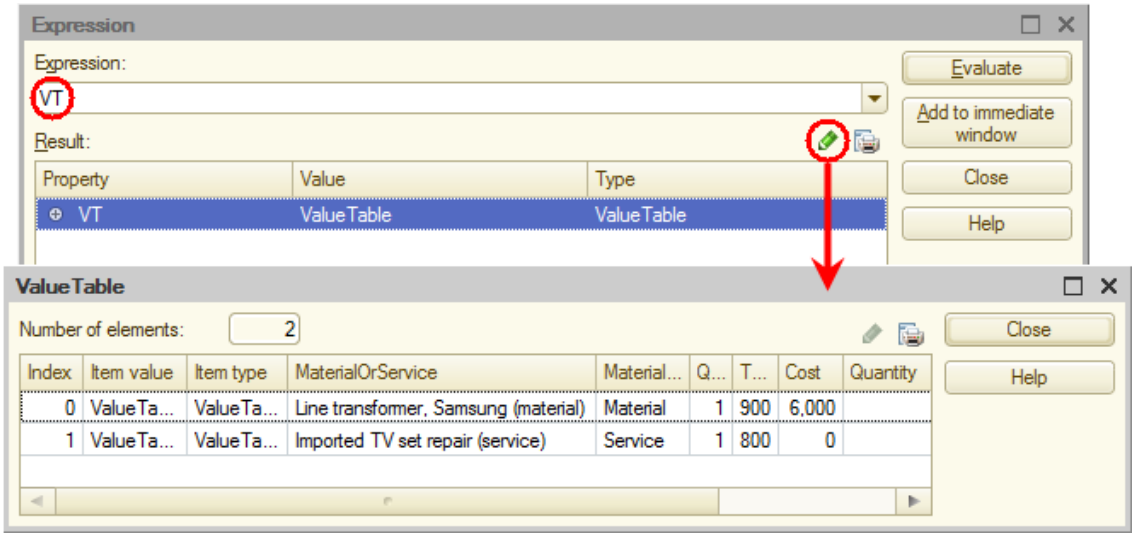


Fig. 14.29. Viewing a value table that stores the query result

The window that is opened contains the entire value table that stores the query execution result.

You can use the **Output list** button to export the value table to a spreadsheet document if you want to analyze this data or save it for later use.

After previewing the query result, remember to remove the breakpoint from the document posting procedure and either comment or remove the line that exports the query result into a value table. You do not need it when you perform regular document operations.

Real-time and regular document posting

Let us discuss some specifics of document posting procedures that check the availability of materials in warehouses.

This check is not required all the time. It is required for real-time document posting and not required for regular document posting. So what are real-time and regular posting modes intended for? And how does real-time posting work?

The development of 1C:Enterprise configurations involves the concept of real-time and regular document posting. This concept helps solving tasks related to real-time accounting, such as inventory accounting, and ensures proper document processing in real-life conditions. But you can disable this feature (forbid real-time document posting) and implement your own algorithms that determine the document order on the timeline and perform other posting-related operations.

This concept implies that users can work in two fundamentally different modes.

Real-time document posting is performed in real time, in other words, it reflects the current changes and facts. Real-time posting is particularly relevant when multiple users work with a single application. Therefore, for this posting mode you need to implement the maximum number of checks to prevent errors when accepting user input.

For example, in real-time posting you need to monitor the on-hand balances for listed products in order to prevent a single item from being sold simultaneously by multiple sales reps.

During the real-time posting the platform first compares the document date with the current session date. The current session date is the computer system date converted to the session time zone. If the date of the document being posted matches the current date, the platform posts the document in real-time mode. The event handler is capable of getting the posting mode, so it can select the appropriate posting algorithm.

If the date of the document being posted is before the current date, the document is posted in regular mode.

Regular posting means that the facts that took place in the past or are certain to happen in the future are reflected in the database. Therefore, regular document posting is intended to simply reflect data on completed operations in the infobase.

For regular posting, many checks do not make sense, especially the balance checks. Naturally, if mistakes were made during regular document posting

(such as issuing a number of products not available in the warehouse on the posting date), analysis of the resulting database state is a separate task, unrelated to regular posting and performed independently of that process. Instead, this process takes place when the database has sufficient data for analysis, for example, when earlier goods receipt documents are entered.

Hence real-time posting is intended to determine whether an operation can be executed in real time in a multiuser environment (and execute the operation if possible). By contrast, regular posting is intended for unconditional recording of operations that are already completed or will certainly be completed to the database.

Sometimes you might want to post a document with some date in the future in order to reflect some events that are certain to happen in the future. In this scenario the platform does not allow posting the document with a future date if the document is allowed to use real-time posting. You will be warned that real-time posting is not available for this document without any other options to choose from. So you will either need to change the document date to the current date (and then you can post the document in real-time mode), or to a date in the past (and then you can post the document in regular mode). So if the accounting logic suggests that some document should be posted with a date in the future, real-time posting mode should be disabled for this document in the metadata (on the Posting tab of the configuration object editor).

The concepts of "real-time timestamp" and "point in time" are related to real-time document posting. We will not use this information right now but this is a good chance to discuss it because we are talking about real-time posting.

"Point in time" concept

The Date document attribute determines its position on the time axis. A date includes time with one second precision. This allows you to control the document writing order. However, when a large number of documents are created, some documents might have identical dates (because they are created within a single second). How does one determine the sequence of document creation in this situation?

The point in time concept has been introduced to deal with this problem. A point in time is composed of date, time, and a reference to a database object. This lets you explicitly identify the position of any reference type database object on the time axis, but it generally only applies to documents. The point in time also allows you to determine the order of nonobject data, such as register records that are subordinate to a recorder.

The point in time concept is implemented in 1C:Enterprise script as the universal PointInTime object. This object has the Date and Ref properties, which provide you with the option to get the point in time "components", and a single Compare() method that compares two points in time. Besides, a PointInTime object has a constructor and can be created explicitly for any database object of reference type.

For multiple documents that have identical dates and times, the platform determines their order on the time axis based on the references to these documents. The order does not always match the order of document

creation, and it is not available to users, so they cannot change the order of documents within a one-second period or determine that one document was created before another.

The platform generates a real-time timestamp at each real-time document posting. The timestamp value is generated based on the current time and the last real-time timestamp generated.

If the last real-time timestamp that was generated is less than the current session time, the current session time is used as the new real-time timestamp.

If the last real-time timestamp is equal to or greater than the current session time, the real-time timestamp is set to a value one second greater than the old real-time timestamp.

So, the platform processes posting of Document configuration objects that have real-time posting enabled (fig. 14.30) as follows:

- When you create a document, the platform assigns the current date and "zero" time to the document.
- When you post a document with the same date (day) as the current session date, the platform assigns a real-time timestamp to the document date.
- When you cancel the document posting and repost it (without changing its date), the platform sets a new real-time timestamp.
- When you attempt to repost a document, the platform sets a new real-time timestamp and posts the document.
- If you attempt to post (or repost) a document with the date (day) earlier than the current session date, the platform posts the document in regular mode.
- If you attempt to post (or repost) a document with the date (day) later than the current session date, the platform prohibits the posting.

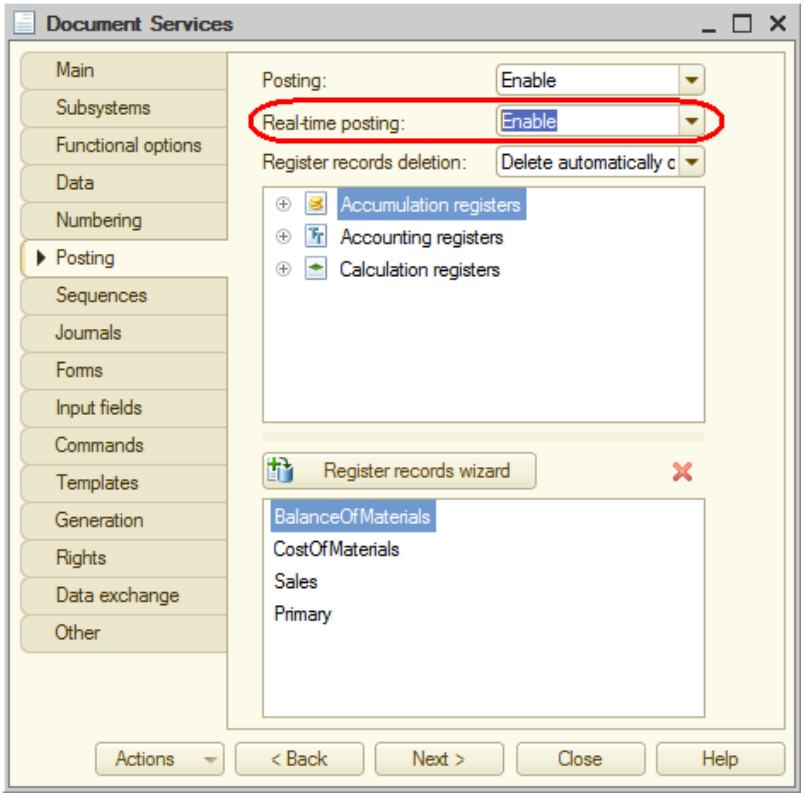


Fig. 14.30. Enabling real-time document posting

Checking the balances

The general method of monitoring item balances during document posting is as follows: first, write the register records generated by the document without any checks, and then read the balance values from the database.

If a negative balance value is detected, the document cannot be posted. Instead the applied solution should inform the user about the missing materials and undo the posting.

If no negative balance values are detected, a document can be posted without hesitation.

You are actually half-finished with this task: you implemented the generation and writing of document register records. The only thing you still need to do is monitor the results of real-time posting and undo the posting if a negative balance value is found.

In Designer mode

Let us add a script that performs real-time posting checks.

1. Add the following lines after the loop that iterates through the query results and before the end of the procedure (listing 14.35).

Listing 14.35. Posting() procedure (fragment)

```
...
    EndDo;
```

```
    RegisterRecords.Write();

    If Mode = DocumentPostingMode.RealTime Then
        // Checking for negative balances
    EndIf;

EndProcedure
```

First, this script writes the register records.

Then it determines the document posting mode. In the Posting() procedure call, the second parameter (Mode) is the document posting mode, and the value of that variable is compared to the value of the DocumentPostingMode system enumeration. In the event of real-time posting the balance check is applied.

Let us start writing the negative balance check.

Since once again you need to get balance values only for the materials available in the document, let us specify that this query uses the same temporary tables manager (TTManager).

2. Add the lines specified in listing 14.36 to the end of the posting procedure.

Listing 14.36. Posting() procedure (fragment)

```
If Mode = DocumentPostingMode.RealTime Then
    // Checking for negative balances
    Query3 = New Query;
    Query3.TempTablesManager = TTManager;
    Query3.Text = "";
EndIf;
```

3. Right-click between the quotation marks and open the query wizard.
4. Confirm that you want to create a new query.
5. Select the **BalanceOfMaterials.Balance** table and two fields from this table: **Material** and **QuantityBalance**.
6. In the **Condition** field, enter the condition specified in listing 14.37.

Listing 14.37. Virtual table condition

```
Material IN (
    SELECT
        DocumentMaterialsAndServices.MaterialOrService
    FROM
        DocumentMaterialsAndServices)
AND Warehouse = &Warehouse
```

This means that you only get totals for the materials available in the temporary table and only for the warehouse specified in the document.

7. Click the **Conditions** tab, move the **QuantityBalance** field to the list of conditions, select the **Arbitrary** check box, and specify that you are only interested in negative balance values (listing 14.38).

Listing 14.38. Query condition

```
BalanceOfMaterialsBalance.QuantityBalance < 0
```

8. Click **OK**.

The query text should look as shown in listing 14.39.

Listing 14.39. Query text

```
// Checking for negative balances
Query3 = New Query;
Query3.TempTablesManager = TTManager;
Query3.Text = "SELECT
  |   BalanceOfMaterialsBalance.Material,
  |   BalanceOfMaterialsBalance.QuantityBalance
  |FROM
  |   AccumulationRegister.BalanceOfMaterials.Balance( , Material IN (
  |      SELECT
  |         DocumentMaterialsAndServices.MaterialOrService
  |      FROM
  |         DocumentMaterialsAndServices)
  |      AND Warehouse = &Warehouse) AS BalanceOfMaterialsBalance
  |WHERE
  |   BalanceOfMaterialsBalance.QuantityBalance < 0";
```

Now you only need to specify the query parameter, iterate through the query results, and display notifications about negative balance values.

9. Modify the posting procedure as shown in listing 14.40.

Listing 14.40. Posting() procedure (fragment)

```
// Checking for negative balances
Query3 = New Query;
Query3.TempTablesManager = TTManager;
Query3.Text = "SELECT
    |    BalanceOfMaterialsBalance.Material,
...
    |WHERE
    |    BalanceOfMaterialsBalance.QuantityBalance < 0";

Query3.SetParameter("Warehouse", Warehouse);

QueryResult = Query3.Execute();
SelectionDetailRecords = QueryResult.Select();
```

```
While SelectionDetailRecords.Next() Do

    Message = New UserMessage();

    Message.Text = String(- SelectionDetailRecords.QuantityBalance)
        + " units shortage for """ + SelectionDetailRecords.Material
        + """.";

    Message.Message();


    Cancel = True;

EndDo;
```

Let us review the added script lines.

In this check the warehouse specified in the document is passed to the query in the Warehouse parameter.

Then the query is executed to retrieve negative balances for materials that are stored in the temporary table and have the appropriate warehouse specified in the Warehouse parameter.

Then the selection of query records is iterated within the loop, and for each record with a negative balance a message is displayed.

Finally, the Cancel parameter of the document posting procedure is assigned the True value, which means that the document is not posted, the transaction is canceled, and the state of data modified in the process of posting is returned to the initial state (as it was before the document posting attempt).

Locking data that is read and modified during posting

It may seem that you completed the task. However, there is still a very important issue you have not taken care of.

Now the procedure is executed as follows:

1. The first query (Query) is executed. It generates a temporary table with the list of document materials.
2. The second query (Query2) is executed. It reads the cost and balance of materials available in the document tabular section.
3. The RegisterRecords.Write() line is executed. It writes the register records.
4. The third query (Query3) is executed. It performs the check for negative balances.

Note that from the beginning of the second query execution and till the end of the procedure you need to ensure that the product cost and balance values accessed in the procedure remain unchanged and are not even available for other transactions to read. Obviously the platform locks this data from modification but the lock is only applied when the register records are written.

But the following scenario is possible. During the second query execution you read that there are 2 units of some material available. Another transaction (another user) that intends to spend the materials also reads

that 2 units of the material are available. Then the platform writes the register records and locks the data. The other transaction waits for the platform to release the data. You post the document, withdraw 2 units of the material, and release the data. The other transaction also attempts to withdraw 2 units of the material but they are not available any more.

A similar situation can occur between steps 3 and 4. So the balance check might be performed incorrectly.

To avoid such collisions, you need to lock balance values from reading by other transactions even before the second query is executed. Before reading any data that you intend to modify, you also need to prevent other transactions from reading the data until your modifications are completed (or until the document posting is canceled).

In Designer mode

How does one do it? This is definitely a good question. Let us look at the Data lock control mode configuration property. This property has the Managed value (fig. 14.31).

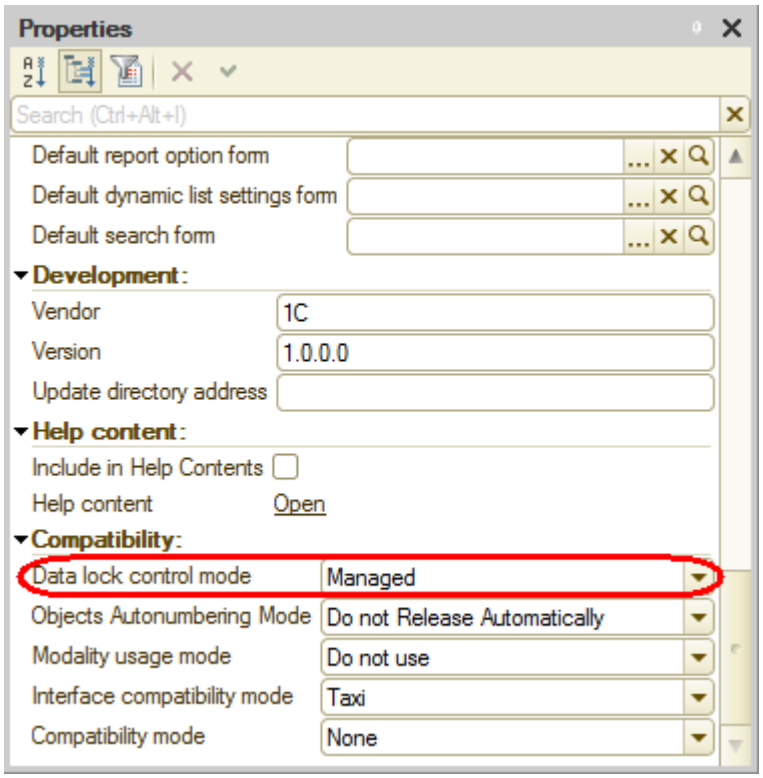


Fig. 14.31. Data lock control mode in configuration properties

This means that you have to use managed locks, which are set using 1C:Enterprise script tools.

You need to lock the data you intend to read and subsequently modify. This is implemented using the LockForUpdate property of register record sets. Let us use this property.

- Insert the lines shown in listing 14.41 before writing empty record sets.

Listing 14.41. Posting() procedure (fragment)

```
...
    |    ON DocumentMaterialsAndServices.MaterialOrService =
    |        BalanceOfMaterialsBalance.Material";


// Setting data locks for the CostOfMaterials and BalanceOfMaterials
registers
RegisterRecords.CostOfMaterials.LockForUpdate = True;
RegisterRecords.BalanceOfMaterials.LockForUpdate = True;


// Writing empty record sets to read balances without the data added
by this document
RegisterRecords.CostOfMaterials.Write();
RegisterRecords.BalanceOfMaterials.Write();


QueryResult = Query2.Execute();
```

A managed lock is applied at the moment of writing these record sets, right before the execution of the second query. This is exactly what we wanted to achieve.

Marking custom module regions

You finally finished editing the Posting() procedure in the Services document module. It is the only procedure in the module but it is quite long, especially from the novice developer perspective, and it does not fit a single screen, which makes it hard to read, understand, and debug.

The module editor provides a handy feature for marking custom text regions in modules. Then you can group and collapse these regions just like you collapse loops, conditions, procedures, and functions. You can name the regions to mark the module parts that have specific purposes.

For example, you can split your posting procedure into three logical parts: the first part generates the temporary table for storing the document materials, the second part calculates the material cost, and the third part performs the check for negative balances if the document is posted in real-time mode.

Mark these parts in the procedure text using the preprocessor instructions #Region <region name> and #EndRegion, as shown in listing 14.42.

Listing 14.42. Posting() procedure

```
Procedure Posting(Cancel, Mode)


    RegisterRecords.BalanceOfMaterials.Write = True;
    RegisterRecords.CostOfMaterials.Write = True;
    RegisterRecords.Sales.Write = True;


    // Creating temporary tables manager
```

```
TTManager = New TempTablesManager;

#Region DocumentMaterialsAndServices
Query = New Query;

// Specifying the temporary tables manager used by the query
Query.TempTablesManager = TTManager;

Query.Text =
   "SELECT
   |  ServicesMaterialsAndServices.MaterialOrService,
   |  ServicesMaterialsAndServices.MaterialOrService.MaterialServiceType
   |     AS MaterialServiceType,
   |  SUM(ServicesMaterialsAndServices.Quantity) AS QuantityInDocument,
   |  SUM(ServicesMaterialsAndServices.Total) AS TotalInDocument
   |INTO DocumentMaterialsAndServices
   |FROM
   |  Document.Services.MaterialsAndServices
   |    AS ServicesMaterialsAndServices
   |WHERE
   |  ServicesMaterialsAndServices.Ref = &Ref
   |GROUP BY
   |  ServicesMaterialsAndServices.MaterialOrService,
   |  ServicesMaterialsAndServices.MaterialOrService.MaterialServiceType";

Query.SetParameter("Ref", Ref);

QueryResult = Query.Execute();
#EndRegion

#Region RegisterRecords
Query2 = New Query;
Query2.TempTablesManager = TTManager;

Query2.Text = "SELECT
   |  DocumentMaterialsAndServices.MaterialOrService,
   |  DocumentMaterialsAndServices.MaterialServiceType,
   |  DocumentMaterialsAndServices.QuantityInDocument,
   |  DocumentMaterialsAndServices.TotalInDocument,
   |  ISNULL(CostOfMaterialsBalance.CostBalance, 0) AS Cost,
   |  ISNULL(BalanceOfMaterialsBalance.QuantityBalance, 0) AS Quantity
   |FROM
   |  DocumentMaterialsAndServices AS DocumentMaterialsAndServices
   |     LEFT JOIN AccumulationRegister.CostOfMaterials.Balance( ,
```

```
      |        Material IN (
      |        SELECT
      |          DocumentMaterialsAndServices.MaterialOrService
      |        FROM
      |          DocumentMaterialsAndServices)) AS CostOfMaterialsBalance
      |      ON DocumentMaterialsAndServices.MaterialOrService =
      |        CostOfMaterialsBalance.Material
      |      LEFT JOIN AccumulationRegister.BalanceOfMaterials.Balance( ,
      |        Material IN (
      |        SELECT
      |          DocumentMaterialsAndServices.MaterialOrService
      |        FROM
      |          DocumentMaterialsAndServices))
      |            AS BalanceOfMaterialsBalance
      |      ON DocumentMaterialsAndServices.MaterialOrService =
      |        BalanceOfMaterialsBalance.Material";

   // Setting data locks for the CostOfMaterials
   // and BalanceOfMaterials registers
   RegisterRecords.CostOfMaterials.LockForUpdate = True;
   RegisterRecords.BalanceOfMaterials.LockForUpdate = True;

   // Writing empty record sets to read balances without the data
   // added by this document
   RegisterRecords.CostOfMaterials.Write();
   RegisterRecords.BalanceOfMaterials.Write();

   QueryResult = Query2.Execute();

   // VT = QueryResult.Unload();

   SelectionDetailRecords = QueryResult.Select();

   While SelectionDetailRecords.Next() Do

      If SelectionDetailRecords.Quantity = 0 Then
         MaterialCost = 0;
      Else
         MaterialCost = SelectionDetailRecords.Cost
            / SelectionDetailRecords.Quantity;
      EndIf;

      If SelectionDetailRecords.MaterialServiceType =
         Enums.MaterialServiceTypes.Material Then
```

```
        // register BalanceOfMaterials Expense
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Warehouse = Warehouse;
        Record.Quantity = SelectionDetailRecords.QuantityInDocument;

        // register CostOfMaterials Expense
        Record = RegisterRecords.CostOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Expense;
        Record.Period = Date;
        Record.Material = SelectionDetailRecords.MaterialOrService;
        Record.Cost = SelectionDetailRecords.QuantityInDocument
            * MaterialCost;

    EndIf;

    // register Sales
    Record = RegisterRecords.Sales.Add();
    Record.Period = Date;
    Record.MaterialOrService = SelectionDetailRecords.MaterialOrService;
    Record.Customer = Customer;
    Record.Technician = Technician;
    Record.Quantity = SelectionDetailRecords.QuantityInDocument;
    Record.Revenue = SelectionDetailRecords.TotalInDocument;
    Record.Cost = SelectionDetailRecords.QuantityInDocument * MaterialCost;

EndDo;

RegisterRecords.Write();
#EndRegion

#Region BalanceCheck
If Mode = DocumentPostingMode.RealTime Then
    // Checking for negative balances
    Query3 = New Query;
    Query3.TempTablesManager = TTManager;
    Query3.Text = "SELECT
        |  BalanceOfMaterialsBalance.Material,
        |  BalanceOfMaterialsBalance.QuantityBalance
        |FROM
        |  AccumulationRegister.BalanceOfMaterials.Balance( , Material IN (
        |    SELECT
```

```
          |      DocumentMaterialsAndServices.MaterialOrService
          |    FROM
          |      DocumentMaterialsAndServices)
          |    AND Warehouse = &Warehouse) AS BalanceOfMaterialsBalance
          |WHERE
          |  BalanceOfMaterialsBalance.QuantityBalance < 0";


      Query3.SetParameter("Warehouse", Warehouse);


      QueryResult = Query3.Execute();
      SelectionDetailRecords = QueryResult.Select();


      While SelectionDetailRecords.Next() Do
         Message = New UserMessage();
         Message.Text = String(- SelectionDetailRecords.QuantityBalance)
            + "units shortage for """ + SelectionDetailRecords.Material
            + """ with """ + SelectionDetailRecords.PropertySet
            + """ property set.";
            Message.Message();


         Cancel = True;


      EndDo;


   EndIf;
   #EndRegion


EndProcedure
```

Now you can collapse the marked regions (fig. 14.32).

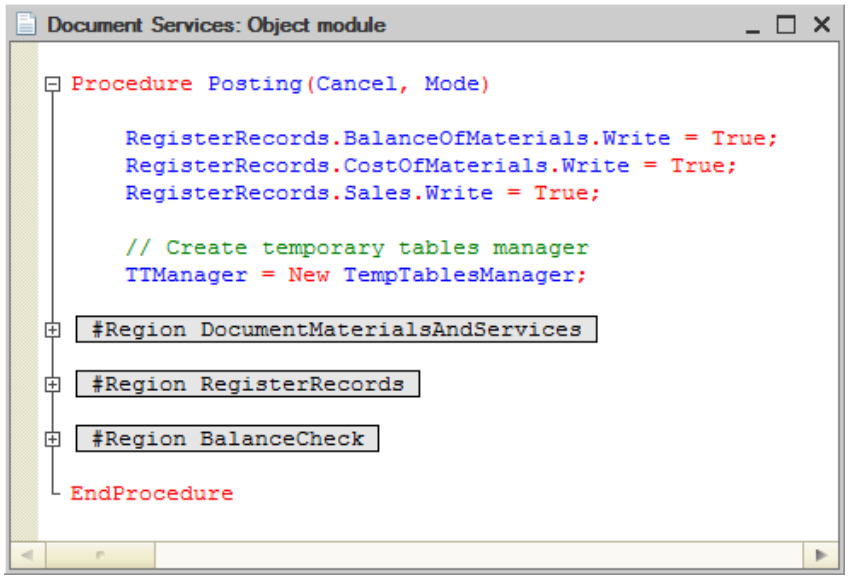


Fig. 14.32. Marked regions in a module

Then you can expand only the region you are working with. This makes the procedure text compact and easier to read.

In this example you split a procedure into three logical parts. But you might often want to do the contrary: when a module contains many procedures, you can group similar procedures. For example, you can name the procedure groups API, EventHandlers, and InternalProceduresAndFunctions, where each name clearly describes the group purpose.

You can create nested regions or include regions into other collapsible structures. When marking regions, ensure that they do not overlap each other or other collapsible structures because overlapping regions cannot be grouped.

In 1C:Enterprise mode

Run 1C:Enterprise in the debug mode and test the new Posting event handler by reposting all of the Services documents.

Nothing has changed from the user perspective, but from the data access perspective the document posting is arranged better and more efficiently.

Theory. Understanding cache

In section "Theory. Referential data usage specifics" on page 375 we have mentioned that the platform has a cache that stores object data read from the database. Now we will discuss this feature in more detail.

1C:Enterprise caches database objects that are read from the database using object operations.

When you access object attributes using a reference, the platform accesses the object cache that is stored in RAM.

The object cache consists of two parts: the transactional cache and the standard cache. Which cache is used depends on whether the reference is made in the context of a transaction or not (fig. 14.33).

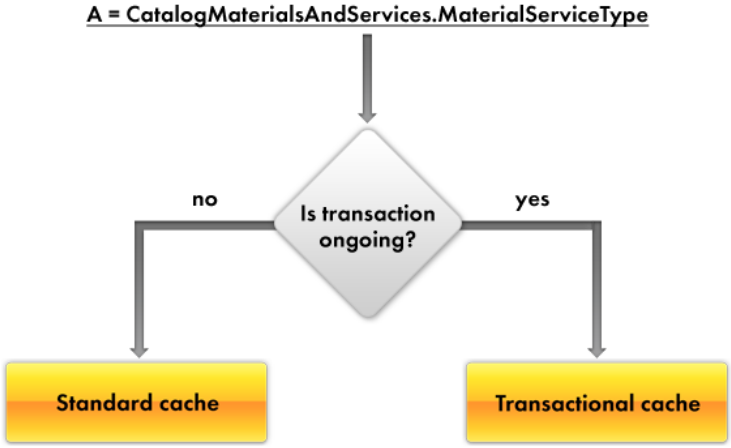


Fig. 14.33. Object cache

All the data stored in the cache is intended to be read-only. This means that all the data that is read by reference is handled using the object cache, while writing is handled using 1C:Enterprise object operations.

Standard cache

If the required data is not found in the standard cache, the object data is read from the database and stored to the cache. In this case a reference to the database object serves as the unique identifier for the cache. Therefore, the data for each read object can exist in the cache in one of two forms: either all the object data, or an object presentation.

This means that if you access the cache to get an object presentation, and the cache contains data for the specified reference, that data is taken from the cache (if the entire object is in the cache, the required presentation is taken from the object data).

If the cache does not contain data for the specified reference, only the fields required to generate the object presentation are read from the database.

If you access the cache to get an object attribute, and the cache contains data for the specified reference, what comes next depends on the cache contents.

If the cache contains the entire object, the attribute value is read from the cache. If the cache contains object presentation, the presentation is deleted from the cache and then all the object data is read from the database and stored to the cache. If the cache does not contain data for the specified reference, all of the object fields are read from the database.

The read data remains in the cache until one of the following events occurs:

- The read data is forced out of the cache by read data that belongs to other objects (cache overflow).
- The next time you access the cache it turns out that the read data has been modified in the database.
- The 20-minute timeout expires.

All the read data is kept in a sequential queue and, since the cache size is limited, the oldest data in the cache is replaced by the most recent data.

Upon subsequent calls for data that has already been read into the cache, the platform checks the elapsed time since the data entered the cache.

If the call is within 20 seconds after the data arrived, the data is considered valid. If the elapsed time is longer than 20 seconds, the platform checks that the version of the data stored in the cache matches the version in the database.

If the data versions do not match (the data in the database has been changed), the data is deleted from the cache and reread from the database. The next 20-second data validity period is counted from that point.

In addition to the listed events, read data is deleted from the cache 20 minutes after it was last read from the database.

So if you execute two operators in sequence (listing 14.43) where MaterialOrService is a reference to a catalog item, the execution of the

second line takes much less time because the execution of the first line includes a database query while the execution of the second line only includes reading data from RAM (from the object cache).

Listing 14.43. Operator sequence

```
A = MaterialOrService.Description;
B = MaterialOrService.MaterialServiceType;
```

Transactional cache

If data is accessed in the context of a transaction, the request is redirected to the transactional cache. In 1C:Enterprise all the operations related to data modifications in the database are executed within transactions. For example, a document posting event handler is executed within a transaction.

A transaction is an indivisible sequence of data manipulations that brings the database from one integral state to a new one. So if for some reason one of the transaction operations cannot be accomplished, the database returns to the state it had before the transaction (transaction rollback).

The transactional cache is essentially the same sequential queue as the standard cache. The difference is that all the data in the transactional cache is definitely valid (up-to-date).

When the platform reads data to the transactional cache, it locks the data in the database, so there is no way it can be changed until the transaction is complete.

The transactional cache stores read data until it is replaced by subsequently read data or until the transaction is completed. Upon completion of a transaction, the cache is cleared while the actions performed at the moment of clearing depend on the completion status of the transaction.

If the transaction is successfully completed (committed), the data of all the objects in the transactional cache is moved to the standard cache and the transactional cache is cleared (fig. 14.34).

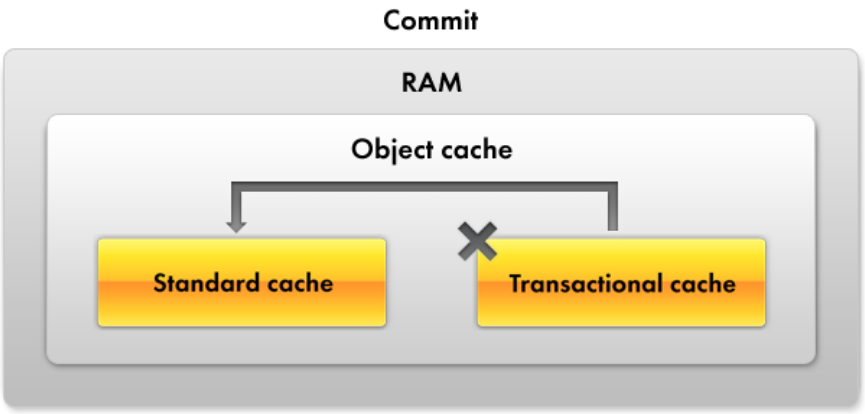


Fig. 14.34. Successful completion of a transaction

If the changes are canceled (rolled back), they are simply cleared from the transactional cache (fig. 14.35).

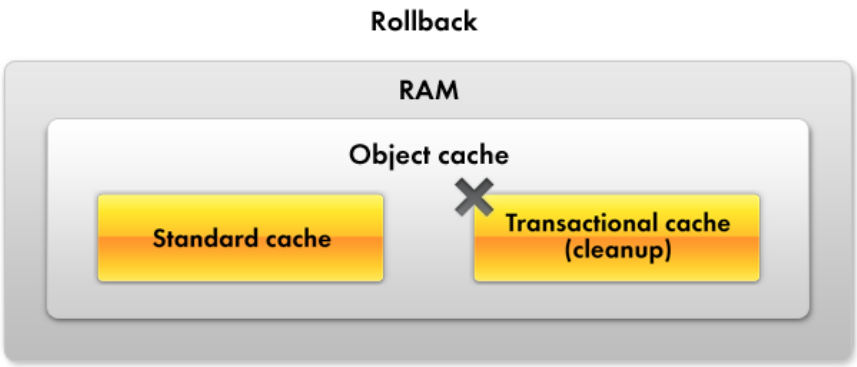


Fig. 14.35. Clearing transactional cache upon rollback

Quiz

- How does 1C:Enterprise access referential data?
- How is object cache used?
- Why is it better to use queries to access large amounts of infobase data?
- What is a point in time?
- What is the difference between real-time document posting and regular posting?
- What is a real-time timestamp?
- How does one use a query to get accumulation register balances?
- What should one pay particular attention to when defining the parameters for virtual query tables?
- Why balance checks are not required for regular posting?
- What are the meaning and purpose of temporary tables?
- What is a temporary tables manager?
- How can one use temporary tables in virtual table parameters? How is it accomplished?
- How does one apply data locks using 1C:Enterprise script?
- How does one preview query results in Debugger?
- How does one mark custom regions in a module?

Lesson 15 (2:50). Charts of characteristic types

In this lesson you will study the Chart of characteristic types configuration object. You will learn how to use this object to expand the functionality of your configuration.

The task at hand

Your next task is to provide users with the option to add custom material descriptions and perform accounting operations based on that user-defined descriptions.

The user-defined descriptions should work as follows: multiple custom characteristics are allowed for each material (such as color or manufacturer). When materials arrive, users can enter specific values for the relevant characteristics (for example, when power supply cables arrive, users can enter that the cables are white, with a cross-section of 2.5 sq mm, and when rubber tubing is delivered, users can enter that it is black and manufactured by Fagumit).

Then you will always be able to find out how many white-colored materials you have or how many black rubber tubing units you have used.

Since you do not know in advance which characteristics users will need to describe a material, you have to provide them with the option to create any characteristics, and, most importantly, specify value types for these characteristics. And when a user enters a characteristic, they select only from the values of the specified type.

You can implement this feature using the Chart of characteristic types configuration object.

Understanding charts of characteristic types

The Chart of characteristic types configuration object is intended to describe data storage structures for user-defined characteristics. Based on a Chart of characteristic types, the platform creates a set of database tables for storing available characteristic types and their value types.

In its nature a chart of characteristic types is very similar to a catalog but it has a more specific goal: it only contains data that describes characteristic types of some database object.

A chart of characteristic types consists of characteristic types. Each characteristic type is defined by its name and value type.

A developer and, most importantly, a user can enter as many characteristic types as they need (fig. 15.1).

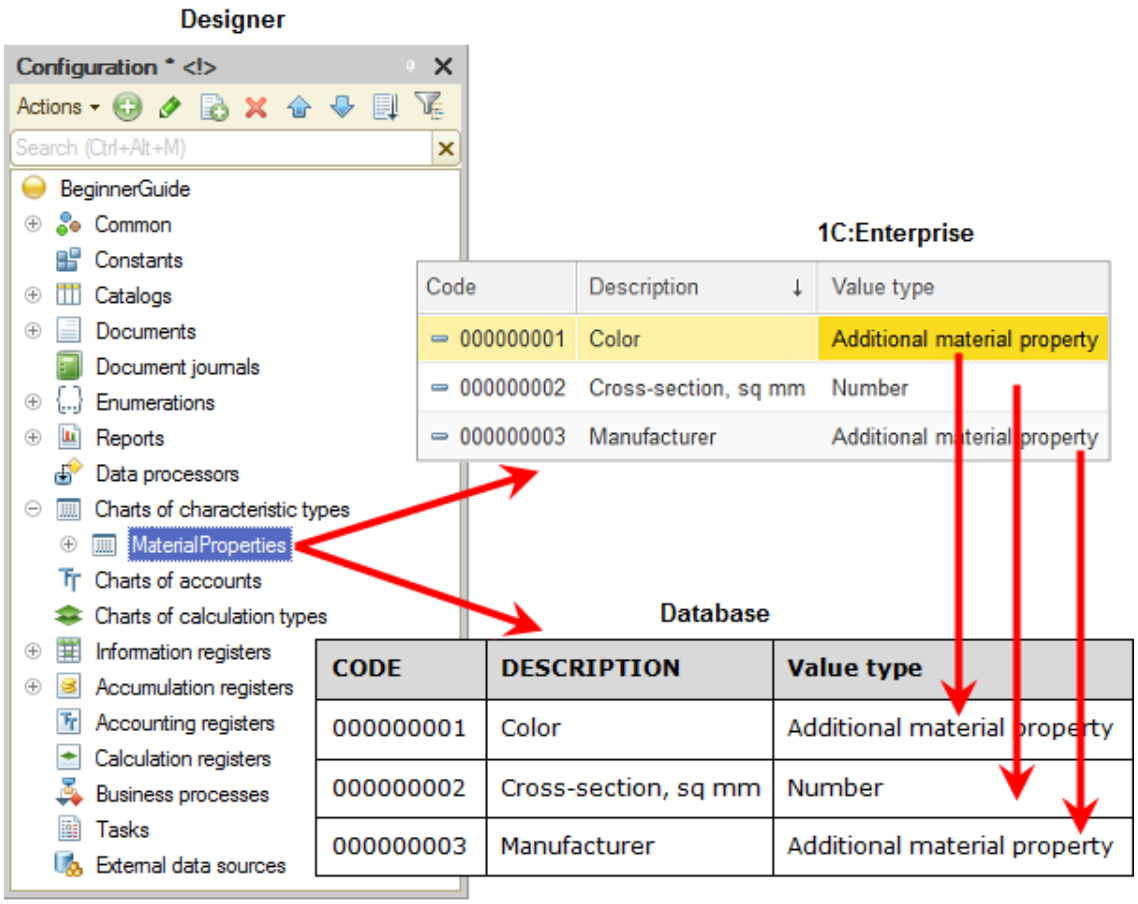


Fig. 15.1. A chart of characteristic types in Designer mode, in 1C:Enterprise mode, and in the database

The Chart of characteristic types configuration object has the Characteristic value type property. Developers use this property to define the composite data type that includes all the value types that can be assigned to characteristics (fig. 15.2).

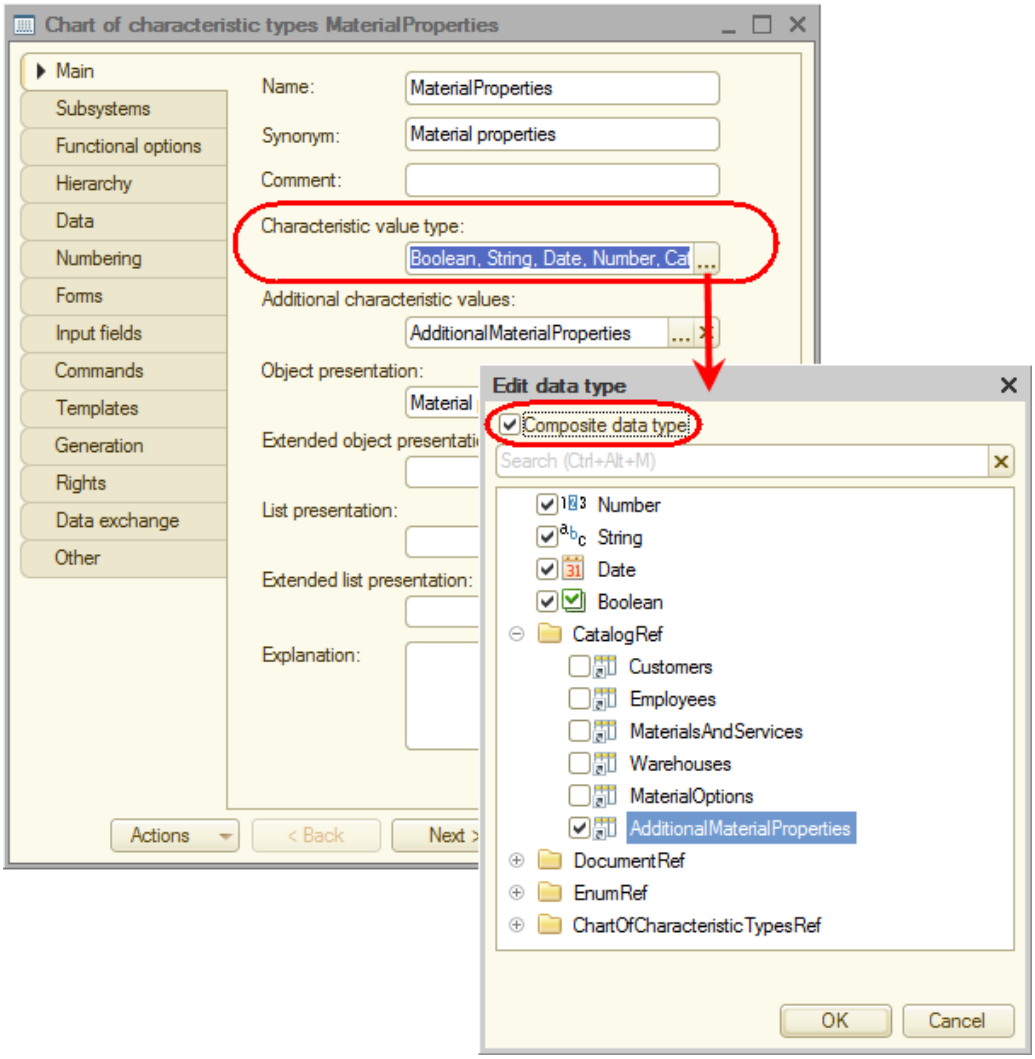


Fig. 15.2. Characteristic value type property

In addition to that, the data types available in the configuration might not be sufficient for a user. For example, they want to include product color in their accounting data but the Color catalog is not available in the configuration.

To handle this scenario, a developer can create an auxiliary catalog and select this catalog in the Additional characteristic values property of the chart of characteristic types (fig. 15.3).

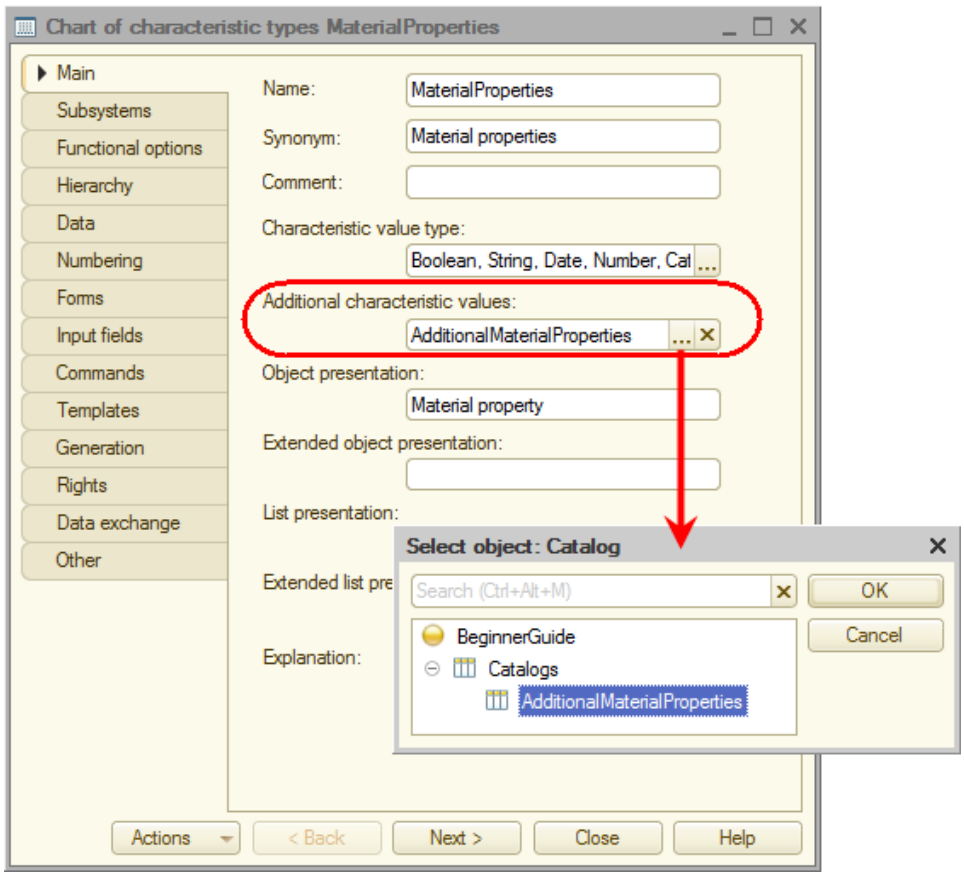


Fig. 15.3. Additional characteristic values property

Then a user can create a new characteristic type named Color and define available values for the color in the catalog of additional characteristic values.

Note that this catalog is subordinate to the chart of characteristic types. So if in the future a user wants to create a characteristic type named Scent and define values for this characteristic, they will create it in the same catalog of additional characteristics and the scent values will not be mixed with the color values.

Learn more! For details on the structure of 1C:Enterprise script objects intended for chart of characteristic types operations, see section "Quick developer reference. Charts of characteristic types" on page 851.

A chart of characteristic types has no internal predefined methods of linking a characteristic type to the object that it describes. It merely enables developers and users to define a certain set of characteristics and specify their types.

Depending on the applied solution at hand, developers decide for themselves how to maintain the correspondence between a specific characteristic type or characteristic value and a specific database object.

The example provided in this lesson is not simple in terms of its implementation. So we will first explain the logical relation between the objects that are used in the example.

Logical relation between the objects

To implement this example, you need three new configuration objects.

The first object is Chart of characteristic types. It will store characteristic types that can generally be used to describe materials.

The second object is a catalog subordinate to the MaterialsAndServices catalog. The catalog items describe material options, where each option has a fixed set of characteristic values.

The third object is an information register that stores the relationships between specific characteristic values and a certain material option (fig. 15.4).

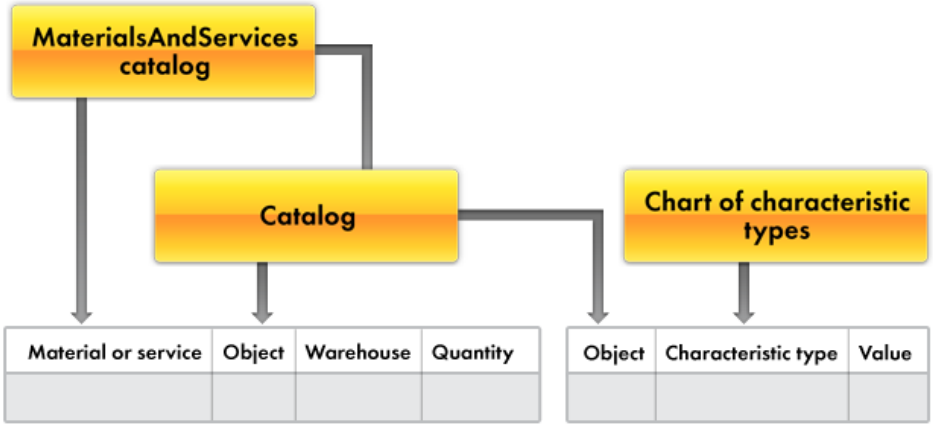


Fig. 15.4. Logical relation between the objects

Using this logical structure, you can describe each lot of delivered materials using any number of characteristic types, since the relationships are stored in the information register.

Besides, you gain the option to perform accounting based on characteristic types by adding one more dimension to the accumulation registers. This dimension will store references to items of the catalog that is subordinate to the MaterialsAndServices catalog (see fig. 15.4).

So to find out the balance of materials that have some specific value of a certain characteristic, all you need to do is select all the items of the subordinate catalog that have this characteristic value from the information register, and then get the accumulation register balances for their owners.

Creating new configuration objects

In Designer mode

As we mentioned above, you need to create some new configuration objects:

- MaterialOptions catalog for describing material options
- AdditionalMaterialProperties catalog for describing custom characteristic value types

- MaterialProperties chart of characteristic types for creating characteristic types
- MaterialPropertyValues information register for storing values of characteristic types for various materials options

Let us begin.

1. In Designer, create a Catalog object named *MaterialOptions*.

   Let us specify that the new catalog is subordinate to the MaterialsAndServices catalog.
2. On the **Owners** tab, add the **MaterialsAndServices** catalog to the list of owners.
3. Create a Catalog object named *AdditionalMaterialProperties*.
4. Create a Chart of characteristic types object named *MaterialProperties*.
5. In the **Characteristic value type** field, click the **Select** ... button and define the composite data type as follows (fig. 15.5):
   - Number, length: 15, precision: 3
   - String, length: 25
   - Date
   - Boolean
   - CatalogRef.AdditionalMaterialProperties

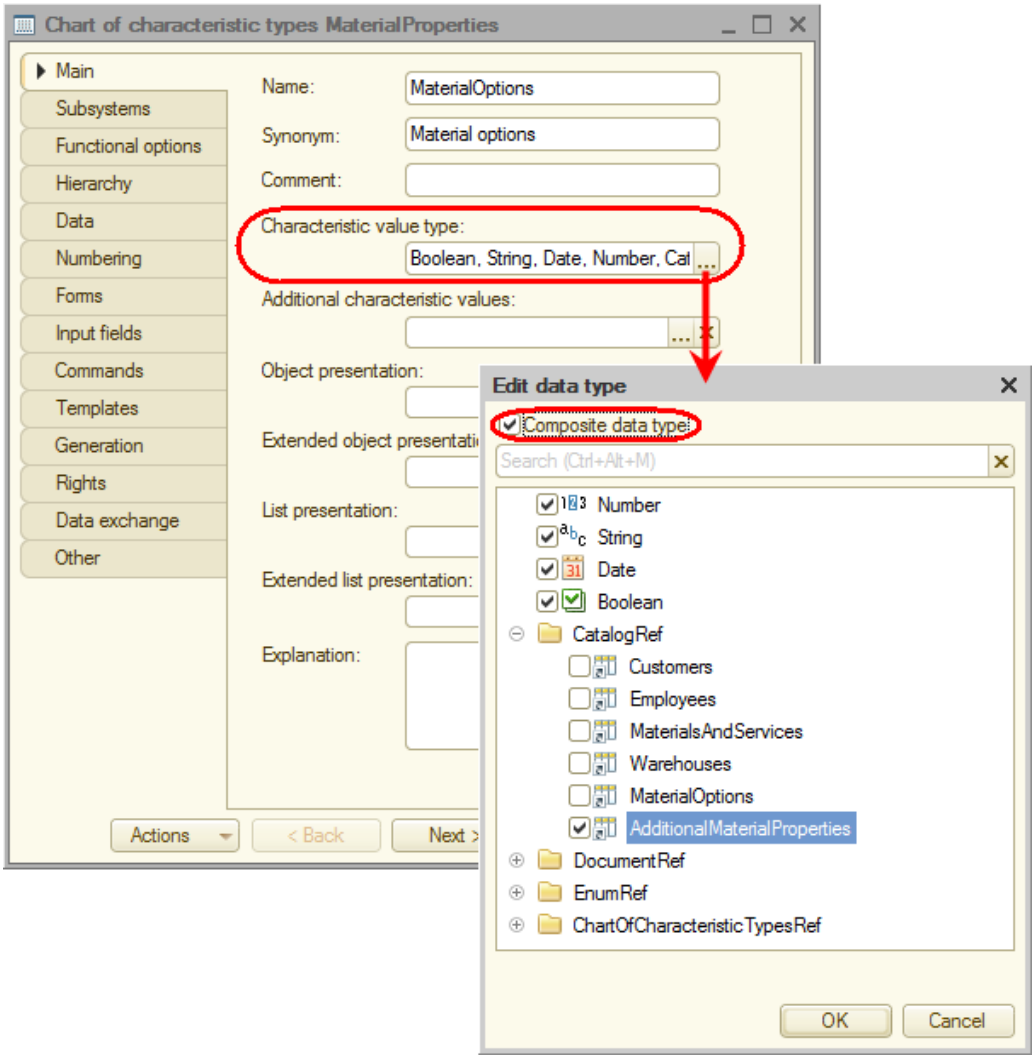


Fig. 15.5. Specifying a composite data type for the characteristic value type of a chart of characteristic types

6. For the **AdditionalMaterialProperties** catalog, specify the **MaterialProperties** catalog as the owner (fig. 15.6).

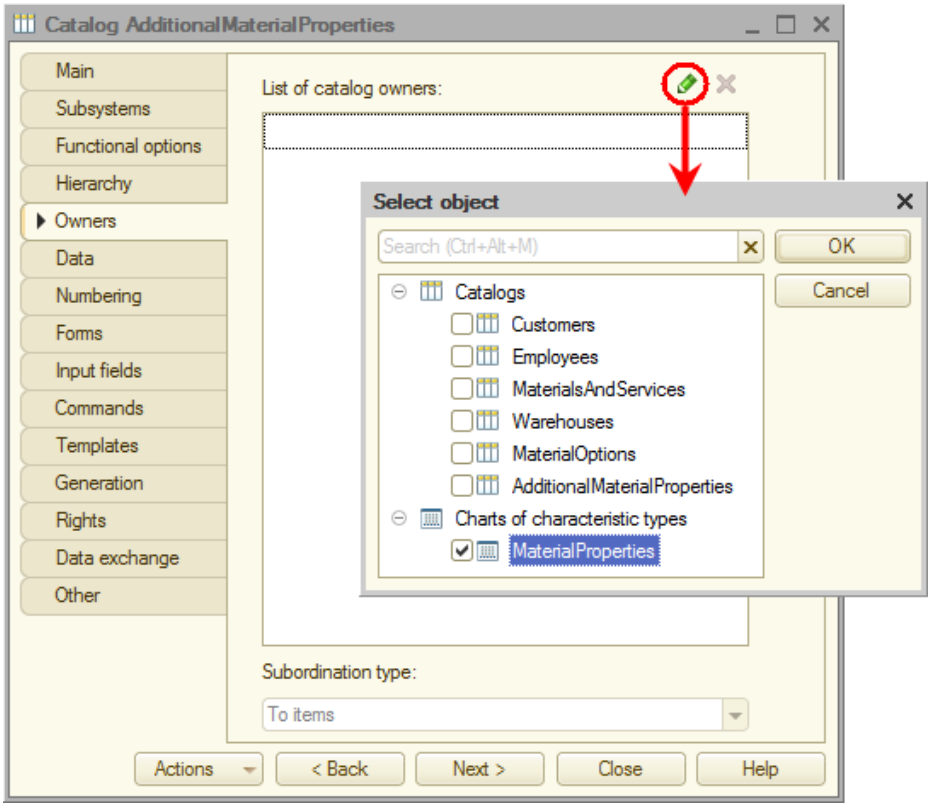


Fig. 15.6. Specifying catalog owner

7. Specify that the additional characteristic values for the chart of characteristic types are stored in the **AdditionalMaterialProperties** catalog (fig. 15.7).

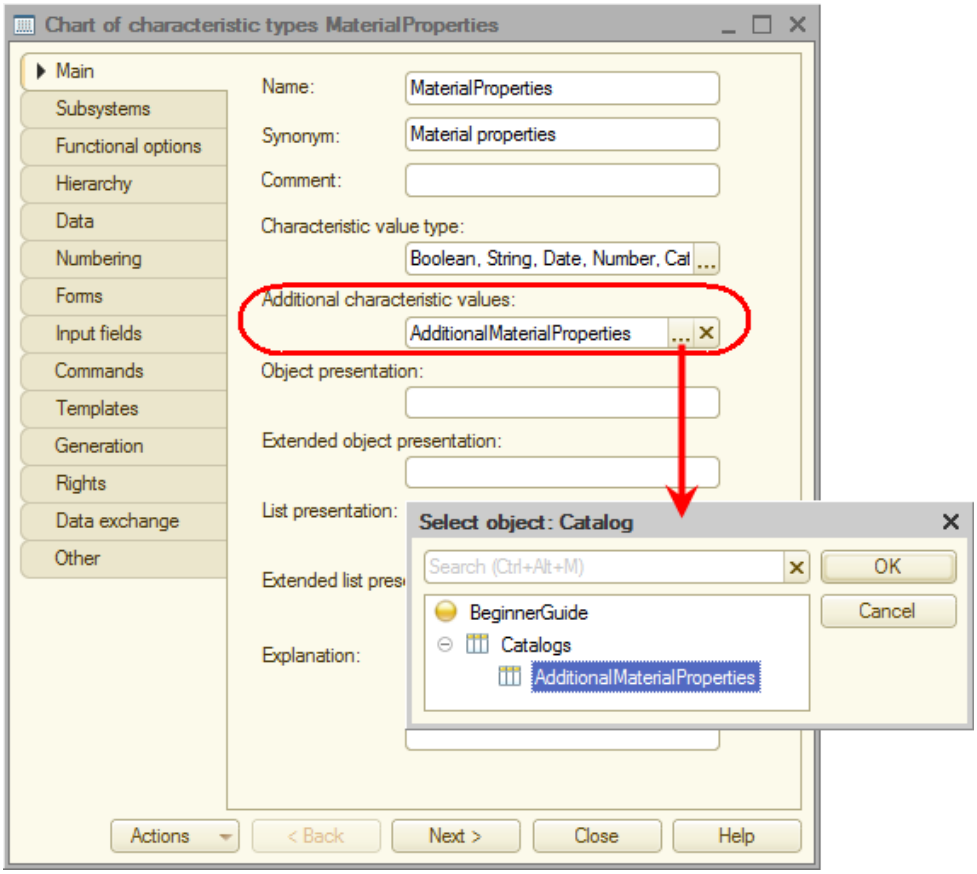


Fig. 15.7. Chart of characteristic types editor window

8. Create an InformationRegister object named *MaterialPropertyValues*.
9. On the **Data** tab, create the register dimensions:
   - **PropertySet**. Master, type: CatalogRef.MaterialOptions
   - **PropertyType**. Type: ChartOfCharacteristicTypesRef.MaterialProperties
10. Create a register resource (fig. 15.8):
    - **Value**. Type: Characteristic.MaterialProperties.

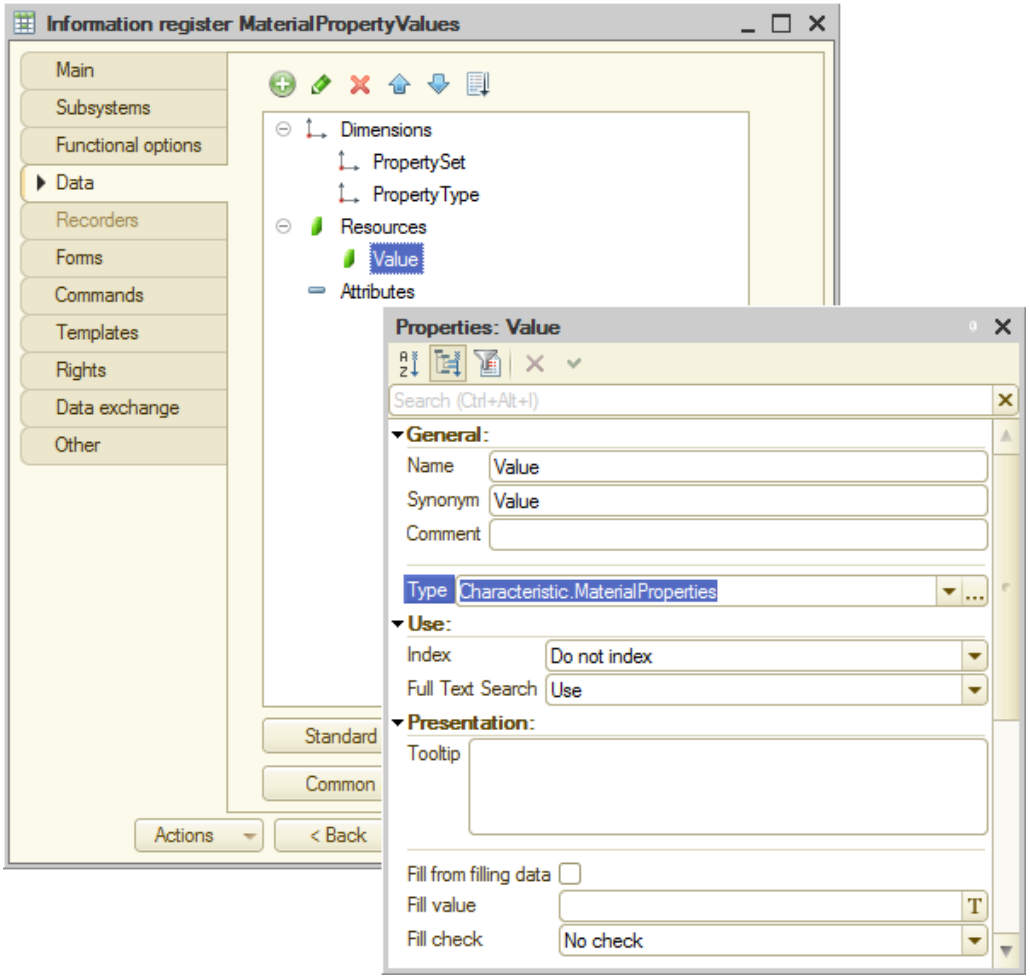


Fig. 15.8. Information register editor

Note that you can define a value type for a register resource as Characteristic.<name>. It is the composite data type that is defined in the value type for the corresponding chart of characteristic types. In other words, a register resource can have a value of any type among those defined in the value type of the chart of characteristic types.

11. In the **Link by type** property of this resource, select the **PropertyType** register dimension.

    A link by type ensures that the value types that can be entered in this field match the characteristic type selected in the **PropertyType** field.

12. For the **Choice parameters links** property, click the **Select** ... button and move the **PropertyType** register dimension from the list of available attributes to the list of parameters.

    Specifying a value for the Choice parameters links property ensures that when values stored in the Additional material properties catalog are being selected, only values related to the selected characteristic are suggested instead of all the values in the catalog (fig. 15.9).

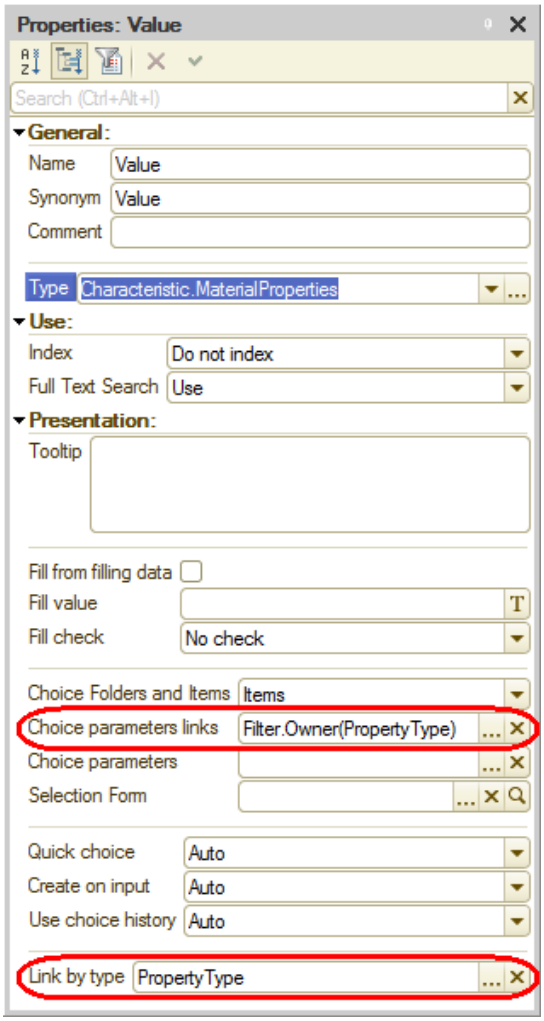


Fig. 15.9. Information register resource properties

Description of material option characteristics

Finally, for the MaterialOptions catalog, let us specify where the material option properties are stored and how one can retrieve their values. The platform will use the description to generate reports and various dynamic lists based on material options.

1. Right-click the **MaterialOptions** catalog and then click **Characteristics** (fig. 15.10).

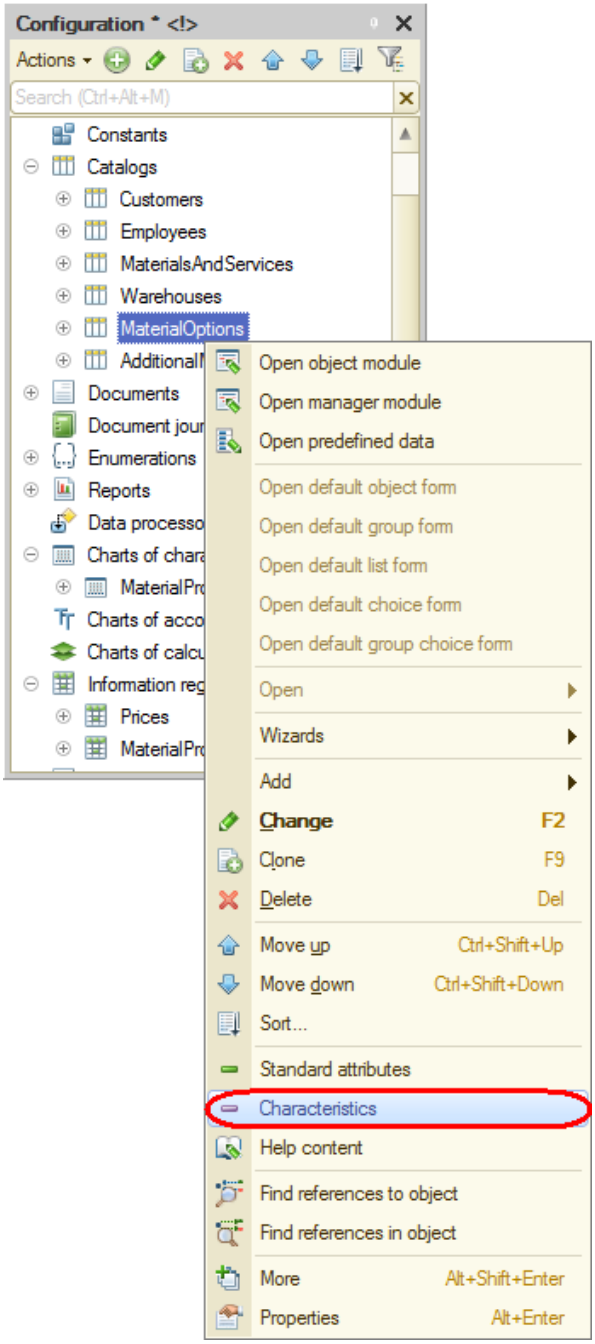


Fig. 15.10. Switching to the characteristics of the MaterialOptions catalog

This opens the **Additional characteristics of metadata object** dialog box.

2. Click the **Add** button in the command bar.

   This adds a new record.

3. Select the **MaterialProperties** chart of characteristic types as a source of characteristics.

   The platform automatically determines that the key field is the Ref field of this configuration object (fig. 15.11).

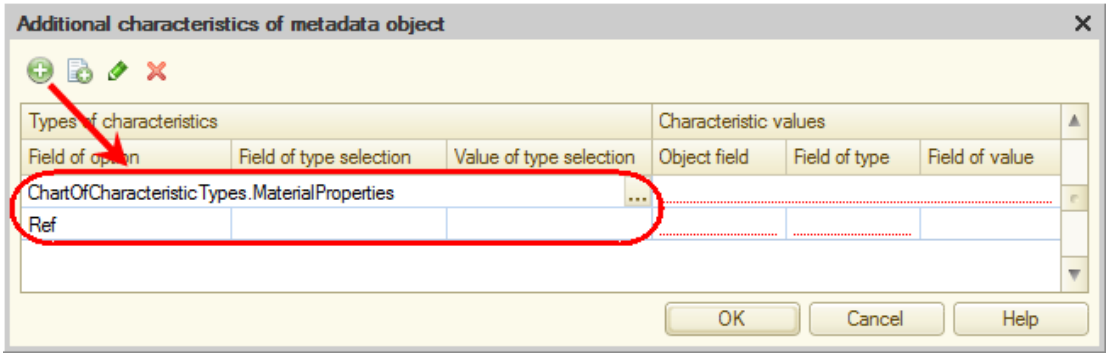


Fig. 15.11. Description of the source of the characteristic types

4. Leave the **Field of type selection** and **Value of type selection** fields empty.

   This scenario does not require filling these fields.

   Let us proceed to describing where the property values are stored.

5. Select the **MaterialPropertyValues** information register as a source of characteristic values.

   The platform automatically determines that in this register the PropertySet dimension is the object field and the PropertyType dimension is the type field. The only thing you have to specify manually is where the property values are stored.

6. In the **Field of value** column, select **Value**.

   The resulting characteristic description for the MaterialOptions catalog should look as shown in fig. 15.12.

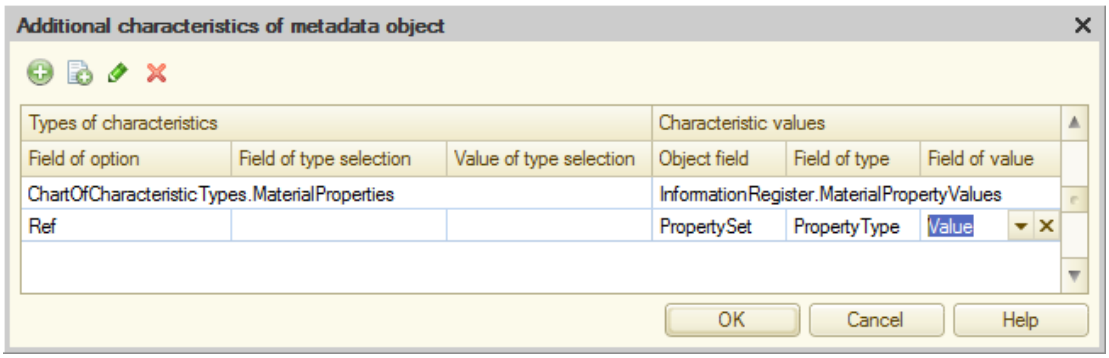


Fig. 15.12. Description of characteristics for the MaterialOptions catalog

Modifying configuration objects

So you have created new configuration objects and defined their basic properties required to implement your task.

But you will further see that you are not fully happy with some properties. In fact, you can never foresee everything in advance in the development process. Often some shortages only become obvious at run time. So when you review some preliminary results in 1C:Enterprise mode, it is important to be able to understand shortages and correct them on the fly.

This is why during this lesson we will demonstrate a backward development process. This is a valuable experience that we think will be very useful for you.

- Run 1C:Enterprise in the debug mode.

  The purpose of this run is checking the interaction between the logically connected configuration objects: MaterialsAndServices catalog, MaterialOptions catalog, MaterialProperties chart of characteristic types, and MaterialPropertyValues information register.

  Note that you have not specified any subsystems for these objects. This is because displaying these objects outside of the logical connections between them has no particular sense. Since you have defined the catalog owners, the master information register dimension, and so on, the required objects are automatically added to the navigation panels of their owners as subordinate data.

  So ignore the system message stating that the created objects are not included into any subsystems.

The MaterialOptions catalog

In 1C:Enterprise mode

Now you need to create the property sets and the characteristics that form the property sets for specific materials. As we mentioned earlier, the property sets are stored in the MaterialOptions catalog, which is subordinate to the MaterialsAndServices catalog.

Let us create a set of properties for the Power supply cable item.

1. In the **Inventory** section, open the **Materials and services** catalog, expand the **Materials** group, expand the **Miscellaneous** group, and double-click the **Power supply cable** item.

   Since the MaterialsAndServices catalog is the owner of the MaterialOptions catalog, the form navigation panel contains the link for switching to the subordinate list. This list contains only the items related to the MaterialsAndServices item being edited.

2. Click **Material options**.

   This opens the list of material property sets (fig. 15.13).

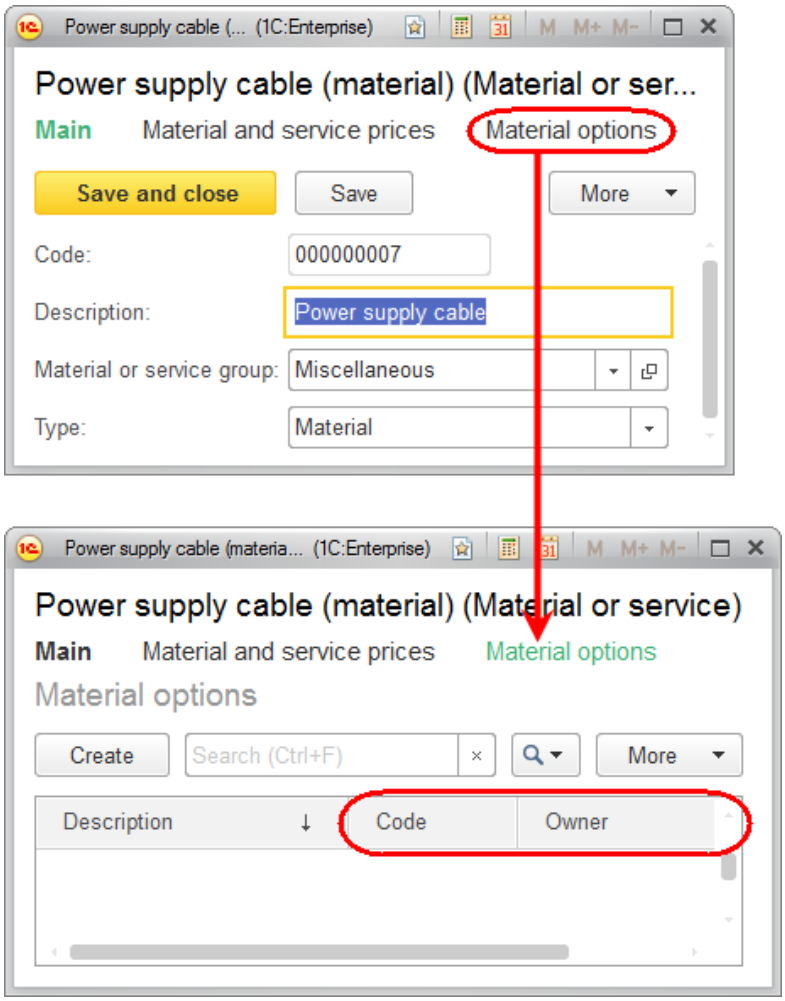


Fig. 15.13. List of material options

We are not quite happy with the material options form that is opened because the Code and Owner columns are obviously not needed.

The material option code is generated automatically for each new material and is meaningless for a user.

The material option owner name is displayed in the form title and is a nonsense in the list as well.

To hide these columns, you have to create a MaterialOptions list form and specify that during the form opening it checks where it is opened from (you can check this using the value in the Filter form parameter).

If filter by owner is in effect (the form is opened from the materials and services list), let us hide the Code and Owner columns.

But if the form is opened from any other location, these columns may be needed so it would not be correct to simply remove them from the form.

Since the form is created on the server, you should do it in the OnCreateAtServer form event handler.

In Designer mode

Let us eliminate the shortages of the list form.

1. In Designer, open the editor of the **MaterialOptions** catalog configuration object.
2. On the **Forms** tab, in the **List** field, click the **Open** button (fig. 15.14).

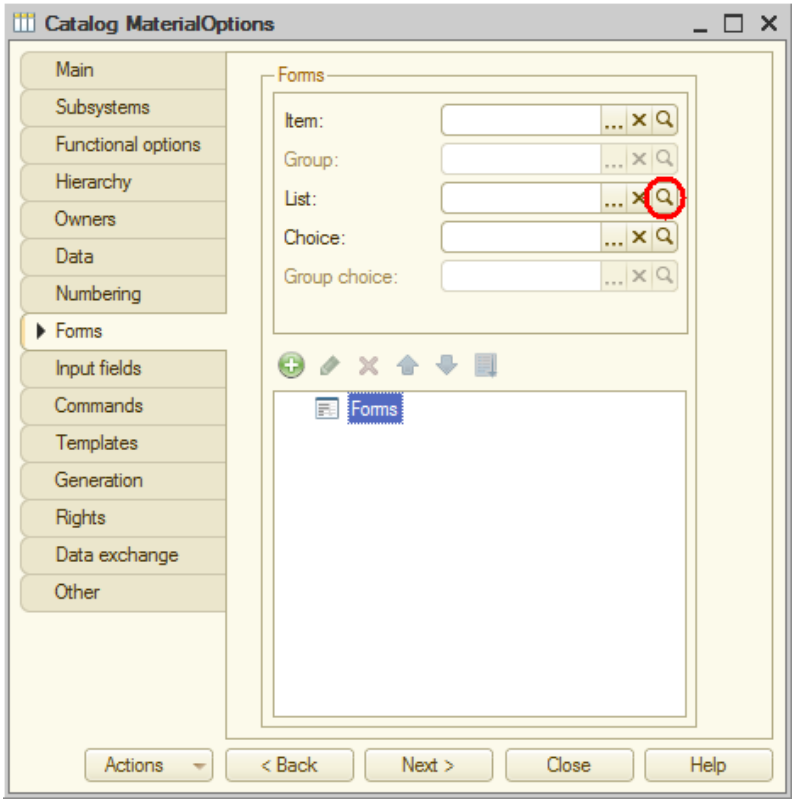


Fig. 15.14. Creating a default list form

3. In the wizard window that is opened, click **Finish**.

   This creates the default list form.

   Unlike the automatically generated form, the form created by the wizard does not include the Owner field. So your task is getting even simpler as you only need to hide the Code field.
4. In the upper left pane of the form editor window, on the **Elements** tab, double-click the **Form** item.

   You have to double-click this item because the event that you need belongs to the entire form. This opens the form property palette.
5. Scroll through the list of form properties to locate the **OnCreateAtServer** event and click its **Open** button (fig. 15.15).

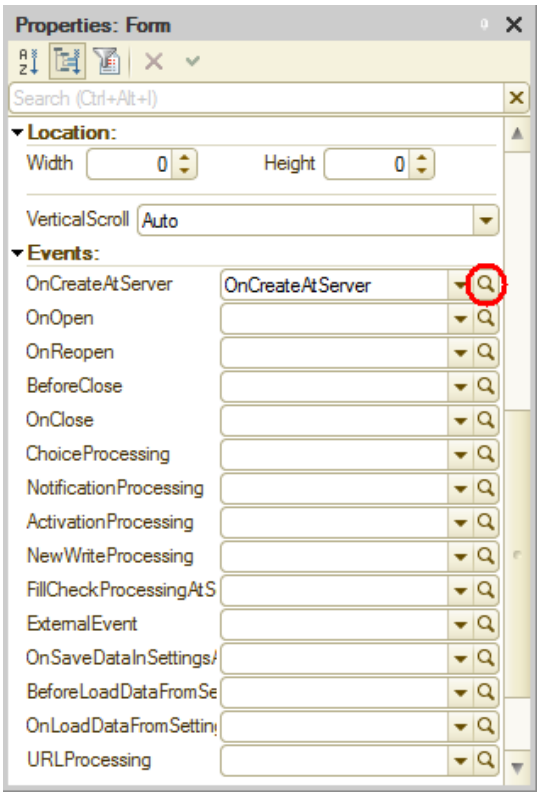


Fig. 15.15. Creating OnCreateAtServer form event handler

This creates the OnCreateAtServer form event handler in the form module.

6. Add the script shown in listing 15.1 to the event handler.

Listing 15.1. OnCreateAtServer() form event handler

```
If Parameters.Filter.Property("Owner") Then
    Items.Code.Visible = False;
EndIf;
```

Let us review this code.

Parameters is a property of the managed form where the module belongs. Using this property, you get the object that contains a collection of form parameters.

The Filter item of this collection is called by its name. Using the Property() method of the filter item structure, you define whether a filter by the Owner field is applied.

If the filter is applied, you set the Code field visibility to False, which hides the field. Here Items is a managed form property that provides access to all form items.

In 1C:Enterprise mode

Let us test the changes.

1. Run 1C:Enterprise in the debug mode.

   The material options form should look as shown in fig. 15.16.

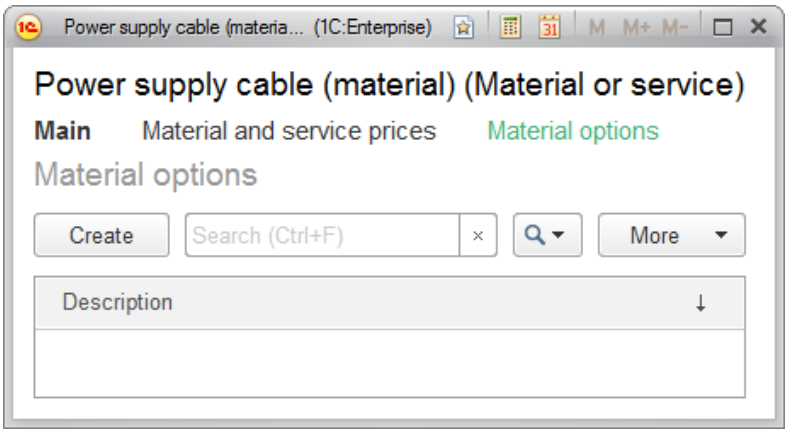


Fig. 15.16. List of material options

You see that you have achieved the goal: where in the fig. 15.13 you had three columns, now you only have one Description column.

2. Click **Create** to create a set of properties for the material.

   This opens the MaterialOptions catalog item form (fig. 15.17).

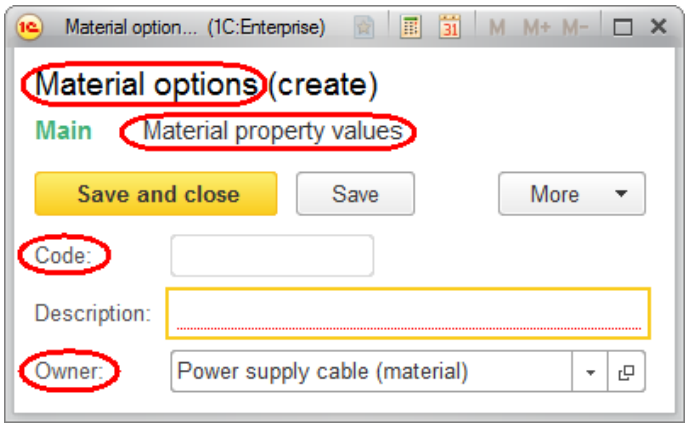


Fig. 15.17. MaterialOptions catalog item form

This form is generated automatically. But there are still some shortcomings:

- The form title should be in the singular.
- The Code and Owner fields are not needed.
- The command for switching to subordinate data should be renamed into something more clear.

Let us return to Designer to correct this.

In Designer mode

Let us rename the form title so that it is clear that the form is intended for creating a single material option.

1. In Designer, open the editor of the **MaterialOptions** catalog configuration object.
2. On the **Main** tab, in the **Object** presentation field, enter *Material option* (fig. 15.18).

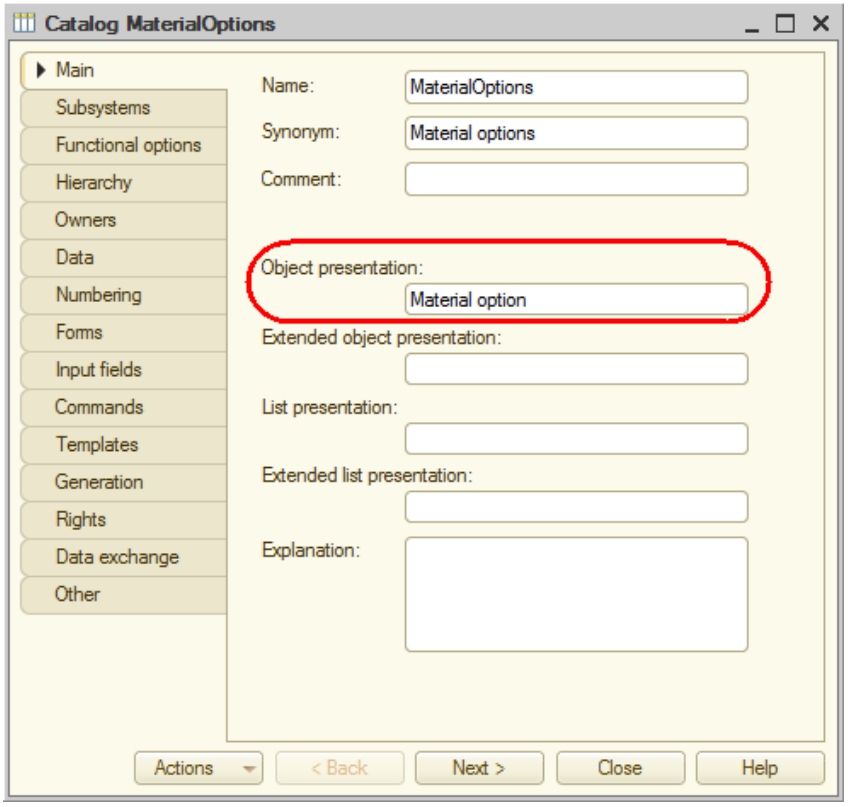


Fig. 15.18. Specifying object presentation

This property will be used in the 1C:Enterprise interface as the title of the catalog item form.

Then you need to remove the Code and Owner fields from the form.

3. On the **Forms** tab, in the **Item** field, click the **Open** button and click **Finish**.

   This creates the default item form.

4. In the upper left pane of the form editor window, on the **Elements** tab, click **Code** and click **Delete**, and then repeat this procedure for **Owner** (fig. 15.19).

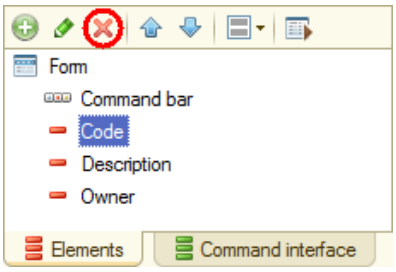


Fig. 15.19. Deleting form controls

So the only catalog attribute remaining in the form is Description. Let us change its presentation.

5. In the **MaterialOptions** catalog configuration object editor, on the **Data** tab, click **Standard attributes** and double-click **Description**.
6. In the property palette that is opened, in the **Synonym** field, enter *Name*.

Finally, the title of the Material option form and the title of its subordinate data Material property values do not look good together (see fig. 15.17). This subordinate data consists of records of the MaterialPropertyValues register that can be opened from the item form.

7. Open the editor of the **MaterialPropertyValues** information register configuration object and then, on the **Main** tab, in the **List presentation** field, enter *Material option details* (fig. 15.20).

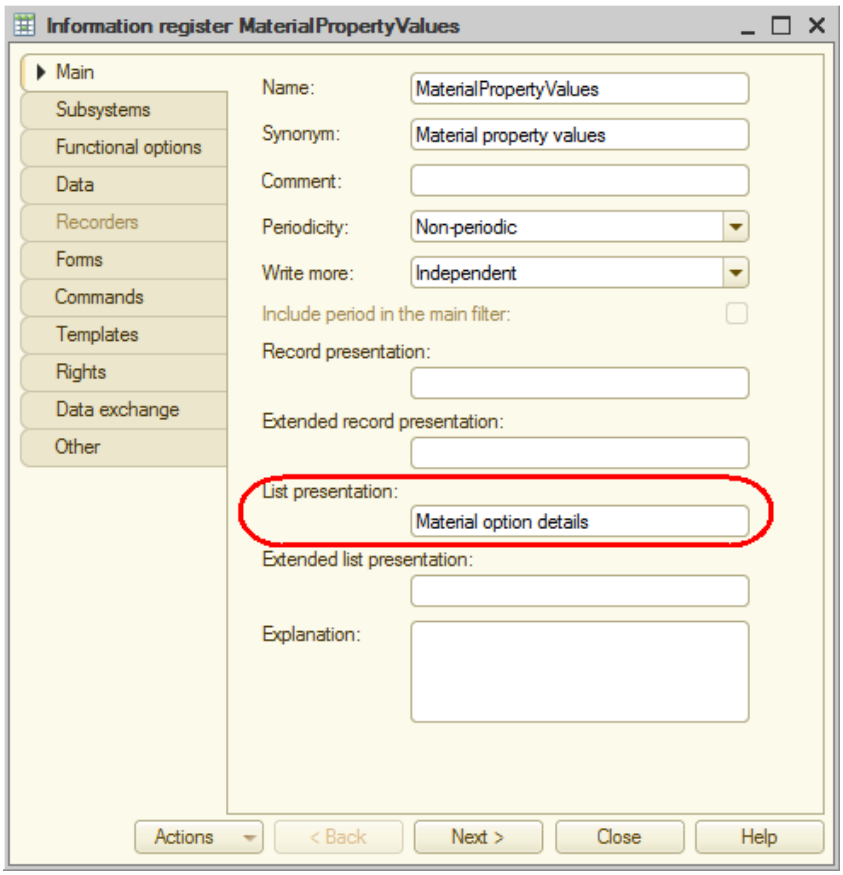


Fig. 15.20. Specifying list presentation for a register

This value will be used in the 1C:Enterprise interface as the title of the register list form.

In 1C:Enterprise mode

Let us test the changes.

1. Run 1C:Enterprise in the debug mode.
2. In the **Inventory** section, open the **Materials and services** catalog, expand the **Materials** group, expand the **Miscellaneous** group, and double-click the **Power supply cable** item.
3. In the item form, click **Material options**.

   This opens the list of material property sets, which is empty for now.
4. Click **Create**.

   Now we are happy with everything about the material options form.

The MaterialPropertyValues information register

In 1C:Enterprise mode

Let us create a material option.

1. In the **Description** field, enter *White cables* (fig. 15.21).

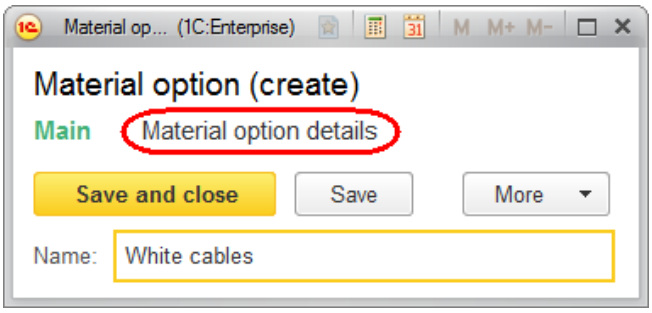


Fig. 15.21. Material options catalog item form

2. Click **Material option details**.

   This opens the form where you can edit the material option content.

   Tip. The reader feedback shows that one might encounter the following issue at that point: the Material option details command, which opens the related MaterialPropertyValues register records, is not available in the form navigation panel. This is probably because you forgot to set the Master property for the PropertySet register dimension, which has the CatalogRef.MaterialOptions type.

   Once a register dimension becomes a master dimension, the link that opens the list of register records that have the reference to the current material option in their PropertySet dimension is added to the form navigation panel.

   Since the material option has not been recorded yet, you are prompted to record the data.

3. Confirm that you want to record the data (fig. 15.22).

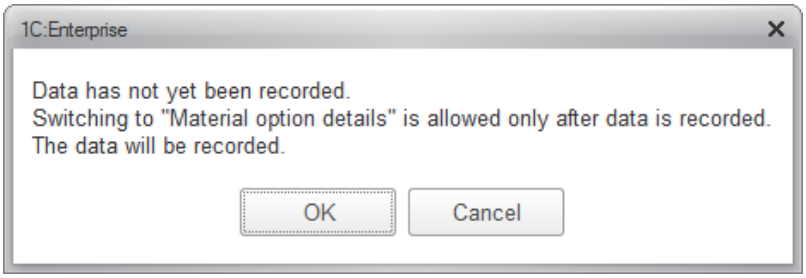


Fig. 15.22. Confirmation of data recording

   This opens the automatically generated list form of the MaterialPropertyValues register (fig. 15.23).

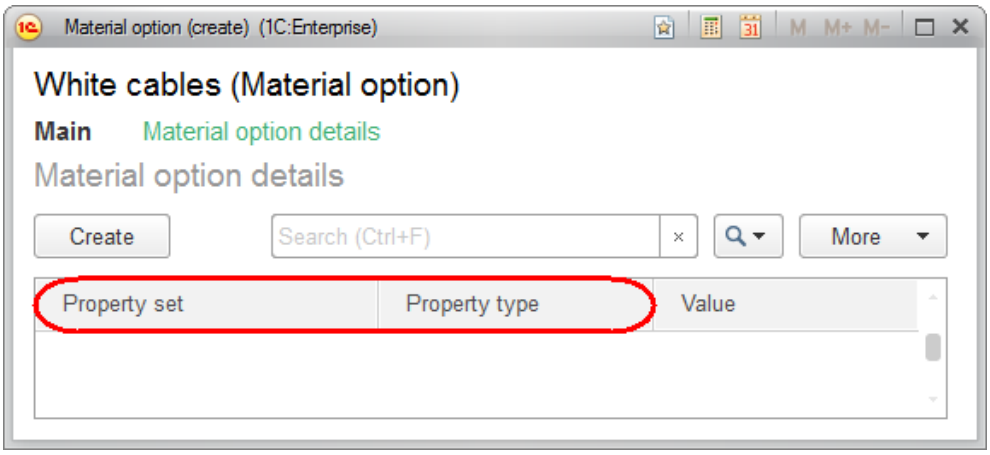


Fig. 15.23. Material option details register list form

We are not happy with everything in the form either:

- The Property type column should better be renamed.
- The Property set column is not needed.

Let us return to Designer and eliminate the shortages of the list form.

In Designer mode

Let us rename the Property type column to Property.

1. In Designer, open the editor of the **MaterialPropertyValues** information register configuration object.
2. On the **Data** tab, open the property palette for the **PropertyType** dimension and set its **Synonym** to *Property* (fig. 15.24).

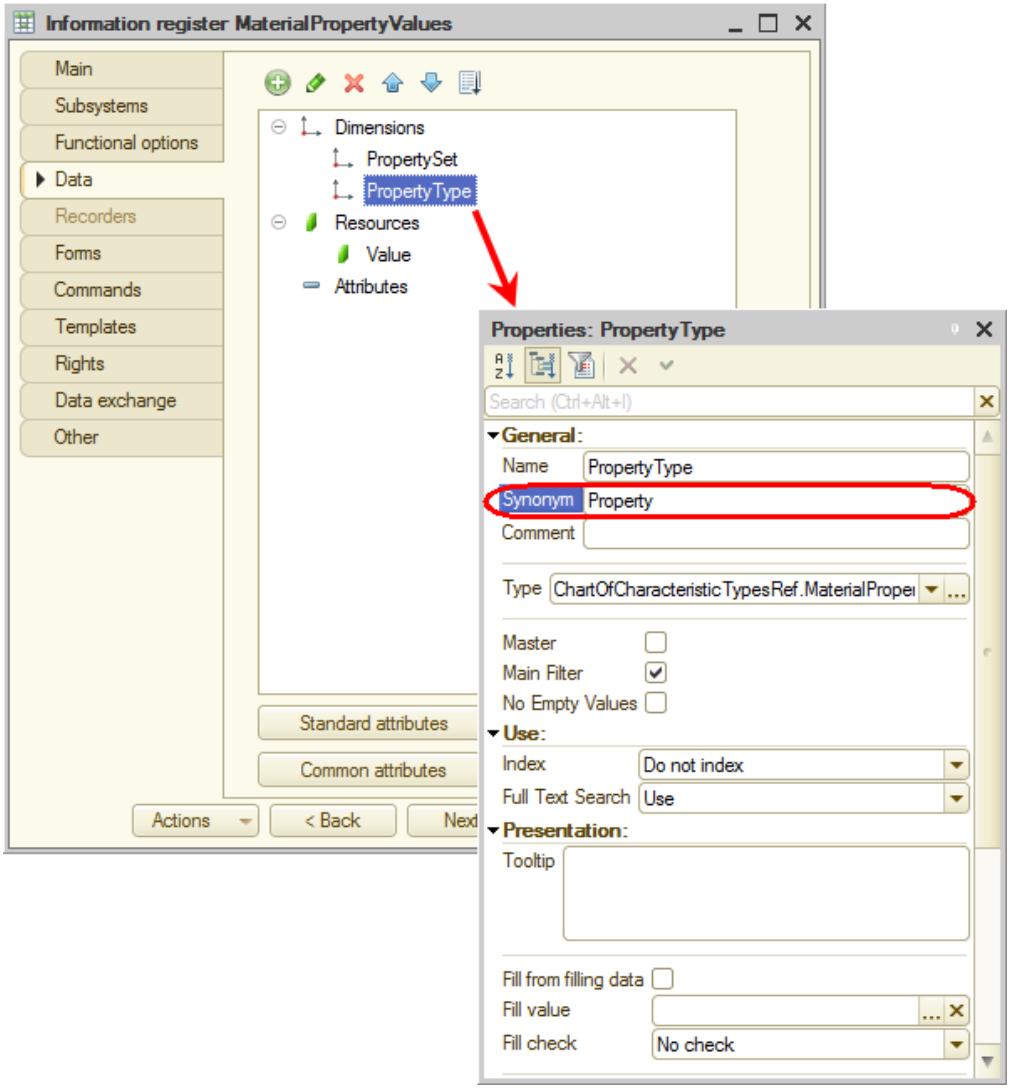


Fig. 15.24. Specifying register dimension synonym

Since the register has the PropertySet master dimension with the CatalogRef.MaterialOptions type, the Property set field is not needed because the owner of this property set is displayed in the form title.

Therefore, in the OnCreateAtServer event handler of the register list form, let us hide the Property set column for scenarios when the form is opened with a filter by this field applied (in other words, when the register list form is opened from the item form of the MaterialOptions catalog).

3. On the **Forms** tab, in the **List** field, click the **Open** button and then click **Finish**.

   This creates the default list form.

4. Create the OnCreateAtServer event handler and add the script shown in listing 15.2 to the handler.

Listing 15.2. OnCreateAtServer() form event handler

```
If Parameters.Filter.Property("PropertySet") Then
    Items.PropertySet.Visible = False;
EndIf;
```

This script is similar to that provided in listing 15.1, so it does not require any further comments.

In 1C:Enterprise mode

Let us test the changes.

1. Run 1C:Enterprise in the debug mode and open the **Material option** form (fig. 15.25).

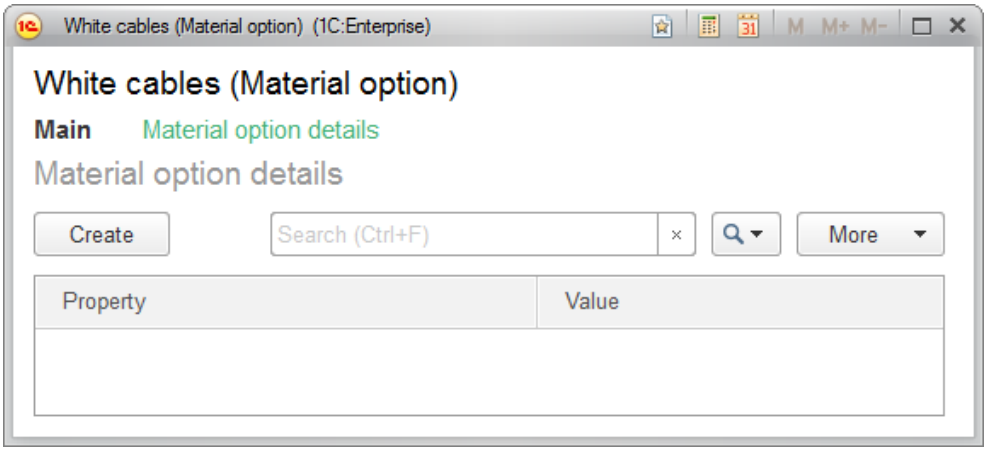


Fig. 15.25. Material option details register list form

Let us add a material option details record.

2. Click **Create**.

   This opens an automatically generated MaterialPropertyValues register record form (fig. 15.26).

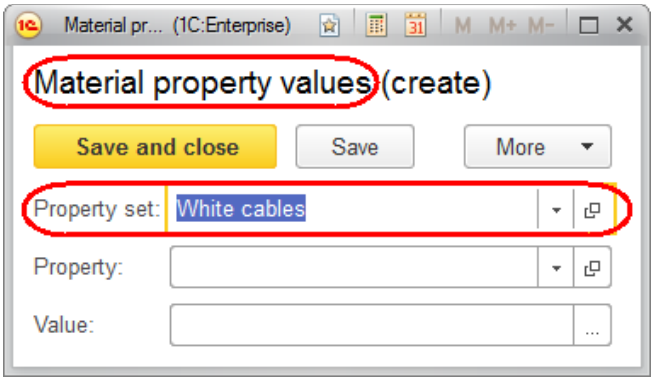


Fig. 15.26. Material property values register record form

But there are still some shortcomings:

- The form title should be in the singular.
- The Property set column is not needed.

Let us return to Designer to correct this.

In Designer mode

Let us rename the form title so that it is clear that the form is intended for creating a single property in material option details and for specifying the property value.

1. In Designer, open the editor of the **MaterialPropertyValues** information register configuration object.
2. On the **Main** tab, in the **Record presentation** field, enter *Property and value* (fig. 15.27).

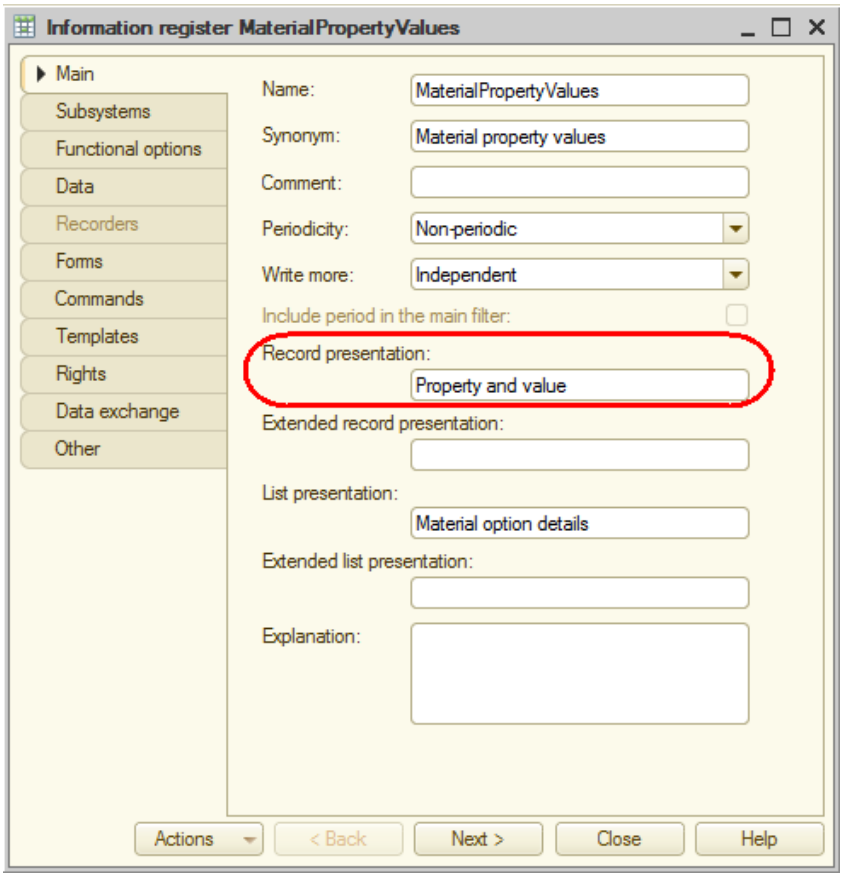


Fig. 15.27. Specifying register record presentation

This property will be used in 1C:Enterprise interface as the title of the register record form.

Then let us remove the PropertySet field from the form.

3. On the **Forms** tab, in the **Record** field, click the **Open** button and then click **Finish**.

   This creates the default record form.

4. In the upper left pane of the form editor window, click the **Elements** tab, then click **PropertySet**, and click **Delete**.

In 1C:Enterprise mode

Let us test the changes.

- Run 1C:Enterprise in the debug mode and open the MaterialPropertyValues register record form.

  The form should look as shown in fig. 15.28.

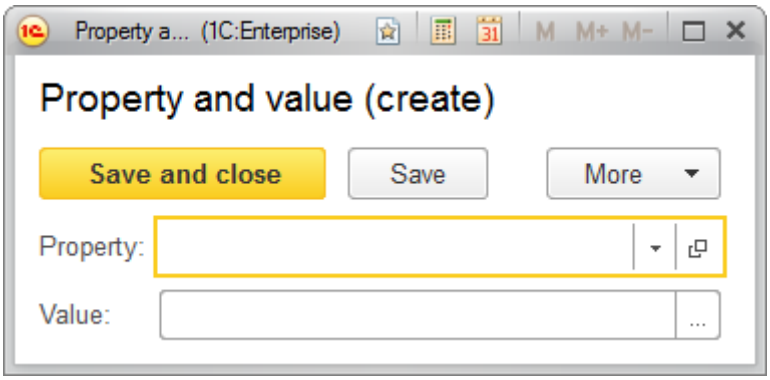


Fig. 15.28. Material option details register record form

Creating material characteristic types

In 1C:Enterprise mode

Let us create various material options in 1C:Enterprise mode.

1. In the **Inventory** section, open the **Materials and services** catalog, expand the **Materials** group, expand the **Miscellaneous** group, and double-click the **Power supply cable** item.
2. In the material item form, click **Material options**.
3. In the material options form, double-click **White cables**.
4. In the material option form, click **Material option details**.

   This opens an empty list of material option details.
5. Click **Create**.

   In the form that is opened (see fig. 15.28), let us create a Color property with the White value.
6. In the **Property** field, click the **Select from list** button and then, in the drop-down list, click **Show all**.

   The PropertyType (Property) dimension of the MaterialPropertyValues register has the ChartOfCharacteristicTypesRef.MaterialProperties type. This is why the choice form of this chart of characteristic types is opened. The list of characteristics is empty for now.
7. Click **Create**.

8. In the chart of characteristic types item form, in the **Description** field, enter *Color*, and in the **Value type** field, leave the default value *Additional material properties* (fig. 15.29).

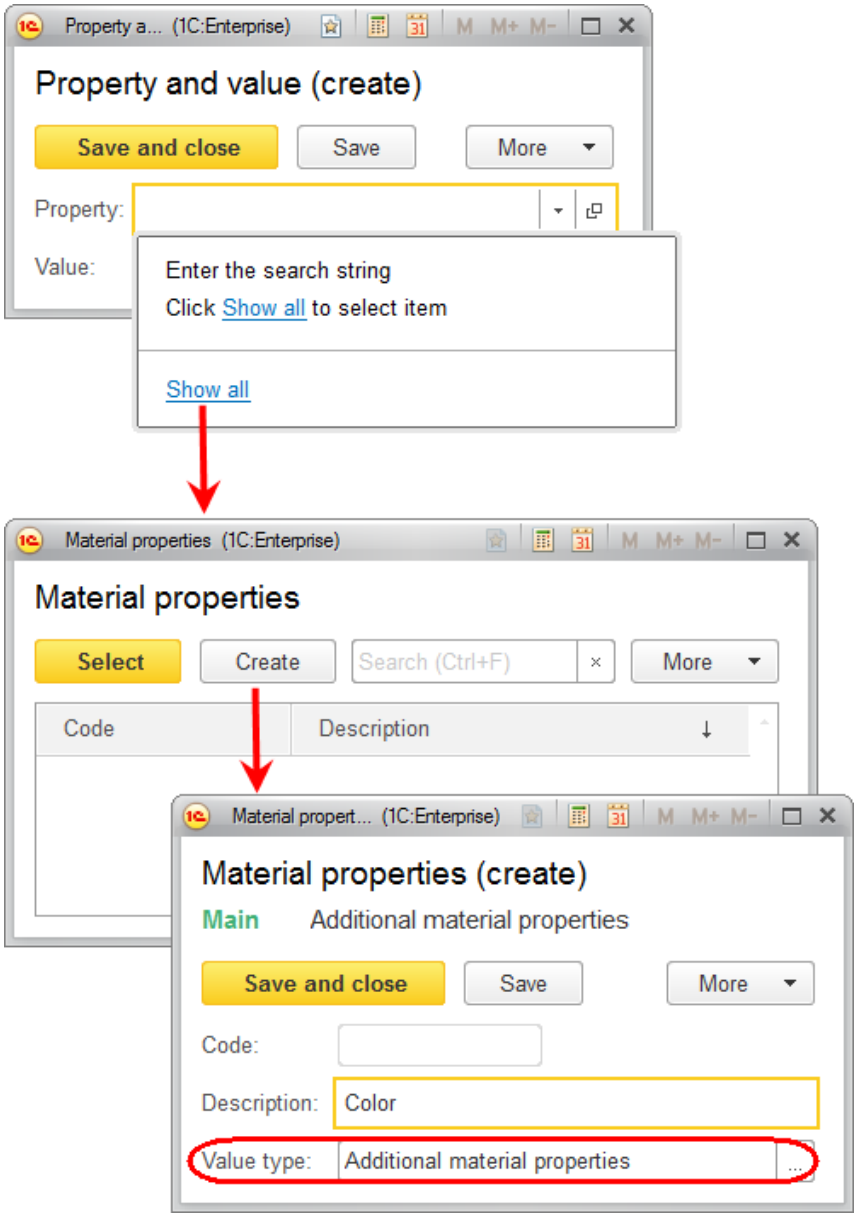


Fig. 15.29. Creating a characteristic type in a chart of characteristic types

Note that the item form of a chart of characteristic types (see fig. 15.29) and the item form of the additional material properties catalog (see fig. 15.30) also have the code field that is not needed. And the form titles should better be in the singular.

This tutorial does not describe adjusting these forms because we have already demonstrated similar actions. You can do it yourself using the steps described for the MaterialOptions catalog item form.

9. Click **Save and close**.

   The created characteristic type is added to the choice form of the chart of characteristic types.

10. Click **Select**.

This returns you to the material option details record form titled **Property and value**.

11. In the **Value** field, click the **Select from list** button and then click the **Create** button.

    The Value resource of the MaterialPropertyValues register has the Characteristic.MaterialProperties type. This is a composite data type that is described in the Characteristic value type property of the MaterialProperties chart of characteristic types.

    Since you selected the CatalogRef.AdditionalMaterialProperties type for the Color characteristic type, the choice form of this catalog is opened.

12. In the additional material properties item form, in the **Description** field, enter *White*, and in the **Owner** field, leave the default value Color (fig. 15.30).

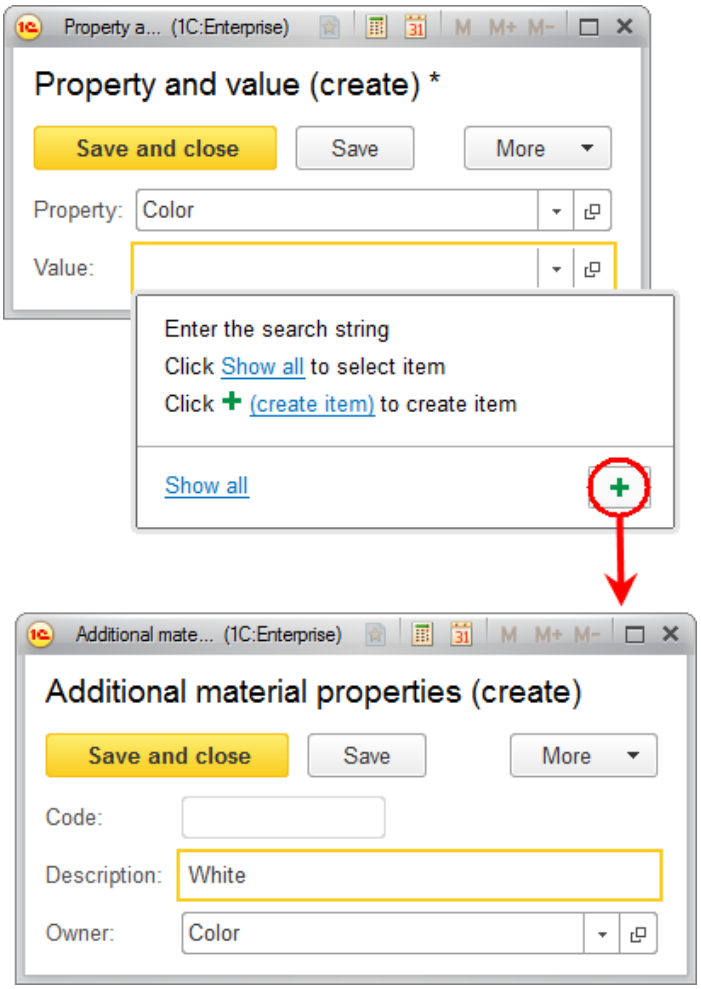


Fig. 15.30. Creating additional material properties

13. Click **Save and close**.

    This returns you to the material option details record form titled **Property and value**, which now contains the Color property with the White value (fig. 15.31).

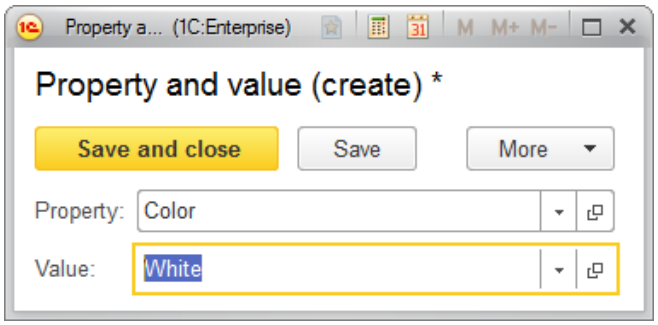


Fig. 15.31. Property and value in material option details

14. Click **Save and close**.

    This returns you to the material option details list form.

    Let us create another property named "Cross-section, sq mm" within the "White cables" material option. This requires repeating the steps that you performed to create the first property.

15. Click **Create** (fig. 15.32).

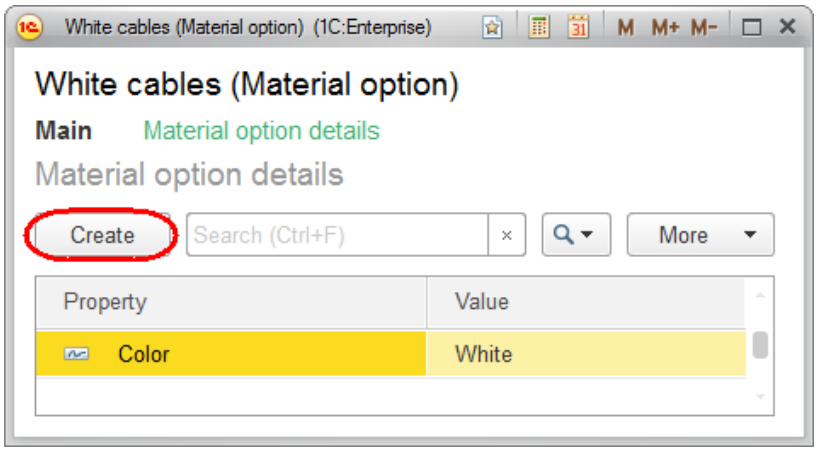


Fig. 15.32. Creating a property in a list of material option details

16. In the material option details record form, in the **Property** field, click the **Select from list** button and then, in the drop-down list, click **Show all**.
17. In the chart of characteristic types choice form, click **Create**.
18. In the chart of characteristic types item form, in the **Description** field, enter *Cross*-section*, sq mm*, and in the **Value type** field, select **Number**, set its Length to *15* and Precision to *3* (fig. 15.33).

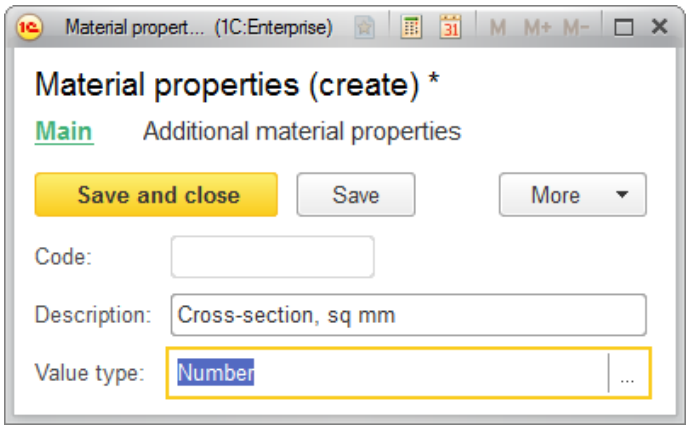


Fig. 15.33. Creating a characteristic type in a chart of characteristic types

19. Click **Save and close**.

    The created characteristic type is added to the choice form of the chart of characteristic types.

20. Click **Select**.

    This returns you to the material option details record form titled **Property and value**.

21. In the **Value** field, enter *2.5* (fig. 15.34).

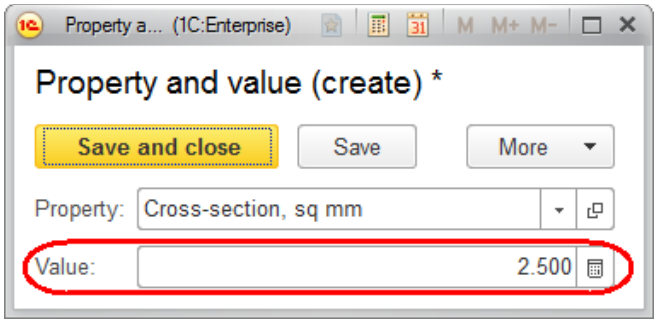


Fig. 15.34. Property and value in material option details

22. Click **Save and close**.

    This returns you to the material option details list form. The form contains two properties and their values for the White cables material option (fig. 15.35).

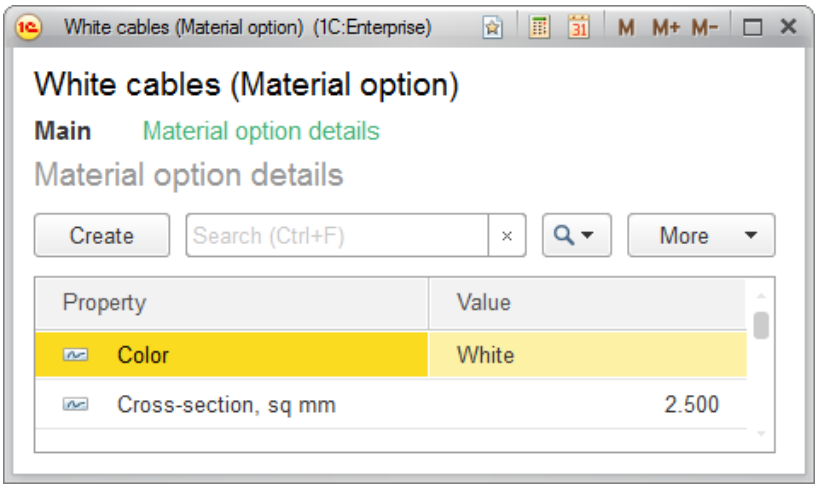


Fig. 15.35. Properties and values in material option details

23. Use the same steps to create a set of properties for **Tubing, rubber** item of the **Materials and services** catalog.

    Name the property set Poland (fig. 15.36) and include the following properties (fig. 15.37):

    - **Color**. Value: Black
    - **Manufacturer**. Value: Fagumit

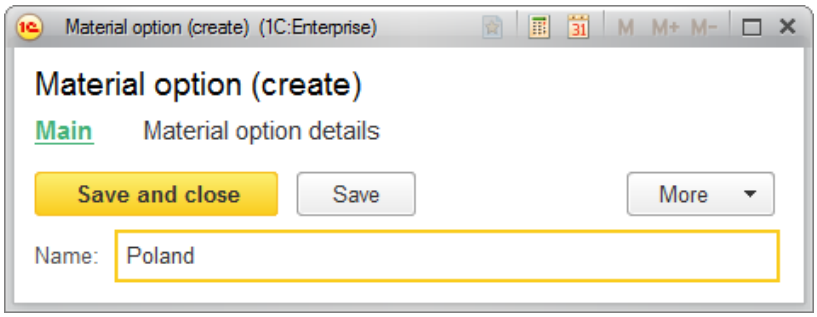


Fig. 15.36. Material option for a rubber tubing material

When you create the Color property, select it from the available properties in the chart of characteristic types.

The value of this characteristic type is Black. First you need to add it to the catalog of additional material properties and then select it from the catalog.

When you create the Manufacturer property that has the Additional material properties value type, first add this property to the chart of characteristic types (set the value type to Additional material properties), and then select it from the chart.

The value of this characteristic type is Fagumit. First you need to add it to the catalog of additional material properties and then select it from the catalog.

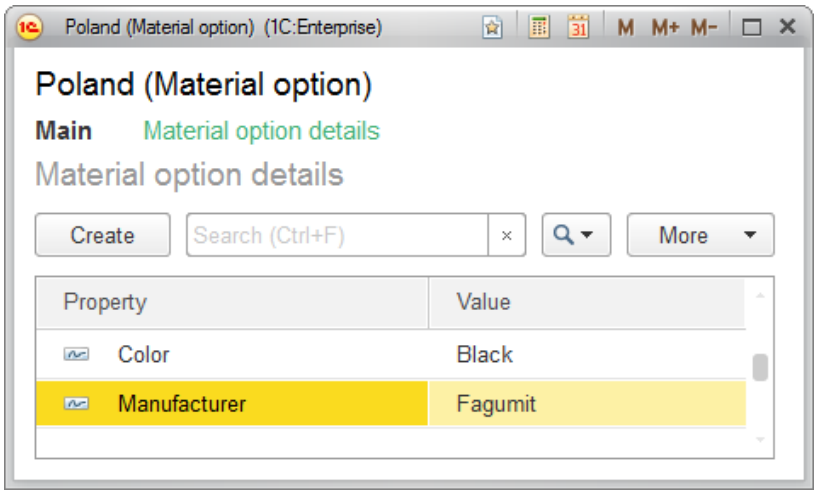


Fig. 15.37. Properties and values in Poland material option details

Let us review everything you have created from the developer perspective rather than from the user perspective.

- Click in the upper left corner of the applied solution window, click **All functions**, and open all of the configuration objects that store material characteristics data, one by one.

  The Material options catalog stores the material property sets you have created. Each property set is subordinate to a specific material item.

  The Material properties chart of characteristic types stores the material characteristic types you have created:

  - **Color**. Value type: CatalogRef.AdditionalMaterialProperties
  - **Cross-section, sq mm**. Value type: Number
  - **Manufacturer**. Value type: CatalogRef.AdditionalMaterialProperties

  The Additional material properties catalog stores the values of these characteristic types (except for the cross-section value, which is a Number).

  The MaterialPropertyValues information register stores the relations between characteristic types and their values for specific property sets.

  The relations between these configuration objects are shown in fig. 15.38.

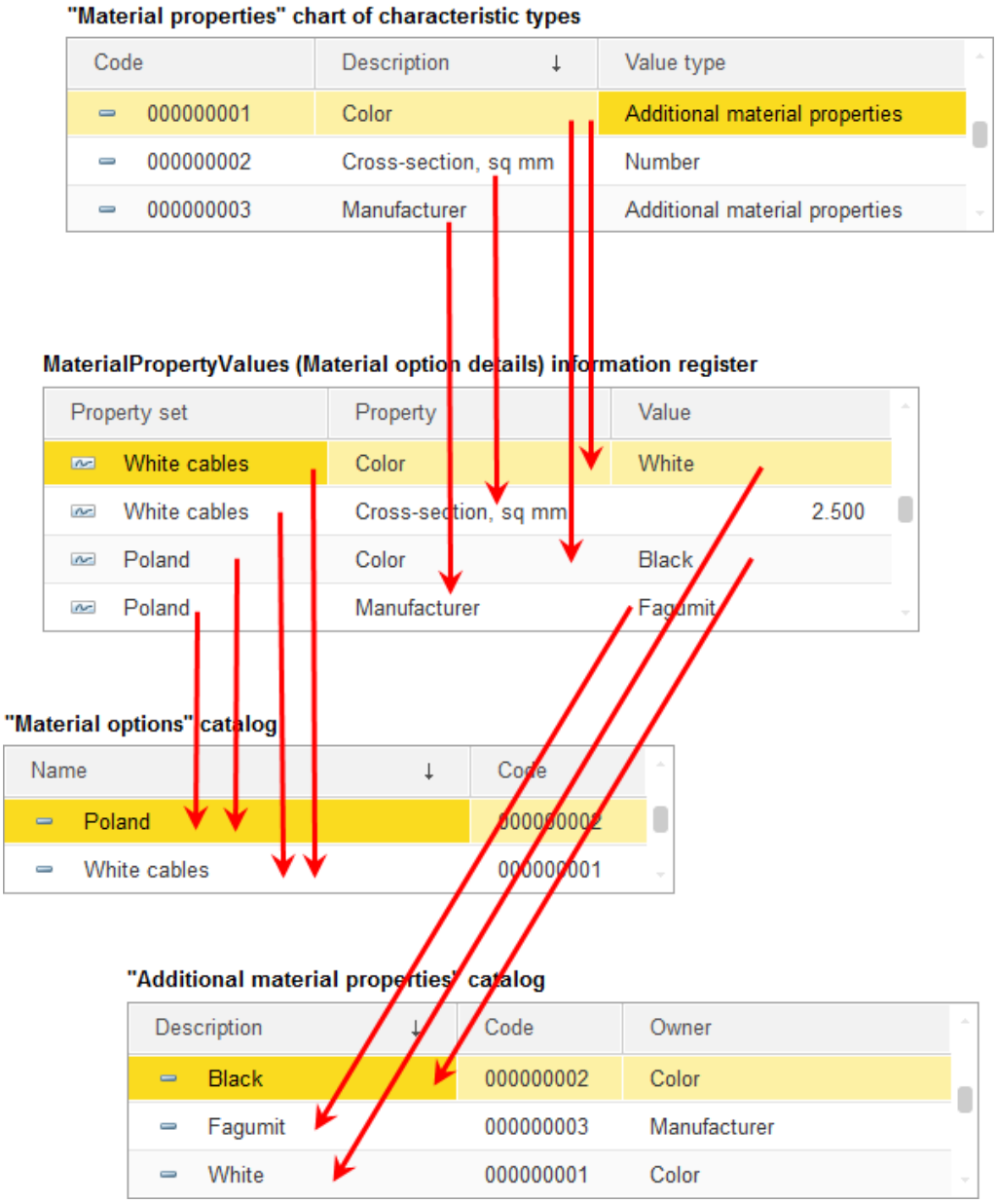


Fig. 15.38. Configuration objects that store material characteristics data

Modifying accounting functionality

So you implemented the option to specify custom material characteristics and then created some characteristics (material options).

But this is only a part of the job. We would also want to perform material accounting based on these characteristics. The goals we want to achieve are:

- Record product receipts with characteristics
- Record product expenses with characteristics
- Generate reports for products with specific characteristics instead of products in general

To do so, you need to adjust the existing registers and create a new report that displays data based on material properties.

The BalanceOfMaterials register

In Designer mode

In order to perform material accounting by characteristic values, you need to edit the structure of the BalanceOfMaterials accumulation register so that it could store data broken down by material property set.

1. In Designer, open the editor of the **BalanceOfMaterials** accumulation register configuration object.
2. On the **Data** tab, add the **PropertySet** dimension that has **CatalogRef.MaterialOptions** type (fig. 15.39).

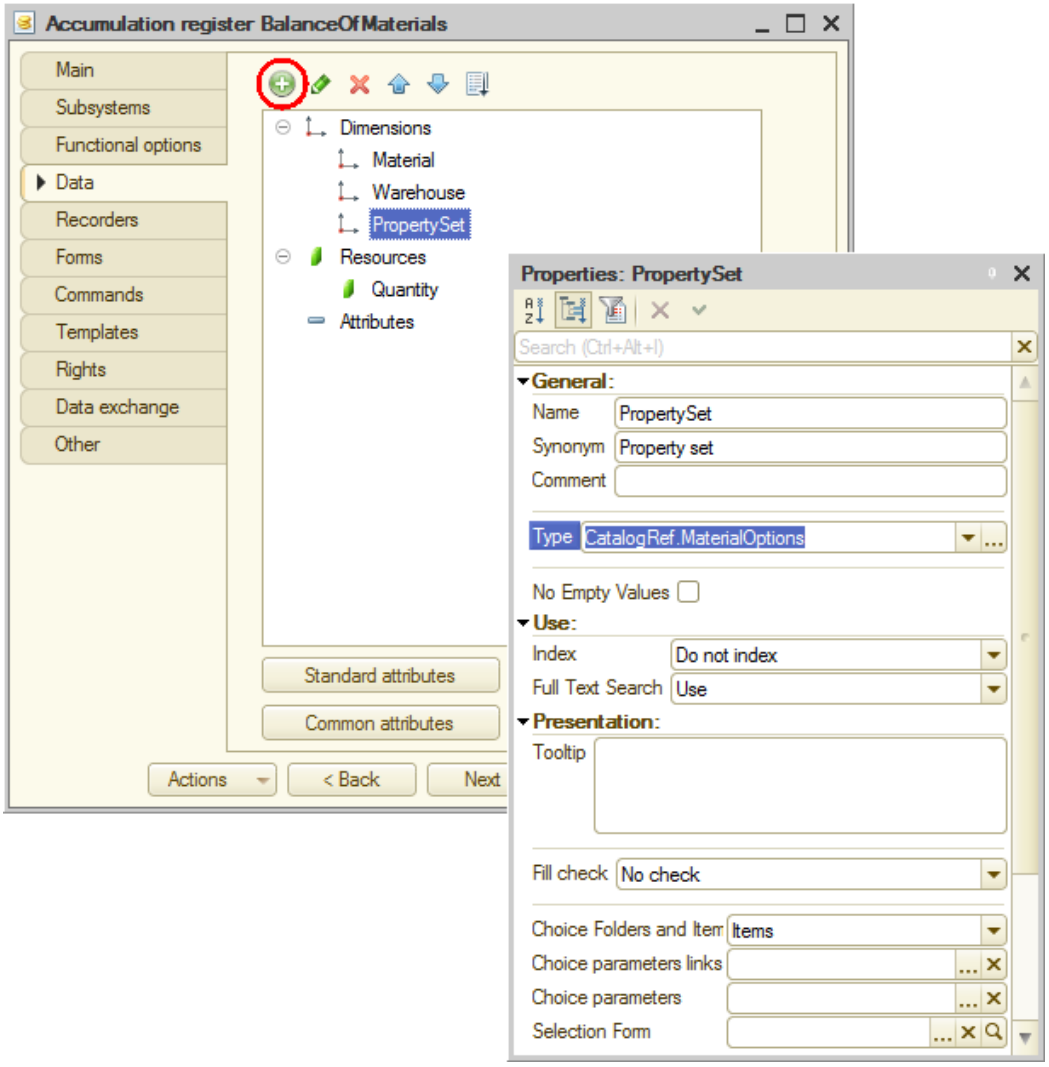


Fig. 15.39. New PropertySet dimension

The GoodsReceipt document

In Designer mode

Then you need to adjust the GoodsReceipt document. You have to add the option to specify a property set in a document and have this property set recorded when the document is posted.

1. In Designer, open the editor of the **GoodsReceipt** document configuration object.
2. On the **Data** tab, add the **PropertySet** attribute with the **CatalogRef.MaterialOptions** type to the document tabular section (fig. 15.40).

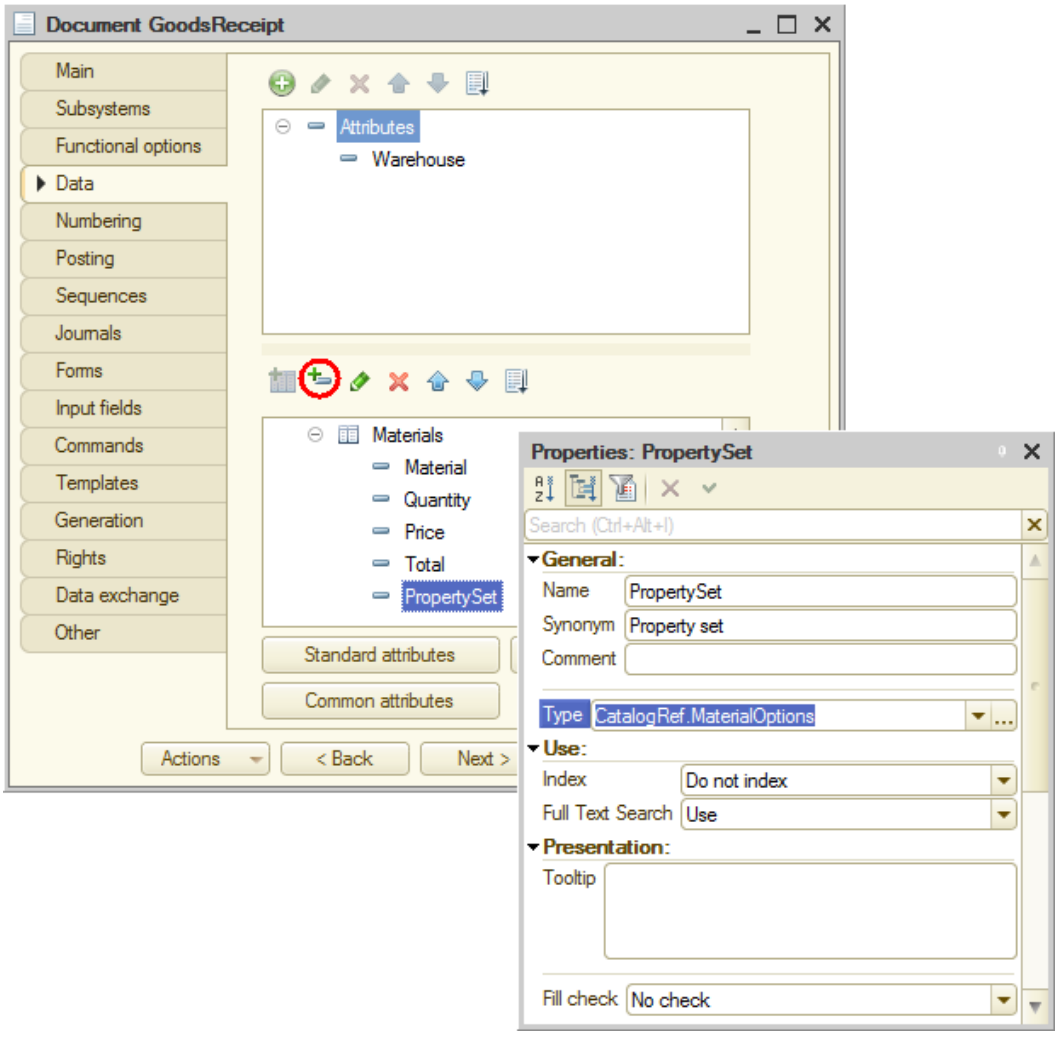


Fig. 15.40. New PropertySet attribute

You need to fill the Choice parameters links property for this attribute to limit the selection of property sets by those available for a specific material.

3. In the property palette, in the **Choice parameters links** field, click the **Select** ... button.
4. Drag the **Materials.Material** attribute from the list of available attributes to the list of parameters (fig. 15.41) and click **OK**.

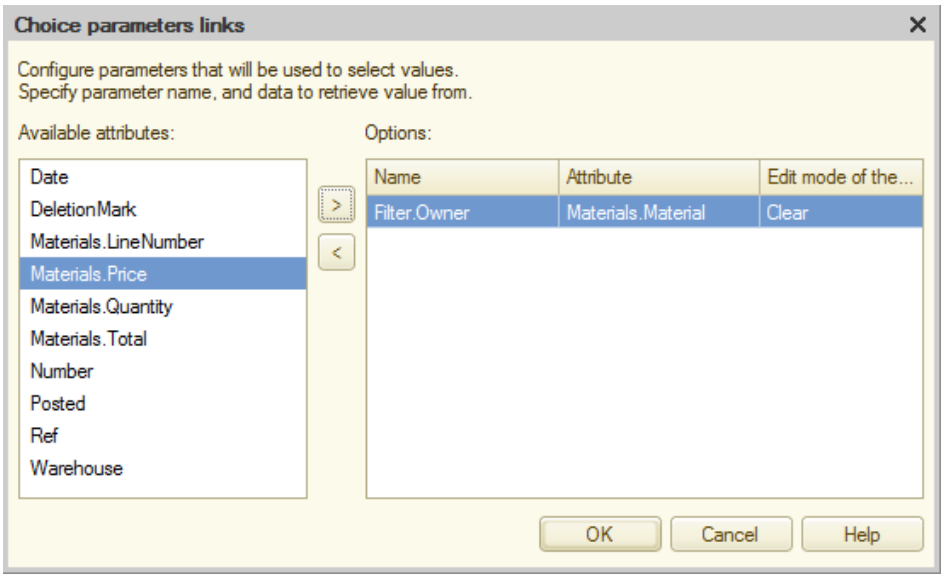


Fig. 15.41. Choice parameters links

So you have defined that when a user attempts to select a value for the PropertySet field, the list of Material options catalog items available for selection only includes the items subordinate to the material selected in the Material column.

Let us add this attribute to the tabular section of the document form.

5. On the **Forms** tab, in the list of forms, double-click **DocumentForm**.
6. In the upper right pane of the form editor, on the **Attributes** tab, expand the **Object** form attribute.

   You can see that it includes all the attributes of the GoodsReceipt document.
7. Expand the **Materials** tabular section and drag the **PropertySet** attribute to the form elements pane (the upper left pane), to the position after the **Material** field (fig. 15.42).

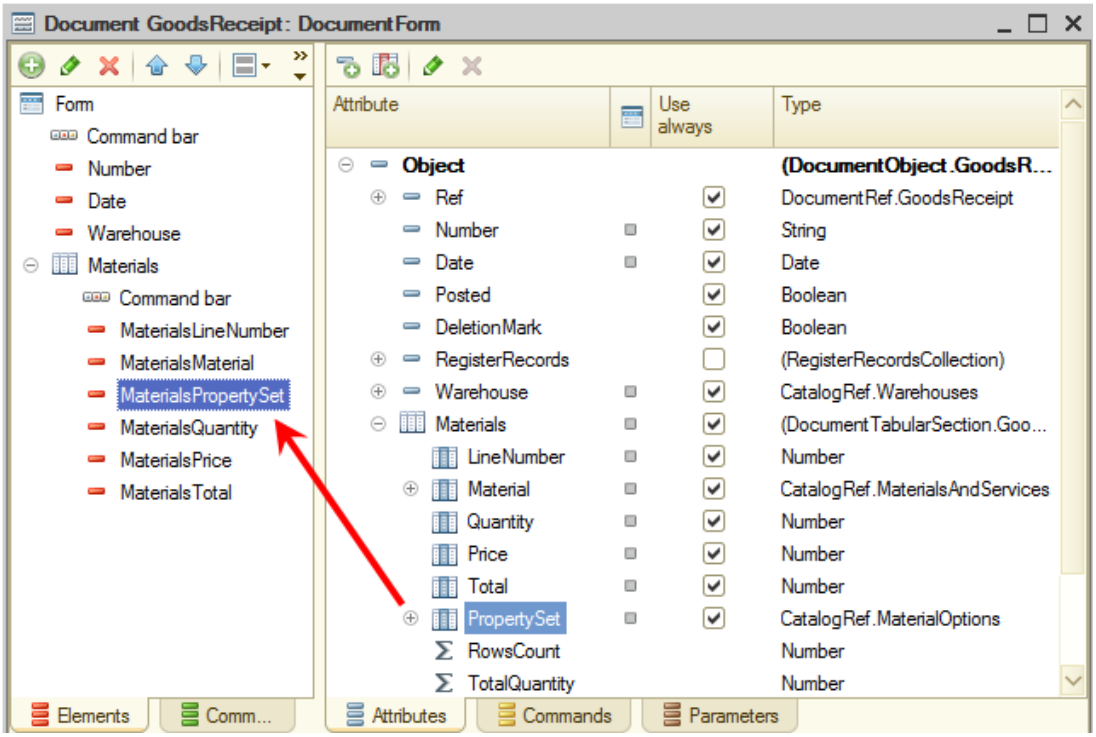


Fig. 15.42. Modification of GoodsReceipt document form

Note that in the property palette of the PropertySet form control you can see that the PropertySet tabular section attribute is selected as the DataPath property value. The platform filled this property automatically when you dragged the attribute to the form.

The DataPath property matches a form control to a form attribute (the data displayed in the form). This property must be filled because otherwise the form control is not displayed.

Note. When you add a form control using the **Add** button, you have to fill its DataPath property manually in order to link the control to a form attribute.

8. In the editor of the **GoodsReceipt** document configuration object, click the **Other** tab and open the object module.
9. Open the **Posting** event handler procedure and, in the section that generates register records, assign a value to the PropertySet dimension of the BalanceOfMaterials register as shown in listing 15.3.

Listing 15.3. Posting() procedure (fragment)

```
...
// register BalanceOfMaterials Receipt
...
Record.Material = CurRowMaterials.Material;
Record.PropertySet = CurRowMaterials.PropertySet;
Record.Warehouse = Warehouse;
...
```

The Services document

In Designer mode

Let us adjust the Services document in a similar manner, giving users the option to specify a property set for each material.

1. In Designer, open the editor of the **Services** document configuration object.
2. On the **Data** tab, add the **PropertySet** attribute with the **CatalogRef.MaterialOptions** type to the document tabular section.
3. Fill the **Choice parameters links** property for this attribute by adding **the MaterialsAndServices.MaterialOrService** attribute from the list of available attributes to the list of parameters.

   By doing this, you specify that the list of MaterialOptions catalog items available for selection in the PropertySet field contains only items subordinate to the material selected in the MaterialOrService column.
4. Drag this attribute from the form attributes pane to the form controls pane, to the tabular section, after the **MaterialOrService** item.
5. In the editor of the **Services** document configuration object, click the **Other** tab and open the object module.
6. Open the **Posting** event handler procedure and, in the section that generates register records, assign a value to the PropertySet dimension of the BalanceOfMaterials register as shown in listing 15.4.

   Listing 15.4. Posting() procedure (fragment)

```
...
// register BalanceOfMaterials Expense
...
Record.Material = SelectionDetailRecords.MaterialOrService;
Record.PropertySet = SelectionDetailRecords.PropertySet;
Record.Warehouse = Warehouse;
...
```

7. Since during the previous lesson you adjusted the document posting procedure to get all the document data using a query, update the query to get the new document attribute (listing 15.5).

   Listing 15.5. Posting() procedure (fragment)

```
...
Query = New Query;

// Specifying the temporary tables manager used by the query
Query.TempTablesManager = TTManager;
```

```
Query.Text = "SELECT
 | ServicesMaterialsAndServices.MaterialOrService,
 | ServicesMaterialsAndServices.MaterialOrService.MaterialServiceType
 |  AS MaterialServiceType,
 | ServicesMaterialsAndServices.PropertySet,
 | SUM(ServicesMaterialsAndServices.Quantity) AS QuantityInDocument,
 | SUM(ServicesMaterialsAndServices.Total) AS TotalInDocument
 |INTO DocumentMaterialsAndServices
 |FROM
 | Document.Services.MaterialsAndServices
 |  AS ServicesMaterialsAndServices
 |WHERE
 | ServicesMaterialsAndServices.Ref = &Ref
 |GROUP BY
 | ServicesMaterialsAndServices.MaterialOrService,
 | ServicesMaterialsAndServices.MaterialOrService.MaterialServiceType,
 | ServicesMaterialsAndServices.PropertySet";
...
Query2 = New Query;
Query2.TempTablesManager = TTManager;
Query2.Text = "SELECT
 | DocumentMaterialsAndServices.MaterialOrService,
 | DocumentMaterialsAndServices.MaterialServiceType,
 | DocumentMaterialsAndServices.PropertySet,
 | DocumentMaterialsAndServices.QuantityInDocument,
 | DocumentMaterialsAndServices.TotalInDocument,
 | ISNULL(CostOfMaterialsBalance.CostBalance, 0) AS Cost,
 | ISNULL(BalanceOfMaterialsBalance.QuantityBalance, 0) AS Quantity
 |FROM
 | DocumentMaterialsAndServices AS DocumentMaterialsAndServices
...
```

You also have to adjust the last query that performs the check for negative balances during real-time posting. Instead of getting general balances for materials specified in the document tabular section, it should get the balances for materials with the property sets specified in the document rows (listing 15.6).

Listing 15.6. Check for negative balances during real-time posting

```
...
Query3.Text = "SELECT
    |    BalanceOfMaterialsBalance.Material,
    |    BalanceOfMaterialsBalance.PropertySet,
    |    BalanceOfMaterialsBalance.QuantityBalance
    |FROM
    |    AccumulationRegister.BalanceOfMaterials.Balance( ,
```

```
    |        (Material, PropertySet) IN (
    |        SELECT
    |            DocumentMaterialsAndServices.MaterialOrService,
    |            DocumentMaterialsAndServices.PropertySet
    |        FROM
    |            DocumentMaterialsAndServices)
    |        AND Warehouse = &Warehouse) AS BalanceOfMaterialsBalance
    |WHERE
    |    BalanceOfMaterialsBalance.QuantityBalance < 0";


Query3.SetParameter("Warehouse", Warehouse);


QueryResult = Query3.Execute();
SelectionDetailRecords = QueryResult.Select();


While SelectionDetailRecords.Next() Do
    Message = New UserMessage();
    Message.Text = String(- SelectionDetailRecords.QuantityBalance)
        + " units shortage for """ + SelectionDetailRecords.Material
        + """ with """ + SelectionDetailRecords.PropertySet
        + """ property set.";
    Message.Message();


    Cancel = True;


EndDo;
...
```

Material receipt and expense with property sets specified

In 1C:Enterprise mode

Let us specify property sets for material receipts.

1. Run 1C:Enterprise in the debug mode.
2. Open the GoodsReceipt document #2 and specify that it includes 2 white power supply cables and Polish rubber tubing.
3. Copy the first row of the document and add 3 black power supply cables.

   In doing so, you need to create another property set for power supply cables: Black cables with Color set to Black and Cross-section set to 2.5 (fig. 15.43).

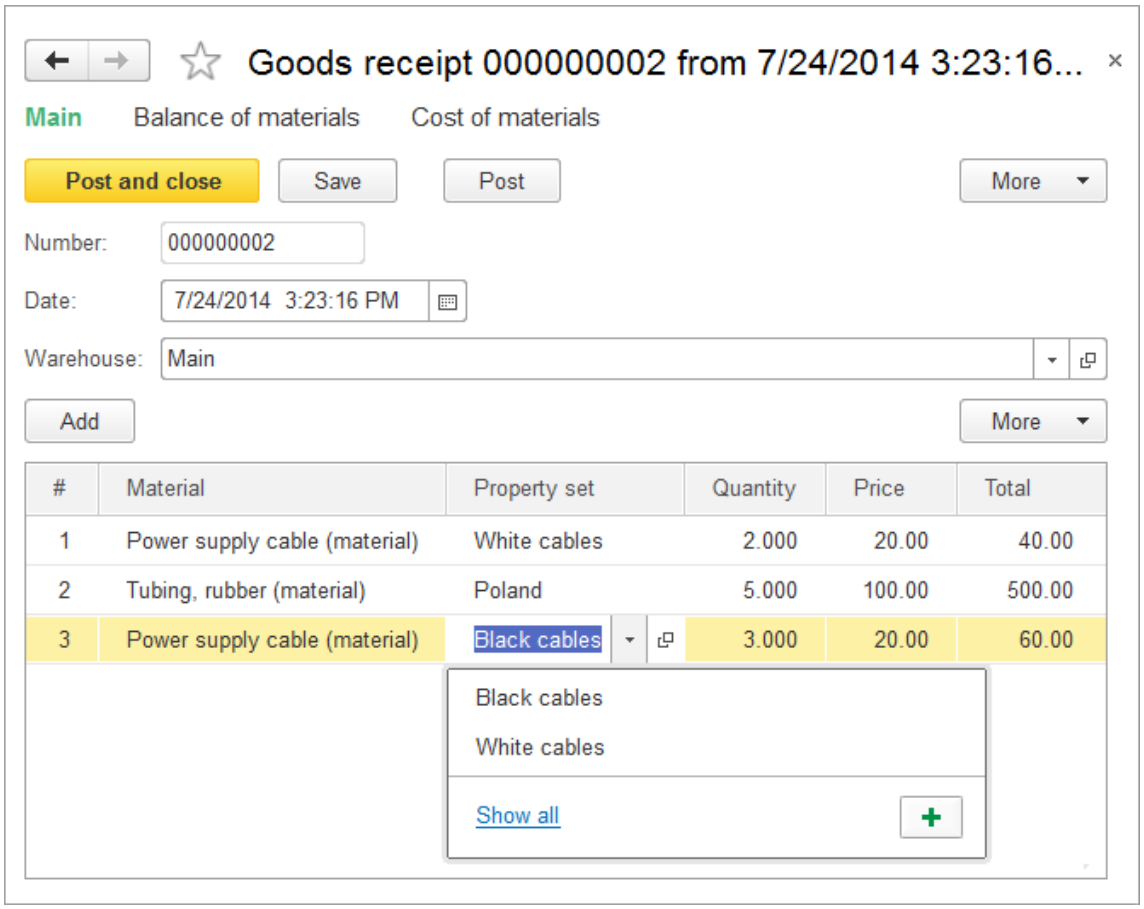


Fig. 15.43. Goods receipt document #2

4. Click **Post** and then, in the navigation panel of the document form, click **Balance of materials**, and check the document records for the BalanceOfMaterials register (fig. 15.44).

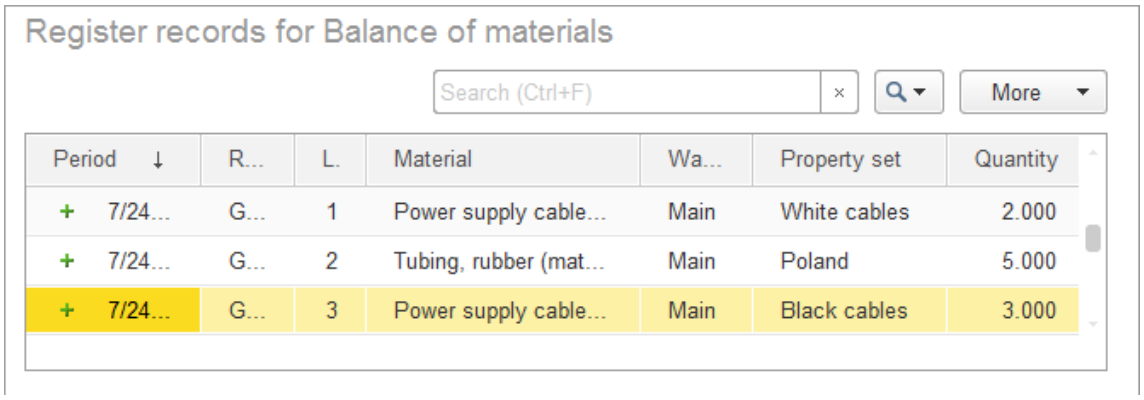


Fig. 15.44. Balance of materials register records for Goods receipt document #2

5. Open the Services document #1 and specify that Polish rubber tubing has been used (fig. 15.45).

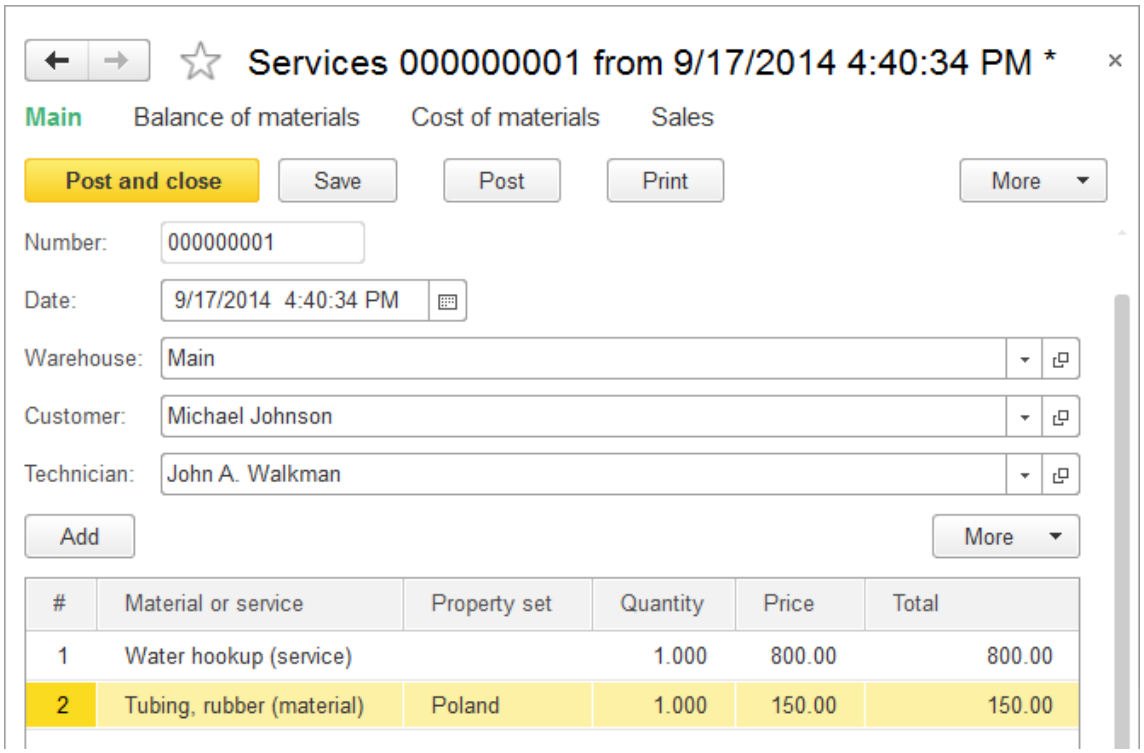


Fig. 15.45. Services document #1

6. Click **Post** and then, in the navigation panel of the document form, click **Balance of materials**, and check the document records for the BalanceOfMaterials register (fig. 15.46).

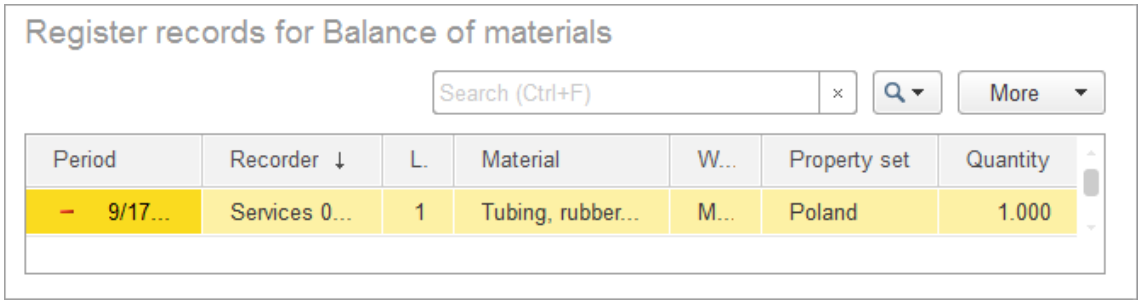
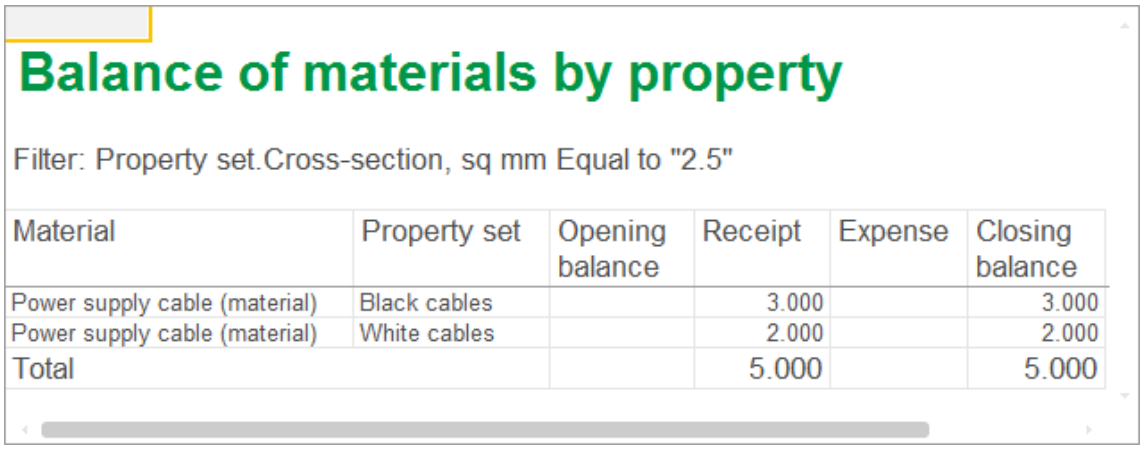


Fig. 15.46. Balance of materials register records for Services document #1

Report with characteristics

To polish the result, let us create a report that shows the availability of materials with certain properties. To create the report, you will use the data composition system features that provide support for characteristics (fig. 15.47).



Balance of materials by property

Filter: Property set.Cross-section, sq mm Equal to "2.5"

Material	Property set	Opening balance	Receipt	Expense	Closing balance
Power supply cable (material)	Black cables		3.000		3.000
Power supply cable (material)	White cables		2.000		2.000
Total			5.000		5.000

Fig. 15.47. Resulting report

Briefly, a data set for the data composition system it retrieved by a simple query to the BalanceOfMaterials register. The platform automatically includes the material option properties to the report based on the description that you created for the MaterialOptions catalog in section "Description of material option characteristics" on page 440.

Based on the description, the platform generates a straightforward and user-friendly interface for selecting characteristics. It also generates the required database queries based on the user-selected values.

In Designer mode

Let us proceed to report creation.

1. Create a report configuration object named *MaterialBalanceByProperty*.
2. Run the Data composition schema wizard.
3. Add **Data set – query** and run the Query builder.

Query for a data set

Let us specify the query data set.

1. Select the **BalanceOfMaterials.BalanceAndTurnovers** virtual table as the data source.
2. Select the following fields from this table (fig. 15.48):
   - **Material**
   - **PropertySet**
   - **QuantityOpeningBalance**
   - **QuantityReceipt**
   - **QuantityExpense**
   - **QuantityClosingBalance**

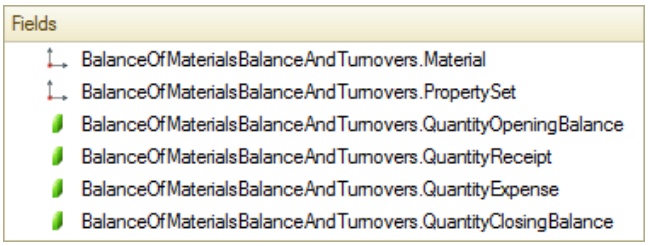


Fig. 15.48. Fields selected for the query

3. Click the **Unions/Aliases** tab and set the aliases for the numeric fields by removing Quantity from their names (fig. 15.49).

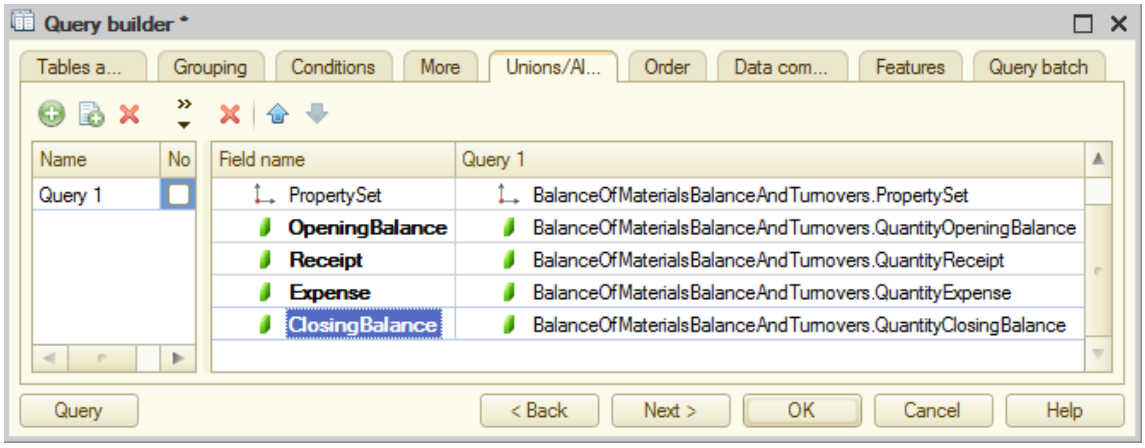


Fig. 15.49. Unions/aliases

4. Click **OK**.

   This completes the query creation.

Resources

Let us add the resources.

- On the **Resources** tab, select all available resources (fig. 15.50).

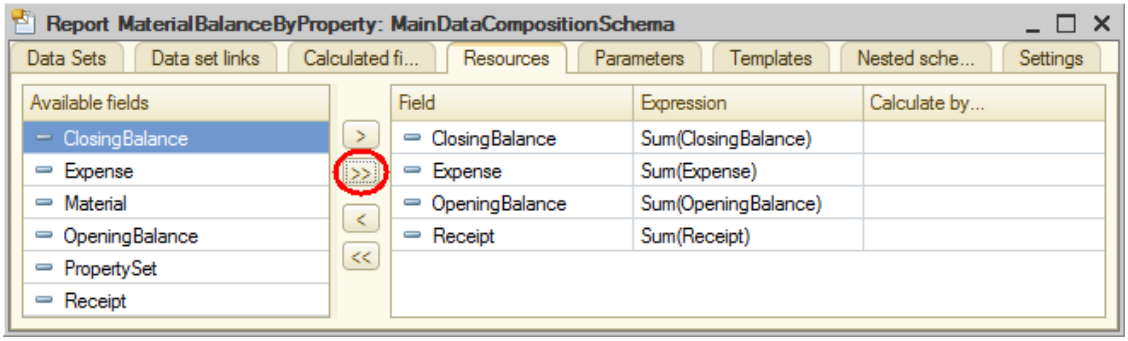


Fig. 15.50. Data composition schema resources

Settings

Let us specify the report structure.

1. On the **Settings** tab, add the <Detailed records> grouping.
2. On the **Selected fields** tab, select the fields that you want displayed in the report (fig. 15.51):
   - **Material**
   - **PropertySet**
   - **OpeningBalance**
   - **Receipt**
   - **Expense**
   - **Closing balance**

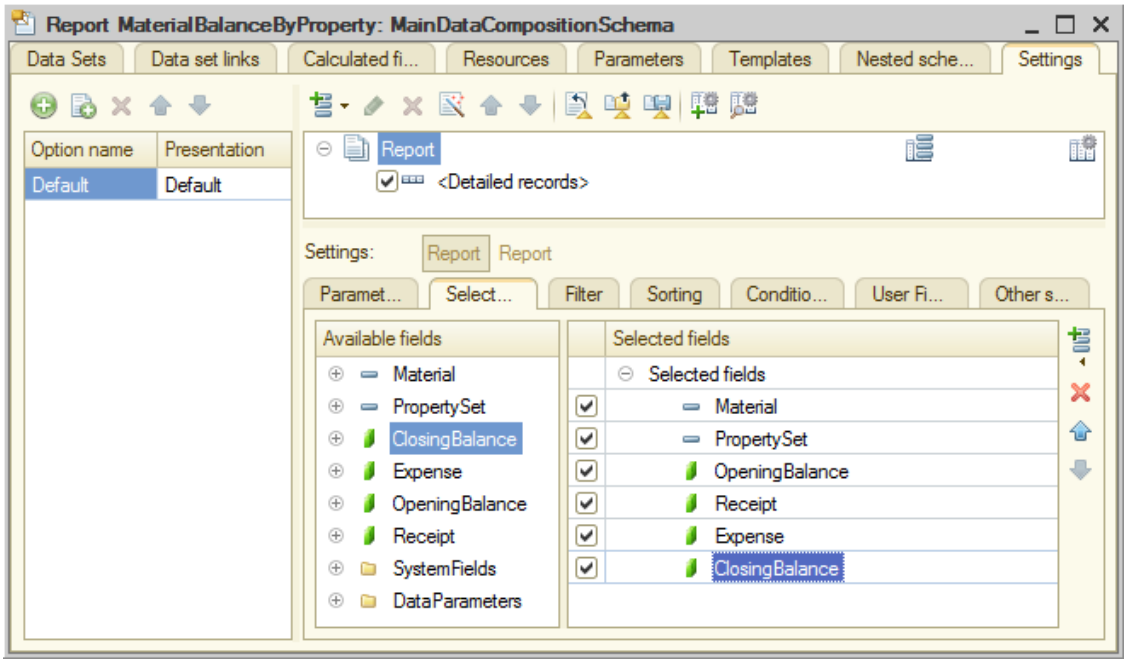


Fig. 15.51. Report groupings and fields

3. On the **Other settings** tab, set the report title to *Balance of materials by property*.

   To be able to test the report, let us include the Filter setting to the list of quick user settings.

4. Click the **Custom settings item properties** button in the command bar at the top of the window.

   In the window that is opened you can edit the assortment of user settings for the report.

5. Select the check box next to **Filter** and, in the **Edit mode** list, keep the default **Quick access value** (fig. 15.52).

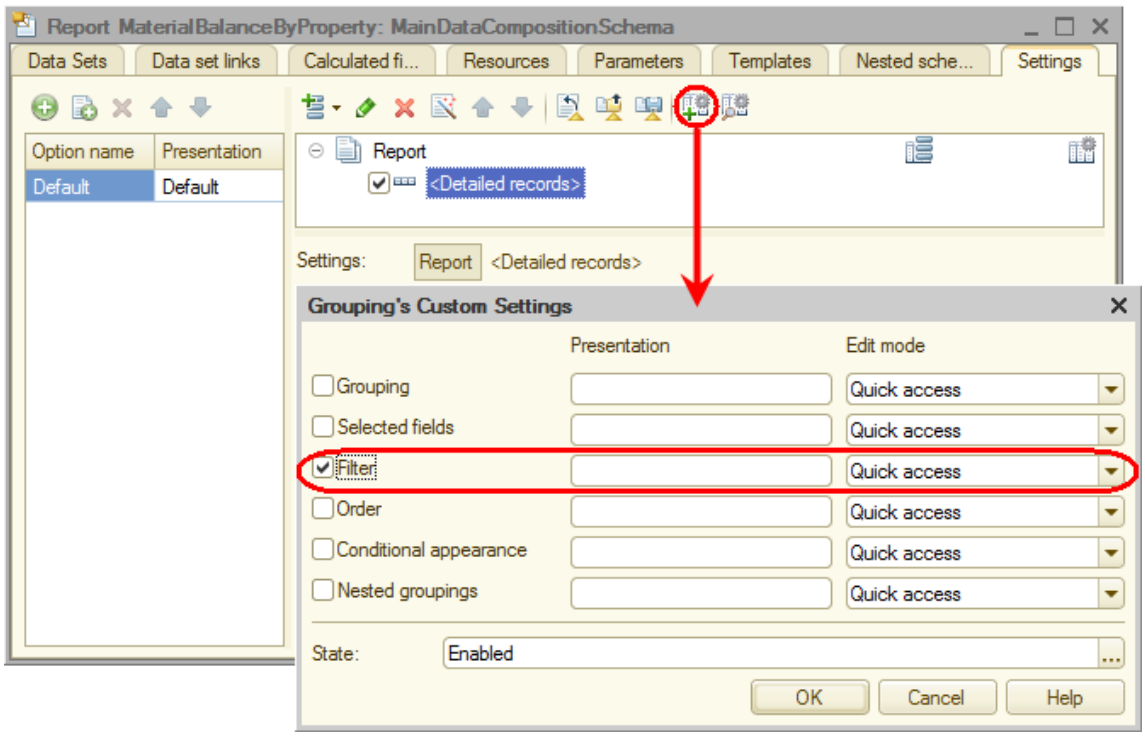


Fig. 15.52. Specifying user settings

Finally, let us specify the subsystems where the report is available.

6. Close the data composition schema wizard and then, in the editor of the **MaterialBalanceByProperty** report configuration object, click the **Subsystems** tab.
7. Select the **Accounting** and **Inventory** check boxes.

   This completes the report creation.

In 1C:Enterprise mode

Let us check the report.

1. Start 1C:Enterprise in the debug mode.
2. In the **Inventory** section, run the **Material balance by property** command (fig. 15.53).

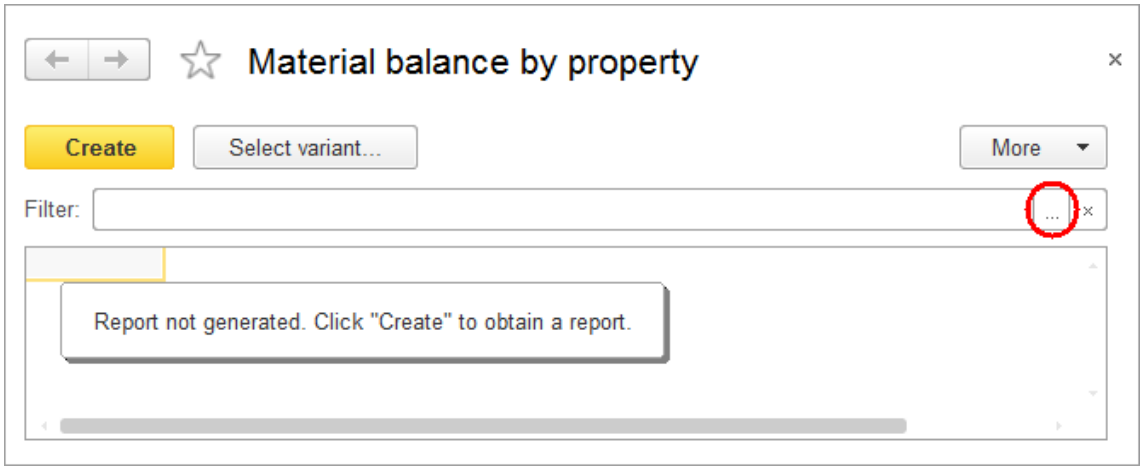


Fig. 15.53. Report form

You can see that the Filter setting is available in the report form. You can use it to obtain the balance of materials based on their characteristics.

First, let us check the availability of materials with a cross-section of 2.5 sq mm.

3. In the **Filter** field, click the **Select** button (see fig. 15.53).

   This opens the **Edit filter** dialog box. The list on the left contains all available report fields.

4. Expand the **Property set** item (fig. 15.54).

   Note that the data composition system added all the characteristics defined for various property sets in the database (Manufacturer, Cross-section, and Color) to the standard attributes of the MaterialOptions catalog. So filtering the report by values of some characteristics is straightforward and intuitive.

5. Select the **Cross-section, sq** mm field and set the condition where its value is equal to *2.5*.

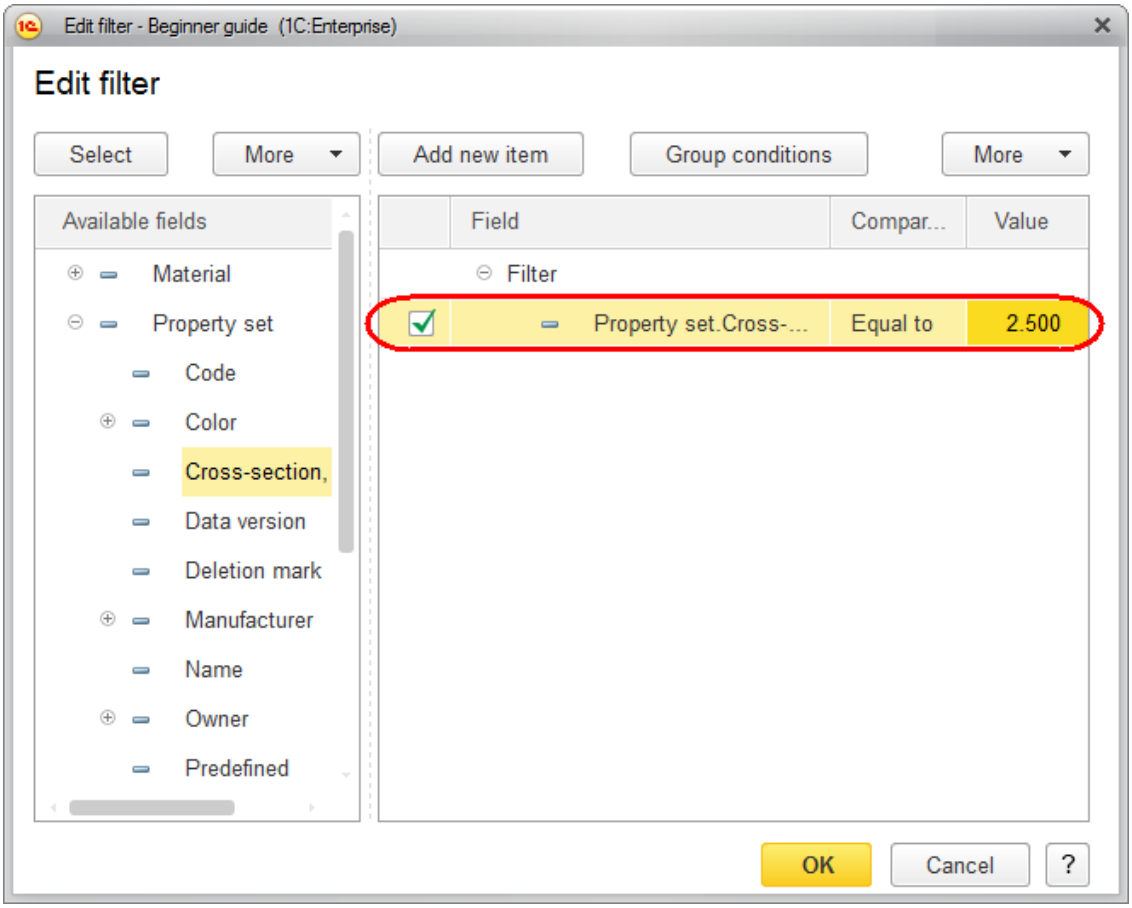


Fig. 15.54. Creating a filter

6. Click **OK**.
7. In the report window, click **Create**.

   The report should look as shown in fig. 15.55.

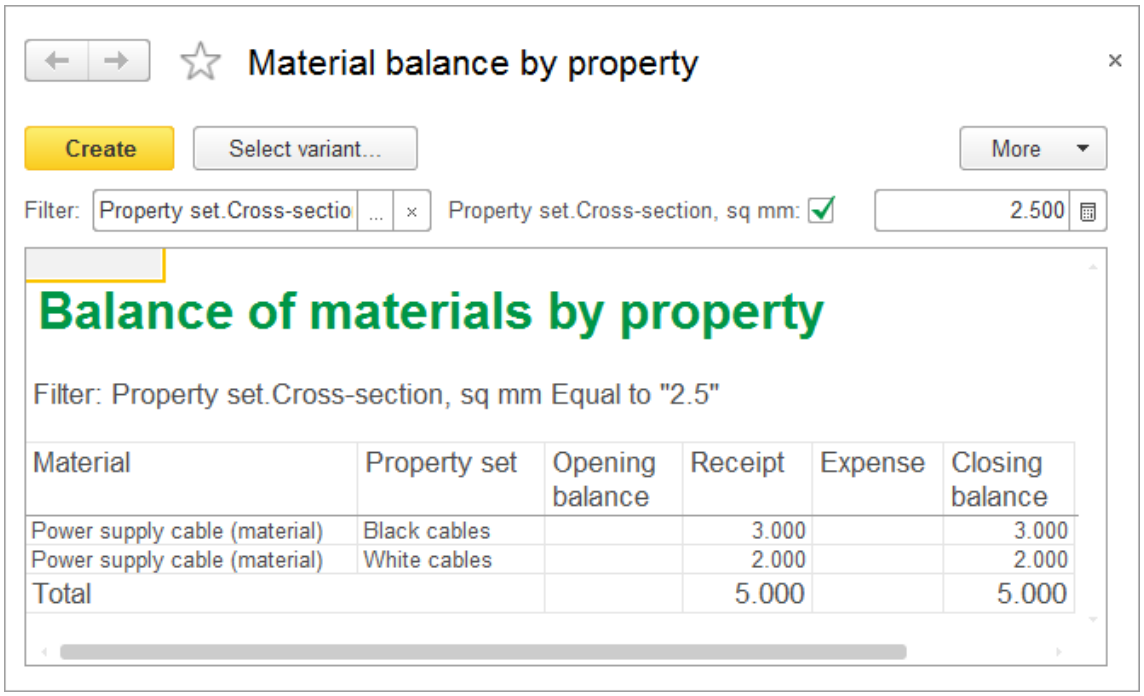


Fig. 15.55. Resulting report

Now let us check which of the available materials are black.

8. In the **Filter** field, click the **Select** button, click the current filter condition, and click the **Delete** button.
9. In the left pane, double-click the **Color** field.

   In the **Value** field, click the **Select from list** button and select Black from the list of available material properties (fig. 15.56).

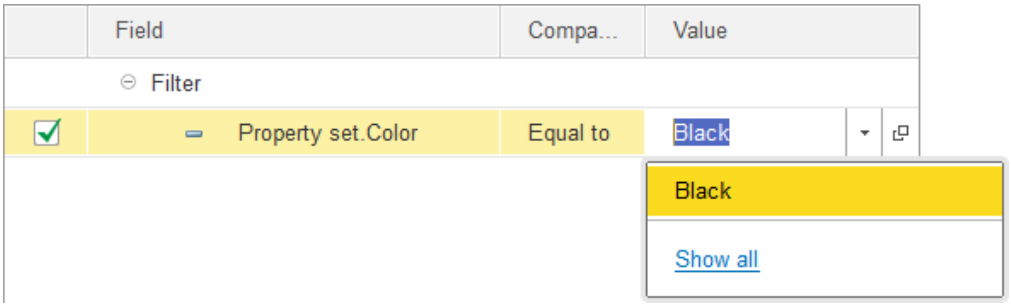


Fig. 15.56. Creating a filter

10. Click **OK**.
11. In the report window, click **Create**.

    The report should look as shown in fig. 15.57.

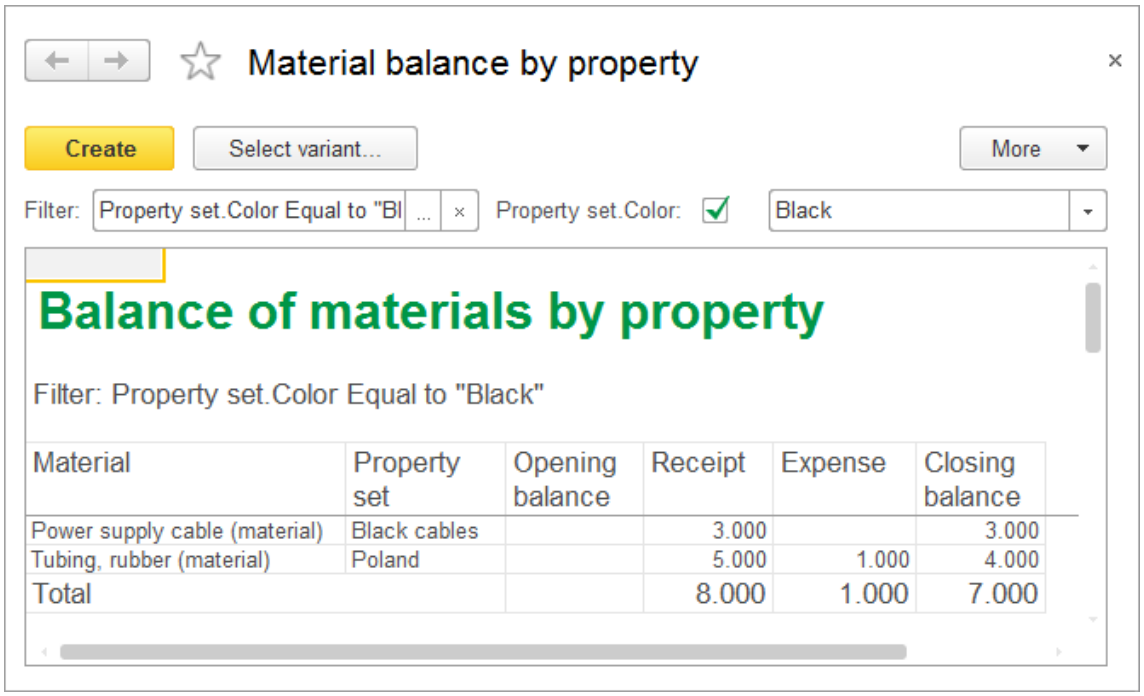


Fig. 15.57. Resulting report

And finally, to ensure that the report functions properly, let us obtain the number of black rubber tubing units.

12. In the **Filter** field, click the **Select** button and then, in the list on the left, double-click the **Material** field.

    This adds another filter condition.

13. In the **Value** field, click the **Select from list** button and select **Tubing, rubber** from the list (fig. 15.58).

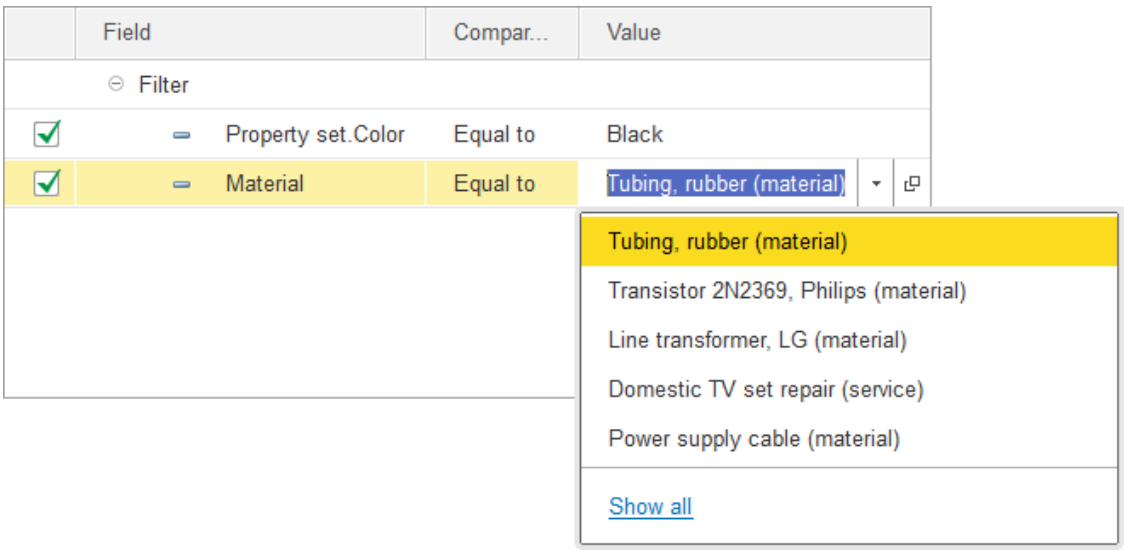


Fig. 15.58. Creating a filter

14. Click **OK**.
15. In the report window, click **Create**.

    The report should look as shown in fig. 15.59.

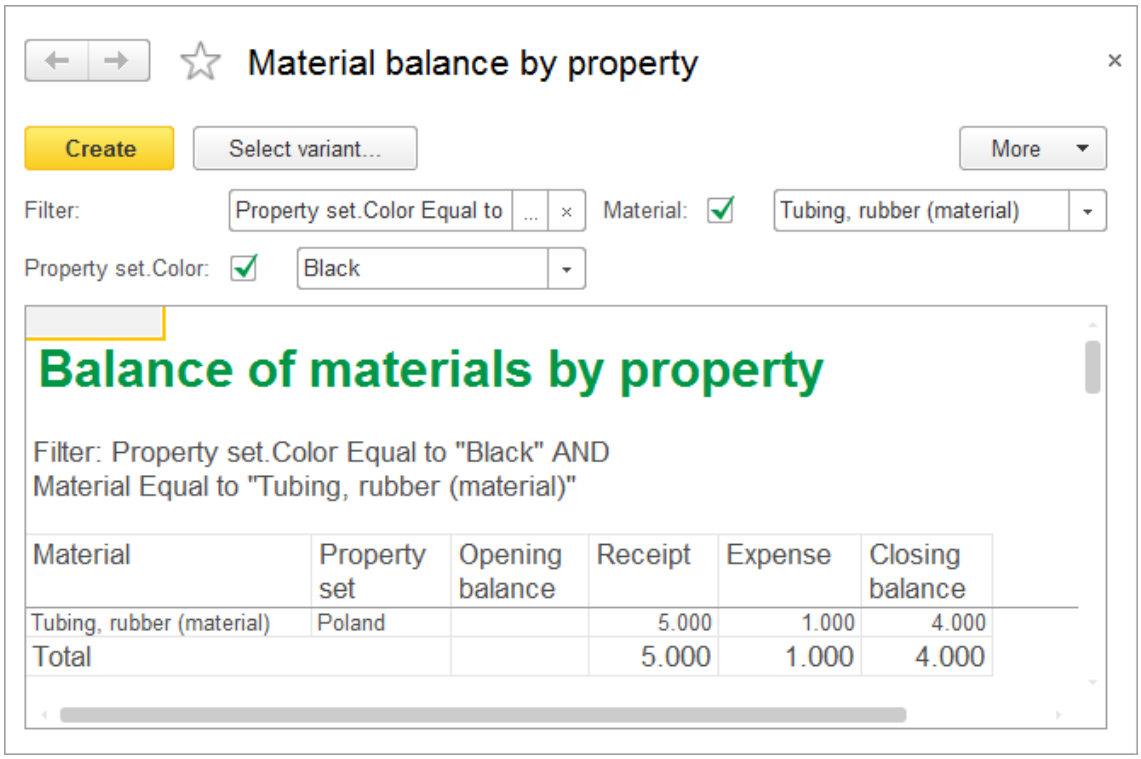


Fig. 15.59. Resulting report

So you can be sure that, by using this logical design, you can perform material accounting based on any number of material properties and property values.

It is worth mentioning that the example we have covered in this chapter is not the final solution for this configuration. We have simply demonstrated the available options. In order for your configuration to fully use material properties, you have to make appropriate changes in the remaining registers, documents, and some reports.

Quiz

- What are the uses of the Chart of characteristic types configuration object?
- What is the fundamental difference between a chart of characteristic types and a catalog?
- What is a type of characteristic value?
- Why does one need additional characteristic values?
- How does one use a chart of characteristic types to implement accounting by a variable number of characteristics?
- How does one create a chart of characteristic types?
- What is a choice parameters link?
- How does one edit a form title?
- How does one implement hiding form controls that contain subordinate data when a form is created?

- How does one define characteristics in metadata?
- How does one use characteristics for report generation?

Lesson 16 (1:50). Accounting

In this lesson you will study the accounting features of 1C:Enterprise. This tutorial does not explain the accounting basics, so if you do not have required understanding of accounting, we recommend that you first read some popular materials on the subject.

To implement accounting operations, you need the Chart of characteristic types configuration object, which you already know, and two new configuration objects: Chart of accounts and Accounting register.

The accounting register accumulates data that describes completed business transactions. The chart of accounts stores account descriptions. The chart of characteristic types defines analytical accounting objects, which introduce additional account dimensions.

Note that the chart of accounts provided in this tutorial is greatly simplified. It only contains a few provisional accounts, but that is enough to study the basics of 1C:Enterprise accounting.

Using charts of characteristic types in accounting

We have discussed the Chart of characteristic types configuration object in detail during the previous lesson (page 431), and now we will only show how to use this object in accounting.

Some of the accounts might use analytical accounting. We will use the "extra dimension types" term to refer to sections of analytical accounting. It means that each account can have multiple dimension types, and accounting is performed based on these types.

We will use the "extra dimensions" term to refer to specific analytical accounting objects.

For example, account 5000 (Inventory) is intended for accounting based on Products and Warehouses, which both are extra dimension types. While a specific "Chocolate spread" product and a specific "Main warehouse" that are specified in some posting within account 5000 are both extra dimensions.

So one of the purposes of a chart of characteristic types is describing extra dimension types. This means that all the analytical accounting sections are described in the respective chart of characteristic types and the types for the values available for some extra dimensions are defined there as well.

Adding a chart of characteristic types

In Designer mode

Let us create a chart of characteristic types that will store the descriptions of analytical accounting sections (extra dimension types).

1. In Designer, create a ChartOfCharacteristicTypes object named *ExtraDimensionTypes*.
2. On the **Subsystems** tab, include the chart of characteristic types in the **Accounting** subsystem.

   You also need a catalog for storing user-defined analytical accounting objects.
3. Create a Catalog object named *ExtraDimensions*.

   Let us specify that the catalog is subordinate to the ExtraDimensionTypes chart of characteristic types.
4. On the **Owners** tab, click the **Edit List Item** button and select the **ExtraDimensionTypes** chart of characteristic types (fig. 16.1).

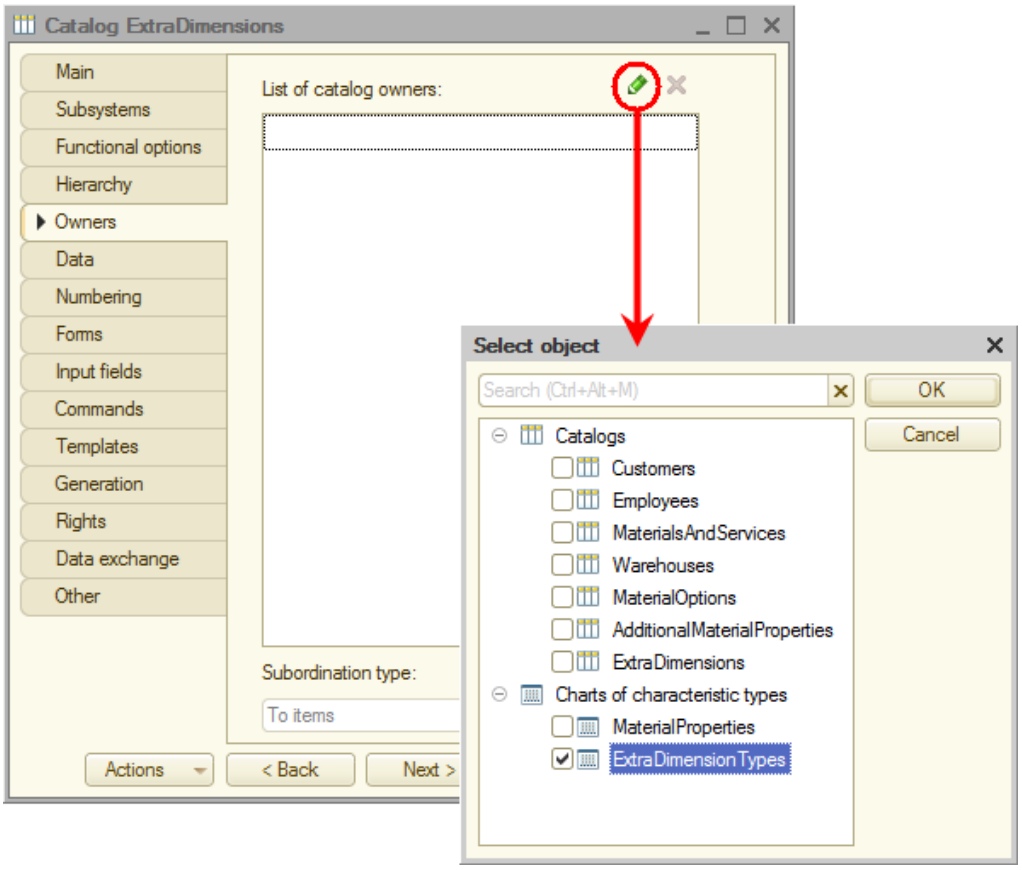


Fig. 16.1. ExtraDimensions catalog editor

5. Close the catalog editor and return to the chart of characteristic types.
6. On the **Main** tab, in the **Characteristic value type** field, click the **Select** ... button and specify the composite data type as follows (fig. 16.2):
   - **CatalogRef.Customers**
   - **CatalogRef.MaterialsAndServices**
   - **CatalogRef.ExtraDimensions**

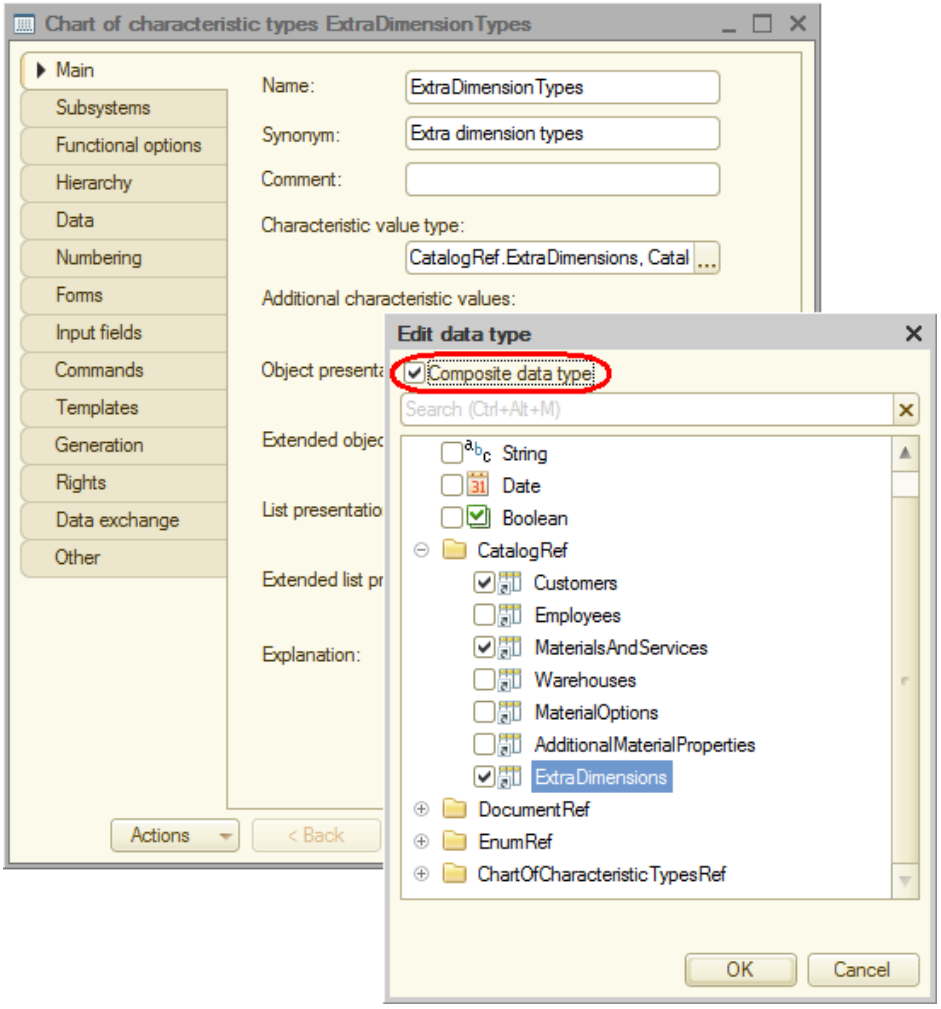


Fig. 16.2. Specifying a value type for a chart of characteristic types

The accounting department at Jack of All Trades maintains records of monetary transactions with material and customer dimensions, but in the future they might need more dimensions (this is why we use the ExtraDimensions catalog).

Note that the catalog that stores additional characteristic values must be present in the composite type that defines the characteristic value type, otherwise Designer displays an error message.

7. Specify that additional characteristic values are stored in the **ExtraDimensions** catalog.
8. On the **Other** tab, click **Predefined data** and enter the predefined values for the chart of characteristic types (fig. 16.3).

They represent available analytical accounting types.

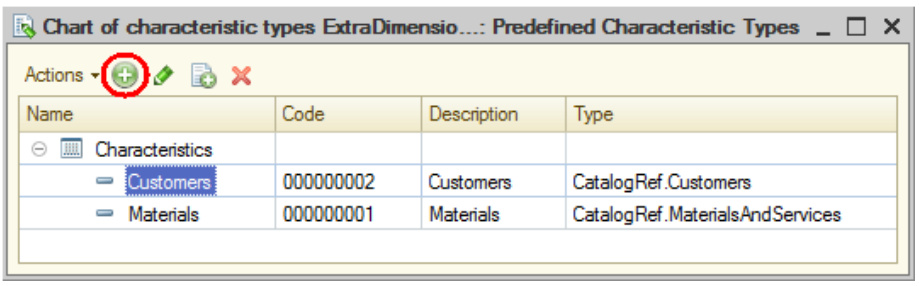


Fig. 16.3. Predefined characteristic types

9. Click **Add** to create the following predefined extra dimension types:
    - **Materials**. Code: 000000001, type: CatalogRef.MaterialsAndServices
    - **Customer**. Code: 000000002, type: CatalogRef.Customers

With that, you are done creating extra dimension types, so we can proceed to discussing the next configuration object: Chart of accounts.

Understanding charts of accounts

The Chart of accounts object is intended to define the set of accounts a company uses to store its accounting data.

Based on the Chart of accounts configuration object, the platform creates database tables that store accounts used by the company and their purposes.

This can be a government-directed accounting system, a chart of management accounts, or a custom set of accounts used in the analysis of various kinds of company activity.

In 1C:Enterprise a chart of accounts supports a hierarchy of subaccounts: each top-level account can have multiple subaccounts, which have their own subaccounts, and so on.

Analytical accounting by dimensions described in a chart of characteristic types is available for any account or subaccount. The relation between a chart of accounts and a chart of characteristic types is specified at the development stage.

To store definitions of extra dimensions, the platform creates a dedicated tabular section named ExtraDimensionTypes within the chart of accounts. The tabular section is not visible in Designer but is accessible using 1C:Enterprise script tools.

For each account you can specify multiple accounting options (for example, quantitative and multicurrency accounting). Each accounting option is defined by means of a subordinate configuration object named accounting flag.

You can also specify multiple extra dimension accounting options (for example, quantitative, multicurrency, or amount-tracking accounting). Each extra dimension accounting option is defined by means of a subordinate configuration object named extra dimension accounting flag.

Learn more! For details on the structure of 1C:Enterprise script objects intended for chart of accounts operations, see section "Quick developer reference. Charts of accounts" on page 852.

In addition to the above, each account can have a set of properties that are defined as attributes of the Chart of accounts configuration object. You can use them to define unique properties for chart of accounts items (such as the DoNotUseInPostings attribute).

Adding a chart of accounts

In Designer mode

As we mentioned in the beginning of this lesson, the accounting in Jack of All Trades is greatly simplified. This is why the chart of accounts used by their accounting department only contains four accounts:

- Inventory
- AccountsPayable
- AccountsReceivable
- Equity

Let us create a chart of accounts for Jack of All Trades.

1. Create a ChartOfAccounts object named *Main*.
2. In the **List presentation** field, enter *Main chart of accounts*.
3. On the **Subsystems** tab, include the chart of accounts in the **Accounting** subsystem.
4. On the **Data** tab, click **Accounting flags**, then click the **Add** button to create an accounting flag named *Quantitative* (fig. 16.4).

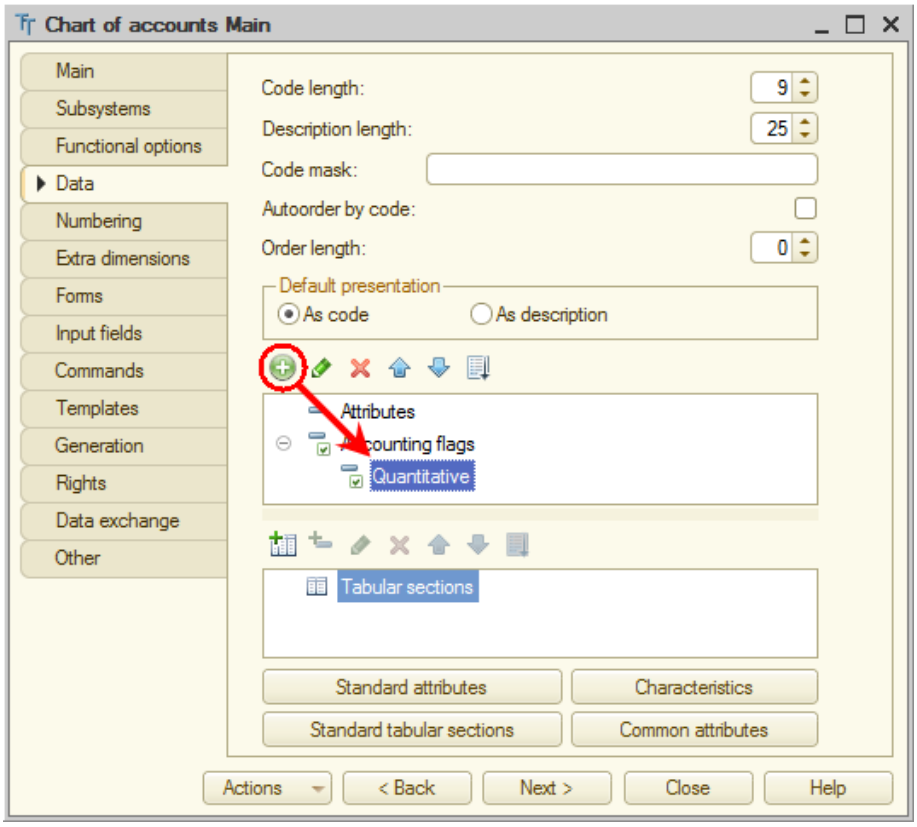


Fig. 16.4. Creating a chart of accounts attribute in the Accounting flags group

5. On the **Extra dimensions** tab, specify that the extra dimension types for the chart of accounts are stored in the **ExtraDimensionTypes** chart of characteristic types.

6. Set **Maximum number of extra dimensions** to *2*.
7. Create the *Quantitative* extra dimension accounting flag (fig. 16.5).

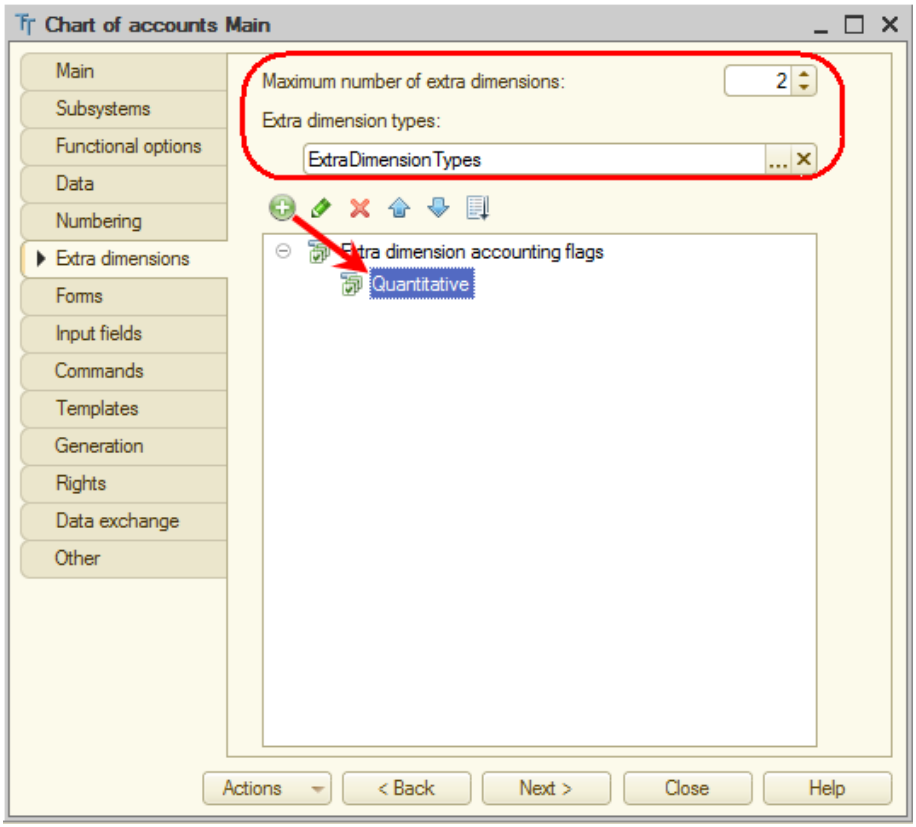


Fig. 16.5. Specifying extra dimension properties for a chart of accounts

8. On the **Other** tab, click **Predefined data**.
9. Create the four predefined accounts (to create each account, click the root **Accounts** item and then click the **Add** button):
   - **Inventory**. Code: 5000, type: Active, with quantitative accounting by material (fig. 16.6).

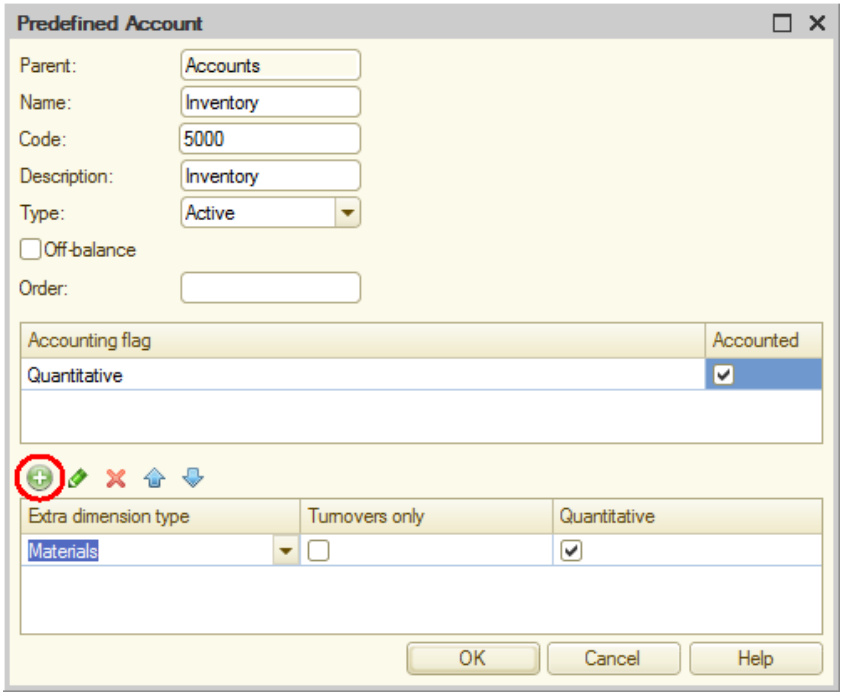


Fig. 16.6. Predefined account: Inventory

- **AccountsPayable**. Code: 4000, type: Active/Passive (fig. 16.7).

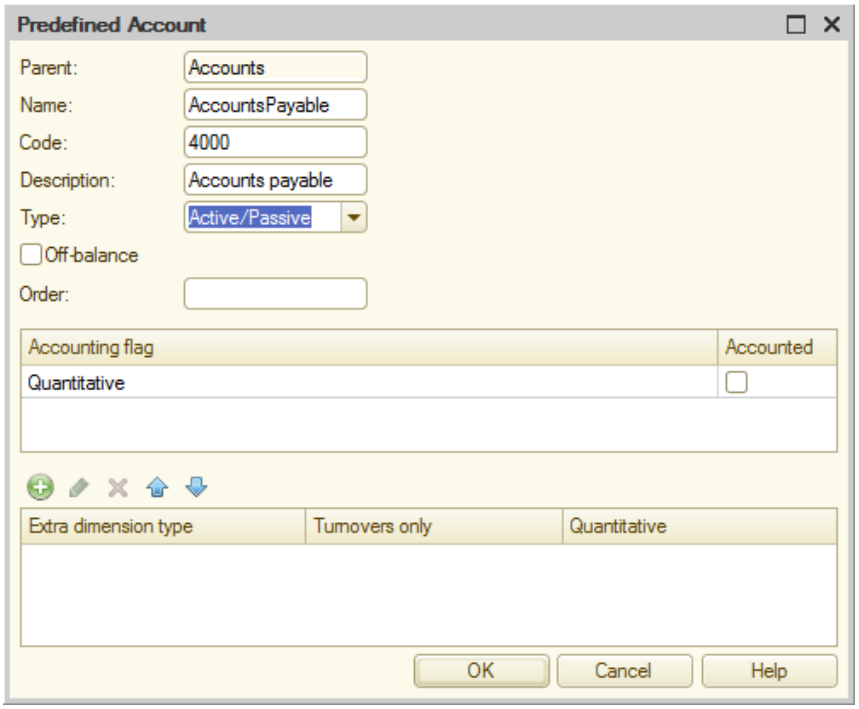


Fig. 16.7. Predefined account: AccountsPayable

- **AccountsReceivable**. Code 2000, Type: Active/Passive, with accounting by customers (fig. 16.8).

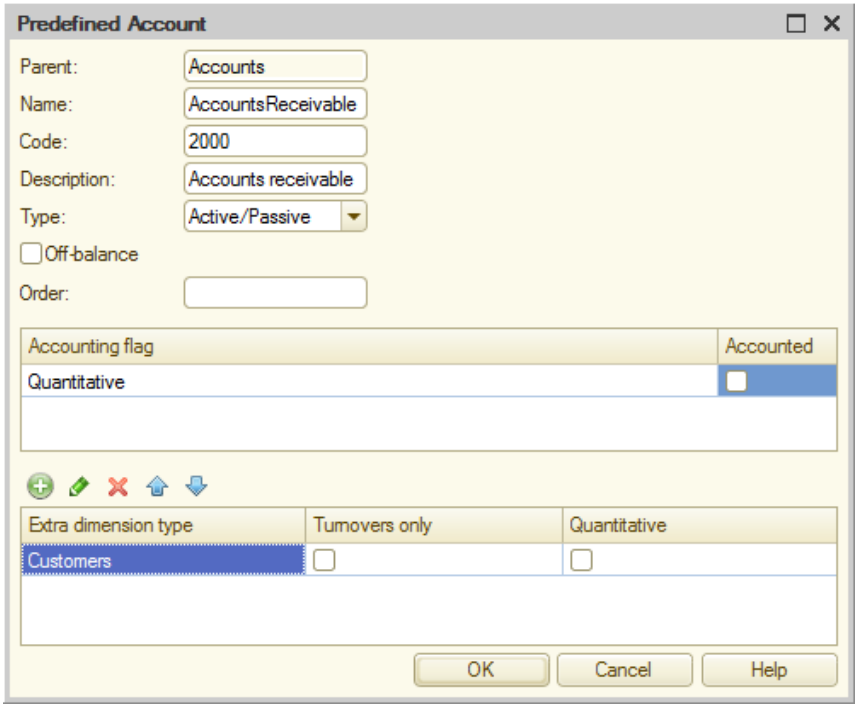


Fig. 16.8. Predefined account: AccountsReceivable

- **Income**. Code: 9000, Type: Active/Passive (fig. 16.9).

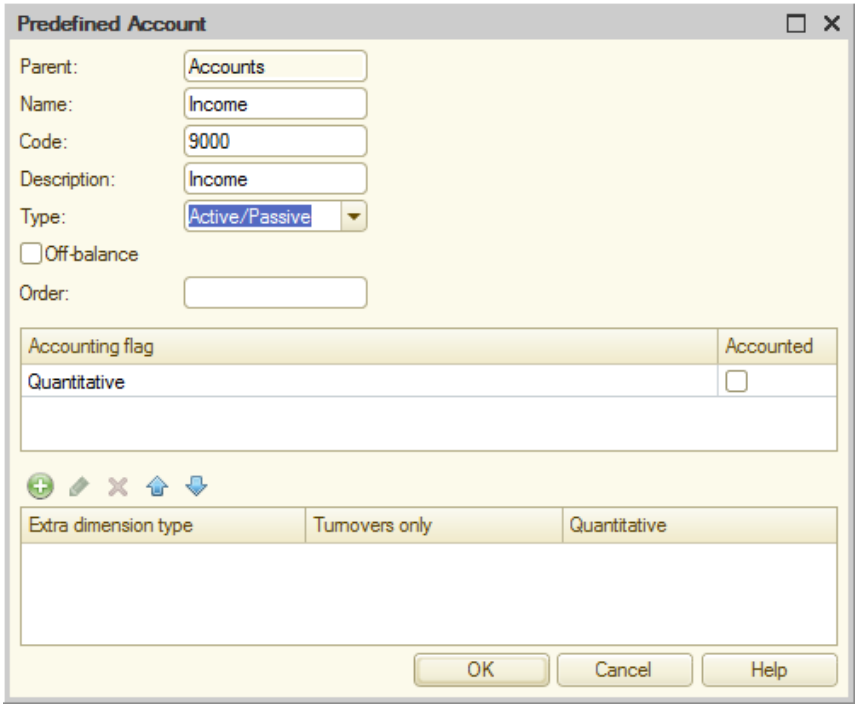


Fig. 16.9. Predefined account: Income

This results in the following arrangement of the chart of accounts for Jack of All Trades (fig. 16.10):

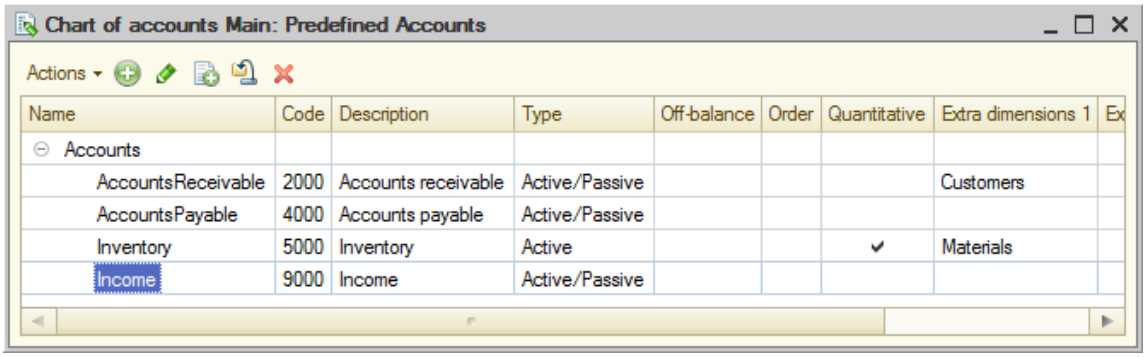


Fig. 16.10. Main chart of accounts

Now you can move on to the last configuration object that you need to implement accounting operations: the Accounting register.

Learn more! You can set the Autoorder by code property for a chart of accounts. This property specifies that sorting by the Order field is always applied when a user or a developer selects sorting by code. We recommend that you use this property when you need to sort a chart of accounts by code with regard to the account code delimiter. For example, when sorting by account code is applied to the accounts 10.11 and 10.2, the accounts are arranged as follows:

10.11
10.2

This is correct from the perspective of row ordering, but this does not comply with the code assignment logic.

But if the values of the Order field are 10.11 and 10. 2 (there is a space before 2) and the Autoorder by code property is set, when a user selects ordering by code, they get an order that takes the delimiters into consideration:

10.2
10.11.

If the property is not set, the user has to select sorting by the Order field explicitly.

Understanding accounting registers

The Accounting register configuration object is intended to describe the data accumulation structure used for storing accounting data based on a chart of accounts. Based on the Accounting register configuration object, the platform creates database tables that accumulate accounting data.

An accounting register looks very similar to an accumulation register. It also has resources, and it can have dimensions and attributes.

Dimensions allow you to break down accounting data (for example, you can use the Company dimension to keep records for multiple companies).

Attributes serve as flags that allow you to distinguish between various register records (for example, you can use a document journal number as

an attribute, which allows you to select postings that have similar purposes).

The main difference between an accounting register and an accumulation register is that the former is tightly linked to a chart of accounts. Therefore, each accounting register record contains additional fields that are defined by the chart of accounts settings.

For example, a register record can contain additional fields to store corresponding accounts, amounts, analytical accounting objects (extra dimensions), quantities, currency types, and so on.

Another distinguishing feature of an accounting register is its support of double-entry accounting, where each register record has fields for specifying debit and credit accounts.

Learn more! For details on the structure of 1C:Enterprise script objects intended for accounting register operations, see section "Quick developer reference. Accounting registers" on page 866.

Adding an accounting register

In Designer mode

Let us create an accounting register.

1. In Designer, create an AccountingRegister object named *Primary*.
2. In the **Extended list presentation** field, enter *Accounting register records*.
3. Specify that the register is linked to the **Main** chart of accounts.
4. Select the **Correspondence** check box (fig. 16.11).

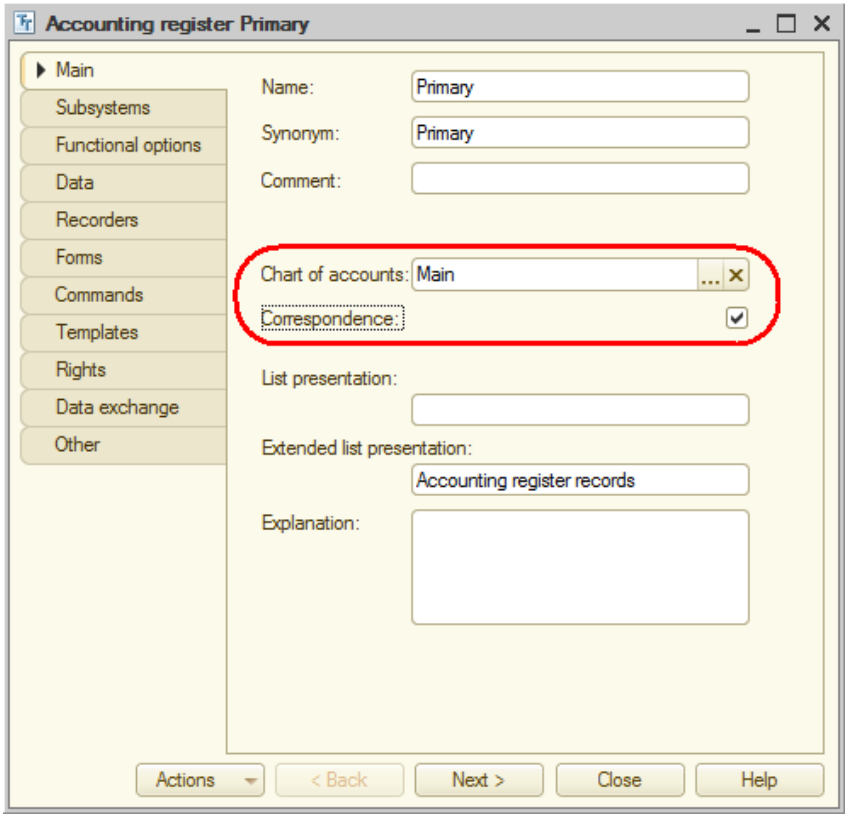


Fig. 16.11. Main properties of an accounting register

The Correspondence check box indicates that the created register supports account correspondence. That means that each register record has both debit and credit parts, which allows you to get not only balance and turnovers by account, but also the correspondence between accounts.

Registers that do not support correspondence are used when double-entry accounting is not required (which means that balancing between debit and credit accounts within an accounting transaction is not required).

5. On the **Subsystems** tab, include the accounting register in the **Accounting** subsystem.
6. On the **Data** tab, create two resources:
   - **Sum**. Length: 15, precision: 2, Balance
   - **Quantity**. Length: 15, precision: 3, not Balance, accounting flag: Quantitative, extra dimension accounting flag: Quantitative (fig. 16.12).

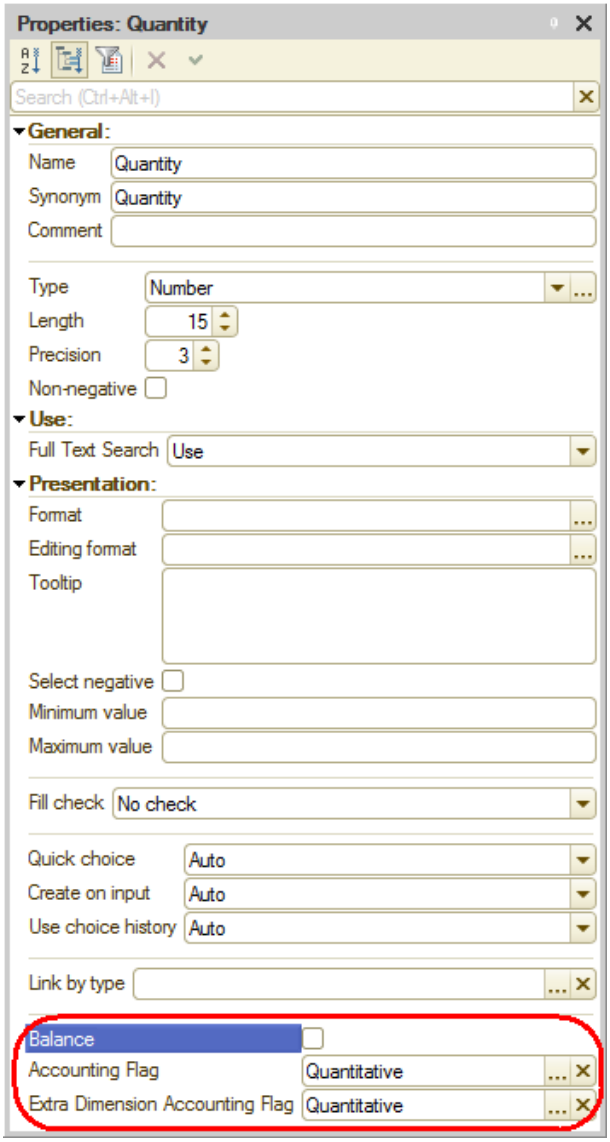


Fig. 16.12. Properties of the Quantity resource of the accounting register

Finally, let us add the command that opens the register list to the Accounting section, after all other register-related commands.

7. In the configuration tree, right-click the **Accounting** subsystem and then click **Open command interface**.
8. In the **Navigation panel.Normal** group, enable visibility for the **Primary** command and drag it to the **Navigation panel.See also** group.

   This completes the creation of the accounting register.

Now let us see how you can use the accumulation register.

Modifying the GoodsReceipt document

So what comes next?

First you will update both your documents (GoodsReceipt and Services) to provide data not only for the accumulation registers, but also for the accounting register.

Then you will create a Trial balance accounting report, which shows the inventory amounts at Jack of All Trades based on the accounting register data.

During the document posting the following accounting postings (records in the accounting register) should be created (table 16.1).

Table 16.1. Postings created by documents

DOCUMENTS	POSTINGS				
	DEBIT		CREDIT		SUM
Goods receipt	5000	Inventory	4000	Accounts payable	Cost
Services	2000	Accounts receivable	9000	Income	Total
	9000	Income	5000	Inventory	Cost

Let us implement the postings to the accounting register.

You have to modify the posting procedure for the GoodsReceipt document and then repost all GoodsReceipt documents in 1C:Enterprise in order to execute the modified posting algorithm.

In Designer mode

Let us modify the posting procedure.

1. In Designer, open the editor of the **GoodsReceipt** document configuration object.
2. On the **Posting** tab, specify that the document creates records in the **Primary** accounting register.
3. On the **Other** tab, open the object module.
4. Open the **Posting** event handler procedure, and at the end of the loop before the EndDo line add the script that generates Primary register records (listing 16.1).

Listing 16.1. Register records of the GoodsReceipt document (fragment)

```
// register Primary
Record = RegisterRecords.Primary.Add();
Record.AccountDr = ChartsOfAccounts.Main.Inventory;
Record.AccountCr = ChartsOfAccounts.Main.AccountsPayable;
Record.Period = Date;
Record.Sum = CurRowMaterials.Total;
```

```
Record.QuantityDr = CurRowMaterials.Quantity;
Record.ExtDimensionsDr[ChartsOfCharacteristicTypes.ExtraDimensionTypes
    .Materials] = CurRowMaterials.Material;
```

5. Before the beginning of the loop, set the Write property of the Primary register record set to True.

   This writes the register changes to the database.

   The resulting Posting procedure should look as shown in listing 16.2.

Listing 16.2. Register records of the GoodsReceipt document

```
Procedure Posting(Cancel, Mode)

    RegisterRecords.BalanceOfMaterials.Write = True;
    RegisterRecords.CostOfMaterials.Write = True;
    RegisterRecords.Primary.Write = True;

    For Each CurRowMaterials In Materials Do

        // register BalanceOfMaterials Receipt
        Record = RegisterRecords.BalanceOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Receipt;
        Record.Period = Date;
        Record.Material = CurRowMaterials.Material;
        Record.PropertySet = CurRowMaterials.PropertySet;
        Record.Warehouse = Warehouse;
        Record.Quantity = CurRowMaterials.Quantity;

        // register CostOfMaterials Receipt
        Record = RegisterRecords.CostOfMaterials.Add();
        Record.RecordType = AccumulationRecordType.Receipt;
        Record.Period = Date;
        Record.Material = CurRowMaterials.Material;
        Record.Cost = CurRowMaterials.Total;

        // register Primary
        Record = RegisterRecords.Primary.Add();
        Record.AccountDr = ChartsOfAccounts.Main.Inventory;
        Record.AccountCr = ChartsOfAccounts.Main.AccountsPayable;
        Record.Period = Date;
        Record.Sum = CurRowMaterials.Total;
        Record.QuantityDr = CurRowMaterials.Quantity;
        Record.ExtDimensionsDr[ChartsOfCharacteristicTypes
           .ExtraDimensionTypes.Materials] = CurRowMaterials.Material;

    EndDo;
EndProcedure
```

Now let us examine this script.

As you can see, accounting register records are generated in the same manner as accumulation register records.

But during the creation of the data storage table the platform added a number of attribute fields that originate from using the Main chart of accounts.

First of all, these are the AccountDr and AccountCr fields. These fields store accounts that have their debit and credit values affected by this posting.

Besides, for each register dimension or attribute that is linked to an accounting flag the platform creates a pair of fields to store resource values separately for posting debits and credits: QuantityDt and QuantityCr. And for the accounts that have extra dimensions the platform creates the ExtDimensionDr and ExtDimensionCr collections.

If you take a look at the table 16.1, you will see that when a goods receipt is posted, the Inventory (5000) account serves as a debit account while AccountsPayable (4000) serves as a credit account.

You access an account using the ChartsOfAccounts global context property. This property provides access to all the charts of accounts available in the configuration. Then you specify the name of the required chart of accounts (Main), separated by a dot. And then you specify the name of a predefined account from this chart of accounts (Inventory) after another dot. It is one of the four predefined accounts that you created earlier.

Since quantitative accounting is only available for the Inventory (5000) account, the QuantityDr register field is filled with the quantity of material units from the document tabular section. The QuantityCr register field is not filled because quantitative accounting is not applied to the credit account of the posting (AccountsPayable).

Now look at the last line of the loop where a value is assigned to the debit extra dimension.

For each posting, the number of extra dimensions on the debit account is not equal to the number of extra dimensions on the credit account. These numbers are taken from the account definitions in the chart of accounts. Therefore, for each accounting register record, the platform stores two value collections: a collection of debit extra dimensions and a collection of credit extra dimensions. Each of them contains a number of elements equal to the number of extra dimension types defined for the respective account (debit or credit) in the chart of accounts.

You can access a collection element by specifying the respective extra dimension type (ChartsOfCharacteristicTypes.ExtraDimensionTypes.Materials) in square brackets.

Another way to access a collection element is specifying the name of a predefined extra dimension type separated by a dot from the debit extra dimension collection.

In other words, the statement Record.ExtDimensionsDr[ChartsOfCharacteristicTypes.ExtraDimensionTypes.Materials] gives the same result as the statement Record.ExtDimensionDr.Materials.

The ExtDimensionCr register collection is not filled because accounting by extra dimensions is not applied to the credit account of the posting (AccountsPayable).

Finally, let us edit the command interface of the document form to add the command that opens the list of the Primary register records related to that document to the form navigation panel.

1. Open the **GoodsReceipt** document form.
2. In the upper left pane, click the **Command interface** tab.
3. In the **Navigation panel** branch, expand the **Go to** node and select the **Visible** check box for the **Primary** command.

In 1C:Enterprise Mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.

   The platform displays a warning stating that the Primary accounting register and the ExtraDimensions catalog are not included in any subsystems. Ignore the warning.
2. Open the GoodsReceipt document #1 and click **Post**.
3. Click **Primary** to open the accounting register records generated by the document (fig. 16.13, 16.14, and 16.15).

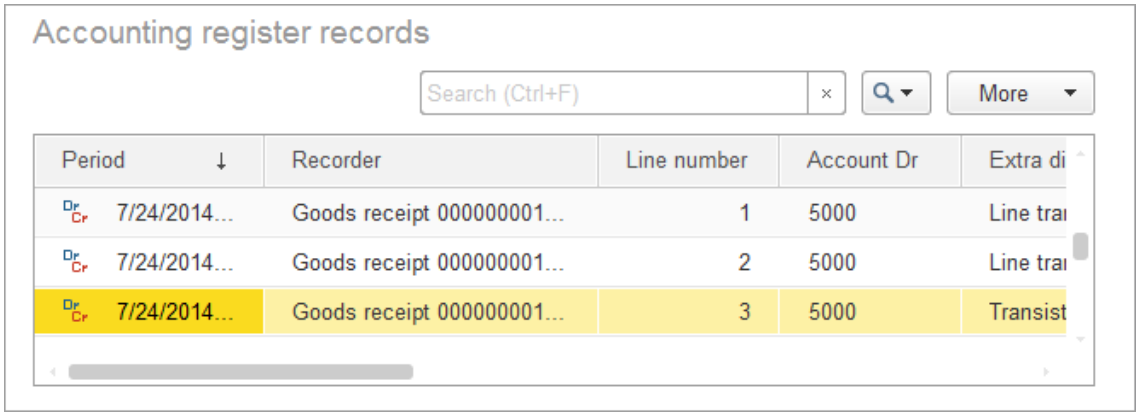


Fig. 16.13. Goods receipt #1 register records in the Primary accounting register

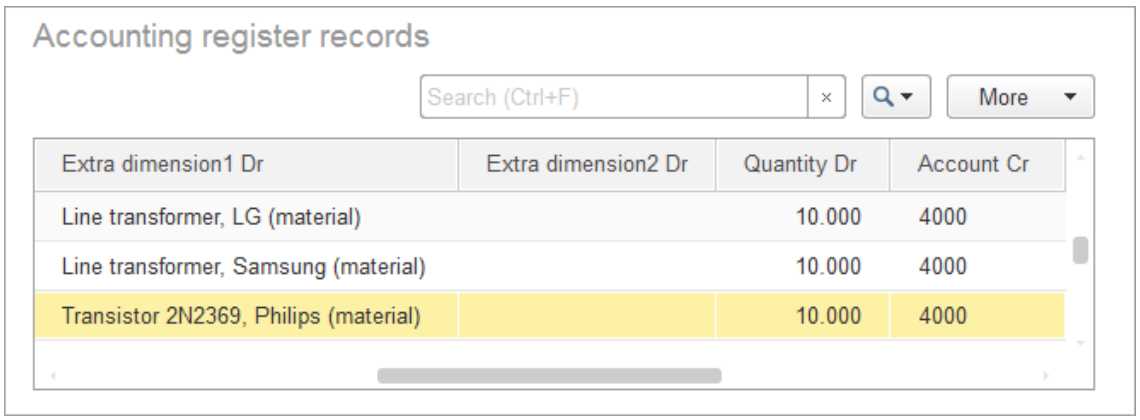


Fig. 16.14. Goods receipt #1 register records in the Primary accounting register

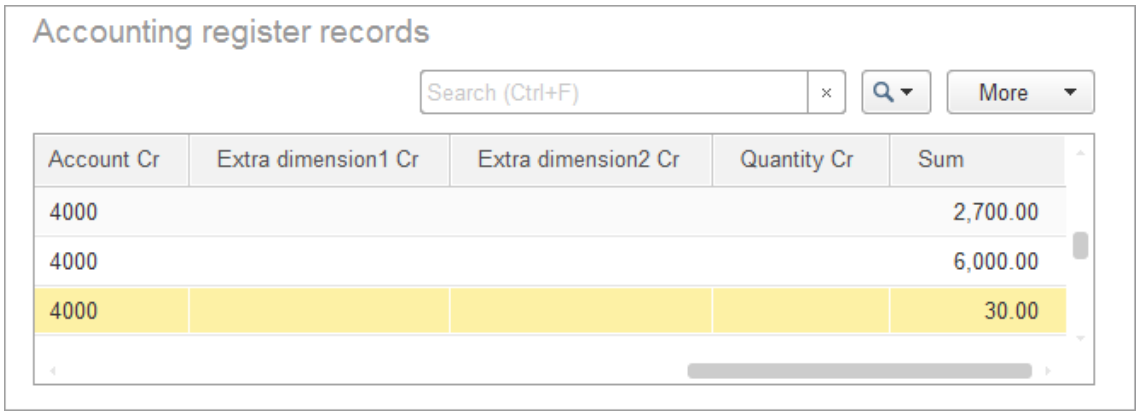


Fig. 16.15. Goods receipt #1 register records in the Primary accounting register

Note: since there is no analytical accounting for account 4000 (AccountsPayable) and all you track is a sum total, there are no records in the register for Extra dimension1 Cr, Extra dimension2 Cr, and Quantity Cr.

4. Repost the GoodsReceipt document #2 and ensure that it also generates correct records in the Primary accounting register.

Now let us proceed to a more complex task: adding records of the Services document to the Primary register.

Modifying the Services document

First let us modify the Service document posting procedure, and then repost all Services documents in 1C:Enterprise mode to execute the updated posting algorithm.

In Designer mode

In contrast to the GoodsReceipt document that generates a single accounting posting, the Services document will generate two postings.

Remember that the accounting process at Jack of All Trades is simpler than real-life accounting, since to make the job easier we only perform accounting for receipts and expenses of materials. The services provided by the company do not exist for the purposes of accounting.

Therefore, the Services document should generate accounting register records only for rows that include expenses of materials.

1. In Designer, open the editor of the **Services** document configuration object.
2. On the **Posting** tab, specify that the document creates records in the **Primary** accounting register.
3. On the **Other** tab, open the object module.
4. Open the **Posting** event handler procedure.

Since you only need to create records related to materials, your additions will perfectly fit the If… clause that includes the generation of records for the BalanceOfMaterials and CostOfMaterials registers.

5. Add the records for the Primary accounting register to the end of the condition, before the EndIf line (listing 16.3).

Listing 16.3. Register records of the Services document (fragment)

```
...
If SelectionDetailRecords.MaterialServiceType =
    Enums.MaterialServiceTypes.Material Then


    // register BalanceOfMaterials Expense

    ...


    // register CostOfMaterials Expense

    ...


    // register Primary

    // First posting: Dr 2000 (AccountsReceivable) – Cr 9000 (Income)

    // Total

    Record = RegisterRecords.Primary.Add();

    Record.AccountDr = ChartsOfAccounts.Main.AccountsReceivable;

    Record.AccountCr = ChartsOfAccounts.Main.Income;

    Record.Period = Date;

    Record.Sum = SelectionDetailRecords.TotalInDocument;

    Record.ExtDimensionsDr[ChartsOfCharacteristicTypes.
        ExtraDimensionTypes.Customers] = Customer;


    // Second posting: Dr 9000 (Income) – Cr 5000 (Inventory) Cost

    Record = RegisterRecords.Primary.Add();

    Record.AccountDr = ChartsOfAccounts.Main.Income;

    Record.AccountCr = ChartsOfAccounts.Main.Inventory;

    Record.Period = Date;

    Record.Sum = MaterialCost
        * SelectionDetailRecords.QuantityInDocument;

    Record.QuantityCr = SelectionDetailRecords.QuantityInDocument;

    Record.ExtDimensionsCr[ChartsOfCharacteristicTypes.
        ExtraDimensionTypes.Materials] =
        SelectionDetailRecords.MaterialOrservice;


EndIf;

...
```

In the first posting you specify the document total as the record amount. The debit extra dimension for Accounts receivable is extended by customer.

In the second posting you specify the cost of materials as the record

amount. Additionally you specify the quantity and the selected items as the credit extra dimension because the Inventory account features quantitative accounting and is divided by material.

6. At the very beginning of the procedure, set the Write property of the accounting register to True (listing 16.4).

   This means that the register changes are recorded to the database.

Listing 16.4. Writing register records

```
Procedure Posting(Cancel, Mode)

    RegisterRecords.BalanceOfMaterials.Write = True;
    RegisterRecords.CostOfMaterials.Write = True;
    RegisterRecords.Sales.Write = True;
    RegisterRecords.Primary.Write = True;
    ...
```

Finally, let us edit the command interface of the document form to add the command that opens the list of the Primary register records related to that document to the form navigation panel.

1. Open the **Services** document form.
2. In the upper left pane, click the **Command interface** tab.
3. In the **Navigation panel** branch, expand the **Go to** node and select the **Visible** check box for the **Primary** command.

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. Open the Services document #1 and click **Post**.
3. Click **Primary** to open the accounting register records generated by the document (fig. 16.16, 16.17, and 16.18).

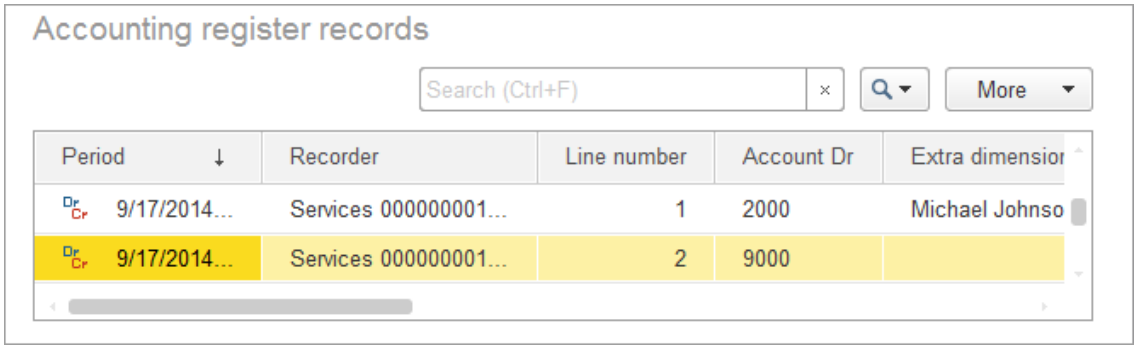


Fig. 16.16. Services #1 register records in the Primary accounting register

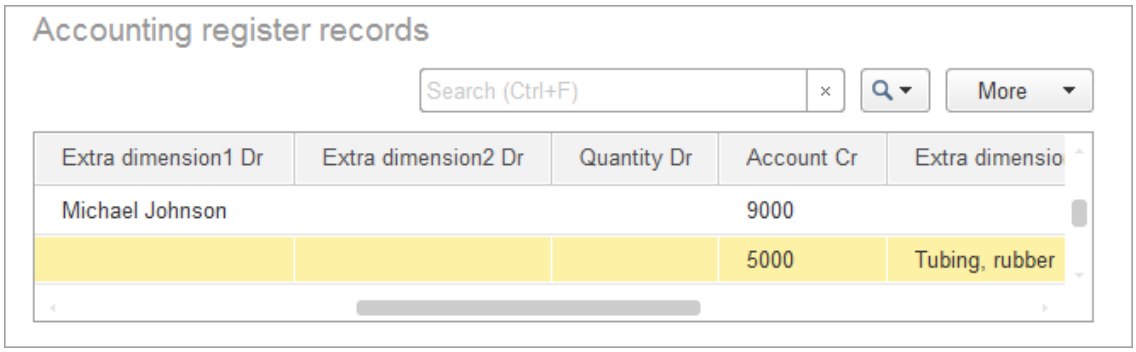


Fig. 16.17. Services #1 register records in the Primary accounting register

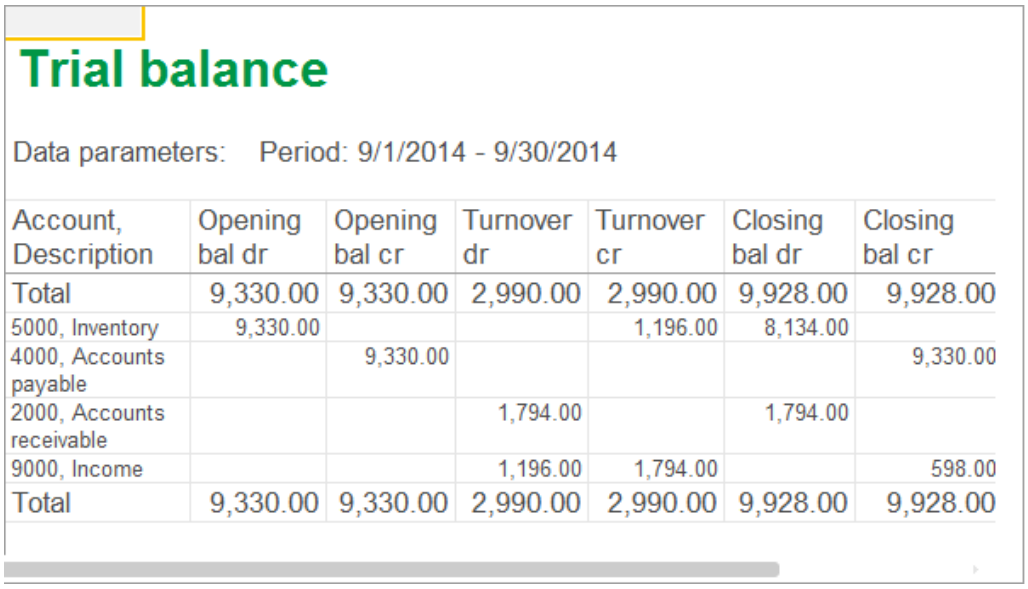
Fig. 16.18. Services #1 register records in the Primary accounting register

4. Repost the remaining Services documents.

Trial balance

Now all that is left for you to do is create a report for the accounting department at Jack of All Trades, and you will be done with learning accounting registers.

The only report the accounting department of your company uses is the Trial balance report (fig. 16.19).



Trial balance

Data parameters: Period: 9/1/2014 - 9/30/2014

Account, Description	Opening bal dr	Opening bal cr	Turnover dr	Turnover cr	Closing bal dr	Closing bal cr
Total	9,330.00	9,330.00	2,990.00	2,990.00	9,928.00	9,928.00
5000, Inventory	9,330.00			1,196.00	8,134.00	
4000, Accounts payable		9,330.00				9,330.00
2000, Accounts receivable			1,794.00		1,794.00	
9000, Income			1,196.00	1,794.00		598.00
Total	9,330.00	9,330.00	2,990.00	2,990.00	9,928.00	9,928.00

Fig. 16.19. Resulting report

In Designer mode

Let us proceed to report creation.

1. Create a report configuration object named *TrialBalance*.
2. Run the Data composition schema wizard.
3. Add **Data set – query** and run the Query builder.

Query for a data set

The Trial balance accounting report is a table with its rows listing all the accounts in the chart of accounts, and its columns displaying the opening balance, turnover, and the closing balance for the debits and credits of each account.

1. Select the following tables as the data sources (fig. 16.20):
   - **Main** chart of accounts object (referential) table
   - **Primary.BalanceAndTurnovers** accounting register virtual table

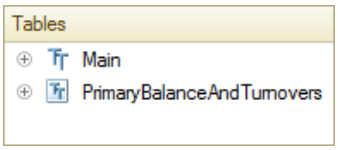


Fig. 16.20. Query tables

2. Select the **Ref** field from the **Main** table.
3. Select the following fields from the **PrimaryBalanceAndTurnovers** table (fig. 16.21):
   - **SumOpeningSplittedBalanceDr**
   - **SumOpeningSplittedBalanceCr**
   - **SumTurnoverDr**
   - **SumTurnoverCr**
   - **SumClosingSplittedBalanceDr**
   - **SumClosingsplittedBalanceCr**

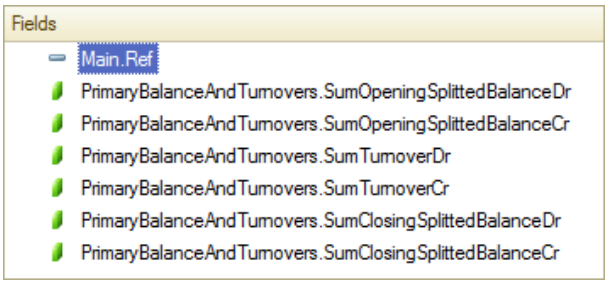


Fig. 16.21. Fields selected for the query

4. Click the **Links** tab and specify that you want to select all records from the main table and only those that match the link condition from the register table (fig. 16.22).

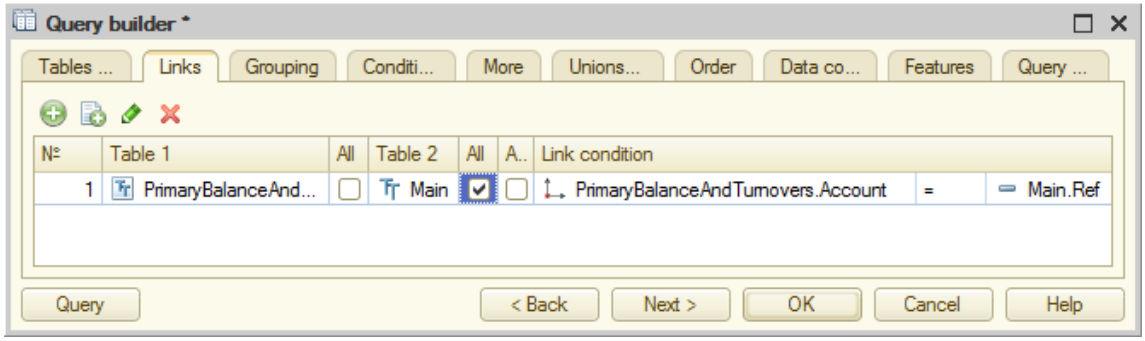


Fig. 16.22. Table link condition

5. Click the **Unions/Aliases** tab and set the following aliases for the report fields (fig. 16.23):
   - **Account**
   - **OpeningBalDr**
   - **OpeningBalCr**
   - **TurnoverDr**
   - **TurnoverCr**
   - **ClosingBalDr**
   - **ClosingBalCr**

Field name	Query 1
Account	Main.Ref
OpeningBalDr	PrimaryBalanceAndTurnovers.SumOpeningSplittedBalanceDr
OpeningBalCr	PrimaryBalanceAndTurnovers.SumOpeningSplittedBalanceCr
TurnoverDr	PrimaryBalanceAndTurnovers.SumTurnoverDr
TurnoverCr	PrimaryBalanceAndTurnovers.SumTurnoverCr
ClosingBalDr	PrimaryBalanceAndTurnovers.SumClosingSplittedBalanceDr
ClosingBalCr	PrimaryBalanceAndTurnovers.SumClosingSplittedBalanceCr

Fig. 16.23. Unions/aliases

6. Click **OK**.

   This completes the query creation.

Resources

Let us add the resources.

- On the **Resources** tab, select all available resources.

Parameters

Normally, accounting reports are generated for a specific period: month, quarter, year, and so on. We will use this report to show how to set the report period using the "standard period" feature.

1. On the **Parameters** tab, add the *Period* parameter of the **StandardPeriod** type.
2. For the **BeginOfPeriod** and **EndOfPeriod** parameters, specify the calculation expressions (listing 16.5) and prohibit user editing.

   Listing 16.5. Expressions for calculating the BeginOfPeriod and EndOfPeriod parameters

   ```
   &Period.StartDate
   &Period.EndDate
   ```

3. For the **Period** parameter, in the **Usage** column, select **Always**.

   As a result, the check box that shows whether the report period is used is hidden from the report form. This is required because accounting reports are always generated for a specific period.

   The resulting data composition parameters should look as shown in fig. 16.24.

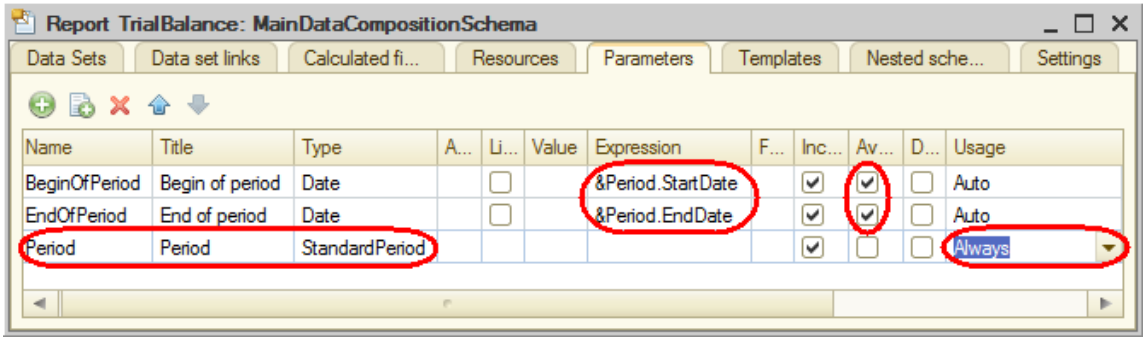


Fig. 16.24. Data composition schema parameters

Note that the beginning and end dates of a standard period include time as well. But here, unlike the BeginOfPeriod and EndOfPeriod parameters, the beginning date has the time 00:00:00 while the end date has the time 23:59:59. Hence, the last day is included in the report and there is no need to use the EndOfPeriod() function.

Settings

Let us specify the report structure.

1. On the **Settings** tab, add the <Detailed records> grouping.
2. On the **Selected fields** tab, select the **Account** field, then expand this field and select its **Description** attribute.
3. Select all register fields and arrange them as shown in fig. 16.25.

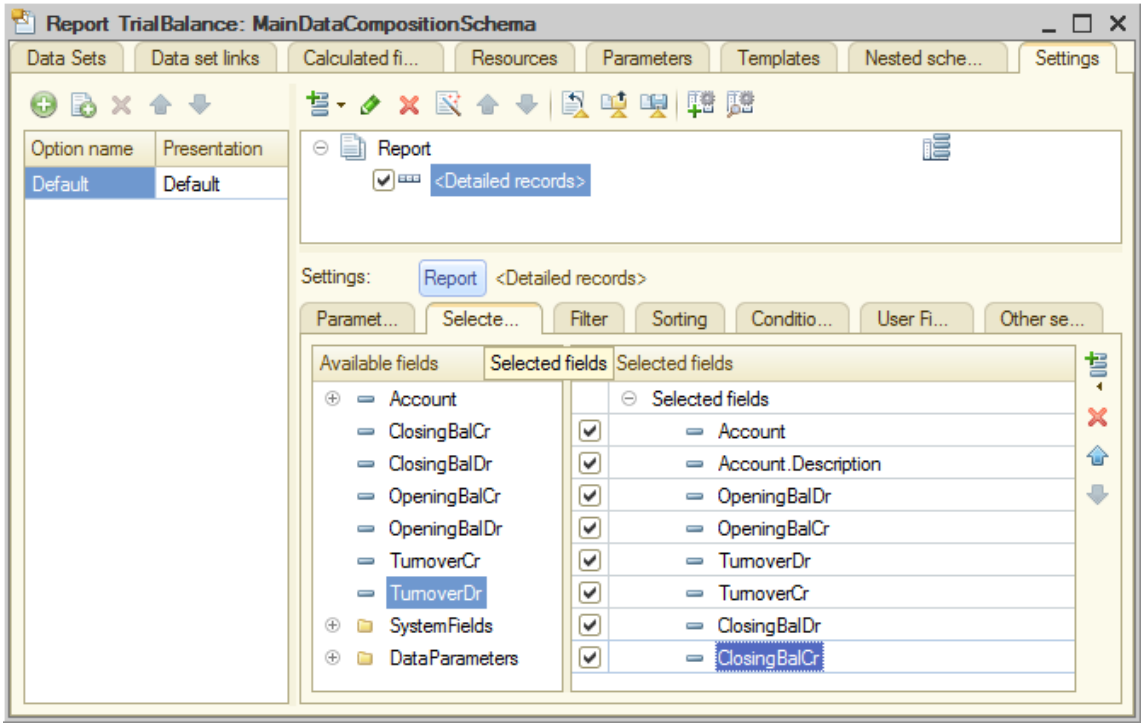


Fig. 16.25. Report groupings and fields

4. On the **Other settings** tab, set the report title to *Trial balance*, and set **Vertical placement of overalls** to **Begin and end**.
5. On the **Parameters** tab, for the **Period** parameter, select **This month** (fig. 16.26).

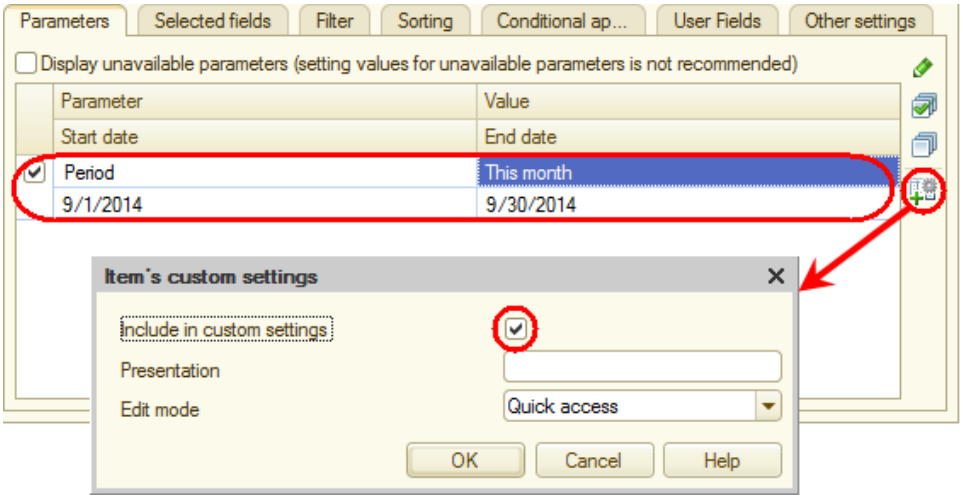


Fig. 16.26. Creating quick settings for the report period

This is how you ensure that the current month is always specified as a report period when the report form is opened. Remember that the beginning and end dates of the period are changed dynamically based on the report generation date.

6. Click the **Custom settings item property** button to specify that **Period** is included in the list of custom settings and this setting is available directly in the report form (**Quick access** edit mode).

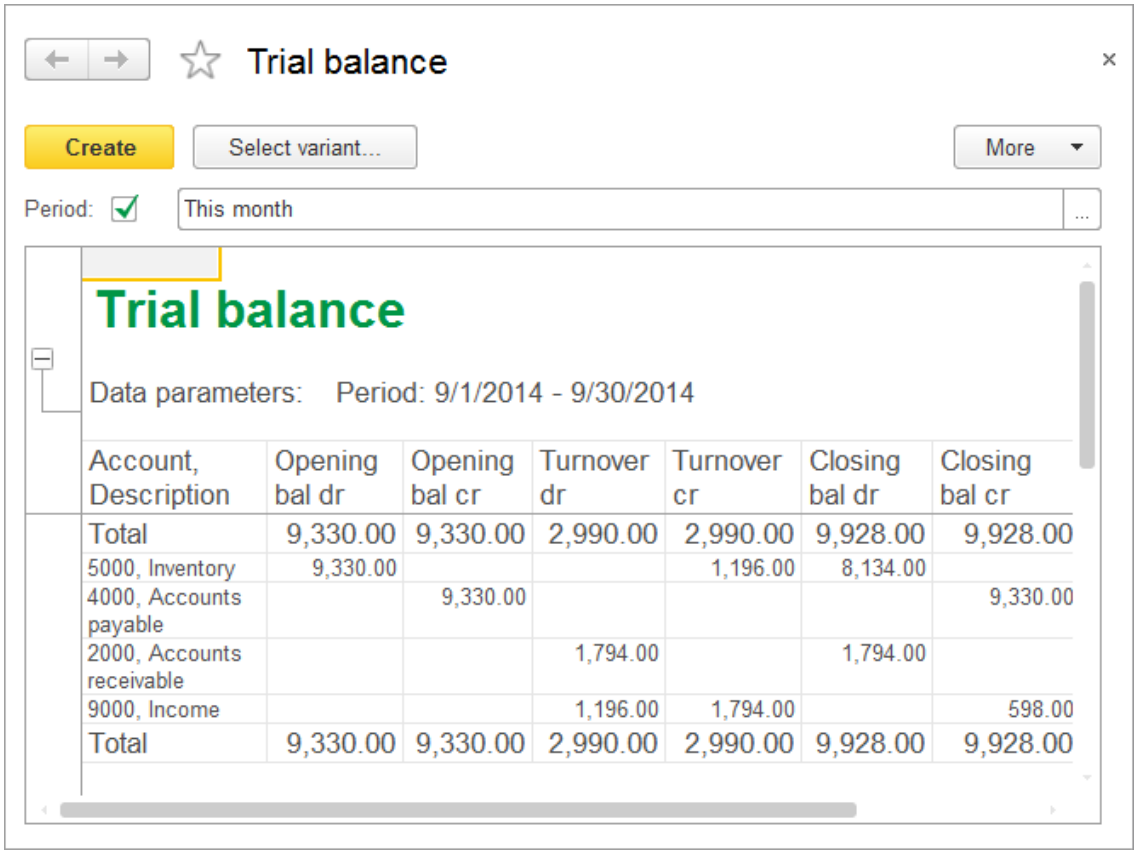
   Let us specify the subsystems where the report is available.

7. Close the data composition schema wizard and then, in the editor of the **TrialBalance** report configuration object, click the **Subsystems** tab.
8. Select the **Accounting** check box.

In 1C:Enterprise mode

Let us check the report.

1. Start 1C:Enterprise in the debug mode.
2. In the **Accounting** section, open the report and click **Create** (fig. 16.27).



Account, Description	Opening bal dr	Opening bal cr	Turnover dr	Turnover cr	Closing bal dr	Closing bal cr
Total	9,330.00	9,330.00	2,990.00	2,990.00	9,928.00	9,928.00
5000, Inventory	9,330.00			1,196.00	8,134.00	
4000, Accounts payable		9,330.00				9,330.00
2000, Accounts receivable			1,794.00		1,794.00	
9000, Income			1,196.00	1,794.00		598.00
Total	9,330.00	9,330.00	2,990.00	2,990.00	9,928.00	9,928.00

Fig. 16.27. Resulting report

You see that the default report period is specified as This month (a standard period). The standard report period feature is very convenient when users frequently need to generate the report for a specific period of time. In this scenario you can predefine the required period in the settings, so the user does not have to specify it before generating the report. But if they want, they can select another period from the list of standard ones in 1C:Enterprise mode.

Quiz

- How does one use a chart of characteristic types for accounting?
- What is an extra dimension?

- What are the uses of the Chart of accounts configuration object?
- How does one create a chart of accounts?
- What are the uses of Accounting register configuration object?
- How does one create an accounting register and specify accounting parameters?
- How does one create records in an accounting register using 1C:Enterprise script?
- How does one create a report based on accounting register data using the composition system?
- How does one define a standard report period?

Lesson 17 (1:00). Charts of calculation types and calculation registers

In this lesson you will study the Chart of calculation types configuration object and the Calculation register configuration object. You will also learn the basic concepts of creating complex periodic calculations.

At the end of the lesson you will create a chart of calculation types and a calculation register. You will use them in the next lesson to study periodic calculations.

Chart of calculation types and calculation register purposes

In this lesson we will discuss 1C:Enterprise features intended for automating complex periodic calculations.

Such calculations are most commonly used to calculate wages. Therefore, we will use the employee payroll at Jack of All Trades as an example for our discussion.

As a rule, employee wages are a function of many components (salary, bonuses, penalties, sick leaves, nonrecurring payments, and so on). Each of these components is calculated using some algorithm that is unique to this particular component.

For example, the penalty amount can be determined as a simple fixed amount, bonuses can be calculated as a percentage of salary, while salary might be calculated based on the number of working days in the month and the number of days the employee actually worked. Therefore, we will refer to each of these components as a calculation type.

An algorithm for each calculation type is generally based on two parameter types: a calculation period and some source data set.

In reality, calculation types generally do not exist in a vacuum, but instead somehow influence other calculations. Since a calculation type depends on two parameter types, this influence is also of a twofold nature.

Dependency by base period

A calculation type can influence the source data of another calculation type.

As an example, consider bonuses calculated as a percentage of salary. When salary changes, a bonus requires recalculation based on the new salary amount. In other words, the modified salary amount serves as the base for bonus calculation.

Additionally, since the salary is calculated for a given period, the bonus calculation requires not just the basic salary amount, but rather the total that is calculated for the period affecting the bonus calculation. We will

refer to that period as a base period, and we will refer to this dependency between calculation types as the dependency by base period.

For example, consider the accrual of bonuses for April. The bonus is calculated as 10% of the total calculated salary. So you need to analyze all the records affecting the accrual of salary for the base period you deal with, in this case for the month of April.

Suppose that the total of salary is $8 000, so the bonus would come to $800 (fig. 17.1).

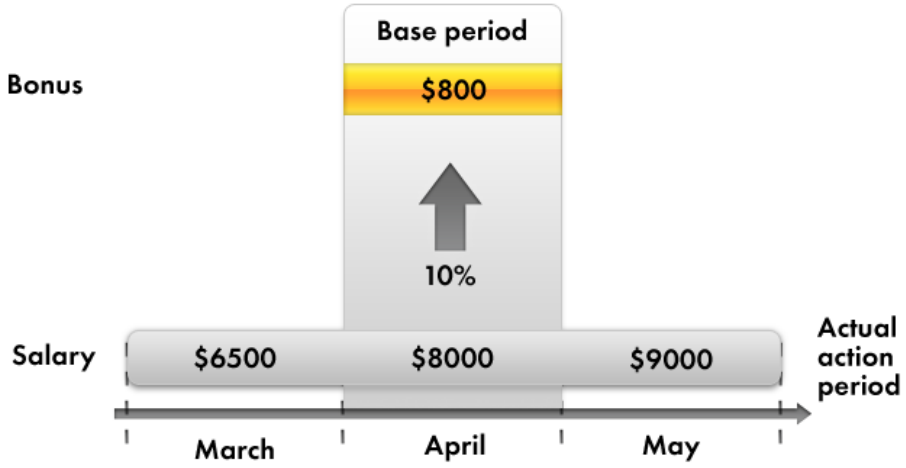


Fig. 17.1. Bonus dependency on salary by base period

Displacement by action period

A calculation type can influence the calculation period of another calculation type instead of its source data.

As an example, consider salary calculation with absences taken into account. Suppose that you calculate an employee's salary for March. In this case the action period for the calculation is from 3/1/2014 to 3/31/2014.

Then a manager informs you that the employee was absent from the 1st to the 10th of March for some unknown reason. So you need to perform the Absence calculation, which might introduce some deductions from the payment.

But you also need to recalculate the employee's salary based on the fact that the actual Salary calculation period is now from 3/11/2014 to 3/31/2014.

We will refer to this influence as the displacement by action period.

As a result, if the employee would have received $9,300 for a full month work, the accrual for the actual period comes to $6,300 (fig. 17.2).

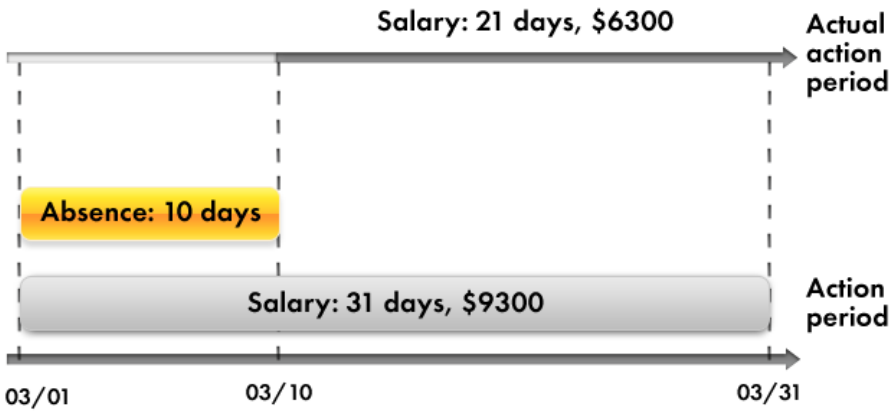


Fig. 17.2. A record of the Absence calculation displaces a record of the Salary calculation by action period

Thus, based on the two types of mutual influence of calculations, one could come to a conclusion that in general there are three periods associated with each calculation type: action period, actual period, and base period.

- Action period is a requested period. This means that when you specify an action period, your request is "I would like the result to be valid for this period."
- Actual period is what remains after analyzing all the calculation action periods that displace your calculation by action period.
- Base period is the period within which you analyze the results of other calculations that influence your calculation by that period.

As you see, the mutual influence of calculation types can vary greatly and can be multilayered, which complicates everything the most. In other words, one calculation type might influence another, which in turn affects a third one, and so on.

Obviously, this situation calls for some kind of universal mechanism that would allow you to define each of these calculation types (its algorithm, influences on other calculation types, and dependencies), to store data obtained as a result of these calculations, and to monitor the need to recalculate the results of dependent calculations upon changes in the results of "primary" calculations.

Within 1C:Enterprise this universal mechanism is implemented using charts of calculation types and calculation registers.

The first configuration object we will discuss during this lesson is the Chart of calculation types.

Understanding charts of calculation types

The Chart of calculation types configuration object is intended to define data storage structures for various calculation types. Based on a Chart of calculation types configuration object, the platform creates a database table that stores available calculation types and their interrelationships.

The distinctive characteristic of a chart of calculation types is that users can add new calculation types as they work. This feature makes the mechanism

of periodic calculations more flexible and allows users to create their own calculation types in addition to those predefined by the developer.

The Chart of calculation types configuration object has the Uses Action Period property. If the property is set, the developer has the option to specify the calculation types that displace the current calculation type by action period.

Another important Chart of calculation types property is Dependence on Base. This property shows whether this chart of calculation types contains calculation types that are dependent by base period.

If the property is set, the developer can specify the chart of calculation types that stores the base calculation types, and also define the dependencies.

There are two types of base dependencies to choose from: dependency by action period and dependency by registration period. Both dependency types are discussed in detail in section "Understanding calculation registers" on page 513.

Another important feature of chart of calculation types is the option to create predefined calculation types and describe how they influence each other. In doing so, the developer can generally define three categories of calculation types that influence the predefined types:

- Base calculation types. Their results are used for recalculation of the current calculation type.
- Displacing calculation types. They displace the current calculation type by action period.
- Leading calculation types. Changes in their results force the recalculation of the current calculation type.

Common sense will tell you that all the leading calculation types should encompass all the base calculation types. Besides, leading calculation types can also contain some other calculation types that indirectly influence the current calculation type.

For example, there are three calculation types: Absence, Salary, and Bonus. Absence displaces Salary by action period, while Bonus depends on Salary by base period.

In this scenario, Salary should be the base calculation type for Bonus, and both Salary and Absence should be leading types for Bonus, since a change of the Absence calculation result forces a change of the Salary result, which in turn forces the change of the Bonus result (fig. 17.3).

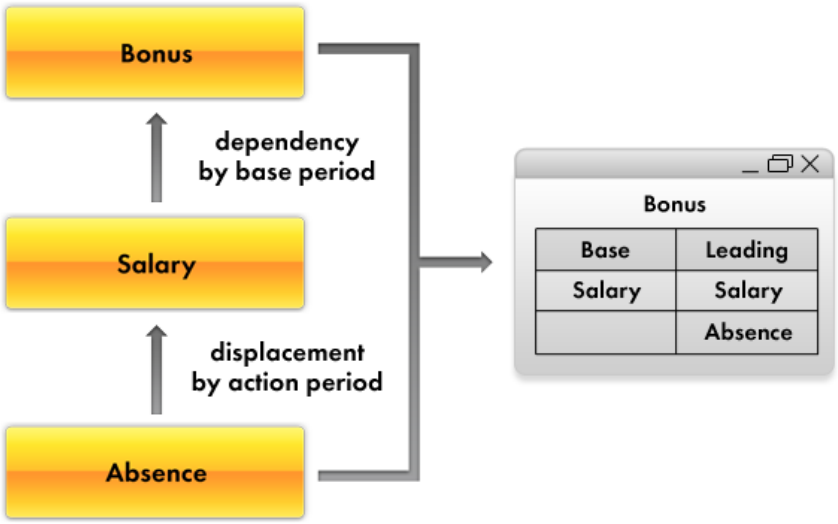


Fig. 17.3. Mutual influence of calculation types

Learn more! For details on the structure of 1C:Enterprise script objects intended for chart of calculation types operations, see section "Quick developer reference. Charts of calculation types" on page 854.

Adding a chart of calculation types

Let us proceed to creating a chart of calculation types named MainAccruals. You will use this chart of calculation types in your configuration.

In Designer mode

Let us create a chart of calculation types.

1. In Designer, create a ChartOfCalculationTypes object named *MainAccruals*.
2. In the **Object presentation** field, enter *Calculation types*.
3. On the **Subsystems** tab, include the chart of calculation types in the **Payroll** subsystem.
4. On the **Calculation** tab, specify that it uses action period and depends on base calculation type by action period.
5. Specify that this chart of calculation types serves as a base chart of calculation types for itself (fig. 17.4).

   Since it is the only chart of calculation types in the configuration, it will store all of the calculation types, including the base ones.

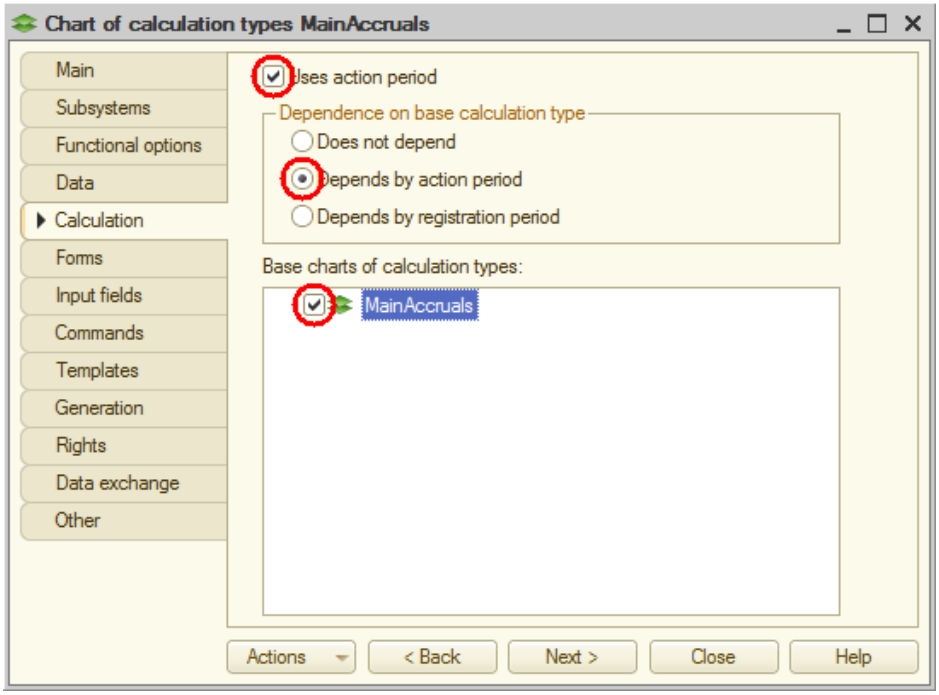


Fig. 17.4. Specifying the use of action period, dependency by base calculation type, and the base chart of calculation types

Let us specify the predefined calculation types. As with accounting, calculations at Jack of All Trades are rather basic, so you will only create three items.

6. On the **Other** tab, create the following predefined items (fig. 17.5):
   - **Absence**. Name, code, and description: Absence
   - **Salary**. Name, code, and description: Salary, displacing calculation type: Absence, leading calculation type: Absence
   - **Bonus**. Name, code, and description: Bonus, base calculation type: Salary, leading calculation types: Absence and Salary

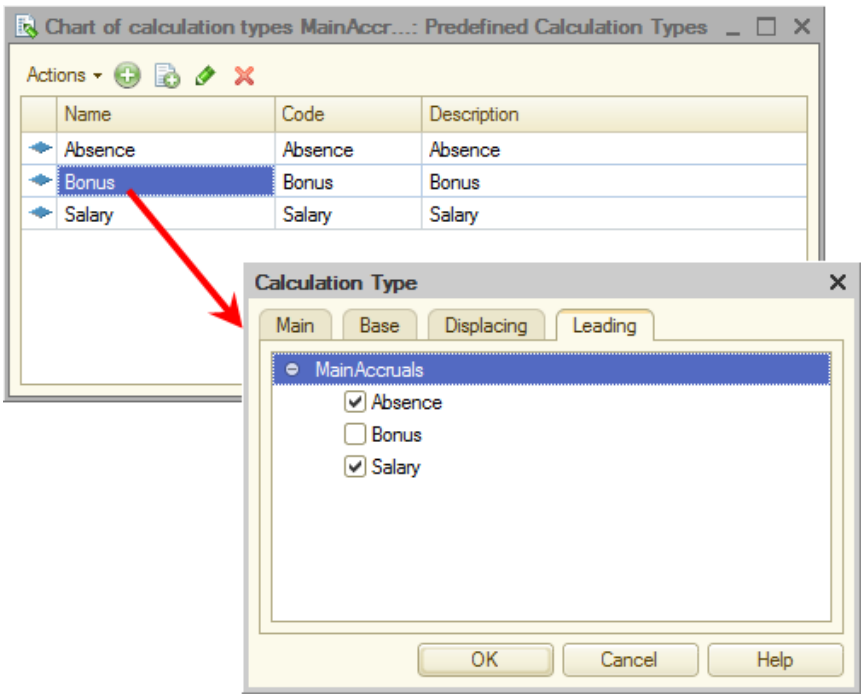


Fig. 17.5. Predefined calculation types for the MainAccruals chart of calculation types

Now let us proceed to discussing the second object used in the implementation of complex periodic calculations: the Calculation register.

Understanding calculation registers

The Calculation register configuration object is intended to describe the structure of accumulated data resulting from calculations. Based on the Calculation register configuration object, the platform creates database tables that store accumulated data generated by various database objects.

Calculation registers are not designed to be edited directly by users. A developer can allow users to edit a calculation register, if required. However, normally calculation registers are modified by algorithms of other database objects, rather than through a direct user intervention.

Just like any other register, a calculation register has resources where it stores numerical data, it has dimensions used for getting register resource values by those dimensions, and it has attributes, which describe each calculation register record.

What makes calculation registers different is their periodicity, the ability to use the algorithms of displacement by action period and dependency by base period, and the connection with a chart of calculation types. We will discuss each of these features in the following sections.

Periodicity

The periodicity of a calculation register can have one of the following values:

- Day
- Month
- Quarter
- Year

The calculation register periodicity defines the time period where each register record belongs.

If the periodicity is set to Day, each record in the register belongs to some day; if periodicity is set to Month, each register record belongs to some month, and so on.

To reflect the fact that a record belongs to a certain period, the register has the RegistrationPeriod system attribute of the Date type. When writing data to the register, the platform always sets the value of that attribute to the beginning of the period where the data belongs.

For example, if data is written to a calculation register with periodicity set to Month, and the RegistrationPeriod for that data is 4/8/2014, the register stores the data with the RegistrationPeriod value set to 4/1/2014 (fig. 17.6).

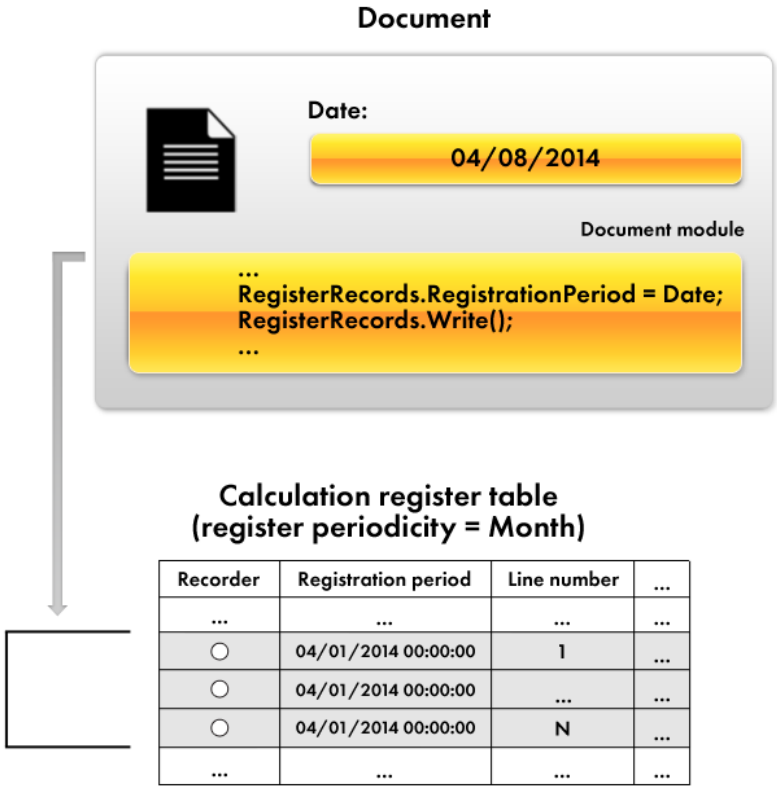


Fig. 17.6. Recording document data to a calculation register

If you set the register periodicity to Year in this scenario, the stored registration period value changes to 1/1/2014 (fig. 17.7).

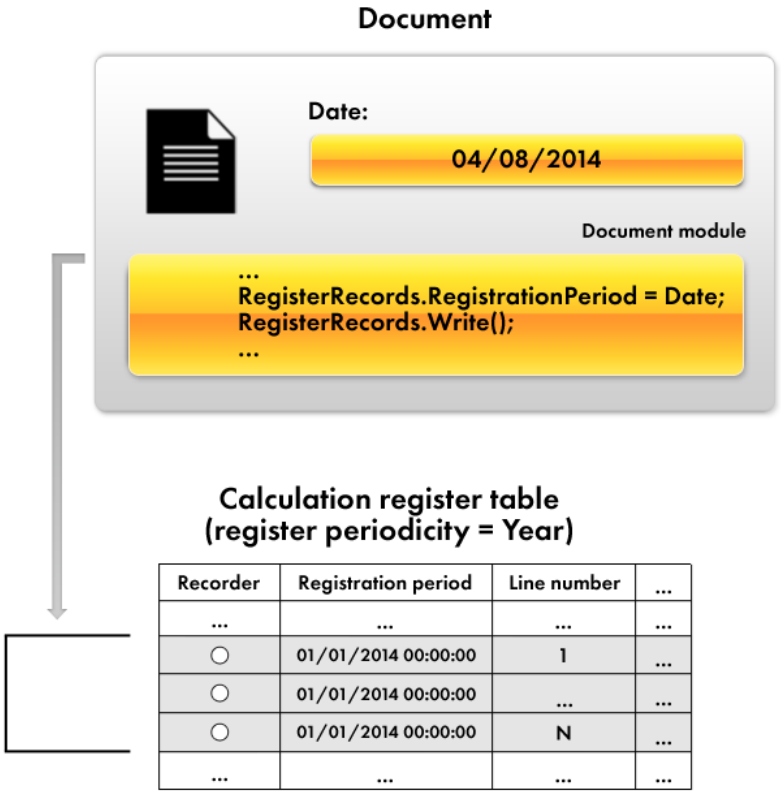


Fig. 17.7. Recording document data to a calculation register

Displacement by action period

Another important feature of a calculation register is the ability to displace certain records with other records by action period.

In doing so, each calculation register record generates an actual action period which, in general, represents the sum total of several periods contained within a single action period (fig. 17.8).

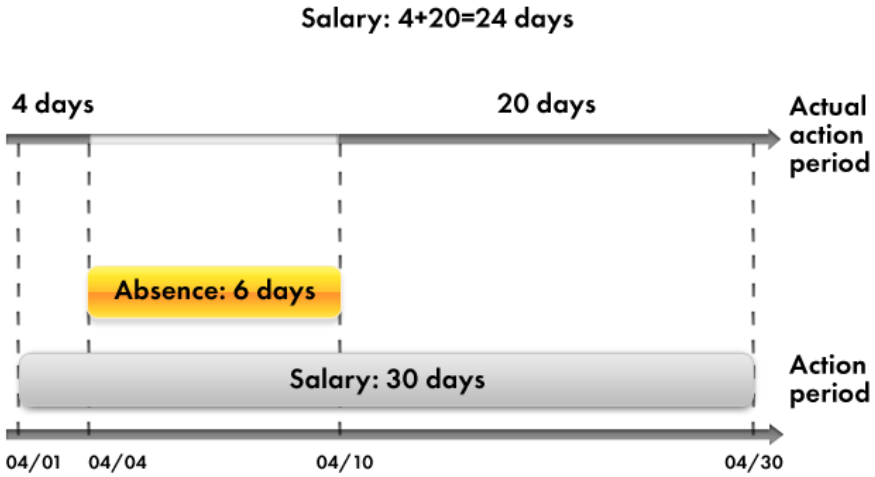


Fig. 17.8. Absence calculation record displaces a Salary calculation record by action period

After adding a salary accrual record, the structure of calculation register table records looks as shown in tables 17.1 and 17.2.

Table 17.1. Calculation register table

...	BEGINNING OF ACTION PERIOD	END OF ACTION PERIOD	CALCULATION TYPE	...
...	...	...	...	...
...	04/01/2014 00:00:00	04/30/2014 23:59:59	Salary	...
...	...	...	...	...

Table 17.2. Actual action period table

...	BEGINNING OF ACTION PERIOD	END OF ACTION PERIOD	CALCULATION TYPE	...
...	...	...	...	...
...	04/01/2014 00:00:00	04/30/2014 23:59:59	Salary	...
...	...	...	...	...

After adding records from the Absence calculation type, which displaces Salary calculations by action period, the records related to salary calculation look as shown in tables 17.3 and 17.4.

Table 17.3. Calculation register table

...	BEGINNING OF ACTION PERIOD	END OF ACTION PERIOD	CALCULATION TYPE	...
...	...	...	...	...
...	04/01/2014 00:00:00	04/30/2014 23:59:59	Salary	...
...	04/04/2014 00:00:00	04/10/2014 23:59:59	Absence	...
...	...	...	...	...

Table 17.4. Actual action period table

...	BEGINNING OF ACTION PERIOD	END OF ACTION PERIOD	CALCULATION TYPE	...
...	...	...	...	...
...	04/01/2014 00:00:00	04/03/2014 23:59:59	Salary	...
...	04/11/2014 00:00:00	04/31/2014 23:59:59	Salary	...
...	...	...	...	...

Dependency by base period

Another calculation register feature is support of register record dependency by base period. This feature is intended for calculating dependent (secondary) register records based on the result of primary record calculation.

A calculation register can support two types of dependency by base period: dependency by action period and dependency by registration period.

Dependency by action period

Dependency by action period means that when the platform analyzes base records, it selects the records where the actual action period and the specified base period overlap.

For example, at the beginning of April, March payroll is calculated. Bonuses for March are calculated based on March salaries. In this scenario, as a rule, developers use dependency by action period (fig. 17.9).

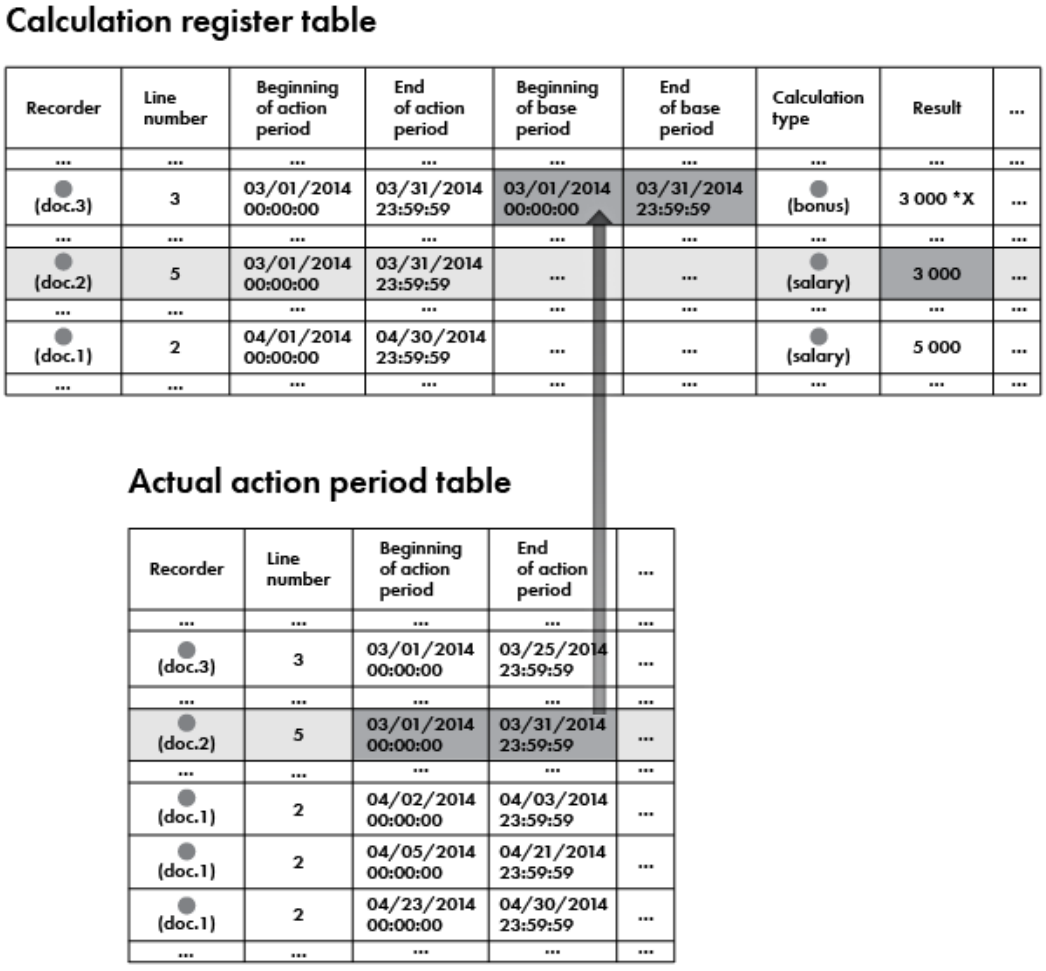


Fig. 17.9. Dependency by action period

There are two comments to be made regarding the figure above.

The Beginning of base period and End of base period fields are applicable only for the records with calculation types that have dependencies by base period specified (in this case, for calculation of bonuses).

The base value received from a specific record that influences a calculation generally is not equal to the result stored within that record. The base value is calculated based on the ratio between the part of the actual period of the influencing record that overlaps the base period and the full actual period of the influencing record, and also on the schedule data linked to record.

Dependency by registration period

Dependency by registration period means that when the platform analyzes base records, it selects the records whose Registration period field value falls within the specified base period.

Let us consider an example of penalty calculation for March salary accruals. Absence records registered in March serve as a base for calculating the penalty amounts (these can include March absence records as well as February absence records. In this scenario, as a rule, developers use dependency by registration period (fig. 17.10).

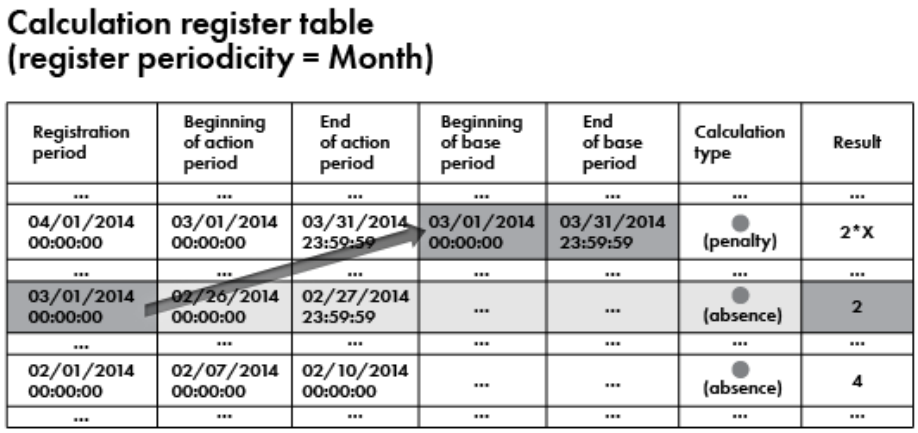


Fig. 17.10. Dependency by registration period

The final important feature of a calculation register is its connection with the chart of calculation types. This connection enables displacement by action period and dependency by base period, since a chart of calculation types describes the interrelationship between calculation types.

A calculation register can have subordinate Recalculation objects. They are used to register the fact of adding register records that affect the calculation results of existing register records. A Recalculation configuration object can have multiple dimensions, each specifying a connection between the dimensions of this calculation register and the dimensions of calculation registers that influence it. In some scenarios it can be the same register.

The platform stores the list of register records that are subject to recalculations in database tables based on Recalculation configuration objects. Recalculation tables are populated automatically based on calculation register records that are affected by leading calculation types, as well as on calculation register records that have their

actual periods changed. Based on this data, a developer can decide whether the register records require recalculation.

The last observation that we need to make regarding calculation registers concerns the ability to link a calculation register to a schedule. The schedule should be represented by an information register (nonperiodic, with a mandatory dimension of Date type, and a resource of Number type), which contains time-based source data used in the calculations. The dimensions of that schedule can be, for example, a work schedule (a reference to a catalog) and a date, and its resource can store the number of working hours for that date. In this scenario you can link a calculation register record to a specific work schedule (by specifying a reference to a work schedule catalog as a record attribute) and then use 1C:Enterprise script tools to get the number of working hours in the action period, actual period, or registration period for each record.

Learn more! For details on the structure of 1C:Enterprise script objects intended for calculation register operations, see section "Quick developer reference. Calculation registers" on page 868.

Adding a calculation register

Before you create a Calculation register configuration object named Accruals, you need to create two auxiliary configuration objects:

- WorkSchedules information register
- WorkScheduleTypes catalog

The catalog will store work schedule types available in Jack of All Trades. The information register will define which days are working days, since the salary amount is calculated based on the number of days employees worked within the payment period.

In Designer mode

Let us create the catalog and the information register.

1. In Designer, create a Catalog object named *WorkScheduleTypes*.
2. On the **Subsystems** tab, include the catalog in the **Payroll** subsystem.
3. On the **Other** tab, create two predefined work schedules: **AdmininstrationSchedule** and **TechnicianSchedule** (fig. 17.11).

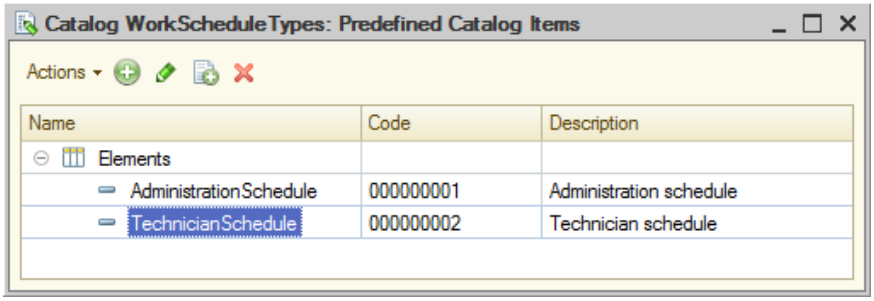


Fig. 17.11. Predefined work schedules

4. Create an InformationRegister configuration object named *WorkSchedules*.
5. Add two register dimensions:
   - **WorkSchedule**. Type: CatalogRef.WorkScheduleTypes
   - **Date**. Type: Date
6. Add a register resource:
   - **Value**. Type: Number, length: 1
7. On the **Subsystems** tab, include the information register in the **Payroll** subsystem.

Then let us populate the WorkSchedules information register with technician working days in September.

In 1C:Enterprise mode

Let us create a work schedule.

1. Start 1C:Enterprise in the debug mode.
2. In the **Payroll** section, run the **Work schedules** command.
3. Create 30 register records, one by one. For the **WorkSchedules** register dimension, select the **Technician schedule** predefined item of the **WorkScheduleTypes** catalog. For the **Value** resource, specify *1* for working days and *0* for holidays (fig. 17.12).

   To optimize this task, copy the register records (on the **More** menu, click **Copy**, or press F9).

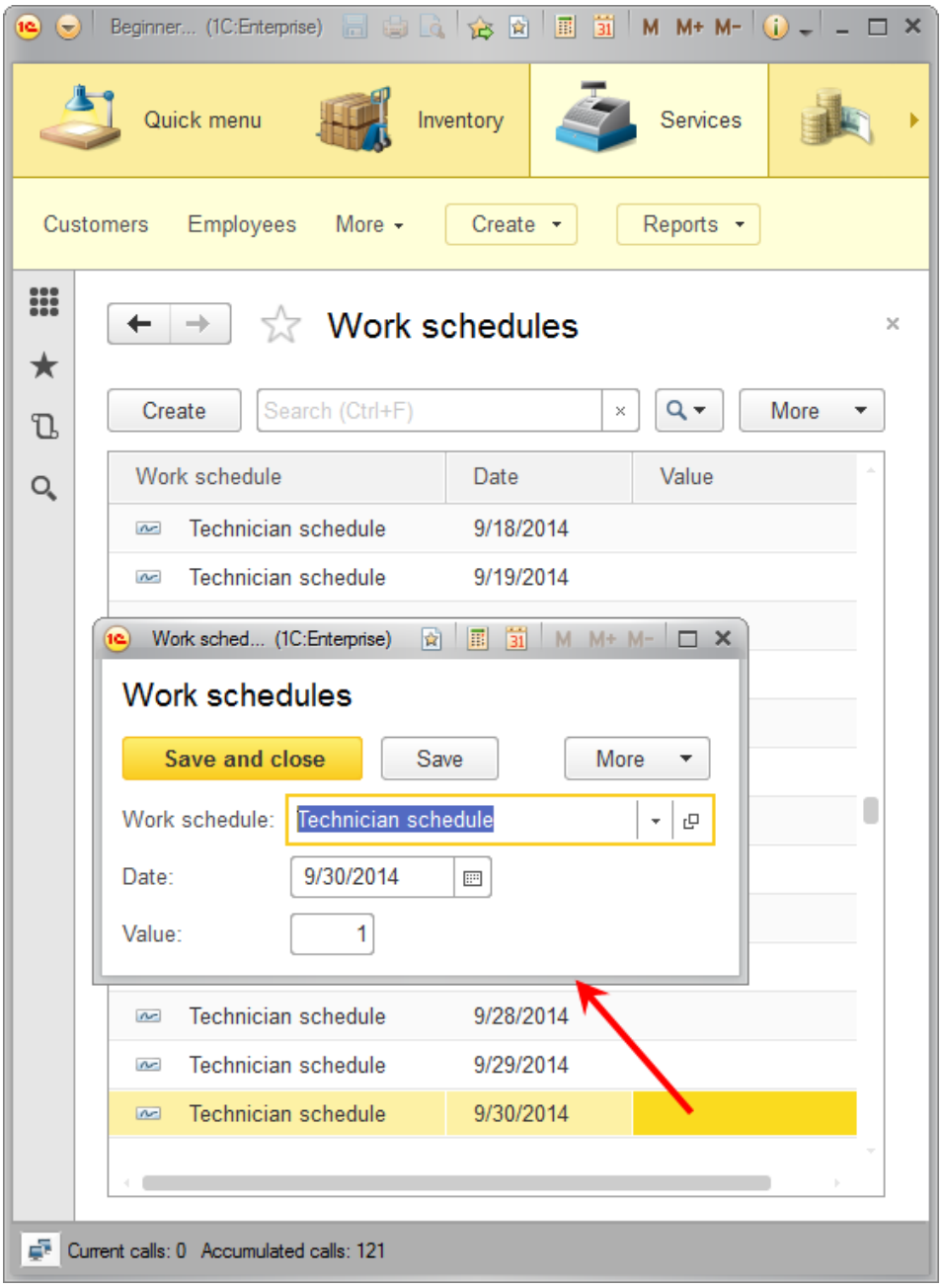


Fig. 17.12. Work schedules register records

Now you are ready to create the calculation register.

In Designer mode

Let us create the calculation register.

1. In Designer, create a CalculationRegister object named *Accruals*.
2. In the **Extended list presentation** field, enter *Accruals register records*.
3. In the **Chart of calculation types** field, select **MainAccruals**.
4. Specify that the register uses an action period.

5. In the **Schedule** field, select the **WorkSchedules** information register.
6. In the **Schedule value** field, select the **Value** resource.
7. In the **Schedule date** field, select the **Date** dimension.
8. Specify that the calculation register uses a base period and has the **Month** periodicity (fig. 17.13).

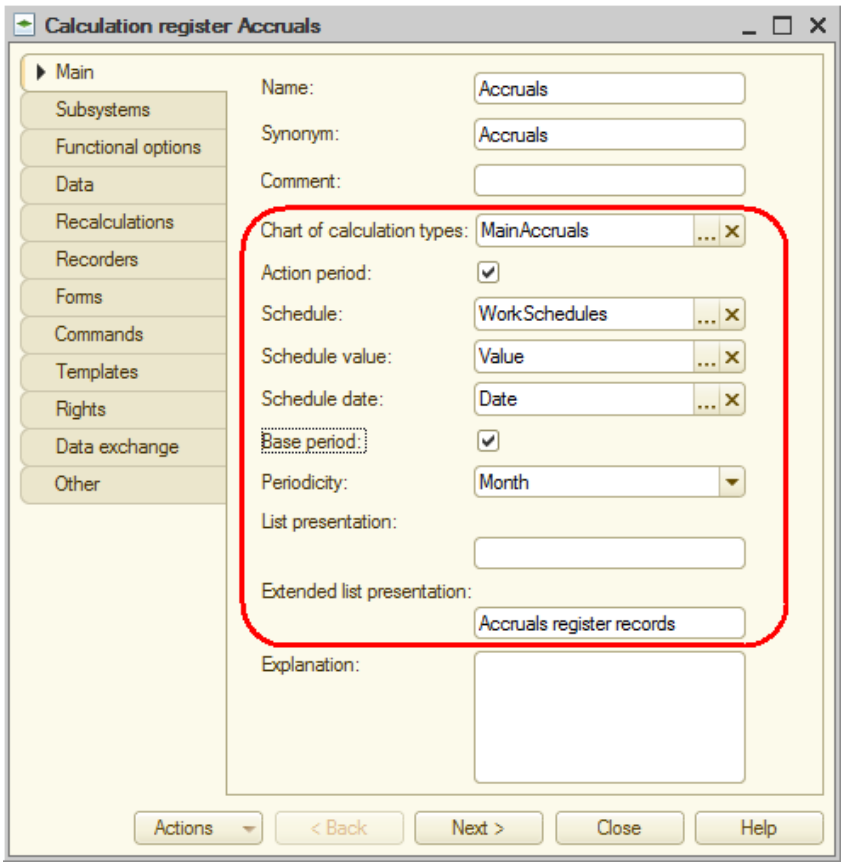


Fig. 17.13. Accruals calculation register editor

9. On the **Subsystems** tab, include the calculation register in the **Payroll** subsystem.
10. On the **Data** tab, create the following items (fig. 17.14):
    - **Employee** dimension. Type: CatalogRef.Employees, Base
    - **Result** resource. Type: Number, length: 15, precision: 2
    - **WorkSchedule** attribute. Type: CatalogRef.WorkScheduleTypes, link to schedule: WorkSchedule
    - **SourceData** attribute. Type: Number, length: 15, precision: 2

    The WorkSchedule attribute links each register record to a work schedule, while the SourceData attribute stores data that you might need for calculating or recalculating (in this example you will calculate salary).

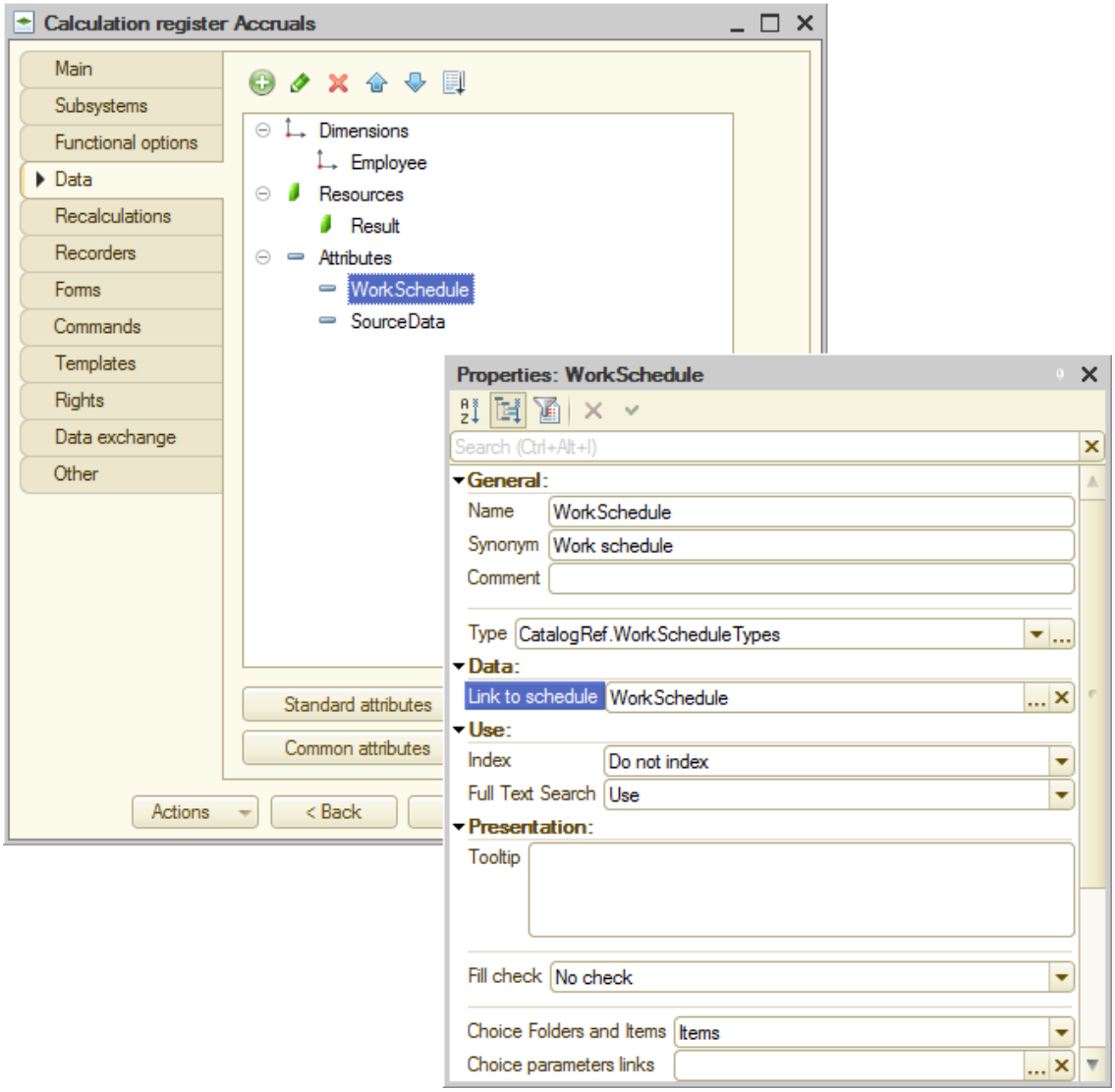


Fig. 17.14 Calculation register dimensions, resources, and attributes

11. On the **Recalculations** tab, create a **Recalculation** configuration object named *Recalculation*.
12. For this object, create a single **Employee** dimension.
13. For the Employee dimension, in the Register Dimension property, select Employee, and in the Leading registers data property, select the same Employee dimension of the Accruals calculation register (fig. 17.15).

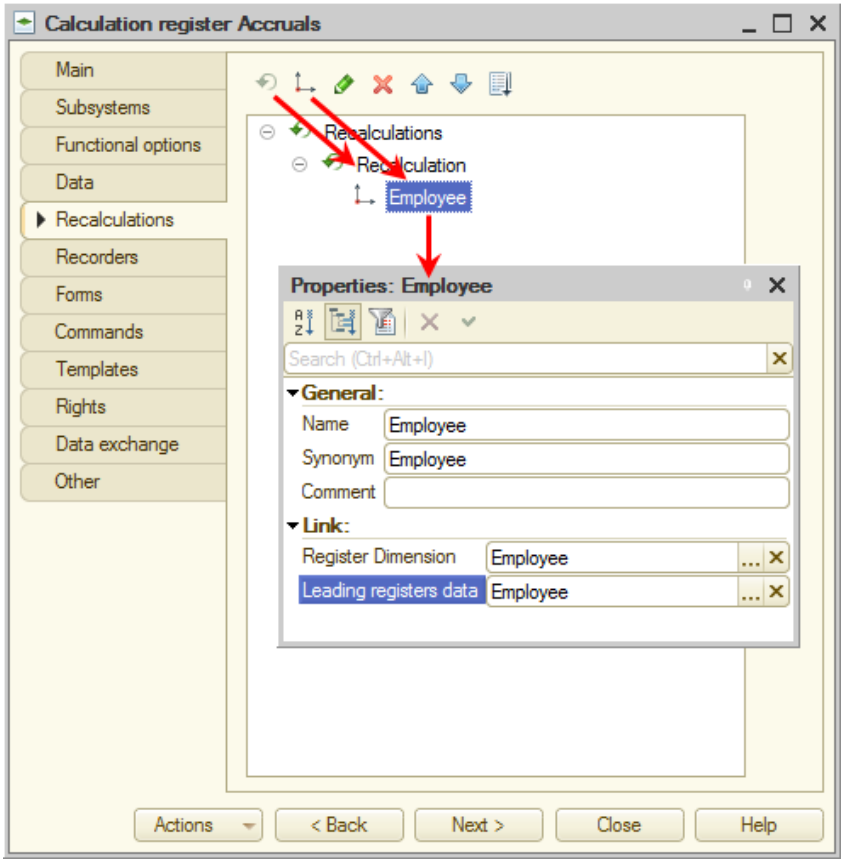


Fig. 17.15. Register recalculations

Finally, let us edit the command interface to make the command that opens the register records available in the Payroll section.

14. In the configuration tree, right-click the **Payroll** subsystem and then click **Open command interface**.

    This opens the window that lists all commands of the selected subsystem.

15. In the **Navigation panel.Normal** group, select the check box next to the **Accruals** command.

    This completes the creation of the Accruals calculation register configuration object.

Quiz

- What are complex periodic calculations?
- What is a base calculation type?
- What is the difference between the base period, actual period, and action period?
- What does dependency by base period mean?
- What is a displacement by action period?
- What is the purpose of a Chart of calculation types object?
- What are the major properties of a chart of calculation types?

- What is the difference between base, displacing, and leading calculation types?
- How does one create a chart of calculation types?
- What is a Calculation register configuration object?
- What are distinctive features of a calculation register?
- What is a time schedule?
- What is a recalculation?
- How are recalculation records generated?
- How does one create a calculation register?

Lesson 18 (3:40). Using calculation registers

Now you have everything you need to start the development of payroll calculation system for Jack of All Trades.

In this chapter you will create a document for performing various accruals, you will see how and when the platform creates recalculation records, and you will also learn how displacement by action period and dependency on base period work.

You will also create a report that shows accruals to employees at Jack of All Trades, and implement the algorithm for keeping calculated data up-to-date.

Finally, you will create a new form item: a Gantt chart. It will illustrate the functionality of some calculation features.

Adding a document that describes accruals

You need a new document for recording accruals to Jack of All Trades employees to the database.

In Designer mode

Let us create the document.

1. In Designer, create a Document object named *EmployeeAccruals*.
2. In the **Object presentation** field, enter Employee accrual.
3. On the **Numbering** tab, set the number type to **Number**, and set the number length to *5* (fig. 18.1).

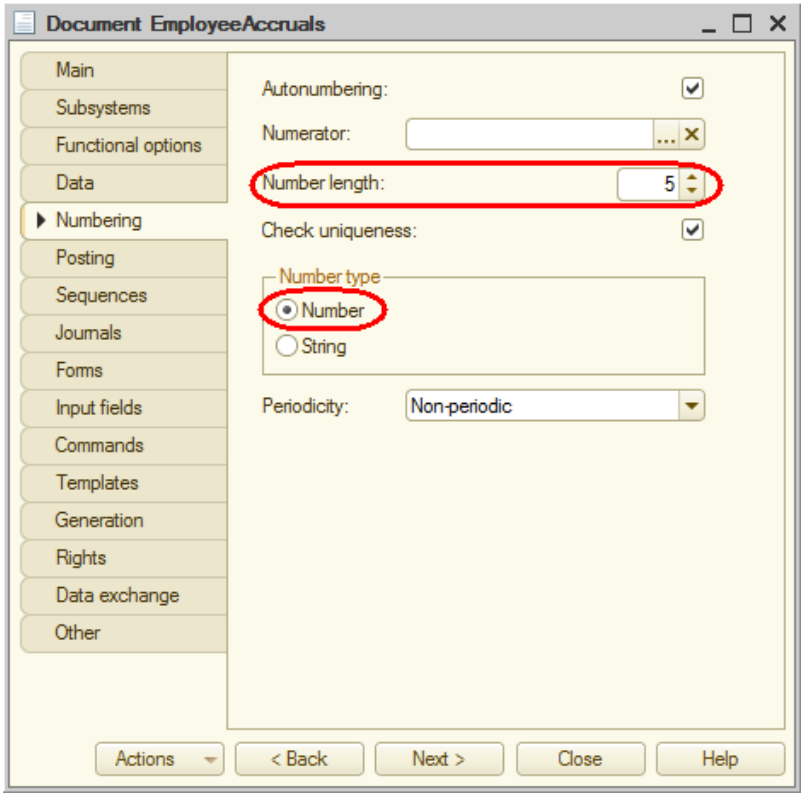


Fig. 18.1. Document numbering

4. On the **Subsystems** tab, specify that the document is displayed in the **Payroll** subsystem.
5. On the **Data** tab, add a tabular section named *Accruals* with the following attributes:
   - **Employee**. Type: CatalogRef.Employees
   - **WorkSchedule**. Type: CatalogRef.WorkScheduleTypes
   - **StartDate**. Type: Date
   - **EndDate**. Type: Date
   - **CalculationType**. Type: ChartOfCalculationTypesRef.MainAccruals
   - **Accrual**. Type: Number, length: 15, precision: 2

   You need the StartDate and EndDate attributes for specifying the period where each calculation record is effective.
6. On the **Posting** tab, prohibit real-time posting.
7. Specify that the document creates records in the **Accruals** calculation register and run the register records wizard (fig. 18.2).

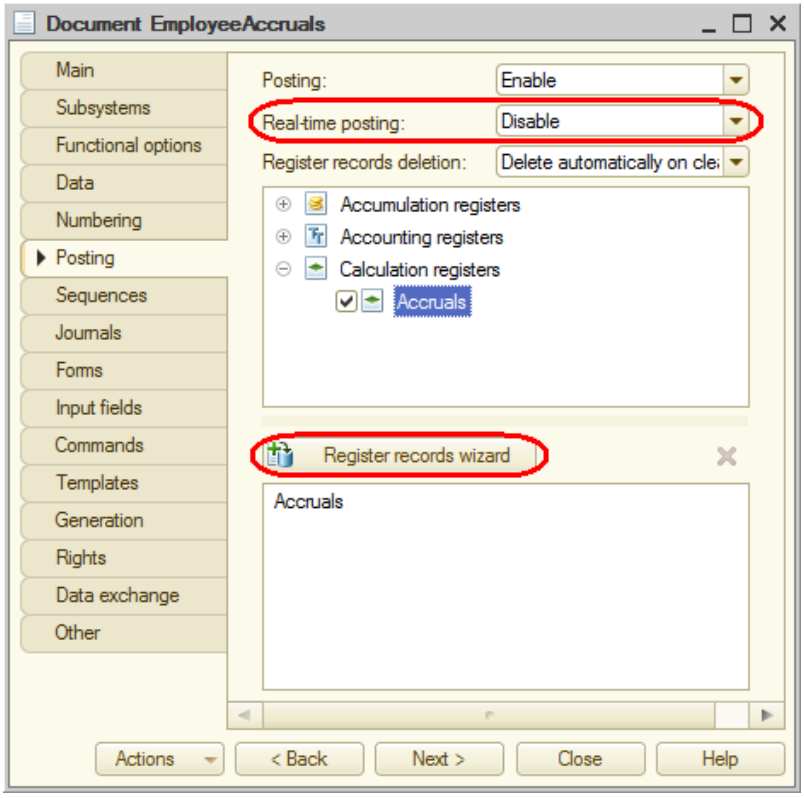


Fig. 18.2. Document register records

8. In the wizard, select the **Accruals** tabular section and then click **Fill Expressions**.
9. For the **EndOfActionPeriod** and **EndOfBasePeriod** attributes, specify the following expression: *EndOfDay(CurRowAccruals.EndDate*).
10. For the **RegistrationPeriod** field, specify the following expression: *Date*.
11. For the **SourceData** attribute, specify the **Accrual** tabular section attribute.
12. For the **Result** resource, leave the expression field empty (fig. 18.3).

    You will calculate this expression later.

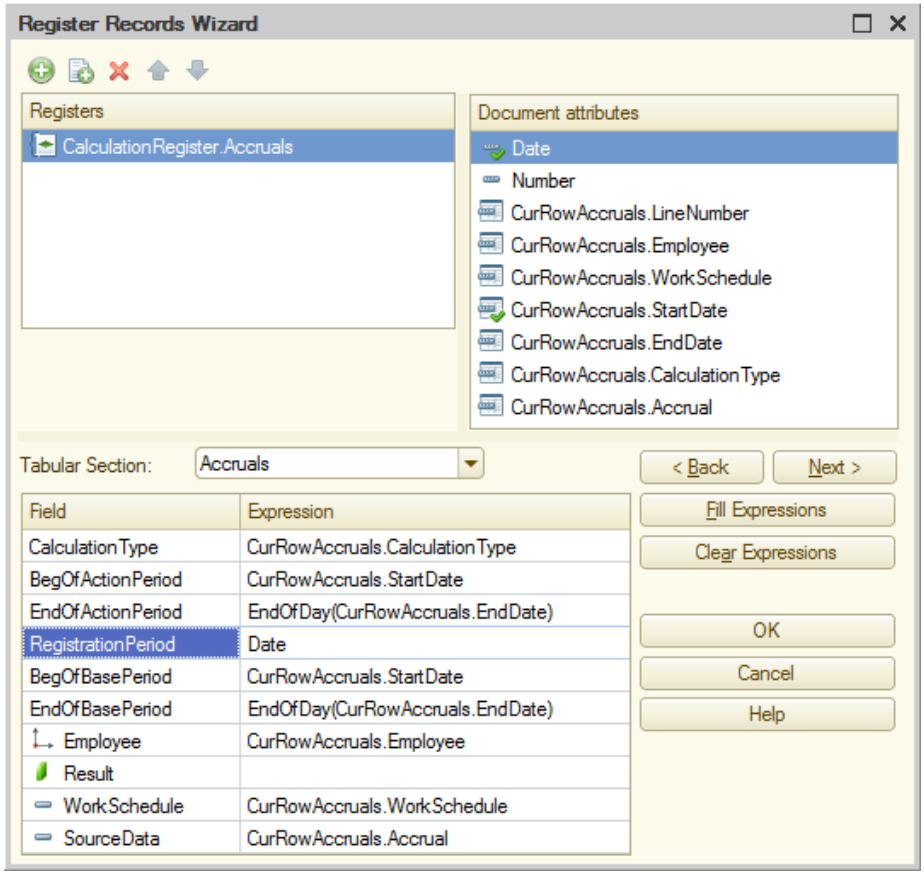


Fig. 18.3. EmployeeAccruals document records in the calculation register

13. Click **OK** and review the handler script generated by the wizard (listing 18.1).

Listing 18.1. Handler script generated by the register records wizard

```
Procedure Posting(Cancel, Mode)
    //{{__REGISTER_REGISTERRECORDS_WIZARD
    // This fragment was built by the wizard.
    // Warning! All manually made changes will be lost next time
    // you use the wizard.

    // register Accruals
    RegisterRecords.Accruals.Write = True;

    For Each CurRowAccruals In Accruals Do

        Record = RegisterRecords.Accruals.Add();
        Record.ReversingEntry = False;
        Record.CalculationType = CurRowAccruals.CalculationType;
        Record.BegOfActionPeriod = CurRowAccruals.StartDate;
        Record.EndOfActionPeriod = EndOfDay(CurRowAccruals.EndDate);
        Record.RegistrationPeriod = Date;
        Record.BegOfBasePeriod = CurRowAccruals.StartDate;
```

```
        Record.EndOfBasePeriod = EndOfDay(CurRowAccruals.EndDate);
        Record.Employee = CurRowAccruals.Employee;
        Record.WorkSchedule = CurRowAccruals.WorkSchedule;
        Record.SourceData = CurRowAccruals.Accrual;

    EndDo;
    //}}__REGISTER_REGISTERRECORDS_WIZARD

EndProcedure
```

Let us review this script.

In general, it does not have any major differences with other event handlers that create register records.

The only thing we will remind you here is that Accruals is the name of the calculation register so the expressions RegisterRecords.Accruals.Write() and RegisterRecords.Accruals.Add() are intended to call the methods of the register record set.

The line For Each CurRowAccruals In Accruals Do calls the document tabular section by its name (Accruals) and iterates through this tabular section.

The script inside the loop adds a register record and assigns values taken from the document tabular section to the record fields.

The EndOfDay() function is used to assign values to the EndOfActionPeriod and EndOfBasePeriod fields. You did the same thing when you created reports in order to include the last day of a period to a report.

Finally, let us edit the command interface to make the document creation command available in the Payroll section.

1. In the configuration tree, right-click the **Payroll** subsystem and then click **Open command interface**.
2. In the **Actions panel.create** group, select the check box next to the **Employee accrual: create** command (fig. 18.4).

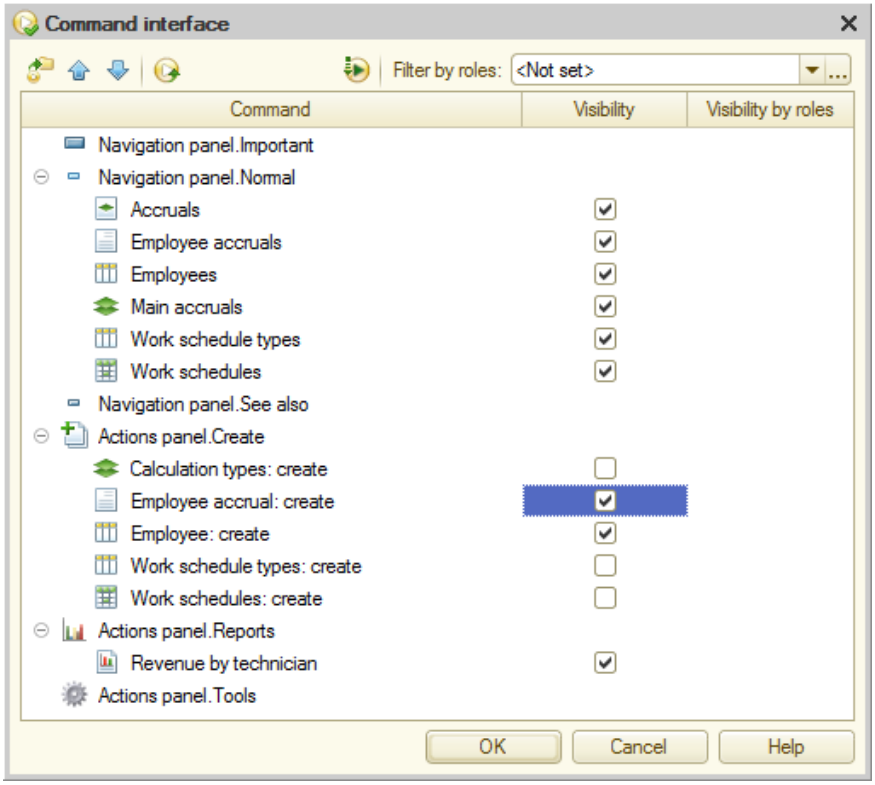


Fig. 18.4. Editing subsystem command interface

In 1C:Enterprise mode

Let us test the document.

1. Start 1C:Enterprise in the debug mode.
2. In the **Payroll** section, run the **Employee accruals** command and record September salary for all Jack of All Trades employees (fig. 18.5).

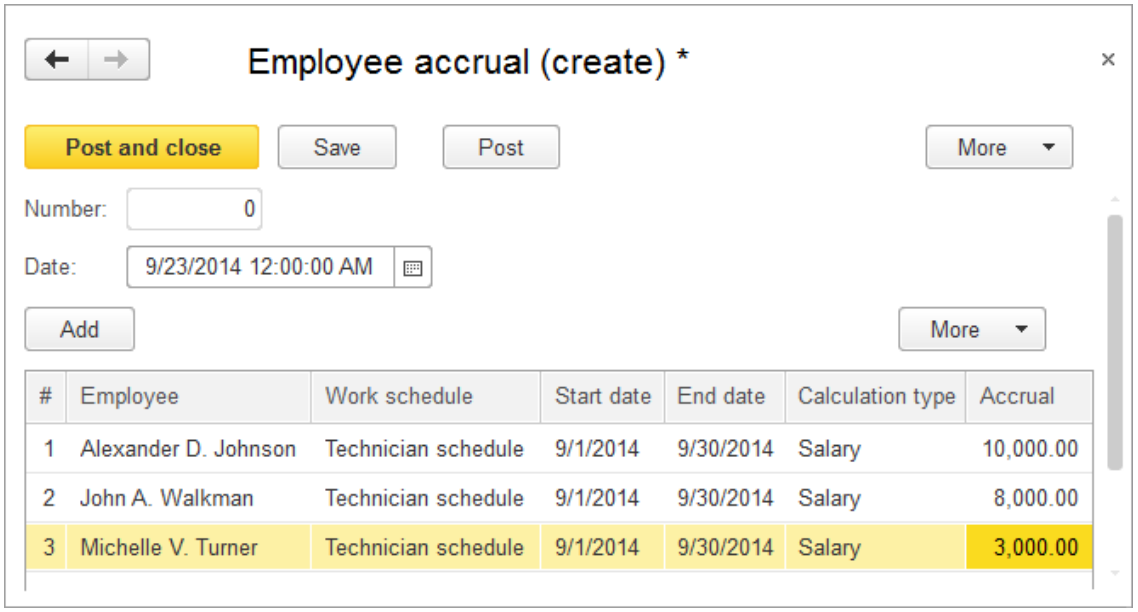


Fig. 18.5. Employee accrual document # 1

3. Post the document and look at the records it generated in the Accruals register (fig. 18.6, 18.7, and 18.8).

Registration period	Recorder	L...	Calculation type	Reversing entry
9/1/2014 12:00:00 AM	Employee accrual 1...	1	Salary	
9/1/2014 12:00:00 AM	Employee accrual 1...	2	Salary	
9/1/2014 12:00:00 AM	Employee accrual 1...	3	Salary	

Fig. 18.6. Employee accrual #1 register records in the Accruals calculation register

Employee	Result	Work schedule	Source data	Action period
Alexander D. Johnson		Technician sched...	10,000.00	9/1/2014 12:00:00 AM
John A. Walkman		Technician sched...	8,000.00	9/1/2014 12:00:00 AM
Michelle V. Turner		Technician sched...	3,000.00	9/1/2014 12:00:00 AM

Fig. 18.7. Employee accrual #1 register records in the Accruals calculation register

Action period start date	Action period end date	Start date of base period	End date of base period
9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM
9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM
9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM

Fig. 18.8. Employee accrual #1 register records in the Accruals calculation register

Note that the platform set the registration period for each record to the beginning of the calculation register period (in the posting event handler you specified that it uses the document date, which is 9/23/2014 in this example).

Also note that the SourceData record attribute stores the employee salary amount specified in the document. This lets you subsequently calculate the total payment, which is based on the salary.

To learn more of calculation register functionality, you need a utility report that shows the contents of recalculation records.

Examples of displacement and dependency by base period

Recalculation report

In Designer mode

Let us create the report.

1. Create a Report configuration object named *Recalculation*.
2. Run the Data composition schema wizard.
3. Add **Data set – query** and run the Query builder.
4. In the **Database** list, expand **Recalculations** and select all the fields from the **Accruals.Recalculation** virtual table (fig. 18.9):
   - **RecalculationObject**
   - **CalculationType**
   - **Employee**

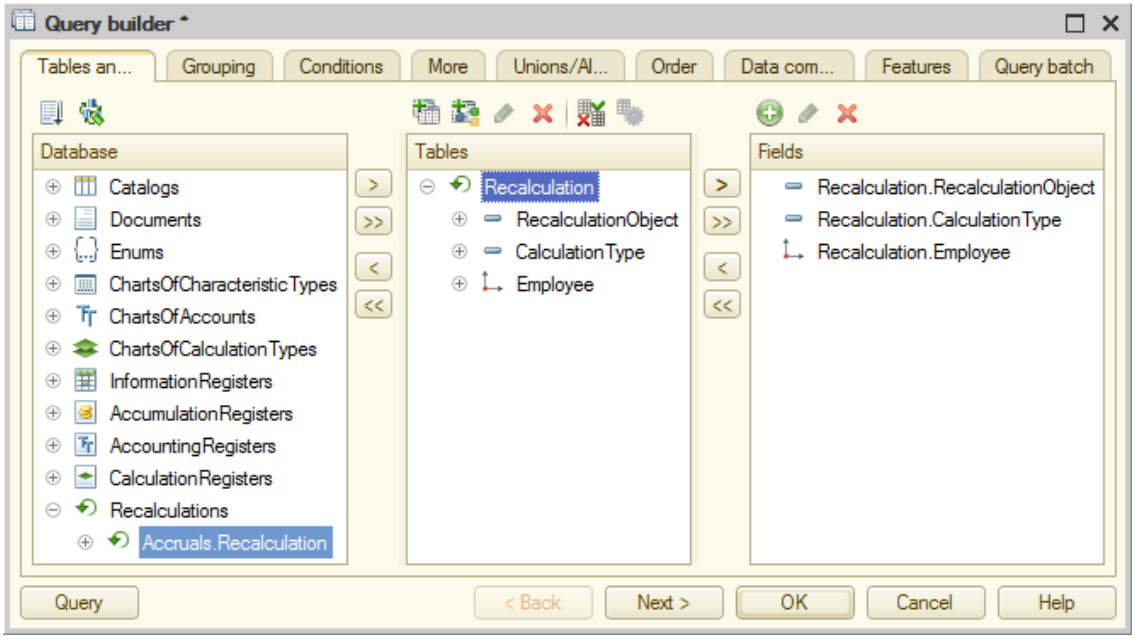


Fig. 18.9. Query fields and tables

5. Click **OK**.

   This completes the query creation.
6. On the **Settings** tab, add the <Detailed records> grouping.
7. On the **Selected fields** tab, select the following fields:
   - **RecalculationObject**
   - **CalculationType**
   - **Employee**
8. Close the data composition schema wizard by clicking **OK**.

9. In the editor of the **Recalculation** report configuration object, on the **Subsystems** tab, specify that the document is displayed in the **Payroll** subsystem.

Dependency by base period

In 1C:Enterprise mode

If you generate the report in 1C:Enterprise mode, you will see that no recalculations have been performed yet. You need another Employee accrual document to test recalculations.

1. Start 1C:Enterprise in the debug mode.
2. Create the Employee accrual #2 document with September bonuses for Jonnson and Walkman (fig. 18.10).

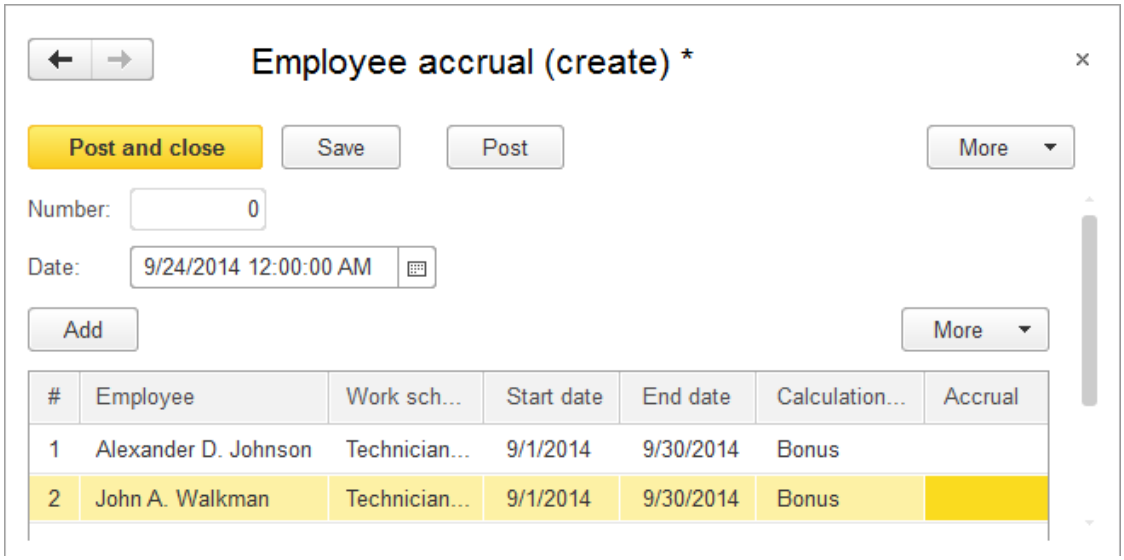


Fig. 18.10. Employee accrual #2 document

This document records the fact that bonuses are due to employees named Johnson and Walkman based on their work in September. Since you do not know the bonus amount (it will be calculated based on a certain algorithm), leave the Accrual field empty.

3. Click **Post and close**.
4. Open the Employee accrual # 1 document and change Johnson's salary from 10 000 to 7 000.
5. Click **Post and close**.
6. Generate the **Recalculation** report (fig. 18.11).

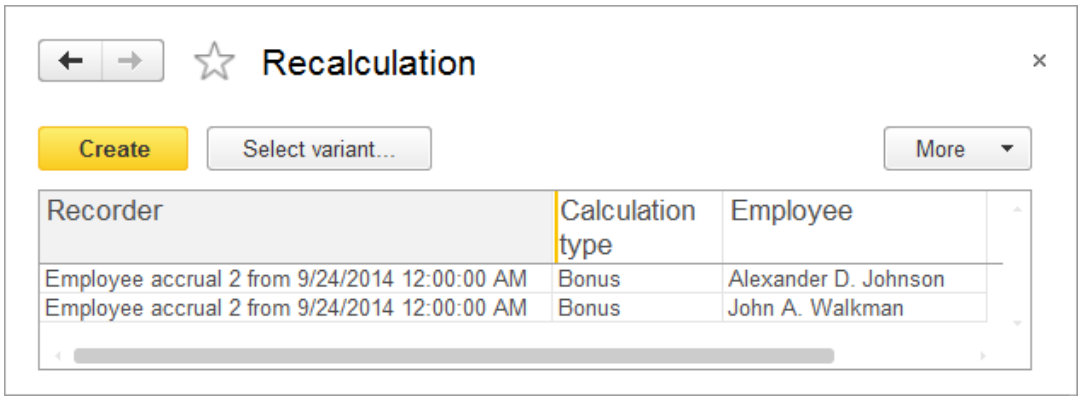


Fig. 18.11. Recalculation report

As you see, it now has some data.

This is because the Bonus calculation type depends on the Salary calculation type by base period. Once you modify the existing records of Salary type, the platform generates a set of recalculation records (because their base is changed).

You may be wondering why the Walkman's record is present in the report, since you only changed Johnson's salary. The reason is that the platform does not track the specific changes to document records. The only thing it tracks is the fact that posting (reposting) a document changes a calculation register record set.

Therefore the recalculation record set includes ALL register records whose values MIGHT change after reposting a document that creates base records in the register.

7. Repost the Employee accrual # 2 document (the one that accrues bonuses), and generate the Recalculation report.

   The report is empty again, as the platform has noted the fact that you recalculated the dependent records and therefore cleared the recalculation table.

   In this example you have studied calculation register dependency by base period.

Displacement by action period

In 1C:Enterprise mode

Let us check how displacement by action period works.

You need a third Employee accrual document for this.

1. Create the Employee accrual #3 document as shown in fig. 18.12.

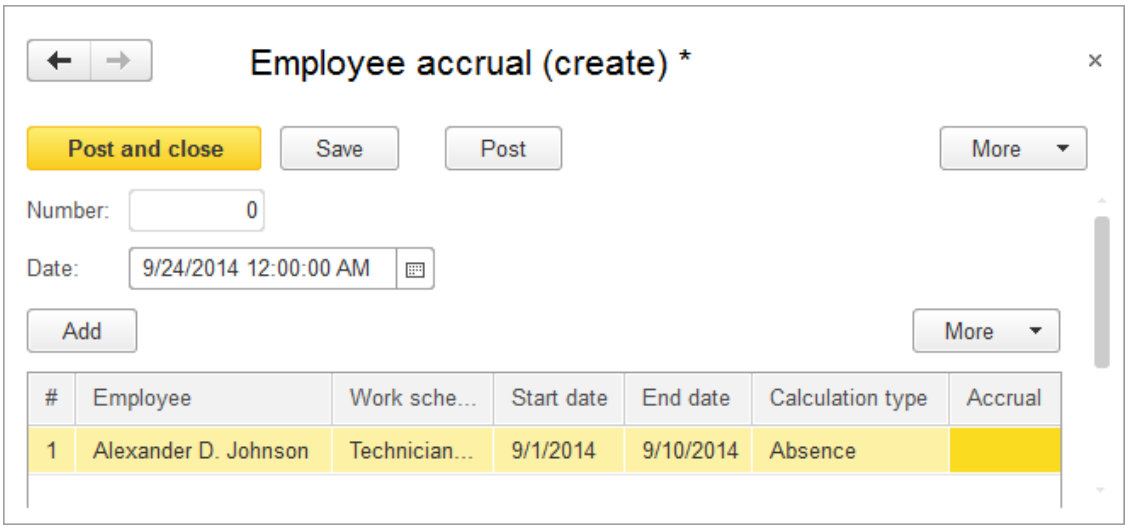


Fig. 18.12. Employee accrual #3 document

This document records the fact that Johnson was not at work from the 1st through the 10th of September.

Obviously, in this case you need to recalculate his salary and also his bonus that depends on the salary.

2. Click **Post and close** and generate the **Recalculation** report (fig. 18.13).

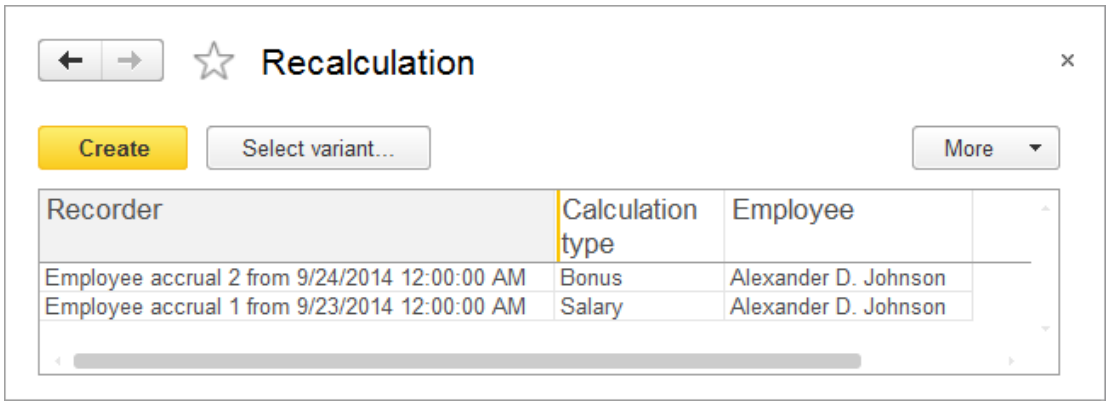


Fig. 18.13. Recalculation report

As you can see, Johnson's salary accrual record is included in the recalculation. This is the result of displacement by action period, since the Absence calculation type displaces the Salary calculation type.

Note that Johnson's bonus accrual record is also included in the recalculation. This is because during the creation of predefined calculation types you specified that the Bonus calculation type result depends on changes of the Absence calculation type result. This dependency is indirect, but since you have explicitly specified this dependency, the platform tracks it.

3. Repost the Employee accrual #1 and Employee accrual #2 documents and ensure that the recalculation table is cleared.

Procedure for calculating calculation register records

In Designer mode

So far, you simply added Accruals calculation register records, which stated that some calculations were required. However, you have not yet implemented these calculations.

Now it is time to describe the algorithms for various calculations types.

Since you will use the algorithms not only in Employee accrual documents, it is wise to store them in a separate common module.

1. In Designer, open the posting event handler of the EmployeeAccruals document and add a call of the CalculateAccruals() procedure from the Calculations common module after the lines that record data to the Accruals register (listing 18.2).

Listing 18.2. EmployeeAccruals document posting procedure

```
Procedure Posting(Cancel, Mode)
    ...
    EndDo;
    //}}__REGISTER_REGISTERRECORDS_WIZARD

    // Writing register records
    RegisterRecords.Accruals.Write();

    // Getting the list of employees referenced in the document
    Query = New Query(
    "SELECT DISTINCT
    |   EmployeeAccrualsAccruals.Employee
    |FROM
    |   Document.EmployeeAccruals.Accruals AS EmployeeAccrualsAccruals
    |WHERE
    |   EmployeeAccrualsAccruals.Ref = &CurDocument");

    Query.SetParameter("CurDocument", Ref);

    // Generating employee list
    ValTable = Query.Execute().Unload();
    EmployeeArray = ValTable.UnloadColumn("Employee");

    // Calling the CalculateAccruals procedure from the common module
    Calculations.CalculateAccruals(RegisterRecords.Accruals,
        ChartsOfCalculationTypes.MainAccruals.Salary, EmployeeArray);
    RegisterRecords.Accruals.Write( , True);
```

```
    Calculations.CalculateAccruals(RegisterRecords.Accruals,
        ChartsOfCalculationTypes.MainAccruals.Bonus, EmployeeArray);
    RegisterRecords.Accruals.Write( , True);

EndProcedure
```

Note that the records generated by the document are first written to the register (RegisterRecords.Accruals.Write()) and then the register record set is passed to the CalculateAccruals() calculation procedure. You will create this procedure in the Calculations common module later in this tutorial.

This procedure is called twice. The first call calculates primary records (Salary), while the second call calculates secondary ones (Bonus).

The calculation procedure generates the resource values for the register based on the calculation algorithms described in the procedure and register data.

Once the resources are calculated, the set of register records is rewritten without generating recalculation records (the second parameter in the Write() method is True).

Before calling the procedure from the common module, a query generates a list of employees referenced in the document in order to pass it to the procedure.

The CurDocument query parameter value is taken from the standard Ref document attribute. The query method Query.Execute().Unload() dumps the query result to a value table (the ValTable variable). Then the EmployeeArray is generated, it contains the Employee column from this value table.

2. In the **Common** branch of the configuration tree, create a module named *Calculations*.
3. For this module, select the **Server call** check box.

   This is required to make the export procedures and functions visible (fig. 18.14).

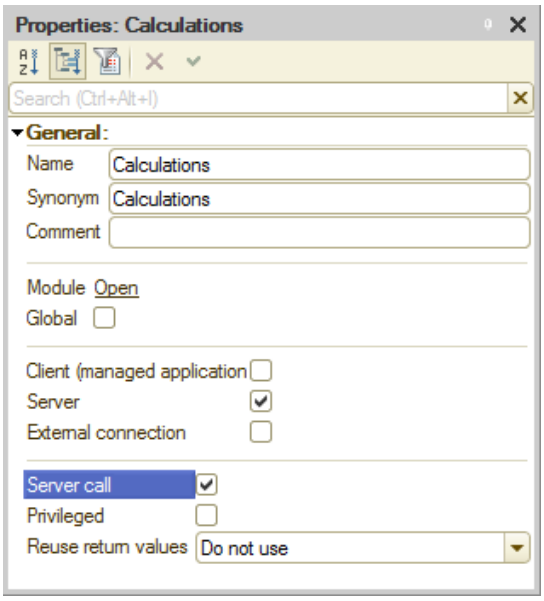


Fig. 18.14. Common module properties

4. Create a template for the CalculateAccruals() procedure (listing 18.3).

Listing 18.3. CalculateAccruals() procedure template

```
Procedure CalculateAccruals (RegisterRecordSet,
    RequiredCalculationType, EmployeeList) Export


    Recorder = RegisterRecordSet.Filter.Recorder.Value;


    // Calculating primary records
    If RequiredCalculationType =
        ChartsOfCalculationTypes.MainAccruals.Salary Then


    // Calculating secondary records
    ElsIf RequiredCalculationType =
        ChartsOfCalculationTypes.MainAccruals.Bonus Then


    EndIf;


EndProcedure
```

The accruals calculation algorithm varies for primary (Salary calculation type) and secondary (Bonus calculation type) records, and each of the algorithm parts is located in a branch of the If... clause.

Calculation of primary records requires the schedule data of the calculation register, so let us add a query to the calculation register virtual table (CalculationRegister.Accruals.ScheduleData) to the first branch of the If... clause (listing 18.4).

Listing 18.4. Modifying the CalculateAccruals() procedure

```
Procedure CalculateAccruals (RegisterRecordSet,
   RequiredCalculationType, EmployeeList) Export


   Recorder = RegisterRecordSet.Filter.Recorder.Value;


   // Calculating primary records
    If RequiredCalculationType =
       ChartsOfCalculationTypes.MainAccruals.Salary Then


       Query = New Query;
       Query.Text =
           "SELECT
           | AccrualsScheduleData.ValueActionPeriod AS Normal,
           | AccrualsScheduleData.ValueActualActionPeriod AS Actual,
           | AccrualsScheduleData.LineNumber
           |FROM
           | CalculationRegister.Accruals.ScheduleData(Recorder =
           |   &Recorder AND
           |   CalculationType = &CalculationType AND
           |   Employee IN (&EmployeeList)) AS AccrualsScheduleData";


       Query.SetParameter("Recorder", Recorder);
       Query.SetParameter("CalculationType", RequiredCalculationType);
       Query.SetParameter("EmployeeList", EmployeeList);


       SelectionResult = Query.Execute().Select();


   // Calculating secondary records
   ElsIf RequiredCalculationType =
       ChartsOfCalculationTypes.MainAccruals.Bonus Then


   EndIf;


EndProcedure
```

The query selects schedule values for the action period and the actual action period from the calculation register schedule data virtual table. The virtual table parameters specify that the selection of schedule values is limited by recorder, calculation type, and employee list.

5. Add the iteration through the record set passed to the procedure and the calculation of records based on the retrieved schedule data (listing 18.5).

Listing 18.5. Adding the iteration through the record set and the calculation of primary records

```
Procedure CalculateAccruals (RegisterRecordSet,
   RequiredCalculationType, EmployeeList) Export


   Recorder = RegisterRecordSet.Filter.Recorder.Value;


   // Calculating primary records
   If RequiredCalculationType =
      ChartsOfCalculationTypes.MainAccruals.Salary Then
      ...
      SelectionResult = Query.Execute().Select();


      For Each RegisterRecord In RegisterRecordSet Do
         StructureNumber = New Structure("LineNumber");
         StructureNumber.LineNumber = RegisterRecord.LineNumber;
         SelectionResult.Reset();


         If SelectionResult.FindNext(StructureNumber) Then


            If SelectionResult.Normal = 0 Then
               Message = New UserMessage;
               Message.Text = "Calculation type: Salary."
                  + "No working days in the specified period";
               Message.Message();
               RegisterRecord.Result = 0;


            Else


               // Calculating salary based on the actual period
               // and the source data
               RegisterRecord.Result = (RegisterRecord.SourceData
                  / SelectionResult.Normal) * SelectionResult.Actual;
               Message = New UserMessage;
               Message.Text = "Calculation completed: "
                  + RegisterRecord.Recorder + " - "
                  + RegisterRecord.CalculationType + " - "
                  + RegisterRecord.Employee;
               Message.Message();
            EndIf;


         EndIf;


      EndDo;


   // Calculating secondary records
   ElsIf RequiredCalculationType =
```

```
      ChartsOfCalculationTypes.MainAccruals.Bonus Then

   EndIf;

EndProcedure
```

For each record of the calculation register record set, a line number is retrieved. The line number identifies an accrual for a specific employee, and this number is used to find the corresponding record in the query result selection.

If a record with that line number is found in the query result, the calculation register record result is calculated. It is a salary accrual calculated by the following formula: the accrued amount (SourceData register field) is divided by the number of working days in the month (Normal) and the result is multiplied by the number of actual working days (Actual).

6. Add a query script to the second branch of the If… clause (listing 18.6).

   This query is similar to the first one, with the following difference: the base values are retrieved from the calculation register virtual table (CalculationRegister.Accruals.AccrualBase).

Listing 18.6. Adding the query script to the second clause branch

```
Procedure CalculateAccruals (RegisterRecordSet,
   RequiredCalculationType, EmployeeList) Export

   Recorder = RegisterRecordSet.Filter.Recorder.Value;

   // Calculating primary records
   If RequiredCalculationType =
      ChartsOfCalculationTypes.MainAccruals.Salary Then

   ...

   // Calculating secondary records
   ElsIf RequiredCalculationType =
      ChartsOfCalculationTypes.MainAccruals.Bonus Then

      Query = New Query;
      Query.Text=
         "SELECT
         |  AccrualsBaseAccruals.ResultBase AS Base,
         |  AccrualsBaseAccruals.LineNumber AS LineNumber
         |FROM
         |  CalculationRegister.Accruals.BaseAccruals(&MainDimensions,
         |  &BaseDimensions, , Recorder = &Recorder AND
         |     CalculationType = &CalculationType AND
         |     Employee IN (&EmployeeList))
```

```
          |  AS AccrualsBaseAccruals";

      Dim = New Array(1);
      Dim[0]="Employee";

      Query.SetParameter("MainDimensions", Dim);
      Query.SetParameter("BaseDimensions", Dim);
      Query.SetParameter("Recorder", Recorder);
      Query.SetParameter("CalculationType", RequiredCalculationType);
      Query.SetParameter("EmployeeList", EmployeeList);

      SelectionResult = Query.Execute().Select();

   EndIf;

EndProcedure
```

The virtual query table parameters include not only recorder, calculation type, and employee list, which are familiar to you, but also the dimensions of the main and base registers. In this case it is a single Accruals register, and what you need is its Employee dimension.

7. Add the iteration through the record set and the calculation of secondary records to the second If... clause (listing 18.7).

Listing 18.7. Adding the iteration through the record set and the calculation of secondary records

```
Procedure CalculateAccruals (RegisterRecordSet,
    RequiredCalculationType, EmployeeList) Export

    Recorder = RegisterRecordSet.Filter.Recorder.Value;

    // Calculating primary records
    If RequiredCalculationType =
        ChartsOfCalculationTypes.MainAccruals.Salary Then

    ...

    // Calculating secondary records
    ElsIf RequiredCalculationType =
        ChartsOfCalculationTypes.MainAccruals.Bonus Then

        ...

        SelectionResult = Query.Execute().Select();

        For Each RegisterRecord In RegisterRecordSet Do
```

```
            StructureNumber = New Structure("LineNumber");
            StructureNumber.LineNumber = RegisterRecord.LineNumber;
            SelectionResult.Reset();

            If SelectionResult.FindNext(StructureNumber) Then
                RegisterRecord.Result = SelectionResult.Base
                    * (10/100);
                Message = New UserMessage;
                Message.Text="Calculation completed: "
                    + RegisterRecord.Recorder + " - "
                    + RegisterRecord.CalculationType + " - "
                    + RegisterRecord.Employee;
                Message.Message();

            EndIf;

        EndDo;

    EndIf;

EndProcedure
```

This script calculates the accrued bonus as 10% of the calculated salary.

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. Open the Employee accrual #3 document and then, on the **More** menu, click **Clear posting**.

   This cancels the document posting.
3. Repost Employee accrual #1 and Employee accrual #2 documents.

   The Accruals calculation register should look as follows (fig. 18.15, 18.16, and 18.17).

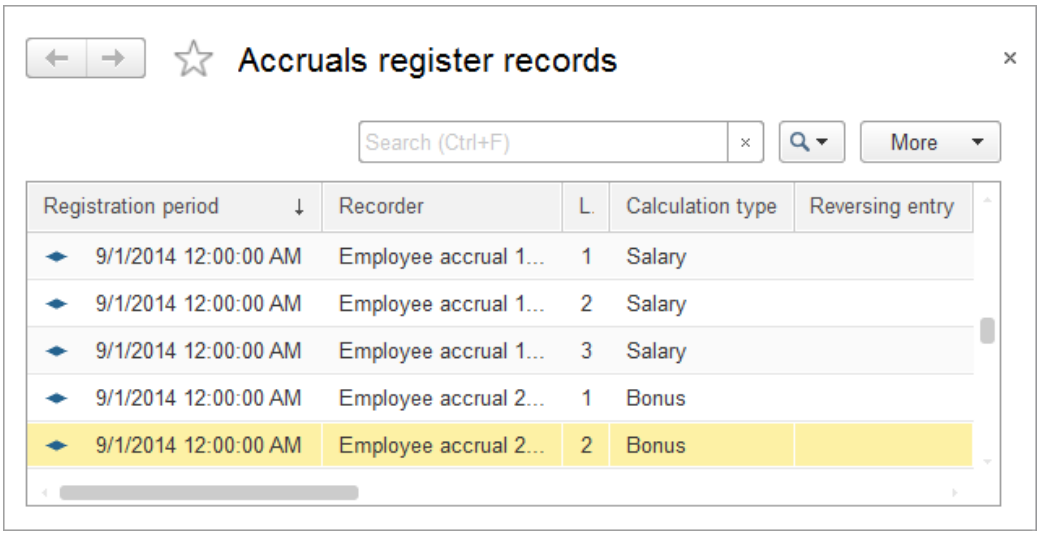


Fig. 18.15. Accruals register records

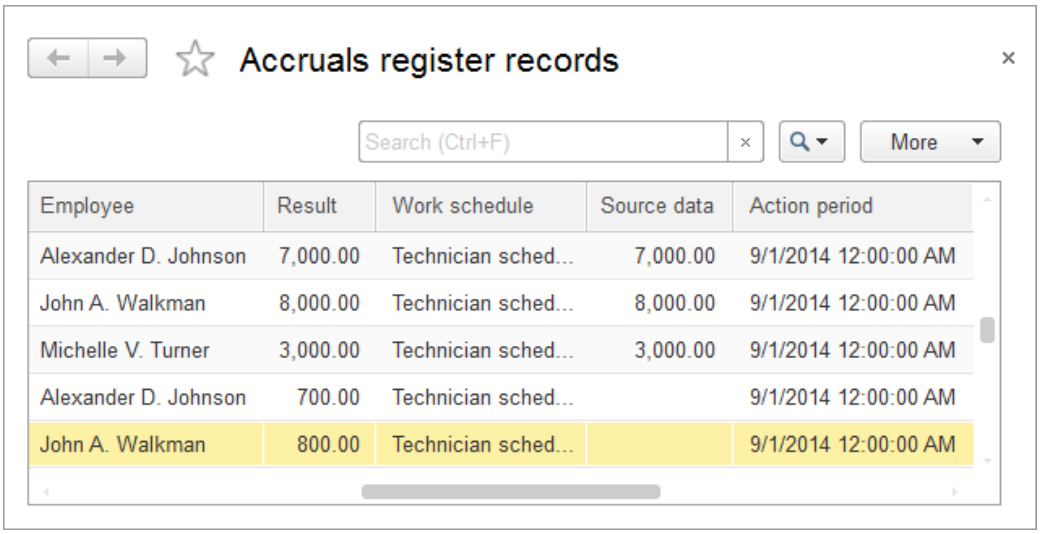


Fig. 18.16. Accruals register records

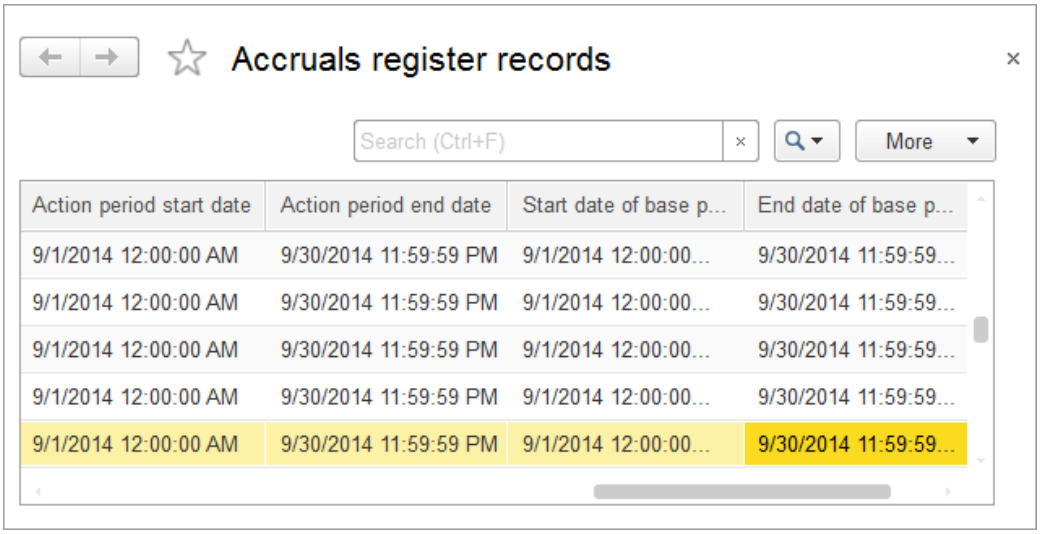


Fig. 18.17. Accruals register records

You can see that salary has been accrued to all employees (the Result field) for the entire month in compliance with the source data

(the Source data field). The employees Johnson and Walkman were accrued a bonus of 10% of their accrued salary.

4. Post Employee accrual #3, and then post Employee accrual #1 and Employee accrual #2.

   At this point the Recalculation report should be empty.

   The register state should change as follows (fig. 18.18, 18.19, and 18.20).

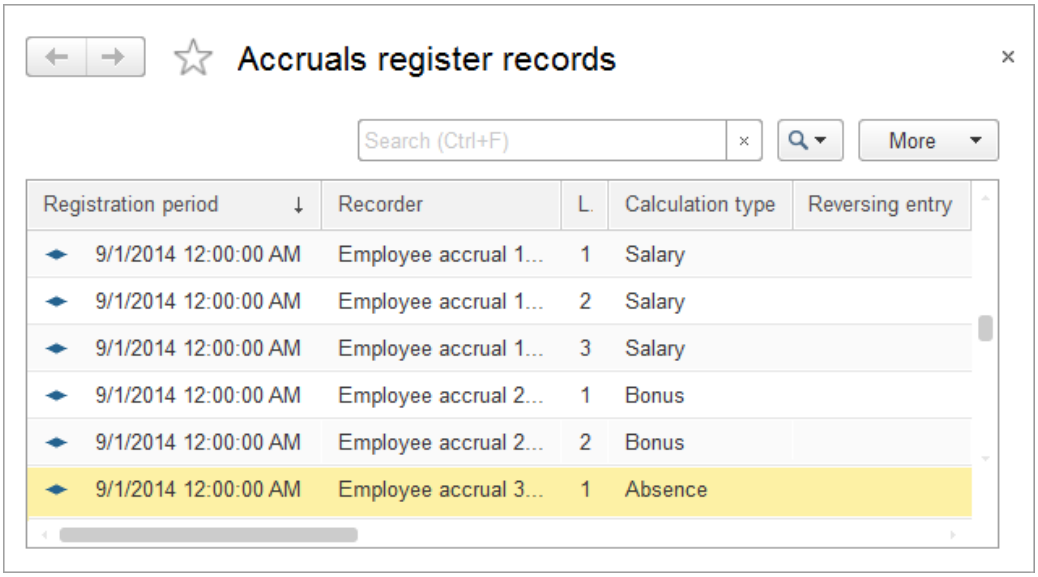


Fig. 18.18. Accruals register records

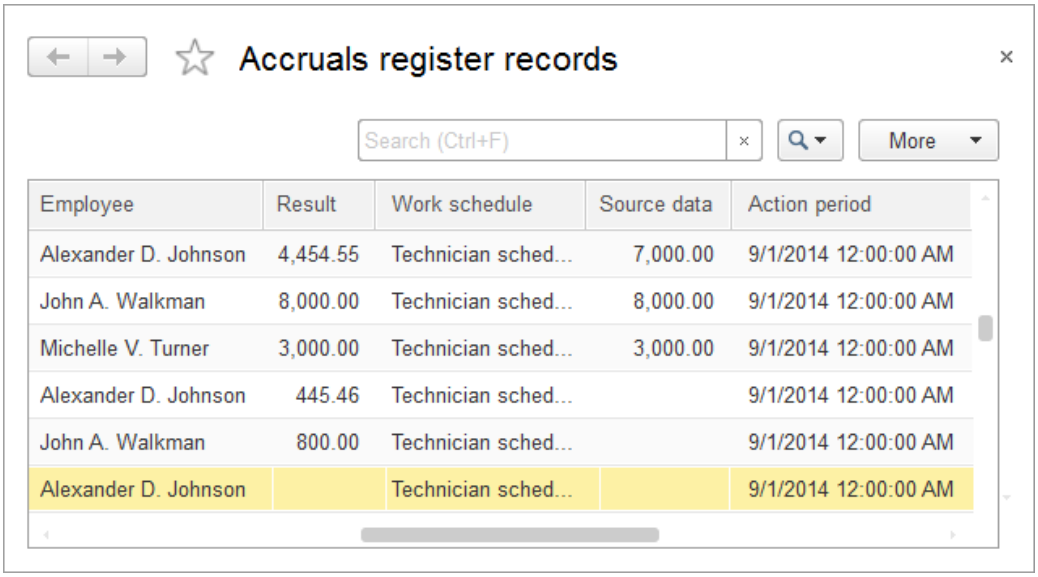


Fig. 18.19. Accruals register records

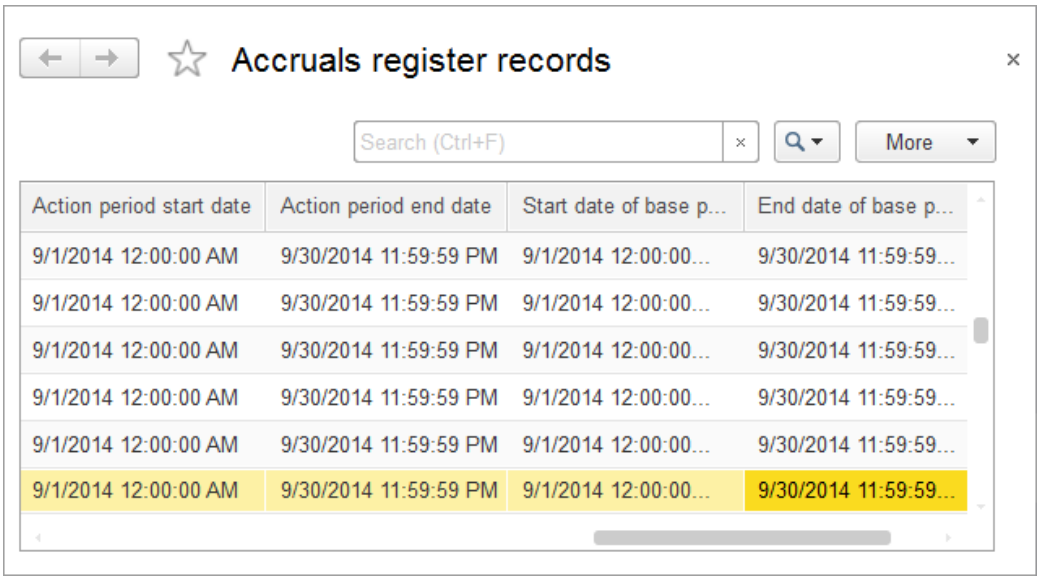
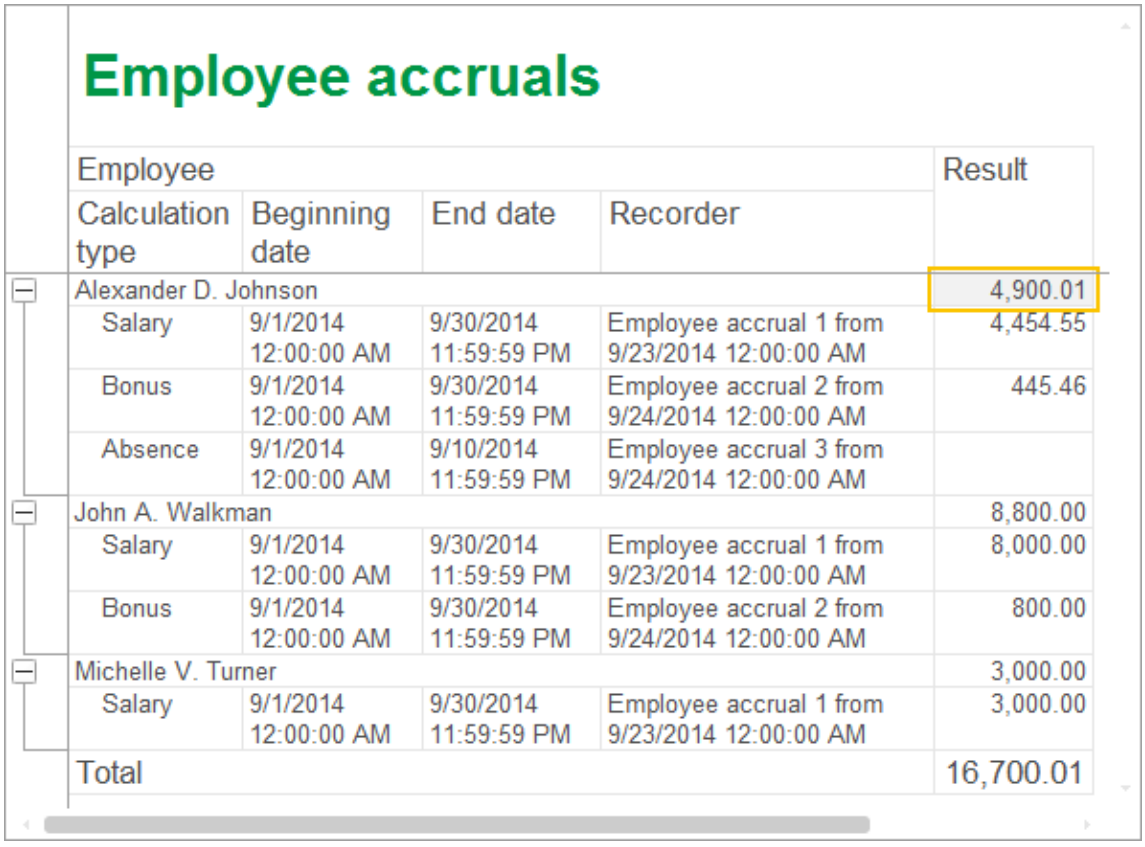


Fig. 18.20. Accruals register records

As a result of Johnson's absence, his salary amount is reduced, and his bonus is reduced accordingly.

Employee accruals report

Let us see how one can use the data that is stored in the calculation register to get an aggregate employee accruals report (fig. 18.21).



Employee / Calculation type	Beginning date	End date	Recorder	Result
Alexander D. Johnson				4,900.01
Salary	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 1 from 9/23/2014 12:00:00 AM	4,454.55
Bonus	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 2 from 9/24/2014 12:00:00 AM	445.46
Absence	9/1/2014 12:00:00 AM	9/10/2014 11:59:59 PM	Employee accrual 3 from 9/24/2014 12:00:00 AM	
John A. Walkman				8,800.00
Salary	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 1 from 9/23/2014 12:00:00 AM	8,000.00
Bonus	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 2 from 9/24/2014 12:00:00 AM	800.00
Michelle V. Turner				3,000.00
Salary	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 1 from 9/23/2014 12:00:00 AM	3,000.00
Total				16,700.01

Fig. 18.21. Resulting report

In Designer mode

Let us proceed to report creation.

1. Create a Report configuration object named *EmployeeAccruals*.
2. Run the Data composition schema wizard.
3. Add **Data set – query** and run the Query builder.

Query for a data set

Let us specify the query data set.

1. Select the **Accruals** register table as the data source (fig. 18.22).

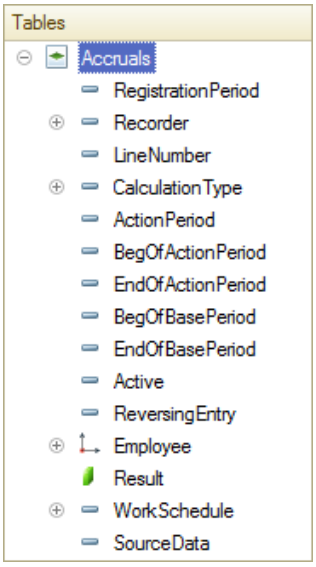


Fig. 18.22. Accruals table fields

2. Select the following fields from this table (fig. 18.23):
   - **Employee**
   - **CalculationType**
   - **BegOfActionPeriod**
   - **EndOfActionPeriod**
   - **Recorder**
   - **Result**

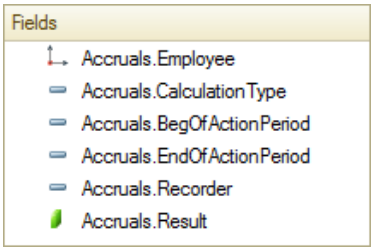


Fig. 18.23. Fields selected for the query

3. Click the **Unions/Aliases** tab and set the aliases for the **BegOfActionPeriod** and **EndOfActionPeriod** fields (fig. 18.24).

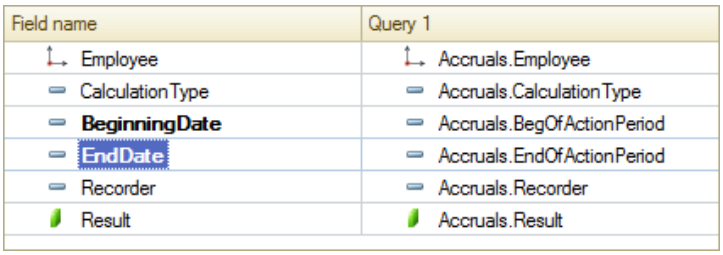


Fig. 18.24. Unions/aliases

4. Click **OK**.

   This completes the query creation.

Resources

Let us add the resources.

- On the **Resources** tab, specify that the total is calculated by the **Result** field.

Settings

Let us specify the report structure.

1. On the **Settings** tab, create a grouping by the **Employee** field.
2. Within this grouping, create a subordinate <Detailed records> grouping.
3. On the **Selected fields** tab, select the fields that you want displayed in the report (fig. 18.25):
   - **CalculationType**
   - **Beginning Date**
   - **EndDate**
   - **Recorder**
   - **Result**

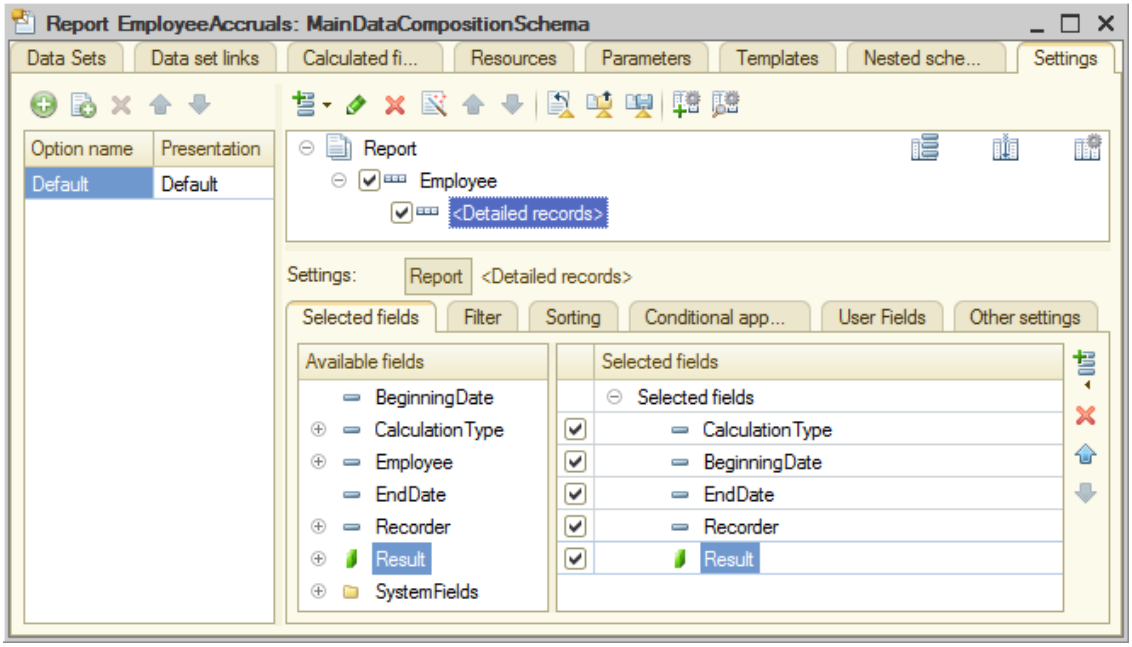


Fig. 18.25. Report structure and selected fields

4. On the **Sorting** tab, specify sorting by **Employee** and **Recorder** fields (fig. 18.26).

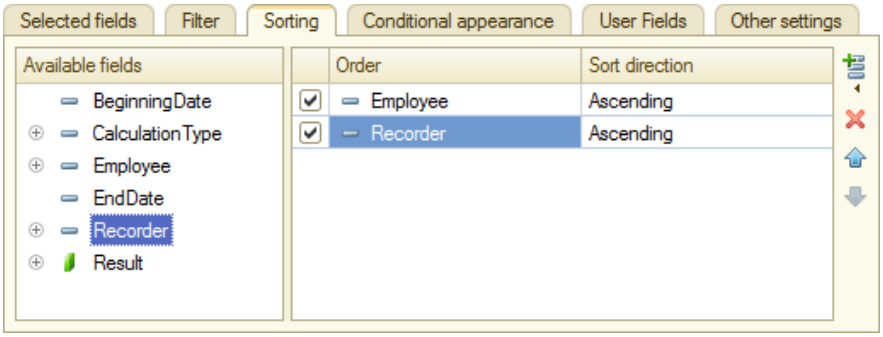


Fig. 18.26. Report sorting settings

5. On the **Other settings** tab, set the report title to *Employee accruals*.
6. Close the data composition schema wizard and then, in the editor of the **EmployeeAccruals** report configuration object, click the **Subsystems** tab.
7. Include the report in the **Payroll** and **Accounting** subsystems.

In 1C:Enterprise mode

Let us check the report.

1. Start 1C:Enterprise in the debug mode.
2. In the **Payroll** section, open the **Employee accruals** report and then click **Create** (fig. 18.27).

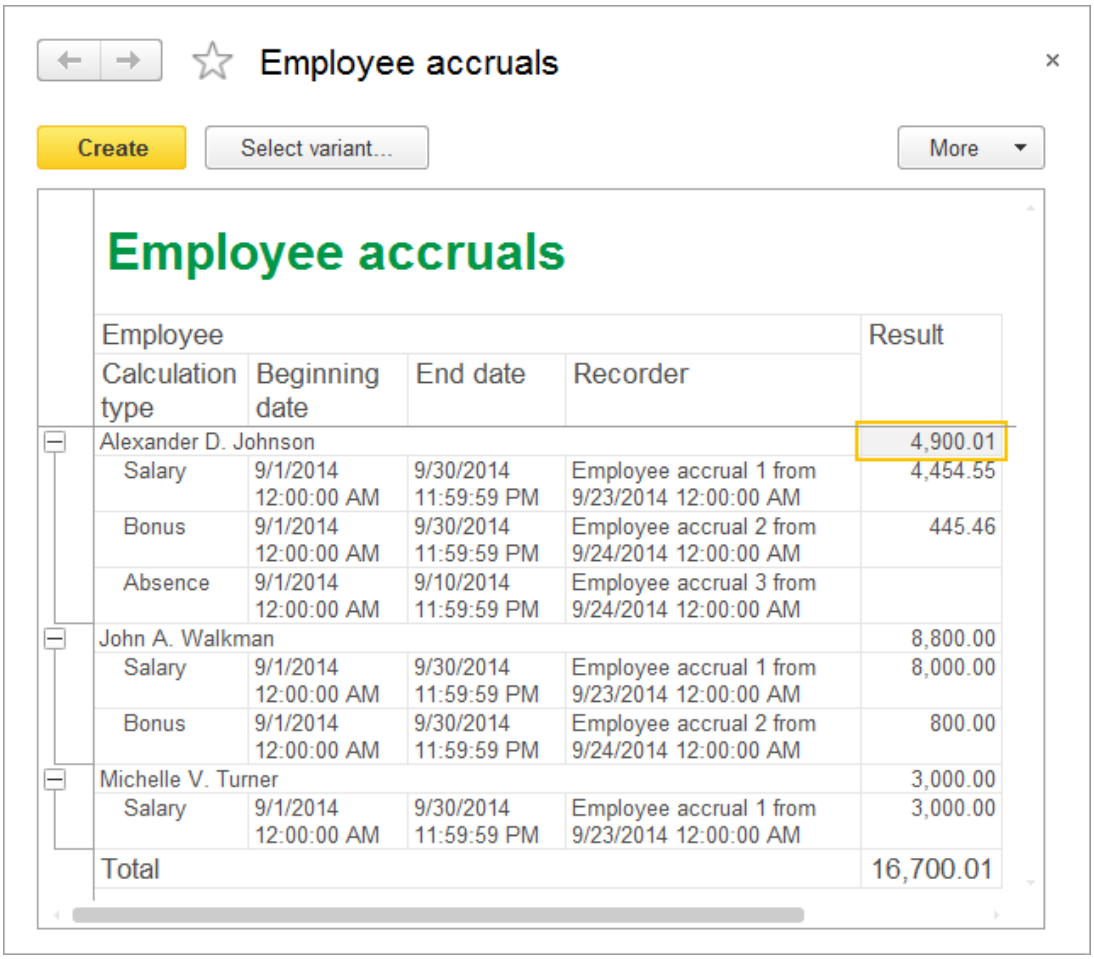


Fig. 18.27. Resulting report

Recalculations

So now your calculation algorithm has a single "bottleneck" left: checking whether the calculation register data is up-to-date.

Up to this point, you have used the utility Recalculation report to determine whether the calculation register data is up-to-date (the report is empty) or requires recalculation.

Let us create a procedure that checks whether calculation register data requires recalculation and performs the recalculation if needed.

Since the only way to get totals for employee accruals in your configuration is the EmployeeAccruals report, in order to call the procedure you will create the report form and add the Recalculate button to the form command bar. This button will run the recalculation of register data.

In Designer mode

Let us create the report form.

1. Open the configuration object editor for the **EmployeeAccruals** report.

2. On the **Forms** tab, in the **Report form** field, click the **Open** button and create the default report form.
3. In the upper right pane of the form editor, click the **Commands** tab, click the **Form command** tab, and create the *Recalculate* command (fig. 18.28).

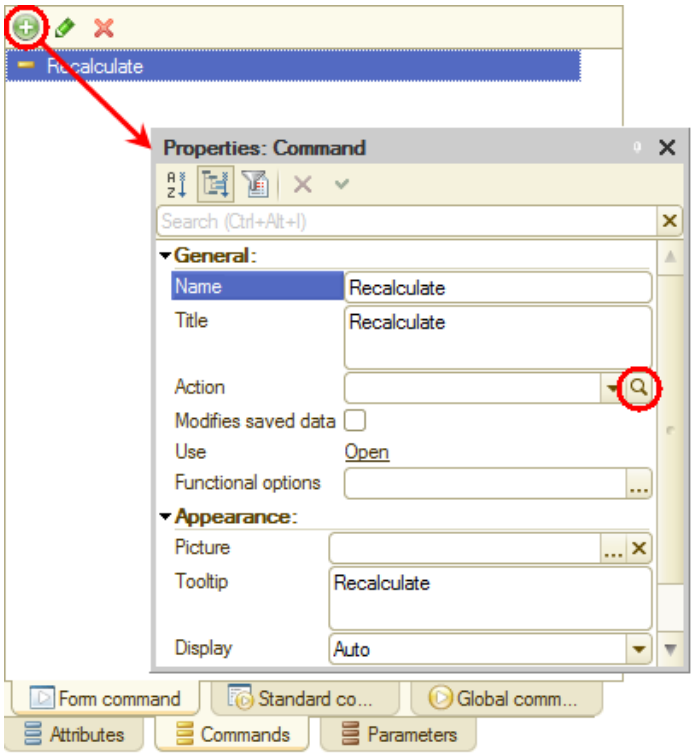


Fig. 18.28. Adding a form command

Let us specify the action for this command.

4. In the **Action** field, click the **Open** button.
5. Click **Create on client** and then click **OK**.

   This adds the Recalculate() procedure template to the form module.
6. Add the call of the RecalculateAccruals() procedure defined in the Calculations common module to the Recalculate() procedure (listing 18.8).

Listing 18.8. Recalculate() command handler

```
&AtClient
Procedure Recalculate(Command)

    Calculations.RecalculateAccruals(PredefinedValue(
        "ChartOfCalculationTypes.MainAccruals.Salary"));
    Calculations.RecalculateAccruals(PredefinedValue(
        "ChartOfCalculationTypes.MainAccruals.Bonus"));

EndProcedure
```

7. Create the RecalculateAccruals() procedure in the Calculations common module (listing 18.9).

Listing 18.9. RecalculateAccruals() procedure

```
Procedure RecalculateAccruals(RequiredCalculationType) Export

    // Selecting records from the recalculation record set
    // in the following order:
    // records from document1 for the listed employees
    // records from document2 for the listed employees, and so on

    Query = New Query(
        "SELECT
        |    AccrualsRecalculation.RecalculationObject,
        |    AccrualsRecalculation.Employee
        |FROM
        |    CalculationRegister.Accruals.Recalculation
        |        AS AccrualsRecalculation
        |WHERE
        |    AccrualsRecalculation.CalculationType =
        | &RequiredCalculationType
        |TOTALS BY
        |    AccrualsRecalculation.RecalculationObject");

    Query.SetParameter("RequiredCalculationType",
        RequiredCalculationType);
    EmployeeList = New ValueList;

    // Iterating through the grouping by recorder
    SelectionByRecorder =
        Query.Execute().Select(QueryResultIteration.ByGroups);
    While SelectionByRecorder.Next() Do
        Recorder = SelectionByRecorder.RecalculationObject;

        // Iterating through the employee grouping
        // for the selected recorder and creating the employee list
        SelectionByEmployee = SelectionByRecorder.Select();
        EmployeeList.Clear();
        While SelectionByEmployee.Next() Do
            EmployeeList.Add(SelectionByEmployee.Employee);
        EndDo;

        // Getting the calculation register record set
        // for the selected recorder
        RecordSet = CalculationRegisters.Accruals.CreateRecordSet();
```

```
        RecordSet.Filter.Recorder.Value = Recorder;

        RecordSet.Read();


        CalculateAccruals(RecordSet, RequiredCalculationType,
            EmployeeList);

        RecordSet.Write( , True);


        // Removing the recalculated records from the recalculation

        RecalculationRecordSet = CalculationRegisters
            .Accruals.Recalculations.Recalculation.CreateRecordSet();

        RecalculationRecordSet.Filter.RecalculationObject.Value =
            Recorder;

        RecalculationRecordSet.Write();


    EndDo;


EndProcedure
```

In the beginning of the procedure a query selects recalculation records that contain the calculation type passed to the procedure, grouped by recalculation object.

Next, the procedure iterates through the query results to generate a list of employees for each recalculation object, reads the corresponding calculation register records, and calls the CalculateAccruals procedure(), which you used earlier to calculate records in the EmployeeAccruals document.

Once the calculation of records is completed, the procedure writes the record set without generating recalculation records, and clears the recalculation records for the recalculation object that has just been processed.

8. Return to the **EmployeeAccruals** report form.

   So you specified the action (the command execution handler) for the Recalculate command. To be able to use the command, you have to create a button in the form and link it to the command (in the Command name button property).

   The simplest way to do it is to drag the command from the **Form command** pane to the form controls pane.

9. Drag the **Recalculate** command to the **MainCommandBar** group of form controls.

   This adds the Recalculate button, which is linked to the command, to the form (fig. 18.29).

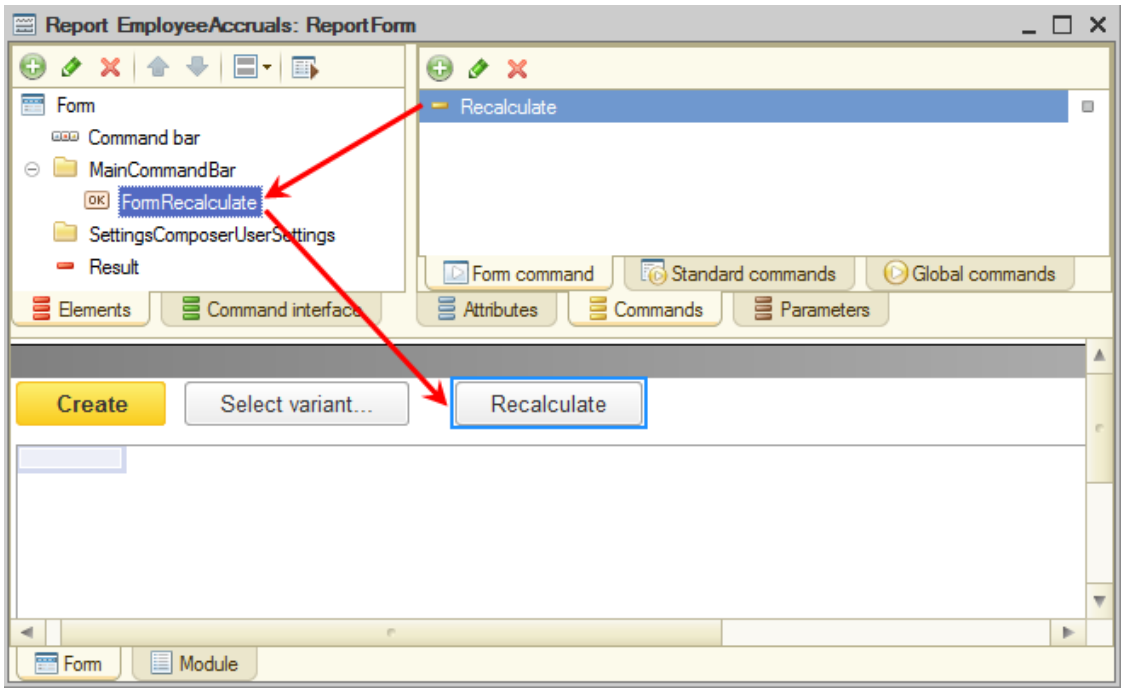
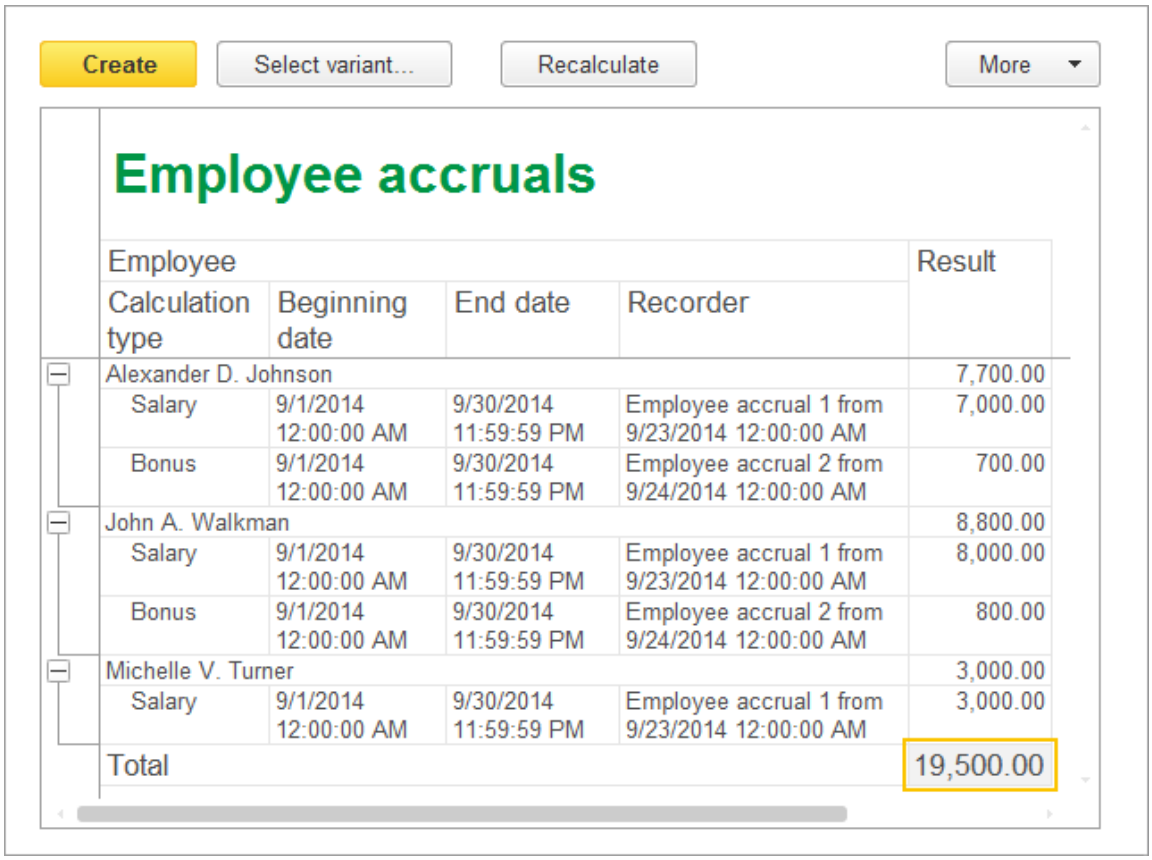


Fig. 18.29. Adding a button to a form

In 1C:Enterprise mode

Let us test the recalculation of calculation register records.

1. Start 1C:Enterprise in the debug mode.
2. Clear the posting of all Employee accrual documents, and then post Employee accrual # 1 and Employee accrual #2.
3. Create the **Employee accruals** report (fig. 18.30).



Employee / Calculation type	Beginning date	End date	Recorder	Result
Alexander D. Johnson				7,700.00
Salary	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 1 from 9/23/2014 12:00:00 AM	7,000.00
Bonus	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 2 from 9/24/2014 12:00:00 AM	700.00
John A. Walkman				8,800.00
Salary	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 1 from 9/23/2014 12:00:00 AM	8,000.00
Bonus	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 2 from 9/24/2014 12:00:00 AM	800.00
Michelle V. Turner				3,000.00
Salary	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 1 from 9/23/2014 12:00:00 AM	3,000.00
Total				19,500.00

Fig. 18.30. Employee accruals report

4. Open the Employee accrual #1 document, change Johnson's salary to 10 000, and post the document.
5. In the **Employee accruals** report, click the **Recalculate** button.

   You can see that the bonuses for Johnson and Walkman are recalculated (fig. 18.31).

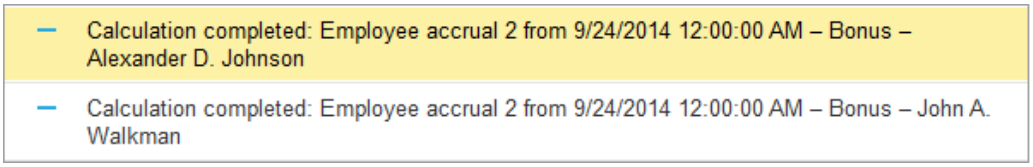


Fig. 18.31. Service message window

6. In the report, click **Create**.

   This is required for the report to display up-to-date data. The resulting report contains the new value for Johnson's bonus (fig. 18.32).

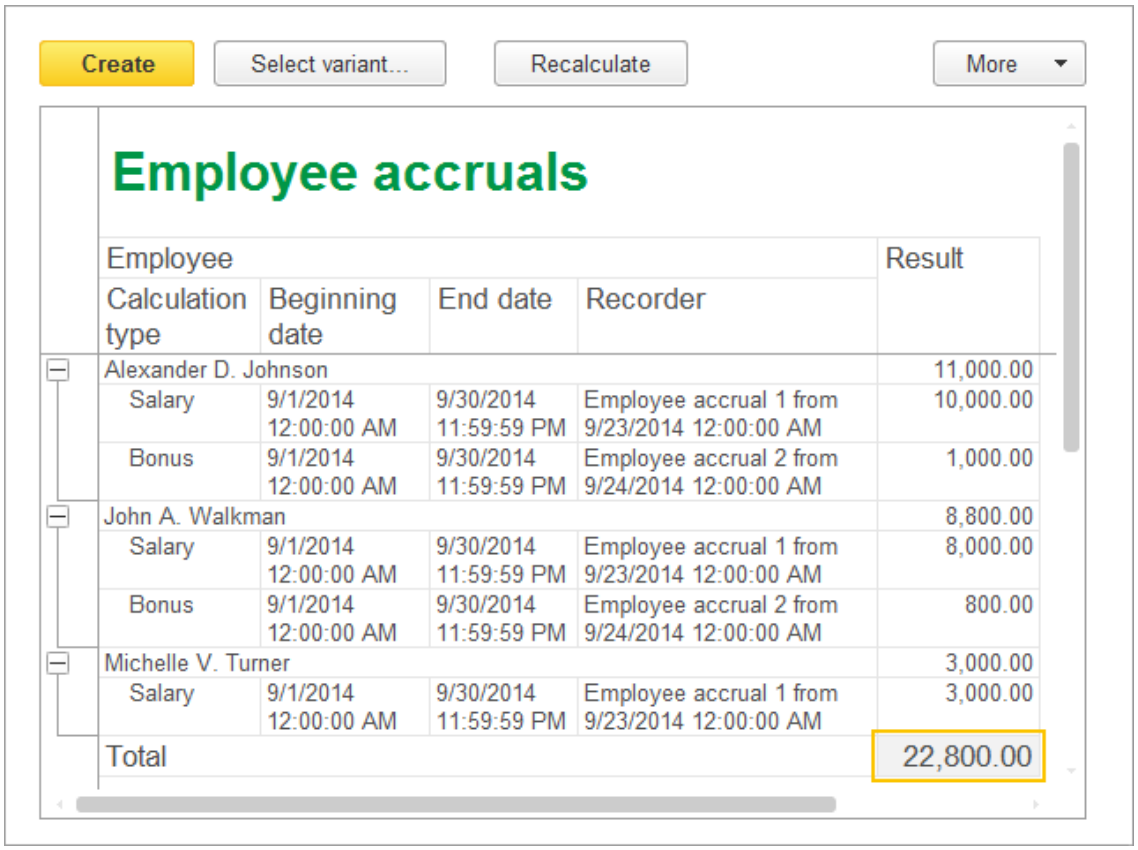


Fig. 18.32. Employee accruals report

7. Post Employee accrual # 3 and then, in the report, click **Recalculate**.

   You can see that Johnson's bonus and salary are recalculated (fig. 18.33).

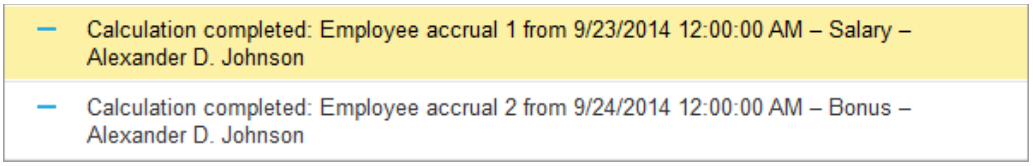
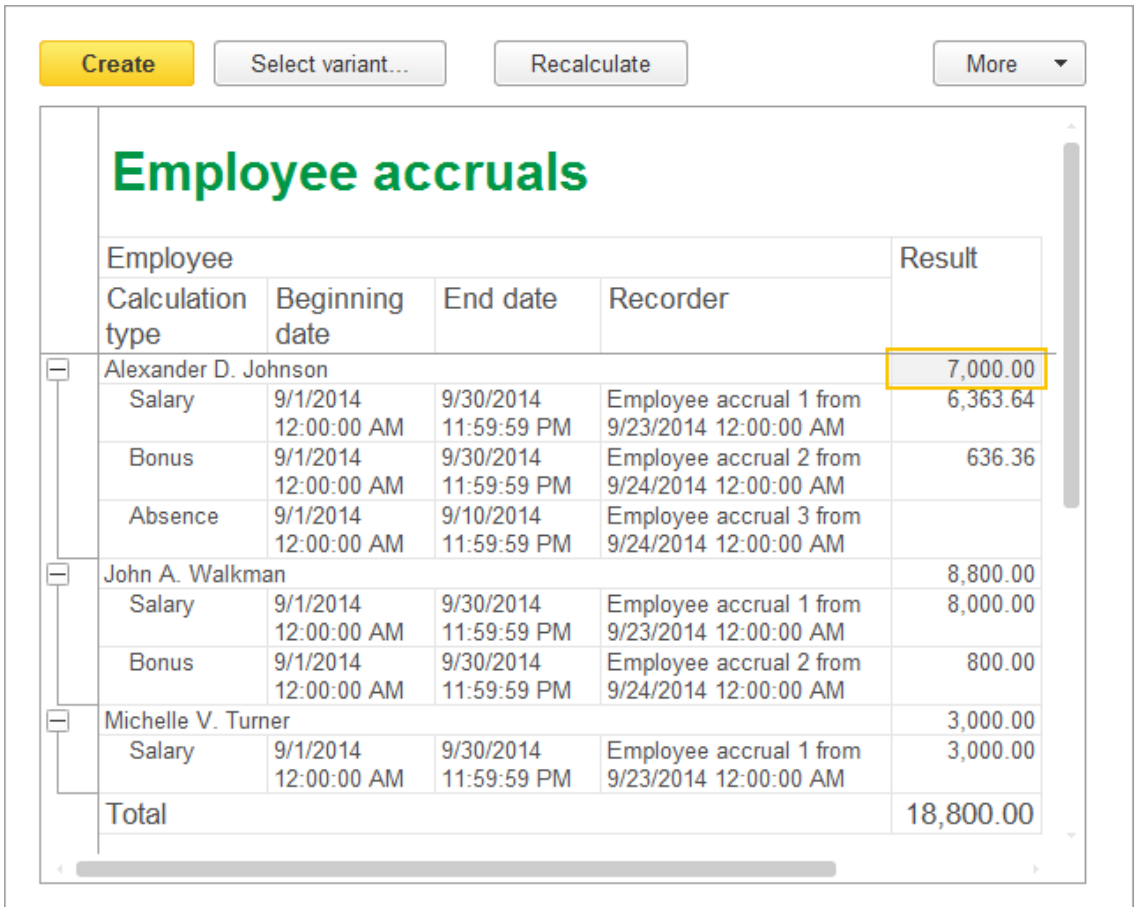


Fig. 18.33. Service message window

8. In the report, click **Create**.

   The resulting report contains up-to-date salary and bonus values (fig. 18.34).



Employee / Calculation type	Beginning date	End date	Recorder	Result
Alexander D. Johnson				7,000.00
Salary	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 1 from 9/23/2014 12:00:00 AM	6,363.64
Bonus	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 2 from 9/24/2014 12:00:00 AM	636.36
Absence	9/1/2014 12:00:00 AM	9/10/2014 11:59:59 PM	Employee accrual 3 from 9/24/2014 12:00:00 AM	
John A. Walkman				8,800.00
Salary	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 1 from 9/23/2014 12:00:00 AM	8,000.00
Bonus	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 2 from 9/24/2014 12:00:00 AM	800.00
Michelle V. Turner				3,000.00
Salary	9/1/2014 12:00:00 AM	9/30/2014 11:59:59 PM	Employee accrual 1 from 9/23/2014 12:00:00 AM	3,000.00
Total				18,800.00

Fig. 18.34. Employee accruals report

Gantt chart

Let us create a report that provides a visual representation of the actual action periods for calculation records.

In addition to a visual demonstration of the displacement of records by action period, this report introduces the Gantt chart form control.

A Gantt chart is a chart of intervals on a time scale (fig. 18.35) that reflects usage of resources (series) by objects (points).

To better understand components of a Gantt cart, let us examine the chart that you will get as a result of this tutorial.

As we have already mentioned, for each employee the chart displays actual action periods for each record of each calculation type available for that employee.

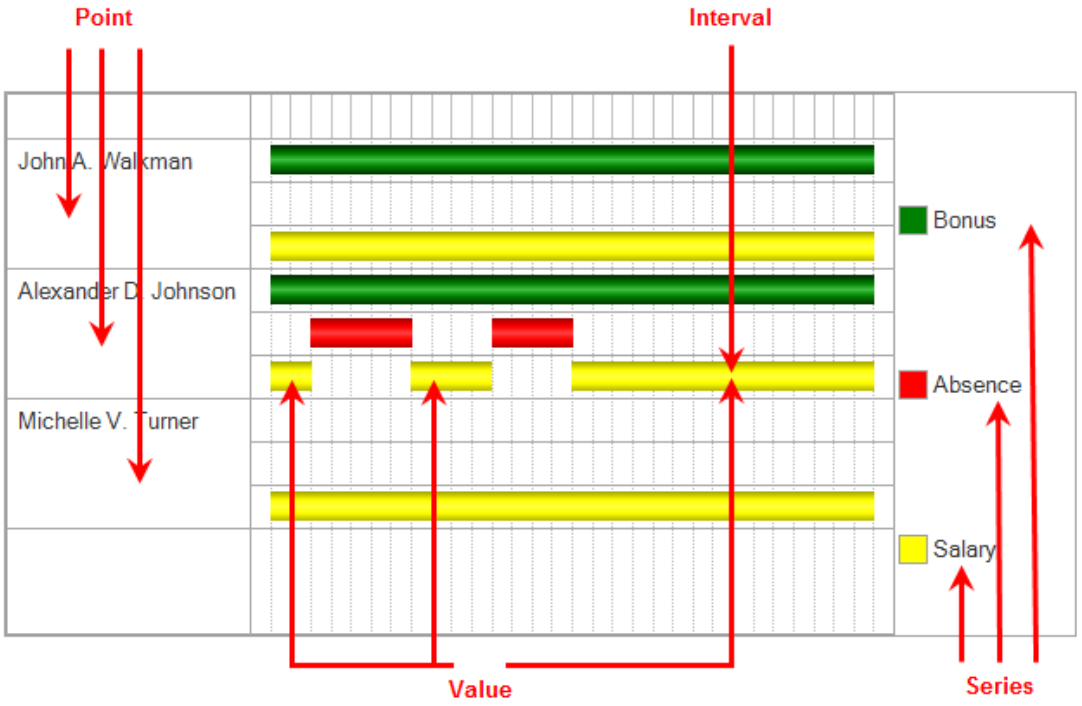


Fig. 18.35. Example Gantt chart

A Gantt chart is a total of points, series, and values for each point-series combination.

In this case, the chart points are employees and the series are calculation types. Thus, for each employee there is a certain chart value for each of the series (for each calculation type).

Gantt chart values are objects generated automatically based on the points and series defined for this chart.

These objects represent a collection of intervals, in other words, they can contain not just one, but multiple intervals, which exist for a point-series combination (by default, GanttChartValue objects are created containing no intervals). A developer can create a chart value by specifying the point and series and then adding any number of intervals to the collection.

All intervals for all chart values are arranged in relation to a single time axis, which shows their relative positions.

Now we will briefly explain what you will do next.

As source data for the chart, you will take data from the Accruals calculation register. Each of the register records contains everything you need to create the chart: employee, calculation type, and the beginning and end of the interval.

All you need to do is use the 1C:Enterprise script tools to arrange all that in a chart.

So, let us proceed.

In Designer mode

Let us create a Gantt chart.

1. Create a report configuration object named *AccrualsChart*.

   This report is not based on a data composition schema. Instead, you will create a default report form and then generate the Gantt chart and specify its settings using 1C:Enterprise script.

2. In the configuration object editor for the AccrualsChart report, on the **Forms** tab, in the **Report form** field, click the **Open** button and create the default report form.

   The upper right pane of the form editor (the **Attributes** tab) lists the form attributes. The default form attribute (Report) is created automatically upon form creation.

3. Click the **Add attribute** button to add a form attribute. Enter *GanttChart* as a name and select **GanttChart** as its type (fig. 18.36).

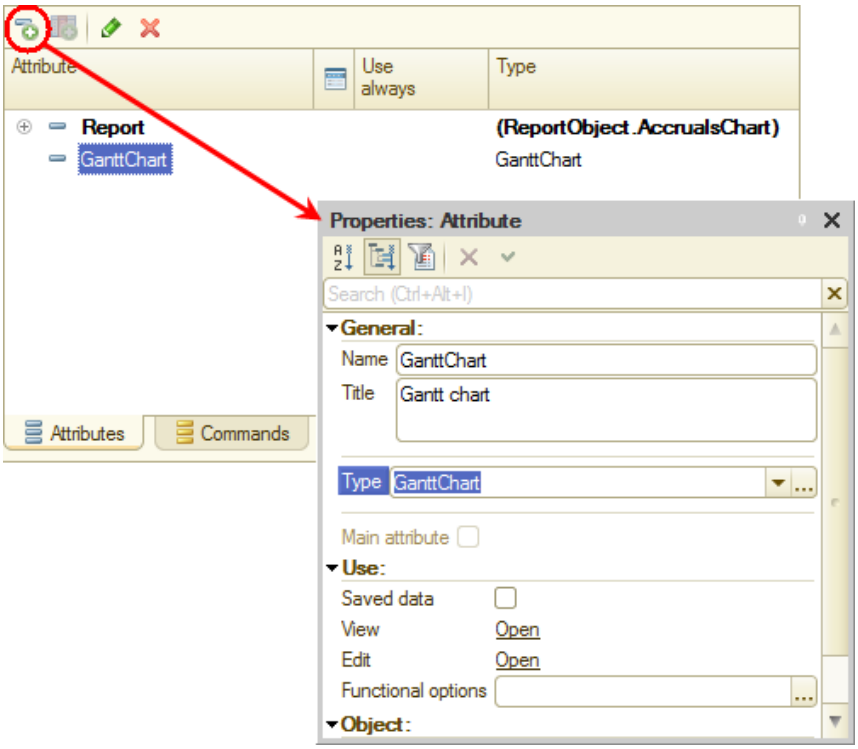


Fig. 18.36. Adding a form attribute

4. Drag the new attribute to the form controls pane.

   Before you perform this step, the controls pane is empty. Once you drag the attribute, a Gantt chart field is added to the form preview pane (fig. 18.37).

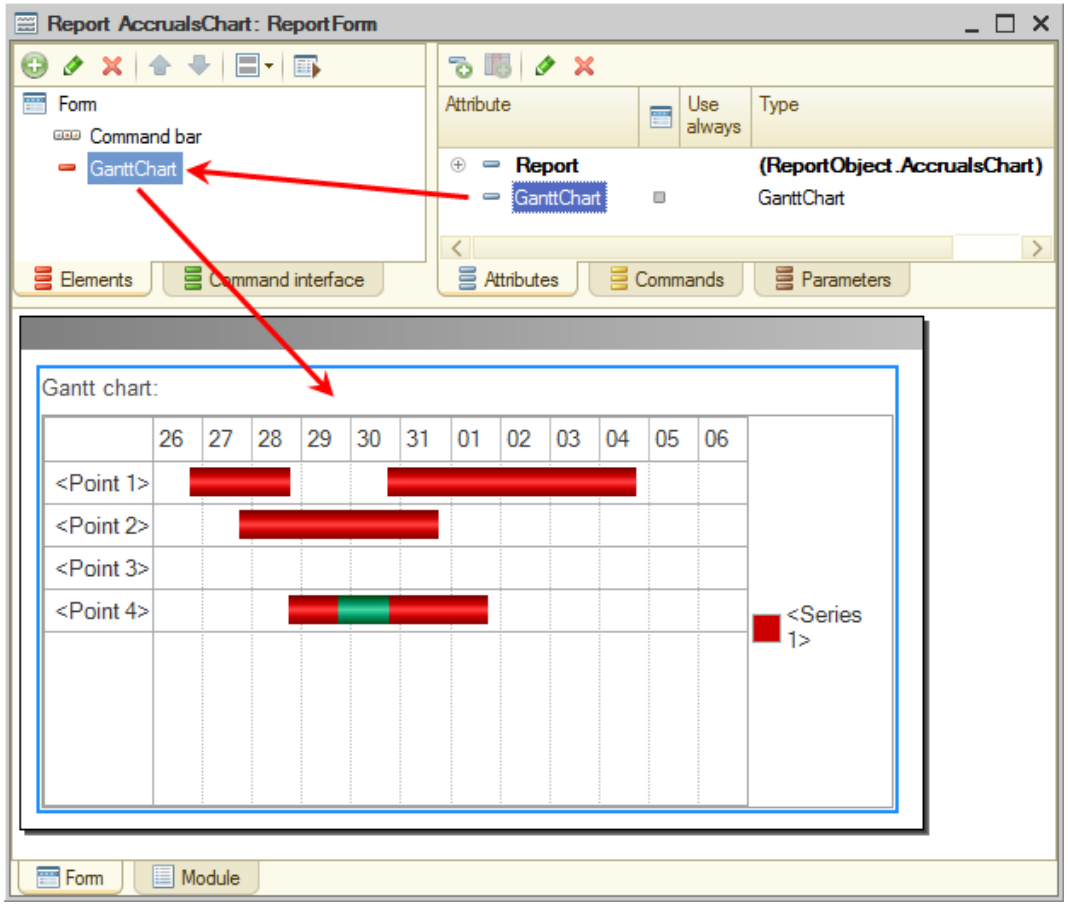


Fig. 18.37. Adding a Gantt chart to a form

5. On the **Commands** tab, create the *Create* command (fig. 18.38).

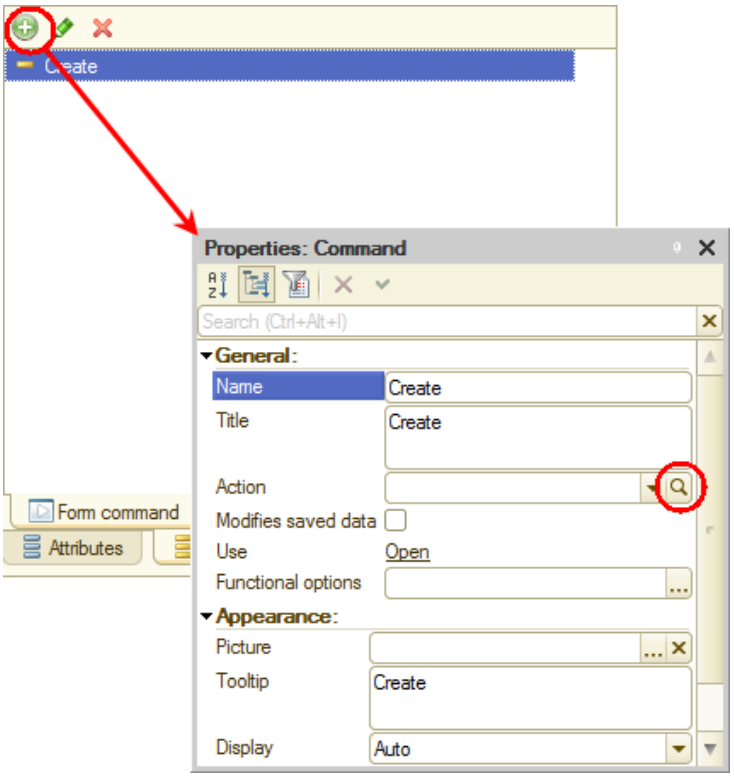


Fig. 18.38. Adding a form command

Let us specify the action for this command.

6. In the **Action** field, click the **Open** button.
7. Click **Create on client and a procedure on server (no context)** (fig. 18.39) and then click **OK**.

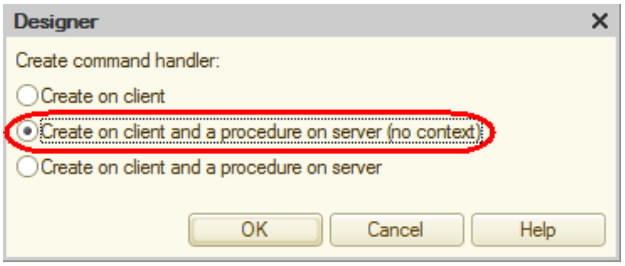


Fig. 18.39. Selecting form command handler type

This adds two procedure templates to the form module: the client Create() procedure and the server out-of-context CreateAtServer() procedure that is called from the Create() procedure.

We will skip the definition of a server out-of-context procedure in this tutorial, only mentioning that their execution is faster because they do not require passing the entire form context from the client to the server.

However, you have to pass the reference to the GanttChart form item as a parameter to the CreateAtServer() procedure to have it filled with data on the server side.

8. Change the module script as shown in listing 18.10.

Listing 18.10. CreateAtServer() procedure

```
&AtServerNoContext
Procedure CreateAtServer(Chart)
    // Insert handler contents.
EndProcedure

&AtClient
Procedure Create(Command)
    CreateAtServer(GanttChart);
EndProcedure
```

9. Add a query template to the CreateAtServer() procedure (listing 18.11).

Listing 18.11. CreateAtServer() procedure

```
&AtServerNoContext
Procedure CreateAtServer(Chart)

    Query = New Query;
    Query.Text="";

EndProcedure
```

10. Right-click between the quotation marks, click **Query Builder**, and confirm that you want to create a query.
11. Select the **Accruals.ActualActionPeriod** virtual register table as the data source.
12. Select the following fields from this table (fig. 18.40):
    - **Employee**
    - **CalculationType**
    - **BegOfActionPeriod**
    - **EndOfActionPeriod**
    - **Result**
    - **Recorder**
    - **Recorder.Presentation**

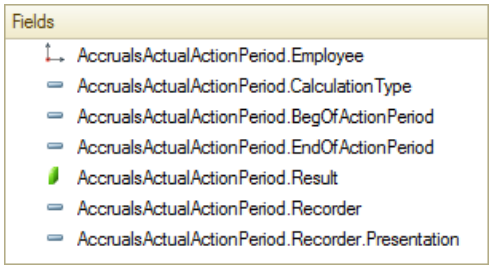


Fig. 18.40. Fields selected for the query

This completes the query creation.

13. Click **OK** and add the following script after the query (listing 18.12).

Listing 18.12. CreateAtServer() procedure

```
&AtServerNoContext
Procedure CreateAtServer(Chart)

    Query = New Query;
    Query.Text=
        "SELECT
        |    AccrualsActualActionPeriod.Employee,
        |    AccrualsActualActionPeriod.CalculationType,
        |    AccrualsActualActionPeriod.BegOfActionPeriod,
        |    AccrualsActualActionPeriod.EndOfActionPeriod,
        |    AccrualsActualActionPeriod.Result,
        |    AccrualsActualActionPeriod.Recorder,
        |    AccrualsActualActionPeriod.Recorder.Presentation
        |FROM
        |    CalculationRegister.Accruals.ActualActionPeriod
        |        AS AccrualsActualActionPeriod";

    SelectionResult = Query.Execute().Select();
```

```
    // Prohibiting chart updates
    Chart.ResfreshEnabled = False;

    Chart.Clear();
    Chart.ShowTitle = False;

    // Filling chart
    While SelectionResult.Next() Do

        // Getting series, point, and value
        CurSeries = Chart.SetSeries(SelectionResult.CalculationType);
        CurPoint = Chart.SetPoint(SelectionResult.Employee);
        CurValue = Chart.GetValue (CurPoint, CurSeries);

        // Creating intervals within a value
        CurInterval = CurValue.Add();
        CurInterval.Begin = SelectionResult.BegOfActionPeriod;
        CurInterval.End = SelectionResult.EndOfActionPeriod;
        CurInterval.Text = SelectionResult.RecorderPresentation;
        CurInterval.Details = SelectionResult.Recorder;

    EndDo;

    // Setting series colors
    For Each Series in Chart.Series Do
        If Series.Value = ChartsOfCalculationTypes.MainAccruals.Salary
            Then
            Series.Color = WebColors.Yellow;

        ElsIf Series.Value =
            ChartsOfCalculationTypes.MainAccruals.Bonus Then
            Series.Color = WebColors.Green;

        ElsIf Series.Value =
        ChartsOfCalculationTypes.MainAccruals.Absence Then
            Series.Color = WebColors.Red;

        EndIf;

    EndDo;

    // Allowing chart updates
    Chart.RefreshEnabled = True;

EndProcedure
```

First, the procedure prohibits chart updates for the period of filling it with data. You need this so that you do not end up with a recalculation for every change to the chart data. Once the chart is filled, the procedure enables updates, and all of the recalculations are performed at once.

In the query selection search loop the chart is filled.

The SetSeries() and SetPoint() methods get either current or new points and series. Points and series are uniquely identified by their values (employee and calculation type obtained by the query).

After that, as a point and series are obtained, the GetValue() method obtains respective chart values.

Then a new interval is added to the chart. Its beginning and end are specified, as well as the interval text, which is displayed when you move the mouse pointer over the interval, and the interval details, which are displayed when you double-click the interval.

Once all the chart values are generated, the fill color is assigned for each series. The chart series represent a collection of values, which are iterated using the For Each … Do statement.

Let us return to the form and add the button that runs the Create command.

14. Drag the **Create** command from the **Form command** pane to the form controls pane (fig. 18.41).

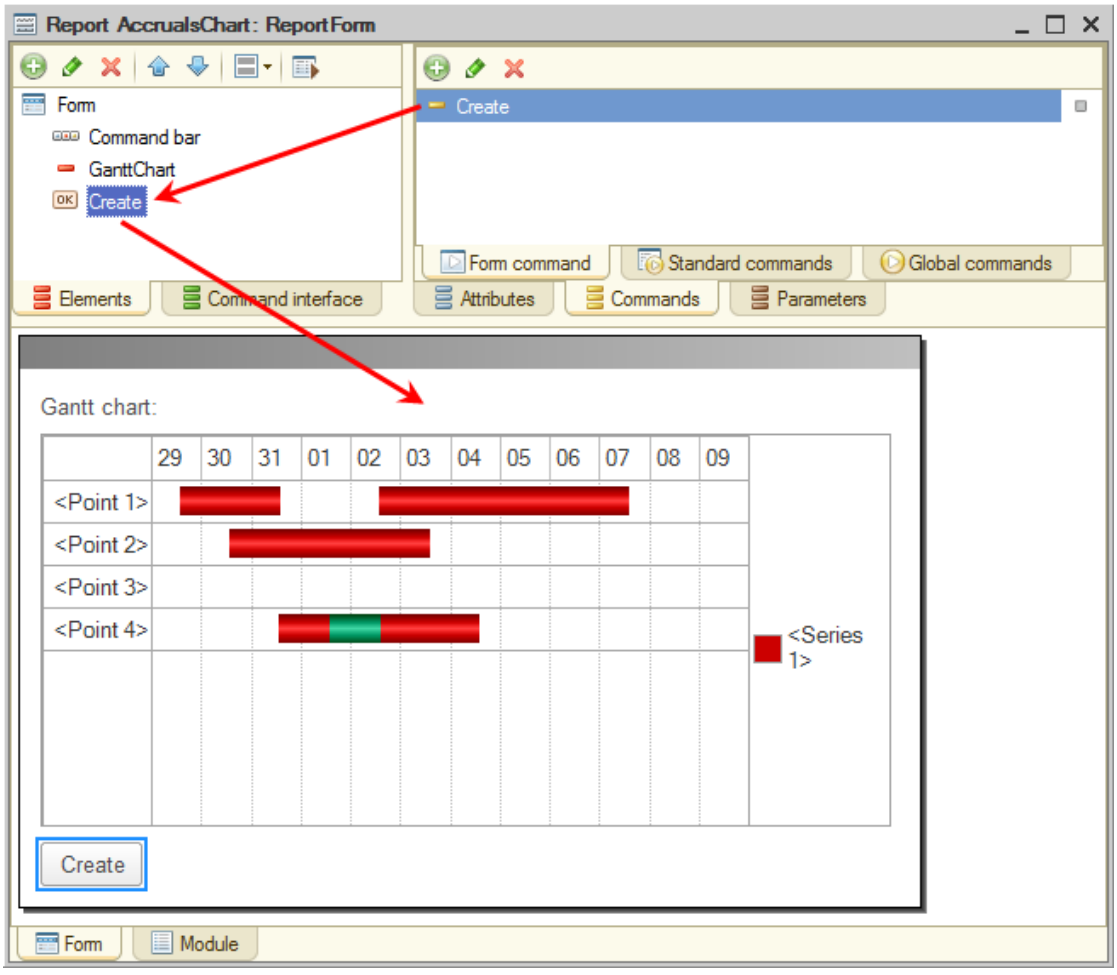


Fig. 18.41. Adding a button to a form

15. In the editor of the **AccrualsChart** report configuration object, click the **Subsystems** tab.
16. Include the report in the **Payroll** subsystem.

In 1C:Enterprise mode

Let us test the report.

1. Start 1C:Enterprise in the debug mode and create the report (fig. 18.42).

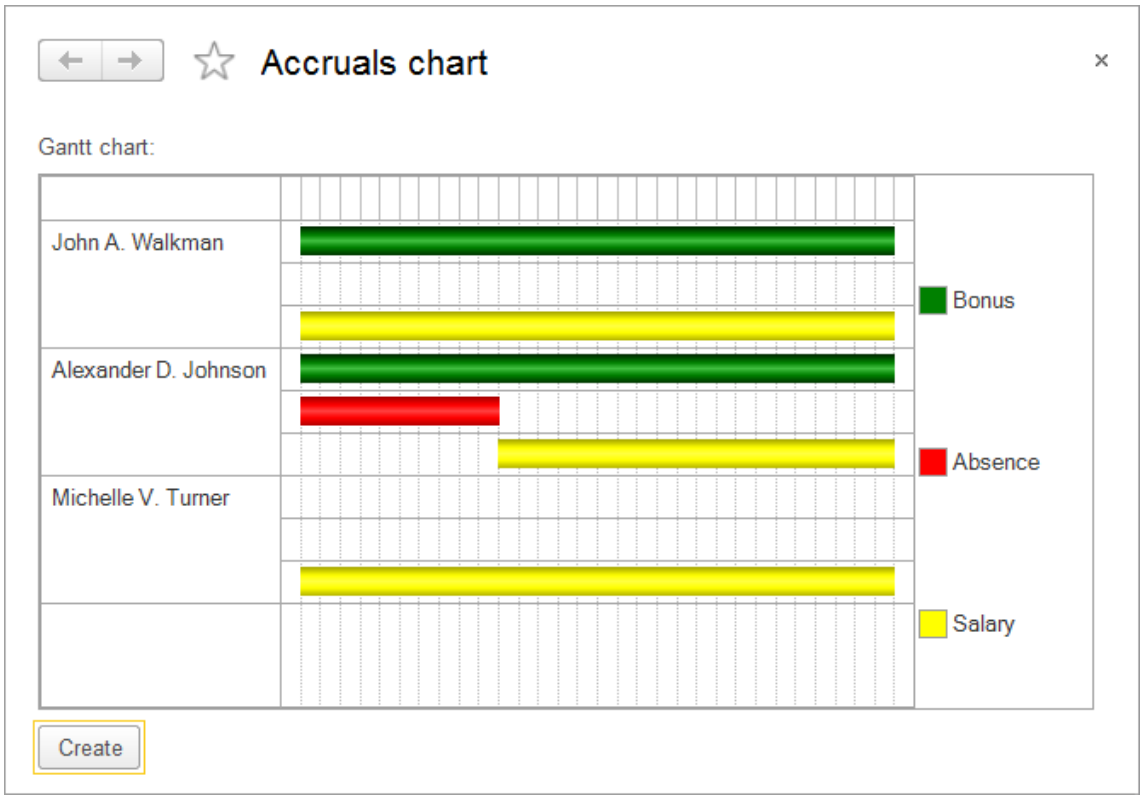


Fig. 18.42. Accruals chart report

Now let us look at how displacement by action period works.

2. Open the Employee accrual #3 document, and instead of one absence from the 1st through the 10th of September, enter two absences for Johnson: from the 3rd through the 7th and from the 12th through the 15th.
3. Repost the document and click the **Create** button in the report (fig. 18.43).

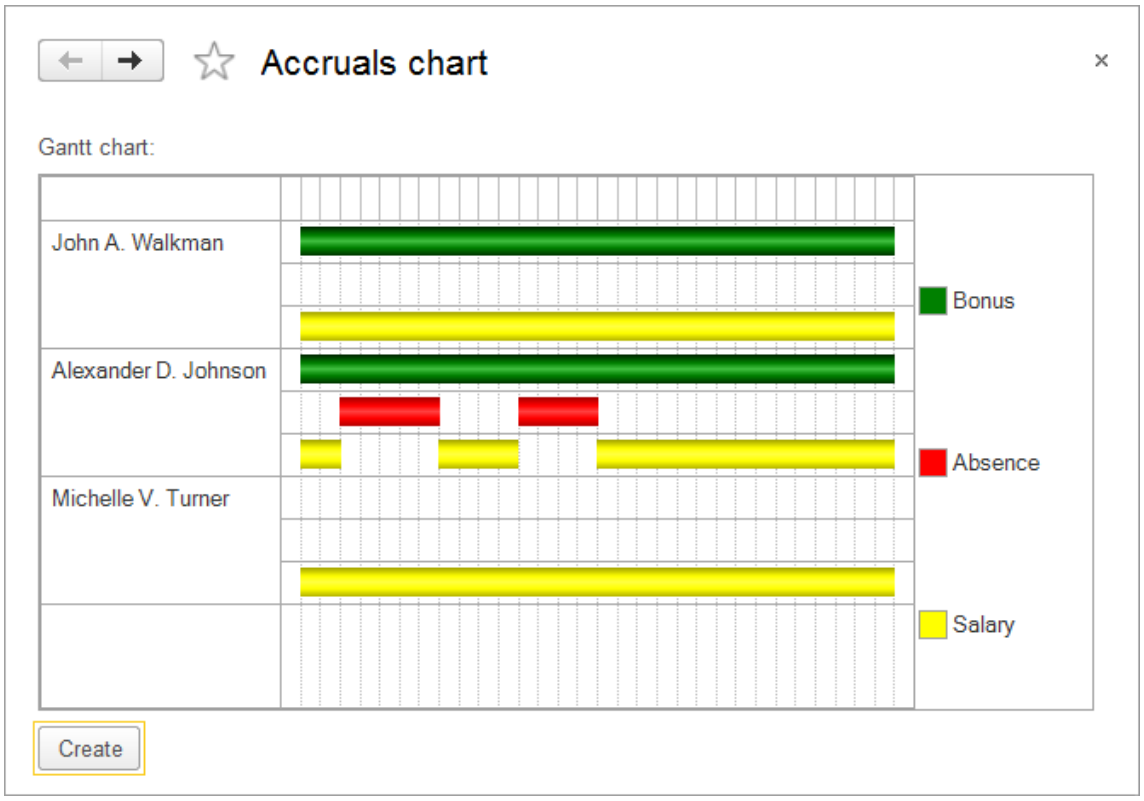


Fig. 18.43. Accruals chart report

You can plainly see how the Absence calculation records displace the Salary calculation record by action period, modifying its actual action period.

Note. You can set up the Gantt chart in 1C:Enterprise mode by right-clicking anywhere in the chart and then clicking **Settings**. Alternatively, you can specify the chart settings in Designer: in the **AccrualsChart** report form editor, in the property palette of the form attribute that represents the chart, for the **Setup** property, click **Open**. You can specify the following settings using this method: Show title, Show legend, Transparent, and more.

Quiz

- How does one create calculation register records for a document?
- How does one use a query to get recalculation records?
- How does recalculation work?
- How does one calculate records of a calculation register?
- How does one use a query to get schedule and base data?
- How does one perform a recalculation for specific calculation register records?
- How does one use a query to get calculation register records?
- How does one use a query to get the actual action periods for calculation register records?

- What is the purpose of a Gantt chart?
- What is the arrangement of a Gantt chart?
- How does one fill the data for a Gantt chart?

Lesson 19 (0:40). Database search

The infobase of Jack of All Trades is very small for now. In the process of creating and testing the configuration you only added a few materials and services and posted a small number of documents.

But real-life infobases contain much more data and sometimes locating required data is a challenging task, especially for a user who is not deeply familiar with the products (services) or with the list of contractors.

To simplify the search for information in the database, 1C:Enterprise offers full-text search. The advantage of this search is the usage of simple and natural search queries, such as "peterson phone". You can also use operators similar to those used for Internet search (AND, OR, NOT, and so on).

The full-text search is handy when you do not know the exact location of the required data (for example, a specific catalog). Full-text search is absolutely vital when you do not know exactly what you need to search for (for example, you do not remember the exact name of a product or a contractor).

Besides, full-text search finds data where other search tools are very time-consuming or require creation of special algorithms and data processors. For example, full-text search works well with lengthy text fields and fields of the ValueStorage type.

In this lesson you will learn the basics of full-text search, create a full-text index, and use the search based on this index to find specific data in the Jack of All Trades database.

Full-text search basics

Full-text search in 1C:Enterprise is based on two components:

- Full-text index
- Full-text search tools

A full-text index is required for full-text search to be available. A full-text index is created once and then should be updated regularly.

The search only covers the data contained in the full-text index. So if a database is used heavily (data is modified, new data is added), the full-text index should be updated as frequently as possible. But if there are moderate volumes of modified or new data, full-text index can be updated less frequently, for example, once a day when the system experiences the least load.

A full-text index can be created and updated both interactively in 1C:Enterprise mode and using 1C:Enterprise script tools. In this lesson we will cover interactive indexing, while the next lesson will show how to update a full-text index automatically.

In a running infobase the platform tracks modifications of data in the configuration objects that can be included in the full-text search. Such objects include exchange plans, catalogs, documents, charts of

characteristic types, charts of accounts, charts of calculation types, registers (information, accumulation, accounting, and calculation), business processes, and tasks.

When the full-text index is created or updated, the platform analyzes the data stored in attributes of such objects and includes this data in the index. Note that it is true only for some attributes: those of the Ref, Number, Date, and ValueStorage types, and those of referential type (for example, CatalogRef.Products).

The full-text search itself is performed using 1C:Enterprise script tools. Leaping ahead, we will note here that full-text search is performed in compliance with user rights. So if some data is not available to a specific user, this user cannot obtain this data using full-text search either.

The results of a full-text search are retrieved in batches and they are sorted in a specific order. This increases the chance to find the required data in the beginning of the first batch. In practice, when a search query is written well, the required data is retrieved in the top three to five results.

Now that you understand how full-text search works in general, let us proceed to the first part of the full-text search tutorial: the creation of a full-text index.

Then you will run the full-text search based on the created index.

Creating a full-text index

First let us review the configuration properties and objects related to full-text search.

In Designer mode

Each configuration object that can be included in full-text indexing has the Full-text search property. The default value of this property is Use.

Hence, right now you do not need to make any changes. But to study the property, let us look at it.

1. Open the property palette of the **MaterialsAndServices** catalog configuration object (fig. 19.1).

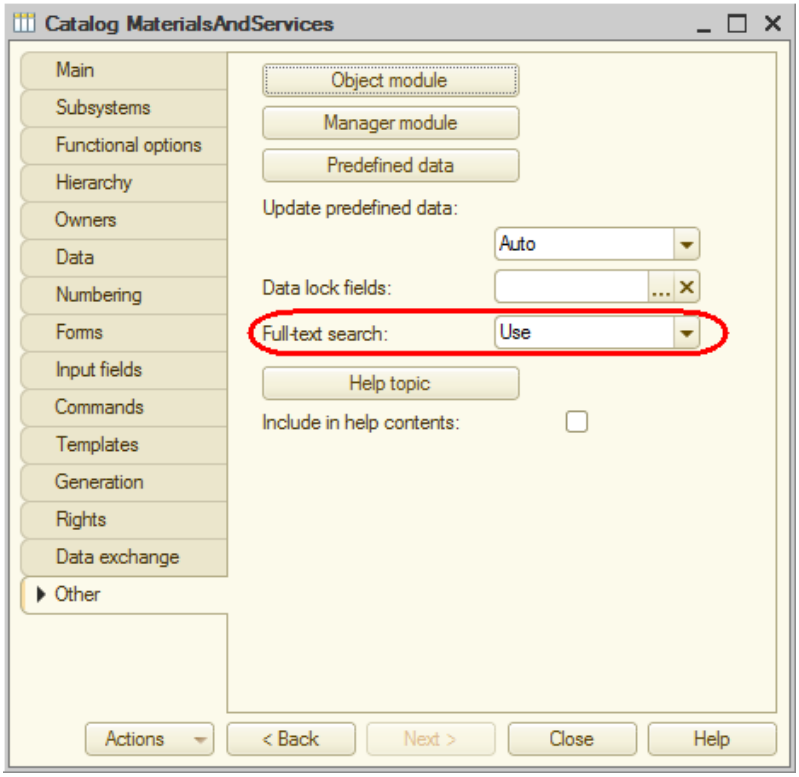


Fig. 19.1. Full-text search property

The Full-text search property is available not only for configuration objects but for their attributes as well. So you can define specific attributes whose data is included in the full-text index.

The default value of the attribute property is also Use, so you do not have to change anything. Let us look at this property.

2. Open the property palette of the **MaterialServiceType** attribute of the **MaterialsAndServices** catalog (fig. 19.2).

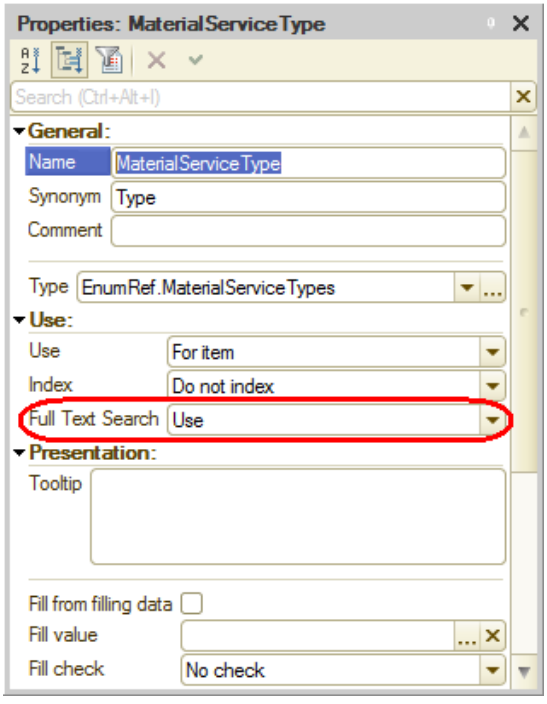


Fig. 19.2. Full-text search property

So by default all attributes of all objects in your configuration are included in the full-text search.

In 1C:Enterprise mode

Let us create the index.

1. Start 1C:Enterprise in the debug mode.
2. On the main menu, click **All functions**, expand **Standard**, and click **Manage Full Text Search** (fig. 19.3).

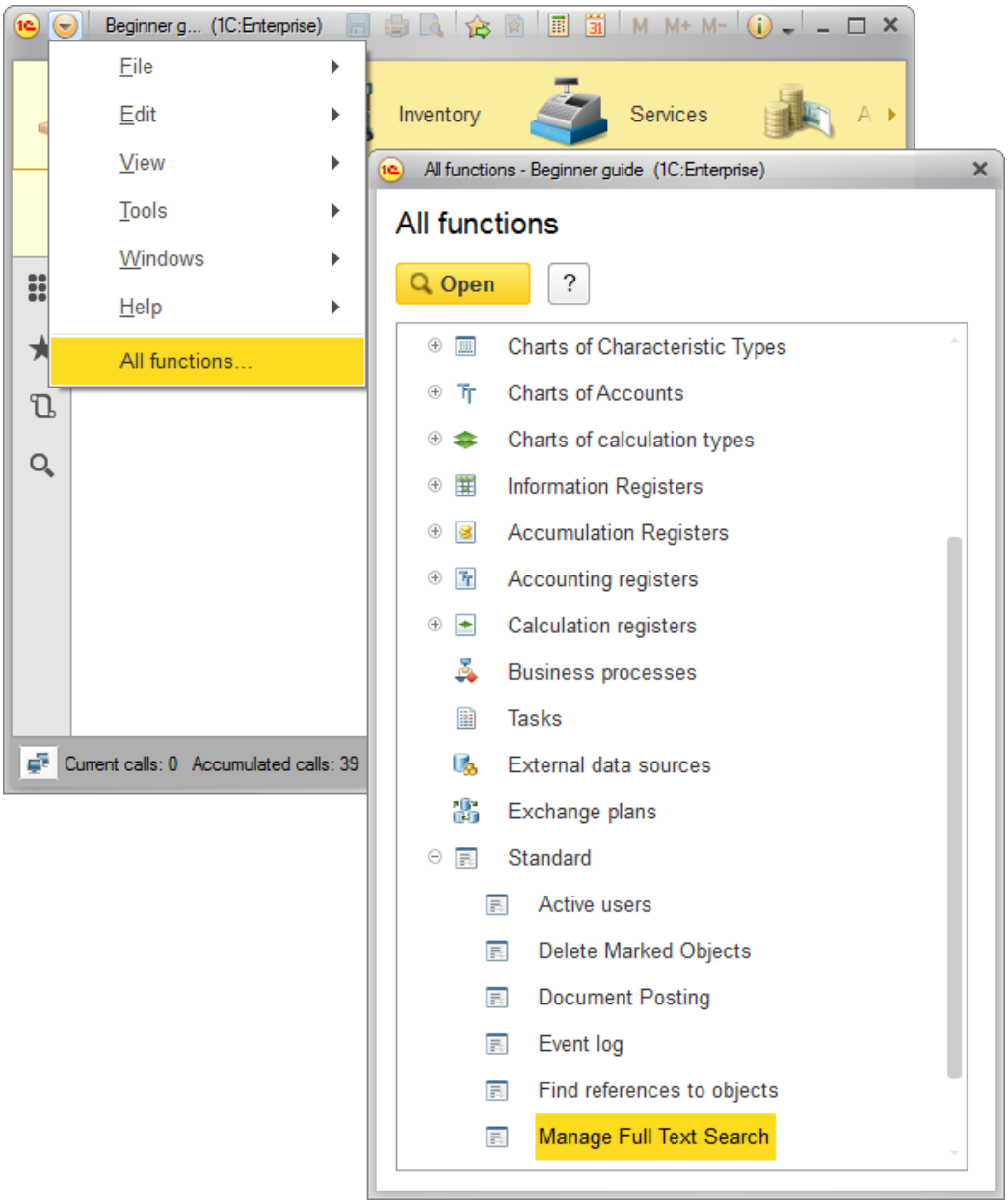


Fig. 19.3. Opening the full-text search management window

This opens the full-text search management window (fig. 19.4).

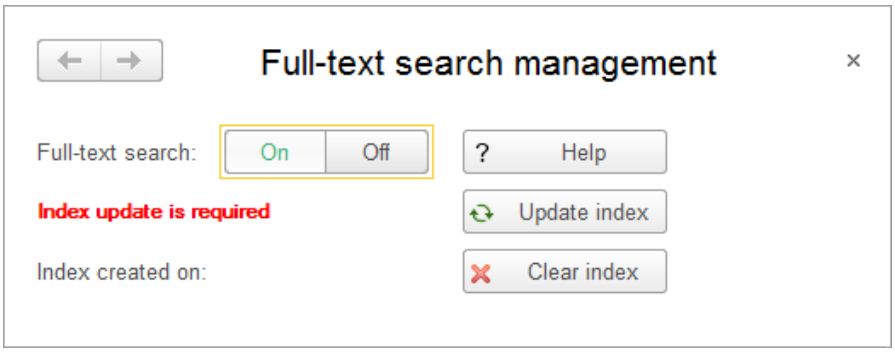


Fig. 19.4. Full-text search management window

In this window you can interactively create and update the full-text index. Besides, here you can allow or prohibit all operations related to the full-text search: updating and clearing the full-text index, and performing the full-text search.

You can see that the full-text search is allowed by default.

As you can see on fig. 19.4, the platform informs you that the index update is required. This is true because the index is not yet available (the index generation date is empty).

3. Click **Update index**.

   You can use this button both to create an index and to update it.

   Note that in a large infobase creating or updating a full-text index can take a few minutes. This is why 1C:Enterprise status bar shows which data portion is being processed during the index update.

   When the full text-index is created, an appropriate message is displayed (fig. 19.5), and the new index generation date is displayed in the full-text search management window (fig. 19.6).

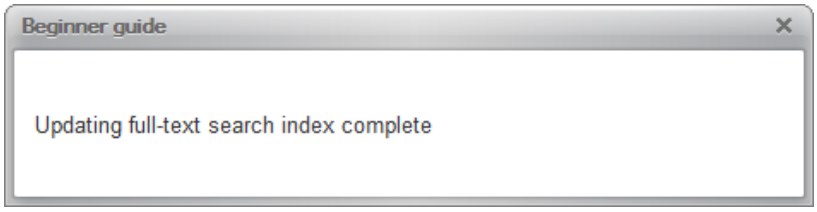


Fig. 19.5. Updating full-text search index complete

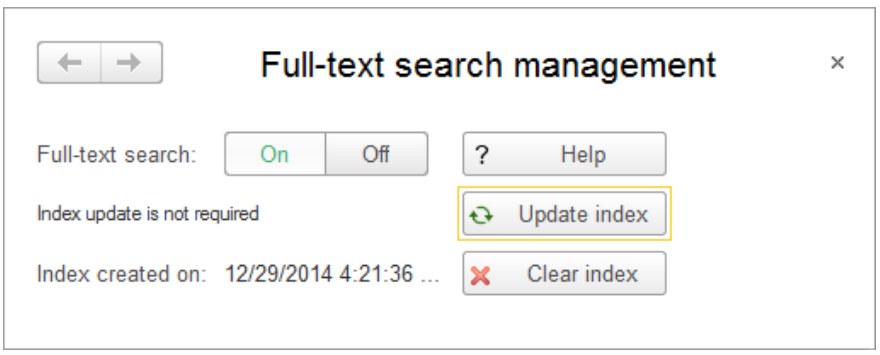


Fig. 19.6. Full-text search management window

So you have created a full-text index for your infobase. Let us proceed to testing the full-text search.

Database search examples

In 1C:Enterprise mode

The search based on the full-text index is a standard 1C:Enterprise platform feature.

1. In the **Tools** pane, click the **Search** button (the one with the magnifying glass, fig. 19.7).

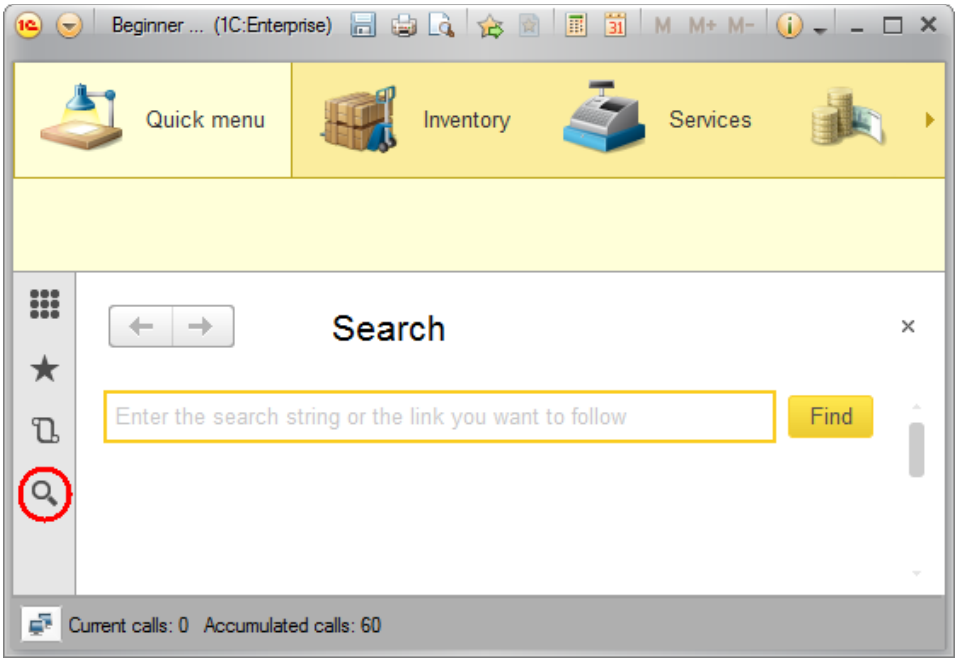


Fig. 19.7. Running the search

   The amount of data in your database is small, still it is enough for studying the basic full-text search features.

2. In the search field, enter *Johnson*.

   -OR-

   In the search field, type *Johnson* and click **Find** (fig. 19.8).

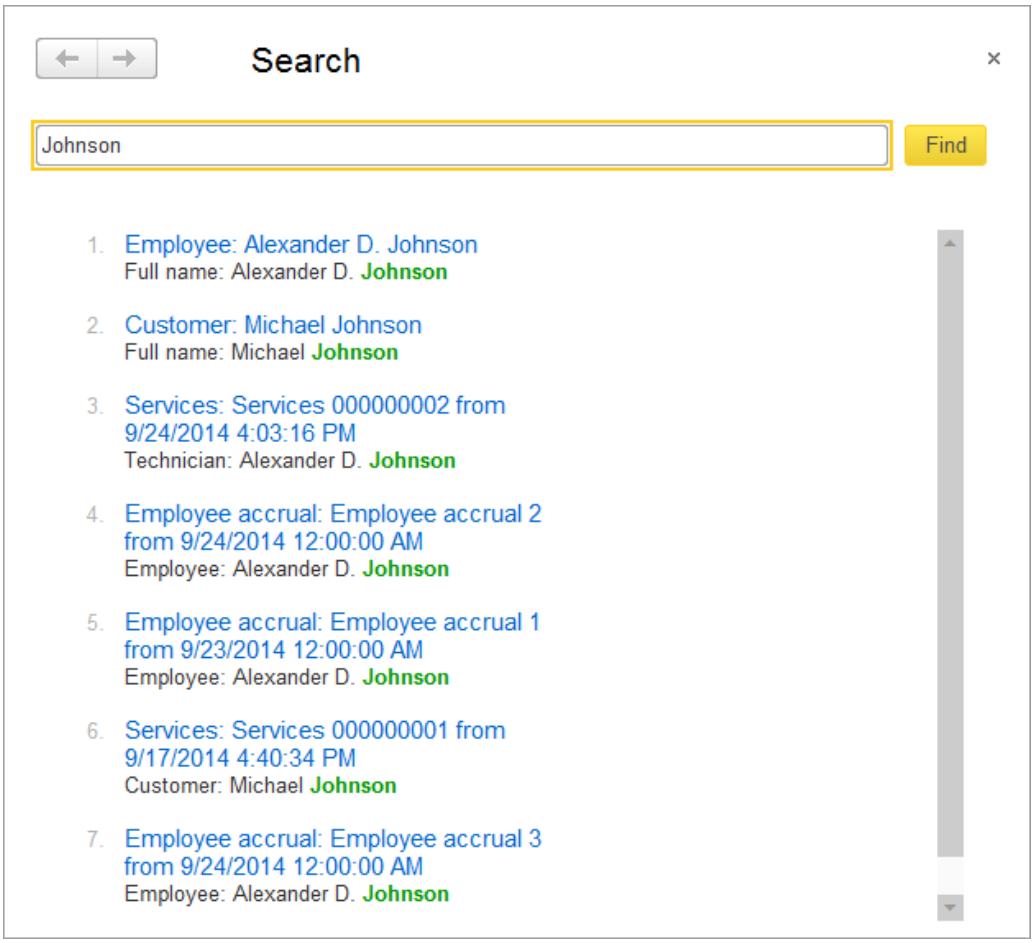


Fig. 19.8. Search for "Johnson"

The search result contains 7 items. The words that match the search string are marked bold and green.

The search can return all word forms. Let us test this.

3. Search for *customers* (fig. 19.9).

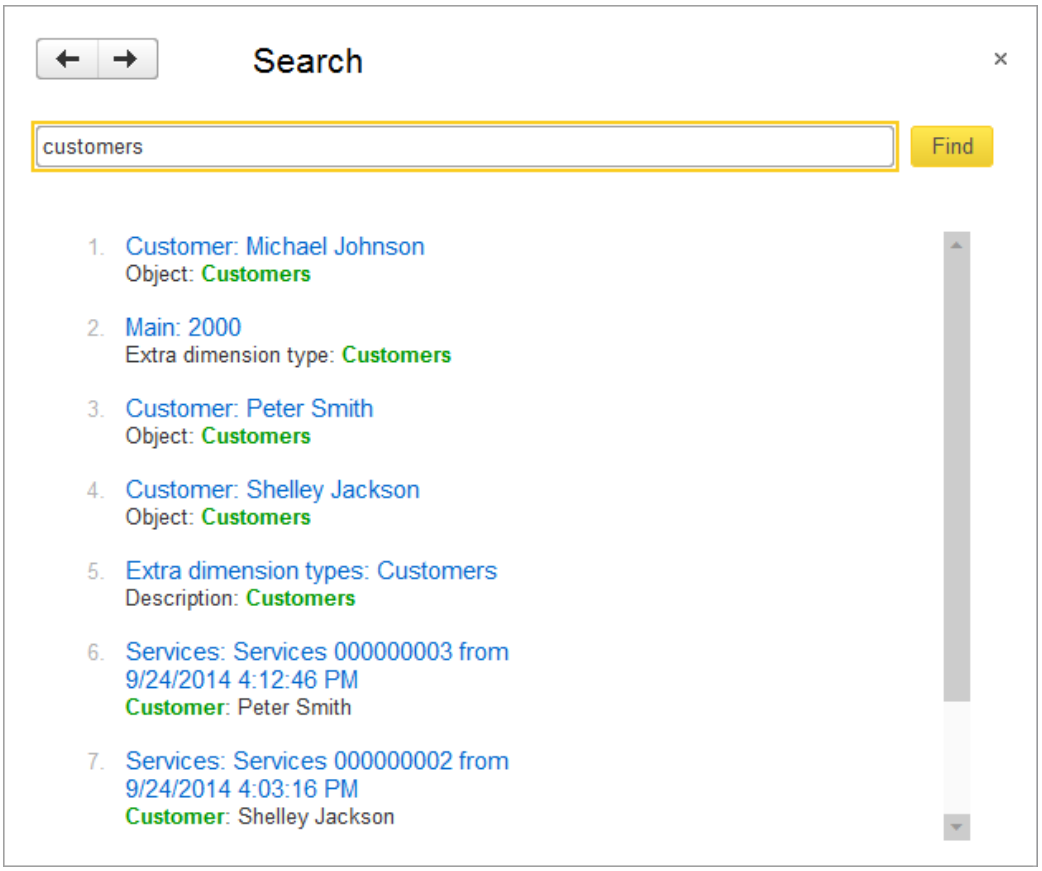


Fig. 19.9. Search for "customers"

Note that different forms of that word are found: "customer" and "customers".

To perform search by the exact phrase, enclose it in double quotation marks.

4. Search for *"customers"* (fig. 19.10).

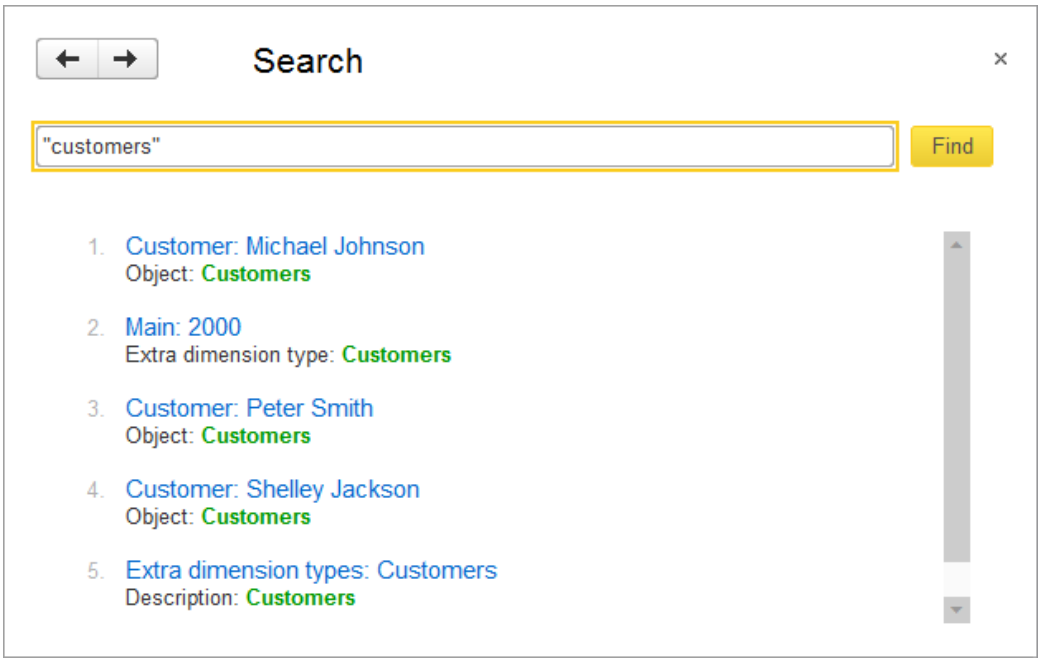


Fig. 19.10. Exact search for "customers"

Sometimes users do not remember the exact material or service names. In this case they have the option to search by the first characters of the word, with the rest of the characters replaced with asterisk (*).

5. Search for *tran** (fig. 19.11).

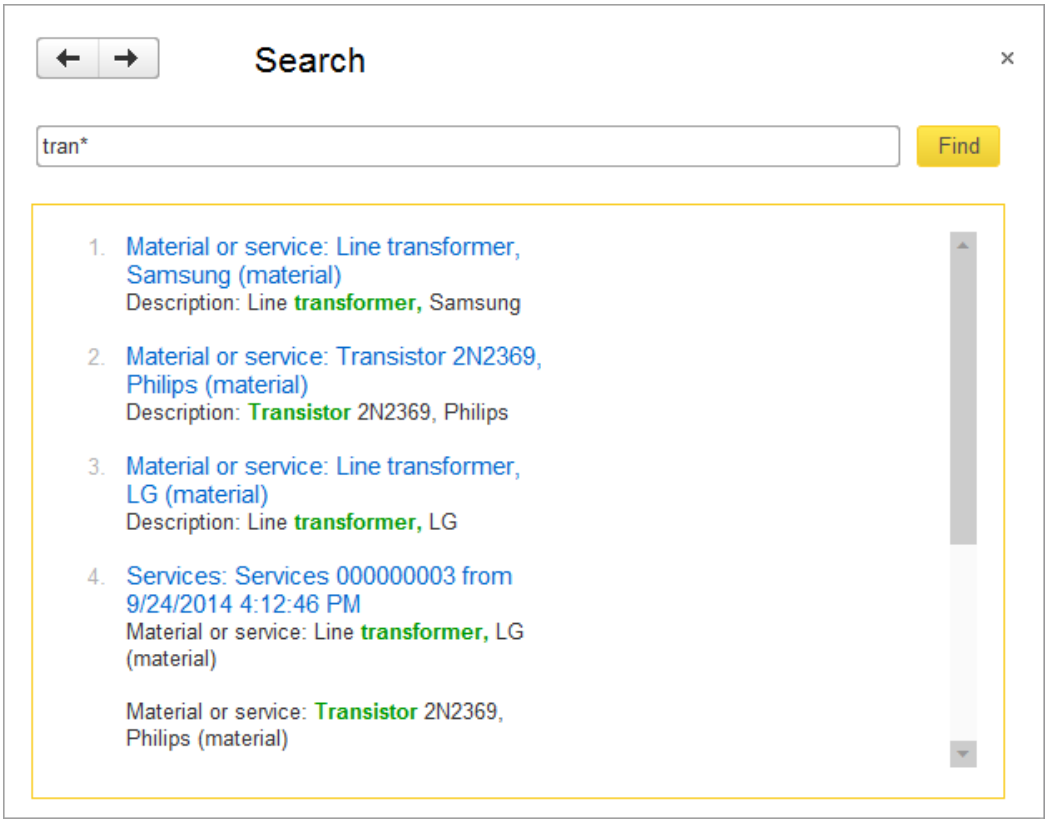


Fig. 19.11. Search for the first characters of a keyword

The search for tran* returns items that include transformers or transistors.

As a rule, using a couple of words gives the best result. For example, if you want to find the date when customer Smith had a transformer replaced in their TV, try "smith transformer".

6. Search for *smith transformer* (fig. 19.12).

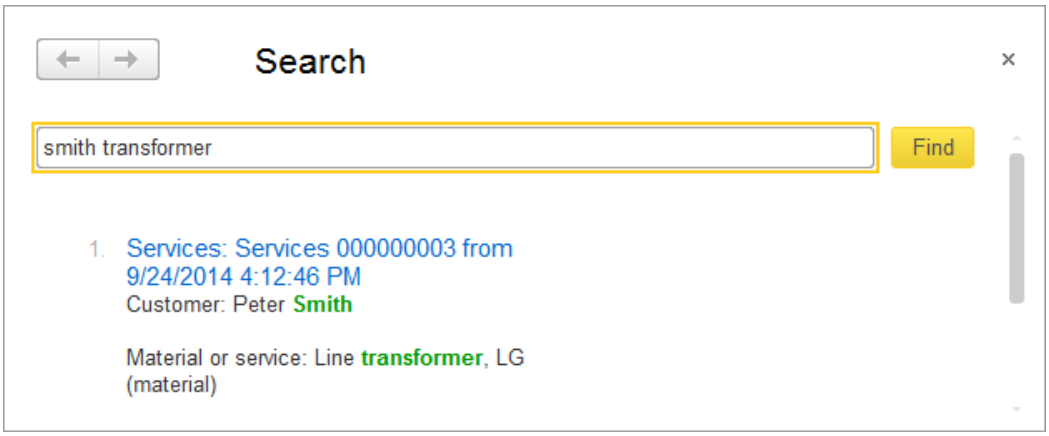


Fig. 19.12. Search for "smith transformer"

Clicking the hyperlink opens the Services #3 document, which contains the full list of services rendered to this customer.

The platform indexes not only the data stored in configuration objects, but also names of attributes and metadata objects. Therefore, if you want to search for Johnson employee, specify the appropriate catalog name or its part.

7. Search for *johnson employee* (fig. 19.13).

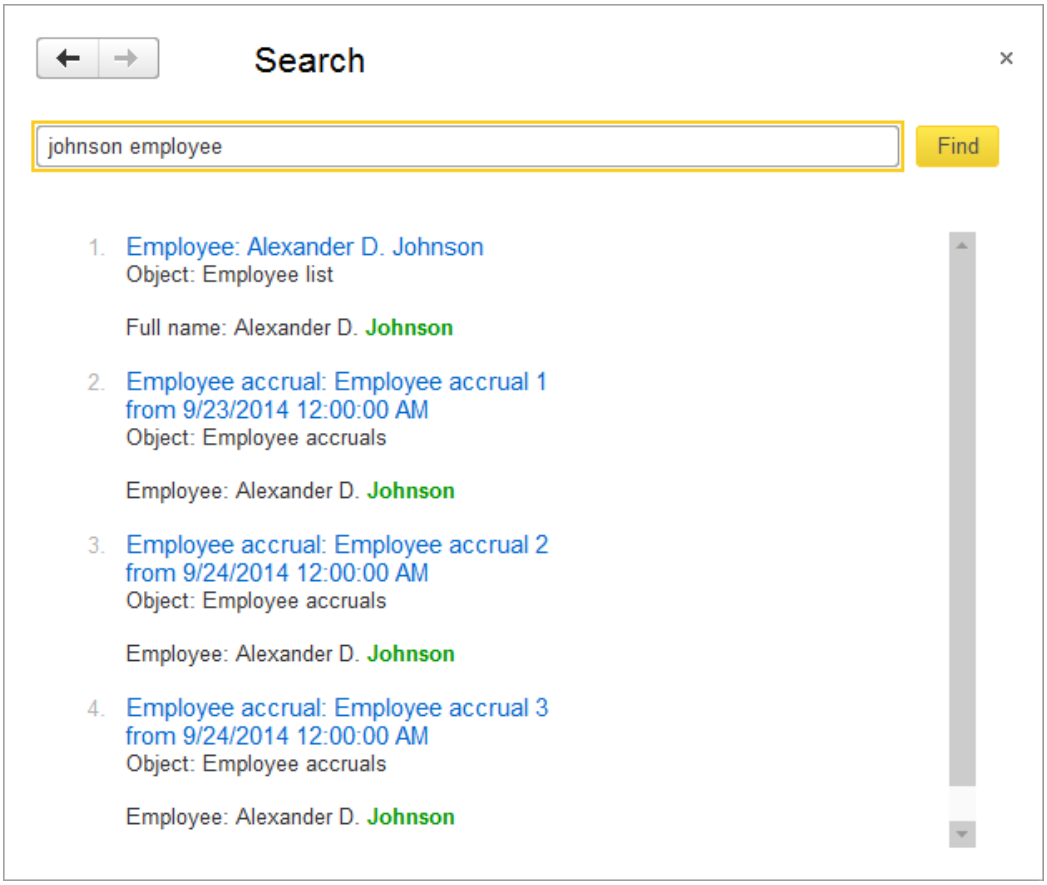


Fig. 19.13. Search for "johnson employee"

As you can see, the platform found several items, including employee accrual documents, but the Employee catalog item is on top of the list.

The platform supports fuzzy search. If you are not sure about the spelling, you can enter number sign (#) followed by the number of characters that can be different. Let us try searching for repair records with incorrect spelling.

8. Search for *reepair#2* (fig. 19.14).

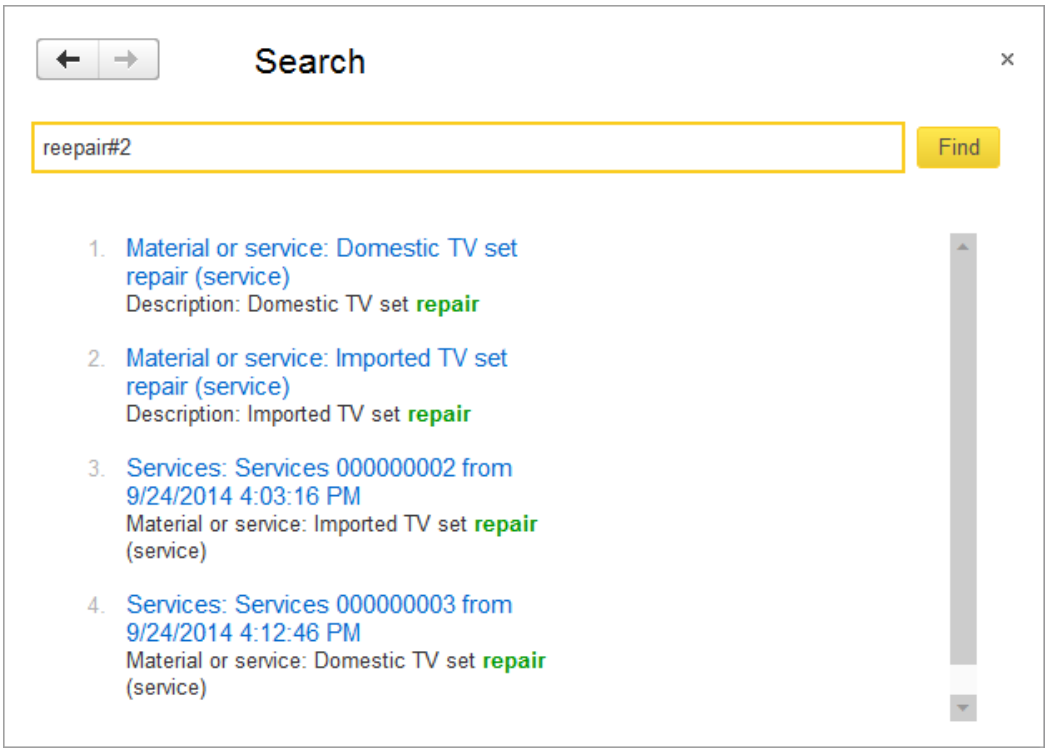


Fig. 19.14. Fuzzy search for "reepair"

Search by synonyms is also available. To use this option, precede the search string by exclamation mark (!). For example, search for !mending gives the same result as search for repair (see fig. 19.14) because they are synonyms.

The search supports a variety of operators, which are described in detail in appendix "Full-text search expressions" of 1C:Enterprise 8.3 Developer guide.

The search form that is displayed when you click the **Find** button (fig. 19.8-19.14) is generated by the platform. This is because the Default search form configuration property value is empty by default.

You can change the default search algorithm if needed by clicking the **Open** button in the property field. This opens the common form wizard, which generates the form that is opened by the platform when users run the search. You can change the form, and then the platform will open your version of that form.

Search during input by string

In Designer mode

1C:Enteprise features another powerful search tool: search during the input in a text field. You encountered this feature earlier when you added materials or services to document tabular sections.

You can set up the input by string feature in the configuration object editor, on the **Input fields** tab. This is applicable to objects that are used as referential fields. The following parameters are available:

- **Search method**. Default value: At beginning of line
- **Full-text search**. Default value: Do not use
- **Search results display mode**. Default value: Regular

Let us edit these parameters.

- For the **MaterialsAndServices** catalog, set the parameters as shown in fig. 19.15.

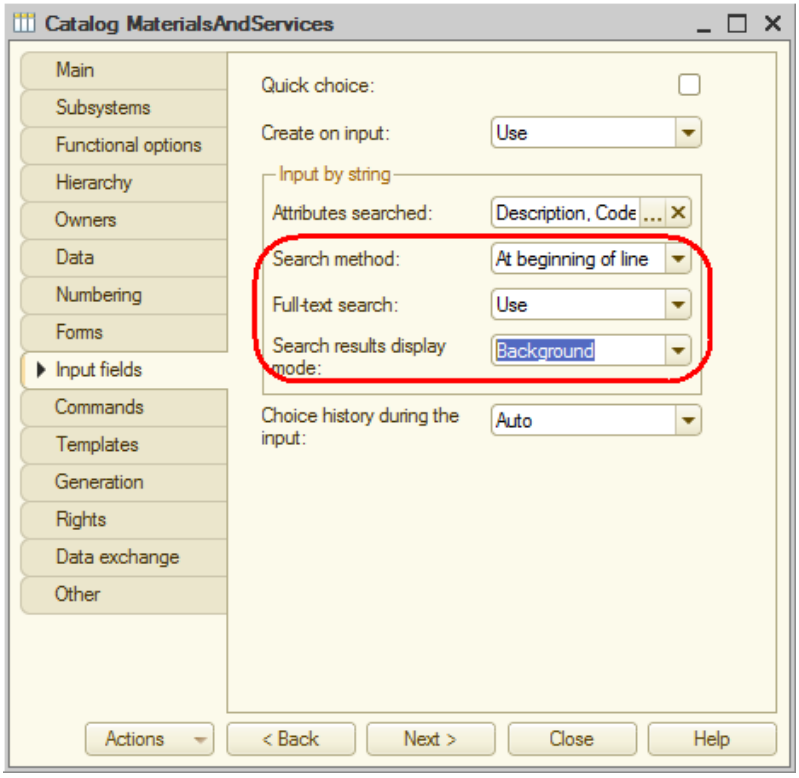


Fig. 19.15. Input by string parameters

So, when a user enters a value to a field that references a material or a service, the platform performs the full-text search in background mode (as a background job). Both the full-text search option and the background search option are especially helpful when your database contains a large amount of data.

If your database is small, using the full-text search is not necessary and we recommend searching anywhere in the line instead of searching at the beginning (set **Search method** to **Anywhere in the line**).

During the applied solution execution you can change the values of these parameters using 1C:Enterprise script tools, namely the AutoComplete and TextEditEnd client event handlers of text fields. You can also implement this in the manager module of the object whose data is to be searched, in the ChoiceDataGetProcessing event handler (except the search results display mode property).

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. Open the **Material and service prices** register and start the creation of a new price.

   Once you start typing in the Material or service field, which has the material or service reference type, the drop-down list displays the material or service items that begin with the typed characters (fig. 19.16).

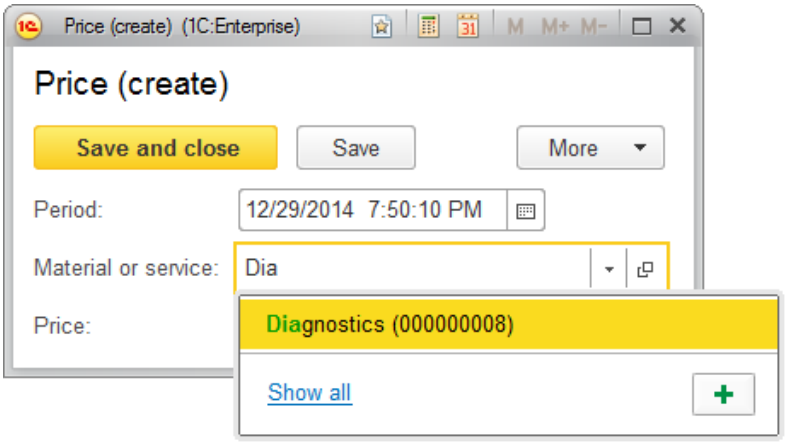


Fig. 19.16. Search by string that is being typed

According to the properties that you specified in Designer, this is performed as a full-text search in background mode.

Quiz

- What is the purpose of full-text search?
- How does one manage full-text search options interactively?
- How does one compose basic search expressions?
- How does one specify search parameters for the input by string feature?

Lesson 20 (0:35). Scheduled jobs

Any 1C:Enterprise infobase requires periodic execution of a specific set of scheduled operations.

For example, when you modify existing data or add new data, you need to create backups. If your infobase gets out of order due to a failure, you can restore the majority of data from a backup copy. So the more frequently you use backups, the less data you need to reenter if a failure occurs.

Another example: in order to be able to use full-text search in a database, you need to index the data. This means that the full-text index should be regularly updated. How frequently? It depends on the intensity of data additions and modifications. But it is obvious that some specific interval should be used.

To automate such operations, 1C:Enterprise offers the scheduled jobs feature. You can create jobs, each of them representing a sequence of actions described using 1C:Enterprise script. You can create a schedule for each job to have it executed automatically in compliance with the schedule.

In this lesson you will study the jobs using two scheduled operations related to the full-text search: full-text indexing and index merging.

You will describe these operations using 1C:Enterprise script, as well as create schedules for automatic execution.

The task at hand

In the previous lesson you learned that a full-text index is required to enable full-text search. A full-text index is only created once and should be updated regularly.

In reality, a full-text index includes two indexes: main and supplementary. When a full-text search is performed, both indexes are searched. The difference is as follows.

The main index is designed to ensure maximum search speed with large data volume. But the disadvantage is that data is added to the main index quite slowly.

The supplementary index is a direct opposite of the main index: data is added to the supplementary index fast but if the data volume in the supplementary index is significant, the search is performed relatively slowly.

The availability of the main and supplementary indexes suggests the following usage strategy. The majority of data is located in the main index, which enables quite a rapid search. New data (either modified or added) is added to the supplementary index in real time with a required periodicity (for example, once per hour or every minute). Data is added rapidly and does not influence the system performance significantly. While the supplementary index does not contain a large volume of data, search in this index is also performed rapidly.

In the periods of low user activity or when scheduled infobase operations are performed (often in the night), the main index is merged with the supplementary index (this can be done once a day). This operation might

generate noticeable load or take a long time (depending on the amount of data accumulated). As a result, new data is moved to the main index while the supplementary index is cleared and prepared for quick filling with new data.

To enable users to search in all the data without impacting the system performance, the supplementary index should be updated quite frequently (for example, once per hour or every minute). Besides, to keep the search rapid enough, the supplementary index should contain as little data as possible, hence the main index should be merged with the supplementary index regularly (for example, in the night during the minimum user activity period).

So to automate full-text indexing, you need two jobs. The first job will perform indexing without merging and will be executed every minute. The second job will merge indexes and will be executed once a day in the night.

Let us proceed to creating the jobs.

Understanding scheduled jobs

Scheduled jobs are located in the configuration object tree, in the Common branch. Each scheduled job includes two major properties: Method name and Schedule.

The Method name property connects the scheduled job to some procedure or function defined in a common module. When the job is started, the procedure or function is executed. It should contain a 1C:Enterprise script algorithm that describes the operations included in the job.

The Schedule property defines the execution periodicity.

In addition to the above properties, a scheduled job also has other properties, such as Repeat interval on failure and Restart count on failure. So if a scheduled job fails for some reason, 1C:Enterprise can run this job automatically for the specified number of times during the specified time period.

Creating scheduled jobs

In Designer mode

Let us create the first scheduled job, which updates the index.

1. In the **Common** branch of the configuration object tree, click **Scheduled jobs** and add a scheduled job object named *UpdateIndex* (fig. 20.1).

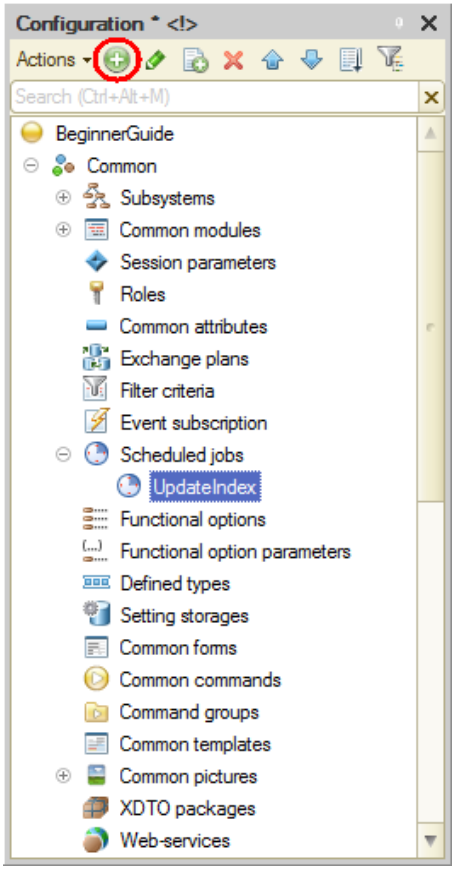


Fig. 20.1. Creating a scheduled job

Let us create a procedure that will update the full-text index of the infobase.

Any procedure or function of a nonglobal common module can serve this goal, provided that it can be called on the server (the common module should have the Server property set).

2. Add a common module named *ScheduleProcedures* to the configuration, and then, in the module properties, select the **Server call** check box.

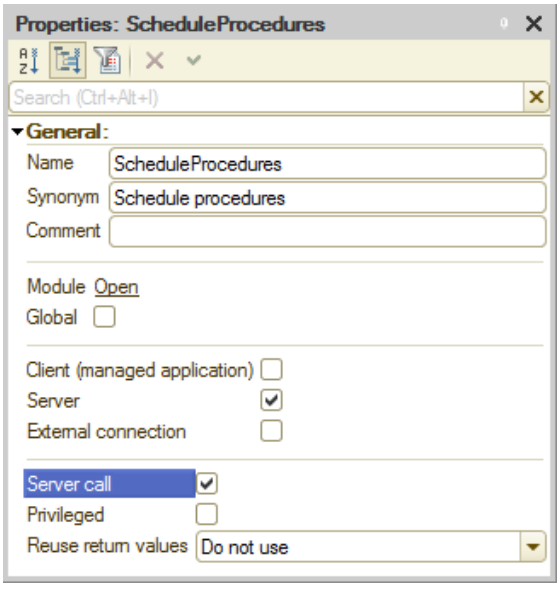


Fig. 20.2. Common module properties

This is required to make its export procedures and functions available for calling (fig. 20.2).

3. In the property palette of the **UpdateIndex** scheduled job, for the **Method name** property, click the **Open** button.

   This opens the common module selection window (fig. 20.3).

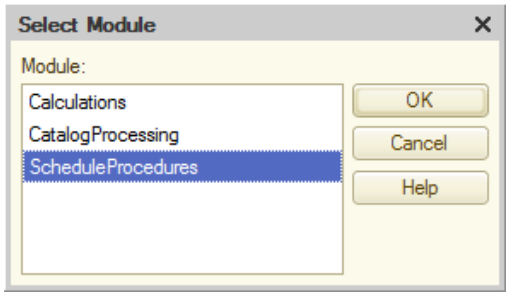


Fig. 20.3. Selecting an event handler

4. Select the **ScheduleProcedures** module.

   This creates the UpdateIndex() procedure template in the module.

5. Populate the template as shown in listing 20.1.

Listing 20.1. UpdateIndex() procedure

```
If FullTextSearch.GetFullTextSearchMode() = FullTextSearchMode.Enable
    Then

    If Not FullTextSearch.IndexTrue()Then

        FullTextSearch.UpdateIndex( , True);

    EndIf;

EndIf;
```

First this procedure checks whether full-text search operations are allowed (as they can be prohibited, for example, interactively, see fig. 19.4).

If full-text search operations are allowed, the procedure checks whether the full-text index is up-to-date (if after the last indexing the data to be included in the full-text indexing has not been changed, the index is still up-to-date and new indexing is not required).

If indexing is required, the procedure calls the UpdateIndex() method of the full-text search manager.

The first method parameter determines whether the indexes should be merged. Its default value is False, this means that the indexes are not merged.

The second parameter defines the volume of data to be indexed: either the entire data volume or batches. Your goal is to make the

indexing as fast as possible, so you specify that the indexing is performed in batches (the True value).

The batch size is fixed to 10 000 objects. So if you need to index 15 000 objects, the first method call indexes the first 10 000 objects (the first batch), while the second method call indexes the remaining objects (during the next run of the scheduled job).

Let us proceed to creating a schedule for the scheduled job execution.

6. In the scheduled job property palette, for the **Schedule** property, click the **Open** link.

   This opens the schedule editor (fig. 20.4).

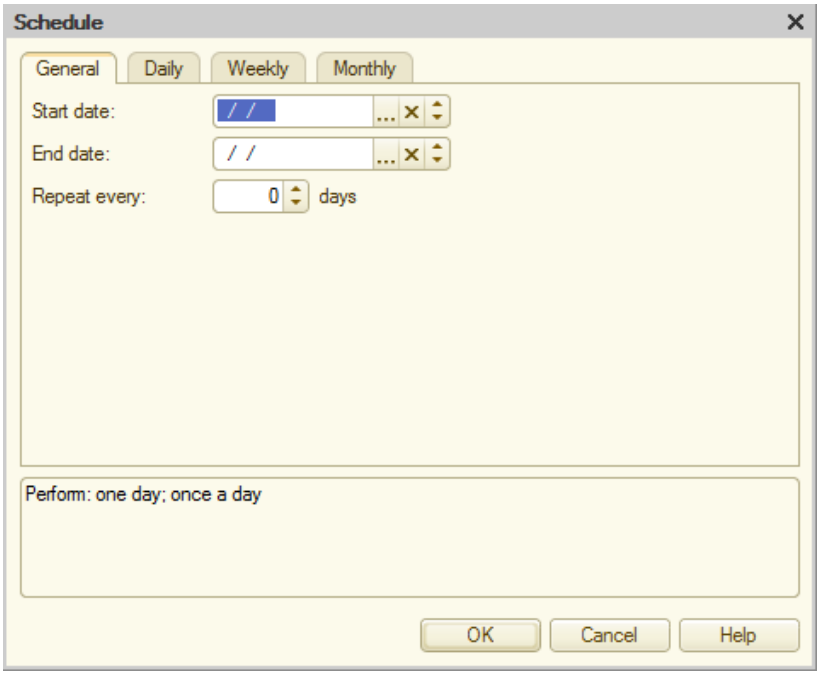


Fig. 20.4. Schedule editor

This dialog box includes multiple tabs to define various schedule types. The bottom of the dialog box displays the full schedule description.

Your goal is to set up the following schedule: daily, every minute.

7. On the **General** tab, specify that the job is repeated daily (fig. 20.5).

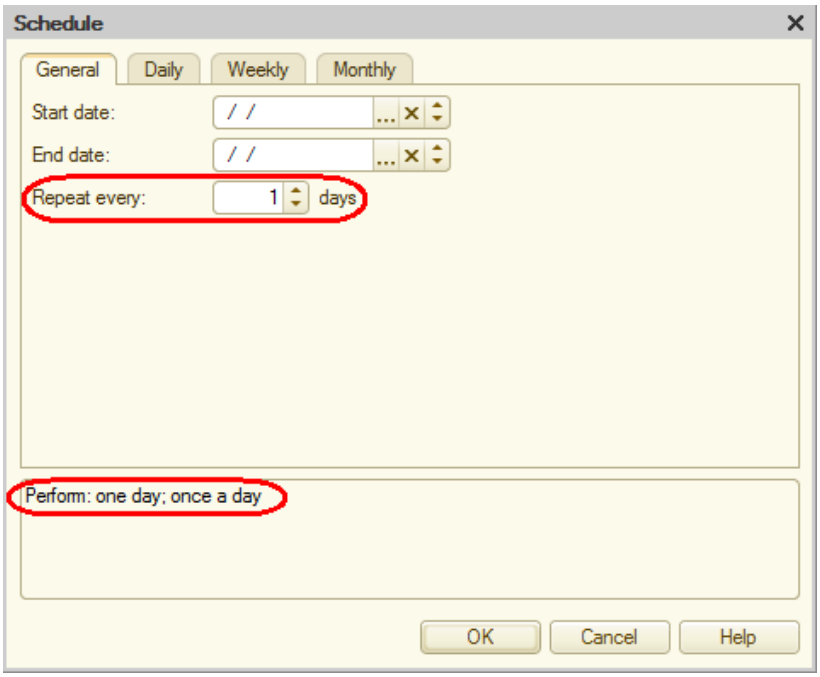


Fig. 20.5. Job schedule: daily

8. On the **Daily** tab, specify that the job is executed every 60 seconds (fig. 20.6).

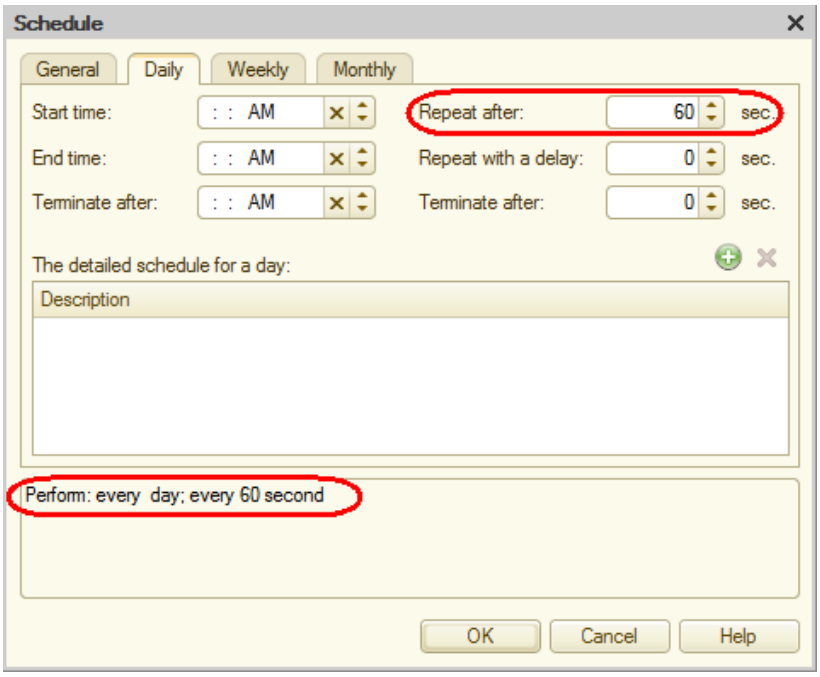


Fig. 20.6. Job schedule: every minute

The bottom of the dialog box displays the description of the created schedule: "Perform: every day; every 60 second".

It seems that you have obtained the result you needed: the scheduled job is executed daily, every minute. But Jack of All Trades does not work 24/7, so executing the job at night is obviously useless because database data is not modified. But some employees might stay late after their workday is over.

9. In the **Start time** field, enter *08:00* (fig. 20.7).

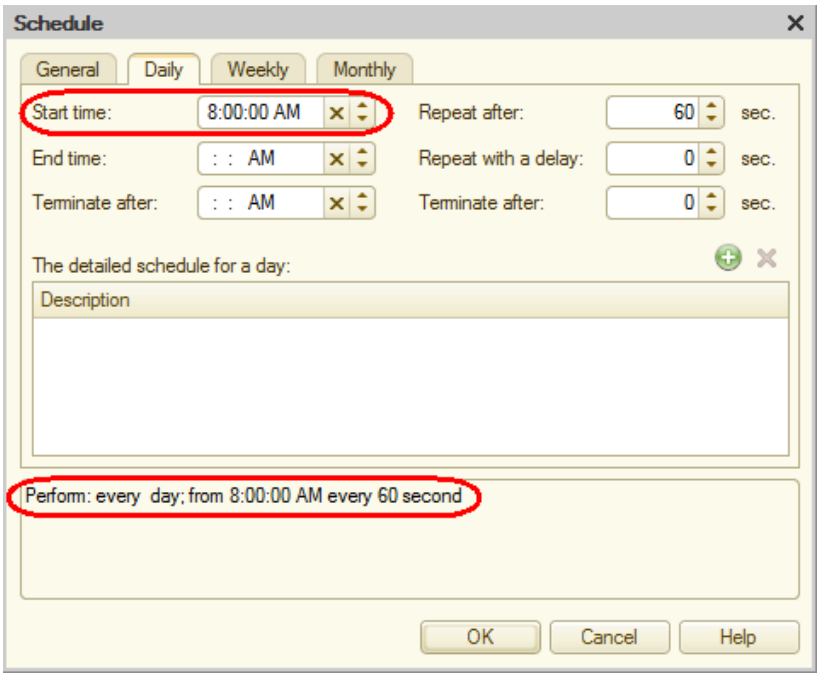


Fig. 20.7. Specifying the execution start time

This specifies that the job is executed starting from 8 a.m. Since no end time is specified, the job execution is stopped when the current day is over. In other words, the job is executed all day except the interval between 00:00 and 08:00.

Now you are done creating a schedule for the scheduled job.

10. Click **OK**.
11. In the scheduled job property palette, select the **Predefined** check box (fig. 20.8).

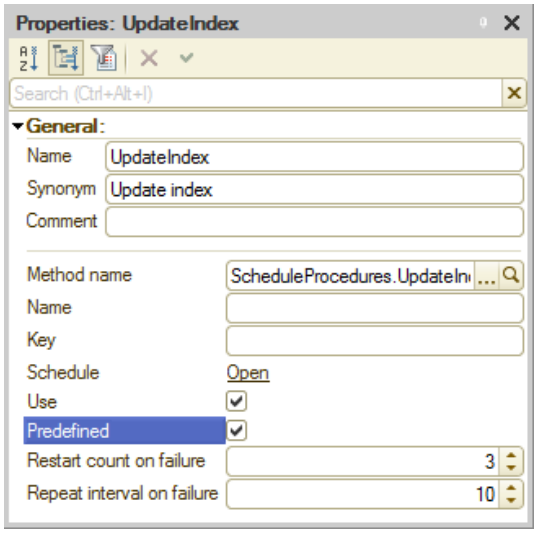


Fig. 20.8. Predefined scheduled job

This means that once the applied solution is started in 1C:Enterprise mode, the platform creates a predefined scheduled job. Otherwise you would have to create the job using 1C:Enterprise script tools.

This completes the creation of the UpdateIndex scheduled job.

Let us use the same steps to create another scheduled job.

12. Create a scheduled job named *MergeIndexes* and then, in its property palette, for the **Method name** property, click the **Open** button.
13. Select the **ScheduleProcedures** module.

    This creates the MergeIndexes() procedure template in the module.
14. Populate the template as shown in listing 20.2.

    Listing 20.2. MergeIndexes() procedure

```
If FullTextSearch.GetFullTextSearchMode() = FullTextSearchMode.Enable
    Then

    If Not FullTextSearch.IndexTrue()Then

        FullTextSearch.UpdateIndex(True);

    EndIf;

EndIf;
```

    This procedure is similar to the one shown in listing 20.1, with the following differences: the indexes are merged during the update (the first parameter is True) and all available data is indexed as a single batch (the default value of the second parameter is False) because in this case indexing time is not vital.
15. In the job properties, select the **Predefined** check box.

    Then let us proceed to editing the schedule.
16. In the scheduled job property palette, for the **Schedule** property, click the **Open** link.
17. On the **General** tab, specify that the job is repeated daily (repeat every: 1 days).
18. On the **Daily** tab, specify the start time 1:00 for the job execution (fig. 20.9).

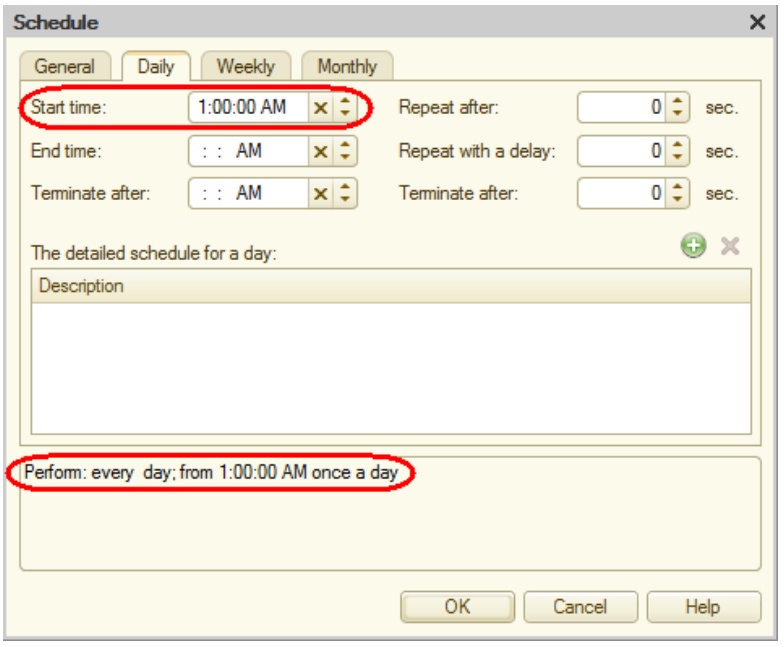


Fig. 20.9. Schedule of the MergeIndex job

This gives you the following schedule: "Perform: every day; from 1:00:00 AM once a day".

You are done creating a schedule for the MergeIndex scheduled job.

19. Click **OK**.

Running scheduled jobs

The scheduled job execution does not require any additional developer actions.

As we explained in the theoretical lesson on page 185, 1C:Enterprise supports two operation modes: file and client/server.

Your tutorial infobase runs in the file mode. When you start the client application, the platform starts another thread that establishes connection to the database and performs sequential scheduled job executions initiated by the client.

Infobases that run in the client/server mode also do not require additional developer actions for automatic scheduled job execution. After updating the database configuration the 1C:Enterprise server cluster manager starts the job execution according to the specified schedule.

In 1C:Enterprise mode

Let us test the scheduled jobs.

1. Start 1C:Enterprise in the debug mode.
2. Change the database state (for example, you can add a material or service and then delete it), and wait for a couple of minutes.

3. On the main menu, point to **All functions**, expand **Standard**, and double-click **Manage Full Text Search**.

   The index creation date and time should match the time of your last database operation plus 1 minute (fig. 20.10).

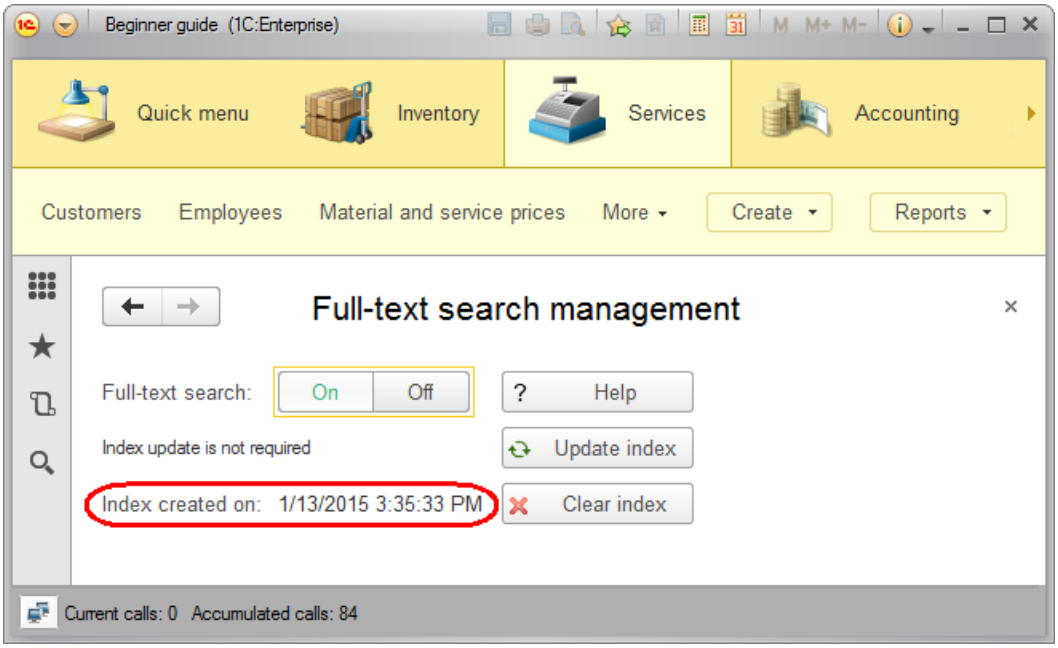


Fig. 20.10. Changing the full-text index date

So you see that the index update job is executed every 60 seconds as specified in the schedule. To ensure that both scheduled jobs are executed, you should have at least one applied solution instance running for a 24-hour period.

Quiz

- What is the main index? What is the supplementary index?
- What strategy is used in the full-text indexing of an infobase?
- What is the purpose of scheduled jobs?
- How does one create a schedule for automatic job execution?

Lesson 21 (0:40). Editing register records in document forms

Your infobase, just like any other infobase, requires the way to enter opening balances for a register. This is needed to provide users with the option to begin their work with the infobase not from scratch but instead from some initial state taken from their previous accounting system (even if that was kept by pen and ink).

The task of entering opening balances is different from other algorithms that modify your infobase registers because it includes making direct changes to the registers, without using any intermediate algorithms (filling document data, posting documents, checking document data for errors, and so on).

Let us review an example of entering opening balances for the BalanceOfMaterials accumulation register.

To perform this task, you will create a document whose records to the BalanceOfMaterials register can be edited manually, within the document form.

In Designer mode

Let us create the document that directly modifies register records.

1. Create a document configuration object named *InputOpeningMaterialBalances*.
2. On the **Posting** tab, disable the document posting.

   You need this because you will develop a custom algorithm for writing register records.
3. Specify that the document records are located in the **BalanceOfMaterials** accumulation register (fig. 21.1)

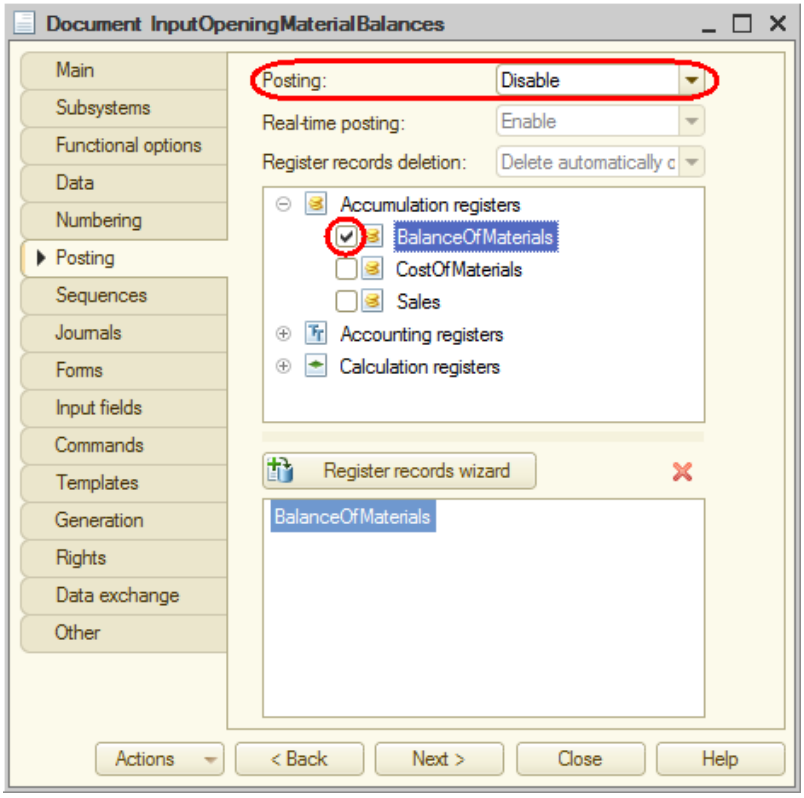


Fig. 21.1. Disabling document posting

4. On the **Forms** tab, create the default document form.
5. In the form editor, on the **Attributes** tab, expand the main form attribute **Object**, expand the **RegisterRecords** collection, locate the **BalanceOfMaterials** item, and drag it to the form controls pane.
6. When prompted to answer the "Add columns to the table?" question, click **Yes** (fig. 21.2).

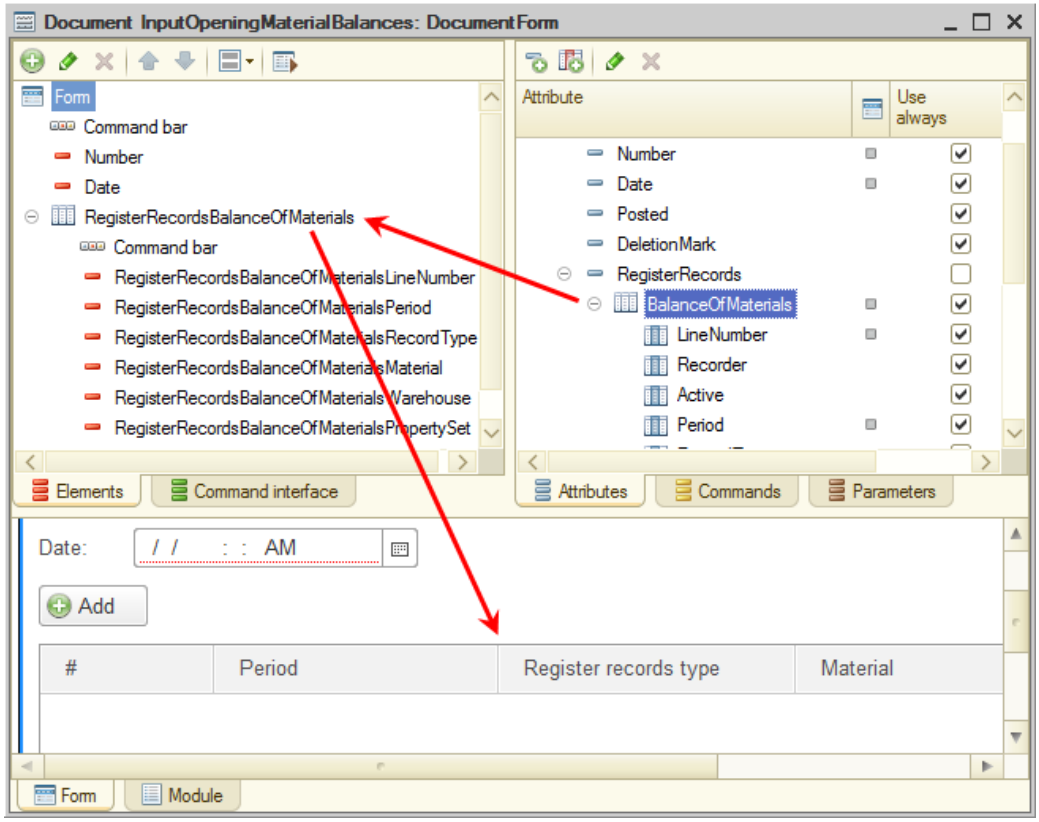


Fig. 21.2. Editing the Input opening material balances document form

Note that in the property palette of this table, in the Data property, a link to the data of the Object.RegisterRecords.BalanceOfMaterials register record set is generated.

Let us slightly modify the form appearance.

7. In the form controls pane, add a group of the **Regular group (no visual presentation)** type with **Horizontal** grouping and drag the **Number** and **Date** document properties to this group.
8. Swap the positions of the **PropertySet** and **Warehouse** table fields (fig. 21.3).

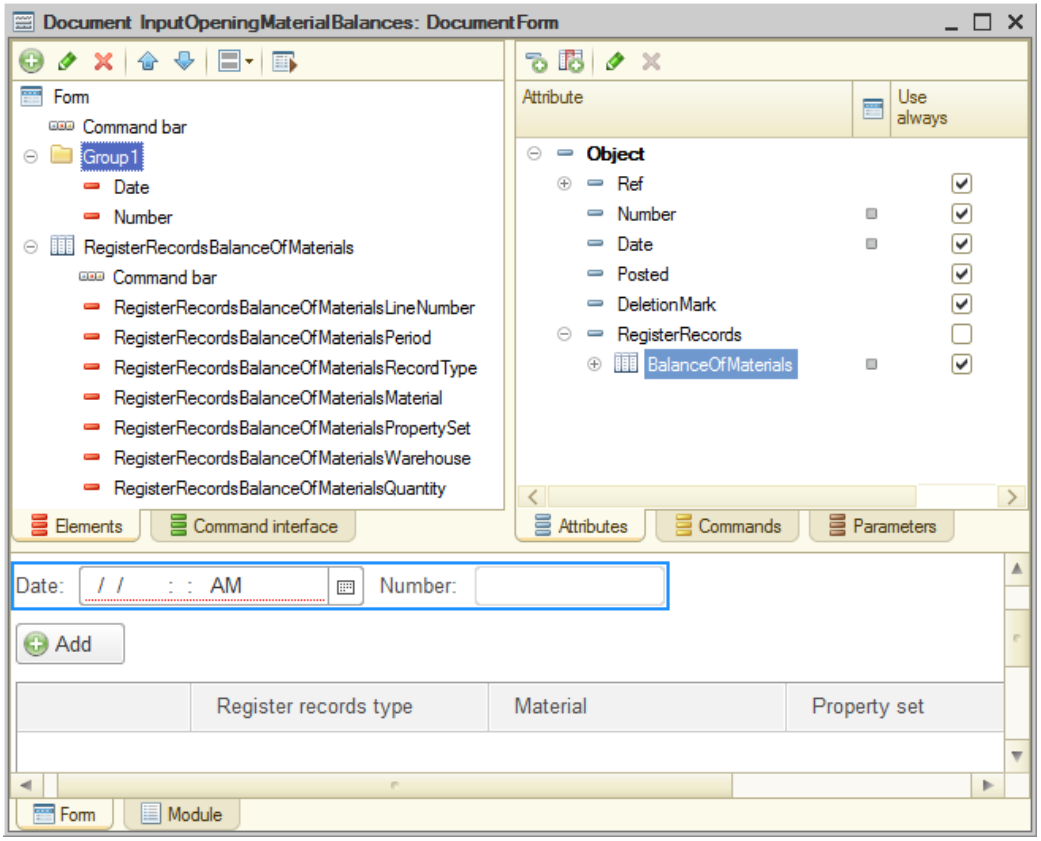


Fig. 21.3. Modified form of the Input opening material balances document

Let us edit the command interface of the document form to add the command that opens the list of BalanceOfMaterials register records related to the document to the navigation panel.

9. In the upper left pane of the form editor, click the **Command interface** tab.
10. In the **Navigation panel** group, in the **Go to** subgroup, enable visibility for the command that opens the **Balance of materials** register (fig. 21.4).

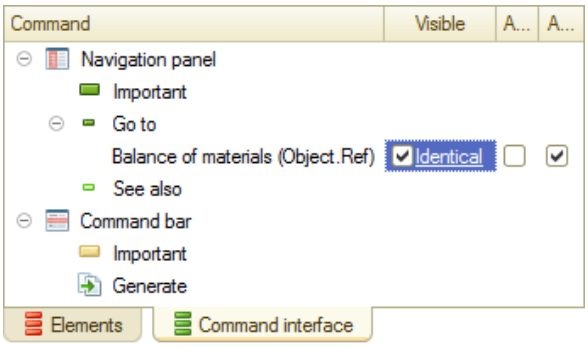


Fig. 21.4. Editing form command interface

11. In the editor of the **InputOpeningMaterialBalances** document configuration object, on the **Subsystems** tab, specify that the object belongs to the **Accounting** subsystem.

    Finally, let us edit the command interface of this subsystem.

12. In the configuration tree, right-click the **Accounting** subsystem and then click **Open command interface**.
13. In the **Actions panel.Create** group, enable visibility for the **Input opening product balances: create** command.

In 1C:Enterprise mode

Let us check the document.

1. Start 1C:Enterprise in the debug mode.
2. In the **Accounting** section, run the **Input opening material balances** command.
3. Fill the document with data as shown in fig. 21.5.

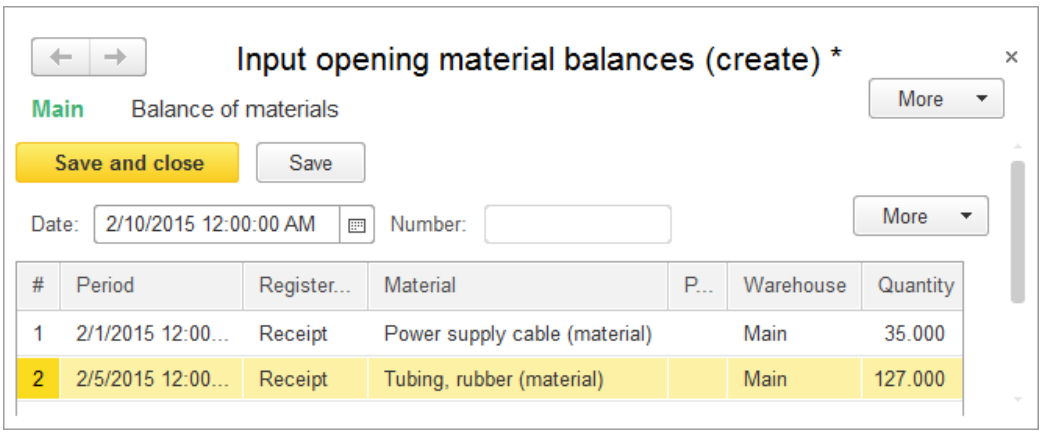


Fig. 21.5. Creating the Input opening material balances #1 document

Note that the document date does not match the dates of the individual register records listed in the document.

4. Click **Save**.
5. In the document form navigation panel, switch to the BalanceOfMaterials register records created by the document (fig. 21.6).

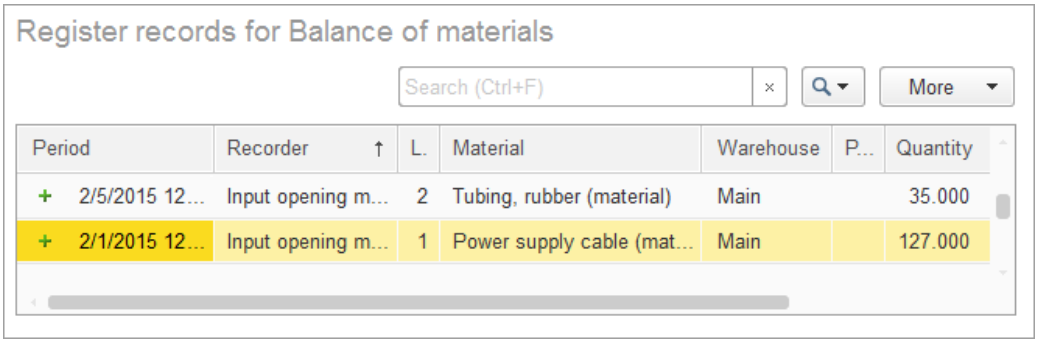


Fig. 21.6. Records of the BalanceOfMaterials register

So you have accomplished your purpose: on the one hand, by providing a document date, you can record the moment when the register records are changed, while on the other hand you can assign a custom value of the Period field to each of the register records.

Now let us apply some stricter requirements to how your document generates register records, and take a look at two typical scenarios.

Editing register records from 1C:Enterprise script

Writing register records from a form

In Designer mode

The first requirement that you will implement is that register records must be generated with the same date as the date of the document. In other words, let us synchronize the register records date with the document date.

- Create the BeforeWrite client event handler for the document form and add the following script to this handler (listing 21.1).

Listing 21.1. BeforeWrite event handler of the document form

```
For Each RegisterRecord In Object.RegisterRecords.BalanceOfMaterials
    Do

    RegisterRecord.Period = Object.Date;

EndDo;
```

In 1C:Enterprise mode

Let us check the document.

1. Start 1C:Enterprise in the debug mode.
2. Open the document and click **Save**.
3. Open the records created by the document in the BalanceOfMaterials register.

   You can see that the value of the Period field of all the records is now equal to the document date (fig. 21.7).

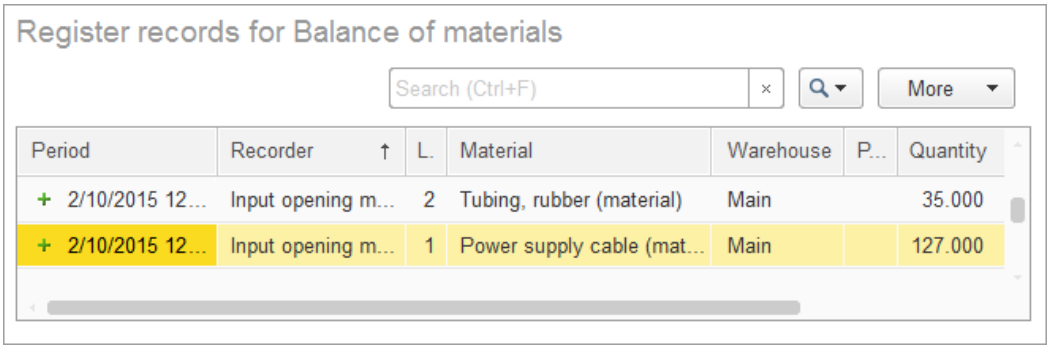


Fig. 21.7. Modified records of the BalanceOfMaterials register

So you achieved the goal, but only for scenarios where the document is saved interactively.

Writing register records from 1C:Enterprise script

If you run the Write() method of the document object, the document is written without involving the document form. This means that the BeforeWrite event of the document form is never called and so the handler script is never executed.

To synchronize the period of document register records with the document date in the scenario where the document is written using 1C:Enterprise script tools, you have to use the BeforeWrite event handler of the document object instead of the BeforeWrite handler of the document form.

During interactive document writing the BeforeWrite event of the document form is called first, and then the BeforeWrite event of the document object is called (see the chart of events in section "Event sequence for writing documents from document forms" on page 847).

In Designer mode

Let us implement the event handler in the InputOpeningMaterialBalances document object module.

1. Return to Designer and comment out the handler script in the form module.
2. In the **InputOpeningMaterialBalances** document object editor, on the **Other** tab, open the object module and add the script shown in listing 21.2.

Listing 21.2. BeforeWrite event handler of the object module

```
Procedure BeforeWrite(Cancel, WriteMode, PostingMode)

    // Determining whether updating register record dates is required
    UpdateRegisterRecordsDate = IsNew() Or
        RegisterRecords.BalanceOfMaterials.Modified();
    If Not UpdateRegisterRecordsDate Then

        // Verifying that the date changed
        Query = New Query;
        Query.SetParameter("CurDocument", Ref);
        Query.Text =
            "SELECT
            |    Date
            |FROM
            |    Document.InputOpeningMaterialBalances
            |WHERE
            |    Ref = &CurDocument";

        Selection = Query.Execute().Select();
        Selection.Next();
```

```
        UpdateRegisterRecordsDate = Selection.Date <> Date;

    EndIf;

    // Assigning the new date to all records, if required
    If UpdateRegisterRecordsDate Then

        If Not RegisterRecords.BalanceOfMaterials.Selected() And Not
            RegisterRecords.BalanceOfMaterials.Modified() Then

            RegisterRecords.BalanceOfMaterials.Read();

        EndIf;

        For Each RegisterRecord In RegisterRecords.BalanceOfMaterials
            Do
            RegisterRecord.Period = Date;
        EndDo;

    EndIf;

EndProcedure
```

As you can see, in this case the handler contains more lines, due to the additional checks that are performed because both interactive and script-based writing of the object is possible.

Let us walk through the handler contents in greater detail. If a new document is written or its register records are changed, you need to update the register record dates. Otherwise the query reads the document date from the database and then the date is compared to the date of the object being written. If the dates do not match, you also need to update the register record dates.

Before a date is set, the script checks whether a record set was read from the RegisterRecords property of the object and whether it has been modified. If both conditions are false, it means that the record set in the RegisterRecords property of the object is empty and that this condition is not due to its modification. In this case, to prevent erroneous deletion of register records (overwriting them with an empty record set), the script preliminary reads the register records from the register into the RegisterRecords property.

Then, as in the previous scenario (writing register records from a form), the required date is assigned to all the records in that set. When the document object is recorded, this record set is written to the accumulation register.

In 1C:Enterprise mode

Let us check the new event handler.

1. Start 1C:Enterprise in the debug mode.
2. Set a new date for the document (for example, 02/03/2015), save it, and ensure that the records in the accumulation register have the new date as well.

While saving a document, you can control not only the period of the accumulation register records, but also the values of other register fields.

For instance, by applying the same principle you can create an AccountingOperation document for entering manual operations into the accounting register.

In that case, in addition to controlling the period of register records, you need to manage the values in the Active field (enabling or disabling document posting), and so on.

Choosing handler locations

We recommend that you choose the handler to store your custom algorithm depending on the object logic. If your configuration does not include the option to save the object using 1C:Enterprise script tools, you can choose the form module handler. But if your configuration includes 1C:Enterprise script algorithms that save the object, choose the object module handler.

Note that neither of these methods prevents modification of register records using the <...>Register.RecordSet.<register name> object. Therefore, if the configuration logic suggests possibility of script-based modifications of a RecordSet object, the handler code should be located in the event handler of this record set. All attempts to modify register data ultimately come down to writing a record set.

Quiz

- What is the purpose of the input opening balance document? How does one create such document?
- How does one modify a register value using 1C:Enterprise script tools when entering opening balances?
- When should one use a form module and when should one use an object module to host event handlers?

Lesson 22 (1:50). Users and their roles

Now that you have created all the major configuration objects, you can proceed to defining user roles. Administration of a 1C:Enterprise user list and assigning user roles according to job responsibilities is an important part of applied solution interface design, it is how you specify access rights for individual users and actions that are available to them. This is the topic of this lesson.

Until now any user had full access to all sections of your applied solution and to all the configuration objects and commands used in these sections. However, in real life restricting user access to some data stored in the infobase is an important requirement.

For example, managers obviously need to have full access to all the data in the database, while stock clerks, by contrast, only need access to the data related to product receipts and expenses in warehouses without being able to access accounting or human resources data.

Additionally, you need a way to restrict the actions that users can perform on database objects. For example, stock clerks can create and modify receipts since it is their responsibility to account for materials within the company. Technicians may need to view the goods receipts in order to know which products have been received and when. However, technicians do not need the ability to make any changes to those receipts.

Understanding roles

A configuration object named Role is intended for defining user rights. Using this object, a developer can describe a set of rights to execute some actions on specific database objects and on the configuration as a whole.

As a rule, a role is created separately for each type of activity, and each user is assigned one or more roles.

When a user is assigned multiple roles, access is granted or denied based on the following rules:

- If any of the roles has the permission, access is granted
- If none of the roles have the permission, access is denied

Creating roles

In Designer mode

When creating roles, one generally starts out by deciding what kind of data access various user groups need. To do so, let us assign roles by subsystems, which will make the task a lot easier.

Administrator

First, let us create the Administrator role. This role should include full rights for operations with infobase data.

1. Expand the **Common** branch of the configuration object tree, click **Roles**, and add a new Role configuration object named *Administrator* (fig. 22.1).

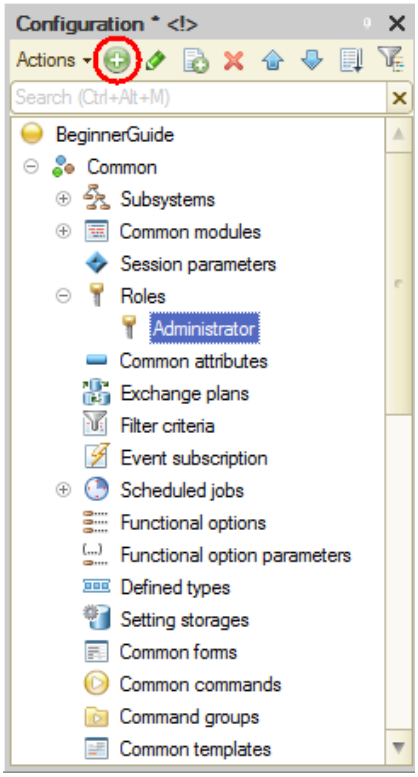


Fig. 22.1. Creating a role

This opens the rights editor for the role (fig. 22.2).

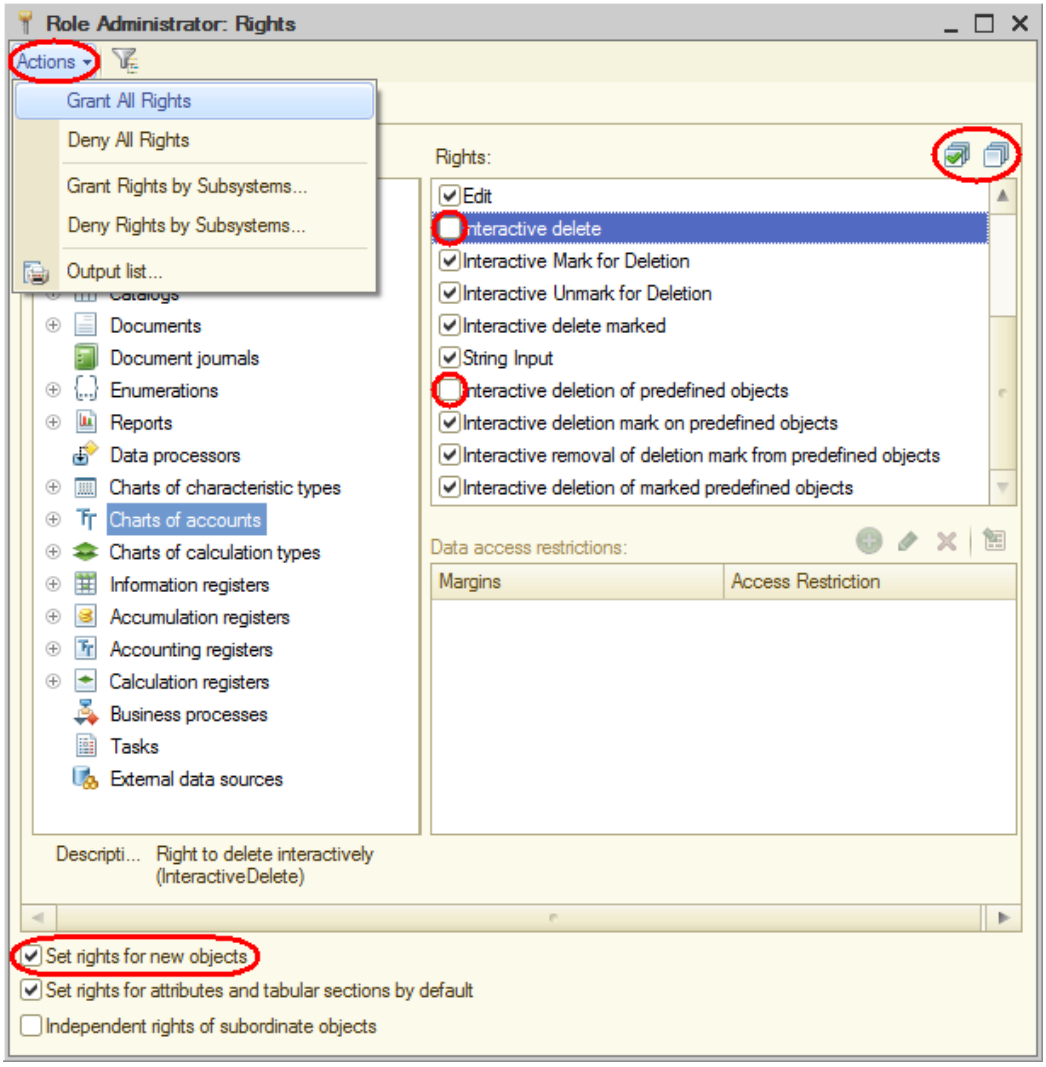


Fig. 22.2. Rights editor window for the Administrator role

The left pane contains the list of configuration objects, grouped by type. The right pane contains the list of rights available for the selected object or object type.

An administrator should have rights for all the objects and all the object types.

2. On the **Actions** menu, click **Grant All Rights**.

   This selects all rights for all objects.

   You can grant or deny all rights for a specific configuration object using the **Mark all elements** and **Remove marks from all elements** buttons above the list of rights.

   The only thing you have to do is deny the interactive deletion right for all the objects. This is required to prevent an administrator from accidental deletion of database objects.

3. Scroll through all the configuration object types (Catalogs, Documents, and so on) and clear the **Interactive delete** check box for each object. Note that this also clears the **Interactive deletion of predefined objects** check box (see fig. 22.2).

4. Select the **Set rights for new objects check box** (see fig. 22.2).

This grants the administrator full access rights for the objects that you create after setting the rights.

You are done with the creation of the Administrator role.

CEO

Next, let us create the CEO role.

1. Create a new Role configuration object named *CEO*.

   It is fine with us that the new role does not have access rights for all the objects except for the configuration object types that do not have any objects created. For such configuration objects, full rights remain in effect.

2. Ensure that the **Output** right is set for the root configuration object.

3. Scroll through all the configuration object types and grant the **View** right for each object.

   This also automatically sets the **Read** and **Use** rights.

4. Expand the **Common** branch and then, in the **Subsystems** branch, grant the **View** right for all the subsystems.

   This grants CEO the right to view all infobase data. Later you will use the command visibility by role settings to hide all the actions that are not related to the configuration applied logic from CEO's interface (fig. 22.3).

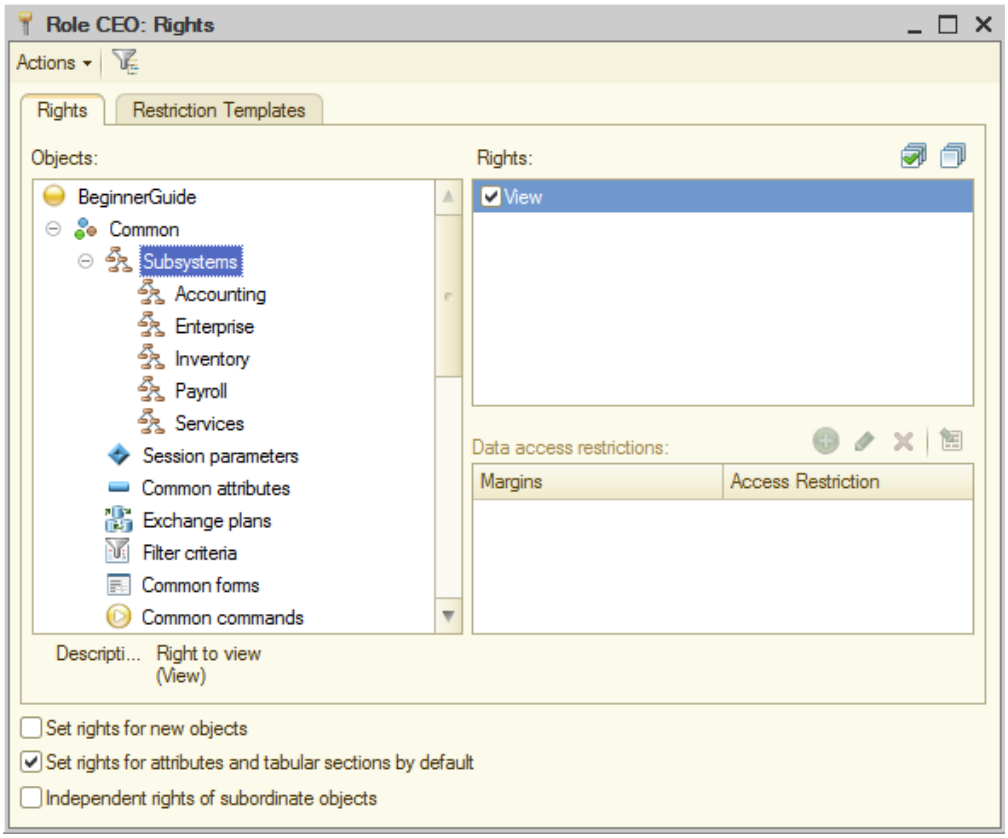


Fig. 22.3. Rights editor window for the CEO role

This completes the creation of the second role.

Technician

Next, let us create the Technician role.

1. Create a new Role configuration object named *Technician*.
2. On the **Actions** menu, click **Grant Rights By Subsystems**, select the **Inventory and Services** subsystems, and click **Set**.

   This grants all the rights for configuration objects that belong to these subsystems.
3. Filter the list of objects by the **Inventory** and **Services** subsystems (fig. 22.4).

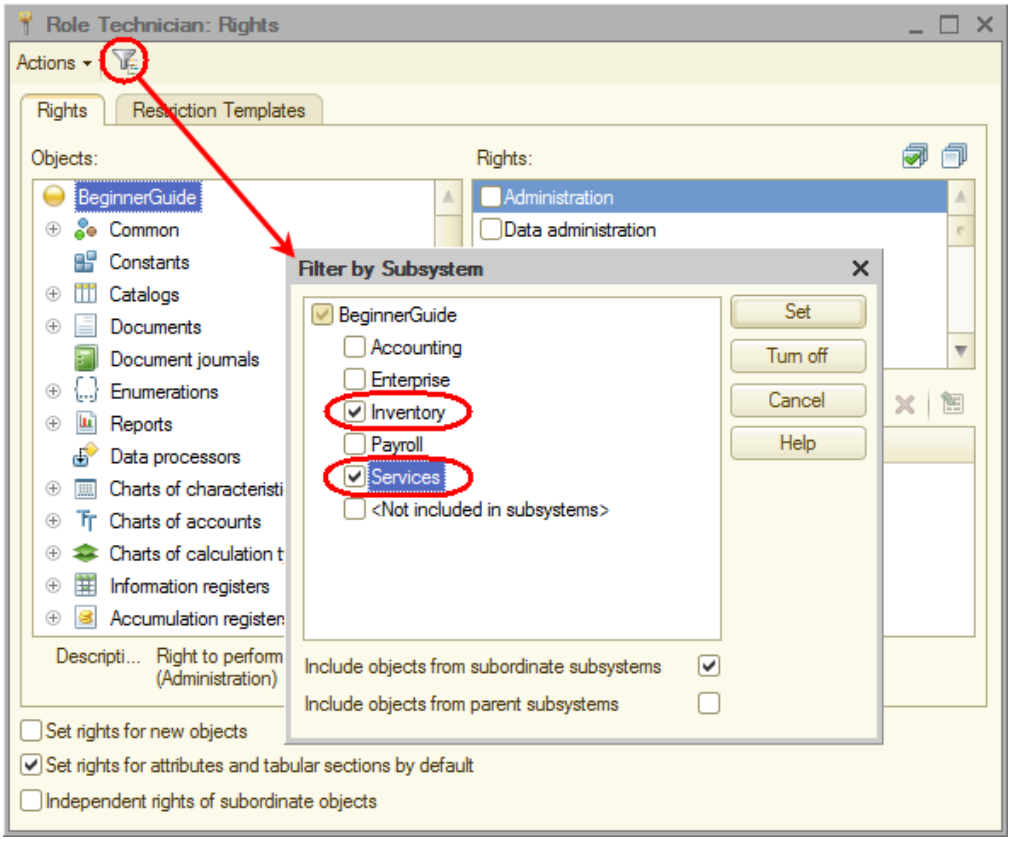


Fig. 22.4. Filtering by subsystem

   This helps you to fine tune the rights for objects belonging to these subsystems.
4. For the **Employees** catalog, deny the **Insert**, **Edit**, and **Delete** rights.

   Note that denying the Insert right removes the check mark from the Interactive insert right because it is a subset of the Insert right. In the same manner, subsets of rights are denied when you deny the Edit and Delete rights.
5. Scroll through all the configuration object types and clear the **Interactive delete** check box for each object.

6. Remove the filter and grant all the rights except **Interactive delete** for the following configuration objects:
   - **MaterialOptions** catalog
   - **AdditionalMaterialProperties** catalog
   - **MaterialProperties** chart of characteristic types
   - **MaterialPropertyValues** information register

   These objects are not included in any subsystems but technicians need them to work with material characteristics.
7. Expand the **Common** branch, expand the **Subsystems** branch, click **Enterprise**, and grant the **View** right.

   This grants access to reference data stored in this subsystem. Later you will hide the functionality that technicians do not need using command visibility by roles.

   This completes the creation of the Technician role.

Payroll accountant

All you have left to do is create two roles: Accountant and Payroll accountant.

We are going to divide the rights for payroll calculation and for accounting. There is an accountant at Jack of All Trades, and an assistant accountant as well. The assistant usually performs payroll calculation, but sometimes the chief accountant does that. Therefore the chief accountant needs to have both roles, while the assistant only needs the Payroll accountant role.

1. Create a new Role configuration object named *PayrollAccountant*.
2. In the rights editor, grant the right for the **Payroll** subsystem (and remember to deny **Interactive delete**).
3. For the **Sales** accumulation register and for the **Customers** catalog, grant the **View** right.
4. For the **Enterprise** subsystem, grant the **View** right.

   This completes the creation of the Payroll accountant role.

Accountant

Finally, let us create the Accountant role.

1. Create a Role configuration object named *Accountant*.
2. In the rights editor, grant the right for the **Accounting** subsystem.
3. Filter the list of objects by that subsystem and deny the **Insert**, **Edit**, and **Delete** rights for the **MaterialsAndServices** catalog.
4. Deny the **Interactive delete** right for all objects.
5. Remove the filter and grant all the rights except **Interactive delete** for the **ExtraDimensions** catalog.

6. Grant the **View** right for the following configuration objects:
   - **Warehouses** catalog
   - **MaterialOptions** catalog
   - **AdditionalMaterialProperties** catalog
   - **MaterialProperties** chart of characteristic types
   - **MaterialPropertyValues** information register
7. Grant the **View** right for the **Enterprise** subsystem.

Rights to run client applications

Finally, grant the rights to run client applications (the thin client and the web client) for each role.

For this task you will use another tool that better suits your goal: the All Roles editor.

1. In the configuration object tree, right-click the **Roles** branch and click **All roles**.

   You can see that the Thin client and Web client rights are granted to all roles (fig. 22.5).

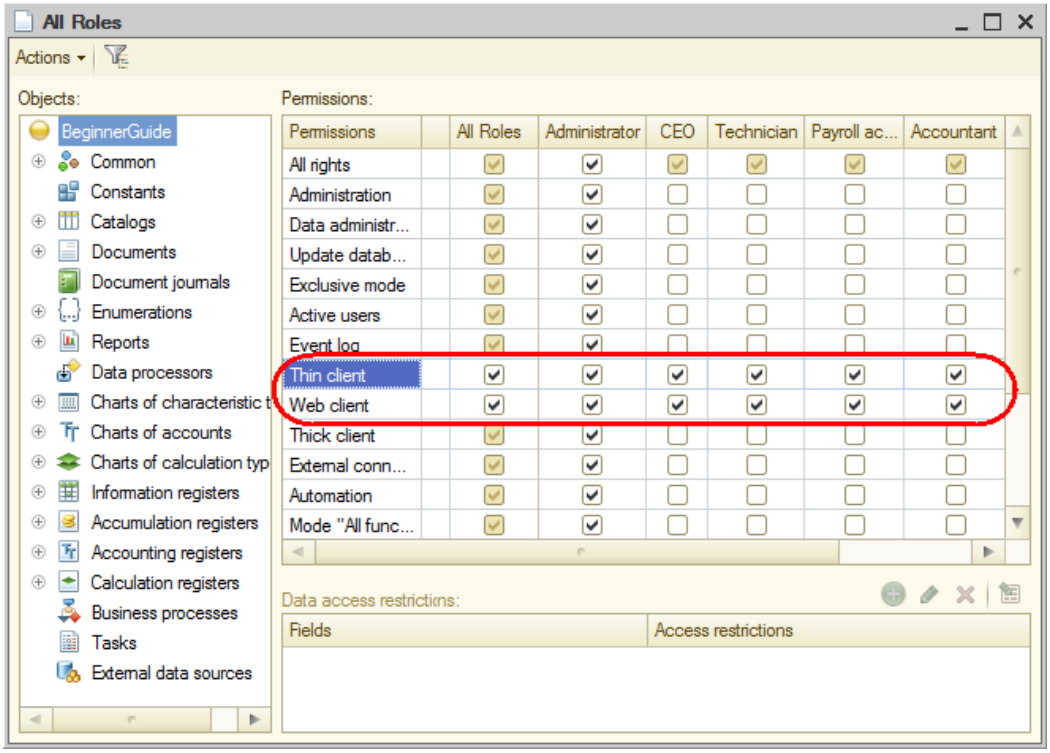


Fig. 22.5. All Roles editor

Administrators also have the right to connect to the infobase using other client application types.

To view the list of rights for each role

- On the **Actions** menu, click **Output list** (fig. 22.6).

New1

	1	2	3
1	Objects	Rights	Accountant
2	Configuration.BeginnerGuide		
3		All rights	
4		Administration	No
5		Data administration	No
6		Update database configuration	No
7		Exclusive mode	No
8		Active users	No
9		Event log	No
10		Thin client	Yes
11		Web client	Yes
12		Thick client	No
13		External connection	No
14		Automation	No
15		Mode "All functions"	No
16		Saving user data	Yes
17		Interactive open external data processors	No
18		Interactive open external reports	No
19		Output	Yes
20	Common		
21	Subsystems		
22		View	No
23	Subsystem.Accounting		
24		View	Yes
25	Subsystem.Enterprise		
26		View	Yes
27	Subsystem.Inventory		
28		View	No
29	Subsystem.Payroll		
30		View	No
31	Subsystem.Services		
32		View	No
33	Session parameters		
34		All rights	No
35		Get	No
36		Set	No
37	Common attributes		
38		All rights	No
39		View	No
40		Edit	No

Fig. 22.6. List of rights for the Accountant role

You can also view the list of rights for all available roles in the All Roles editor.

Adding users

To be able to tell one user from another when they work with the infobase, 1C:Enterprise features a user list. You can create and delete users, assign roles to users, and so on.

In Designer mode

Before you proceed to creating users, you first need to update the database configuration.

1. On the **Configuration** menu, click **Update database configuration**.

   -OR-

   Click the **Update database configuration** button on the toolbar.

You need this because a user can only be assigned the roles that exist in the database configuration.

2. On the **Administration** menu, click **Users**.

   This opens the list of users available in the system. This list is empty so far. Let us add a user.

3. On the **Actions** menu, click **Add**.

   -OR-

   Click the **Add** button (fig. 22.7).

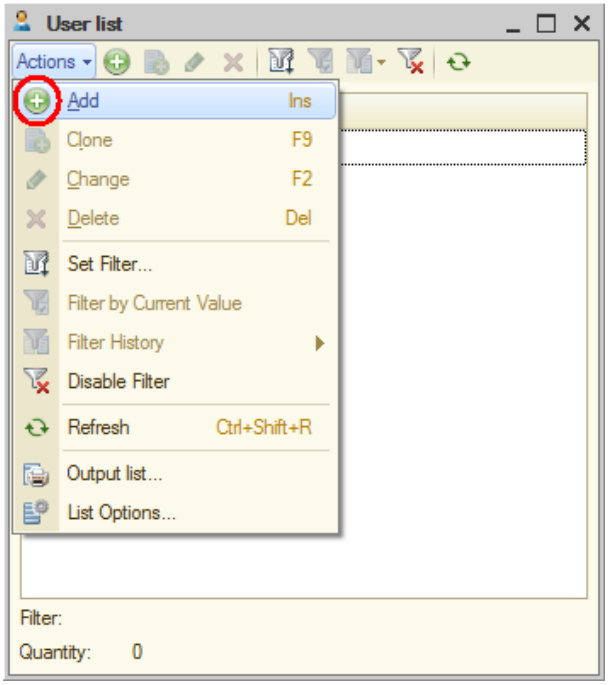


Fig. 22.7. User list

This opens the user editor window (fig. 22.8).

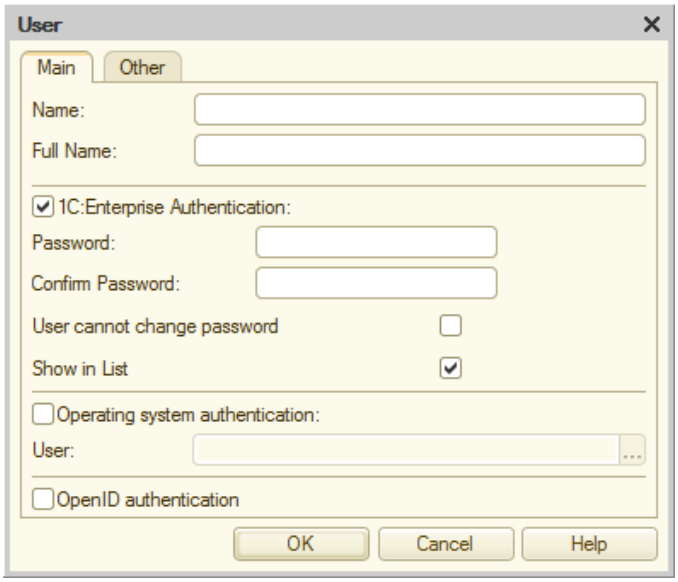


Fig. 22.8. User editor window

Attention! If you use the training version of the 1C:Enterprise platform, you cannot specify user passwords or use OS authentication. These are the limitations of the training version.

Name (the user name) is an identifier that appears in the user selection list when the applied solution is started in 1C:Enterprise mode.

Full Name is a string that can be used within the configuration for displaying various reference data. It is generally recommended that a full name includes the first name, middle name, and last name of a user (without abbreviations).

The next two panes of the window contain authentication settings.

1C:Enterprise Authentication means that during the applied solution startup a user is prompted to select a user name and enter a password. If the provided password matches the one stored in the system for that user ID, the applied solution is started with the rights that have been set for that user. After logging on the user can change their password, unless the **User cannot change password** check box is selected.

Operating system authentication means that during the applied solution startup no additional information is required from the user. 1C:Enterprise determines the operating system account of the user and then searches its own list of users. If a user that corresponds to the current operating system user is found in the list, the infobase is opened with the rights specified for that user.

Let us proceed to creating users.

4. In the **Name** field, enter *Administrator*.
5. In the **Full Name** field, enter *Administrator*.
6. On the **Other** tab, select the **Administrator** role and select **English** for the configuration language (fig. 22.9).

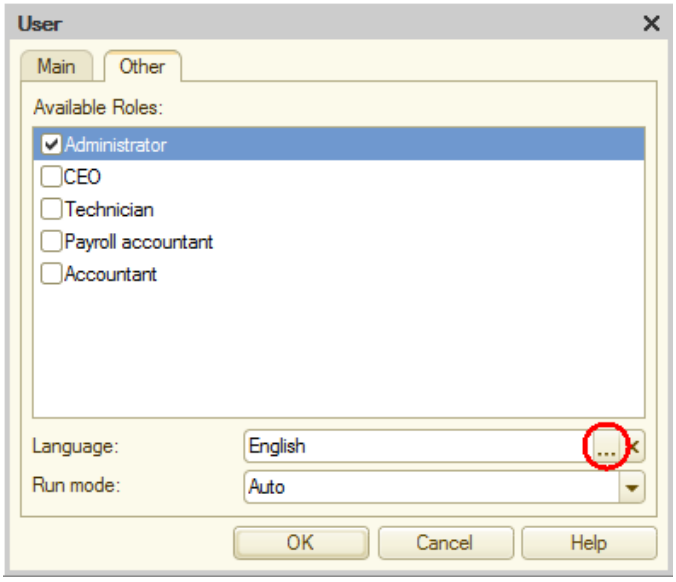
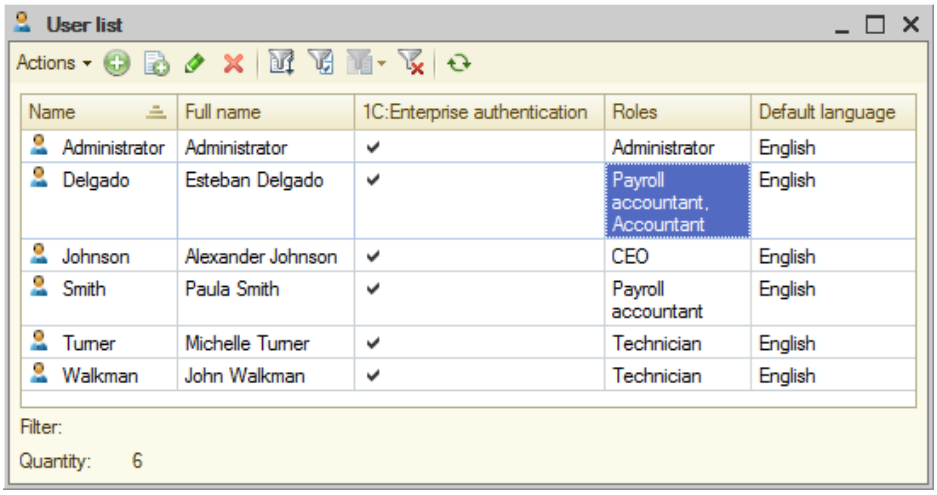


Fig. 22.9. The Other tab of the user editor window

7. Click **OK**.

8. Create the rest of the system users as shown in fig. 22.10. For each user select 1C:Enterprise authentication and the English language.



22.10. User list

Note. If some columns, such as Roles, are not visible in the user list, on the **Actions** menu, click **List Options** and add the required columns.

Note that the chief accountant Esteban Delgado is assigned two roles: Payroll accountant and Accountant because in addition to accounting operations he might perform payroll calculation operations.

You can also view the list of registered users by clicking **Output list** on the **Actions** menu.

Record and database field level data access restrictions

Next, we will demonstrate how you can restrict data access with further precision, based on the infobase data itself.

1C:Enterprise features data access restrictions at record and database field level. For four major rights (read, insert, edit, and delete) you can specify which infobase data is available to users.

The restrictions are defined in a language that is a subset of the query language.

In the next section you will implement the following addition to the Employee accruals document: technicians can view their own salaries but cannot view their bonuses. In other words, technicians are not allowed to view Employee accruals documents that include bonus accrual records.

In Designer mode

Let us modify the technician rights.

1. Grant the **Technician** role the **View** right for the **EmployeeAccruals** document, and also the **View** right for the **Payroll** subsystem.

   The right to view the Payroll subsystem is required because the document belongs to the subsystem.

2. Grant the **View** right for the **WorkScheduleTypes** catalog and for the **MainAccruals** chart of calculation types.

   These rights are required because the EmployeeAccruals document references these objects.

3. In the list of configuration objects, click the **EmployeeAccruals** document, and then click its **Read** right.

4. This right was granted automatically when you granted the View right,

   In the **Data access restrictions** list, click the **Add** button (fig. 22.11).

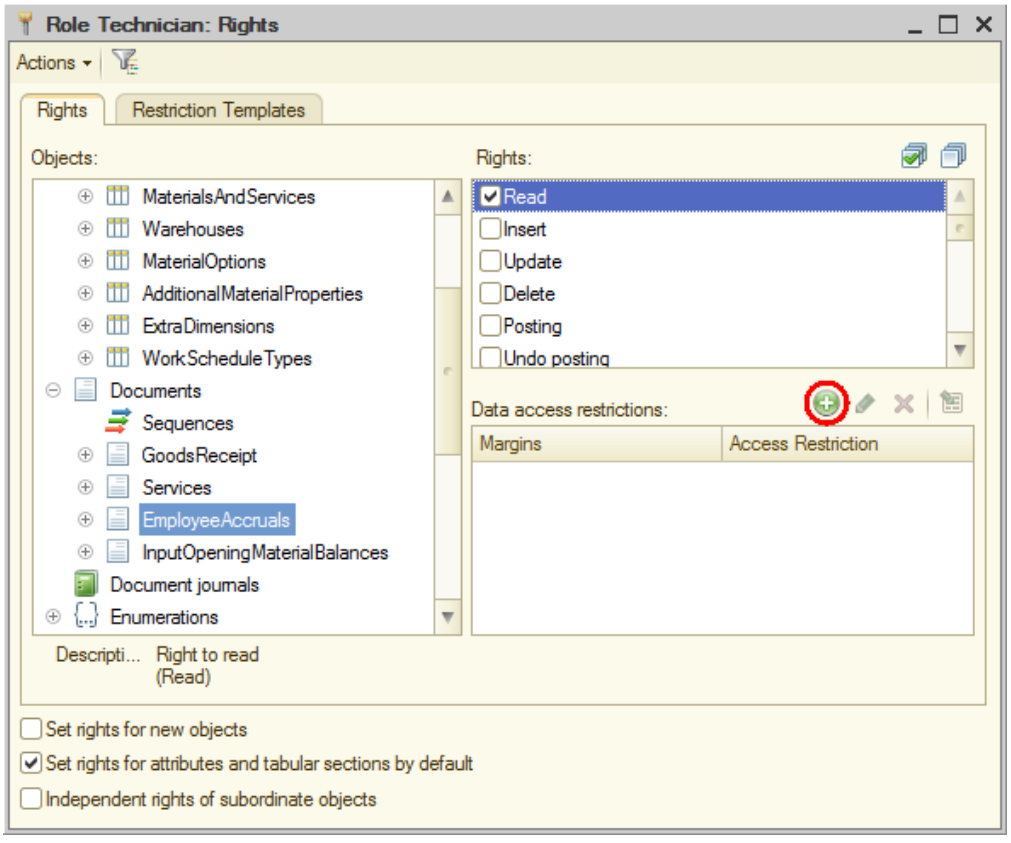


Fig. 22.11. Adding an access restriction for the Technician role

The idea is to deny access to all the fields of the EmployeeAccruals document. So you do not have to select fields here.

5. Click the selection button ... in the **Access Restriction** column (fig. 22.12).

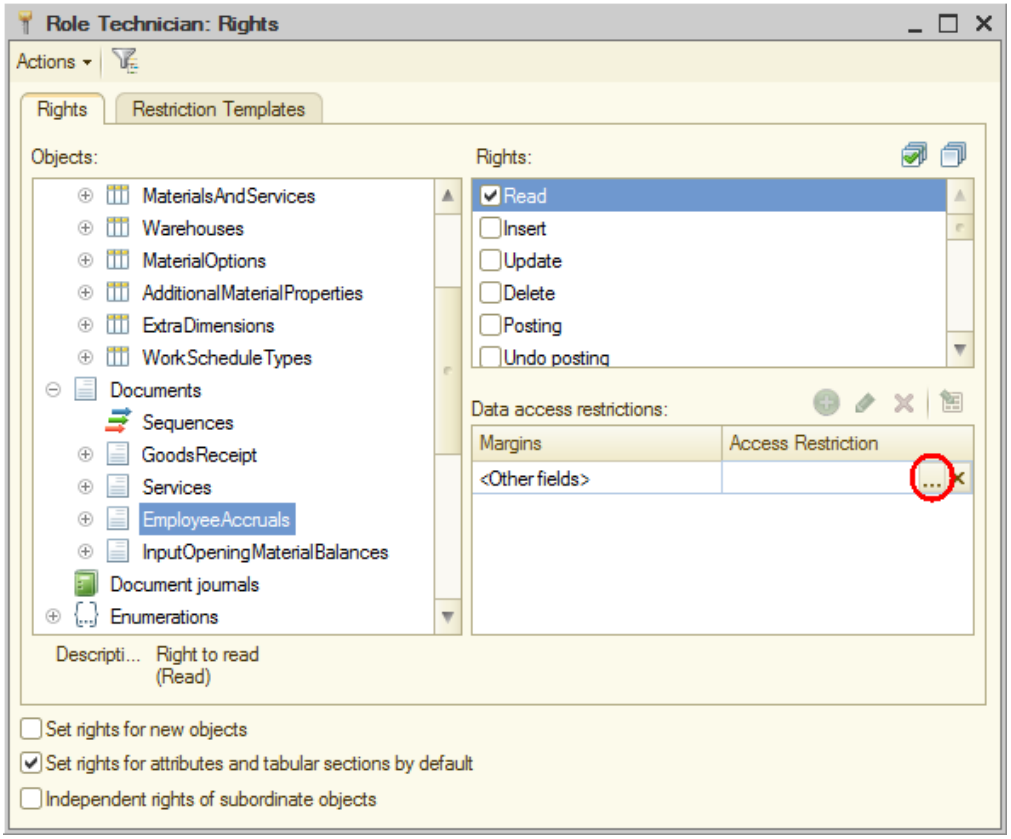


Fig. 22.12. Opening access restriction editor

This opens the **Restrict access** dialog box where you can describe the restrictions in a language that is a subset of the query language.

To simplify the process, let us use the query wizard.

6. Click **Query Builder**.

   Actually this opens the data access restriction wizard, which is very similar to the query wizard (see fig. 22.13).

   Note that the **EmployeeAccruals** table is automatically seleted on the **Tables and fields** tab and the wizard is opened on the **Conditions** tab.

7. Drag the **CalculationType** field of the **Accruals** tabular section to the list of conditions, select the **Arbitrary** check box, and populate the right-hand part of the condition as shown in listing 22.1 (fig. 22.13).

Listing 22.1. Data access restriction

```
EmployeeAccruals.Accruals.CalculationType <>
    VALUE(ChartOfCalculationTypes.MainAccruals.Bonus)
```

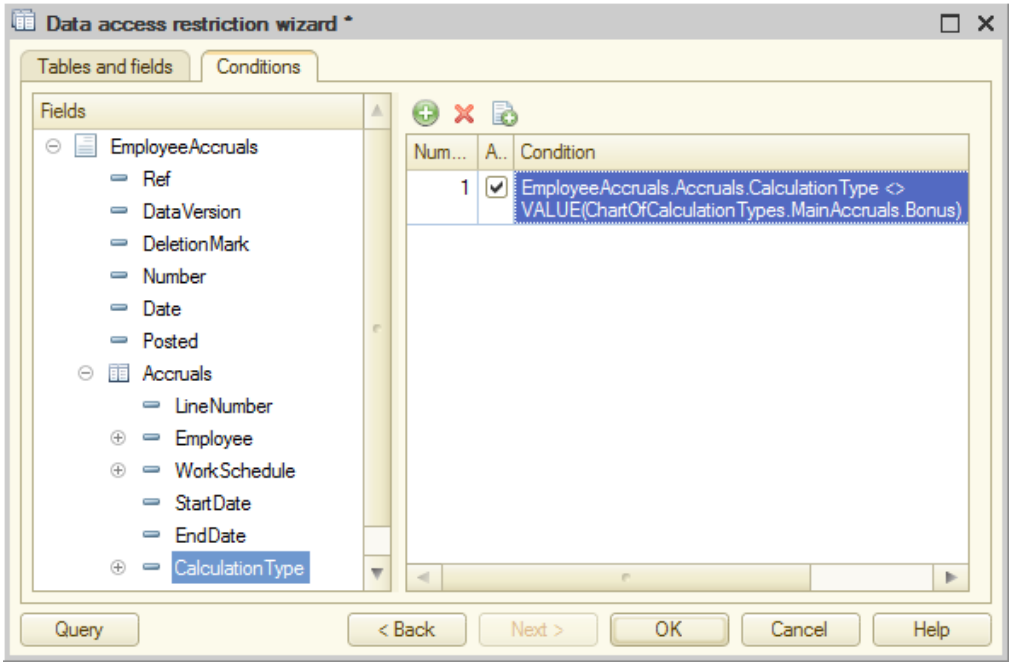


Fig. 22.13. Data access restriction wizard

8. Click **OK** (fig. 22.14).

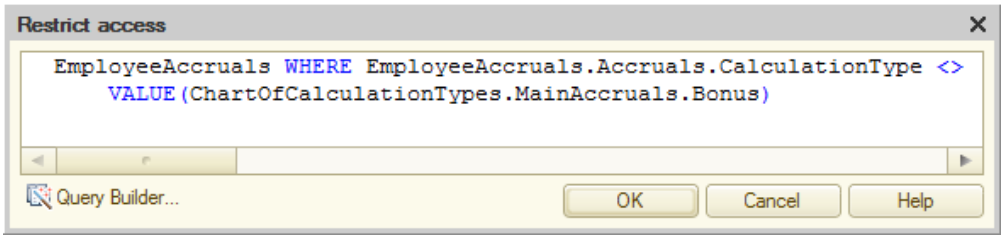


Fig. 22.14. Data access restriction script

The current user can only read or edit some database object if a data access restriction grants them the right to perform this action (if the restriction condition is true).

In this example a user can read the Employee Accruals document

```
EmployeeAccruals WHERE ...
```

only if its Accruals tabular section

```
... WHERE EmployeeAccruals.Accruals ...
```

contains calculation types

```
... WHERE EmployeeAccruals.Accruals.CalculationType ...
```

that are not Bonus calculation type

```
... <> VALUE(ChartOfCalculationTypes.MainAccruals.Bonus)
```

9. Click **OK**.

The data access restrictions pane for the Technician role should look as shown in fig. 22.15.

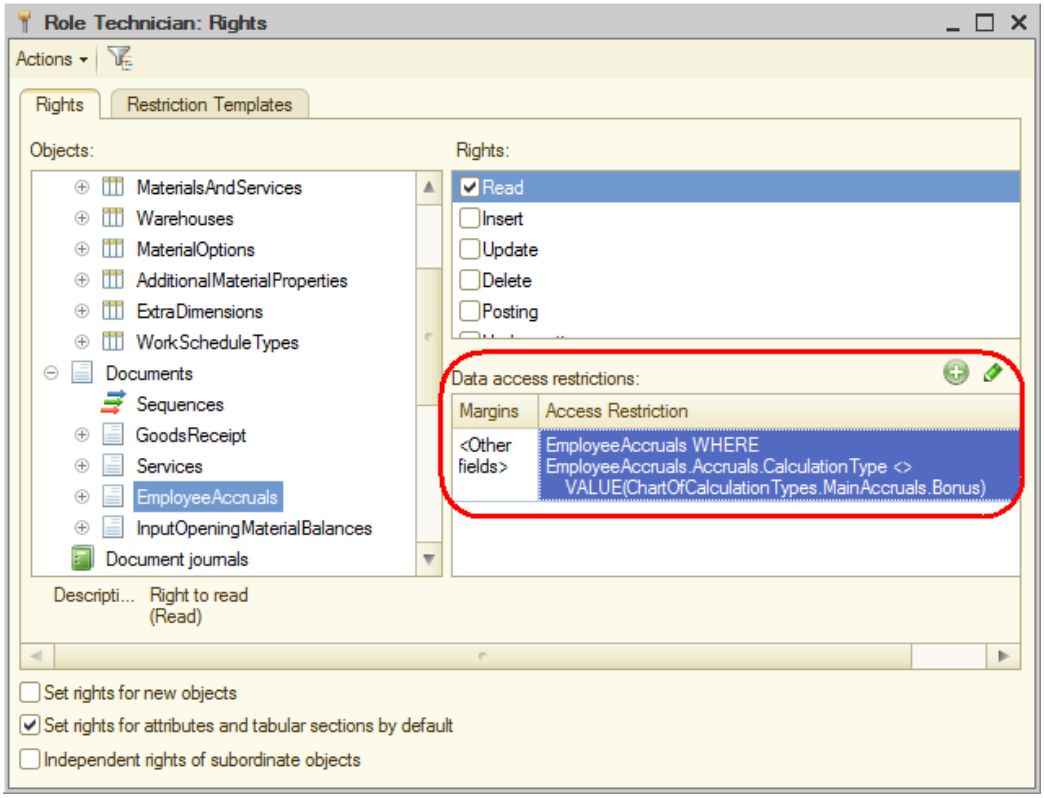


Fig. 22.15. Access restriction for the Technician role

In 1C:Enterprise mode

Let us test the result.

1. Update the infobase by pressing F7 and start 1C:Enterprise for user **Turner**.

   This user has the Technician role.

2. In the **Payroll** section, open the list of **Employee accruals** documents (fig. 22.16).

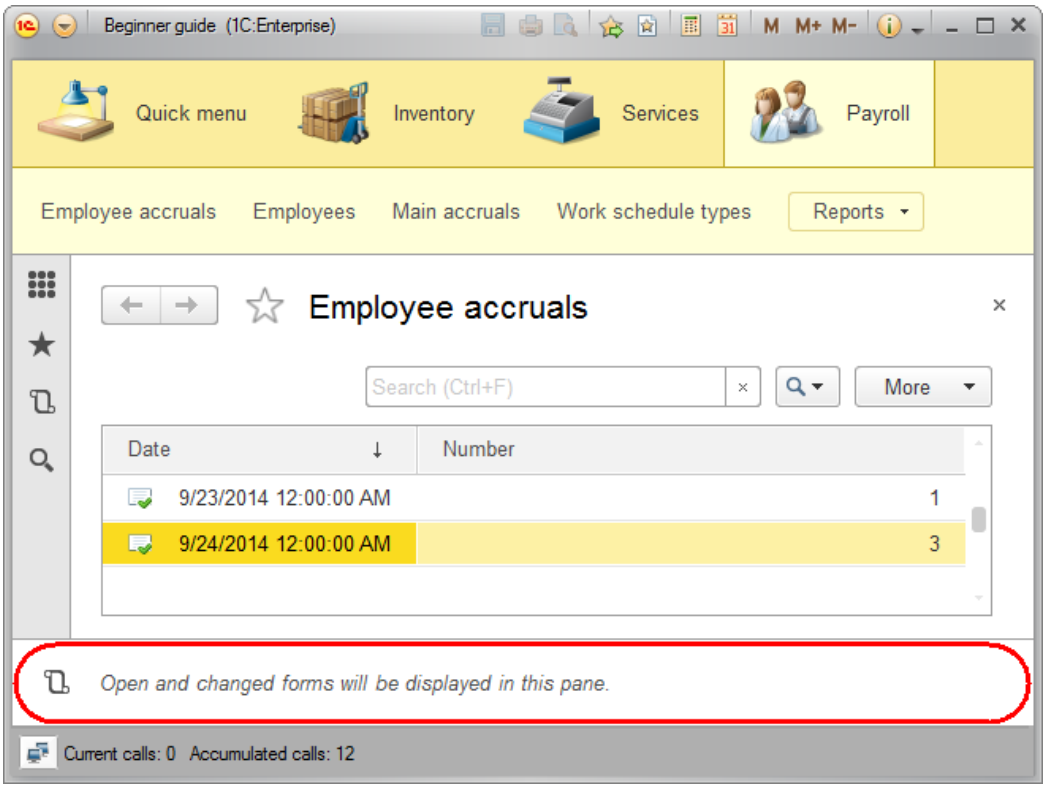


Fig. 22.16. List of Employee accruals documents

As you can see, the list only includes documents #1 and #3 because document #2 accrues bonuses.

This also illustrates a feature that is not directly related to data access restrictions.

At the bottom of the window you can see the history panel. In lesson three, during the creation of your first catalog, you added this panel to the applied solution window in Designer. Then you removed this panel in 1C:Enterprise mode.

But when a user makes a change in 1C:Enterprise mode, the change is only made for that user, while the panel settings specified in Designer are applied to the entire configuration. At the time you made that change your configuration did not have any users defined, that is why the panel is now visible. It will be visible to each new user.

To correct this, let us return to Designer and hide the history panel from there. This change will be applied to all users.

In Designer mode

Let us hide the history panel.

1. Right-click the configuration tree root object and click **Open client application interface**.
2. In the **Client application interface** window, in the left pane, click **History pane**, and then click the **Delete current item** button (fig. 22.17).

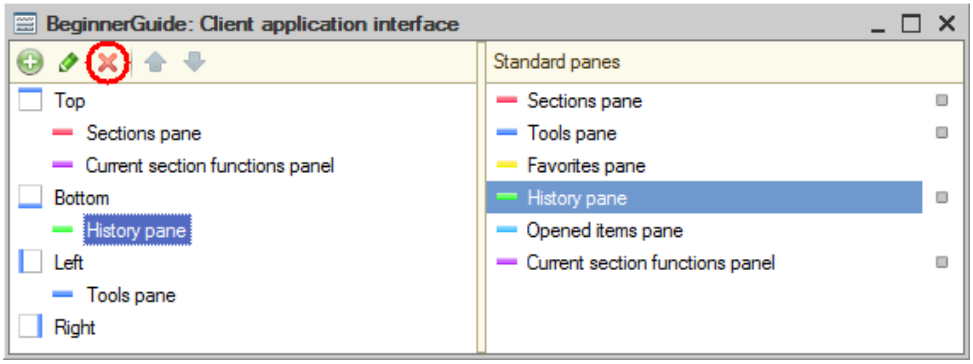


Fig. 22.17. Specifying applied solution panel settings

Now let us return to access restrictions and complicate the task. We still do not want a technician to see the accrued bonuses but at the same time we do not want to hide the fact that such a document exists at all.

In other words, a technician should be able to see the document in the list of documents but should be unable to open it.

Let us look at the Employee accruals document access restriction that you set for the Technician role. You have not specified any fields there, so the restriction is applied to all the document fields (see fig. 22.15).

Let us unconditionally allow reading the document fields that are required to display the document in the list.

This is how you enable the display of the document in the list. But since you do not remove the existing condition for other fields, users still can open the document only if its tabular section includes calculation types other than Bonus.

Let us add another condition to the access restriction.

3. In the list of fields, select the following fields:
   - **Ref**
   - **DeletionMark**
   - **Number**
   - **Date**
   - **Posted**

4. In the access restriction column, enter WHERE TRUE (fig. 22.18).

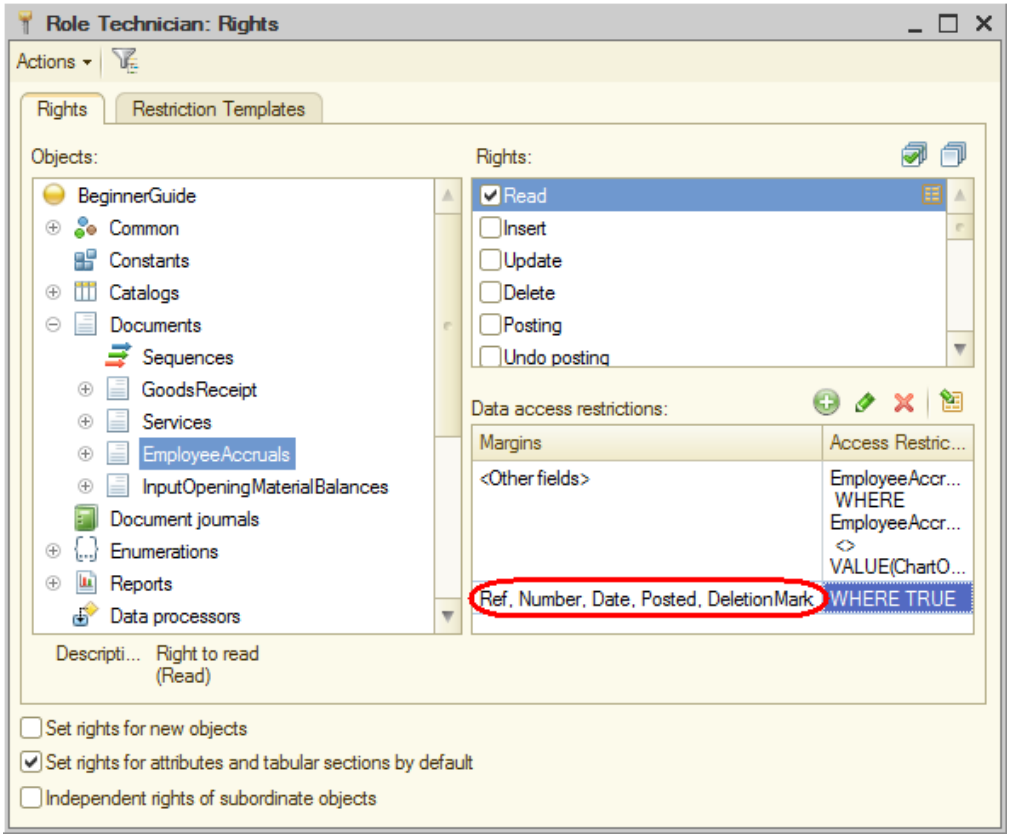


Fig. 22.18. Specifying data access restrictions for the Technician role

5. Close the rights editor.

In 1C:Enterprise mode

Let us test the result.

1. Update the infobase by pressing F7 and start 1C:Enterprise as technician **Turner**.
2. In the **Payroll** section, open the list of **Employee accruals** documents (fig. 22.19).

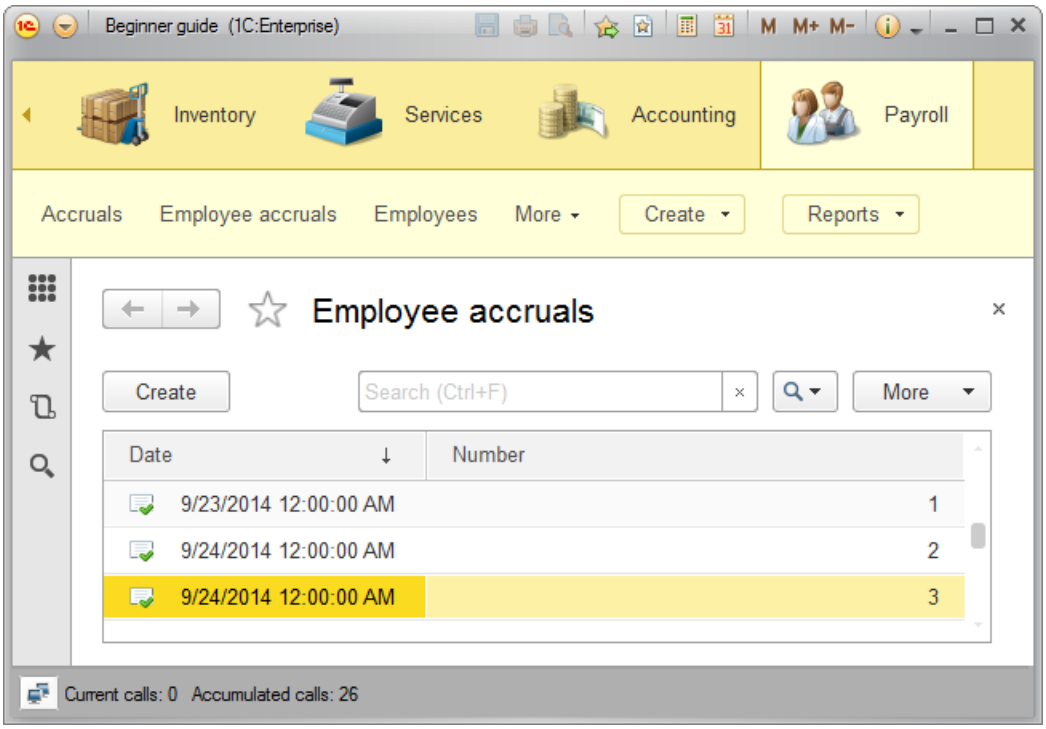


Fig. 22.19. List of Employee accruals documents

The list of documents includes all the documents related to accruals. You can open and view documents #1 and #3 but if you attempt to open document #2, an access violation message is displayed (fig. 22.20).

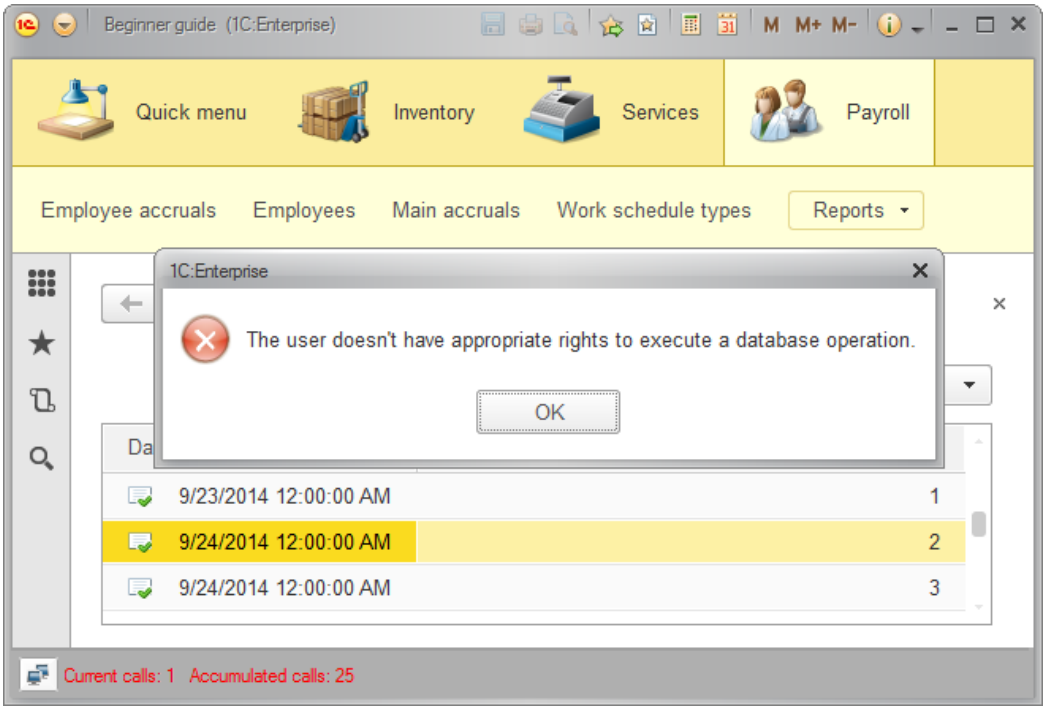


Fig. 22.20. Access violation message

So you have reached the goal.

Now pay attention to the following issue. Everything is fine while document #2 only contains records related to bonus calculation. But remember the idea of the access restriction: a user

can only read the Employee accruals document if its tabular section named Accruals includes calculation types other than Bonus. This means that if such a document contains calculation types other than Bonus, a technician can open and view this document.

Let us try this.

3. Run 1C:Enterprise as **Administrator**.
4. In the **Payroll** section, open the list of **Employee accruals** documents.
5. Open the document #2 and copy any of its rows.
6. In the new row, change the calculation type to **Salary**.
7. Post and close the document.
8. Close the session.
9. Run 1C:Enterprise as **Turner**.
10. In the **Payroll** section, open the list of **Employee accruals** documents.
11. Open the document #2.

    The document is opened and you can see all its rows.

In Designer mode

Let us correct the access restriction.

1. Return to Designer.

   To prevent viewing the document, you have to edit the data access restriction.

   The new condition will be more complex, so we will also demonstrate how you can use templates in data restrictions.

2. Open the **Technician** role and click the **Templates** tab (fig. 22.21).

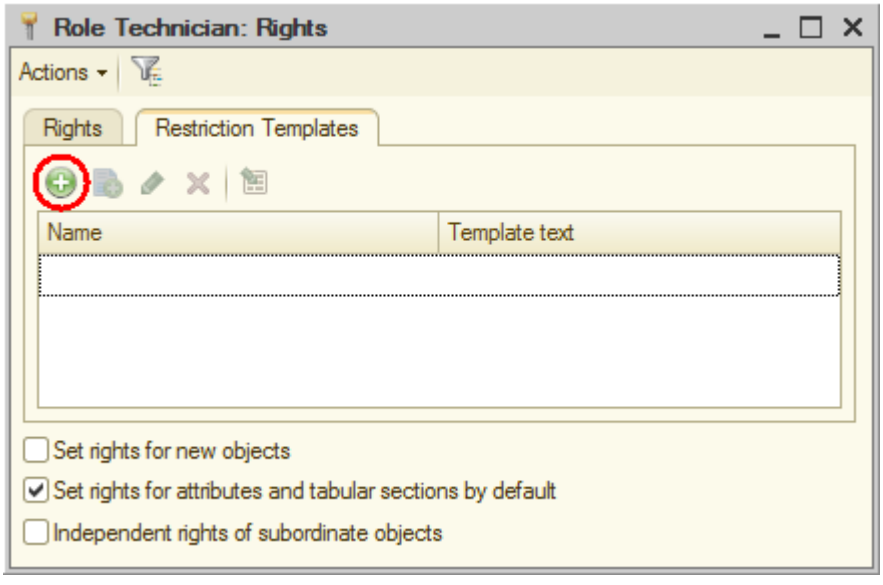


Fig. 22.21. Adding an access restriction template to the Technician role

3. Add a template named *BonusIncluded*, with the text shown in listing 22.2.

Listing 22.2. Data access restriction

```
SELECT
    1
FROM
    Document.EmployeeAccruals.Accruals
WHERE
    Document.EmployeeAccruals.Accruals.CalculationType =
        VALUE (ChartOfCalculationTypes.MainAccruals.Bonus)
        AND Document.EmployeeAccruals.Accruals.Ref = #Parameter(1).Ref
```

Actually it is a query to the tabular section of the EmployeeAccruals document, which returns either nothing or a single-field record that contain 1 as its value.

This record is returned if the document tabular section includes the Bonus calculation type.

The second condition in the query specifies the document whose tabular section is accessed. This condition uses a template parameter.

Listing 22.3. Data access restriction

```
AND Document.EmployeeAccruals.Accruals.Ref = #Parameter(1).Ref
```

#Parameter(1) will be replaced with the string specified when this template is called from the data access restriction.

4. Return to the **Rights** tab.
5. In the access restriction for other fields (the first row) replace the restriction with the text shown in listing 22.4 (fig. 22.22).

Listing 22.4. Data access restriction

```
DocAccruals WHERE NOT 1 IN (#BonusIncluded("DocAccruals"))
```

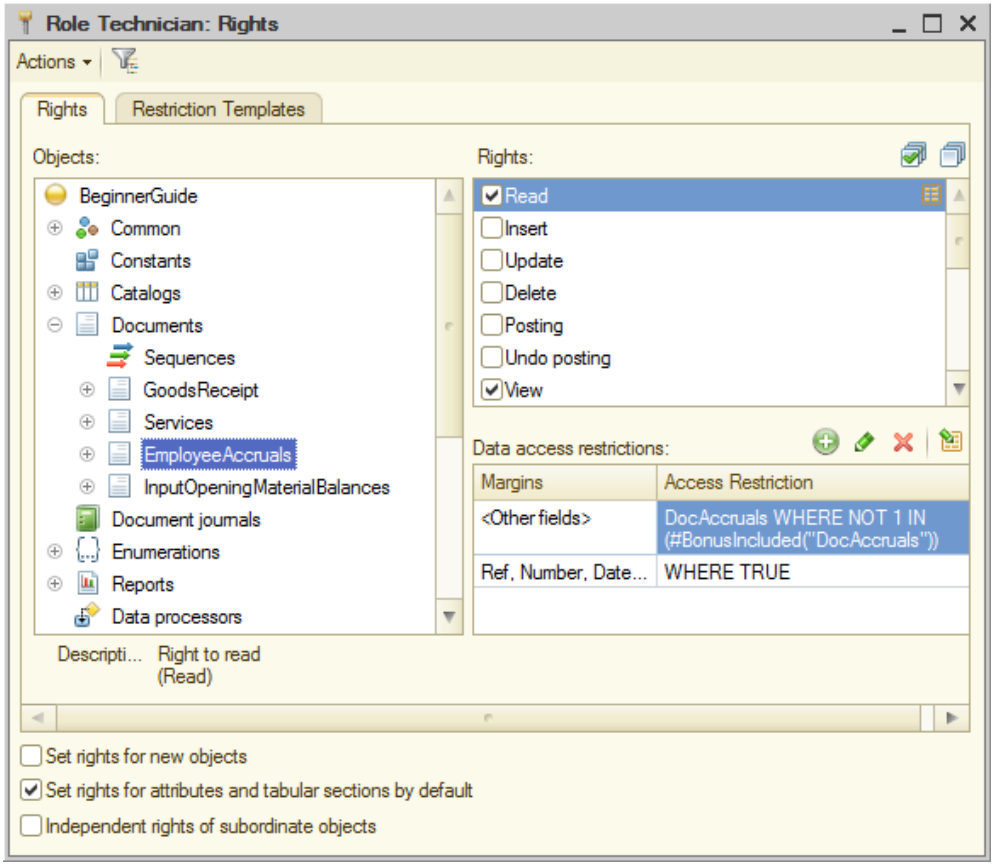


Fig. 22.22. Setting an access restriction for the Technician role

The #BonusIncluded("DocAccruals") statement calls the template. The template script is simply inserted here and the DocAccruals line replaces the first template parameter (#Parameter(1)).

As we have mentioned earlier, if a tabular section includes an accrual named Bonus, the query in the template returns a single record with 1 as its value.

This is why the condition shown in listing 22.4 allows reading DocAccruals when the template query does not return 1:

```
... WHERE NOT 1 IN (#BonusIncluded("DocAccruals"))
```

So it happens when the tabular section does not contain any Bonus accrual.

You could specify this restriction condition directly, without using a template. But the resulting script would have been more complicated (listing 22.5). And templates also provide the option to separate some parts of restriction conditions for reuse in other conditions.

Listing 22.5. Data access restriction

```
DocAccruals WHERE NOT 1 IN (
    SELECT
        1
    FROM
        Document.EmployeeAccruals.Accruals
```

```
WHERE Document.EmployeeAccruals.Accruals.CalculationType =
    VALUE(ChartOfCalculationTypes.MainAccruals.Bonus)
    AND Document.EmployeeAccruals.Accruals.Ref = DocAccruals.Ref)
```

6. Close the rights editor.

In 1C:Enterprise mode

Let us test the result.

1. Update the infobase by pressing F7 and start 1C:Enterprise as **Turner**.
2. In the **Payroll** section, open the list of **Employee accruals** documents.

   As you remember, document #2 includes rows both with the Bonus calculation type and with the Salary calculation type. Until now you could open the document.
3. Try to open the document #2.

   The platform displays an access violation message, which is exactly what we expect (see fig. 22.20).

In Designer mode

Since this example, which includes access restrictions at database record and field level, was intended solely for demonstration purposes, let us revert to the previous configuration state.

1. For the **Technician** role, for the **EmployeeAccruals** document, remove the **Read** right.
2. For the **Payroll** subsystem, remove the **View** right.
3. For the **WorkScheduleTypes** catalog and for the **MainAccruals** chart of calculation types, remove the **Read** right.
4. Run 1C:Enterprise as **Administrator**.
5. In the **Payroll** section, open the list of **Employee accruals** documents.
6. Open the document #2 and remove the last row (the recently added one).
7. Post and close the document.

Quiz

- What are the uses of the Role configuration object?
- How does one create a role using configuration subsystems?
- How does one create a user list and specify user rights?

- What is the difference between 1C:Enterprise authentication and operating system authentication?
- How does one create database record- and field-level restrictions?

Lesson 23 (1:10). Start page and command interface customization

In this lesson you will improve the configuration look and feel by fine tuning the applied solution command interface, specifying command visibility by user roles, and customizing the start page and the Quick menu section for each role.

This will clean up the applied solution interface and make it more user-friendly, which is very important.

The appearance and arrangement of the application interface heavily influence the user feedback to your applied solution.

Customizing section command interfaces

Until now you have virtually ignored the appearance of section (subsystem) command interfaces because you dealt with issues that were more important from a developer perspective.

Now it is time to deal with this aspect, which is important from a user perspective, so let us pay attention to the order of commands and combine them into groups based on their priorities and usage frequency.

You will also use the command visibility by roles feature to refine the section interfaces, making them more transparent for certain users.

In Designer mode

If you look at the section interfaces, you might find them quite messy and overloaded with commands that open lists and create catalog items.

Since all users need to work with catalogs more or less often, moving all the catalogs to the Enterprise section, which you made available to all users in the previous lesson, makes a lot of sense. Of course some users cannot change the catalog content (for example, CEO cannot), still it is handy to keep all reference data in a single location. This also simplifies the interfaces of other sections.

1. For each catalog, open its editor window and then, on the **Subsystems** tab, select the **Enterprise** subsystem and clear all other subsystems (fig. 23.1).

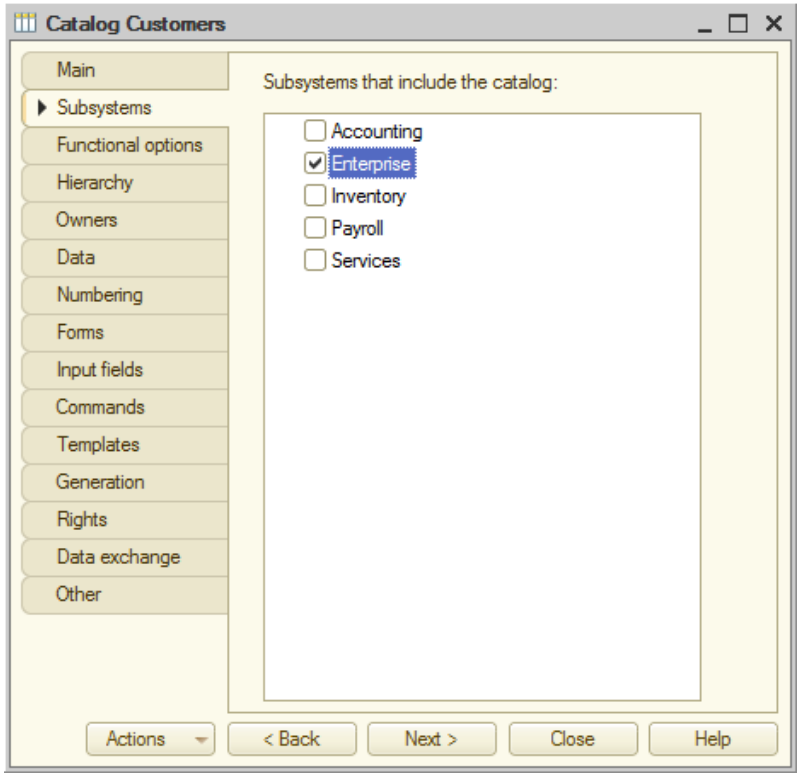


Fig. 23.1. Selecting a subsystem for a catalog

Make this change even for the catalogs that were not previously included in any subsystem (such as MaterialOptions and ExtraDimensions).

2. Add the **MaterialPropertyValues** information register to the **Enterprise** subsystem.

   It is the register that stores the mapping between material properties and their values.

   Now let us customize the section command interfaces. You will order the commands according to their priorities, as well as show or hide them for specific roles.

3. Expand the **Subsystems** branch, right-click the **Inventory** subsystem, and click **Open command interface** (fig. 23.2).

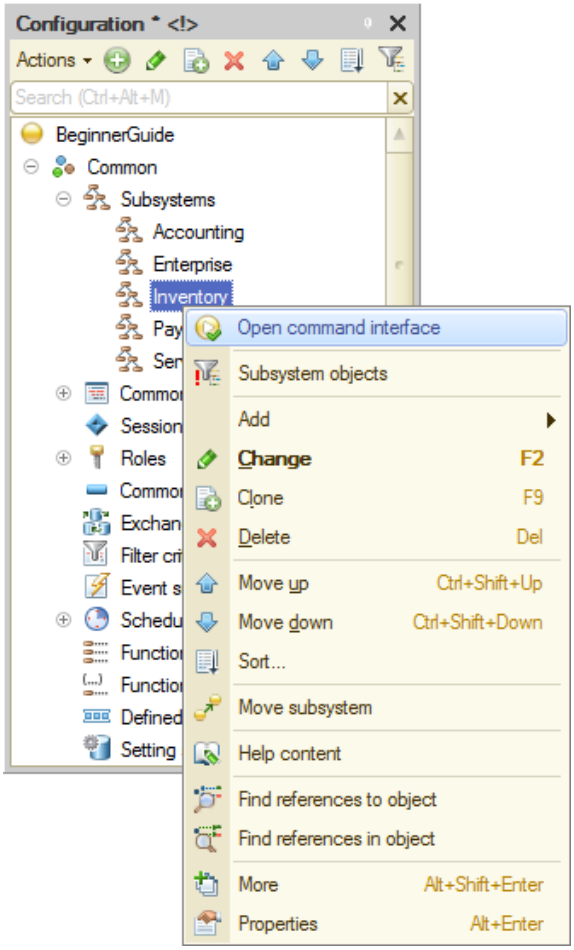


Fig. 23.2. Opening the subsystem command interface editor

The following roles have the right to access the Inventory subsystem: Administrator, CEO, and Technician.

4. Drag the **Goods receipts** command from the **Navigation Panel.Normal** group to the **Navigation Panel.Important** group.

   This is because creating and viewing goods receipts are the most frequently used operations in the Inventory section. Commands belonging to the Important group are displayed in bold in the section commands pane.

5. In the **Actions panel.Create** group, enable visibility of the **Price: create** command and arrange the commands as follows:

   - **Navigation panel.Important**
     - **Goods receipts**
   - **Navigation panel.Normal**
     - **Material and service prices**
   - **Navigation panel.See also**
     - **Balance of materials**
     - **Cost of materials**
     - **Sales**

- **Actions panel.Create**
  - **Goods receipt: create**
  - **Price: create**
- **Actions panel.Reports**
  - **Materials**
  - **Material balance by property**

This order of commands is based on their importance and usage frequency.

Now let us fine tune the command visibility to roles.

6. In the **Navigation panel.See also** group, hide all commands from **CEO** and **Technician**.

   This is because commands for viewing all accumulation register records (Navigation panel.See also group) are only required by Administrator.

7. Hide the **Goods receipt: create** and **Price: create** commands from **Administrator** and **CEO**.

   This is because commands for creating goods receipts and prices (Navigation panel.Create group) are only required by Technicians. CEO does not need them because they can only view documents. Administrator does not enter large amounts of data and in rare cases when they need to create a document they can do it from a document list.

   The resulting command interface settings should look as shown in fig. 23.3.

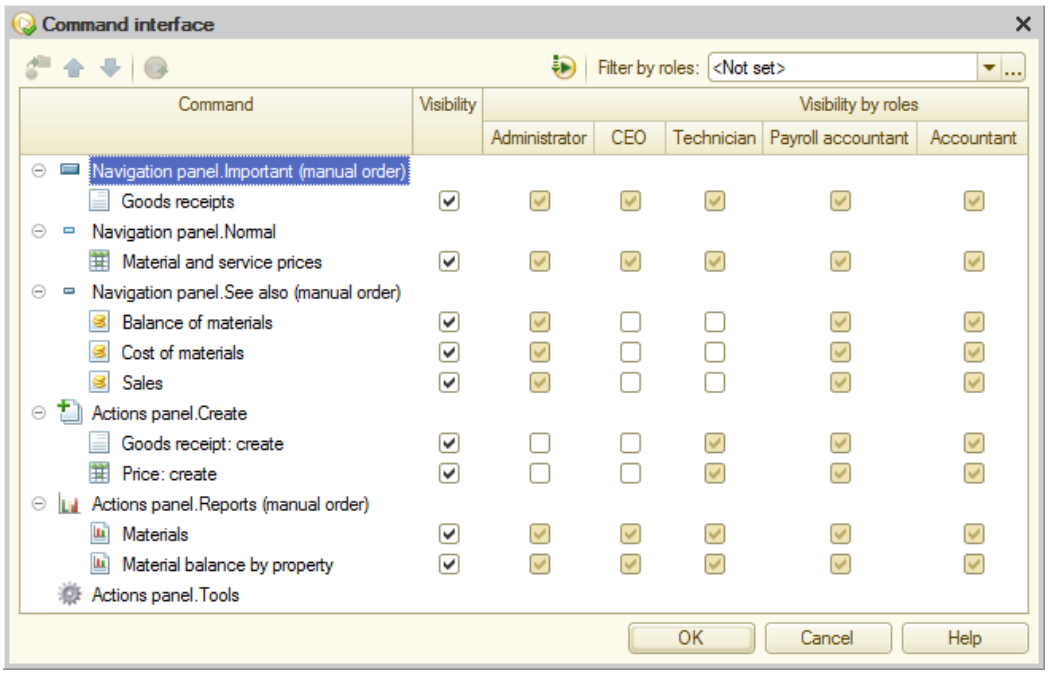


Fig. 23.3. Inventory subsystem command interface

Let us customize the other subsystems in a similar manner.

The following roles have the right to access the Services subsystem: Administrator, CEO, and Technician.

8. For the **Services** subsystem, arrange the commands and their general visibility as follows:
    - **Navigation panel.Important**
        - **Services**
    - **Navigation panel.Normal**
        - **Material and service prices**
    - **Navigation panel.See also**
        - **Balance of materials**
        - **Cost of materials**
        - **Sales**
    - **Actions panel.Create**
        - **Services: create**
        - **Prices: create**
    - **Actions panel.Reports**
        - **Materials**
        - **Services document register**
        - **Service evaluation**
        - **Revenue by technician**
        - **Service list**
        - **Profit by customer**
        - **Generic report**

    Let us fine tune the command visibility to roles.

9. In the **Navigation panel.See also** group, hide all commands from **CEO** and **Technician**.

    This is because commands for viewing all accumulation register records (Navigation panel.See also group) are only required by Administrator.

10. Hide the **Services: create** and **Price: create** commands from **Administrator** and **CEO**.

    This is because commands for creating Services documents and prices (Navigation panel.Create group) are only required by Technicians. CEO does not need them because they can only view documents. Administrator does not enter large amounts of data and in rare cases when they need to create a document they can do it from a document list.

    The resulting command interface settings should look as shown in fig. 23.4.

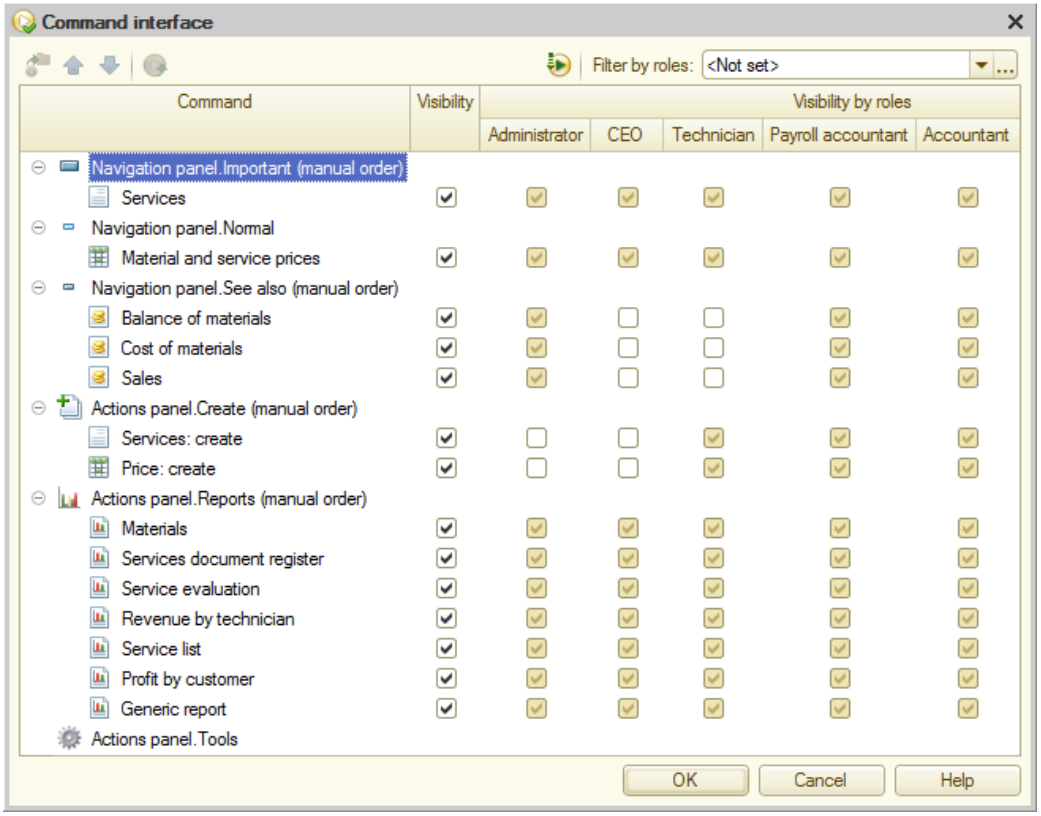


Fig. 23.4. Services subsystem command interface

The following roles have the right to access the Accounting subsystem: Administrator, CEO, and Accountant.

11. For the **Accounting** subsystem, arrange the commands and their general visibility as follows:
    - **Navigation panel.Important**
        - **Primary**
        - **Main chart of accounts**
        - **Input opening material balances**
    - **Navigation panel.Normal**
        - **Goods receipts**
        - **Services**
        - **Material and service prices**
    - **Navigation panel.See also**
        - **Extra dimension types**
        - **Balance of materials**
        - **Cost of materials**
        - **Sales**
    - **Actions panel.Create**
        - Hide all commands because bulk object addition is not expected in this interface section.

- **Actions panel.Reports**
  - **Trial balance**
  - **Employee accruals**
  - **Materials**
  - **Service evaluation**
  - **Service list**
  - **Profit by customer**
  - **Material balance by property**

Let us fine tune the command visibility to roles.

12. Hide the **Balance of materials**, **Cost of materials**, and **Sales** commands from **CEO** and **Accountant**.

    This is because commands for viewing all accumulation register records (Navigation panel.See also group) are only required by Administrator.

13. Hide the **Extra dimension types** command from CEO.

    This is because CEO does not need the command for viewing extra dimension types (Navigation panel.See also group), since it is an accounting operation.

    The resulting command interface settings should look as shown in fig. 23.5.

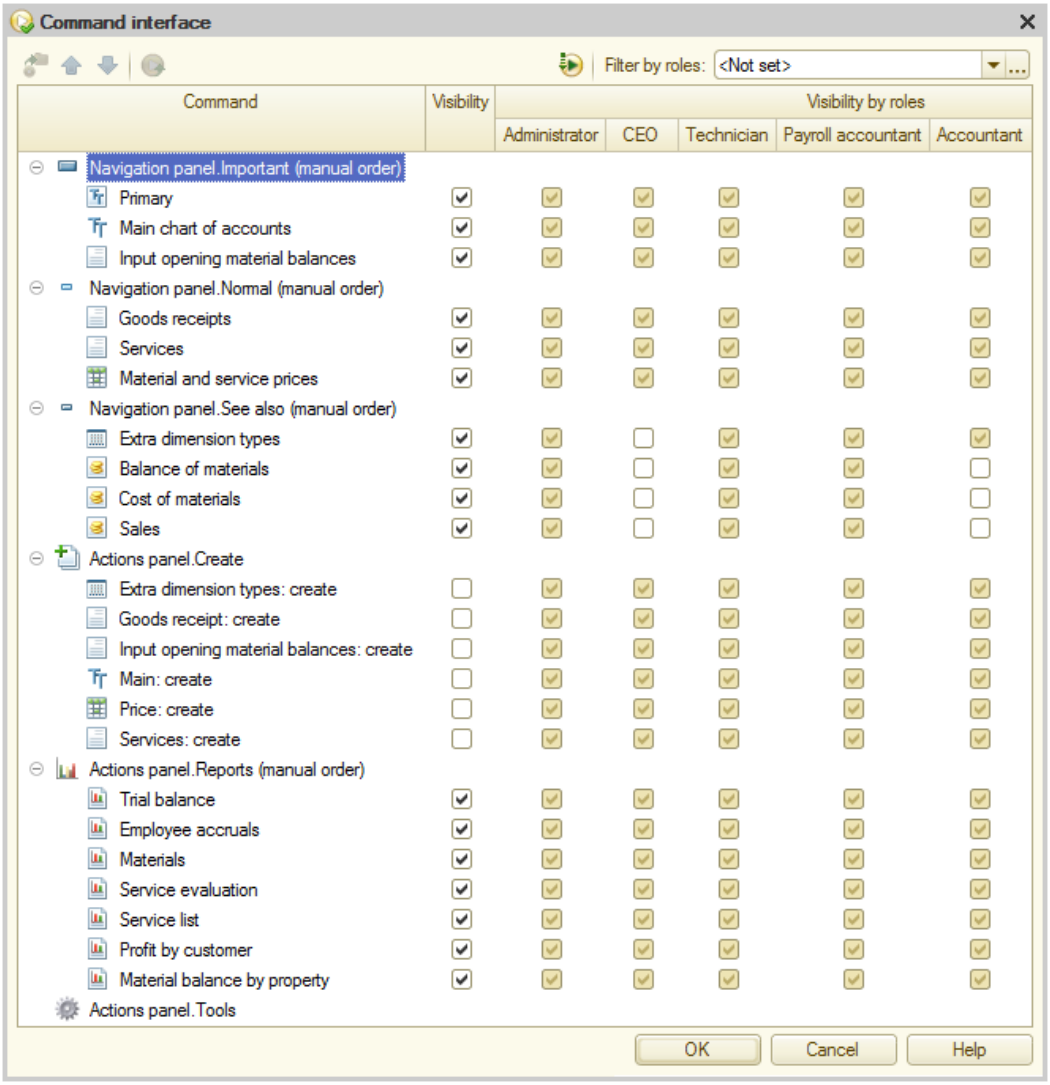


Fig. 23.5. Accounting subsystem command interface

The following roles have the right to access the Payroll subsystem: Administrator, CEO, and Payroll accountant.

14. For the **Payroll** subsystem, arrange the commands and their general visibility as follows:
    - **Navigation panel.Important**
        - **Work schedules**
        - **Employee accruals**
    - **Navigation panel.Normal**
        - **Calculation types**
        - **Accruals**
    - **Actions panel.Create**
        - **Employee accrual: create** (and hide the Work schedules: create and Main accruals: create commands because bulk object addition is not expected in this interface section).

- **Actions panel.Reports**
  - **Employee accruals**
  - **Accruals chart**
  - **Revenue by technician**

Let us fine tune the command visibility by roles.

15. Hide the **Calculation types** command from CEO.

    This is because CEO does not need the command for viewing the chart of calculation types (Navigation panel.Normal group).

16. Hide the **Accruals** command from **CEO** and **Payroll accountant**.

    This is because the command for viewing the Accruals register records (Navigation panel.Normal group) is only required by Administrator.

17. Hide the **Employee accruals: create** command from **CEO** and **Administrator**.

    This is because the command for creating Employee accruals documents (Navigation panel.Create group) is only required by Payroll accountant.

18. Disable general visibility for the **Recalculation** report.

    It is the auxiliary report that you used in lesson 18, you no longer need it.

    The resulting command interface settings should look as shown in fig. 23.6.

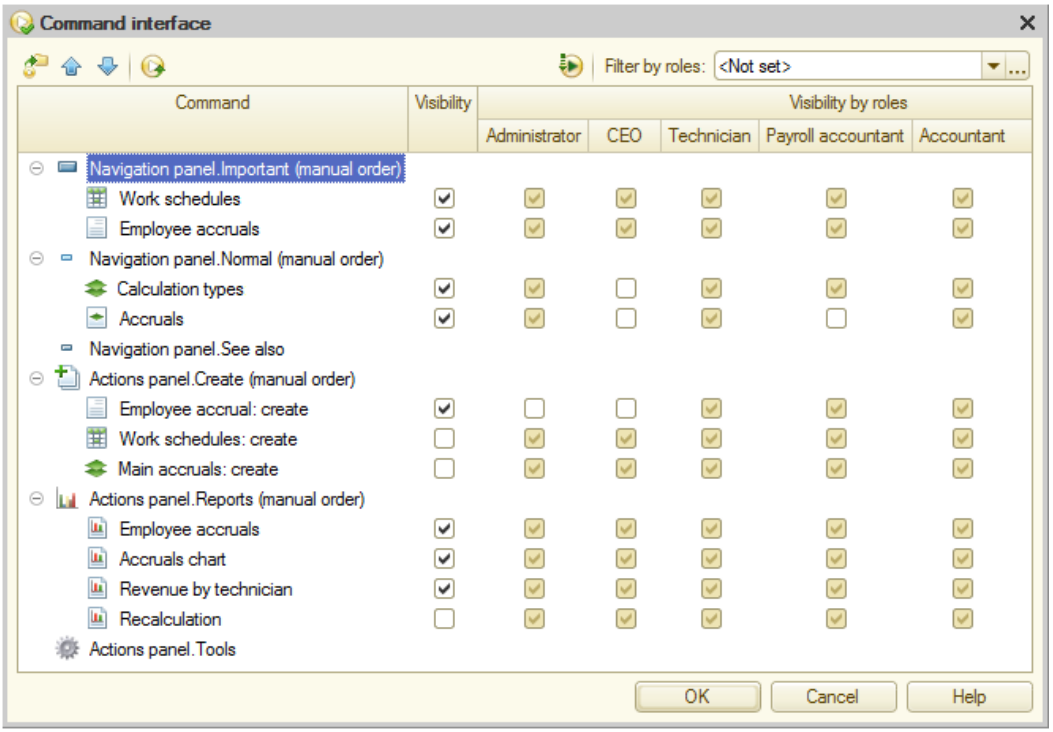


Fig. 23.6. Payroll subsystem command interface

As for the Enterprise subsystem, all users have the right to access it. This subsystem displays reference data.

Let us arrange the commands that open catalog lists according to their priorities and usage frequency, and disable the general visibility of catalog item creation commands. This makes sense because bulk input of catalog data is not expected and whenever a user needs to enter a catalog item, they have a handy option to create it in an item selection drop-down list.

19. For the **Enterprise** subsystem, arrange the commands and their general visibility as follows:
    - **Navigation panel.Important**
        - **Materials and services**
        - **Customers**
        - **Employees**
        - **Warehouses**
    - **Navigation panel.Normal**
        - **Material options**
        - **Additional material properties**
        - **Extra dimensions**
        - **Work schedule types**
        - **Material option details**
20. In the **Navigation panel.Normal** group, hide the commands that open catalog lists from **CEO**.

    These lists provide detailed data, and a CEO has no need to go into all these details.

    The resulting command interface settings should look as shown in fig. 23.7.

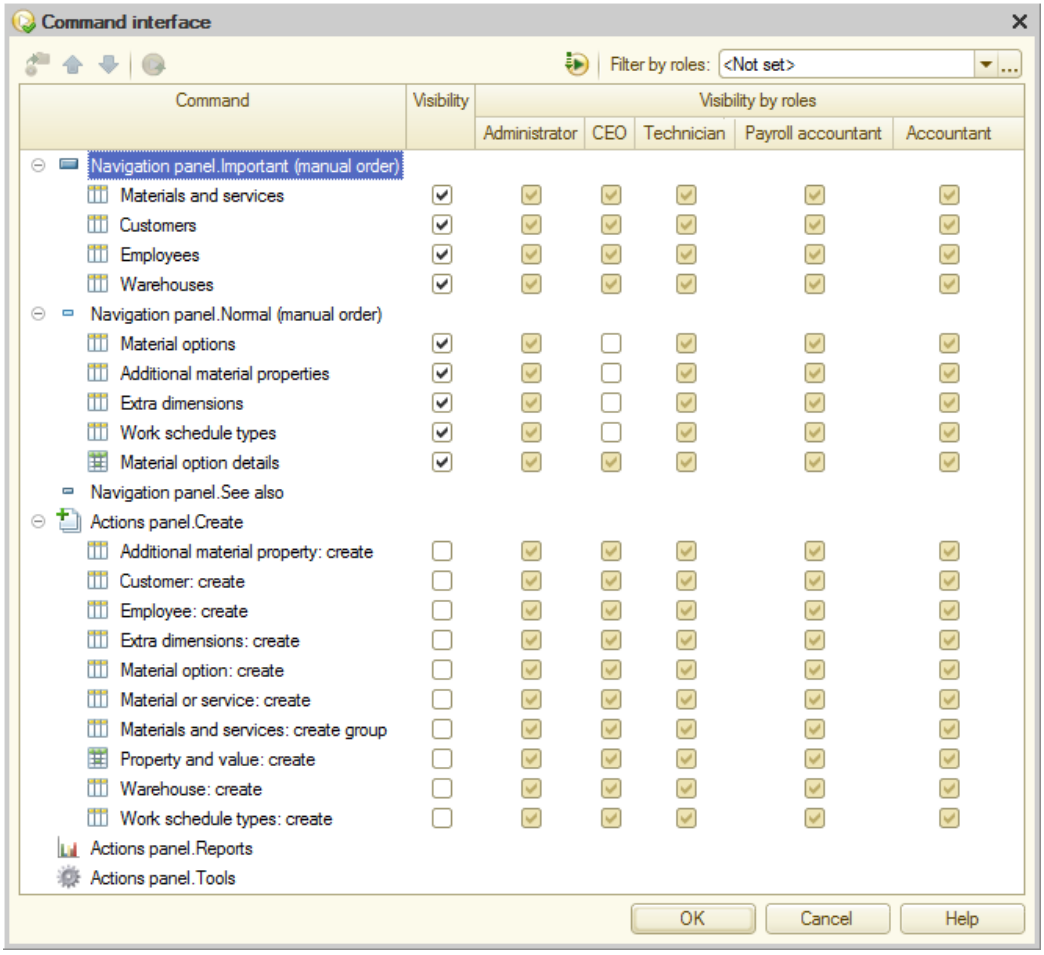


Fig. 23.7. Enterprise subsystem command interface

In 1C:Enterprise mode

Let us test the result.

- Start 1C:Enterprise in the debug mode.

  Since you started Designer as Administrator, the applied solution is started for that user, so you do not have to enter the user name here.

Let us look at the interface, which is now very clean and simple. For example, the Payroll section looks as shown in fig. 23.8.

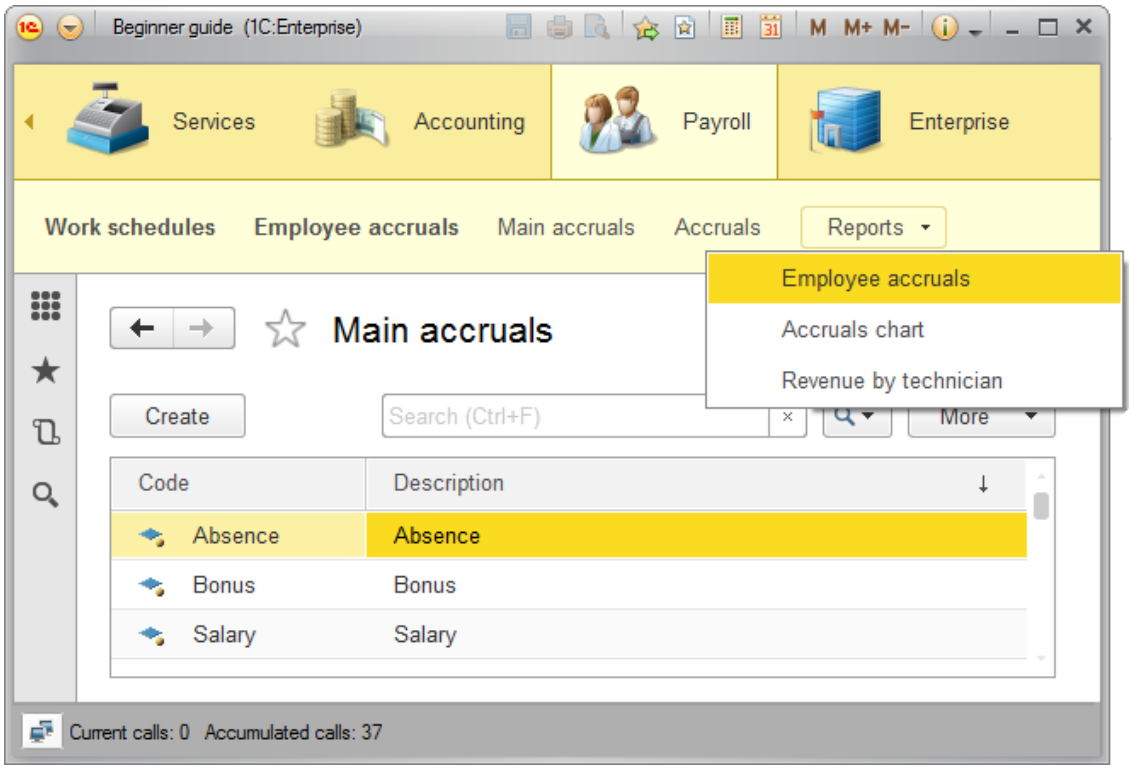


Fig. 23.8. Payroll section command interface

As you can see, all the command groups are visually separated from each other while the most important commands are displayed in bold in section navigation panels. Commands that do not fit a navigation panel are available on the More menu.

So you have customized the section command interface in compliance with your understanding of the applied solution usage scenarios.

But users have the option to customize the command interface according to their needs.

To customize user interface

1. On the **Tools** panel, click the **Function menu** button.

   -OR-

   Double-click the section name.

2. Click **Navigation settings**.

   -OR-

   Click **Action settings** (see fig. 23.9).

3. Open the section function menu and mark frequently used commands.

   They are marked with stars to the left of command names (fig. 23.9).

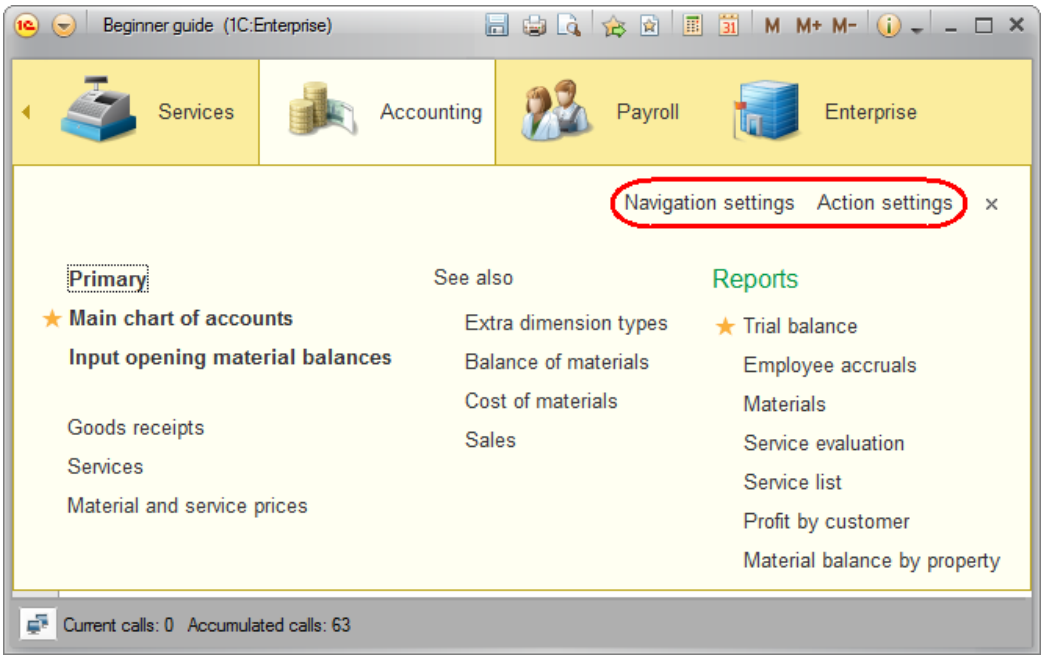


Fig. 23.9. Accounting section function menu

4. On the **Tools** panel, click the **Favorites** item and mark frequently used commands.

   They are marked with pins to the left of command names (fig. 23.10), and displayed in bold at the top of the favorites list, so that a user can easily find them.

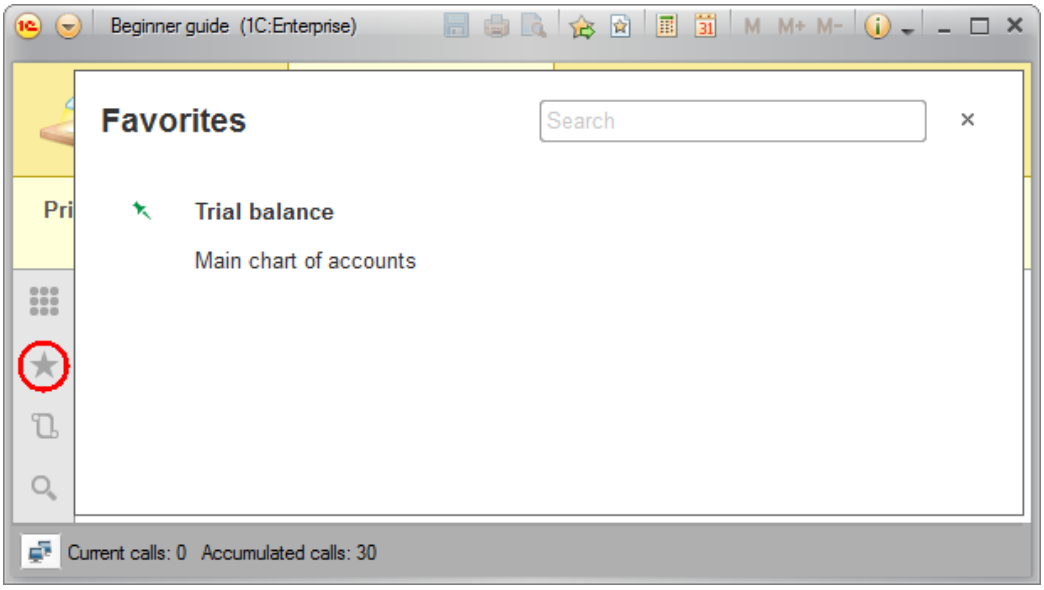


Fig. 23.10. Favorites list of the Accountant user

Quick menu. Customizing the start page

The Quick menu section of an applied solution is intended to encompass the documents, reports, catalogs, and other items that are frequently used by a specific user.

The work area of the Quick menu section contains a user-specific start page, which includes the forms of documents, reports, and other entities that a user accesses on a daily basis. The command bar of the Quick menu section contains the most important commands that are chosen based on the user role.

For example, a stock clerk needs easy access to the list of materials and the list of goods receipts, while a manager needs the list of customers, the list of Services documents, and so on.

When 1C:Enterprise is started, the Quick menu section is active and the forms available in this section are displayed in the applied solution work area (fig. 23.11).

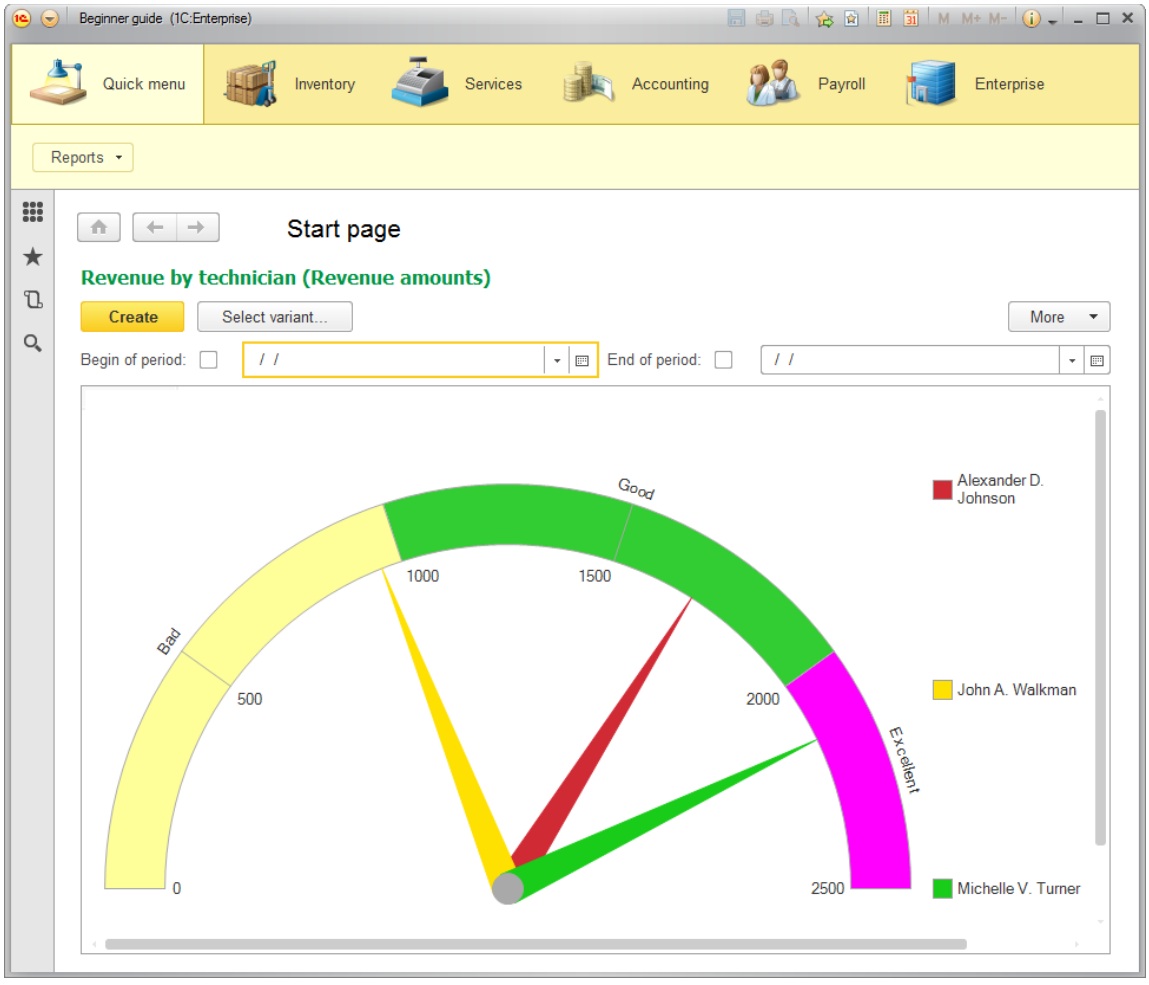


Fig. 23.11. The Quick menu for a user that has the CEO role

So a user does not have to browse through the sections in search of a command they need. Instead, they can immediately start working once they start 1C:Enterprise.

But the Quick menu should not be overloaded with forms and commands because this can reduce the applied solution usability.

In Designer mode

Let us proceed to the customization of the Quick menu.

1. In the configuration object tree, right-click the **BeginnerGuide** root item and then click **Open main section command interface** (fig. 23.12).

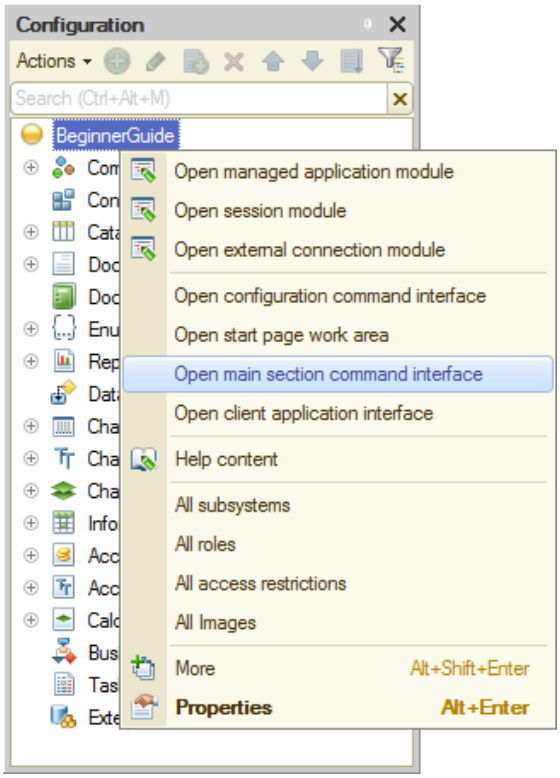


Fig. 23.12. Opening the main sectioncommand interface settings

The dialog box that is opened lists all of the configuration objects grouped by type, and also the commands that perform various operations on these objects. You can double-click the commands to add them to the right pane, which represents the main section command interface, and then specify their visibility for each role.

For the Technician role the most important commands are creation of Goods receipt documents and Services documents, as well as creation of the Materials report and the Revenue by technician report.

2. Move the following commands to the command interface:
   - **Goods receipt: create**
   - **Services: create**
   - **Materials**
   - **Revenue by technician**
3. Set general visibility for each of these commands.
4. Hide the document creation commands from all roles except **Technician**.

5. Hide the report creation commands from all roles except **CEO** and **Technician** (fig. 23.13).

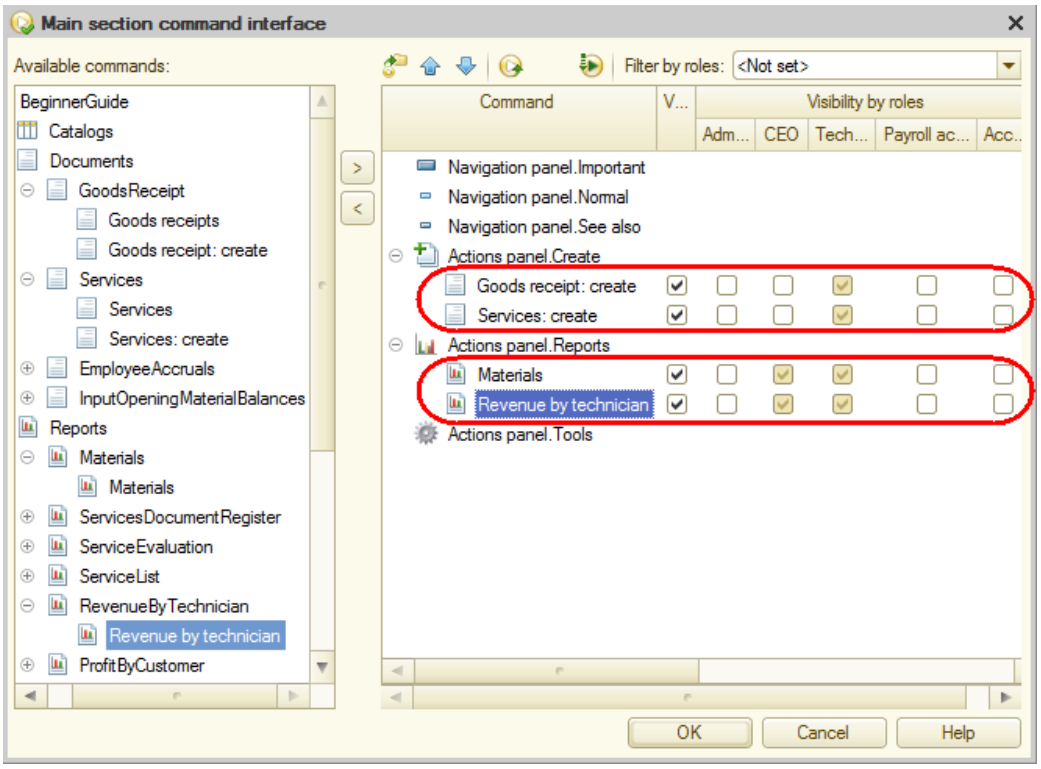


Fig. 23.13. Main section command interface settings window

For the Payroll accountant role the most important commands are creation of Employee accruals documents and creation of the Employee accruals report.

6. Move the following commands to the command interface:
   - **Employee accruals: create** (document)
   - **Employee accruals** (report)
7. Set general visibility for each of these commands.
8. Hide the **Employee accruals: create** command for all roles except **Payroll accountant**.
9. Hide the **Employee accruals** command for all roles except **Payroll accountant** and **CEO**.

   Let us add commands for the Accountant role.

10. Move the following commands to the command interface:
    - **Input opening material balances: create**
    - **Trial balance**
11. Set general visibility for each of these commands.
12. Hide the **Input opening material balances: create** command for all roles except **Accountant**.
13. Hide the **Trial balance** command for all roles except **Accountant** and **CEO**.

For the CEO role the most important commands are creation of the following reports: Materials, Revenue by technician, Employee accruals, and Trial balance.You have already added all of these commands to the main section commands panel and made them visible for CEO.

For the Administrator the main section commands panel remains empty. In the next lesson you will fill it with commands related to data exchange.

The resulting main section command interface settings should look as shown in fig. 23.14.

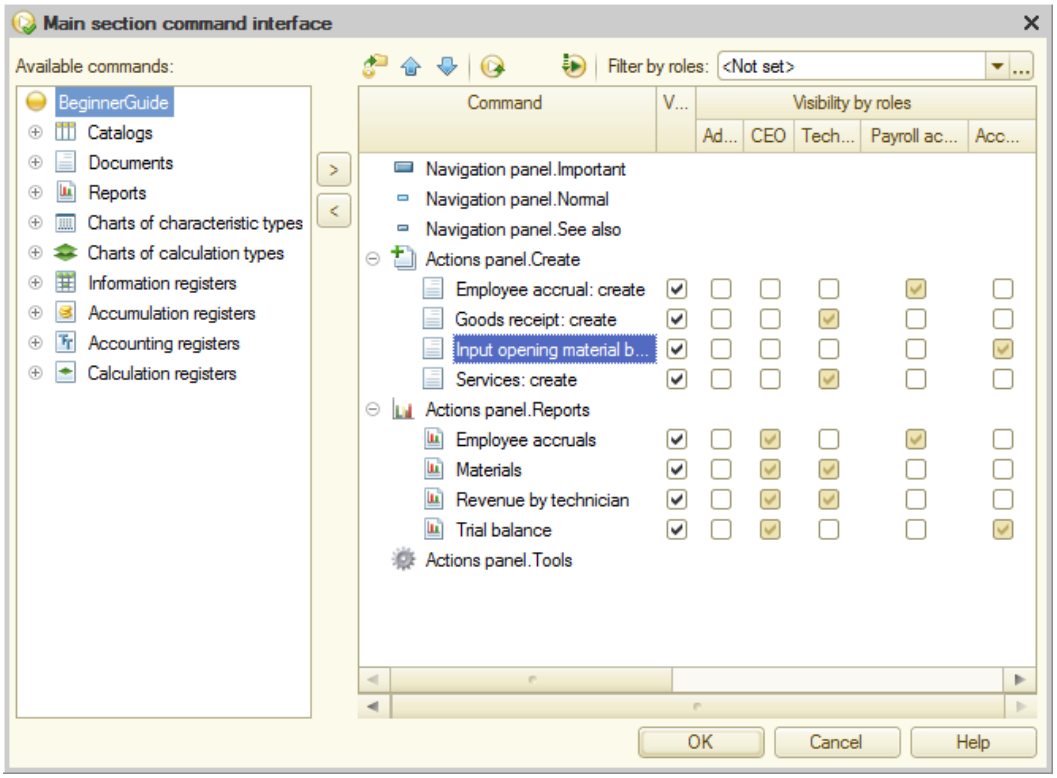


Fig. 23.14. Main section command interface settings window

Now let us customize the start page.

14. In the configuration object tree, right-click the **BeginnerGuide** root item and then click **Open start page work area** (see fig. 23.12).

    This opens the start page settings window (fig. 23.15).

15. In the **Starting page template list**, select **Two columns, different width** (2:1).

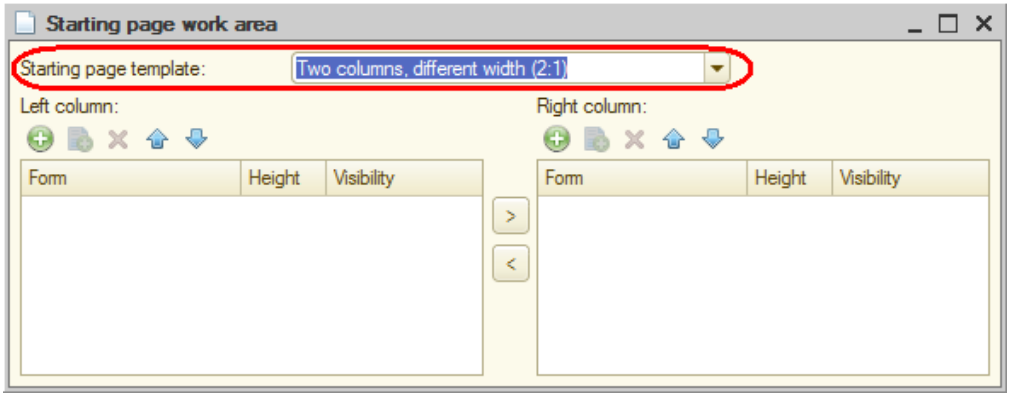


Fig. 23.15. Start page settings window

It means that the forms on the start page are arranged in two columns, and the left column is twice as wide as the right one.

You have the option to select another template: two columns of equal width or a single column. But the first option (2:1) better suits your needs because you can put the most frequently used forms to the left column so that they easily catch the eye.

Note that you cannot add automatically generated forms to the start page. So, before adding a form to the start page, you have to explicitly create it in the configuration. You will create each form when you actually need it.

Attention! To be able to create forms or any other configuration objects, you have to run Designer as Administrator. Other roles do not have rights to access newly created objects.

Let us customize the start page for the Technician role by adding the list of Services documents to the left column and the list of Goods receipt documents to the right column. These list forms are not available in the configuration, so you have to create them first.

16. Create the list forms for the following configuration objects:
    - **Services** document
    - **GoodsReceipt** document

    By default a list form only includes the Date and Number columns.
17. In the **GoodsReceipt** document list form, add the **Warehouse** column (fig. 23.16).

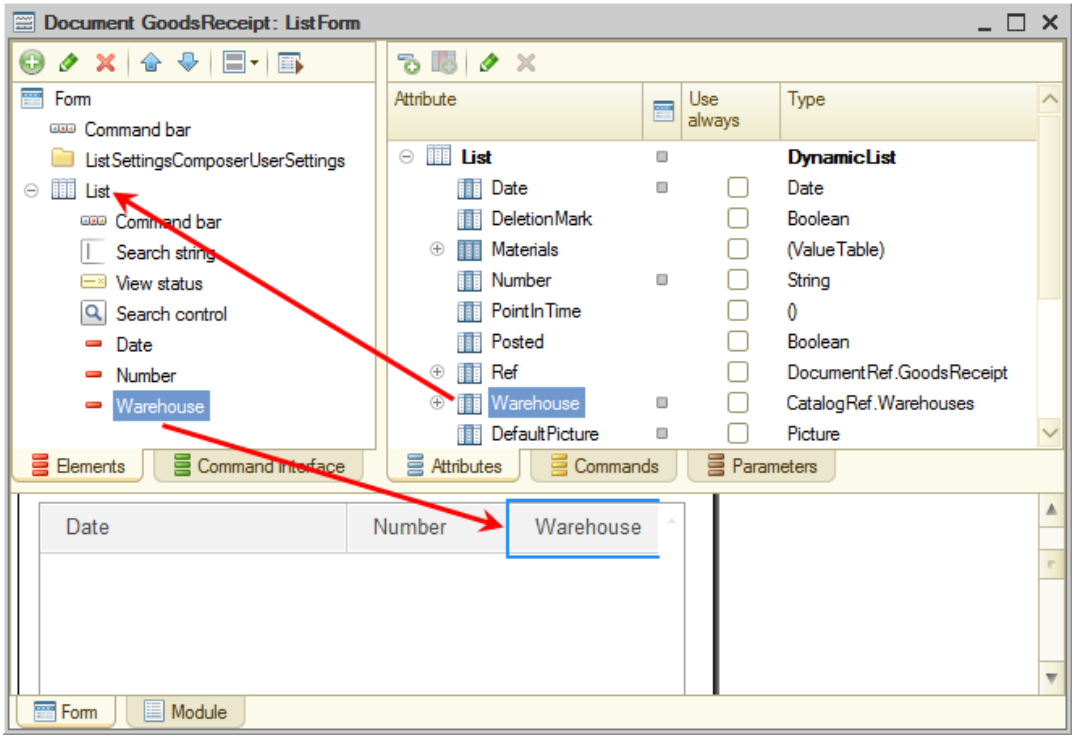


Fig. 23.16. Creating the Goods receipt document list form

18. In the **Services** document list form, add the following columns: **Warehouse**, **Customer**, and **Technician**.
19. In the **Starting page work area** window, in the left pane, click **Add** .
20. Select the **Services** document list form (fig. 23.17).

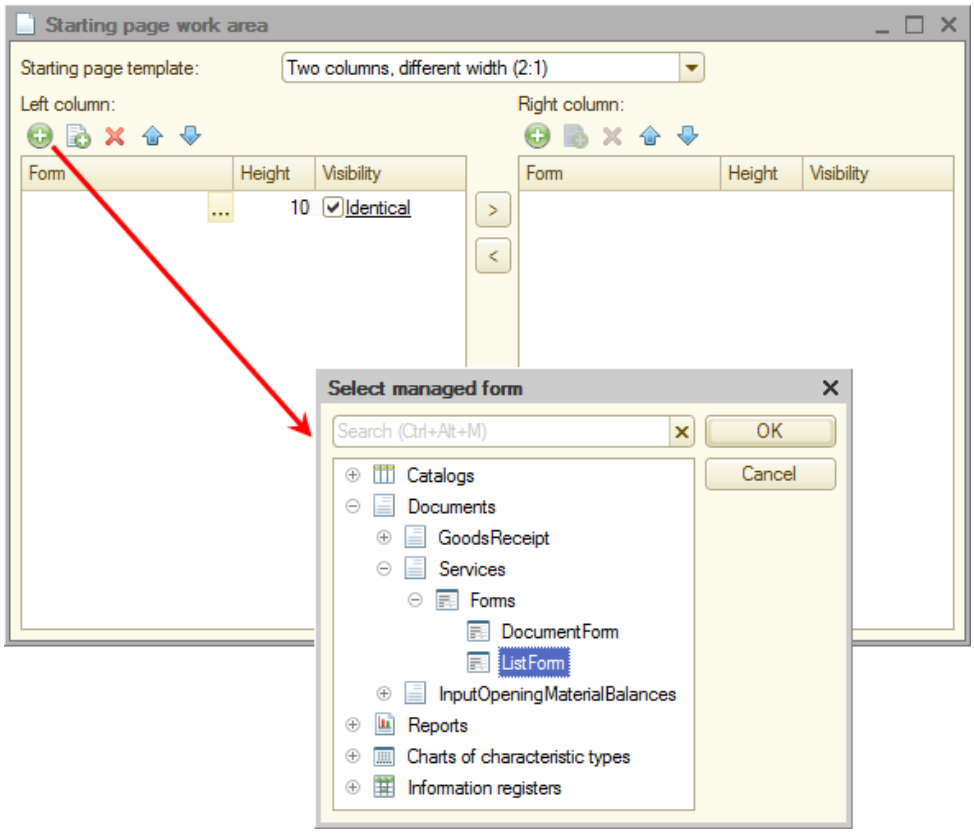


Fig. 23.17. Customizing the start page for users with Technician role

21. In a similar manner, add the **GoodsReceipt** list form to the right column.

    For each of the two forms, click the link in the **Visibility** column and make them visible to the **Technician** role only (fig. 23.18).

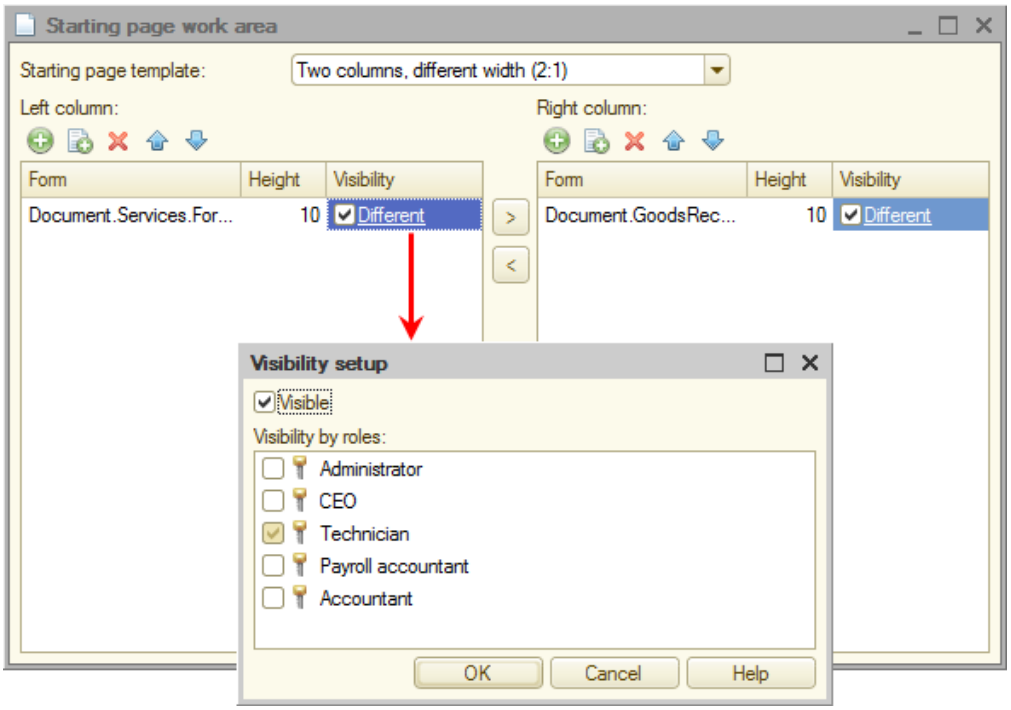


Fig. 23.18. Customizing the start page for users with Technician role

Now let us customize the start page for the Accountant role. The accountant will probably often need the trial balance report, so let us add it to the left column of their start page, leaving the right column empty.

22. For the **TrialBalance** report, create a report form.
23. In the **Starting page work area** window, add the form to the left column and make is visible for the **Accountant** role only.

    Then let us customize the start page for the Payroll accountant role. Payroll accountants mostly use documents and reports related to employee accruals. So let us add the list of Employee accruals documents to the right column. As for the accruals report, they can always open it from the main section commands panel.

24. For the **EmployeeAccruals** document, create a list form.
25. In the **Starting page work area** window, add the form to the right column and make it visible for the **Payroll accountant** role only.

    Then let us customize the start page for the CEO role. This is a role for someone with supervisor responsibilities. They do not need to enter any documents (and they do not even have rights for this). But to make decisions, they need to view reports.

    The commands that open the most important reports required by CEO are located in the main section commands panel. Let us add a single Revenue by technician report to the CEO start page.

26. For the **RevenueByTechnician** report, create a report form.
27. In the **Starting page work area** window, add the form to the left column and make is visible for the **CEO** role only.

    Let us keep the Administrator start page empty for now. Later you will add a data exchange management form there.

    As a result, the start page customization window should look as shown in fig. 23.19.

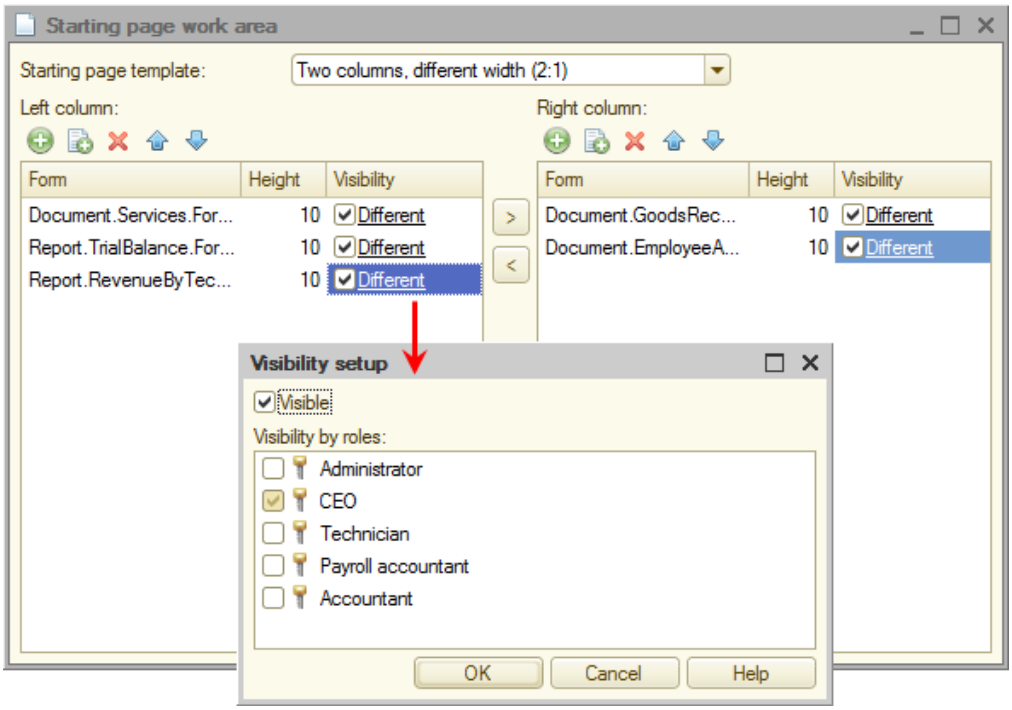


Fig. 23.19. Start page settings window

If a user only has a single form visible on the start page (for example, CEO has a single Revenue by technician report, while Payroll accountant has a single list of employee accruals), the form is scaled to fit the window width, regardless of which column it originally belongs, left or right.

If the start page includes multiple forms visible to a single role and all of the forms belong to a single column, they are arranged vertically on the start page.

In 1C:Enterprise mode

Now if you switch to 1C:Enterprise mode and select user Delgado (who has the Accountant and Payroll accountant roles), you will see the start page shown in fig. 23.20.

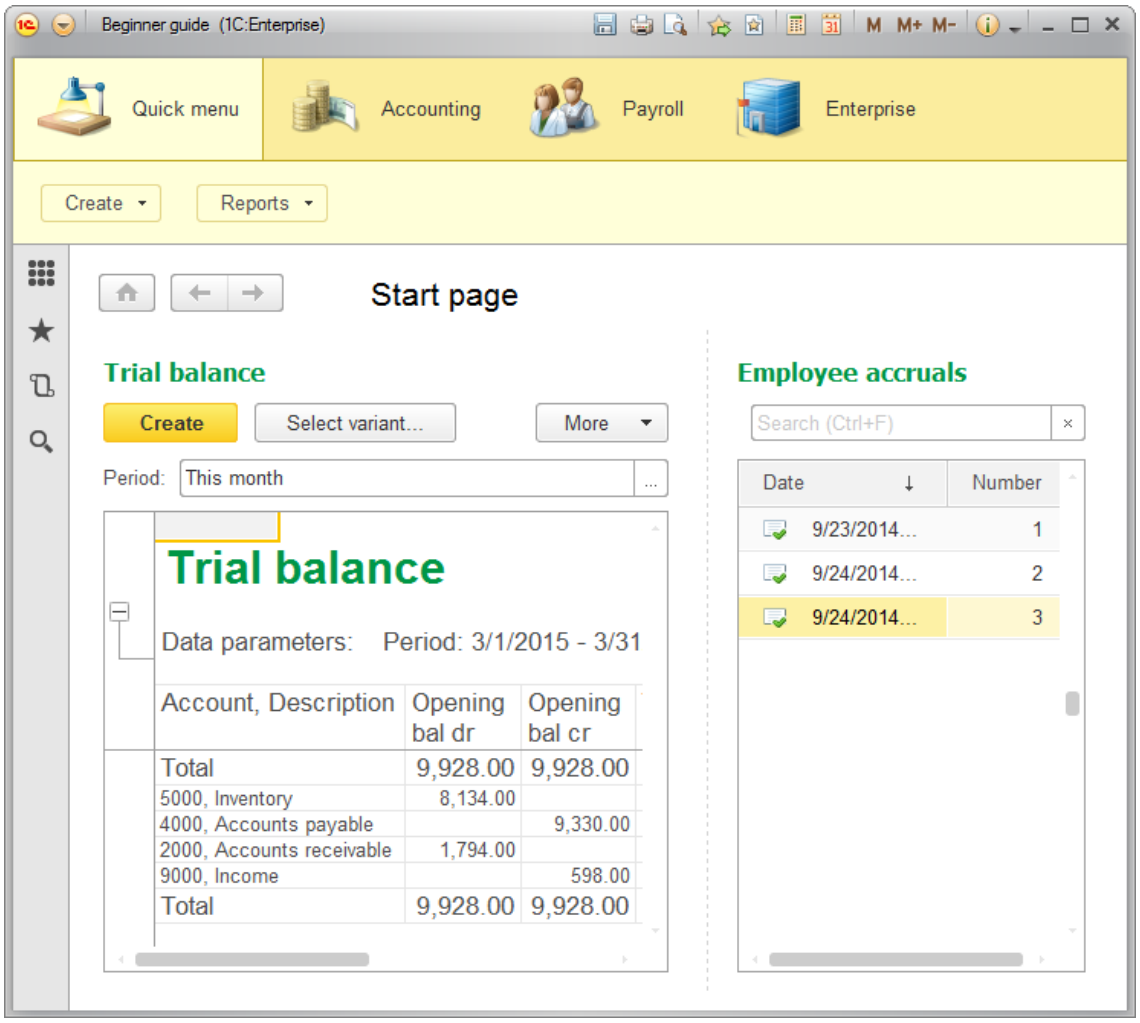


Fig. 23.20. Start page of a user with Accountant and Payroll accountant roles

Attention! To run 1C:Enterprise as a specific user, you have to update the database configuration, restart 1C:Enterprise, select the infobase, click the **1C:Enterprise** button (this starts the thin client), and select the required user. If you run 1C:Enterprise in the debug mode as you usually do (from Designer), the platform considers that you log on using the same user account that you used to start Designer (the Administrator account).

Note that the start page also contains the list of Employee accruals documents because user Delgado has two roles: Accountant and Payroll accountant. And this user also has access to the Accounting, Payroll, and Enterprise sections in compliance with the rights that you specified for the roles during the previous lesson.

And for user Johnson (who has the CEO role) the start page looks as shown in fig. 23.21.

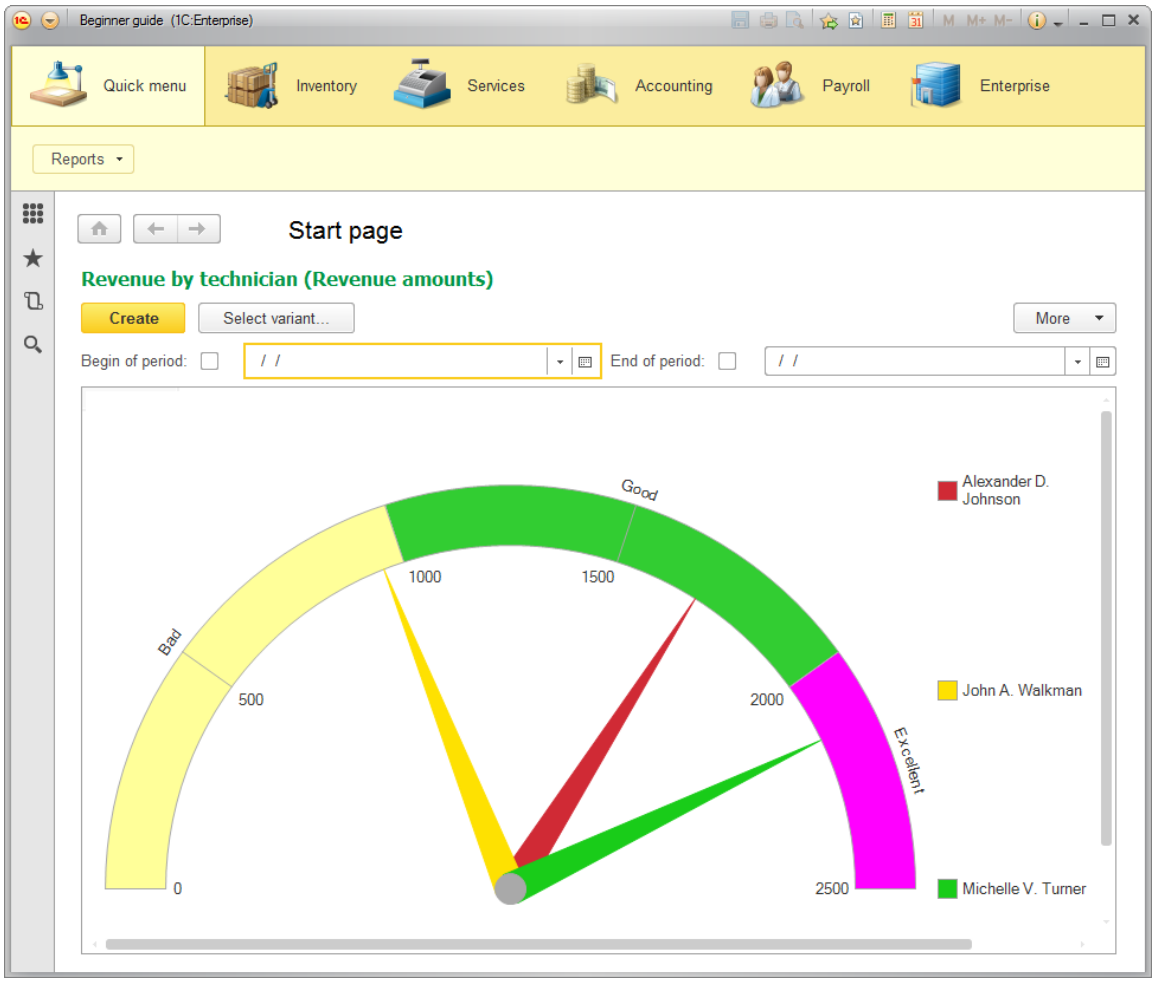


Fig. 23.21. Start page of a user with the CEO role

Note that a CEO needs the "pretty" gauge chart report view. You cannot specify a default report option in the configuration, so you have to run the applied solution as CEO and select the Revenue amounts report option. This makes the option the default one for the current user.

For user Turner (who has the Technician role) the start page looks as shown in fig. 23.22.

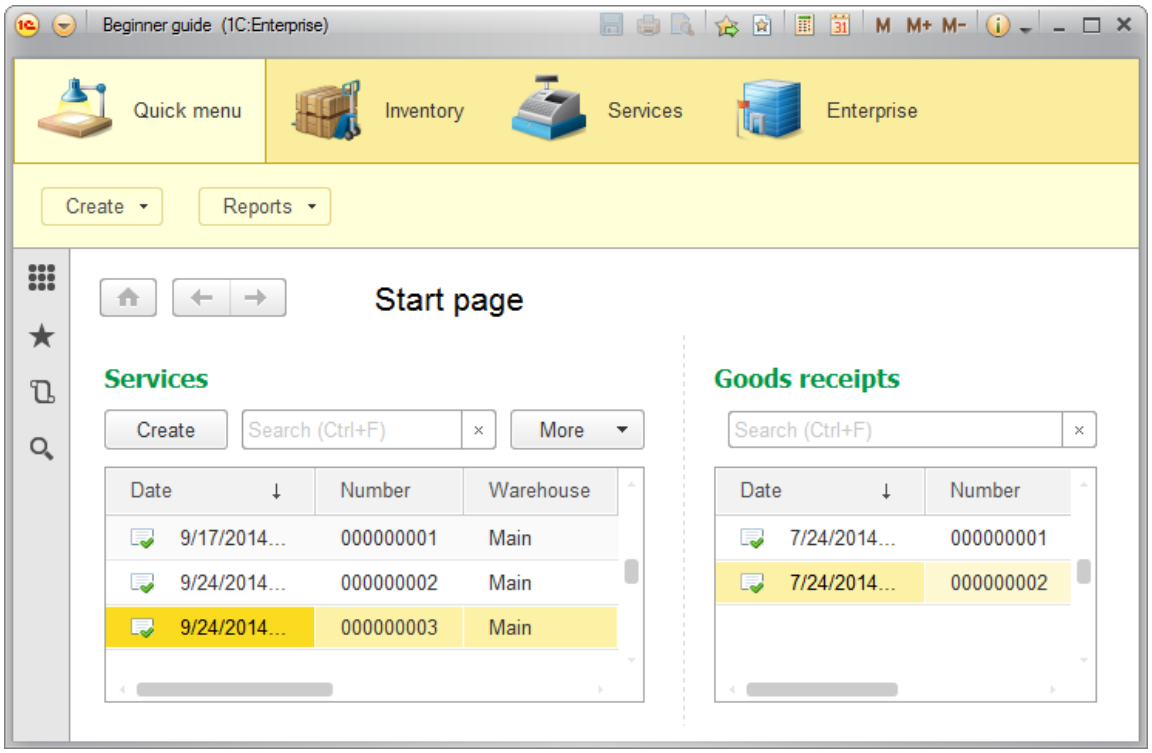


Fig. 23.22. Start page of a user with the Technician role

This user has access to the Inventory, Services, and Enterprise sections in compliance with the rights that you specified for the role in the previous lesson.

When Technician opens a document, the **Start page** button (with a house icon) is available to the left of the document title. This button opens the start page (fig. 23.23).

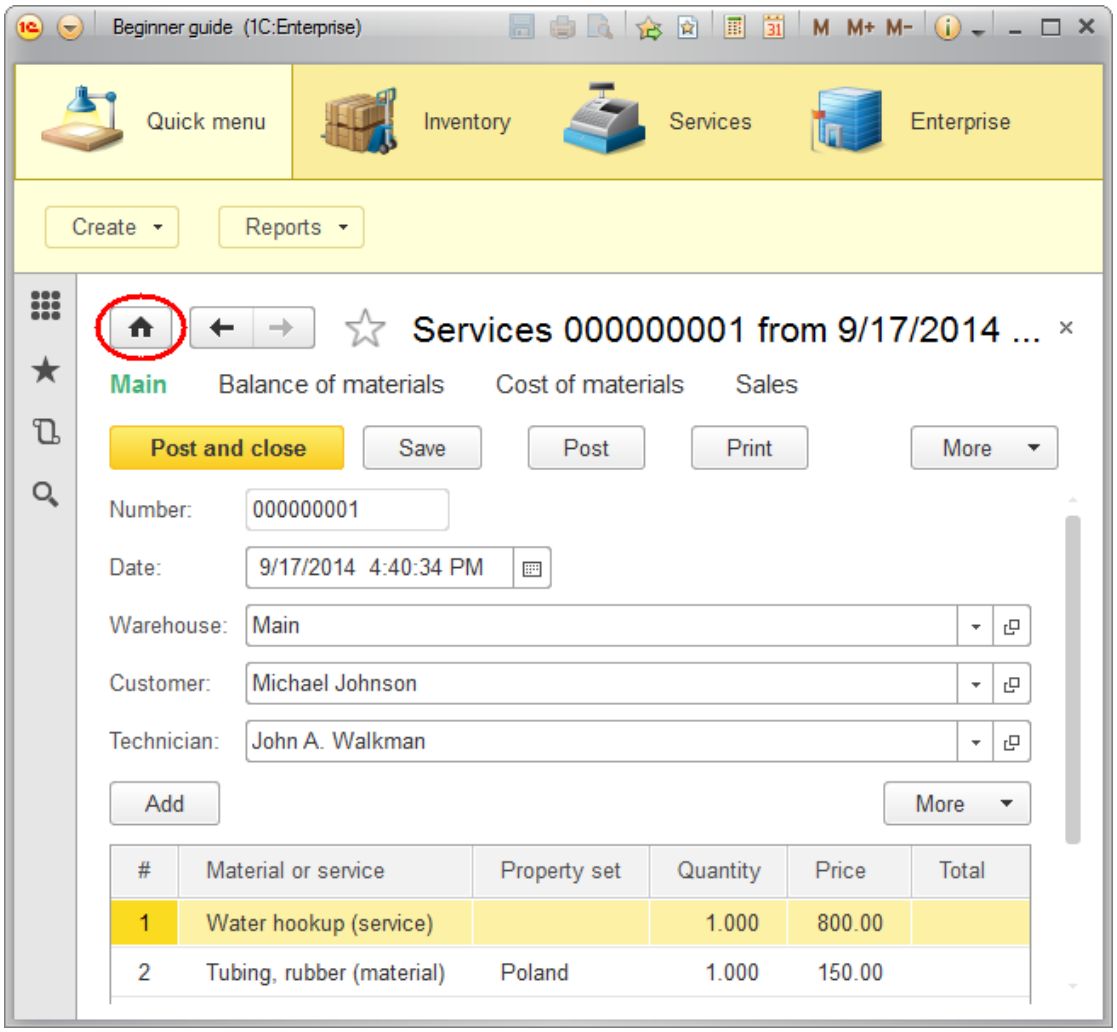


Fig. 23.23. Returning to the start page

So you customized the start page for each role. When you open the applied solution in 1C:Enterprise mode as a user with a specific role (for example, Technician), the start page displays the document list forms that they need for their work. And the Quick menu section commands panel contains the commands that create these documents and open reports that are most important to technicians.

Besides the start page, each user can access the functionalities available in other applied solution sections, provided that they have the rights for this.

And finally, users can customize their start pages (on the main menu, point to **View** and click **Start page** setup). But note that users only can add forms explicitly created by developers to their start pages.

If a user wants to minimize the command interface to expand the work area, they can hide all of the applied solution panels except for the Tools panel (on the main menu, point to **View** and click **Hide all panes**, see fig. 23.24).

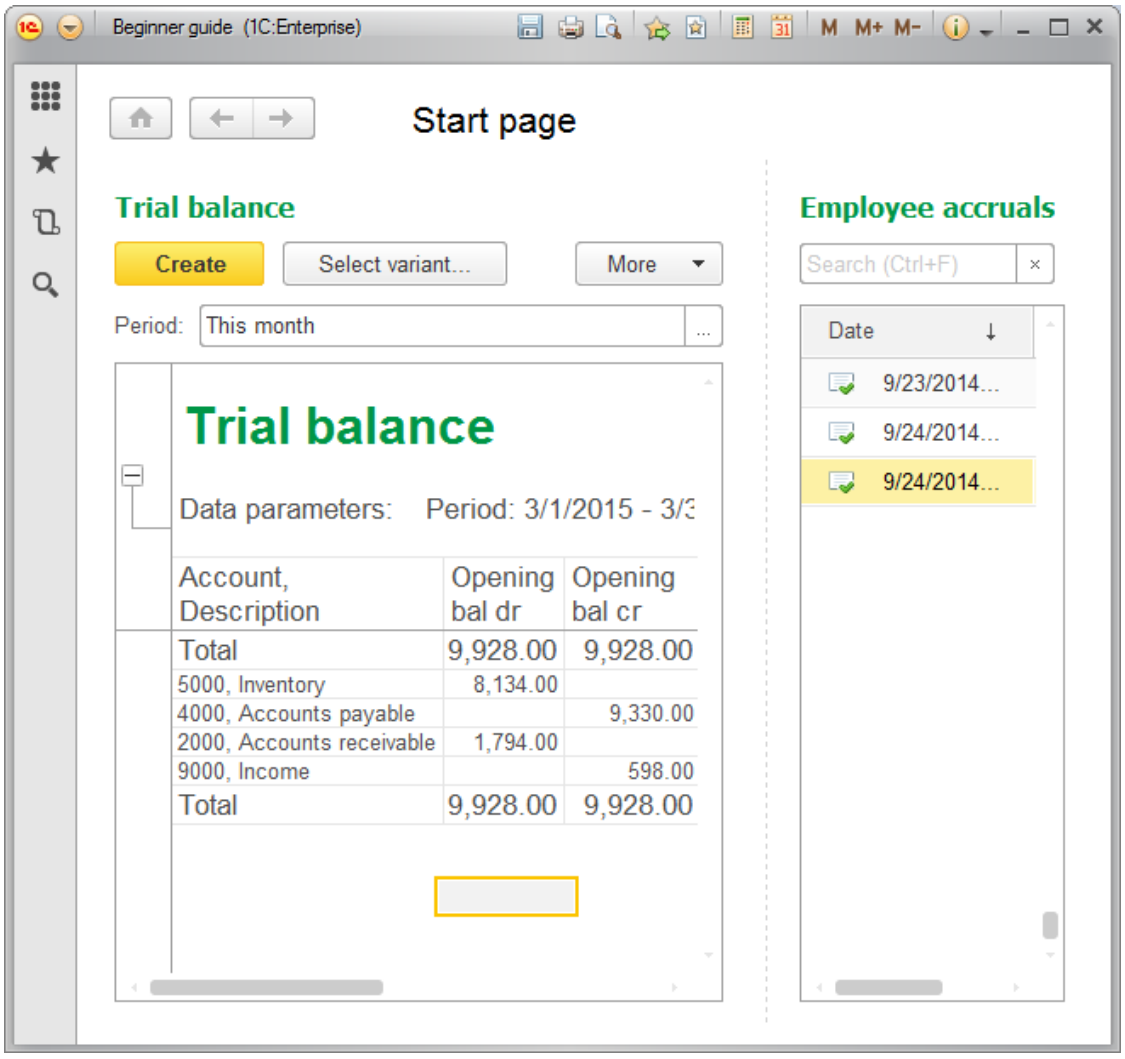


Fig. 23.24. Minimalistic applied solution interface

Of course commercial solutions have different user roles and start page arrangements, which are determined by typical work routines and customer requests.

In the last two lessons we demonstrated the principles of role-based applied solution interface arrangement, as well as user rights-based access restrictions that are applied to individual commands or entire sections.

Quiz

- How does one customize applied solution section command interfaces?
- What is the "Quick menu" section?
- How does one customize the main section command interface?
- What is a start page?
- How does one customize the start page for specific users?
- How does one specify command visibility by roles?

Lesson 24 (6:10). Data exchange

In this lesson you will study the data exchange tools available in 1C:Enterprise, and you will add the option to exchange data with remote branches and departments.

Understanding data exchange

The data exchange feature of 1C:Enterprise is used to exchange data stored in the database with other software systems.

These can be other 1C:Enterprise infobases (and the configurations of participating infobases do not have to be identical), as well as non-1C:Enterprise systems.

Such a flexibility in data exchange is possible because data exchange tools available in 1C:Enterprise can be used in various combinations. Besides, data exchange format is based on the XML language, which is a generally accepted method of data presentation.

The data exchange features include:

- Exchange plans
- XML serialization
- Tools for reading and writing XML documents

Generally speaking, the interaction between these three components can be visualized as shown in fig. 24.1.

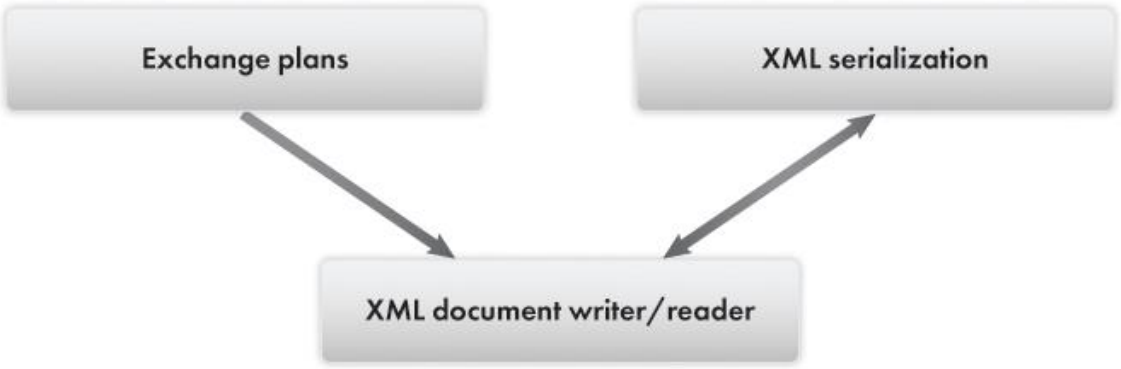


Fig. 24.1. Interaction between data exchange features

Using *exchange plans*, you can get information regarding which data items have been changed and which data exchange node they need to be sent to.

This is possible because exchange plans include a tool that records changes. Data changes are transferred using messages, and exchange plans also support this messaging infrastructure.

XML serialization enables conversion of a 1C:Enterprise object into an XML-formatted data sequence. XML serialization also provides the backward conversion of an XML-formatted data sequence into a 1C:Enterprise object, provided that a matching 1C:Enterprise data type is available.

Reading or writing XML documents is performed using the XML read/write functions of 1C:Enterprise script.

When implementing a data exchange algorithm, a developer can use these features either all together or in various combinations. In each specific case a developer decides which features they want to use.

In the example provided in this book you will use all the three features. So before you proceed to writing the script, let us take a closer look at each of them.

Understanding exchange plans

To implement exchange of any data with any system, you need to somehow identify the entities that will participate in the exchange in that system, and then define an exchange list for each of the entities. This is what the Exchange plan configuration object is used for.

Much like catalog data items are represented by catalog items, exchange plan data items are represented by exchange plan nodes.

Each node represents an exchange participant included in the exchange plan, and each exchange plan contains a predefined node, which represents this infobase.

A single configuration can include multiple exchange plans. Each exchange plan defines a set of data to be exchanged within the context of that plan, as well as the exchange scenario itself.

Multiple exchange plans may be needed when various nodes exchange different sets of data, or when the exchange scenarios for some nodes differ from the exchange scenarios for other nodes.

Learn more! For details on the structure of 1C:Enterprise script objects related to exchange plans, see section "Developer quick reference. Exchange plans" on page 871.

The following entities can participate in data exchange:

- Database objects: catalog items, documents, and so on
- Nonobject data: register record sets, sequence record sets, and constants
- A special 1C:Enterprise script object: ObjectDeletion

We will subsequently refer to these data structure items as *exchange objects*.

A developer can determine the content of each exchange plan by specifying the configuration objects whose data participates in data exchange under this plan.

When you define a data set for an exchange plan, you can specify the AutoRecord property for each object type. It defines how the exchange plan tracks data changes.

Exchange plans track data changes using the *change registration* feature.

This feature is based on the fact that each of the exchange objects has a DataExchange property, which lets you specify the nodes for which changes

to that object are recorded. All changes of an exchange object ultimately boil down to writing or deleting that object. The change registration feature analyzes the writing and deletion of exchange objects and generates change registration records based on the data exchange parameters stored in each exchange object. Note that the DataExchange property is not stored in the database, but is used only while a data exchange object is being written.

So the AutoRecord property, which is set together with the exchange plan data set, means that the data exchange parameters are generated by the change registration feature based on the exchange plan data.

You have the option to modify automatically generated exchange parameters. This is done using the BeforeWrite and BeforeDelete event handlers of the objects that participate in exchanges. You can modify the list of target nodes (the nodes for which changes are recorded) within the handlers.

You can also disable automatic change registration. If you do, you have to generate data exchange parameters from scratch using 1C:Enterprise script tools. This could theoretically be done in any script segment, but to make the configuration easier to understand, it is better to consistently use the BeforeWrite and BeforeDelete event handlers for this purpose, so that the script that generates data exchange parameters is concentrated in straightforward locations instead of being scattered throughout the configuration.

So, now you know that when exchange objects are written or deleted, an exchange plan generates *change registration records*. These records are stored in change registration tables, a separate table is maintained for each exchange object.

When an exchange object is modified, the number of records generated in the change registration table is equal to the number of target nodes specified in the data exchange parameters of that exchange object. Each of the generated records includes a link to its target node. Change registration tables are only created for a metadata object if the object is included in at least one of the exchange plans.

Besides a link to the exchange node for which changes are recorded, each record in the change registration table contains the number of the *message* that initally passed the change to that node. Before the initial message is passed, the field value is Null.

From the exchange plan perspective, a *message* is a basic unit of data exchange. Therefore, one of the most important components of an exchange plan (aside from the change registration services) is the *message infrastructure*.

Since exchange plan messages are transferred from one node to another within an exchange plan, each message is explicitly associated with an exchange plan and has a unique number, a single sender, and a single recipient. The message infrastructure provides message numbering, so that change registration records can store the numbers of the messages that initially transferred such changes.

The message infrastructure also supports sending message delivery confirmations from target nodes. A confirmation is included in each message coming from a target node as a number of the last received message.

Subsequently, by analyzing the number of the last received message and the numbers of the messages stored in the change registration records, the developer can delete change registration records whose delivery is confirmed.

XML serialization

The *XML serialization* term describes a feature that enables conversion of a 1C:Enterprise object into an XML-formatted data sequence. XML serialization also provides the backward conversion of an XML-formatted data sequence into a 1C:Enterprise object, provided that a matching 1C:Enterprise data type is available.

The thing to understand is that each exchange object, which exists as a single object within 1C:Enterprise, is actually comprised of various types of interconnected data.

For example, in addition to its code and description, a catalog item might contain a certain number of attributes of various types and a certain number of tabular sections, which in turn contain a certain number of attributes of various types.

Through XML serialization the entire body of data is represented as an XML-formatted data sequence.

Through backward conversion, the object is reassembled, provided that a matching data type is available in 1C:Enterprise.

Reading and writing XML documents

In contrast to XML serialization, the tools for *reading and writing XML documents* allow you to work with XML data on a basic level, without mapping that data to 1C:Enterpise objects.

Specifically, they allow you to open XML files for reading, read data from these files, create new XML files, and write data to these files.

Universal data exchange

Suppose that Jack of All Trades established a branch in town N, and installed a configuration identical to yours to manage the branch operations.

You need to establish data exchange between these two databases, so that each database stores full information about materials and services, while accounting and payroll are managed independently in each database.

To do so, you will create an exchange plan, specify which data is included in the exchange, and create a number of procedures that write exchange files to the hard disk and load data from these exchange files.

To simplify the example, you will not develop any automatic file exchange between the two databases. Instead, you will start the exchange procedure manually.

Before you start programming the exchange algorithm, let us discuss some enhancements that you need to make in your database to prepare it for the exchange.

These enhancements are related to the fact that, up to this point, you have only worked with a single database, and have relied on uniqueness of catalog code numbers and document numbers. Now, new catalog items and new documents will be created independently in two databases at the same time, and you still need to ensure uniqueness of catalog item code numbers and document numbers, but this time it's across two databases.

If you do not do that, there can be a situation when new documents created in both infobases have identical numbers. This will cause a data exchange conflict because the platform attempts to write a document to the database using a number that is already used by another document.

To prevent such situations, let us add a unique prefix that unambiguously identifies the database to document numbers and catalog codes.

Hence even if the numbers of new documents in the two databases match, they will still have different prefixes and no conflict will occur.

You will use the Constant configuration object to store number prefixes.

Creating a constant for data exchange

In Designer mode

Attention! Since during the previous lesson you created a list of users in the configuration, when you start Designer you are prompted for a user name. Enter Administrator as the user name because this user has full access to all the configuration objects. There is no need to enter a password because you have not created one.

The Constant configuration object is used to create database tables for storing data that is never changed over time or is rarely changed.

Each Constant configuration object defines a table that stores a single value.

Let us proceed to creating a constant for storing the number prefix value.

1. Open Designer and create a Constant configuration object named *NumberingPrefix*.
2. Set its data type to **String** with a fixed length of **2** characters.

Updating configuration objects involved in data exchange

In Designer mode

The first thing you need to do is modify the modules of all the objects that are involved in exchanges (these objects include documents, catalogs, and charts of characteristic types).

After the changes document numbers, catalog codes, and chart of accounts codes will be generated based on the value of the NumberingPrefix constant to ensure that such numbers and codes are unique.

Let us keep the function that generates the numbering prefix in a common module because in the future you might need to change the algorithm used to generate document prefixes.

1. Add a common module named *Exchange*.
2. Add the function shown in listing 24.1 to the module.

   Listing 24.1. Function that generates a number prefix

   ```
   Function GetNumberingPrefix() Export
       Return Constants.NumberingPrefix.Get();
   EndFunction
   ```

   As you can see, the function merely returns the value of the NumberingPrefix constant.

   Now let us update the Customers catalog.

3. In the configuration object tree, right-click the **Customers** catalog and then click **Open object module**.
4. Add the OnSetNewCode event handler as shown in listing 24.2.

   Listing 24.2. OnSetNewCode event handler

   ```
   Procedure OnSetNewCode(StandardProcessing, Prefix)
       Prefix = Exchange.GetNumberingPrefix();
   EndProcedure
   ```

   The OnSetNewCode event occurs when a new code is set for a catalog item. Note that you are creating this script in the object module instead of the form module because this event occurs for the object in general instead of some particular form.

   Prefix is the second parameter passed to the handler. A prefix is populated in this procedure, and the platform generates a code based on the prefix.

   The event handler calls a function of a common module. Since the module is not global, you address the function using the module name and the function name: Exchange.GetNumberingPrefix. In this procedure you assign the value of the NumberingPrefix constant to the prefix.

5. Add similar handlers to all the catalogs and charts of characteristic types involved in exchanges:

   Catalogs:

   - **Employees**
   - **MaterialsAndServices**
   - **Warehouses**
   - **MaterialOptions**
   - **AdditionalMaterialProperties**

   Chart of characteristic types:

   - **MaterialProperties**

6. For all these objects and for the **Customers** catalog, in the configuration object property palette increase code length to **11** characters.

   Now let us update the documents.

7. In the **GoodsReceipt** document module, add the OnSetNewNumber event handler as shown in listing 24.3.

   Listing 24.3. OnSetNewNumber event handler

```
Procedure OnSetNewNumber(StandardProcessing, Prefix)
    Prefix = Exchange.GetNumberingPrefix();
EndProcedure
```

8. Add similar handlers to all the documents involved in exchanges (in your configuration it is only the **Services** document).
9. For both documents, in the configuration object property palette increase the number length to **11** characters.

   This completes the preparations for existing configuration objects, and you can move on to the development of data exchange procedures.

Adding an exchange plan

In Designer mode

Let us create the central part of a data exchange algorithm: the exchange plan.

1. In the configuration object tree, expand the Common branch and add an ExchangePlan configuration object named *Branches*.
2. In the **Object presentation** field, enter *Branch*.
3. On the **Data** tab, create an exchange plan attribute named *Main*, of the **Boolean** type (fig. 24.2).

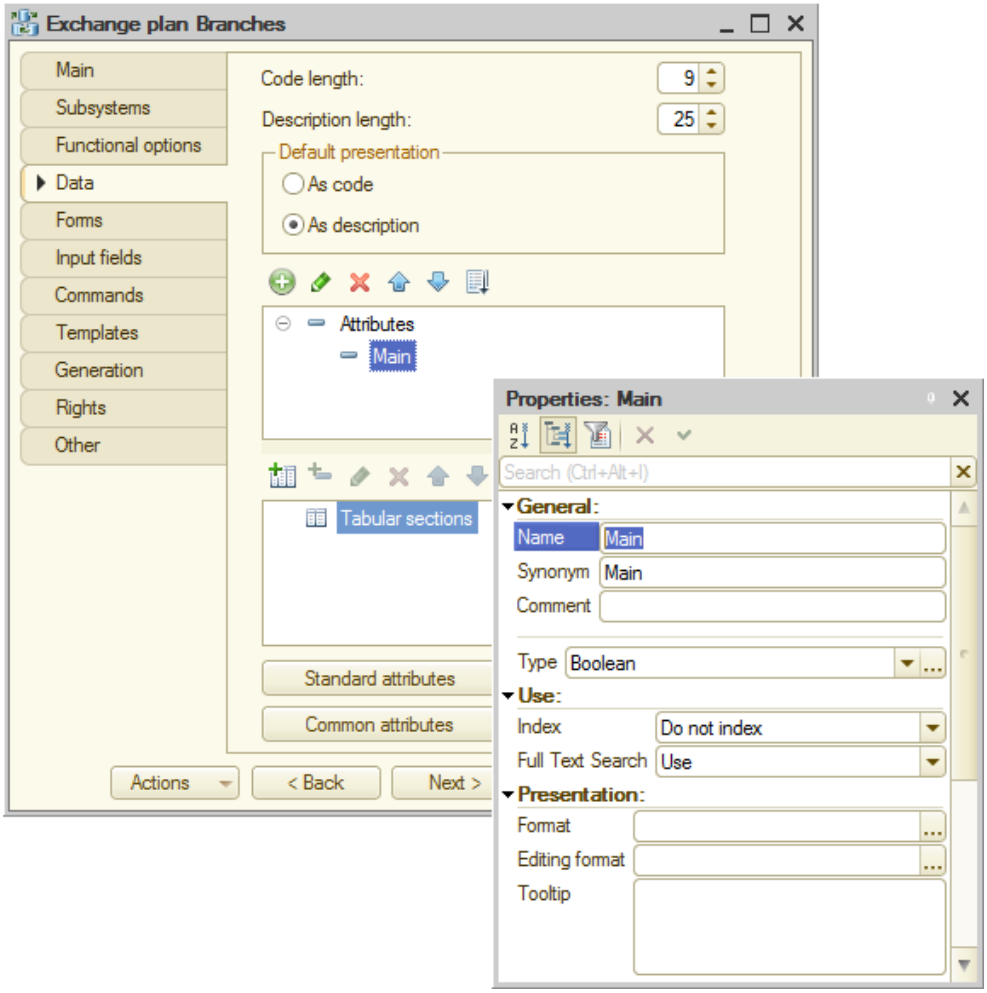


Fig. 24.2. Exchange plan attribute

You will need this attribute to resolve collisions during data exchanges. A collision is a situation where a data exchange object is simultaneously modified in two nodes.

In this case you will look into the value of the Main attribute and confirm the changes only if they are made in the main node. In the event of a collision, you will reject the changes made anywhere but in the main node.

Let us define the set of objects that participate in the exchange.

4. On the **Main** tab, click **Content**.
5. Include all the objects not related to accounting and payroll calculation into the exchange (fig. 24.3).

   Note that the NumberingPrefix constant is not included in the exchange, since its value should be unique for each database that takes part in the exchange.

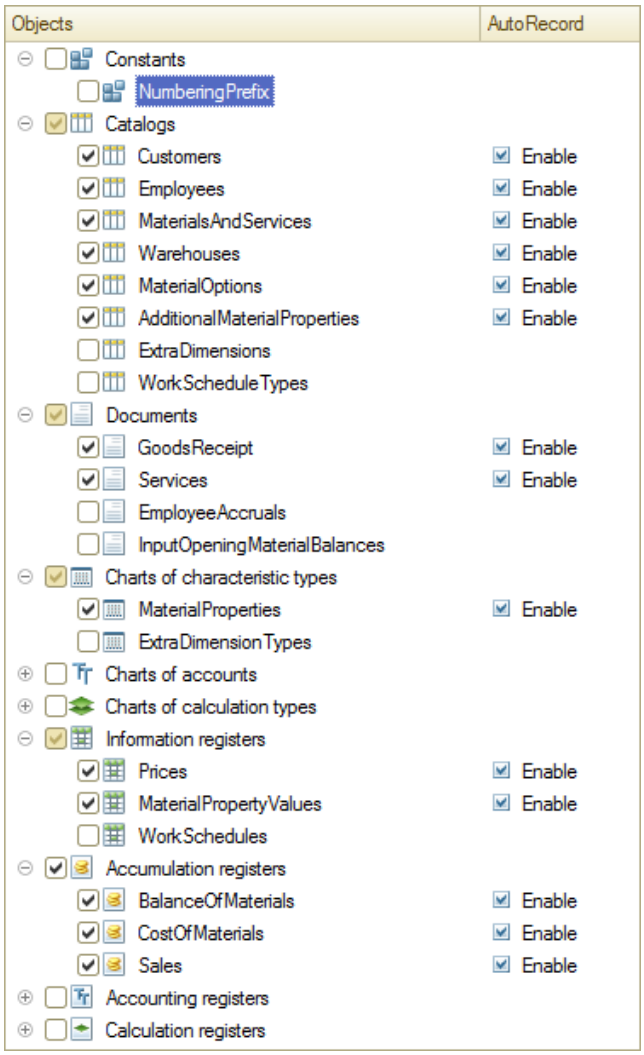


Fig. 24.3. Data exchange content

6. On the **Forms** tab, in the **Node** field, click the **Open** button and create the default node form (fig. 24.4).

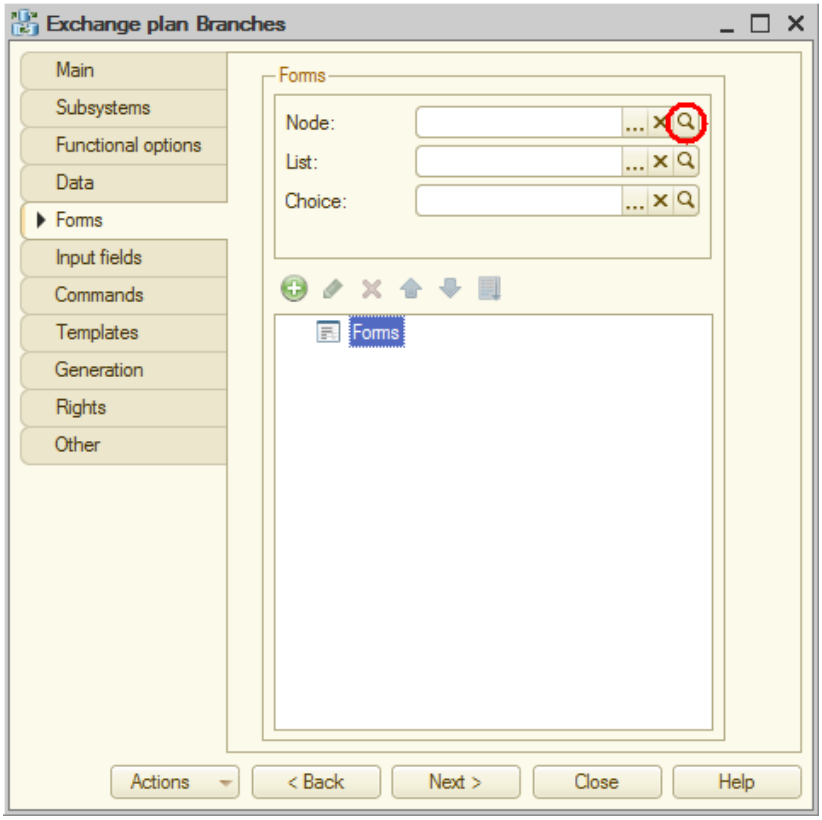


Fig. 24.4. Creating an exchange plan node form

7. In the form controls pane, right-click the **Form** root item, point to **Events**, and click **<OnCreateAtServer>**.

   This creates the OnCreateAtServer event handler template. You need this handler to prohibit setting the Main attribute for the predefined node that corresponds to this infobase.

8. Add the script shown in listing 24.4 to the handler.

   Listing 24.4. OnCreateAtServer form event handler

```
&AtServer
Procedure OnCreateAtServer(Cancel, StandardProcessing)

    If Object.Ref = ExchangePlans.Branches.ThisNode() Then
        Items.Main.Enabled = False;
    EndIf;

EndProcedure
```

   This procedure calls the ThisNode() method of the exchange plan manager, which returns a reference to the exchange plan node that corresponds to the current infobase.

9. Create a default exchange plan list form.

   You need this form to define some operations related to registering a new exchange node in this form.

   The main goal of these operations is generating all the change registration records for all configuration objects included in the

exchange plan during the registration of a new exchange node. This basically serves as initial synchronization of the exchange node with all the exchange data.

10. On the **Commands** tab, create a command named *WriteChanges*.
11. In the property palette that is opened, in the **Action** field, click the **Open** button.
12. In the dialog box that propmts you to select handler type, click **Create on client and a procedure on server (no context)**.

    This creates two procedures in the form module: a client WriteChanges() procedure and a server out-of-context WriteChangesAtServer procedure, which is called from the client procedure.

    As you already know, out-of-context server procedures are executed much faster than those having a context because they do not require passing the entire form context from the client to the server.

    You need to pass a reference to the ExchangePlan.Branches object as a parameter of the WriteChangesAtServer procedure. You can do it using the CurrentRow property of the List table (the table uses a dynamic list of nodes of the Branches exchange plan as its data source).
13. Update the module text as shown in listing 24.5.

Listing 24.5. WriteChanges() command event handler

```
&AtServerNoContext
Procedure WriteChangesAtServer(Node)
    // Insert handler contents.
EndProcedure


&AtClient
Procedure WriteChanges(Command)
    WriteChangesAtServer(Items.List.CurrentRow);
EndProcedure
```

14. Add the script shown in listing 24.6 to the WriteChangesAtServer() procedure.

Listing 24.6. WriteChangesAtServer() procedure

```
&AtServerNoContext
    Procedure WriteChangesAtServer(Node)
    // Writing all data changes for a node
    ExchangePlans.RecordChanges(Node);
EndProcedure
```

This procedure utilizes the change registration feature by calling the RecordChanges() method of the exchange plan manager.

A reference to the current exchange plan node (Branches) is passed to this method.

The procedure execution creates change registration records in the infobase. These records include the changes that will be sent to the new infobase for each object included in the exchange plan content.

15. Click the **Form** tab and drag the **WriteChanges** command from the **Commands** tab to the form command bar (fig. 24.5).

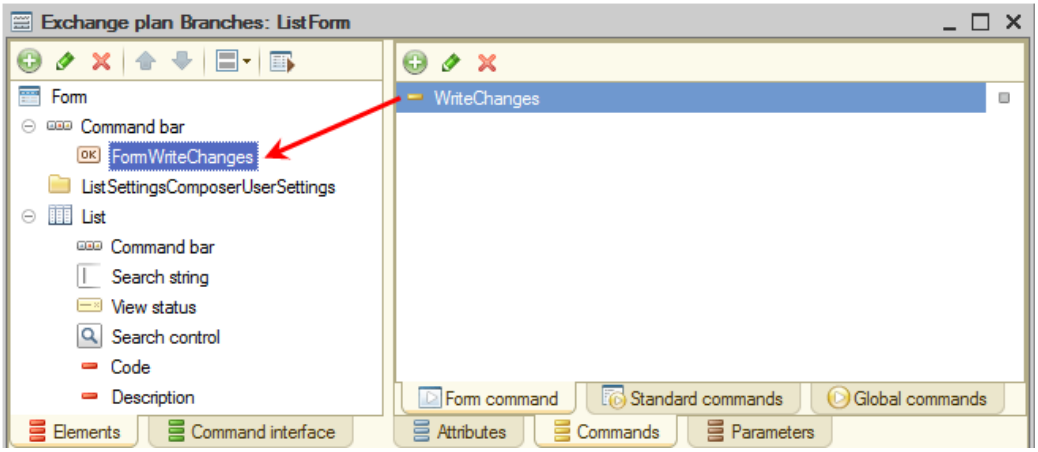


Fig. 24.5. Exchange plan list form editor

Note that the command name in the tree of form items does not exactly match the actual command name. The platform added the "Form" prefix to it, and the button linked to the command also has this name. You will use this name to access the button from 1C:Enterprise script (see listing 24.8).

Also note that the button should only be available if the current node is not a predefined node for this infobase because otherwise it is impossible to record changes.

To ensure such button behavior, let us create a function in the list form module that is executed on the server and returns True when the node passed to the function is a predefined one.

16. Add the function shown in listing 24.7 to the list form module.

Listing 24.7. PredefinedNode() function

```
&AtServerNoContext
Function PredefinedNode(Node)
    Return Node = ExchangePlans.Branches.ThisNode();
EndFunction
```

17. In the form controls pane, right-click the **List** item, open its property palette, and create the OnActivateRow event handler that is executed on the client.

18. Populate the event handler as shown in listing 24.8.

Listing 24.8. OnActivateRow() event handler of a list form item

```
&AtClient
Procedure ListOnActivateRow(Item)

    If PredefinedNode(Item.CurrentRow) Then
```

```
        Items.FormWriteChanges.Enabled = False;

    Else
        Items.FormWriteChanges.Enabled = True;

    EndIf;

EndProcedure
```

This procedure sets the availability of the WriteChanges button based on the return value of the PredefinedNode() function, which is in turn based on the reference to the current node (Item.CurrentRow) passed to the function.

This completes the exchange plan creation and now you can proceed to the creation of data exchange procedures.

Creating data exchange procedures

In Designer mode

Let us create a data processor that starts data exchange.

1. In the configuration object tree, add a DataProcessor configuration object named *DataExchange*.
2. On the **Forms** tab, create a default data processor form.
3. In the form editor, on the **Commands** tab, create a form command named *StartDataExchange*.
4. In the command property palette, in the **Action** field, click the **Open** button.
5. In the dialog box that prompts you to select handler type, click **Create on client and a procedure on server (no context)**.

   This creates two procedures in the form module: a client StartDataExchange() procedure and a server out-of-context StartDataExchangeAtServer procedure, which is called from the client procedure.

   The StartDataExchange() procedure has the following text (listing 24.9).

   Listing 24.9. StartDataExchange() command handler

```
&AtClient
Procedure StartDataExchange(Command)
    StartDataExchangeAtServer();
EndProcedure
```

6. Add the following script to the StartDataExchangeAtServer procedure (listing 24.10).

Listing 24.10. Creating the StartDataExchangeAtServer procedure

```
&AtServerNoContext
Procedure StartDataExchangeAtServer() Export

    NodeSelection = ExchangePlans.Branches.Select();

    While NodeSelection.Next() Do

        // Exchanging data with all nodes, except for the current
        // node (ThisNode)
        If NodeSelection.Ref <> ExchangePlans.Branches.ThisNode() Then
            NodeObject = NodeSelection.GetObject();

            // Receiving message
            NodeObject.ReadMessageWithChanges();

            // Generating message
            NodeObject.WriteMessageWithChanges();

        EndIf;
    EndDo;

EndProcedure
```

The procedure has the following algorithm: in the loop it iterates through the nodes of the Branches exchange plan, and for all the nodes except the local node it first reads the messages received from other exchange nodes (you will create the ReadMessagesWithChanges procedure later).

Next it generates the messages to be sent that contain the modified data for this node (you will create the WriteMessagesWithChanges procedure later).

7. Drag the **StartDataExchange** command from the **Commands** tab to the form controls pane.

   The resulting data processor form should look as follows (fig. 24.6).

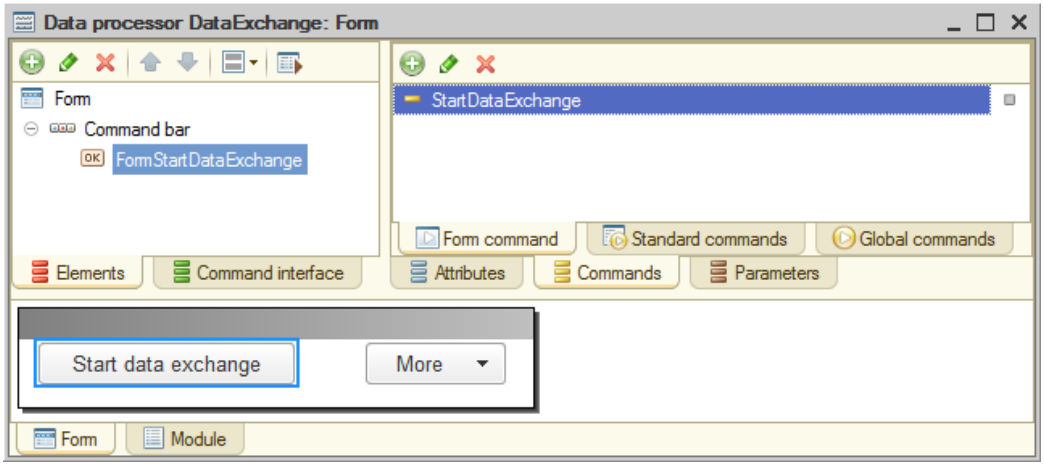


Fig. 24.6. Data processor form

Data writing procedure

Let us store the procedures that write and read exchange data in the object module of the exchange plan.

1. In the editor of the **Branches** exchange plan configuration object, click the **Other** tab and then click **Object module**.

   Let us create the WriteMessageWithChanges procedure. This will take several steps.

   First, let us implement the name generation for the file that will store the exchange data, and user notifications for the beginning and end of data export.

2. Add the script shown in listing 24.11 to the module.

   Listing 24.11. Generating a file name in the data writing procedure

```
Procedure WriteMessageWithChanges() Export


    Message = New UserMessage;

    Message.Text = "-------- Starting export to node: "
        + String(ThisObject) + " --------";

    Message.Message();

    Directory = TempFilesDir();


    // Generating temporary file name

    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + TrimAll(ExchangePlans.Branches.ThisNode().Code)
        + "_" + TrimAll(Ref.Code) + ".xml";

    Message = New UserMessage;

    Message.Text = "-------- Export completed --------";

    Message.Message();


EndProcedure
```

To simplify the example, let us exchange messages through a temporary files directory. The message names are standardized and follow the pattern: MessageSourceNodeCode_TargetNodeCode.xml.

Now let us use the XML document read/write feature. The next procedure part creates an XMLWriter object. Then it uses the created object for opening a new XML file for writing and writes an XML declaration to this file. Finally, it closes the file.

3. Update the procedure as shown in listing 24.12.

Listing 24.12. Creating an XMLWriter object in the data writing procedure

```
Procedure WriteMessageWithChanges() Export


    Message = New UserMessage;

    Message.Text = "-------- Starting export to node: "
        + String(ThisObject) + " --------";

    Message.Message();

    Directory = TempFilesDir();


    // Generating temporary file name

    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")/
        + "Message" + TrimAll(ExchangePlans.Branches.ThisNode().Code)
        + "_" + TrimAll(Ref.Code) + ".xml";


    // Creating XMLWriter object

    // *** Writing XML documents

    XMLWriter = New XMLWriter;

    XMLWriter.OpenFile(FileName);

    XMLWriter.WriteXMLDeclaration();

    XMLWriter.Close();


    Message = New UserMessage;

    Message.Text = "-------- Export completed --------";

    Message.Message();


EndProcedure
```

Now let us implement the message infrastructure. The next procedure part creates an ExchangeMessageWriter object. Its BeginWrite() method creates the next message number and writes the message header to the XML file. Finally, the procedure ends the message writing.

4. Update the procedure as shown in listing 24.13.

Listing 24.13. Creating the message sequential number and writing the message header to the XML file

```
Procedure WriteMessageWithChanges() Export


    Message = New UserMessage;
```

```
    Message.Text = "-------- Starting export to node: "
        + String(ThisObject) + " --------";

    Message.Message();

    Directory = TempFilesDir();


    // Generating temporary file name

    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + TrimAll(ExchangePlans.Branches.ThisNode().Code)
        + "_" + TrimAll(Ref.Code) + ".xml";


    // Creating XMLWriter object

    // *** Writing XML documents

    XMLWriter = New XMLWriter;

    XMLWriter.OpenFile(FileName);

    XMLWriter.WriteXMLDeclaration();


    // *** Message infrastructure

    ExchangeMessageWriter = ExchangePlans.CreateMessageWriter();

    ExchangeMessageWriter.BeginWrite(XMLWriter, Ref);

    Message = New UserMessage;

    Message.Text = "Message number: "
        + ExchangeMessageWriter.MessageNo;

    Message.Message();

    ExchangeMessageWriter.EndWrite();


    XMLWriter.Close();


    Message = New UserMessage;

    Message.Text = "-------- Export completed --------";

    Message.Message();


EndProcedure
```

Since the procedure is located in the object module, you can use the standard Ref attribute as a reference to the Branches exchange plan object.

In order to get the data for storing to the file, let us use the change registration feature. The next procedure part gets a selection of change registration records intended for this node. The method that generates the selection accepts the message number as its second parameter.

5. Update the procedure as shown in listing 24.14.

Listing 24.14. Getting a selection of change registration records intended for this node

```
Procedure WriteMessageWithChanges() Export


    Message = New UserMessage;
```

```
    Message.Text = "-------- Starting export to node: "
        + String(ThisObject) + " --------";

    Message.Message();

    Directory = TempFilesDir();


    // Generating temporary file name

    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + TrimAll(ExchangePlans.Branches.ThisNode().Code)
        + "_" + TrimAll(Ref.Code) + ".xml";


    // Creating XMLWriter object

    // *** Writing XML documents

    XMLWriter = New XMLWriter;

    XMLWriter.OpenFile(FileName);

    XMLWriter.WriteXMLDeclaration();


    // *** Message infrastructure

    ExchangeMessageWriter = ExchangePlans.CreateMessageWriter();

    ExchangeMessageWriter.BeginWrite(XMLWriter, Ref);

    Message = New UserMessage;

    Message.Text = "Message number: "
        + ExchangeMessageWriter.MessageNo;

    Message.Message();


    // Getting the selection of changed data

    // *** Change registration

    SelectionOfChanges =
        ExchangePlans.SelectChanges(ExchangeMessageWriter.Recipient,
        ExchangeMessageWriter.MessageNo);


    ExchangeMessageWriter.EndWrite();

    XMLWriter.Close();


    Message = New UserMessage;

    Message.Text = "-------- Export completed --------";

    Message.Message();


EndProcedure
```

All that is left to do is adding a loop for selecting records and serializing them to the XML file that is open.

6. Update the procedure as shown in listing 24.15.

Listing 24.15. Iterating through the record selection and serializing records to the XML file that is open

```
Procedure WriteMessageWithChanges() Export


    Message = New UserMessage;
```

```
    Message.Text = "-------- Starting export to node: "
        + String(ThisObject) + " --------";

    Message.Message();

    Directory = TempFilesDir();


    // Generating temporary file name

    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + ErimAll(ExchangePlans.Branches.ThisNode().Code)
        + "_" + TrimAll(Ref.Code) + ".xml";


    // Creating XMLWriter object

    // *** Writing XML documents

    XMLWriter = New XMLWriter;

    XMLWriter.OpenFile(FileName);

    XMLWriter.WriteXMLDeclaration();


    // *** Message infrastructure

    ExchangeMessageWriter = ExchangePlans.CreateMessageWriter();

    ExchangeMessageWriter.BeginWrite(XMLWriter, Ref);

    Message = New UserMessage;

    Message.Text = "Message number: "
        + ExchangeMessageWriter.MessageNo;

    Message.Message();


    // Getting the selection of changed data

    // *** Change registration

    SelectionOfChanges =
        ExchangePlans.SelectChanges(ExchangeMessageWriter.Recipient,
        ExchangeMessageWriter.MessageNo);


    While SelectionOfChanges.Next() Do

        // Writing data to the message

        // ***XML serialization

        WriteXML(XMLWriter, SelectionOfChanges.Get());

    EndDo;


    ExchangeMessageWriter.EndWrite();

    XMLWriter.Close();


    Message = New UserMessage;

    Message.Text = "-------- Export completed --------";

    Message.Message();


EndProcedure
```

This completes the procedure for writing exchange data.

Data reading procedure

The steps for creating the procedure that reads exchange data will be similar to the steps for creating the procedure that writes exchange data. First, let us generate the name of the file that will store exchange data.

1. Add the procedure shown in listing 24.16 to the Branches configuration object module.

Listing 24.16. Generating a data exchange file name

```
Procedure ReadMessageWithChanges() Export

    Directory = TempFilesDir();

    // Generating file name
    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + ErimAll(Ref.Code) + "_"
        + TrimAll(ExchangePlans.Branches.ThisNode().Code) + ".xml";
    File = New File(FileName);
    If Not File.Exist() Then
        Return;
    EndIf;

    DeleteFiles(FileName);

    Message = New UserMessage;
    Message.Text = "-------- Import completed --------";
    Message.Message();

EndProcedure
```

The procedure generates the name of the file that it expects to find in the directory and then creates a File object with that name and checks whether the file exists. If there is no such file, the procedure execution is terminated. If the file is found, the procedure deletes the file once it processes all its data.

Let us add the procedure part that reads the exchange data file.

2. Update the procedure as shown in listing 24.17.

Listing 24.17. Adding the reading of the exchange data file

```
Procedure ReadMessageWithChanges() Export

    Directory = TempFilesDir();

    // Generating file name
    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + TrimAll(Ref.Code) + "_"
        + TrimAll(ExchangePlans.Branches.ThisNode().Code) + ".xml";
```

```
    File = New File(FileName);
    If Not File.Exist() Then
        Return;
    EndIf;

    // *** Reading XML document
    // Trying to open the file
    XMLReader = New XMLReader;
    Try
        XMLReader.OpenFile(FileName);

    Except
        Message = New UserMessage;
        Message.Text = "Cannot open the data exchange file.";
        Message.Message();

        Return;

    EndTry;

    Message = New UserMessage;
    Message.Text = "-------- Starting import from: "
        + String(ThisObject) + "---------";
    Message.Message();
    Message = New UserMessage;
    Message.Text = "- Reading file: " + FileName;
    Message.Message();

    XMLReader.Close();

    DeleteFiles(FileName);

    Message = New UserMessage;
    Message.Text = "-------- Import completed --------";
    Message.Message();

EndProcedure
```

The added procedure part utilizes the XML document read/write feature, which works with files at the basic level.

First, it creates an XMLReader object that opens the file for reading. If the file is successfully opened, a message is displayed informing the user that the data import begins. At the end the procedure stops reading data from the XML file using the Close() method.

The data received in this manner should be some sort of data exchange message. To present such data in message terms, let us add the next procedure part.

3. Update the procedure as shown in listing 24.18.

Listing 24.18. Adding the reading of XML message header

```
Procedure ReadMessageWithChanges() Export


    Directory = TempFilesDir();


    // Generating file name
    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + TrimAll(Ref.Code) + "_"
        + TrimAll(ExchangePlans.Branches.ThisNode().Code) + ".xml";
    File = New File(FileName);
    If Not File.Exist() Then
        Return;
    EndIf;


    // *** Reading XML document
    // Trying to open the file
    XMLReader = New XMLReader;
    Try
        XMLReader.OpenFile(FileName);


    Except
        Message = New UserMessage;
        Message.Text = "Cannot open the data exchange file.";
        Message.Message();


        Return;


    EndTry;


    Message = New UserMessage;
    Message.Text = "-------- Starting import from: "
        + String(ThisObject) + "---------";
    Message.Message();
    Message = New UserMessage;
    Message.Text = "- Reading file: " + FileName;
    Message.Message();


    // Importing data from the file
    // *** Message infrastructure
    ExchangeMessageReader = ExchangePlans.CreateMessageReader();
```

```
    // Reading the data exchange message header from the XML file
    ExchangeMessageReader.BeginRead(XMLReader);


    ExchangeMessageReader.EndRead();


    XMLReader.Close();


    DeleteFiles(FileName);


    Message = New UserMessage;
    Message.Text = "-------- Import completed --------";
    Message.Message();


EndProcedure
```

Here the procedure utilizes the exchange plan message infrastructure and creates an ExchangeMessageReader object. The BeginRead() method of this object reads the XML message header, which contains information about the message sender among other data. Once the entire message is processed, the reading is stopped.

Now that exchange data is presented as a message and the header is received, let us make one more check before actual data processing begins.

4. Update the procedure as shown in listing 24.19.

Listing 24.19. Adding a message check

```
Procedure ReadMessageWithChanges() Export


    Directory = TempFilesDir();


    // Generating file name
    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + TrimAll(Ref.Code) + "_"
        + TrimAll(ExchangePlans.Branches.ThisNode().Code) + ".xml";
    File = New File(FileName);
    If Not File.Exist() Then
        Return;
    EndIf;


    // *** Reading XML document
    // Trying to open the file
    XMLReader = New XMLReader;
    Try
        XMLReader.OpenFile(FileName);


    Except
```

```
        Message = New UserMessage;
        Message.Text = "Cannot open the data exchange file.";
        Message.Message();

        Return;

    EndTry;

    Message = New UserMessage;
    Message.Text = "-------- Starting import from: "
        + String(ThisObject) + "---------";
    Message.Message();
    Message = New UserMessage;
    Message.Text = "- Reading file: " + FileName;
    Message.Message();

    // Importing data from the file
    // *** Message infrastructure
    ExchangeMessageReader = ExchangePlans.CreateMessageReader();

    // Reading the data exchange message header from the XML file
    ExchangeMessageReader.BeginRead(XMLReader);

    // The message is not for this node
    If ExchangeMessageReader.Sender <> Ref Then
        Raise "Incorrect node";
    EndIf;

    ExchangeMessageReader.EndRead();
    XMLReader.Close();
    DeleteFiles(FileName);

    Message = New UserMessage;
    Message.Text = "-------- Import completed --------";
    Message.Message();

EndProcedure
```

Here the procedure checks whether the message sender is exactly the exchange plan node processed by the current procedure call.

If the node is correct, before actual data reading begins, you have to delete all the change registration records made for this node that correspond to message numbers less than or equal to the received message number specified in the message being processed. The idea is to prevent duplicating data that is already sent to the node and processed by it.

5. Update the procedure as shown in listing 24.20.

Listing 24.20. Deleting change registration records for the source node

```
Procedure ReadMessageWithChanges() Export

    Directory = TempFilesDir();

    // Generating file name
    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + TrimAll(Ref.Code) + "_"
        + TrimAll(ExchangePlans.Branches.ThisNode().Code) + ".xml";
    File = New File(FileName);
    If Not File.Exist() Then
        Return;
    EndIf;

    // *** Reading XML document
    // Trying to open the file
    XMLReader = New XMLReader;
    Try
        XMLReader.OpenFile(FileName);

    Except
        Message = New UserMessage;
        Message.Text = "Cannot open the data exchange file.";
        Message.Message();

        Return;

    EndTry;

    Message = New UserMessage;
    Message.Text = "-------- Starting import from: "
        + String(ThisObject) + "---------";
    Message.Message();
    Message = New UserMessage;
    Message.Text = "- Reading file: " + FileName;
    Message.Message();

    // Importing data from the file
    // *** Message infrastructure
    ExchangeMessageReader = ExchangePlans.CreateMessageReader();

    // Reading the data exchange message header from the XML file
    ExchangeMessageReader.BeginRead(XMLReader);
```

```
    // The message is not for this node
    If ExchangeMessageReader.Sender <> Ref Then
        Raise "Incorrect node";
    EndIf;

    // Deleting change records for the source node
    // *** Change registration service
    ExchangePlans.DeleteChangeRecords(ExchangeMessageReader.Sender,
        ExchangeMessageReader.ReceivedNo);

    ExchangeMessageReader.EndRead();
    XMLReader.Close();
    DeleteFiles(FileName);

    Message = New UserMessage;
    Message.Text = "-------- Import completed --------";
    Message.Message();

EndProcedure
```

Note that the new procedure part accesses the change registration service and uses the DeleteChangeRecords() method to delete the messages.

Now you can proceed to reading the data stored in the message.

6. Update the procedure as shown in listing 24.21.

Listing 24.21. Reading the message data

```
Procedure ReadMessageWithChanges() Export

    Directory = TempFilesDir();

    // Generating file name
    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + TrimAll(Ref.Code) + "_"
        + TrimAll(ExchangePlans.Branches.ThisNode().Code) + ".xml";
    File = New File(FileName);
    If Not File.Exist() Then
        Return;
    EndIf;

    // *** Reading XML document
    // Trying to open the file
    XMLReader = New XMLReader;
    Try
        XMLReader.OpenFile(FileName);
```

```
Except
    Message = New UserMessage;
    Message.Text = "Cannot open the data exchange file.";
    Message.Message();

    Return;

EndTry;

Message = New UserMessage;
Message.Text = "-------- Starting import from: "
    + String(ThisObject) + "---------";
Message.Message();
Message = New UserMessage;
Message.Text = "- Reading file: " + FileName;
Message.Message();

// Importing data from the file
// *** Message infrastructure
ExchangeMessageReader = ExchangePlans.CreateMessageReader();

// Reading the data exchange message header from the XML file
ExchangeMessageReader.BeginRead(XMLReader);

// The message is not for this node
If ExchangeMessageReader.Sender <> Ref Then
    Raise "Incorrect node";
EndIf;

// Deleting change records for the source node
// *** Change registration service
ExchangePlans.DeleteChangeRecords(ExchangeMessageReader.Sender,
    ExchangeMessageReader.ReceivedNo);

// Reading data from the message
// *** XML serialization
While CanReadXML(XMLReader) Do

EndDo;

ExchangeMessageReader.EndRead();
XMLReader.Close();
DeleteFiles(FileName);

Message = New UserMessage;
```

```
    Message.Text = "-------- Import completed --------";
    Message.Message();


EndProcedure
```

The data reading is executed in the loop and XML serialization is involved again. The CanReadXML() global context method gets the next XML data type from the XMLReader object and checks whether a matching 1C:Enterprise data type is available. If it is available, the loop execution continues.

You have to present XML data as a value that has a type available in 1C:Enterprise. Let us use the ReadXML() global context method for this purpose.

7. Update the procedure as shown in listing 24.22.

Listing 24.22. Presenting XML data as a value that has a type

```
Procedure ReadMessageWithChanges() Export


    Directory = TempFilesDir();


    // Generating file name
    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + TrimAll(Ref.Code) + "_"
        + TrimAll(ExchangePlans.Branches.ThisNode().Code) + ".xml";
    File = New File(FileName);
    If Not File.Exist() Then
        Return;
    EndIf;


    // *** Reading XML document
    // Trying to open the file
    XMLReader = New XMLReader;
    Try
        XMLReader.OpenFile(FileName);


    Except
        Message = New UserMessage;
        Message.Text = "Cannot open the data exchange file.";
        Message.Message();


        Return;


    EndTry;


    Message = New UserMessage;
    Message.Text = "-------- Starting import from: "
        + String(ThisObject) + "---------";
```

```
    Message.Message();
    Message = New UserMessage;
    Message.Text = "- Reading file: " + FileName;
    Message.Message();

    // Importing data from the file
    // *** Message infrastructure
    ExchangeMessageReader = ExchangePlans.CreateMessageReader();

    // Reading the data exchange message header from the XML file
    ExchangeMessageReader.BeginRead(XMLReader);

    // The message is not for this node
    If ExchangeMessageReader.Sender <> Ref Then
        Raise "Incorrect node";
    EndIf;

    // Deleting change records for the source node
    // *** Change registration service
    ExchangePlans.DeleteChangeRecords(ExchangeMessageReader.Sender,
        ExchangeMessageReader.ReceivedNo);

    // Reading data from the message
    // *** XML serialization
    While CanReadXML(XMLReader) Do

        // Reading next value
        Data = ReadXML(XMLReader);

    EndDo;

    ExchangeMessageReader.EndRead();
    XMLReader.Close();
    DeleteFiles(FileName);

    Message = New UserMessage;
    Message.Text = "-------- Import completed --------";
    Message.Message();

EndProcedure
```

The ReadXML() method assigns the 1C:Enterprise object that matches the XML data to the Data variable.

Next you have to resolve a possible collision.

8. Update the procedure as shown in listing 24.23.

Listing 24.23. Resolving possible collisions

```
Procedure ReadMessageWithChanges() Export

    Directory = TempFilesDir();

    // Generating file name
    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + TrimAll(Ref.Code) + "_"
        + TrimAll(ExchangePlans.Branches.ThisNode().Code) + ".xml";
    File = New File(FileName);
    If Not File.Exist() Then
        Return;
    EndIf;

    // *** Reading XML document
    // Trying to open the file
    XMLReader = New XMLReader;
    Try
        XMLReader.OpenFile(FileName);

    Except
        Message = New UserMessage;
        Message.Text = "Cannot open the data exchange file.";
        Message.Message();

        Return;

    EndTry;

    Message = New UserMessage;
    Message.Text = "-------- Starting import from: "
        + String(ThisObject) + "---------";
    Message.Message();
    Message = New UserMessage;
    Message.Text = "- Reading file: " + FileName;
    Message.Message();

    // Importing data from the file
    // *** Message infrastructure
    ExchangeMessageReader = ExchangePlans.CreateMessageReader();

    // Reading the data exchange message header from the XML file
    ExchangeMessageReader.BeginRead(XMLReader);

    // The message is not for this node
```

```
    If ExchangeMessageReader.Sender <> Ref Then
        Raise "Incorrect node";
    EndIf;

    // Deleting change records for the source node
    // *** Change registration service
    ExchangePlans.DeleteChangeRecords(ExchangeMessageReader.Sender,
        ExchangeMessageReader.ReceivedNo);

    // Reading data from the message
    // *** XML serialization
    While CanReadXML(XMLReader) Do

        // Reading next value
        Data = ReadXML(XMLReader);

        // Do not apply the change received from a subordinate node
        // to the master one if a change record is found
        If Not ExchangeMessageReader.Sender.Main And
          ExchangePlans.IsChangeRecorded(ExchangeMessageReader.Sender,
          Data) Then

          Message = New UserMessage;
          Message.Text = "- Changes rejected";
          Message.Message();

          Continue;
        EndIf;

    EndDo;

    ExchangeMessageReader.EndRead();
    XMLReader.Close();
    DeleteFiles(FileName);

    Message = New UserMessage;
    Message.Text = "-------- Import completed --------";
    Message.Message();

EndProcedure
```

A possible collision is resolved as follows: the procedure checks whether the sender node is the main node and whether any records regarding changes of this object for this node are available in the database. If the object has been changed in the database and the sender is not a main node, the change record of the received object is rejected. Otherwise the changes of the received object are accepted.

Now the only thing that is left is writing the received data.

9. Update the procedure as shown in listing 24.24.

Listing 24.24. Writing the received data

```
Procedure ReadMessageWithChanges() Export

    Directory = TempFilesDir();

    // Generating file name
    FileName = Directory + ?(Right(Directory,1)= "\", "", "\")
        + "Message" + TrimAll(Ref.Code) + "_"
        + TrimAll(ExchangePlans.Branches.ThisNode().Code) + ".xml";
    File = New File(FileName);
    If Not File.Exist() Then
        Return;
    EndIf;

    // *** Reading XML document
    // Trying to open the file
    XMLReader = New XMLReader;
    Try
        XMLReader.OpenFile(FileName);

    Except
        Message = New UserMessage;
        Message.Text = "Cannot open the data exchange file.";
        Message.Message();

        Return;

    EndTry;

    Message = New UserMessage;
    Message.Text = "-------- Starting import from: "
        + String(ThisObject) + "---------";
    Message.Message();
    Message = New UserMessage;
    Message.Text = "- Reading file: " + FileName;
    Message.Message();

    // Importing data from the file
    // *** Message infrastructure
    ExchangeMessageReader = ExchangePlans.CreateMessageReader();

    // Reading the data exchange message header from the XML file
```

```
    ExchangeMessageReader.BeginRead(XMLReader);

    // The message is not for this node
    If ExchangeMessageReader.Sender <> Ref Then
        Raise "Incorrect node";
    EndIf;

    // Deleting change records for the source node
    // *** Change registration service
    ExchangePlans.DeleteChangeRecords(ExchangeMessageReader.Sender,
        ExchangeMessageReader.ReceivedNo);

    // Reading data from the message
    // *** XML serialization
    While CanReadXML(XMLReader) Do

        // Reading next value
        Data = ReadXML(XMLReader);

        // The change received from a subordinate node is not applied
        // to the master one if a change record is found
        If Not ExchangeMessageReader.Sender.Main And
          ExchangePlans.IsChangeRecorded(ExchangeMessageReader.Sender,
          Data) Then
          Message = New UserMessage;
          Message.Text = "- Changes rejected";
          Message.Message();

          Continue;
        EndIf;

        // Writing received data
        Data.DataExchange.Sender = ExchangeMessageReader.Sender;
        Data.DataExchange.Load = True;
        Data.Write();

    EndDo;

    ExchangeMessageReader.EndRead();
    XMLReader.Close();
    DeleteFiles(FileName);

    Message = New UserMessage;
    Message.Text = "-------- Import completed --------";
    Message.Message();
EndProcedure
```

Before writing the received object, the procedure records the source node to the data exchange parameters of the object, so that when the object is written to the database, no change registration record is generated for the node it is just received from.

The procedure also sets the Load property in the data exchange parameters to True. This states that the object is written during the update of the data received during the exchange, which simplifies the object writing procedure for the system because it can avoid some standard checks and prevent modifications of related data that should happen for normal writing.

This completes the procedure of receiving and processing exchange data.

Testing data exchange

In Designer mode

To be able to edit the NumberingPrefix constant, you have to create a form.

1. In the configuration object tree, expand the **Common** branch, click the **Common forms** branch, and use the form wizard to create a constants form named *GeneralSettings*.
2. In the **More: GeneralSettings** window that is opened, specify that the form belongs to the **Enterprise** subsystem (fig. 24.7).

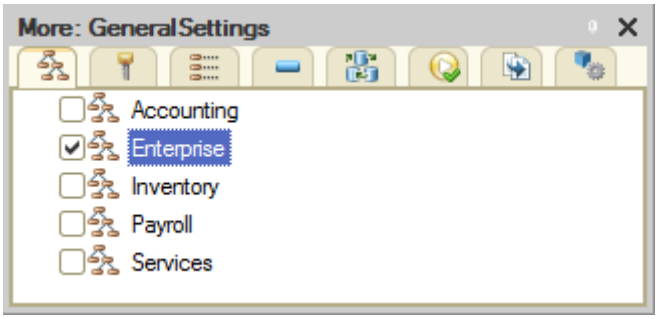


Fig. 24.7. The More window of the GeneralSettings constants form

3. In the configuration object editors for the **Branches** exchange plan and the **DataExchange** data processor, on the **Subsystems** tab, specify that they belong to the **Enterprise** subsystem.
4. In the **Enterprise** subsystem command interface, specify that only **Administrator** has access to the commands that open the exchange plan, the data processor, and the constants form.
5. Move the **Branches** command to the top of the **Navigation panel.Important** group.
6. In the **Actions panel.Create** group, for the **Branch: create** command, enable visibility to the **Administrator** role only.
7. In the **Actions panel.Tools** group, order the commands as follows:
   - **General settings**
   - **Data exchange**

8. Update the database configuration by pressing F7.
9. Create a directory for storing the branch infobase.
10. On the **Configuration** menu, click **Save configuration to file** and save the configuration to that directory.

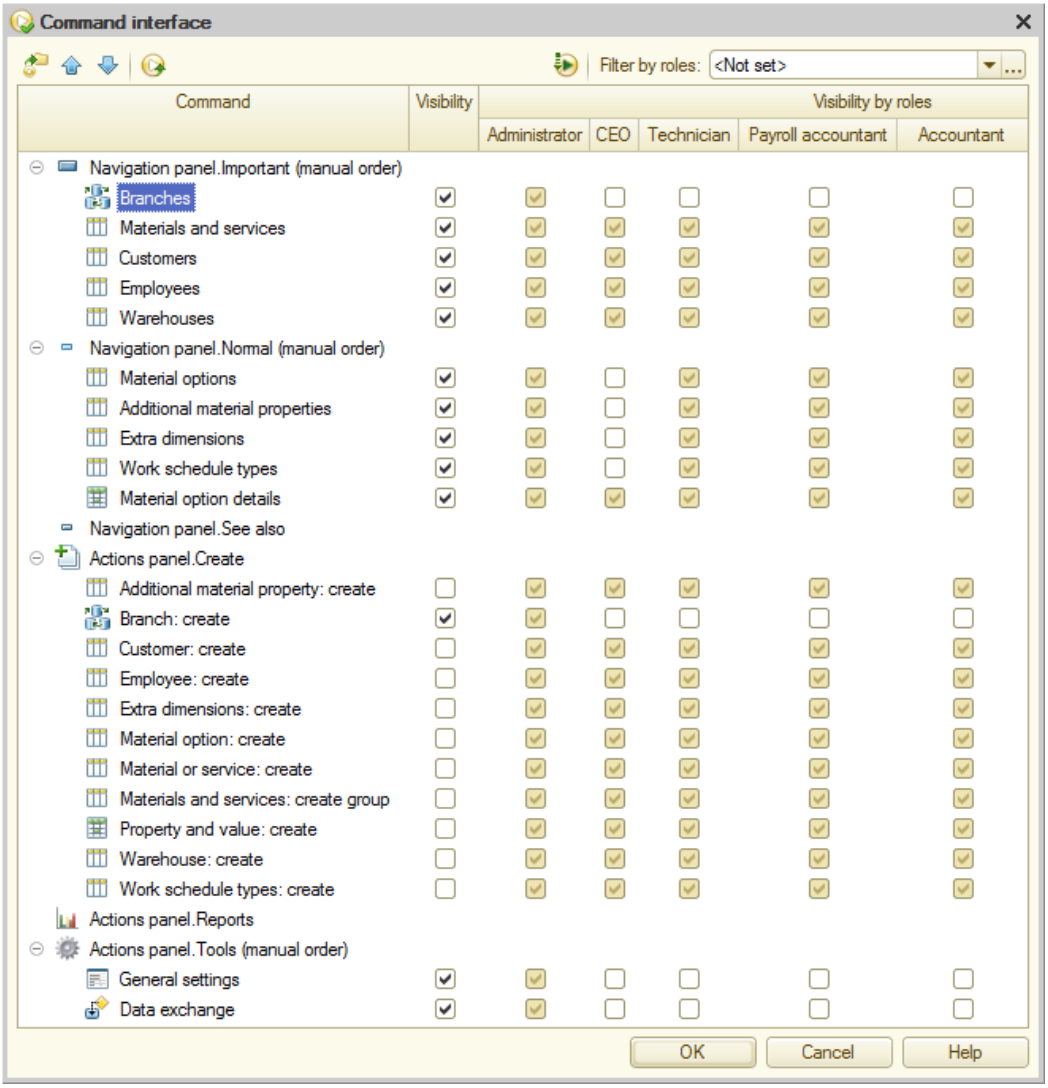


Fig. 24.8. Enterprise subsystem command interface

In 1C:Enterprise mode

Let us enter the values required for data exchange in the central infobase.

1. Start 1C:Enterprise in the debug mode.

   First, let us set the numbering prefix.

2. In the **Enterprise** section, on the **Tools** menu, click **General settings**.
3. In the **Numbering prefix** field, enter *CI* and then click **Save and close** (fig. 24.9).

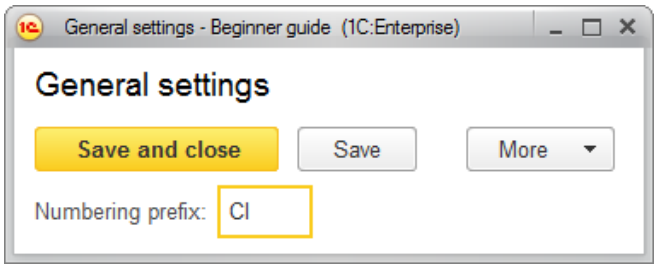


Fig. 24.9. Value of the NumberingPrefix constant

Then let us open the exchange plan and specify the default node parameters, i.e. the parameters of the current infobase.

4. In the **Enterprise** section, in the section commands panel, click **Branches**.

   You can see that the list of exchange plans contains a single empty row.

5. Open that row, enter *CI* for the infobase code and *Central infobase* for its description (fig. 24.10), then click **Save and close**.

   Remember that a code serves as a unique node ID across all infobases. Later you will create nodes with the same codes in the branch infobase.

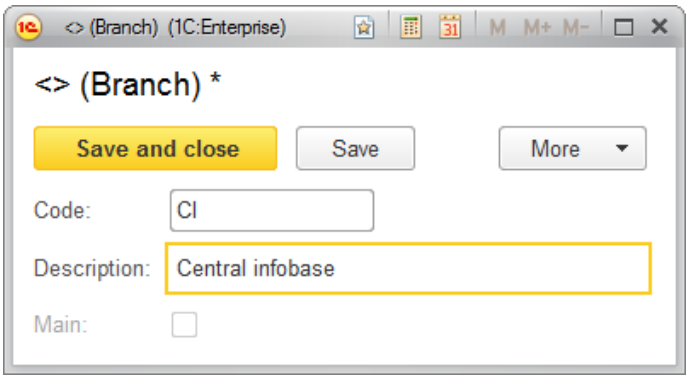


Fig. 24.10. Creating an exchange plan node

Then let us create the branch infobase node.

6. Click the **Create** button.

   -OR-

   On the **Create** menu, click **Branch**.

7. Enter *BR* for the infobase code and *Branch* for its description (fig. 24.11), then click **Save and close**.

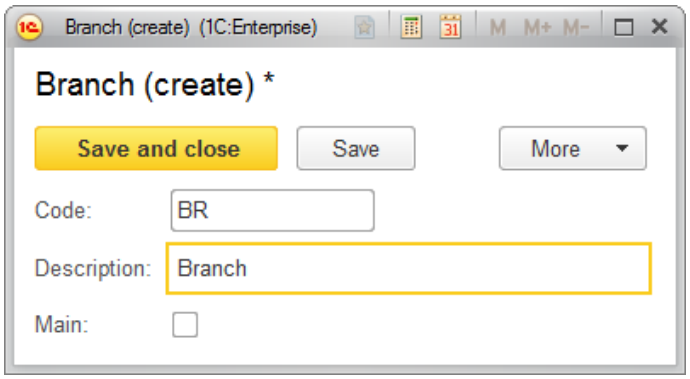


Fig. 24.11. Creating an exchange plan node

Note that the predefined infobase node (Central database) is marked with a special icon in the list of exchange nodes, and the **Write changes** button is not available for this node.

8. Click the **Branch** node and click **Write changes**.
9. Open the **Data exchange** data processor and click **Start data exchange**.

The text shown in fig. 24.12 is displayed in the message window.

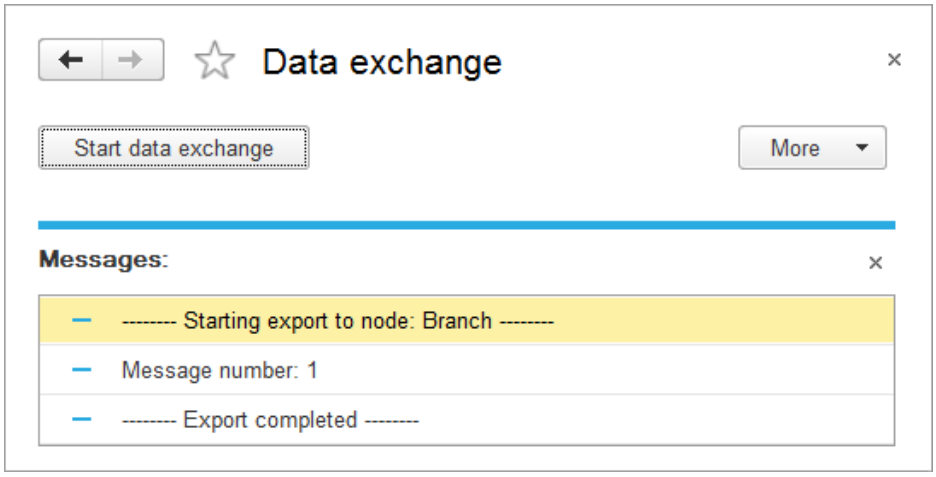


Fig. 24.12. Service message window

So as a result of the data exchange operation the central infobase generated an exchange file containing the changes of all the data it exchanges with the branch.

Running the branch infobase

Let us proceed to the branch infobase.

You have to add a new infobase with an empty configuration to the infobase list. The infobase will be located in the directory that you created earlier in this lesson.

1. Run 1C:Enterprise.
2. In the 1C:Enterprise startup window, click **Add**, click **Creating an infobase**, and then click **Next**.

3. Select **Creating an infobase without configuration...** and click **Next**.
4. Enter the infobase name (for example, *Branch infobase*), then click **Next**.
5. Specify the infobase directory where the saved configuration is located (for example, *D:\Practical developer guide\branch*), click **Next**, and click **Ready**.

In Designer mode

Let us prepare the branch infobase for the exchange.

1. Open the branch infobase configuration in Designer mode.
2. On the **Configuration** menu, click **Open configuration**.

   You can see that the list of configuration objects is empty.
3. On the **Configuration** menu, click **Load configuration from file**, select the file that you created earlier (for example, D:\Practical developer guide\branch\1Cv8.cf), and click **Open**.
4. When prompted to confirm the configuration update, click **Yes**.
5. In the list of configuration changes, click **Accept**.

   All the configuration objects are now transferred from the central infobase.

   There is one thing that requires special attention. The data exchange objects include the Warehouses catalog that has a predefined item named Main.

   When you create catalogs or other configuration objects that can have predefined items, their **Update predefined data** property is set to **Auto**. It means that, once a database is restructured or a table that stores the configuration object data is accessed for the first time, data items linked to predefined data items by name are created or updated.

   In this scenario, once you open the list of warehouses, the platform automatically creates an item in the Warehouses catalog, which is linked to the predefined catalog item named Main. And the catalog item property PredefinedDataName is set to Main. This link can only be changed using 1C:Enterprise script.

   Then, once you load data from the central infobase to the Warehouses catalog, the platform adds another data item linked to the Main predefined item of the Warehouses catalog. This will cause an error because a configuration cannot have two items linked to a single predefined item.

   To avoid this, you have to change the property value.
6. Open the object editor window of the **Warehouses** catalog, click the **Other** tab, and set **Update predefined data** to **Do not update automatically** (fig. 24.13).

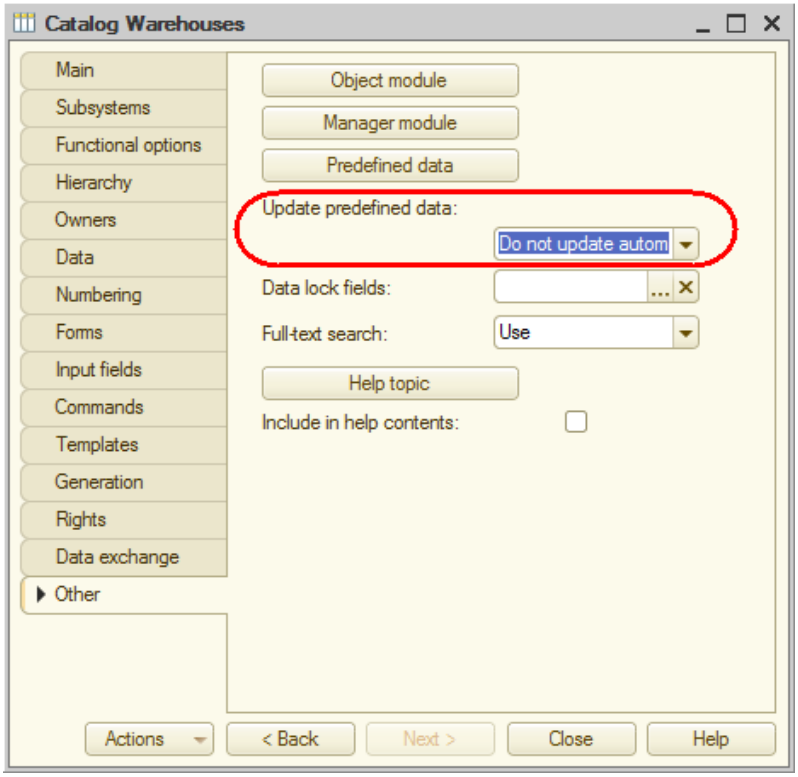


Fig. 24.13. Editing the properties of the Warehouse catalog

7. On the **Administration** menu, click **Users** and create a single user: *Administrator* with **Administrator** role.

   This is because you need to create users from scratch in each infobase.

In 1C:Enterprise mode

Let us enter the values required for data exchange.

1. Start 1C:Enterprise in the debug mode.
2. Set the **Numbering prefix** to *BR* (fig. 24.14).

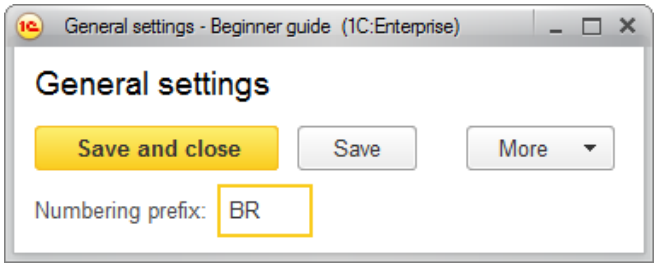


Fig. 24.14. Editing the constant

Then let us open the exchange plan and describe the predefined node (the current infobase node).

3. In the **Enterprise** section, on the navigation panel, click **Branches**.

   You can see that the list of exchange plans contains a single empty row.

4. Open that row, enter *BR* for the infobase code and *Branch* for its description (fig. 24.15), then click **Save and close**.

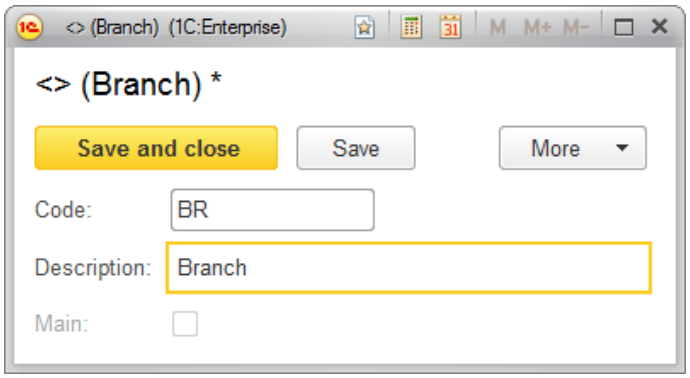


Fig. 24.15. Creating an exchange plan node

5. Create an exchange plan node with *CI* for the infobase code and *Central infobase* for its description, and the **Main** check box selected (fig. 24.16), then click **Save and close**.

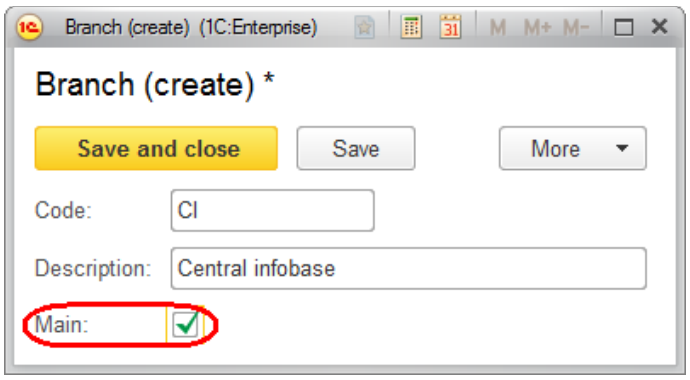


Fig. 24.16. Creating an exchange plan node

6. Click the new **Central infobase** node and then click **Write changes**.

   To better understand the process, let us check some infobase catalogs.

7. Open the list of the **Customers** catalog and ensure that it is empty for now.

8. Open the list of **Warehouses** and ensure that it is also empty.

   It does not contain the predefined item because you set its Update predefined data property to Do not update automatically.

9. Open the **Data exchange** data processor and click **Start data exchange**.

   This fills the catalogs with items, and also fills other objects that are included in the exchange. And the message window displays the messages shown in fig. 24.17.

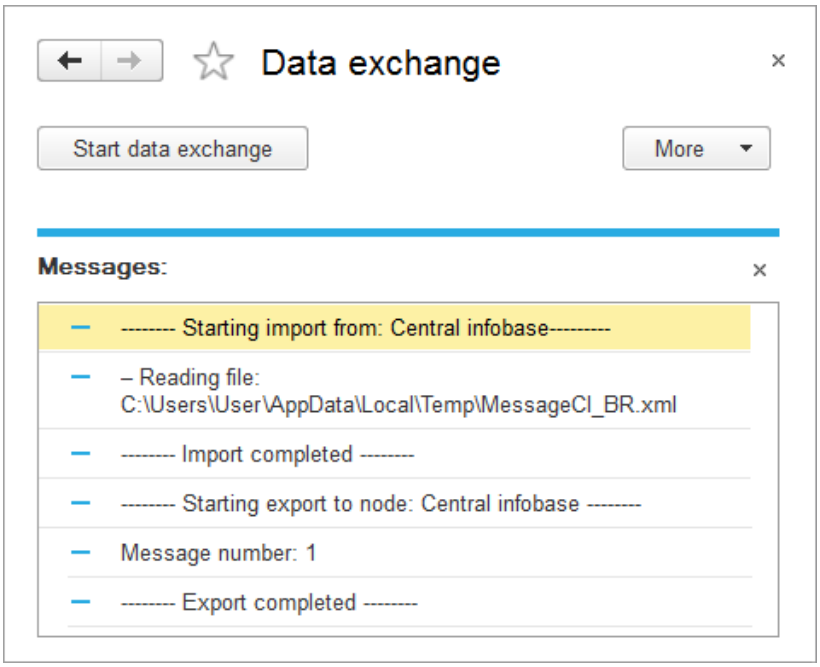


Fig. 24.17. Exchange plan messages

Note. To view the list of customers replicated from the central infobase, refresh the list (press F5).

Now let us test how exchange is processed on the other side.

10. In the **Customers** catalog, create a new customer with an arbitrary name.

    Note that the customer code has the BR prefix and the code numbering starts from one.

11. Return to the **Data exchange** data processor and click **Start data exchange**.

12. Return to the central infobase, perform the data exchange there, and ensure that the customer created in the branch infobase has been transferred to the central one.

Distributed infobases

The distributed infobase scenario is a form of universal data exchange.

Attention! If you use the training version of the 1C:Enterprise platform, you will be unable to reproduce this example because distributed infobases are not supported by the training version.

Distributed infobases have identical configurations at all nodes and a tree-like node structure. They support not only exchange of data changes but also exchange of configuration changes.

The distributed infobase scenario employs exchange plans. Exchange plan configuration objects have the Distributed infobase property to support this scenario.

If you set the property to True, it enables the distributed infobase scenario for the exchange plan, and then you can create a distributed infobase using solely interactive means, without writing any script.

However, this feature does not prevent you from managing exchange using 1C:Enterprise script tools as they are also available for distributed infobase operations.

The following example describes both exchange implementation methods in a distributed infobase scenario.

Overview

As we have already mentioned above, a distributed infobase should have a clearly defined tree structure. The number of levels in that structure is unlimited. An explicit master-subordinate relationship between each pair of linked nodes is required (fig. 24.18).

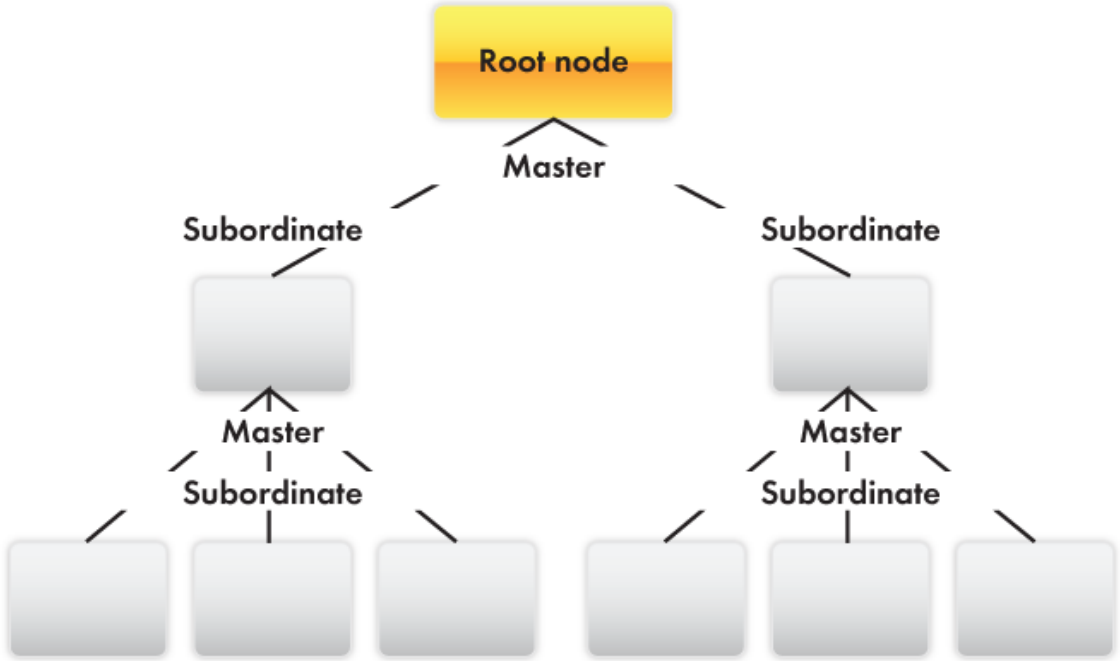


Fig. 24.18. Distributed infobase structure

So, each node in that structure can have an unlimited number of subordinate nodes (or none at all). Additionally, each node except one should have a master node, and the root node is the only one with no master. This rigidly defined node structure is essential for specifying the migration pattern for data and configuration changes.

A configuration can only be changed in a node that has no master node (in the root node). Data changes can be made in any node.

Configuration changes are transferred down from the master node to subordinate ones. Data changes can be transferred between any linked nodes.

Collisions are resolved on the basis of master/subordinate relationships as well. If changes are made simultaneously in a master node and in a

subordinate one, only the changes to the master node are accepted and changes to the subordinate node are discarded.

For any subordinate node an initial image can be created. An initial image is an infobase created on the basis of the configuration and data of the master node in compliance with the rules specified in the exchange plan. The initial node image creation procedure can be performed repeatedly. Every time it is performed, it deletes all the change records for the subordinate node from the master node infobase. Once an initial image is created, it is ready for exchange with the master node.

Creating an initial image is the recommended method of subordinate node creation in a distributed infobase.

The task at hand

As an example of distributed infobase operations, let us create several departments for Jack of All Trades.

Unlike branches that are located in different cities and act as independent entities with relative independence in how they manage their books, the departments are located in the same city, have no independent legal status, and keep their books in strict compliances with practices of the headquarters.

Therefore, they all use exactly the same configuration as the one used in the headquarters, and if the headquarters make any changes to the configuration, all of those changes should be rapidly disseminated to the configurations of the departments.

To implement this scenario, a distributed infobase is a perfect fit. First let us set up data exchange with departments using exclusively interactive means.

Interactive exchange

In Designer mode

To set up a distributed infobase, you need to create another Exchange plan configuration object.

1. In the configuration object tree, add an ExchangePlan configuration object named *Departments*.
2. In the **Object presentation** field, enter *Department*.
3. Select the **Distributed infobase** check box (fig. 24.19).

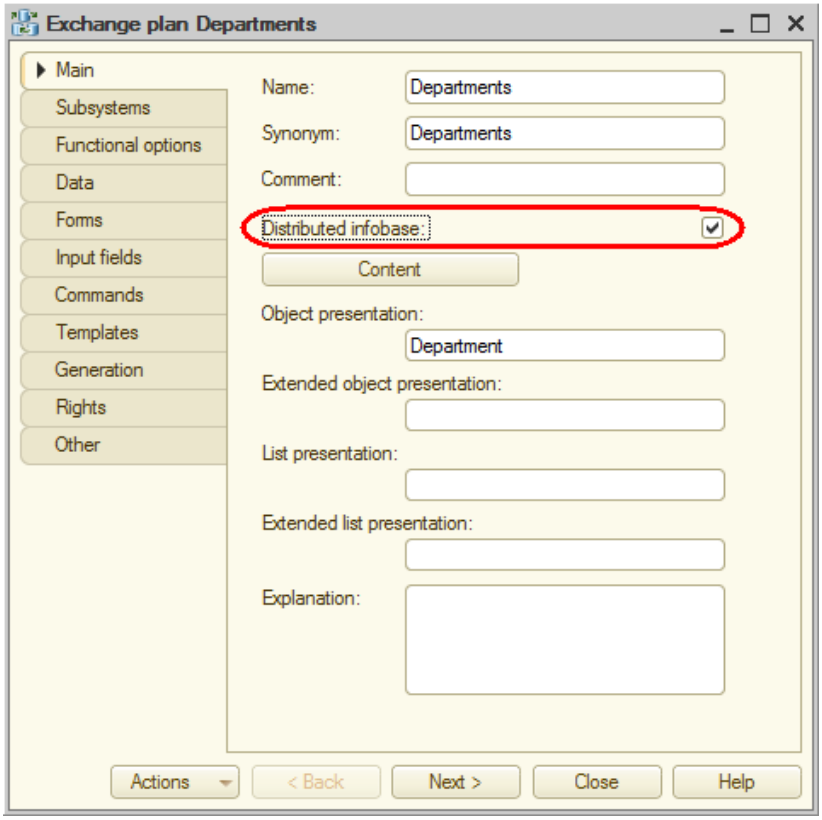


Fig. 24.19. Specifying Distributed infobase property

4. On the **Input fields** tab, select the **Quick choice** check box.

   This allows selection of exchange plan nodes from a drop-down list. Next let us define the set of objects that participate in the exchange.

5. On the **Main** tab, click the **Content** button and specify the same data exchange content as the one used in the Branches exchange plan.

   Basically you should include all the objects that are not related to accounting and payroll into the exchange. The resulting content of exchange data should look as shown in fig. 24.20.

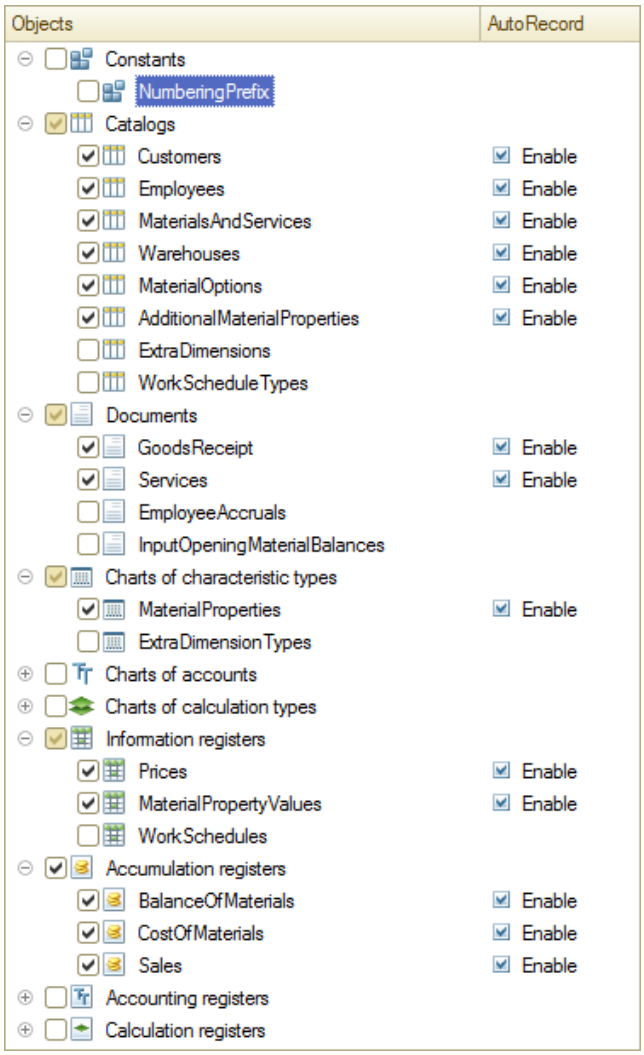


Fig. 24.20. Exchange data content

6. On the **Subsystems** tab, include the exchange plan in the **Enterprise** subsystem.
7. In the **Enterprise** subsystem command interface settings window, move the command that opens the **Departments** exchange plan to the **Navigation panel.Important** group, after the **Branches** command, and make it available to the **Administrator** role only.
8. In the **Actions panel.Create** group, enable visibility of the **Department: create** command for the **Administrator** role only.

In 1C:Enterprise mode

Let us create the department infobase.

1. Start 1C:Enterprise in the debug mode.

   Let us specify the parameters of the central node (which is a predefined item of the exchange plan).
2. In the **Enterprise** section, in the navigation panel, click **Departments**.

You can see that the list of exchange plans contains a single empty row. This is the predefined infobase node.

3. Open that row, enter *CI* for the infobase code and *Central infobase* for its description, then click **Save and close**.
4. Create an exchange plan node with *Dep* for the infobase code and *Department* for its description.

   The created node has three buttons available in the command bar of the exchange plan form**: Write changes**, **Read changes**, and **Create initial image** (fig. 24.21).

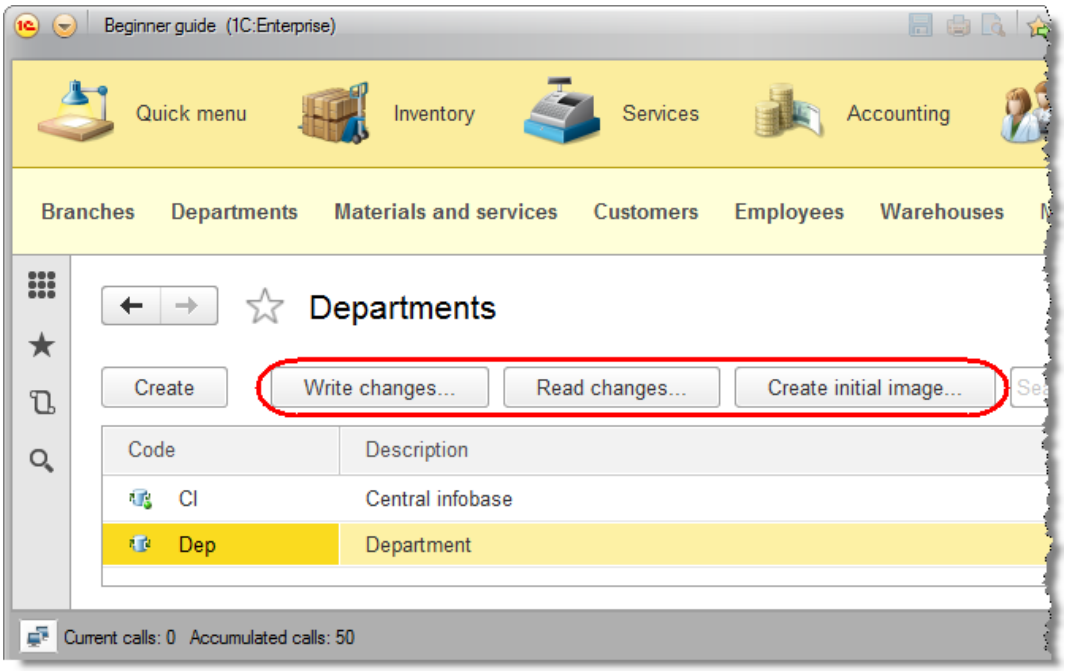


Fig. 24.21. Commands for distributed infobase operations

Let us use one of these buttons to create an initial image of the department infobase.

5. Create a new directory on the hard disk to store the department infobase.
6. In the list of data exchange nodes, click **Departments** and then click **Create initial image**.
7. In the **Create initial infobase image** dialog box, specify that the infobase will be located on this computer, and click **Create initial image**.
8. At the next step, click **Save** and specify the directory that you created for the infobase (fig. 24.22).

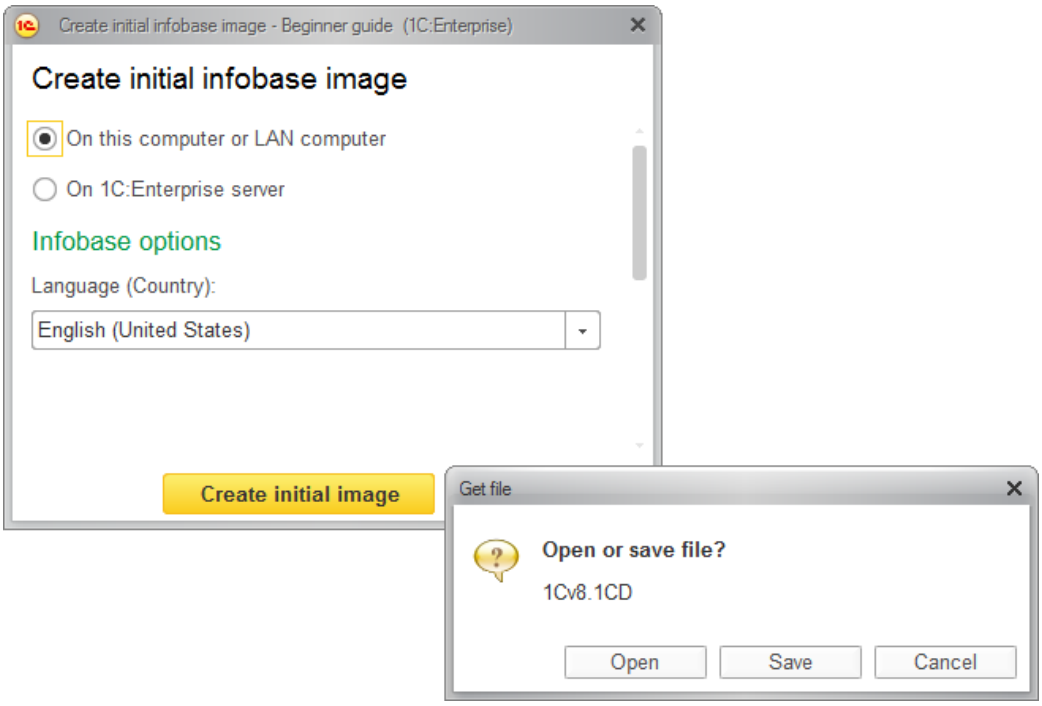


Fig. 24.22. Creating an initial infobase image

9. Return to the **Create initial infobase image** dialog box and click **Close**.

   The platform creates the initial image of the department infobase in the specified directory.

Running the department infobase

Let us switch to the department infobase.

1. Run 1C:Enterprise.

   Then let us add the created infobase to the list.

2. In the 1C:Enterprise startup window, click **Add**, click **Adding an existing infobase to the list**, and then click **Next**.
3. Enter the infobase name (for example, *Department infobase*), then click **Next**.
4. Specify the infobase directory where the saved configuration is located (for example, *D:\Practical developer guide\department*), click **Next**, and click **Ready**.

In Designer mode

Let us prepare the department infobase for the exchange.

1. Open the department infobase configuration in Designer mode.
2. On the **Configuration** menu, click **Open configuration**.

   You can see that the department configuration is protected from modifications via distributed infobase management tools (fig. 24.23).

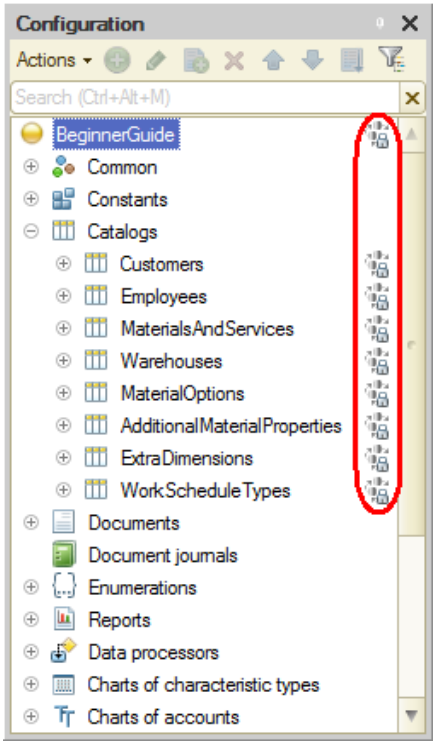


Fig. 24.23. Subordinate node configuration protected from modifications via distributed infobase management tools

3. On the **Administration** menu, click **Users** and create a single user: *Administrator* with **Administrator** role.

In 1C:Enterprise mode

Let us enter the values required for data exchange.

1. Start the department infobase in the debug mode and open the **Departments** exchange plan (fig. 24.24).

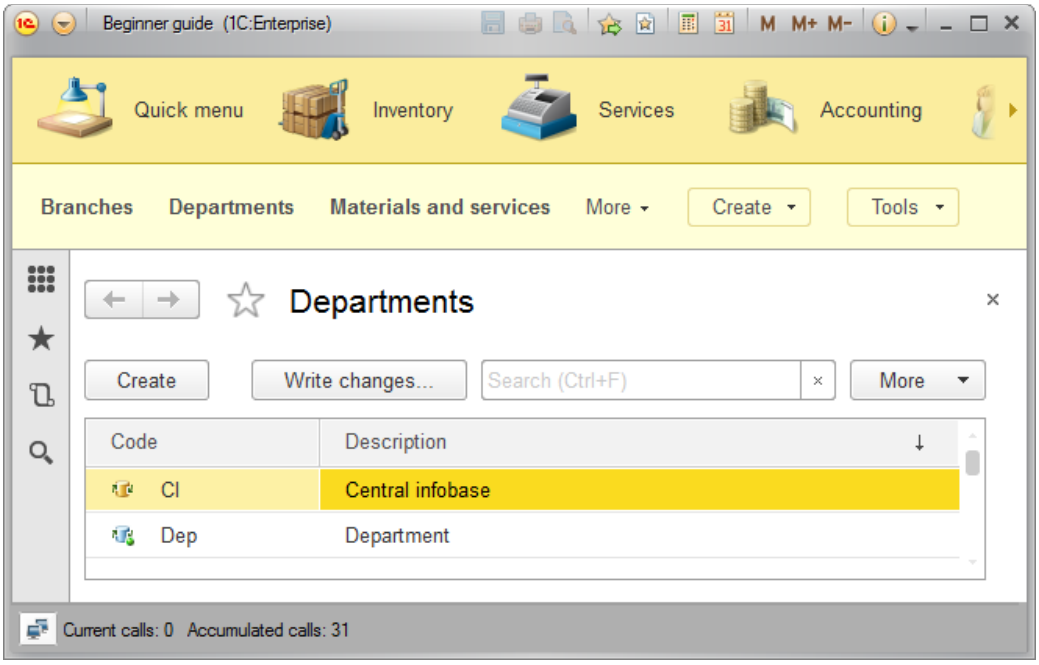


Fig. 24.24. Nodes of the Departments exchange plan

Note that in the infobase of the subordinate node the subordinate node itself (Department) is a predefined exchange plan node, while the central infobase node is marked by a yellow icon, indicating that it is the master of the department infobase. Additionally, the only available commands for the central infobase are Write changes and Read changes.

Now let us test the data exchange.

2. Set the **Numbering prefix** to *DP*.

   Note. If you completed Lesson 25 (Functional options) prior to lesson 24, some sections might be hidden from the sections panel (for example, the Accounting section). If this is the case, open the General settings form and check the values of the functional options that define section visibility (for example, the Accounting option). Note that functional options are not transferred between nodes, you have to specify their values independently for each node.

3. Open the **Customers** catalog and add a new customer.
4. Open the **Departments** exchange plan, select the **Central infobase** node (the node where you need to transfer the changes), and click **Write changes**.
5. In the **Write message with changes** dialog box, click **Write and save to file**.
6. Click **Save** and specify the message file name (fig. 24.25).

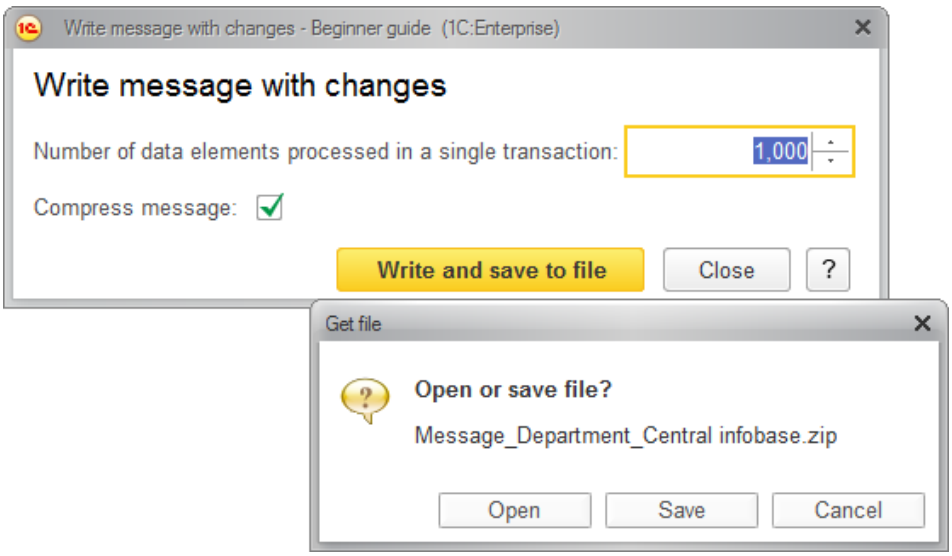


Fig. 24.25. Write message with changes dialog box

7. In the **Write message with changes** dialog box, click **Close**.
8. Start the central infobase in the debug mode, open the **Departments** exchange plan, click the **Department** node (the node whose changes you need to transfer to the central infobase), and click **Read changes**.
9. In the **Read message with changes** dialog box, click **Select file and read changes**.
10. Specify the message file name and click **Open**.

11. In the **Read message with changes** dialog box, click **Close**.
12. Ensure that the new customer created in the department infobase is now available in the central infobase as well.
13. Close the 1C:Enterprise window of the central infobase.

    Next, let us test how configuration changes are transferred between the master node and subordinate ones.
14. In Designer of the central infobase, create a constant named *NewConstant*.
15. In the **Warehouses** catalog, add a predefined item named *Office* with code 000000003.

    When you start creating the predefined item, the platform provides 000000002 as the default code value. But the catalog already includes a Retail warehouse with code 000000002, which was created in 1C:Enterprise mode, so you have to change the code to make it unique.
16. Update the database configuration and start 1C:Enterprise in the debug mode.
17. Open the **Departments** exchange plan and write the changes for the **Department** subordinate node.
18. Close the 1C:Enterprise window and close Designer of the department infobase.
19. Run the department infobase in 1C:Enterprise mode and read the changes from the **Central infobase** node.

    Once the changes are read, the following message is displayed (fig. 24.26).

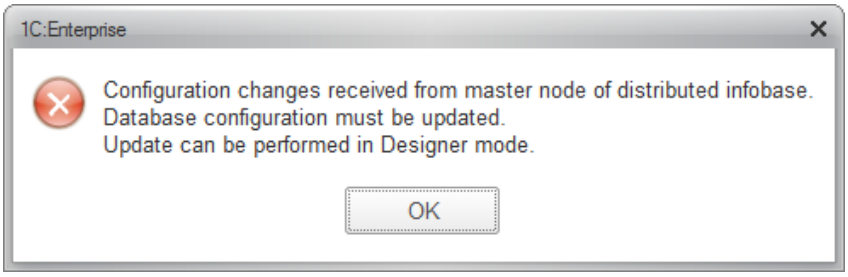


Fig. 24.26. System message

20. Close the 1C:Enteprise window, open Designer of the department infobase, and ensure that the NewConstant constant is now available in the configuration, as well as the Office predefined item of the Warehouses catalog.

    In other words, the changes made to the central infobase configuration have been replicated to the configuration of the subordinate node.

    Now everything you need to do is update the infobase configuration in the subordinate node.

    We should probably say a few words concerning the order of applying changes when a single message contains changes to both the configuration and the data.

In this case, the base configuration is changed first, and then a message is displayed prompting you to update the database configuration.

But at that point the new data is not yet added. For example, the Office predefined item is not yet present in the Warehouses catalog.

Unlike universal data exchange, the distributed infobase scenario is free of the issue related to duplication of predefined data. This is because automatic creation of predefined data is not performed in subordinate nodes of a distirbuted infobase.

After synchronizing the configurations, you have to reread the data, and this time all of the data changes stored in the message are applied. The order of applying changes remains the same regardless of whether the changed data refers to existing configuration objects or new ones.

21. Delete the **NewConstant** object and the **Office** predefined item of the **Warehouses** catalog.

Script-based exchange

All the distributed infobase data exchange activities that you performed manually in the previous part of the lesson can be programmed as well.

Let us create a data processor that performs all these operations for a specified node.

In Designer mode

Let us implement script-based data exchange.

1. Open the central infobase in Designer.
2. Create a DataProcessor configuration object named *DepartmentsExchange*.
3. On the **Forms** tab, create a default data processor form.
4. In the form editor, on the **Attributes** tab, add a form attribute named *DepartmentTextBox* of the **ExchangePlan.Departments** type.
5. Drag the attribute to the form controls pane.
6. In the field property palette that is opened, set the field title to *Department* and keep the default field type (**Text box**, fig. 24.27).

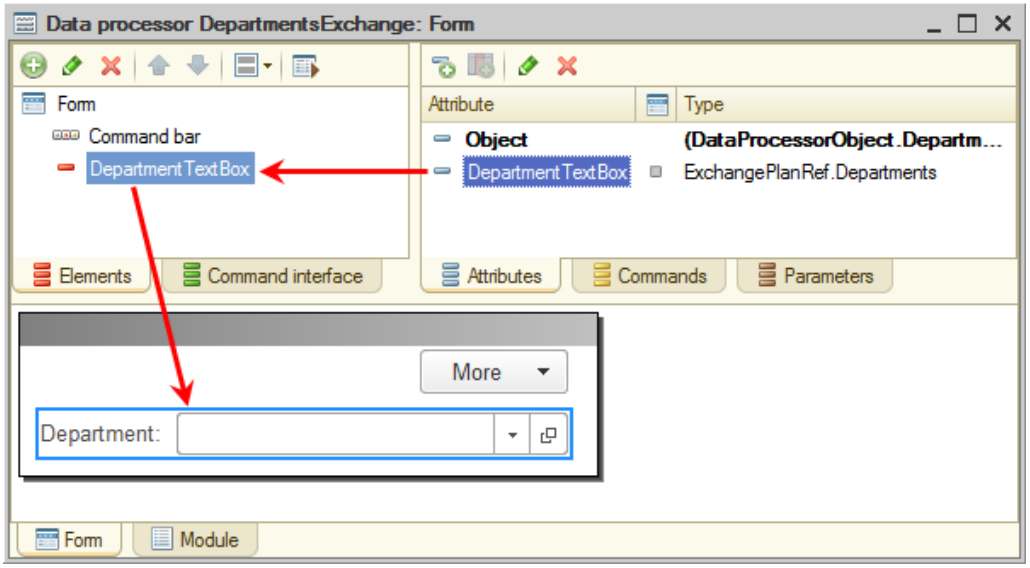


Fig. 24.27. Editing data processor form

7. On the **Commands** tab, create the following commands: **CreateInitialImage**, **WriteChanges**, and **ReadChanges**.
8. For each of the commands, in the property palette, in the **Action** field, click the **Open** button and create a client handler.

   Do not fill the event handlers for now.
9. Click the **Form** tab and drag all three commands to the form controls pane.

   The resulting data processor form should look as shown in fig. 24.28.

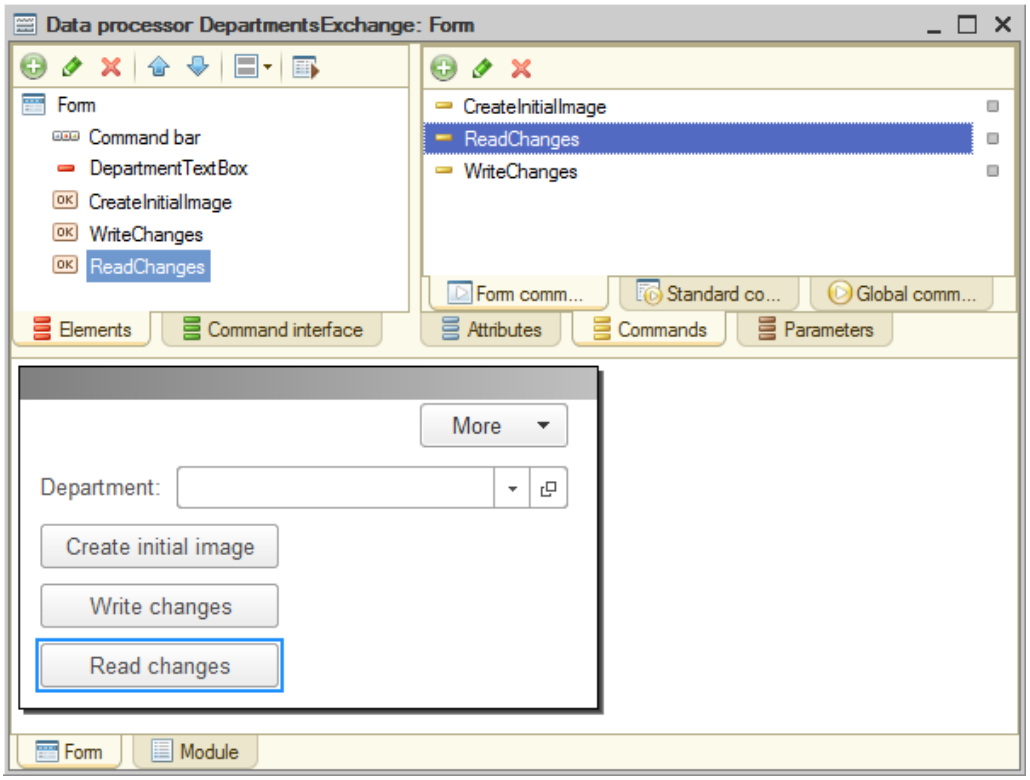


Fig. 24.28. Data processor form

10. Open the property palette of the **CreateInitialImage** button and clear the **Availability** check mark.

    When you open the data processor, the button should not be available until you select an exchange plan node in the DepartmentTextBox field. And this button should not be available if you select the predefined node of the infobase. In other words, initial image creation should not be available if the selected node is a predefined one.

    Let us implement this button behavior.

11. In the data processor form module, create a function that is executed on the server and returns True if the node passed to the function is the predefined one (listing 24.25).

Listing 24.25. PredefinedNode() function

```
&AtServerNoContext
Function PredefinedNode(Node)
    Return Node = ExchangePlans.Departments.ThisNode();
EndFunction
```

In the form controls pane, open the property palette of the **DepartmentTextBox** item, create the **Selection processing** event handler, and fill it as shown in listing 24.26.

Listing 24.26. TextBoxDepartmentChoiceProcessing() procedure

```
&AtClient
Procedure TextBoxDepartmentChoiceProcessing(Item, SelectedValue,
    StandardProcessing)


    If PredefinedNode(SelectedValue) Then
        Items.CreateInitialImage.Enabled = False;
    Else
        Items.CreateInitialImage.Enabled = True;
    EndIf;


EndProcedure
```

The procedure sets the availability of the CreateInitialImage button depending on the value of the PredefinedNode() function. This value, in turn, depends on the reference to the selected node passed to the function (SelectedValue).

12. Fill the CreateInitialImage command handler as shown in listing 24.27.

Listing 24.27. CreateInitialImage button click handler

```
&AtClient
Procedure CreateInitialImage(Command)


    Dialog = New FileDialog(FileDialogMode.ChooseDirectory);
```

```
    Dialog.Title = "Specify infobase directory";
    If Dialog.Choose() Then
        CreateInitialImageAtServer(DepartmentTextBox,
            Dialog.Directory);

        Message = New UserMessage;
        Message.Text = "Initial image created.";
        Message.Message();
    EndIf;

EndProcedure
```

First, the procedure opens the dialog box for selecting the directory to store the infobase image. Then it calls the CreateInitialImageAtServer() procedure, which is executed on the server without context. This procedure calls the CreateInitialImage() method of the ExchangePlansManager object.

This method creates an image of a subordinate distributed infobase node. Its first parameter is a reference to the node (DepartmentTextBox form attribute) for which the initial image is created, and the second parameter is a connection string that specifies the infobase.

13. Create the procedure shown in listing 24.28.

Listing 24.28. CreateInitialImageAtServer() procedure

```
&AtServerNoContext
Procedure CreateInitialImageAtServer(Node, ConnectionDirectory)
    ExchangePlans.CreateInitialImage(Node, "File =""
        + ConnectionDirectory + """");
EndProcedure
```

14. Create an event handler for the WriteChanges command (listing 24.29).

Listing 24.29. WriteChanges button click handler

```
&AtClient
Procedure WriteChanges(Command)

    Dialog = New FileDialog(FileDialogMode.Save);
    Dialog.Title = "Specify exchange file";
    If Dialog.Choose() Then
        WriteChangesAtServer(DepartmentTextBox, Dialog.FullFileName);

        Message = New UserMessage;
        Message.Text = "Writing changes completed.";
        Message.Message();
    EndIf;
 EndProcedure
```

First, the procedure opens the dialog box for selecting the file to store the changes. Then it calls the WriteChangesAtServer() procedure that is executed on the server without context. The first parameter of the procedure is a reference to the node (DepartmentTextBox form attribute) for which the changes are recorded.

The procedure creates an XMLWriter object to work with the file.

Next it creates an ExchangeMessageWriter object that will be used to create an exchange message. The second parameter of the BeginWrite() method is the exchange node for which the message is created.

Then the procedure calls the WriteChanges() method of the ExchangePlansManager object, which writes the changes intended for the selected node to the specified exchange message.

Finally, it ends the exchange message writing and closes the file.

15. Create the procedure shown in listing 24.30.

Listing 24.30. WriteChangesAtServer() procedure

```
&AtServerNoContext
Procedure WriteChangesAtServer(Node, FileName)

    // Creating and initializing XMLWriter object
    XMLWriter = New XMLWriter;
    XMLWriter.OpenFile(FileName);

    // Creating ExchangeMessageWriter object and starting
    // message writing
    ExchangeMessageWriter = ExchangePlans.CreateMessageWriter();
    ExchangeMessageWriter.BeginWrite(XMLWriter, Node);

    // Writing message body content for the
    // distributed infobase data exchange message
    ExchangePlans.WriteChanges(ExchangeMessageWriter);

    // Ending the writing and closing XMLWriter
    ExchangeMessageWriter.EndWrite();
    XMLWriter.Close();

EndProcedure
```

Learn more! It is worth noting that the WriteChanges method lets you specify the limit of data items stored to a message within a single transaction. By default, all data is stored within a single transaction.

This is the recommended mode because it ensures the consistency of data stored to a message.

However, if a message is created in multiuser mode, lock conflicts may occur between transactions that store data to messages and transactions performed by other users. To reduce the chance of such conflicts, you can adjust the parameter value. The lesser the parameter value, the lesser the chance of conflict, but the higher the chance of storing inconsistent data to the message.

Considering all of the above, the ideal solution is to perform data exchanges in exclusive mode. However, this option is not always available given the way a specific infobase can be set up.

16. Create an event handler for the ReadChanges command (listing 24.31).

Listing 24.31. ReadChanges button click handler

```
&AtClient
Procedure ReadChanges(Command)

    Dialog = New FileDialog(FileDialogMode.Open);
    Dialog.Title = "Select exchange file";
    If Dialog.Choose() Then
        ReadChangesAtServer(Dialog.FullFileName);

        Message = New UserMessage;
        Message.Text = "Reading changes completed.";
        Message.Message();
    EndIf;

EndProcedure
```

First, the procedure opens the file selection dialog box to select the file to be read. Then is calls the ReadChangesAtServer() procedure that is executed on the server without context.

This procedure creates an XMLReader object to work with the file. Then it creates an ExchangeMessageReader object to read the message stored in the specified file.

Then it uses the ReadChanges() method of the ExchangePlansManager object to read the received message.

Finally, it ends the exchange message reading and closes the file.

17. Create the procedure shown in listing 24.32.

Listing 24.32. ReadChangesAtServer() procedure

```
&AtServerNoContext
Procedure ReadChangesAtServer(FileName)

    // Creating and initializing XMLReader object
    XMLReader = New XMLReader;
    XMLReader.OpenFile(FileName);
```

```
    // Creating ExchangeMessageReader object and starting
    // message reading

    ExchangeMessageReader = ExchangePlans.CreateMessageReader();

    ExchangeMessageReader.BeginRead(XMLReader);


    // Reading message body content

    ExchangePlans.ReadChanges(ExchangeMessageReader);


    // Ending the reading and closing XMLReader

    ExchangeMessageReader.EndRead();

    XMLReader.Close();


EndProcedure
```

18. On the **Subsystems** tab, include the **DepartmentsExchange** data processor in the **Enterprise** subsystem.

    Then let us set up the main section command interface for Administrator.

19. Right-click the root configuration object and then click **Open main section command interface**.

20. Add the commands that open **General settings**, **Data exchange**, and **Departments exchange** forms to the list of main section commands and enable their visibility for Administrator only (fig. 24.29).

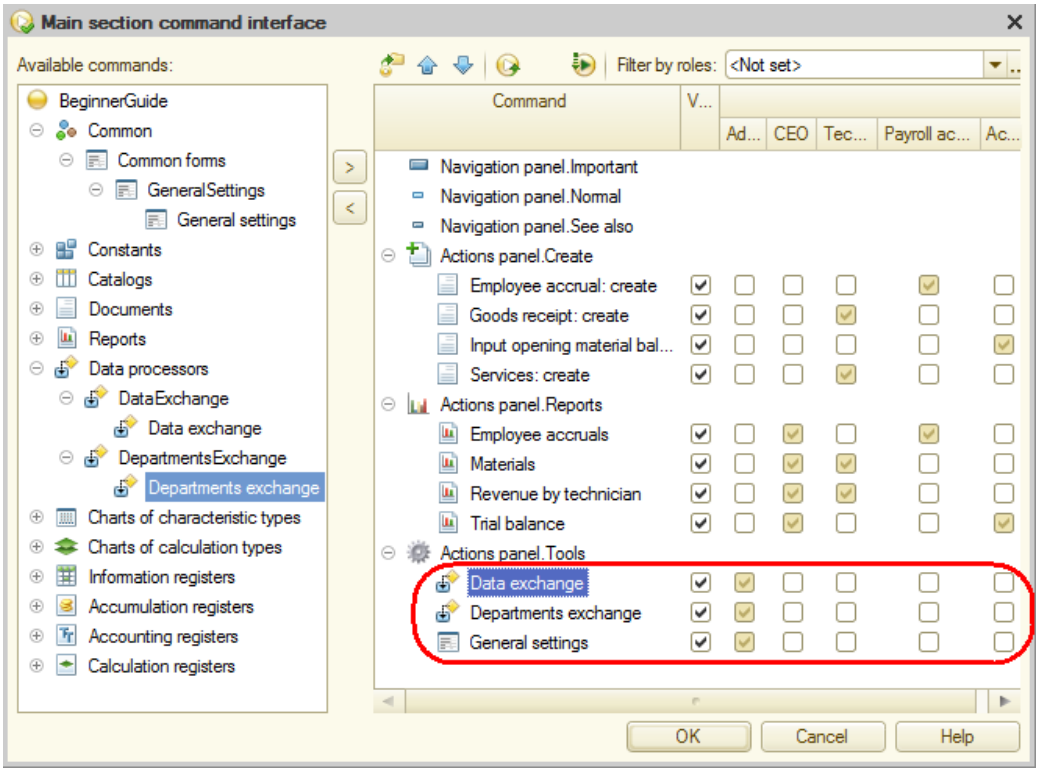


Fig. 24.29. Customizing main section command interface for Administrator

Let us add the list forms of Branches and Departments exchange plans to Administrator's start page.

21. Right-click the root configuration object and then click **Open start page work area**.
22. For the **Departments** exchange plan, create a list form.
23. Add the list forms of the **Branches** and **Departments** exchange plans to the left part of the start page and make them visible to the **Administrator** role only (fig. 24.30).

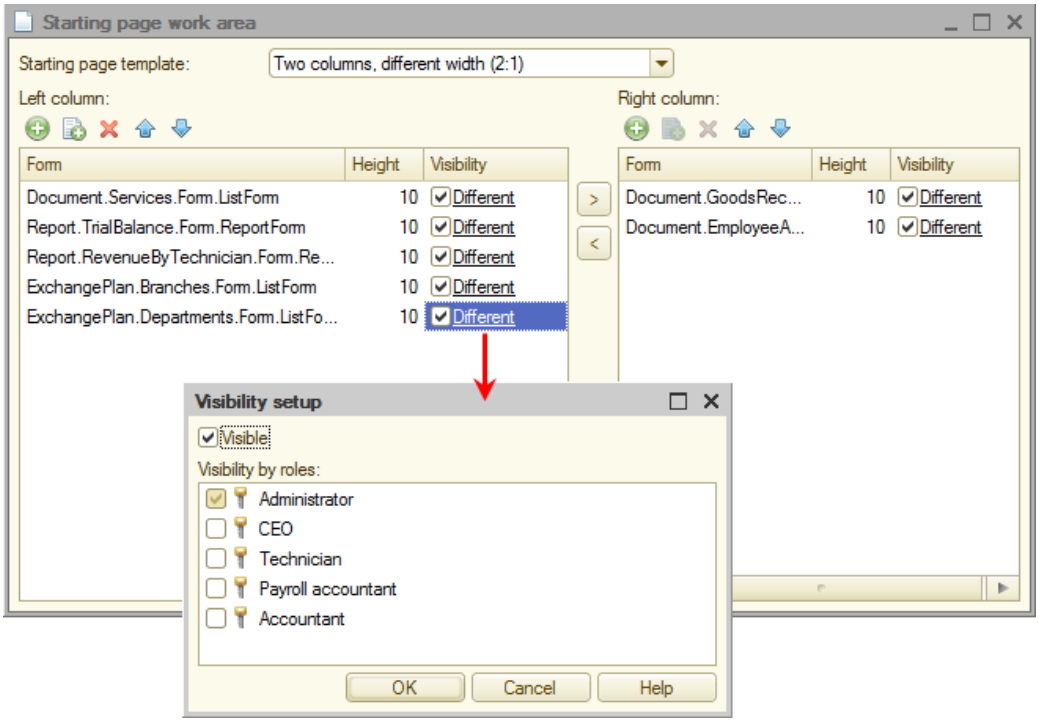


Fig. 24.30. Start page settings for Administrator

Then let us test the script-based exchange.

Note. You have to specify the source and target nodes correctly. If data is modified in the central infobase, you have to select the target subordinate node (**Department**) from the list of exchange plan nodes and then click **Write changes**. And in the target node infobase you have to select the source node (**Central infobase**) and then click **Read changes**.

1. Run 1C:Enterprise as **Administrator**.

   This opens the applied solution start page (fig. 24.31).

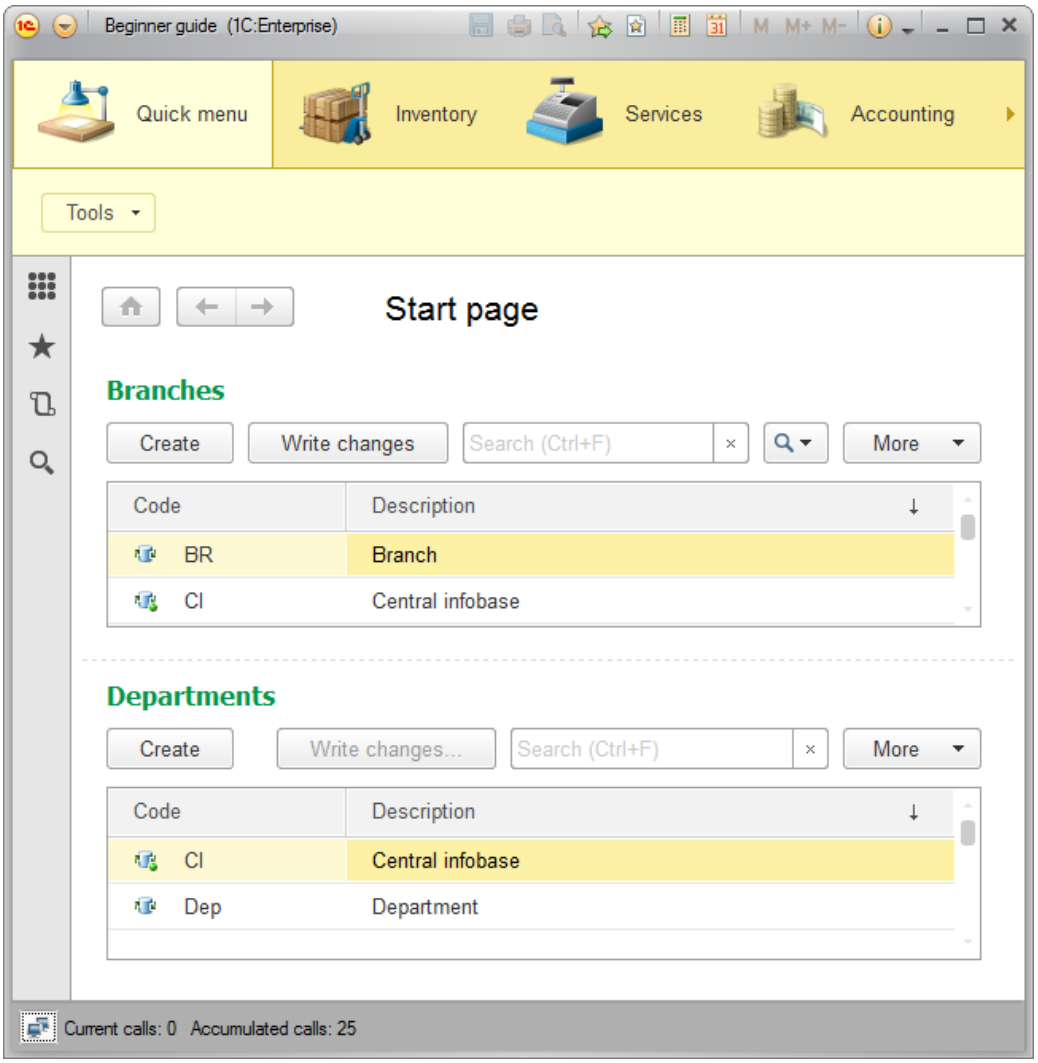


Fig. 24.31. Start page of a user with Administrator role

2. In the **Quick menu** section, on the **Tools** menu, click **Departments exchange**.
3. In the **Department** field, select the **Department** node (fig. 24.32).

   It is the node for which you will create an initial image.

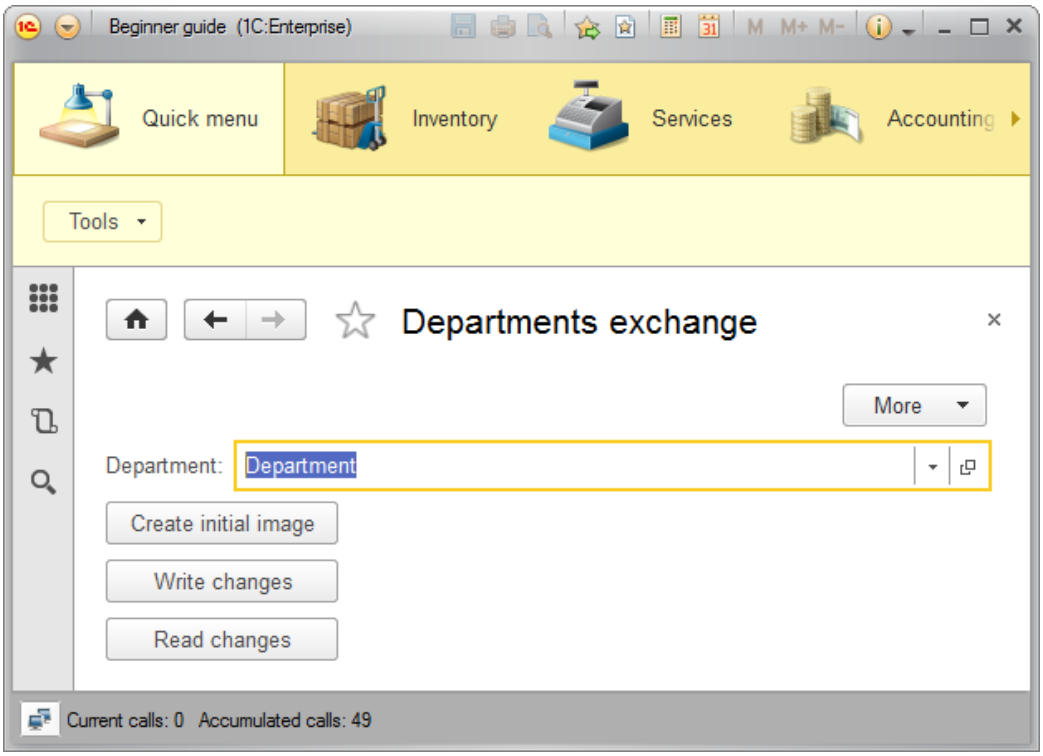


Fig. 24.32. Example of script-based exchange with departments

4. Click **Create initial image**.
5. Create a directory for storing the initial image, select it in the directory selection dialog box, and click **Select Folder**.
6. Create a new employee in the central infobase.
7. In the **Departments exchange** form, in the **Department** field, select **Department**.

   It is the node where you need to transfer the changes.
8. Click **Write changes** and specify the name of the file where you want to store the changes.
9. Open the department infobase (based on the initial image that you created in steps 4-5) in Designer.
10. On the **Administration** menu, click **Users** and create a single user: *Administrator* with **Administrator** role.
11. Open the department infobase in 1C:Enterprise mode.
12. In the **Quick menu** section, in the **Departments exchange** form, in the **Department** field, select **Central infobase**.

    It is the node whose changes you need to transfer to the current infobase.
13. Click **Read changes** and specify the name of the file that stores the changes.
14. Ensure that the new employee with a code having the CI prefix is present in the department infobase.

And now we will make a few final observations.

In a distributed infobase scenario four events become available for the ExchangePlanObject.<name> object:

- OnSendDataToMaster()
- OnSendDataToSlave()
- OnReceiveDataFromMaster()
- OnReceiveDataFromSlave()

You can use these events to manage sending and receiving data at the level of individual data items. They are called for each data item included in the message.

You can watch these events in action by adding the following script to the module of the Departments exchange plan object (listing 24.33).

Listing 24.33. Watching the execution of the ExchangePlanObject object events

```
Procedure OnSendDataToMaster(DataItem, ItemToSend)

    Message = New UserMessage;
    Message.Text = "OnSendDataToMaster: " + DataItem;
    Message.Message();

EndProcedure

Procedure OnSendDataToSlave(DataItem, ItemToSend)

    Message = New UserMessage;
    Message.Text = "OnSendDataToSlave: " + DataItem;
    Message.Message();

EndProcedure

Procedure OnReceiveDataFromMaster(DataItem, ItemToReceive, SendBack)

    Message = New UserMessage;
    Message.Text = "OnReceiveDataFromMaster: " + DataItem;
    Message.Message();

EndProcedure

Procedure OnReceiveDataFromSlave(DataItem, ItemToReceive, SendBack)

    Message = New UserMessage;
    Message.Text = "OnReceiveDataFromSlave: " + DataItem;
    Message.Message();

EndProcedure
```

The first parameter in all of these events is the data item for which the event is called.

The ItemToSend parameter defines which data is stored to the message. It can have one of the three values:

- **Auto**. The data item is included in the message. It is the default value.
- **Delete**. A value indicating that the data item that should be deleted is included in the message.
- **Ignore**. No data related to the data item is included in the message.

The ItemToReceive parameter defines whether the read data item is recorded to the database. It can have one of the three values:

- **Auto**. If the data item is received from the master node, it is always recorded. If the data item is received from a subordinate node, it is only recorded if no changes are registered for that data item. It is the default value.
- **Accept**. A received data item is always recorded.
- **Ignore**. A received data item is ignored (so it is not recorded).

Data receipt events also have a third Boolean parameter: SendBack. This parameter enables forced registration of changes for the received item in the target infobase.

You might need this feature if an inconsistency is detected in the data received from the source node (for example, an error occurred during data modification in the source node).

If this is the case, you can ignore the received change and set the SendBack flag to force recording the changes of the received data items for the source node in the target infobase.

After the subsequent exchange, the state of that data item in the source node is taken from the target infobase.

Modifying distributed infobase node structure

In conclusion, we need to point out that the distributed infobase feature includes software tools for reconfiguring the node structure.

The SetMasterNode() method of the ExchangePlansManager object is used for this purpose.

A reference to the exchange plan node of a distributed infobase that becomes the master for the current infobase is passed as a method parameter. You can pass the Undefined value to indicate that the current infobase has no master node.

Suppose you need to move one of the subordinate nodes to the root (fig. 24.33).

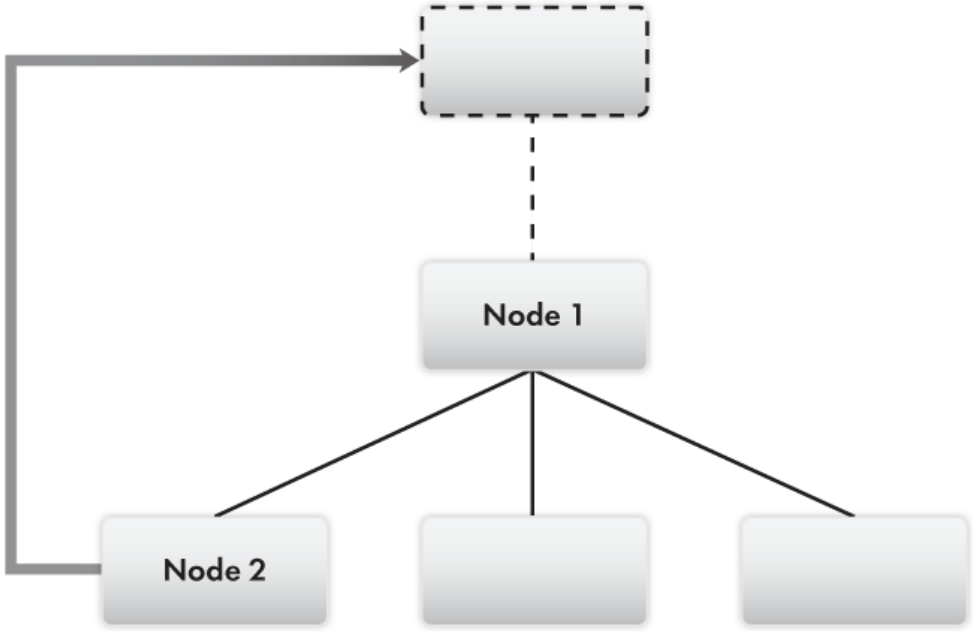


Fig. 24.33. Reconfiguring node structure

To do so, you have to run the script shown in listing 24.34.

Listing 24.34. Moving Node2 to the tree root

```
// In the Node2 infobase
ExchangePlansManager.SetMasterNode(Undefined);

// In the Node1 infobase
ExchangePlansManager.SetMasterNode(Node2);
```

This script deletes all the Node1 configuration change records for Node2 because after its execution the transfer of configuration changes is only allowed from Node2 to Node1. Data change records are not deleted because data exchange between these nodes is still allowed.

In a similar manner you can use the Undefined value of the method parameter to detach an individual infobase or a subtree from the tree structure (fig. 24.34, listing 24.35).

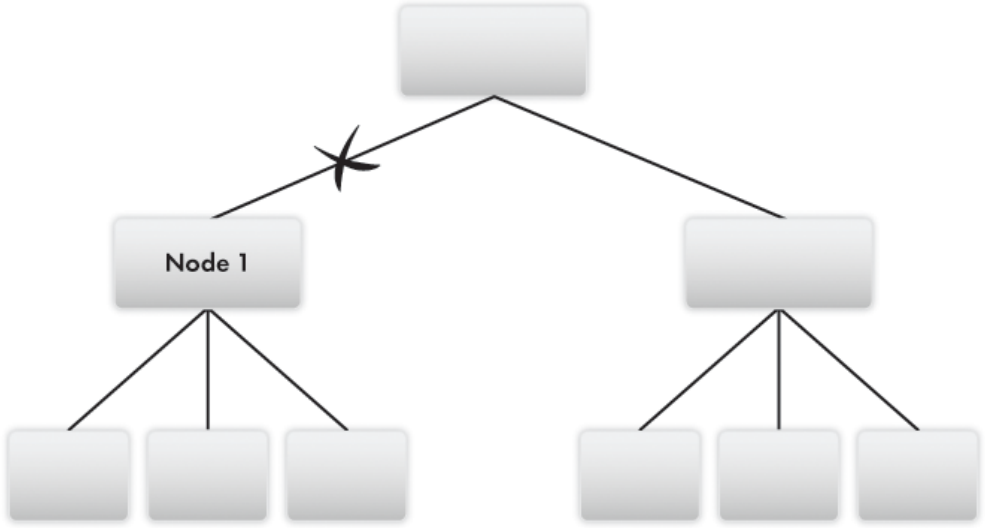


Fig. 24.34. Detaching a subtree from a distributed infobase

Listing 24.35. Detaching a subtree

```
// In the Node1 infobase
ExchangePlansManager.SetMasterNode(Undefined);
```

In addition, you can assemble a distributed infobase from individual infobases with identical configurations (fig. 24.35, listing 24.36).

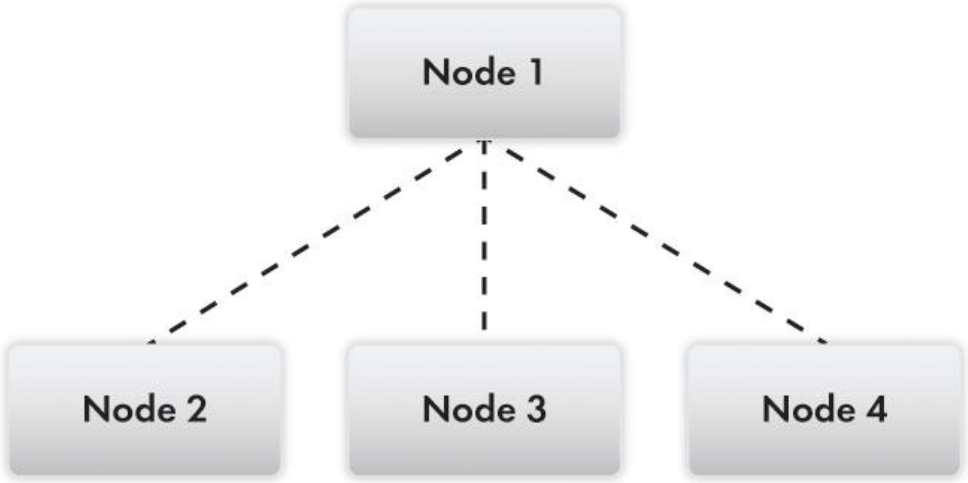


Fig. 24.35. Infobase composed of individual infobases with identical configurations

Listing 24.36. Assembling a distributed infobase from infobases with identical configurations

```
// In the infobases of Node2, Node3, and Node4
ExchangePlansManager.SetMasterNode(Node1);
```

Quiz

- What features are included in the universal exchange feature?
- What is the purpose of the Exchange plan configuration object?
- What are the main components of an exchange plan?
- What are the exchange plan nodes?
- What is the content of an exchange plan? What data items support data exchange?
- What is AutoRecord?
- What is the puropse of change registration?
- How does message infrastructure work?
- What is the purpose of XML serialization?
- What is the purpose of XML document read/write feature?
- How does one create an exchange plan?
- How does one prepare a configuration for data exchange?
- How does one implement data exchanges in the general case?
- How does one implement data exchanges in a distributed infobase?
- How does one use 1C:Enterprise script to manage data exchanges in a distributed infobase?
- What are the specifics of exchanging data that includes predefined items?
- How does one change the tree structure of a distributed infobase?

Lesson 25 (0:30). Functional options

So now you have created a small-sized applied solution that automates operations at Jack of All Trades maintenance company. And here comes the change-up pitch.

The staff at Jack of All Trades liked your applied solution so much that they told their neighbors at the Beauty Queen makeup studio about it. The staff at the studio saw how your applied solution works and then asked you to automate their operations as well.

Naturally, you were delighted to accept, for one simple reason: what you have created is a universal configuration, which is suitable for automation of almost any kind of service activities.

All that you need to do to adjust your confuguration for a makeup studio is simply create a new infobase with your configuration and populate it with new data, including employees, new services, and so on. All the accounting tools you have created are not tied to any specific features of a company so they can easily be used in any other company that has similar activities.

So, even if the makeup studio management wants you to add some new functionality, you can do it by editing just a few modules in your configuration. This is much more efficient that recreating an applied solution from scratch for this specific company.

But at the same time a makeup studio might not need some of the features that are already available in your configuration.

So what can you do? Do you have to delete unwanted configuration objects and script?

This can take a lot of time and effort. Real-life configurations can include a large number of configuration objects, which can refer each other in a complex way.

This is why 1C:Enterprise supports the functional options feature. Functional options serve for enabling or disabling functionality blocks during the deployment without changing the configuration itself.

Functional options allow developers to separate some applied solution functionality, so that it can be enabled or disabled during the deployment or at run time.

Accounting and Payroll options

Suppose that for some reason the makeup studio does not have accounting and payroll calculation. To disable these features, let us create functional options named Accounting and Payroll, assign these options to appropriate configuration objects, and disable them in 1C:Enterprise mode.

So, while the makeup studio has exactly the same configuration as Jack of All Trades, their applied solution does not display any accounting or payroll options, as if they do not exist at all.

In Designer mode

Let us create the functional options.

1. Add constants named *Accounting* and *Payroll* of the **Boolean** type (fig. 25.1).

   This is where you will store functional option values.

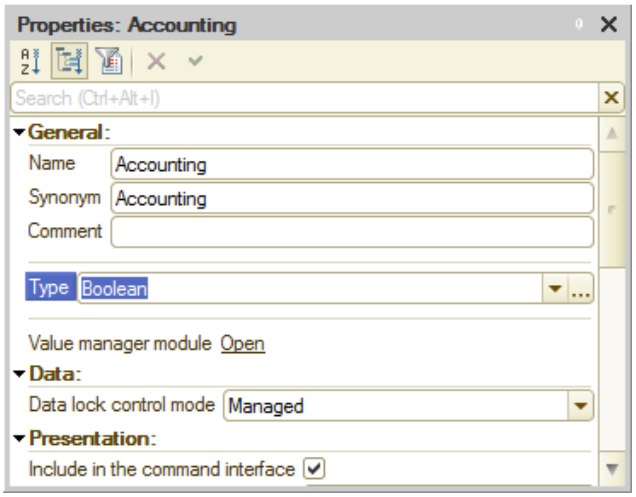


Fig. 25.1. Creating a constant

   If a constant value is True, the functional option is enabled. If the value is False, the functional option is disabled.

2. In the configuration tree, expand the **Common** branch, click the **Functional options** branch, and create functional options named *Accounting* and *Payroll* with their respective constants specified in the **Data path property** (fig. 25.2).

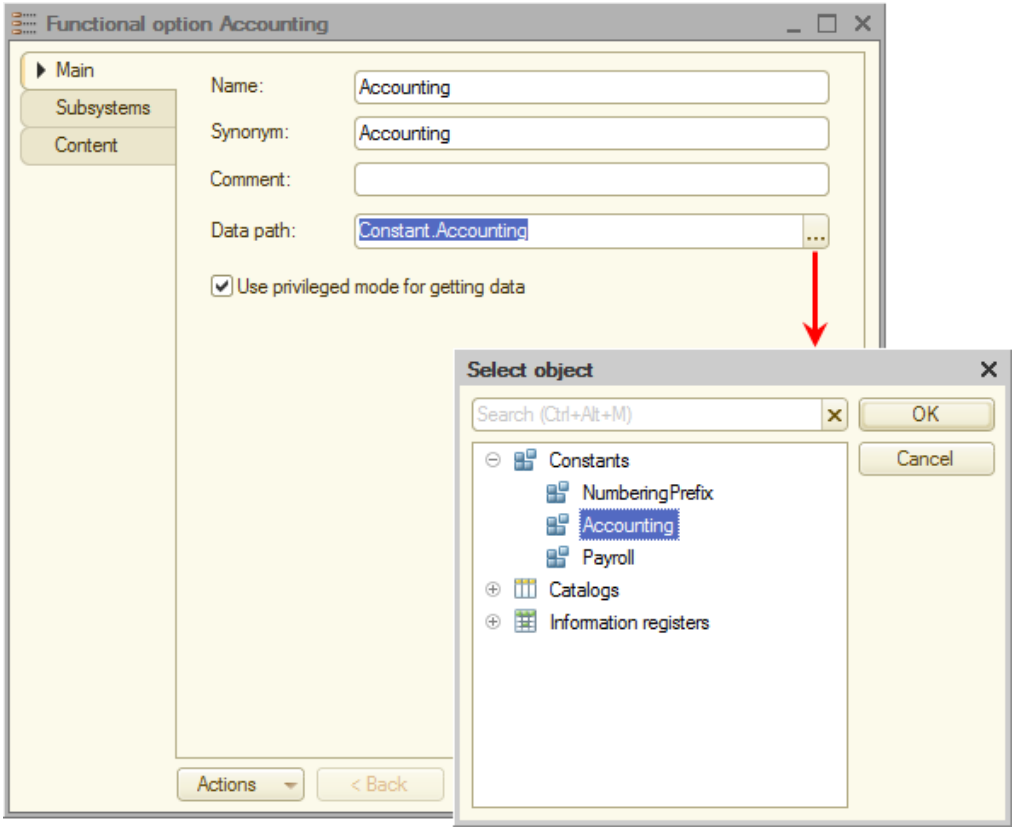


Fig. 25.2. Creating a functional option

Then let us associate configuration objects with functional options.

The following configuration objects are related to accounting:

- **ExtraDimensions** catalog
- **InputOpeningMaterialBalances** document
- **TrialBalance** report
- **ExtraDimensionTypes** chart of characteristic types
- **Main** chart of accounts
- **Primary** accounting register

3. In the **Payroll** functional option configuration object editor, on the **Content** tab, select these objects (fig. 25.3).

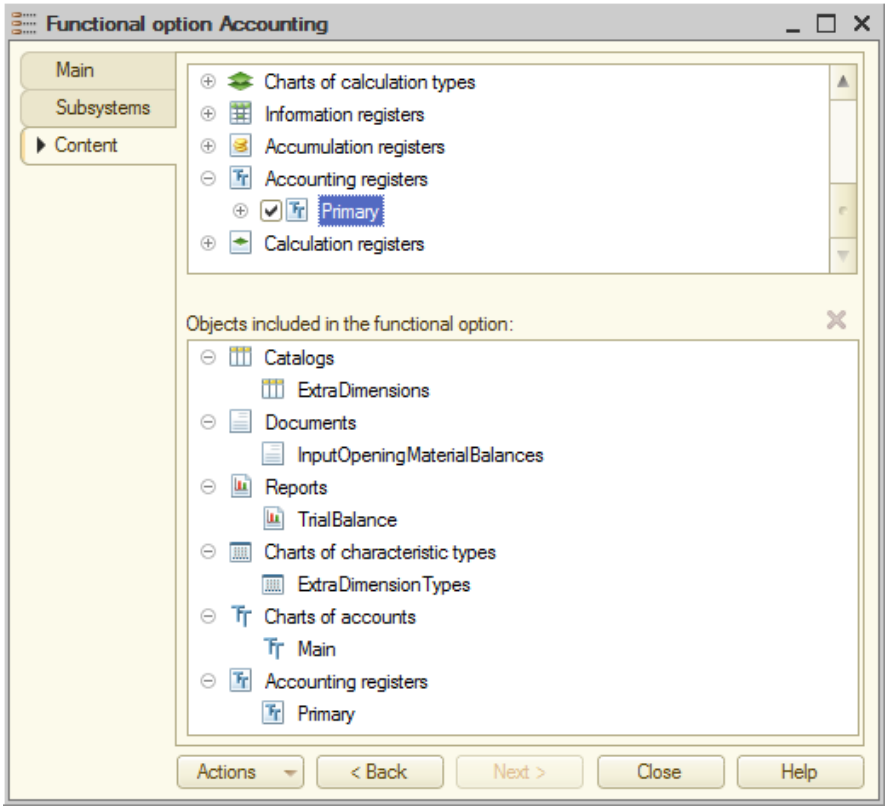


Fig. 25.3. Accounting functional option content

The following configuration objects are related to payroll:

- **WorkScheduleTypes** catalog
- **EmployeeAccruals** document
- **EmployeeAccruals** report
- **Recalculation** report
- **AccrualsChart** report
- **MainAccruals** chart of calculation types
- **WorkSchedules** information register
- **Accruals** calculation register

4. In the **Accounting** functional option configuration object editor, on the **Content** tab, select these objects.

   Now if you open the configuration object editor for the ExtraDimension catalog or for any other configuration object that belongs to the Accounting functional option, you can see that the functional option is selected on the Functional options tab (fig. 25.4).

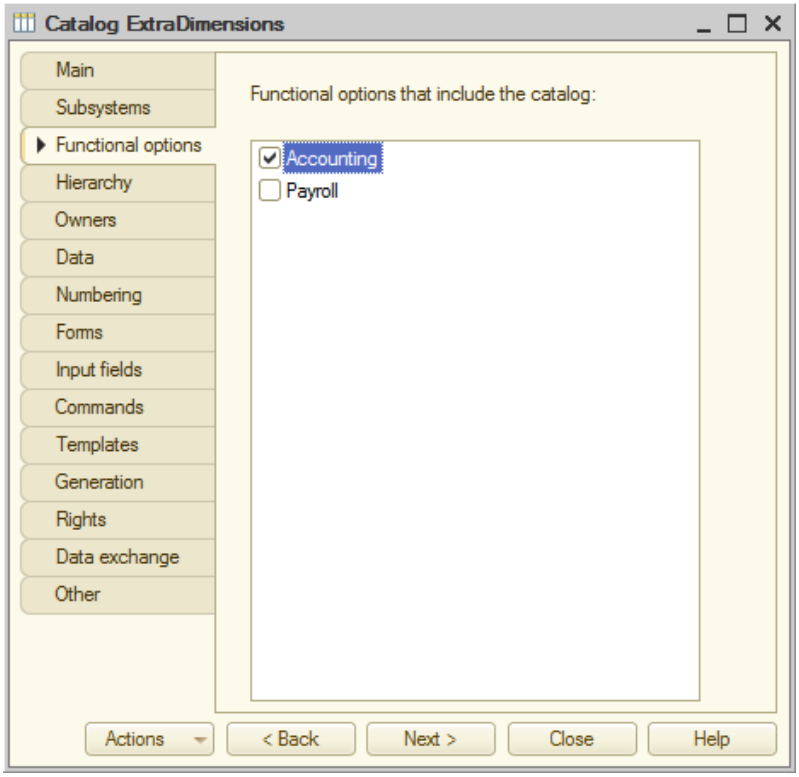


Fig. 25.4. Functional option value for an accounting object

So configuration objects are displayed in the applied solution interface if their respective functional options are enabled (the appropriate constants are True).

And for objects related to payroll you can see that the Payroll functional option is selected on the Functional options tab of the object editors.

The rest of the configuration objects do not have any items selected on the Functional options tab. If a functional option is disabled for an object, this object does not depend on the value of this functional option and is always displayed (fig. 25.5).

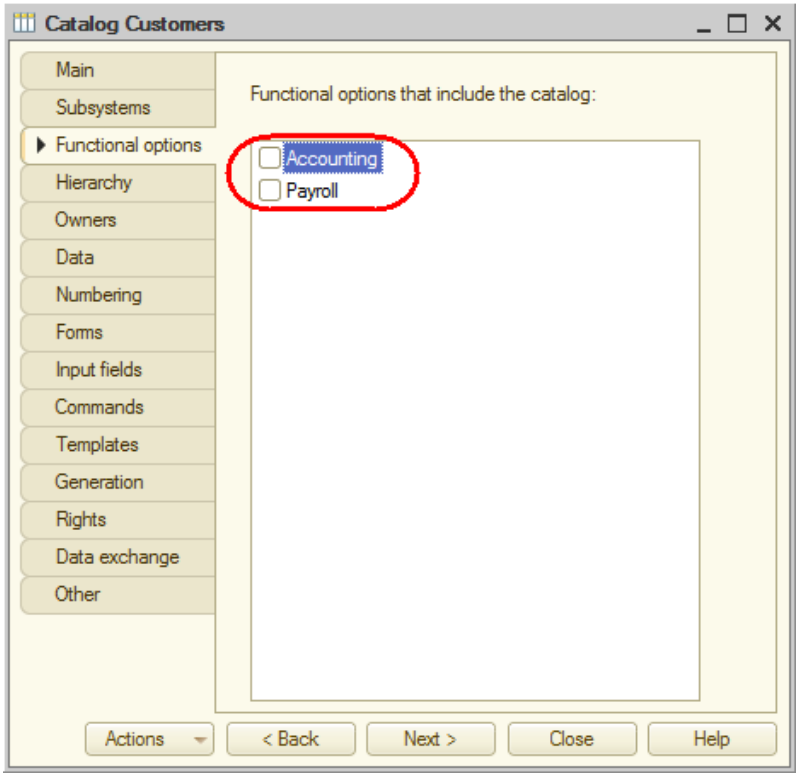


Fig. 25.5. Functional option settings for independent objects

Note. If you include a subsystem into a functional option, you will not see the respective section in 1C:Enterprise mode at all when this functional option is disabled.

5. Expand the **Common forms** branch and open the common constants form.

   It is the GeneralSettings form that you created in the previous lesson. Currently it contains the NumberingPrefix constant.

   Now you need to add the new constants to this form to be able to change the functional option values at run time.

6. In the **GeneralSettings** form, on the **Attributes** tab, expand the main **ConstantsSet** attribute and drag the Accounting and Payroll constants to the form controls pane (fig. 25.6).

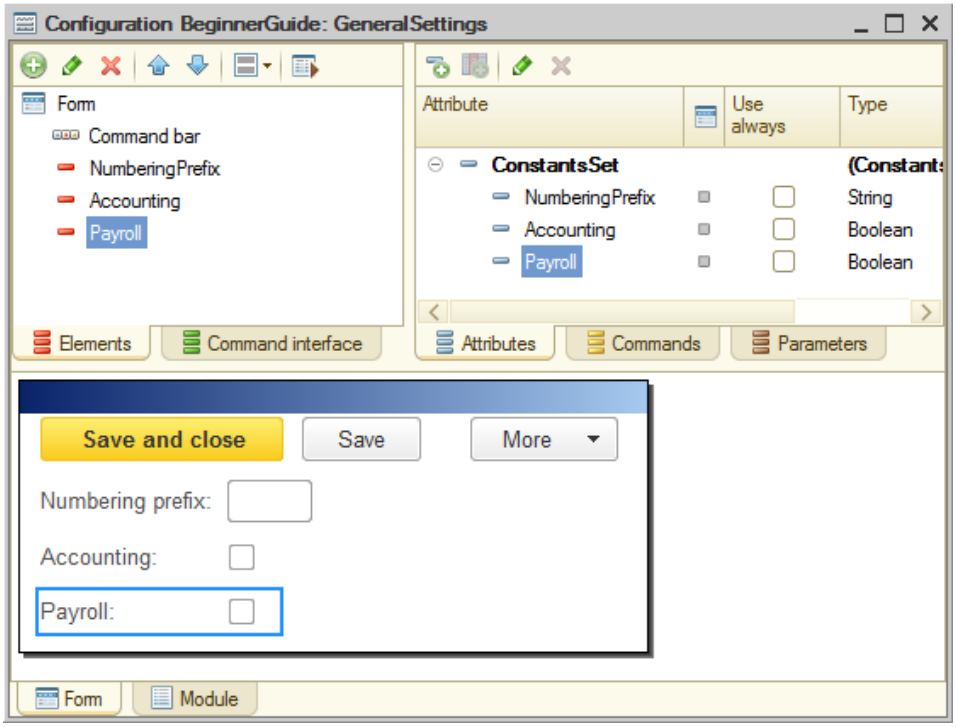


Fig. 25.6. Editing common constants form

In 1C:Enterprise mode

Let us test the functional options.

1. Start 1C:Enterprise in the debug mode.
2. In the **Quick menu** section, click **General settings**.

   This opens the constants form and you can see that both constants are False (fig. 25.7).

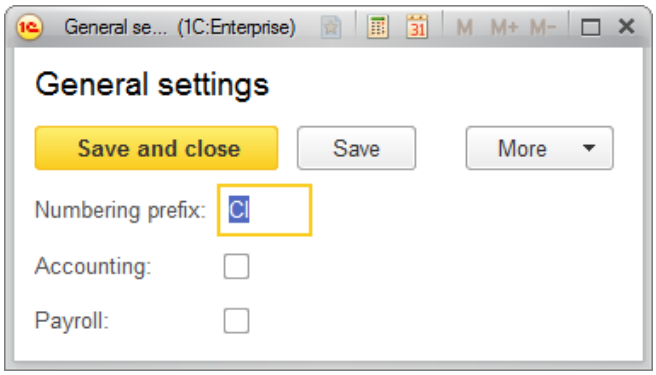


Fig. 25.7. Constants form

It means that the respective functional options are disabled.

True, you can see that the commands related to accounting and payroll are missing from the Accounting and Payroll sections (fig. 25.8).

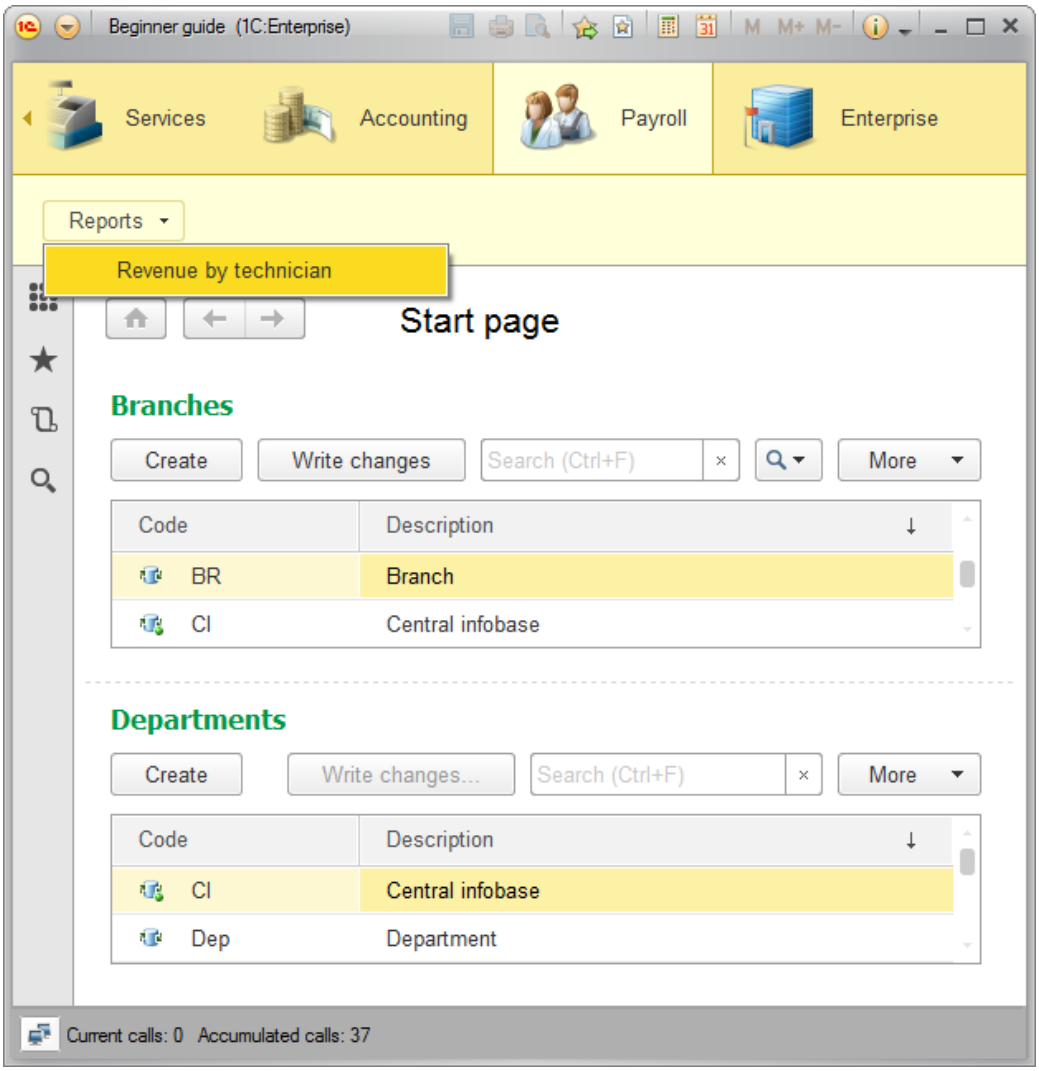


Fig. 25.8. Payroll section interface

So if after a while the management of the makeup studio decides to start using payroll, their administrator will simply enable the Payroll option (fig. 25.9).

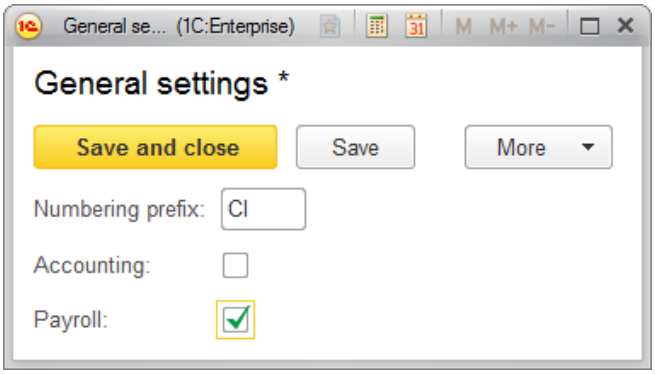


Fig. 25.9. Changing the value of the Payroll functional option

3. In the **General settings** form, enable the **Payroll** accounting option, and then restart 1C:Enterprise.

   The restart is required to display the new applied solution interface.

After the restart the Payroll section should look as shown in fig. 25.10.

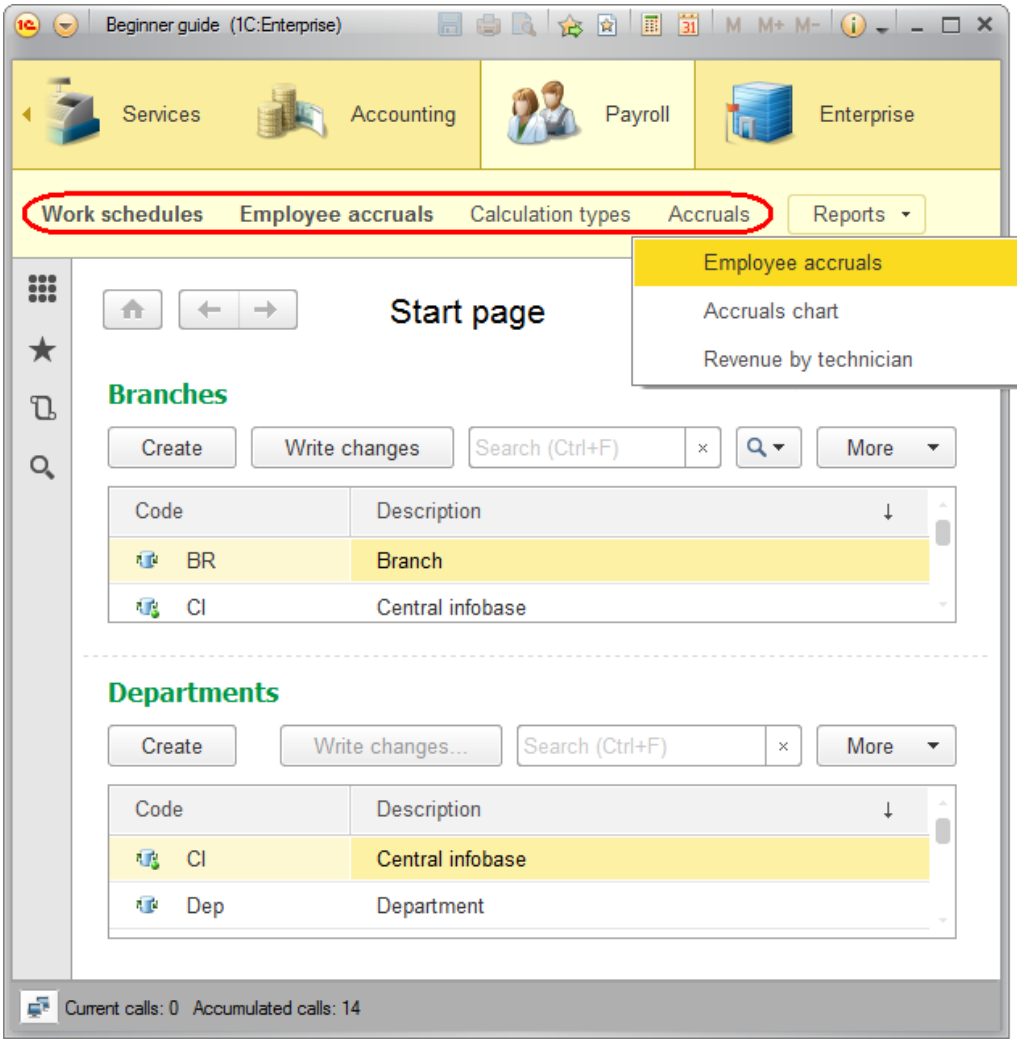


Fig. 25.10. Payroll section interface

And if you enable the Accounting functional option as well, this restores the applied solution interface initially developed for Jack of All Trades.

This is how an applied solution can be quickly and easily customized in compliance with customer requirements.

Accounting by customers option

Note that functional options can affect not only the applied solution command interface but also the appearance of applied solution forms. Besides, functionality can be enabled or disabled without restarting the client application. You can also work with functional options using 1C:Enterprise script. With all that, the functional options feature makes deployment and customization simple and straightforward even for beginner users.

Now let us look into another example.

Some companies do not need to record customer names. Often only the fact that a service is rendered matters while the personality of a customer does not.

So let us implement the option to disable maintenance of a customer list to prevent users from having to specify the customer every time a service is rendered.

Let us also update existing functional options by including the Accounting and Payroll subsystems in these options. This is needed for the solution to look complete: if accounting is not needed, it should not be present anywhere.

In Designer mode

First, let us add subsystems to the functional options.

1. Open **Accounting** functional option content and add the **Accounting** subsystem.
2. Open **Payroll** functional option content and add the **Payroll** subsystem (fig. 25.11).

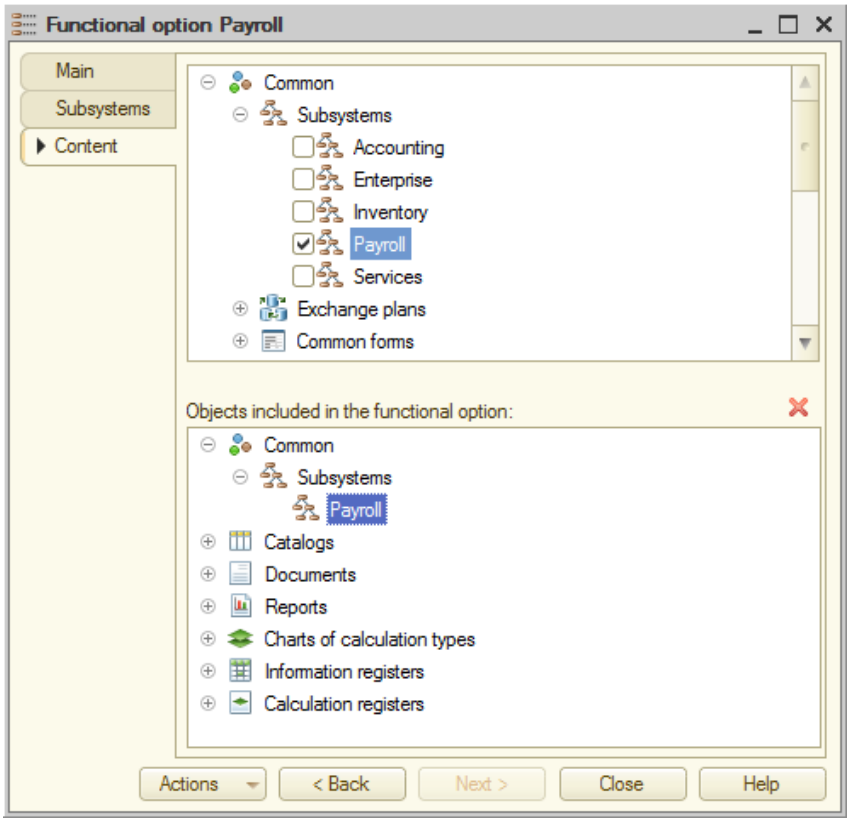


Fig. 25. 11. Payroll functional option content

Now let us proceed to creating a new functional option. You need a constant to store the option value.

3. Add a constant named *AccountingByCustomers* of **Boolean** type (fig. 25.12).

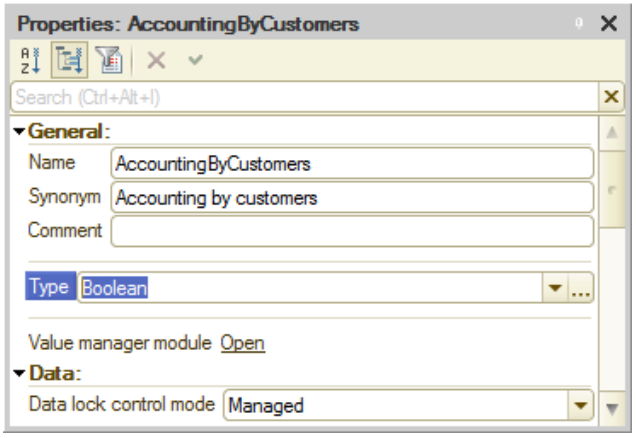


Fig. 25.12. AccountingByCustomers constant

4. Add a functional option named *AccountingByCustomers* and specify that its value is stored in the **AccountingByCustomers** constant (fig. 25.13).

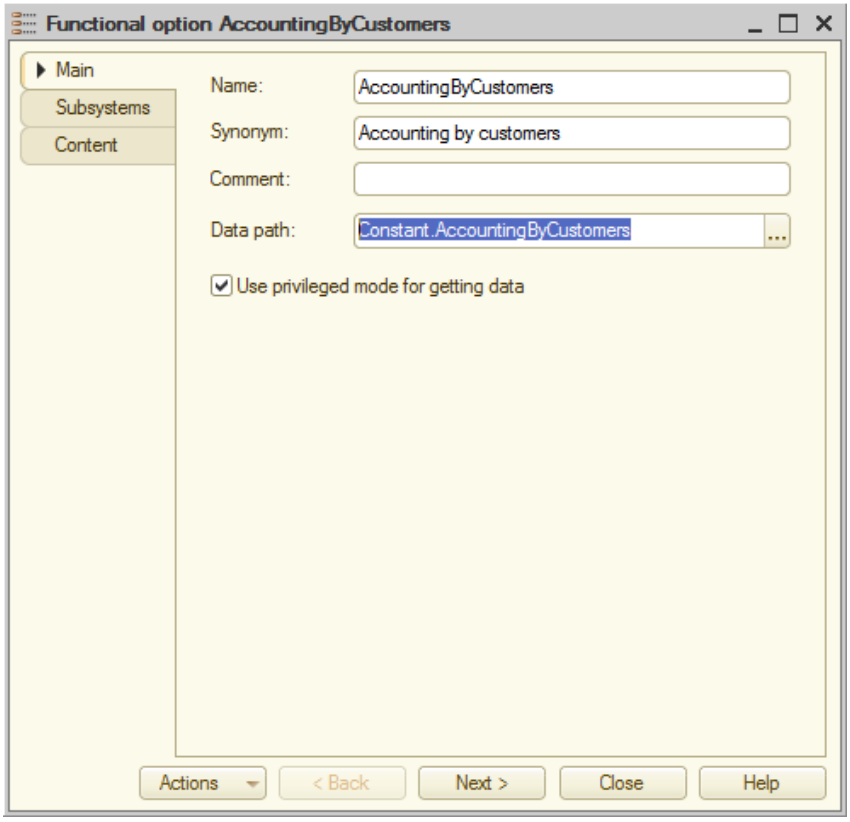


Fig. 25.13. AccountingByCustomers functional option

5. On the **Content** tab, specify the objects to be included in the functional option (fig. 25.14):
   - **Customers** catalog
   - **Customer** attribute of the **Services** document
   - **Customer** dimension of the **Sales** accumulation register

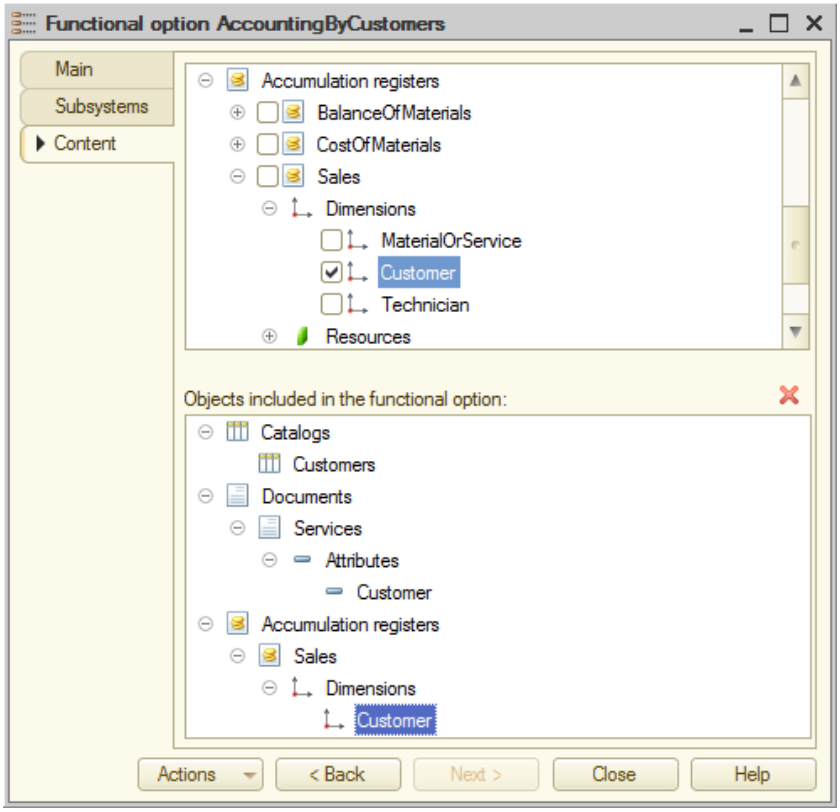


Fig. 25.14. AccountingByCustomers functional option content

Let us update the GeneralSettings common form that is used to specify functional option values.

6. In the **GeneralSettings** form editor, drag the **AccountingByCustomers** constant to the form controls (fig. 25.15).

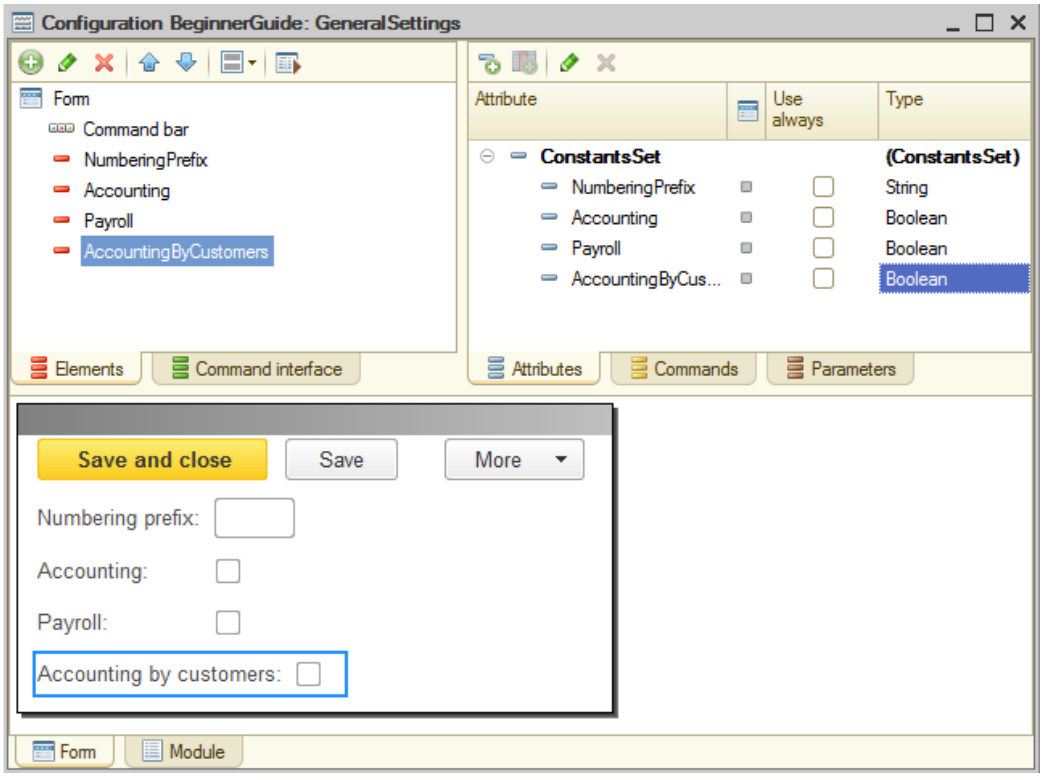


Fig. 25.15. GeneralSettings form

Next you have to ensure that the applied solution interface is refreshed when new functional option values are applied.

7. In the form controls tree, double-click the root **Form** item.
8. In the property palette, find the **After write at client** event and click the **Open** button in that field.
9. Specify that the handler is only executed on the client.
10. In the form module that is opened, add a single line to the handler body, as shown in listing 25.1.

Listing 25.1. AfterWrite form event handler

```
&AtClient
Procedure AfterWrite(WriteParameters)

    RefreshInterface();

EndProcedure
```

RefreshInterface() is a global context method that refreshes the command interface, the start page, and opened forms according to the current functional option values and their parameters.

To simplify testing the functional options, let us modify the common form so that it does not block the main applied solution window.

Currently, when you open the GeneralSettings form in 1C:Enterprise mode, you cannot change anything in the main applied solution window because it is blocked while the General Settings form is open. This happens because the form wizard selected the default Lock parent window value for the Window open mode property.

11. Click the **Form** tab and then, in the form controls tree, double-click the root **Form** item.
12. In the property palette, set **Window open mode** to **Independent** (fig. 25.16).

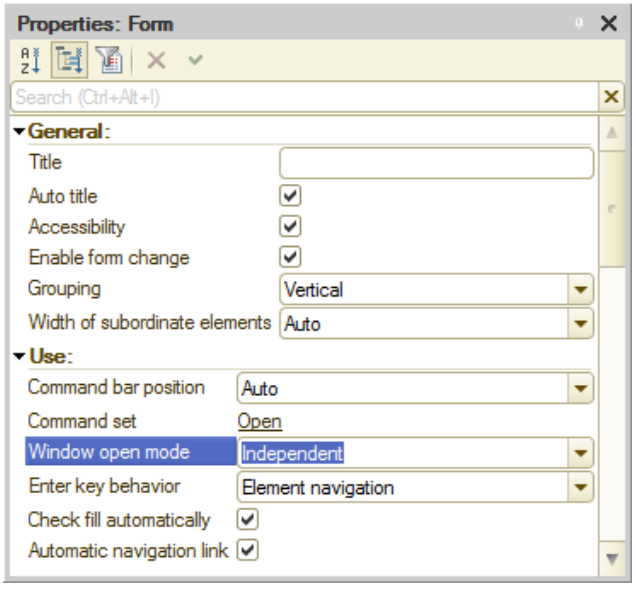


Fig. 25.16. Window open mode property

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. In the **Quick menu** section, click **General settings**.

   You can see that the Accounting by customers option is disabled (fig. 25.17).

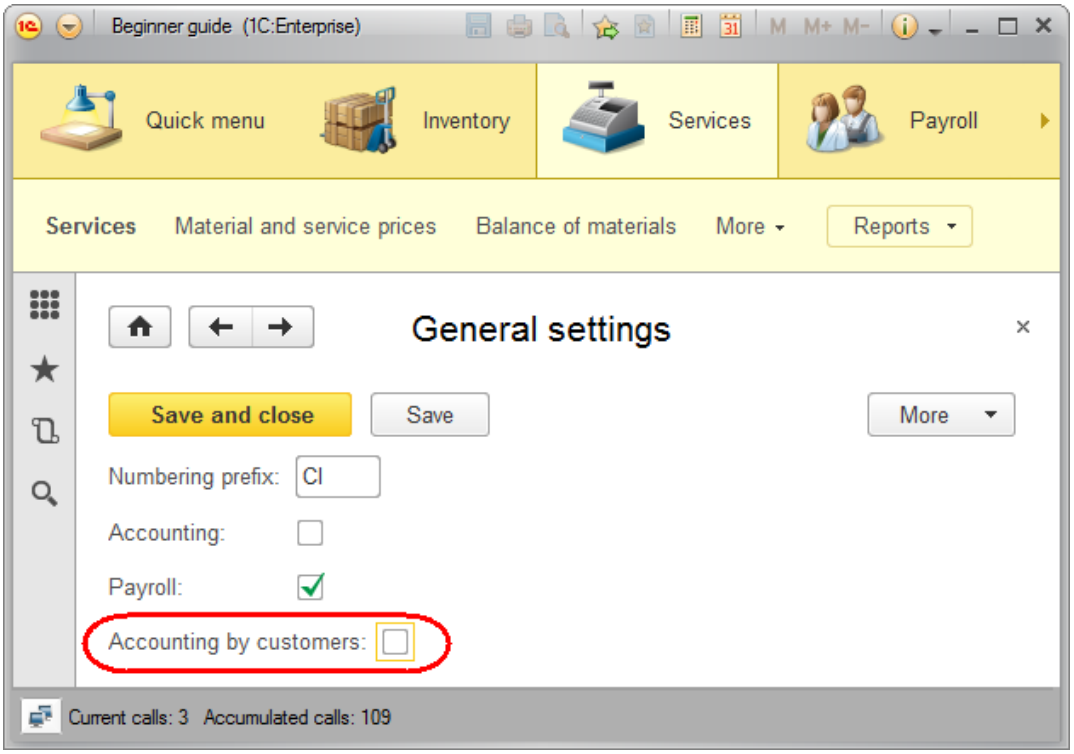


Fig. 25.17. Accounting by customers functional option is disabled

Let us check how this affects the Services document.

3. In the **Services** section, click **Services** and open any of the documents, for example, Services #2 (fig. 25.18).

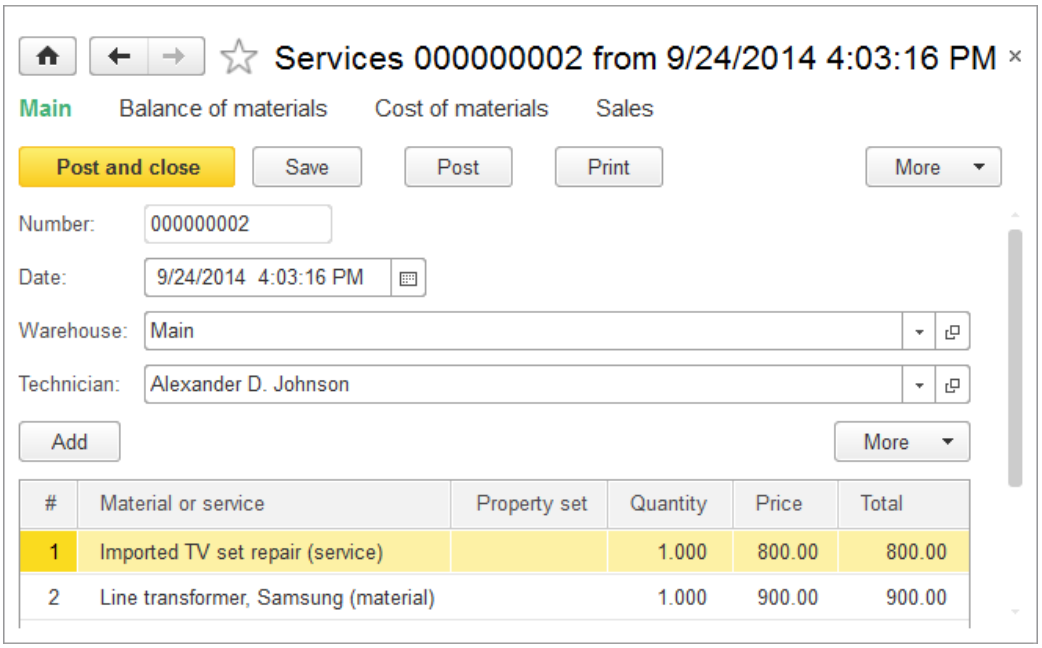


Fig. 25.18. Services document

The document does not have the Customer field, and if you switch to the list of Sales register records using the navigation panel command, you can see that it no longer has the Customer column. Also, the Customers command is absent from the Enterprise section.

4. Return to the **General settings** form, select the **Accounting by customer** check box, and click **Save**.

   This introduces some changes to the applied solution interface. Return to the Services #2 document (fig. 25.19).

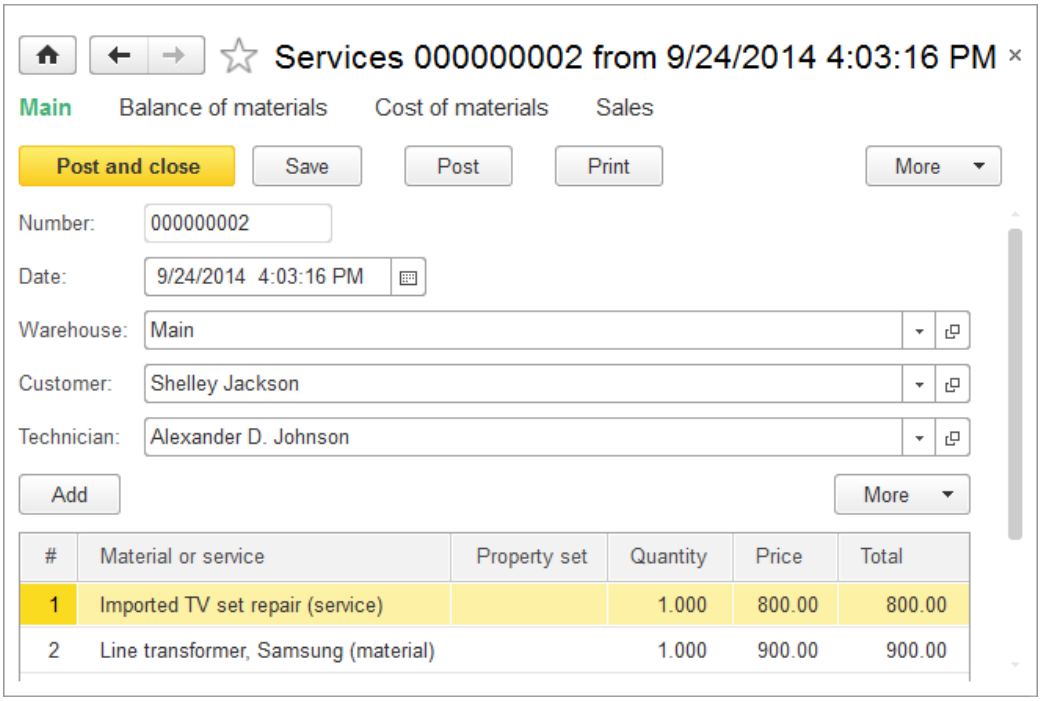


Fig. 25.19. Services document

The document has the Customer field, the list of register records has the Customer column, and the Customers command is again available in the Enterprise section.

5. In the **General settings** form, enable all functional options and click **Save** (fig. 25.20).

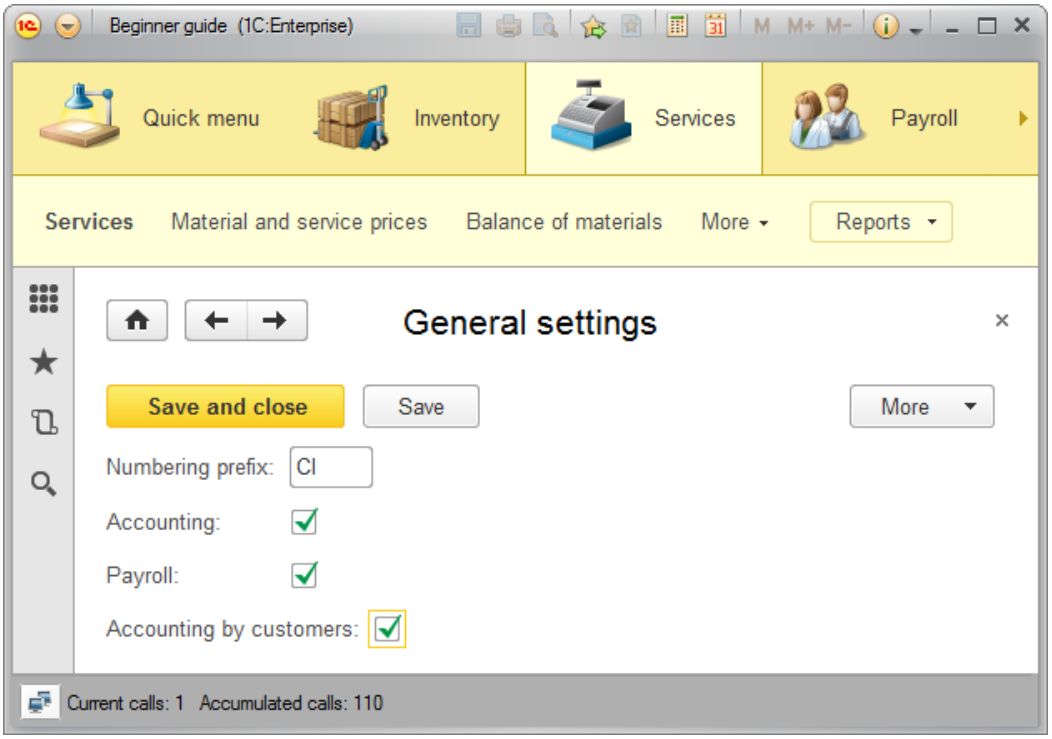


Fig. 25.20. General settings

This enables all of the applied solution features. Note that the applied solution interface is refreshed immediately once you save the settings.

This completes the configuration development.

The next two lessons are devoted to standard 1C:Enterprise development techniques. For some examples we will provide multiple implementation options, so it is up to you to decide which one to choose.

Quiz

- What is a functional option? What is the purpose of a functional option?
- How does one modify applied solution interface using functional options?

Lesson 26 (2:00). Picking list items, avoiding modal windows, and generating data based on other data

In this lesson you will learn the development techniques for picking items from lists, avoiding modal windows, and generating data based on other data available in the infobase.

Picking list items

Picking items from a list is typically used for filling document tabular sections with data selected by user from some object list.

We will illustrate this feature with a task of picking catalog items for filling a document tabular section, which is the most widely used scenario.

Since picking list items is implemented at the form level, it is easy to adapt it to other scenarios. These scenarios can involve other applied object types, but the algorithm is still the same.

For picking items, you have to open a catalog form as subordinate to a document form (or to some document form item). You can use any catalog form and any method of getting that form. What is important is that the form must be opened as a subordinate one.

The result of picking is available in the ChoiceProcessing event handler of the document form (or the form item, if the catalog item form is subordinate to that form item).

The ChoiceProcessing event occurs when any of the following conditions are met:

- Interactive selection is performed in the catalog form
- The NotifyChoice() method is called in the catalog form

We will illustrate various item picking options using the following example: picking items of the MaterialsAndServices catalog to fill GoodsReceipt documents.

Picking a single item

Picking a single item works as follows: once you select an item, the catalog form is closed. In order to select another item one must perform a new pick operation.

In Designer mode

Let us implement picking a single item.

1. In Designer, open the **GoodsReceipt** document form.
2. On the **Commands** tab, create the **Pick** command.

3. In the command property palette, in the **Action** field, click the **Open** button and specify that the command is executed on the client.
4. Click the **Form** tab and drag the command to the form controls pane, to the command bar of the **Materials** table (fig. 26.1).

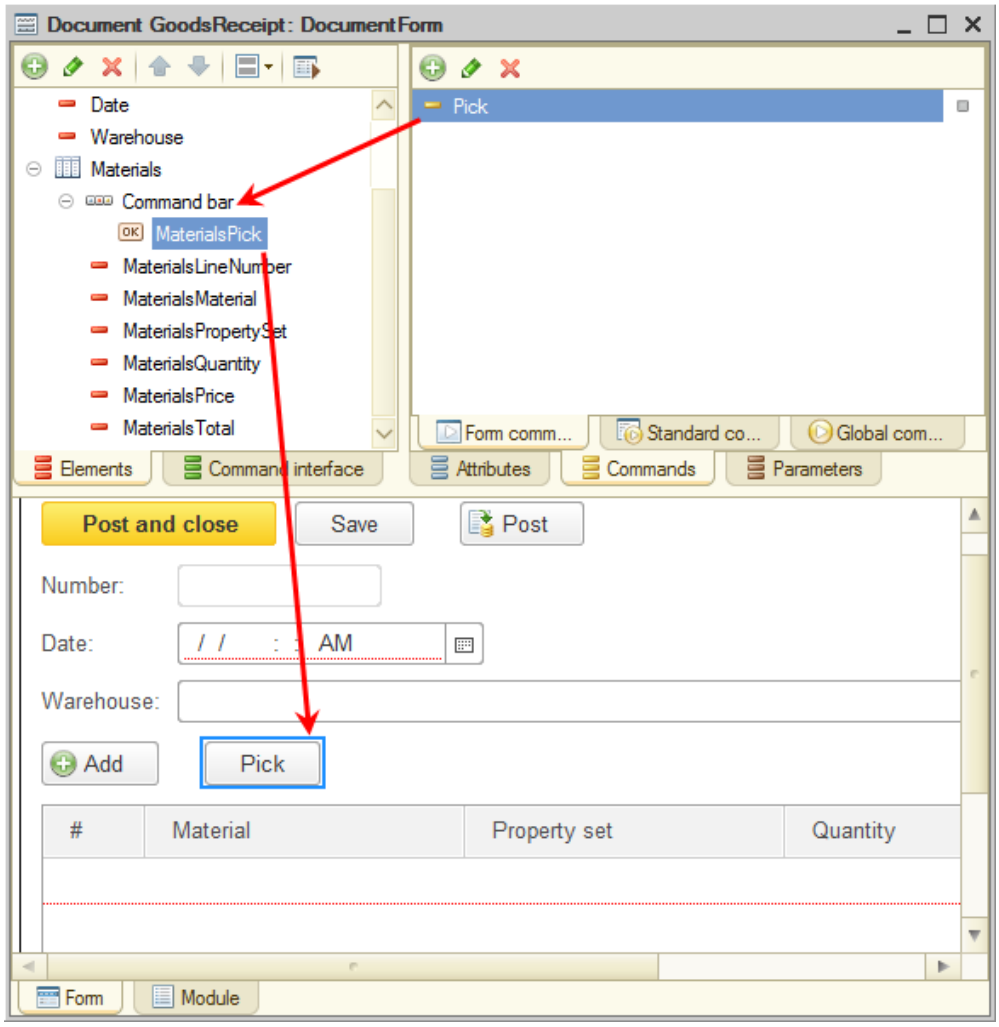


Fig. 26.1. Creating the Pick button

5. Click the **Module** tab and add the script shown in listing 26.1 to the Pick command handler.

Listing 26.1. Pick command handler

```
&AtClient
Procedure Pick(Command)
    OpenForm("Catalog.MaterialsAndServices.ChoiceForm", ,
        Items.Materials);
EndProcedure
```

The procedure opens the MaterialsAndServices catalog selection form, which is subordinate to the Materials table of the GoodsReceipt document (Items.Materials).

Once a user selects a value from the catalog selection form, the value is passed to the ChoiceProcessing event handler of the Materials table because the table is the owner of the selection form.

6. Open the property palette of the **Materials** table and create a ChoiceProcessing event handler (fig. 26.2) that is executed on the client (listing 26.2).

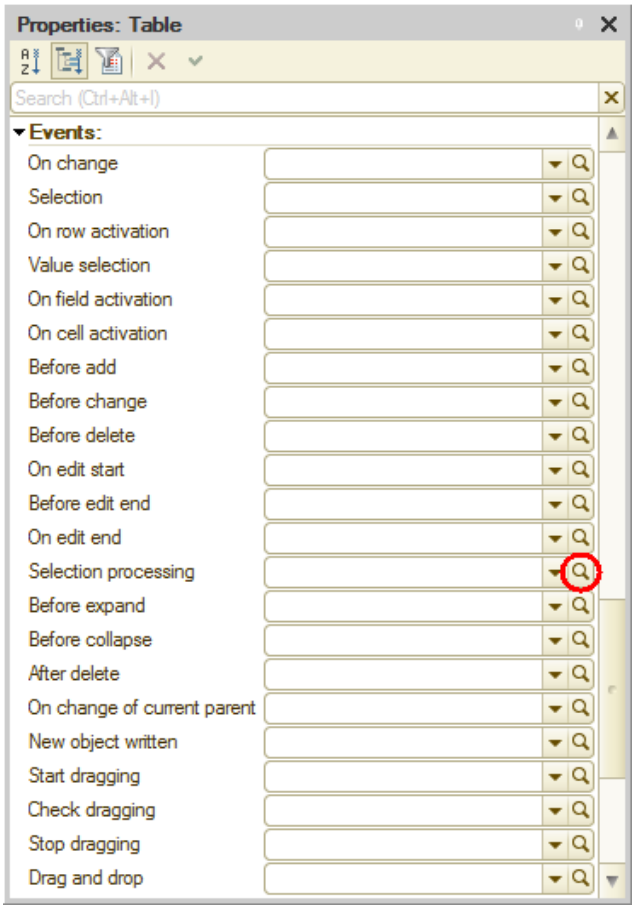


Fig. 26.2. Creating the ChoiceProcessing event handler for the Materials table

Listing 26.2. ChoiceProcessing event handler for the Materials table

```
&AtClient

Procedure MaterialsChoiceProcessing(Item, SelectedValue,
    StandardProcessing)


    Items.Materials.AddRow();

    Items.Materials.CurrentData.Material = SelectedValue;


EndProcedure
```

The procedure adds a row to the Materials table and assigns the value selected in the catalog selection form to the Material column of that row. This value is passed to the handler as the SelectedValue parameter.

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. In the **Inventory** section, open the list of goods receipts and start creating a new one.
3. In the command bar, click the **Pick** button and then double-click an item to select it.

Picking multiple items

In Designer mode

Picking multiple items works as follows: the catalog form is open until the user closes it interactively or until the Close() form method is executed.

- In the **GoodsReceipt** document form, update the **Pick** command handler as shown in listing 26.3.

Listing 26.3. Pick button click handler

```
&AtClient
ProcedurePick(Command)


    FormParameters = New Structure("CloseOnChoice", False);
    OpenForm("Catalog.MaterialsAndServices.ChoiceForm", FormParameters,
        Items.Materials);


EndProcedure
```

In this procedure the form is opened with parameters.

Form parameters are used for opening the form in a specific state. The parameters are stored in a structure. Each structure item describes a single form parameter. The parameter name serves as a structure item key.

The structure is passed to the OpenForm() method as its second parameter (the FormParameters variable).

The structure is generated right before it is passed to the method. It contains a single item with the CloseOnChoice key.

By passing the structure to the OpenForm method, you set the CloseOnChoice form parameter to False.

It means that the form is not closed when a user selects a single item by double-clicking it.

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. In the **Inventory** section, open the list of goods receipts and start creating a new one.
3. In the command bar, click the **Pick** button.
4. Double-click a material to select it, then double-click another material.
5. Open the **Services** group and pick several services.
6. Close the selection form window.

Picking with multiple selection

In Designer mode

There is another picking method available: you can pick multiple items at once.

The option to select multiple items is enabled by default in all list forms. But the option to pick the selected items is usually disabled.

To enable the option, let us use the MultipleChoice parameter of the dynamic list form extension.

1. In the **GoodsReceipt** document form, update the Pick command handler as shown in listing 26.4.

   Listing 26.4. Pick button click handler

   ```
   &AtClient
   Procedure Pick(Command)


       FormParameters = New Structure ("MultipleChoice", True);
       OpenForm("Catalog.MaterialsAndServices.ChoiceForm",
           FormParameters, Items.Materials);


   EndProcedure
   ```

   In the multiple selection mode the form returns an array of items instead of a single item. So you have to iterate through the array in the ChoiceProcessing handler.

2. Update the ChoiceProcessing event handler as shown in listing 26.5.

   Listing 26.5. ChoiceProcessing event handler for the Materials table

   ```
   &AtClient
   Procedure MaterialsChoiceProcessing(Item, SelectedValue,
       StandardProcessing)


   ```

```
    For Each SelectedItem In SelectedValue Do
        NewRow = Object.Materials.Add();
        NewRow.Material = SelectedItem;
    EndDo;

EndProcedure
```

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. In the **Inventory** section, open the list of goods receipts and start creating a new one.
3. In the command bar, click the **Pick** button.

   Let us switch to the flat list view to simplify the picking.
4. On the **More** menu, point to **View mode** and click **List**.
5. Select multiple items by clicking them while holding Ctrl.
6. Click **Select**.

   This adds the selected items to the document tabular section.

Picking multiple items with multiple selection

In Designer mode

The last example combines both methods that you learned. It includes selecting multiple items and picking them without closing the selection form, then selecting some more and picking them, and so on.

This requires passing both form parameters to a form being opened: CloseOnChoice and MiltipleChoice.

- Update the Pick button click handler as shown in listing 26.6.

Listing 26.6. Pick button click handler

```
&AtClient
Procedure Pick(Command)

    FormParameters = New Structure ("CloseOnChoice, MultipleChoice",
        False, True);
    OpenForm("Catalog.MaterialsAndServices.ChoiceForm", FormParameters,
        Items.Materials);

EndProcedure
```

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. In the **Inventory** section, open the list of goods receipts and start creating a new one.
3. In the command bar, click the **Pick** button.
4. Expand the **Services** group, expand the **TV sets** group, select all services in that group, and pick them.
5. Expand the **Materials** group, expand the **Miscellaneous** group, select all materials in that group, and pick them.
6. Close the **Materials and services** selection form window.

Using the NotifyChoice() method

Use the NotifyChoice() form method when the algorithm that generates selection data is complex and you need the user to enter or select some data in addition to selecting the catalog item. In this scenario you have to call the NotifyChoice() method when all the data required for picking is generated.

The NotifyChoice() method alerts the form owner that picking or selection is finished, passes the selected value to the owner, and closes the form unless it is opened in multiple selection mode.

You can also use the NotifyChoice() method when you need to pass a custom data structure in addition to the selected catalog item (or array of items) to the document form.

Avoiding modal windows

We will use the example from the previous section (picking materials and services for a goods receipt tabular section) to explain how to avoid modal windows in the user interface.

In new configurations modal windows are not allowed by default. This is defined by the Modality usage mode configuration property.

Theory

Before you proceed to the practical examples, we will explain why this interface mode is recommended.

As a rule, applications for desktop computers use modal windows. 1C:Enterprise is no exception. Modal windows are handy in scenarios where user input is required for further execution of an algorithm. Such windows block the entire applied solution interface and pause the script execution until they are closed.

But 1C:Enterprise is no longer a desktop-only application, it eventually acquired a web client and mobile device support, which introduced some issues related to modal windows.

In order to display modal windows properly, desktop browsers require specific setup, while mobile browsers do not support modal windows at all.

This is why a special interface mode that does not use modal windows is implemented in 1C:Enterprise. In this mode a window that should normally be modal is displayed within the parent window area and blocks the entire interface. From the user perspective it looks exactly like a modal window because a user cannot perform any other operations until they close the window.

But it does not work this way from the developer perspective. That "blocking window" does not pause the script execution. This means that an algorithm that was previously implemented as a solid script fragment must be divided into two parts: one part is executed before opening the window and the other one is executed when the window is closed.

To inform the platform which script should be executed once the window is closed, the name of the procedure to be executed is passed to the window.

The following example demonstrates how it works. While it is obviously artificial and not suitable for commercial solutions, it prepares you for understanding the second example.

Requesting user input in a form command

Suppose that you need to request a user confirmation before picking materials and services for a goods receipt tabular section.

In Designer mode

Let us implement the confirmation window.

1. Open the configuration property palette and look at **the Modality usage mode** property (fig. 26.3).

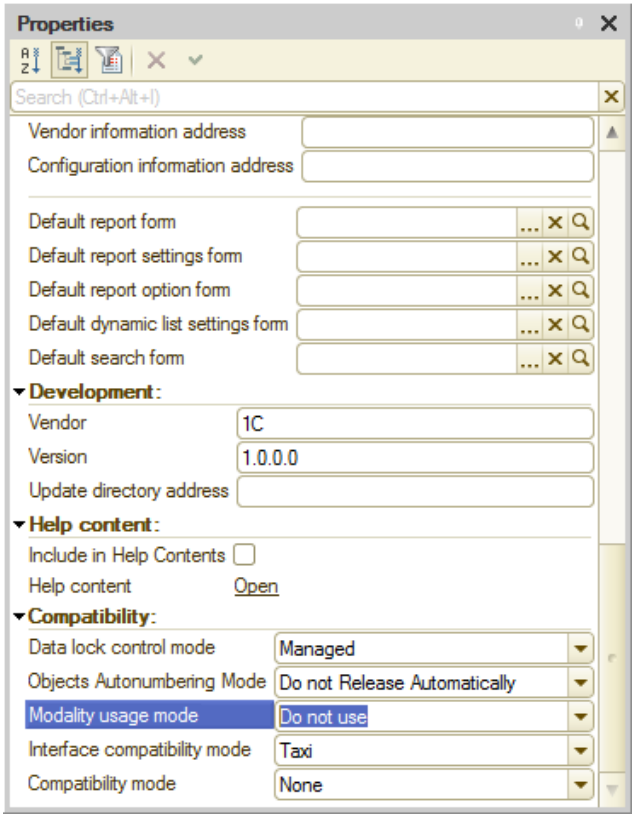


Fig. 26.3. Modality usage mode configuration property

It has the default value "Do not use", which is the recommended mode.

An attempt to open a modal window in this mode returns an error.

To avoid this error, let us use the nonmodal method ShowQueryBox() instead of the modal method DoQueryBox() in the Pick command handler.

For each modal method, the platform has a matching nonmodal method that does not block script execution and does not use a modal window. Nonmodal methods do not cause any web application issues. They have the following specifics:

- Method names begin with Show or Begin. Examples: ShowQueryBox() instead of DoQueryBox(), BeginPutFile instead of PutFile, and so on.
- The first parameter is a NotifyDescription object that points to the module procedure to be executed once the user input is received.
- There is no return value. Istead, the user input is passed to the module procedure described by the NotifyDescription object.

Learn more! The full mapping between modal and nonmodal methods is available in section "Forms. Different approaches to modality. Method mapping" of 1C:Enterprise 8.3 Developer Guide.

2. Add the script shown in listing 26.7 to the **GoodsReceipt** document form module.

Listing 26.7. Pick button click handler with a confirmation dialog box

```
&AtClient
Procedure PickConfirmation(Command)


    Notification = New NotifyDescription("PickCompletion",
        ThisObject);
    ShowQueryBox(Notification, "Do you want to pick materials and
        | services?",
        QuestionDialogMode.YesNo);


EndProcedure


&AtClient
Procedure PickCompletion(Result, Parameters) Export


    If Result = DialogReturnCode.Yes Then
        FormParameters = New Structure ("MultipleChoice", True);
        OpenForm("Catalog.MaterialsAndServices.ChoiceForm",
            FormParameters,
            Items.Materials);
    EndIf


EndProcedure
```

The PickConfirmation procedure first creates a NotifyDescription object. The first parameter of the object constructor is the name of the notification handler procedure, which is executed once the user input is received. The second parameter is the name of the module where the procedure is located.

A notification handler procedure can be stored in a managed form module, in a nonglobal common client module, or in a command module. In this example the procedure is located in the module of this form, which is described by the ThisObject reference.

Then a nonmodal method ShowQueryBox() is executed. It displays the confirmation dialog box. The method accepts the NotifyDescription object as a parameter. This object points to the module procedure (PickCompletion) that is executed once a user makes a selection in the blocking dialog box.

A notification handler procedure must be declared as an export procedure (using the Export keyword). The Result parameter contains the user answer. If the user answer is Yes, the procedure opens the materials and services list for picking.

Let us replace the old Pick command handler with a new one.

3. In the Pick command property palette, in the **Action** field, select the **PickConfirmation** procedure (fig. 26.4).

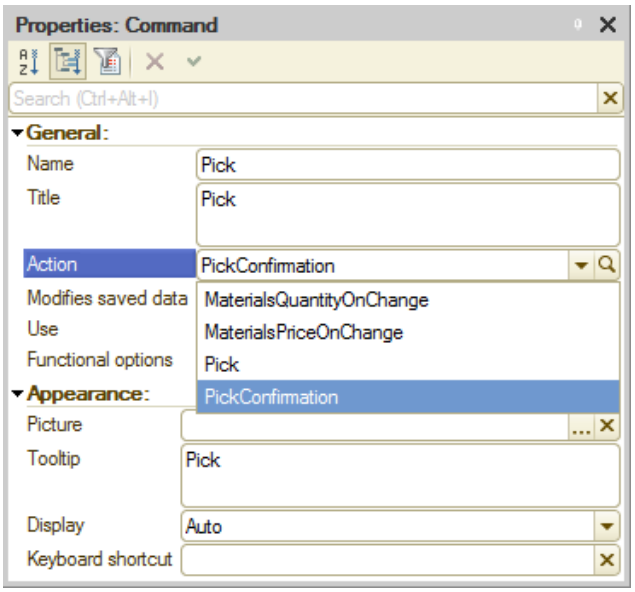


Fig. 26.4. Specifying the action for the Pick command

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. In the **Inventory** section, open the list of goods receipts and start creating a new one.
3. In the command bar, click the **Pick** button.

   This opens the confirmation dialog box. The dialog box blocks the interface but does not block the applied solution execution. You can only proceed to picking materials and services when you confirm that you want to perform this operation.

Requesting user input in an event handler

Let us implement a more complex example where user input is requested in an event handler. Suppose that you need a user confirmation before adding materials to a goods receipt tabular section, and a single confirmation is required for each document.

Just like the previous example, this one does not fit any commercial solution. Still, both examples perfectly illustrate the general approach to avoiding modal windows, which is described earlier in this section.

The new procedure will include single picking with multiple selection available.

1. In the **GoodsReceipt** document form, open the **Pick** command property palette and then, in the **Action** field, select the **Pick** procedure.
2. Update the Pick command handler as shown in listing 26.8.

Listing 26.8. Pick button click handler

```
&AtClient
Procedure Pick (Command)


    FormParameters = New Structure ("MultipleChoice", True);
    OpenForm("Catalog.MaterialsAndServices.ChoiceForm",
        FormParameters, Items.Materials);


EndProcedure
```

3. Add a blocking confirmation dialog box to the ChoiceProcessing handler of the Materials table (listing 26.9).

Listing 26.9. ChoiceProcessing event handler of the Materials table with a confirmation dialog box

```
&AtClient
Procedure MaterialChoiceProcessing(Item, SelectedValue,
    StandardProcessing)


    If AnswerBeforeAdd <> True Then
        StandardProcessing = False;
        Notification = New NotifyDescription("AddCompletion",
            ThisObject, SelectedValue);
        ShowQueryBox(Notification, "Do you want to add materials to
            the tabular section?",
            QuestionDialogMode.YesNo);
    Else
        For Each SelectedItem In SelectedValue Do
            NewRow = Object.Materials.Add();
            NewRow.Material = SelectedItem;
        EndDo;
    EndIf;


EndProcedure
```

The AnswerBeforeAdd variable is used to determine whether a user answered the question.

If a user has not answered the question, the procedure cancels the standard processing and displays the blocking confirmation dialog box. If a user answered the question earlier, the list of materials is added to the tabular section without any confirmations.

If the AnswerBeforeAdd variable value is not true, the procedure creates a NotifyDescription object. The first parameter of the object constructor is the name of the notification handler procedure, which is executed once the user input is received. The second parameter is the name of the module where the procedure is located (in this example it is the form module). And the third optional parameter contains an array of selected material values (SelectedValue).

Then a nonmodal method ShowQueryBox() displays the confirmation dialog box. The method accepts the NotifyDescription object as a parameter. This object points to the exported module procedure (AddCompletion) that is executed once a user makes a selection in the blocking dialog box.

The MaterialsList parameter of the AddCompletion procedure contains the array of selected material values. The Result parameter contains the user input. If the user answer is Yes, the procedure sets AnswerBeforeAdd to True and then adds the selected materials to the document tabular section.

4. Add the procedure shown in listing 26.10 to the form module.

Listing 26.10. AddCompletion() notification handler procedure

```
&AtClient
Procedure AddCompletion(Result, MaterialsList) Export

    If Result = DialogReturnCode.Yes Then
        AnswerBeforeAdd = True;

        For Each selectedItem In MaterialsList Do
            NewRow = Object.Materials.Add();
            NewRow.Material = SelectedItem;
        EndDo;

    EndIf;

EndProcedure
```

Note that once a user confirms that they want to add materials to the document, the confirmation is never requested again because the value of the AnswerBeforeAdd variable is set to True.

The last thing to do is declare that variable.

5. Add the script from listing 26.11 to the beginning of the form module.

Listing 26.11. Variable declaration in a form module

```
&AtClient
Var AnswerBeforeAdd;
```

In 1C:Enterprise mode

Let us test the changes.

- Start 1C:Enterprise in the debug mode.

  You can see that the application behavior is almost identical to the previous example, with the following differences: first, the confirmation is requested after the materials are selected, and second, the confirmation is only requested once.

Note. To check whether your configuration includes any modal methods, in Designer, on the **Configuration** menu, click **Check configuration** and select the **Search for methods that open modal windows** check box.

Generating data based on other data

You can use this feature to generate new objects (such as documents, catalogs, or charts of characteristic types). We will explain this feature on the example of document generation, which is the most common use case.

For each Document configuration object you can allow its generation based on other database objects. And the document itself can serve as a basis for generation of other objects.

The operations that fill document attributes during the document generation must be described in the object module of the document, in the Filling event.

You can either do it manually or use the generation settings wizard that offers visual tools for handler script generation.

In this example a Services document is generated based on a Customers catalog item.

Creating a document generation command

In Designer mode

Let us implement the document generation.

1. Open the configuration object editor for the **Services** document.
2. Add a document attribute named *BasisObject* of **CatalogRef.Customers** type.

   Creating the attribute is not required for document generation, it is only needed for building a chain of dependent documents in the future.

   Let us specify the documents that can serve as a basis for generating the Services document, and the list of documents that can be generated based on the Services document.
3. On the **Generation** tab, in **the Can be generated based on** pane, click the **Edit List item** button and select the **Customers** catalog (fig. 26.5).

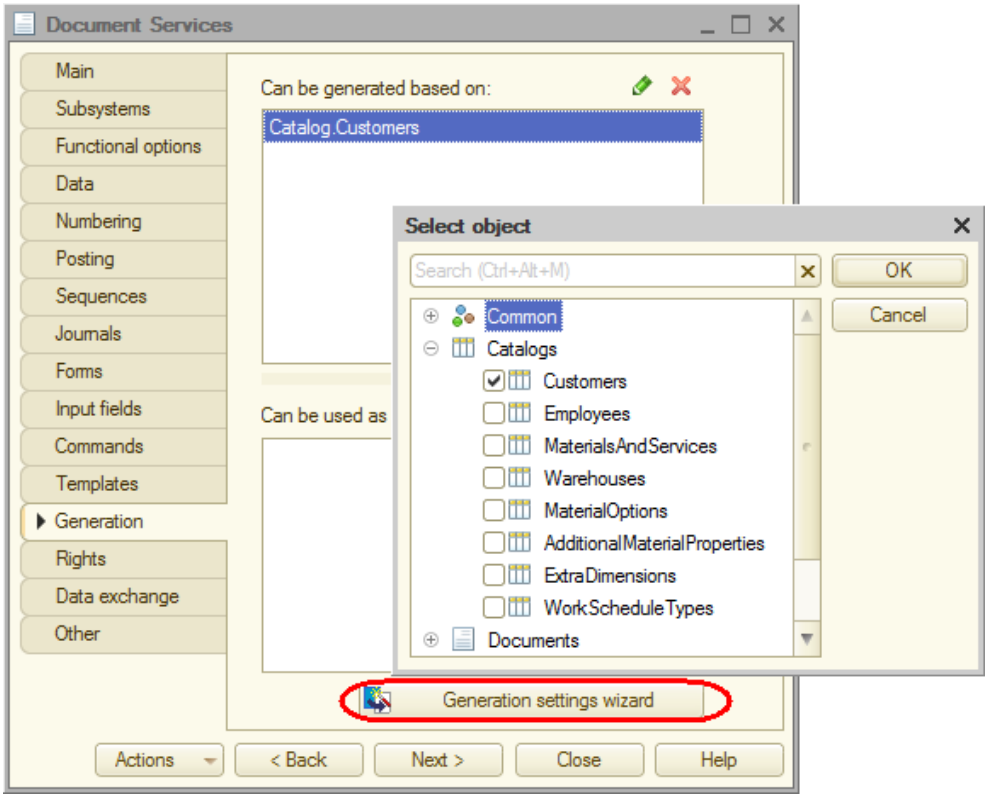


Fig. 26.5. Defining the assortment of objects that can serve as a basis for the document generation

4. Click **Generation settings wizard**.

   Let us specify the attribute values for a document to be generated.

5. Click **Fill Expressions** (fig. 26.6).

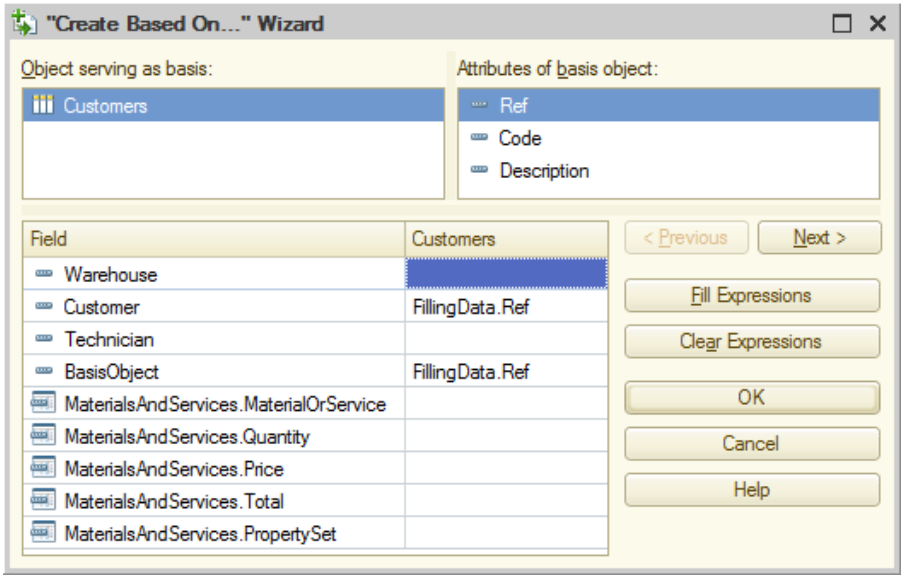


Fig. 26.6. Filling attribute values for a generated document

Note that the wizard suggests using the FillingData.Ref value for filling the BasisObject attribute. In this scenario this record is redundant because a reference to a catalog is passed as a basis.

However, in general, the Filling event comes up when an object is created based on a passed value (which might not have the reference type).

6. Click **OK**.

   This generates the Filling event handler script in the document module (listing 26.12).

Listing 26.12. Filling event handler

```
Procedure Filling(FillingData, StandardProcessing)
    //{{__CREATE_BASED_ON_WIZARD
    // This fragment was built by the wizard.
    // Warning! All manually made changes will be lost next time
    // you use the wizard.
    If TypeOf(FillingData) = Type("CatalogRef.Customers") Then
        // Filling the headline
        Customer = FillingData.Ref;
        BasisObject = FillingData.Ref;
    EndIf;
    //}}__CREATE_BASED_ON_WIZARD
EndProcedure
```

As you can see, an If... condition is generated for each type of basis object, and the attributes of the new document are filled within that condition.

In 1C:Enterprise mode

Let us test the document generation.

1. Start 1C:Enterprise in the debug mode.
2. Open the list of customers.

   Note that the command bar of the Customers catalog list now contains the Generate button.

3. Select a customer, click **Generate**, and click **Services**.

   This creates a Services document where the Customer field is filled with the selected customer (fig. 26.7).

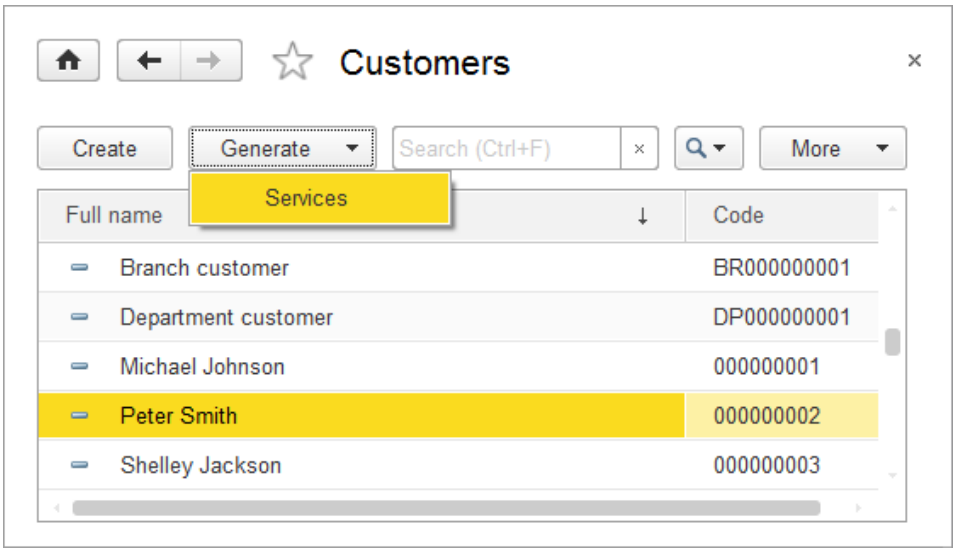


Fig. 26.7. Generating a Services document based on a customer

4. Enter a few more documents based on customer **Peter Smith**.

Generated objects

While the platform allows the generation of objects based on other objects, it has no special means to analyze chains of related objects. We will provide a few tips on the subject that you can use in your development.

To form chains of related objects, add an auxiliary attribute to each generated object. The attribute will store a reference to the basis object. Then create a FilterCriterion object for filtering by specific values of that attribute.

Then, to get all generated objects, you can simply assign the required value to the filter criterion.

Understanding filter criteria

The FilterCriterion configuration object is used to establish the rules for filtering objects.

You can use it to search for data of any kind. For example, you can apply a filter to find all documents whose attributes or tabular sections include a specific contractor.

You can apply multiple filters (for example, filter only posted documents or only documents belonging to a specific time interval).

Getting generated objects

Since the task of getting all the objects generated on the basis of some other object comes up most often in document analysis, we will illustrate the methodology described above using the example of getting a list of documents created on the basis of a Customers catalog item.

In Designer mode

Let us implement the filter.

1. In the configuration tree, expand the **Common** branch and create a **FilterCriterion** configuration object named *Services*.
2. On the **Data** tab, select the criterion type: **CatalogRef.Customers**.
3. On the **Content** tab, select the **BasisObject** attribute of the **Services** document (fig. 26.8).

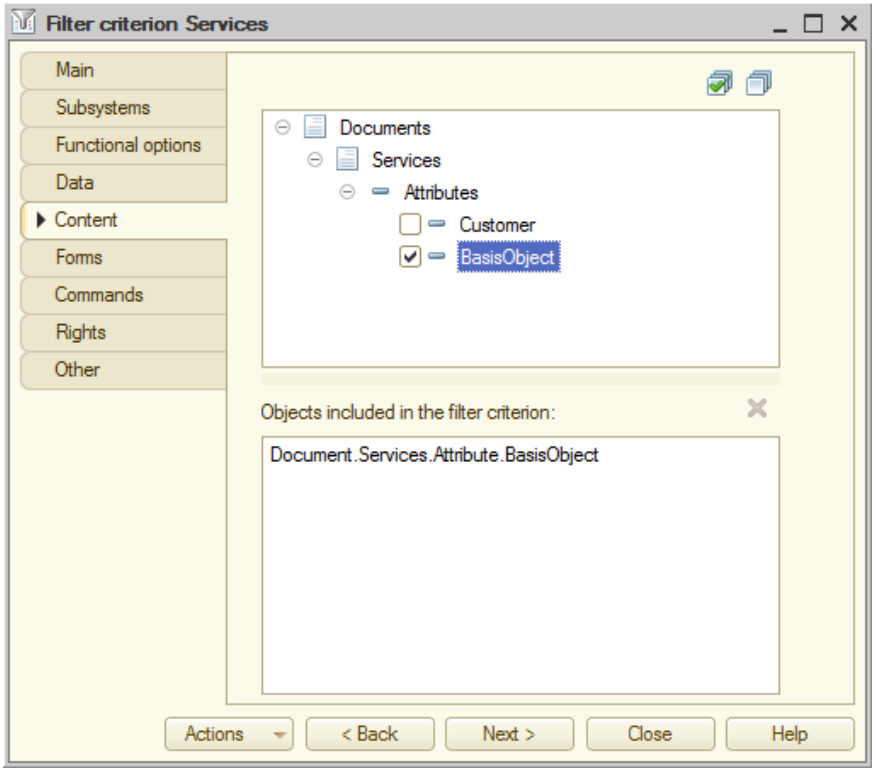


Fig. 26.8. Content of the Services filter criterion

This adds the command that opens the filter criterion to the navigation panel of the Customers item form (to the Go to group).

4. Create the **Customers** catalog item form and then, on the **Command interface** tab, set the visibility of the **Services** command (fig. 26.9).

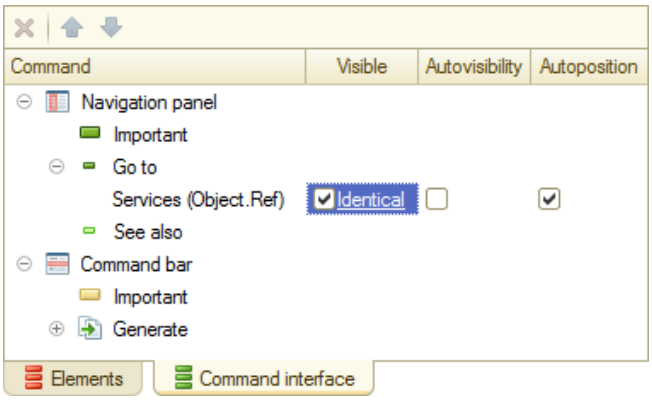


Fig. 26.9. Command that opens the filter criterion from the Customers catalog item form

In 1C:Enterprise mode

Let us test the filter.

1. Start 1C:Enterprise in the debug mode.
2. In the customer list form, select customer **Peter Smith**.

   Previously you created a few documents based on this customer.
3. Open the customer form.

   The navigation panel now has the Services command. This command opens the list form of the filter criterion, where a filter by the selected Customers catalog item is applied.
4. Run this command (fig. 26.10).

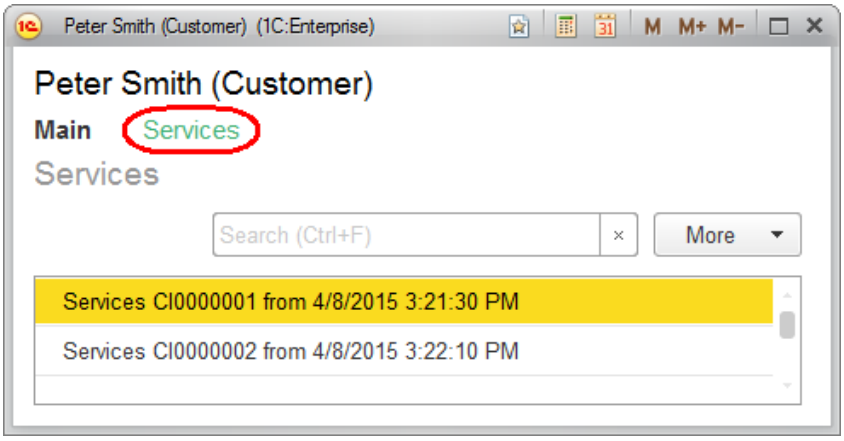


Fig. 26.10. The Services filter criterion list with a filter by customer

The resulting list contains the Services documents generated based on customer Peter Smith. Double-clicking a document opens it.

Quiz

- What is picking?
- How does one implement various picking types in a tabular section of a document form?
- How does one ask user a question in a command?
- How does one ask user a question in a form event handler?
- What is the difference between modal windows and blocking windows?
- What is object generation on the basis of other objects?
- How does one implement object generation?
- How does one use a filter criterion to display the list of objects generated based on the current object?

Lesson 27 (2:10). Form development techniques

In this lesson we will introduce some standard object form development techniques.

First, we will explain how data is displayed in forms and what data types are used here.

Form data and controls

It is important to understand the principles of displaying data in 1C:Enterprise platform forms.

The fact that a form belongs to a certain configuration object does not determine which data is available for displaying in the form.

For example, you can create a common form that is not subordinate to any configuration object, but displays a catalog list, or a document that can be edited, or something else. However, you cannot use this form as a default form for a specific operation.

A form and its controls are inherently separated from configuration objects. To have a form to display certain data, you have to link the form itself or its controls to some data. If you do not specify the links, the controls are not displayed in the form at all (except for the controls that define form appearance).

When you use a form generation wizard, Designer creates the links automatically. When you create a form manually, you can define these links by specifying form properties or form control properties (fig. 27.1).

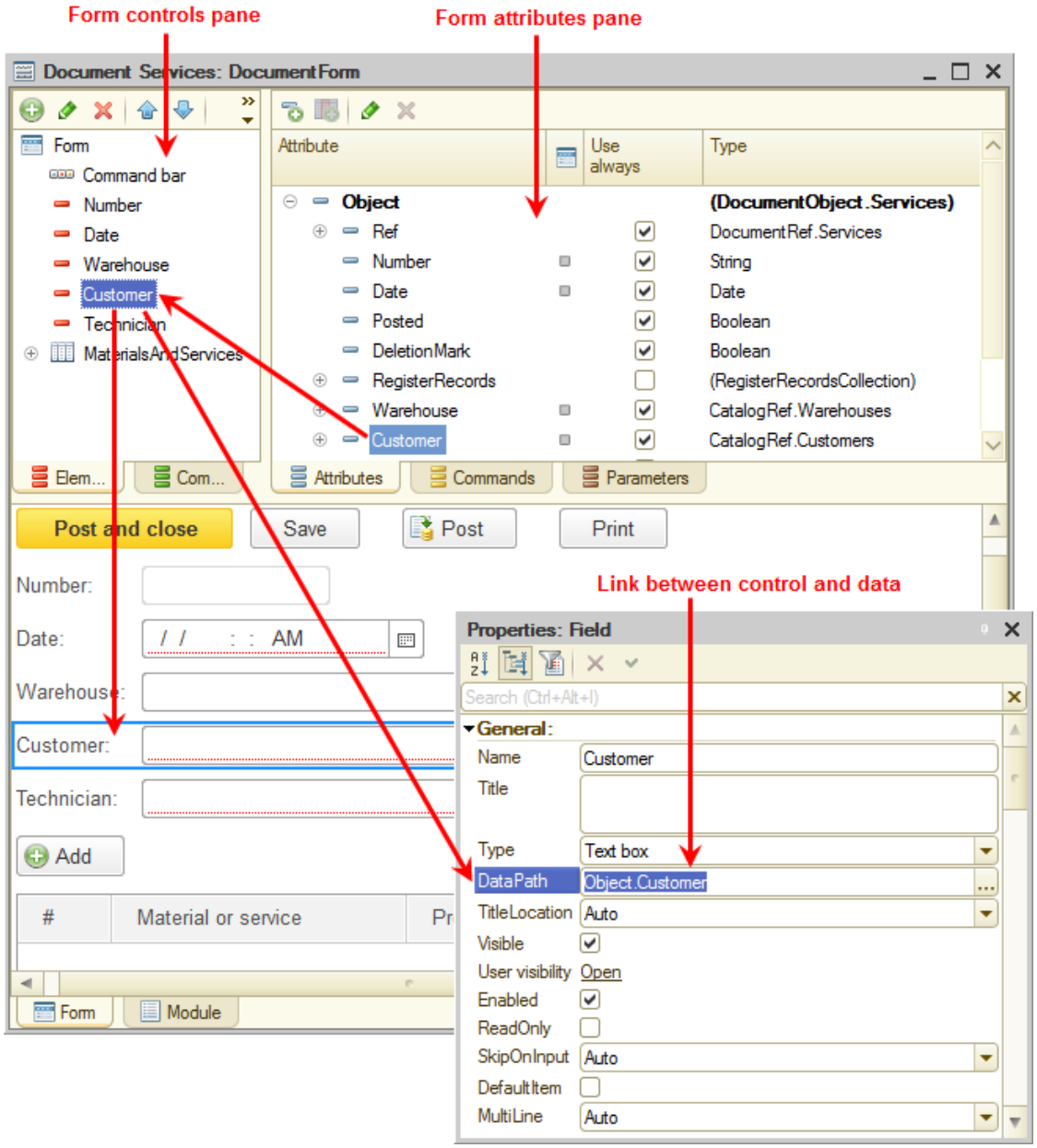


Fig. 27.1. Linking form controls to data

The link between a form control and data that it displays is defined in the DataPath property.

A link between a data item and a form or its control is established using form attributes. The list of available form attributes is displayed on the Attributes tab in the form editor.

Among all the form attributes usually there is a main attribute (displayed in bold type). This attribute defines the data source for the form in general. The type of the main form attribute determines not only the data type to be displayed in the form controls, but also the form behavior.

For example, if the main form attribute has DocumentObject.GoodsReceipt type, when the form is closed, the platform displays a confirmation request before saving and posting the document. However, if the main form attribute has CatalogObject.Customers type, the confirmation request is not displayed when the form is closed.

Data sources have similar effect on form controls. For example, if a table uses a DynamicList form attribute as its data source, its set of columns varies depending on the object used as the main table of this dynamic list (for example, AccumulationRegister.BalanceOfMaterials or Catalog.MaterialsAndServices).

The same is true for the Command bar control. If the AutoFill property of a command bar is set to True, changing the data source (or more precisely, the action source) changes the set of commands displayed in the bar.

The fact that a form or its controls can be linked to data items of various types is the reason for the forms and controls to have multiple extensions.

An *extension* is a set of additional properties, methods, and events belonging to a form or form control. Availability of an extension is determined either by the type of data displayed in the form or control, or by the position of the form control within other controls.

To better understand how it works, let us create a form.

1. Start creating a default list form for the **MaterialsAndServices** catalog but do not click the **Finish** button in the form wizard as you have done earlier.
2. Click **Next**.
3. Add the **MaterialServiceType** field to the **List** table (in addition to the **Description** and **Code** fields), then click **Finish** (fig. 27.2).

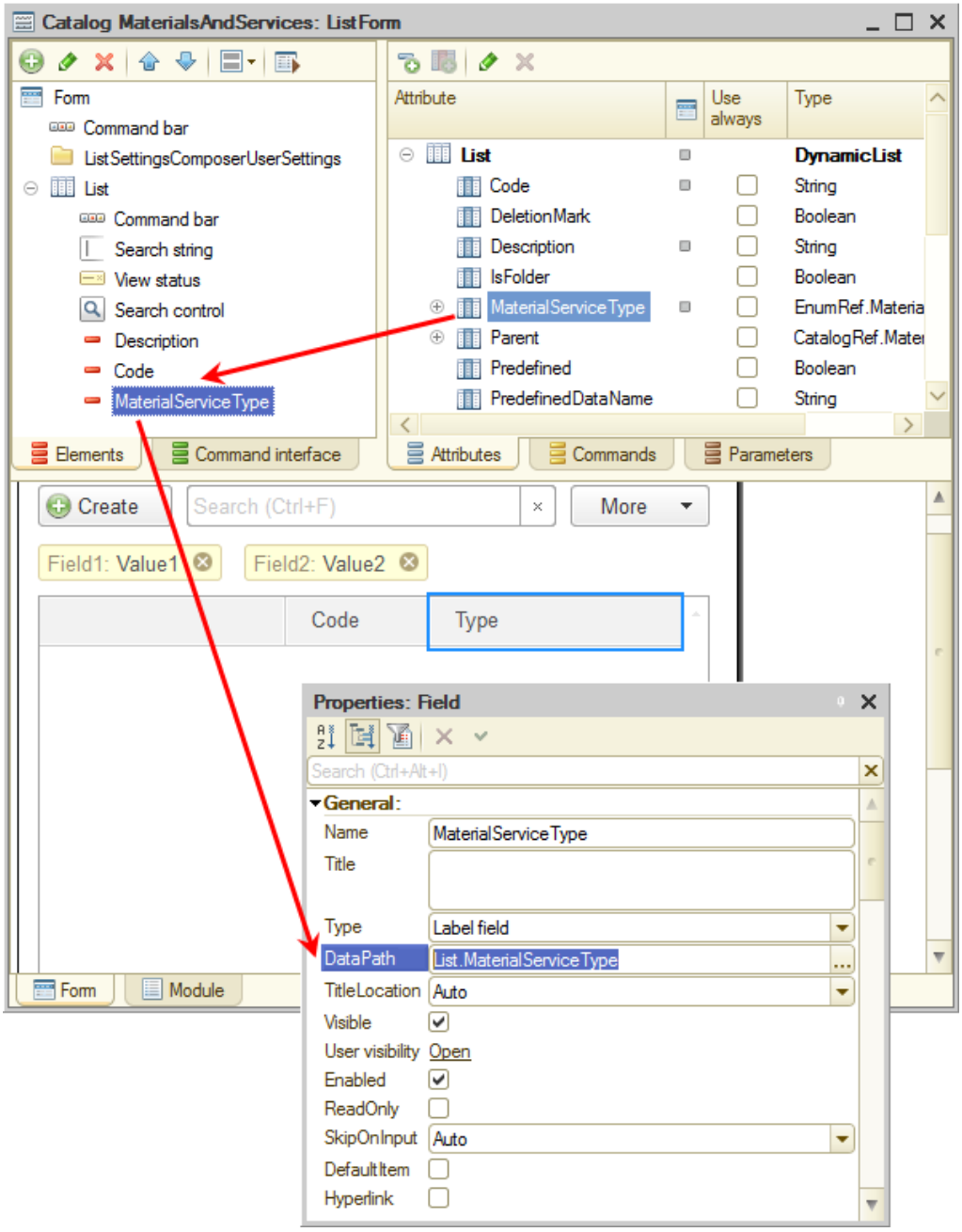


Fig. 27.2. MaterialsAndServices catalog list form

So we will explain the extensions on the example of the MaterialServiceType field located in the List table of the MaterialsAndServices catalog form.

The form itself displays the data of a DynamicList object (the main form attribute has DynamicList type, fig. 27.3).

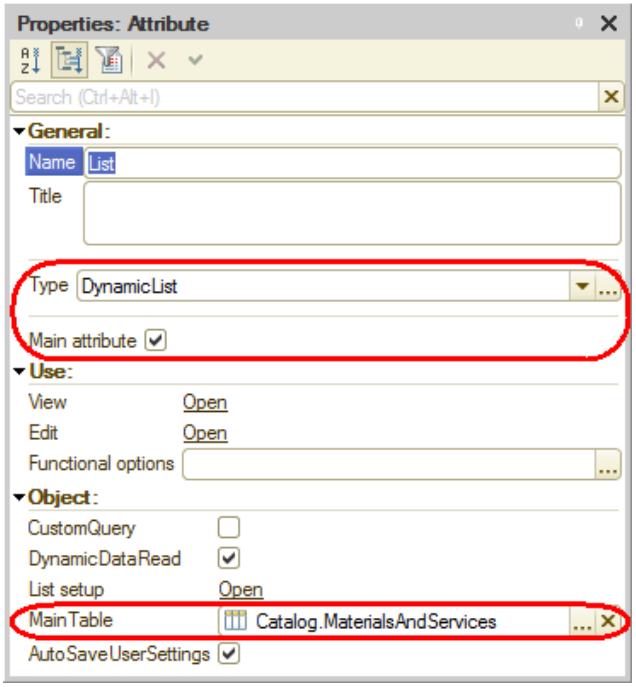


Fig. 27.3. Main list form attribute

This is why the properties, methods, and events of Managed form extension for dynamic lists are added to the properties, methods, and events of the ManagedForm 1C:Enterprise script object (fig. 27.4).

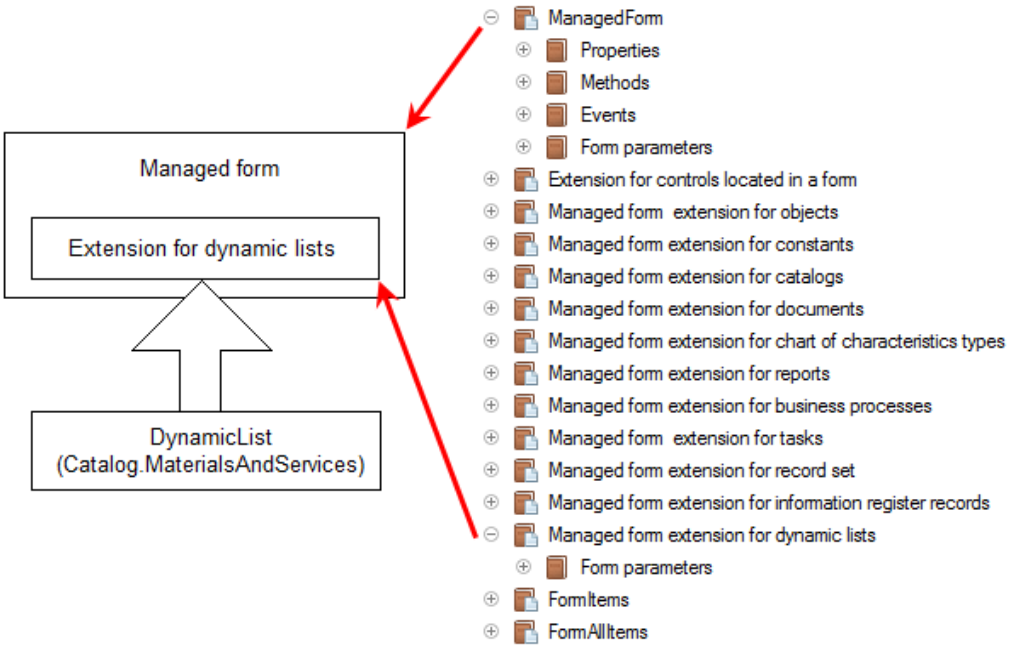


Fig. 27.4. Extension for dynamic lists context added to form context

This adds new properties to the form: CurrentRow, Filter, and more.

Now take a look at the List table.

Since the table displays a dynamic list, the properties, methods, and events of Managed form extension for dynamic lists are added to the properties, methods, and events of the FormTable 1C:Enterprise script object (fig. 27.5).

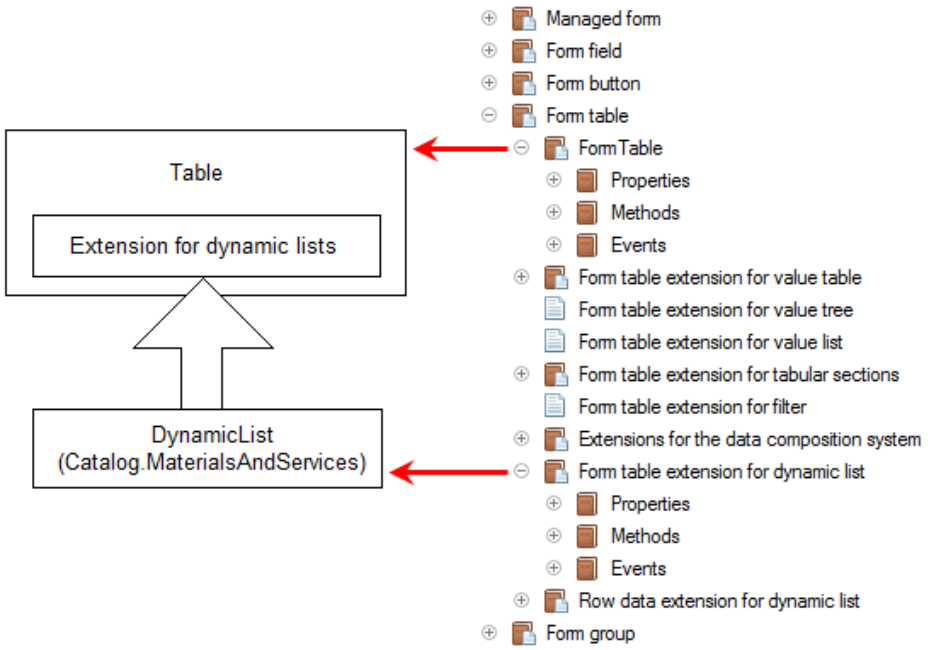


Fig. 27.5. Dynamic list extension context added to table context

This adds new properties to the List table: AutoRefresh, ShowRoot, and more.

Finally, look at the MaterialServiceType field.

This field is a text box linked to an attribute of the EnumRef.MaterialServiceTypes type.

This adds the properties, methods, and events of Form field extension for a text box to the properties, methods, and events of the FormField 1C:Enterprise script object (fig. 27.6).

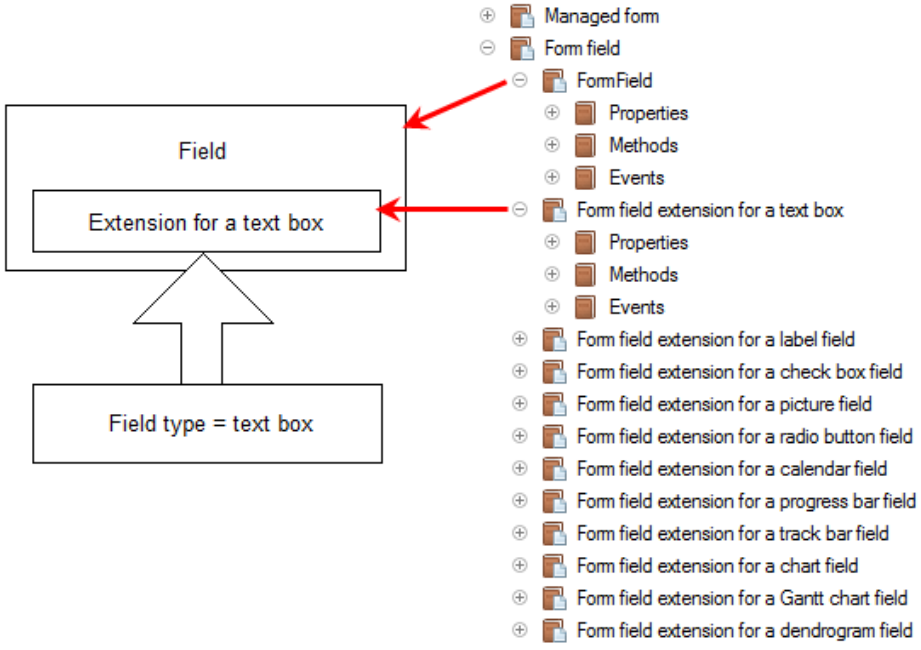


Fig. 27.6. Text box extension context added to form field context

This adds new properties to the MaterialServiceType field: QuickChoice, SelectedText, and more.

Form data types

The following data type categories are available in managed forms:

- 1C:Enterprise script types intended for use both in managed forms and outside of them. Examples: Number, CatalogRef.<name>, GraphicalSchema, SpreadsheetDocument.
- 1C:Enterprise script types intended exclusively to represent data of applied objects (catalogs, documents, and so on) in forms. Examples: FormDataStructure, FormDataCollection.

DynamicList type, which is used in managed forms to display lists of applied objects.

All the applied object types (such as CatalogObject and others) do not exist on the thin and web client ends as they are only available on the server end. But developers need to display data of such objects in managed forms.

The following managed form data types are used for solving this task:

- **FormDataStructure**. This object stores a set of properties that can have any types. Available types include structures, collections, or structures with collections. For example, this type represents CatalogObject in forms.
- **FormDataCollection**. This object stores a list of typed values, similar to an array. Its elements can be accessed by index or by identifier. In some cases access by identifier is not available (this is determined by the type of applied object represented by the collection). Any integer can serve as an identifier. For example, this type represents tabular sections in forms.
- **FormDataStructureAndCollection**. This object is a structure and a collection simultaneously. It can be handled as any of these entities. For example, this type represents record sets in a form.
- **FormDataTree**. This object is used to store hierarchical data.

An applied object is presented by one or more form data items. In general, the hierarchy and content of form data depend on complexity of applied objects and on their interconnections.

For example, a document that contains a tabular section is represented by an object of the FormDataStructure type (the document itself) that has FormDataCollection (document tabular section) as a subordinate object.

Attention! Remember that applied objects are only available on the server, while you can use form data objects both on the server and on the client.

Actually form data is a unified presentation of various applied objects that is available both on the server and on the client. Forms use unified methods of processing this data.

The form attributes pane of the form editor usually displays the applied types stored in the attributes rather than form data types.

For example, if an Object attribute contains data of a Customers catalog item, the Type column does not display the actual form attribute type (DataFormStructure) and instead displays the type of the applied object whose data is stored in the attribute (CatalogObject.Customers). To indicate

that it is not the actual attribute type, the applied object type is enclosed in parentheses.

Hence, a form contains some projection of applied object data as its own data types and automatically makes conversions between these types if required.

But if you implement a custom data processing algorithm, you have to include data conversion between applied object types and form data types to that algorithm.

A set of global methods is available for converting applied objects into form data and back:

- ValueToFormData() converts an applied object to a form data item
- FormDataToValue() converts a form data item to an applied object

A managed form itself also has similar methods for converting values of form attributes into applied objects and back:

- ValueToFormAttribute() converts an applied object into a managed form attribute
- FormAttributeToValue() converts a managed form attribute into an applied object value

The methods that process applied objects are only available in server form procedures.

When you perform standard form operations on the main attribute (opening the form, execution of the standard Write command, and so on), the conversion is performed automatically.

Let us review an example of data conversion within a custom algorithm.

Suppose you have a special form where one of the attributes (ProductToModify) uses data of the Products catalog. When a form is created on the server, some algorithm determines the product whose data should be stored to the form attribute, reads the product data, converts its type, and stores it to the form attribute. This includes a call of the ValueToFormData() method (listing 27.1).

Listing 27.1. Example of converting applied object data into form data

```
&AtServer
Procedure OnCreateAtServer(Cancel, StandardProcessing)


    ObjectProduct = Catalogs.Products.FindByDescription(
        "Coffee pot").GetObject();
    ValueToFormData(ObjectProduct, ProductToModify);


EndProcedure


&AtClient
Procedure Write()


    WriteAtServer();
```

```
EndProcedure


&AtServer
Procedure WriteAtServer()


    ObjectProduct = FormDataToValue(ProductToModify,
        Type("CatalogObject.Products"));
    ObjectProduct.Write();


EndProcedure
```

At some point of form operations you decide that modified product data should be recorded to the database. So you convert form data back into an applied object by calling FormDataToValue(), and then write it.

As we mentioned above, a form also has methods for converting applied data into form attributes and back.

Such methods are generally more convenient because they have access to form attribute type. Besides, the FormAttributeToValue() method sets a correspondence between form data and the object that is used to generate messages.

Let us review an example of using this method.

Suppose you get an applied object from a form attribute in a server form procedure and then use the Recalculate() method of this applied object. Then the object data is modified according to the recalculation and converted back into a form attribute (listing 27.2).

Listing 27.2. Example of converting applied object data into form data

```
&AtServer
Procedure RecalculateAtServer()


    // Converting the Object attribute to an applied object.
    Document = FormAttributeToValue("Object");


    // Recalculating using a method defined in the document module
    Document.Recalculate();


    // Converting the applied object back to an attribute.
    ValueToFormAttribute(Document, "Object");


EndProcedure
```

Related lists

During applied solution development you might often need to implement navigation from an applied object form to data that is logically connected to that object.

It can be a list of a subordinate catalog, or a register where this object creates records, or a register where a dimension with this object type is specified as the leading one, or filter criteria that include this type, or objects that can be generated based on this object type, and more.

A typical related list usage scenario is navigation from a document form to the list of register records created by the document in a specific register.

All the scenarios described above are managed by the platform automatically. The commands for navigation to the related data are generated based on the data stored in configuration objects. Some of the commands are made visible immediately and they appear in forms. Some commands are created but are not made visible by default.

So if you do not see the command for navigating to some related data, it is likely that all you have to do is enable the command visibility in the form interface.

In Designer mode

The following example illustrates the concept described above.

1. Open the form of the **Services** document.
2. In the upper left pane, click the **Command interface** tab.

   You can see that the Go to section of the Navigation panel includes a set of commands for navigating to the register records created by the document.

   You can either enable general visibility for these commands or fine-tune the command visibility for each role in the configuration (fig. 27.7).

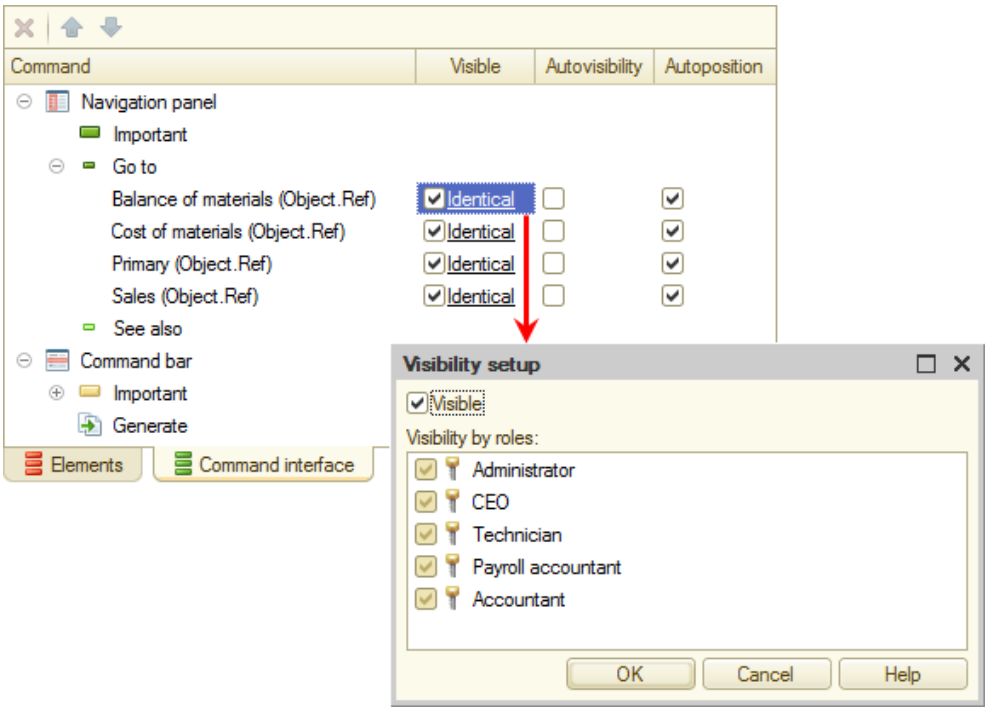


Fig. 27.7. Form command interface

In 1C:Enterprise mode

Let us look at the navigation commands.

1. Run 1C:Enterprise in the debug mode.
2. Open any of the **Services** documents (fig. 27.8).

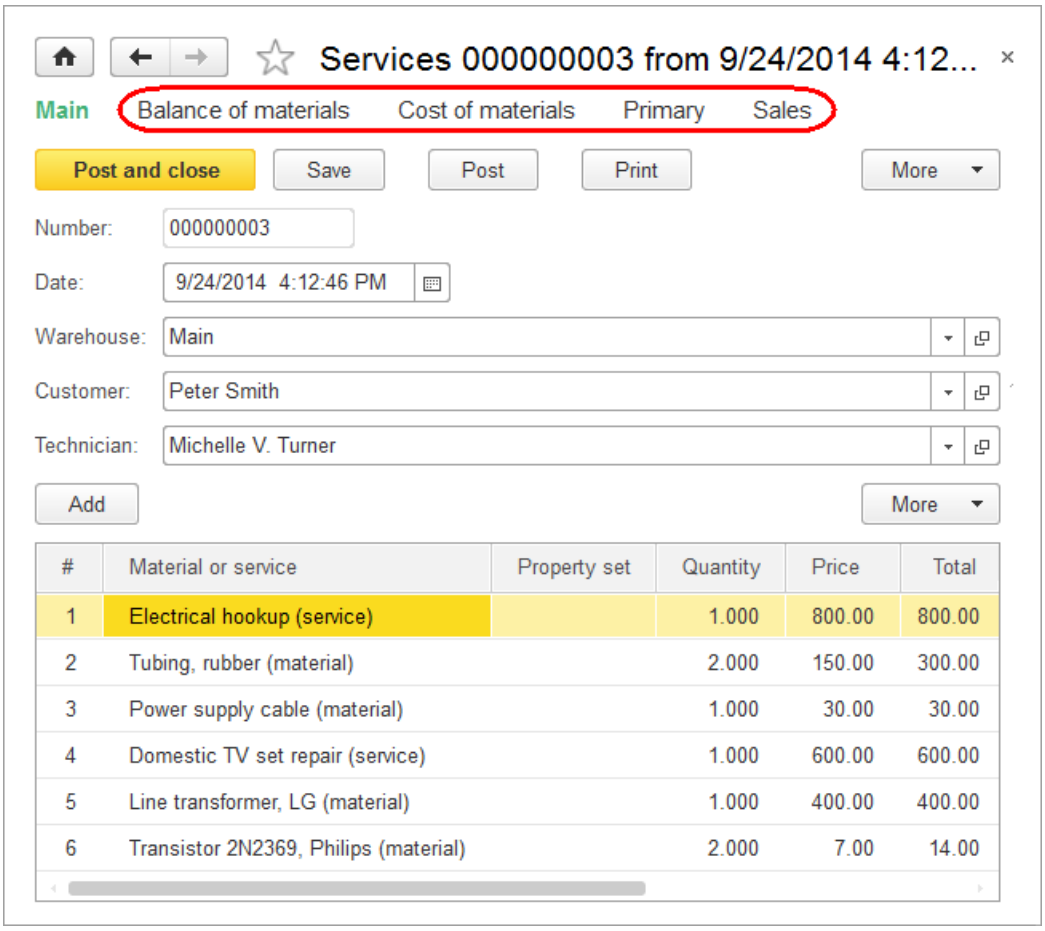


Fig. 27.8. Services document #3

You can see that the form navigation panel contains commands for navigating to register record lists related to the current document.

3. Click the **Cost of materials** command to view the records created in this register by this document (fig. 27.9).

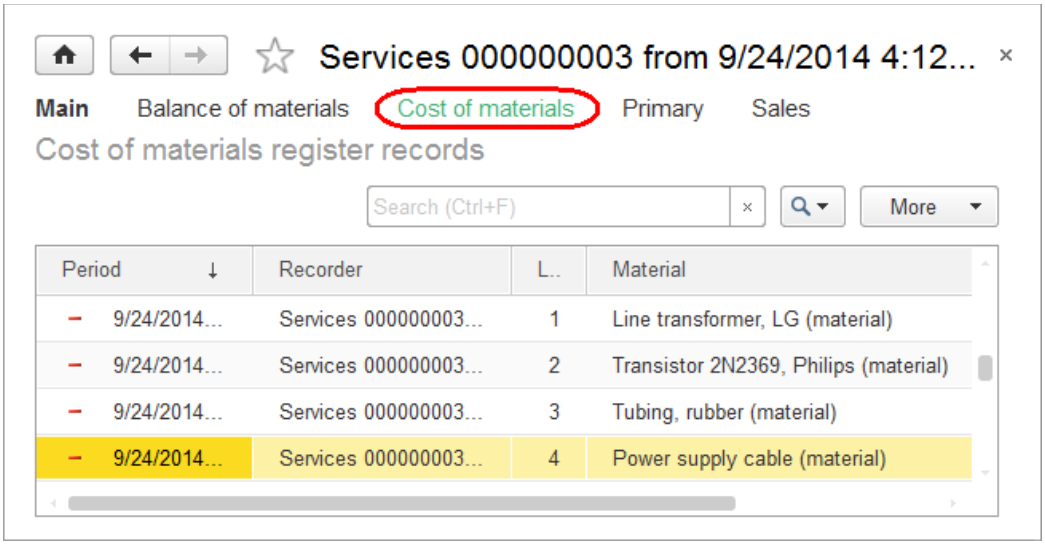


Fig. 27.9. Records of the Services document #3 in the Cost of materials register

Formatting rows in list forms

List forms have a handy feature: you can apply conditional formatting to their rows. We will explain this feature on the example of custom appearance implementation for the MaterialsAndServices catalog list form.

In Designer mode

Let us create the conditional formatting.

1. In Designer, open the **MaterialsAndServices** list form and create the **OnCreateAtServer** form event handler.
2. Add the script shown in listing 27.3 to the handler.

   Listing 27.3. OnCreateAtServer form event handler

```
&AtServer
Procedure OnCreateAtServer(Cancel, StandardProcessing)
    CatalogList = Items.List;

    // Setting appearance for the catalog
    CatalogList.Representation = TableRepresentation.List;

    // Hiding grid lines
    CatalogList.VerticalLines = False;
    CatalogList.HorizontalLines = False;

EndProcedure
```

   This procedure is executed on the server when the form is created. First, it changes the list view from hierarchical to flat, so that you can see materials and services at the same time.

   Then it hides the grid lines that separate columns and rows of the list table.

   Now let us proceed to specifying conditional appearance for the list rows.

3. Open the property palette of the **List** form attribute (it is the main form attribute).
4. In the **List setup** field, click **Open** (fig. 27.10).

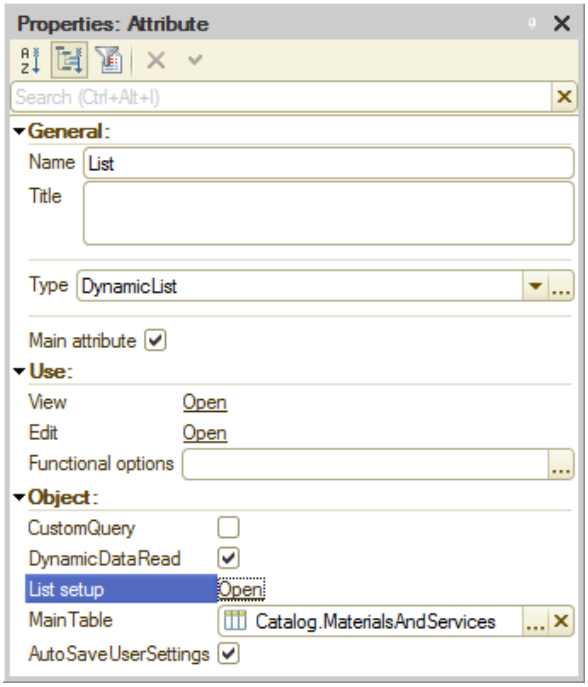


Fig. 27.10. Opening dynamic list setup

Since the attribute has DynamicList type, which is based on a data composition system, you can apply filter, sorting, grouping, and conditional appearance to it in the same manner as you did in reports.

5. In the dynamic list settings window that is opened, click the **Conditional appearance** tab and then click **Add** .

   First, let us specify the formatting for highlighting fields.

6. In the **Format** field, click the selection button ... and select lavender color for the background (fig. 27.11).

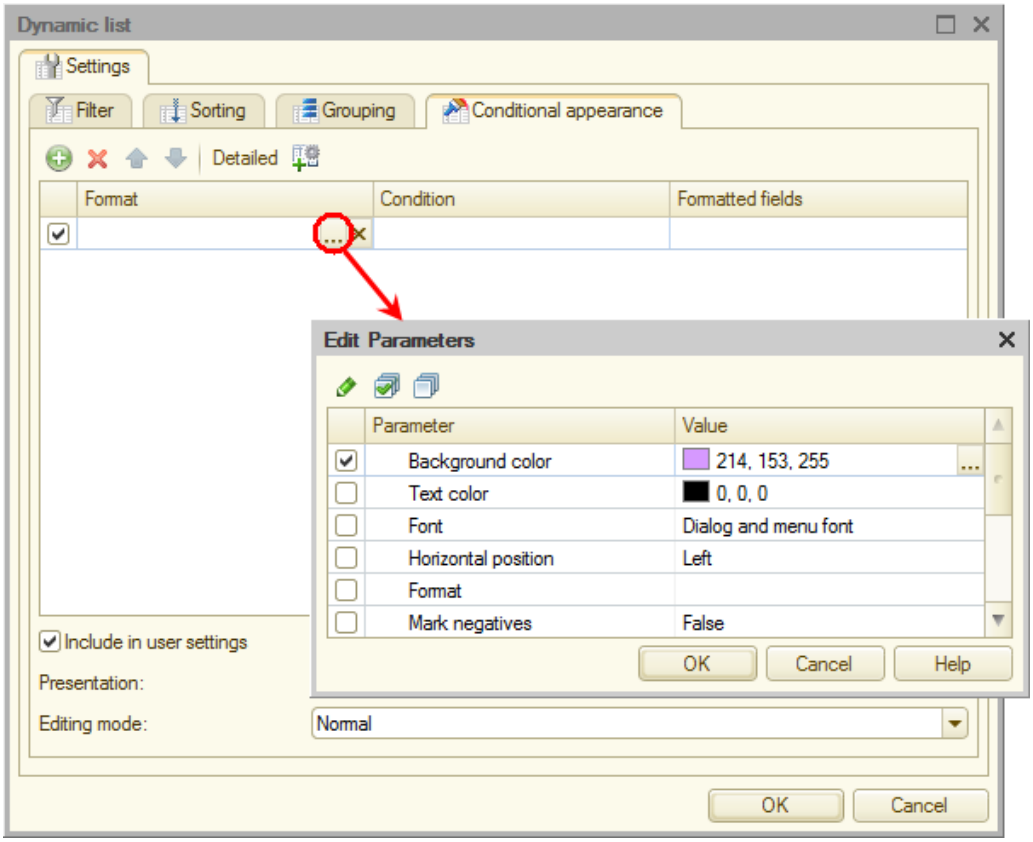


Fig. 27.11. Conditional appearance settings

7. Click **OK**.

   Then let us specify the condition for the appearance to be applied (when list rows should have lavender color).

8. In the **Condition** field, click the selection button ....
9. In the window that is opened, click the **Add** button and then click **New item**.
10. In the **Left value** column, select **MaterialServiceType**.
11. In the **Comparison type** column, select **Equal to**.
12. In the **Right value** column, select **Enum.MaterialServiceTypes.Service** (fig. 27.12).

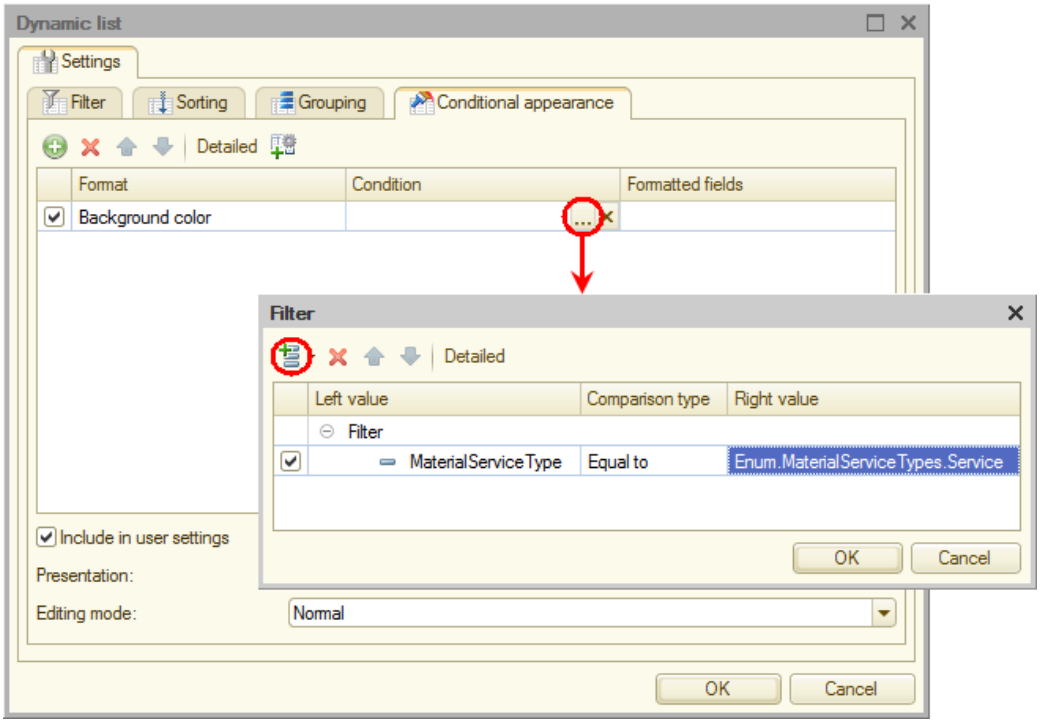


Fig. 27.12. Conditional appearance settings

13. Click **OK**.

    So you have defined that services have lavender background in the list.

    Since there is no need to highlight individual cells, you can leave the list of formatted fields empty.

14. Click **OK**.

In 1C:Enterprise mode

Let us test the conditional appearance.

1. Start 1C:Enterprise in the debug mode.
2. Open the list of materials and services.

   You can see that the list has a flat view, services are highlighted with lavender color, and there are no grid lines for rows and columns (fig. 27.13).

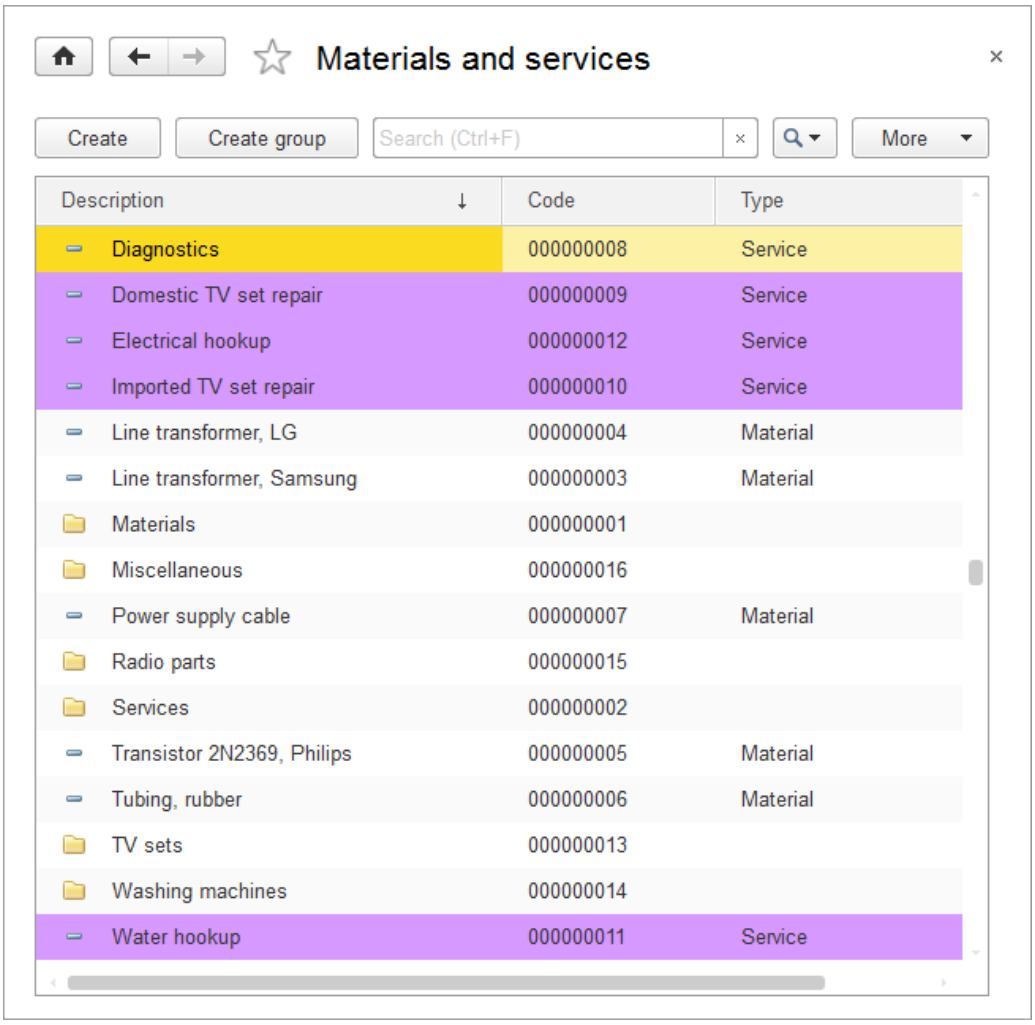


Fig. 27.13. List of materials and services with custom appearance

Now let us review the user settings available in the list. They are similar to user settings available in reports: filter, sorting, grouping, and conditional appearance. And dynamic list settings in the platform are very similar to report settings generated using a data composition schema.

In Designer mode

Each of the dynamic list settings tabs (Filter, Sorting, Grouping, and Conditional appearance) has the **Include in user settings** check box, which is selected by default, and the **Editing mode** field, which is set to **Normal** (see fig. 27.11).

This means that users can change the list appearance by clicking **Configure list** on the **More** menu. This opens the dynamic list settings window, which is similar to the report settings window, where they can specify a custom conditional appearance or other settings (see fig. 27.15).

You have the option to display popular list settings directly in the list form.

- In the **Editing mode** field, select **Quick access** (fig. 27.14).

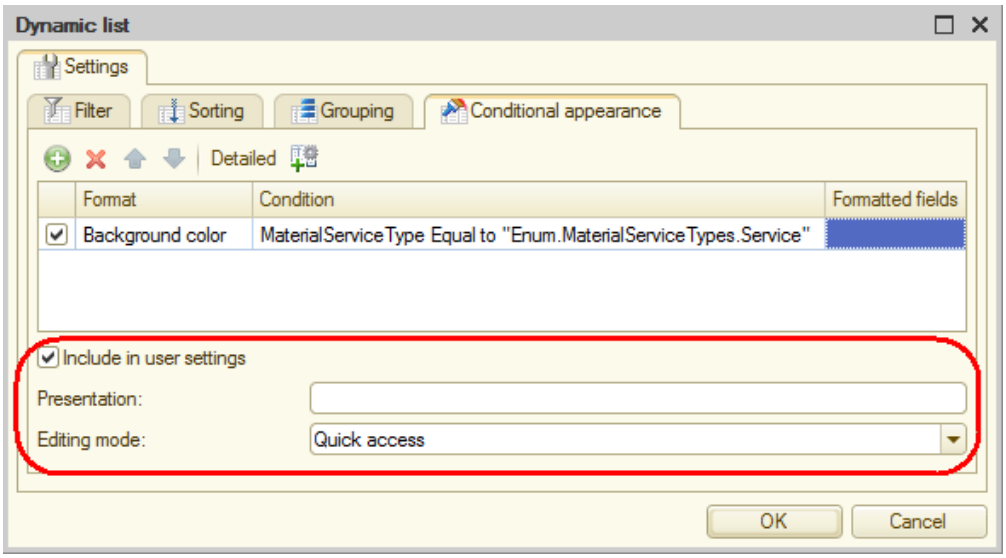


Fig. 27.14. Conditional appearance settings

In 1C:Enterprise mode

Let us test the changes.

- Open the list in 1C:Enterprise mode (fig. 27.15).

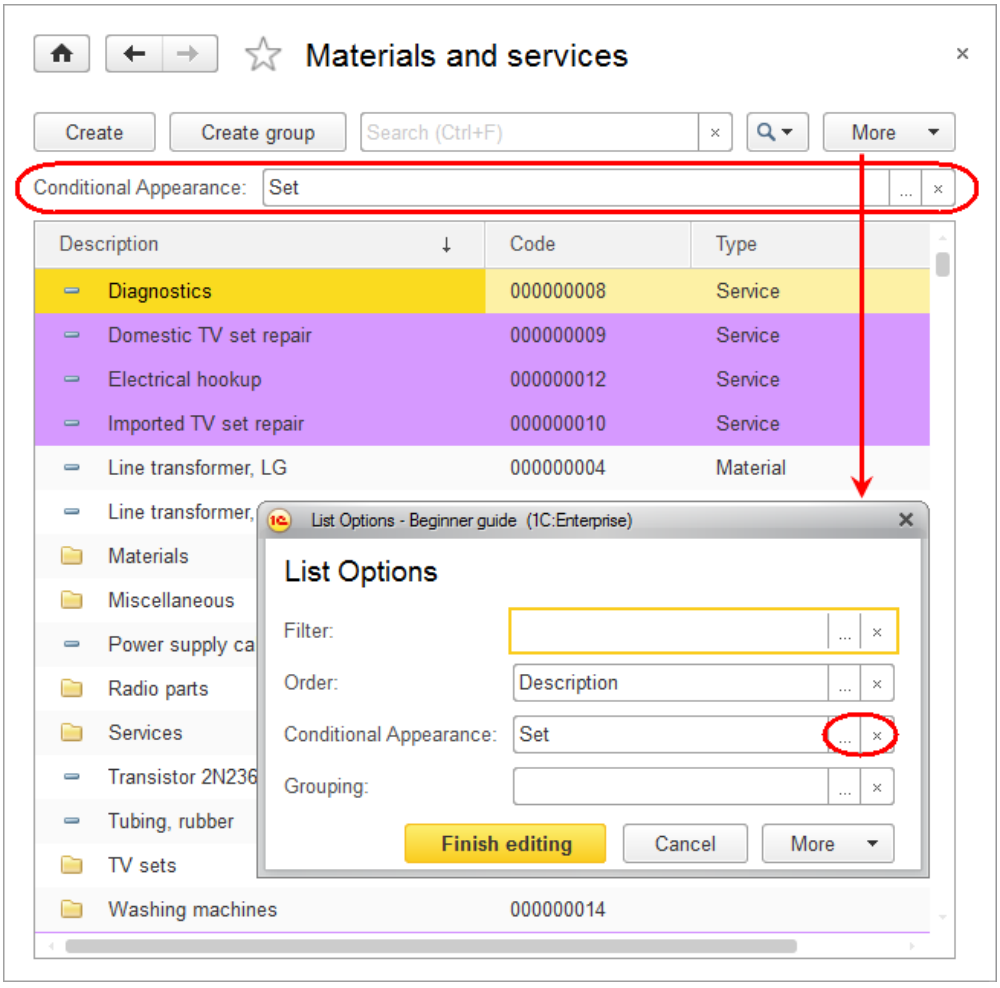


Fig. 27.15. User settings of the materials and services list

Note that the conditional appearance setting is available to users in the list form itself, above the list of materials and services. And users have the option to open the user settings window, which contains not only conditional appearance settings but also other appearance settings, by clicking **Configure list** on the **More** menu.

Users can also customize the settings (move them between quick settings and normal settings) by clicking **Customize displayed settings** on the **More** menu.

The **Include in user settings**, **Presentation**, and **Editing mode** property values can be applied either to the general filter, sorting, grouping, or conditional appearance list settings, or to individual settings. For example, you can include a filter in the user settings but exclude another filter. Properties of individual settings take precedence over the general properties. You can specify properties of individual settings by clicking the **Custom settings item properties** button above the settings list.

You can set conditional apperance for lists that are not based on dynamic lists (for example, document tabular sections) in a similar manner, using the conditional appearance of the form where they are located. In the form controls tree, open the property palette of the root item and then click the **Conditional appearance** link.

To customize the appearance of the entire tabular section row, add the form table that contains the tabular section (for example, Materials) to the Formatted fields list. Otherwise specify the table fields where you want to apply conditional formatting.

Calculated columns in lists

You might want to display custom data in list columns. For example, you might want to add a column with a calculated value.

We will explain this feature using the following example: displaying effective prices in the MaterialsAndServices catalog list.

You can get this data from the Prices.SliceLast information register table. So the Price field of the table should be added to the List dynamic list, which is the main attribute of the materials and services list form and serves as a data source for the list table.

In Designer mode

Let us add the prices to the list of materials and services.

1. In Designer, open the list form of the **MaterialsAndServices** catalog.
2. Open the property palette of the main **List** attribute.

   Up to this point the Main table property of the dynamic list was set to Catalog.MaterialsAndServices (see fig. 27.10) and the list was generated using a query to the table.

   Now you also need the related data from the Prices.SliceLast information register table.

3. Select the **CustomQuery** check box and then, in the **List setup** field, click **Open** (fig. 27.16).

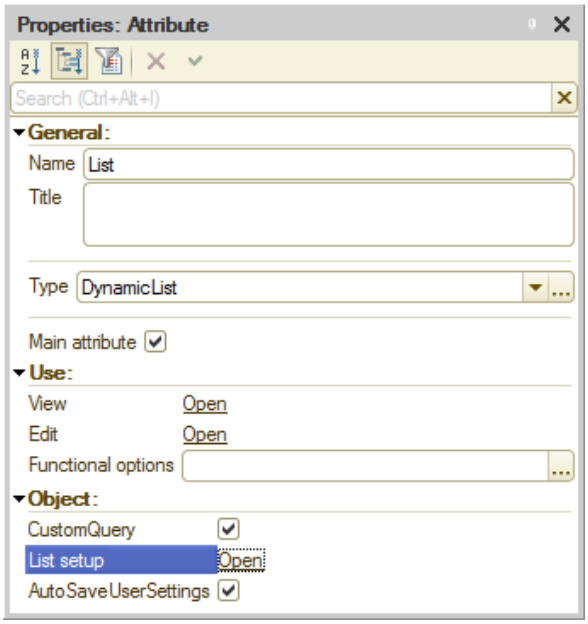


Fig. 27.16. Opening dynamic list settings

This opens the dynamic list settings window. On the Query tab you can see the query that selects all the fields from the Catalog.MaterialsAndServices table. Let us modify it.

4. Click the **Query Builder** button (fig. 27.17).

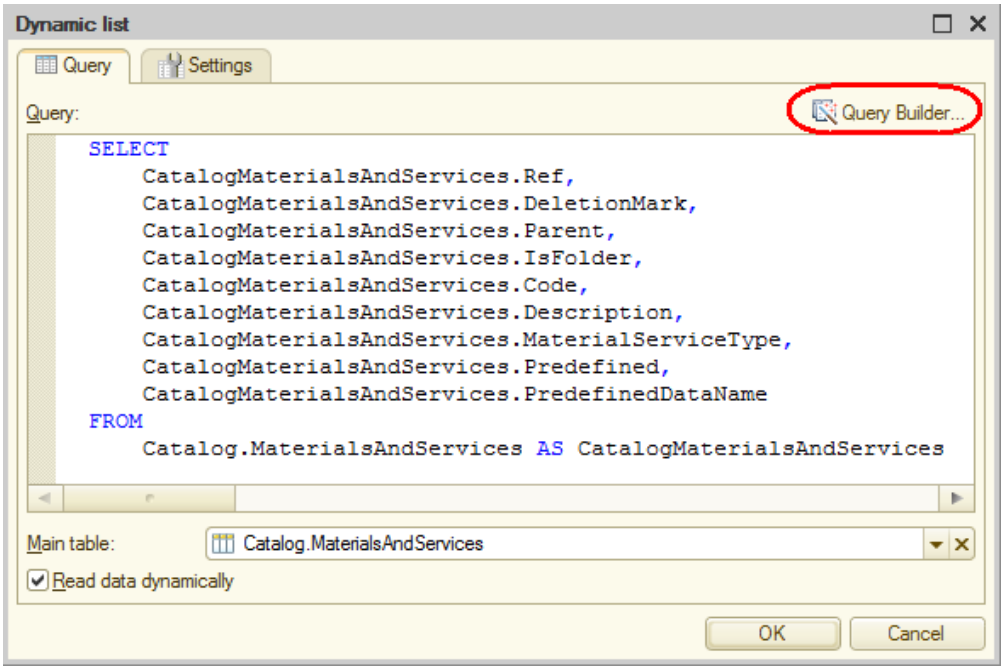


Fig. 27.17. Creating a custom query for a dynamic list

5. Add **Prices.SliceLast** to the list of tables and select the **Price** field from this table (fig. 27.18).

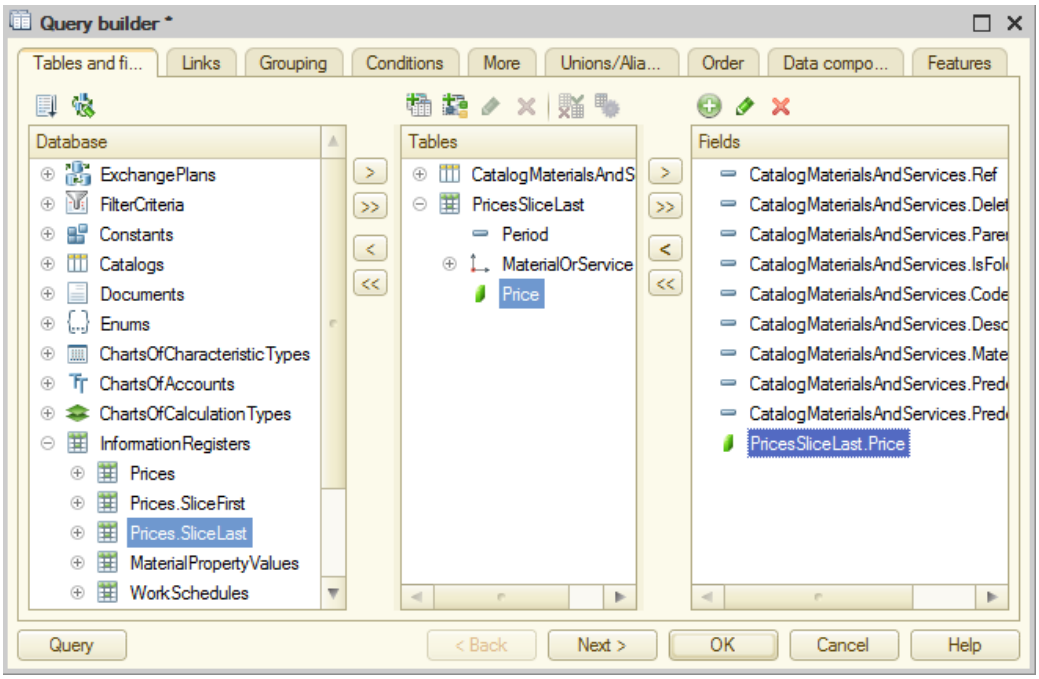


Fig. 27.18. Adding a second table

Let us edit the automatically generated relation between the tables.

6. On the **Links** tab, select the **All** check box for the **Catalog.MaterialsAndServices** table and clear the **All** check box for the **Prices.SliceLast** table (fig. 27.19).

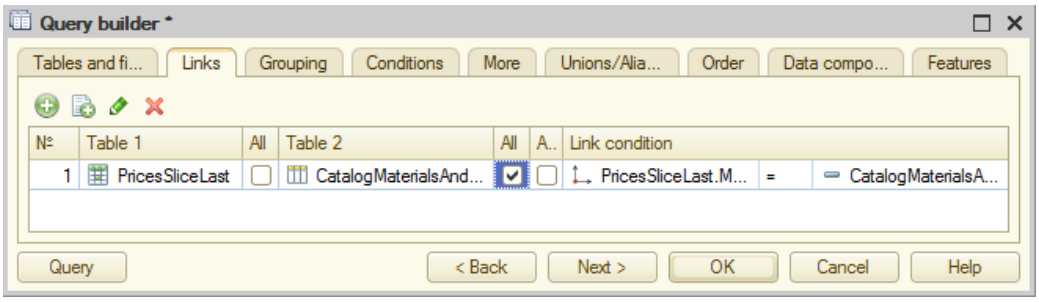


Fig. 27.19. Specifying the relation between the tables

This is how you specify that the list of materials and services displays all the items, including those that have no prices assigned. You performed similar steps in lesson 13 "Reports" for the Service list report on page 353.

This completes the query creation.

7. Click **OK**.
8. You are already familiar with the query script so we will not discuss it here.

   So you added the option to display the Price column, which contains the current prices, in the list of materials and services, but the column is not yet added to the list form.

   Now let us specify the conditional appearance of the Price column by highlighting low prices with a color.

9. In the dynamic list settings window, click the **Settings** tab.

10. Click the **Conditional appearance** tab.

    This tab contains the conditional appearance of the list rows that you created earlier in this lesson.

11. Click the **Add** button.

    First, let us specify the formatting for highlighting fields.

12. In the **Format** field, click the selection button and select blue text color.

    Then let us specify the condition for the appearance to be applied (when the text in the Price column should be blue).

13. In the **Condition** field, click the selection button.

14. In the window that is opened, click the **Add** button and then click **New item**.

15. In the **Left value** column, select **Price**.

16. In the **Comparison type** column, select **Less than**.

17. In the **Right value** column, enter *500* (fig. 27.20).

    Finally, let us specify the fields where the formatting is applied.

18. In the **Formatted fields** column, click the selection button.

19. Click **Add** and select **Price** (fig. 27.20).

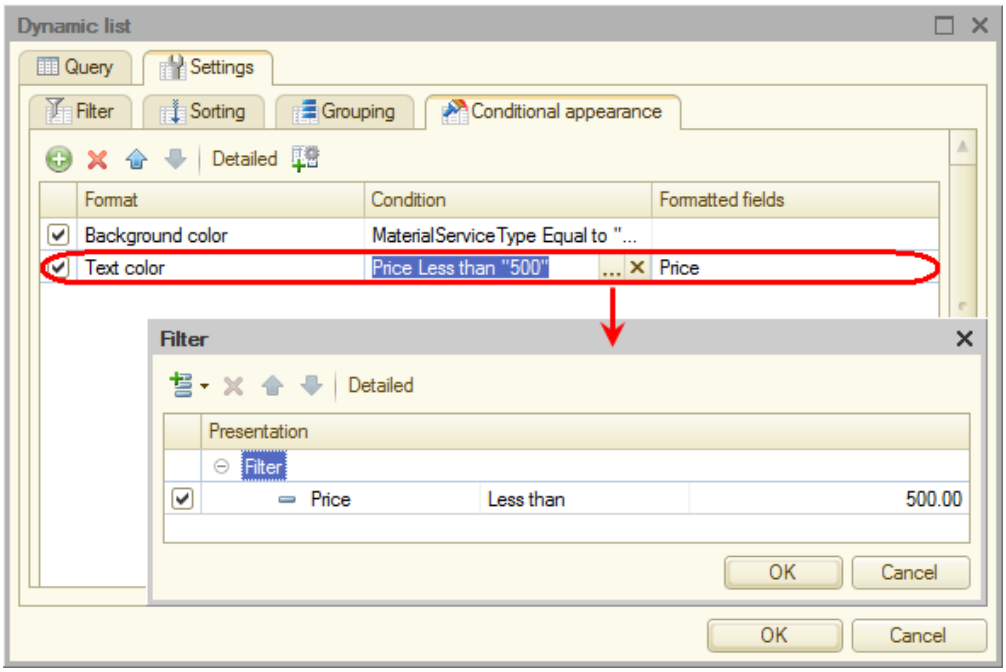


Fig. 27.20. Dynamic list conditional formatting settings

20. Click **OK**.

21. Drag the **Price** field from the attributes pane to the controls pane (fig. 27.21).

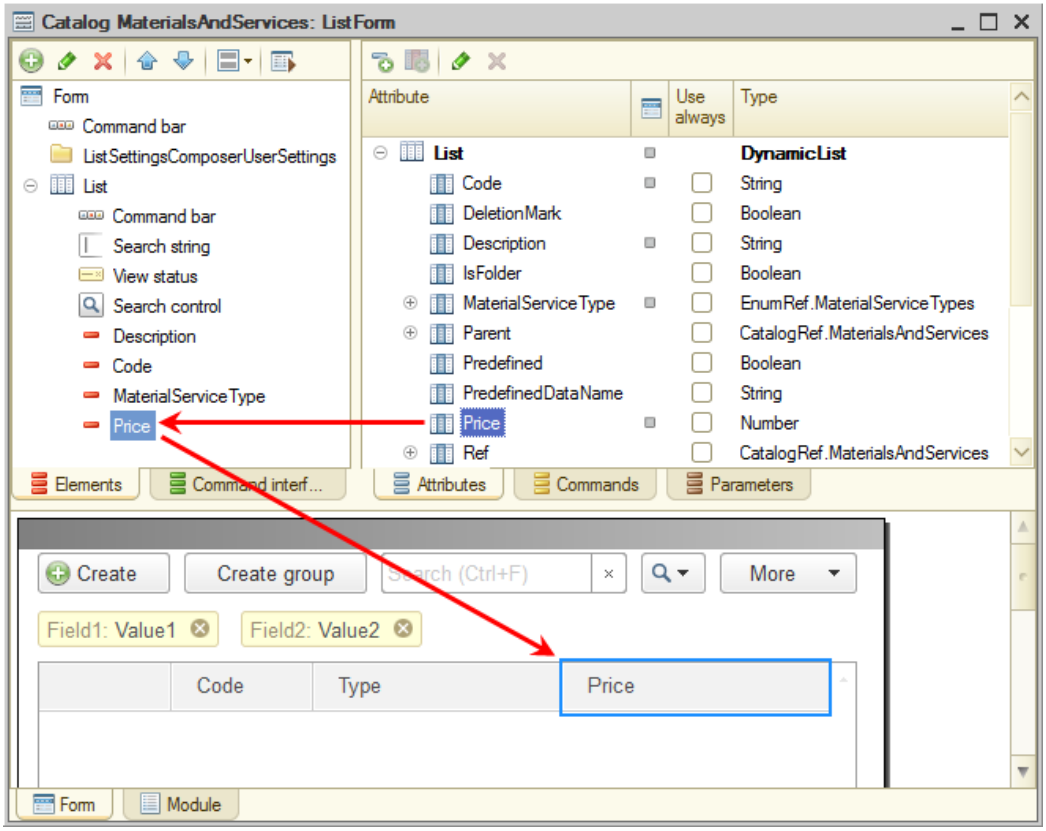


Fig. 27.21. Adding a column to the list form

Now let us specify individual properties for each conditional appearance settings item.

22. In the dynamic list settings window, on the **Settings** tab, click the first settings item and then click the **Custom settings item properties** button.
23. Select the **Include in custom settings** check box, set **Presentation** to *Services*, and in the **Edit mode** field, leave the **Quick access** value.
24. Click **OK**.
25. For the second settings item, select the **Include in custom settings** check box, set **Presentation** to *Low price*, and in the **Edit mode** field, leave the **Quick access** value.
26. At the bottom of the tab, set the general **Editing mode** property to **Normal** (fig. 27.22).

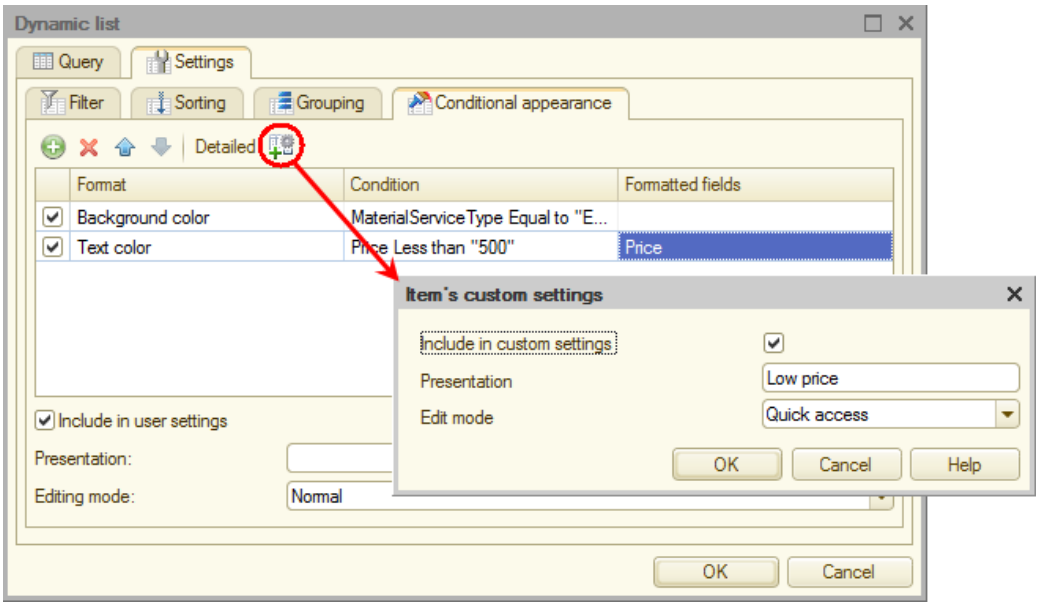


Fig. 27.22. Conditional appearance settings item properties

In 1C:Enterprise mode

Let us test the changes.

- Start 1C:Enterprise in the debug mode and open the list of materials and services.

  You can see that materials and services are accompanied by their current prices, prices under 500 are displayed in blue color, and rows that represent services have lavender background.

  Both conditional appearance settings are displayed above the list of materials and services, they have human-readable names, and you can easily enable or disable them (provided that you remove the general setting that adds the backgound color, if it is still present in the user settings), fig. 27.3.

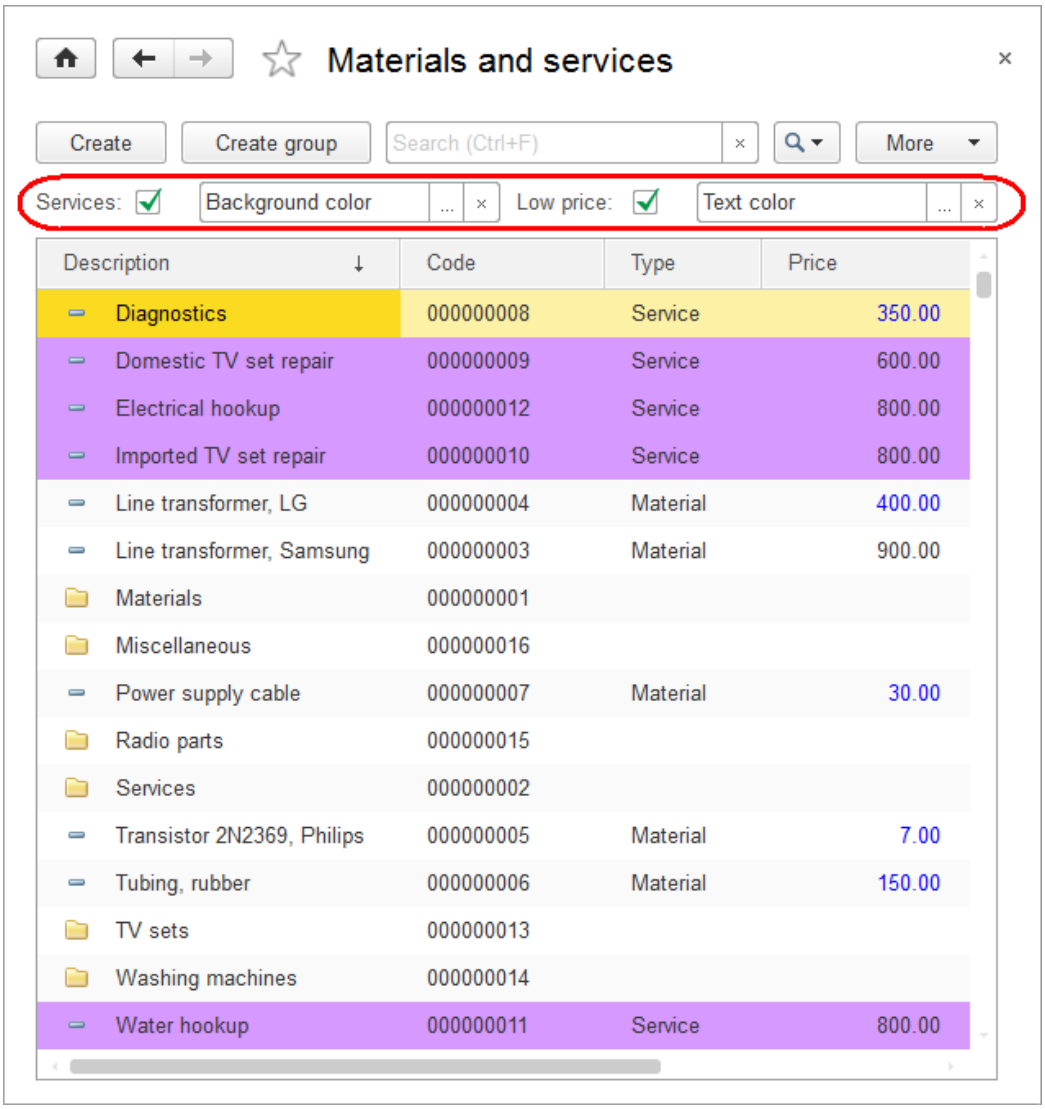


Fig. 27.23. List of materials and services with the specified formatting

Finally, note the Auto save user settings property of a dynamic list (see fig. 27.16). It is enabled by default.

This is why, once a user closes the form, its list settings are saved to a system storage, and once this user opens this form again, the settings are loaded from the storage. In addition to that, users can save list settings manually with custom names by clicking **Save settings** or **Select settings** on the **More** menu. You can use this feature to store multiple settings options for a single list.

You can also revert to the list settings specified by a developer in Designer by clicking **Set default settings** on the **More** menu.

Selection list in a text box

You can use selection lists in text boxes, which improves their usability.

A text box that has a reference type, such as Warehouse or Employee, by default includes a selection button that opens the selection form for a referenced object.

You might need to implement a similar selection from a set of predefined values for fields of other types.

Let us review a very simple example where a user needs to enter customer addresses starting with city names.

In Designer mode

Let us add a selection list to a text box.

1. In Designer, open the configuration object editor of the **Customers** catalog.
2. On the **Data** tab, add the **Address** attribute of the **String** type with length *25*.
3. Open the **Customers** catalog item form.
4. Expand its main **Object** attribute and drag the **Address** field from the attributes pane to the form controls pane.
5. In the property palette that is opened, set the **ListChoiceMode** check box.
6. In the **ChoiceList** field, click the selection button and then use the **Add** button to add cities: *New York*, *New Rochelle*, and *Boston* (fig. 27.24).

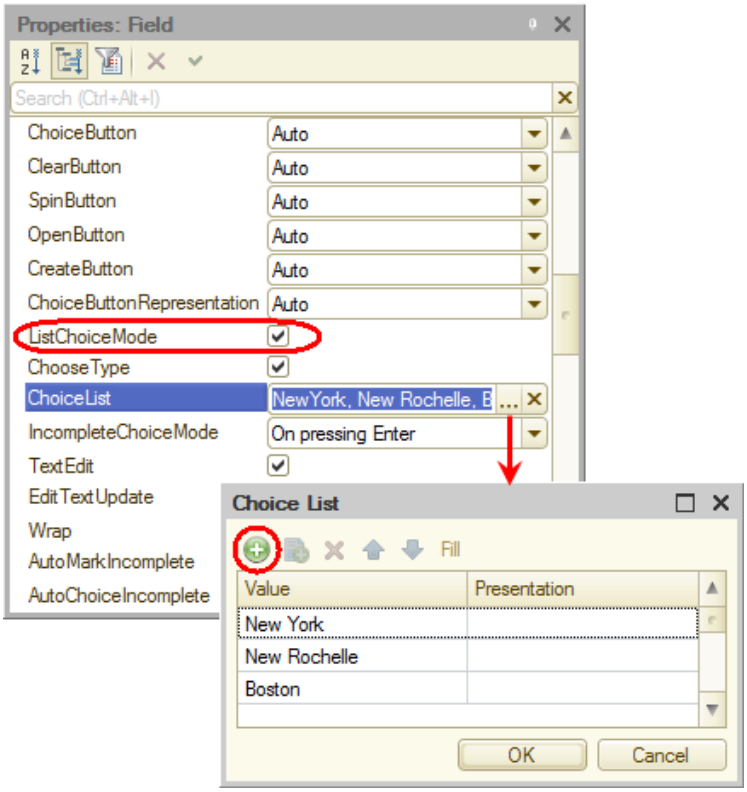


Fig. 27.24. Creating a selection list for a text box

In 1C:Enterprise mode

Let us test the text box.

1. Start 1C:Enterprise in the debug mode and open the customer form.
2. In the **Address** field, type *n*.

   The platform suggests two cities starting with this letter (fig. 27.25).

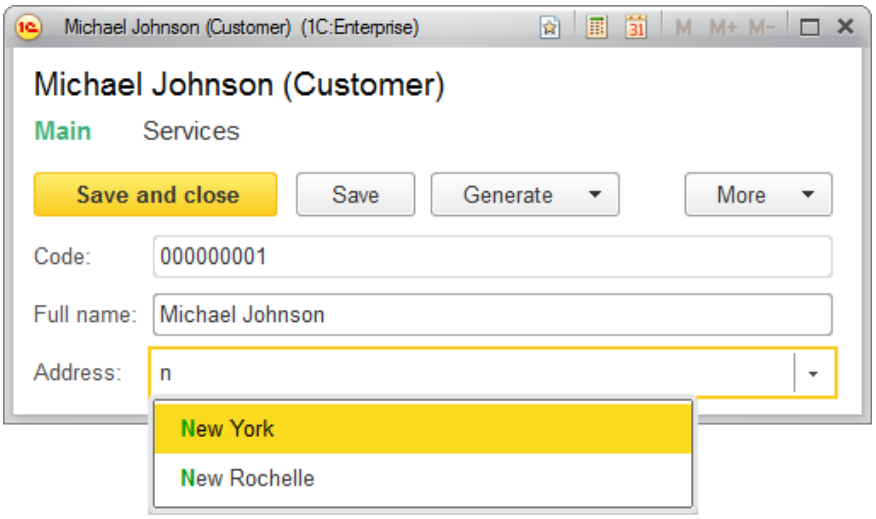


Fig. 27.25. Selecting from a set of predefined values in a text box

The operations that you performed in Designer are also available in 1C:Enterprise script. This means that you can use some algorithm for dynamic generation of a selection list for a text box.

Moreover, you can use 1C:Enterprise script to modify an automatically generated selection list (for example, a selection list in an automatically generated form). To do so, use the ChoiceDataGetProcessing event handler that is located in the object manager module. For example, to implement picking a catalog item in a text box, you can put the event handler to the catalog manager module.

Choice form for a field with a reference attribute

Users often enter data in fields that store reference attributes (attributes that reference some configuration objects).

For example, in the Services document they fill the following reference fields: Customer (type: CatalogRef.Customers), Technician (type: CatalogRef.Employees), and so on.

Clicking the selection button in a reference field opens the default choice form of the respective configuration object (Customers catalog, Employees catalog, and more).

You might need to use a custom choice form for a reference attribute. To do so, in the Choice form property of the attribute, specify the custom form.

That custom form will be opened every time this attribute is selected in any form.

In Designer mode

Let us use the Services document as an example. Suppose that you do not want to use the default choice form of the Employees catalog for selecting a value in the Technician field and you want to use a custom form instead.

First you have to create this form.

1. Open the editor of the **Employees** catalog configuration object and click the **Forms** tab.

   This object has no forms at all. This means that all the catalog forms used in 1C:Enterprise mode (including the choice form) are generated automatically.

2. Click **Add** (fig. 27.26).

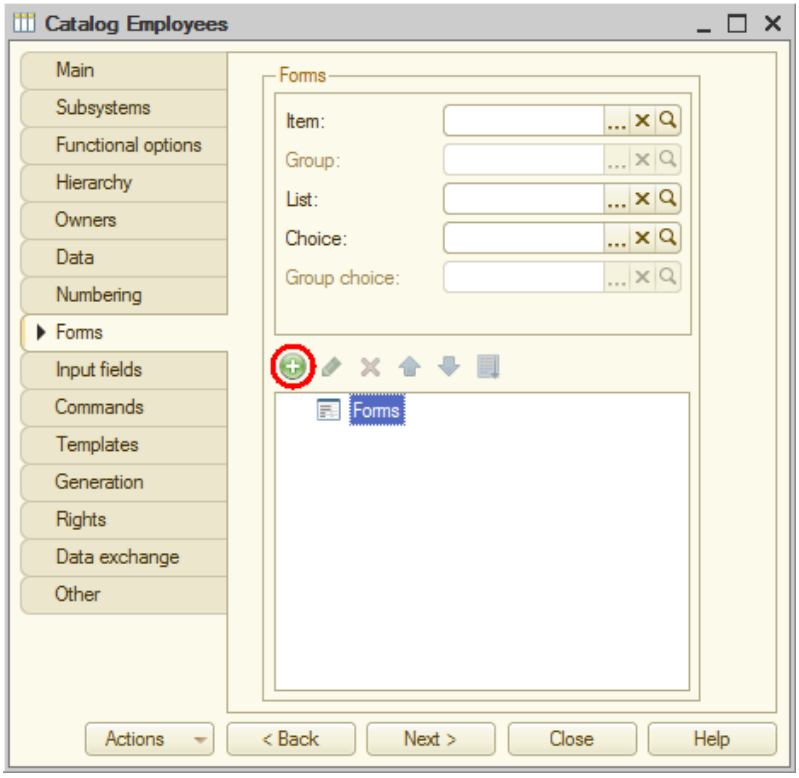


Fig. 27.26. Creating an Employees catalog form

3. In the form wizard that is opened, select **Generic form** and enter *ChoiceForm* for its name (fig. 27.27).

Fig. 27.27. Creating a generic form for the Employees catalog

4. Click **Finish**.

   The form has no controls or data because it is a generic form and you can populate it with data and controls as you want.

5. On the **Attributes** tab, create a main form attribute named *List* with **DynamicList** type, and set its **Main table** property to **Catalog.Employees** (fig. 27.28).

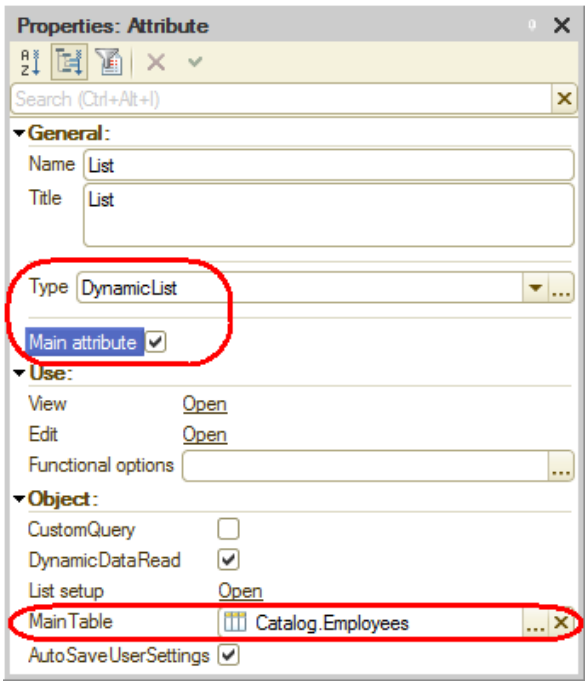


Fig. 27.28. Creating a main form attribute

Attention! It is important that you select the **Main attribute** check box. If you skip this step, the form is unable to determine its data source.

6. Drag the **List** attribute to the form controls window.
7. When asked "Add columns to the table "List"?", click **Yes**.

   This adds the List table, which contains a list of employees, to the form.

   Now let us slightly modify the form appearance.

8. In the form controls pane, expand the **List** table and delete the **Code** field.

   Since the table has its own command bar, let us hide the form command bar.

9. Right-click the root **Form** item, open its property palette, and then, in the **CommandBarLocation** field, select **None** (fig. 27.29).

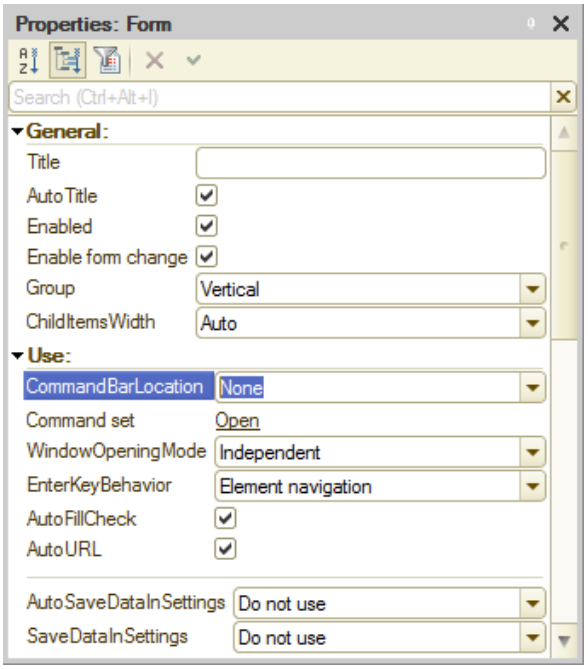


Fig. 27.29. Specifying form properties

Next, let us enable selecting values from the table that contains the list of employees.

10. In the **List** table property palette, select the **ChoiceMode** check box (fig. 27.30).

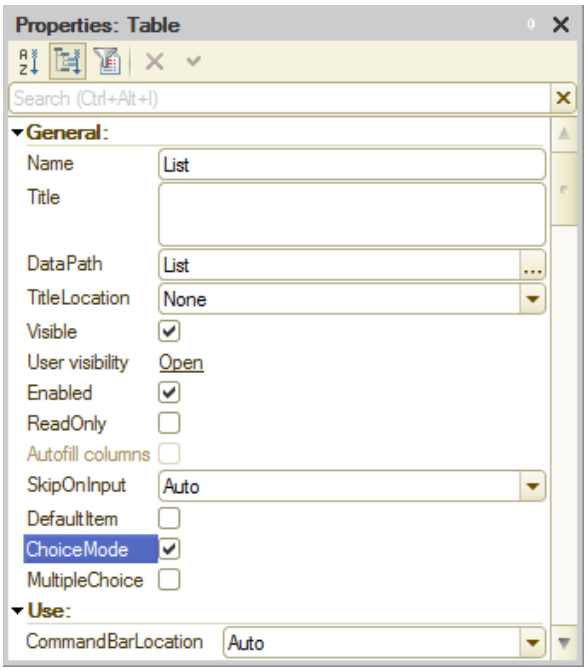


Fig. 27.30. Table properties

Finally, let us change the form title.

11. In the form property palette, in the **Title** field, enter *Select employees*.
12. Clear the **AutoTitle** check box.

    This prohibits displaying the extended presentation of the Employees catalog as the form title (fig. 27.31).

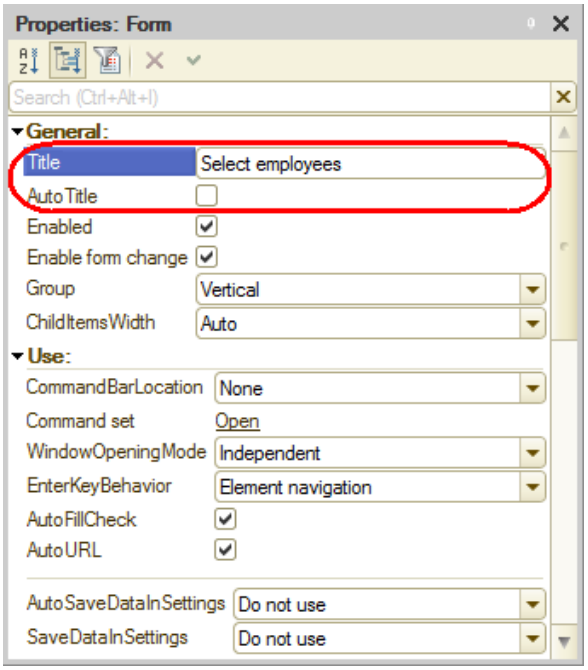


Fig. 27.31. Specifying form properties

So you have created a form that is very similar to the choice form. You have removed the Code field from the list and modified the form title to make it different from the choice form.

Obviously in reality generic form functionality should be different from that of the choice form because otherwise making it the choice form does not make sense. But this is beyond your current task. The only thing we want to demonstrate is how a generic form is used as a choice form.

13. Open the configuration object editor for the **Services** document.
14. Open the property palette of the **Technician** attribute and click the selection button in the **Selection Form** field.

    This opens a list of custom forms created in the configuration for the object referenced by this attribute. The Tecnhician attribute references the Employees catalog, which has a single custom form: ChoiceForm.
15. Select **ChoiceForm** (fig. 27.32).

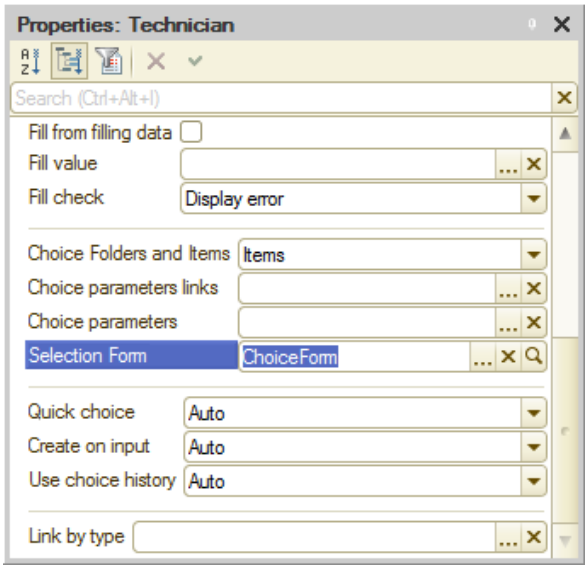


Fig. 27.32. Specifying a choice form for a reference attribute

In 1C:Enterprise mode

Let us test the new form.

1. Start 1C:Enterprise in the debug mode.
2. Open any of the **Services** documents.
3. In the **Technician** field, click the **Select from list** button and then, in the drop-down list, click **Show all**.

   This opens the custom form that you created, with Select employees as a title (fig. 27.33).

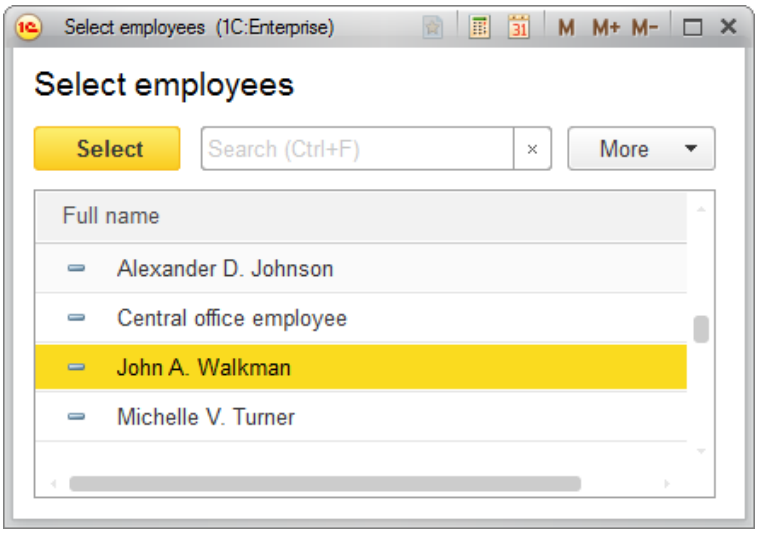


Fig. 27.33. Employee choice form

Since you have defined the Selection Form property at the Technician attribute level instead of the form level, any form of the Services document that includes the attribute opens this custom form to select a technician.

If you clear the Selection Form property of the Technician attribute in the Services document, an attempt to select a value in the Technician field in the 1C:Enterprise mode opens the automatically generated default form of the Employees catalog.

Attribute fill check

Attributes of configuration objects support both automatic and 1C:Enterprise script-based fill checks. Instead of form level, this check is performed at the attribute property level or in the module of the object where the attribute belongs. This ensures that the fill check is performed in all forms that include the attribute.

Automatic fill check

In Designer mode

Let us implement a fill check in the Services document configuration object. Suppose that you want to ensure that the PropertySet attribute of the document tabular section is filled.

1. Open the configuration object editor for the Services document.
2. Open the property palette of the **PropertySet** tabular section attribute and set the **Fill check** property to **Display error** (fig. 27.34).

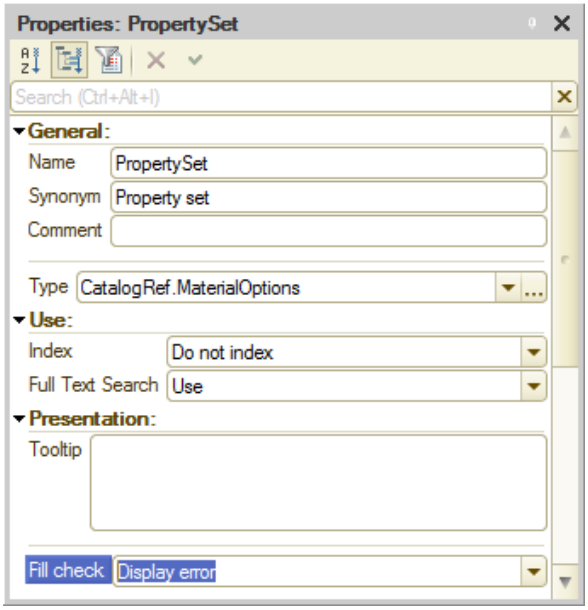


Fig. 27.34. The Fill check property of the PropertySet attribute

This value defines that when a document is written, the platform checks whether this attribute value is empty. And if it is empty, it displays an error message and does not save the document.

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. Open the Services document #CI0000001.

   The tabular section of the document contains two rows, both having empty values in the Property set column.
3. Click **Post**.

   Instead of posting, two error messages are displayed and the changes are not saved (fig. 27.35).

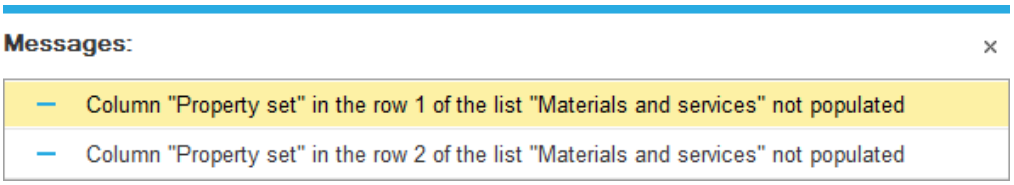


Fig. 27.35. Error messages

1C:Enterprise script-based fill check

In Designer mode

Your application logic might require custom attribute fill checks. In this scenario, if the Fill check property of an attribute is set to Display error, you can use 1C:Enterprise script to remove the attribute from the array of

attributes to be checked and then implement a custom check. Alternatively, you can add an attribute that is not normally checked to the array of attributes to be checked.

1C:Enterprise-based fill checks for configuration objects are executed in FillCheckProcessing() event handlers that are located in object modules. A FillCheckProcessing event handler is called every time an object form is saved or a document is posted. Fill checks for values that are entered interactively must be performed in this event handler instead of when the object is being saved.

- Open the **Services** document module and add the procedure shown in listing 27.4.

Listing 27.4. FillCheckProcessing event handler

```
Procedure FillCheckProcessing(Cancel, AttributesToCheck)

   Index = AttributesToCheck.Find("MaterialsAndServices.PropertySet");
   If Index <> Undefined Then
      AttributesToCheck.Delete(Index);
   EndIf;
   Index = 0;

   For Each CurRowMaterialOrService In MaterialsAndServices Do
     If CurRowMaterialOrService.MaterialOrService.MaterialServiceType =
        Enums.MaterialServiceTypes.Material Then

        If Not ValueFilled(CurRowMaterialOrService.PropertySet) Then
           Message = New UserMessage();
           Message.Text = "The ""Property set"" column is not filled
              | in row " + String(Index+1) + " of the ""Materials and
              | services"" list";
           Message.Field = "MaterialServiceList[" + String(Index)
              + "].PropertySet";
           Message.SetData(ThisObject);
           Message.Message();

           Cancel = True;

         EndIf;
      EndIf;
      Index = Index + 1;
   EndDo;

EndProcedure
```

First the procedure searches the AttributesToCheck array for the PropertySet tabular section attribute. This array is passed to the event handler and contains all the attributes that have the Fill Check property set to Display error. If the PropertySet attribute is found, it

is deleted from the array because a custom check is performed for that attribute.

Next the procedure iterates through the document tabular section rows and generates an error message for each tabular section row that has an empty value in the PropertySet column.

Setting the Cancel parameter to True means that the document is not posted if at least one row that contains a material has an empty PropertySet value. If you comment this script line out, error messages will be displayed but the document will be posted.

To simplify the example, we used the MaterialsOrServices.MaterialOrService.MaterialServiceType call, though using a query is optimal here. This issue has been discussed in detail in lesson 14 so we will not discuss it here again.

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. Open the Services document #CI0000001 and click **Post**.

   An error message is displayed for the second row of the tabular section (fig. 27.36).

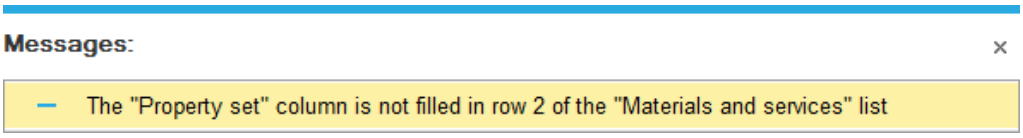


Fig. 27.36. Error message upon posting a document

You can also use 1C:Enterprise script to add an attribute to the array of attributes that are checked (listing 27.5).

Listing 27.5. Adding an attribute to the array of attributes that are checked

```
AttributesTocheck.Add("MaterialsAndServices.PropertySet");
```

Using parameterized commands

You can use parameterized commands in object forms to pass some parameter (for example, a value of a reference attribute) to a command handler. This parameter will be available for further use, for example, you can open a report form using the parameter.

We will explain this on the example of a command that opens a Balance of materials report for a warehouse specified in a document. The report is opened from the Services document form, and the Warehouse document attribute value is passed to the report. When the report is opened, a filter by the Warehouse parameter is added to the report settings.

In Designer mode

First, let us create a parameterized command for the Materials report configuration object.

1. Open the configuration object editor of the **Materials** report.
2. On the **Commands** tab, click **Add** and create a command named *BalanceByWarehouse*.
3. In the property palette that is opened, set **Command parameter type** to **CatalogRef.Warehouses**.
4. Set the **Group** property to **Form command bar.Important** (fig. 27.37).

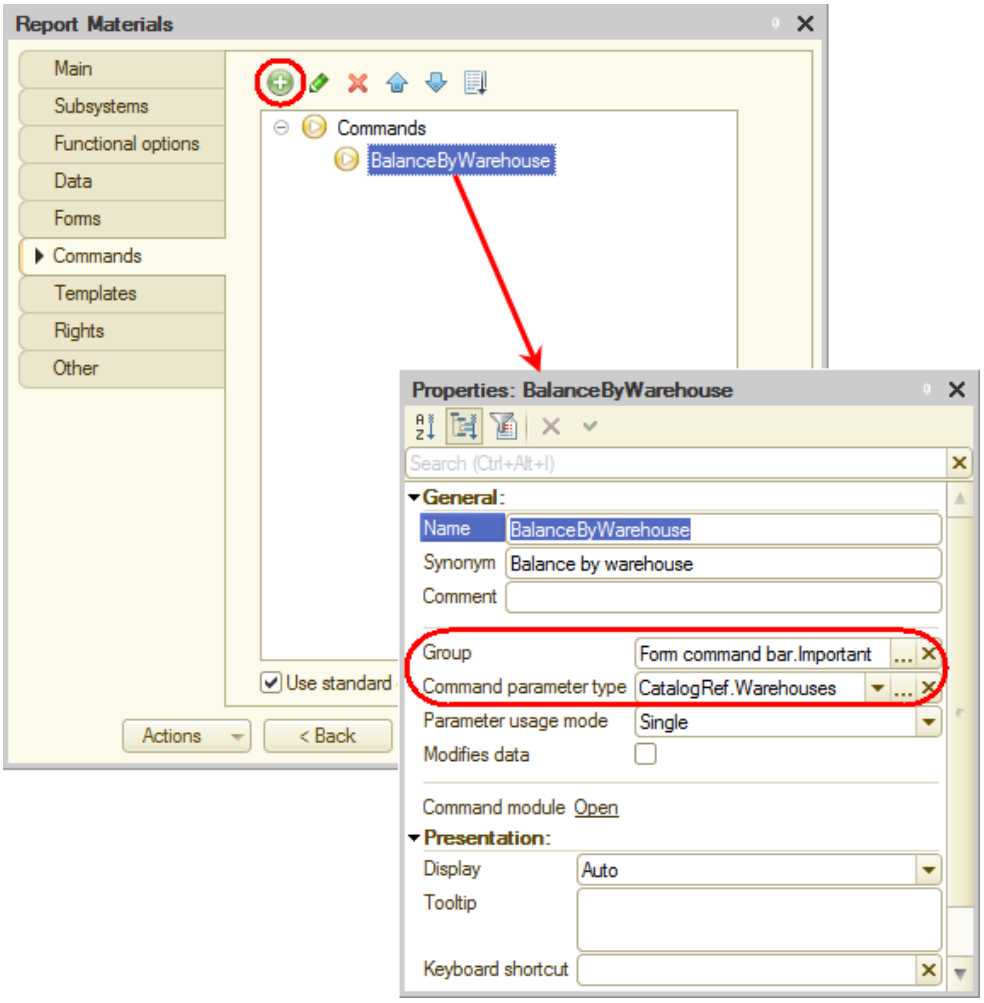


Fig. 27.37. Creating a parameterized command

So you have created a command with a parameter of CatalogRef.Warehouses type. This command is automatically added to the list of available parameterized commands of each form that includes an attribute of that type.

5. In the command module that is opened, fill the handler as shown in listing 27.6.

Listing 27.6. Parameterized command event handler

```
&AtClient

Procedure CommandProcessing(CommandParameter,
    CommandExecuteParameters)


    FormParameters = New Structure("Filter, PurposeUseKey,
        GenerateOnOpen", New Structure("Warehouse", CommandParameter),
        "BalanceByWarehouse", True);

    OpenForm("Report.Materials.Form", FormParameters,
        CommandExecuteParameters.Source, True,
        CommandExecuteParameters.Window);


EndProcedure
```

Let us examine this script.

The CommandProcessing() procedure receives a CommandParameter that contains a value of the CatalogRef.Warehouses type. Next the structure of form parameters (FormParameters) is created. It includes the following parameters: Filter, PurposeUseKey, and GenerateOnOpen.

A structure that contains the Warehouse item with the value passed in the command parameter (CommandParameter) is assigned to the Filter parameter. The PurposeUseKey parameter, which defines the form purpose, is set to BalanceByWarehouse. The GenerateOnOpen parameter is set to True, which defines that the report is generated immediately after opening.

Next the form parameter structure is passed to the global OpenForm() method and the form specified in the first parameter of the method is opened with the Warehouse parameter. Since the fourth method parameter Uniqueness is set to True, every time the form is opened it is a new report form and the report is generated based on the warehouse filter passed in the command parameter.

Now let us add the command to the document form.

As we mentioned earlier, the forms of the GoodsReceipt and Services documents include the BalanceByWarehouse command because they include the Warehouse attribute of the CatalogRef.Warehouses type.

6. Open the **Services** document form.
7. On the **Commands** tab, click the **Global commands** tab.

   Here you can see the list of global parameterized commands available in the form.
8. In the **Parametrizable** group, expand the **Object** item and drag the **Report.Materials.Command.BalanceByWarehouse(Object. Warehouse)** command to the command bar of the form controls.

   The part enclosed in parentheses (Object.Warehouse) is the value of the Warehouse attribute passed to the command (fig. 27.38).

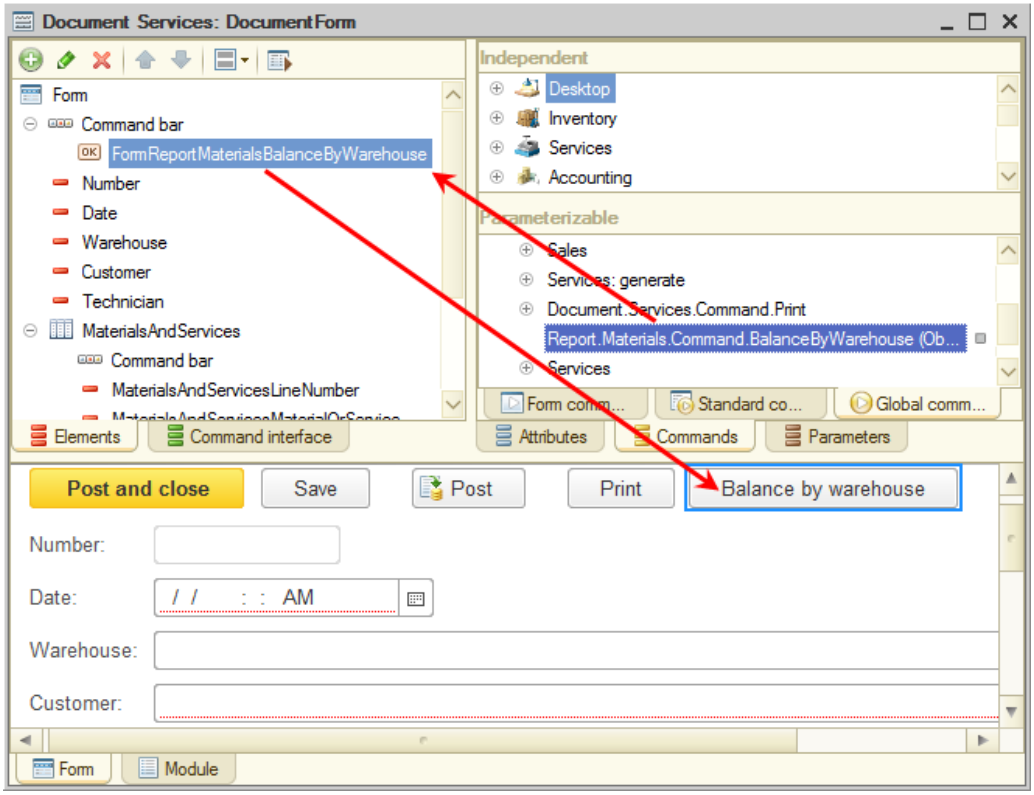


Fig. 27.38. Adding a parameterized command to a form

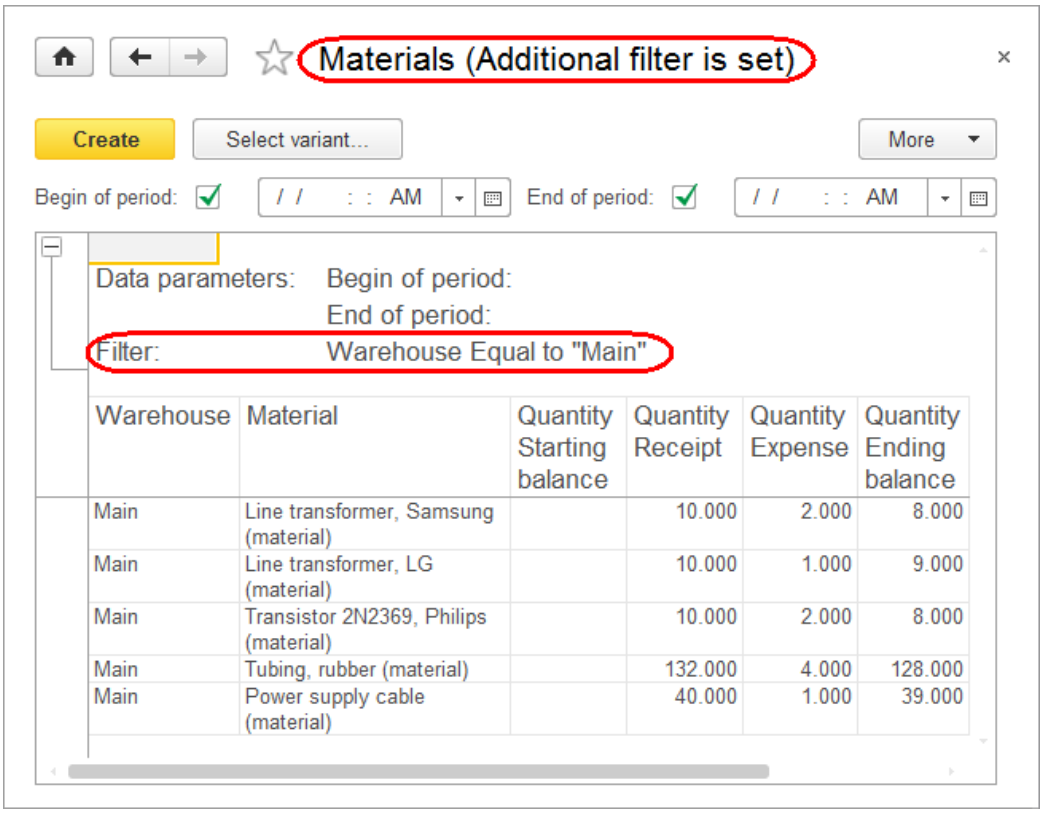
Note that you have not created a report form nor have you added a filter by the Warehouse form parameter to the report settings. It is done automatically when the BalanceByWarehouse event handler is executed.

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. Open the Services document #CI0000001.
3. Click **Balance by warehouse**.

   This opens the Materials report. Its form is generated automatically. The report is generated with a filter by Main warehouse once the form is opened (because this warehouse is specified in the document form, fig. 27.39).



Warehouse	Material	Quantity Starting balance	Quantity Receipt	Quantity Expense	Quantity Ending balance
Main	Line transformer, Samsung (material)		10.000	2.000	8.000
Main	Line transformer, LG (material)		10.000	1.000	9.000
Main	Transistor 2N2369, Philips (material)		10.000	2.000	8.000
Main	Tubing, rubber (material)		132.000	4.000	128.000
Main	Power supply cable (material)		40.000	1.000	39.000

Fig. 27.39. Materials report with a filter by warehouse specified in the document

4. Close the report.
5. In the document form, change **Warehouse** to **Retail**.
6. If this document contains any material rows, delete them.

   This is required because earlier in this lesson you added a fill check for material rows and once you change anything in the document, the platform cannot save it and display the report because of the unfilled Property set field.
7. Click **Balance by warehouse**.

   This opens the Materials report with a filter by Retail warehouse (in this case the report is empty because you have not entered any materials stored in this warehouse).

   So you have implemented a user-friendly feature: opening a report that displays material balances for a warehouse directly from the document form with a filter by warehouse specified in the document.

Opening a list form with the specified filter

Finally, let us review an example of opening a list form with a filter that is specified in advance. This example shows the difference between the fixed (1C:Enterprise script-based) user settings and dynamic list settings specified in Designer.

Suppose that you need to open the list of Service documents from the list of employees, filtered by technician selected in that list.

You have solved a similar task earlier using a filter criterion and the generation feature (see section "Getting generated objects" on page 748). And now let us solve it using 1C:Enterprise script.

In Designer mode

So you need to open the Services document list form with a filter by technician (the Technician field value is a reference to the employee selected in the list of employees).

1. Create a list form of the **Employees** catalog.
2. In this form, create the **Services** command and drag it to the form command bar.
3. In the command property palette, in the **Action** field, create a client command handler (fig. 27.40).

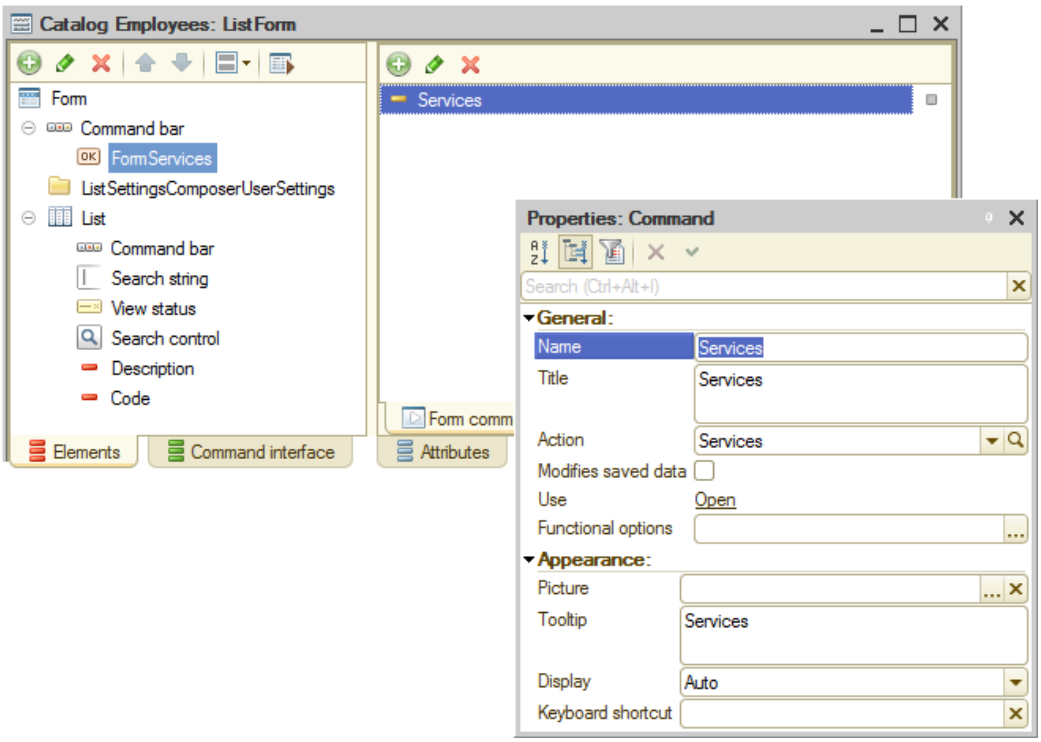


Fig. 27.40. List form of the Employees catalog

4. In the form module, fill the command handler as shown in listing 27.7.

Listing 27.7. Services command handler

```
&AtClient
Procedure Services(Command)


    FormParameters = New Structure("Filter", New
        Structure("Technician", Items.List.CurrentRow));

    OpenForm("Document.Services.ListForm", FormParameters,, True);


EndProcedure
```

This script is similar to the previous listing 27.6 where a report form is opened with a filter by parameterized command value. But here a reference to the current item of the Employees catalog is passed to the Technician field filter. This reference is stored in the CurrentRow property of the List form table, which displays dynamic list data. The filter is passed to the form parameters when the Services document list form is opened.

In 1C:Enterprise mode

Let us test the new form.

1. Start 1C:Enterprise in the debug mode.
2. Open the list of employees.

   You can see that the Services button is now available.
3. Click the **Services** button.

   This opens the list of Services documents with technician equal to the selected employee (fig. 27.41).

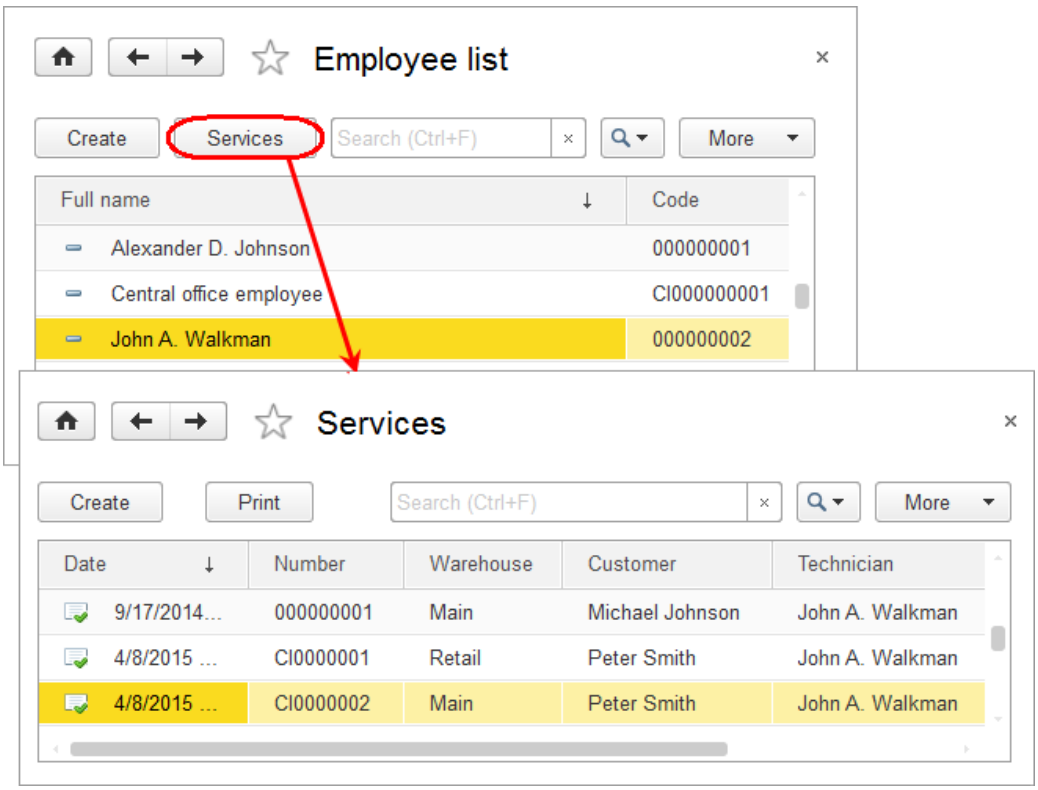


Fig. 27.41. Filter by technician in the Services document list.

However, it is not that simple. To create filters and other dynamic list settings, you have to know the settings types and their priorities.

As we have mentioned earlier, dynamic lists are based on the data composition system. When a form is opened, the list settings are stored to the SettingsComposer property of the dynamic list.

Three types of dynamic list settings are available:

- **Fixed settings**, which are specified using 1C:Enterprise script and stored to the SettingsComposer.FixedSettings dynamic list property.
- **Settings** specified in Designer, which are stored to the SettingsComposer.Settings dynamic list property.
- **User settings**, which are specified in 1C:Enterprise mode and stored to the SettingsComposer.UserSettings dynamic list property.

The user settings are loaded from the system storage and added to the settings specified in Designer. If a settings conflict occurs, the user settings override the settings specified in Designer.

Then the fixed settings are added to the resulting settings. If a settings conflict occurs, an error is displayed. For example, when a list form is opened with a filter (in this case it is a filter by Technician field), the filter is stored to the fixed settings. Therefore, users cannot set a list filter by this field.

So, if you want to give users the option to filter the list by field included in the fixed list settings, you have to remove the filter from the collection of fixed settings (SettingsComposer.FixedSettings.Filter) and then add it to the main list settings (SettingsComposer.Settings.Filter).

In Designer mode

Let us implement opening the list with a filter.

1. Open the list form of the **Services** document.
2. In the form editor, add the following form attributes:
   - **ScriptFilter**. Type: **Boolean**.
   - **FilterField**. Type: **Arbitrary**.
   - **FilterValue**. Type: **Arbitrary**.

   These attributes will store the script filter data in the list form.
3. Create the **OnCreateAtServer** event handler and fill it as shown in listing 27.8.

   Listing 27.8. OnCreateAtServer event handler

```
&AtServer
Procedure OnCreateAtServer(Cancel, StandardProcessing)

    If Parameters.Filter.Property("Technician") Then

        ScriptFilter = True;
        FilterField = New DataCompositionField("Technician");
        Parameters.Filter.Property("Technician",FilterValue);

    EndIf
EndProcedure
```

During the creation of the list form the procedure uses a collection of form parameters to check whether a filter by Technician field is set. If the filter is set, the procedure sets the flag showing that the form is opened using a script filter, sets FilterField to Technician data composition field, and assigns the appropriate filter value from the collection of form parameters to FilterValue.

4. Create the **OnOpen** form event handler and fill it as shown in listing 27.9.

Listing 27.9. OnOpen event handler

```
&AtClient
Procedure OnOpen(Cancel)


    If ScriptFilter = True Then
        ScriptSettings = List.SettingsComposer.FixedSettings;
        For Each SettingsItem In ScriptSettings.Filter.Items Do
            If SettingsItem.LeftValue = FilterField Then
                ScriptSettings.Filter.Items.Delete(SettingsItem);
            EndIf;
        EndDo;


        Settings = List.SettingsComposer.Settings;


        FilterItem =
            Settings.Filter.Items.Add(
            Type("DataCompositionFilterItem"));
        FilterItem.LeftValue = FilterField;
        FilterItem.ComparisonType =
            DataCompositionComparisonType.Equal;
        FilterItem.RightValue = FilterValue;


        List.SettingsComposer.LoadSettings(Settings);
    EndIf;


EndProcedure
```

The handler script is executed if the form is opened with the script filter.

First the procedure iterates through the collection of filter items of the fixed settings of dynamic list settings composer. If the collection includes a filter item where LeftValue is equal to the FilterField attribute value (a script filter by Technician field), this item is deleted.

Then this filter is added to the dynamic list settings collection (SettingsComposer.Settings.Filter) and the modified settings are loaded to the settings composer.

The only thing left is to delete the created filter item when the Services document list form is closed.

5. Create the **OnClose** form event handler and fill it as shown in listing 27.10.

Listing 27.10. OnClose event handler

```
&AtClient
Procedure OnClose(Cancel, StandardProcessing)

    If ScriptFilter = True Then
        Settings = List.SettingsComposer.Settings;
        For Each SettingsItem In Settings.Filter.Items Do
            If SettingsItem.LeftValue = FilterField Then
                Settings.Filter.Items.Delete(SettingsItem);
            EndIf;
        EndDo;
        List.SettingsComposer.LoadSettings(Settings);
    EndIf;

EndProcedure
```

The handler script is executed if the form is opened with the script filter.

The procedure iterates through the collection of filter items of the settings of dynamic list settings composer (SettingsComposer.Settings.Filter). If the collection includes a filter item where LeftValue is equal to the FilterField attribute value, this item is deleted. Then the modified settings are loaded to the settings composer.

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode.
2. Open the list of employees and click the **Services** button.

   This opens the list of Services documents with technician equal to the selected employee.

   Let us view the list settings.
3. On the **More** menu, click **Configure list**.

   The settings window includes a filter by Technician field, and you can edit or clear this filter (fig. 27.42).

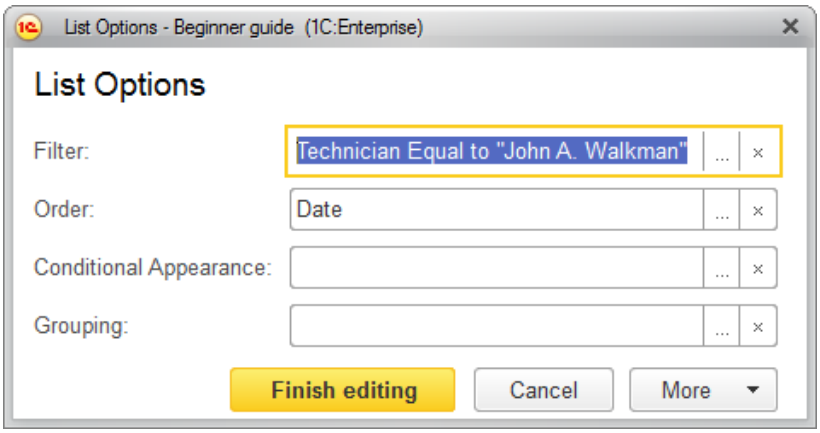


Fig. 27.42. List settings window with a filter specified

Once you close the form, the filter is deleted.

So you learned how to open a list form with a specified filter and how to give users the option to edit this filter.

Quiz

- How are form data and controls connected?
- What is a main form attribute?
- What are extensions of forms and form controls?
- What data types are available in forms?
- How does one convert applied object data into form data?
- What is related data and how does one navigate to related data from a form?
- How does one specify conditional appearance for list form rows?
- How does one define a choice form for a reference attribute?
- How does one implement automatic and 1C:Enterprise script-based fill checks for attributes?
- What is a parameterized command?
- How does one use parameterized commands in forms?
- How does one open a list form with a specific filter?

Lesson 28 (2:00). Form customization techniques

So far you mostly used item or list forms generated by the form wizard. Sometimes you customized the forms by adding tabular section columns, buttons for running custom commands, or other controls.

In this lesson we will summarize the form customization knowledge and you will implement this knowledge to design forms in the form editor.

Form editor

You know the form editor well from the previous lessons. Developers use the form editor to specify a set of controls (fields, buttons, tables, and so on), attributes (data), commands, and parameters used in a form. To make a form interactive, developers specify the relations between form controls and form data, and also customize the interface properties of the form controls.

So developers do not "draw" forms but describe them using a dedicated editor. The platform generates a form based on that description with the option to customize form behavior by adding 1C:Enterprise script algorithms to the form module.

A form editor window is split into several interconnected areas (fig. 28.1).

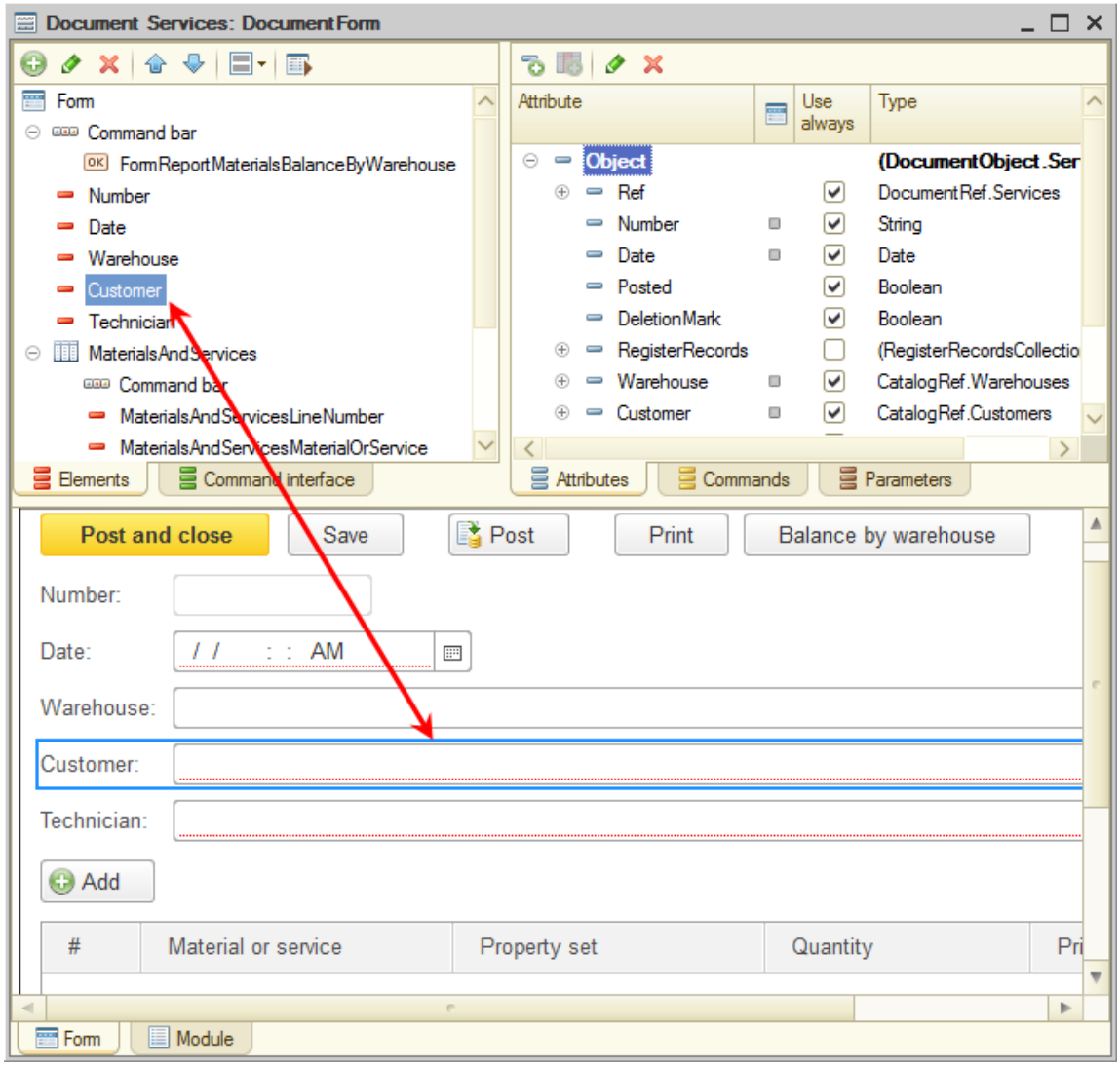


Fig. 28.1. Form editor window

The Elements tab in the upper left pane is used to edit the assortment and order of form controls and combine them into groups. You can change the appearance of the controls by assigning values to their interface properties.

A form is defined as a hierarchical structure with the root Form item. The assortment of form properties, standard commands, and events is defined by the type of the main form attribute. For example, if the main attribute is a document, the Post and close command is included in the command bar. For other types of main attributes, the Save and close command is included instead.

Properties of parent items in the tree of form controls define the assortment of subordinate item properties and their property values. For example, when you add a button to a command bar, the Type property of the button is automatically set to Command bar button.

A form control property value can affect the value of the same property of a parent or subordinate form control. For example, the Width of a parent control can affect the Width of a subordinate control, and vice versa.

The form control positions are defined as follows: controls that are higher in the tree are closer to the top and left form borders (fig. 28.2).

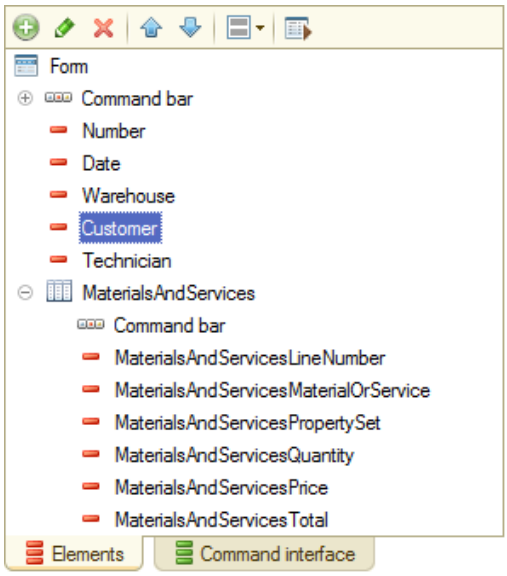


Fig. 28.2. Elements tab of the form editor

The bottom pane of the form editor window displays a form preview. Clicking an item in the tree of form controls highlights it in the preview pane, and vice versa (see fig. 28.1). You can add, edit, delete, or drag form controls both in the form controls pane and in the preview pane.

The Command interface tab in the upper left pane of the form editor window is used to specify the assortment and order of commands in the form navigation panel and command bar. For example, you can set visibility for commands that open related lists or generate objects (fig. 28.3).

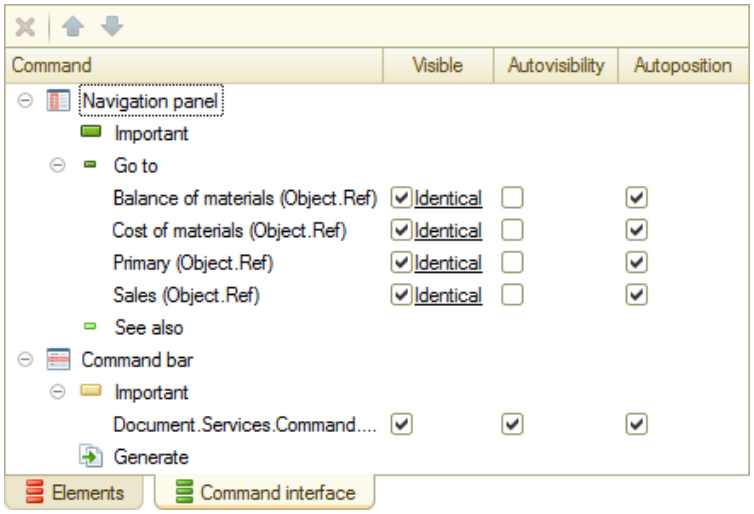


Fig. 28.3. Command interface tab of the form editor

The Attributes tab in the upper right pane of the form editor window is used for specifying the assortment of form attributes and their properties. The main form attribute (the one with the Main attribute property set to True) is displayed at the top of the list in bold font. This attribute defines the form functionality (fig. 28.4).

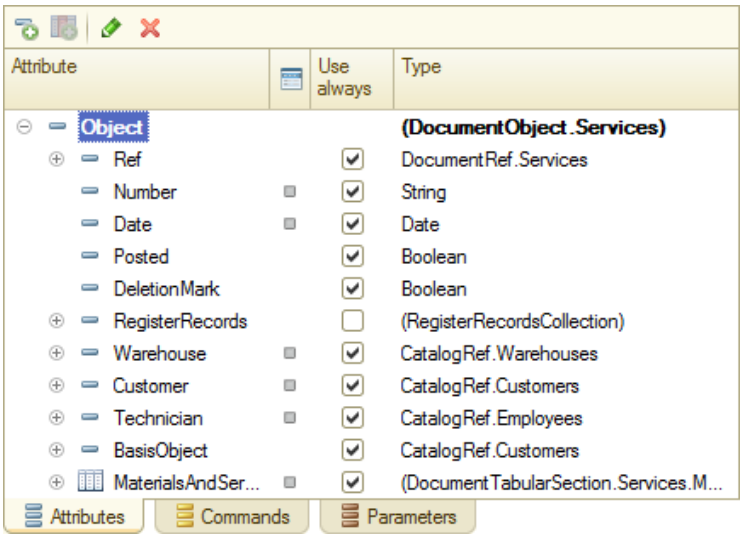


Fig. 28.4. Attributes tab of the form editor

Once you link a form attribute to a form control (using the DataPath property of the form control), attribute data is displayed in the form. The attribute data path is filled automatically when you drag an atribute to the form controls pane, and the platform also creates an appropriate control for displaying the attribute data. This is why dragging attributes is the best way to compose a form. You can switch to the attribute that serves as a data source for a form control by right-clicking the control and then clicking **Go to**.

If you delete a form attribute (and confirm the deletion), form controls linked to the attribute are also deleted from both the controls tree and the preview pane.

The Commands tab in the upper right pane of the form window is used to define the assortment of commands available in the form. This includes developer-created commands (Form command tab), form-specific and form extension commands (Standard commands tab), and global commands that belong to the entire configuration (Global commands tab, fig. 28.5).

Fig. 28.5. Commands tab of the form editor

The Action property of a developer-created command links the command to a handler procedure (a 1C:Enterprise script algorithm described in a form module, see fig. 28.6).

Dragging a command to the form controls pane creates a form control (a button or a hyperlink) that is linked to the command through the CommandName property of the form control. Clicking the control executes

the command handler. You can switch to the command that is linked to a form control by right-clicking the control and then clicking **Go to**.

If you delete a form command (and confirm the deletion), form controls linked to the command are also deleted both from the controls tree and the preview pane.

The Parameters tab in the upper right pane of the form window is used to define the assortment of form parameters. Parameters are used to fine tune form functionality when the form is opened in 1C:Enterprise mode.

The Module tab at the bottom of the form editor window opens the form module, which contains the form algorithms written in 1C:Enterprise script. This includes form command handlers, form control event handlers, and custom procedures and functions (fig. 28.6).

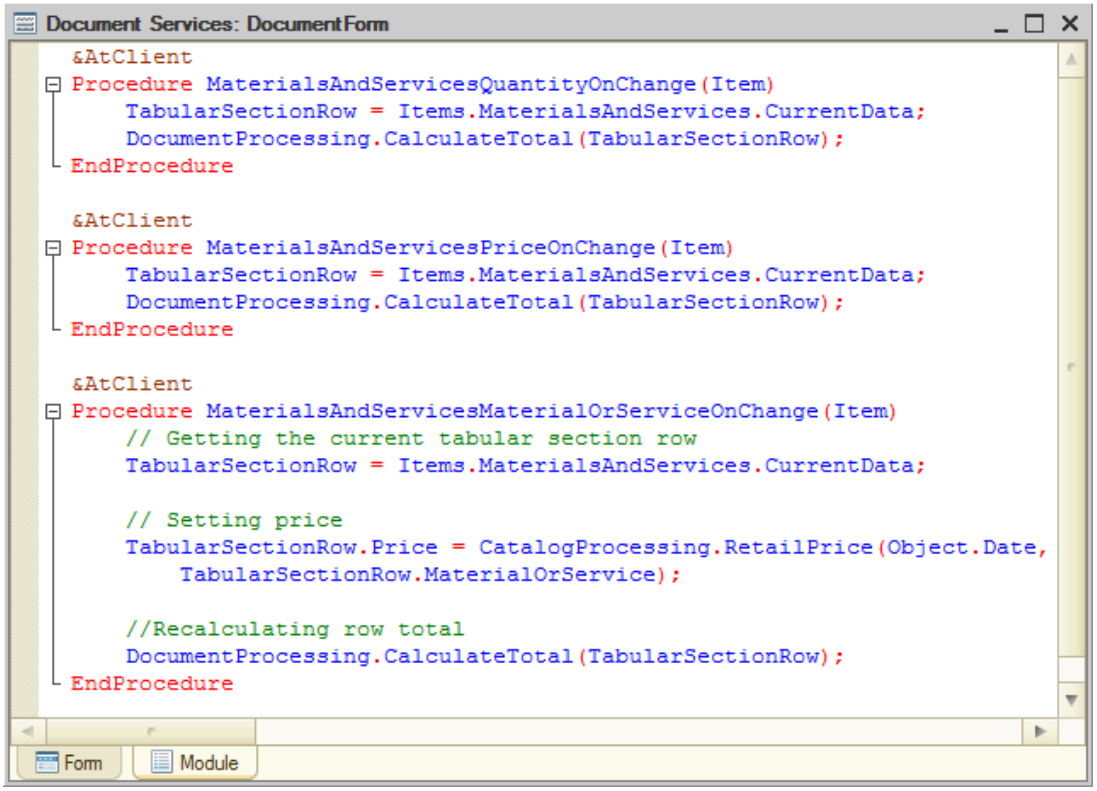


Fig. 28.6. Form module

You can switch to the command handler linked to a form control using the context menu of the form control. You can use the Events context menu item to create or open event handlers of the form control.

Implementing a custom form for the Services document

In Designer mode

Your configuration includes a default Services document form that you created in section "Document form" of lesson 4 on page 130 using the form wizard (fig. 28.7).

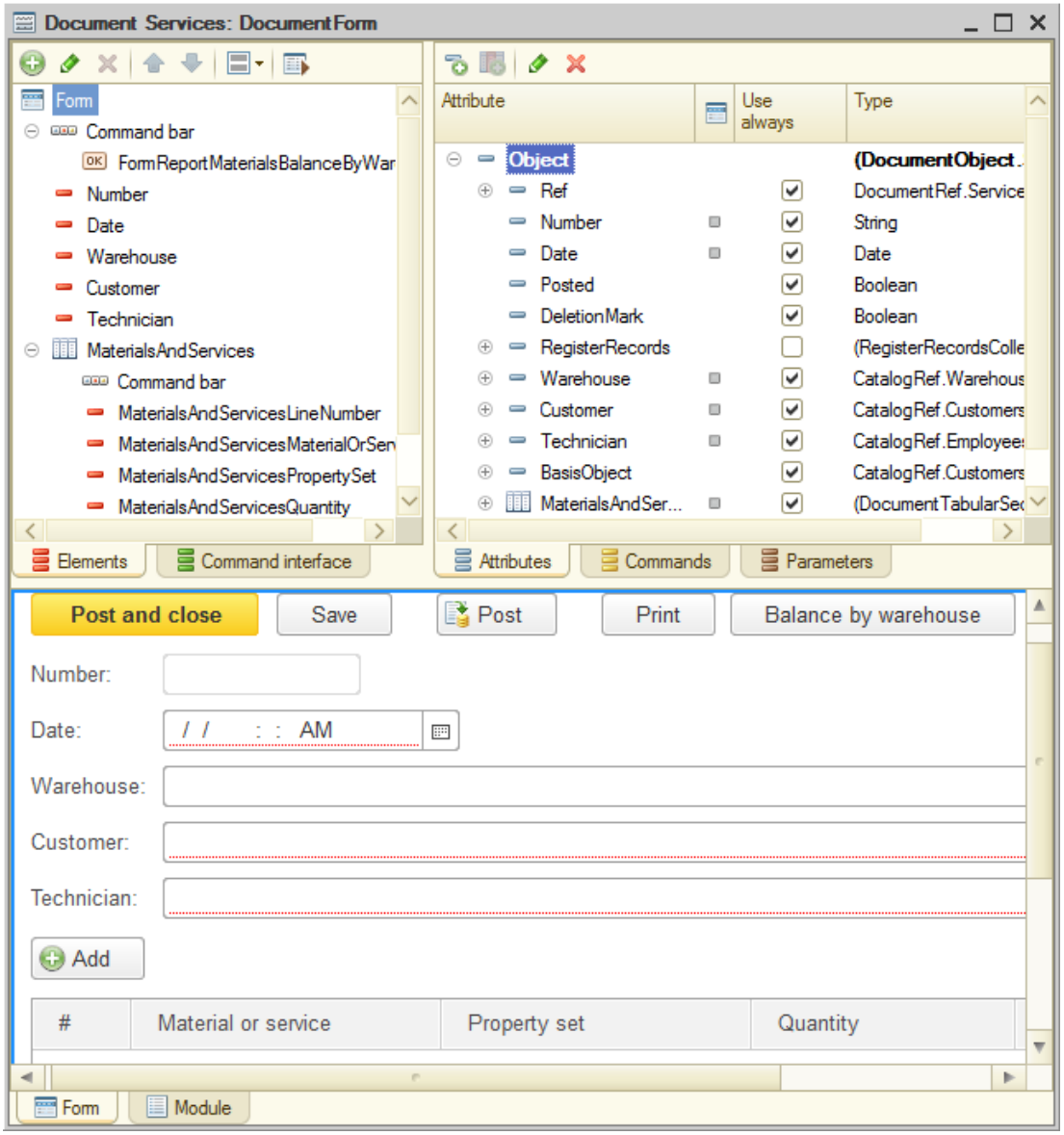


Fig. 28.7. Services document form in the form editor

You can see that the document attributes and its tabular section are displayed in the order that matches the order of form attributes. This provides easy access to all of the document fields but the form lacks usability.

The form looks messy because there are too many fields and columns, and also too big, especially when its tabular section contains many rows.

You can improve the usability by focusing user attention on specific data groups and highlighting the most important data, grouping fields by their purpose, and making the form more compact.

Let us develop a new Services document form from scratch. During the form development you will gain or renew the knowledge of the following form development techniques:

- Adding a main attribute to a form
- Adding a form control linked to a specific attribute
- Grouping form controls

- Specifying group settings and making a group collapsible
- Adding a pane with multiple tabs to a form
- Adding a tabular section with conditional row highlighting
- Displaying a tabular section column total

Let us begin the form development.

1. Open the configuration object editor for the **Services** document.
2. On the **Forms** tab, click **Add** (fig. 28.8).

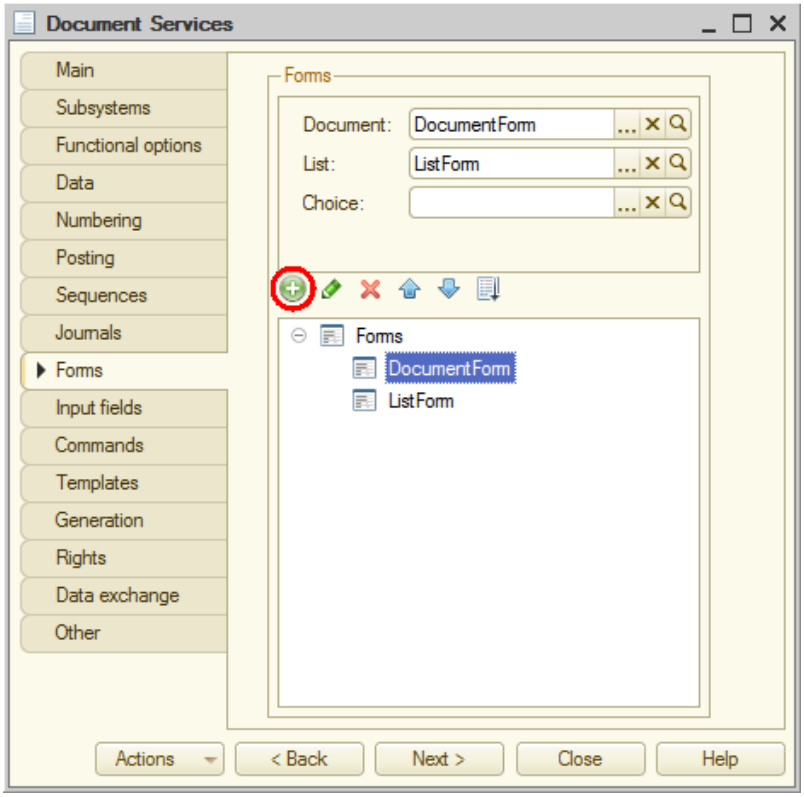


Fig. 28.8. Creating a document form

3. In the document form wizard, click **Generic form**.
4. In the **Name** field, enter *FormForEditing* (fig. 28.9).

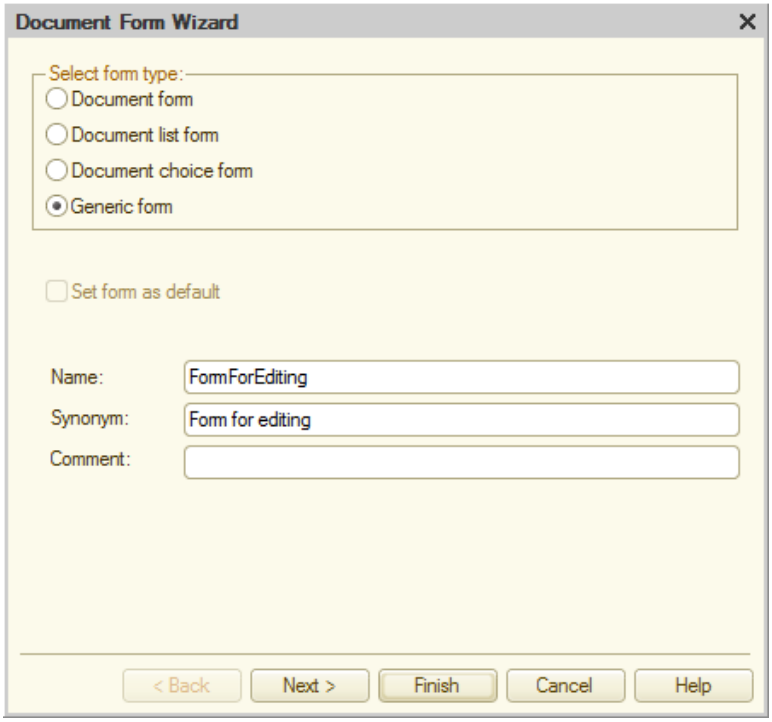


Fig. 28.9. Creating a generic document form

5. Click **Finish**.

   This opens the form editor. You can see that the only controls available in the form are the command bar and the standard More menu. Since it is a generic form, you have to fill it with custom controls and custom data.

6. On the **Attributes** tab, create an attribute named *DocumentObject* with **DocumentObject.Service type** and then, in the attribute properties, select the **Main attribute** check box (fig. 28.10).

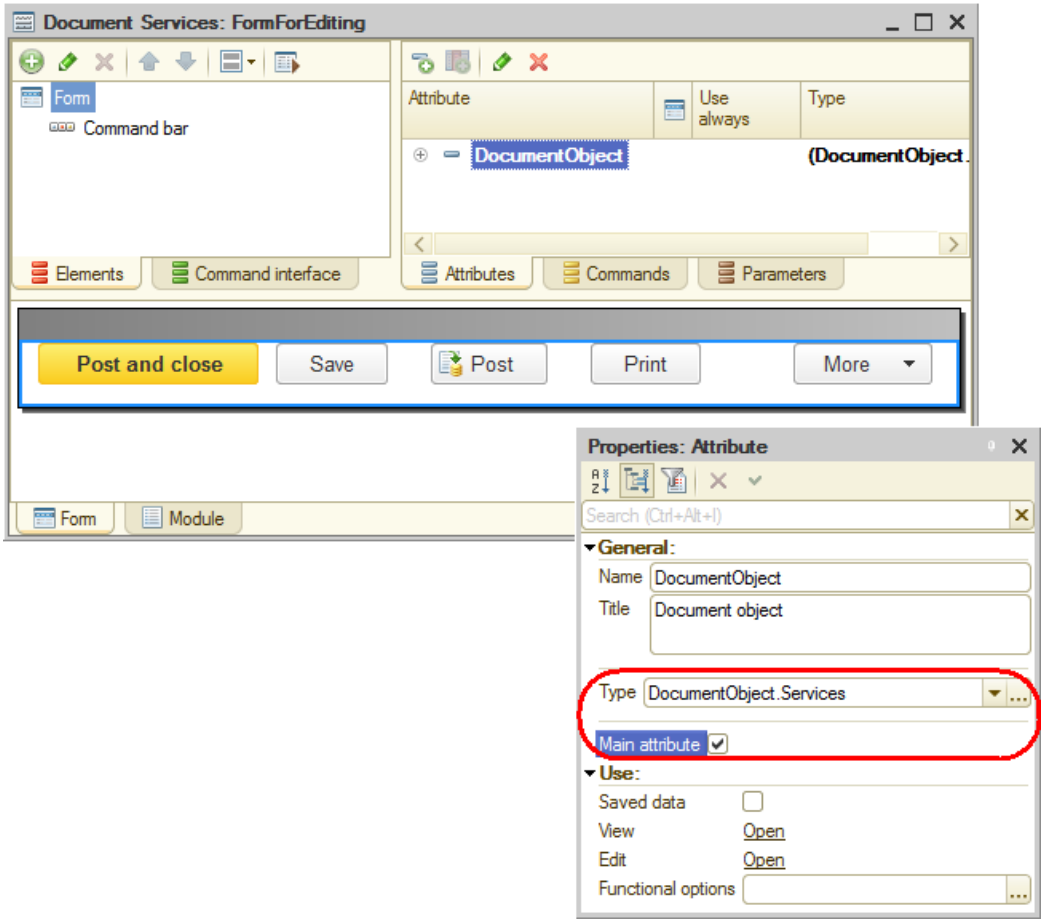


Fig. 28.10. Form editor window

Once you specify the main form attribute, buttons for standard document operations (Post and close, Save, and Post) are added to the form preview.

Also, the data of the DocumentObject.Services object becomes available for editing. You have to link this data to appropriate form controls. The simplest way to link is dragging the attributes to the form controls pane, which automatically writes the links to the DataPath properties of the form controls.

7. In the **Attributes** pane, expand the **DocumentObject** item and drag the following attributes, exactly in this order, to the form controls pane: **Number**, **Date**, **Warehouse**, **Customer**, and Technician.

   For each attribute, a form control is added. The controls are arranged vertically and each control occupies a new row, just like in the form generated by the wizard (fig. 28.11).

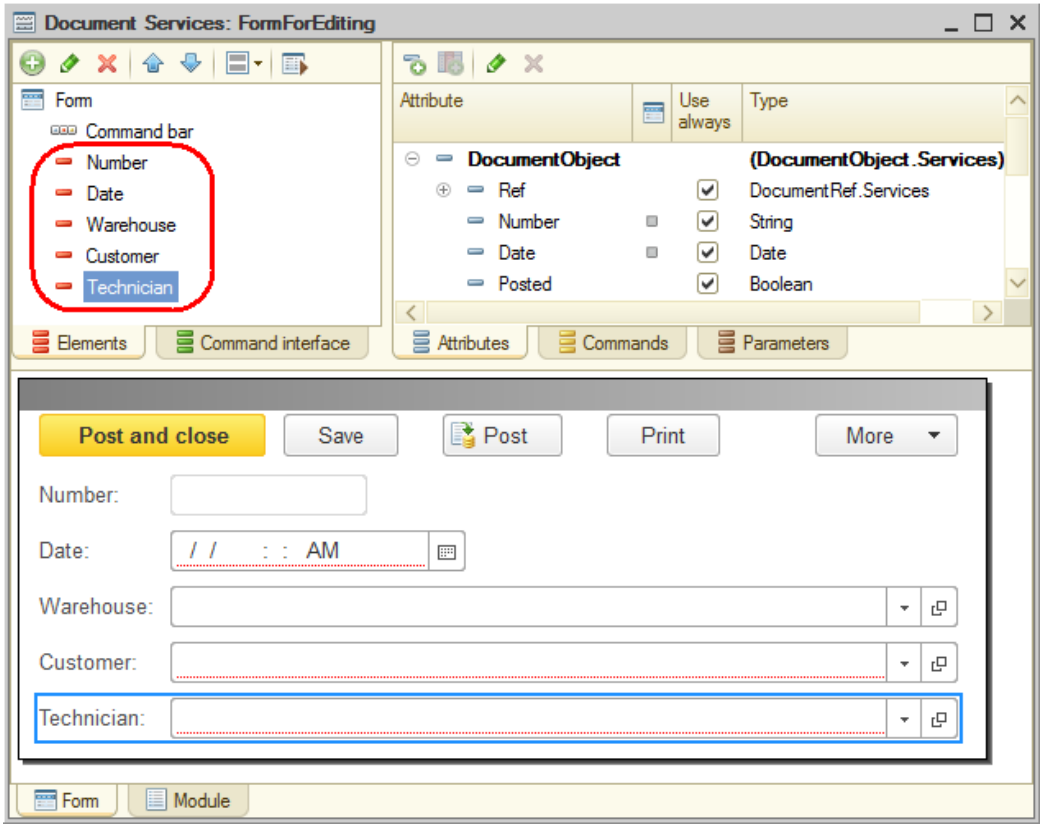


Fig. 28.11. Form editor window

Let us group the form controls.

First, let us group the standard Date and Number document attributes, highlight them with background color, and display them in a single row.

8. In the form controls pane, click **Add**, select **Group - Regular group (no visual presentation)**, and click **OK**.

   This group type suits our needs best because it is not highlighted (you will add custom highlighting later) and it does not have a title (fig. 28.12).

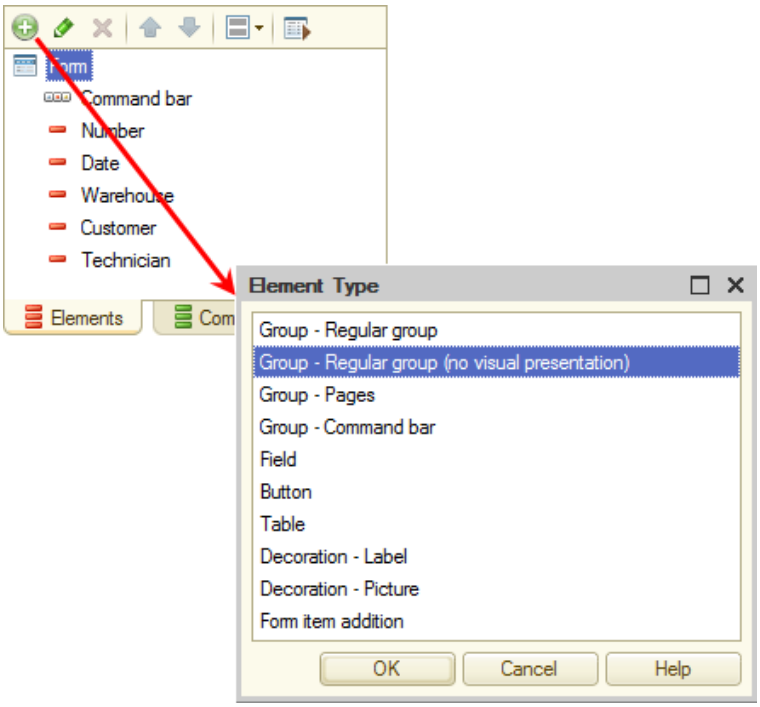


Fig. 28.12. Adding a group to a form

9. In the group property palette, in the **Group** list, select **Horizontal**, clear the **EnableContentChange** check box, and specify a background color (fig. 28.13).

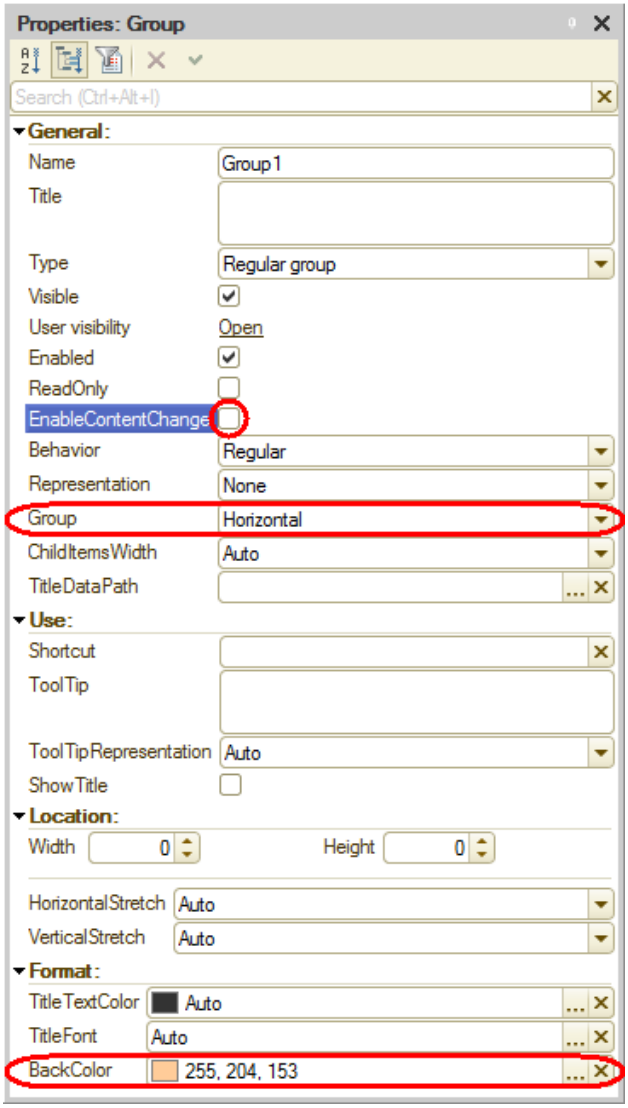


Fig. 28.13. Group property palette

10. Drag the **Number** and **Date** fields to the group and move the group to the upper part of the form (just below the command bar).

    You can see the result of the changes in the preview pane at the bottom (fig. 28.14).

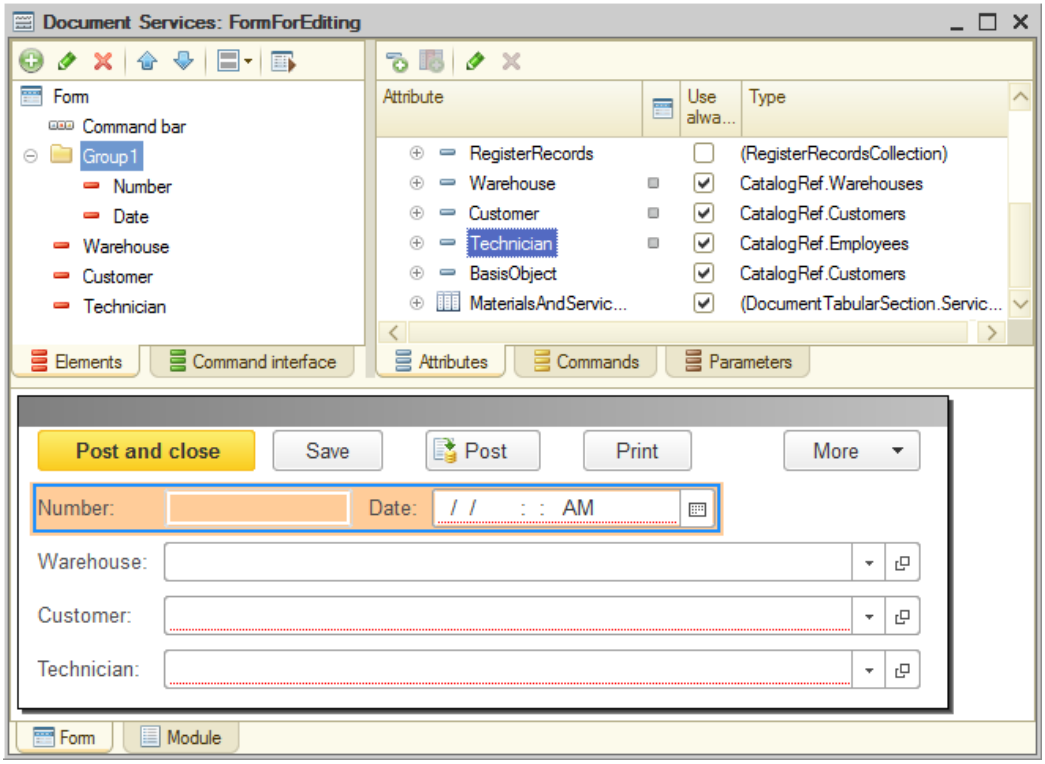


Fig. 28.14. Form editor window

The highlighted group catches the eye and both fields are located in the same row. By clearing the EnableContentChange check box you disabled the group customization in 1C:Enterprise mode. And the form looks more compact because two fields occupy a single row.

Let us add detailed customer info to the form, such as address, phone number, or email. This block should be optional and collapsible. To make the form more compact, it should be collapsed by default.

Adding attributes of a reference attribute to a form is very simple. The type of the Customer document attribute is a reference to the Customers catalog. So you can simply expand the Customer attribute and drag the attributes of the Customers catalog to the form.

11. Drag the **Address** catalog attribute to the form controls window.

    Note that its data path is an expression that includes a dot: DocumentObject.Customer.Address.

12. Move the **Warehouse** field to the bottom (fig. 28.15).

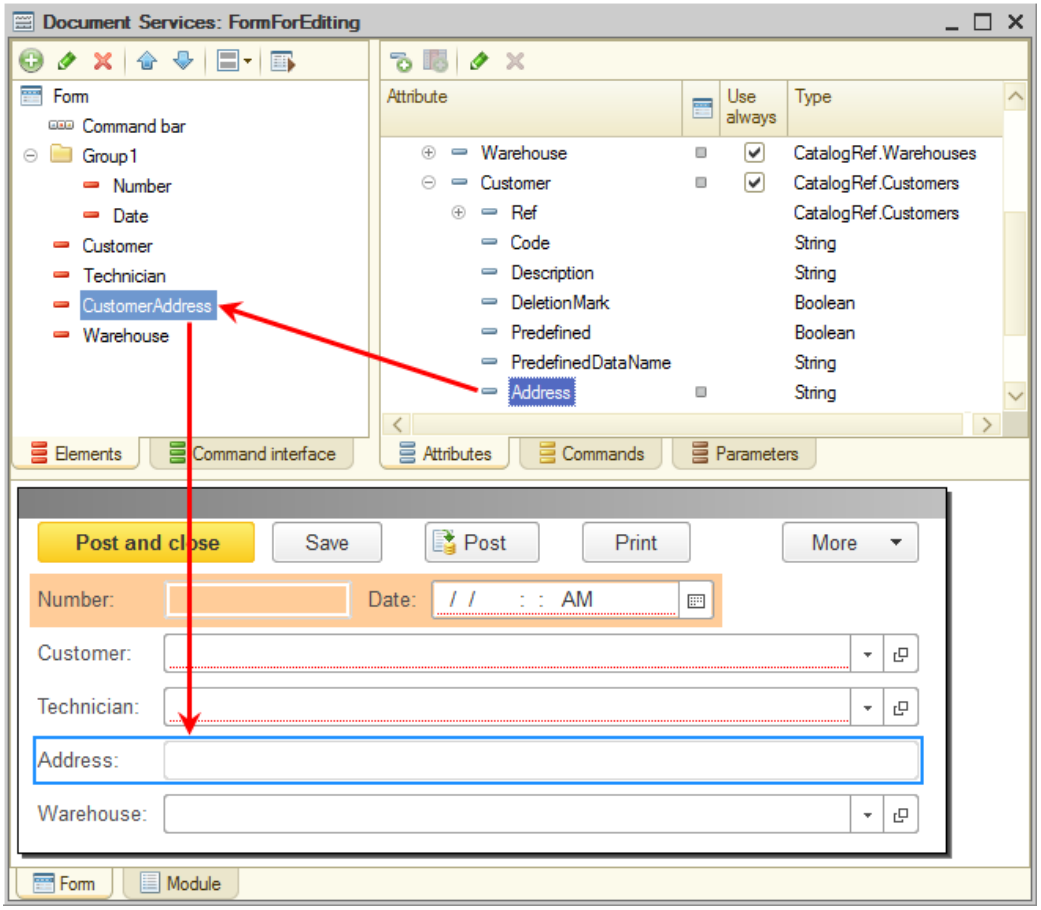


Fig. 28.15. Form editor window

Let us create another group for viewing the address and other customer info and make it collapsible.

13. In the form controls pane, click **Add**, select **Group - Regular** group, and click **OK**.
14. In the group property palette, in the **Title** field, enter *Hide customer info*.
15. In the **Behavior** list, select **Collapsible**.
16. In the **CollapsedRepresentationTitle** field, enter *View customer info*.
17. In the **Representation** list, select **None**.
18. Select the **Collapsed** check box (fig. 28.16).

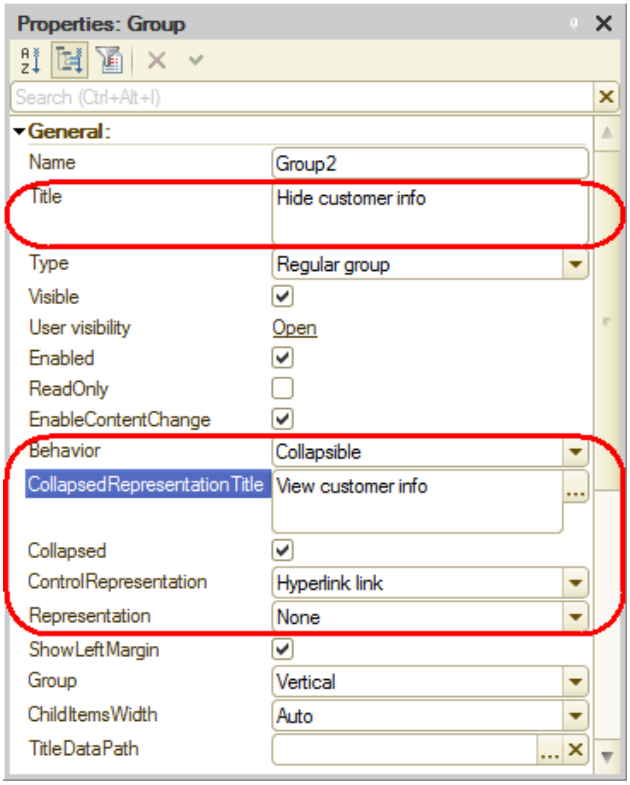


Fig. 28.16. Group property palette

19. Drag the **CustomerAddress** field to the group and move the group to the position below the **Customer** field.

    You can see the result of the changes in the preview pane (fig. 28.17). Note that the preview displays an expanded group with the "Hide customer info" title, but when you open the form in 1C:Enterprise mode, the group will be collapsed and will have the "Show customer info" title.

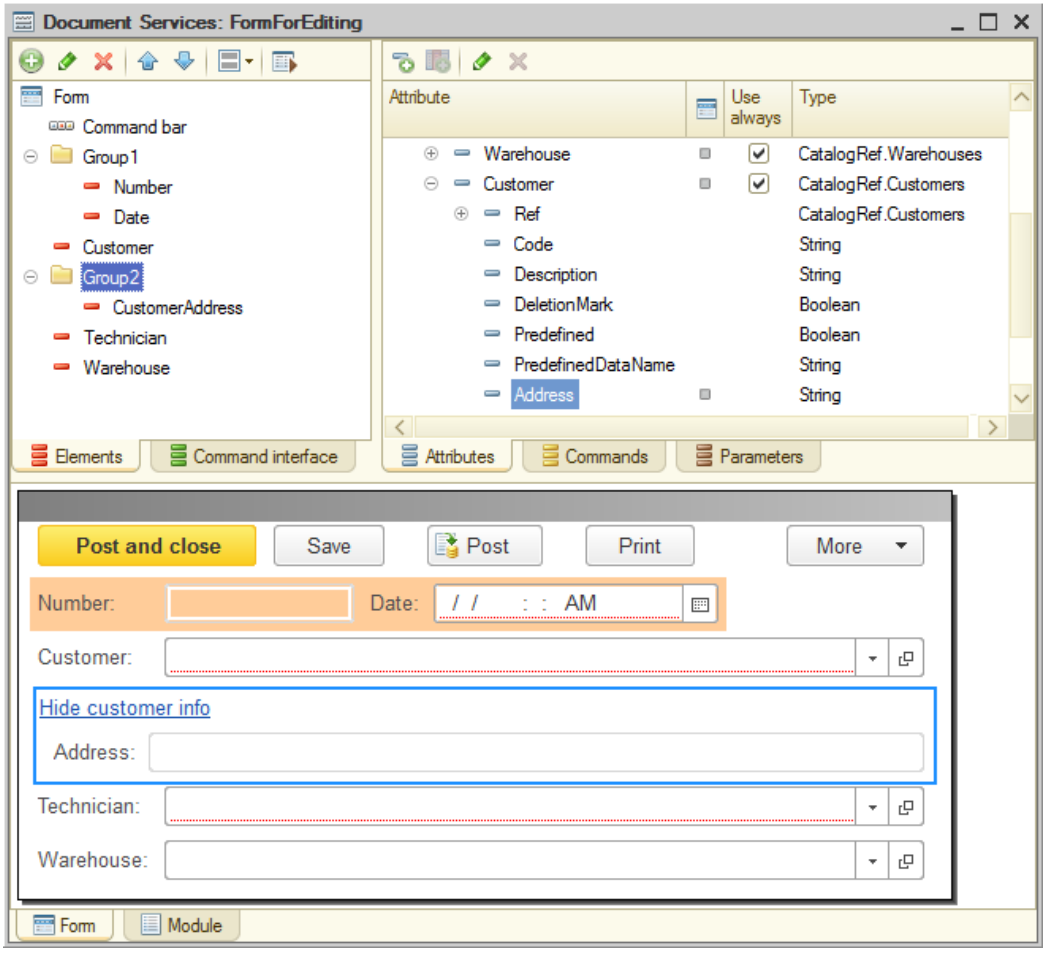


Fig. 28.17. Form editor window

In general, to make a group collapsible, you have to set its Behavior property to Collapsible and specify the expanded group title and/or the collapsed group title. You can specify a picture instead of title text if the ControlRepresentation property is set to Picture (its default value is Hyperlink link).

So you have added the option to expand or collapse customer info by clicking the group title in 1C:Enteprise mode.

Then let us group the Warehouse field with the command that opens the Materials report filtered by warehouse. You created this command in the "Using parameterized commands" section of the previous lesson.

20. In the form controls pane, click **Add**, select **Group - Regular group (no visual presentation)**, and click **OK**.
21. In the group property palette, in the **Group** list, select **Horizontal**.
22. Drag the **Warehouse** field and the **BalanceByWarehouse** global parameterized command to the group and move the group to the position below the **Technician** field.

    You can see the result of the changes in the preview pane (fig. 28.18).

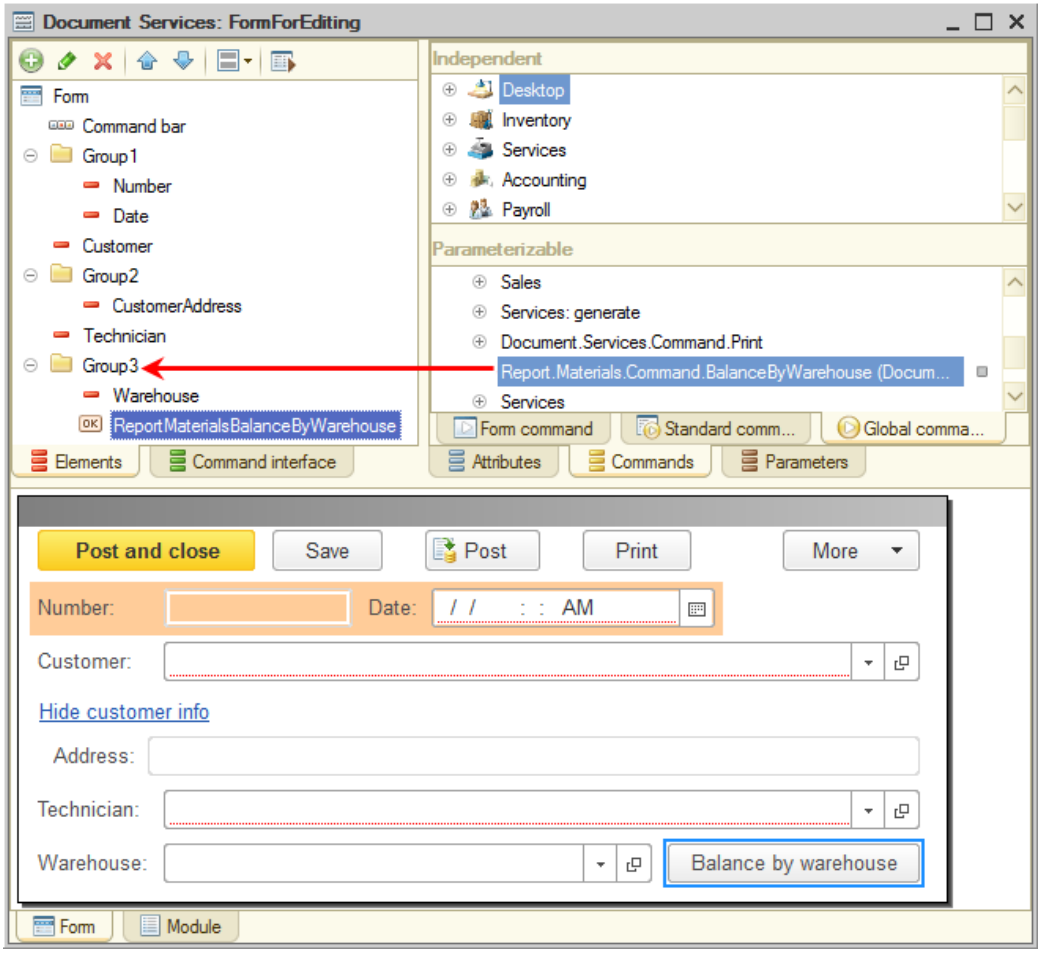


Fig. 28.18. Form editor window

The form is not yet finished but it is a good time to test it in 1C:Enterprise mode. To be able to test it, let us set the custom form as the default Services document form.

23. In the **Services** configuration object editor, on the **Forms** tab, in the **Document form** field, click the selection button and select **FormForEditing** (fig. 28.19).

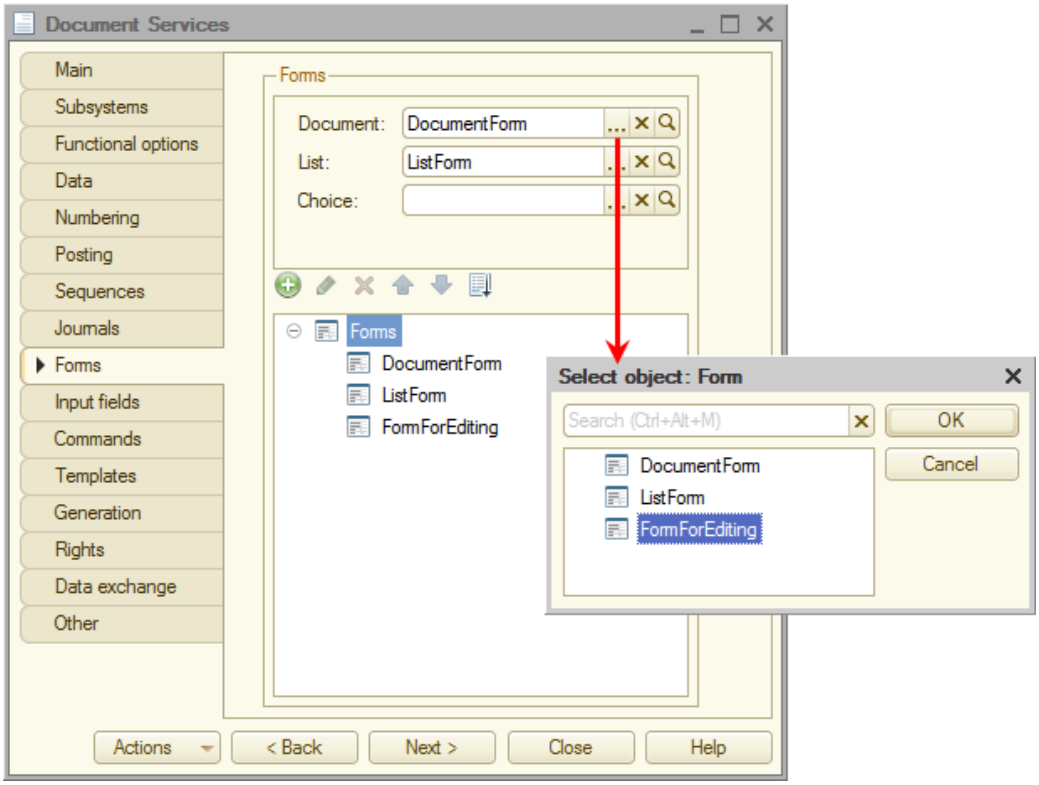


Fig. 28.19. Specifying default document form

In 1C:Enterprise mode

Let us test the form.

- Start 1C:Enterprise in the debug mode and open any of the Services documents.

  The document form looks exactly as planned (fig. 28.20).

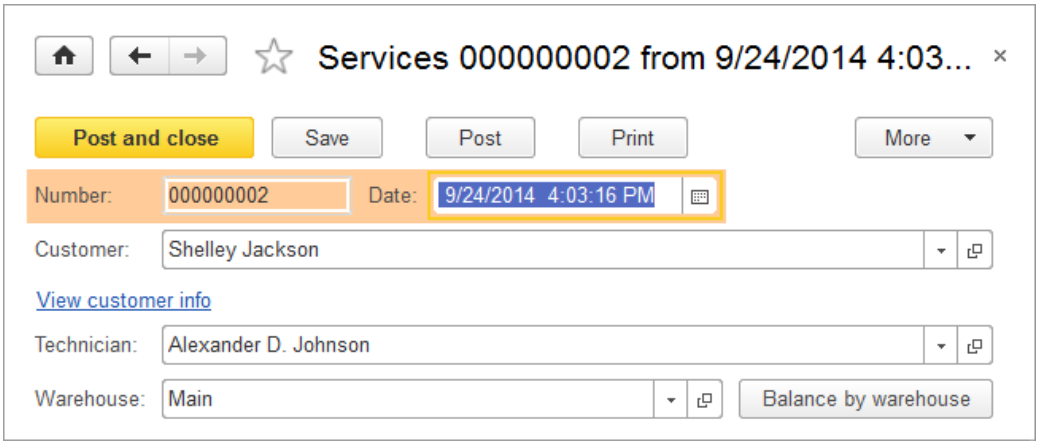


Fig. 28.20. Document form in 1C:Enterprise mode

However, the form is incomplete, it lacks the tabular section. Let us return to Designer, add the tabular section, and customize its view.

In Designer mode

Before you add the tabular section, let us split the form into two tabs. The first tab will display the major document attributes (Number, Data, Customer, and so on) and the second tab will display the table of services.

1. In the form controls pane, click **Add**, select **Group - Pages**, and click **OK** (fig. 28.21).

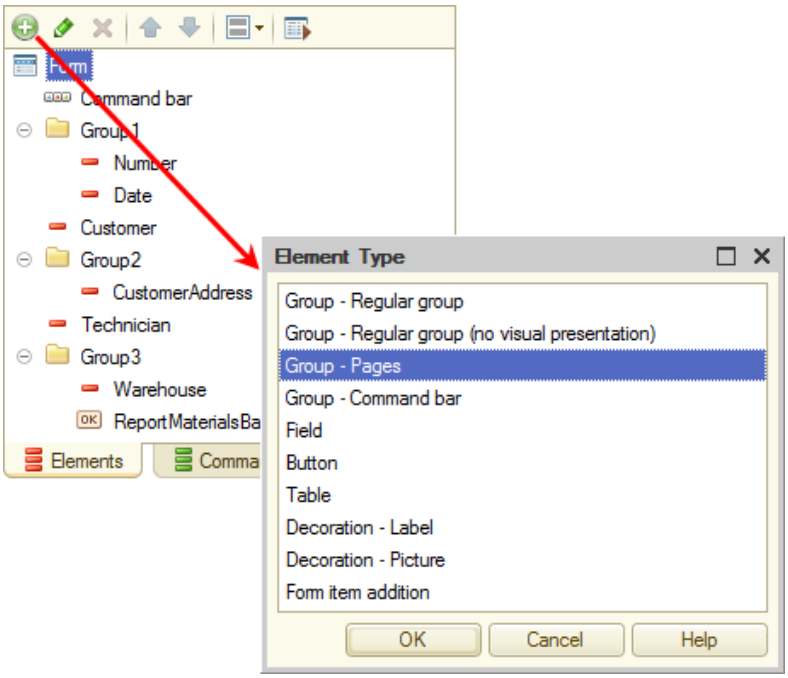


Fig. 28.21. Adding a page group to a form

2. In the group property palette, in the **Title** field, enter *Pane*.
3. In the form controls pane, click the **Pane** group and add another group of the **Pages** type to it (fig. 28.22).

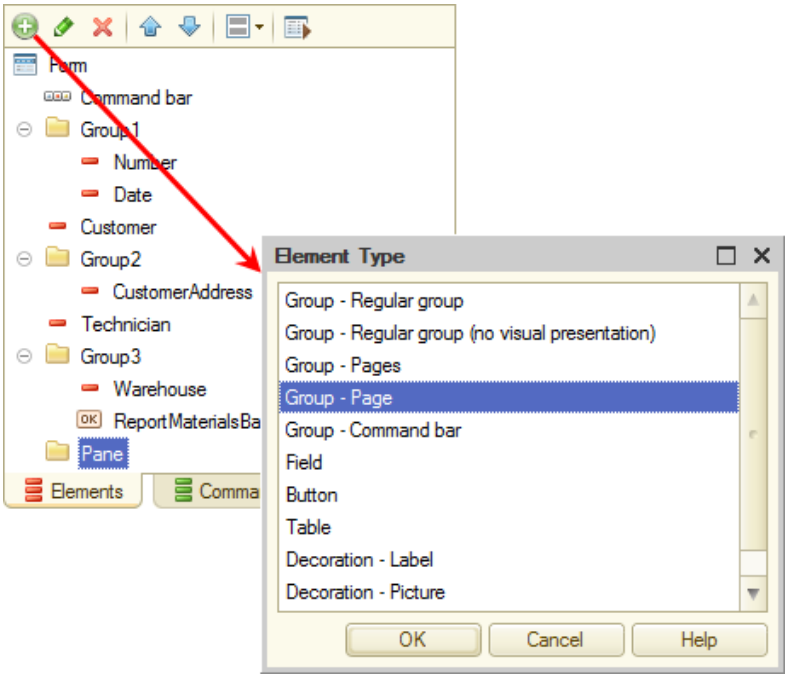


Fig. 28.22. Adding a page to a page group

4. In the group property palette, set the title to *Main*.
5. Add another group of the **Page** type with the *Services* title.

   Attention! When you click the **Add** button, the added control is subordinate to the selected one (in this example this adds a group subordinate to the selected group).
6. Drag all of the form controls (**Group1**, **Customer**, **Group2**, **Technician**, and **Group3**) to the **Main** group.

   The resulting form editor window should look as shown in fig. 28.23.

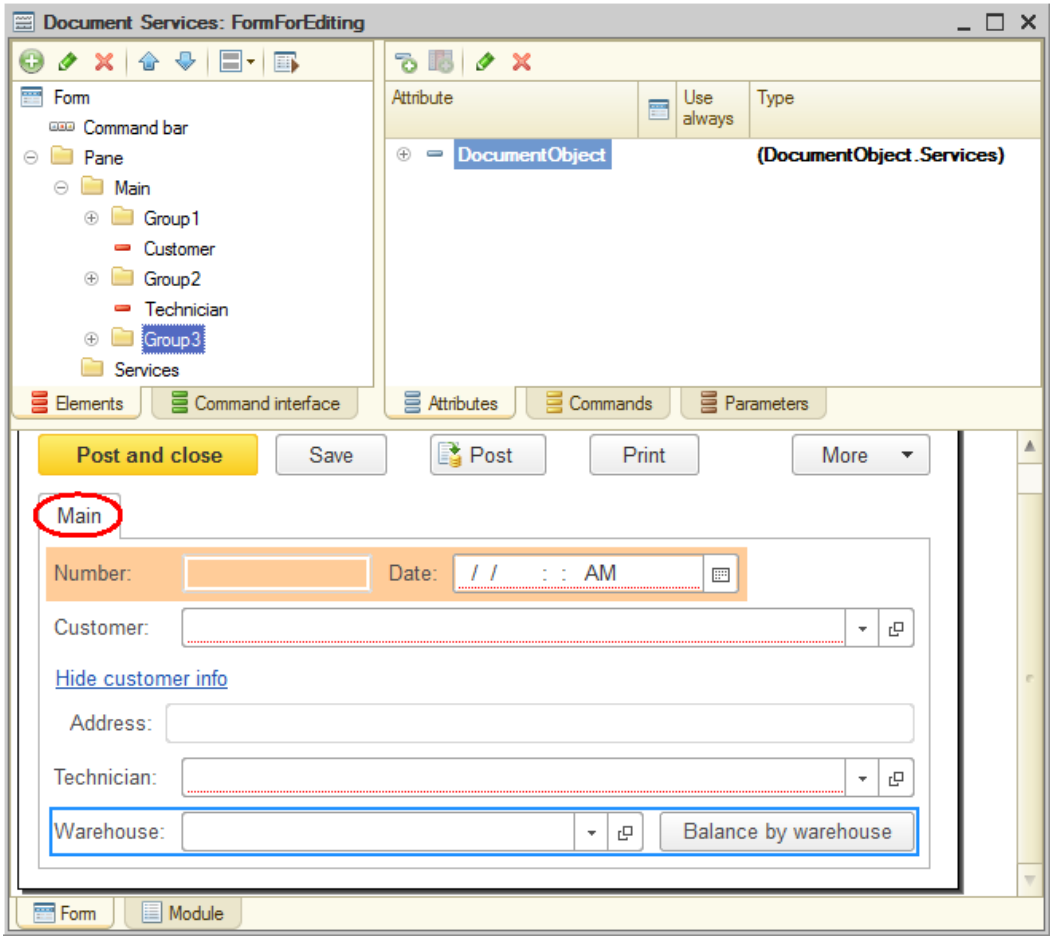


Fig. 28.23. Form editor window

The Services tab is not displayed because it does not contain any controls.

7. In the form attributes pane, expand the **DocumentObject** attribute and drag the **MaterialsAndServicestabular** section to the **Services** group.
8. Confirm that you want to add columns to the table.

   This adds the Services tab with a table that displays tabular section data to the form preview (fig. 28.4).

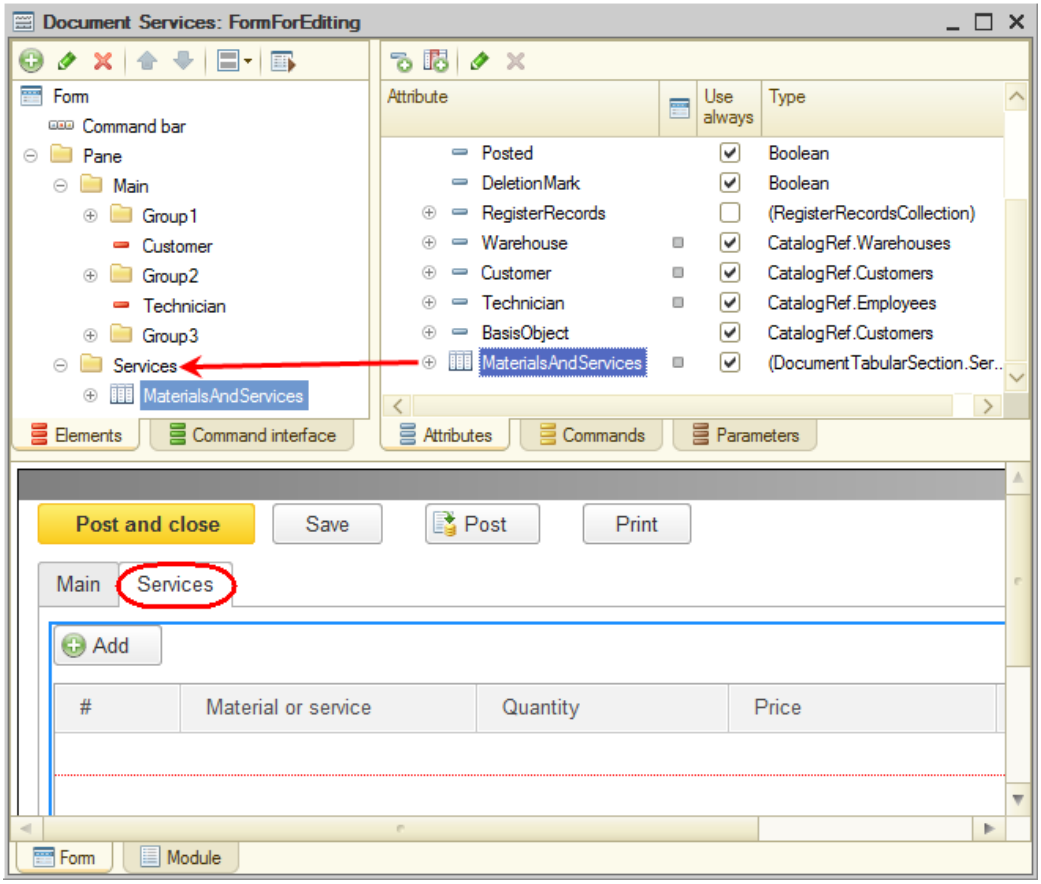


Fig. 28.24. Form editor window

Let us customize the table view. The PropertySet column is not important, so you can delete it.

9. In the form controls pane, expand the **MaterialsAndServices** table, click the **MaterialsAndServicesPropertySet** item, and click **Delete current item** (fig. 28.25).

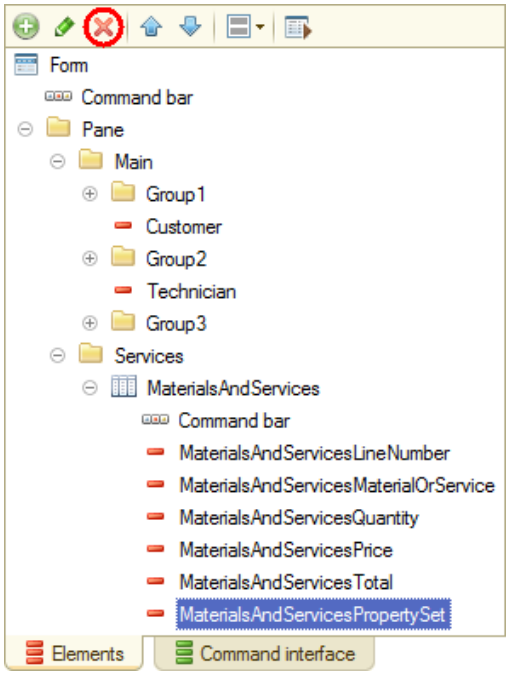


Fig. 28.25. Form controls pane

Then let us apply conditional formatting to the table by highlighting rows with total greater that 300 USD. You need the ConditionalAppearance form property for this.

10. In the form controls tree, double-click the root **Form** item.
11. In the property palette, in the **ConditionalAppearance** field, click the **Open** link (fig. 28.26).

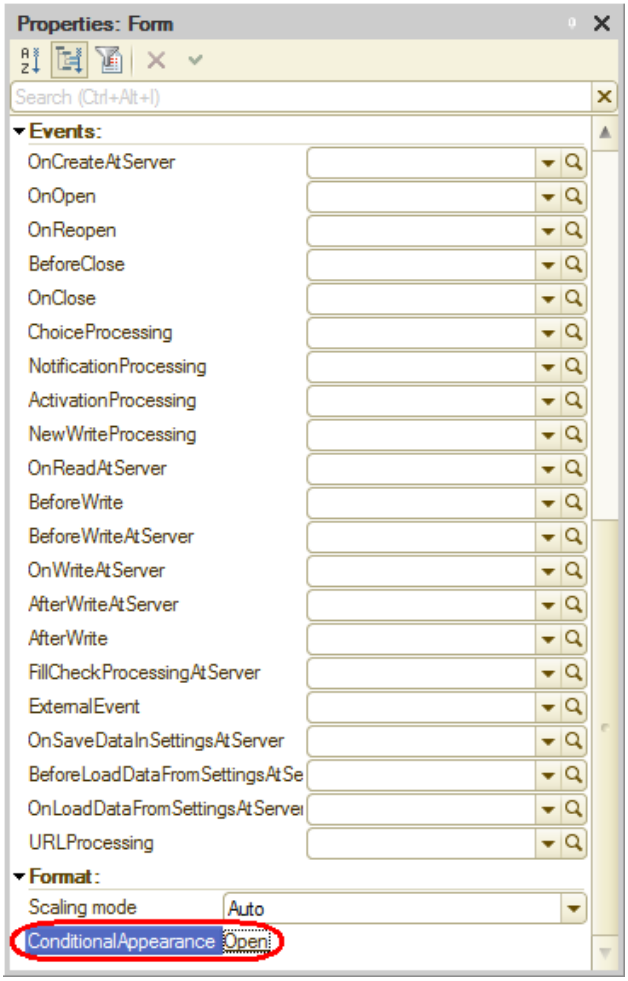


Fig. 28.26. Form property palette

12. In the **Conditional Appearance Settings** dialog box, click **Add** .

    First, let us specify the formatting for highlighting fields.
13. In the **Format** field, click the selection button and select light blue color for the background.

    Then let us specify the condition for the appearance to be applied (when list rows should have light blue color).
14. In the **Condition** field, click the selection button.
15. In the window that is opened, click the **Add** button and then click **New item** (fig. 28.27).
16. In the **Left value** column, select the **Total** tabular section field.
17. In the **Comparison type** column, select **Greater than**.
18. In the **Right value** column, enter *300* and click **OK**.

Finally, let us specify the list of formatted fields.

19. In the **Formatted fields** field, click the selection button.
20. In the **Formatted Fields** dialog box, click **Add**.

    Let us specify that the entire table row (all columns) is highlighted.
21. Select the **MaterialsAndServices** table and click **OK** (fig. 28.27).

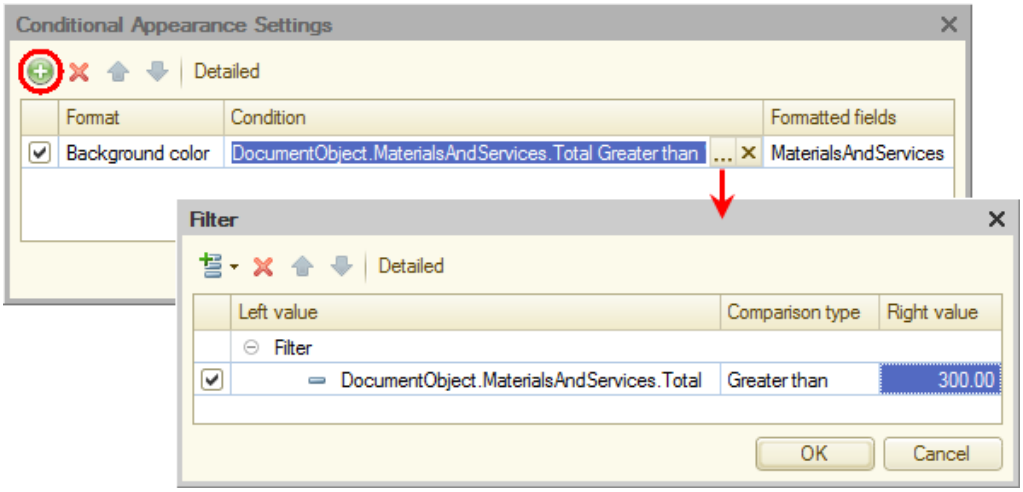


Fig. 28.27. Form conditional appearance settings window

You can specify conditional formatting for any form control (field, group, and so on) in a similar manner.

In 1C:Enterprise mode

Let us test the form.

1. Start 1C:Enterprise in the debug mode and open any of the Services documents.

   The document form looks exactly as planned.

   The Main tab displays major document attributes, the link that opens additional customer info, and the button that opens the Balance of materials report. The document number and date are displayed in a single row and highlighted with background color (fig. 28.28).

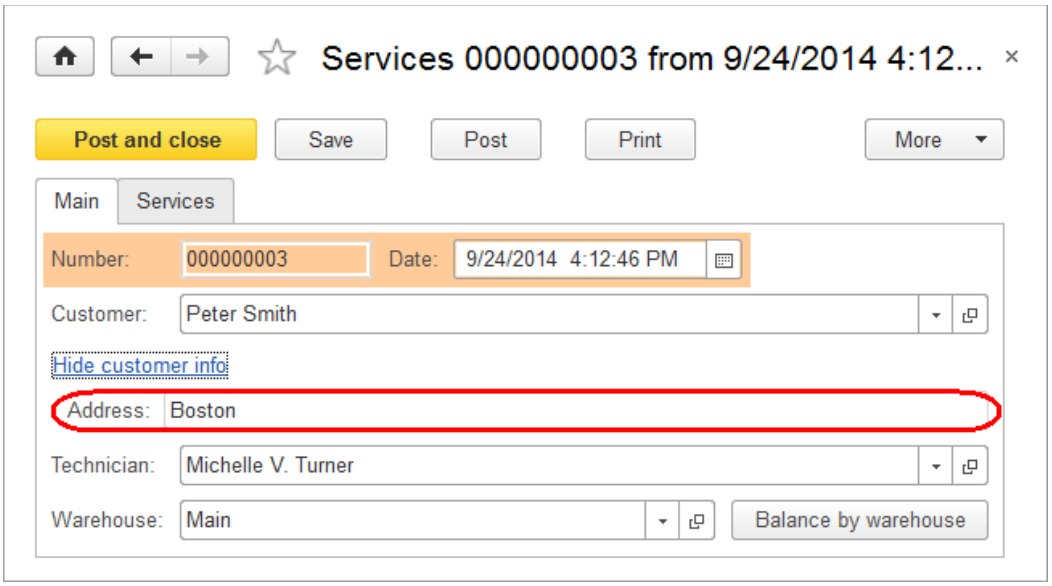


Fig. 28.28. Document form in 1C:Enterprise mode

2. Switch to the **Services** tab.

   It displays the document tabular section where rows with total greater than 300 are highlighted with light blue (fig. 28.29).

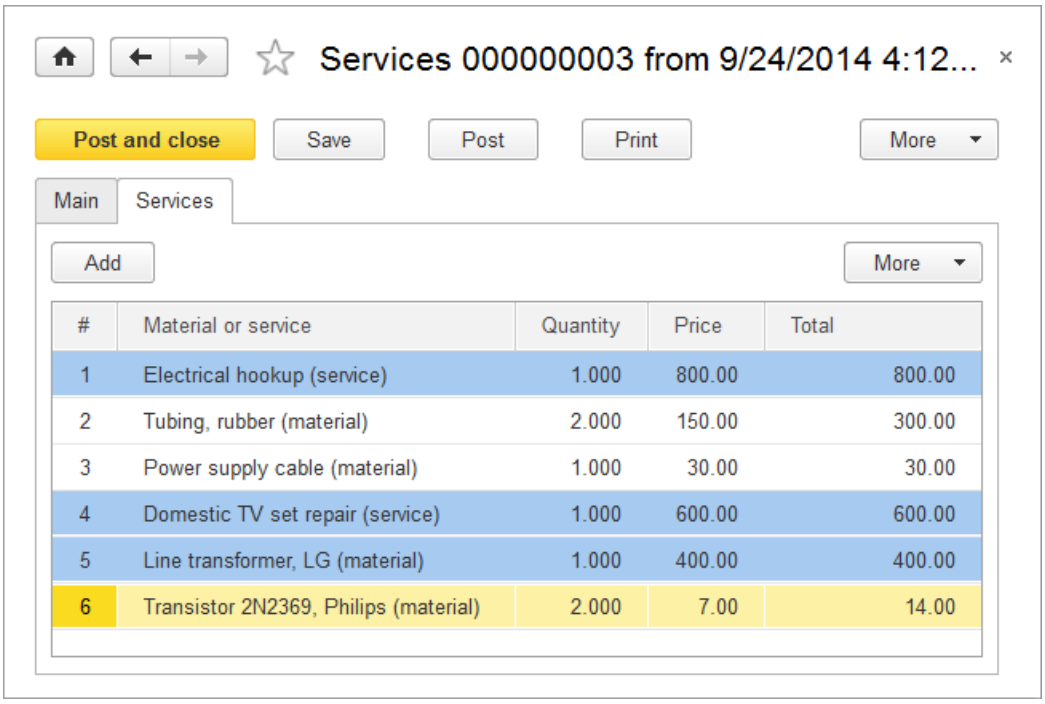


Fig. 28.29. Document form in 1C:Enterprise mode

The only thing that is missing is the total amount. It is logical to add it to the tab that displays the major document attributes.

In Designer mode

Let us return to Designer and add the total amount to both form tabs.

1. In the form controls pane, double-click the **MaterialsAndServices** table.
2. In the property palette, select the **Footer** check box (fig. 28.30).

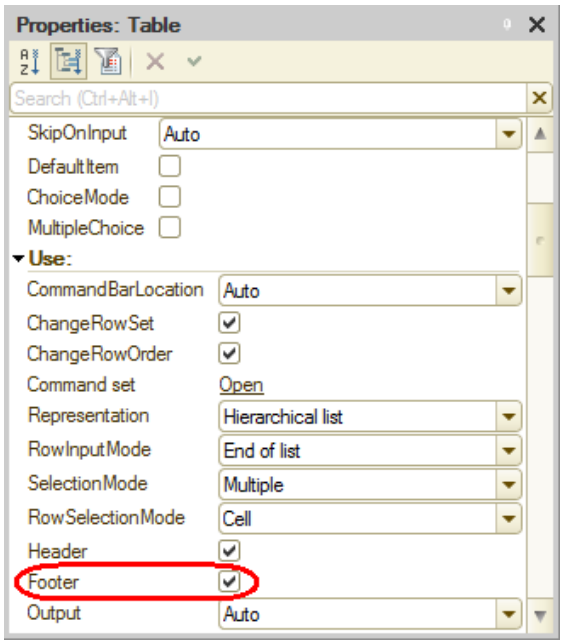


Fig. 28.30. Table property palette

3. Click the **MaterialsAndServicesTotal** table field.
4. In the property palette, in the **FooterDataPath** field, select **DocumentObject.MaterialsAndServices.TotalTotal** (fig. 28.31).

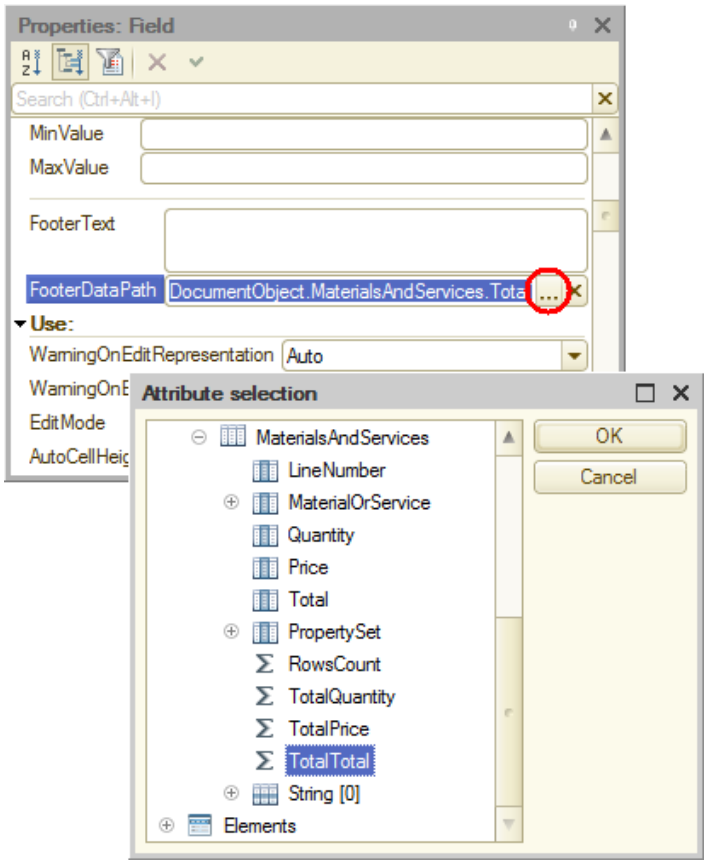


Fig. 28.31. Table field property palette

This adds the total by the Total field to the table footer. And now let us add it to the Main tab.

5. Drag the **TotalTotal** tabular section attribute from the form attributes pane to the **Main** group (fig. 28.32).

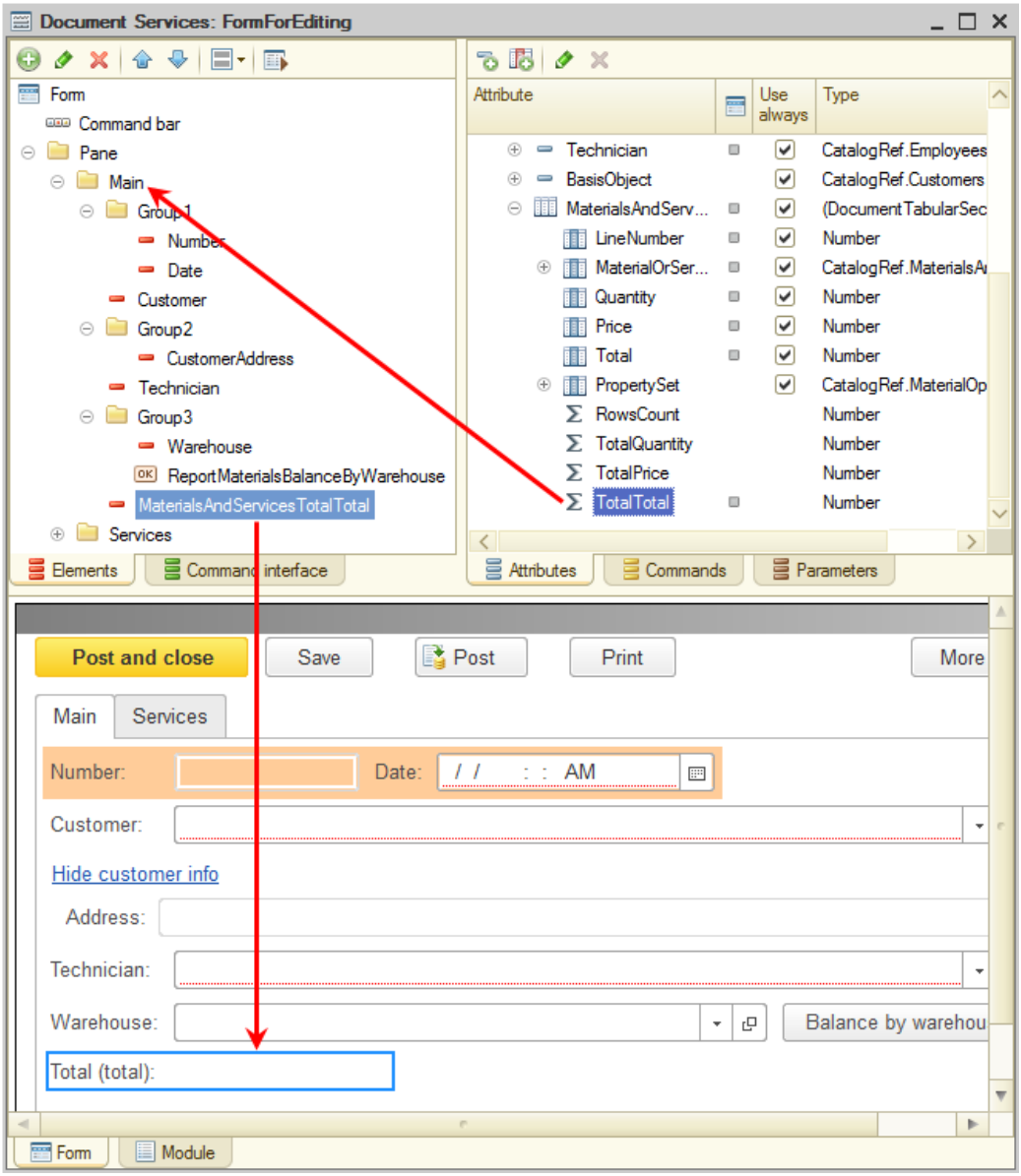


Fig. 28.32. Document form editor

6. In the form controls window, double-click the **MaterialsAndServicesTotalTotal** field.
7. In the property palette, in the **Font** and **TitleFont** fields, select **LargeTextFont** (fig. 28.33).

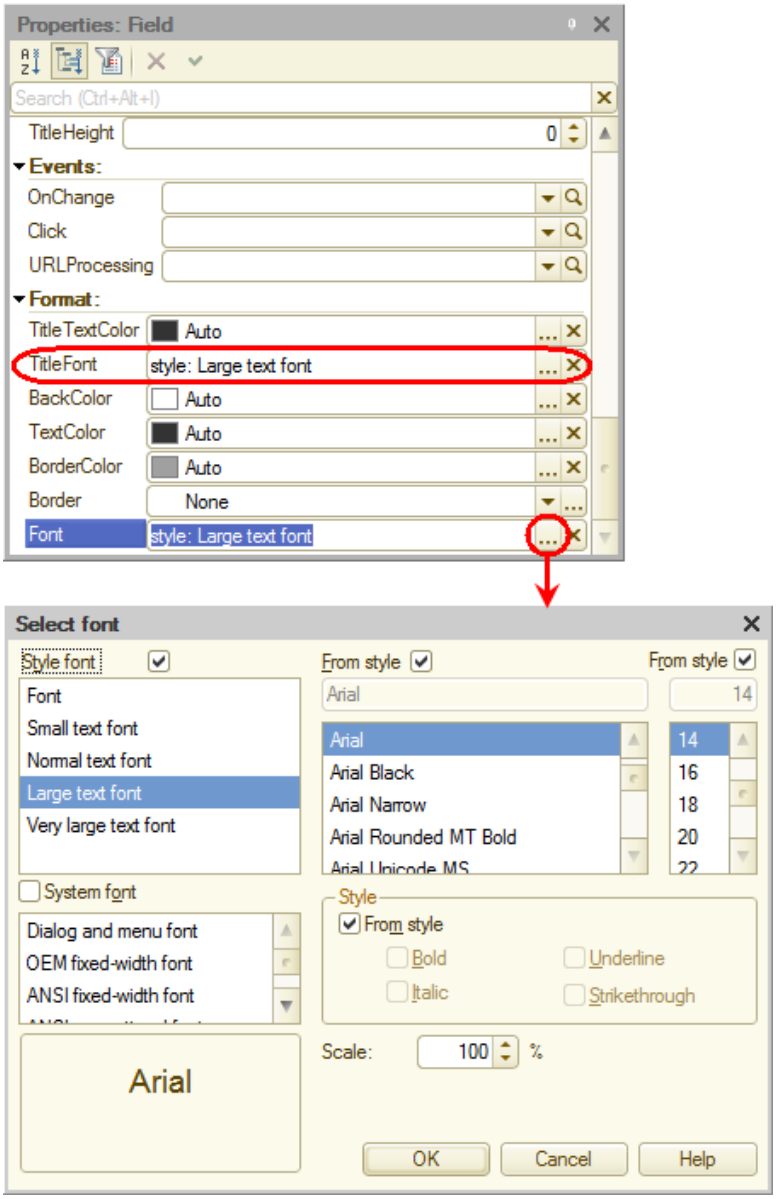


Fig. 28.33. Form field property palette

In 1C:Enterprise mode

Let us test the changes.

1. Start 1C:Enterprise in the debug mode and open any of the Services documents.

   You can see that the Main tab displays the document total in large font (fig. 28.34).

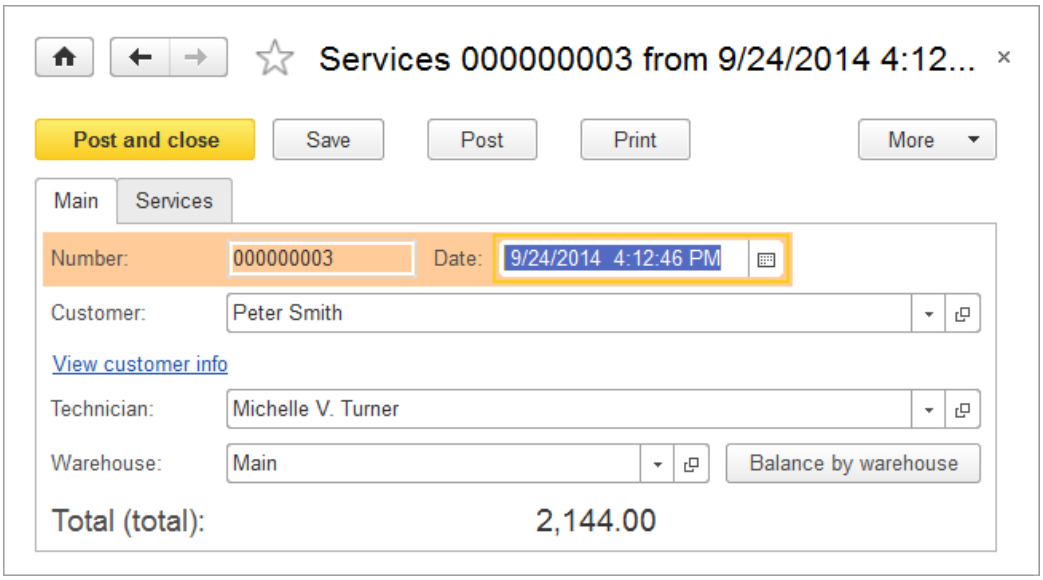


Fig. 28.34. Document form in 1C:Enterprise mode

2. Click the **Services** tab.

   The total amount is displayed at the bottom (fig. 28.35).

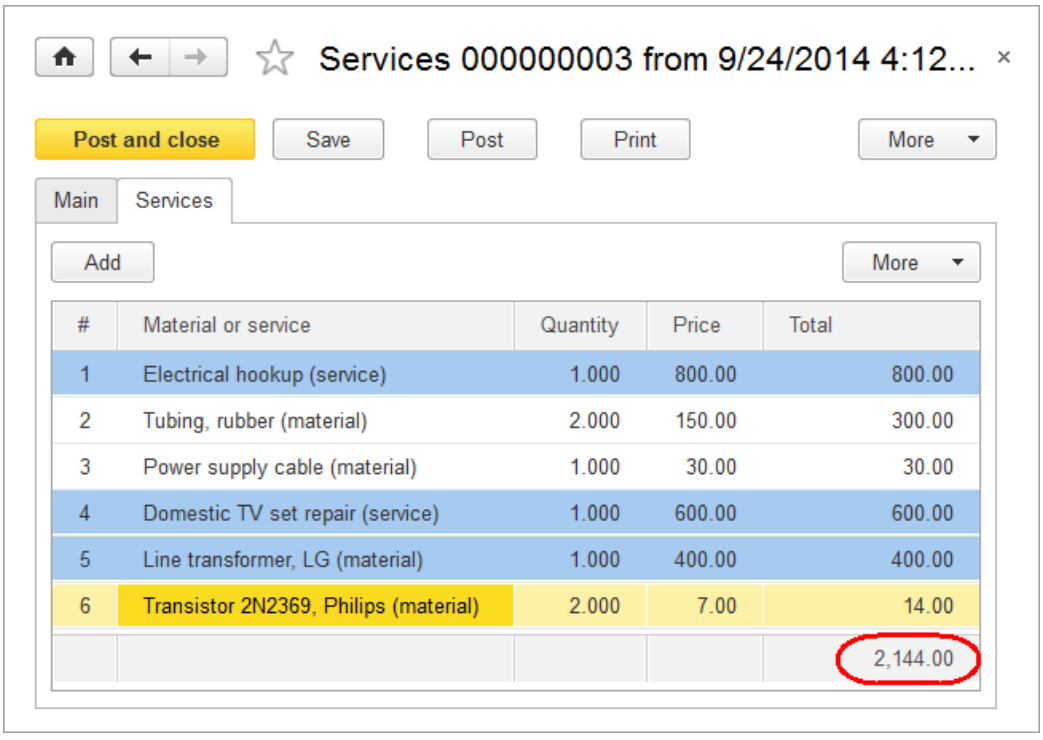


Fig. 28.35. Document form in 1C:Enterprise mode

This is the final form view. It is compact and user-frendly, and has a better look and feel than before.

Of course in commercial application development, in addition to your own form design concept, you have to follow customer requirements and user feedback.

You have completed the example that includes several development techniques aimed to improve form design and usability. In the

following sections we will examine a few more short examples of form customization.

Adding a tumbler switch to a form

An automatically generated material or service item form includes the MaterialServiceType field. Its values Material and Service are stored in the MaterialServiceTypes enumeration. The default field view is a text box where values are selected from a drop-down list.

If the number of enumeration items is small (2-3 items), you can display the field as a switch. A switch can have one of the following types: a set of radio buttons or a tumbler. The second option better suits modern application interfaces.

In Designer mode

Let us implement a switch.

1. Create a custom form for the **MaterialsAndServices** catalog item.
2. In the form controls tree, double-click the **MaterialServiceType** field.
3. In the property palette, in the **Type** field, select **Radio buttons**.
4. In the **RadioButtonType** field, select **Tumbler**.
5. In the **ChoiceList** field, provide a list that consists of **MaterialServiceTypes** enumeration values (fig. 28.36).

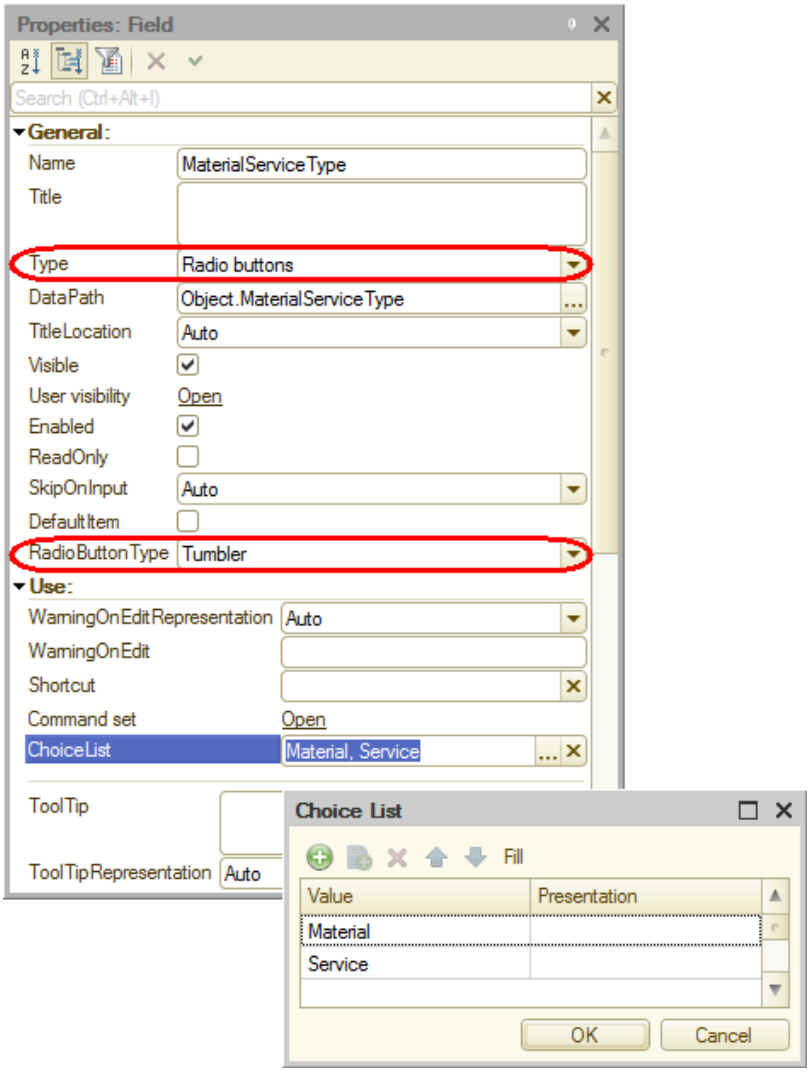


Fig. 28.36. Switch field property palette

Let us also make another change. The Parent field stores the group where a catalog item belongs. And to move an item to the root of the hierarchy, a user should clear this field.

However, the clear button is disabled by default. There are other ways to clear the field, such as using the mouse or pressing Delete, but some users really need this button. Let us enable it.

6. In the **Parent** field property palette, in the **ClearButton** field, select **Yes** (fig. 28.37).

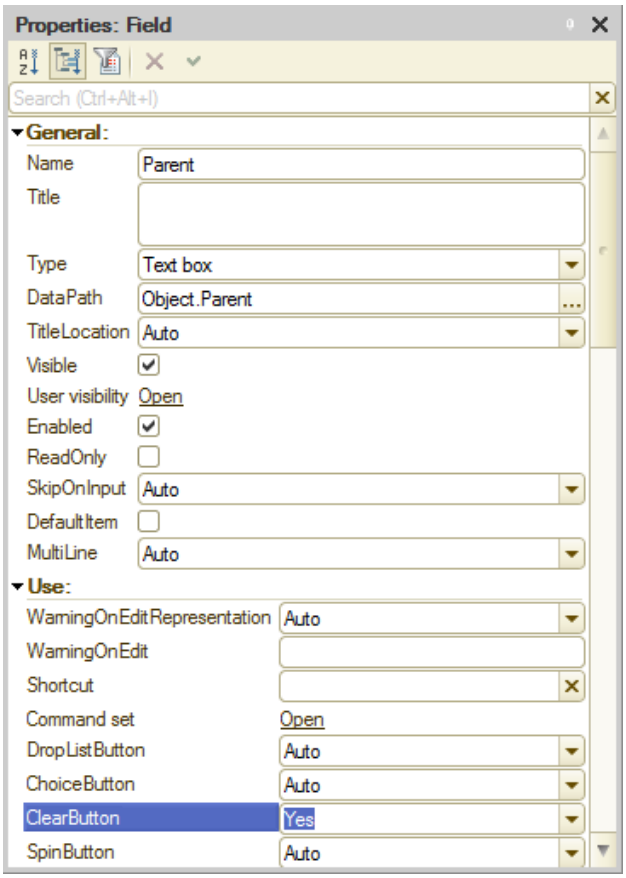


Fig. 28.37. Text box property palette

Note that text boxes have other properties that define button visibility: OpenButton, CreateButton, and more. The platform defines their visibility automatically but you can explicitly enable or disable them.

After the changes the form preview should look as shown in fig. 28.38.

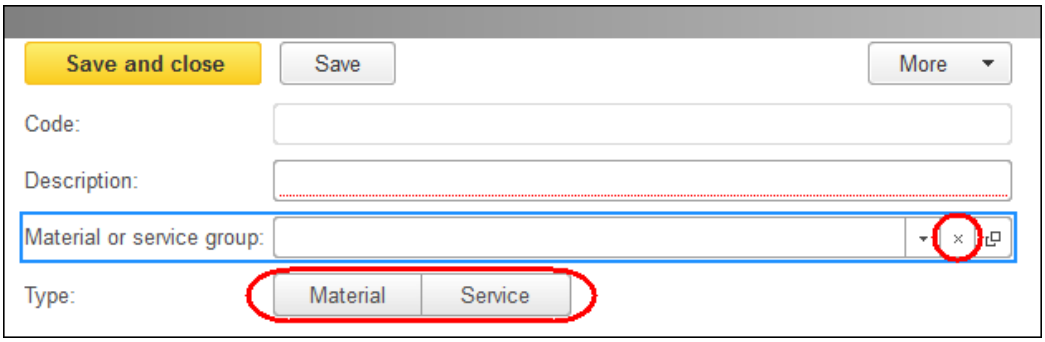


Fig. 28.38. Catalog item form preview in the form editor

In 1C:Enterprise mode

Let us test the changes.

- Start 1C:Enterprise in the debug mode and open any item of the **Materials and services** catalog.

You can see that a tumbler switch is used for selecting between material and service. The selected value is displayed in green. And the Material or service group field has the Clear button (fig. 28.39).

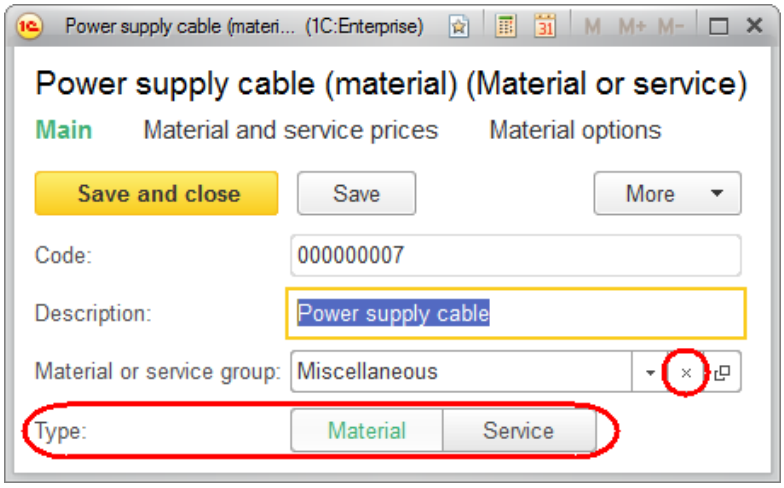


Fig. 28.39. MaterialsAndServices catalog item form in 1C:Enterprise mode

Grouping list data

Sometimes grouping list data by one or several field values significantly improves list usability. For example, you can group the list of Services documents by customer. This operation is available both in Designer and in 1C:Enterprise mode (on the **More** menu, click **Configure list**).

We recommend that you design list grouping as follows: if a grouping benefits all users, add it in Designer. List groupings added by users are only available to users who added them. Note that the same principle is applied to report customization.

In Designer mode

Let us specify the grouping of the Services document list in Designer.

1. Open the list form of the **Services** document and open the property palette of the main **List** attribute.
2. In the **List setup** field, click the **Open** link (fig. 28.40).

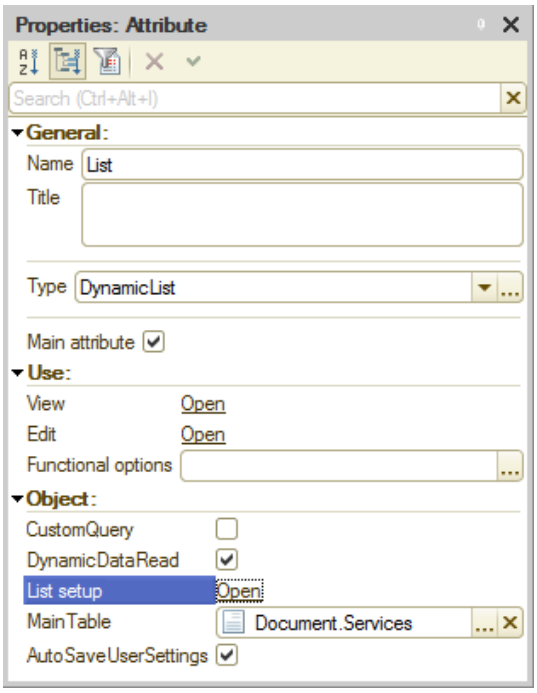


Fig. 28.40. Dynamic list property palette

3. In the **Dynamic list** dialog box, on the **Grouping** tab, drag the **Customer** field from the list of available fields to the list of grouping fields (fig. 28.41).

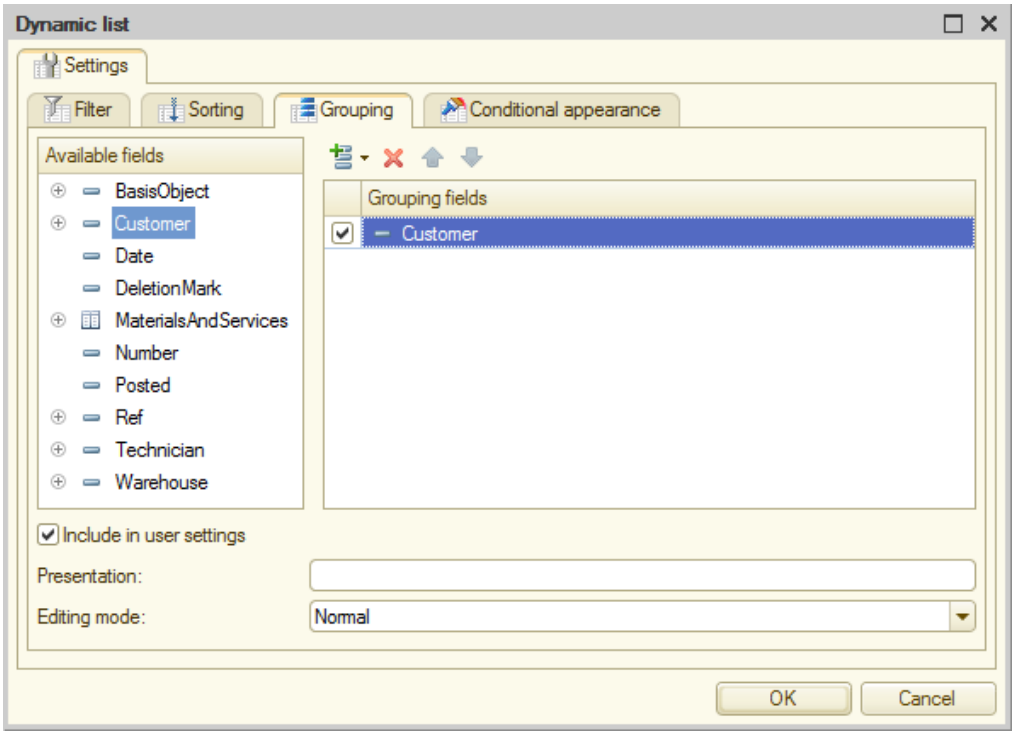


Fig. 28.41. Dynamic list grouping settings

This setting is also added to the list of user settings in 1C:Enterprise mode (to edit this setting, on the **More** menu, click **Configure list**).

In 1C:Enterprise mode

Let us test the changes.

- Start 1C:Enterprise in the debug mode and open the **Services** document list.

  You can see that the list is grouped by customer (fig. 28.42). You can change, save, or edit this setting using the More menu.

  Attention! This is how the list looks for new users. The list view settings for existing users were fixed earlier, when the list was not yet grouped. To view a grouped list while logged on as one of the existing users, edit the list settings in 1C:Enterprise mode.

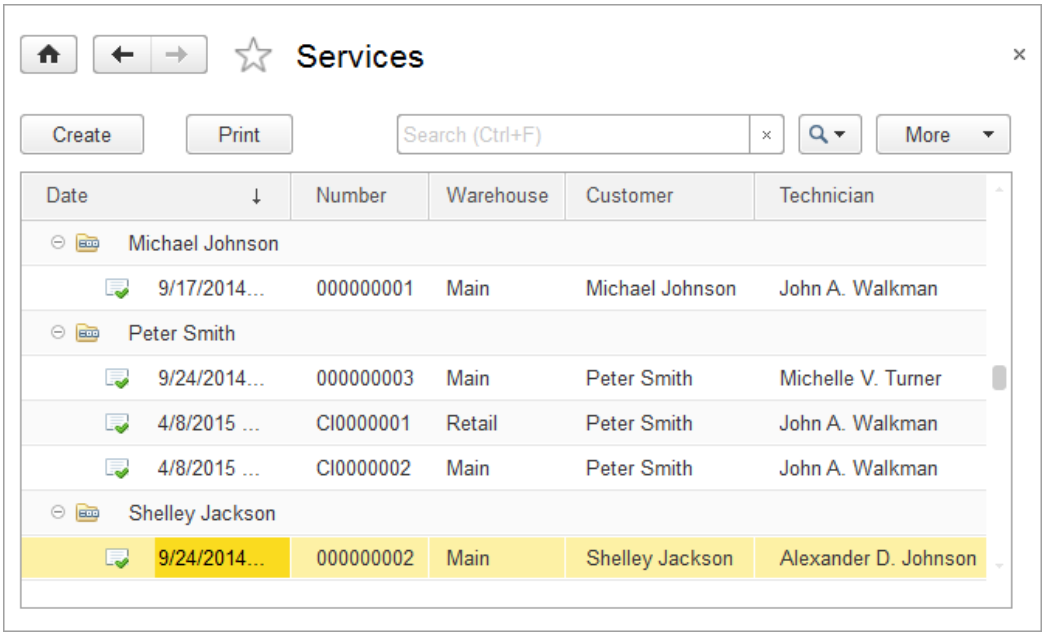


Fig. 28.42. Services document list in 1C:Enterprise mode

Quiz

- How does one add a main attribute to a form?
- How does one add a form control related to a specific attribute?
- How does one group form controls?
- How does one specify group properties and make a group collapsible?
- How does one add a pane with tabs to a form?
- How does one add a tabular section to a form and apply conditional formatting to its rows?
- How does one display a total for a tabular section column?
- How does one add a tumbler switch to a form?
- How does one show or hide text box buttons?
- How does one group list data?

Quick developer reference

1C:Enterprise script objects used for operations with applied data

1C:Enterprise script includes a set of standardized objects for accessing data stored in the database. They can be broken down into several types based on their purposes.

1. Manager of data structures of specific type. Examples:

- CatalogsManager
- DocumentsManager
- ReportsManager
- ChartsOfAccountsManager

Each object is a value collection, which contains managers of all the data structures of this type available in the database.

For example, the manager of catalogs (CatalogsManager) is a collection of values that contains CatalogManager.<name> objects. Each of these objects is used to access a specific data structure manager.

2. Manager of a specific data structure. Examples:

- CatalogManager.Customers
- CatalogManager.MaterialsAndServices
- DocumentManager.GoodsReceipt
- DocumentManager.Services

Each object provides management tools for a specific data structure.

For example, the manager of the GoodsReceipt document (DocumentManager.GoodsReceipt) is used to find specific documents of GoodsReceipt type, create objects of such documents, and so on.

3. Object. Examples:

- CatalogObject.Customers
- CatalogObject.MaterialsAndServices
- DocumentObject.GoodsReceipt
- DocumentObject.Services

Such objects are used to read, edit, write, and delete data within data structures. They provide access to data structure objects and enable modifications of data in the database. They are applicable to data structures storing objects that can be referenced (CatalogObject.<name> for catalogs, DocumentObject.<name> for documents, and so on).

4. Record set. Examples:

- InformationRegisterRecordSet.Prices
- AccumulationRegisterRecordSet.BalanceOfMaterials
- AccountingRegisterRecordSet.Primary

Such objects are also used to read, edit, write, and delete data within data structures. They provide access to data structure objects and enable modifications of data in the database. They are applicable to data structures storing objects that cannot be referenced (AccumulationRegisterRecordSet.<name> for registers, RecalculationRecordSet.<name> for recalculations, and so on).

5. Reference. Examples:

- CatalogRef.Customers
- CatalogRef.MaterialsAndServices
- DocumentRef.GoodsReceipt
- DocumentRef.Services

Such objects are used to specify a reference to a database object and also provide a certain amount of data related to this object (DocumentRef.<name> for documents, and so on).

6. Selection. Examples:

- CatalogSelection.Customers
- DocumentSelection.GoodsReceipt
- InformationRegisterSelection.Prices
- AccumulationRegisterSelection.BalanceOfMaterials

Such objects represent data sets that contain object data selected from a single data structure according to a specific criterion (CatalogSelection.<name> for catalogs, and so on). A selection can be iterated using the Next() method. Data is read from the database dynamically, as iteration proceeds. You can get a reference to such object using the Ref property, and you can get the object itself using the GetObject method.

Manipulating object data

Despite the wide variety of 1C:Enterprise script objects used to process data stored in the database, only a small number of objects can be used to edit that data. We will refer to such objects as *data manipulation objects*.

Each data manipulation object type has a respective module in the configuration. It is referred to as an *object module* or a *record set module*, depending on the configuration object it belongs to. For a constant this module is referred to as a *value manager module*.

A module of a data manipulation object is always executed when a data manipulation object is created. It is also executed every time a user interactively accesses the data structure because this initiates the creation

of the corresponding data manipulation object. For example, when a catalog item form is opened, the CatalogObject.<name> object is created.

A data manipulation object module can include procedure definitions with the Export keyword, which implies that the procedures are called as methods of the corresponding data manipulation object. Here it is important not to confuse the data manipulation object with other objects that provide access to data within the data structure.

For example, if you create a procedure (listing 29.1) in the object module of the Customers catalog configuration object, in the future you will be able to call this procedure as a method of the CatalogObject.Customers object (listing 29.2).

Listing 29.1. Check() procedure in a catalog module

```
Procedure Check() Export
…
EndProcedure;
```

Listing 29.2. Calling the procedure as a method of the Customers object

```
Customer = Catalogs.Customers.FindByCode(1).GetObject();
Customer.Check();
```

However, the following script would generate an error, since the CatalogRef.Customers object does not have a Check method (listing 29.3).

Listing 29.3. The Check procedure call that generates an error

```
Customer = Catalogs.Customers.FindByCode(1);
Customer.Check();
```

The following table lists available data manipulation objects. As always, for every rule there is an exception, and there are two exceptions here as well.

Table 29.1. Objects used for data manipulation

CONFIGURATION OBJECT	DATA MANIPULATION STRUCTURE IN THE DATABASE	DATA MANIPULATION STRUCTURE IN 1C:ENTERPRISE SCRIPT
Constant	Constant	ConstantValueManager.<name> (ConstantManager.<name>, ConstantsSet)
Catalog	Catalog item	CatalogObject.<name>
Document	Document	DocumentObject.<name>
Sequence	Sequence record set	SequenceRecordSet.<name>
ChartOfCharacteristic Types	Characteristic type	ChartOfCharacteristicTypesObject.<name>
ChartOfAccounts	Account	ChartOfAccountsObject.<name>

CONFIGURATION OBJECT	DATA MANIPULATION STRUCTURE IN THE DATABASE	DATA MANIPULATION STRUCTURE IN 1C:ENTERPRISE SCRIPT
ChartOfCalculation Types	Calculation type	ChartOfCalculationTypesObject.<name>
InformationRegister	Information register record set	InformationRegisterRecordSet.<name> (InformationRegisterRecordManager.<name>)
AccumulationRegister	Accumulation register record set	AccumulationRegisterRecordSet.<name>
AccountingRegister	Accounting register record set	AccountingRegisterRecordSet.<name>
CalculationRegister	Calculation register record set	CalculationRegisterRecordSet.<name>

First, there are three data manipulation objects listed for constants: ConstantValueManager.<name>, ConstantManager.<name>, and ConstantsSet. In practice, the data stored in constants is manipulated using the ConstantValueManager.<name> object.

The other two objects (ConstantManager.<name> and ConstantsSet) are also used for editing values of constants stored in the database. However, when their Set() and Write() methods are executed, they initiate the creation of a ConstantValueManager.<name> object, which modifies the data.

When the Set() method of the ConstantManager.<name> object is executed, the value manager module of the constant is called, as well as its OnWrite() and BeforeWrite() event handlers. When the Write() method of the ConstantsSet object is executed, the value manager module and the appropriate handlers are called for each constant in the set.

Second, there are two data manipulation objects listed for information registers. In practice, the data stored in information registers is manipulated using the InformationRegisterRecordSet.<name> object.

The InformationRegisterRecordManager.<name> object is also used for manipulating information register data. But the InformationRegisterRecordManager.<name> object does not manipulate the data directly: instead it uses the InformationRegisterRecordSet.<name> object.

So, calling the InformationRegisterRecordManager.<name> object executes the record set module, along with the BeforeWrite() and OnWrite() event handlers. However, the procedures and functions that are defined in the record set module with the Export keyword are not available as methods of the InformationRegisterRecordManager.<name> object.

Constants

1C:Enterprise script objects used for operations with constants

The following chart shows the interaction between 1C:Enterprise script objects used for operations with constants (fig. 29.1).

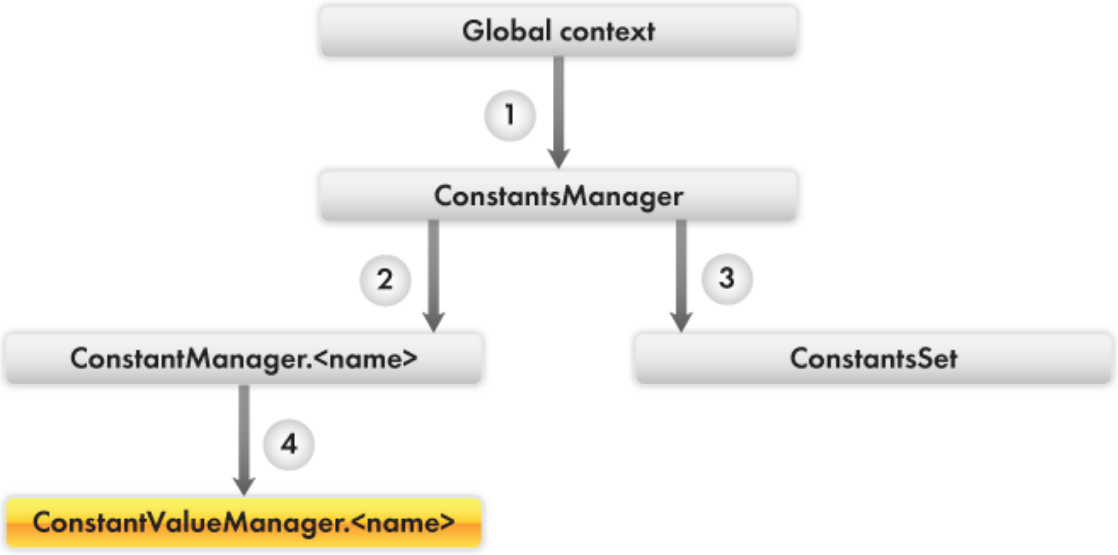


Fig. 29.1. 1C:Enterprise script objects used for operations with constants

Note. The yellow box indicates the data manipulation object. The object method from which the arrow originates is marked with the respective number in the listing, while the target object of the arrow is the type of returned object.

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

ConstantsSet is used to perform read and write operations on a set of constants (which can include all available constants). It is also used in constant forms to store, read, and write constants.

ConstantValueManager.<name> is used to access constants. Writing a constant (interactively in a form, or using ConstantsSet, or using ConstantManager.<name>) creates an object of this type and calls the module of that object and its event handlers.

For operations with constants that involve 1C:Enterprise script objects, see listing 29.4.

Listing 29.4. Object usage examples

```
1.  // Global context
    // Constants


// Example: assigning a value to a constant.
```

```
Constants.Accountant.Set("Patrick Henry Brewster");

2.  // ConstantsManager object
    // .<constant name>
    // [<constant name>]
    // For Each … In … Do … EndDo;

// Example: reading a constant value.
Result = Constants.NumberingPrefix.Get();
Message("NumberingPrefix constant value = "+ Result + ".");

// Example: setting NumberingPrefix constant value to CI.
Constants["NumberingPrefix"].Set("CI");
Message("New value = " + Constants["NumberingPrefix"].Get() + ".");

// Example: clearing values of all constants.
For Each NextConstant In Constants Do
    NextConstant.Set(Undefined);
EndDo;

3.  // ConstantsManager object
    // CreateSet()

// Example: assigning new values to several constants.
Set = Constants.CreateSet("CEO, Accountant");
Set.CEO = "Thaddeus Addison";
Set.Accountant = "Alexandra Swain";
Set.Write();

4.  // ConstantsManager.<name> object
    // CreateValueManager()

// Example: displaying values of all constants available in
// the configuration.
For Each NextConstant In Constants Do
    ConstName = NextConstant.CreateValueManager().Metadata().Name;
    ConstValue = NextConstant.Get();
    Message("Constant "+ ConstName + " = " + ConstValue + ".");
EndDo;
```

Event sequence for writing constants from constant forms (save and close)

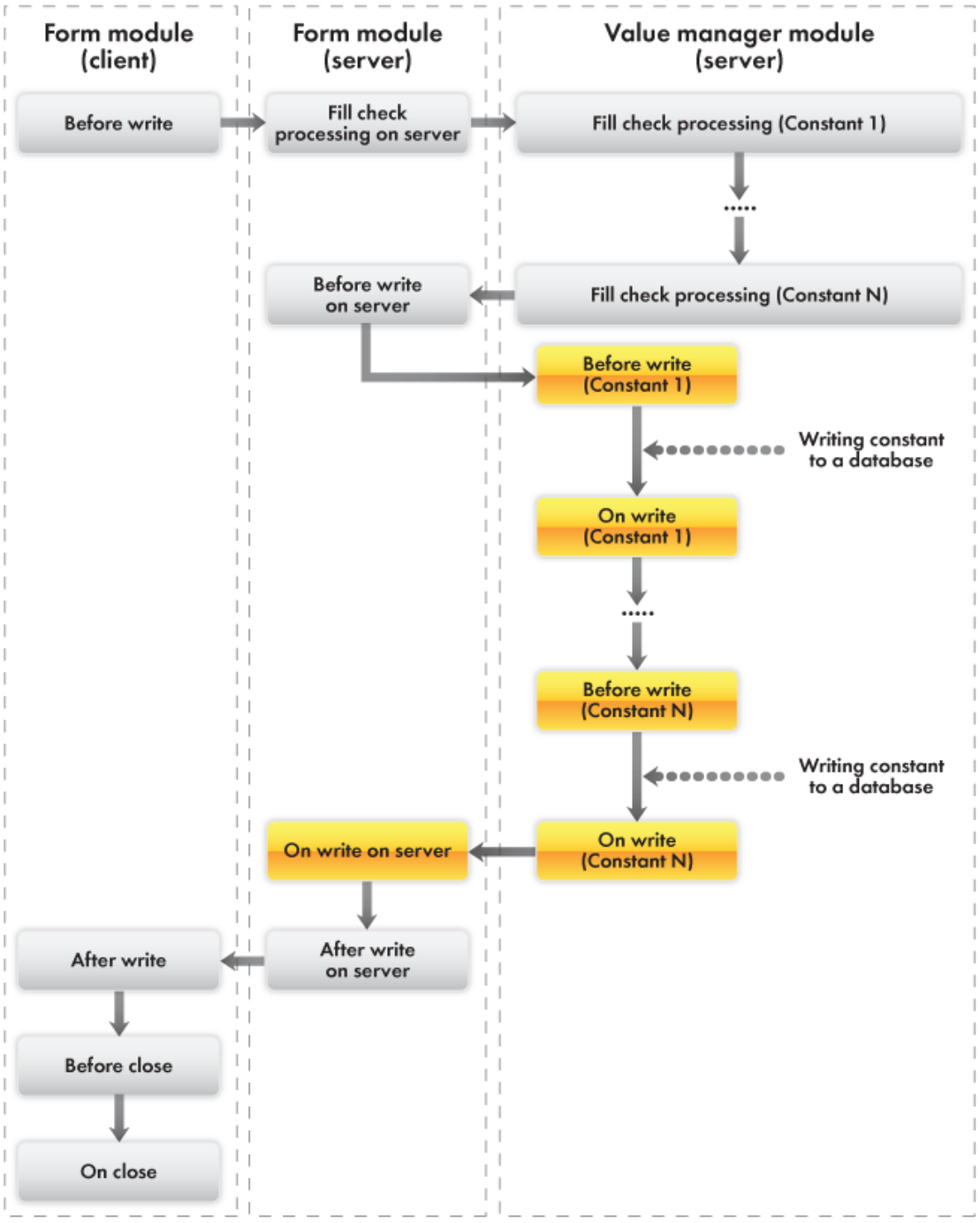


Fig. 29.2. Event sequence for writing constants from a constant form

Note. Yellow boxes indicate events executed in the writing transaction.

Constant forms are accessed using the ConstantsSet object, which in turn uses the ConstantValueManager.<name> object.

The internal implementation of the ConstantsSet object defines that when a set of constants is being written the FillCheckProcessing(), BeforeWrite(), and OnWrite() event handlers from the constant value manager module are called for each constant in the set.

Catalogs

1C:Enterprise script objects used for operations with catalogs

The following chart shows the interaction between 1C:Enterprise script objects used for operations with catalogs (fig. 29.3).

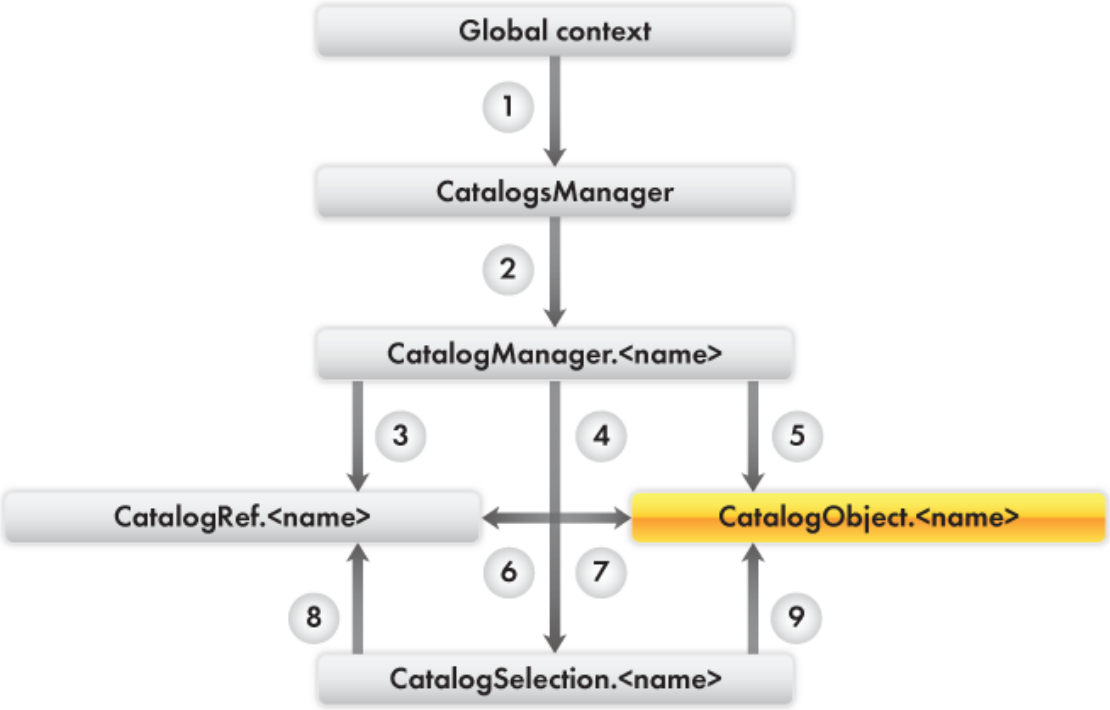


Fig. 29.3. 1C:Enterprise script objects used for operations with catalogs

Note. The yellow box indicates the data manipulation object. The object method from which the arrow originates is marked with the respective number in the listing (for example, number 3 stands for the FindByCode() method of the CatalogManager.<name> object), while the target object of the arrow is the type of returned object (for example, CatalogRef.<name>).

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

For examples of operations with catalogs that involve 1C:Enterprise script objects, see listing 29.5.

Listing 29.5. Object usage examples

```
1.  // Global context
    // Catalogs


// Example: displaying all types of references to catalog items available
// in the configuration.
Array = Catalogs.AllRefsType().Types();
```

```
For Each NextType In Array Do
    Message(NextType);
EndDo;

2.  // CatalogsManager object
    // .<catalog name>
    // [<catalog name>]
    // For Each … In … Do … EndDo;

// Example: creating a folder in the MaterialsAndServices catalog.
NewFolder = Catalogs.MaterialsAndServices.CreateFolder();
NewFolder.Description = "My new folder";
NewFolder.Write();

// Example: getting a reference to the MaterialsAndServices catalog.
Catalogs["MaterialsAndServices"].GetRef();

3.  // CatalogsManager.<catalog name> object
    // FindByCode()
    // FindByDescription()
    // FindByAttribute()
    // EmptyRef()
    // GetRef()
    // .<predefined catalog item name>

// Example: checking whether a MaterialsAndServices catalog item with code 13
// is marked for deletion.
If Catalogs.MaterialsAndServices.FindByCode(13).DeletionMark Then
    Message("Item with code 13 is marked for deletion.");
EndIf;

// Example: checking whether a MaterialsAndServices catalog item
// named "Services" is a folder.
If Catalogs.MaterialsAndServices.FindByDescription("Services", True).IsFolder
Then
    Message("The item named Services is a folder.");
EndIf;

// Example: checking whether all catalog items have a type (material
// or service).
EmptyEnumRef = Enums.MaterialServiceTypes.EmptyRef();
If Not Catalogs.MaterialsAndServices.FindByAttribute("ProductType",
    EmptyEnumRef).IsEmpty() Then
    Message("Items with no type assigned are found.");
EndIf;
```

```
// Example: passing an empty reference to a method parameter.
Selection = Catalogs.MaterialsAndServices.Select(
    Catalogs.MaterialsAndServices.EmptyRef());


4.  // CatalogManager.<catalog name> object
    // Select()
    // SelectHierarchically()


// Example: displaying the list of items located in the root folder of
// the catalog.
Selection = Catalogs.MaterialsAndServices.Select(
    Catalogs.MaterialsAndServices.EmptyRef());
While Selection.Next() Do
    If Not Selection.IsFolder Then Message(Selection);
    EndIf;
EndDo;


// Example: deleting all items from a hierarchical catalog.
Selection = Catalogs.MaterialsAndServices.SelectHierarchically();
While Selection.Next() Do
    Selection.Delete();
EndDo;


5.  // CatalogManager.<catalog name> object
    // CreateFolder()
    // CreateItem()


// Example: creating an item in the Employees catalog.
NewItem = Catalogs.Employees.CreateItem();
NewItem.Description = "Andrew A. Smith";
// Filling the PastEmployment tabular section.
NewTableRow = NewItem.PastEmployment.Add();
NewTableRow.Company = "NTC Corp.";
NewTableRow.StartDate = Date(2013,05,01);
NewTableRow.EndDate = Date(2013,12,31);
NewTableRow.JobTitle = "Programmer";
NewItem.Write();


6.  // CatalogObject.<catalog name> and CatalogRef.<catalog name> objects
    // Owner
    // Parent
    // Ref


// Example: prohibiting changes to subordinate items in the catalog item
```

```
// form module if the ProhibitChanges property of the owner item is True.
Procedure BeforeWrite(Cancel)
    If Owner.ProhibitChanges Then Cancel = True;
    EndIf;
EndProcedure

7.  // CatalogRef.<catalog name>
    // GetObject()
    // CatalogObject.<catalog name>
    // Copy()

// Example: changing catalog item name.
Item = Catalogs.MaterialsAndServices.FindByCode(10).GetObject();
Item.Description = "My new description";
Item.Write();

// Example: filling a catalog with test data.
Item = Catalogs.MaterialsAndServices.CreateItem();
Item.Description = "Test item";
Item.Write();
For Z = 1 To 1000 Do
    NewItem = Item.Copy();
    NewItem.Write();
EndDo;

8.  // CatalogSelection.<catalog name> object
    // Ref

// Example: filling the tabular section of a GoodsReceipt document
// with all the items from the specified MaterialsAndServices catalog folder.
Selection = Catalogs.MaterialsAndServices.SelectHierarchically(TextBox1);
While Selection.Next() Do
    MaterialOrServiceReference = Selection.Ref;
    If MaterialOrServiceReference.IsFolder Then Continue;
    EndIf;
    NewRow = Materials.Add();
    NewRow.Material = MaterialOrServiceReference;
EndDo;

9.  // CatalogSelection.<catalog name> object
    // GetObject()

// Example: marking all items of a nonhierarchical catalog for deletion.
Selection = Catalogs.Customers.Select();
```

```
While Selection.Next() Do
    Selection.GetObject().SetDeletionMark(True);
EndDo;
```

Event sequence for writing catalog items from item forms (save and close)

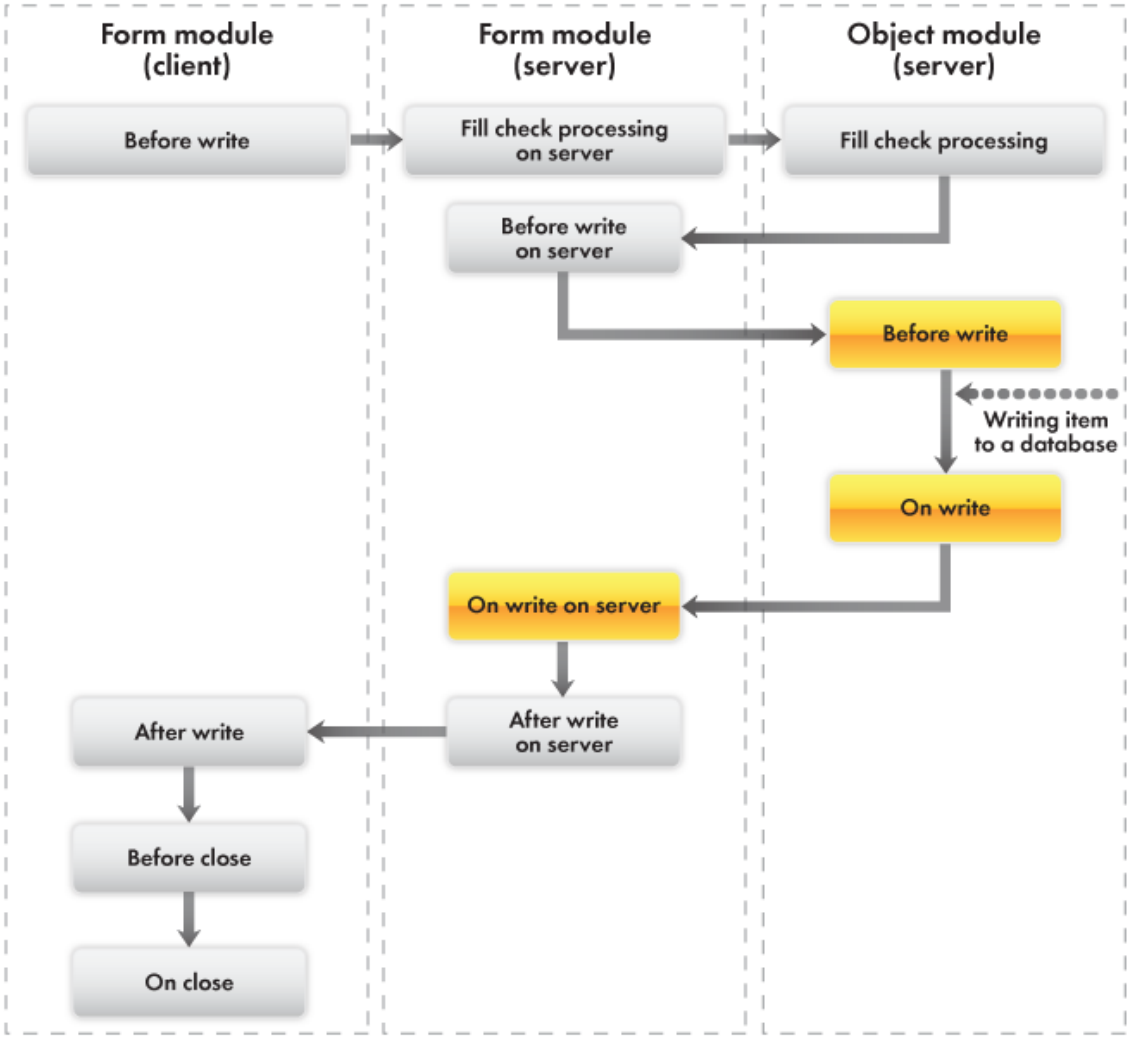


Fig. 29.4. Event sequence for writing a catalog item from an item form

Note. Yellow boxes indicate events executed in the writing transaction.

Documents

1C:Enterprise script objects used for operations with documents

The following chart shows the interaction between 1C:Enterprise script objects used for operations with documents (fig. 29.5).

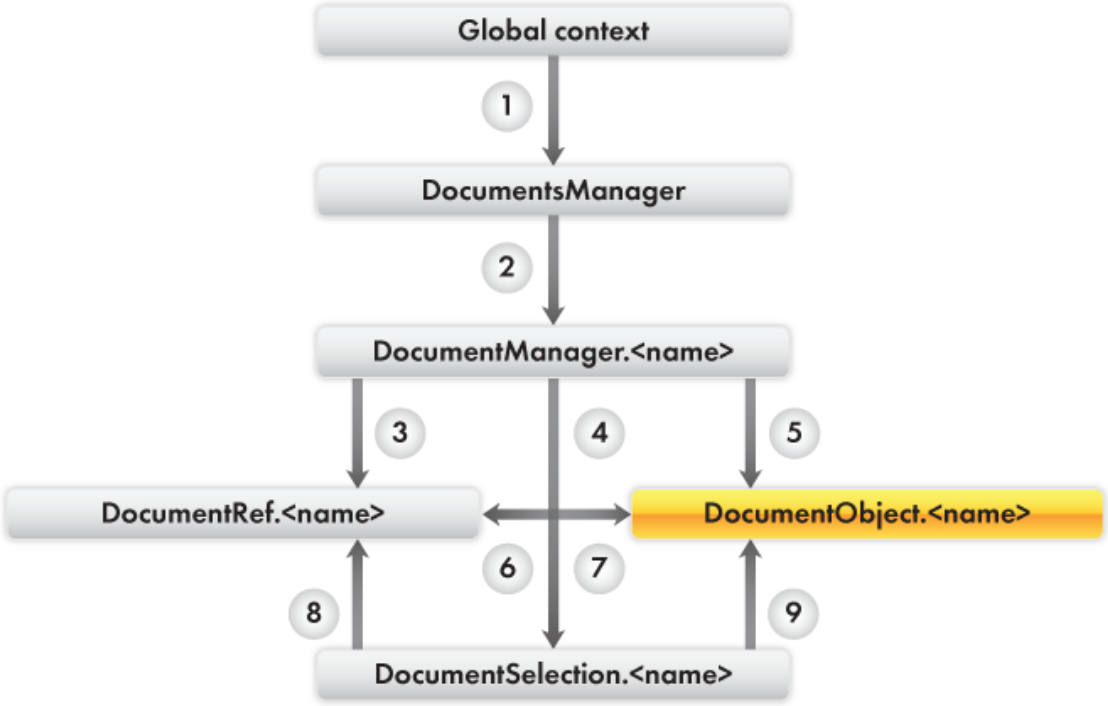


Fig. 29.5. 1C:Enterprise script objects used for operations with documents

Note. The yellow box indicates the data manipulation object. The object method from which the arrow originates is marked with the respective number in the listing, while the target object of the arrow is the type of returned object.

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

For examples of operations with documents that involve 1C:Enterprise script objects, see listing 29.6.

Listing 29.6. Object usage examples

```
1.  // Global context
    // Documents

// Example: displaying all types of references to documents available in
// the configuration.
Array = Documents.AllRefsType().Types();
For Each NextType In Array Do
    Message(NextType);
EndDo;

2.  // DocumentsManager object
    // .<document name>
    // [<document name>]
    // For Each … In … Do … EndDo;
```

```
// Example: getting the print template for the Services document
Template = Documents["Services"].GetTemplate("Print");


// Example: getting references for all documents available in
// the configuration.
For Each NextDocument In Documents Do
    Ref = NextDocument.GetRef();
    …
EndDo;


3.  // DocumentManager.<document name> object
    // FindByNumber()
    // FindByAttribute ()
    // EmptyRef ()


// Example: checking whether the GoodsReceipt document #3 has been posted.
If Documents.GoodsReceipt.FindByNumber(3).Posted Then
    Message("Document #3 is posted.");
EndIf;


// Example: checking whether the Warehouse attribute is filled
// in all GoodsReceipt documents.
WarehouseEmptyRef = Catalogs.Warehouses.EmptyRef();
If Not Documents.GoodsReceipt.FindByAttribute(
    "Warehouse",WarehouseEmptyRef).IsEmpty() Then
    Message("Documents with blank Warehouse attribute are found.");
EndIf;


4.  // DocumentManager.<document name> object
    // Select()


// Example: selecting all GoodsReceipt documents for the current month.
Selection = Documents.GoodsReceipt.Select(BegOfMonth(CurrentDate()),
    EndOfMonth(CurrentDate()));
While Selection.Next() Do
    Message(Selection);
EndDo;


5.  // DocumentManager.<document name> object
    // CreateDocument()


// Example: creating a GoodsReceipt document.
NewDocument = Documents.GoodsReceipt.CreateDocument();
NewDocument.Date = CurrentDate();
NewDocument.Warehouse = Catalogs.Warehouses.Main;
```

```
// Filling the Materials tabular section
NewTableRow = NewDocument.Materials.Add();
NewTableRow.Material = Catalogs.MaterialsAndServices.FindByCode(6);
NewTableRow.Quantity = 10;
NewTableRow.Price = 22.5;
NewTableRow.Total = 225;
NewDocument.Write();

6.  // DocumentObject.<document name> and DocumentRef.<document name> objects
    // Ref

// Example: calling a procedure within an object module to check
// whether document attributes are filled.
If Not CheckAttributesFilling(ThisObject.Ref) Then
    Message("Document attributes are not filled.");
EndIf;

7.  // DocumentRef.<document name> and DocumentObject.<document name> objects
    // GetObject()
    // Copy()

// Example: marking a document for deletion.
OldDocument = Documents.Services.FindByNumber(13).GetObject();
OldDocument.SetDeletionMark(True);

8.  // DocumentSelection.<document name> object
    // Ref

// Example: generating a list of references to all GoodsReceipt documents
// for the current month.
ReceiptList = New ValueList;
Selection = Documents.GoodsReceipt.Select(BegOfMonth(CurrentDate()),
EndOfMonth(CurrentDate()));
While Selection.Next() Do
    ReceiptList.Add(Selection.Ref);
EndDo;

9.  // DocumentSelection.<document name> object
    // GetObject()
// Example: deleting all GoodsReceipt documents.
Selection = Documents.GoodsReceipt.Select();
While Selection.Next() Do
    Selection.GetObject().Delete();
EndDo;
```

Event sequence for writing documents from document forms

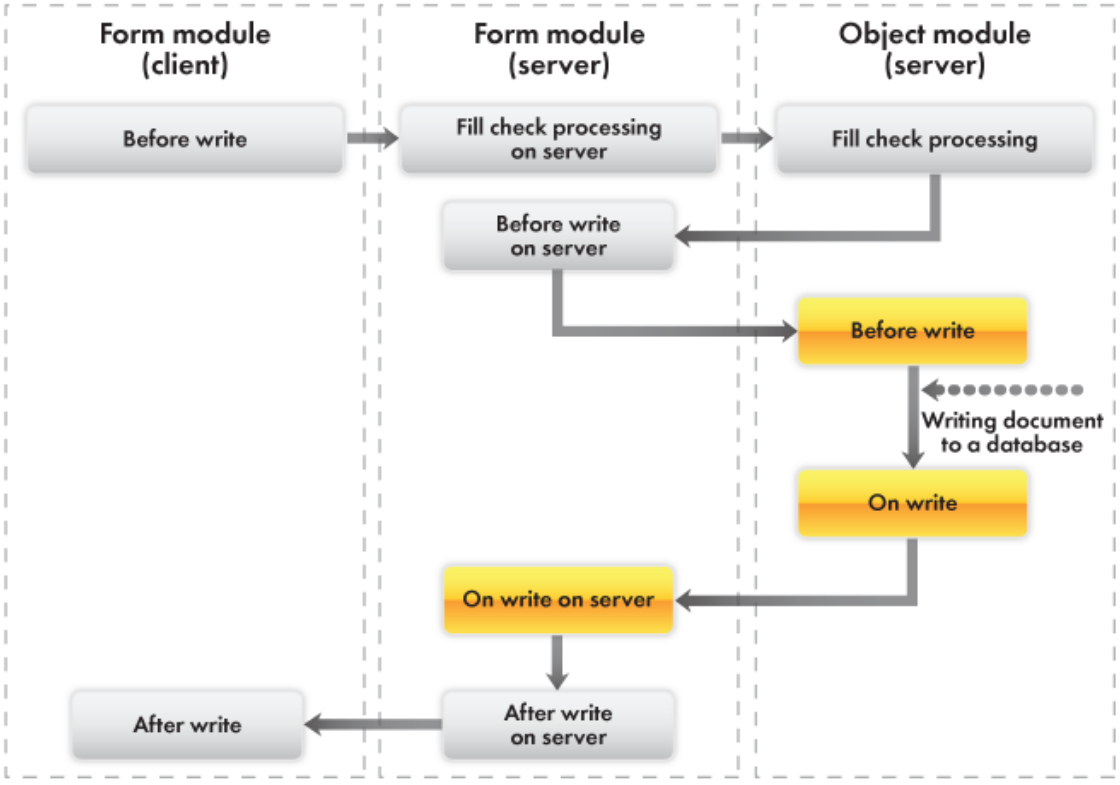


Fig. 29.6. Event sequence for writing a new document from a document form

Note. Yellow boxes indicate events executed in the writing transaction.

Event sequence for posting documents from document forms (post and close)

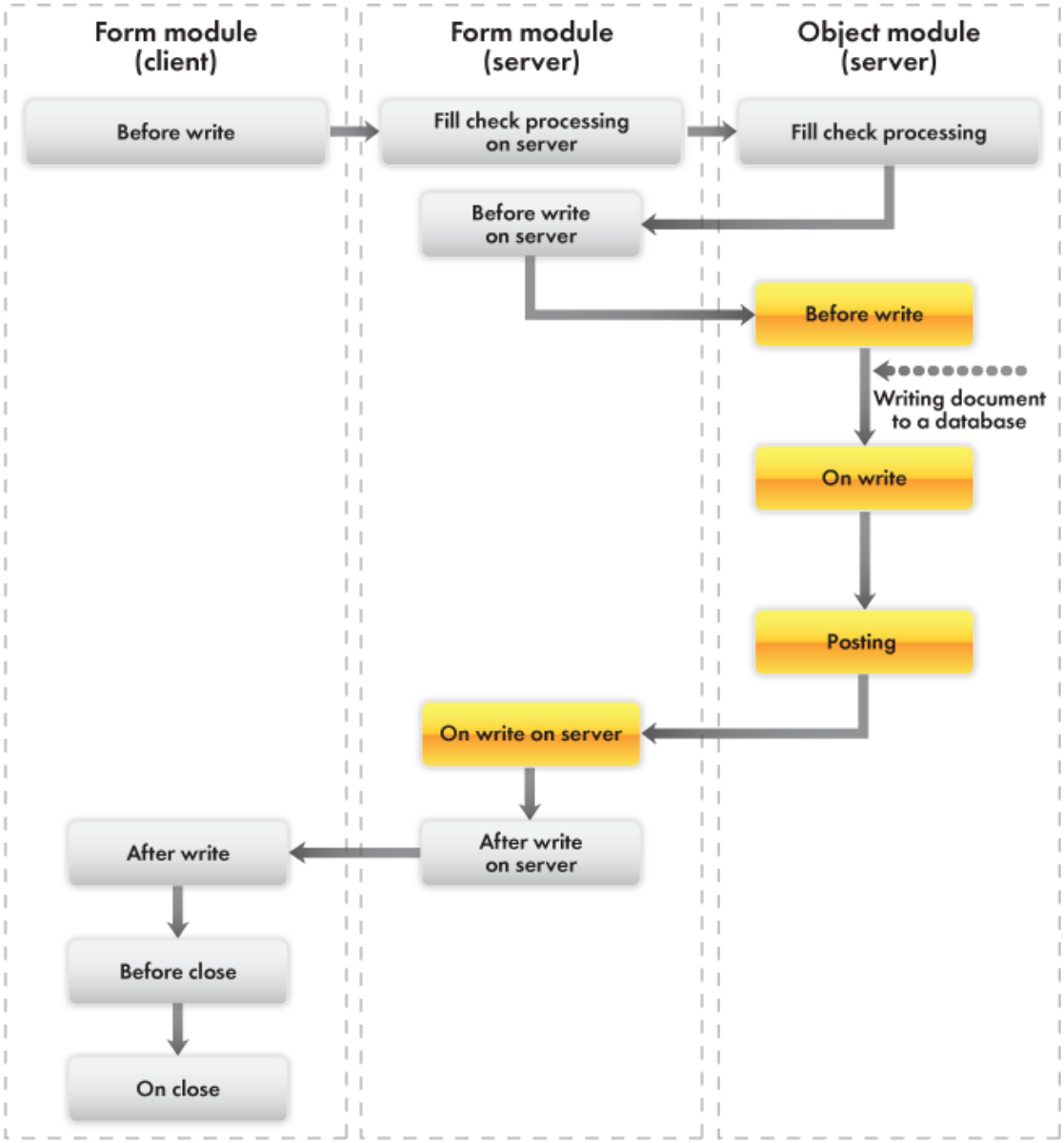


Fig. 29.7. Event sequence for posting a document from a document form

Note. Yellow boxes indicate events executed in the writing transaction.

Event sequence for canceling document posting from document forms

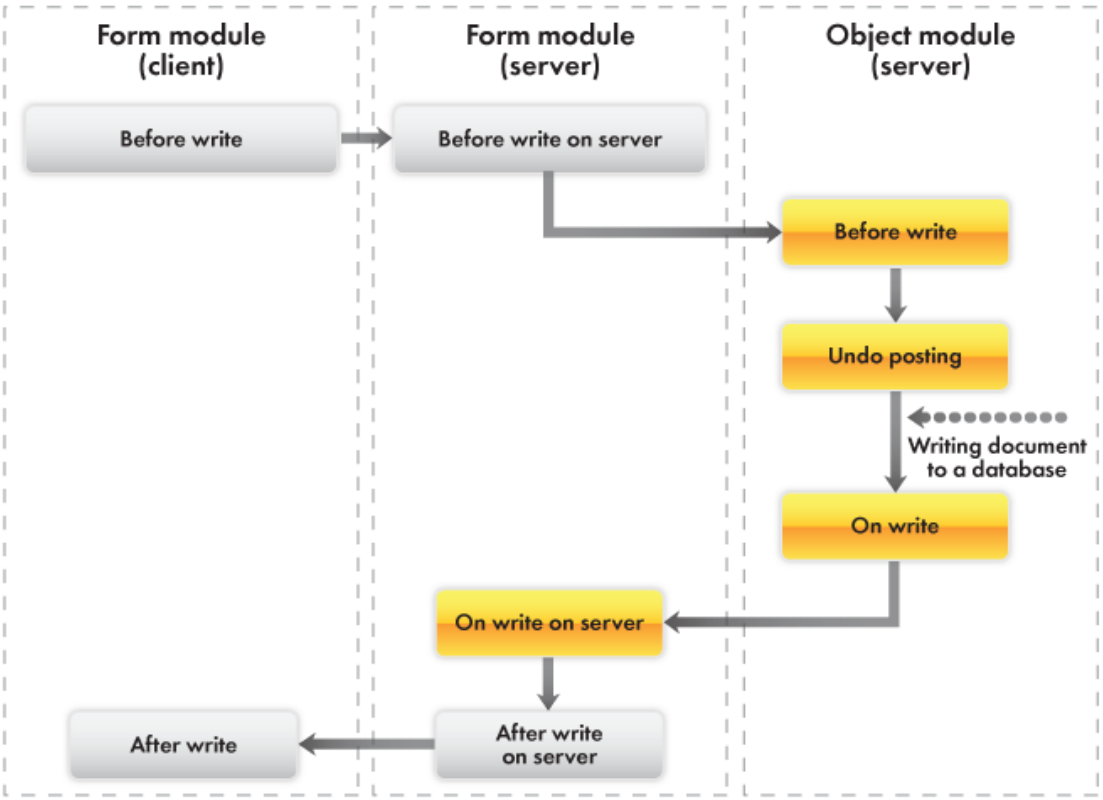


Fig. 29.7. Event sequence for canceling document posting from a document form

Note. Yellow boxes indicate events executed in the writing transaction.

Enumerations

1C:Enterprise script objects used for operations with enumerations

The following chart shows the interaction between 1C:Enterprise script objects used for operations with enumerations (fig. 29.9).

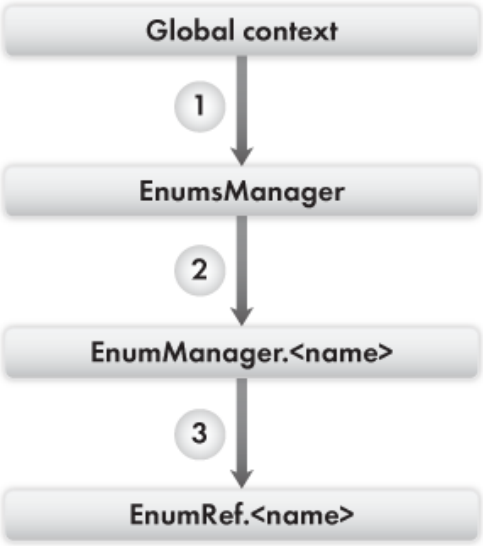


Fig. 29.9. 1C:Enterprise script objects used for operations with enumerations

Note. The object method from which the arrow originates is marked with the respective number in the listing, while the target object of the arrow is the type of returned object.

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

For examples of operations with enumerations that involve 1C:Enterprise script objects, see listing 29.7.

Listing 29.7. Object usage examples

```
1.  // Global context
    // Enums

// Example: getting an enumeration value by index.
Enums.MaterialServiceTypes.Get(0);

2.  // EnumsManager object
    // .<enumeration name>
    // [<enumeration name>]
    // For Each … In … Do … EndDo;

// Example: getting the number of enumeration values.
Enums.["MaterialServiceTypes"].Count();

3.  // EnumManager.<enumeration name> object
    // .<enumeration value name>
    // [<enumeration value name>]
    // [<collection element index>]
    // For Each … In … Do … EndDo;
    // EmptyRef ()
```

```
// Example: getting an empty reference to an enumeration value.
…
EnumEmptyRef = Enums.MaterialServiceTypes.EmptyRef();
If CurrentMaterialOrService.MaterialServiceType = EnumEmptyRef Then
   // Prompting to fill the type.
    …
EndIf;
…
```

Charts of characteristic types

1C:Enterprise script objects used for operations with charts of characteristic types

The following chart shows the interaction between 1C:Enterprise script objects used for operations with charts of characteristic types (fig. 29.10).

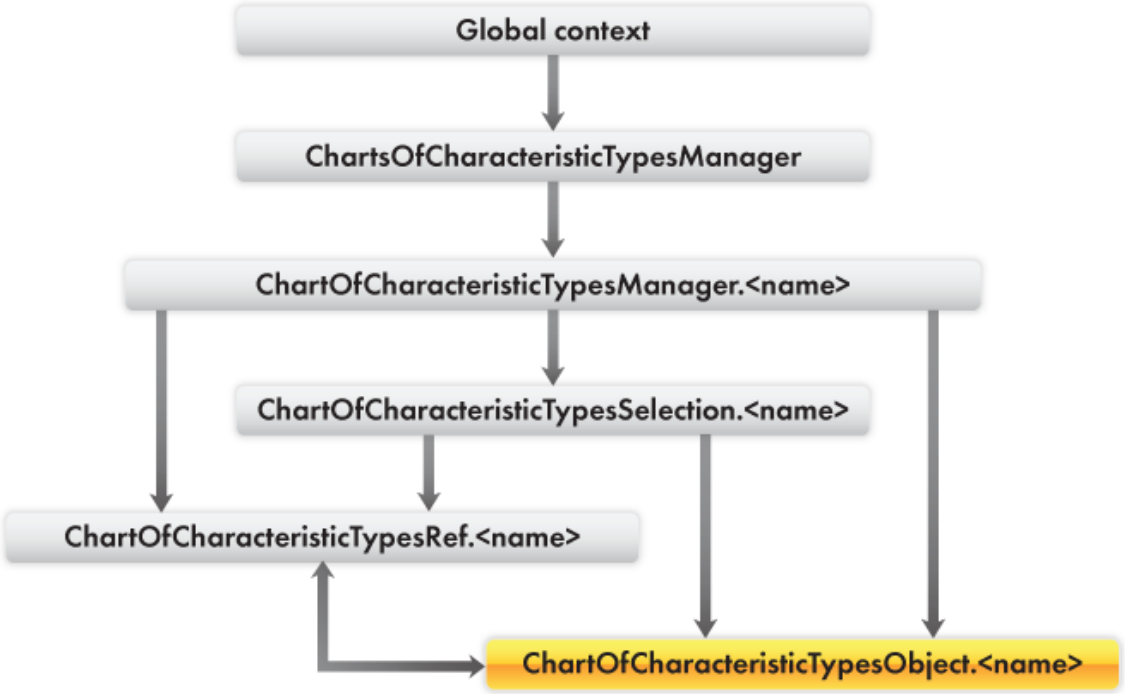


Fig. 29.10. 1C:Enterprise script objects used for operations with charts of characteristic types

Note. The yellow box indicates the data manipulation object.

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

Properties and methods used for operations with charts of characteristic types are similar to properties and methods used for operations with catalogs. See section "1C:Enterprise script objects used for operations with catalogs" on page 839.

Event sequence for writing characteristic types from item forms (save and close)

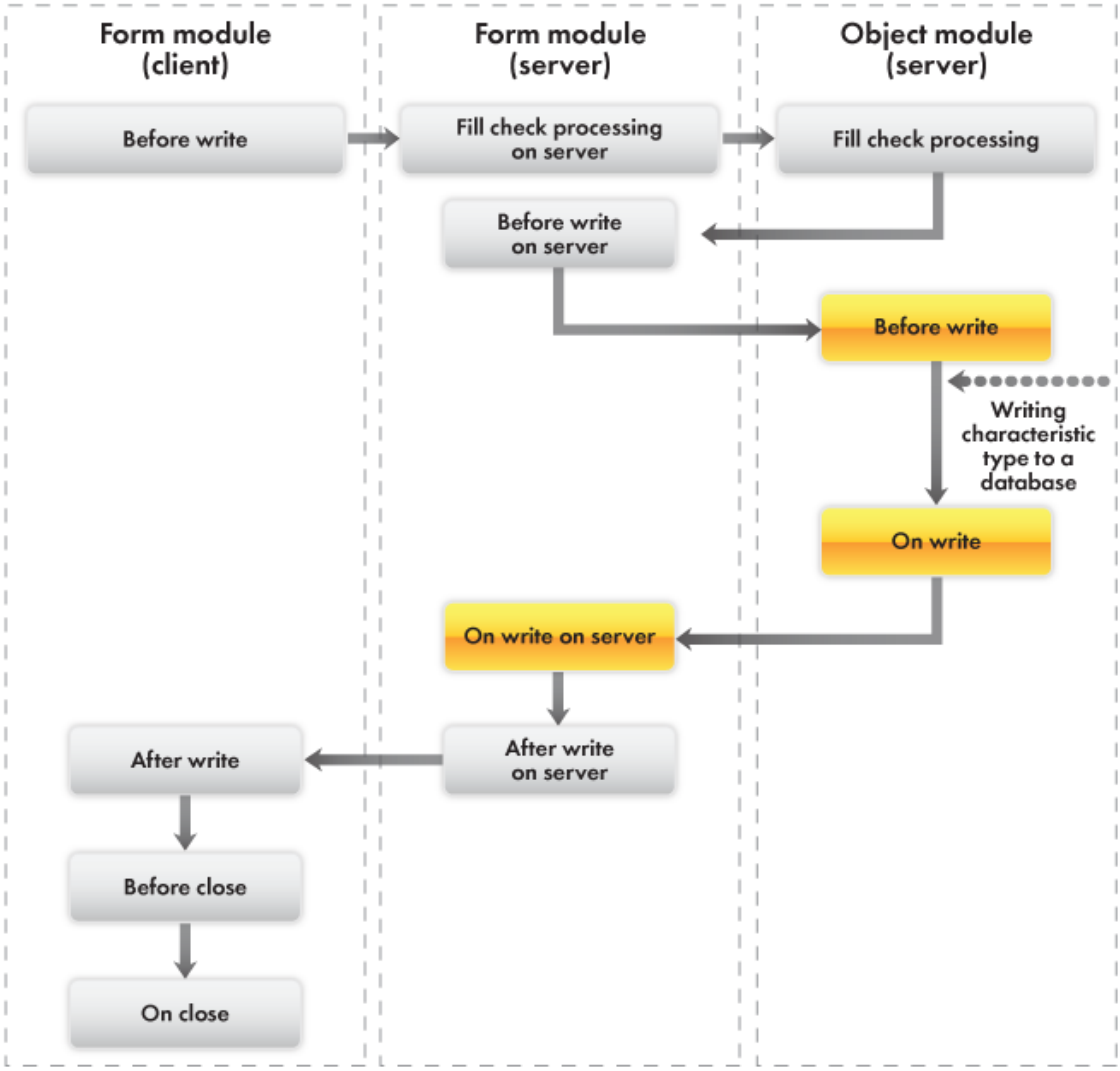


Fig. 29.11. Event sequence for writing a characteristic type from an item form

Note. Yellow boxes indicate events executed in the writing transaction.

Charts of accounts

1C:Enterprise script objects used for operations with charts of accounts

The following chart shows the interaction between 1C:Enterprise script objects used for operations with charts of accounts (fig. 29.12).

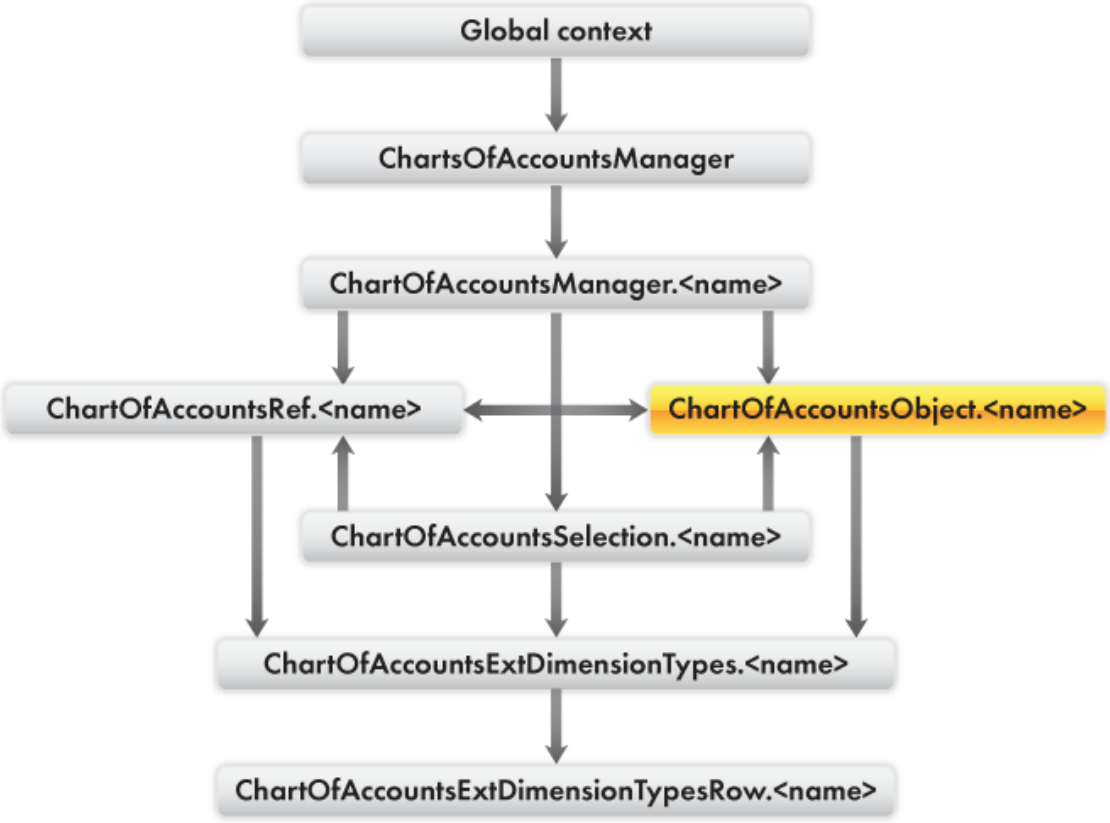


Fig. 29.12. 1C:Enterprise script objects used for operations with charts of accounts

Note. The yellow box indicates the data manipulation object.

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

ChartOfAccountsExtDimensionTypes.<name> is used to access methods of the DimensionTypes tabular section of account objects.

ChartOfAccountsExtDimensionTypesRow.<name> is a row of the DimensionTypes tabular section.

Properties and methods used for operations with charts of accounts are similar to properties and methods used for operations with catalogs. See section "1C:Enterprise script objects used for operations with catalogs" on page 839.

Event sequence for writing accounts from account forms (save and close)

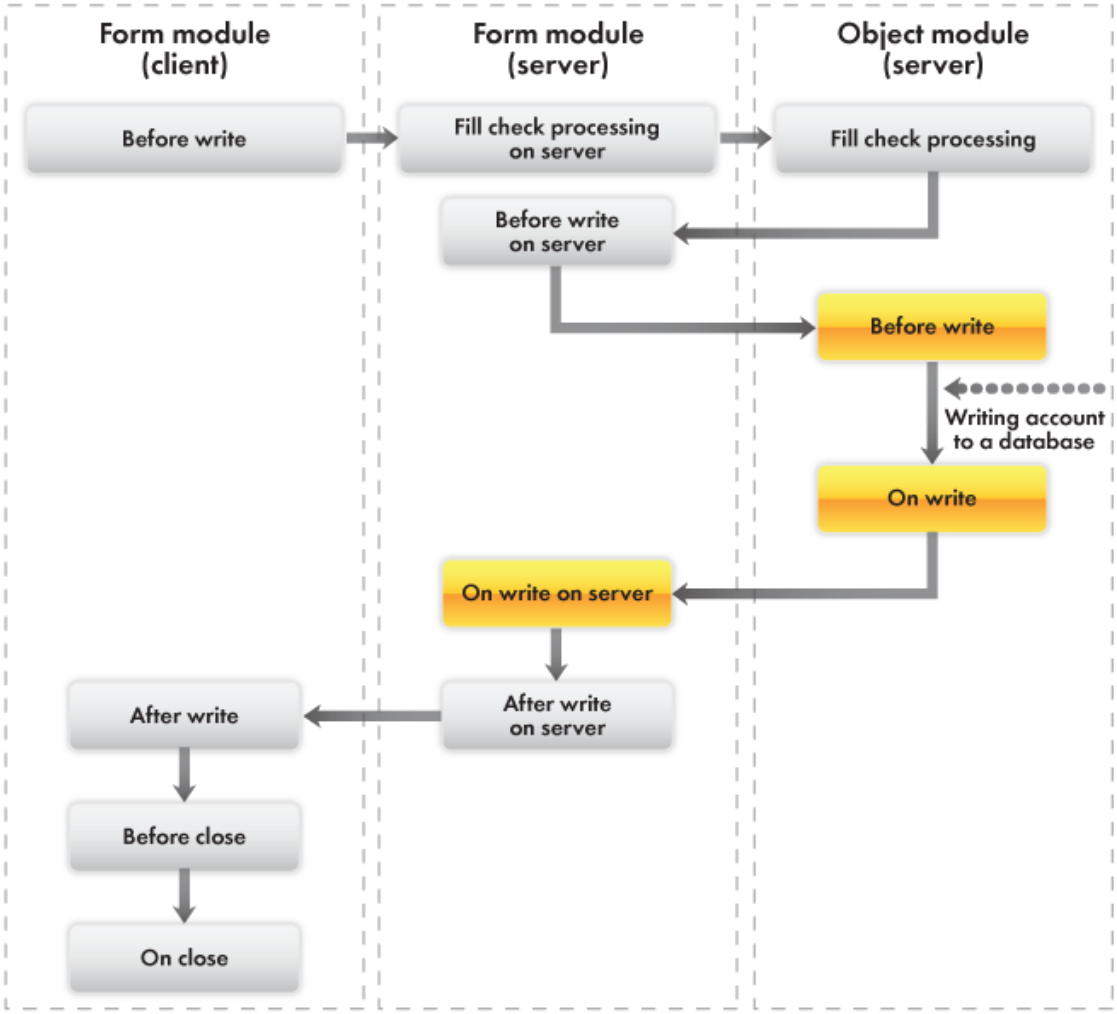


Fig. 29.13. Event sequence for writing an account from an account form

Note. Yellow boxes indicate events executed in the writing transaction.

Charts of calculation types

1C:Enterprise script objects used for operations with charts of calculation types

The following chart shows the interaction between 1C:Enterprise script objects used for operations with charts of calculation types (fig. 29.14).

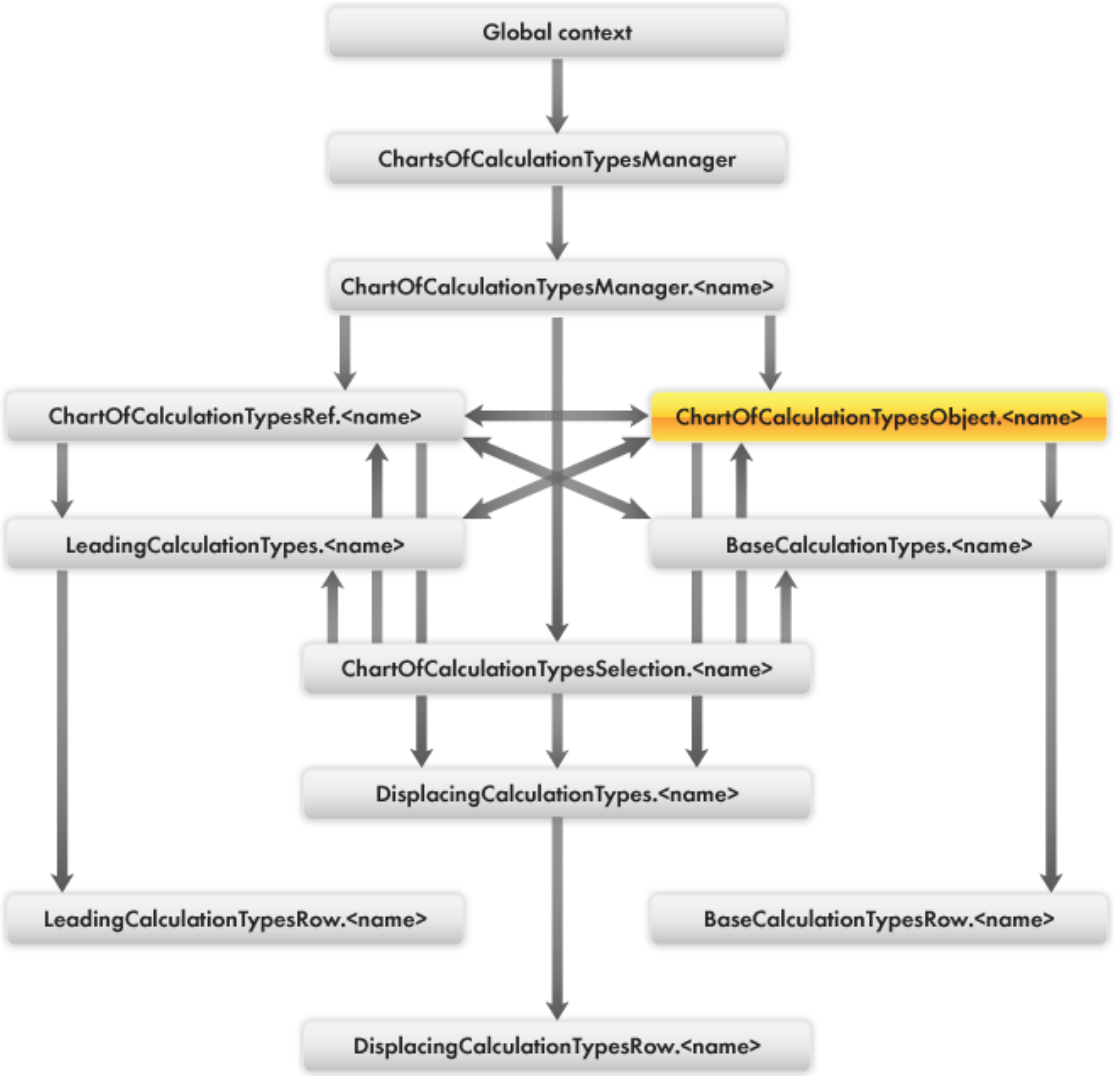


Fig. 29.14. 1C:Enterprise script objects used for operations with charts of calculation types

Note. Yellow boxes indicate data manipulation objects.

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

DisplacingCalculationTypes.<name> is a predefined tabular section of a calculation type, which stores the list of displacing calculation types. The tabular section is defined only for charts of calculation types with the "Uses action period" property set to True. It contains a single column: CalculationType, which has the ChartOfCalculationTypesRef.<name> type.

DisplacingCalculationTypesRow.<name> is a row of the DisplacingCalculationTypes.<name> table.

LeadingCalculationTypes.<name> is a predefined tabular section of a calculation type, which stores the list of leading calculation types. It contains a single column: CalculationType, which has the ChartOfCalculationTypesRef.<name> type.

LeadingCalculationTypesRow.<name> is a row of the LeadingCalculationTypes.<name> table.

BaseCalculationTypes.<name> is a predefined tabular section of a calculation type, which stores the list of base calculation types. The tabular section is defined only for charts of calculation types with the "Dependency on base calculation type" property not equal to "Does not depend". It contains a single column: CalculationType, which has the ChartOfCalculationTypesRef.<name> type.

BaseCalculationTypesRow.<name> is a row of the BaseCalculationTypes.<name> table.

Properties and methods used for operations with charts of calculation types are similar to properties and methods used for operations with catalogs. See section "1C:Enterprise script objects used for operations with catalogs" on page 839.

Event sequence for writing calculation types from calculation type forms (save and close)

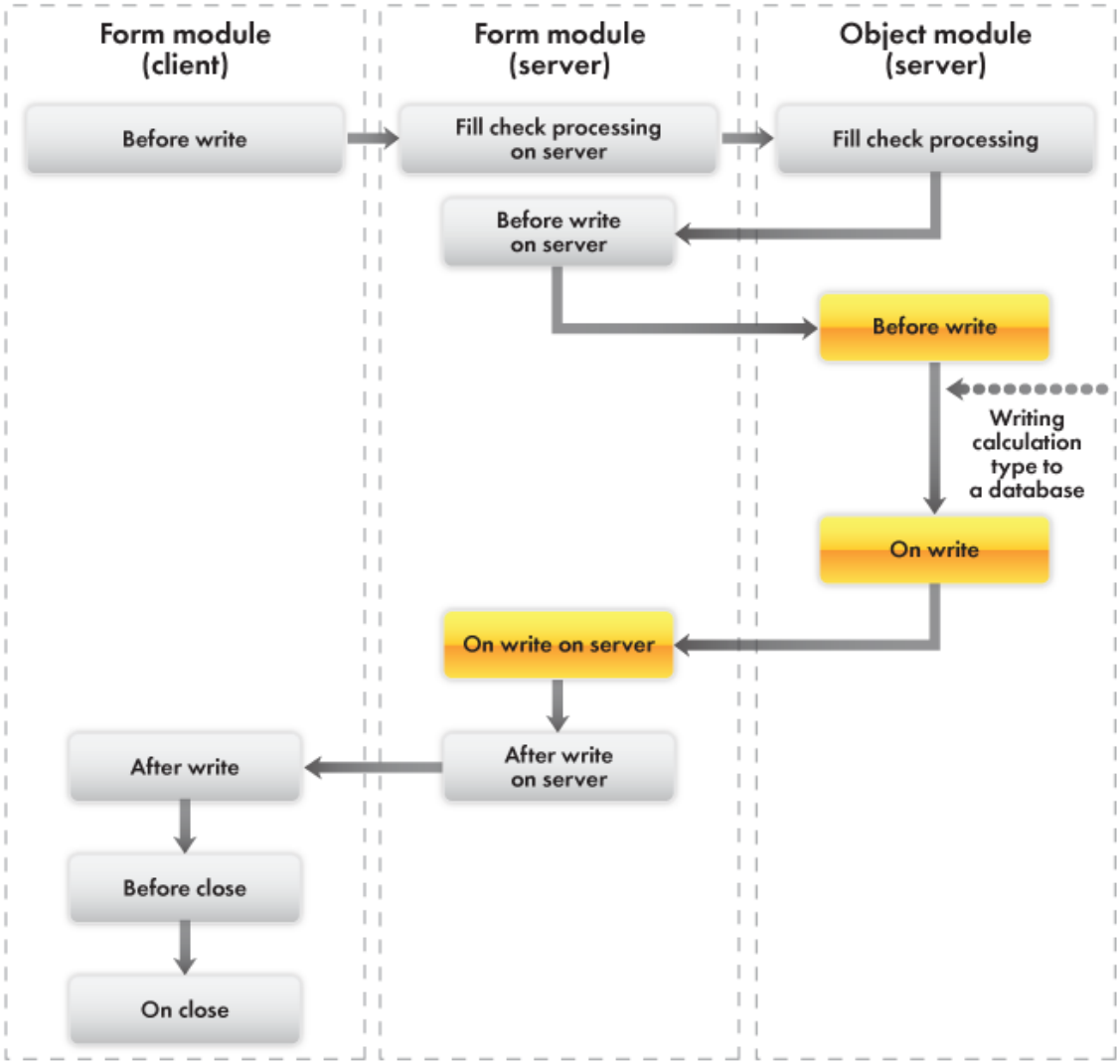


Fig. 29.15. Event sequence for writing a calculation type from a calculation type form

Note. Yellow boxes indicate events executed in the writing transaction.

Information registers

1C:Enterprise script objects used for operations with information registers

The following chart shows the interaction between 1C:Enterprise script objects used for operations with information registers (fig. 29.16).

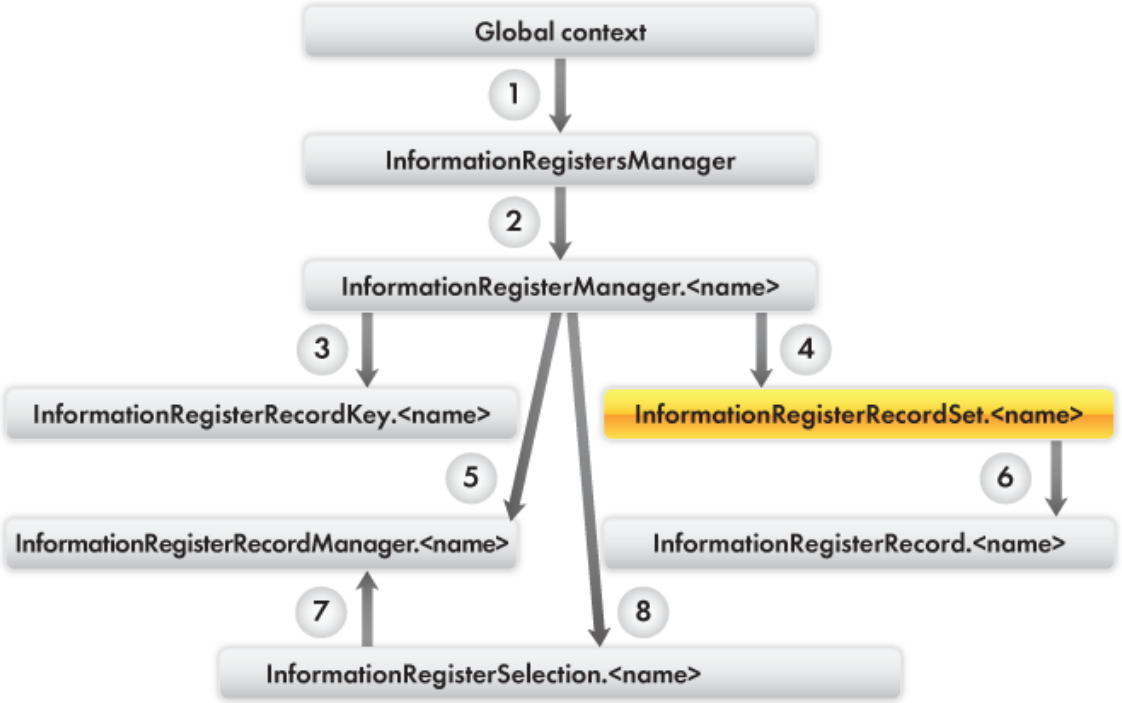


Fig. 29.16. 1C:Enterprise script objects used for operations with information registers

Note. The yellow box indicates the data manipulation object. The object method from which the arrow originates is marked with the respective number in the listing, while the target object of the arrow is the type of returned object.

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

InformationRegisterRecordManager.<name> is used to read, write, and delete individual information register records. It is available only for information registers that cannot be changed by recorders (those having the "Write mode" property set to Independent in Designer).

InformationRegisterRecord.<name> provides access to information register records. This object is not created directly. Instead, it is provided by other objects related to information registers. For example, this object represents register records in a record set.

InformationRegisterRecordKey.<name> is a set of values that uniquely identify a register record. This object is used for referencing a specific record. For example, this object represents a value of the CurrentRow property of the tabular section that stores a list of register records.

For examples of operations with information registers that involve 1C:Enterprise script objects, see listing 29.8.

Listing 29.8. Object usage examples

```
1.  // Global context
    // InformationRegisters

// Example: getting the current price from the Prices periodic information
// register.
Item = Catalogs.MaterialsAndServices.FindByCode(4);
Filter = New Structure("MaterialOrService", Item);
ResourceValues = InformationRegisters.Prices.GetLast(CurrentDate(), Filter);
Price = ResourceValue.Price;

2.  // InformationRegistersManager object
    // .<information register name>
    // [<information register name>]
    // For Each … In … Do … EndDo;

// Example: getting the initial price from the Prices periodic information
// register.
RegisterName = "Prices";
Service = Catalogs.MaterialsAndServices.FindByDescription("Diagnostics");
Filter = New Structure;
Filter.Insert("MaterialOrService", Service);
Price = InformationRegisters[RegisterName].GetFirst(CurrentDate(),
    Filter).Price;

3.  // InformationRegisterManager.<name> object
    // CreateRecordKey()

// Example: activating a row in the information register list.
KeyFieldStructure = New Structure;
KeyFieldStructure.Insert("Period", Date("20140331000000"));
KeyFieldStructure.Insert("MaterialOrService",
    Catalogs.MaterialsAndServices.FindByCode("0000006"));
Items.Materials.CurrentRow =
    InformationRegisters.Prices.CreateRecordKey(KeyFieldStructure);

4.  // InformationRegisterManager.<name> object
    // CreateRecordSet()

// Example: displaying materials and services whose price was set at
// the given date and time.
Set = InformationRegisters.Prices.CreateRecordSet();
Set.Filter.Period.Set(SpecifiedDate, True);
```

```
Set.Read();
For Each NextRecord In Set Do
    Message("Material or service = " + NextRecord.MaterialOrService
        + ", price = " + NextRecord.Price + ".");
EndDo;


5.  // InformationRegisterManager.<name> object
    // CreateRecordManager()


// Example: adding a price value to the Prices register.
Record = InformationRegisters.Prices.CreateRecordManager();
Record.Period = CurrentDate();
Record.MaterialOrService                                                    =
Catalogs.MaterialsAndServices.FindByCode("0000005");
Record.Price = 568;
Record.Write();


6.  // InformationRegisterRecordSet.<name> object
    // [<collection element index>]
    // For Each … In … Do … EndDo;


// Example: displaying materials and services whose price was set at
// the given date and time.
Set = InformationRegisters.Prices.CreateRecordSet();
Set.Filter.Period.Set(SpecifiedDate, True);
Set.Read();
For Each NextRecord In Set Do
   Message("Material or service = " + NextRecord.MaterialOrService
       + ", price = " + NextRecord.Price + ".");
EndDo;


7.  // InformationRegisterSelection.<name> object
    // GetRecordManager()


// Example: Deleting all information register records for the current month.
Selection = InformationRegisters.Prices.Select(BegOfMonth(CurrentDate()),
    EndOfMonth(CurrentDate()));
While Selection.Next() Do
    Selection.GetRecordManager().Delete();
EndDo;


8.  // InformationRegisterManager.<name> object
    // Select()
    // SelectByRecorder()
```

```
// Example: displaying price changes for a material or service over

// the course of a year.

Filter = New Structure("MaterialOrService",
    Catalogs.MaterialsAndServices.FindByCode("0000005"));

Selection = InformationRegisters.Prices.Select(BegOfYear(CurrentDate()),
    CurrentDate(), Filter);

While Selection.Next() Do

    Message("Date = " + Selection.Period + ", price = " + Selection.Cost
        + ".");

EndDo;
```

Event sequence for writing data from information register record forms (save and close)

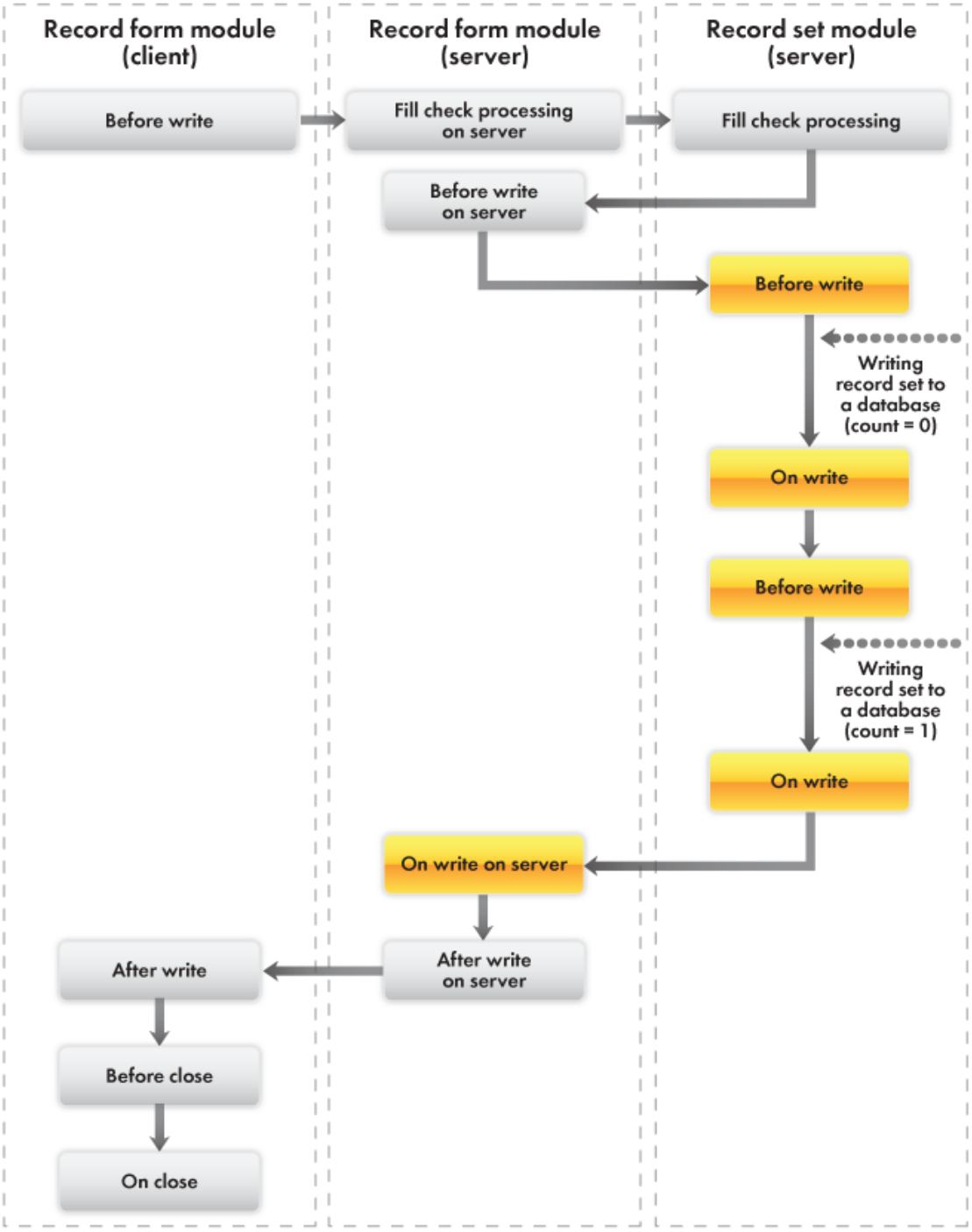


Fig. 29.17. Event sequence for writing data from an information register record form

Note. Yellow boxes indicate events executed in the writing transaction.

Information register record forms are accessed using the InformationRegisterRecordManager.<name> object, which in turn uses the InformationRegisterRecordSet.<name> object.

The internal implementation of the InformationRegisterRecordManager.<name> object defines that when an existing information register record is being written the BeforeWrite() and OnWrite() event handlers from the record set module are called twice: first they are called for the "old" record set (with a record count of 0), and then for the "new" record set (with a record count of 1).

Event sequence for writing data from information register record set forms (save and close)

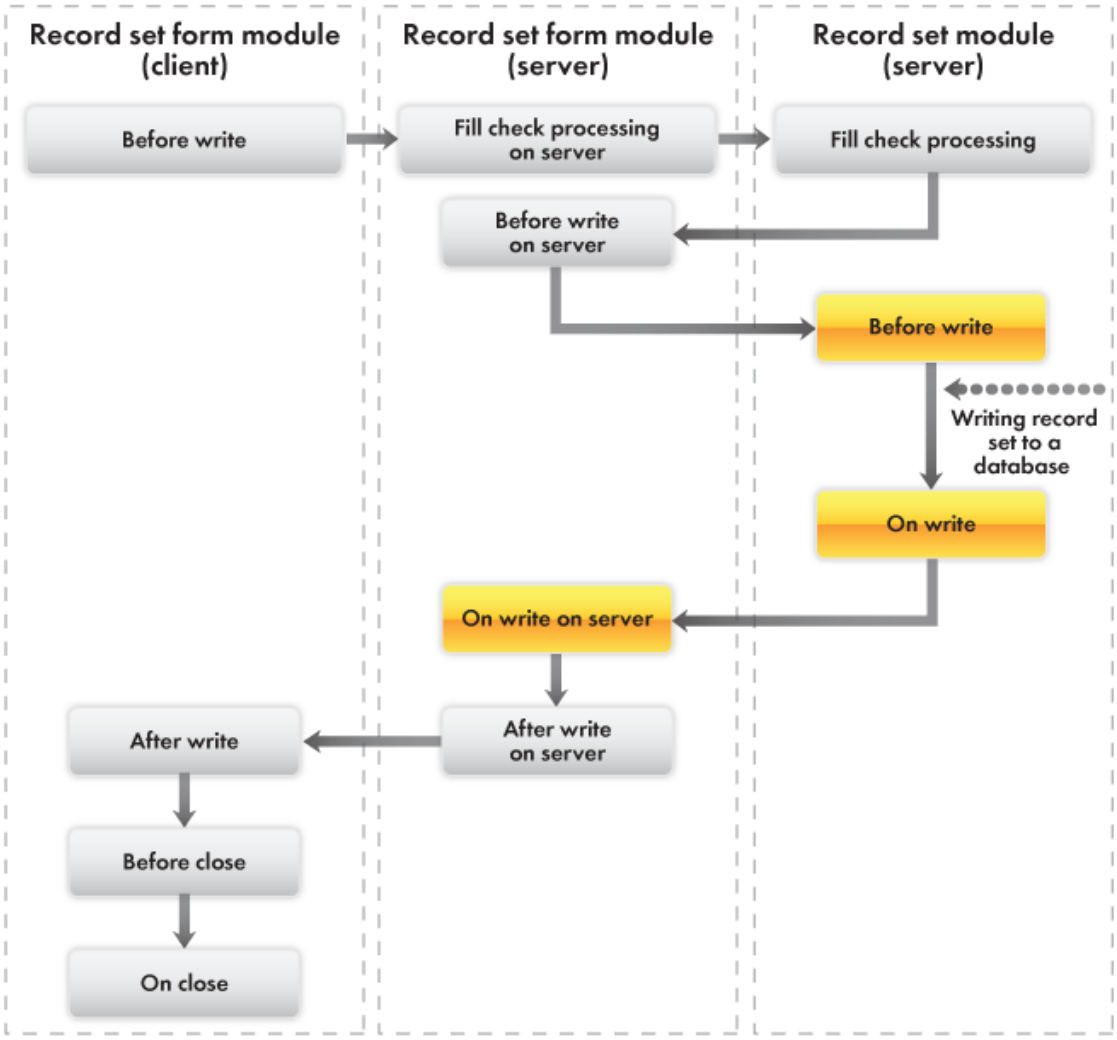


Fig. 29.18. Event sequence for writing data from an information register record set form

Note. Yellow boxes indicate events executed in the writing transaction.

Accumulation registers

1C:Enterprise script objects used for operations with accumulation registers

The following chart shows the interaction between 1C:Enterprise script objects used for operations with accumulation registers (fig. 29.19).

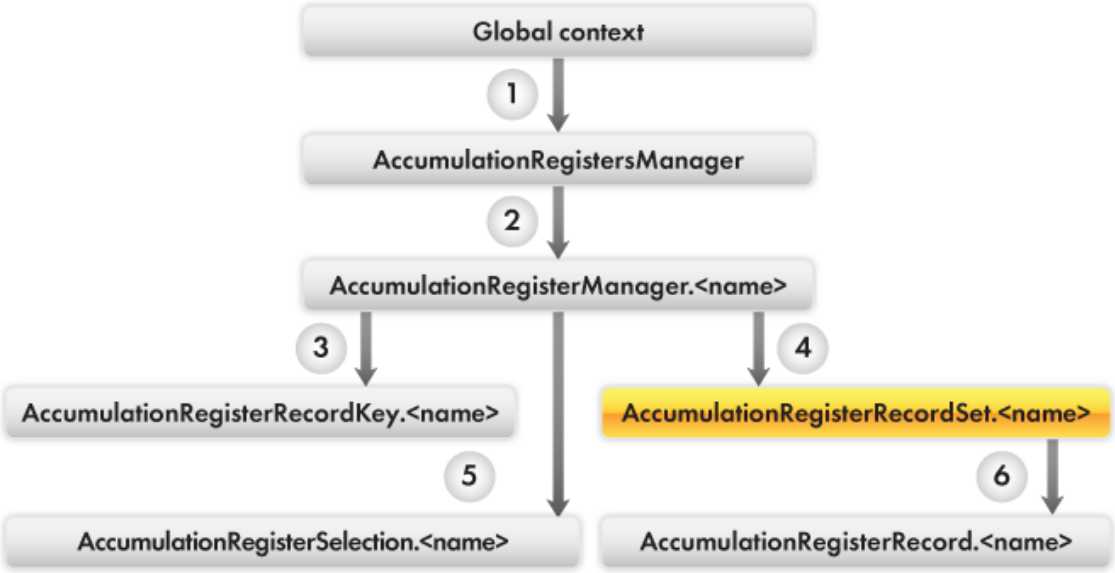


Fig. 29.19. 1C:Enterprise objects used for operations with accumulation registers

Note. The yellow box indicates the data manipulation objects. The object method from which the arrow originates is marked with the respective number in the listing, while the target object of the arrow is the type of returned object.

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

AccumulationRegisterRecord.<name> provides access to accumulation register records. This object is not created directly. Instead, it is provided by other objects related to accumulation registers. For example, this object represents register records in a record set.

AccumulationRegisterRecordKey.<name> is a set of values that uniquely identify a register record. This object is used for referencing a specific record. For example, this object represents a value of the CurrentRow property of the tabular section that stores a list of register records.

For examples of operations with accumulation registers that involve 1C:Enterprise script objects, see listing 29.9.

LISTING 29.9. OBJECT USAGE EXAMPLES

```
1.  // Global context
    // AccumulationRegisters
```

```
// Example: performing full recalculation of all totals in the
// BalanceOfMaterials register.
AccumulationRegisters.BalanceOfMaterials.RecalculateTotals();


2.  // AccumulationRegistersManager object
    // .<accumulation register name>
    // [<accumulation register name>]
    // For Each … In … Do … EndDo;


// Example: calculating totals of the BalanceOfMaterials register for
// the specified date.
RegisterName = BalanceOfMaterials;
AccumulationRegisters[RegisterName].SetMaxTotalsPeriod(SpecifiedDate);


3.  // AccumulationRegisterManager.<name> object
    // CreateRecordKey()


// Example: activating a specific row in the accumulation register list.
KeyFieldStructure = New Structure;
KeyFieldStructure.Insert("Recorder",
    Documents.GoodsReceipt.FindByNumber("0000002"));
KeyFieldStructure.Insert("LineNumber", 2);
Items.Materials.CurrentRow =
    AccumulationRegisters.BalanceOfMaterials.CreateRecordKey(
    KeyFieldStructure);


4.  // AccumulationRegisterManager.<name> object
    // CreateRecordSet()


// Example: getting register records created by a document.
RequiredDocument = Documents.GoodsReceipt.FindByNumber(4);
RegisterRecords = AccumulationRegisters.BalanceOfMaterials.CreateRecordSet();
RegisterRecords.Filter.Recorder.Value = RequiredDocument;
RegisterRecords.Read();


5.  // AccumulationRegisterManager.<name> object
    // Select()
    // SelectByRecorder()


// Example: selecting all records for the current month from
// the BalanceOfMaterials register.
Selection = AccumulationRegisters.BalanceOfMaterials.Select(
    BegOfMonth(CurrentDate()), EndOfMonth(CurrentDate()));


6.  // AccumulationRegisterRecordSet.<name> object
    // [<collection element index>]
```

```
    // For Each … In … Do … EndDo;

// Example: getting register records created by a document.
RequiredDocument = Documents.GoodsReceipt.FindByNumber(4);
RegisterRecords = AccumulationRegisters.BalanceOfMaterials.CreateRecordSet();
RegisterRecords.Filter.Recorder.Value = RequiredDocument;
RegisterRecords.Read();

For Each NextRecord In RegisterRecords Do
    // Register record processing algorithm
    …
EndDo;
```

Event sequence for writing accumulation register record sets from record set forms

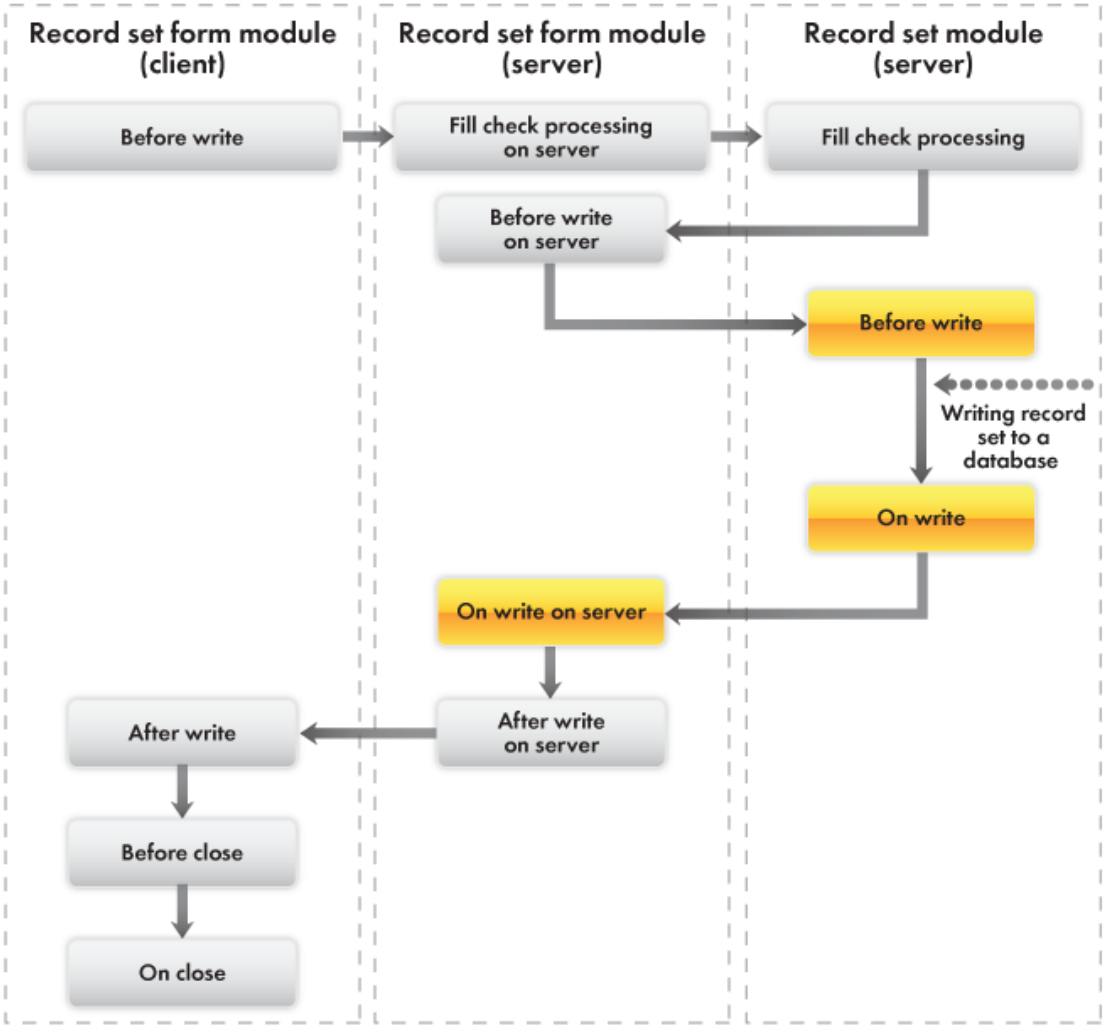


Fig. 29.20. Event sequence for writing an accumulation register record set from a record set form

Note. Yellow boxes indicate events executed in the writing transaction.

Accounting registers

1C:Enterprise script objects used for operations with accounting registers

The following chart shows the interaction between 1C:Enterprise script objects used for operations with accounting registers (fig. 29.21).

Fig. 29.21. 1C:Enterprise objects used for operations with accounting registers

Note. The yellow box indicates the data manipulation object.

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

AccountingRegisterRecord.<name> provides access to accounting register records. This object is not created directly. Instead, it is provided by other objects related to accounting registers. For example, this object represents register records in a record set.

AccountingRegisterExtDimensions.<name> is a collection of extra dimension values of an accounting register record. Setting and getting values of a specific extra dimension are performed using the [] operator that accepts extra dimension type as a parameter, or using a predefined extra dimension name.

AccountingRegisterRecordKey.<name> is a set of values that uniquely identify a register record. This object is used for referencing a specific record. For example, this object represents a value of the CurrentRow property of the tabular section that stores a list of register records.

Properties and methods used for operations with accounting registers are similar to properties and methods used for operations with accumulation registers. See section "1C:Enterprise script objects used for operations with accumulation registers" on page 863.

Event sequence for writing accounting register record sets from record set forms

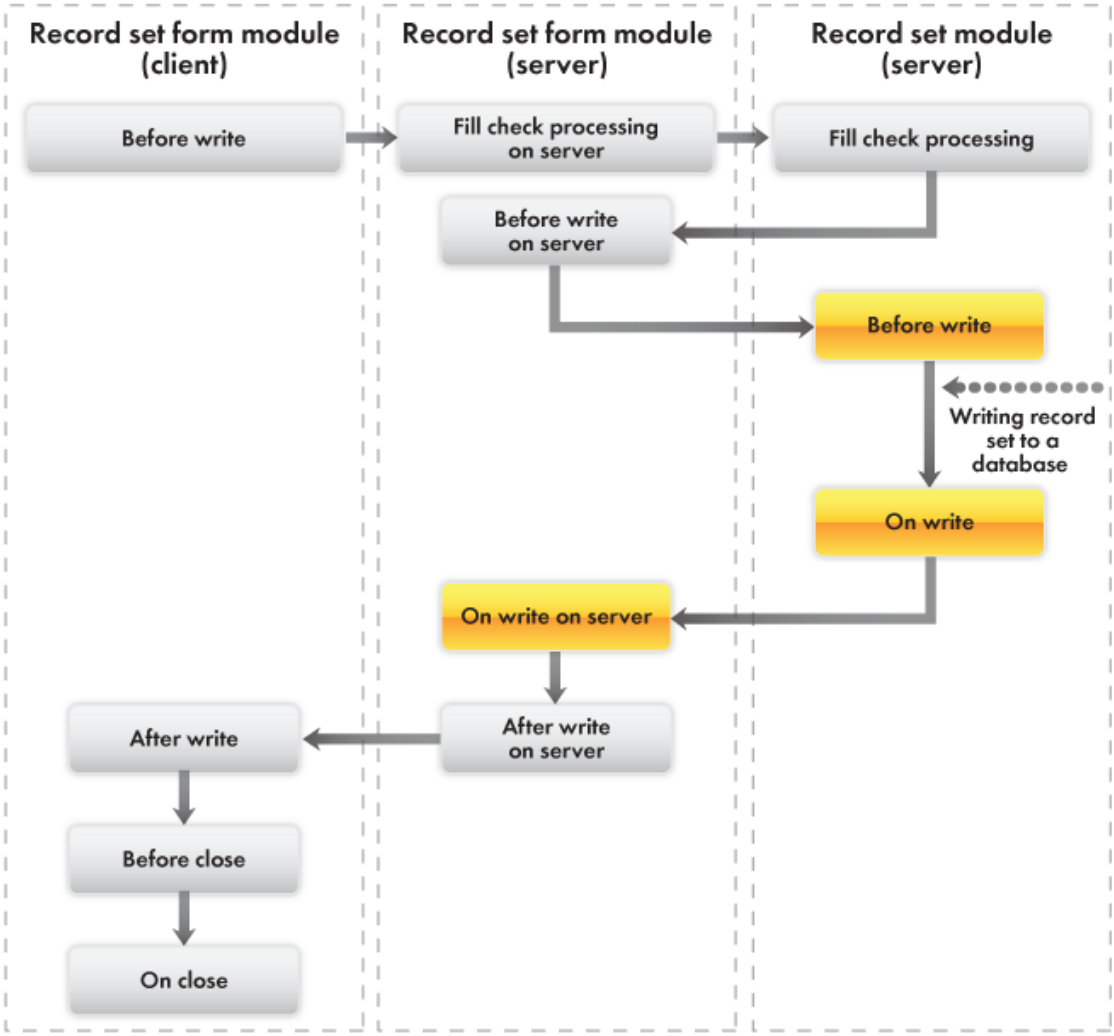


Fig. 29.22. Event sequence for writing an accounting register record set from a record set form

Note. Yellow boxes indicate events executed in the writing transaction.

Calculation registers

1C:Enterprise script objects used for operations with calculation registers

The following chart shows the interaction between 1C:Enterprise script objects used for operations with calculation registers (fig. 29.23).

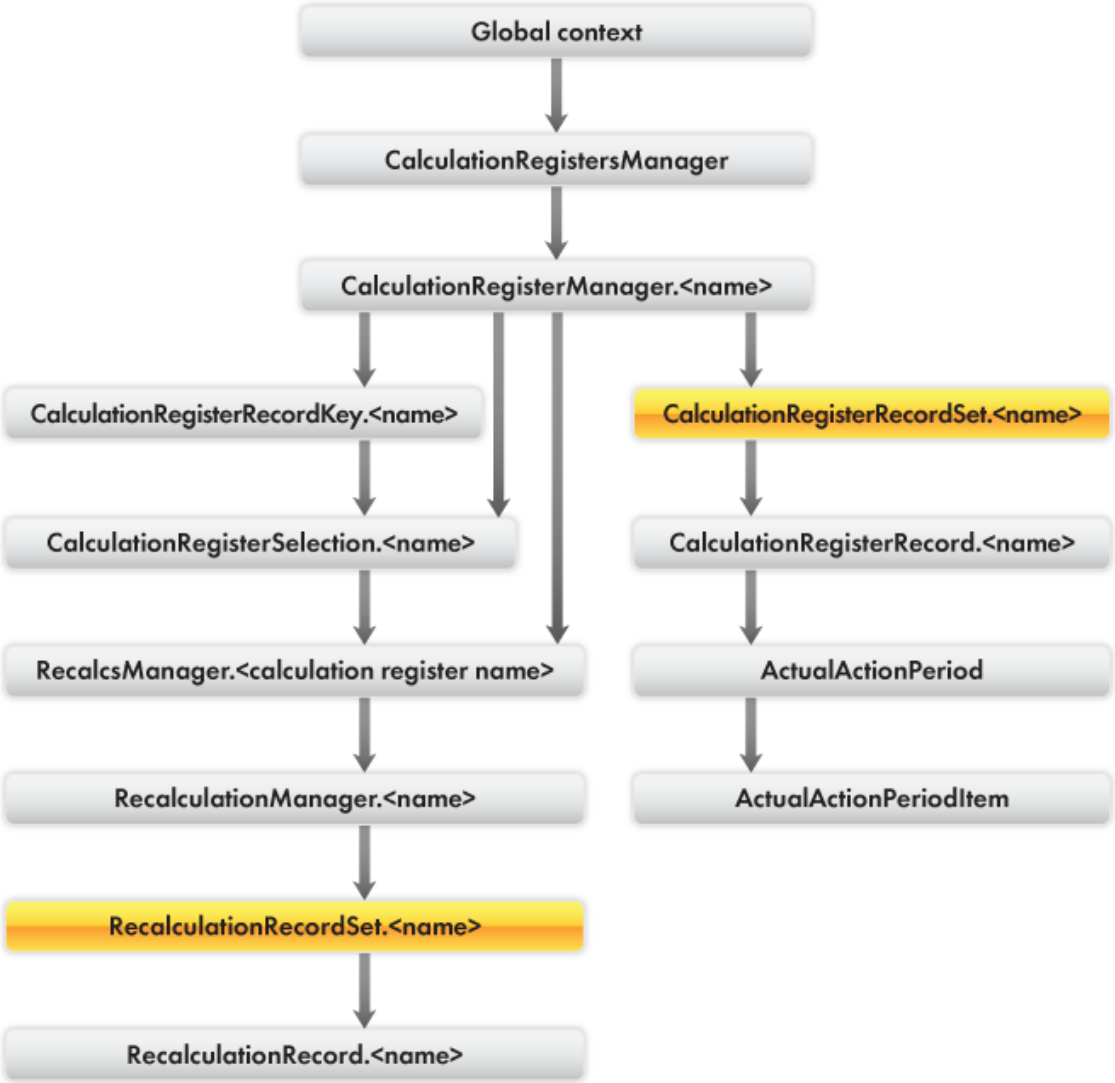


Fig. 29.23. 1C:Enterprise script objects used for operations with calculation registers

Note. Yellow boxes indicate data manipulation objects.

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

CalculationRegisterRecord.<name> provides access to calculation register records. This object is not created directly. Instead, it is provided by other objects related to calculation registers. For example, this object represents register records in a record set.

CalculationRegisterRecordKey.<name> is a set of values that uniquely identify a register record. This object is used for referencing a specific record. For example, this object represents a value of the CurrentRow property of the tabular section that stores a list of register records.

RecalcsManager.<calculation register name> is a manager of all recalculation managers in a calculation register.

RecalculationManager.<recalculation name> is used to get a recalculation record set.

RecalculationRecordSet.<recalculation name> represents a recalculation record set.

RecalculationRecord.<recalculation name> is used to access a recalculation record.

ActualActionPeriod is an array of values that have ActualActionPeriodItem type.

ActualActionPeriodItem is an item of the actual action period.

Properties and methods used for operations with calculation registers are similar to properties and methods used for operations with accumulation registers. See section "1C:Enterprise script objects used for operations with accumulation registers" on page 863.

Event sequence for writing calculation register record sets from record set forms

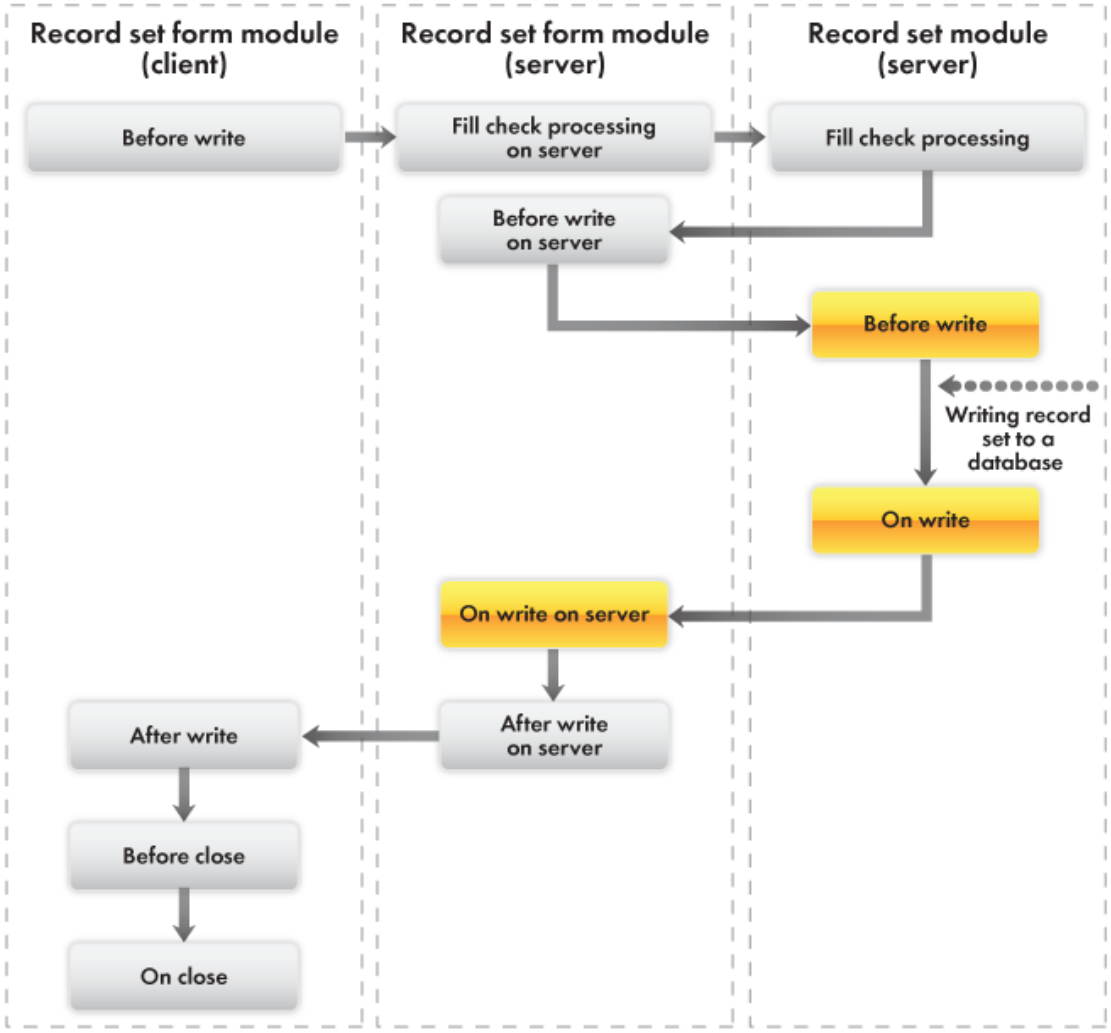


Fig. 29.24. Event sequence for writing a calculation register record set from a record set form

Note. Yellow boxes indicate events executed in the writing transaction.

Exchange plans

1C:Enterprise script objects used for operations with exchange plans

The following chart shows the interaction between 1C:Enterprise script objects used for operations with exchange plans (fig. 29.25).

Fig. 29.25. 1C:Enterprise script objects used for operations with exchange plans

Note. The yellow box indicates the data manipulation object.

Learn more! For details on major types of 1C:Enterprise script objects, see section "1C:Enterprise script objects used for operations with applied data" on page 832.

ExchangeMessageWriter is used to write data exchange messages.

ExchangeMessageReader is used to read data exchange messages. At the start of the reading it checks the message header and rejects messages with invalid headers. At the end of the reading it modifies the ReceivedNo attribute of the exchange plan based on the number of the read message.

Properties and methods used for operations with exchange plans are similar to properties and methods used for operations with catalogs. See section "1C:Enterprise script objects used for operations with catalogs" on page 839.

Event sequence for writing exchange plan nodes from node forms (save and close)

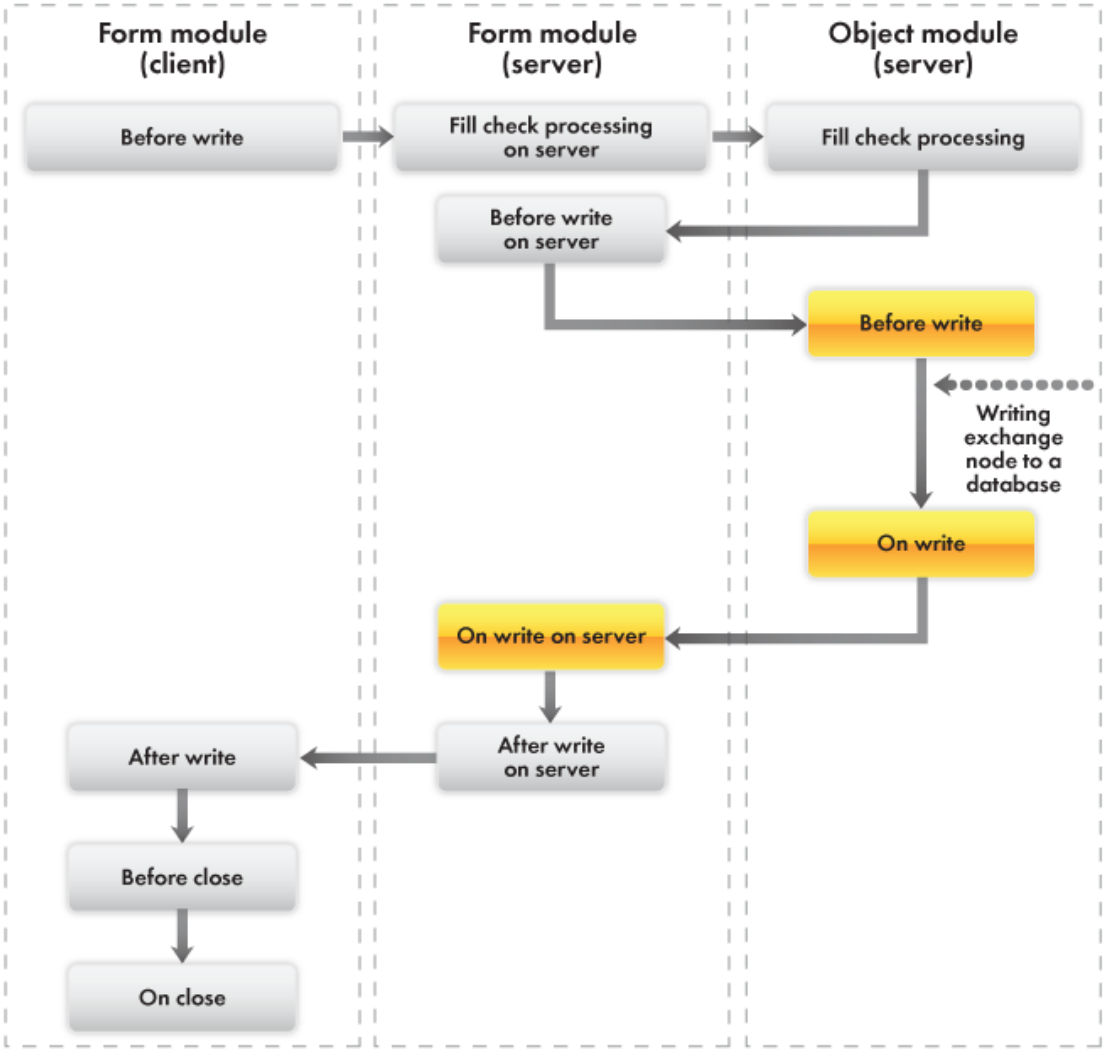


Fig. 29.26. Event sequence for writing an exchange plan node from a node form

Note. Yellow boxes indicate events executed in the writing transaction.

Glossary

1C:Enterprise script

An important part of 1C:Enteprise platform used for implementation of custom applied solution algorithms.

1C:Enterprise server

A part of a 1C:Enteprise system that passes the client application queries to the database server and returns the query results to the client. The server runs the majority of 1C:Enterprise script algorithms, prepares data to be displayed in forms and reports, and so on.

1C:Enterprise mode

The applied solution operation mode intended for end users. In this mode users add data, have it processed, and obtain the data processing results.

Accounting flag

A configuration object that is subordinate to a chart of accounts. It stores the accounting type for a specific account. For example, it can show whether the accounting is quantitative, or whether accounting is performed in foreign currency.

Accounting register

A configuration object used to describe the structure of accumulated accounting data based on a chart of accounts.

Accumulation register

A configuration object used to describe data accumulation structure.

Action period of a calculation type

A user-defined period when the calculation type is in effect.

Actual action period

A period that is determined based on the action period of a calculation type by applying all of the action periods of the calculation types that displace that calculation type by action period.

Applied solution

A component of a 1C:Enterprise system that is visible to end users. It encompasses the entire enterprise automation functionality.

Balance register

An accumulation register that accumulates balance values.

Base calculation period

A period within which the results of other calculation types that affect a specific calculation type by base period are analyzed.

Base calculation type

A calculation type that produces a result that is used in the recalculation of a specific calculation type. For example, Salary serves as a base calculation type for the Bonus calculation type.

Base configuration (or simply Configuration)

A configuration intended for developers. They use Designer to edit this configuration.

Blocking window

A window that blocks the entire application interface or its part but does not block the script execution.

Breakpoints

A tool for interrupting program execution at specific points during debugging.

Calculated fields

Additional fields of a data composition schema whose values are calculated according to some formula.

Calculation register

A configuration object used to describe the storage structure for accumulated results of complex periodic calculations.

Calculation type

An algorithm used to calculate a certain component of the wages, such as Salary, Bonus, and so on.

Catalog

A configuration object used for managing lists of data.

Chart

A report structure item used to display data as a chart.

Chart of accounts

A configuration object that defines the set of accounts a company uses to store its accounting data.

Chart of characteristic types

A configuration object that describes a storage structure for user-defined characteristics.

Client

The applied solution component the users interact with.

Client/server architecture

An architecture consisting of three components interacting with each other: a client, a 1C:Enteprise server, and a database server.

Configurability of 1C:Enterprise system

The ability to customize the system to fit the specific needs of a company or to perform a specific set of tasks.

Configuration

A set of developer-created objects, their properties, methods, and behavior algorithms, which reflect enterprise business activities. A configuration is developed in Designer mode.

Configuration object attributes

Developer-defined properties that describe a configuration object. For example, Color or Manufacturer.

Configuration object properties editor

An editor window that is available for complex configuration objects. It is intended for step-by-step creation of such objects.

Configuration object tree

A hierarchical structure of all configuration objects.

Configuration objects

Logical units that form a configuration.

Current section commands panel

A panel that contains commands of the current section: first the commands that open lists, and then commands that create new data items, generate reports, and run data processors.

Current section function menu

A list of all commands available in the current section. A user can specify the list and order of commands in 1C:Enteprise mode, mark commands as favorites, and so on.

Data composition schema

The basis for report building that contains initial data required to compose the report.

Data composition schema settings

The settings that define hierarchical report structure (grouping, table, or chart) and the report view (the list of report fields, filter, sorting, conditional formatting of report records, and so on).

Data composition schema resources

Fields whose values are calculated based on detailed records included in a grouping. The resources are group or overall totals of a report.

Data composition system

A powerful and flexible report generation tool that streamlines all operations required for report generation: from retrieving data from various sources to displaying that data in a user-friendly manner.

Database configuration

A configuration where users operate.

Database server

A third-party software. Its primary purpose is maintenance of databases.

Debugger

A tool that simplifies the development and debugging of 1C:Enterprise script modules.

Dependency by base period

The effect that base calculation types have on the base action period of a specific calculation type.

Designer mode

Developers use this mode for creating or modifying configurations.

Detailed report records

Records obtained when a query is executed without totals.

Displacement by action period

The effect that displacing calculation types have on the action period of a specific calculation type.

Displacing calculation type

A calculation type that displaces a specific calculation type by action period. For example, Absence serves as a displacing calculation type for the Salary calculation type.

Document

A configuration object used for entering events related to the enterprise business.

Document posting

Reflects the fact that the events described by a document affect the accounting records.

Enumeration

A configuration object that describes a storage structure for value sets that cannot be changed at run time.

Event handlers

A 1C:Enterprise script procedure that is executed when an event of a configuration object occurs.

Events

Various situations that occur when an applied solution is running. Events are connected to specific configuration objects. For example, the OnOpen event of a Form configuration object occurs when the form is opened.

Exchange plan

A configuration object that is used to describe nodes and configuration objects participating in the exchange (exchange nodes and exchange objects).

Exchange objects

Configuration objects that participate in the exchange described by a specific exchange plan.

Extra dimension accounting flag

A configuration object that is subordinate to a chart of accounts. It stores accounting options (for example, quantitative or multicurrency accounting) for a specific account.

Extra dimension types

Account characteristics used in analytical accounting.

Extra dimensions

Objects that represent accounting options.

Favorites panel

A panel that contains the most important lists, reports, database objects, commands, and other entities frequently accessed by a user.

Fixed settings of a dynamic list or report

Settings defined using 1C:Enterprise script (for example, when a list form is opened with a specific filter).

Folder and item hierarchy

A hierarchy type in a hierarchical catalog: an item or a group of items can be subordinate to another group of items in that catalog.

Form editor

A set of interconnected windows used for editing form data, items, commands, module, and other form components.

Form navigation panel

A panel that contains commands for opening lists with data related to the database object displayed in the form. For example, a document form can contain commands that open register records generated by the document.

Form wizard

A visual developer tool used to create configuration object forms.

Functional options

Configuration objects that provide developers with the option to separate some applied solution functionality, so that it can be easily enabled or disabled at the deployment stage or during the regular applied solution operation.

Gantt chart

A chart that consists of intervals positioned on a time scale. It depicts the usage of resources (series) by objects (points).

Grouping

A report structure item used to display data as a regular linear report.

History panel

A panel that contains the history of user actions, grouped by date.

Independent information register

An information register that is not subordinate to a recorder.

Information panel

A panel in the bottom part of the applied solution window, which displays the system performance indicators: server call details, volume of data transferred between the client and the server, and so on.

Information register

A configuration object used to describe a multidimensional data storage structure.

Information register record key

A set of register dimension values, which also includes a period value for periodic registers. An information register cannot contain two records with identical keys.

Item hierarchy

A hierarchy type in a hierarchical catalog: an item can be subordinate to another item.

Leading calculation type

A calculation type that produces a result that forces the recalculation of a specific calculation type when the result is changed. For example, Absence and Salary serve as leading calculation types for the Bonus calculation type.

Main section (Quick menu)

An applied solution section used to store forms and commands related to frequently used documents, reports, catalogs, and so on.

Modal method

A 1C:Enteprise script method that opens a modal window. Examples of modal methods are DoQueryBox() and DoMessageBox().

Modal window

A window that blocks the execution of the applied solution logic. This ensures that the applied solution algorithms are executed in the right order and provide results based on recent user input.

Module

A storage for a program text written in 1C:Enteprise script.

Module context

A set of objects, variables, procedures, and functions available during the module execution.

Name of a configuration object

A unique object name that serves for accessing the object properties and methods in 1C:Enterprise script.

Object data access method

The data access method that is based on 1C:Enteprise script objects. For example, the DocumentObject.Services object contains the values of all attributes and tabular sections of the Services document.

Owner

A configuration object that has a subordinate configuration object. For example, a Products catalog can be the owner of a UnitsOfMeasure catalog.

Parent

A catalog item or a group of items that has subordinate items in that catalog (the exact definition depends on the catalog hierarchy type).

Period addition of a data composition schema template

One can specify a period addition for a grouping, with certain periodicity within a certain interval. This is used for adding details to report data.

Periodic information register

An information register that uses time stamps.

Platform

The basis of any 1C:Enterprise system. It provides the environment for configuration execution, development, and modification.

Point in time

A combination of date, time, and a reference to a database object. This concept is intended to explicitly identify the order of reference database objects in time. It makes sense mostly for documents.

Predefined items

Configuration object items created by a developer. They can be used in 1C:Enterprise script algorithms.

Primitive data types

Number, String, Date, and Boolean. Primitive data types are predefined within the system and are limited in number.

Print wizard

A visual developer tool used to create configuration object print form templates.

Property palette

An auxuliary window used to edit all of the configuration object properties and other data related to the object.

Query language

A language used to describe the algorithm for selecting data from database query tables. This algorithm is stored in the query text.

Query wizard

A visual developer tool used to build queries.

Quick user settings item

A settings item often accessed by users, available directly in a report form.

Real-time document posting

Posting that is performed in real time. The purpose of real-time posting is recording changes and events that occur right now.

Real-time timestamp

A timestamp that is generated every time a document is posted in real time. Its value is generated based on the current session date and the last real-time timestamp generated.

Recalculation

A configuration object that is subordinate to a calculation register. When a newly added register record affects a result of a calculation that is based on existing register records, a recalculation object records that fact.

Register attributes

A set of register properties that store additional data.

Register dimensions

Configuration objects that serve as categories for data accumulated in a register. For example, Material or Warehouse.

Register recorder

A configuration object used to generate records in the register.

Register records

A set of records that describes the changes of the register state. Each record includes dimension values, resource increment values, and more.

Register records of a document

Register records that are created during document posting. They reflect the changes introduced by the document.

Register resources

Types of data accumulated in the register. For example, Quantity or Amount.

Regular document posting

Posting that reflects the facts that took place in the past or are certain to happen in the future in the database. The purpose of regular document posting is reflecting completed operations in the infobase.

Role

A configuration object intended to describe user rights for performing specific operations with specific data stored in the infobase.

Sections panel

A panel in the top part of the applied solution window that reflects the functional structure of the applied solution and provides navigation between the functional parts.

Standard attributes

Configuration object properties that are automatically generated by the platform. Examples: Code and Description.

Subordinate configuration objects

Configuration objects that are logically connected with another configuration object and are subordinate to it. Attributes and Forms are examples of subordinate configuration objects.

Subsystems

Configuration objects that logically divide an applied solution into functional parts.

Synonym of a configuration object

Stores a presentation of a configuration object that is used in the applied solution interface.

Syntax Assistant

A developer tool that stores descriptions of all software objects available in the system along with their methods, properties, events, and so on.

System command area

The area in the applied solution header, next to the configuration name. It contains commands available in all applied solutions, such as opening the main menu, opening the About window, or printing.

Table

A report structure item used for displaying data as a table.

Table access to data

Data access implemented using database queries. Developers use it to access individual fields of database tables.

Tabular section

A data set with a fixed structure and a variable number of rows. For example, a list of past employments in the Employees catalog.

Template

A configuration object intended to store various forms of data presentation. It can also store auxiliary data used by some configuration objects or by the entire configuration.

Temporary tables

Software objects created by developers and populated with data. Queries can use temporary table data.

Tools panel

A panel that contains commands for opening the Favorites panel, the History panel, and the current section function menu, and for running the full-text search in the database.

Transaction

An indivisible sequence of data manipulations that brings the database from one integral condition to a new one. If for some reason one of the operations included in a transaction cannot be performed, the database reverts to the state prior to the transaction.

Turnover register

An accumulation register that accumulates turnover values.

Type-defining objects

Configuration objects that can generate new data types.

User settings of a dynamic list or a report

Settings specified in 1C:Enterprise mode. They are automatically saved for the current user.

Virtual tables

Tables that the platform generates based on queries to actual database tables.

Work area of an applied solution

The applied solution window that displays list forms, report forms, document forms, and so on.

XML reader

A 1C:Enterprise script object used for reading XML documents.

XML serialization

Conversion of a 1C:Enterprise object into an XML data sequence, or an opposite conversion (provided that a matching object type is available in 1C:Enterprise).

XML writer

A 1C:Enterprise script object used for writing XML documents from 1C:Enterprise script.